

gog: A Grammar of Graphics

One engine. Every language.

psychometrician	Claude Sonnet 4.6	Claude Opus 4.8
	Claude Opus 5	Claude Fable 5

2026-07-28

Table of contents

Preface	20
<i>A</i> grammar, not <i>the</i> grammar	20
Where this began: 1443	21
What made the letters learnable	22
The same problem, in graphics	24
What the grammar refuses	27
The promise, and the book's rules	28
The cast	29
How this book runs	30
Installing gog	31
Who wrote this	33
Thanks to Posit	35
Thanks to Anthropic	35
I A morning	37
1 A first plot	39
1.1 Three steps to run any example	41
1.2 Add a word, ask a richer question	41
1.3 An incomplete sentence is refused	42
1.4 Refine, don't rebuild	43
1.5 The shape of every sentence	44
2 The grammar	46
2.1 The kernel	46
2.2 The colors of the code	47
2.3 Sources: where the rows come from	49
2.4 Marks: the shape drawn for each row	49
2.5 Channels: a column made visible	50
2.6 Transforms: values the data does not hold	53
2.7 The + operator: layering	54
2.8 The * operator: combining	57
2.9 The and / operators: faceting, or composing	58
2.10 Scales: how a number becomes a position	61

2.11	Spaces: the surface a plot is drawn on	63
2.12	Settings: values you <i>set</i> rather than <i>map</i>	63
2.13	Labels: the words on the plot	66
2.14	Facets: one panel per category	67
2.15	Selections: choosing rows in a drawn plot	67
2.16	Names the host language already uses	67
3	Reading practice	70
3.1	Gold medals by country	70
3.2	The same bars, lying down	71
3.3	One measurement's shape	73
3.4	The same question, smoothly	74
3.5	One dot per country, grouped by continent	75
3.6	The typical value per continent	77
3.7	Five countries across the decades	78
3.8	Three species on one axis	80
3.9	One panel per continent	82
3.10	Three measurements on three axes	83
3.11	What you just did	85
4	Writing practice	86
4.1	Which continents live longest?	86
4.2	How do petal lengths differ by species?	90
4.3	Where is the world's wealth, and its health?	94
4.4	Five questions, nothing in front of you	96
4.5	When the engine says no	103
4.6	The morning checkpoint	105
II	The letters	106
5	Data	108
5.1	Binding a table	108
5.2	The table has to be tidy	110
5.3	Plot-level vs. layer-level data	113
5.4	When the second table names its columns differently	122
5.5	A table in a database	124
5.5.1	Three things to know	125
5.5.2	How much data comes back	126
5.5.3	Two directions, and which one suits you	127
5.6	Declaring the order of categories	128
5.7	Missing values	136
5.8	Special characters and other writing systems	140

6	The book's data	144
6.1	Reading a table	144
6.2	Two things a CSV cannot record	145
6.2.1	A declared order	146
6.2.2	Labels that look like numbers	150
6.3	The tables	151
7	Choosing a mark	153
7.1	One variable	153
7.1.1	Continuous variable	154
7.1.2	Categorical variable	156
7.2	Two variables	159
7.2.1	Continuous $\times$ Continuous	160
7.2.2	Categorical $\times$ Continuous	168
7.3	Quick reference	174
7.4	Mark families	175
7.5	Unpronounceable combinations	176
8	Point	178
8.1	Basic usage	178
8.2	The dot as a default	179
8.3	A categorical x: the strip plot	183
8.3.1	Either axis: a horizontal strip	186
8.4	Color	188
8.5	Shape	190
8.6	Size	191
8.7	Opacity	193
8.8	All channels at once	196
8.9	What you can set	198
8.10	What it refuses	200
9	Line	202
9.1	Basic usage	202
9.2	Automatic sorting by x	203
9.3	Multiple series	205
9.4	Layering line over other marks	207
9.4.1	Giving each layer its own appearance	208
9.5	A category on the domain: the profile	212
9.6	Smoothed trend line	216
9.7	Two boundaries from a pair	217
9.8	Stroke width and opacity	219
9.9	What you can set	223
9.10	What it refuses	225

10 Bar	228
10.1 Basic usage	228
10.2 Color	229
10.3 Ordering bars	231
10.4 Duplicate categories	240
10.5 Horizontal bars	245
10.6 Texture	249
10.7 Histogram	250
10.7.1 Overlaid histograms: one distribution per group	254
10.8 Grouped bars: side by side	260
10.9 Stacked bars: piled up	262
10.9.1 The 100% stacked bar	264
10.10 What you can set	267
10.11 What it refuses	269
 11 Area	 270
11.1 Basic usage	270
11.2 The baseline has to be zero	271
11.3 Multiple series	274
11.4 Giving an area an edge	279
11.5 One region, one fill	281
11.6 With transforms	282
11.7 A category on the domain	288
11.8 What you can set	291
11.9 What it refuses	292
 12 Step	 294
12.1 Histogram outline	294
12.2 A step function	296
12.3 A value that holds until it changes	297
12.4 Multiple series	299
12.5 A category on the domain	300
12.6 Stroke width and opacity	302
12.7 What you can set	303
12.8 What it refuses	305
 13 Interval	 308
13.1 A range per group	308
13.2 The middle half of each group	310
13.3 Three layers instead of one mark	311
13.4 A confidence interval, with a center	314
13.5 The spread of the data	315
13.6 Why <code>interval</code> needs a transform	319

13.7	Whiskers on their side	319
13.8	Splitting by color	324
13.9	Caps, or a bare linerange	328
13.10	Hiding the center dot	330
13.11	A dashed whisker	332
13.12	What you can set	334
13.13	What it refuses	337
14	Box	338
14.1	A summary per group	338
14.2	Distribution across many groups	340
14.3	Boxes on their side	341
14.4	Whiskers to the extremes	345
14.5	Why <code>box</code> needs no transform	346
14.6	Coloring the boxes	347
14.7	What you can set	350
14.8	What it refuses	353
15	Ribbon	354
15.1	A band from a range	354
15.2	The mean and its band	355
15.3	A fan chart of nested bands	357
15.4	Why a ribbon needs a transform	360
15.5	A confidence band	360
15.6	A pre-computed band	362
15.7	Filled, or two dashed edges	366
15.8	One band per group	368
15.9	One region, one fill	369
15.10	A category on the domain	370
15.11	The violin: a distribution per category	373
15.11.1	What the width means	376
15.11.2	Sideways, and halved	377
15.11.3	Letting the ridges overlap	382
15.12	What you can set	388
15.13	What it refuses	390
16	Text	392
16.1	The mark and its content	392
16.2	Both at once: the labeled scatter	395
16.2.1	Nudging the label off its point	396
16.2.2	<code>repel</code> : moving labels off each other	398
16.3	Numeric labels: the value on each mark	403
16.4	Color	405

16.5 Styling: one color, one size	407
16.6 Why <code>label</code> is a channel	408
16.7 A note is a layer	409
16.8 What you can set	410
16.9 What it refuses	412
17 Path	414
17.1 The connected scatterplot	414
17.2 Why <code>line</code> cannot draw it	416
17.3 Which end is the last row?	417
17.4 An arrow is a layer	420
17.5 Two transforms, and the rest refused	422
17.6 The contour: <code>path * density</code>	422
17.7 In polar, a route is a spiral	428
17.8 In space, the route is unchanged	430
17.9 What a path is for	435
17.10What you can set	436
17.11What it refuses	438
18 Rule	439
18.1 A line at a threshold	439
18.2 Which axis it lands on	441
18.3 A rug is the same mark	442
18.3.1 The other axis	445
18.4 Coloring the lines	446
18.5 Thickening a rule	448
18.6 In polar: the ring and the spoke	452
18.7 What you can set	454
18.8 What it refuses	457
19 Zone	459
19.1 A band across the panel	459
19.2 A band up the panel	461
19.3 Both pairs: a rectangle	464
19.4 Which axis the pair bounds	465
19.5 Coloring the zones	467
19.6 Rectangles that <i>are</i> the data	469
19.7 Cells cut from the data: the heatmap	475
19.8 Hexagons: the other mesh	480
19.8.1 Why the mesh belongs to <code>bin</code>	484
19.8.2 One caveat	485
19.9 Cells the data estimated: <code>zone * density</code>	485
19.10Bands the data cut: <code>zone * density(levels =)</code>	490

19.11	Cells the axes already made: the tile plot	493
19.11.1	<code>bin</code> cuts, <code>count</code> tallies	495
19.11.2	<code>count</code> tallies rows, <code>mean</code> reduces a column	500
19.11.3	Cells cut, then summarized: the summary heatmap	502
19.11.4	One of each: the mixed mesh	505
19.11.5	Half a mesh: bounded by one slot	508
19.12	Sides the data drew: the choropleth	511
19.13	What you can set	512
19.14	What it refuses	518
20	Surface	525
20.1	A sheet through the samples	525
20.2	One row per crossing	526
20.3	A continuous floor	529
20.4	The third geometry of one field	529
20.5	When each cell holds one value	534
20.6	The mesh, and what the shading means	540
20.7	Color by height, per face	543
20.8	Opacity by a measure, per face	549
20.9	One sheet, four angles	553
20.10	What a surface is for	556
20.11	What you can set	556
20.12	What it refuses	557
21	Edge	559
21.1	Color and opacity, one per row	561
21.2	What you can set	564
21.3	What it refuses	565
22	Scales	566
22.1	A scale is not a transform	566
22.2	The order of a scale and a transform	569
22.2.1	Scaling the grouping axis	569
22.2.2	The measured value keeps its own units	570
22.2.3	<code>smooth</code> fits inside its groups	572
22.3	What a log scale refuses	574
22.4	Choosing the base	575
22.4.1	The base changes the labels, not the picture	575
22.4.2	Natural log	579
22.5	Time: a date column gets a calendar axis	580
22.5.1	Bars sit on dates; they never measure one	584
22.5.2	What a time axis refuses	585
22.5.3	Time on the other channels	586

22.6	The scales	588
22.7	The axis fits your data	588
22.8	A scale is not only for axes	591
22.9	limits: when your data is not the authority	596
22.10	The column's type picks the axis	604
22.11	tick_count: how many ticks the axis aims for	606
22.11.1	Crowding in the plane and in the cube	610
23	Channels and legends	611
23.1	color	612
23.2	size	618
23.3	shape	620
23.4	pattern	623
23.5	opacity	627
23.6	group	633
23.7	Multiple channels	635
23.8	Palettes	637
23.8.1	Categorical palettes	637
23.8.2	Ramps	642
23.8.3	Diverging ramps	645
23.8.4	Custom colors	652
23.9	Legend placement	656
24	Setting vs mapping	657
24.1	Why a constant is not a channel	660
24.2	What you can set	660
24.2.1	One color, any mark	669
24.2.2	Several at once	674
24.2.3	Colors are CSS colors	675
24.3	What settings are for	677
24.4	line takes settings it refuses as channels	680
24.5	You cannot map and set the same property	682
24.6	style applies to one layer only	683
24.7	palette: choosing the colors a mapping uses	684
24.8	theme: settings that belong to no mark	687
24.9	The journal look	697
24.10	The strip, and how it prints	700
24.10.1	The label color follows the band	705
25	Transforms	713
25.1	One variable, or two?	714
25.2	bin: histogram	715
25.2.1	Choosing the bin count	716

25.2.2	Why <code>bar * bin</code> , not a <code>histogram()</code> shortcut?	719
25.2.3	The same <code>bin</code> , cutting two axes	720
25.2.4	One of each: the mixed mesh	721
25.2.5	Which mesh: <code>tiling</code>	723
25.3	<code>count</code> : frequency aggregation	724
25.3.1	The same <code>count</code> , tallying two axes	726
25.4	<code>proportion</code> : relative frequency	729
25.4.1	The total is the whole plot, always	730
25.4.2	Normalizing something other than a tally	734
25.5	<code>density</code> : kernel density estimate	737
25.5.1	Choosing the bandwidth	739
25.5.2	One transform, four readings	742
25.6	What <code>bin</code> , <code>density</code> and <code>smooth</code> refuse	744
25.6.1	A fit needs rows to fit through	745
25.7	Chaining transforms	746
25.7.1	What cannot be chained, and why	750
25.8	Output columns	754
25.9	<code>smooth</code> : LOESS trend line	755
25.10	Aggregation family: <code>sum</code> , <code>mean</code> , <code>median</code> , <code>max</code> , <code>min</code> , <code>quantile</code>	757
25.10.1	The same six, grouping by a pair	765
25.10.2	Why these names?	767
25.11	<code>quantile</code> : one point in the distribution	767
25.12	<code>range</code> : a low/high pair	771
25.12.1	A band between two quantiles	773
25.13	<code>confidence</code> : the interval of the mean	776
25.14	<code>deviation</code> : the spread of the data	778
25.15	<code>bounds</code> : a <i>pre-computed</i> low/high pair	780
25.16	<code>partition</code> : a whole among nested parts	782
25.16.1	The mosaic	782
25.17	<code>flow</code> : a magnitude through its stages	790
25.18	<code>layout</code> : a graph placed from an edge table	792
25.19	<code>cluster</code> : the closest leaves joined first	792
25.20	<code>dodge</code> : side by side instead of on top	794
25.21	<code>stack</code> : piled up, not side by side	799
25.21.1	Stacked areas: the case <code>stack</code> was built for	800
25.21.2	<code>stack(share = TRUE)</code> : every pile filled to one	802
25.21.3	<code>stack(baseline =)</code> : where the pile sits	805
25.21.4	A pile has one direction	812
25.21.5	The dot plot	814
25.21.6	How many rows is too many	817
25.21.7	One column, split into its parts	822
25.22	<code>jitter</code> : spread sideways, along the category	824
25.22.1	How much to spread: <code>jitter(amount)</code>	827

25.23	<code>repel</code> : move the labels off each other	829
III	The sentences	834
26	Operators	836
26.1	<code>+</code> : compose	836
26.1.1	Layering with <code>+</code>	836
26.1.2	The order of <code>+</code> : data, mark, positions, refinements	838
26.2	<code>*</code> : derive	843
26.2.1	Why the order is fixed	843
26.2.2	What <code>*</code> is rigid about	844
26.2.3	Live examples	844
26.3	<code> </code> and <code>/</code> : facet, or compose	849
26.4	Which orders matter	853
26.4.1	What groups first	856
26.4.2	What parentheses may group	857
26.5	Why <code>+</code> , and not a pipe	858
26.6	Summary	860
27	Encoding scope	861
27.1	The rule	861
27.2	One axis, and which column each layer reads	862
27.3	Channels: plot-scoped or layer-scoped, by position	867
27.3.1	Why binding only goes forward	870
27.3.2	A plot-scoped channel and the marks it skips	872
27.4	<code>data()</code> mid-expression	875
28	Faceting	877
28.1	<code> </code> : panels side by side	877
28.2	<code>/</code> : panels stacked	879
28.3	Crossing into a grid	880
28.4	Folding a long line of panels	881
28.5	Giving a panel its own scale	886
28.6	A layer without the facet column	895
28.7	The facet column's type	896
29	Composition	898
29.1	Two plots, one page	898
29.2	A shared column is a shared axis	901
29.2.1	One axis, but not one legend	903
29.3	The marginal plot	906
29.4	A clustered panel decides the order	909

29.5	How thin is the marginal histogram? <code>theme(width =, height =)</code>	910
29.6	How big is the page itself?	912
29.7	What a page is not	913
29.8	Where these operators come from	914
29.9	Layering, faceting, or composition?	914
30	Flow	916
30.1	One row is one path	916
30.2	Adding a stage	918
30.3	Color comes from a stage	919
30.4	The slots and their names	921
30.5	What flow refuses	923
30.6	Where the limits are	924
31	Cluster	925
31.1	The tree	925
31.2	The order	928
31.3	The clustered heatmap	932
31.4	What cluster refuses	934
31.5	Where the limits are	935
32	Space	936
32.1	<code>z</code> : one more vowel	936
32.2	The viewing angle	939
32.3	Turning the plot yourself	941
32.4	<code>path</code> : the route through the cube	942
32.5	The 3-D histogram	944
32.6	Transforms in the cube	947
32.7	<code>interval</code> and <code>box</code> in the cube	952
32.8	<code>z_label()</code> : naming the third axis	957
32.9	What the cube refuses	958
32.10	What the cube blocks	961
32.11	Faceting: one cube per panel	962
33	Polar	968
33.1	The axis that breaks a circle	968
33.2	What <code>polar()</code> changes, and what it leaves alone	971
33.3	Where the circle starts	975
33.4	One turn, whatever the data spans	976
33.5	The radar: a categorical angle	982
33.6	An angle that is measured, not named	987
33.7	The pie: where you put the category	991
33.8	What a circle makes harder to read	998

33.9	The marks in a circle	998
33.10	The sunburst: depth is the radius	1005
33.11	The donut: one level is still a tree	1010
33.12	The same sentence, unbent	1014
33.13	Naming the nodes	1016
33.14	What a tree lets the grammar check	1019
34	Nest	1020
34.1	One sentence, three spaces	1020
34.2	Why area, when a column already works	1025
34.3	Two levels, from a position	1025
34.4	What a packed panel does not have	1027
34.5	Naming the regions	1027
34.6	The other refusals	1030
34.7	Sorting the regions	1031
35	Map	1034
35.1	Flattening the sphere loses something	1036
35.2	Drawing the outlines	1038
35.3	The second layer is the data	1038
35.4	A quantity goes on a channel	1040
35.5	Meridians and parallels	1044
35.6	Naming places	1045
35.7	What a map refuses	1048
35.8	Where Mercator stops	1049
35.9	Filling the regions: the choropleth	1050
35.10	A value on the map	1052
35.11	Rings, and the country inside another country	1054
36	Globe	1056
36.1	The view	1057
36.2	The graticule	1061
36.3	A route bends with the sphere	1062
36.4	Many routes are a network	1064
36.5	Meridians and parallels	1067
36.6	The choropleth, kept round	1068
36.7	A measure stands on the radius	1070
36.8	The depth goes to the cube	1072
36.9	What a globe refuses	1073
37	Network	1075
37.1	One row is one edge	1075
37.2	The nodes and their names	1077

37.3	An edge carries its own columns	1079
37.4	An angle asks for the cube	1080
37.5	What the network refuses	1083
37.6	Where the limits are	1084
38	Play	1085
38.1	The same sentence, in space	1086
38.2	The column's order	1089
38.3	Too many panels for one page	1091
38.4	What every frame shares	1094
38.5	A layer with no frames	1098
38.6	How fast	1102
38.7	The column that cannot do both jobs	1103
38.8	Crossed with a facet	1105
38.9	Frames in the cube	1107
38.10	Frames in polar	1115
38.11	Taking the sequence with you	1119
38.11.1	Why a GIF and not a video	1121
38.12	What it refuses	1121
38.13	What the page adds	1122
38.14	Cuts between frames	1123
39	Selection	1124
39.1	Saying where the selection starts	1126
39.2	Highlighting, never removing rows	1128
39.3	Two bounds make a rectangle	1130
39.4	A column of categories selects slots	1133
39.5	Two views, one selection	1136
39.6	One bound reaches every panel	1139
39.7	Which one is that?	1140
39.8	What can be brushed	1143
39.9	Selecting while the plot plays	1146
39.9.1	What the count includes	1148
39.9.2	Choosing a bound the data crosses	1148
39.10	All three at once	1149
39.11	Moving the bound yourself	1151
39.12	What the page adds	1151
39.12.1	Selecting	1152
39.12.2	Drawing around a group	1152
39.12.3	Reading what you selected	1153
39.12.4	Stamping a point	1153
39.12.5	Where pointing cannot answer	1155
39.12.6	Looking closer	1155

39.12.7 Three words, three things	1155
39.12.8 Your data on the page	1156

IV The cookbook 1157

40 One variable 1159

40.1 How many of each?	1159
40.2 What share of the whole is each?	1161
40.3 How is a measurement distributed?	1163
40.4 What is its smooth shape?	1166
40.5 How does the distribution look in steps?	1167
40.6 When there are only thirty rows	1169
40.7 What this section refuses	1172

41 Two variables 1173

41.1 Are two measurements related?	1173
41.2 What is the trend through the cloud?	1175
41.3 How much was there at each time?	1177
41.4 What is the typical value per group?	1179
41.5 What does <i>every</i> value per group look like?	1181
41.6 What if the table already has the answer?	1184
41.7 What this section refuses	1187

42 A third variable, and more 1188

42.1 Which group does each mark belong to?	1188
42.2 Will it read in gray?	1189
42.3 How big is each observation?	1191
42.4 What if four variables matter?	1192
42.5 What if the third variable is a number?	1194
42.6 What if three measurements matter equally?	1196
42.7 What if the rows form a route?	1199
42.8 What if the shape needs three dimensions?	1202
42.9 Which rows and columns are alike?	1207
42.10 What this section refuses	1209

43 Over time 1210

43.1 How did it change?	1210
43.2 How did several things change?	1211
43.3 What about a date column?	1213
43.4 What happened, and what comes next?	1214
43.5 What this section refuses	1216

44 Many groups	1217
44.1 How do the groups' distributions compare?	1217
44.2 What if the overlap gets worse?	1218
44.3 Grouping without a legend	1220
44.4 One panel per group	1221
44.5 Two grouping variables at once	1223
44.6 Which group is biggest?	1224
44.7 Two groups, mirrored	1225
44.8 What this section refuses	1228
 V The reasons	 1229
45 Design laws	1231
45.1 Law 1: Orthogonality	1231
45.2 Law 2: No Exceptions	1231
45.3 Law 3: Plain Names	1234
45.3.1 One spelling of English	1234
45.4 Law 4: Bind Once	1235
45.5 Law 5: Explicit Over Implicit	1237
45.5.1 Setting is not mapping	1237
45.5.2 A missing name must not become a default	1238
45.5.3 Illegal is not the same as unsupported	1239
45.5.4 Drawing it anyway	1241
45.6 Law 6: Compositional Invariance	1241
45.7 Law 7: Minimum Syllable	1244
45.8 Law 8: Pronounceable $\neq$ Usable	1245
45.9 Law 9: Universal Transcription	1247
45.9.1 Why that renderer writes SVG	1247
45.9.2 Why there is one renderer	1248
45.10 The settable rule	1249
45.11 Why these laws matter	1249
 VI The host language	 1250
46 R	1252
46.1 Pipes	1252
46.2 What %>% does to the table's name	1255
46.3 Why the name is worth a warning	1259
46.4 What the pipe does not change	1260
46.5 Twenty names base R already uses	1260
46.6 Five spellings that still need a prefix	1263

46.7 Reading from a database	1263
46.8 Getting it	1264
47 Python	1266
47.1 A column is written <code>col.gdp</code>	1266
47.2 A sentence over several lines takes parentheses	1267
47.3 The table	1267
47.4 Computing a column first	1268
47.5 Showing the plot	1269
47.6 Six names Python already uses	1269
47.7 What does not change	1270
47.8 Reading from a database	1271
47.9 Getting it	1271
48 Julia	1273
48.1 A column is a symbol	1273
48.2 The operators, and their precedence	1274
48.3 Nine names Julia already uses	1275
48.4 The table	1276
48.5 Reading from a database	1276
48.6 Getting it	1277
49 JavaScript	1279
49.1 The comma is <code>+</code>	1279
49.2 <code>layer()</code> is <code>*</code>	1280
49.3 <code>across()</code> and <code>down()</code> are <code> </code> and <code>/</code>	1280
49.4 Composing two plots: <code>beside()</code> and <code>below()</code>	1281
49.5 A column is written <code>col.gdp</code>	1282
49.6 A named argument is one trailing object	1282
49.7 The table, and its name	1283
49.8 A sub-expression is a value	1283
49.9 Reading from a database	1284
49.9.1 Asynchronous drivers are refused	1284
49.10 Getting it	1284
49.11 A plot for a web page: <code>html_block()</code>	1286
Afterword	1287
The ten-day challenge	1287
Growth without new words	1287
One goal left	1288
Writing quickly is not writing well	1288
The gap	1289

References	1290
Appendices	1293
A The kernel card	1293
B Functions around the grammar	1295
B.1 <code>render_svg()</code>	1295
B.1.1 What the file carries	1296
B.1.2 Plots that carry the table	1297
B.1.3 What “byte for byte” means	1298
B.2 <code>save_gif()</code>	1299
B.3 <code>gog_table()</code>	1300
C Combinations	1301
C.1 Marks and channels	1301
C.2 Marks and transforms	1303
C.2.1 Statistics: a value per group	1303
C.2.2 Ranges and bands	1304
C.2.3 Collision modifiers: an offset, not a statistic	1305
C.3 Transforms and transforms	1306
C.3.1 The shapes a long chain can take	1308
C.4 Marks and spaces	1309
D Chart names, translated	1312
D.1 Names with a one-line answer	1312
D.2 Names that need more	1317
E The ggplot2 phrasebook	1323
E.1 Side by side	1323
E.2 Translations	1323
E.3 Six differences to expect	1325
F Coverage, both directions	1326
F.1 What <code>gog</code> draws that <code>ggplot2</code> does not	1326
F.1.1 Anything in the cube	1326
F.1.2 Texture, mapped from a column	1328
F.1.3 The same plot, without a word for it	1329
F.2 What is different rather than missing	1334
F.3 What <code>ggplot2</code> draws that <code>gog</code> does not	1336
F.3.1 Refused, with the ruling on record	1336
F.3.2 The host language’s job	1339
F.3.3 Missing, with the direction already settled	1340

F.3.4	Not decided either way	1340
F.4	If your plot is not here	1344

Preface

Be agog. Use gog.

`gog` is a single graphics engine written in [Rust](#), callable from [R](#), [Python](#), [Julia](#), and [JavaScript](#). Its discipline comes from Hangeul (한글), the Korean alphabet; its vocabulary from Wilkinson’s *The Grammar of Graphics* (Wilkinson, 2005); and its working shape from [ggplot2](#) (Wickham et al., 2026). Behind all three is one promise: a beginner should be able to read any plot specification aloud on day one and write it themselves on day two.

This preface is unusually long. It explains where the grammar comes from, and why it refuses what it refuses. You do not need any of it to draw a plot. If you would rather start now, take the install command for your language from [Installing gog](#). Then go straight to [Part I, A morning](#), which teaches the whole grammar in one sitting. Nothing in it depends on this page, and you can come back here later.

A grammar, not *the* grammar

Wilkinson opens his book by defending the definite article in his title, the word *The*. His grammar is a formal system: a fixed set of rules that states what it can build and where it stops. Those stated limits are what make the claim testable. A looser approach, such as a list of chart types, has no such boundary. Nothing says where that list ends, so there is nothing to measure it against. A rival cannot even claim to cover more than such a list, because the list can always grow. Wilkinson wrote his limits down, and that is why *the* is a claim he earned.

This book takes the indefinite article on purpose. `gog` is not a restatement of Wilkinson’s system, and it is not a subset of it either. His grammar rests on an algebra of variable sets, in which `*` crosses, `/` nests and `+` blends, and the four operators you will meet in this book mean none of those things. What `gog` took from him is the inventory rather than the algebra. Most of the names in its vocabulary come from his chapters on statistics, geometry, coordinates and aesthetics. That is a real debt, and a large one. What `gog` is instead is a deliberately small **kernel** of sources, marks, channels, transforms and operators, with a few more kinds that refine a plot. It grew one decision at a time, and it answers to [nine laws](#) of its own. The kernel is that whole vocabulary: every name the grammar has, and nothing outside it. The five kinds that build a plot are spelled out [later in this preface](#), and the rest wait for [the kernel chapter](#). When a design question is genuinely open, Wilkinson is still the tiebreaker, and the package

carries his book's name. But the cover says *a*, not *the*. **gog** is one of the grammars that could be built, and it claims nothing more.

The small article also does something the definite one cannot. *A Grammar of Graphics* abbreviates to **agog**, English for eager, expectant, impatient to see what comes next. The acronym is not merely lucky; it names the mental state this grammar asks for. A plot is a specification, not drawing code: you never place a stroke yourself. You declare what you want to see, and then there is a gap (brief, but real, and felt every time) before the engine shows you what you said.

Everyone who writes a specification in **gog** waits in that gap. Reading this book, you arrive after the waiting is done. Every plot on these pages was drawn by the engine while the page was being built, not pasted in as a screenshot. That waiting is over by the time the page reaches you: the plot is already sitting under the code. Run one of these code blocks on your own computer, and you do the waiting yourself. An analyst stands in it while exploring data, after swapping one channel and before the picture changes. The people who build **gog** stand in it too, for whole sessions, waiting to see what a new mark or transform will draw. The package is **gog**; the state of working with it is *agog*.

gog's hexagonal sticker says that without a sentence. What sits inside it is a face. Its eyes are + and *. Its nose is / and |, and its mouth is the package name. Those four symbols are every operator **gog** has.

The two pairs sit where they do for a reason. + and * build a single plot, and read as “...and also...” and “...derived by...”. / and | arrange finished plots, and the panels one plot is split into, and read as “...below...” and “...beside...”. One pair composes inside a picture, the other between pictures. The face is the state this section has been describing: someone who has written a specification and is waiting to see it drawn.

Where this began: 1443

gog's discipline comes from Hangeul, so the story starts with an alphabet rather than with a chart. Nearly every writing system in use today grew slowly, without a plan. Traders borrowed a neighbor's letters and reused them for the sounds of their own language. Scribes copied picture-signs so often, and so quickly, that the pictures turned into simple strokes. Nobody recorded why any of it happened. Hangeul is the great exception. In the winter of 1443, [King Sejong](#) announced twenty-eight letters of deliberate design. The 1446 book that introduced them, the *Hunminjeongeum* (“The Correct Sounds for the Instruction of the People”), opens by saying why (Sejong the Great and the scholars of the Hall of Worthies, 1446). The spoken language differed from written Chinese, so ordinary people “who have something they wish to say are, in the end, unable to express their feelings.” Therefore, new letters, few and simple, “wishing only that everyone may learn them with ease and use them daily, with convenience.”



Figure 2: The mouth saying ㄱ (*g*). The back of the tongue rises to block the throat, and the red line traces the shape the letter draws.²

Twenty-four of Sejong’s twenty-eight letters are still in use today. That small number is the usual explanation for how quickly Hangeul is learned. English has a small alphabet too, and takes years, because its composition is full of exceptions. The same four letters, *ough*, are pronounced six different ways, and none of the six is a rare word:

word	<i>ough</i> is pronounced
<i>though</i>	like <i>o</i> in <i>go</i>
<i>through</i>	like <i>oo</i> in <i>too</i>
<i>thought</i>	like <i>aw</i> in <i>law</i>
<i>tough</i>	like <i>uff</i> in <i>stuff</i>
<i>cough</i>	like <i>off</i>
<i>bough</i>	like <i>ow</i> in <i>cow</i>

Hangeul is learnable in a morning because its composition has **no exceptions**. An **atom** is a piece that nothing divides further, here a single letter. The power is not the atoms. It is the regularity with which they combine.

The opposition understood exactly what would be lost. In 1444 a senior scholar of the court academy petitioned against the new script. Civilized writing meant Chinese characters. The clerks already had a workaround, *이두* (*idu*), and it served everyone who had trained in it. And if letters were easy, people would abandon the hard study through which learning had always been earned. It is the eternal argument of the fluent expert: *the difficulty earns its keep; my fluency is capital*. You have heard its modern form: *everyone already knows what **alpha** means*.

²Diagram by [Paranocean](#) on Wikimedia Commons, used unchanged under [CC BY-SA 4.0](#). It keeps that license rather than this book’s.

In plotting libraries **alpha** means opacity, and the argument assumes everyone has already spent the time to learn that. Sejong overruled it. The script outlived a royal ban in 1504 and survived five centuries in private letters and popular novels. It was named *Hangeul* around 1912, and it is today the alphabet of one of the most literate countries on earth. Systems that are well defined and easy to use win, in the end, because every new learner is one more person choosing them.

The same problem, in graphics

Now look at the tools we draw data with. Most are logographies, writing systems in which each symbol stands for a whole word. They are a pile of chart types, each memorized as a whole. A histogram function here, a pie function there, each with its own arguments in its own order. That is the logographic problem of graphics: not a grammar but a stockpile of special cases, usable only after long training.

That problem already had its reformers. Wilkinson found the grammar underneath the pile, and **ggplot2** carried it into the daily work of hundreds of thousands (Wickham, 2008, 2016). The two of them did for graphics what Sejong did for writing. They traded a stack of memorized charts for a real, generative language, and a better one than anything built before it. **gog** is built on that achievement and in its debt. It adds a single ambition of its own, the one *Hangeul* is famous for: no exceptions in how the pieces compose. That is the standard **gog** holds itself to, and the aim is to shorten the distance between the plot you have in mind and the words that draw it.

That debt is made of specific things, and naming them is better than a vague mention. The **+** that joins the parts of a sentence is **ggplot2**'s, not Wilkinson's, whose **+** blends variables and is a different operation entirely. The split between *mapping* a column and *setting* a fixed value is **ggplot2**'s. So is drawing several marks over one pair of axes. So is the idea that a plot is a specification you build and hand over, rather than a set of drawing commands. Wickham named that last one himself. "One of the most important features of the grammar is its declarative nature," he wrote, and **ggplot2** uses **+** to keep it that way in R (Wickham, 2008, p. 54). This package inherits both the property and the operator that protects it. Wickham has since said publicly that the pipe (R's **|>**, which chains one call into the next) would have been the better interface. In the same answer he says **ggplot2** cannot switch to it, and that a new package just to make the change would not be worth it (Wickham, 2018). **gog** keeps **+**, and the [operators](#) chapter says why. A reader who already writes **ggplot2** will recognize the shape of almost every sentence in this book. That is on purpose. After years of **ggplot2**, its grammar is how I think about visualization, so it is the shape this one took.

What changed is narrower, and it is why this package exists at all. **ggplot2** already derives a histogram, from its bar geom and its binning statistic (Wickham, 2016, Section 3.13). What it adds is a shortcut name for that pair. **gog** adds no such name, so writing the pair is the only way to draw a histogram. One engine answers four languages, so the sentence you learn in R is

the sentence you write in Python. And the rules for combining the pieces hold everywhere, with no exceptions, which is what the rest of this preface is about.

So **gog** invents no new charts, the way Hangeul invented no new sounds. It repeats Sejong's question, asked in full: not *which charts should exist*, but *what is the smallest set of letters, and the most regular rule of composition, from which every chart can be derived?* Wilkinson's grammar answers what can be drawn (Wilkinson, 2005). **ggplot2** found the shape that made that answer usable, and most of how a sentence here is assembled is **ggplot2**'s idea. Hangeul disciplines how the pieces compose. Marks are the consonants, silent until placed. Positions are the vowels that give them somewhere to stand. A mark and its positions together are a syllable, and nothing smaller draws. A histogram is nothing to memorize: it is **bar * bin**, a bar mark derived by binning, the way $\neg(k)$ is derived from $\neg(g)$ rather than invented fresh.

R

```
data(gapminder_2007) + bar * bin + x(life)
```

Python

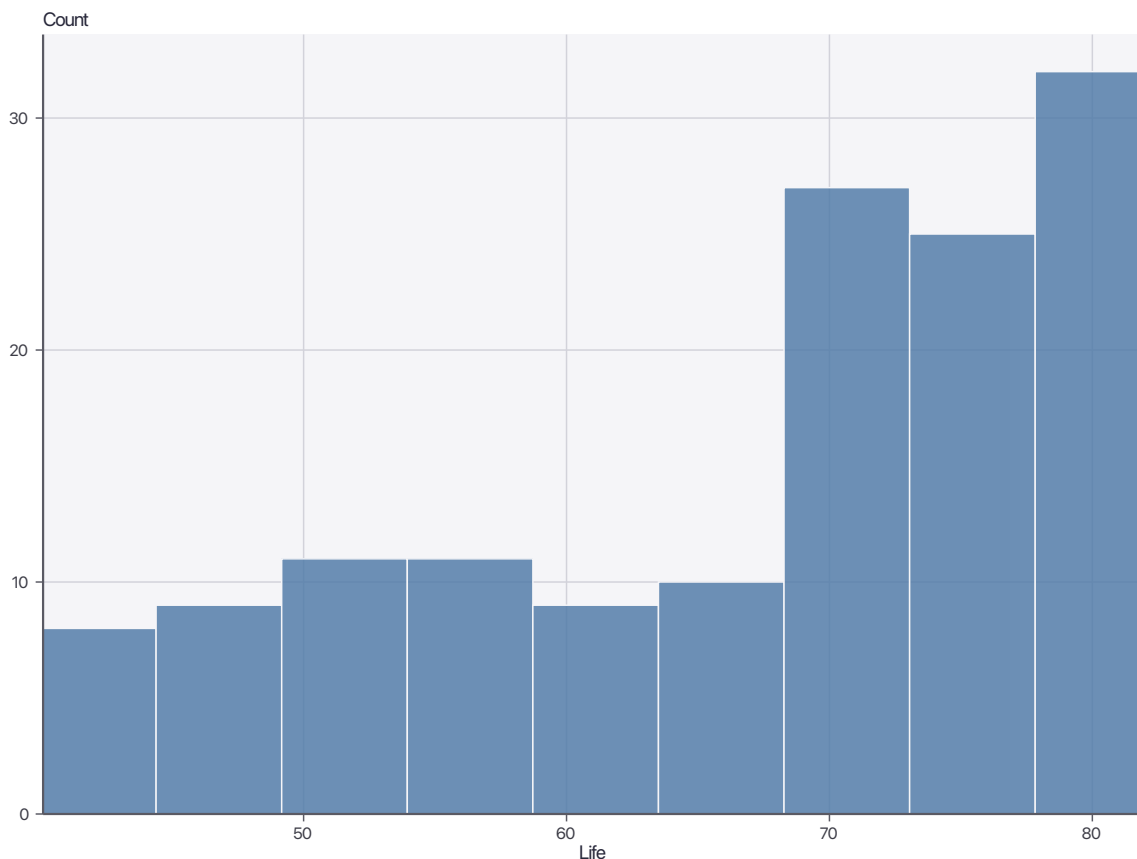
```
data(gapminder_2007) + bar * bin + x(col.life)
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life))
```



“Given gapminder 2007: bars derived by bin, x is life.” That is the whole program, and the plot above was drawn from those words as this page built. There is no histogram function in this package and no argument that turns bars into one. `data(gapminder_2007)` is the source: it names the table (142 countries, one year) that supplies the rows and the columns. `bar` is the mark, the shape drawn for each row, the way `⌈` shows the shape of the mouth that says it. `bin` is the transform, which derives new values from the columns you named: here it cuts a continuous column into intervals and counts the rows falling into each. `x` is the channel, the visual property a column is mapped to. And `*` is the operator that puts a transform on a mark. It is the *derived by* in the sentence above. Read `*` as a spinning wheel rather than as a multiplication sign. Its spokes show one thing being turned into another.

Nothing there is memorized. The names are ordinary English words, and the rule that joins them never changes. Knowing those five parts is knowing this plot, and every other statistical chart in this book is the same five kinds of part in a different arrangement.

And one law stands over the whole package: *no exceptions*. A rule you learn on one mark holds on every mark, or the package has failed you.

What the grammar refuses

The *no exceptions* law is easy to read as *everything combines with everything*. Here is the other half of it, and it is the half that matters more.

Hangeul can write the crowing of a cock. Hangeul cannot write $\neg \cup$ (*g* then *n*). A syllable is not a pile of letters; it is a set of slots, an initial consonant, then a vowel, then an optional final consonant, and each class of letter belongs to one of them. Two consonants with no vowel between them fill nothing. $\neg$ on its own is silent, and $\cup$ (*a*) cannot stand alone either. To write that bare sound you supply a carrier for the empty initial slot, and what you write is $\circ \cup$ (*a*). The script that boasts it can capture the barking of a dog refuses most of the arrangements you could make of its own letters.

Those are not two facts. They are one. If the slots were removed, and any letters could stack in any order, the script would not become more expressive. It would become unreadable. A block would no longer tell you how to say it, and you would return to memorizing whole shapes, which is how writing began. The refusals are what make the script readable at first sight. A reader meeting a word for the first time still knows which part is the consonant, because nothing else could have been in that slot.

A plot is the same. **gog** will not draw a **point** from an **x** alone. A mark with one position has not said where to put it. **gog** will not map a column to **shape** on a **bar** either. A bar is a rectangle, and there is no other shape for it to be. Neither refusal is a missing feature. Both exist for the same reason: they let you read a specification you have never seen before. If every mark accepted every channel, knowing the mark would tell you nothing. A plot would again be something you memorize. The engine says no out loud, and every no points to what you meant instead. That is why this book renders its refusals live, on the page, beside the plots that work.

The goal is that one small set of atoms draws every plot, the way one small set of letters writes every Korean sound. Neither is a claim of completeness.

Hangeul does not write everything. It does not mark tone, and it does not mark how fast a syllable is said. Some sounds it cannot spell at all. Those gaps were left as gaps, because the sounds behind them are not the ones most speakers need. The letters were not built to be perfect. They were built to be a means of writing sound.

gog is built the same way. It does not promise that every visualization is possible. Reaching some of them would mean stepping outside the grammar's own rules and adding an exception, and no chart is worth that.

What the grammar will not do is refuse a plot for being a bad idea. **꺄** (*kkwek*) breaks no rule of Korean, and no one has ever needed to write it. Its graphics equivalent is a color scale (the rule that turns a column's values into colors) stretched over three hundred categories. That plot is legal here, because well-formedness and usefulness are different judgments, made by

different judges. The engine guarantees well-formedness, always. Usefulness is yours, and it is what most of this book is about.

The promise, and the book's rules

This book inherits the 1446 promise, restated for graphics: read the whole grammar in a morning; write it fluently within ten days. Part I is built to be finished in one sitting. The rest of the book deepens what the morning introduced; it never replaces it.

Five rules govern every page, and you can hold the book to them:

- **Every plot is live.** Each is drawn by the engine as the page builds; no plot in this book is a screenshot. The only static images in this book sit in this preface (the *Hunminjeongeum* page, the mouth saying $\neg$, and in the web edition the cover and the DOI badge) and none of them is a plot.
- **Questions first.** Every teaching chapter opens with a question about data, never with a feature name.
- **A small cast.** Eight families of table carry more than two out of three plots in this book. A chapter that needs a frame of its own introduces it where it is drawn. New columns cost the reader attention, so the variety stays in the questions.
- **Read it aloud.** The first specification in every teaching chapter is followed by its English sentence. Read that one aloud, and the rest of the chapter reads the same way. If a sentence cannot be read aloud cleanly, that is a bug in the grammar, not in you.
- **Errors on stage.** Refusals render live in these pages, because the engine's *no*, always with a direction, is half of what it teaches.

One thing to be clear about before you start: this package is young. The version number is 0.1.0, and it means what it says. The grammar is settled enough to build on, and the implementation is not finished. What exists is a first working version, so do not expect everything to work the first time. If you use it for an experiment or a paper, you are taking a risk. This warning is here so that the risk is one you chose, and not one you found out about later.

Expect it to move quickly. New features will arrive often. New atoms will not: the kernel is meant to stay small, and it grows one decision at a time. A plot you write today may have a shorter way to say it next month. What I will try hard not to do is take anything away. A sentence that draws today should still draw a year from now. If that ever has to break, I will say why rather than let it break quietly. I cannot promise it, though, and below version 1.0 nobody honestly could.

This book moves with the package. Because every plot here is live, a change to the engine reaches these pages the next time they are built. The text is rewritten whenever it stops matching. Expect the book to change as often as the package does. What each release changed is recorded in the package's [changelog](#).

One consequence matters before a sentence refuses on your machine. These pages are built from the newest source, and a released package is not. So the book can describe something your copy does not have yet. R is the closest, because its builds follow every change and a fresh install has them. Python, JavaScript and Julia move only when a new version is published. If a sentence here refuses for you, and the message does not know a word this page uses, check your version before you doubt the page.

Please [report the bugs](#) you find. A package this young improves mainly by being used, and that is what I am hoping for.

One kind of report is worth more than the others, and it is the one I cannot write myself. Take a dataset you know well and draw it in `gog`. Then draw the same thing in a tool you already trust. Any tool at all: [ggplot2](#), [matplotlib](#) or [seaborn](#), [Excel](#) or [Tableau](#), [Stata](#) or [SPSS](#), [Vega-Lite](#), [Plotly](#), [Bokeh](#) or [D3](#). Put the two pictures side by side and tell me where they differ. A standard dataset is ideal, because the picture it should produce is already familiar to everybody.

Here is why that comparison is worth so much. This package tests itself thoroughly, but in one direction only. Every sentence in this book is drawn in all four languages, and the four files have to match to the byte. Every refusal on these pages is run again on every build. All of that proves `gog` agrees with itself. None of it proves the picture is right. A box whose whiskers reached the wrong value would pass every test this project has, four times over, in four languages. This book would print it on the page as though it were correct.

A difference is not automatically a defect, and it is fair to know that before you spend the time. Some differences are decisions. A `gog` box spans the quartiles, as most software does; Tukey's original spans the hinges, and the two agree on most samples. The [box](#) chapter says so and says why. So a comparison gets one of two answers: this is a bug and thank you, or this is a choice and here is the reasoning behind it. Both are worth the message.

Not everything reported will be built. I am this project's [benevolent dictator for life](#). The title is a joke, and each word in it means something. *Dictator*, because the founder decides instead of the project voting. *For life*, because there is no election to lose. *Benevolent*, because the decisions are expected to serve the users rather than the founder. It was first used in 1995 for [Guido van Rossum](#), who created Python. Many people would say I am not qualified to hold it. I am holding it until this package is in the hands of a user community and of people better than me.

The cast

The third rule promised a small cast: eight families of table. Meet them once, here:

- `gapminder_2007`, 142 countries in 2007: `gdp` (per capita), `life` (expectancy in years), `population`, `continent`, `country`. The table this book uses most, and an excerpt of the

Gapminder Foundation's data (Bryan, 2025). `gapminder_asia` is five Asian countries traced 1952–2007, and `gm_all` is every country and every year.

- `iris_flowers`, 150 flowers: `sepal_length`, `sepal_width`, `petal_length` (all in cm) and `species`. Three measurements on one scale, and a category that separates cleanly. The measurements are Edgar Anderson's, and they reach most software through Fisher's 1936 paper (Fisher, 1936) and R's own `datasets` package (R Core Team, 2026).
- `medals`, five countries' Olympic gold, silver, bronze counts. Small enough to check by eye.
- `actuals` and `forecast`, yearly sales: history in one table, expectation in another. The two-table pair.
- `six_weeks`, 42 consecutive days of orders, with `day` a real Date, for calendar axes.
- `winds`, 264 wind observations: `direction` (eight compass points), `bearing` (in degrees), `speed` and `season`. The wind rose in `polar` reads it, and so do the book's reference-line and band examples.
- `titanic`, the passengers and crew of the Titanic as 32 groups: `class`, `sex`, `age`, `survived`, and `n`, how many people were in the group. R's own classic table (R Core Team, 2026), reshaped to long form, and the book's table of movement: every flow diagram reads it.
- `trade_partners`, 22 trade relations between 15 economies: `exporter`, `importer`, and `billions`, the value of the goods traded, in billions of dollars, rounded. The book's one table of relations, and every network diagram reads it.

Those eight families answer most of the questions asked here. A handful of chapters need a shape none of them has, and each brings its own small frame, introduced on the page that first draws it rather than in this list. A mountain's elevation grid for `surface`. Two gliders' coiling flight paths for the third dimension in `space`. Country boundaries for `map`, and earthquakes under the Pacific for `globe`. Foods and their nutrients for the `cluster` transform. Thresholds for `rule`, and shaded spans for `zone`. None of them arrives to add variety. A new column costs the reader attention, so the variety in this book stays in the questions.

How this book runs

The shape of the book borrows from the *Hunminjeongeum* itself. That book stated its whole system in a few pages, then explained each letter class, then showed assembly, then ended with ninety-four ordinary words in use:

1. **A morning**, the whole grammar, fast: a first plot, the kernel on one page, reading practice, writing practice.
2. **The letters**, each atom family in depth: the data you bind once, the marks, the scales, the refining channels, setting vs mapping, and the transforms.
3. **Sentences**, composition: layering, deriving, scope, faceting, whole plots arranged on a page, movement traced from place to place, likeness computed into a tree, the third dimension, the plane bent into a circle, the panel packed into regions, the earth flattened,

the earth kept round, relations drawn as a network, a plot played in time, and the reader's own selections.

4. **The cookbook**, recipes organized by your data's shape and your question, never by chart name; each recipe differs from the next by a single atom.
5. **The reasons**, the nine laws the package never breaks, and why.
6. **The host language**, one short chapter apiece for [R](#), [Python](#), [Julia](#) and [JavaScript](#). A binding is the package that lets one language write `gog` sentences and send them to the engine. These chapters hold what belongs to the host language rather than to the grammar.

Six appendices follow. The [kernel card](#) is the whole vocabulary on one page. [Functions around the grammar](#) documents the few helpers that stand outside it, `gog_table()` among them. The [combination grids](#) cross marks with channels, with transforms and with spaces, and transforms with each other. Each shows what combines and what the engine refuses, generated from the engine's own rule table on every build so it cannot drift from the code. A [dictionary](#) runs from chart names to sentences. A [phrasebook](#) translates `ggplot2` into `gog`. And [coverage](#) closes the book, listing what `gog` and `ggplot2` each draw that the other does not.

This book is written in [R](#). [Python](#), [Julia](#) and [JavaScript](#) speak the same sentences into the same engine. The four binding chapters, [R](#) included, are the whole of the difference. They cover how each language captures a column name, and, for JavaScript alone, the four words it needs because it cannot give `+` `*` `|` `/` a meaning of its own. Most code blocks are executed as the page builds: the plots you see are drawn by the engine at that moment.

Installing gog

You have watched the engine draw every plot on this page. All four bindings work. Copy the command for your language, and read the notes that follow.

R

```
install.packages("gog", repos = c("https://psychometrician.r-universe.dev",  
                                   "https://cloud.r-project.org"))
```

Python

```
pip install gog
```

Julia

```
using Pkg; Pkg.add("GrammarOfGraphics")
```

JavaScript

```
npm install grammar-of-graphics
```

The package has two names, and the registries are the reason rather than the grammar. In R and in Python it is `gog`, because a three-letter name is allowed in both. Julia requires a package name of at least five letters, in capitalized words, and npm already had a package called `gog`. So those two spell the name out instead, each in its own style: `GrammarOfGraphics` in Julia, and `grammar-of-graphics` on npm, which does not allow capital letters.

The Python and JavaScript packages carry the engine inside them, already built for your computer, and the R package carries it on macOS and Windows. For those you do not need a Rust (Klabnik & Nichols, 2023) toolchain, and nothing has to be added to your `PATH`. Two of the four commands have a note to read before you run them: the R command and the Julia command.

The R package is not on CRAN yet. It is published on [r-universe](#) instead, which is why the `repos` argument names two addresses. The second address is an ordinary CRAN mirror, and it is there so your other packages still install as usual. On Linux the command builds the engine from source, which needs Rust and a network connection. The [R](#) chapter gives the command that installs Rust.

The Julia package is the one binding that does not carry the engine yet: it installs and loads, but it cannot draw until you add the engine. One more command installs an engine, and the [Julia](#) chapter gives it.

Each binding has its own chapter at the end of this book, and each of those chapters closes with the longer version of this section: [R](#), [Python](#), [Julia](#) and [JavaScript](#).

Every one of these packages is at version 0.1.0. The grammar is settled and the engine draws this whole book, but the packaging is new, so releases will be frequent, and most of them will not change the vocabulary. One promise holds at every version: every page shows what the engine actually did. If a page shows a sentence, the engine drew it. If a page shows a refusal, the engine refused it.

The prose in this book is licensed [CC BY-NC-SA 4.0](#). The code is licensed [Apache 2.0](#), the same license as the engine, so the examples can be used in your own work.

This book is archived on Zenodo and can be cited by its DOI, [10.5281/zenodo.21807652](https://doi.org/10.5281/zenodo.21807652). That address always resolves to the newest deposited edition. The archived copy stays available even if this site does not.

Who wrote this

This book carries five names, and the division of labor behind them is worth stating plainly rather than leaving in a footnote.

One of the five is not a name. I publish as **psychometrician**. I learned what a pen name was in 1992, when I took introductory statistics in the psychology department. That is where I met *Student*. Student was [William Sealy Gosset](#), an English chemist and statistician, and the t-distribution carries his pen name instead of his own. My own papers are few, and no academic journal would let me publish under a pen name. This book is the first chance to use one.

The word does a second job. Most people do not know the job title, and anyone who uses this package now knows that a [psychometrician](#) started it and maintains it. What they are less likely to know is how much psychometricians care about data visualization. As far as I know, only one academic journal runs a visualization competition, and it is ours: [Educational Measurement: Issues and Practice](#), published by Wiley for the [National Council on Measurement in Education](#) (NCME). Every year it asks its members for thoughtful and creative diagrams that communicate a clear message about educational or psychological data. The winning entries become the journal's covers, and the next best entries are shown inside. The [cover gallery](#) is worth your time. This package may not be able to draw all of them.

Psychometricians have cared about data visualization for a long time. My own history with it starts in 2005. I was writing my dissertation, and every conditional plot in its simulation chapters was drawn by the `lattice` package (Sarkar, 2008). It was the first graphics tool that impressed me. After that I was a heavy `ggplot2` user for years. What changed was the day I understood that `ggplot2` had a grammar. From then on it felt like there was no end to the plots I could make.

I owe [Hadley Wickham](#) more than a citation for that. Before his package and his book, a plot was a task to get through. You looked up the function for the chart you had been asked for, learned whatever arguments it happened to want, and moved on. The grammar changed what the work was. Visualization became a creative act rather than an item on a list. It is creativity inside a boundary, the one the grammar draws. That is the part I most want to pass on.

The design is mine. The founding question, the Hangeul discipline, the nine laws, and the ruling at every fork about what this grammar would and would not become: those are the parts I am answerable for. The hex sticker is mine as well. I designed it myself in [Canva](#), with no sticker package. I am not an artist, and what I wanted was not a beautiful sticker but an unmistakable one. The Rust engine was written entirely by the four Claude models on the cover, as were all four language bindings and the test suites. They also did the work nobody is ever thanked for, which is running every sentence in this book through four languages, again and again, until all four agreed to the byte.

The text is a third case, and the honest comparison is a ghostwritten autobiography. The public figure whose name is on the cover usually did not type it. A professional writer interviews them, and what comes back is their thinking in another person's prose. These pages work the same way, with one difference. Not every idea in this book started with me. Claude Code often suggested something better than what I first brought, and my task was to accept it or reject it. Claude Code put all of it into English. The sentences are not mine; the decisions behind all of them are.

Naming them as co-authors is meant literally. A line reading "with assistance from" would understate the work many times over. This whole book teaches you to say exactly what you mean, and the cover follows the same rule.

They were not one kind of collaborator, and it would be a poor tribute to pretend they were. On some days they were assistants: fast, tireless, and so eager to be useful that "done" tended to arrive some time before done did. A change to how bars are drawn was once confirmed by checking a stroke color instead of looking at the plot. The report said all was well while every bar on the page was still wrong. Most of the checks now guarding this repository exist because of an afternoon like that, and not one of them was added to catch a careless human.

On other days they were advisors, and that is the part I did not expect to be writing about here. A design question asked at the wrong hour, asked badly, asked for the third time in a worse form than the first: it got taken as seriously as if it were a good one. A fair number of the refusals in this engine exist because I said "why not just" and got a patient answer rather than an agreeable one. Some of the reasoning in [Part V](#) was settled in an argument I was not winning. An assistant that agrees with you is the easy thing to build. This grammar would be a good deal worse if that is what I had had.

Taken together, it felt like a research meeting. Smart researchers filled the room, eager to listen to me and eager to carry out the plan once I had decided. I have not had both of those at once before. It was always a good time, and sometimes an expensive one. A small question would open into a long detour, and I would lose several hours in it. Accepting that invitation was my decision every time, so none of this is a complaint.

Responsibility does not divide the same way. Every error in these pages, every refusal the engine gets wrong, and every design decision that turns out to have been the wrong one is mine, because I chose it and I kept it. I organized this project and I maintain it. The credit is shared; the fault is not.

One last item belongs in an honest accounting. I work as a psychometrician, and this package was never part of that job. It was built late at night, on weekends, and on vacation days. That time belonged to my wife and daughter, and I spent it talking to the four Claude models on the cover instead. I am not going to call that devotion to the craft. To my wife and daughter: sorry. I would like to end by saying it is finished. It is not. The version is 0.1.0, and what comes next will cost the same nights, weekends and vacation days.

Thanks to Posit

[Posit PBC](#), called RStudio PBC until 2022, builds a large part of what a working R user touches in a day. The parts I depend on are free, and there are three of them.

The [tidyverse](#) (Wickham et al., 2019) is the one I would miss first. It is the vocabulary I think in when I handle data, and has been since long before this project. You will not find it in this book, whose tables are built in base R because `gog` imports only `jsonlite` (Ooms, 2014) and I want it to install anywhere. That is a rule for the package. It is not how I work the rest of the day.

Their editors are the second. I worked in [RStudio](#) for years, and I work now in [Positron](#), which is newer and holds R and Python in one window. The two are not the same program under different names: RStudio is still here and still maintained, and Positron sits beside it.

[Quarto](#) is the third, and it belongs here rather than in a colophon, because the first of the five rules above rests entirely on it. *Every plot is live* is a claim about honesty rather than convenience. Every code block on these pages ran while the page was being built. One that draws a plot really drew it, and one that shows a refusal was really refused. Quarto is what executes those blocks. Without it, every plot here would be a screenshot and every code block would be text that nobody had run. The book would slowly stop matching the package it describes, and no one would notice.

Quarto gives the book three more things it could not have faked. Four languages spell the same sentence in one tabbed block, so a reader sees the parity instead of taking it on trust. One source builds both an HTML site and a printable PDF, Korean letters and all, from the same executed chunks. And a block is allowed to fail without stopping the build, so the engine's refusals appear beside the plots that work instead of in an appendix.

None of that announces itself while you read, which is the mark of a tool doing its job well. Without Quarto there would be no book here at all.

I will not be modest about what the [tidyverse](#) and their editors are worth either. My work as a psychometrician would be harder, and a good deal more miserable, without what this company gives away. Thank you.

Thanks to Anthropic

Soon after Claude was released, I built a web application. It was [React](#) on the front, [Supabase](#) for the database, and the [Google APIs](#) behind it. I did not know React well, and I did not know Supabase at all. The application got finished anyway.

Then it failed, and the reason had nothing to do with the code. I could not market it. I did not know how to reach the people it was built for, and after a while I shut it down. The application

is gone. Its tutorial videos are still online: eighty of them, in most major languages. Two are the English ones, if you would like to see what it did: [tutorial one](#) and [tutorial two](#). Claude Code did the translating for the rest, and with it I could do the same for this book.

The lesson was not the one I went looking for. Building the thing was no longer the hard part. If I plan carefully, keep going, and work through the problem with Claude Code, I can finish what I set out to build. So I began to want something else: my own packages, in the two languages I actually work in, R mainly and Python occasionally. This package came from that wish.

The application left me a skill as well. It is the one part of this project I did not ask Claude Code for. A web application is mostly interface, so I had to learn to design one. I drew every screen on paper before it was built. I did the user-experience work myself, and tested it over and over. This package asks for almost none of that skill. A plot is not an interface, and the engine has no screens. The exception is [selection](#). The controls under a brushed plot are mine. So are their names, what a single click does, and which pointer means which mode. It is a very small piece of design next to that application. I hope you like it anyway.

A grammar of graphics is a larger promise than a web application. It is an engine, and an engine has to be written, tested, and kept working. I am a psychometrician, with no team on this project, no funding, and no ability to write Rust. Without [Claude Code](#), this package would still be a plan.

One more item belongs on that list. English is not my first language. Every conversation behind this package was held in my own broken English, from the first design question to the last bug report. I chose the wrong words and put them in the wrong order. I left out the small words that hold an English sentence together. Claude Code understood what I meant nearly every time. Twenty years of thinking about this field reached the engine through sentences full of mistakes. None of that thinking was lost on the way. You can do better than I did.

I think this may have been luck, and I would rather say so than not. I happened to be thinking about data visualization. I happened to be thinking about Hangeul. Then Claude Code arrived. What I did was stand between the two and let them meet.

There is nothing rare about that position. Anyone could be standing in it. If you have been thinking about your own subject for a long time, and you are willing to play with this tool, you are already there. Much bigger things than this package will come out of that.

So, thank you to the people at [Anthropic](#) who built Claude Code, and to the four models named on this book's cover. I do not know most of your names. I know two of them from YouTube: [Boris Cherny](#) and [Cat Wu](#). You built the thing that let a psychometrician write a graphics engine in Rust. How cool is that? Thank you.

Part I

A morning

In 1446, the book that introduced Hangeul (한글) opened by stating the entire system in a few pages (every letter, every assembly rule) before a single page of explanation. The scholars who wrote its postface made a claim about that brevity: *a wise man can learn the letters before the morning is over*.

This part makes the same claim. It is short on purpose, and it is meant to be read in one sitting: a first plot, then the kernel on one page, then practice. The practice has two halves: first reading finished sentences, then writing your own and meeting the refusals you are most likely to see.

Nothing in this part is exhaustive. Nearly every idea you meet here has a full chapter later in the book. The goal of the morning is different. When it is over, you should be able to read any *gog* sentence, in this book or in a colleague's script.

1 A first plot

Do people in richer countries live longer?

That is a question about data: 142 countries, each with a GDP per capita and a life expectancy.

Table 1.1: `gapminder_2007`: first 5 of 142 rows

country	continent	year	life	population	gdp
Afghanistan	Asia	2007	43.828	31889923	974.5803
Albania	Europe	2007	76.423	3600523	5937.0295
Algeria	Africa	2007	72.301	33333216	6223.3675
Angola	Africa	2007	42.731	12420476	4797.2313
Argentina	Americas	2007	75.320	40301927	12779.3796

In `gog` you answer a question like that by writing what the picture should show, rather than by drawing anything:

R

```
data(gapminder_2007) + point + x(gdp) + y(life)
```

Python

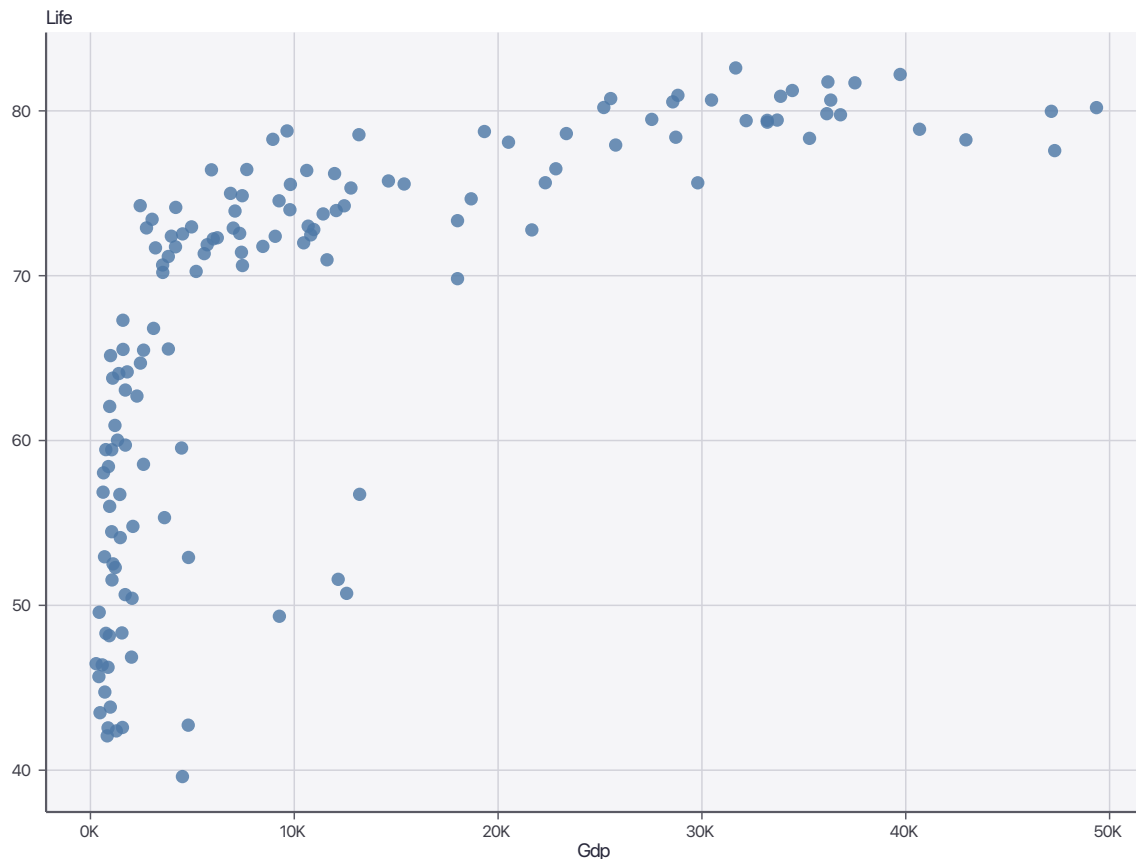
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life))
```



Read it aloud: “*Given gapminder 2007: points, x is gdp, y is life.*”

That sentence is the entire program. You named a table, a mark, and two positions; the engine chose the axes, the ticks, the margins, and drew one point per row. You never call a drawing function, because a plot here is a *specification*: a sentence about the picture, not a claim about the data. Rendering it is the engine’s job, not yours.

None of those four words is yours to define. They arrive with the package, so loading `gog` is what makes `data`, `point`, `x` and `y` available. A word is written bare when it has nothing to take, which is why `point` has no parentheses. Parentheses appear when there is something to put in them, like the column in `x(gdp)`.

That sentence has four spellings, and this edition prints all of them under every example: the R above, then Python, Julia and JavaScript. They are not translations of each other. Each is its own language writing the same specification, which one engine then draws, so the plot below is rendered once, and any of the four would produce the same file. Read whichever is yours and ignore the rest; nothing else in this book is language-specific.

They are in that order because the first three are very nearly one sentence. The operators are the same and only the way a column is named changes: a bare `gdp` in R, `col.gdp` in Python, `:gdp` in Julia. JavaScript is the one that reads differently, because it cannot give `+` `*` `|` `/` new meanings and spells them as words instead. It comes last, so the pattern the other three share is seen first. Each language also has a chapter of its own at the end, a difference list about a page long: [R](#), [Python](#), [Julia](#), [JavaScript](#).

1.1 Three steps to run any example

Everything in this book runs on your own computer, and it takes three steps:

1. **Install.** The command for your language is in [Installing gog](#).
2. **Load it.** `library(gog)` in R, `from gog import *` in Python, using `GrammarOfGraphics` in Julia, and an `import` from `grammar-of-graphics` in JavaScript.
3. **Read a table.** `gog_table("gapminder_2007")` fetches the one above; assign it that name, and `data()` takes it. It comes with the package, and [Reading a table](#) shows the line in all four languages.

Every example after this one works from those three steps, and the table each needs is named in the sentence itself.

1.2 Add a word, ask a richer question

Which continent is which? Say so, one more word in the sentence:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent)
```

Python

```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
↪ color(col.continent)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
     color(col.continent))
```



“...and color by continent.” The `color()` channel maps a column to a visual property. A mapping raises a question (*which continent is this dot?*), so the engine draws the legend that answers it. You never asked for a legend. A mapped column earns one when there is something to decode.

1.3 An incomplete sentence is refused

What happens if you forget a position? An incomplete sentence is refused the way a syllable missing its vowel cannot be pronounced, and the refusal tells you what is missing:

```
data(gapminder_2007) + point + x(gdp)
```

Error:

```
! gog: `point` needs `y()` but none is set. Add `y(<column>)` - a point
  ↳ cannot be drawn without it.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

You meet this on your first page because refusals are half of what makes **gog** learnable; the small vocabulary in the next chapter is the other half. A **binding** is a channel with a column in it, like `x(gdp)`. The engine never accepts a binding only to silently drop it, and every *no* comes with a direction. The errors here do the teaching; the [writing practice](#) chapter collects the ones you will meet first.

1.4 Refine, don't rebuild

The **gdp** axis is crowded on the left: most countries have far lower incomes than the richest few. A logarithmic view is a property of *how x is read*, so it is written on the binding, and nothing else changes:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +
  color(continent) +
  title("The health and wealth of nations, 2007")
```

Python

```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +
  color(col.continent) +
  title("The health and wealth of nations, 2007"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +
  color(:continent) + title("The health and wealth of nations, 2007")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),
     y(col.life), color(col.continent),
     title("The health and wealth of nations, 2007"))
```



“Given *gapminder 2007*: *points*, *x* is *gdp* on a *log* scale, *y* is *life*, color by *continent*.” Three refinements later, the sentence still reads left to right, and the four words you learned first are unchanged. The last word, `title()`, puts a title above the plot. It is a label: a word that names the plot or an axis instead of binding a column, and the [Grammar](#) chapter lists the four of them.

1.5 The shape of every sentence

Every plot in this book, through the globes, networks and brushed plots at the end, is this same shape:


```
data(table) + mark + positions + refinements
```

A table bound once, so columns are bare names. A mark, the visible form. Positions, where the marks stand. Then any number of optional refinements: channels like `color()`, and labels like `title()`.

Two words describe putting a column into a channel, and the book uses both. A column is **bound** to a channel, and the binding is where a scale gets written. The same act is called **mapping** to emphasize that a channel received a column rather than a fixed value. The difference shows on the page: a mapped column earns a **guide** to decode it, and a value you merely set earns none. A table is bound too, by `data()`, which is the Bind-Once law: name it once, and its columns stay bare names.

Two more parts join with an operator of their own. A transform attaches to its mark with `*`, which is how the preface's `bar * bin` made a histogram. A `facet` splits the finished plot into panels with `|` or `/`. A scale is not a separate part at all: it is written on the position it belongs to, the way `x(gdp, scale = "log")` was written above. So the shape in full is:

```
data(table) + mark * transform + positions + refinements | facet(column)
```

The transform and the facet are optional, and so are the refinements. Delete every one of them and a legal plot remains. What cannot go is the table, the mark, and the positions that mark needs. That is the sentence you wrote at the top of this chapter.

Written grammar works the same way, which is why this one is called a grammar at all. “Birds sing” is already a complete sentence in English. “The small birds sing loudly in the morning” says much more, and every word added to it can be taken out again. Remove the verb instead and there is no sentence left to refine. A plot is not a different kind of object. Some of its parts are required, and the rest refine it. That division is the same in every plot in this book, so you learn it one time.

The next chapter states the kernel on one page, and the page is short.

2 The grammar

What do you have to say about your data before a plot can be drawn? `gog`'s whole vocabulary is a short list of ordinary English words. Learn them once and you can read, and write, any plot. This book calls each word an **atom**, a piece that nothing divides further.

This chapter is a small dictionary, and a dictionary is not read the way a story is. Nobody memorizes one before writing, and these lists ask for no memorizing either. Draw your own data with the few words your question needs, and the plot shows you what each one does. Each plot you write builds the skill, and the skill brings more of these words into use.

2.1 The kernel

Kind	The words
Sources	<code>data query</code>
Marks	<code>point line area bar step interval box ribbon text path rule zone surface edge</code>
Channels	<code>x y z color size shape pattern opacity group label play</code>
Transforms	<code>bin smooth count density proportion sum mean median max min quantile range confidence deviation bounds partition flow layout cluster, and the four collision modifiers, dodge stack jitter repel</code>
Operators	<code>+ layers, * derives, and / arrange</code>
Scales	<code>linear log time category order</code>
Spaces	<code>flat space polar nest map globe network</code>
Settings	<code>style theme palette</code>
Labels	<code>title x_label y_label z_label</code>
Facets	<code>facet</code>
Selections	<code>brush</code>

Every word in it is a common English word: no abbreviations, no acronyms, no silent letters. A common word is often taken already. Several of these name something in R, Python or Julia before `gog` is loaded, and [the last section of this chapter](#) says what happens then.

Read the kernel as two groups: what a plot needs, and what refines or arranges it. Five kinds build a plot. A **source** names the table that supplies the rows and the columns. A **mark** is the shape drawn for each row. A **channel** maps a column to something you can see, such as a position or a color. A **transform** derives new values from the columns you named. An **operator** joins the words into one sentence.

The other six refine a plot or arrange it, and none of them is required. A **scale** decides how a number becomes a position. A **space** is the surface the plot is drawn on: a flat page, a cube, a circle, a map, a network. A **setting** changes how the plot looks without saying anything about the data, and a **label** names it. A **facet** cuts one plot into panels. A **selection** picks rows out of a plot that is already drawn.

So **the minimum sentence** is a source, a mark, and the position channels that mark needs:

```
data(table) + mark + x(column) + y(column)
```

Two promises hold across the whole kernel. The engine draws every word in it, though not yet every *pairing* of words. And **gog** never accepts a word and then quietly ignores it: a sentence it cannot draw is refused, and the refusal names the rule. See [Design laws](#) for the difference between **illegal** (the grammar forbids it) and **unsupported** (the grammar allows it; the engine cannot draw it yet).

2.2 The colors of the code

In every specification in this book, each word of the kernel is printed in the color of its kind. These colors belong to the code, never to a plot. They are not decoration: they show the two groups above.

Four of the five kinds that build a plot are words, and each takes a color of its own. The fifth is the operator, which is punctuation in most of these languages, so it is set in heavier type instead of a color. The six kinds that refine or arrange a plot share one muted color, because none of them is required.

Your own names keep the color of the text around them. That is the first thing the coloring tells you: which words belong to **gog**, and which are yours.

R

```
data(gapminder_2007) + bar * bin + x(life) + style(color = "tomato")
```

Python

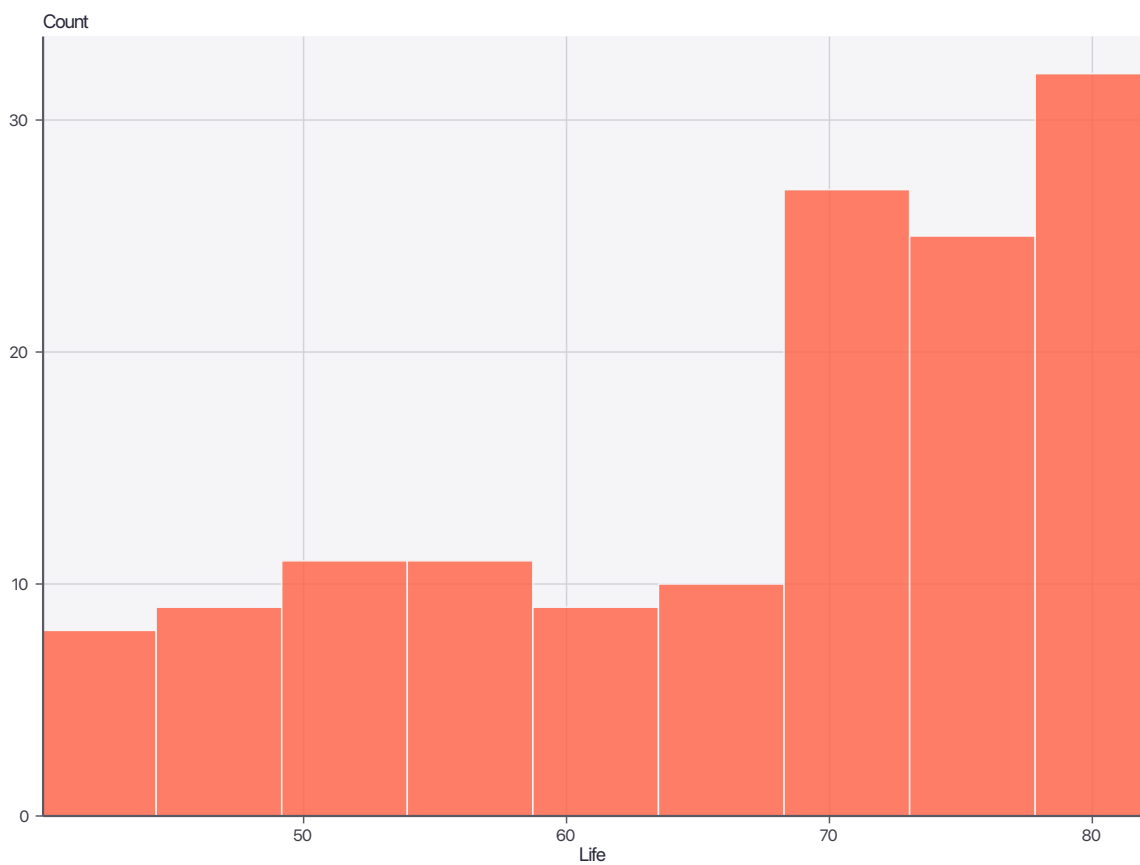
```
data(gapminder_2007) + bar * bin + x(col.life) + style(color = "tomato")
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) + style(color = "tomato")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life),  
  style({ color: "tomato" })))
```



“Given gapminder 2007: bars derived by bin, x is life, colored tomato.”

`data` names the table, `bar` is the mark, `bin` is the transform, and `x` is the channel. Those four build the plot. `style` refines it, so it takes the quiet color, and `gapminder_2007`, `life` and `"tomato"` are yours.

Inside `style`, the word `color` stays the color of the text. It names a property here rather than a column, so it is not the `color` channel: a word you write as an argument name is never one of `gog`'s words.

An ordinary syntax highlighter could not have drawn these colors. It colors code by the host language's own categories, and the four languages disagree about `gog`'s words. R sees `data` as a function, Python sees a plain name, and none of the four has a category for a mark. So the four spellings of one specification would be colored four different ways. Coloring by kind instead gives a word the same color in all four languages. Your own editor highlights the ordinary way, so do not expect these colors there: on your screen a `gog` sentence looks like ordinary code.

Each kind has a section below, in the order of the kernel above: the five that build a plot first, then the six that refine or arrange one. Each section gives the words and what they do, and names the chapter that teaches them in full.

2.3 Sources: where the rows come from

Every sentence begins with a table, and a source is the word that names one. There are two.

Source	Where the rows come from
<code>data</code>	A table already in memory
<code>query</code>	A table a database returns, named by SQL

`query()` stands exactly where `data()` stands, and everything after it is unchanged:

```
query(connection, "SELECT ...") + mark + x(column) + y(column)
```

The SQL stays inside `query()`. The columns after it are still bare names, because a database returns a table, and every sentence begins with a table. See [Data](#).

2.4 Marks: the shape drawn for each row

A mark is the geometric form drawn for each row of data. There are fourteen, and you do not need to learn them all now. A few of them draw most of the plots in this book. Each mark has a chapter of its own in [Part II](#), where its plots show what a one-line description cannot.

Mark	What it draws
<code>point</code>	One glyph (a small drawn symbol, a dot by default) per row
<code>line</code>	Rows connected in <i>x order</i> : one value per x, read left to right
<code>bar</code>	Rectangle from baseline to value, vertical or horizontal
<code>area</code>	Filled region between a line and the baseline
<code>step</code>	A line that holds each value until it changes (cumulative curves, histogram outlines)
<code>interval</code>	A whisker spanning low→high at each x: error bars, ranges; needs a range transform
<code>box</code>	Distribution per group: box, median, whiskers to 1.5 times the interquartile range, outliers as dots; carries its own summary
<code>ribbon</code>	A filled band from a low boundary to a high one across x: confidence / spread bands; needs a range transform
<code>text</code>	A string at each position: its content comes from the <code>label</code> channel
<code>path</code>	Rows connected in <i>the table's order</i> : a route that may return the way it came; takes <code>style(arrow =)</code>
<code>rule</code>	A line at one position, spanning the other axis: reference lines, and rugs via <code>style(reach = "edge")</code>
<code>zone</code>	A shaded region: bounded where its sides are known, spanning the whole panel where they are not. Five things can fix the sides: a categorical position (each category gets its own slot: the tile plot), bounds (names them), bin (cuts them: the heatmap), density (cuts or traces them), and a boundary (the data draws them: the choropleth, in <code>map()</code>)
<code>surface</code>	A sheet above the x-y plane: one row per (x, y) crossing, with a face between each block of four, or one row per cut cell, drawn flat at its own height. The only mark that can be drawn in the cube by itself
<code>edge</code>	A stroke between two named things: one row per relation of a graph. Its endpoints come from <code>layout</code> , and it draws in <code>network()</code>

2.5 Channels: a column made visible

A channel maps a data column to a visual property of the mark. Adding one word to the sentence adds one thing the reader can see:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent)
```

Python

```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ color(col.continent)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent))
```



“Given *gapminder 2007: points, x is gdp, y is life, color by continent.*” You never asked for the legend. A mapped column earns one.

Channels do not all do the same work, and what separates them is what each one *earns*:

Channel	What it does	Earns
x	Horizontal position, from a numeric or categorical column	an axis
y	Vertical position, from a numeric or categorical column	an axis
z	Depth, the third position: it makes a plot 3-D. Drawn by point , bar , interval , box , path and surface	an axis
color	Fill or stroke color, from a categorical or continuous column	a legend
size	Radius, on point	a legend
shape	Glyph (circle, square, triangle) on point	a legend
pattern	Texture: a fill’s hatch, a stroke’s dash. Every mark but point , text and surface	a legend
opacity	Transparency, on point , bar , edge and surface	a legend
group	Splits a mark into one line per category, without encoding it. On line , area , step , interval , box , ribbon , path , zone and surface	nothing
label	The string a text mark draws, its content	nothing
play	Cuts the plot into frames and plays them in sequence, from a numeric or categorical column	a strip

The **Earns** column sorts the channels into four groups, and a Korean syllable shows how they fit together. It has three slots: an initial consonant, then a vowel, then an optional final consonant. The mark is the initial consonant, so the channels are the other two.

x, **y** and **z** are the **vowels**: a mark without one of them is silent, so at least one is required, and each earns an axis to be read against.

The five that earn a legend are the **final consonants**, the optional $\square$ (*m*) that turns 가 (*ga*) into 감 (*gam*). None is ever required, and each changes what the plot says, which is exactly why each needs a key to decode it.

`group` and `label` are neither. `group` splits without encoding: on a `line` it performs the very same `split color` does, one line per category, and then stops. Take away `color`'s visible differences and its legend, and what remains is `group`. So it works like the space between words rather than like a letter: it separates, but it shows nothing. `label` supplies a `text` mark's content rather than refining it.

`play` is the fourth kind, and it stands outside the syllable the way faceting does. It does not refine one picture. It cuts the plot into frames and plays them in order, so what it earns is a `strip` naming the frame on screen. See [Play](#).

Not every channel applies to every mark: a `line` is drawn with a single stroke, so it takes no per-row `size` or `opacity`. `gog` states the rule rather than guessing; see [Design laws](#), and [Combinations](#) for the whole mark-by-channel grid on one page.

2.6 Transforms: values the data does not hold

A transform derives new values from the columns you named, and attaches to a mark with `*`. The mark stays what it is: `bar * bin` is still a bar, drawn over counted bands.

Eleven of them replace values with a computed statistic.

Transform	What it computes
<code>bin</code>	Cuts a continuous column into bands and counts the rows in each
<code>count</code>	Tallies the rows in each category
<code>proportion</code>	The same tally, as a share of the whole
<code>density</code>	A smooth curve of where the rows lie
<code>smooth</code>	A trend line fitted through the points
<code>sum mean median max min</code>	Reduce a column to one value per group
<code>quantile</code>	Reduce a column to the value that a given share of the rows fall below, <code>quantile(0.9)</code>

Four produce a low and a high together, which is what a span needs:

Transform	What it computes
<code>range</code>	A band between two quantiles, the whole group unless you name them
<code>confidence</code>	The interval around the mean, which shows how exactly the mean is known
<code>deviation</code>	A band of standard deviations around the mean, which is how spread the data is

Transform	What it computes
bounds	A low/high pair the table already holds, in two named columns

Four are the **collision modifiers**: they compute nothing, and they move marks that would otherwise land on top of each other:

Transform	What it does
dodge	Puts them side by side
stack	Piles them on top of one another
jitter	Spreads them sideways along a category
repel	Moves labels off one another when they overlap

And four transforms compute an entire picture:

Transform	What it computes
partition	Divides a whole among nested parts: the treemap and the mosaic
flow	Carries a magnitude through its stages: the flow diagram
layout	Places a graph from an edge table: the network diagram
cluster	Joins the closest categories of an axis one pair at a time: the cluster tree. It also puts a tile plot's slots in a useful order

See [Transforms](#) for each one in detail, and [Combinations](#) for which marks take which.

2.7 The + operator: layering

+ joins one word to the next. Every sentence in this book is built with it, and the words can be read left to right in the order they were written.

R

```
# One layer
data(gapminder_2007) + point + x(gdp) + y(life)
```

Python

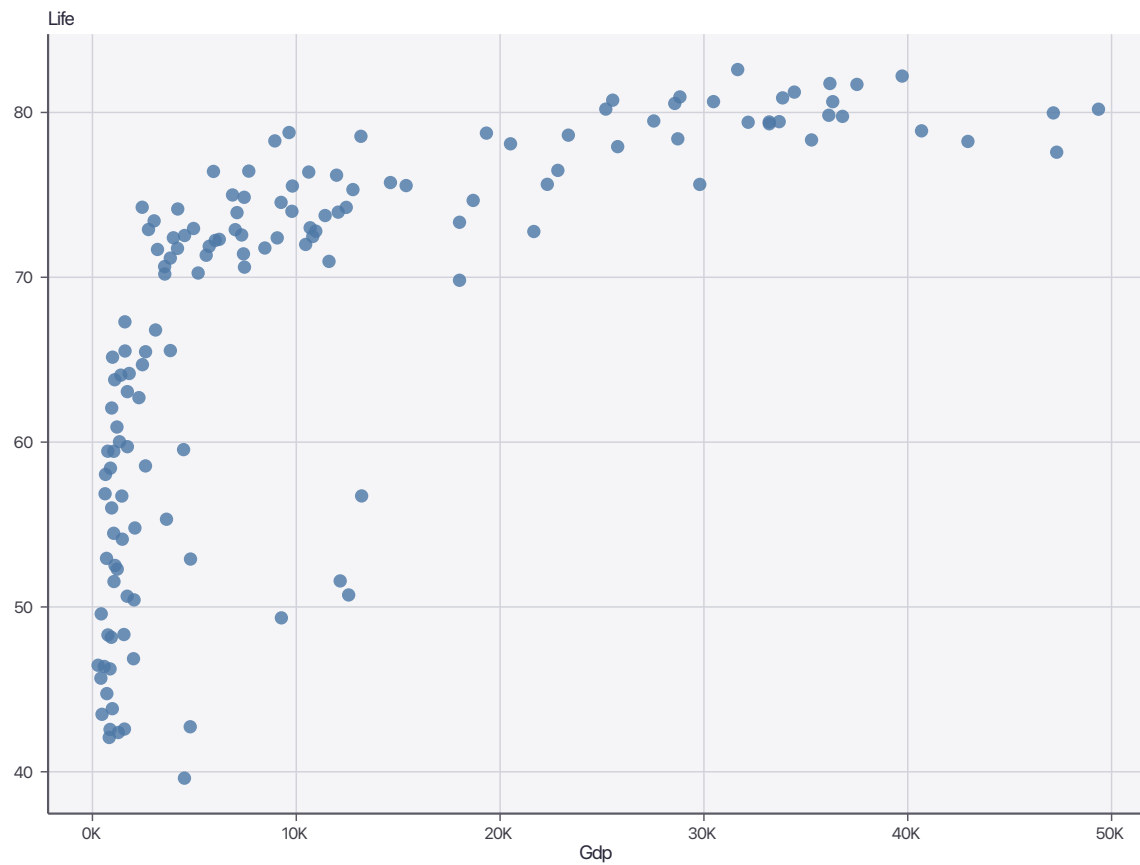
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life))
```



“Given gapminder 2007: points, x is gdp, y is life.” One mark, one layer.

R

```
# Two layers: bars behind, points on top
data(actuals) + x(year) + y(sales) +
  bar + style(color = "lightsteelblue") +
  point + style(color = "steelblue", size = 6)
```

Python

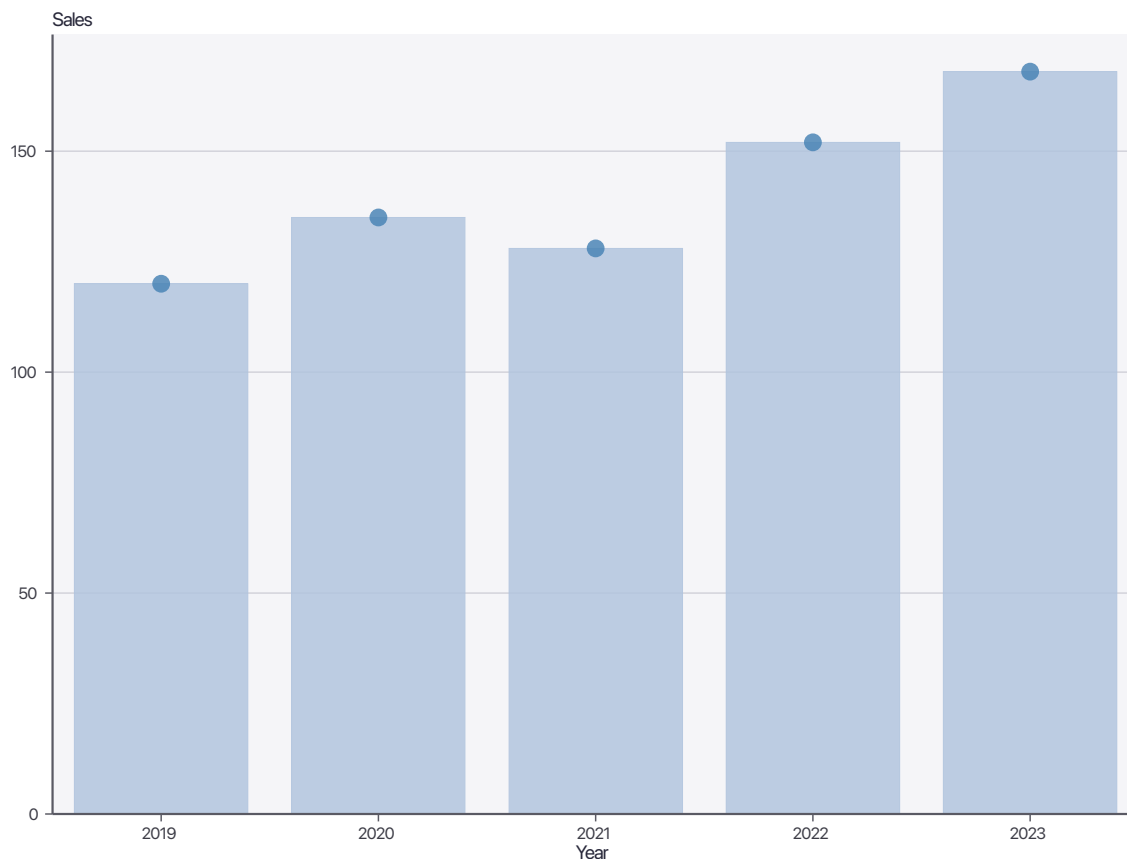
```
(data(actuals) + x(col.year) + y(col.sales) +
  bar + style(color = "lightsteelblue") +
  point + style(color = "steelblue", size = 6))
```

Julia

```
data(actuals) + x(:year) + y(:sales) + bar +
  style(color = "lightsteelblue") + point +
  style(color = "steelblue", size = 6)
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), bar,
  style({ color: "lightsteelblue" }), point,
  style({ color: "steelblue", size: 6 }))
```



“Given the actuals: x is year, y is sales, bars and also points.”

Each mark opens a new layer. Layers are rendered in order: first mark drawn first (at the bottom).

Order appears more than once in this grammar, and it does not mean the same thing every time. [Which orders matter](#) gathers all of it in one place: what you may rearrange freely, what the engine refuses, and the two arrangements that change the picture with no warning.

2.8 The `*` operator: combining

`*` binds a mark to a transform, creating a derived mark type. The mark is always on the left; the transform is on the right. Where two transforms follow one mark, write them in the order they happen: `bar * bin * mean` cuts the axis into bands, then averages inside each one.

```
bar * bin      # histogram
line * smooth  # smoothed trend line
zone * bin     # heatmap: the same `bin`, cutting both axes
```

The last line, `zone * bin`, shows the pattern: one transform means one thing, and the mark decides where it applies. In a histogram the bar’s height carries the count, so the count takes one axis and `bin` cuts the other. A `zone` carries its count by color instead, so both axes are free, and the same `bin` cuts both. When one of a zone’s positions is categorical, `bin` cuts only the other one, because a category is already cut into slots.

The six summaries (`sum`, `mean`, `median`, `max`, `min`, `quantile`) follow the same rule, read in the other direction. A summary groups the rows by every position the mark does *not* measure with. It then reduces the named column on the position the mark does measure with: `y` on a bar, `color` on a zone, `z` in `space()`. So `zone * mean + x(a) + y(b) + color(v)` averages `v` inside every cell two categories cross.

See [Operators](#) for the full rules, and [Transforms](#) for each transform in detail.

2.9 The | and / operators: faceting, or composing

`|` puts things side by side and `/` puts one above the other. Whether those are panels or whole plots is decided by what you write on the right.

With `facet()` on the right they split *one* plot into panels, one per category of the column it names:

R

```
# One panel per continent, side by side
data(gapminder_2007) + point + x(gdp) + y(life) | facet(continent)
```

Python

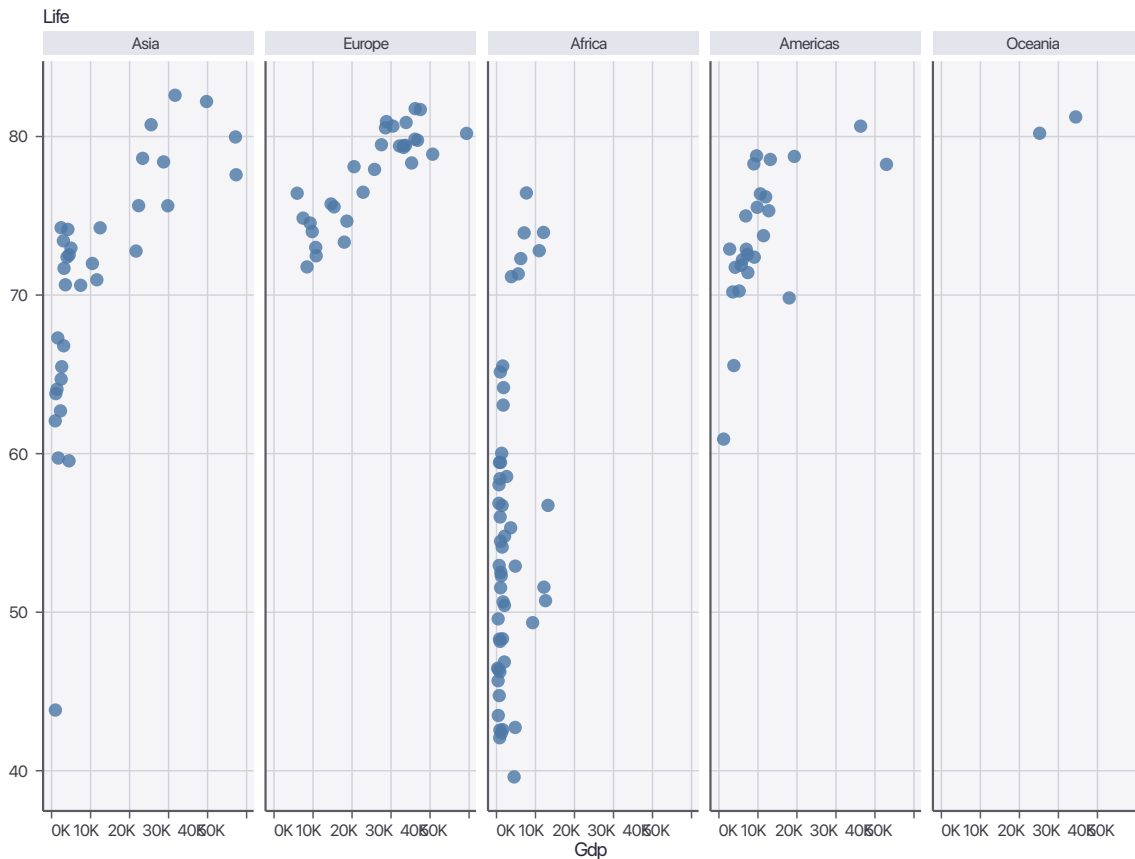
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) |
↪ facet(col.continent)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  across(col.continent))
```



“Given gapminder 2007: points, x is gdp, y is life, split into panel columns by continent.”

Every panel shares one scale, that is what makes them comparable, and both operators together cross into a grid. See [Faceting](#) for the full rules.

With another *plot* on the right they arrange the two on one page, each keeping its own coordinate space:

R

```
# Two plots, side by side  
(data(gapminder_2007) + point + x(gdp) + y(life)) |  
  (data(gapminder_2007) + bar * count + x(continent))
```

Python

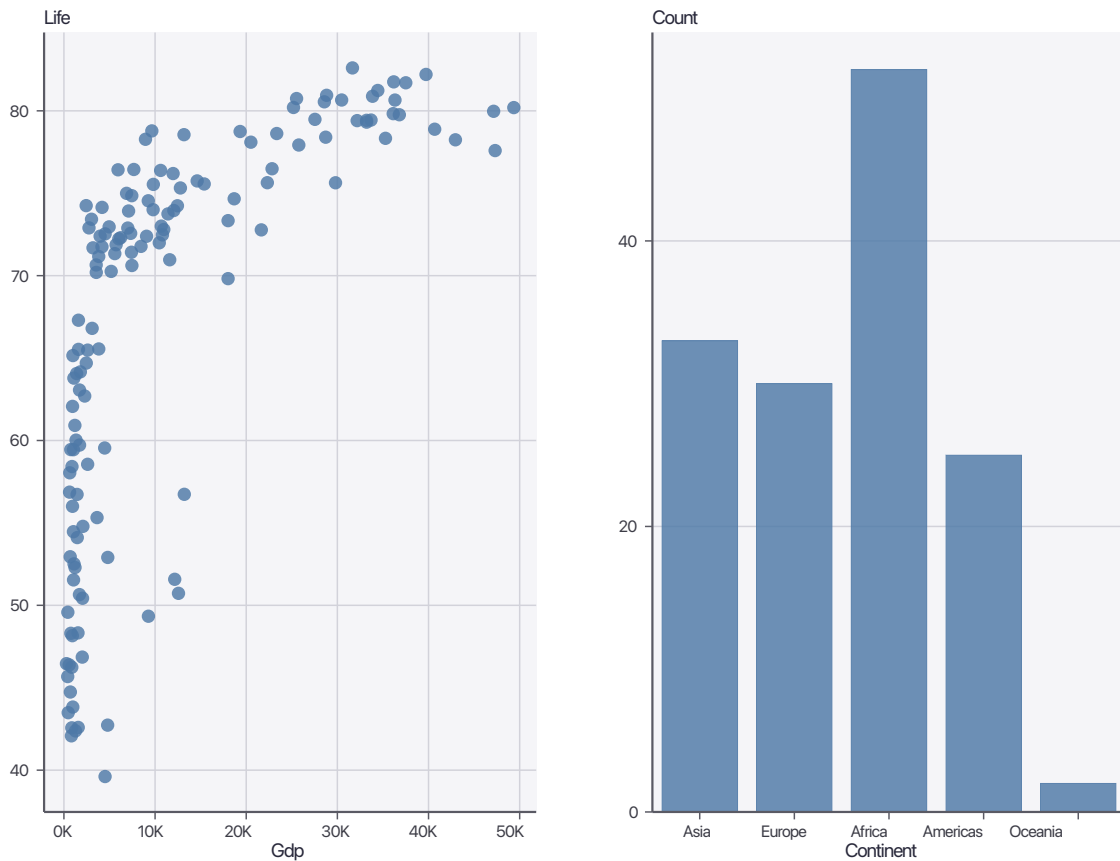
```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) |  
 (data(gapminder_2007) + bar * count + x(col.continent)))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp) + y(:life)) |  
 (data(gapminder_2007) + bar * count + x(:continent))
```

JavaScript

```
beside(plot(data(gapminder_2007), point, x(col.gdp), y(col.life)),  
 plot(data(gapminder_2007), layer(bar, count), x(col.continent)))
```



“Given gapminder 2007: points, x is gdp, y is life, beside bars derived by count, x is continent.”

Composed plots share nothing until they name the same column on the same axis, and then they share that axis outright: one scale, one extent, drawn once. That single rule is what makes a marginal plot (a scatter with a small distribution drawn along each edge) two operators rather than a chart type. See [Composition](#).

2.10 Scales: how a number becomes a position

A scale is written on the **binding**, the channel with its column in it. It is not a word of its own, because it belongs to one channel and nothing else can combine with it:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  x_label("GDP per capita")
```

Python

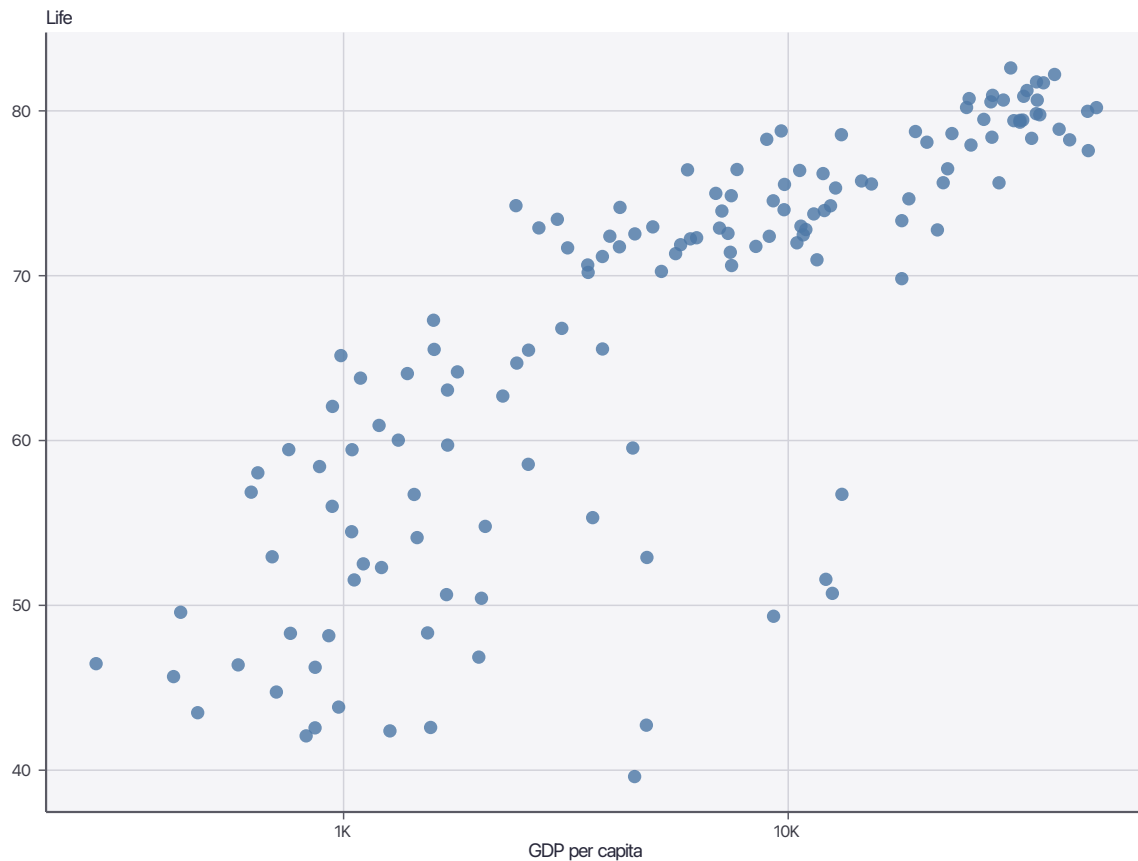
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  x_label("GDP per capita"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  x_label("GDP per capita")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), x_label("GDP per capita"))
```



“Given gapminder 2007: points, x is gdp on a log scale, y is life.”

`x_label()` writes your own words under the axis in place of the column name. It is a label, not a scale, and [Labels](#), below, covers the four of them.

Scale	What it does
<code>linear</code>	Equal steps for equal differences. The default
<code>log</code>	Equal steps for equal ratios, on <code>x</code> , <code>y</code> , <code>color</code> , <code>size</code> and <code>opacity</code> , with any <code>base</code>
<code>time</code>	Positions dates and times, chosen from the column type: <code>Date</code> and <code>POSIXct</code>
<code>category</code>	One slot per distinct value, chosen from the column type: text columns, and columns with a declared order
<code>order</code>	Which way a categorical axis runs, or declare it on the column : a factor in R, <code>ordered()</code> in the other three

A scale is not a transform: it produces no new values, only different positions for the ones you have. Where the two meet, what a log scale means for `bar * bin` or `bar * sum`, is set out in [Scales](#).

The same binding takes two more words for the same reason. `limits` says what range the channel runs over when the data is not the authority on that, and `tick_count` says how densely an axis should be labeled. Both describe the scale, so both are written where the scale is written. [Scales](#) has all three.

2.11 Spaces: the surface a plot is drawn on

A space decides what a position means. Every plot so far has been drawn on the plane. That space is `flat`, and you never have to write it.

Space	Where the marks stand
<code>flat</code>	The plane: <code>x</code> across, <code>y</code> up. The default
<code>space</code>	The cube: <code>x</code> , <code>y</code> and <code>z</code> , projected onto the page
<code>polar</code>	The plane bent into a circle: one position becomes an angle, the other a radius
<code>nest</code>	Rectangles inside rectangles, each sized by its share of the one holding it
<code>map</code>	The earth flattened: longitude across, latitude up
<code>globe</code>	The earth itself, viewed: marks stand on the half of the sphere that faces you
<code>network</code>	The space of a graph: positions come from <code>layout</code> and their numbers mean nothing, so it draws no axes. State a viewing angle in <code>network()</code> and the layout is computed in the cube instead

One space holds the whole plot. See [Space](#), [Polar](#), [Nest](#), [Map](#), [Globe](#) and [Network](#).

2.12 Settings: values you *set* rather than *map*

Every channel above answers a question about the data. `color(species)` asks “which species?”, and earns a legend so the reader can answer it.

Making every point one color asks nothing. It maps no column, needs no scale, and has nothing to decode, so it is not a channel. That is `style`:

Setting	What it fixes
<code>style(color =)</code>	One color for the whole layer
<code>style(opacity =)</code>	One opacity, 0–1
<code>style(size =)</code>	Point radius, or line stroke width
<code>style(shape =)</code>	One glyph for every point
<code>style(pattern =)</code>	The texture of a mark's paint: a stroke's dash (solid/dashed/dotted), a fill's hatch (hatch/crosshatch/grid/dots). Every mark but point , text and surface
<code>palette()</code>	Which colors the <code>color</code> channel uses
<code>theme()</code>	The page around the plot: grid, background, fonts, size

R

```
# map: color carries the continent, and a legend decodes it
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent)
```

Python

```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
    color(col.continent)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
    color(col.continent))
```



R

```
# set: one color, no question asked, no legend drawn
data(gapminder_2007) + point + x(gdp) + y(life) +
  style(color = "tomato", opacity = 0.5)
```

Python

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  style(color = "tomato", opacity = 0.5))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  style(color = "tomato", opacity = 0.5)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  style({ color: "tomato", opacity: 0.5 })))
```



“Given gapminder 2007: points, x is gdp, y is life, colored tomato, with opacity 0.5.”

See [Setting vs mapping](#) for the full rules.

2.13 Labels: the words on the plot

An axis carries the name of the column bound to it. A label replaces that name when the column name is not what a reader should see.

Label	What it names
<code>title</code>	The plot
<code>x_label</code> <code>y_label</code> <code>z_label</code>	One axis each

`title()` is the one that adds words rather than replacing them, because a plot has no other way to say what it is about.

2.14 Facets: one panel per category

Facet	What it cuts
<code>facet</code>	Splits one plot into panels, one per category of the column it names

A facet is written on the right of `|` or `/`, which is why you have already seen it drawn in the operator section: `... | facet(continent)`. See [Faceting](#).

2.15 Selections: choosing rows in a drawn plot

Selection	What it does
<code>brush</code>	Dims the rows you did not pick, and removes none of them

A selection is the only kind that acts on a plot after it is drawn. Dragging changes what you see, never the sentence. Draw the sentence again and you get the selection it names, not the one you dragged. See [Selection](#).

2.16 Names the host language already uses

Every word in the kernel is an ordinary English word, and ordinary words are already taken. `mean` names a statistic in every language that has one. `data`, `order`, `range`, `text` and `box` all name something in R before `gog` arrives. That is a consequence of choosing plain names rather than a sign that something has gone wrong, and in ordinary code almost nothing changes.

When a word is both `gog`'s and the language's own, the language decides which one your call reaches. `gog` does not change that, and each language decides differently, so the answer is different in each one:

Language	What the language does	To reach the other one
R	Twenty kernel words collide with R names. Nine are objects, and a call skips them; eleven are functions, and those mask R's own	<code>base::order(x)</code> , <code>stats::density(x)</code>
Python	A star-import rebinds six builtins, because Python has no rule that protects them	<code>from builtins import sum</code>
Julia	Two modules exporting one name is an error, never a silent choice	<code>Base.sum([1, 2, 3])</code>
JavaScript	Nothing happens, because no name reaches your scope unless you import it	nothing to do

R's case is the one to understand, because it looks the worst and matters the least. A mark takes no argument, and neither does a statistic like `mean` or `max`, so those words are plain objects rather than functions. When R evaluates a call it looks for a *function* of that name, and skips any binding that is not one. Base R therefore still answers `mean(c(1, 2, 3))`, and `sapply(df, mean)` works for the same reason. The message R prints when the package loads names every collision and cannot show you this distinction, which is why the message looks worse than the problem is.

The other eleven words take arguments, so they are functions, and a function does mask R's own. Once `gog` loads, `range(0.25, 0.75)` is `gog`'s band; to reach base R's, write `base::range()`. Say you forget the masking and write `range(heights)` for a vector's smallest and largest. The call reaches `gog`'s `range()`, which takes two numbers between 0 and 1, not a vector. For eight of the eleven functions, a wrong argument like that proves the mistake: the call is refused, and the refusal points you to `base::range()`. Now write `data(mtcars)`, meaning to load R's example dataset. That call is also correct `gog`, naming a plot's table, so `gog` answers it and cannot warn you. `box()` and `title("...")` are the same case. A refusal needs a mistake the engine can detect, and these three hold none.

The collision has no good ending. Prefixing the vocabulary, `gog_mean` and `gog_range`, would remove the reason for the vocabulary: a reader who has to learn a prefix learns every word twice, and none of the words is English any more. Renaming `order` to `sort_by` would take away a word every reader already knows. So the words stay, and the few places that need a qualified name write one.

Each binding chapter carries three things for its language: the full list of collisions, the rule above as code you can run, and the few spellings the rule cannot save. R has five. All five share

one shape: a word is handed to one function, which passes it to a second, the way `aggregate(v ~ g, df, mean)` passes `mean` on. See [R](#), [Python](#), [Julia](#) and [JavaScript](#).

The next two chapters use this vocabulary: first you read ten plots, then you write your own.

3 Reading practice

Can you tell what a plot shows before you see it? You have the vocabulary; now read with it. Ten plots follow, ending in a cube. For each one: read the specification aloud before looking at the picture, then check the picture against what you said. The English sentence is given under each plot. Try to say it yourself first.

3.1 Gold medals by country

`medals` holds one row per country, with its counts of gold, silver and bronze:

Table 3.1: `medals`: all 5 rows

country	gold	silver	bronze
USA	46	37	38
China	38	31	22
Great Britain	29	17	19
Russia	19	18	9
Germany	17	10	15

R

```
data(medals) + bar + x(country) + y(gold)
```

Python

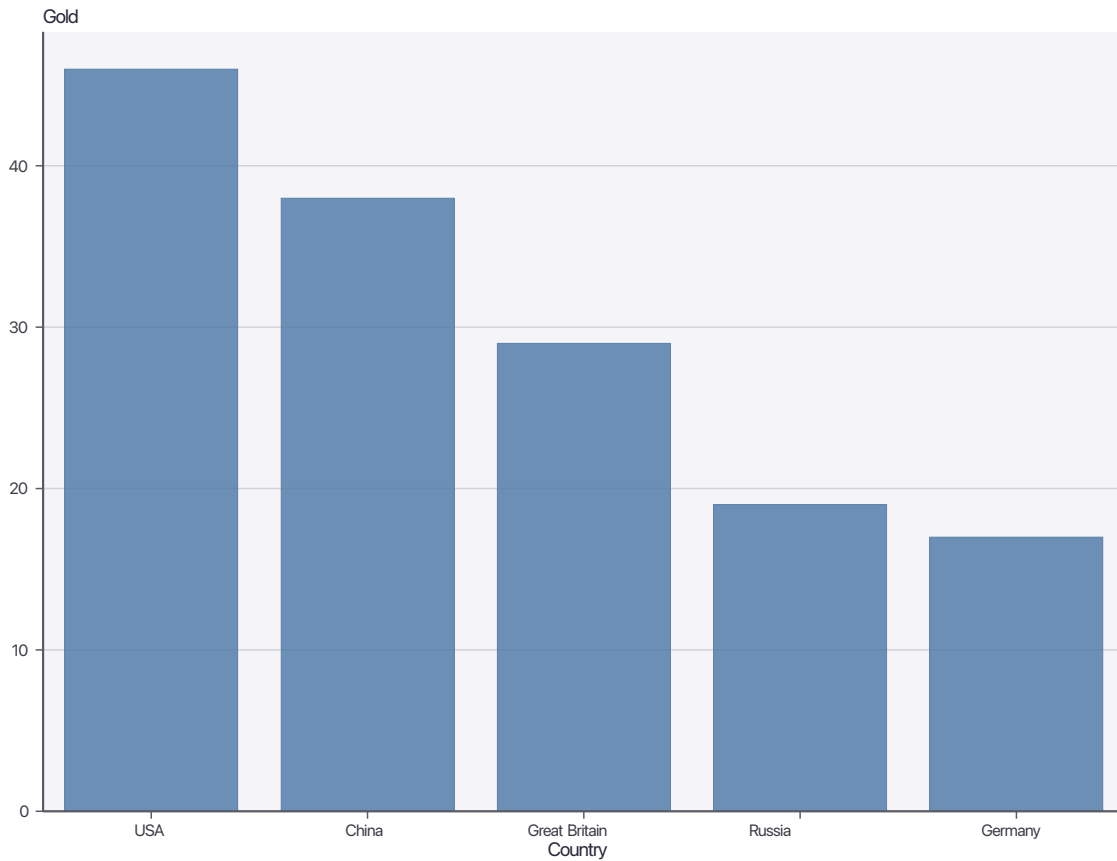
```
data(medals) + bar + x(col.country) + y(col.gold)
```

Julia

```
data(medals) + bar + x(:country) + y(:gold)
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold))
```



“Given the medals: bars, x is country, y is gold.” A bar measures an amount from zero, so the categories sit on x and the number on y . No transform anywhere: **bar** draws the numbers exactly as they arrive in the table.

3.2 The same bars, lying down

R

```
data(medals) + bar + x(gold) + y(country)
```

Python

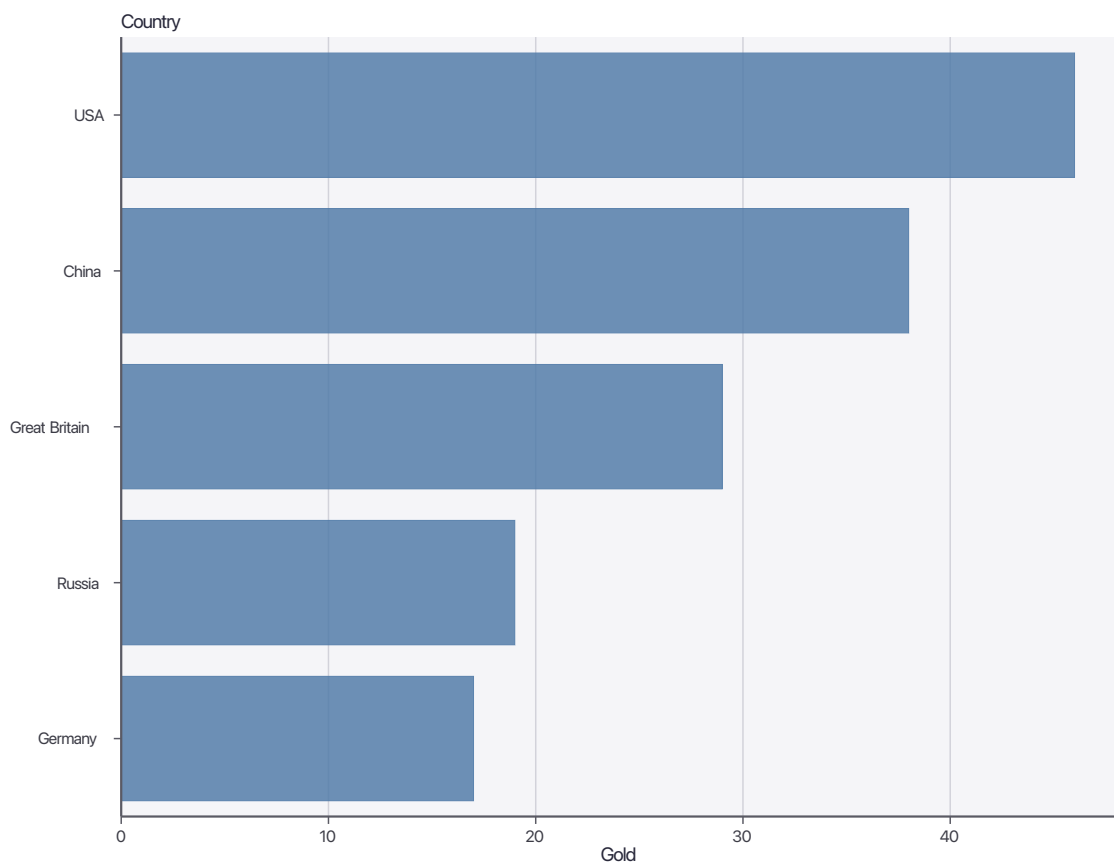
```
data(medals) + bar + x(col.gold) + y(col.country)
```

Julia

```
data(medals) + bar + x(:gold) + y(:country)
```

JavaScript

```
plot(data(medals), bar, x(col.gold), y(col.country))
```



“Given the medals: bars, x is gold, y is country.” Swap which axis carries the category and the bars lie down. There is no “horizontal bar” atom, and no argument that flips one: orientation is read off the bindings, because the sentence already says which axis holds the category.

3.3 One measurement's shape

`iris_flowers` holds one row per flower: four measurements, and the species the flower belongs to.

Table 3.2: `iris_flowers`: first 5 of 150 rows

sepal_length	sepal_width	petal_length	species
5.1	3.5	1.4	setosa
4.9	3.0	1.4	setosa
4.7	3.2	1.3	setosa
4.6	3.1	1.5	setosa
5.0	3.6	1.4	setosa

R

```
data(iris_flowers) + bar * bin + x(petal_length)
```

Python

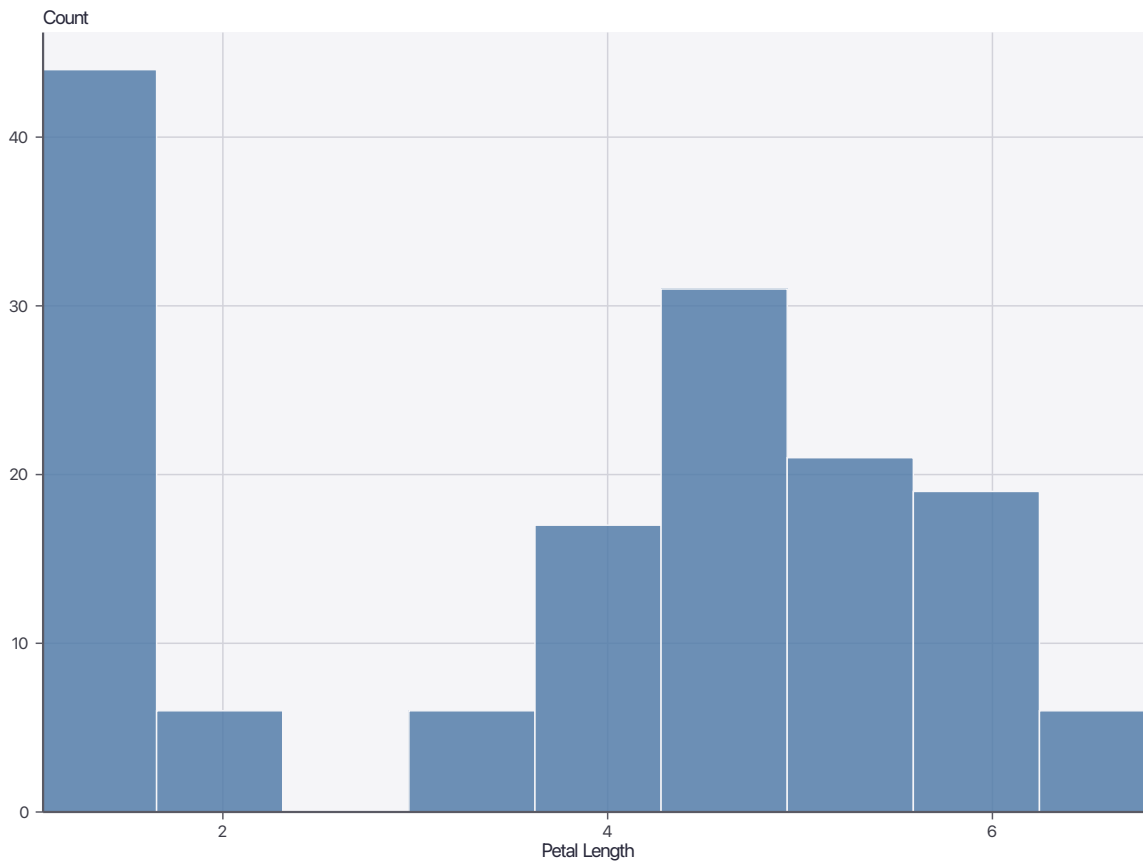
```
data(iris_flowers) + bar * bin + x(col.petal_length)
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length)
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length))
```



“Given the iris flowers: bars derived by *bin*, *x* is petal length.” The `*` derives: `bin` cuts the axis into intervals and counts rows into them, and `bar` draws the counts. You did not bind `y`; `bin` invents the count, and the axis says so. Note the two peaks. Plot eight explains them.

3.4 The same question, smoothly

R

```
data(gapminder_2007) + line * density + x(life)
```

Python

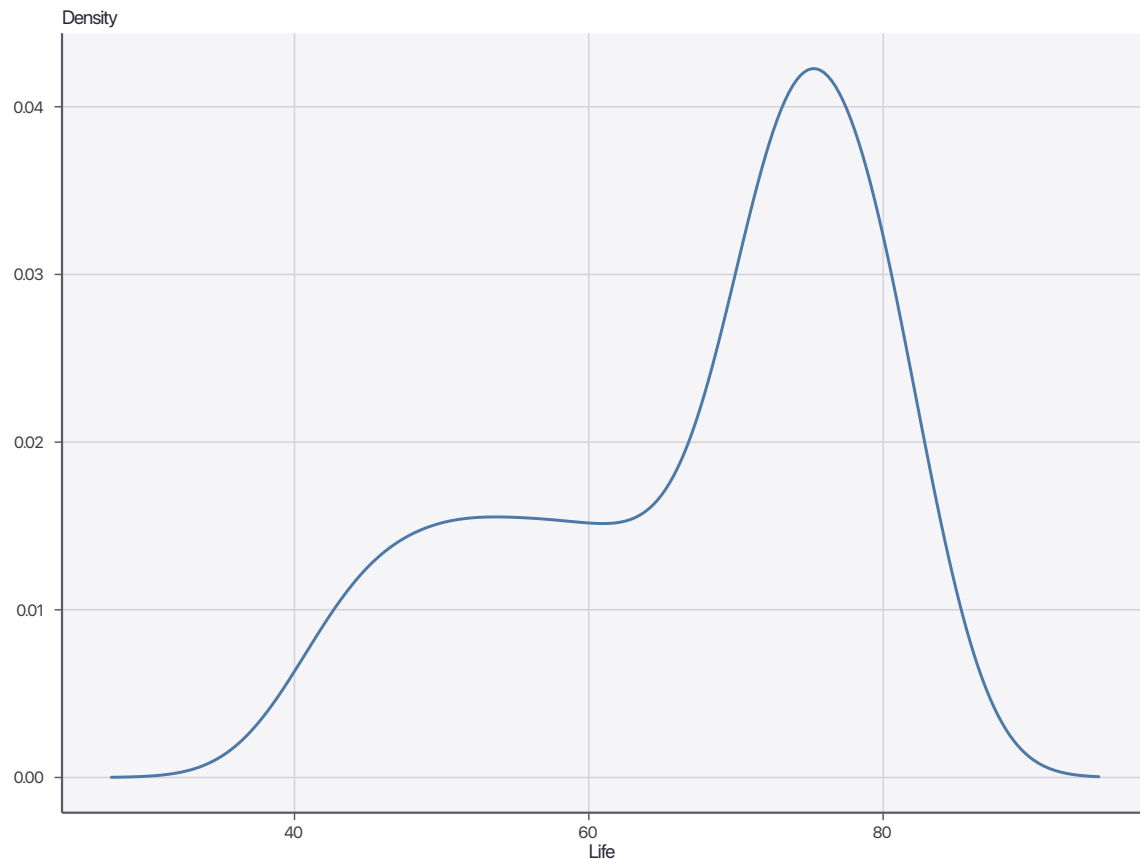
```
data(gapminder_2007) + line * density + x(col.life)
```

Julia

```
data(gapminder_2007) + line * density + x(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density), x(col.life))
```



“Given gapminder 2007: a line derived by density, x is life.” Same question as the histogram (*how is this measurement distributed?*), different derivation. Swapping one atom gives the smooth estimate instead of the binned one.

3.5 One dot per country, grouped by continent

R

```
data(gapminder_2007) + point + x(continent) + y(life)
```

Python

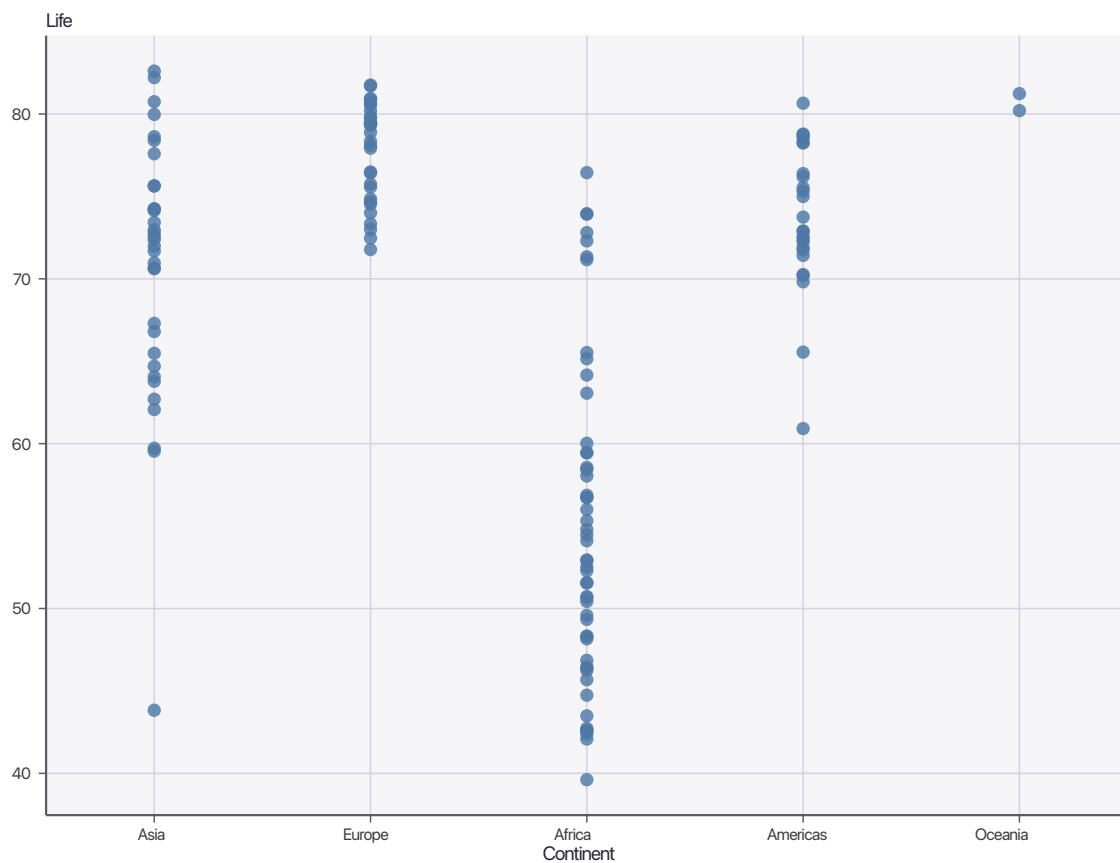
```
data(gapminder_2007) + point + x(col.continent) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:continent) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.continent), y(col.life))
```



“Given *gapminder 2007*: *points*, *x* is *continent*, *y* is *life*.” A categorical *x* gives each category a slot, and the points for that category sit in a strip above it. That is the rule that placed bars on country slots in plot one, working here on a different mark. A plot shaped like this one has a name: the **strip plot**.

3.6 The typical value per continent

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life)
```

Python

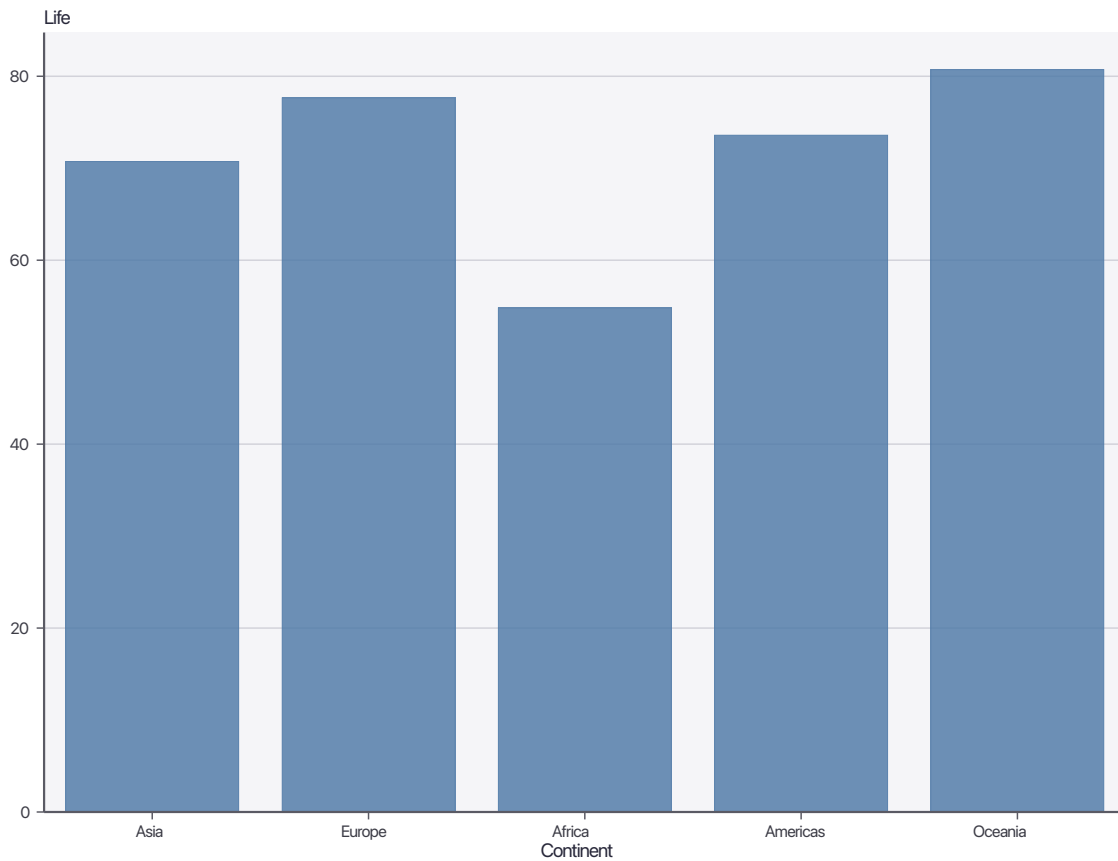
```
data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life)
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),  
      y(col.life))
```



“Given *gapminder 2007*: bars derived by mean, *x* is continent, *y* is life.” The strip plot above showed every country; `* mean` collapses each continent to its average. Here `y(life)` names the column the mean *reads*: a transform that reads a column makes you say which one.

3.7 Five countries across the decades

`gapminder_asia` holds one row per country *and year*, so each country appears many times.

Table 3.3: `gapminder_asia`: first 5 of 60 rows

country	continent	year	life	population	gdp
China	Asia	1952	44.00000	556263527	400.4486
China	Asia	1957	50.54896	637408000	575.9870
China	Asia	1962	44.50136	665770000	487.6740
China	Asia	1967	58.38112	754550000	612.7057

country	continent	year	life	population	gdp
China	Asia	1972	63.11888	862030000	676.9001

R

```
data(gapminder_asia) + line + x(year) + y(life) + color(country)
```

Python

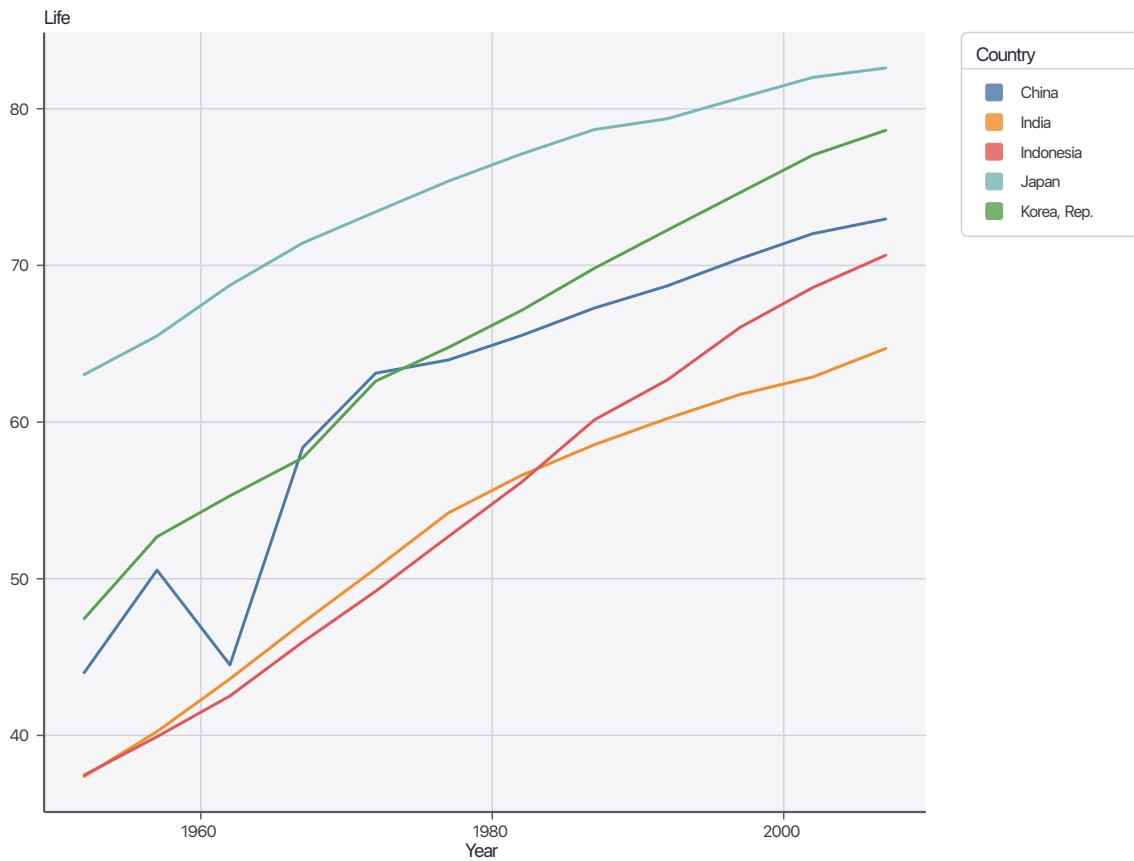
```
data(gapminder_asia) + line + x(col.year) + y(col.life) + color(col.country)
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + color(:country)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),
      color(col.country))
```



“Given *gapminder Asia*: *lines*, *x* is year, *y* is life, color by country.” On a line, a categorical color first *splits* the rows into one line per category, then colors them. One channel does two things, and you write it once.

3.8 Three species on one axis

R

```
data(iris_flowers) + bar * bin + x(petal_length) + color(species)
```

Python

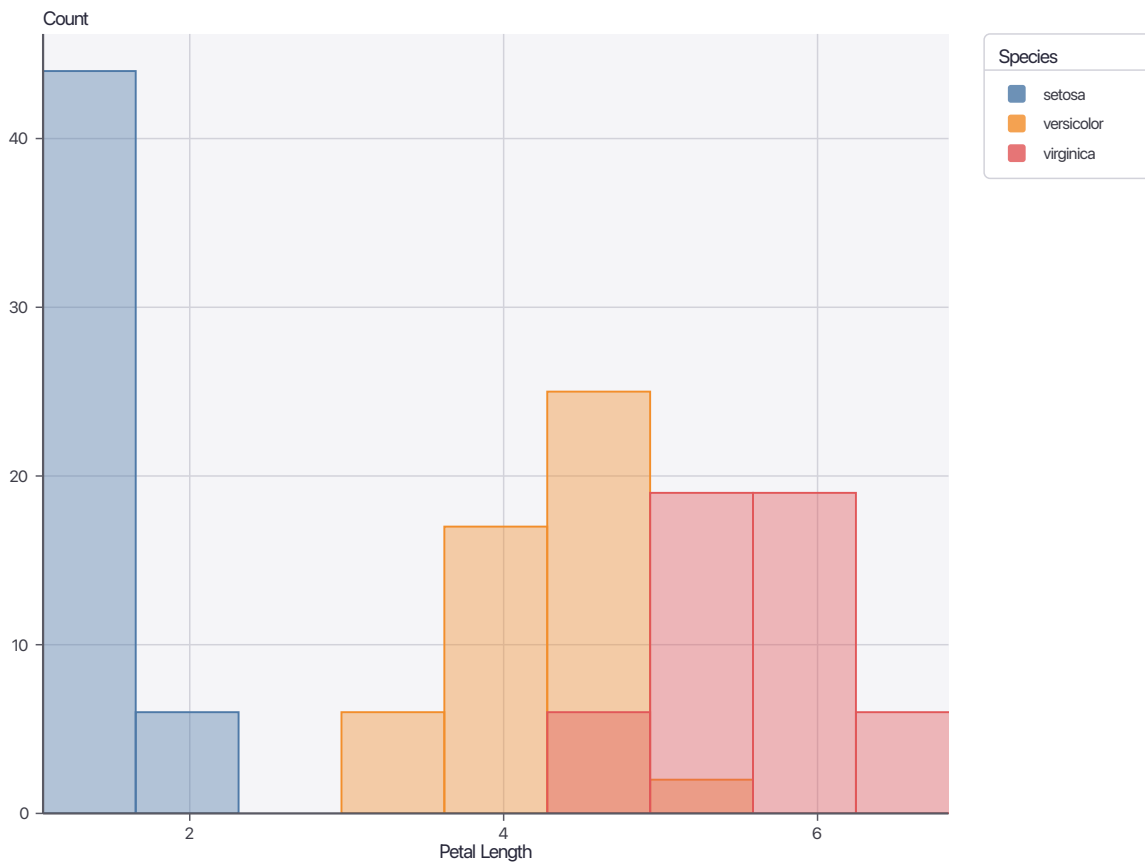
```
data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species)
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length),  
     color(col.species))
```



“Given the iris flowers: bars derived by bin, x is petal length, color by species.” The two peaks from plot three now sit in separate histograms, one per species. All three use the same bin edges, and each is drawn where its own values fall. Each fill is translucent under a solid outline, so every histogram stays visible. `color` does the splitting here, the way it split the lines in plot seven: same word, same rule.

3.9 One panel per continent

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) |  
  ↪ facet(continent)
```

Python

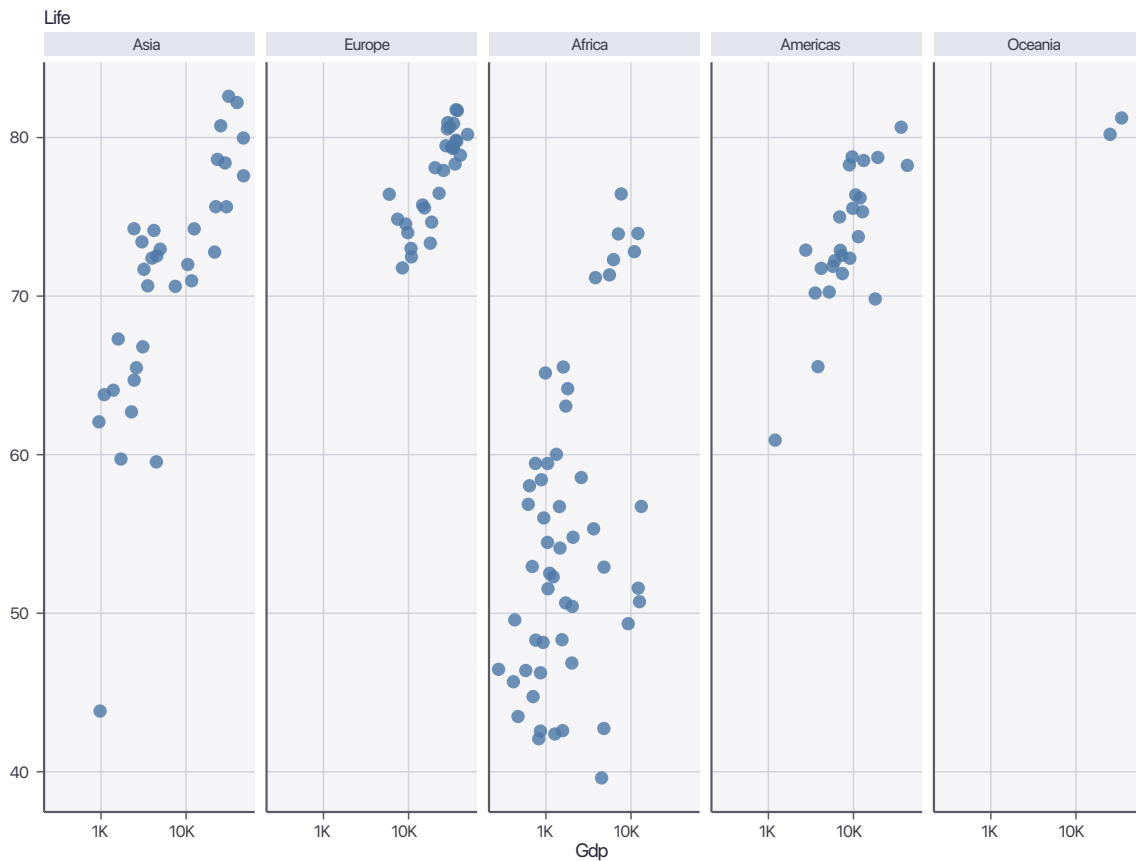
```
data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) |  
  ↪ facet(col.continent)
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) |  
  facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), across(col.continent))
```



“Given *gapminder 2007*: *points*, *x* is *gdp* on a *log scale*, *y* is *life*, *split* into *panel columns* by *continent*.” The `|` operator splits the plot into panel columns, one panel per category. All the panels share one pair of scales, so they can be compared.

3.10 Three measurements on three axes

R

```
data(iris_flowers) + point +
  x(sepal_length) + y(sepal_width) + z(petal_length) +
  color(species)
```

Python

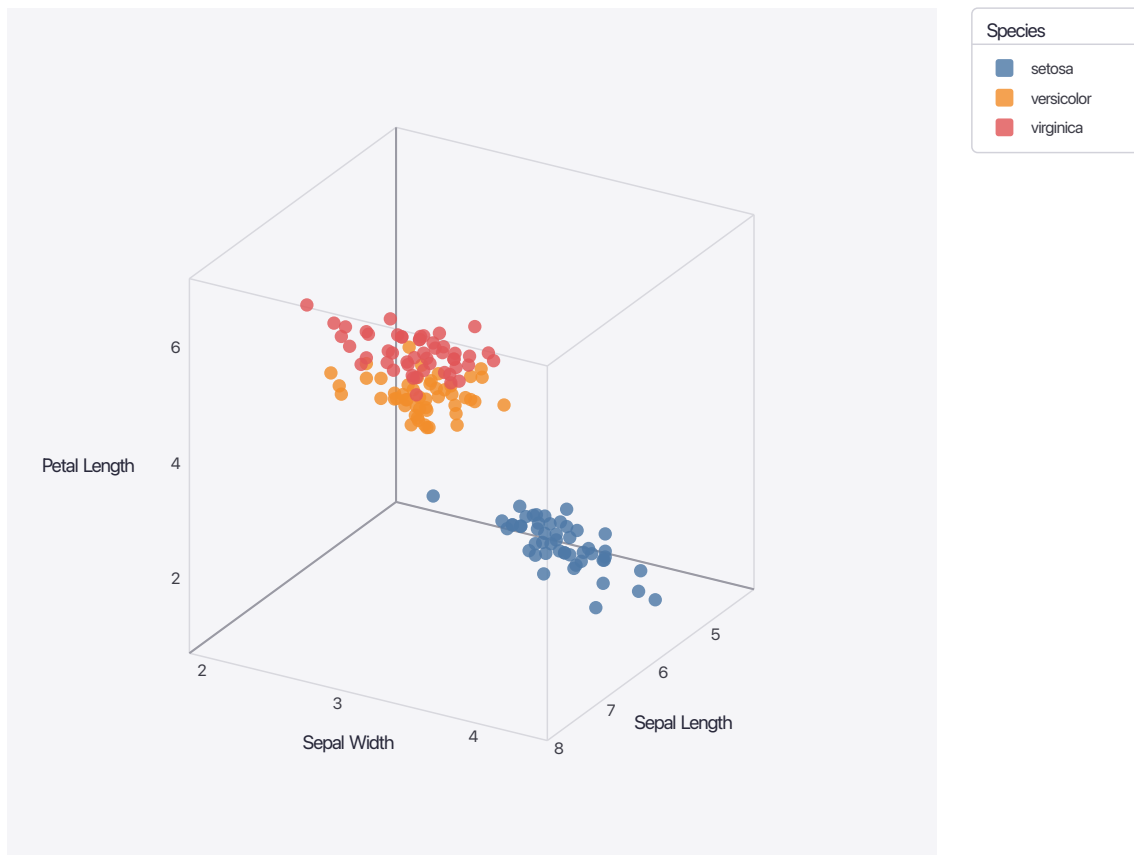
```
(data(iris_flowers) + point +  
  x(col.sepal_length) + y(col.sepal_width) + z(col.petal_length) +  
  color(col.species))
```

Julia

```
data(iris_flowers) + point + x(:sepal_length) + y(:sepal_width) +  
  z(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), point, x(col.sepal_length), y(col.sepal_width),  
  z(col.petal_length), color(col.species))
```



“Given the iris flowers: points, x is sepal length, y is sepal width, z is petal length, color by species.” The third dimension is not a different package, and not a different mark. It is one more position channel, and the sentence grew by one word. In the web edition the cube turns with a drag: drag right and the side facing you moves right, drag down and it tips. Each angle shows what another hides, so try a few.

3.11 What you just did

Ten plots, and twelve distinct words. Every one read left to right as *data, mark, positions, refinements*. Nothing you learned for one plot had to be unlearned for the next. The same `color` split bars, lines and points. The same `*` derived counts, means and densities. Now you write.

4 Writing practice

How does a question about your data turn into a specification? Reading was recognition; writing is recall. Psychologists use those two words for two kinds of remembering. Recognition starts from something in front of you, and asks only whether you have seen it before. That is what the last chapter did: it gave you finished sentences and asked what they say. Recall starts from nothing in front of you: you produce the answer yourself. It is the harder of the two, and it is what writing a specification asks of you.

This chapter works in the recall direction from here on. Three builds come first, showing the method you will actually use: start with the minimum sentence, look, refine. Five questions follow with nothing in front of you, each answered by a sentence you met in the last chapter: write yours before you scroll, then check it against the printed one. Last comes the other half of writing: what happens when a sentence is wrong, and how quickly a refusal teaches the rule.

4.1 Which continents live longest?

Before reading on, try to write the sentence that will answer it: which mark, which transform, which positions? Then build it the way this chapter always will. Start with the minimum sentence, the smallest one that shows the data:

R

```
data(gapminder_2007) + point + x(continent) + y(life)
```

Python

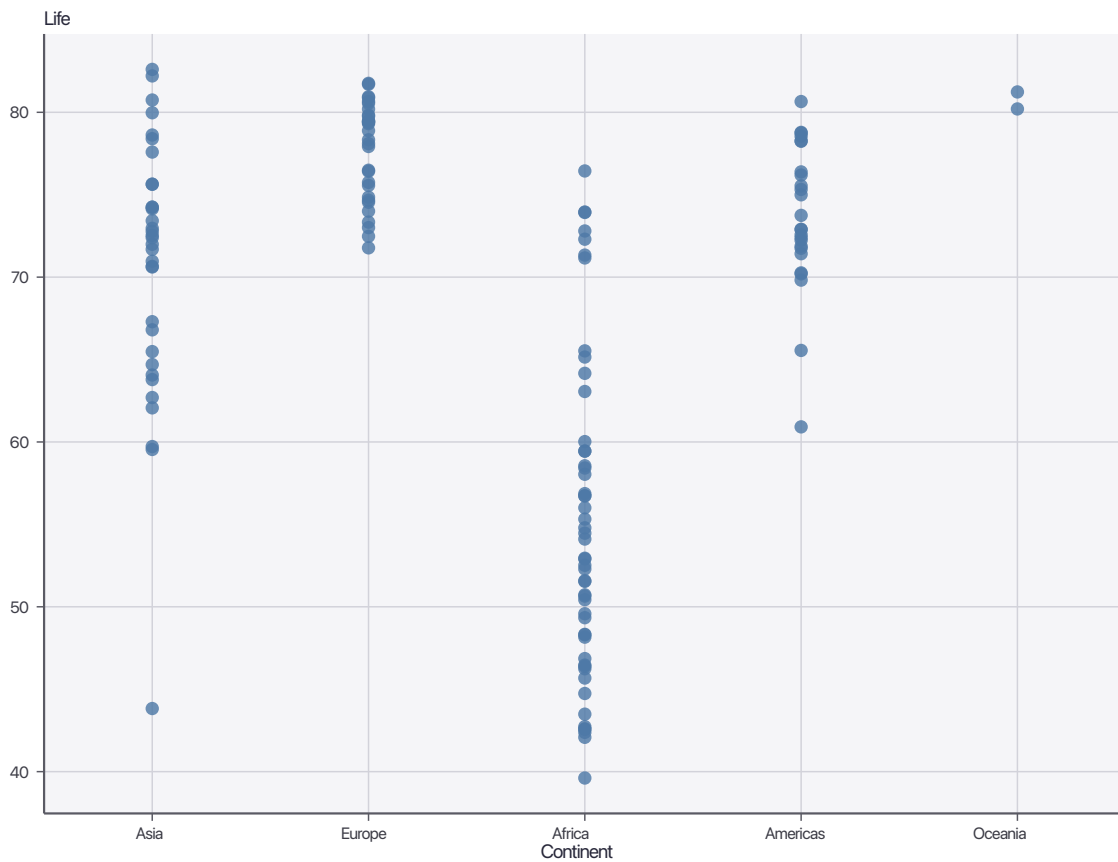
```
data(gapminder_2007) + point + x(col.continent) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:continent) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.continent), y(col.life))
```



“Given *gapminder 2007*: points, *x* is continent, *y* is life.” Informative, but the question asked for a comparison of *typical* values, and this shows every country. Swap the mark and give it a derivation; everything else stays:

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life)
```

Python

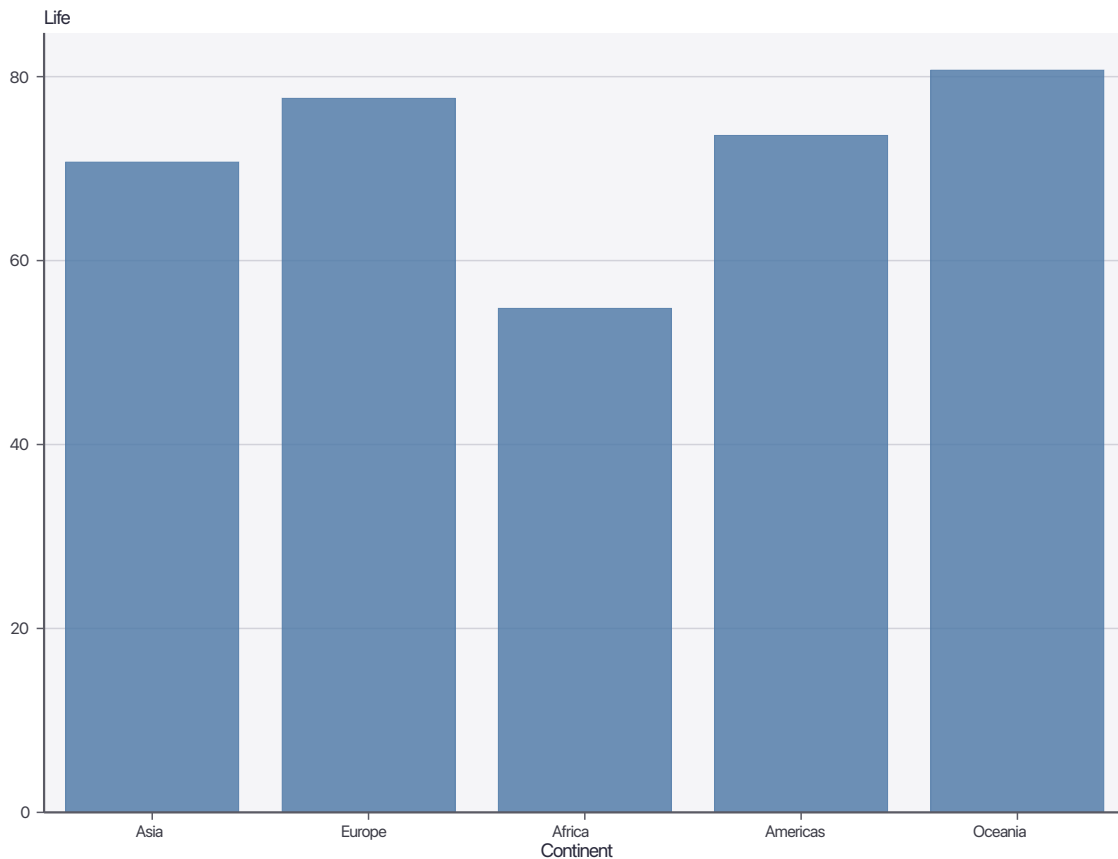
```
data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life)
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),  
      y(col.life))
```



“Given gapminder 2007: bars derived by mean, x is continent, y is life.”

A comparison reads best in order. A categorical column can carry a fixed order, declared where the data is made. The order wanted here depends on the means this plot just computed, so only the sentence can ask for it: `order()` sorts the axis by the column you name:

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life) +  
  order(life, desc = TRUE)
```

Python

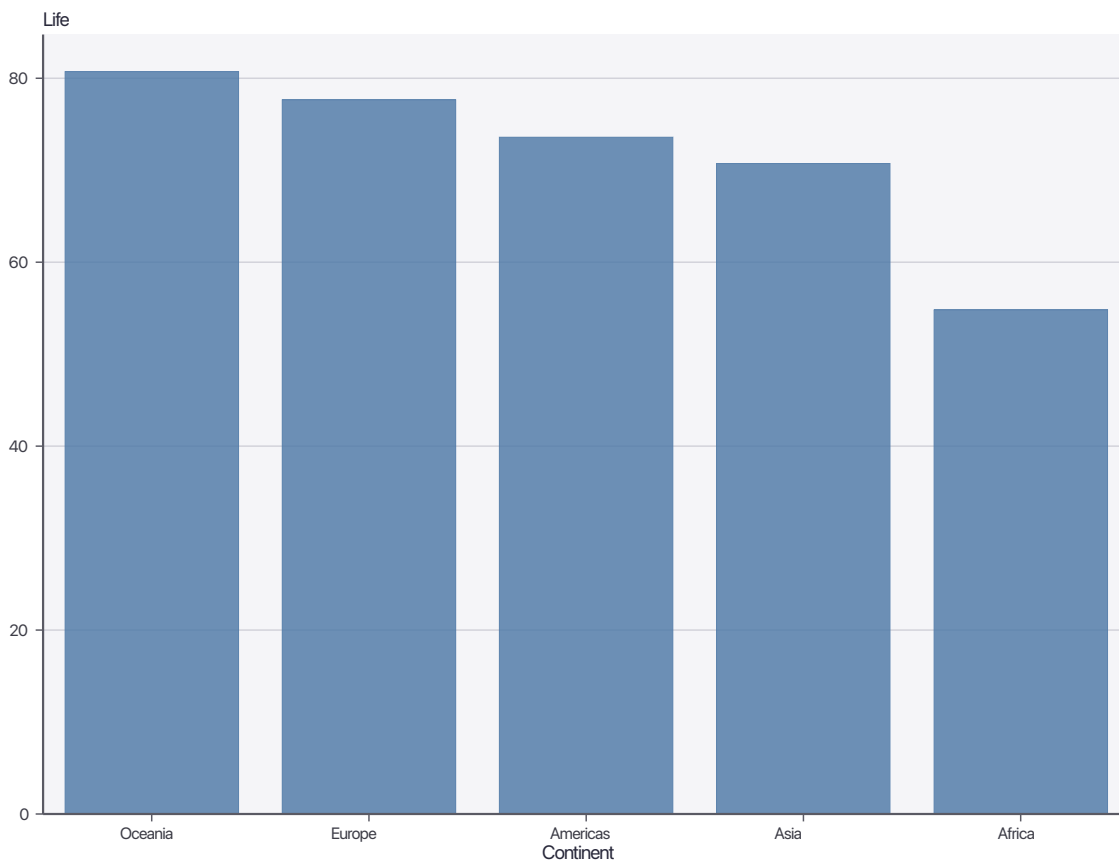
```
(data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life) +  
  order(col.life, desc = True))
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life) +  
  order(:life, desc = true)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),  
  y(col.life), order(col.life, { desc: true }))
```



“Given gapminder 2007: bars derived by mean, x is continent, y is life, ordered by life, largest first.”

Three steps, and no step rewrote the sentence: words were added or swapped, never restructured. That is what composition gives you: each step keeps every word you already wrote.

4.2 How do petal lengths differ by species?

The last chapter drew both of these. Write the two sentences from memory: the one-variable question first, then the same question split by species.

R

```
data(iris_flowers) + bar * bin + x(petal_length)
```

Python

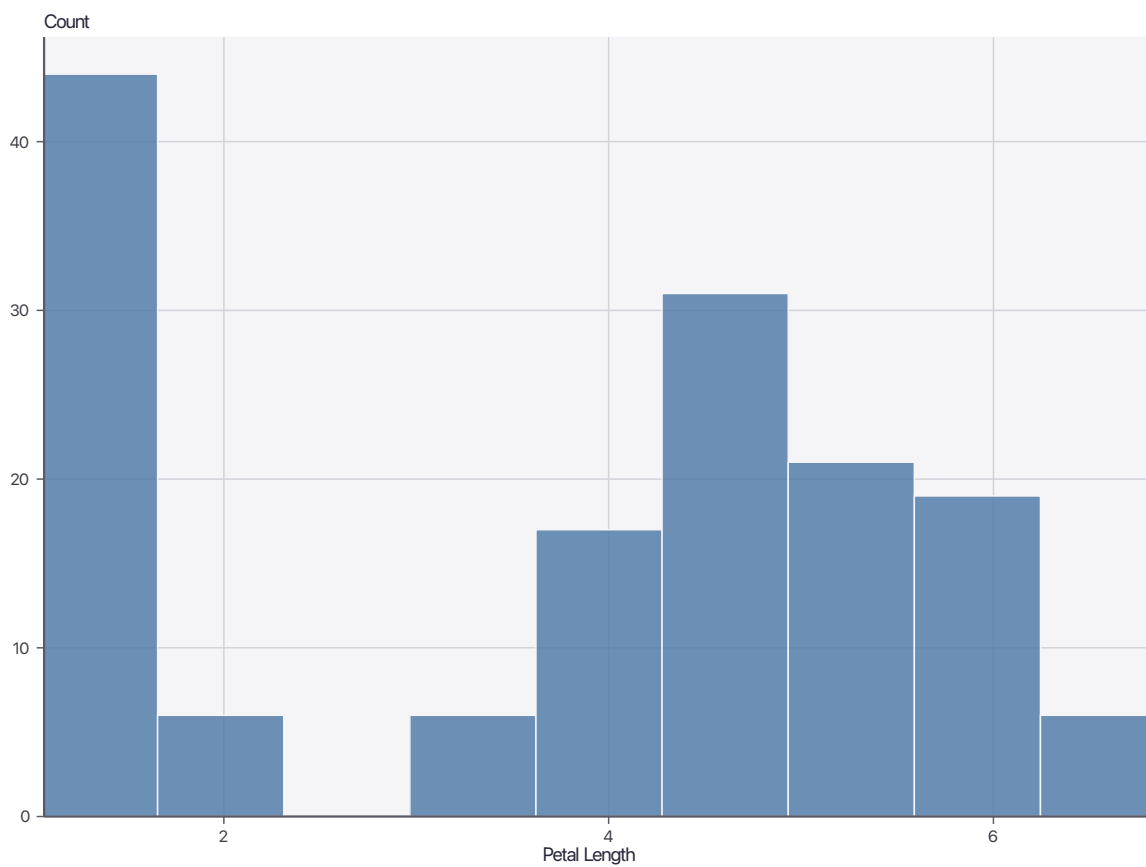
```
data(iris_flowers) + bar * bin + x(col.petal_length)
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length)
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length))
```



“Given the iris flowers: bars derived by bin, x is petal length.”

R

```
data(iris_flowers) + bar * bin + x(petal_length) + color(species)
```

Python

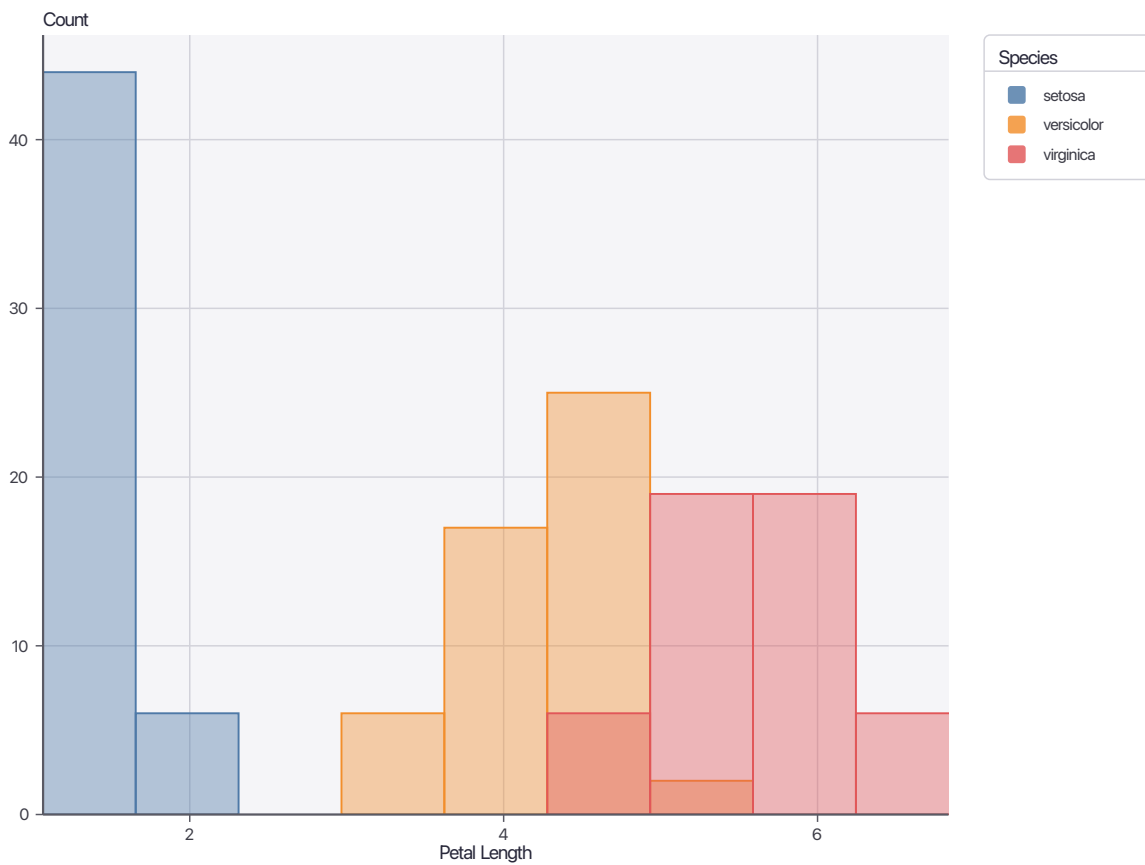
```
data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species)
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length),  
     color(col.species))
```



“Given the iris flowers: bars derived by bin, x is petal length, color by species.”

And if the overlap bothers you, one mark changes it: **step** draws each bin’s height as an unfilled line, so the three species never cover each other:

R

```
data(iris_flowers) + step * bin + x(petal_length) + color(species)
```

Python

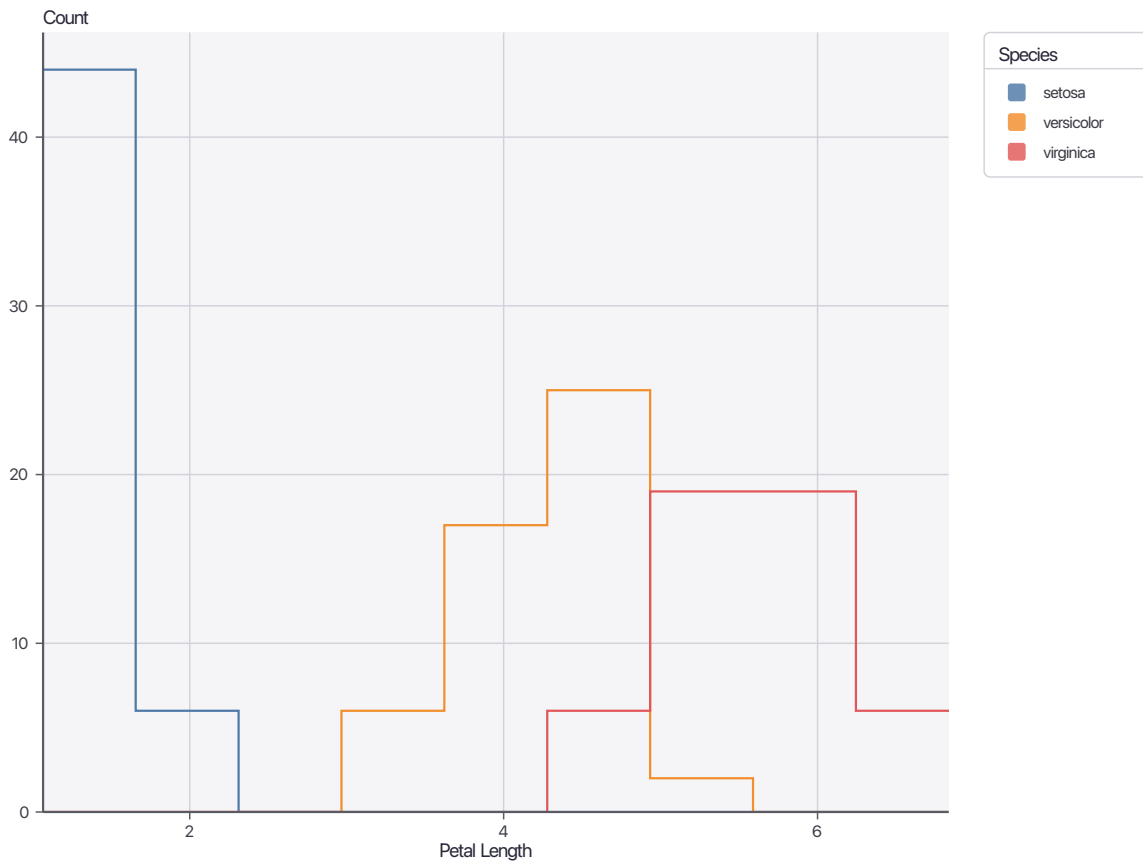
```
data(iris_flowers) + step * bin + x(col.petal_length) + color(col.species)
```

Julia

```
data(iris_flowers) + step * bin + x(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), layer(step, bin), x(col.petal_length),  
      color(col.species))
```



“Given the iris flowers: step outlines derived by bin, x is petal length, color by species.”

The transform never changed. `bin` cut the same intervals all three times; only the mark drawing them changed. A derivation you learn once works with every mark that can take it.

4.3 Where is the world’s wealth, and its health?

Chapter one built this sentence step by step. Write it in one pass now, adding `size` for population, then check:

R

```
data(gapminder_2007) + point +
  x(gdp, scale = "log") + y(life) +
  color(continent) + size(population) +
  title("The health and wealth of nations, 2007")
```

Python

```
(data(gapminder_2007) + point +  
  x(col.gdp, scale = "log") + y(col.life) +  
  color(col.continent) + size(col.population) +  
  title("The health and wealth of nations, 2007"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + size(:population) +  
  title("The health and wealth of nations, 2007")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.continent), size(col.population),  
  title("The health and wealth of nations, 2007"))
```



“Given *gapminder 2007*: *points*, *x* is *gdp* on a *log scale*, *y* is *life*, *color* by *continent*, *size* by *population*.” Three atoms added to the minimum sentence (*color*, *size* and *title*; the log scale rides inside *x* rather than counting as a word), and still no new grammar in it.

4.4 Five questions, nothing in front of you

Each question below matches one plot from the last chapter. Write your sentence first; the printed one is there to check against, not to read first.

Which countries won the most gold? Every country name has to stay readable, so lay the bars down.

R

```
data(medals) + bar + x(gold) + y(country)
```

Python

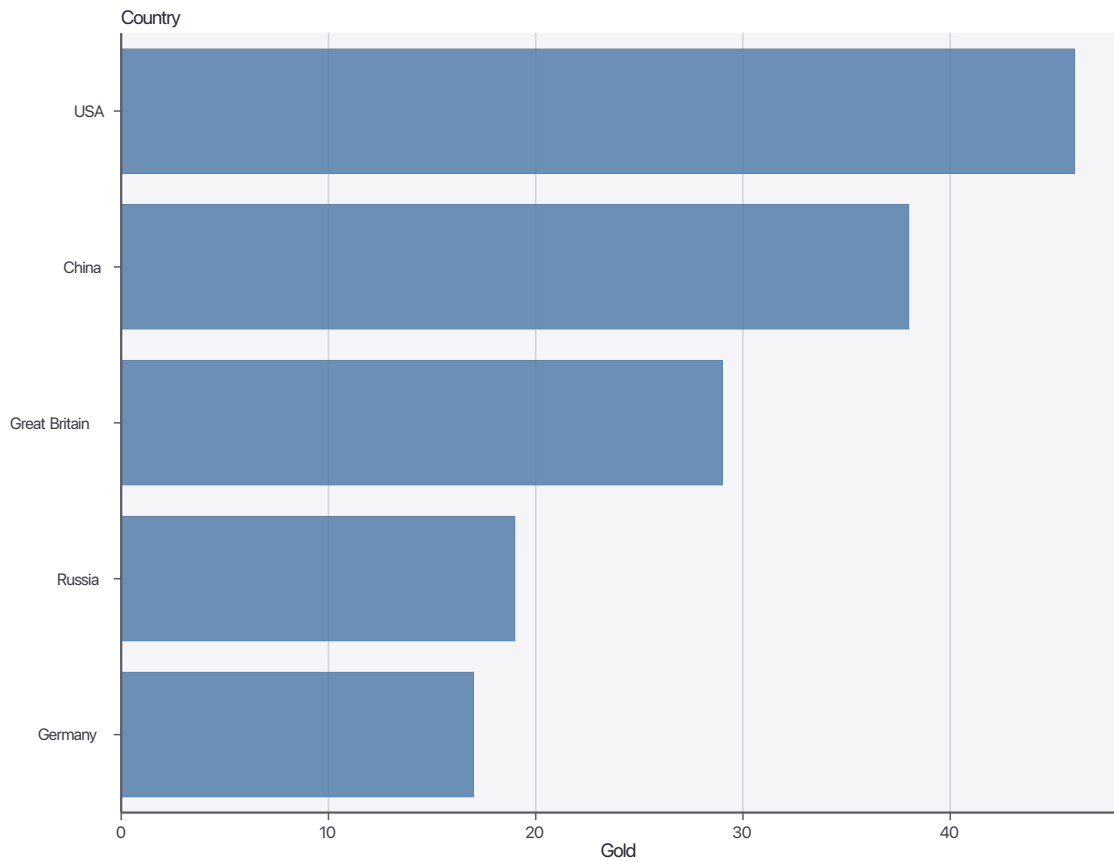
```
data(medals) + bar + x(col.gold) + y(col.country)
```

Julia

```
data(medals) + bar + x(:gold) + y(:country)
```

JavaScript

```
plot(data(medals), bar, x(col.gold), y(col.country))
```



“Given the medals: bars, x is gold, y is country.” One swap of the two bindings lies the bars down, and no new word was needed.

How is life expectancy distributed, drawn as a smooth curve rather than counted into bins?

R

```
data(gapminder_2007) + line * density + x(life)
```

Python

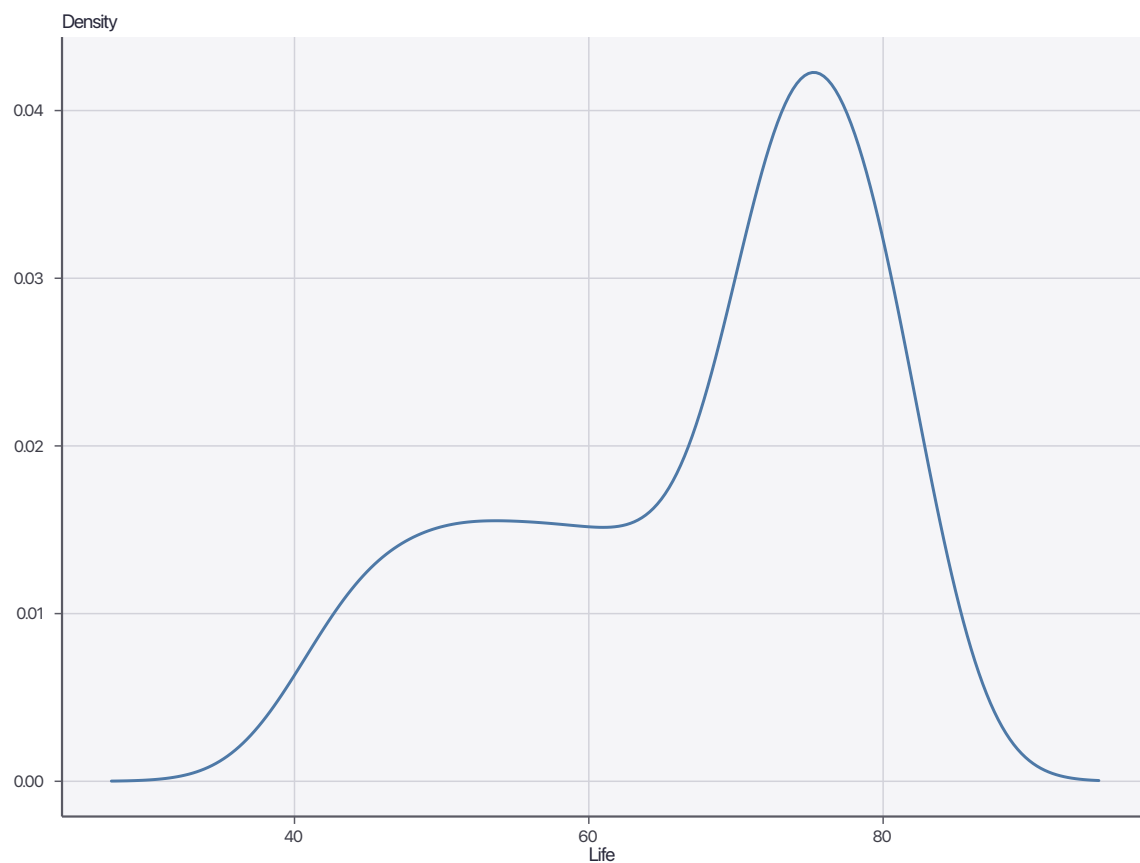
```
data(gapminder_2007) + line * density + x(col.life)
```

Julia

```
data(gapminder_2007) + line * density + x(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density), x(col.life))
```



“Given *gapminder 2007: a line derived by density, x is life.*” The histogram’s sibling: one atom swapped, same question.

How did life expectancy move across the decades in the five Asian countries, one line per country?

R

```
data(gapminder_asia) + line + x(year) + y(life) + color(country)
```

Python

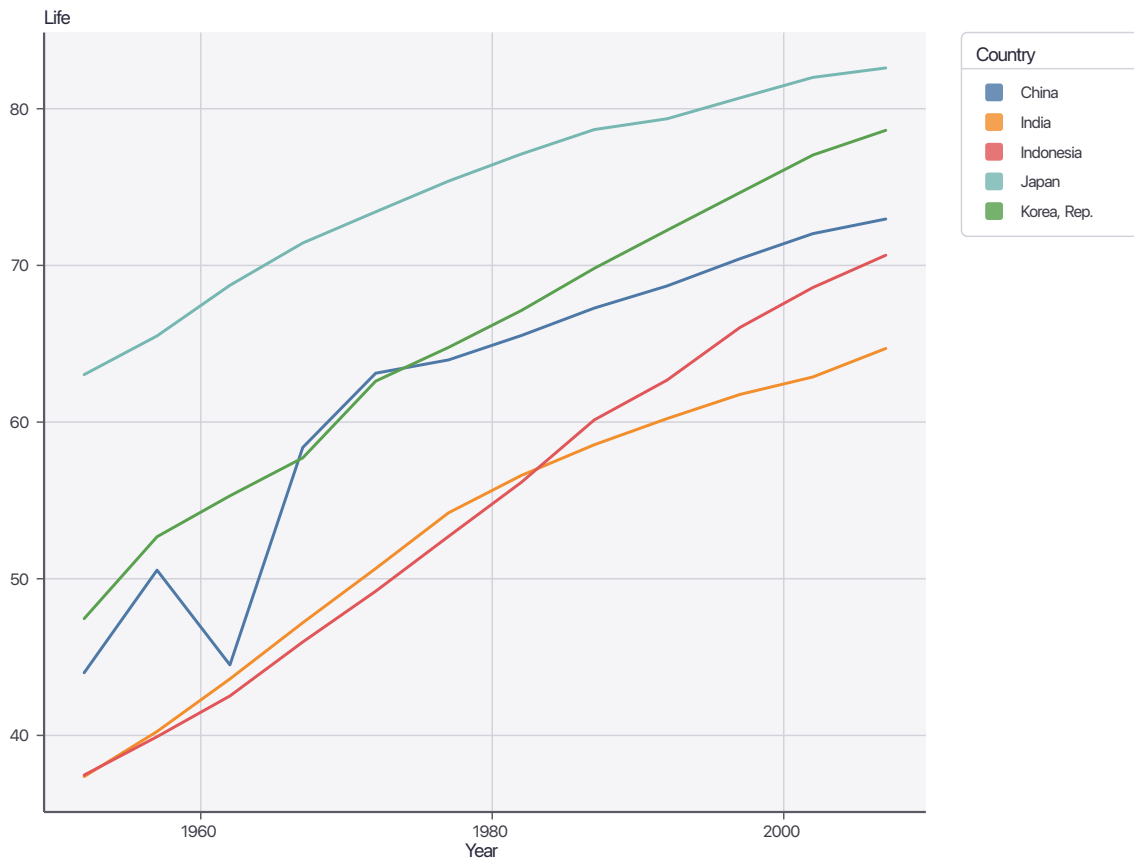
```
data(gapminder_asia) + line + x(col.year) + y(col.life) + color(col.country)
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + color(:country)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
      color(col.country))
```



“Given `gapminder Asia`: `lines`, `x` is `year`, `y` is `life`, `color` by `country`.” `color` splits first and `colors` second, exactly as it did in the last chapter.

Does the wealth and health relationship hold inside each continent, one panel per continent?

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) |
  ↪ facet(continent)
```

Python

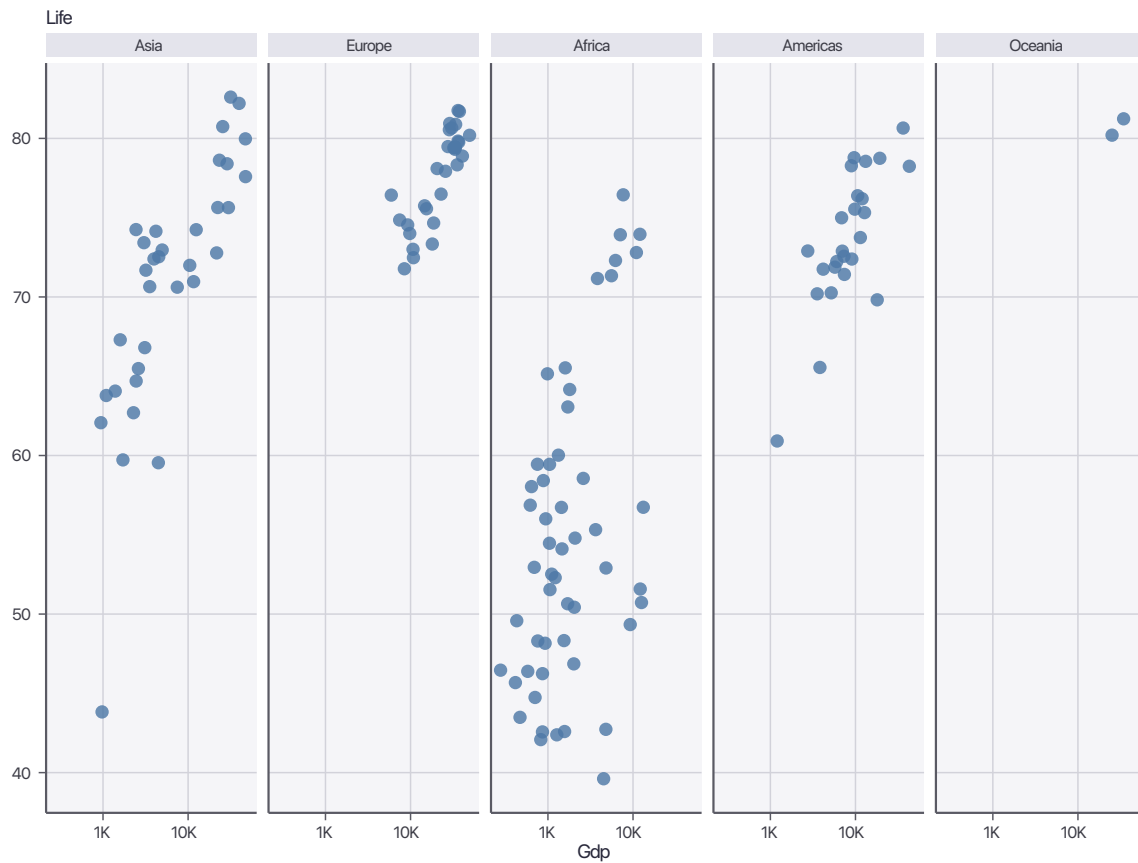
```
data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) |
  ↪ facet(col.continent)
```

Julia


```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) |
  facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),
  y(col.life), across(col.continent))
```



“Given *gapminder 2007*: *points*, *x* is *gdp* on a *log* scale, *y* is *life*, split into panel columns by *continent*.” The `|` and its `facet`, shared scales and all.

Do the three iris measurements separate the species when seen together?

R

```
data(iris_flowers) + point +  
  x(sepal_length) + y(sepal_width) + z(petal_length) +  
  color(species)
```

Python

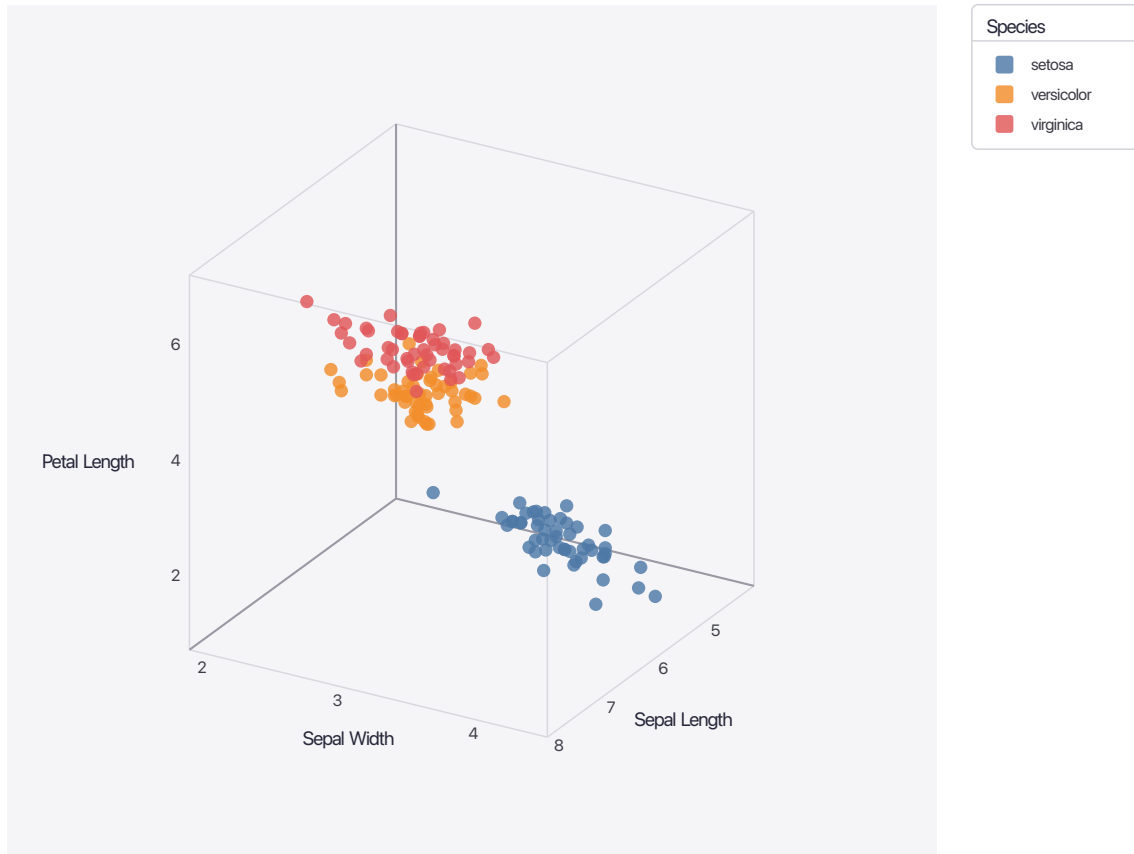
```
(data(iris_flowers) + point +  
  x(col.sepal_length) + y(col.sepal_width) + z(col.petal_length) +  
  color(col.species))
```

Julia

```
data(iris_flowers) + point + x(:sepal_length) + y(:sepal_width) +  
  z(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), point, x(col.sepal_length), y(col.sepal_width),  
  z(col.petal_length), color(col.species))
```



“Given the iris flowers: points, x is sepal length, y is sepal width, z is petal length, color by species.” The cube again, and you wrote it yourself this time.

4.5 When the engine says no

You will write wrong sentences. Everyone does. `gog`’s design promise is that a wrong sentence is refused *with a direction*, never drawn with the wrong part quietly left out. Here are the refusals you are most likely to see first.

Forgetting to bind the table. The sentence must start with the data:

```
gapminder_2007 + point + x(gdp) + y(life)
```

Error:

! gog: a plot starts with ``data()``, which names the table - columns are bare
↪ names and the nearest named table wins, so the name matters. Write
↪ ``data(gapminder_2007) + point + ...``.

Quoting a column name, a habit from other packages. The refusal names the fix:

```
data(gapminder_2007) + point + x("gdp") + y("life")
```

Error:

! gog: ``x("gdp")`` binds a **value**, and a channel takes a **column**: ``x(gdp)``
↪ maps the column called ``gdp``.

Mapping a channel to a column that cannot answer its question. `size` encodes magnitude, and a category has none:

```
data(gapminder_2007) + point + x(gdp) + y(life) + size(continent)
```

Error:

! gog: ``size(continent)`` maps a categorical (text) column, but ``size`` on
↪ ``point`` needs a continuous (numeric) column. Use ``color``, ``shape``, or
↪ ``pattern`` to distinguish categories.

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

Forgetting the column a summary reads. The refusal names the missing y first, because the bar cannot stand without it:

```
data(gapminder_2007) + bar * mean + x(continent)
```

Error:

! gog: ``bar`` needs ``y()`` but none is set. Add ``y(<column>)`` - a bar cannot be
↪ drawn without it.

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

Binning a category. `bin` cuts a continuous axis, and the refusal points at the atom that does count categories:

```
data(gapminder_2007) + bar * bin + x(continent)
```

Error:

! gog: ``bin`` cuts a continuous axis into intervals, and ``x(continent)`` is
↪ categorical - a category is one slot, with no width to cut. To tally rows
↪ per category, ``count`` is the transform that does it: ``bar * count``.

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

Each message names the rule and the fix. Nothing was drawn without telling you, and no binding was dropped to complete the plot. Read any refusal outside this list the same way: it names the law the sentence broke, and it names the fix.

The last three refusals close with a second offer: set `GOG_STRICT=0` and draw the plot anyway. The first two never reach the engine, so there is nothing to draw. `GOG_STRICT` is an environment variable, and [Drawing it anyway](#) shows what it does to each kind of refusal, with the one line that sets it in each language. For now, make the fix the message asks for: at this stage the refusals are the lesson.

4.6 The morning checkpoint

If you started this part in one sitting, check yourself against its promise:

- Read any sentence in this book aloud before looking at its plot.
- Given a question (*distribution? comparison? relationship? change over time?*), name the mark and transform that answer it.
- Predict whether a sentence will render or refuse, and roughly what the refusal will say.

If that mostly works, the morning worked. Part II gives every letter its full chapter. Or go straight to [the cookbook](#) and answer your own questions, following a link back whenever a recipe uses a word you want explained.

Part II

The letters

After stating its system in brief, the 1446 book explained the letters one class at a time: the initial consonants, the vowels, the final consonants. Each class was explained by the same method.

This part does the same for `gog`'s letters. The marks are the initial consonants: visible forms that draw nothing until a position says where they go. The positions are the vowels: `x`, `y`, and `z`, plus the scales that decide how a number, a category, or a date becomes a place on an axis. The refining channels are the finals: `color`, `size`, `shape`, `pattern`, `opacity`. Each is optional, and each earns the legend that decodes it. `group` and `label` are neither. `group` splits a mark without encoding anything, so it earns no legend, and it works like the space between words. `label` supplies a `text` mark's words rather than refining them. `play` stands outside the syllable, the way faceting does: it cuts the plot into frames and plays them in order, and its chapter waits in the next part. Around the letters sit the chapters that finish a plot. `Data`, the table you bind once, opens this part, because nothing can be drawn until a table is named. `The book's data` follows, saying where this book's own tables come from and how to read one in your language. `Setting vs mapping` separates a column you *map* from a constant you *set*, and `Transforms`, which derives new values from your columns, closes the part.

`Choosing a mark` opens the letters themselves, and then each mark has a chapter of its own, fourteen in a row from `point` to `edge`. Every one follows the same template: what it draws, what it requires, what refines it, what derives from it, what you can set on it, and what it refuses. The last two are always the last two, under those names. That sameness is deliberate: it is the `No Exceptions` law applied to a book's structure. Learning the second mark is faster than the first, and by the fifth you already know the shape.

5 Data

Where does a plot get its data? Every sentence starts by naming a table, and everything after that depends on the name. This chapter covers the two ways to name one, the shape the table must have, and the declarations that travel with its columns.

5.1 Binding a table

`data()` takes a table you already hold in your session. Column names after that are bare words: no quotes, no table prefix, no indexing.

R

```
data(gapminder_2007) + point + x(gdp) + y(life)
```

Python

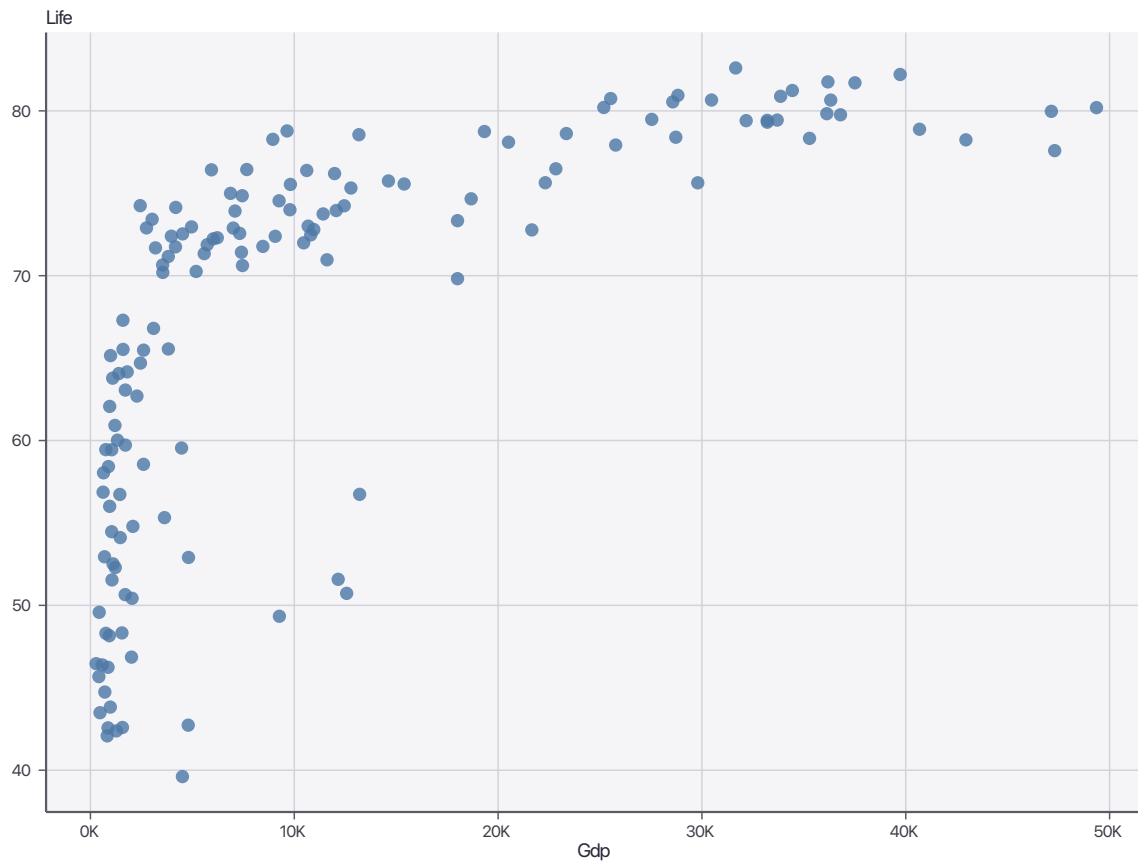
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life))
```

“Given gapminder 2007: points, x is gdp, y is life.”

`gog` looks up `gdp` and `life` inside the `gapminder_2007` table automatically. This is the **Bind-Once Law**: name the table once, then use columns freely.

`gapminder_2007` here is an ordinary table in your session, whatever your language calls one: a data frame in R, a dict of columns or a `pandas` `DataFrame` in Python, a named tuple of columns or a `DataFrame` in Julia, an object of columns in JavaScript. No registration step, no special wrapping required. Each chapter in the bindings part shows its own. The [R](#) chapter also lists the ten names that loading `gog` masks in R, with the qualified form of each.

Writing `data()` does more than supply the numbers. When you write `data(gapminder_2007)`, `gog` keeps the name `gapminder_2007`, not only the numbers inside it. The name matters as soon as a plot has more than one table, because each layer looks up its bare column names in the nearest table by name. Leave `data()` out and the table arrives with no name, so the plot refuses and tells you what to write:

```
gapminder_2007 + point + x(gdp) + y(life)
```

Error:

```
! gog: a plot starts with `data()`, which names the table - columns are bare
  ↳ names and the nearest named table wins, so the name matters. Write
  ↳ `data(gapminder_2007) + point + ...`.
```

gog gives the same advice for a sentence with no data at all, `point + x(gdp)` with nothing before it.

5.2 The table has to be tidy

Every table in this book has the same shape. Each row is one observation, and each column is one variable. That shape has a name, **tidy data** (Wickham, 2014), and gog assumes it in every sentence you write.

The reason is the rule that defines a channel in [Grammar](#): a channel maps a column to a visual property, so anything you want to map has to *be* a column. Suppose you record sales for two regions, and you give each region a column of its own:

```
wide <- data.frame(year = 2019:2023,
                   north = c(120, 135, 128, 152, 168),
                   south = c( 90, 101, 140, 133, 155))
```

Table 5.1: wide: all 5 rows

year	north	south
2019	120	90
2020	135	101
2021	128	140
2022	152	133
2023	168	155

Now ask for one line per region:

```
data(wide) + line + x(year) + y(sales) + color(region)
```

Error:

```
! gog: `y(sales)` refers to a column that is not in the data. Check the
  ↳ spelling of `sales`.
gog: `color(region)` refers to a column that is not in the data. Check the
  ↳ spelling of `region`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The message names two problems. There is no column called `sales`, and no column called `region`. Your two regions are column *names*, and a name is not a column. Your sales numbers sit in two columns instead of one. The sentence asks for columns the table does not have, so `gog` draws nothing.

The same numbers, one row per observation, look like this:

```
long <- data.frame(year = rep(2019:2023, times = 2),
                   region = rep(c("north", "south"), each = 5),
                   sales = c(120, 135, 128, 152, 168,
                             90, 101, 140, 133, 155))
```

Table 5.2: `long`: all 10 rows

year	region	sales
2019	north	120
2020	north	135
2021	north	128
2022	north	152
2023	north	168
2019	south	90
2020	south	101
2021	south	140
2022	south	133
2023	south	155

Now `sales` is a column and `region` is a column, so the sentence draws the plot and earns the legend:

R

```
data(long) + line + x(year) + y(sales) + color(region)
```

Python

```
data(long) + line + x(col.year) + y(col.sales) + color(col.region)
```

Julia

```
data(long) + line + x(:year) + y(:sales) + color(:region)
```

JavaScript

```
plot(data(long), line, x(col.year), y(col.sales), color(col.region))
```



“Given the long table: lines, x is year, y is sales, color by region.”

`gog` never reshapes a table, and no transform turns columns into rows. You do that work before you write the sentence, in the language you are already using. R has `tidyr::pivot_longer()`.

Python has `pandas.melt()`. Julia has `DataFrames.stack()`. JavaScript has no standard table library, so you write a short loop instead.

`gog` is stricter than `ggplot2` here, but not about the shape. `ggplot2` does not reshape tables either, and it refuses the wide table above just as `gog` does. The real difference is narrower. `ggplot2` lets you calculate a value while you map it, so `aes(x = log(gdp))` works there. `gog` does not allow that, because a channel takes a column name and nothing else. Some of those calculations are already part of the grammar. A log axis is one of them, written `x(gdp, scale = "log")`. For anything else, add the column to your table first. So tidy is not quite enough on its own. Every column your sentence names has to be in the table already.

This book does not teach you how to tidy data, and it should not, because there is a better book for it. Read *R for Data Science* by Hadley Wickham, Mine Çetinkaya-Rundel and Garrett Grolemund (Wickham et al., 2023), which is free online at <https://r4ds.hadley.nz/>. Its chapter on data tidying is the one that matters here. If the shape of your data is not settled yet, read that chapter before you read this one.

5.3 Plot-level vs. layer-level data

`data()` at the start of the expression is the **plot-level default**: it applies to all layers that don't declare their own table.

R

```
# Both layers read from "gapminder_2007"
data(gapminder_2007) + x(gdp) + y(life) +
  line +
  point
```

Python

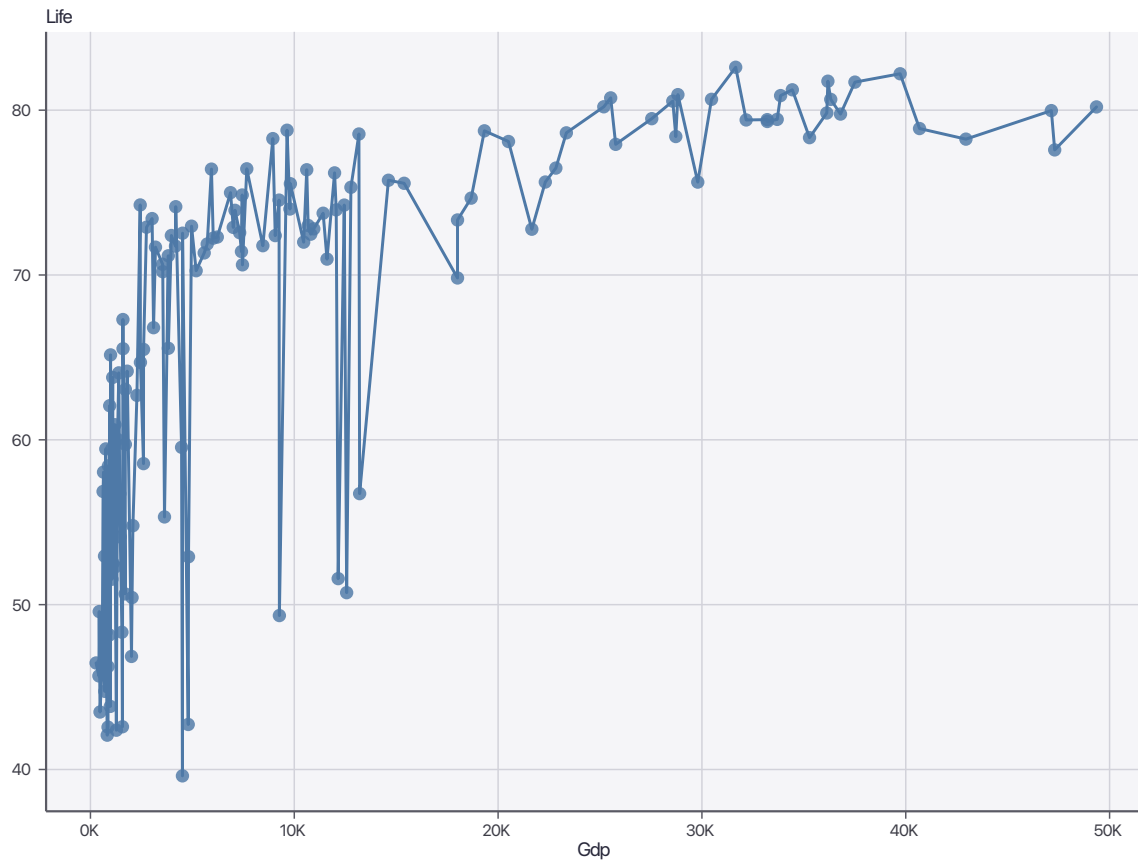
```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +
  line +
  point)
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + line + point
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), line, point)
```



“Given gapminder 2007: x is gdp, y is life, a line and also points.”

The next plots use two tables. **actuals** holds the sales of five years that have passed, and **forecast** holds the three years still to come:

Table 5.3: **actuals**: all 5 rows

year	sales
2019	120
2020	135
2021	128
2022	152
2023	168

Table 5.4: forecast: all 3 rows

year	sales
2024	180
2025	195
2026	210

A `data()` that appears **between two marks** overrides the table for the layer that follows it:

R

```
# line uses "actuals"; point uses "forecast"
data(actuals) + x(year) + y(sales) +
  line +
  data(forecast) + point
```

Python

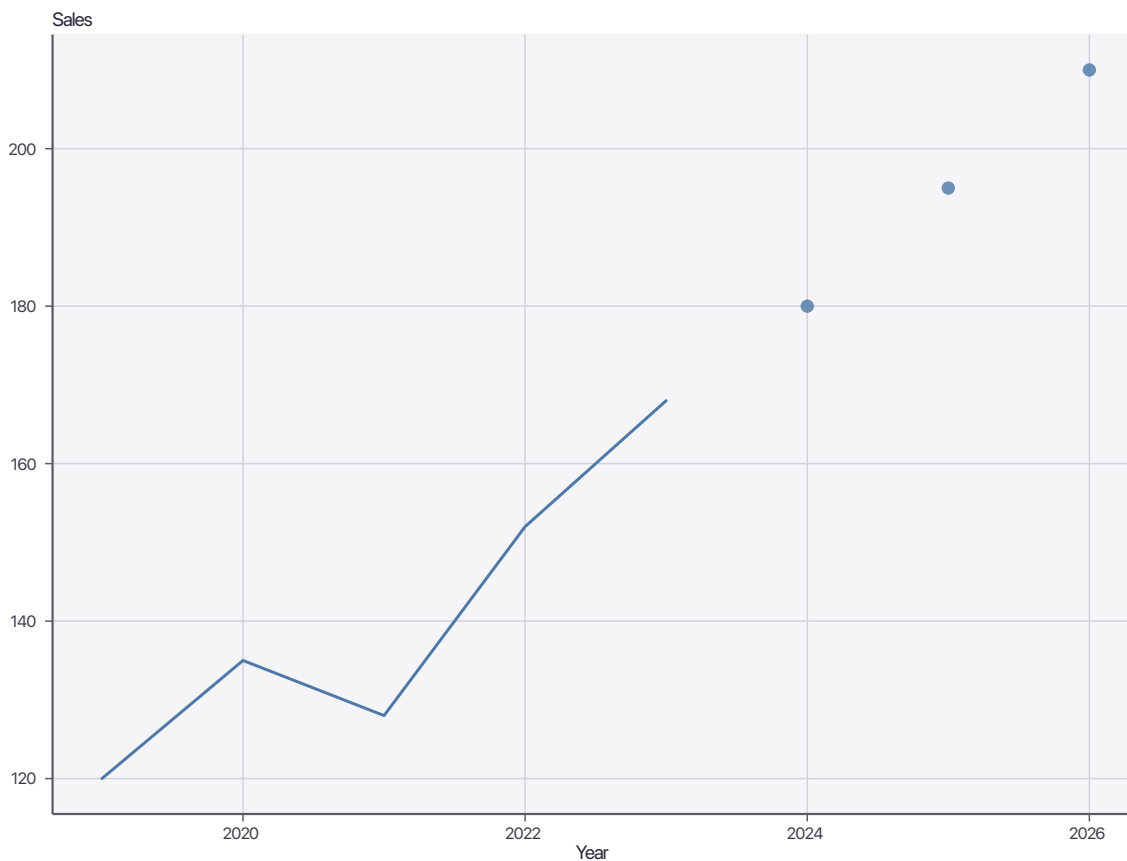
```
(data(actuals) + x(col.year) + y(col.sales) +
  line +
  data(forecast) + point)
```

Julia

```
data(actuals) + x(:year) + y(:sales) + line + data(forecast) + point
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), line, data(forecast),
  point)
```



“Given the actuals: x is year, y is sales, a line; then given the forecast: points.”

The rule: the first `data()` is the plot’s table, and each later `data()` applies to the mark written directly after it. A mark with no `data()` of its own reads the plot’s table. The refusal at the start of this chapter holds for the second table too: write `+ data(forecast)`, not `+ forecast`.

Two things follow from that rule, and both are easy to miss.

A `data()` at the end of a sentence applies to nothing, because no mark comes after it. The plot is the same as it would be without that line:

R

```
# `forecast` is never drawn, because no mark comes after it
data(actuals) + x(year) + y(sales) +
  line +
  point +
  data(forecast)
```


Python

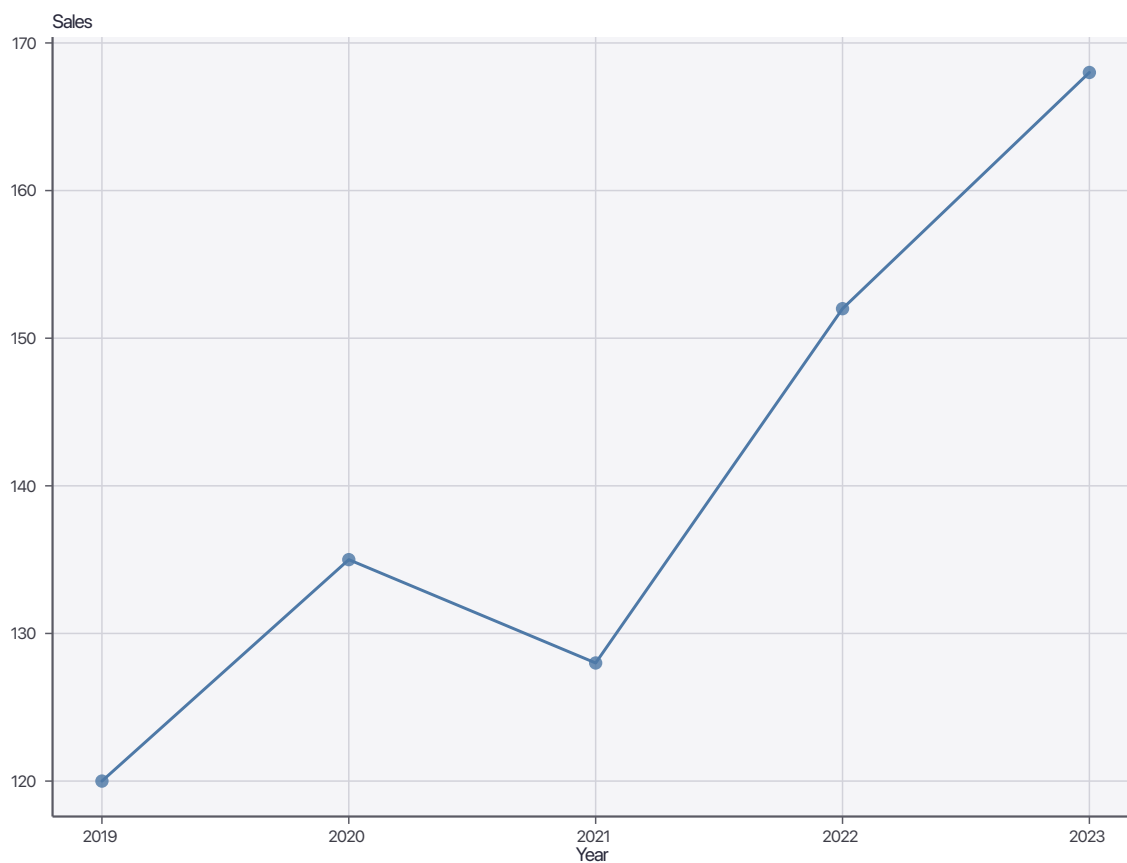
```
(data(actuals) + x(col.year) + y(col.sales) +  
    line +  
    point +  
    data(forecast))
```

Julia

```
data(actuals) + x(:year) + y(:sales) + line + point + data(forecast)
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), line, point,  
    data(forecast))
```



A `data()` also stops after that one mark. The mark after it reads the plot-level table again:

R

```
# line reads actuals. point reads forecast. area reads actuals again.
data(actuals) + x(year) + y(sales) +
  line +
  data(forecast) + point +
  area
```

Python

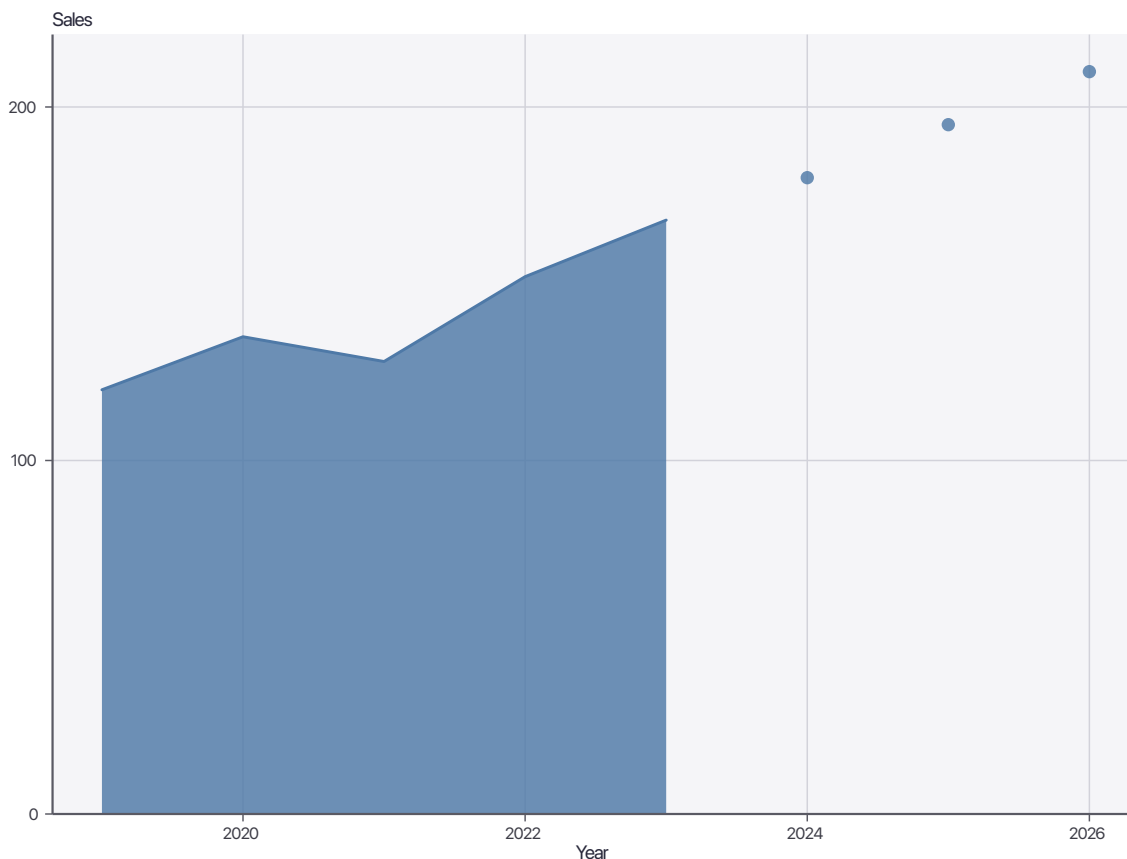
```
(data(actuals) + x(col.year) + y(col.sales) +
  line +
  data(forecast) + point +
  area)
```

Julia

```
data(actuals) + x(:year) + y(:sales) + line + data(forecast) + point +
  area
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), line, data(forecast),
  point, area)
```



“Given the actuals: x is year, y is sales, a line; then given the forecast: points; then given the actuals again: an area.”

The two tables cover different years, which makes the change easy to see. **actuals** runs from 2019 to 2023, and **forecast** from 2024 to 2026. The area and the line both sit on the left. The points sit alone on the right.

That rule fits what a second table is usually for. The first table builds the frame, and a later one annotates it: a threshold, a band, a note, a forecast. An annotation is one mark, so a **data()** that reaches one mark is enough. Every second table in this book is one of those.

To draw two marks from the second table, write **data()** before each of them.

The specification above is correct, and it still reads badly. **area** sits on the line below **data(forecast)**, so it looks like part of the forecast. Only the rule says otherwise. The same plot reads better with the marks grouped by the table they read. Drawn again, it is the same picture:

R

```
data(actuals) + x(year) + y(sales) +  
  line + area +  
  data(forecast) + point
```

Python

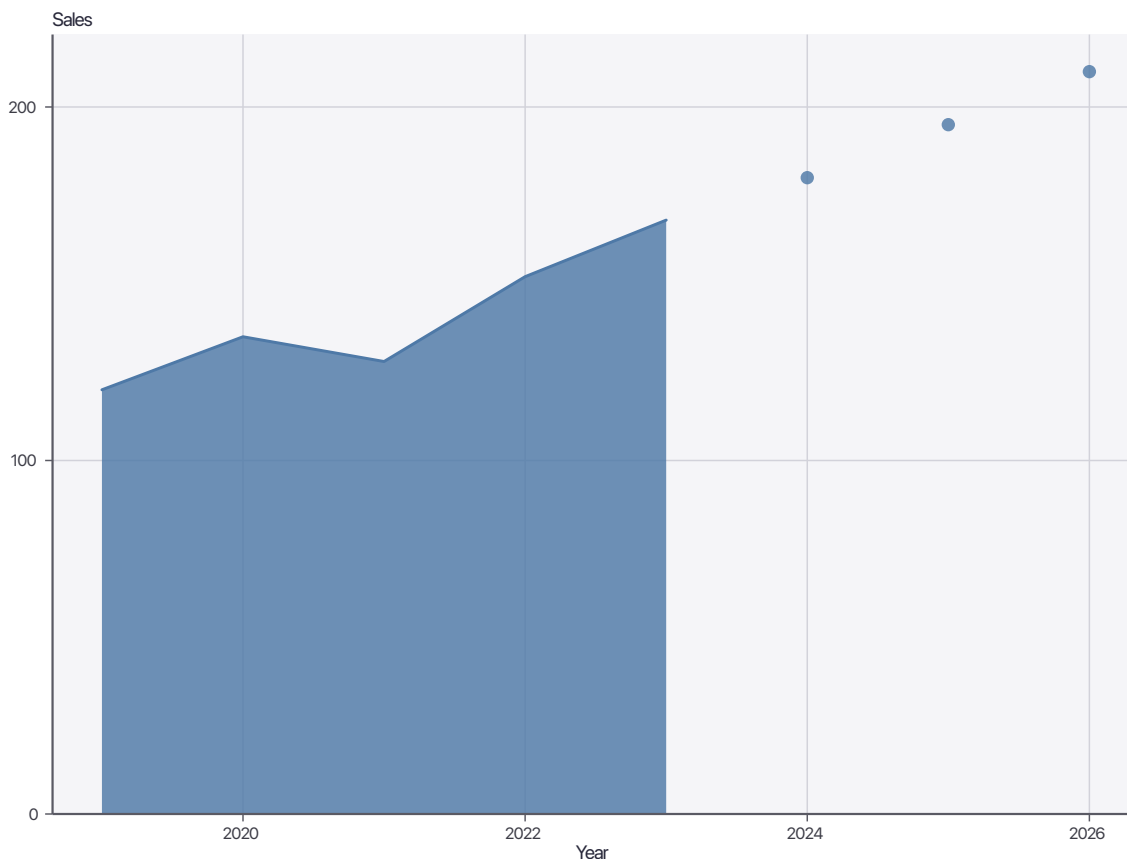
```
(data(actuals) + x(col.year) + y(col.sales) +  
  line + area +  
  data(forecast) + point)
```

Julia

```
data(actuals) + x(:year) + y(:sales) + line + area + data(forecast) +  
  point
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), line, area, data(forecast),  
  point)
```



“Given the actuals: x is year, y is sales, a line and also an area; then given the forecast: points.”

This is style, not grammar. Both spellings are legal, and both draw the same plot. The order of **area** and **point** has changed, and here that changes nothing. The two tables share no year, so neither mark is drawn over the other. **gog** will never refuse either spelling, because legal is not the same as clear ([Law 8](#)). Two sentences can mean the same thing and still be very different to read. That difference is yours to control.

So group the marks by the table they read. Keep each extra **data()** on the same line as the mark it applies to, and keep the marks that read the plot-level table together. Every line is then one table and the marks that read it, and no mark sits under a table it does not read. The rest of this book follows that layout.

The first **data()** is the one exception. It applies to every mark below it, so it takes a line of its own. A later **data()** applies to one mark only. On a line of its own, it would look like it applies to every mark below it, as the first one does. That is why **data(actuals)** stands alone above and **data(forecast)** shares its line with **point**.

5.4 When the second table names its columns differently

The plot above was the easy case: `actuals` and `forecast` both call their columns `year` and `sales`, so one pair of positions written at the front served both layers.

A second table usually does not match. The `milestones` table marks two points on the income axis:

Table 5.5: `milestones`: all 2 rows

at	value	note
10000	72	income takes off
40000	80	the long plateau

Its columns are `at`, `value` and `note`. None of those names appear in `gapminder_2007`, whose columns are `gdp` and `life`. A layer may name its own columns, and the axis does not change:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  data(milestones) + text + x(at) + y(value) + label(note) +  
  style(size = 20, color = "red")
```

Python

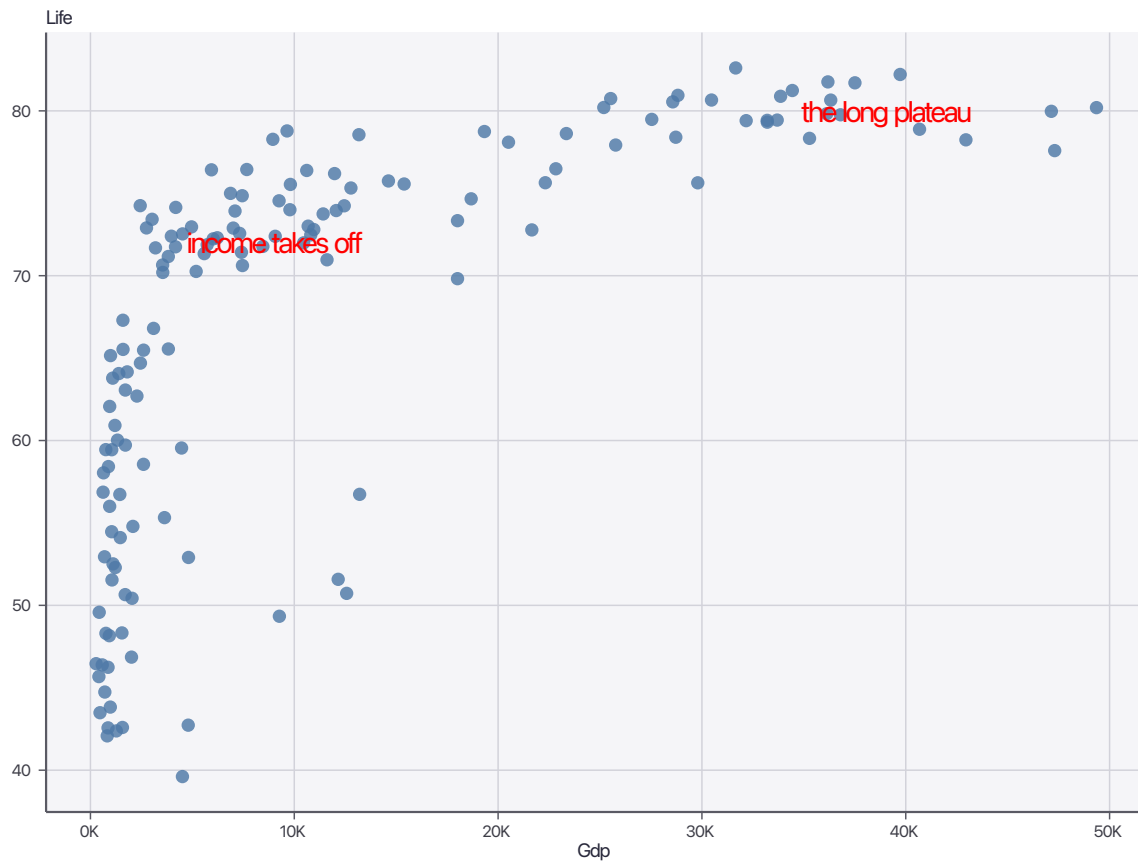
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  data(milestones) + text + x(col.at) + y(col.value) + label(col.note) +  
  style(size = 20, color = "red"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + data(milestones) +  
  text + x(:at) + y(:value) + label(:note) +  
  style(size = 20, color = "red")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  data(milestones), text, x(col.at), y(col.value), label(col.note),  
  style({ size: 20, color: "red" })))
```



“Given gapminder 2007: points, x is gdp, y is life; then given the milestones: text, x is at, y is value, label by note.”

There is still one x axis and one y axis, one scale each, one set of ticks. Only the *names* differ, because the two tables were written by different people for different reasons.

What is shared is the **measurement**, and that is what an axis is. **at** holds income and **value** holds years of life, the same two quantities **gdp** and **life** hold. A table whose numbers mean something else does not belong on these axes, however its columns are spelled: put sales figures on an income axis and the plot is drawn correctly and reads as nonsense. Matching names are not the test. Matching meanings are.

The same test says when a second table does not belong in the plot at all. A layer is drawn on the axes the first table set, so its values must measure what those axes measure. Values that measure something else need axes of their own, and axes of their own means a plot of its own. Put that plot beside the first one instead. [Composition](#) shows how, with | and /.

A layer that names no position of its own reads the axis that is already there. That is why the earlier plot worked with the positions written once at the front.

5.5 A table in a database

Not every table fits on one computer. Most working data sits in a database, and a **warehouse** (a database built to hold a company's whole history) can hold billions of rows. `gog` reaches those tables with one word, `query`, and it stands exactly where `data` stands.

Nothing else in the sentence changes:

```
data(orders)                                + bar + x(status)
query(connection, "SELECT ...")             + bar + x(status)
```

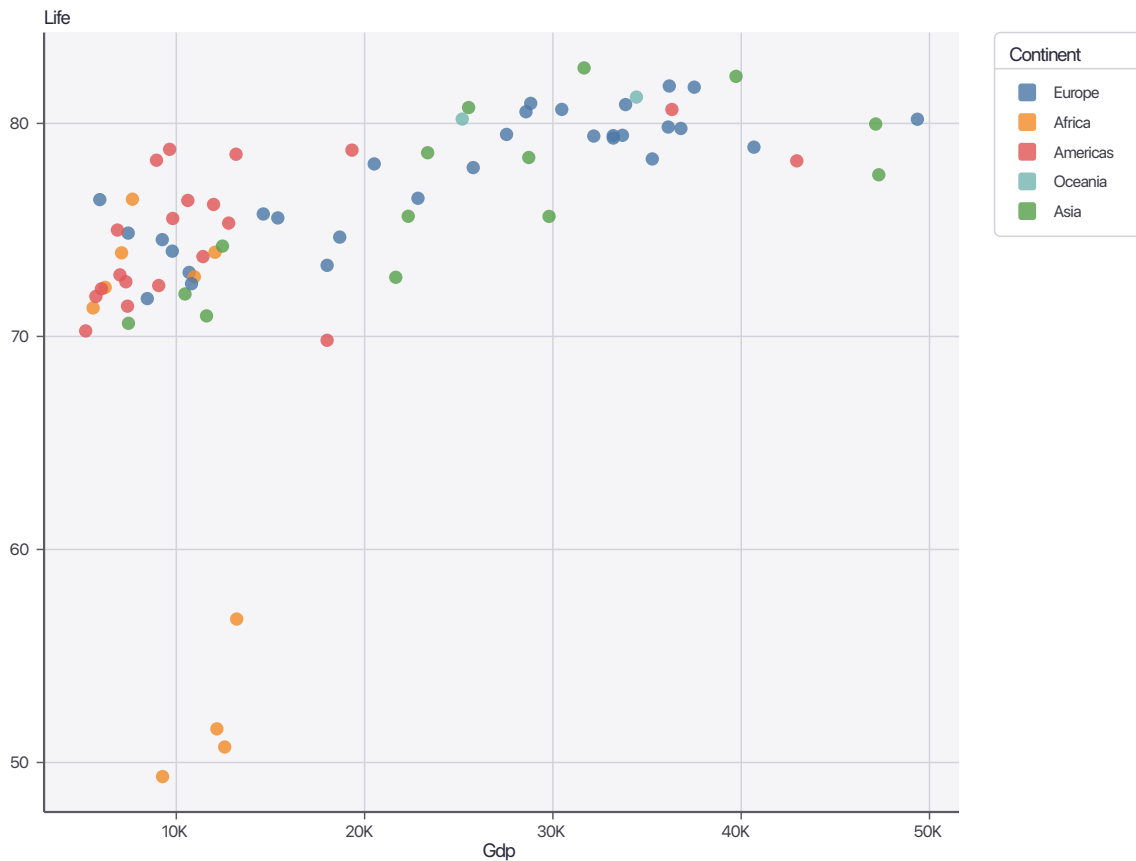
The SQL stays inside that one word; nothing after it is SQL. After it, the columns are still bare names. The marks, channels, transforms and operators are the ones you have already learned. You are not writing a second language for databases. You are naming the rows a different way.

Here is a real database. The code below builds a small one in memory, so it needs no server and no password:

```
con <- DBI::dbConnect(RSQLite::SQLite(), ":memory:")
DBI::dbWriteTable(con, "countries", gapminder_2007)
```

Now the sentence. The connection is the only new part:

```
query(con, "SELECT gdp, life, continent FROM countries WHERE gdp > 5000") +
  point + x(gdp) + y(life) + color(continent)
```

“Given the query: points, x is gdp, y is life, color by continent.”

The plot is an ordinary scatter plot, because the grammar did an ordinary thing. The database chose which rows to send. `gog` drew what arrived.

A query and a loaded table produce *the same picture, byte for byte*, when they hold the same rows. Every binding tests exactly that. So `query` is never a different way of drawing; it is a different way of finding the rows to draw.

5.5.1 Three things to know

The connection is yours. `gog` never opens one, and it never stores a password. You connect with whatever your language already uses, and hand the open connection to `query`. That also means `gog` adds no database driver to your install. A reader who never writes SQL never installs one.

The query runs when the plot is drawn, not when the sentence is written. Writing `query(con, "...")` asks the database nothing. The sentence is a specification, exactly like every other sentence in this book, and the engine draws it later. The delay is not only neatness; it makes something else possible. A specification the engine can read *before* the database runs is one where the engine could rewrite the question, and ask for a summary instead of the rows. Counting is a job a database does well, and the design allows for that later.

Every language reaches a different set of databases. The word is the same in all four. What differs is the connection you hand it, because each language has its own database convention. Your language's chapter says which: [R](#), [Python](#), [Julia](#), [JavaScript](#).

5.5.2 How much data comes back

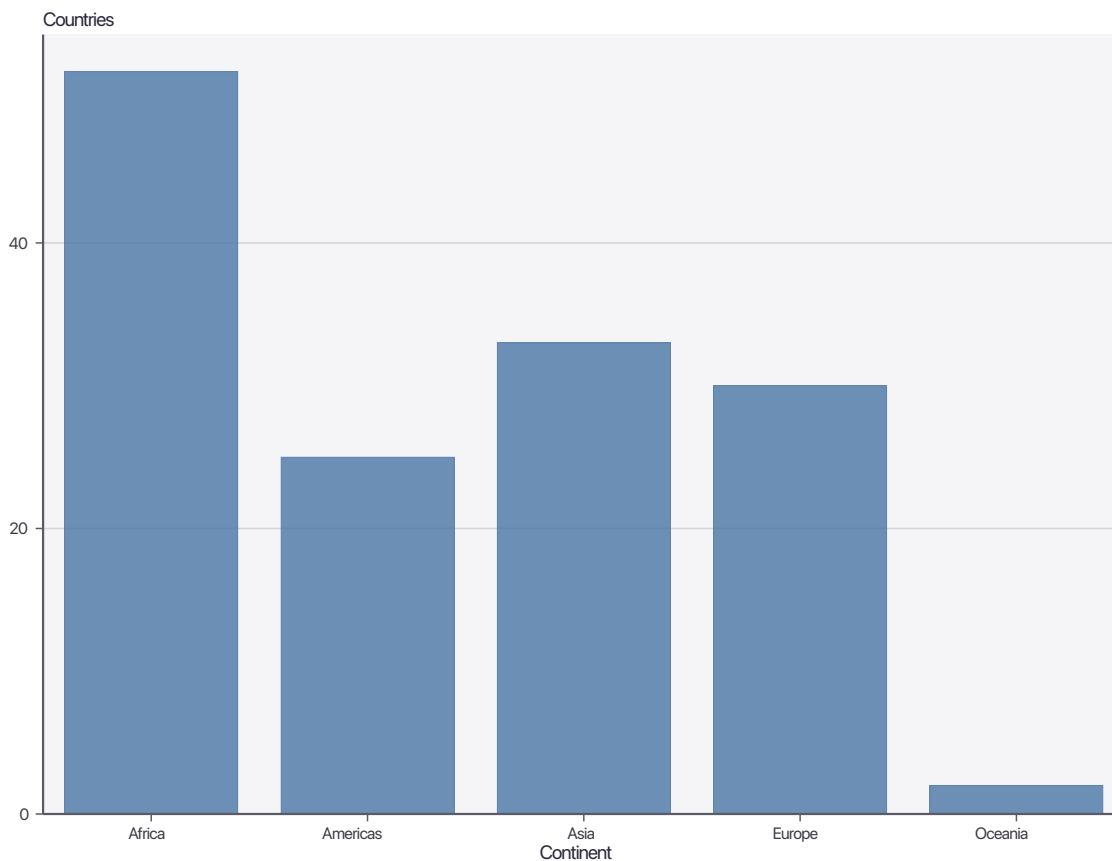
`query()` brings *every row the query returns* to the computer running `gog`. The database does the selecting. Your machine does the drawing. Know that before you run this against a large table.

For most tables it does not matter. A few thousand rows arrive quickly, so write the simple query and stop there. Selecting the columns you need and letting `gog` do the rest is the normal case, and the example above is exactly that.

It starts to matter somewhere around a hundred thousand rows. Past that, moving the rows costs more than drawing them. A warehouse table can hold billions, and `SELECT * FROM that_table` would try to bring all of them to you.

The way to avoid it is already familiar to anyone who works with a warehouse: *summarize in SQL, then draw the summary*. This works because a plot is nearly always a summary anyway. A bar chart of a billion orders shows only a few bars, and a database can produce those bars far more cheaply than your laptop can.

```
query(con, "SELECT continent, COUNT(*) AS countries
          FROM countries GROUP BY continent") +
  bar + x(continent) + y(countries)
```



“Given the query: bars, x is continent, y is countries.”

Five rows crossed the connection, not 142. The plot is the same one you would get by counting locally.

5.5.3 Two directions, and which one suits you

There is another way to give SQL users a grammar of graphics. Posit’s **ggsql** (Posit Software, PBC, 2026) adds visual clauses to SQL itself: you write **VISUALIZE**, **DRAW** and **SCALE** inside the query, and the database returns a picture.

It has two strengths. Its clauses read like SQL, so someone who writes SQL all day stays in the language they already think in, and no host language is installed at all. And it runs the whole pipeline in the database, one query per layer, so a bar chart of ten billion rows fetches only the bar heights. On that second point **ggsql** does something **gog** does not. **gog** brings the rows to you, so on a large table you write the summary in SQL yourself, as the count above did.

If SQL is where you work, `ggsql` is probably the better fit. `gog` would first ask you to learn R, Python, Julia or JavaScript, which is a real cost for someone who uses none of them.

`gog` is built in the opposite direction. The grammar stays in one place and the data comes to it, so one sentence draws a loaded table, a table in Postgres, and a table in a warehouse, in any of four languages. One syntax, in as many languages and places as we can reach.

You work	The lower cost is
Only in SQL	ggsql : nothing to install, and the syntax is the one you use
Across several languages	gog : one grammar covers all of them, and the database too

5.6 Declaring the order of categories

A plain text column has no order of its own, so `gog` draws its categories in the order the rows happen to arrive:

R

```
severity <- data.frame(
  level = c("High", "Low", "Medium"),
  count = c(30.0, 10.0, 20.0)
)

data(severity) + bar + x(level) + y(count) +
  title("Text column: the order the rows came in")
```

Python

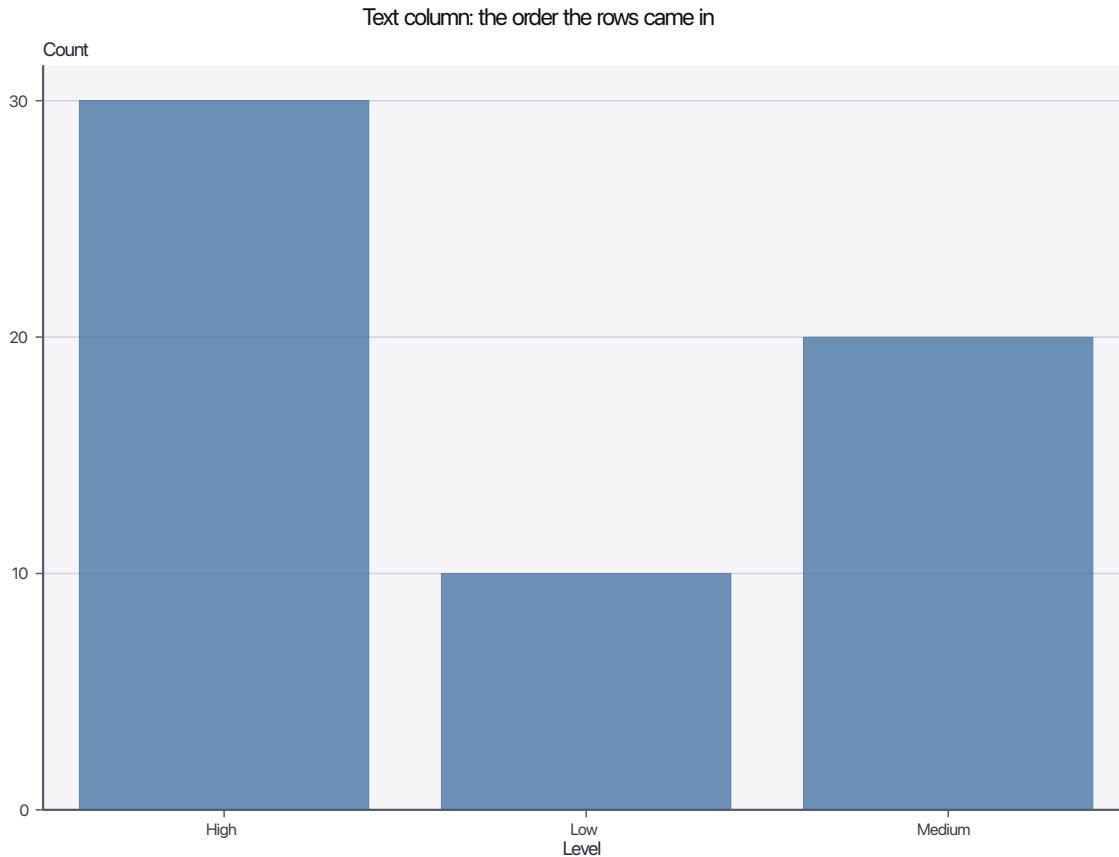
```
severity = {"level": ["High", "Low", "Medium"], "count": [30.0, 10.0, 20.0]}
(data(severity) + bar + x(col.level) + y(col.count) +
  title("Text column: the order the rows came in"))
```

Julia

```
severity = (level = ["High", "Low", "Medium"], count = [30, 10, 20],)
data(severity) + bar + x(:level) + y(:count) +
  title("Text column: the order the rows came in")
```

JavaScript

```
const severity = { level: ["High", "Low", "Medium"], count: [30, 10, 20] };
plot(data(severity), bar, x(col.level), y(col.count),
  title("Text column: the order the rows came in"))
```



That is rarely what you want here. Declare the order you mean, where you build the table. In R the declaration is a **factor**. Python, Julia and JavaScript have no factor type, so **gog** supplies the declaration itself, under one name: **ordered()**. It takes the column's values and the order its categories go in, and everything after it reads the same in all four languages:

R

```
severity <- data.frame(
  level = factor(c("High", "Low", "Medium"), levels = c("Low", "Medium",
↪ "High")),
```

```

    count = c(30.0, 10.0, 20.0)
)

data(severity) + bar + x(level) + y(count) +
  title("Declared: the order you asked for")

```

Python

```

severity = {"level": ordered(["High", "Low", "Medium"], ["Low", "Medium",
↪ "High"]) , "count": [30.0, 10.0, 20.0]}
(data(severity) + bar + x(col.level) + y(col.count) +
  title("Declared: the order you asked for"))

```

Julia

```

severity = (level = ordered(["High", "Low", "Medium"], ["Low", "Medium",
↪ "High"]), count = [30, 10, 20],)
data(severity) + bar + x(:level) + y(:count) +
  title("Declared: the order you asked for")

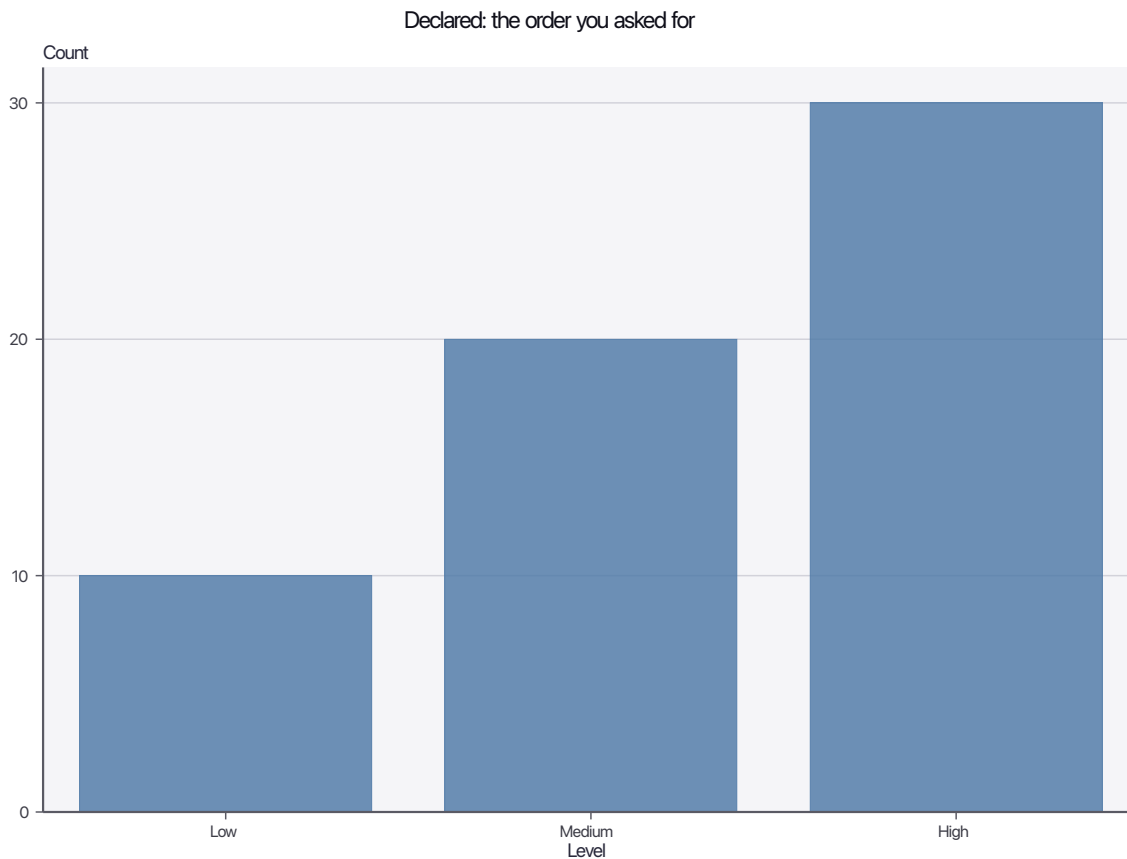
```

JavaScript

```

const severity = { level: ordered(["High", "Low", "Medium"], ["Low",
↪ "Medium", "High"]), count: [30, 10, 20] };
plot(data(severity), bar, x(col.level), y(col.count),
  title("Declared: the order you asked for"))

```



The order goes with the column, so *every part of the plot agrees*: the axis, the color assignment, and the legend all read in the same order.

R

```
data(severity) + point + x(count) + y(count) + color(level) +
  title("The legend follows the same order")
```

Python

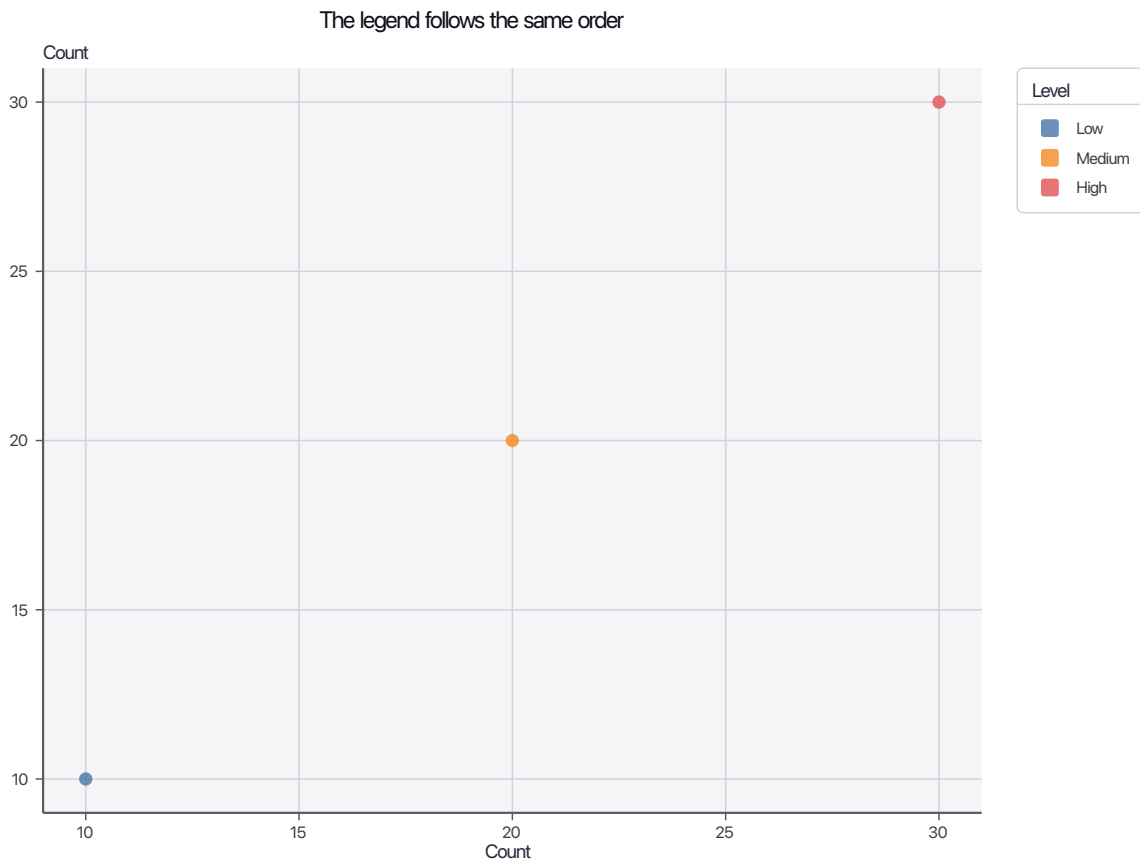
```
(data(severity) + point + x(col.count) + y(col.count) + color(col.level) +
  title("The legend follows the same order"))
```

Julia

```
data(severity) + point + x(:count) + y(:count) + color(:level) +
  title("The legend follows the same order")
```

JavaScript

```
plot(data(severity), point, x(col.count), y(col.count), color(col.level),
  title("The legend follows the same order"))
```



The order holds even when `gog` does the counting for you. These tickets arrive **High** first, but `bar * count` still reads Low, Medium, High. A summary carries the declared order onto its axis. It does not return to the order the rows arrived in:

R


```

tickets <- data.frame(
  level = factor(
    c("High", "Low", "High", "Medium", "High", "Low", "Medium", "High"),
    levels = c("Low", "Medium", "High")
  )
)

data(tickets) + bar * count + x(level) +
  title("bar * count: the levels still order the axis")

```

Python

```

tickets = {"level": ordered(["High", "Low", "High", "Medium", "High", "Low",
↪ "Medium", "High"], ["Low", "Medium", "High"])}
(data(tickets) + bar * count + x(col.level) +
  title("bar * count: the levels still order the axis"))

```

Julia

```

tickets = (level = ordered(["High", "Low", "High", "Medium", "High", "Low",
↪ "Medium", "High"], ["Low", "Medium", "High"]),)
data(tickets) + bar * count + x(:level) +
  title("bar * count: the levels still order the axis")

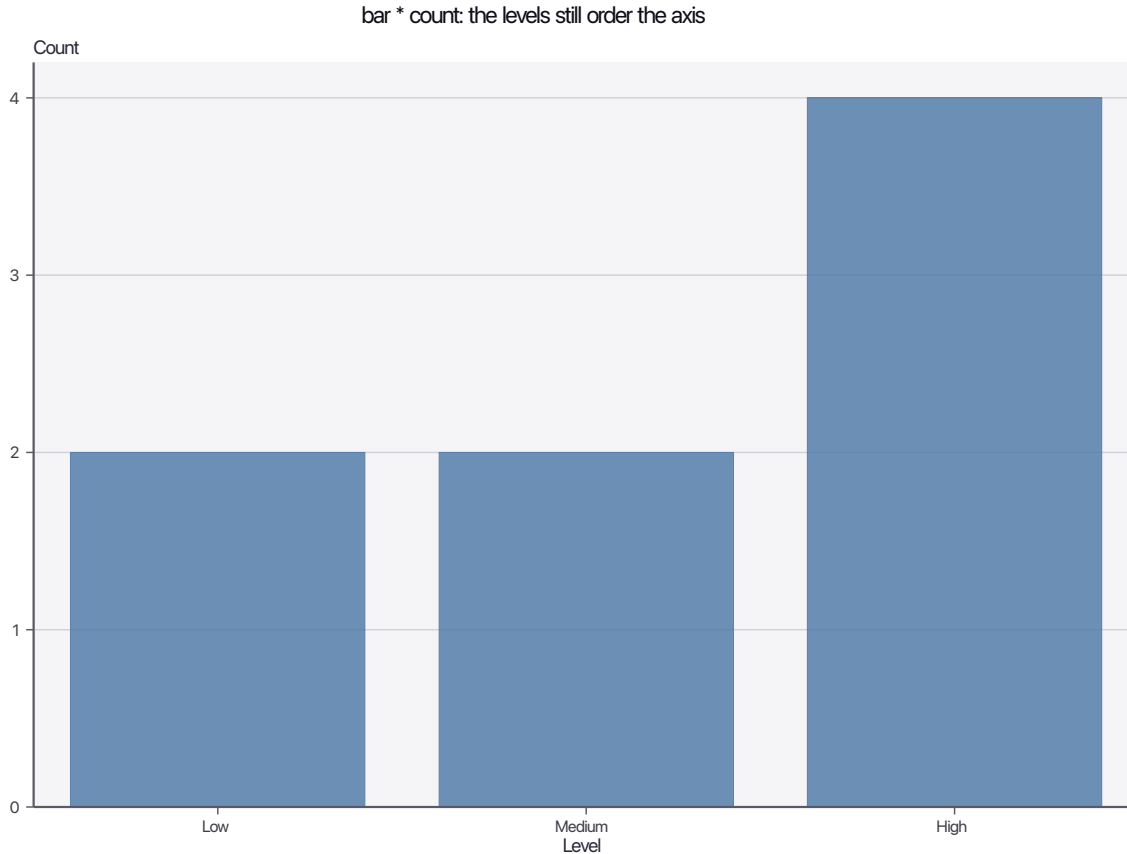
```

JavaScript

```

const tickets = { level: ordered(["High", "Low", "High", "Medium", "High",
↪ "Low", "Medium", "High"], ["Low", "Medium", "High"]) };
plot(data(tickets), layer(bar, count), x(col.level),
  title("bar * count: the levels still order the axis"))

```



“Given the tickets: bars derived by count, x is level.”

In R, the list of levels can mean two things. `factor(x, levels = ...)` says only which order the categories appear in. Adding `ordered = TRUE` says more: that Low really is *less than* High. `gog` draws the categories in the listed order either way. Most people list levels only to set the order on the page, and a plot needs nothing more than that.

In Python, a `pandas` table can carry the same declaration as a `Categorical` column, and `gog` reads its categories, in their order, as the levels. `ordered()` is for a plain dictionary of lists, so that you can declare an order without `pandas` installed.

Two rules worth knowing:

- **Levels set the order.** The data sets which categories appear. A level with no rows in the data gets no bar and no slot on the axis, because an empty labeled gap is harder to read than a shorter axis. A value in the data that the levels do not mention is not dropped. It is drawn after the listed ones.

- **order() in the plot wins.** The levels came with the table, so they apply to every plot drawn from it. `order()` is written in this one plot, so it applies here, and the more specific instruction wins. Name a different column in `order()`, and the categories follow that column's values instead. That is how you sort by value:

R

```
data(severity) + bar + x(level) + y(count) + order(count, desc = TRUE) +  
  title("order() overrides the levels")
```

Python

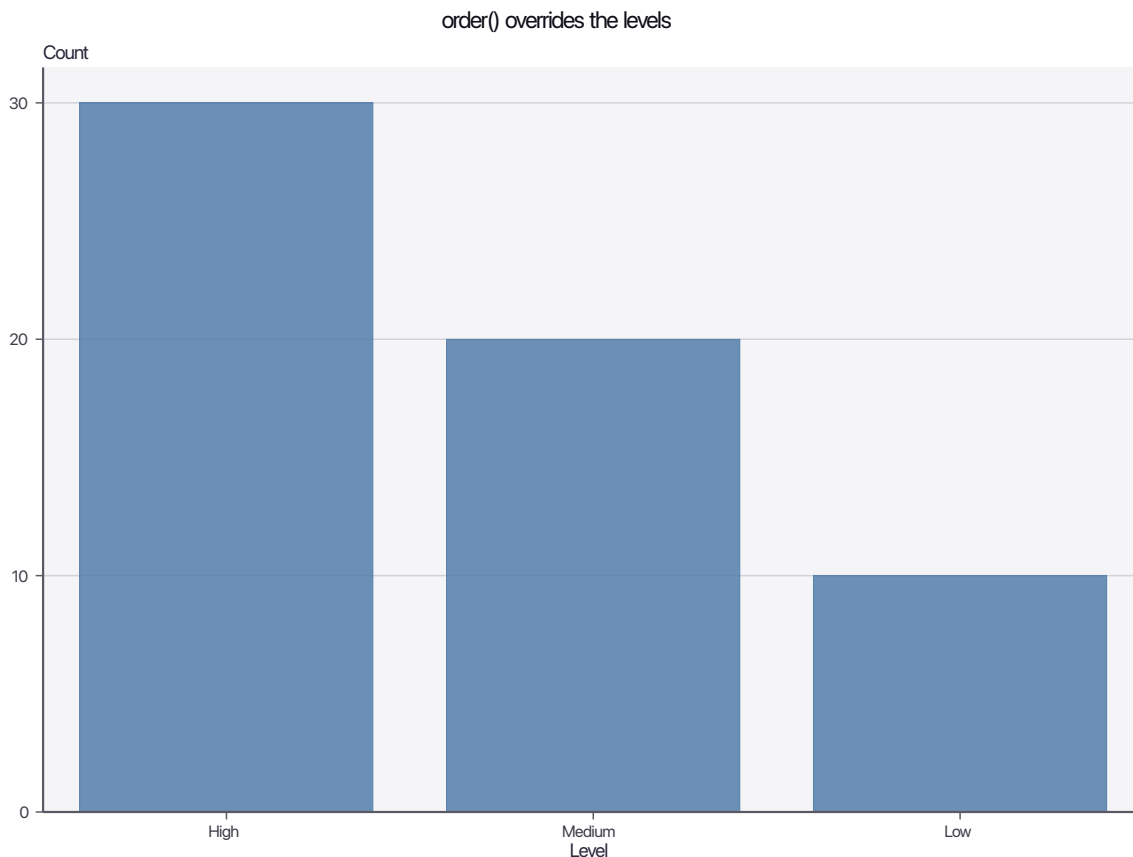
```
(data(severity) + bar + x(col.level) + y(col.count) + order(col.count, desc =  
↪ True) +  
  title("order() overrides the levels"))
```

Julia

```
data(severity) + bar + x(:level) + y(:count) +  
  order(:count, desc = true) + title("order() overrides the levels")
```

JavaScript

```
plot(data(severity), bar, x(col.level), y(col.count),  
  order(col.count, { desc: true }), title("order() overrides the levels"))
```



“Given the severity table: bars, x is level, y is count, ordered by count, largest first.”

`order()` can also name the categorical column itself, as in `order(level)`. That asks for the column’s own order. For a plain text column, that order is alphabetical. For a column with declared levels, it is the declared order. `desc = TRUE` reverses whichever applies. So a declared order cannot be turned alphabetical from inside the plot. To change it, change the levels where the table was built, in the language you are already using. The declaration belongs to the table, and so does the change.

5.7 Missing values

Real data has gaps, and survey tables are usually missing many values. A row with no value where the plot needs one cannot be placed, so `gog` drops it and *says how many*, rather than changing the count without telling you:

R

```

patchy <- data.frame(
  weight = c(3.1, 4.2, NA, 5.8, 6.0, NA, 4.9, 5.1, 3.7, 5.5),
  batch  = c("A", "A", "B", "B", NA, "B", "A", "B", "A", "B")
)
data(patchy) + bar * bin + x(weight) +
  x_label("Weight") + title("Two rows have no weight")

```

Python

```

patchy = {"weight": [3.1, 4.2, None, 5.8, 6.0, None, 4.9, 5.1, 3.7, 5.5],
  ↪ "batch": ["A", "A", "B", "B", None, "B", "A", "B", "A", "B"]}
(data(patchy) + bar * bin + x(col.weight) +
  x_label("Weight") + title("Two rows have no weight"))

```

Julia

```

patchy = (weight = [3.1, 4.2, missing, 5.8, 6, missing, 4.9, 5.1, 3.7, 5.5],
  ↪ batch = ["A", "A", "B", "B", missing, "B", "A", "B", "A", "B"],)
data(patchy) + bar * bin + x(:weight) + x_label("Weight") +
  title("Two rows have no weight")

```

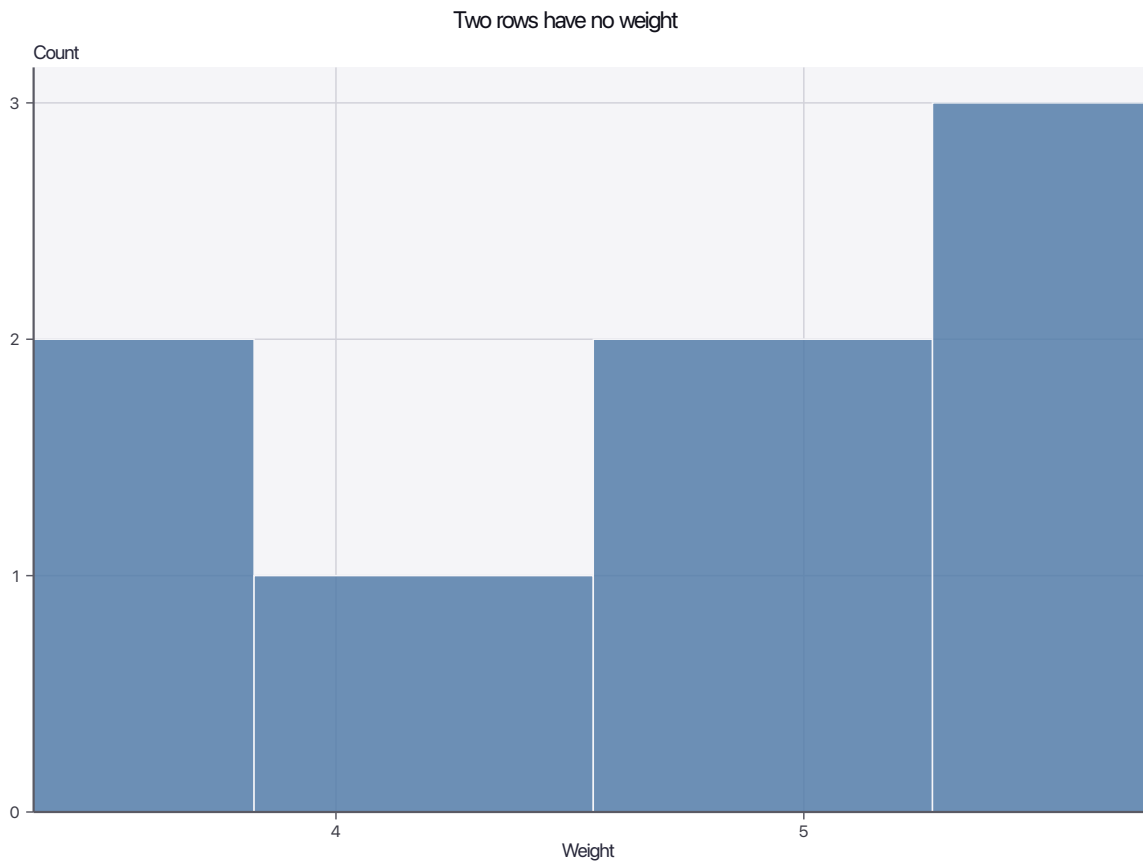
JavaScript

```

const patchy = { weight: [3.1, 4.2, null, 5.8, 6, null, 4.9, 5.1, 3.7, 5.5],
  ↪ batch: ["A", "A", "B", "B", null, "B", "A", "B", "A", "B"] };
plot(data(patchy), layer(bar, bin), x(col.weight), x_label("Weight"),
  title("Two rows have no weight"))

```

gog: dropped 2 rows of `patchy` with a missing value in `weight` - a row with
 ↪ no value in a column the plot maps cannot be placed, so it is left out
 ↪ (the same as other plotting tools drop NA).



“Given the patchy table: bars derived by bin, x is weight.”

Only the columns the plot maps decide which rows are dropped. This plot maps `weight`, so the message names `weight`, and the two rows with no weight are left out. The table also has one row with no `batch`. The plot never reads `batch`, so that row stays in the histogram. Bind `batch` to `color`, and the same row is dropped as well, because it now has no color to take. The message then names both columns:

R

```
data(patchy) + bar * bin + x(weight) + color(batch) +
  x_label("Weight") + title("One more column mapped, one more row dropped")
```

Python

```
(data(patchy) + bar * bin + x(col.weight) + color(col.batch) +
  x_label("Weight") + title("One more column mapped, one more row dropped"))
```

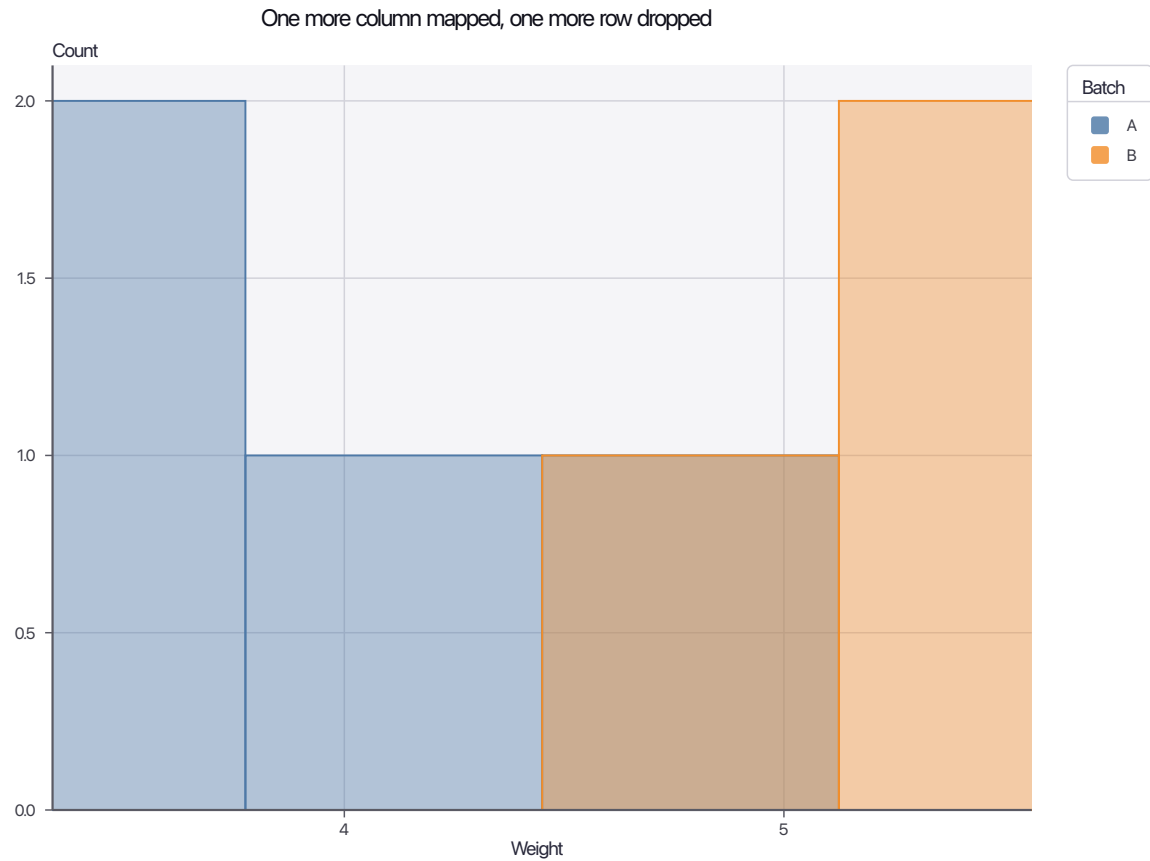
Julia

```
data(patchy) + bar * bin + x(:weight) + color(:batch) +
  x_label("Weight") +
  title("One more column mapped, one more row dropped")
```

JavaScript

```
plot(data(patchy), layer(bar, bin), x(col.weight), color(col.batch),
  x_label("Weight"),
  title("One more column mapped, one more row dropped"))
```

gog: dropped 3 rows of `patchy` with a missing value in `batch`, `weight` - a
 ↪ row with no value in a column the plot maps cannot be placed, so it is
 ↪ left out (the same as other plotting tools drop NA).



“Given the patchy table: bars derived by bin, x is weight, color by batch.”

So a missing value in a column the plot never reads changes nothing. A missing value in a column the plot maps always drops the row, and the message says which column.

5.8 Special characters and other writing systems

Category names, titles, and axis labels are drawn exactly as you wrote them. Three characters have a special meaning inside an SVG file: `&`, `<` and `>`. `gog` writes them in the form the file needs, so they print as themselves, and a name like **R&D** needs no special handling. The plot below puts all three into its names and its title:

R


```
special <- data.frame(
  firm = c("R&D", "Sales & Ops", "<10% unit"),
  spend = c(10, 25, 15)
)
data(special) + bar + x(firm) + y(spend) + title("Q3 <spend> & forecast")
```

Python

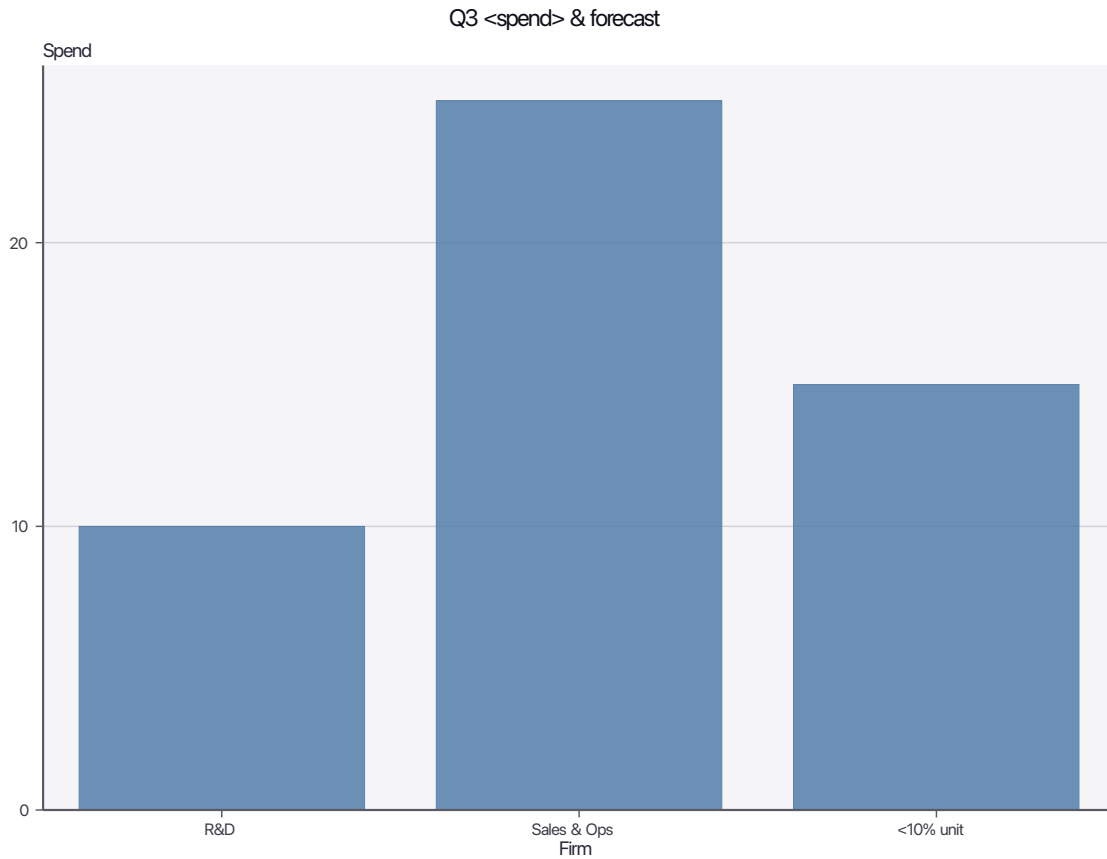
```
special = {"firm": ["R&D", "Sales & Ops", "<10% unit"], "spend": [10, 25,
↪ 15]}
data(special) + bar + x(col.firm) + y(col.spend) + title("Q3 <spend> &
↪ forecast")
```

Julia

```
special = (firm = ["R&D", "Sales & Ops", "<10% unit"], spend = [10, 25, 15],)
data(special) + bar + x(:firm) + y(:spend) +
  title("Q3 <spend> & forecast")
```

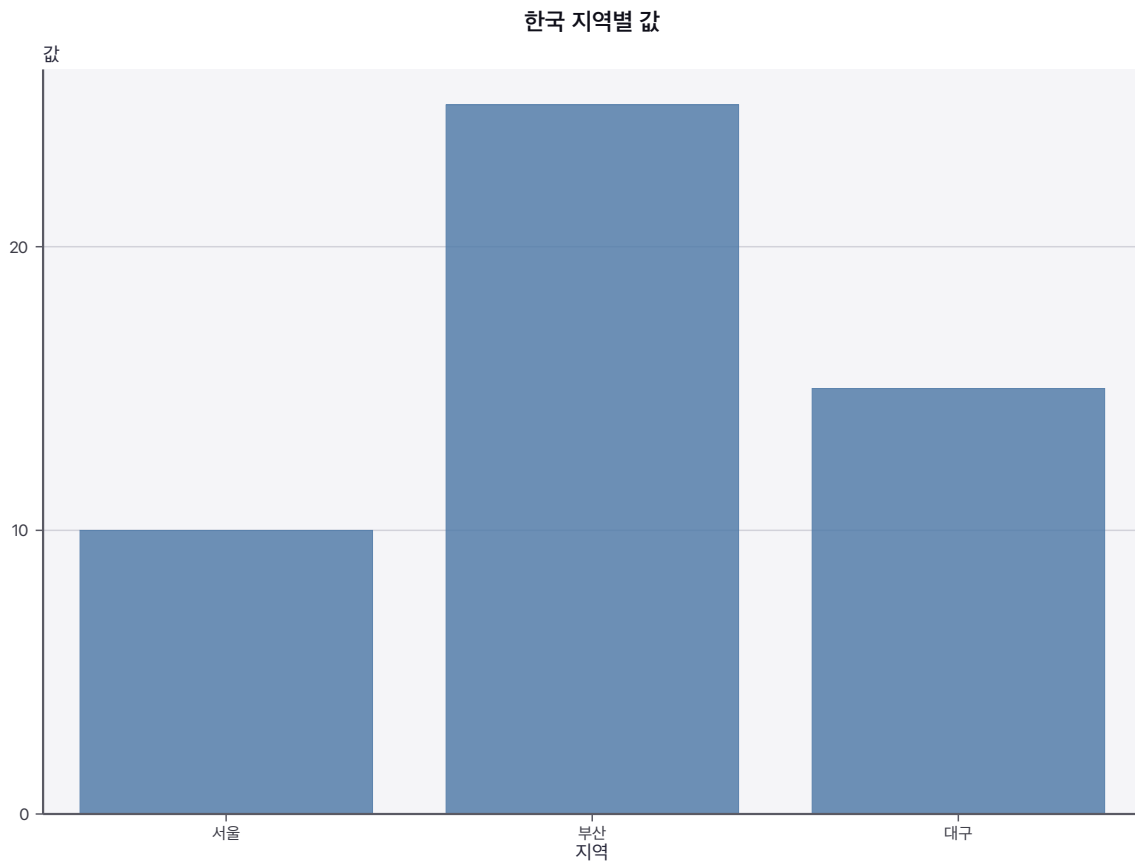
JavaScript

```
const special = { firm: ["R&D", "Sales & Ops", "<10% unit"], spend: [10, 25,
↪ 15] };
plot(data(special), bar, x(col.firm), y(col.spend),
  title("Q3 <spend> & forecast"))
```



A name in another writing system is also drawn as written, and it can be a bare column name as well as a label. The table below is in Korean. Its name, 지역별 (*jiyeokbyeol*), means by region. Its two columns are 지역 (*jiyeok*), region, and 값 (*gap*), value. The three rows are the cities 서울 (*Seoul*), 부산 (*Busan*) and 대구 (*Daegu*). The plot title 한국 지역별 값 (*Hanguk jiyeokbyeol gap*) reads *value by region, Korea*.

```
지역별 <- data.frame(
  지역 = c("서울", "부산", "대구"),
  값 = c(10, 25, 15)
)
data(지역별) + bar + x(지역) + y(값) + title("한국 지역별 값")
```



A label's width is measured from its characters, and a Korean or Chinese character is about twice as wide as a Latin one, so these labels get the margin they need and no more.

That is everything a sentence needs from its table. The next chapter shows where this book's own tables come from, and the mark chapters follow it.

6 The book's data

Where do `gapminder_2007`, `medals` and `winds` come from, and how do you get them?

Every chapter of this book uses the same shared tables, listed in full at the end of this page. They are published beside the book as CSV files, one per table, so you can run any example in this book yourself. Each file has a fixed address:

`https://psychometrician.github.io/gog-book/data/<name>.csv`

`gapminder_2007.csv` holds the 142 countries the first chapter plots. `medals.csv` holds the medal counts the reading chapter draws, and `winds.csv` holds the wind observations the polar chapter draws.

Reading one takes a single call, `gog_table()`, which comes with the package in all four languages. Nothing else needs installing. You do not have to use it. The files are ordinary CSVs at a fixed address, and any reader that takes a web address reads them.

6.1 Reading a table

`gog_table()` reads any of these tables by name:

R

```
gapminder_2007 <- gog_table("gapminder_2007")
data(gapminder_2007) + point + x(gdp) + y(life)
```

Python

```
gapminder_2007 = gog_table("gapminder_2007")
data(gapminder_2007) + point + x(col.gdp) + y(col.life)
```

Julia

```
gapminder_2007 = gog_table("gapminder_2007")
data(gapminder_2007) + point + x(:gdp) + y(:life)
```

JavaScript

```
const gapminder_2007 = await gog_table("gapminder_2007");  
plot(data(gapminder_2007), point, x(col.gdp), y(col.life));
```

Those four calls draw the same picture, because the table is the same file and the engine is the same program.

`gog_table()` also gives each column the right type. A CSV is text, so numbers arrive as text and have to be converted. A column becomes numbers when every value in it is a number, and stays text otherwise. The `text` argument forces a column to stay text. Three columns in these tables need that, and [Labels that look like numbers](#), below, explains which and why.

Why does a graphics package carry a table reader at all? R reads a CSV in one call, and Python and Julia in a few. JavaScript has no CSV reader in its standard library, so the same work there takes dozens of lines. One country in `gapminder_2007.csv` is "Congo, Dem. Rep.", and its name holds a comma, so splitting each line on commas gives that row seven fields where the header has six. Nobody should have to paste dozens of lines of parsing before drawing, so each package carries `gog_table()` instead.

R's `read.csv()`, `urllib` in Python, `Downloads` in Julia and `fetch` in JavaScript are all standard, so `gog_table()` needs nothing beyond the package you already installed.

Read a file with a reader of your own, and the rest of this page still applies, because `data()` takes any table your session holds. You then do two things yourself. Your reader decides each column's type. The three columns named below then need whatever it offers for keeping a column as text. And a declared order is never in the file, so you declare it after reading, whichever reader you use. [Two things a CSV cannot record](#) covers both.

Reading a different table is the same call with a different name: `gog_table("winds")`, `gog_table("medals")`. A misspelled name is refused, and the refusal suggests the nearest published name, or points to the list at the end of this page.

6.2 Two things a CSV cannot record

A CSV records what a value is, and never what kind of thing it is. Two kinds of information are therefore lost when the file is written. You restore them when you read the file back. Both change the plot, and this page takes each in turn.

6.2.1 A declared order

The `winds` table records one wind observation per row:

Table 6.1: `winds`: first 5 of 264 rows

direction	bearing	speed	season
N	19.612260	10.1	Winter
N	357.049499	11.0	Winter
N	349.709737	13.8	Winter
N	2.602499	7.4	Winter
N	338.210715	3.7	Summer

The `direction` column of `winds` holds the eight points of the compass. Their order is N, NE, E, SE, S, SW, W, NW, clockwise from north. The [Data](#) chapter shows how a declared order sets the axis, the color assignment and the legend. A CSV cannot record that declaration. `gog_table("winds")` therefore returns `direction` as plain text, and the declaration is yours to make after reading.

Until you declare it, only the file's own row order keeps the plot correct. The `winds` file lists its rows in compass order, so a plot made without the declaration reads correctly. Sort the rows by `speed`, and the same plot changes. The directions now follow the sorted rows, and that order has nothing to do with the compass. The plot still draws, and nothing on it shows that the compass is in the wrong order.

The example below reads the table, sorts it by `speed`, and then declares the compass order:

R

```
winds <- gog_table("winds")
# `base::order`, because gog's `order()` masks it
winds <- winds[base::order(winds$speed), ]
compass <- c("N", "NE", "E", "SE", "S", "SW", "W", "NW")
winds$direction <- factor(winds$direction, levels = compass)

data(winds) + bar * count + x(direction)
```

Python

```
winds = gog_table("winds")
by_speed = sorted(enumerate(winds["speed"]), key=lambda pair: pair[1])
winds = {name: [column[i] for i, _ in by_speed]
```

```

        for name, column in winds.items()}
compass = ["N", "NE", "E", "SE", "S", "SW", "W", "NW"]
winds["direction"] = ordered(winds["direction"], compass)

data(winds) + bar * count + x(col.direction)

```

Julia

```

winds = gog_table("winds")
by_speed = sortperm(winds["speed"])
winds = Dict{String,Any}(name => column[by_speed] for (name, column) in
    ↪ winds)
compass = ["N", "NE", "E", "SE", "S", "SW", "W", "NW"]
winds["direction"] = ordered(winds["direction"], compass)

data(winds) + bar * count + x(:direction)

```

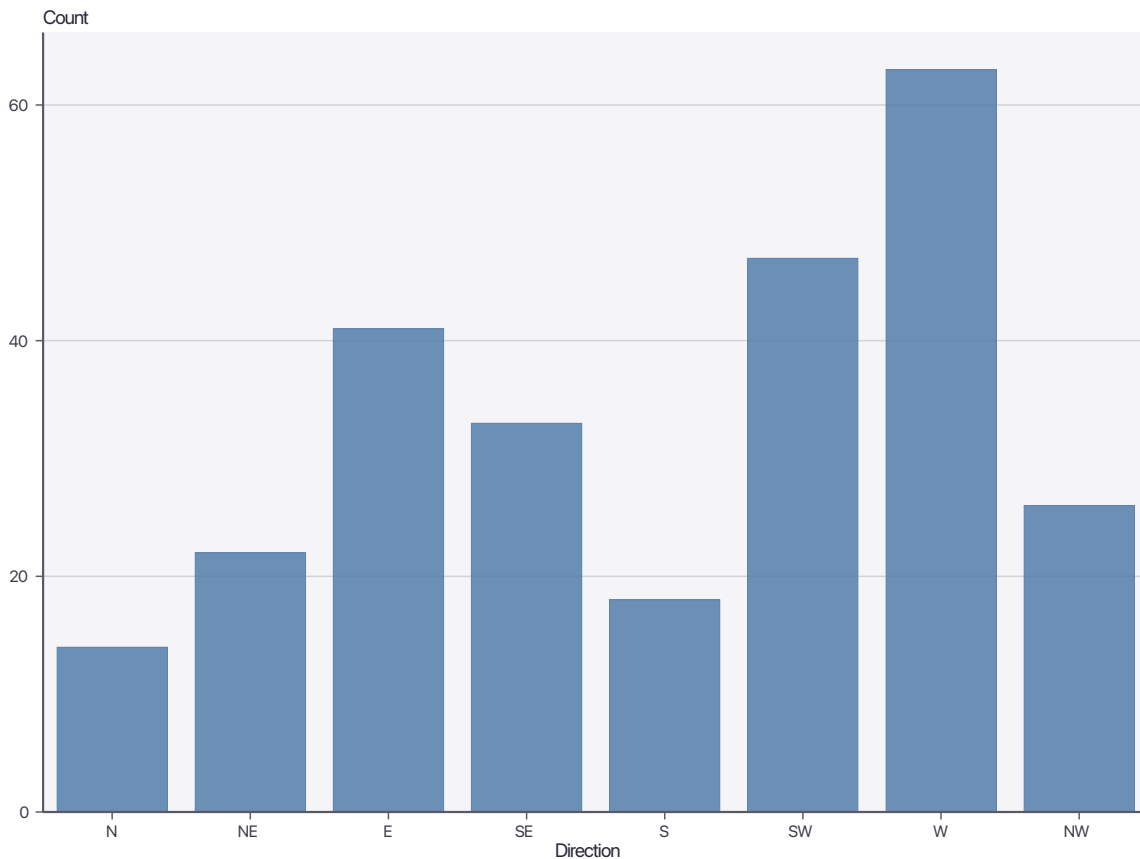
JavaScript

```

const winds = await gog_table("winds");
const by_speed = winds.speed.map((_, i) => i)
    .sort((a, b) => winds.speed[a] - winds.speed[b]);
for (const name in winds) winds[name] = by_speed.map((i) => winds[name][i]);
const compass = ["N", "NE", "E", "SE", "S", "SW", "W", "NW"];
winds.direction = ordered(winds.direction, compass);

plot(data(winds), layer(bar, count), x(col.direction));

```



“Given the winds: bars derived by count, x is direction.”

The rows are in speed order, and the axis still reads N to NW. The declared order decides the axis, not the order of the rows. The [polar](#) chapter draws this same count as a wind rose, and the change is one word. Add `polar()`, and the axis bends into a circle, with N at the top and the rest clockwise from it. The declared order goes with it:

R

```
data(winds) + bar * count + x(direction) + polar()
```

Python

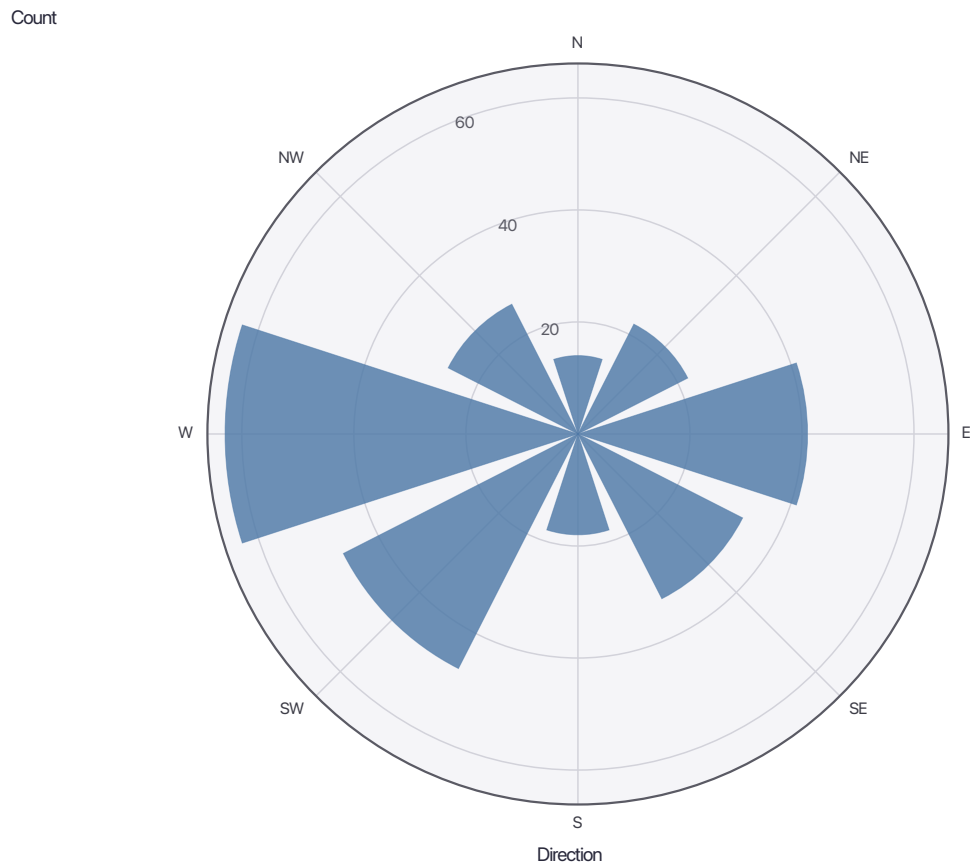
```
data(winds) + bar * count + x(col.direction) + polar()
```

Julia


```
data(winds) + bar * count + x(:direction) + polar()
```

JavaScript

```
plot(data(winds), layer(bar, count), x(col.direction), polar())
```



“Given the winds: bars derived by count, x is direction, in polar.”

The polar chapter explains that plot. Here it shows that a declared order survives a change of space. The circle starts at the first category and runs clockwise through the rest, so the compass reads correctly in both.

The sort exists only to undo the file’s compass order. The declaration is the part you keep. Declare it once after each `gog_table("winds")`, and every plot drawn from `winds` in the rest of the book reads the compass in order.

6.2.2 Labels that look like numbers

The second thing a CSV cannot record is which columns are text. Three columns in these tables hold text that `gog_table()` would read as numbers unless you name them. `census.csv` has an `age` column of 0, 5, 10, which are the names of age bands rather than quantities. `sessions.csv` has a `session` column of 01, 02, 03, and a number cannot keep a leading zero, so 01 comes back as 1. `gm_eras.csv` has an `era` column of 1957 and 2007. Those two values name the eras the rows compare; they do not measure anything. That is the whole reason `gog_table()` takes a `text` argument:

R

```
census <- gog_table("census", text = "age")
sessions <- gog_table("sessions", text = "session")
gm_eras <- gog_table("gm_eras", text = "era")
```

Python

```
census = gog_table("census", text=("age",))
sessions = gog_table("sessions", text=("session",))
gm_eras = gog_table("gm_eras", text=("era",))
```

Julia

```
census = gog_table("census"; text = ["age"])
sessions = gog_table("sessions"; text = ["session"])
gm_eras = gog_table("gm_eras"; text = ["era"])
```

JavaScript

```
const census = await gog_table("census", ["age"]);
const sessions = await gog_table("sessions", ["session"]);
const gm_eras = await gog_table("gm_eras", ["era"]);
```

Without it, `age` becomes numbers, and an axis that should show one slot per age band is drawn as a numeric scale from 0 to 85. The plot looks reasonable, so nothing tells you the axis is wrong.

Each call above names one column, and a `text` argument can name more than one. `gm_eras` holds the same values in two columns: `year` as a number, `era` as a label. The book reads only

`era` as text, so `year` stays a number. To read the year as a label too, name both columns in one `text` argument:

R

```
gm_eras <- gog_table("gm_eras", text = c("year", "era"))
```

Python

```
gm_eras = gog_table("gm_eras", text=("year", "era"))
```

Julia

```
gm_eras = gog_table("gm_eras"; text = ["year", "era"])
```

JavaScript

```
const gm_eras = await gog_table("gm_eras", ["year", "era"]);
```

Each language writes more than one name in its own way. R puts them inside `c()`, Python inside parentheses, and Julia and JavaScript inside square brackets. Only R's spelling changes from the block above. The other three already put one name inside parentheses or brackets, so a second name joins it there.

6.3 The tables

Those three tables need a column named in a `text` argument, and no other shared table does. The list below names every shared table this book reads, with its rows and columns. The [preface](#) introduces the eight families of table that most plots read. A family can cover more than one name below, so the list is longer than eight. Every other table is introduced on the page that first draws it. Each name goes in place of `<name>` in the address at the top of this page. The same name is what you give `gog_table()`:

Table	Rows	Columns
actuals	5	2
cashflow	6	3
census	36	3
commutes	16	3

Table	Rows	Columns
day_cycle	25	2
forecast	3	2
gapminder_2007	142	6
gapminder_asia	60	6
gdp_threshold	1	1
gm_all	1704	6
gm_continents	1056	6
gm_eras	284	7
gm_europe	30	6
iris_flowers	150	4
life_bands	3	2
listening	104	3
maunga_whau	1364	3
medals	5	4
milestones	2	3
nutrients	40	3
quakes_fiji	1000	5
quarterly	19	2
recessions	2	3
ripples	5400	5
score_band	21	4
sessions	14	5
six_weeks	42	4
speed_target	1	1
spending	12	4
target_band	1	2
thermal_marks	34	5
thermals	340	5
tide	8	2
titanic	32	5
trade_partners	22	3
winds	264	4
world_borders	4150	5

Not every table here holds real data. The tables whose names begin with `gapminder` or `gm` hold real figures from the Gapminder Foundation, released into the public domain. `iris_flowers`, `maunga_whau`, `quakes_fiji` and `titanic` are reshaped from tables included with R, and carry R's own license. `world_borders` is Natural Earth's public-domain outline of the countries. Everything else was written for this book, each table to show one thing; `census` is two plausible city profiles, not a real census. The terms for each are in [data/LICENSE.md](#).

7 Choosing a mark

Which visible form should your data take? A **mark** is that visible form: a dot, a bar, a line. Choosing the mark is the first decision in every plot. The answer depends on the *types of your variables* and the *question you are asking*.

There are fourteen marks, and they are not a list to learn before you draw. This chapter is a guide to the choice: it shows which mark answers which question, draws the common cases, and stops. Each mark then has a chapter of its own, with examples. The table at the end is for later, when you have data in front of you and want the row that fits.

If you have used `ggplot2`, a mark is close to a geom, and one difference is worth knowing early. Many geoms compute a statistic before they draw. `geom_histogram()` bins your data and then draws bars. `geom_smooth()` fits a curve and then draws a line. A mark never does that. A mark is only the shape, and the statistic is a separate word: a histogram is `bar * bin`, a trend line is `line * smooth`. A geom tends to name a chart. A mark names a shape.

`gog` reads the type of each column from your data:

- **Continuous.** Any numeric column (`gdp`, `temperature`, `score`).
- **Categorical.** Any text column (`continent`, `country`, `species`), or a column with a declared order.
- **Date/time.** Any date or date-time column; the axis reads as a calendar (see [Scales](#)).

7.1 One variable

With one variable there is one question: how are its values spread out? The answer depends on whether the column is continuous or categorical.

7.1.1 Continuous variable

A continuous column can be cut into bands, or a smooth curve can be estimated over it. `bin` cuts the column into bands and counts the rows in each. `bar * bin` draws one bar per band, which is the histogram:

R

```
# How is life expectancy distributed? → histogram
```

```
data(gapminder_2007) + bar * bin + x(life) +  
  x_label("Life expectancy (years)") + title("Distribution: bar * bin")
```

Python

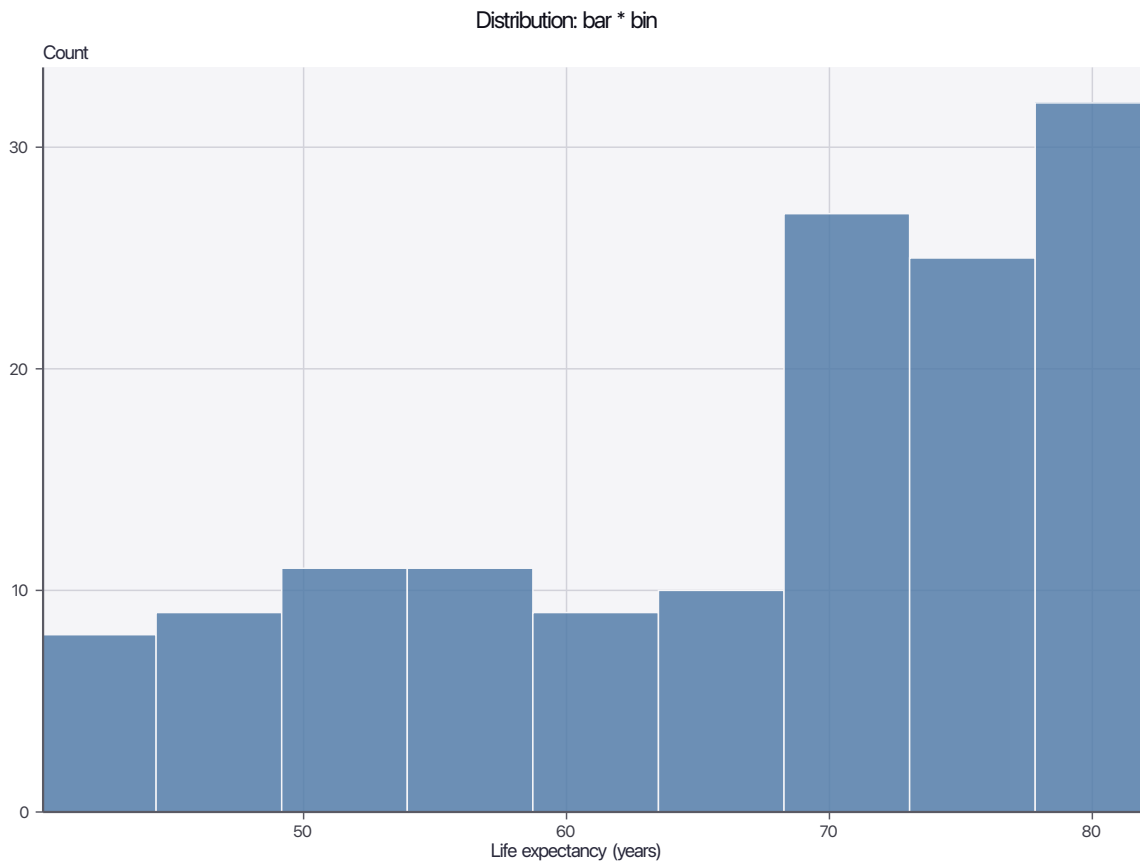
```
(data(gapminder_2007) + bar * bin + x(col.life) +  
  x_label("Life expectancy (years)") + title("Distribution: bar * bin"))
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) +  
  x_label("Life expectancy (years)") + title("Distribution: bar * bin")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life),  
  x_label("Life expectancy (years)", title("Distribution: bar * bin"))
```



“Given gapminder 2007: bars derived by bin, x is life.”

`density` estimates a smooth curve over the same column, and `line * density` draws that curve:

R

```
# Smooth shape of the distribution → density curve
data(gapminder_2007) + line * density + x(life) +
  x_label("Life expectancy (years)") + title("Density: line * density")
```

Python

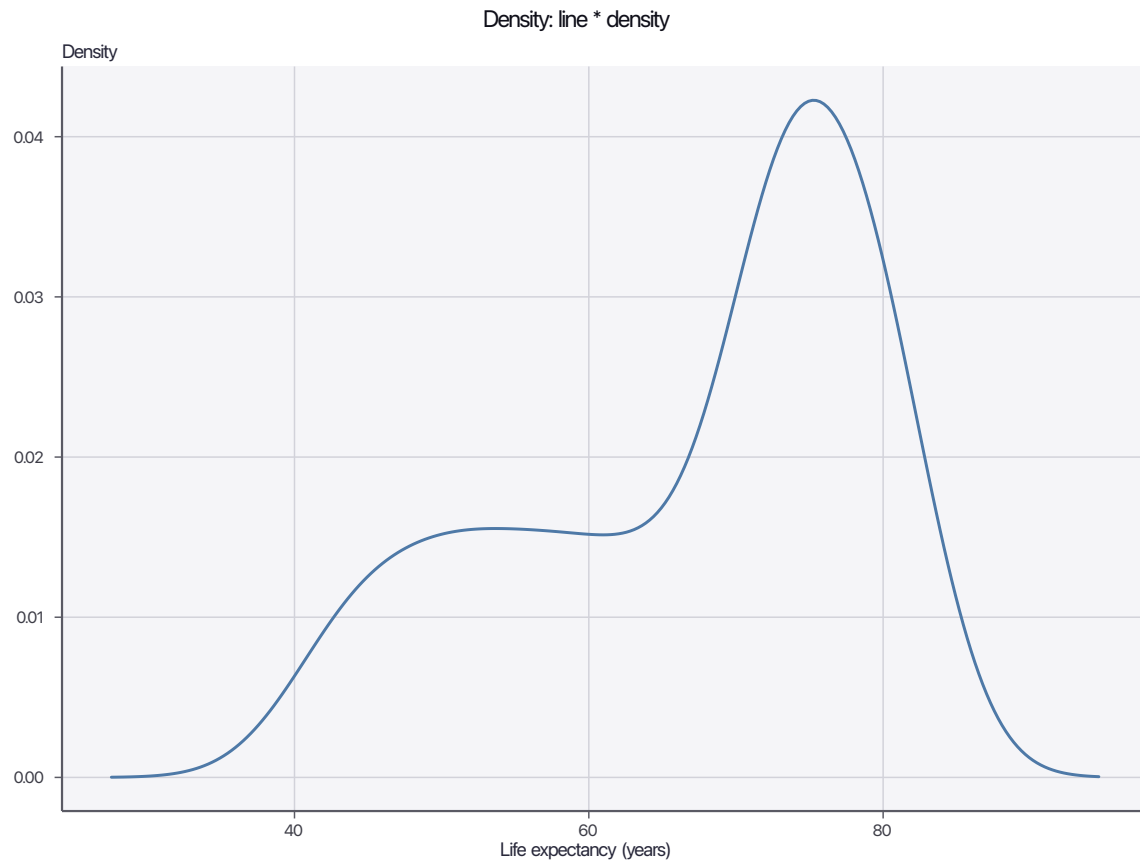
```
(data(gapminder_2007) + line * density + x(col.life) +
  x_label("Life expectancy (years)") + title("Density: line * density"))
```

Julia

```
data(gapminder_2007) + line * density + x(:life) +  
  x_label("Life expectancy (years)") + title("Density: line * density")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density), x(col.life),  
  x_label("Life expectancy (years)"), title("Density: line * density"))
```



“Given gapminder 2007: a line derived by density, x is life.”

7.1.2 Categorical variable

A categorical column cannot be cut into bands, so the same question reads as how often each category occurs. `count` tallies the rows in each category, and `bar * count` draws one bar per category:

R

```
# How many countries per continent? → frequency bar  
data(gapminder_2007) + bar * count + x(continent) +  
  title("Frequency: bar * count")
```

Python

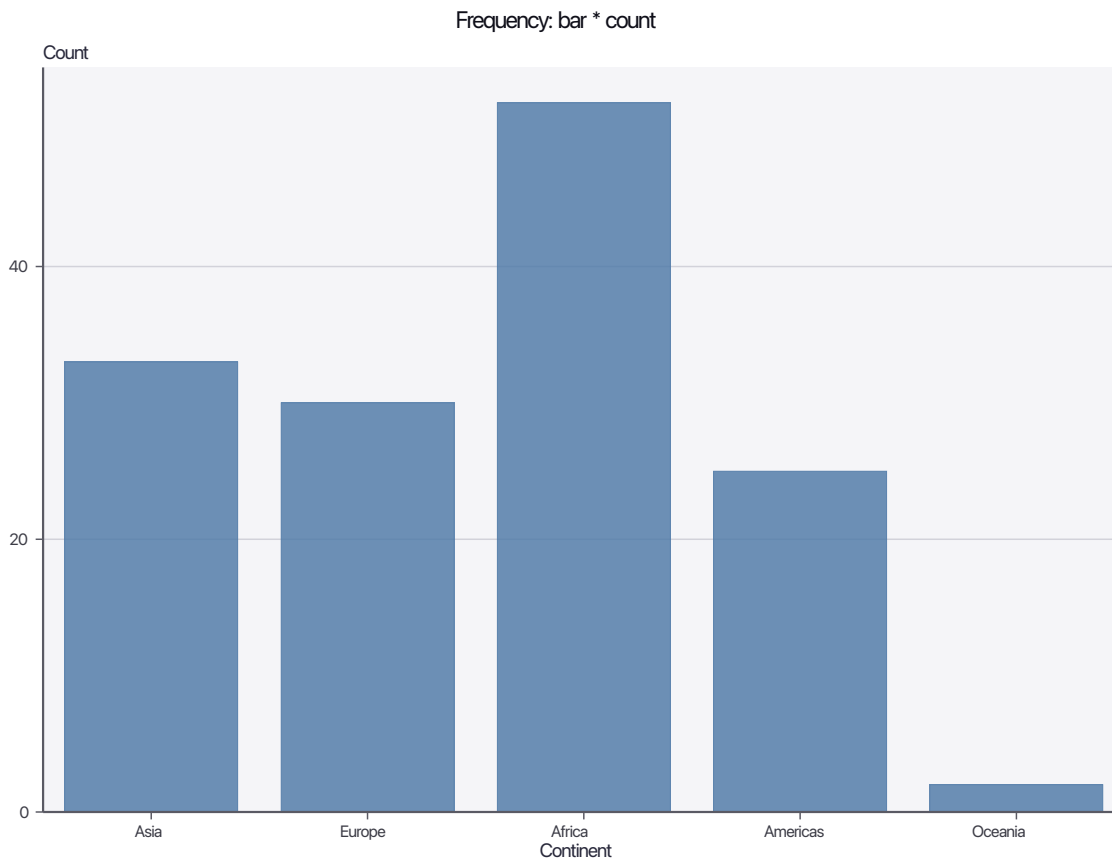
```
(data(gapminder_2007) + bar * count + x(col.continent) +  
  title("Frequency: bar * count"))
```

Julia

```
data(gapminder_2007) + bar * count + x(:continent) +  
  title("Frequency: bar * count")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, count), x(col.continent),  
  title("Frequency: bar * count"))
```



“Given gapminder 2007: bars derived by count, x is continent.”

proportion is the same tally as a share of the whole, so the bars sum to 1:

R

```
# What share of countries per continent? → proportion bar
data(gapminder_2007) + bar * proportion + x(continent) +
  title("Relative frequency: bar * proportion")
```

Python

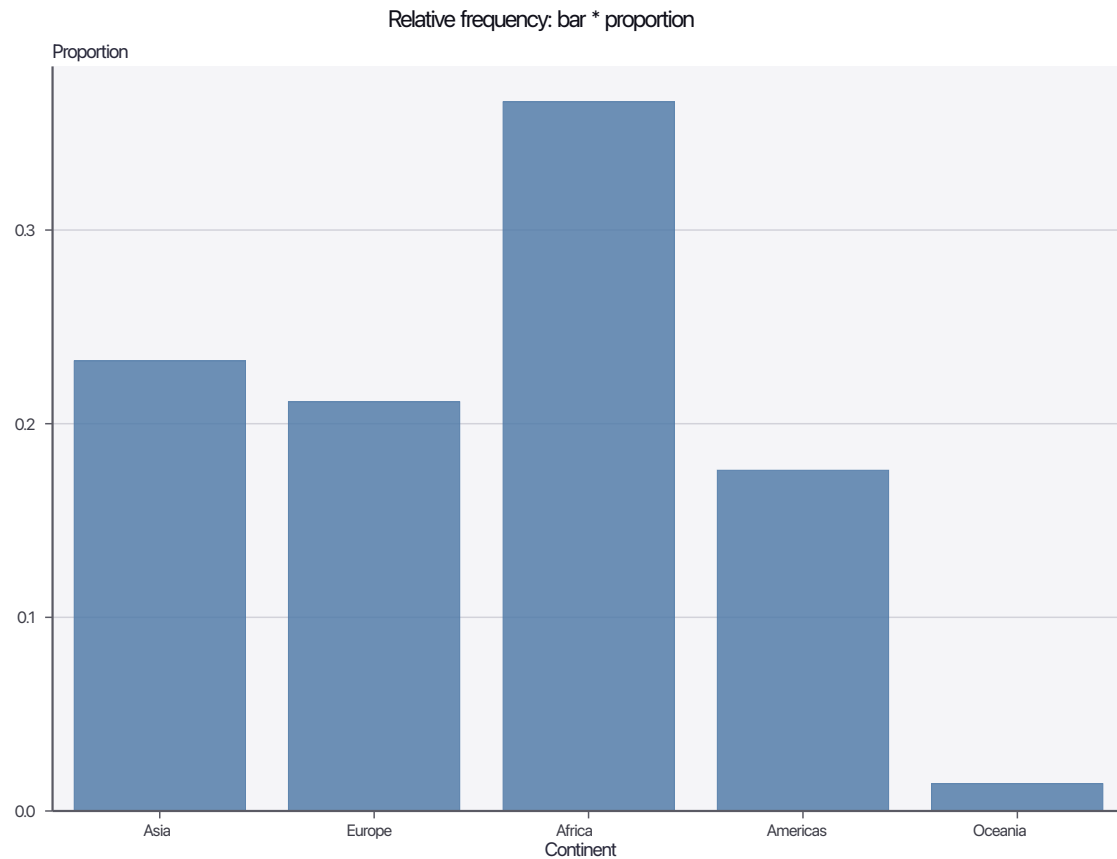
```
(data(gapminder_2007) + bar * proportion + x(col.continent) +
  title("Relative frequency: bar * proportion"))
```

Julia

```
data(gapminder_2007) + bar * proportion + x(:continent) +
  title("Relative frequency: bar * proportion")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, proportion), x(col.continent),
  title("Relative frequency: bar * proportion"))
```



“Given gapminder 2007: bars derived by proportion, x is continent.”

7.2 Two variables

With two variables, the pair of column types decides the mark.

7.2.1 Continuous × Continuous

Both variables are numeric. The natural questions are about relationships, trends, and patterns.

R

```
# Is there a relationship? → scatter
data(gapminder_2007) + point + x(gdp) + y(life) + x_label("GDP per capita") +
  y_label("Life expectancy") + title("Relationship: point")
```

Python

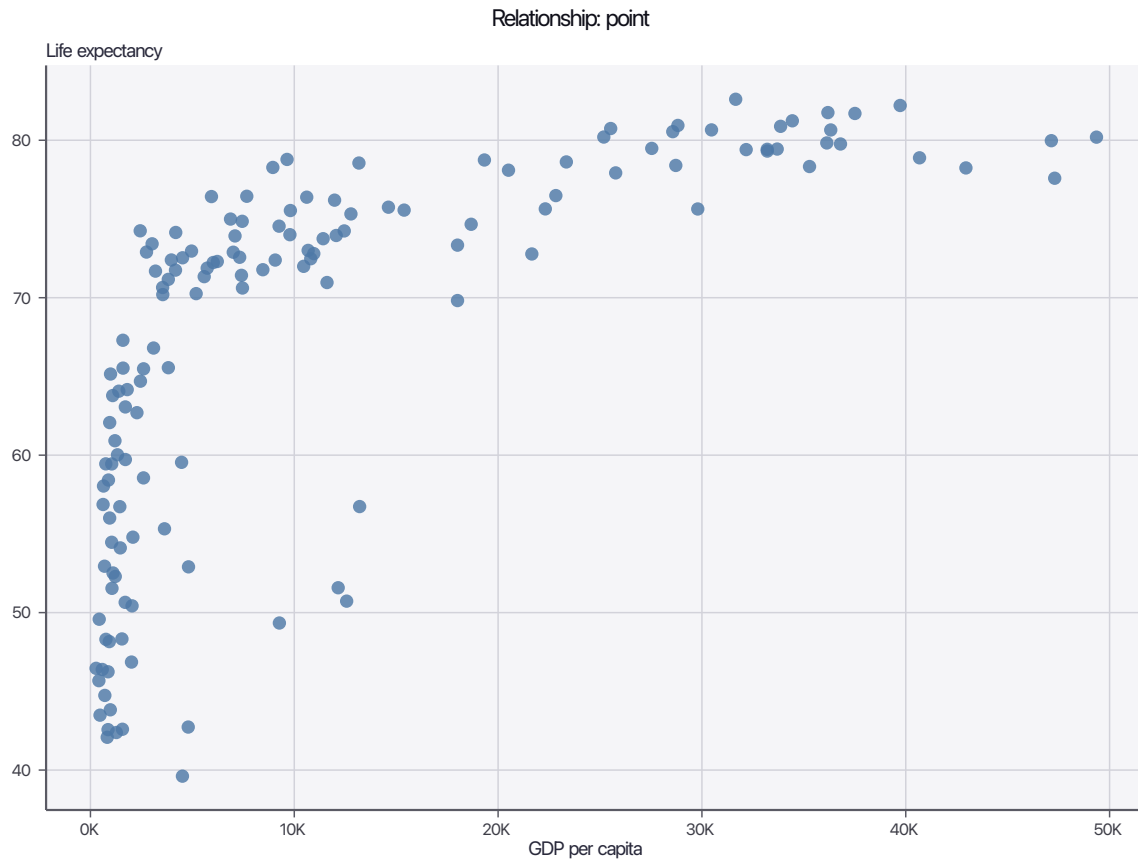
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + x_label("GDP per
↪ capita") +
  y_label("Life expectancy") + title("Relationship: point"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Relationship: point")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  x_label("GDP per capita"), y_label("Life expectancy"),
  title("Relationship: point"))
```



“Given gapminder 2007: points, x is gdp, y is life.”

`smooth` fits a trend line through the points, and `line * smooth` draws it over the scatter. The points are colored by continent here, and the one line runs through all of them:

R

```
# Smooth the relationship → fitted trend line
data(gapminder_2007) + x(gdp) + y(life) +
  point + color(continent) +
  line * smooth +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Trend: point + line * smooth")
```

Python

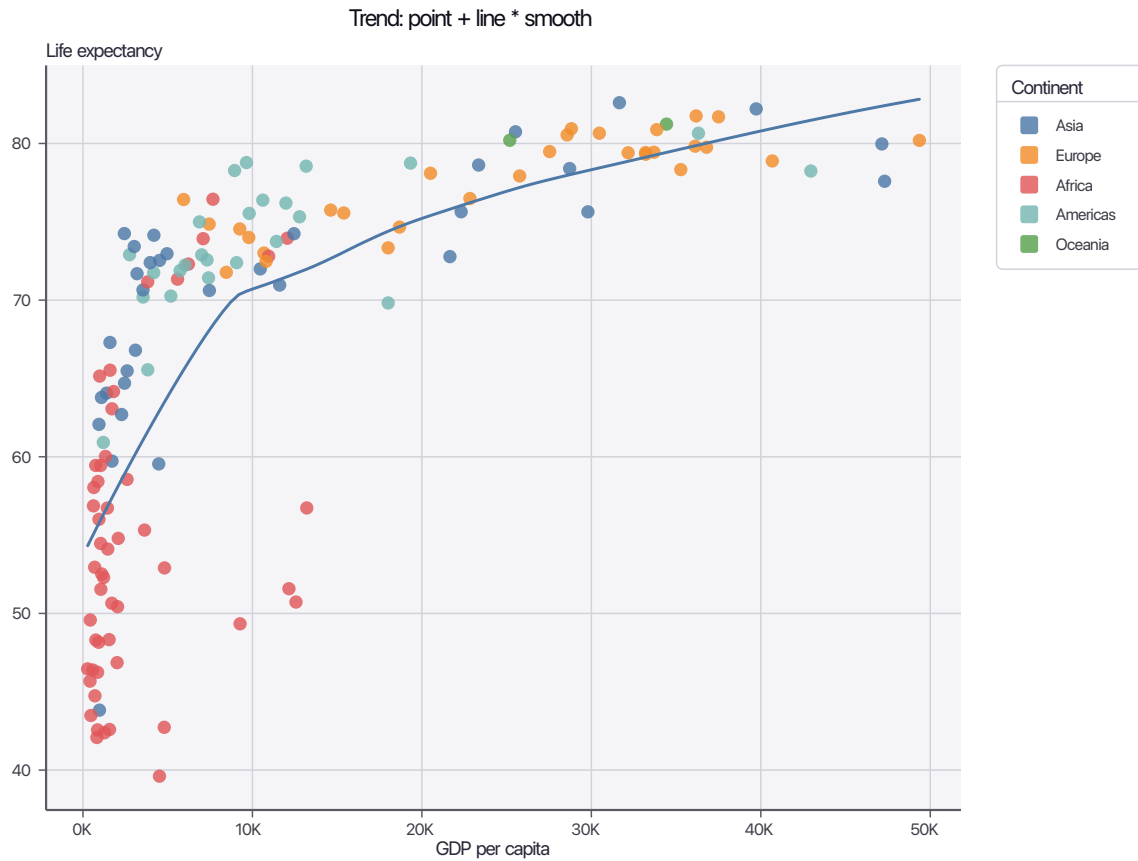
```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +
  point + color(col.continent) +
  line * smooth +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Trend: point + line * smooth"))
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + point + color(:continent) +
  line * smooth + x_label("GDP per capita") + y_label("Life expectancy") +
  title("Trend: point + line * smooth")
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), point,
  color(col.continent), layer(line, smooth), x_label("GDP per capita"),
  y_label("Life expectancy"), title("Trend: point + line * smooth"))
```



“Given gapminder 2007: x is *gdp*, y is *life*, points, color by continent, and also a line derived by *smooth*.”

R

```
# How much, not just where → filled area
data(actuals) + area + x(year) + y(sales) + y_label("Sales") +
  title("Quantity: area")
```

Python

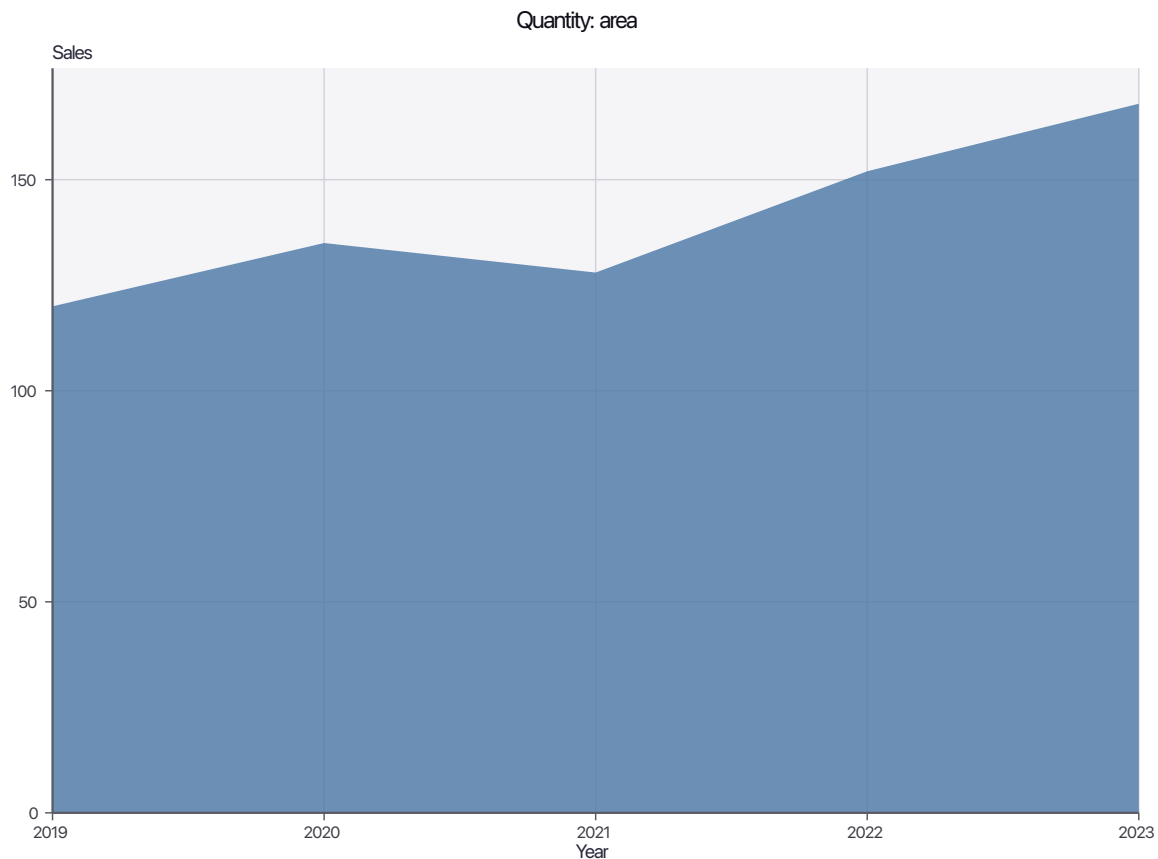
```
(data(actuals) + area + x(col.year) + y(col.sales) + y_label("Sales") +
  title("Quantity: area"))
```

Julia

```
data(actuals) + area + x(:year) + y(:sales) + y_label("Sales") +  
  title("Quantity: area")
```

JavaScript

```
plot(data(actuals), area, x(col.year), y(col.sales), y_label("Sales"),  
  title("Quantity: area"))
```



“Given the actuals: an area, x is year, y is sales.”

line and **area** draw the same boundary and give it different meanings. A line says *where* the value was; an area says *how much* there was, so it fills to zero and the filled region is the quantity. Choose **area** when zero is a true baseline for your variable, and **line** when it is not.

`ribbon` fills between a low boundary and a high one. A `range` transform supplies that pair: the minimum and maximum at each `x`, unless you name two quantiles. Use `ribbon` when each `x` has a *spread* to show. Add a `line * mean` on top and you get a trend line inside its band.

When the *order of the rows* is what you want to show, `path` connects them in that order. `line` sorts its points by `x` before it joins them. `path` does not, so the stroke can move left as well as right, cross itself, or return to where it started. That is the **connected scatterplot**: the two axes carry measurements, and the joining line carries time.

R

```
# The reading order is the table's → path
data(gapminder_asia) + path + x(gdp) + y(life) + color(country) +
  style(arrow = "end") +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("Route: path (the arrow points to 2007)")
```

Python

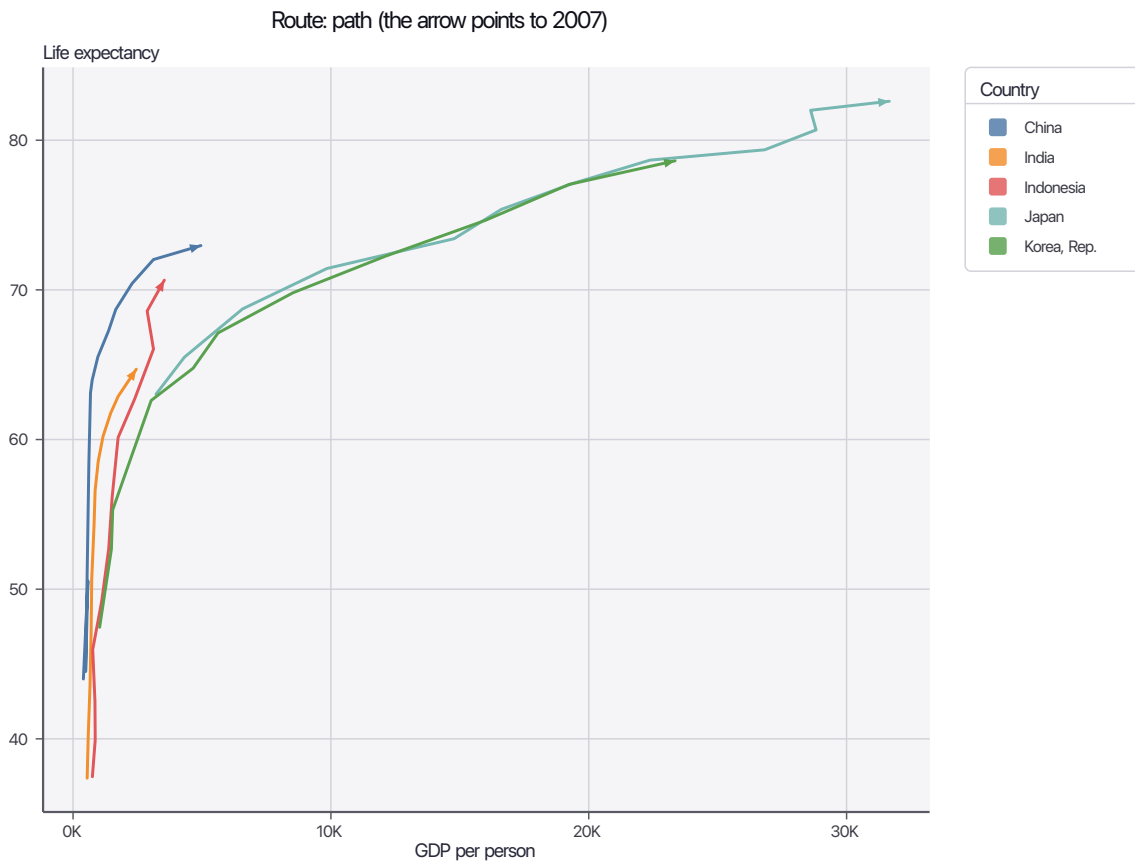
```
(data(gapminder_asia) + path + x(col.gdp) + y(col.life) + color(col.country)
↪ +
  style(arrow = "end") +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("Route: path (the arrow points to 2007)"))
```

Julia

```
data(gapminder_asia) + path + x(:gdp) + y(:life) + color(:country) +
  style(arrow = "end") + x_label("GDP per person") +
  y_label("Life expectancy") +
  title("Route: path (the arrow points to 2007)")
```

JavaScript

```
plot(data(gapminder_asia), path, x(col.gdp), y(col.life),
  color(col.country), style({ arrow: "end" }), x_label("GDP per person"),
  y_label("Life expectancy"),
  title("Route: path (the arrow points to 2007)"))
```



“Given gapminder Asia: paths, x is gdp, y is life, color by country, with arrow end.”

Layered *over* any of these, `rule` marks a value on one axis and spans the other: a threshold line, or a rug. A rug reaches only a little way in from the edge, which is `style(reach = "edge")`, and shows where the observations are. Layered *under* them, `zone` shades a rectangle the same way: bounded where you give it a pair of columns, spanning the panel where you do not. Give `zone` a `bin` instead of the pair of columns, and the same mark tiles the panel with counted cells: the heatmap.

`life_bands` holds three named bands of life expectancy:

Table 7.1: `life_bands`: all 3 rows

life	band
60	Low
70	Middle
80	High

R

```
# One position, the panel supplies the other → rule
data(gapminder_2007) + point + x(gdp) + y(life) + style(opacity = 0.45) +
  rule + x(gdp) + style(reach = "edge") +
  data(life_bands) + rule + color(band) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("Threshold and rug: rule")
```

Python

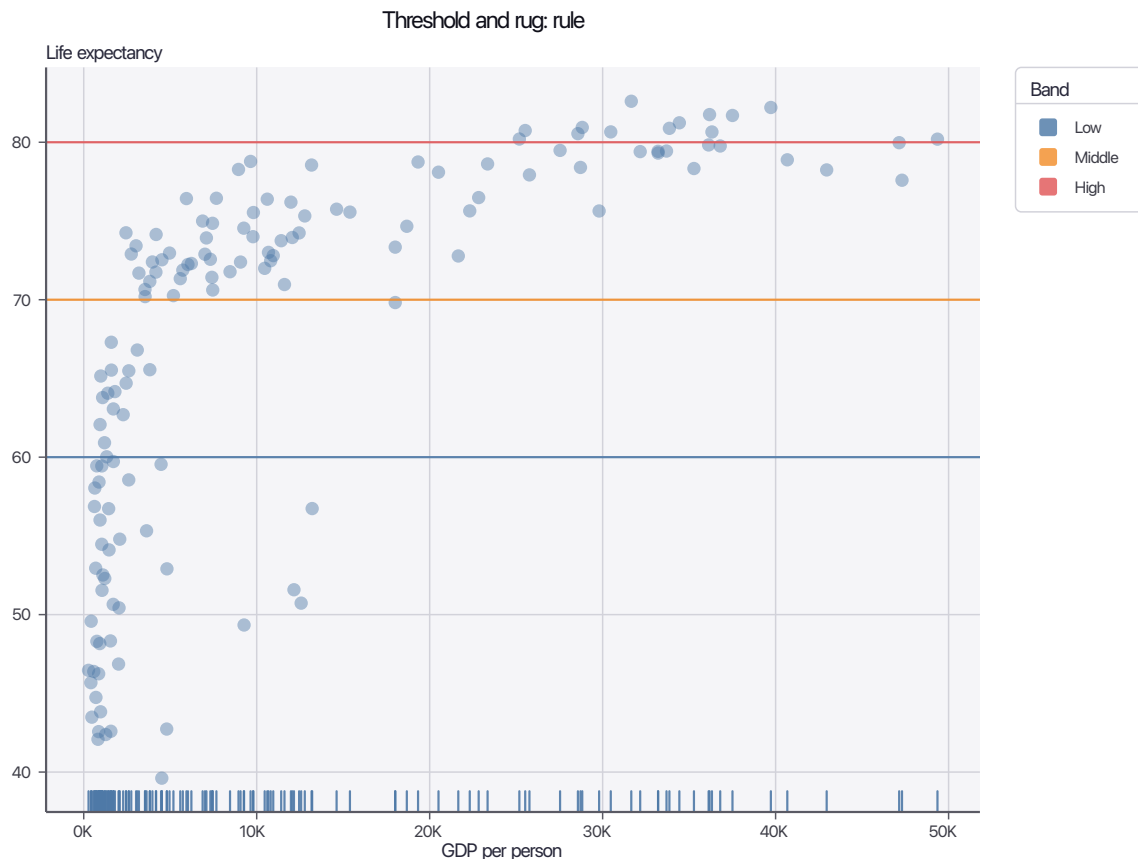
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + style(opacity =
↪ 0.45) +
  rule + x(col.gdp) + style(reach = "edge") +
  data(life_bands) + rule + color(col.band) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("Threshold and rug: rule"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  style(opacity = 0.45) + rule + x(:gdp) + style(reach = "edge") +
  data(life_bands) + rule + color(:band) + x_label("GDP per person") +
  y_label("Life expectancy") + title("Threshold and rug: rule")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  style({ opacity: 0.45 }), rule, x(col.gdp), style({ reach: "edge" }),
  data(life_bands), rule, color(col.band), x_label("GDP per person"),
  y_label("Life expectancy"), title("Threshold and rug: rule"))
```



“Given gapminder 2007: points, x is gdp, y is life, and also rules, x is gdp, with reach edge, and also rules from the life bands, color by band.”

The rule names `x(gdp)` even though the points already named it. That second `x(gdp)` does not add an axis. Every layer shares the plot’s one x and one y. This rule reads the scatter’s table, which holds both `gdp` and `life`, so both axes answer. A rule sits on one axis and spans the other, so it has to say which. Leave that out and `gog` refuses rather than guessing; the [Rule](#) chapter shows the refusal. The three rules from `life_bands` need no position, because that table holds `life` and no `gdp`, so only one axis can answer.

7.2.2 Categorical × Continuous

One axis groups, the other measures. The natural question is *how the measurement differs across groups*. `mean` reduces each group to one value, its average. `bar * mean` draws one bar per group, and the bar’s height is that value:

R

```
# Mean value per group → bar
data(gapminder_2007) + bar * mean + x(continent) + y(life) +
  y_label("Mean life expectancy") + title("Group summary: bar * mean")
```

Python

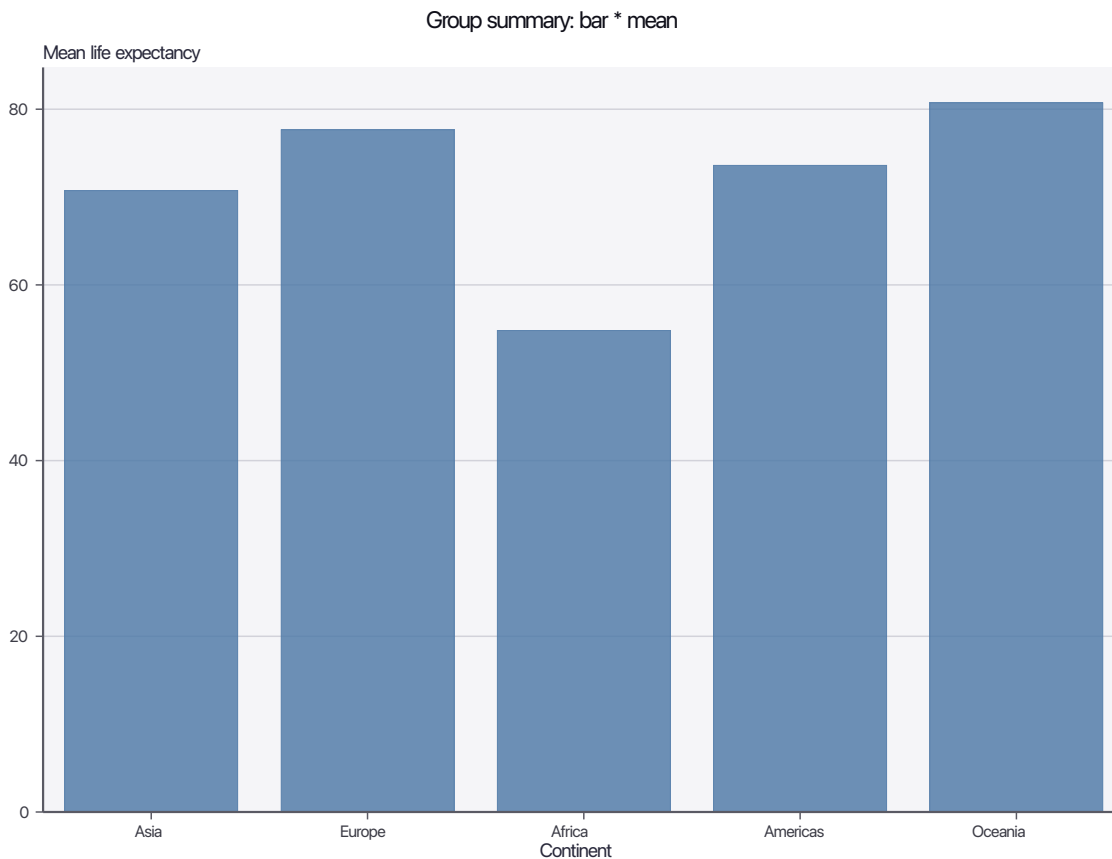
```
(data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life) +
  y_label("Mean life expectancy") + title("Group summary: bar * mean"))
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life) +
  y_label("Mean life expectancy") + title("Group summary: bar * mean")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),
  y(col.life), y_label("Mean life expectancy"),
  title("Group summary: bar * mean"))
```



“Given gapminder 2007: bars derived by mean, x is continent, y is life.”

Drop the transform, and **point** draws every country in its continent’s slot:

R

```
# All individual points per group → strip plot
data(gapminder_2007) + point + x(continent) + y(life) +
  y_label("Life expectancy") +
  title("Strip plot: point (one dot per country)")
```

Python

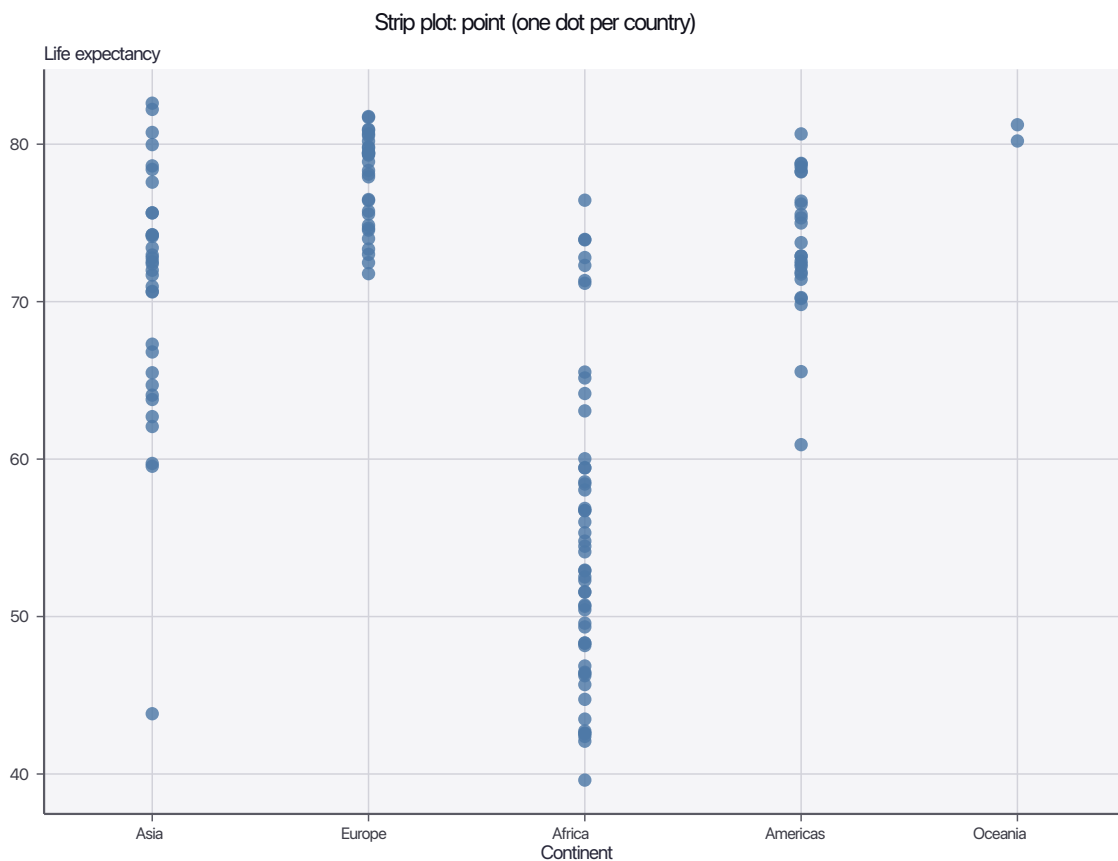
```
(data(gapminder_2007) + point + x(col.continent) + y(col.life) +
  y_label("Life expectancy") +
  title("Strip plot: point (one dot per country)"))
```

Julia

```
data(gapminder_2007) + point + x(:continent) + y(:life) +  
  y_label("Life expectancy") +  
  title("Strip plot: point (one dot per country)")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.continent), y(col.life),  
  y_label("Life expectancy"),  
  title("Strip plot: point (one dot per country)"))
```



“Given gapminder 2007: points, x is continent, y is life.”

Nothing is summarized: each dot is one country, placed by its continent and its life expectancy. Countries with the same life expectancy land on the same spot, so a crowded column hides two

things: how many dots it holds, and how they are spread. The next paragraph names three marks that show both.

For the *shape* of each group rather than every point, `box` draws the five-number summary as `box + x(continent) + y(life)`: median, quartiles, and whiskers, with far values drawn as separate dots. `ribbon * density` draws the whole estimated distribution that those five numbers summarize: the violin plot. When every point matters but the points overlap, `point * jitter` spreads them sideways: the jittered strip plot.

`line`, `area`, `step` and `ribbon` take a category here too, joining one summary per group instead of drawing a bar on each:

R

```
# One value per group, joined → profile
data(gapminder_2007) + x(continent) + y(life) +
  line * mean + point * mean +
  y_label("Mean life expectancy") +
  title("Profile: line * mean")
```

Python

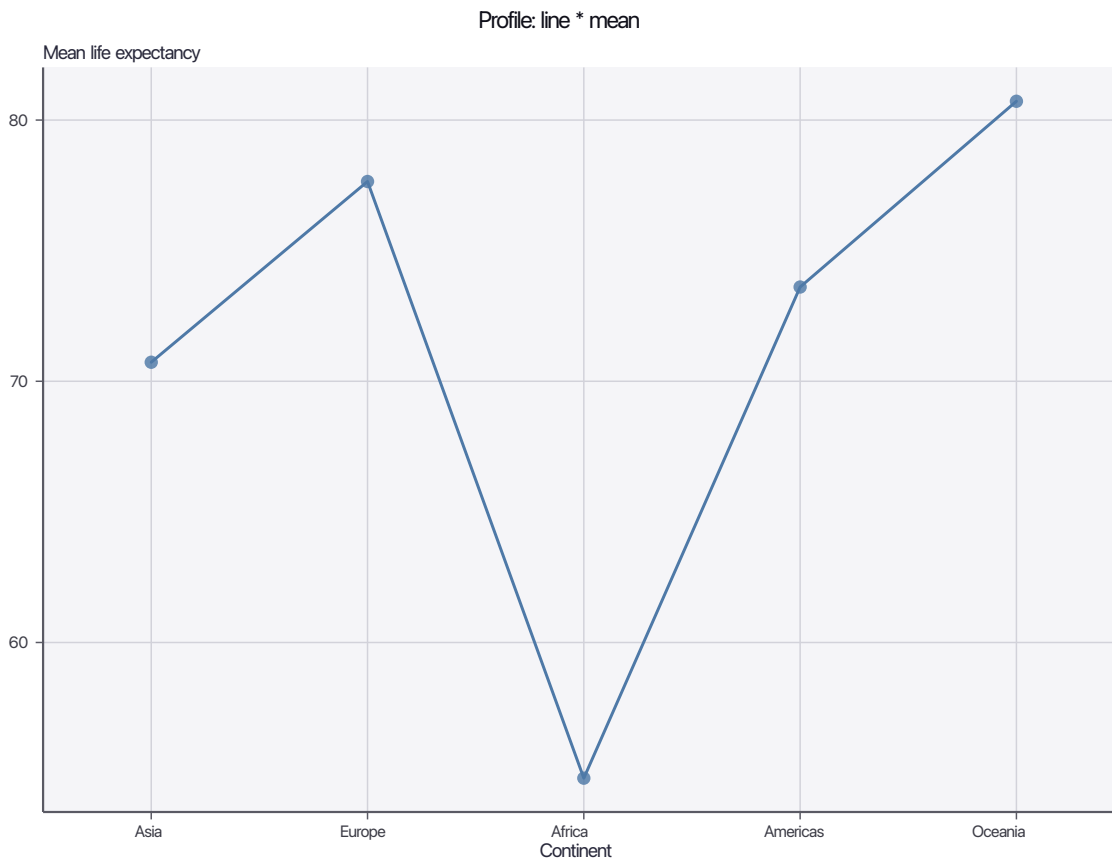
```
(data(gapminder_2007) + x(col.continent) + y(col.life) +
  line * mean + point * mean +
  y_label("Mean life expectancy") +
  title("Profile: line * mean"))
```

Julia

```
data(gapminder_2007) + x(:continent) + y(:life) + line * mean +
  point * mean + y_label("Mean life expectancy") +
  title("Profile: line * mean")
```

JavaScript

```
plot(data(gapminder_2007), x(col.continent), y(col.life),
  layer(line, mean), layer(point, mean), y_label("Mean life expectancy"),
  title("Profile: line * mean"))
```

“Given gapminder 2007: x is continent, y is life, a line derived by mean and also points derived by mean.”

A bar compares one group with another; a profile traces a change across the groups. The profile is the weaker claim of the two, since its segments cross parts of the panel where nothing was measured. Use it when the categories have an order, such as a rating scale or size groups from smallest to largest. Continents have no order, so this plot does what that advice says not to do, and **gog** draws it anyway. The grammar refuses a sentence that is ill-formed, never one that is only unwise; whether a profile suits your categories is your call ([Law 8](#)). The same categories drawn around a circle give the radar chart, in [Polar](#). [area](#) fills the same boundary, [step](#) holds it flat across each slot, and [ribbon * range](#) fills the spread as a band.

7.3 Quick reference

Everything above, in one table. Start from the types of your two columns, x and y, find the rows that match, and read the mark and transform that answer each question. The chapter drew the common rows; every mark has a chapter of its own for the rest. A — means the question needs no variable in that column.

x	y	Mark + transform	Question
Continuous	—	<code>bar * bin</code>	Distribution?
Continuous	—	<code>line * density</code>	Smooth distribution?
Continuous	—	<code>point * bin * stack</code>	Distribution, every observation shown?
Categorical	—	<code>bar * count</code>	Frequency?
Categorical	—	<code>bar * proportion</code>	Share?
Continuous	Continuous	<code>point</code>	Relationship?
Continuous	Continuous	<code>line</code>	Trend / time series?
Continuous	Continuous	<code>line * smooth</code>	Smoothed trend?
Continuous	Continuous	<code>area</code>	How much, over a range?
Continuous	Continuous	<code>ribbon * range</code>	Spread / band over a range?
Categorical	Continuous	<code>bar</code>	One value per group, already in the table?
Categorical	Continuous	<code>bar * mean/sum/...</code>	Value per group?
Categorical	Continuous	<code>point</code>	All values per group?
Categorical	Continuous	<code>box</code>	Distribution per group, summarized?
Categorical	Continuous	<code>ribbon * density</code>	Distribution per group, in full?
Categorical	Continuous	<code>interval * range</code>	Spread per group?
Categorical	Continuous	<code>line * mean</code>	Value per group, as a profile?
Categorical	Continuous	<code>area * mean</code>	Value per group, filled?
Categorical	Continuous	<code>line * mean + polar()</code>	Value per group, as a radar?

One mark is not in the table, because its question has no x and no y: what is connected to what? `edge` draws one line for each connection in a table of connections, such as who trades with whom. Nothing in the data says where the two ends of that line should sit, so a `layout`

transform inside the `network()` space places them. The [Network](#) chapter builds the whole diagram.

A third variable takes the third position, `z`, rather than a new mark. The last row below is different: it uses two categories, and `space()` gives the counts an axis of their own.

x	y	z	Mark	Question
Continuous	Continuous	Continuous	<code>point</code>	Relationship, in three?
Continuous	Continuous	Continuous	<code>path</code>	A route through three?
Continuous	Continuous	Continuous	<code>surface</code>	A height over a plane? (needs a grid : a <code>z</code> value at every <code>x</code> and <code>y</code>)
Categorical	Categorical	—	<code>bar * count + space()</code>	How many, per pair?

Every Categorical / Continuous row above can also be read with the two axes swapped. `bar`, `box` and `interval` sit in a slot on one axis and measure along the other. The column types decide which is which: the categorical column gets the slots. Swapping the two positions therefore draws the same plot horizontally. `box + x(life) + y(continent)` is the horizontal box plot, and `bar + x(gold) + y(country)` the horizontal bar. Nothing else is needed. If you know `ggplot2`'s `coord_flip()` or `matplotlib`'s `barh()`, this is the same result with no extra word. `gog` reads the orientation from which axis carries the categories, as `ggplot2` itself now does.

7.4 Mark families

The mark list looks like a set to memorize, one entry at a time. It is not. The fourteen consonants of Hangeul (한글) are not fourteen unrelated shapes either. The letters are *featural*: most of them are another letter with a single feature changed. $\rhd$ (*k*) is $\neg$ (*g*) with one stroke added, so learning $\neg$ teaches you most of $\rhd$.

The marks are built the same way. Some pairs have the same rules for every channel: whatever one mark requires, accepts or refuses, for `x`, `y`, `color`, `size` and the rest, the other does too. They differ only in the geometry they draw:

Family	Same rules for every channel	They differ only in
<code>area</code> and <code>ribbon</code>		a region closed on the baseline , against one closed on a second data boundary
<code>box</code> and <code>interval</code>		a body that computes its own five-number summary , against a span whose ends you supply
<code>line</code> and <code>step</code>		a stroke that slopes between values, against one that holds each value until it changes

The second member of a pair takes almost nothing to learn. It asks exactly the questions its sibling asks, so you already know all of them. One sentence about what it draws is all that is left. The two marks of a family also sit near each other on the grid in [Combinations](#), which is where the structure you can see on that page comes from.

When does `gog` add a mark? Only for a shape that no existing mark can draw. `step` was added because “hold the value until it changes” is such a shape: no channel, transform or setting can say it. A thick line was not, because `style(size = )` already says it. A new chart is never the reason. A histogram, a radar and a pyramid are arrangements of marks that already exist. A new word is spelled with the letters you have, and a new chart is written with the marks you have. The families make the marks quick to learn. They are not a reason to add more marks.

7.5 Unpronounceable combinations

Just as some letter combinations cannot be pronounced, some mark + variable type combinations cannot be rendered meaningfully. Each is refused outright, and nothing is drawn: if a plot was drawn, the grammar accepted it.

`bin`, `density` and `smooth` all work along a continuous axis, so the column they measure must be numbers. A category on that axis is refused, and each refusal names the atom that asks the same question of categories:

Combination	Problem	Use instead
<code>bar * bin + x(category)</code>	<code>bin</code> cuts a continuous axis into intervals	<code>bar * count + x(category)</code>
<code>bar * density + x(category)</code>	<code>density</code> estimates a continuous distribution	<code>bar * proportion + x(category)</code>
<code>point * smooth + x(category)</code>	<code>smooth</code> fits a curve along a continuous axis	<code>bar * mean + x(category) + y(value)</code>

All three are rendered as live refusals in [What bin, density and smooth refuse](#); the table above is this page's summary of them.

The next fourteen chapters give each mark its own page, starting with `point`.

8 Point

How does one measurement change as another changes? **point** draws one glyph, a small symbol, at the (x, y) position of each row. The glyph is a dot unless you choose another.

The plot it makes is the **scatterplot**, and it was invented later than the other basic charts. Playfair published the line chart and the bar chart in 1786 (Playfair, 1786), and the pie chart in 1801 (Playfair, 1801). The scatterplot came about thirty years after the pie chart. A picture that plots one measured column against another is usually dated to John Herschel. In 1833 he plotted the position of twin stars against the year, then drew a smooth curve through the dots so the trend could be seen (Friendly & Denis, 2005). Other names for the same picture are scatter diagram and scattergram. When **size** maps a third column, the same plot is called the bubble chart.

None of those chart names is a word in **gog**, and that is deliberate. **point** is a **mark**, not a chart. **point + x(gdp) + y(life)** is a scatterplot, and **point + x(continent) + y(life)** is a strip plot. The mark is the same in both. The chart name belongs to the whole sentence, not to any word in the vocabulary. That is why the names live in an appendix, [Chart names](#), instead of in the kernel.

8.1 Basic usage

R

```
data(gapminder_2007) + point + x(gdp) + y(life)
```

Python

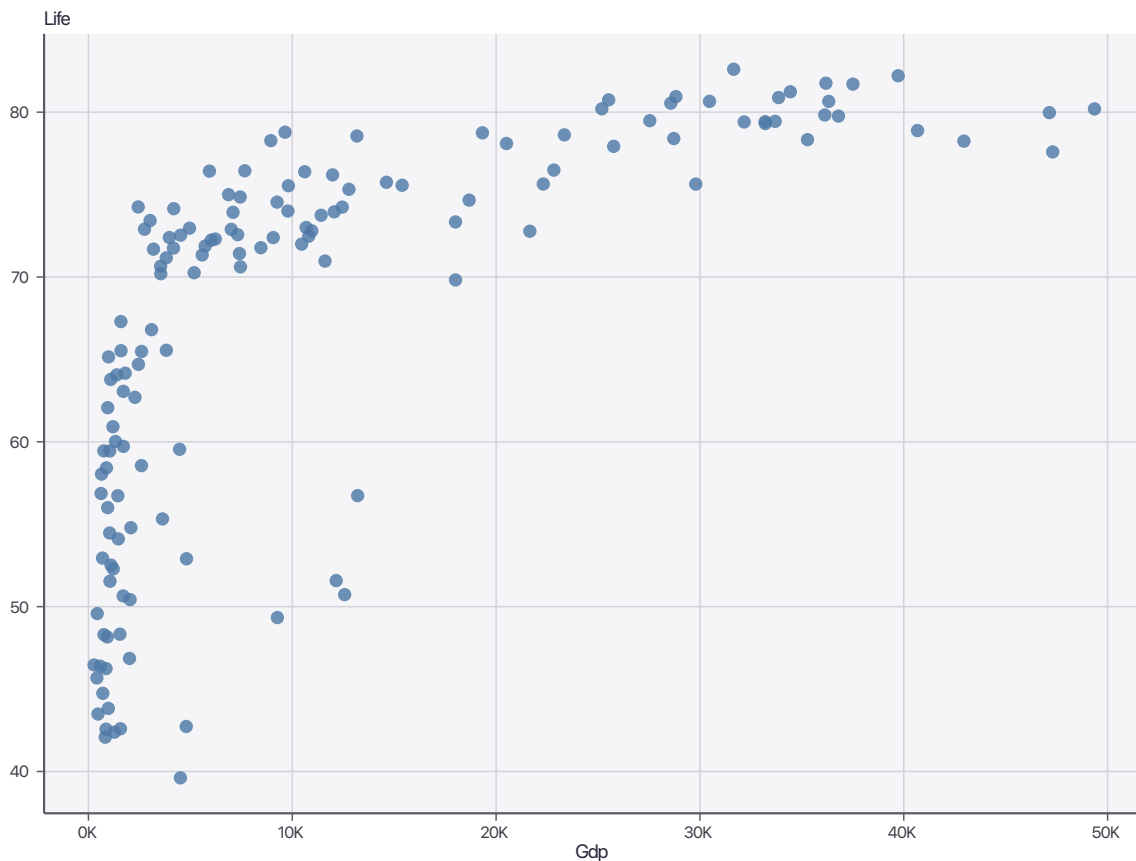
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life))
```



“Given gapminder 2007: points, x is gdp, y is life.”

This is the minimum sentence for a point: a source, the mark, and its two positions. Each dot is one row, one country, placed by its **gdp** and its **life**. Nothing else about the row is shown yet. Both columns hold numbers, so each axis is a linear scale. Most of the dots sit on the left, because most countries have far lower incomes than the richest few. The [Scales](#) chapter shows the log scale that spreads them out.

8.2 The dot as a default

Write **point** and you get a dot, and treating it as *the dot mark* is right almost every time. But the dot is only a *default*, and you can change it. A point is a *glyph at a position*, and **shape**

replaces the dot with one of the five glyphs **gog** draws. In order, they are the circle, the square, the triangle, the diamond and the cross, one per category. Categories are taken as they first appear in the table, or in the [order you declare](#). Asia comes first in this table, so Asia is the circle, and the legend reads the same way. The table has five continents, so each gets its own.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + shape(continent) +  
  title("The glyph is not always a dot")
```

Python

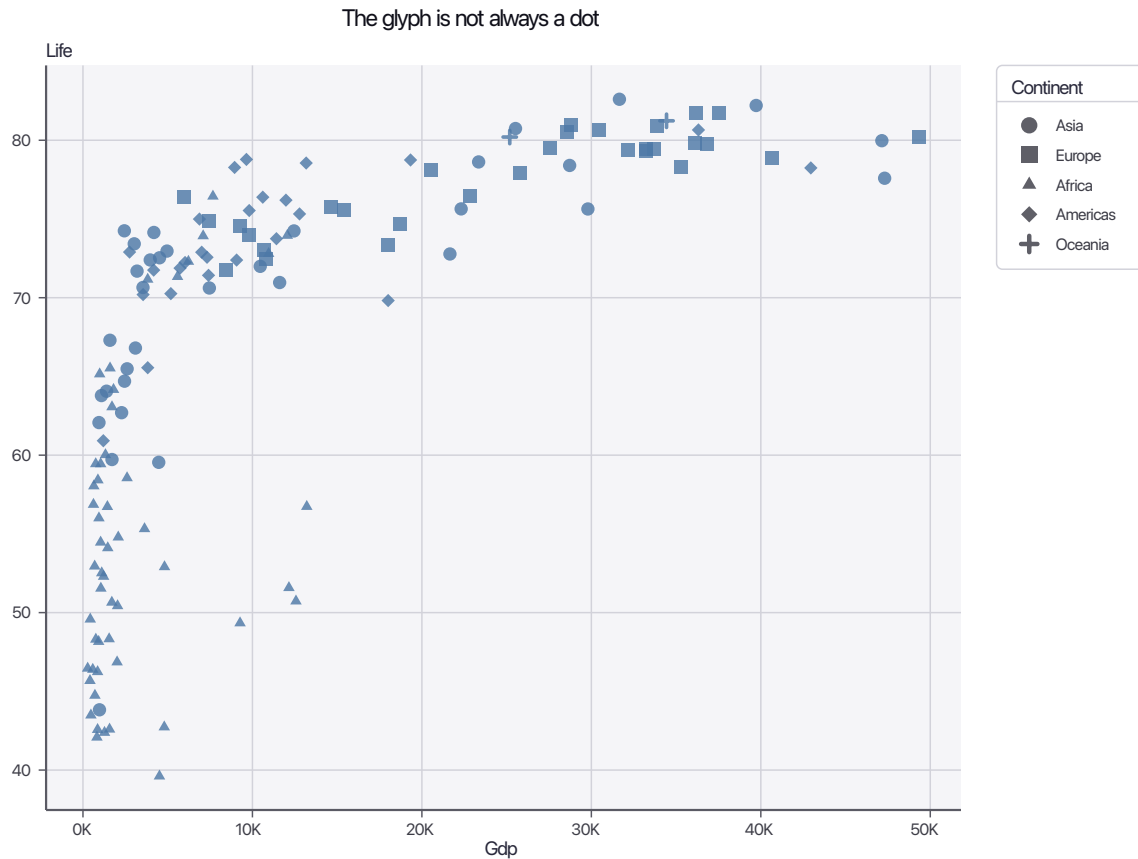
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ shape(col.continent) +  
  title("The glyph is not always a dot"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + shape(:continent) +  
  title("The glyph is not always a dot")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  shape(col.continent), title("The glyph is not always a dot"))
```

“Given gapminder 2007: points, x is gdp, y is life, shape by continent.”

So “a dot” means “the first of the five shapes, drawn when you name no shape.” That is what makes `point` and `text siblings`: both draw one glyph per row at (x, y), differing only in where the glyph comes from. A point picks its glyph from the fixed set of five, with `shape`. A text mark takes its glyph from a column, with `label`, so what you see is the value in that column. Put both on one plot and each row is located by a dot and named by its label:

R

```
data(medals) + point + style(size = 5, color = "#9e9e9e") +
  text + x(gold) + y(silver) + label(country) + style(nudge = "up") +
  x_label("Gold") + y_label("Silver") +
  title("point and text, siblings on one plot")
```

Python

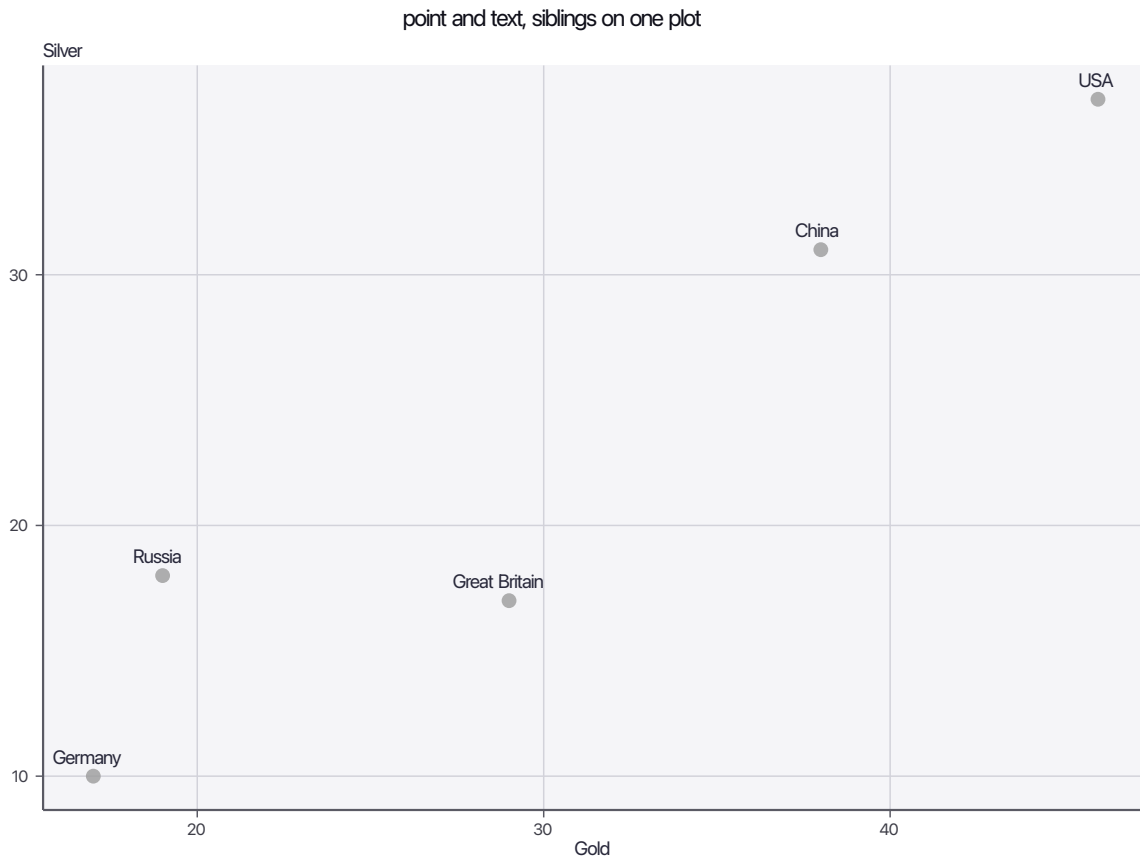
```
(data(medals) + point + style(size = 5, color = "#9e9e9e") +
  text + x(col.gold) + y(col.silver) + label(col.country) + style(nudge =
↪ "up") +
  x_label("Gold") + y_label("Silver") +
  title("point and text, siblings on one plot"))
```

Julia

```
data(medals) + point + style(size = 5, color = "#9e9e9e") + text +
  x(:gold) + y(:silver) + label(:country) + style(nudge = "up") +
  x_label("Gold") + y_label("Silver") +
  title("point and text, siblings on one plot")
```

JavaScript

```
plot(data(medals), point, style({ size: 5, color: "#9e9e9e" }), text,
  x(col.gold), y(col.silver), label(col.country), style({ nudge: "up" }),
  x_label("Gold"), y_label("Silver"),
  title("point and text, siblings on one plot"))
```



“Given the medals: points and also text, x is gold, y is silver, label by country.”

8.3 A categorical x: the strip plot

`x` accepts a text column as readily as a numeric one. Each category gets a position on the axis, and every row is drawn there. You see the whole distribution rather than a summary of it:

R

```
data(gapminder_2007) + point + x(continent) + y(life) +  
  y_label("Life expectancy") + title("One dot per country")
```

Python

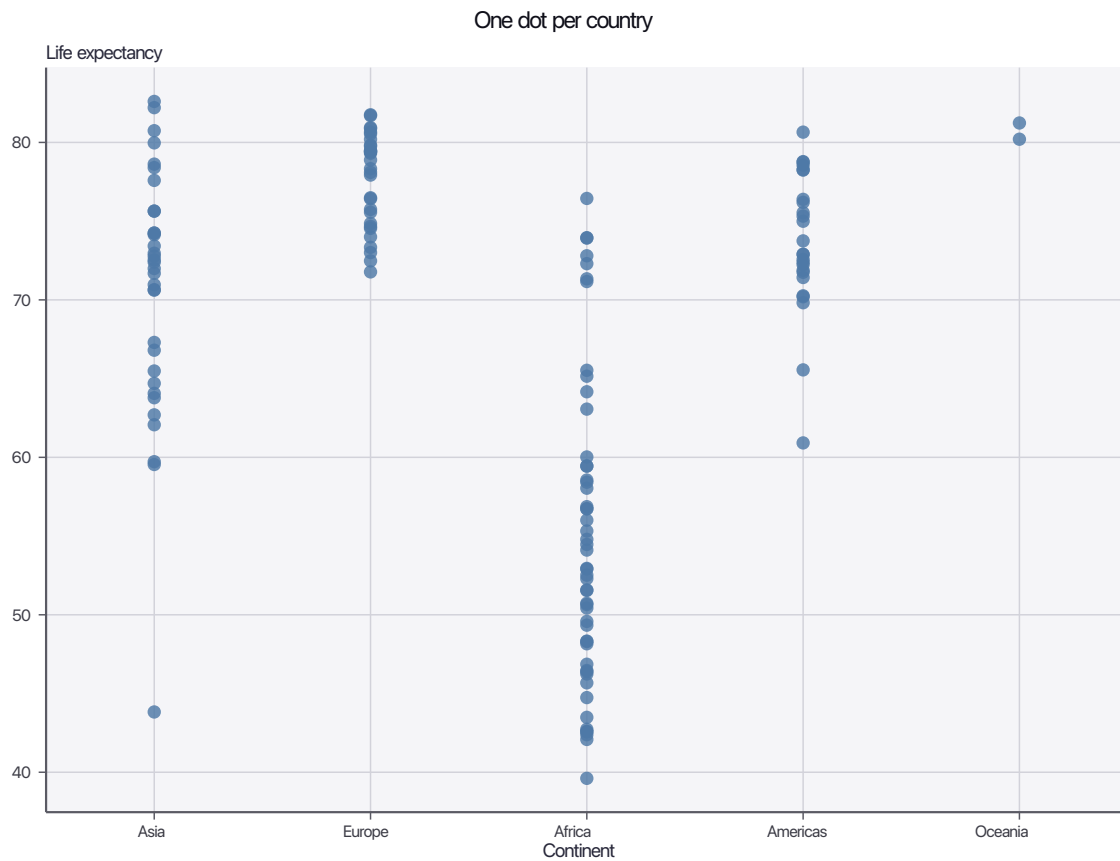
```
(data(gapminder_2007) + point + x(col.continent) + y(col.life) +
  y_label("Life expectancy") + title("One dot per country"))
```

Julia

```
data(gapminder_2007) + point + x(:continent) + y(:life) +
  y_label("Life expectancy") + title("One dot per country")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.continent), y(col.life),
  y_label("Life expectancy"), title("One dot per country"))
```



This is the same rule `bar` uses for a categorical axis, and no mark gets a special case. Compare it with `bar * mean`, which collapses each group to one number:

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life) +  
  y_label("Mean life expectancy") + title("The same data, summarized")
```

Python

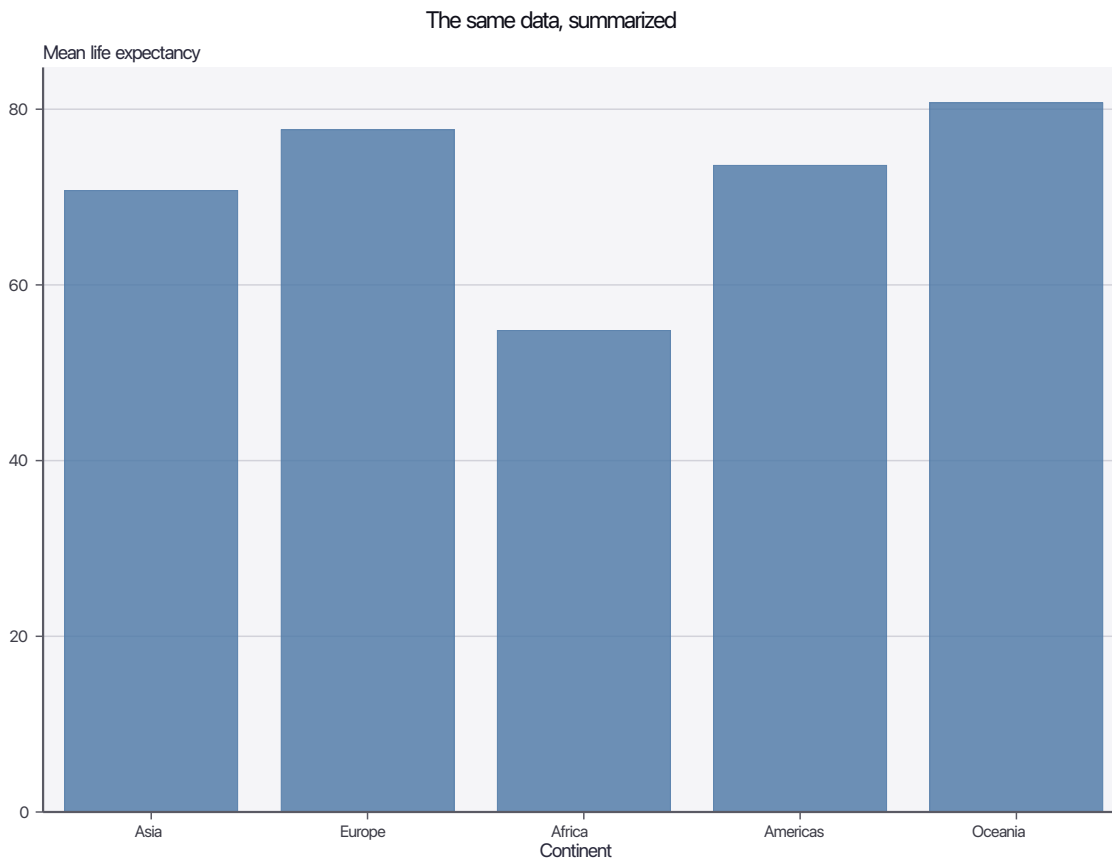
```
(data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life) +  
  y_label("Mean life expectancy") + title("The same data, summarized"))
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life) +  
  y_label("Mean life expectancy") + title("The same data, summarized")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),  
  y(col.life), y_label("Mean life expectancy"),  
  title("The same data, summarized"))
```



“Given gapminder 2007: bars derived by mean, x is continent, y is life.”

The strip plot shows Africa’s spread and its overlap with Asia; the bar chart shows neither. The strip plot has one weakness: every point in a category sits on the same vertical line, so heavy overlap can hide how many there are. `opacity` makes each point partly transparent, so overlap reads as a darker patch. `point * jitter` spreads overlapping points along the category axis, never off their measured value.

A second answer is to count the overlapping points instead of separating them. `point * bin * stack` stacks one dot per row along the measure axis, which is the dot plot. The cookbook draws it with `thirty rows`, where every row still shows. One mark handles overlap in two ways: spread along a category, or count along a measure.

8.3.1 Either axis: a horizontal strip

Because the categorical-axis rule is `bar`’s rule, the category can sit on y instead of x. The strips then lie down. The countries and the distributions are the same, and each row now reads left

to right:

R

```
data(gapminder_2007) + point + x(life) + y(continent) +  
  style(opacity = 0.6) +  
  x_label("Life expectancy") +  
  title("The same distributions, lying down")
```

Python

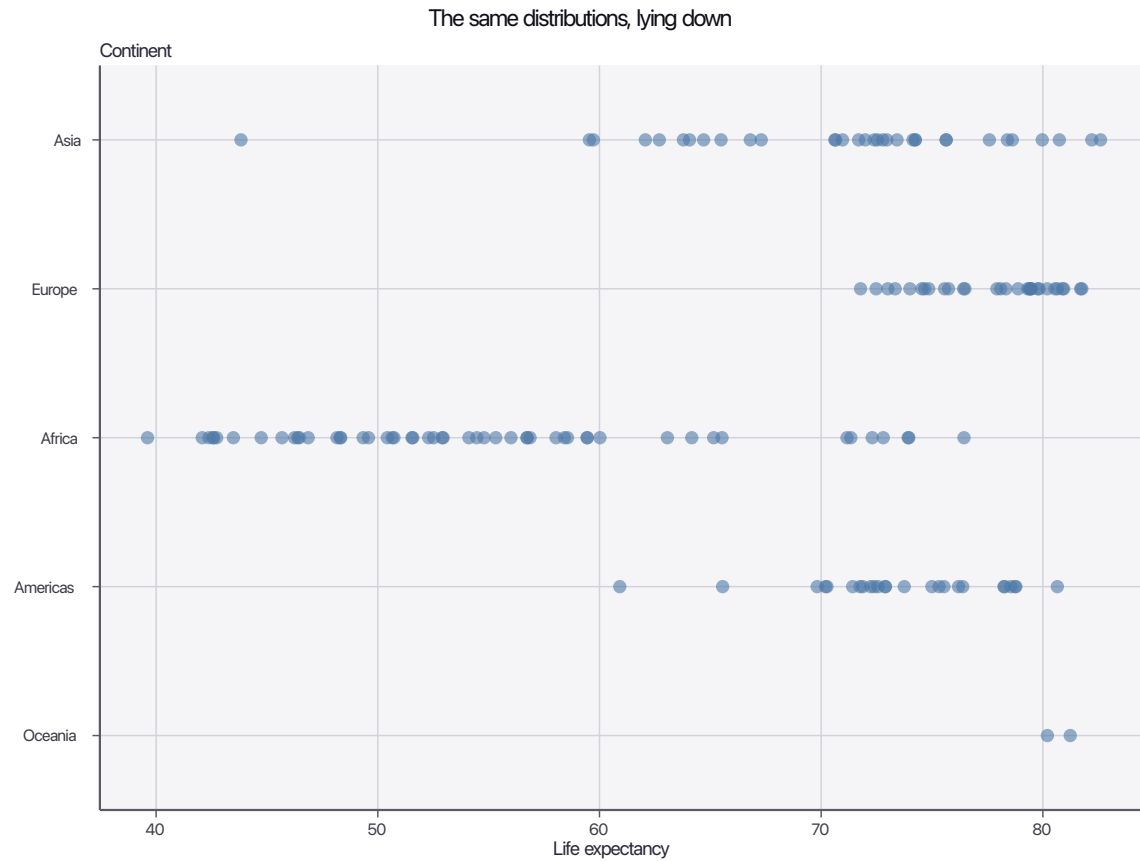
```
(data(gapminder_2007) + point + x(col.life) + y(col.continent)) +  
  style(opacity = 0.6) +  
  x_label("Life expectancy") +  
  title("The same distributions, lying down"))
```

Julia

```
data(gapminder_2007) + point + x(:life) + y(:continent) +  
  style(opacity = 0.6) + x_label("Life expectancy") +  
  title("The same distributions, lying down")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.life), y(col.continent),  
  style({ opacity: 0.6 }), x_label("Life expectancy"),  
  title("The same distributions, lying down"))
```



`bar` already works this way, and `gog` reads a point the same way. Whichever axis carries the category decides the direction, and there is no `flip` atom to set. Use the horizontal form when the category names are too long for the bottom axis. It also helps when there are many categories, since a list of rows is easier to read than a list of columns.

8.4 Color

Bind `color` to a categorical column to color points by group. `gog` assigns a distinct color from a 20-color palette automatically.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent)
```


Python

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent.”

A color legend is generated automatically whenever a column is mapped to `color`.

8.5 Shape

shape maps a categorical column to one of five glyphs: circle, square, triangle, diamond, or cross. Assignment is automatic and consistent: the first category in the table's order gets the circle, the second the square, and so on. Five is the whole set, and about as many as a reader can separate on one plot. Do not map **shape** to a column with more than five categories: the sixth starts over at the circle, two categories then share a glyph and look the same, and **gog** does not warn. The book shows four ways to draw more groups than that:

- **color**, whose default palette has twenty colors, as [Color](#) above does with the continents.
- One panel per category with **facet**, folded into rows so the panels stay wide: [Folding a long line of panels](#).
- The name on the point instead of a glyph: a **text** layer with **label**, the [labeled scatter](#), and [repel](#) when the names overlap. This is the one route that reaches a hundred categories.
- Every category in one pale color, with the few that matter drawn over it in a second layer: [What settings are for](#).

In the web edition a [brush](#) dims the points and names outside a selection, so the reader chooses which twenty are readable. And a coarser column often carries the encoding: **continent** fits the five glyphs, and **country** goes on the label or the facet.

Combine **color** and **shape** on the same column so that colorblind readers, and black-and-white printing, can still separate the groups:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  color(continent) + shape(continent)
```

Python

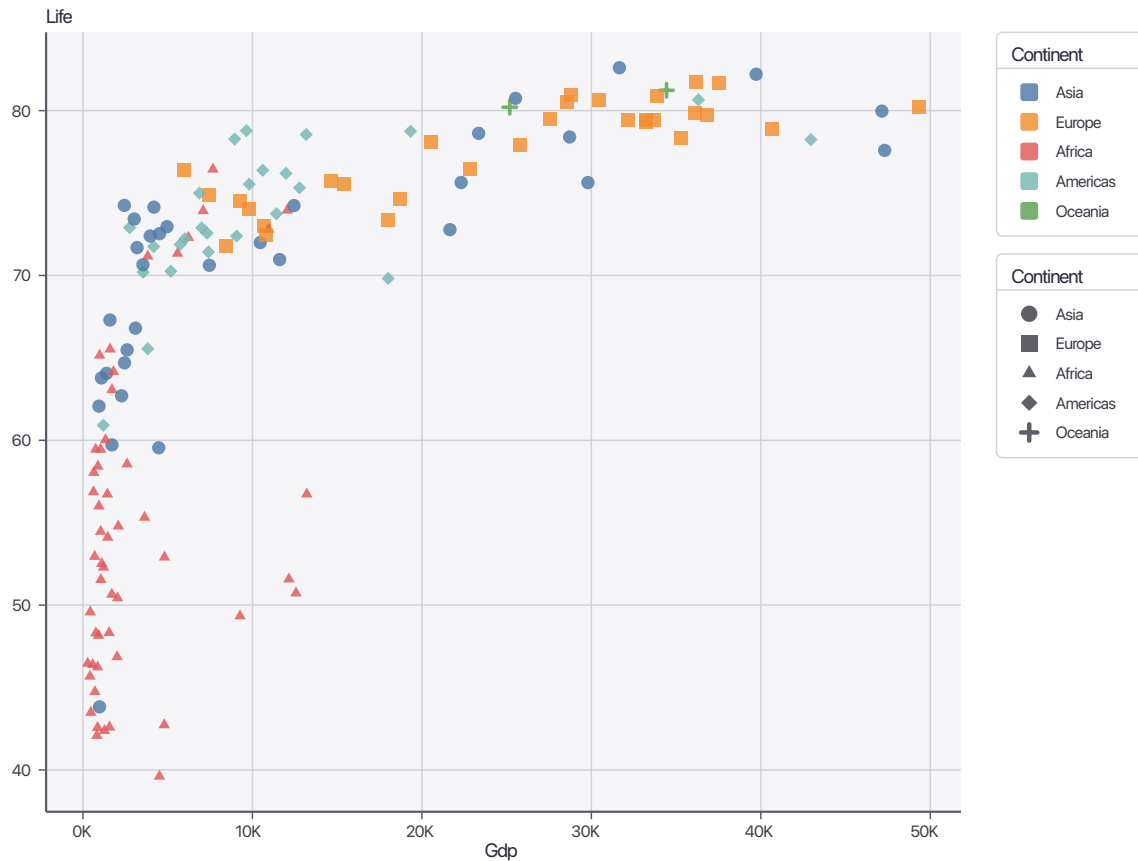
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.continent) + shape(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  shape(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
     color(col.continent), shape(col.continent))
```



8.6 Size

`size` maps a numeric column to the point radius, from 3 pixels at the smallest value to 12 at the largest.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + size(population)
```

Python

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.continent) + size(col.population))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  size(:population)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent), size(col.population))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent, size by population.”

A size legend showing the minimum, midpoint, and maximum of the column is generated automatically.

8.7 Opacity

`opacity` maps a numeric column to transparency. It helps in a dense scatterplot, where marks cover one another and hide the shape of the data.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + opacity(population) +  
  title("Opacity: population")
```

Python

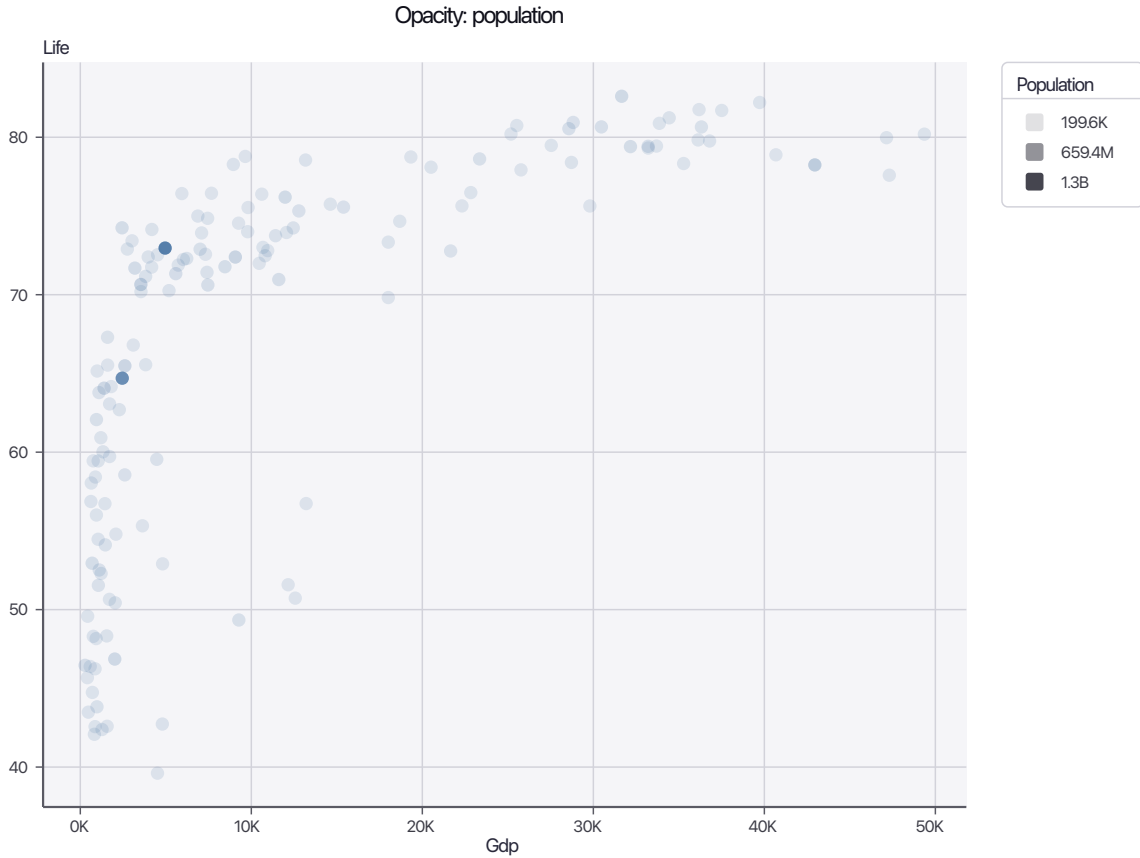
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ opacity(col.population) +  
  title("Opacity: population"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + opacity(:population) +  
  title("Opacity: population")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  opacity(col.population), title("Opacity: population"))
```



“Given *gapminder 2007*: *points*, *x* is *gdp*, *y* is *life*, *opacity* by *population*.”

See [Channels and legends](#) for the opacity range and why `line` does not accept this channel.

That example maps opacity to a column. When points simply cover one another, you want one opacity on every point instead. That is a *setting* rather than a mapping, so you write `style(opacity = )` and no legend appears; see [Setting vs mapping](#). The table below, `gm_all`, holds every *gapminder* country in every year, so the points overlap heavily:

Table 8.1: `gm_all`: first 5 of 1704 rows

country	continent	year	life	population	gdp
Afghanistan	Asia	1952	28.801	8425333	779.4453
Afghanistan	Asia	1957	30.332	9240934	820.8530
Afghanistan	Asia	1962	31.997	10267083	853.1007
Afghanistan	Asia	1967	34.020	11537966	836.1971
Afghanistan	Asia	1972	36.088	13079460	739.9811

R

```
data(gm_all) + point + x(gdp) + y(life) +  
  style(opacity = 0.2, size = 3) +  
  title("Every point at one opacity")
```

Python

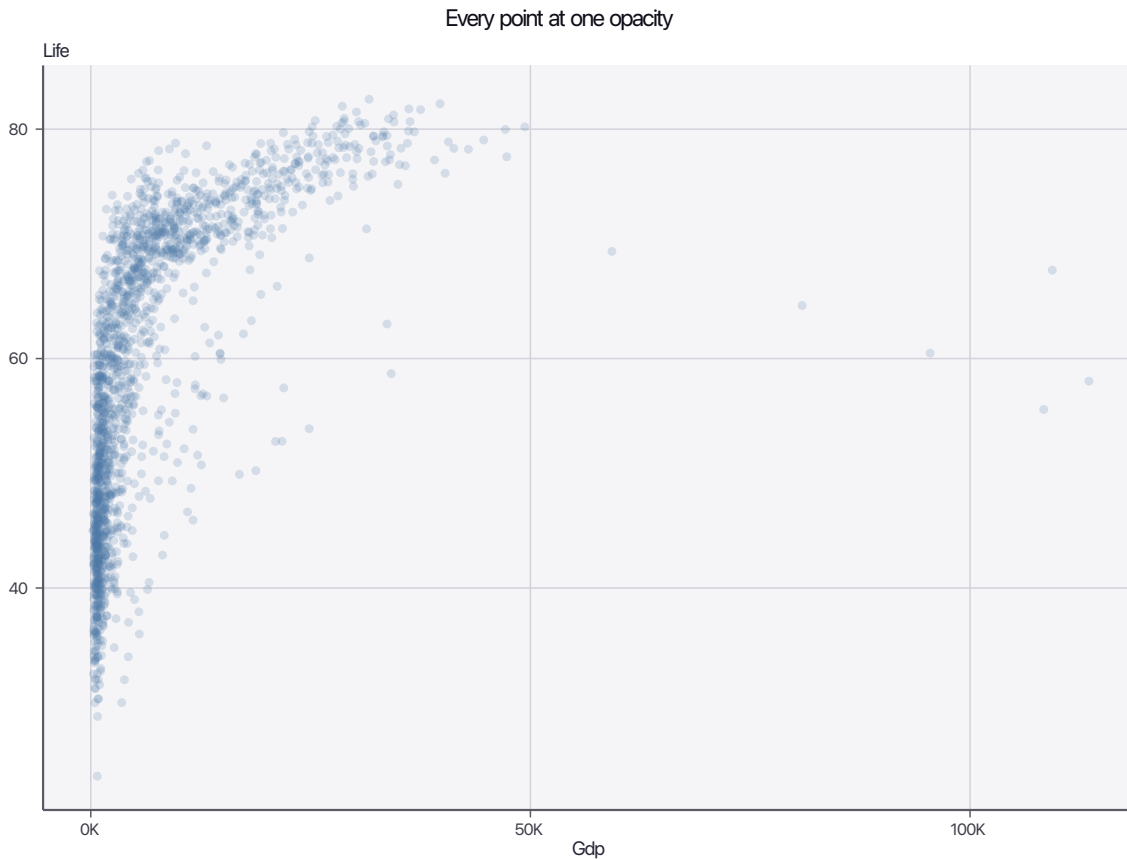
```
(data(gm_all) + point + x(col.gdp) + y(col.life) +  
  style(opacity = 0.2, size = 3) +  
  title("Every point at one opacity"))
```

Julia

```
data(gm_all) + point + x(:gdp) + y(:life) +  
  style(opacity = 0.2, size = 3) + title("Every point at one opacity")
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp), y(col.life),  
  style({ opacity: 0.2, size: 3 }), title("Every point at one opacity"))
```



“Given all the gapminder years: points, x is gdp, y is life, with opacity 0.2.”

8.8 All channels at once

The refining channels can all be combined on one plot:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  color(continent) +  
  shape(continent) +  
  size(population) +  
  title("GDP, life expectancy, and population")
```

Python

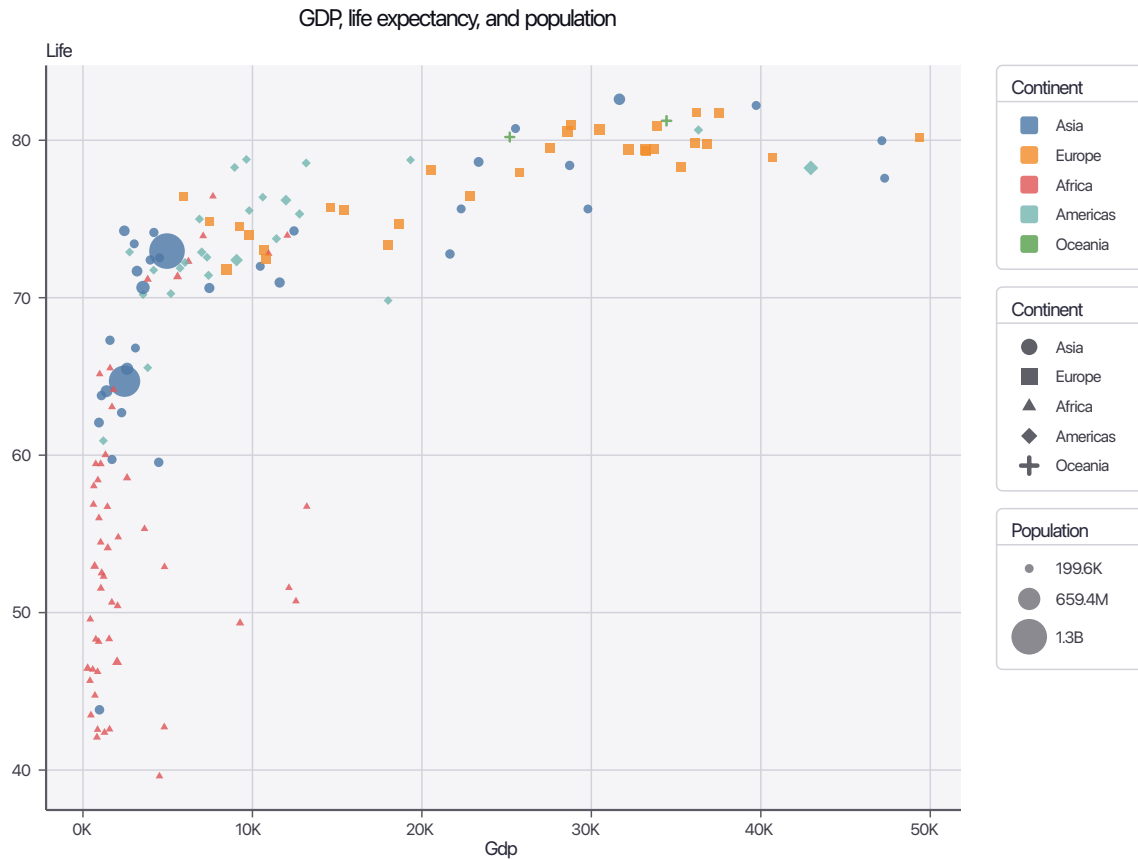

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.continent) +  
  shape(col.continent) +  
  size(col.population) +  
  title("GDP, life expectancy, and population"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  shape(:continent) + size(:population) +  
  title("GDP, life expectancy, and population")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent), shape(col.continent), size(col.population),  
  title("GDP, life expectancy, and population"))
```



8.9 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(shape =)</code>	circle, square, triangle, diamond, cross
<code>style(border_color =)</code>	any CSS color name or hex
<code>style(border_size =)</code>	pixels

And these vary per row if you map them to a column instead: `color()` (either), `size()` (numbers), `shape()` (categories), `opacity()` (numbers), `play()` (either).

A point is a closed glyph, a shape with an inside to fill, so it takes more settings than most marks. It is the only mark that picks a **shape**, and one of the five marks that draw a border. **size** is the radius in pixels, and **border_*** draws that border distinct from the fill.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  style(shape = "diamond", size = 7, color = "steelblue",
        border_color = "white", border_size = 1.5) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("A shape, a size, and a border")
```

Python

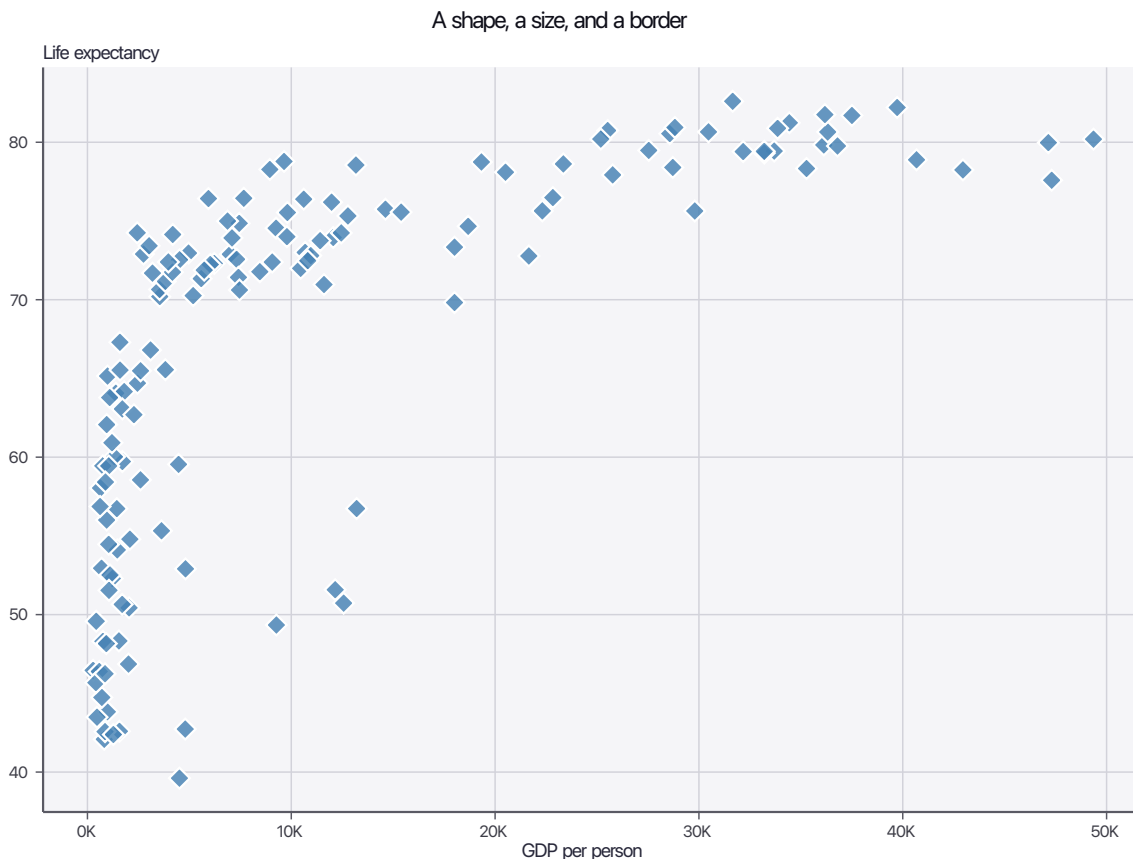
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  style(shape = "diamond", size = 7, color = "steelblue",
        border_color = "white", border_size = 1.5) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("A shape, a size, and a border"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  style(shape = "diamond", size = 7, color = "steelblue", border_color =
  ↪ "white", border_size = 1.5) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("A shape, a size, and a border")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  style({ shape: "diamond", size: 7, color: "steelblue",
  border_color: "white", border_size: 1.5 }), x_label("GDP per person"),
  y_label("Life expectancy"), title("A shape, a size, and a border"))
```



“Given gapminder 2007: points, x is gdp, y is life, colored steelblue, with shape diamond, size 7, border color white and border size 1.5.”

The white border is what separates the overlapping points, which is what a border is for in a dense cloud. A `cross` has no fill to outline, so `border_color` and `border_size` change nothing on that shape.

There is no **pattern**. A texture needs an area to fill or a stroke to dash, and a small glyph is neither; a point’s form is **shape**. The other way to handle points that cover one another is **jitter**, one of the four collision modifiers. It takes an amount: `point * jitter(0.4)` spreads overlapping points along a categorical axis.

8.10 What it refuses

A point is a glyph, and the two refusals below both follow from that.

`size` shows an amount, so it needs a column of amounts:

```
data(gapminder_2007) + point + x(gdp) + y(life) + size(continent)
```

Error:

```
! gog: `size(continent)` maps a categorical (text) column, but `size` on
  ↳ `point` needs a continuous (numeric) column. Use `color`, `shape`, or
  ↳ `pattern` to distinguish categories.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Continents are names, not amounts, and a larger dot would claim an order the data does not have. The refusal names three channels that separate categories: **color**, **shape** and **pattern**. A point takes the first two, and the next section shows why it refuses **pattern**.

The second refusal is texture. A pattern is a stroke's dash or a fill's hatch, and a small closed glyph is neither:

```
data(gapminder_2007) + point + x(gdp) + y(life) + style(pattern = "dashed")
```

Error:

```
! gog: `style(pattern = )` is a stroke's dash or a fill's texture, and a
  ↳ `point` is a glyph, not either - a point's form is set by `shape`, a
  ↳ text's by its content. The strokes
  ↳ (`line`/`step`/`interval`/`path`/`rule`/`edge`) take a dash; the fills
  ↳ (`area`/`bar`/`box`/`ribbon`/`zone`) a texture.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Both messages send you to **shape**, which is the one form a point can change.

9 Line

How did one value change over time? **line** connects the rows in **x order**, drawing one unbroken stroke (a polyline) through all the (x, y) points.

9.1 Basic usage

R

```
data(gapminder_asia) + line + x(year) + y(life)
```

Python

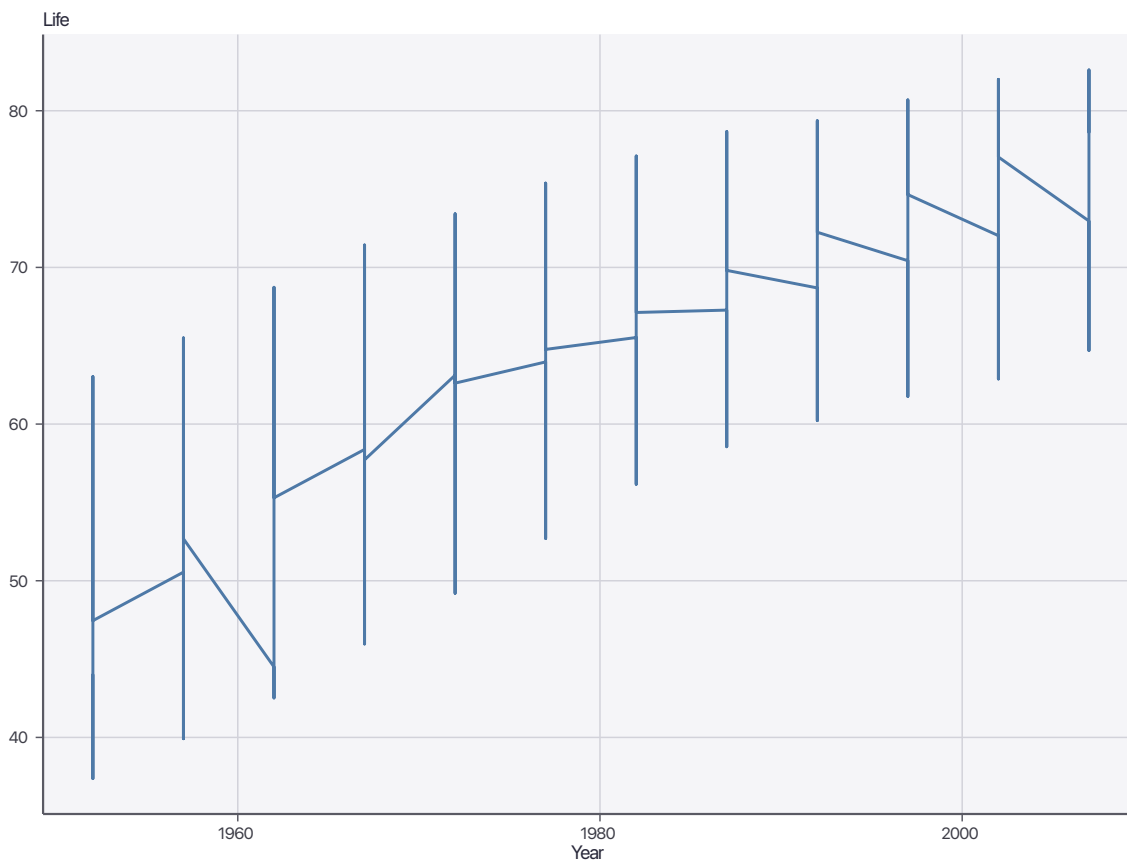
```
data(gapminder_asia) + line + x(col.year) + y(col.life)
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life))
```



“Given gapminder Asia: a line, x is year, y is life.”

All five countries have a row for each year, so the stroke passes through five values at every year. The vertical segments join the five countries inside one year. That is what a line does when nothing splits the rows: it draws one **series**, a single stroke in x order. `gog` warns about this sentence and says what to add, a `color` or a `group`. [Multiple series](#) below gives each country its own line.

9.2 Automatic sorting by x

`line` sorts its own vertices, the points it joins, by `x`, so the rows may arrive in any order. The table below is deliberately shuffled, and the line still runs from left to right:

R

```
scrambled <- data.frame(year = c(1992, 1962, 2002, 1972, 1982),  
                        life = c( 68,  52,  71,  58,  63))  
  
data(scrambled) + line + x(year) + y(life)
```

Python

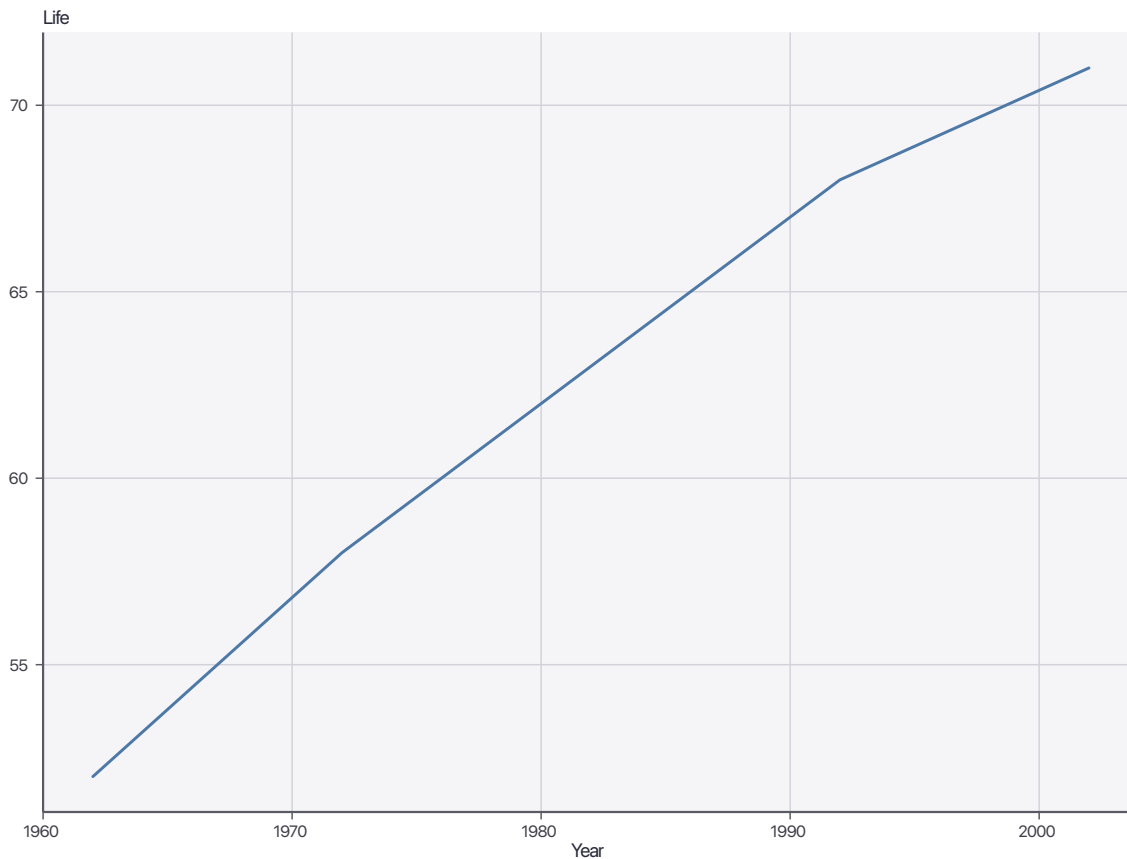
```
scrambled = {"year": [1992, 1962, 2002, 1972, 1982], "life": [ 68,  52,  
    ↪ 71,  58,  63]}  
data(scrambled) + line + x(col.year) + y(col.life)
```

Julia

```
scrambled = (year = [1992, 1962, 2002, 1972, 1982], life = [68, 52, 71, 58,  
    ↪ 63],)  
data(scrambled) + line + x(:year) + y(:life)
```

JavaScript

```
const scrambled = { year: [1992, 1962, 2002, 1972, 1982], life: [68, 52, 71,  
    ↪ 58, 63] };  
plot(data(scrambled), line, x(col.year), y(col.life))
```

That sorting is part of what the mark means. It is not an extra step that the engine performs for you. A line says *this value changed along this axis*, and an axis has one direction, so there is only one order the vertices could be joined in.

`path` is the mark that does the opposite. It joins the rows in the order your table holds them, which is what draws a route or a loop. So choosing between the two is choosing whether the order of your rows carries meaning: With `line` the row order cannot carry meaning; with `path` it decides the whole shape.

9.3 Multiple series

Use `color` to split into one line per group:

R

```
data(gapminder_asia) + line + x(year) + y(life) +
  color(country)
```

Python

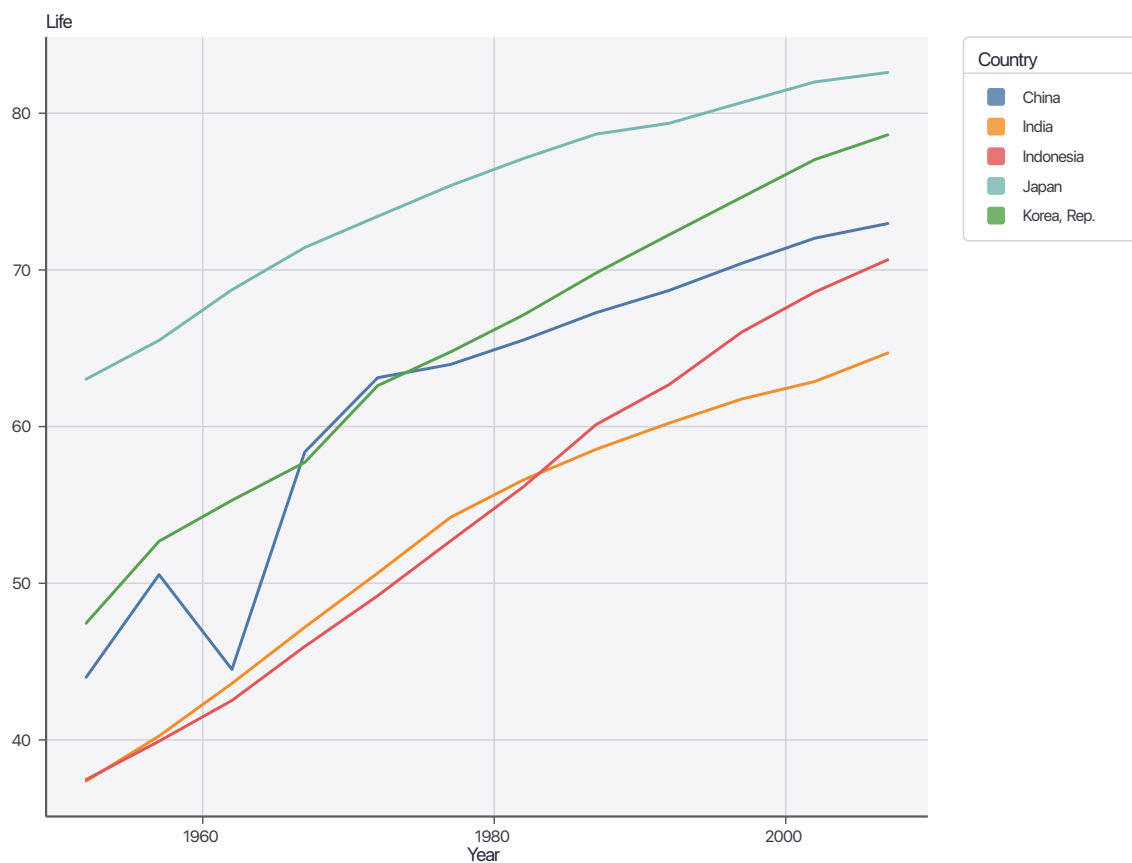
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) +
  color(col.country))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + color(:country)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),
  color(col.country))
```



“Given gapminder Asia: lines, x is year, y is life, color by country.”

Each unique value in `country` becomes a separate line with its own color. A color legend is generated automatically.

9.4 Layering line over other marks

`line` and `point` are often combined: the point layer adds a dot at each data position. Write `color` *before* the marks and it belongs to the plot, so both layers use it:

R

```
data(gapminder_asia) + x(year) + y(life) + color(country) +  
  line + point
```

Python

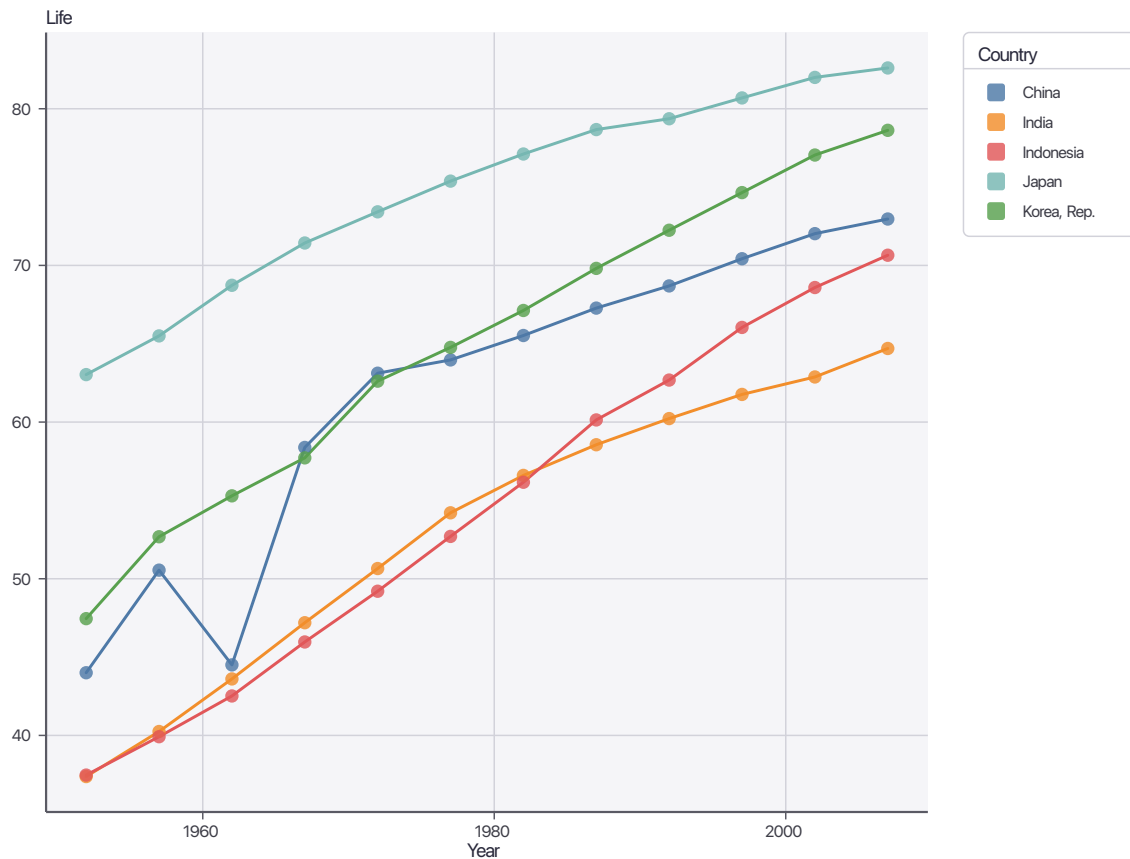
```
(data(gapminder_asia) + x(col.year) + y(col.life) + color(col.country) +  
  line + point)
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + color(:country) + line +  
  point
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), color(col.country),  
  line, point)
```



“Given `gapminder Asia`: x is year, y is life, color by country, lines and also points.”

Both the lines and the points share the same color per country.

9.4.1 Giving each layer its own appearance

`style()` attaches to the nearest preceding mark and *stops there*, so each layer can be styled independently:

R

```
data(gapminder_asia) + x(year) + y(life) +
  line + group(country) + style(color = "steelblue") +
  point + style(color = "firebrick", size = 3) +
  title("Blue line, red points: styled per layer")
```

Python

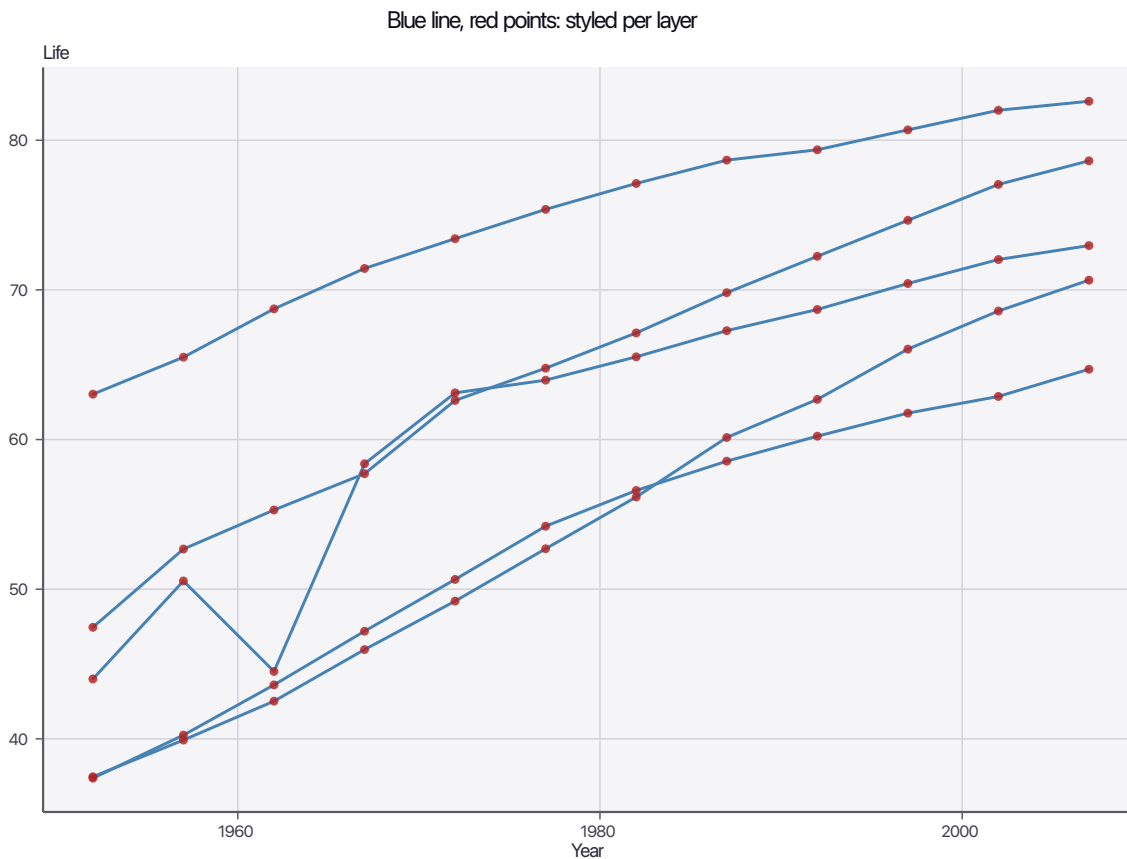
```
(data(gapminder_asia) + x(col.year) + y(col.life) +  
  line + group(col.country) + style(color = "steelblue") +  
  point + style(color = "firebrick", size = 3) +  
  title("Blue line, red points: styled per layer"))
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + line + group(:country) +  
  style(color = "steelblue") + point +  
  style(color = "firebrick", size = 3) +  
  title("Blue line, red points: styled per layer")
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), line,  
  group(col.country), style({ color: "steelblue" }), point,  
  style({ color: "firebrick", size: 3 }),  
  title("Blue line, red points: styled per layer"))
```



“Given gapminder Asia: x is year, y is life, lines grouped by country, colored steelblue, and also points, colored firebrick, with size 3.”

Channels scope the same way. A channel written after a mark binds to that mark alone, so each layer can carry a different encoding:

R

```
data(gapminder_asia) + x(year) + y(life) +
  line + color(country) +
  point + size(population) +
  title("Color on the lines, size on the points")
```

Python

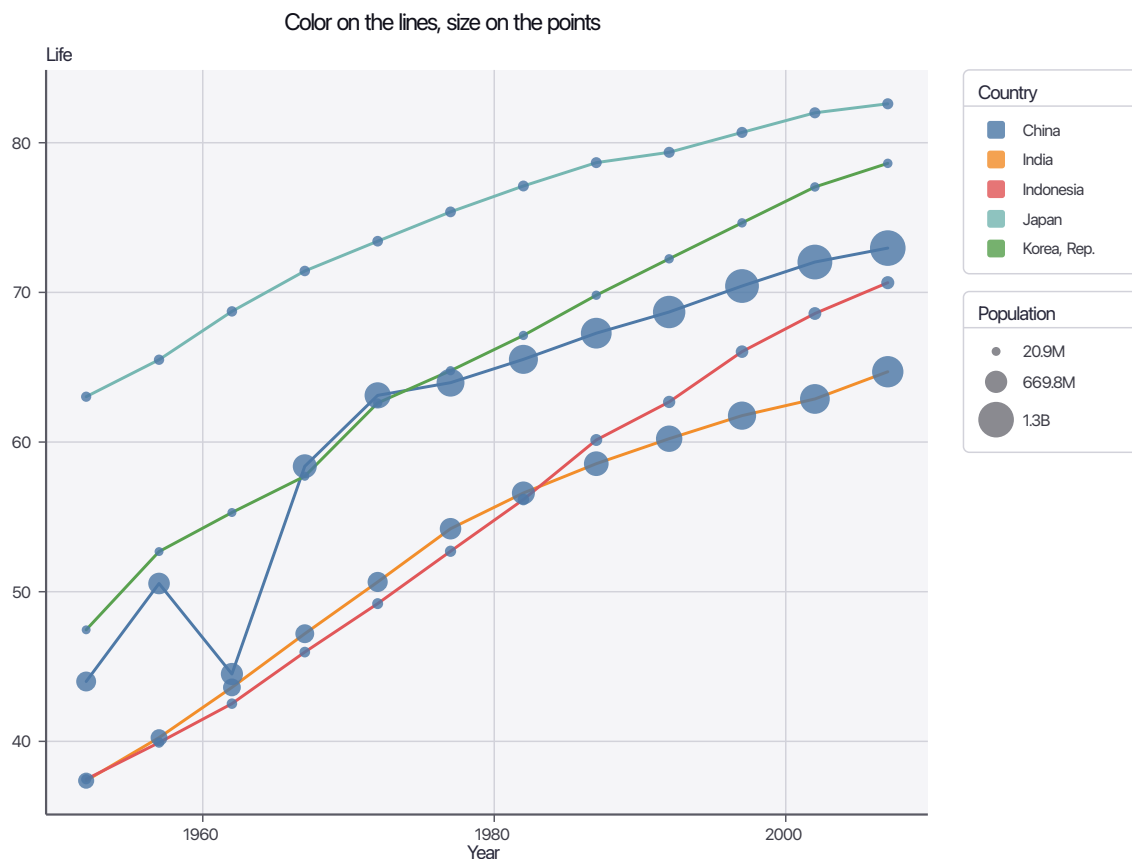
```
(data(gapminder_asia) + x(col.year) + y(col.life) +
  line + color(col.country) +
  point + size(col.population) +
  title("Color on the lines, size on the points"))
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + line + color(:country) +
  point + size(:population) +
  title("Color on the lines, size on the points")
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), line,
  color(col.country), point, size(col.population),
  title("Color on the lines, size on the points"))
```



“Given gapminder Asia: *x* is year, *y* is life, lines, color by country, and also points, size by population.”

That works because channels bind *forward only*. A channel that also bound backwards would put `size` on the `line`, which takes no per-row size, and the plot would be refused. See [Encoding scope](#).

9.5 A category on the domain: the profile

A line’s two axes are not the same kind of thing. `x` is the **domain**, the axis the line is read along; `y` is the **measure**, the quantity it traces. The domain takes a category as readily as a number, and connecting one summary per category is the profile plot:

R

```
data(gapminder_2007) + line * mean + point * mean + x(continent) + y(life) +  
  y_label("Mean life expectancy") +  
  title("line * mean + x(continent): one value per category, joined")
```

Python

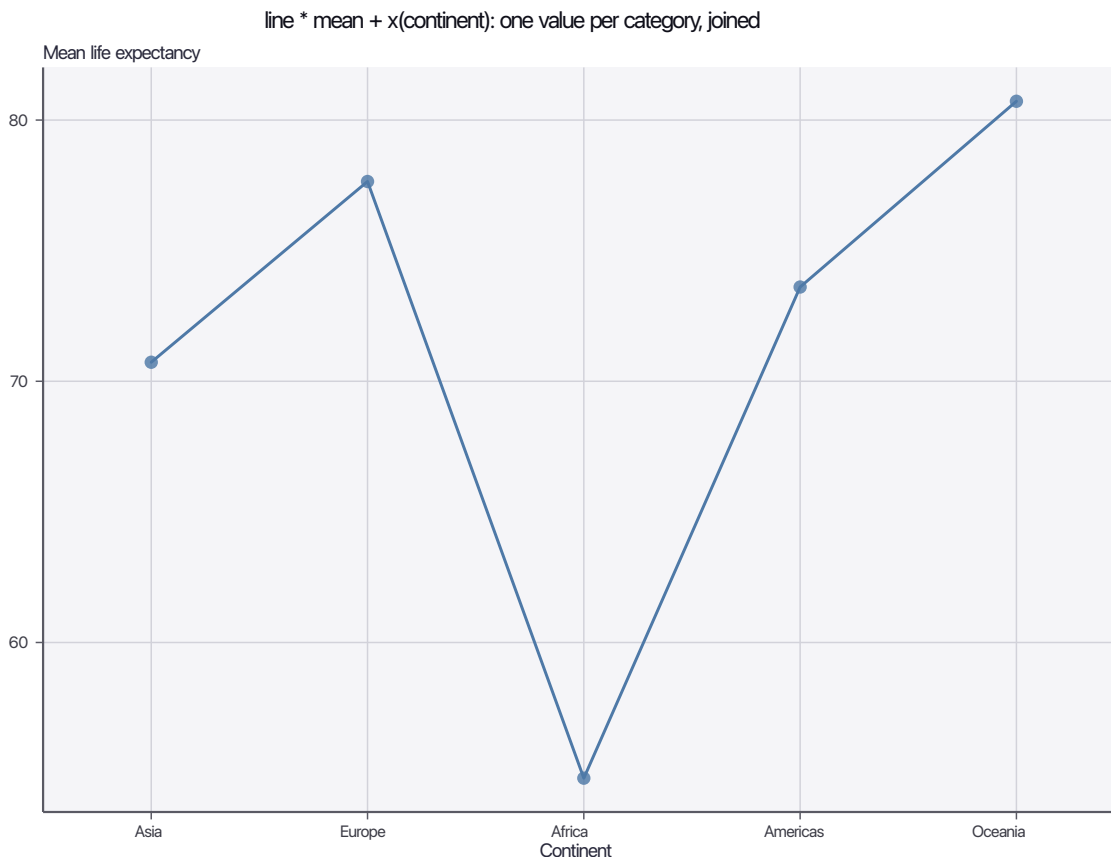
```
(data(gapminder_2007) + line * mean + point * mean + x(col.continent) +  
  ↪ y(col.life) +  
  y_label("Mean life expectancy") +  
  title("line * mean + x(continent): one value per category, joined"))
```

Julia

```
data(gapminder_2007) + line * mean + point * mean + x(:continent) +  
  y(:life) + y_label("Mean life expectancy") +  
  title("line * mean + x(continent): one value per category, joined")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, mean), layer(point, mean),  
  x(col.continent), y(col.life), y_label("Mean life expectancy"),  
  title("line * mean + x(continent): one value per category, joined"))
```

“Given gapminder 2007: a line derived by mean and also points derived by mean, x is continent, y is life.”

The transform is what makes that plot readable. Without one, a **line** would connect all 142 rows across five category slots, and no reader can follow that. The engine prints a warning on your console when a line has many ungrouped rows. **mean** leaves exactly one point per category, which is what makes the connected line mean anything.

Does the connection mean anything? That depends on your data. Nothing was measured between Africa and the Americas, and the segment is still drawn there. If you read only the vertices, the segments help you compare the five heights. If you read the segments themselves, they claim a change that never happened. The grammar draws what you asked for. You decide whether your data supports the connection. Two cases make the judgment easy. An ordered category, such as a grade, a size band or a survey scale, genuinely has a direction for the line to follow. A *circular* category has no ends at all, and that is the next plot.

Drop the point layer, add **polar()**, and the same profile bends into a circle. That is the **radar chart**, and it needs no new atom:

R

```
data(gapminder_2007) + line * mean + x(continent) + y(life) + polar() +  
  title("line * mean + polar(): the radar")
```

Python

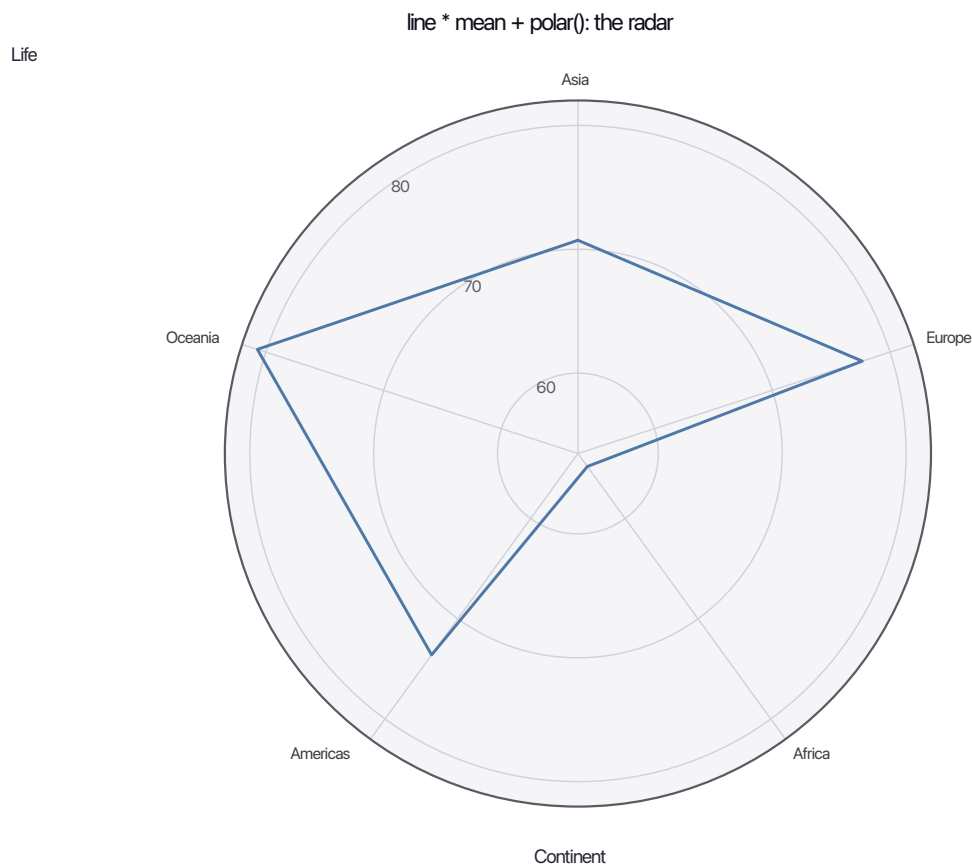
```
(data(gapminder_2007) + line * mean + x(col.continent) + y(col.life) +  
  ↪ polar() +  
  title("line * mean + polar(): the radar"))
```

Julia

```
data(gapminder_2007) + line * mean + x(:continent) + y(:life) + polar() +  
  title("line * mean + polar(): the radar")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, mean), x(col.continent),  
  y(col.life), polar(), title("line * mean + polar(): the radar"))
```



“Given gapminder 2007: a line derived by mean, x is continent, y is life, in polar.”

Look at the segment from Oceania back to Asia. On the flat profile the line has two ends, because a straight axis has two ends. In the circle the five categories are spread around the whole ring, so the last one sits beside the first and the line closes. Nothing was added to say so: a categorical angular axis wraps, and the mark takes that from the space it is drawn in. Full treatment in [Polar](#).

A category on the measure axis is refused, because a line traces a quantity and category names are not a quantity:

```
render_svg(data(gapminder_2007) + line * mean + x(life) + y(continent))
```

Error:

! gog: ``y(continent)`` maps a categorical (text) column, but ``y`` on ``line`` needs a continuous (numeric) column. On these marks ``x`` is the domain and ``y`` the measure, and a category is not a quantity to measure: a mean of category names is not a number, and a region has no categorical baseline to close on. Put the category on ``x`` instead - ``line * mean + x(<category>) + y(<number>)`` is the profile plot, and ``area * mean`` fills it. Unlike ``bar`/`box`/`interval``, these marks do not read their orientation off the bindings, because their two axes do not have the same role.

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

9.6 Smoothed trend line

Combine `line` with the `smooth` transform to draw a fitted trend instead of raw data:

R

```
data(gapminder_2007) + x(gdp) + y(life) +
  point + color(continent) +
  line * smooth
```

Python

```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +
  point + color(col.continent) +
  line * smooth)
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + point + color(:continent) +
  line * smooth
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), point,
  color(col.continent), layer(line, smooth))
```



“Given `gapminder 2007`: *x* is `gdp`, *y* is `life`, points, color by continent, and also a line derived by `smooth`.”

The point layer keeps every country visible, and `smooth` reduces the same rows to one fitted curve. Only the `line` layer carries the transform, so the points are untouched.

9.7 Two boundaries from a pair

Most transforms give a `line` one value per *x*. A **pair** transform (`range`, `confidence`, `deviation`, or `bounds`) gives it two (a low and a high), and a `line` then draws *both boundaries* as two separate lines rather than connecting them into one zigzag. It is the *unfilled* counterpart to a `ribbon` filling the same pair. Here are the lowest and highest life expectancies among five countries, drawn as two lines:

R

```
data(gapminder_asia) + line * range + x(year) + y(life) +  
  y_label("Life expectancy") +  
  title("The lowest and highest life expectancy, as two lines")
```

Python

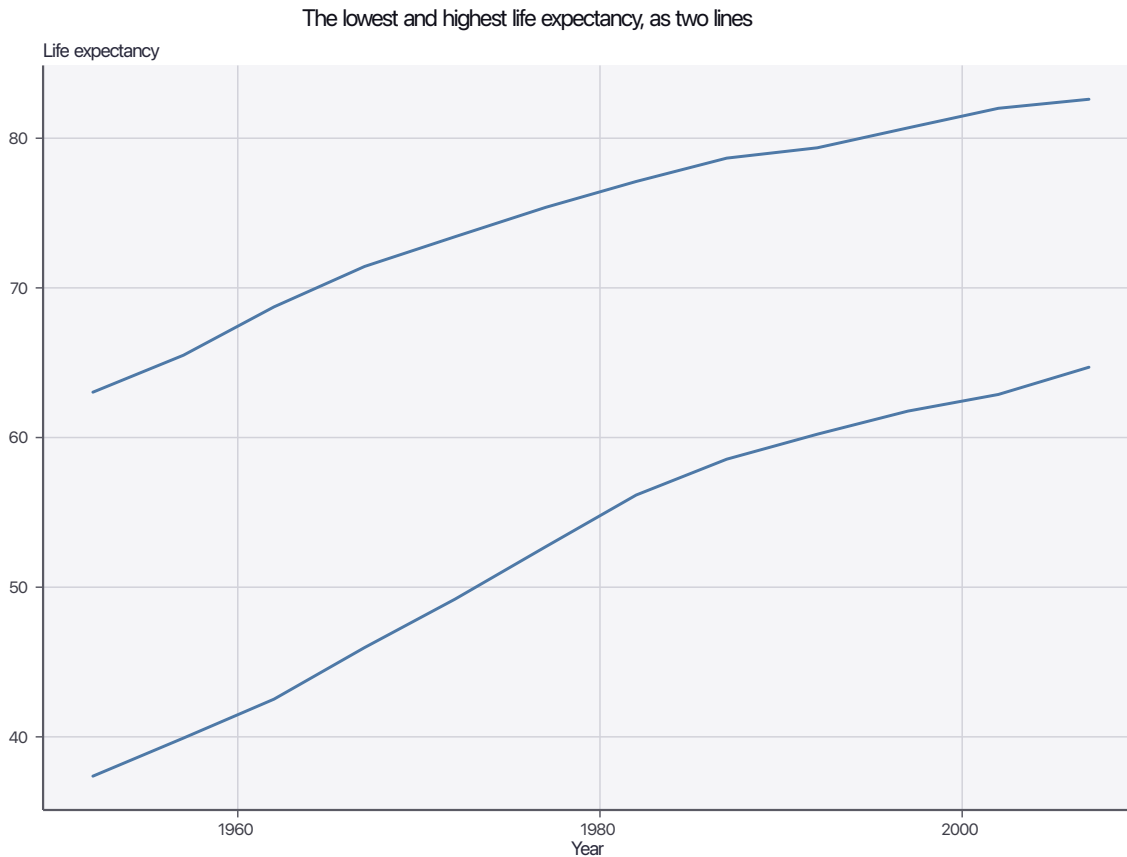
```
(data(gapminder_asia) + line * range + x(col.year) + y(col.life) +  
  y_label("Life expectancy") +  
  title("The lowest and highest life expectancy, as two lines"))
```

Julia

```
data(gapminder_asia) + line * range + x(:year) + y(:life) +  
  y_label("Life expectancy") +  
  title("The lowest and highest life expectancy, as two lines")
```

JavaScript

```
plot(data(gapminder_asia), layer(line, range), x(col.year), y(col.life),  
  y_label("Life expectancy"),  
  title("The lowest and highest life expectancy, as two lines"))
```



“Given gapminder Asia: a line derived by range, x is year, y is life.”

Add `style(pattern = "dashed")` for the dashed band that publications often draw. [Ribbon](#) shows the filled and unfilled versions next to each other.

9.8 Stroke width and opacity

A polyline is drawn with a *single* stroke, so `size` and `opacity` are not channels here: there is nothing per row for them to vary along. That one stroke still has a width and an opacity, and `style()` sets them:

R

```
data(gapminder_asia) + line + x(year) + y(life) + group(country) +
  style(size = 4, opacity = 0.45) +
  title("One width, one opacity, for the whole layer")
```

Python

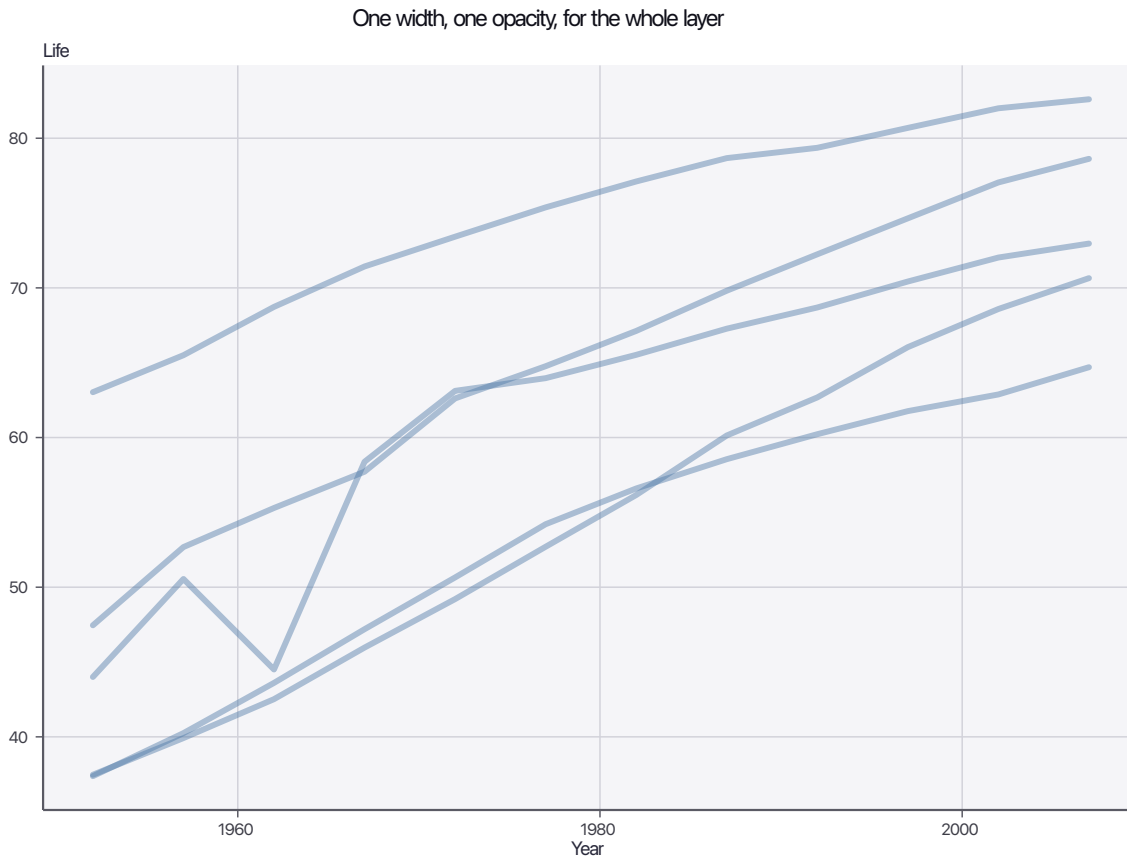
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + group(col.country)
↪ +
  style(size = 4, opacity = 0.45) +
  title("One width, one opacity, for the whole layer"))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country) +
  style(size = 4, opacity = 0.45) +
  title("One width, one opacity, for the whole layer")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),
  group(col.country), style({ size: 4, opacity: 0.45 }),
  title("One width, one opacity, for the whole layer"))
```

“Given gapminder Asia: lines, x is year, y is life, grouped by country, with size 4 and opacity 0.45.”

Two questions look alike and are not. “Can a column be mapped here?” and “Does this mark have this property?” get different answers on a `line`, which is why [Setting vs mapping](#) uses it as its clearest case.

Setting one pale color is the usual way to keep a layer behind the one you want read:

R

```
data(gapminder_asia) + x(year) + y(life) +
  line + group(country) + style(color = "lightgray") +
  line * smooth + style(color = "firebrick", size = 3) +
  title("Every country in gray, the trend in red")
```

Python

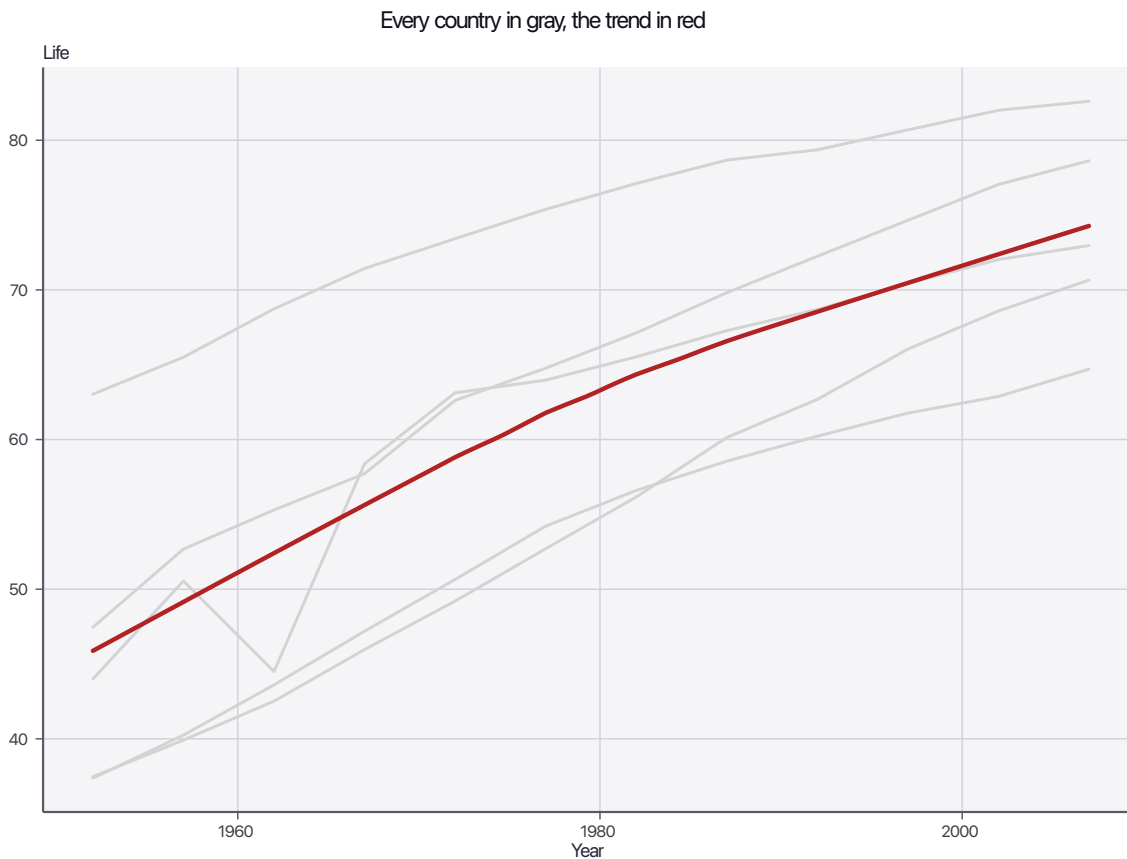
```
(data(gapminder_asia) + x(col.year) + y(col.life) +
  line + group(col.country) + style(color = "lightgray") +
  line * smooth + style(color = "firebrick", size = 3) +
  title("Every country in gray, the trend in red"))
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + line + group(:country) +
  style(color = "lightgray") + line * smooth +
  style(color = "firebrick", size = 3) +
  title("Every country in gray, the trend in red")
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), line,
  group(col.country), style({ color: "lightgray" }), layer(line, smooth),
  style({ color: "firebrick", size: 3 }),
  title("Every country in gray, the trend in red"))
```



`style()` scopes like a channel: it attaches to the mark it is written after. Unlike a channel it has no plot-scoped form. Writing it before any mark is an error, because a plot has no appearance of its own to style.

9.9 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(pattern =)</code>	solid, dashed, dotted

And these vary per row if you map them to a column instead: `color()` (either), `pattern()` (categories), `group()` (categories), `play()` (either).

A line is a stroke, so `size` is its width and `pattern` its dash. Neither varies *along* one line, because a width and a dash belong to the whole stroke. A mapped `pattern` still gives each series its own dash, and `size` is a setting here even though it maps per row on `point`. `color` is the one that *can* vary along the stroke, because a color belongs to a place on the stroke rather than to the whole stroke. The last section of this chapter shows that case.

R

```
data(gapminder_asia) + line + x(year) + y(life) + color(country) +  
  style(size = 3, pattern = "dashed") +  
  y_label("Life expectancy") + title("One width and one dash, for every  
↪ series")
```

Python

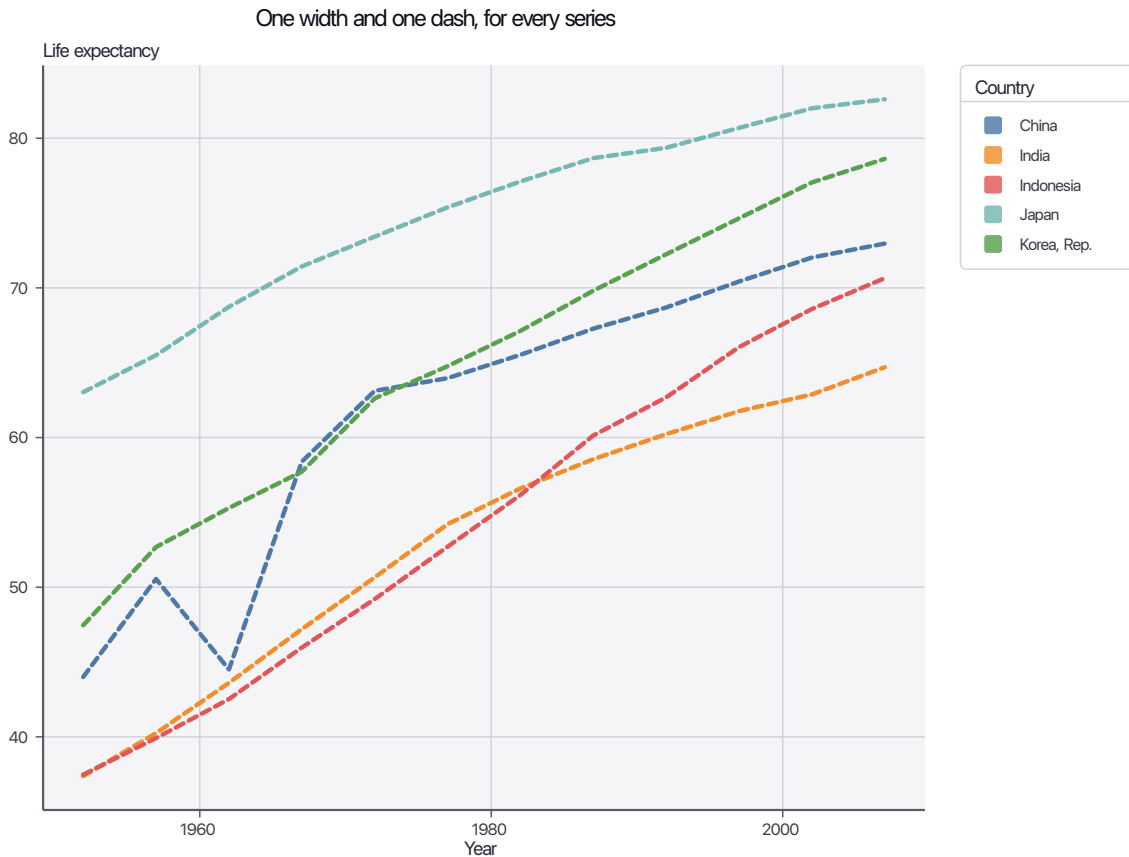
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + color(col.country)  
↪ +  
  style(size = 3, pattern = "dashed") +  
  y_label("Life expectancy") + title("One width and one dash, for every  
↪ series"))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + color(:country) +  
  style(size = 3, pattern = "dashed") + y_label("Life expectancy") +  
  title("One width and one dash, for every series")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
  color(col.country), style({ size: 3, pattern: "dashed" }),  
  y_label("Life expectancy"),  
  title("One width and one dash, for every series"))
```



“Given gapminder Asia: lines, x is year, y is life, color by country, with size 3 and pattern dashed.”

`color` is mapped there, so it is not also set. A layer that both maps and sets the same feature is refused, because one of the two would have to be dropped in silence.

The transforms a line takes carry their own parameters, and those give you the rest of the control. `line * density(adjust = 1.5)` makes the density curve smoother, and `line * bounds(lower, upper)` traces a band’s two edges.

9.10 What it refuses

A dash is the only texture a stroke can carry. A hatch fills an area, and a line has no area to fill.

```
data(gapminder_asia) + line + x(year) + y(life) + style(pattern = "hatch")
```

Error:

```
! gog: `style(pattern = "hatch")` is not a stroke pattern. Use "solid" (the  
↪ default), "dashed", or "dotted".
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The refusal lists the three values a stroke's pattern does take. The two other refusals this chapter shows appeared in earlier sections. One is shown above: a category on the measure axis, which a line cannot trace. The other is stated under Stroke width and opacity: a mapped width or opacity, which one stroke cannot vary row by row even though it can be set. That second pair is the worked example in [Setting vs mapping](#).

color is deliberately *not* in that pair, and the difference is worth stating exactly, because it is the reason none of this is arbitrary. A width and an opacity are properties of the stroke as a whole (there is no answer to “how wide is the line *here*”). A color has an answer everywhere: point at a place on the line and it has one. So a measure can be mapped to color and vary along the stroke, and it earns a gradient legend rather than one color per series:

R

```
data(gapminder_asia) + line + x(year) + y(life) + group(country) +  
  color(gdp, scale = "log") + palette("viridis") +  
  y_label("Life expectancy") + title("Each country's line, colored by its  
↪ income")
```

Python

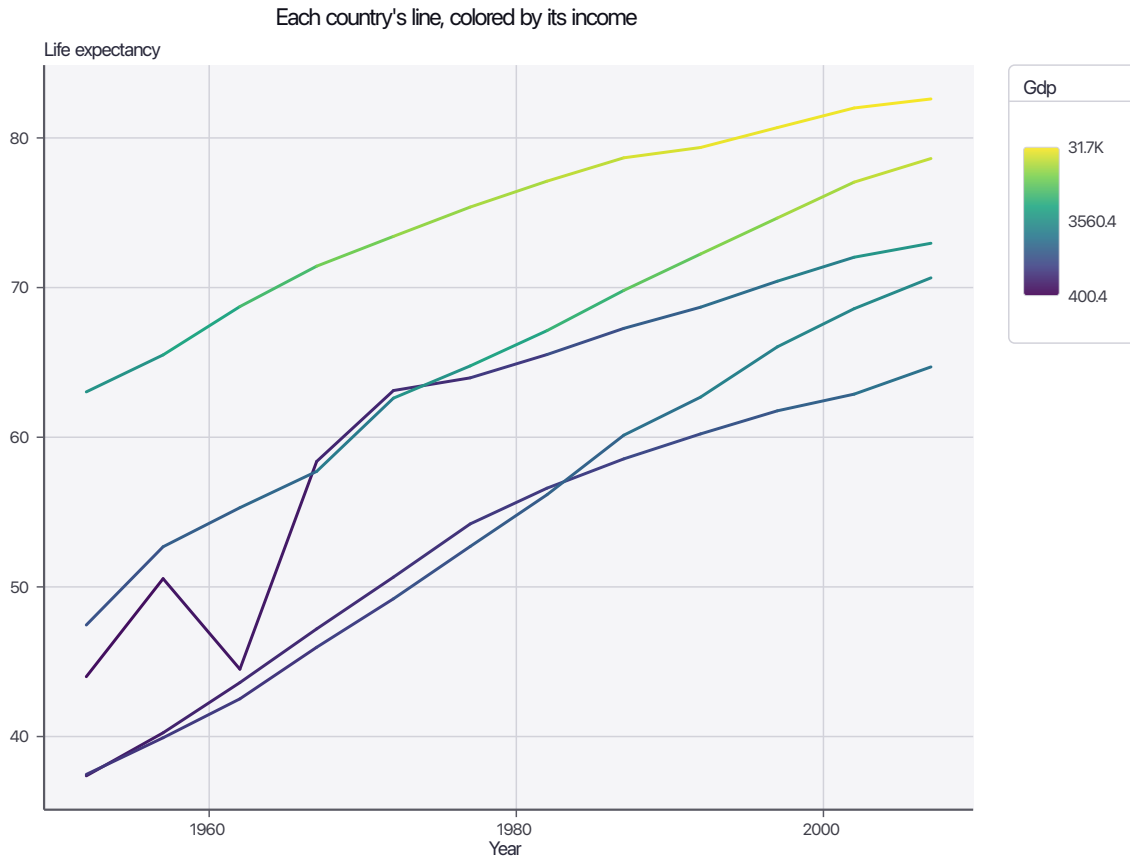
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + group(col.country)  
↪ +  
  color(col.gdp, scale = "log") + palette("viridis") +  
  y_label("Life expectancy") + title("Each country's line, colored by its  
↪ income"))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country) +  
  color(:gdp, scale = "log") + palette("viridis") +  
  y_label("Life expectancy") +  
  title("Each country's line, colored by its income")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),
     group(col.country), color(col.gdp, { scale: "log" }),
     palette("viridis"), y_label("Life expectancy"),
     title("Each country's line, colored by its income"))
```



“Given gapminder Asia: lines, x is year, y is life, grouped by country, color by gdp on a log scale, with the viridis palette.”

Read one country’s line from left to right and you see two variables. The height is how long people live, and the color is how rich the country was then. **group** still splits the mark into one line per country; **color** answers the other question. A *categorical color* splits and colors at the same time, which is what Multiple series showed above.

10 Bar

How large is each category? **bar** draws a rectangle from the baseline (zero) to the value for each row. Bars stand up when the categories are on **x** and lie down when they are on **y**; see [Horizontal bars](#).

The bar chart exists because William Playfair had too little data. His *Commercial and Political Atlas* of 1786 drew England's imports and exports as two curves across thirty years (Playfair, 1786). For Scotland he had a single year, and no curve can show one year. So he drew a bar for each of Scotland's trading partners instead, and wrote that the result was the weaker picture for having no time in it. The form he apologized for became the one everyone knows.

Two names for it are still in use, and they name opposite orientations. Much of the business world, Excel included, calls this a **column chart** when the bars stand up, and keeps **bar chart** for when they lie down. **gog** has one **bar**, and its direction comes from the sentence rather than from a choice. The categorical axis settles it, which is what [Horizontal bars](#) is about. Two names for one mark would be an exception, and **gog**'s rule against exceptions is what refuses it. A **histogram** is a third name, and it is a genuinely different plot drawn with the same mark. Karl Pearson coined the word in the 1890s (Pearson, 1895) for the picture of a *distribution*, where the bars are cut from a continuous axis and touch; that is **bar * bin**, in [Histogram](#) below.

10.1 Basic usage

R

```
data(medals) + bar + x(country) + y(gold)
```

Python

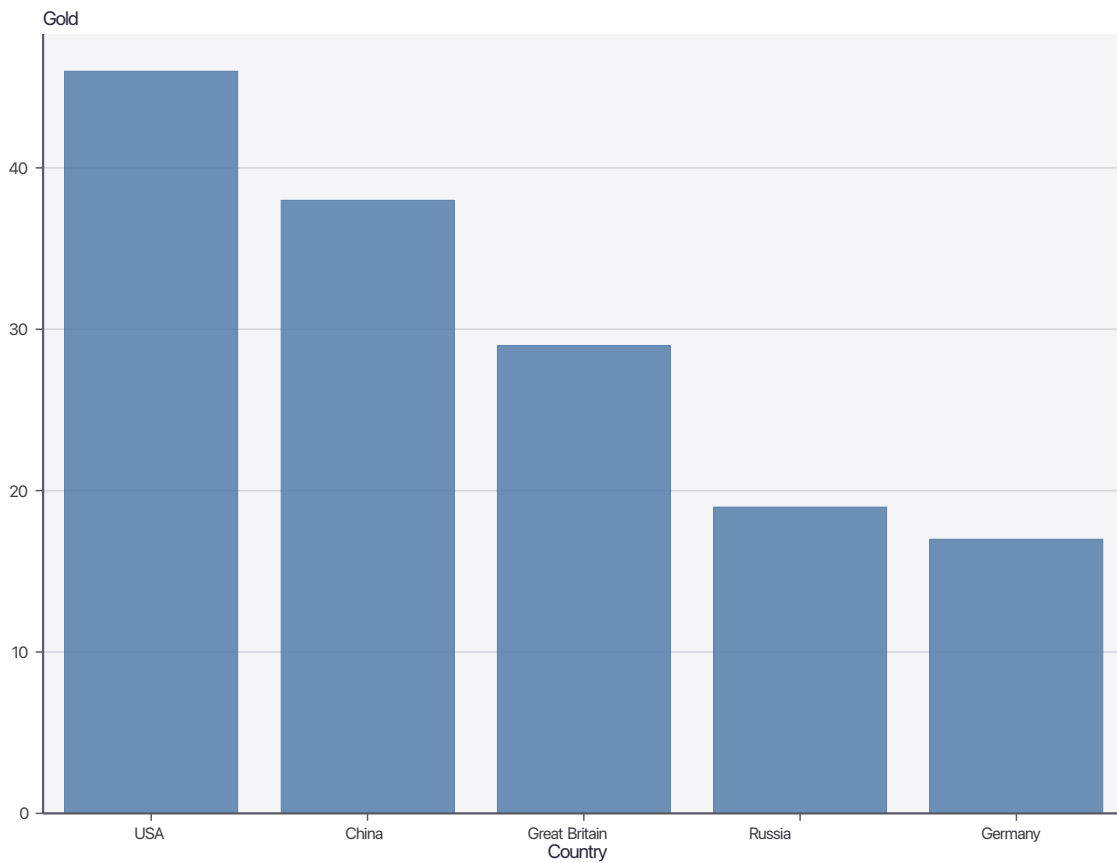
```
data(medals) + bar + x(col.country) + y(col.gold)
```

Julia


```
data(medals) + bar + x(:country) + y(:gold)
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold))
```



“Given the medals: bars, x is country, y is gold.”

The x axis places a tick at each bar position automatically. The y axis reaches zero unless you state your own `limits`, so a bar’s length is its value.

10.2 Color

`color` fills each bar with a distinct color:

R

```
data(medals) + bar + x(country) + y(gold) +  
  color(country)
```

Python

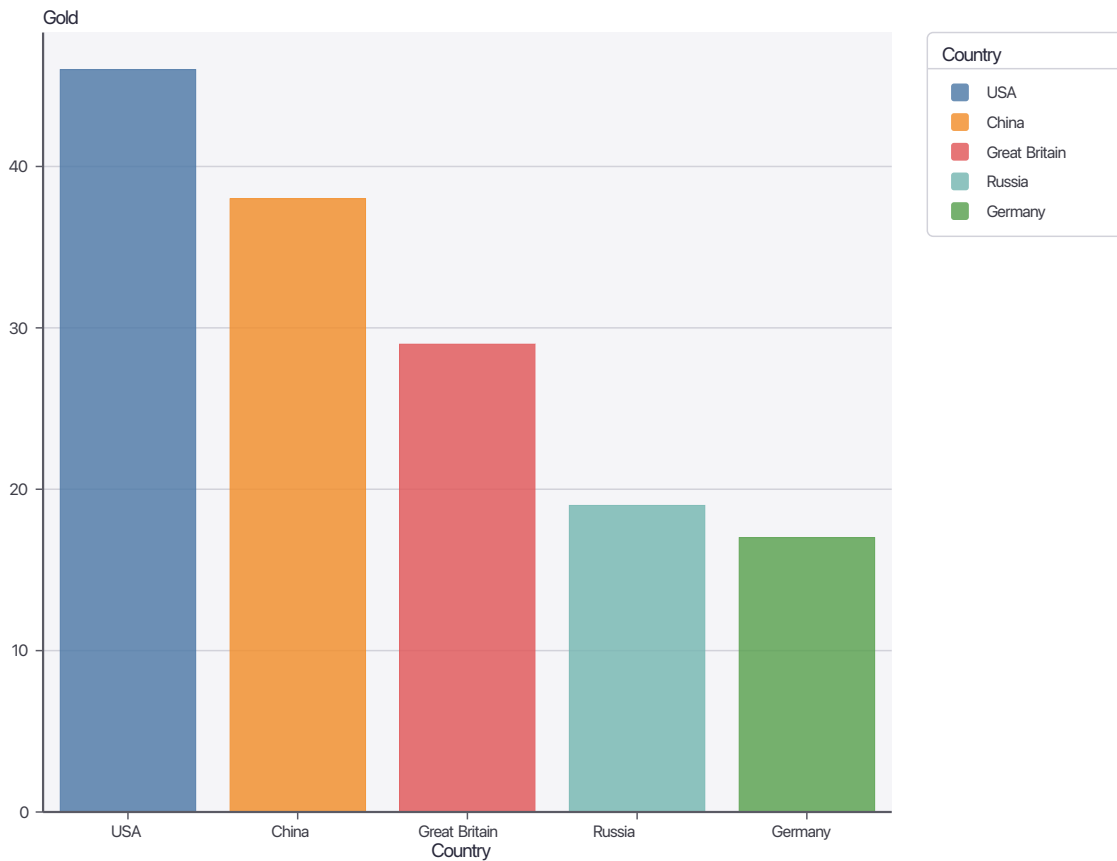
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  color(col.country))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) + color(:country)
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold), color(col.country))
```



“Given the medals: bars, x is country, y is gold, color by country.”

10.3 Ordering bars

`order()` controls the sequence of categories along whichever axis carries them: `x` on a vertical plot, `y` on a horizontal one. You can sort on any column in the table: the categorical column itself, the measured column, or a third column entirely. Naming the categorical column asks for that column’s own order. Here that is alphabetical, because `country` is plain text. A column with a [declared order](#) uses that order instead.

Default, data order (rows as they appear in the table):

R

```
data(medals) + bar + x(country) + y(gold) +
  title("Gold medals: data order")
```

Python

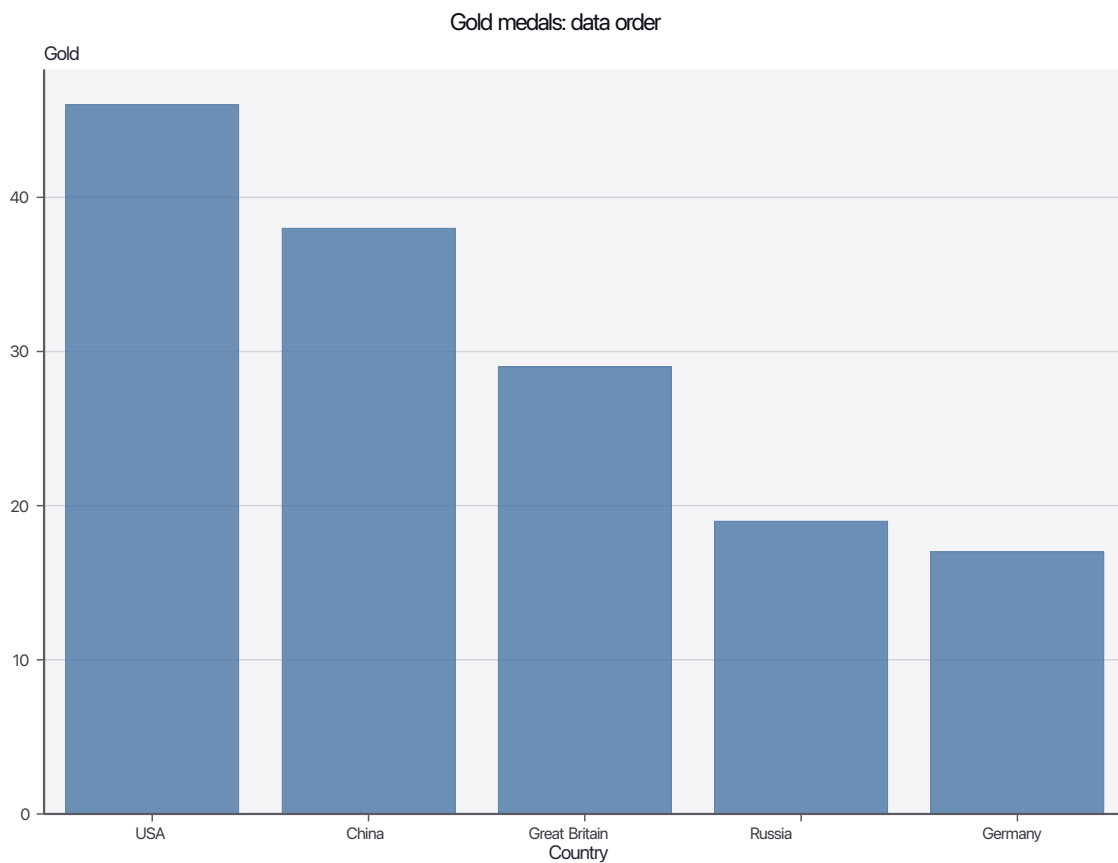
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  title("Gold medals: data order"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) +  
  title("Gold medals: data order")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),  
  title("Gold medals: data order"))
```



By value, descending, tallest bar first, the most common pattern for ranked comparisons:

R

```
data(medals) + bar + x(country) + y(gold) +  
  order(gold, desc = TRUE) +  
  title("Gold medals: largest first")
```

Python

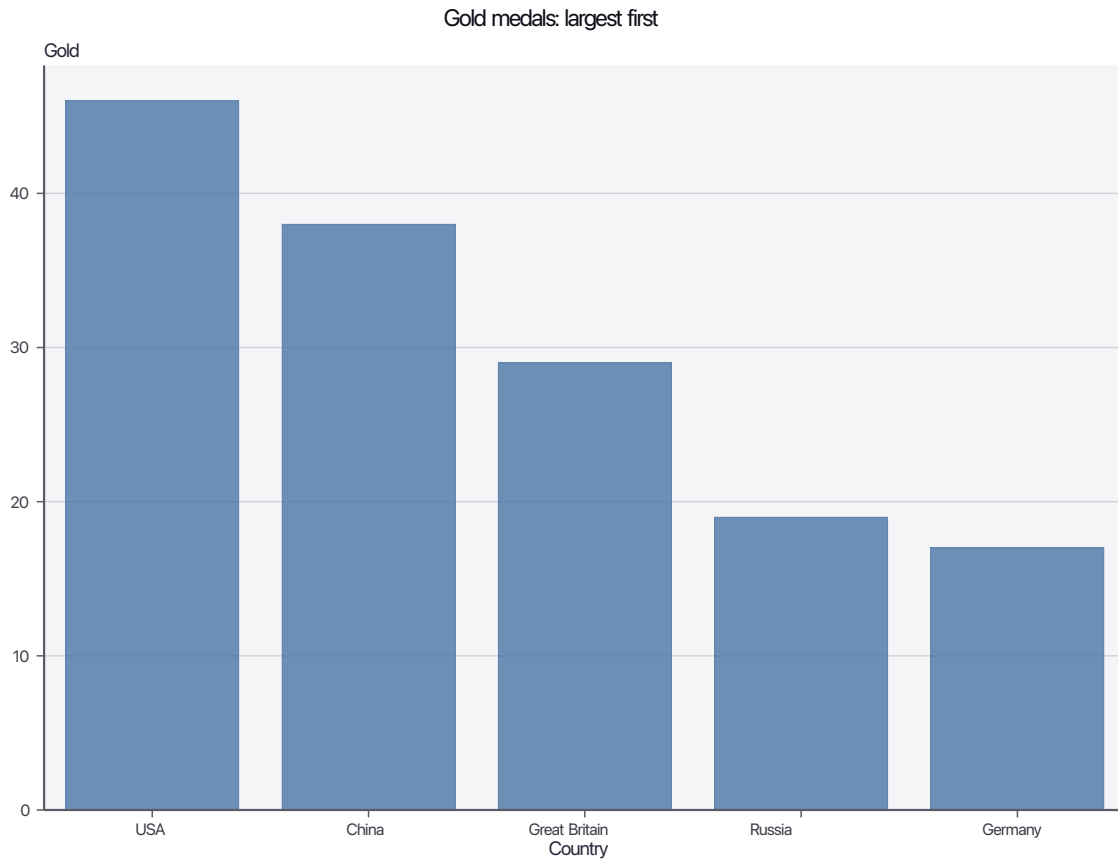
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  order(col.gold, desc = True) +  
  title("Gold medals: largest first"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) + order(:gold, desc = true) +  
  title("Gold medals: largest first")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),  
  order(col.gold, { desc: true }), title("Gold medals: largest first"))
```



“Given the medals: bars, x is country, y is gold, ordered by gold, largest first.”

By value, ascending, smallest first, useful when scanning for the lowest values:

R

```
data(medals) + bar + x(country) + y(gold) +  
  order(gold) +  
  title("Gold medals: smallest first")
```

Python

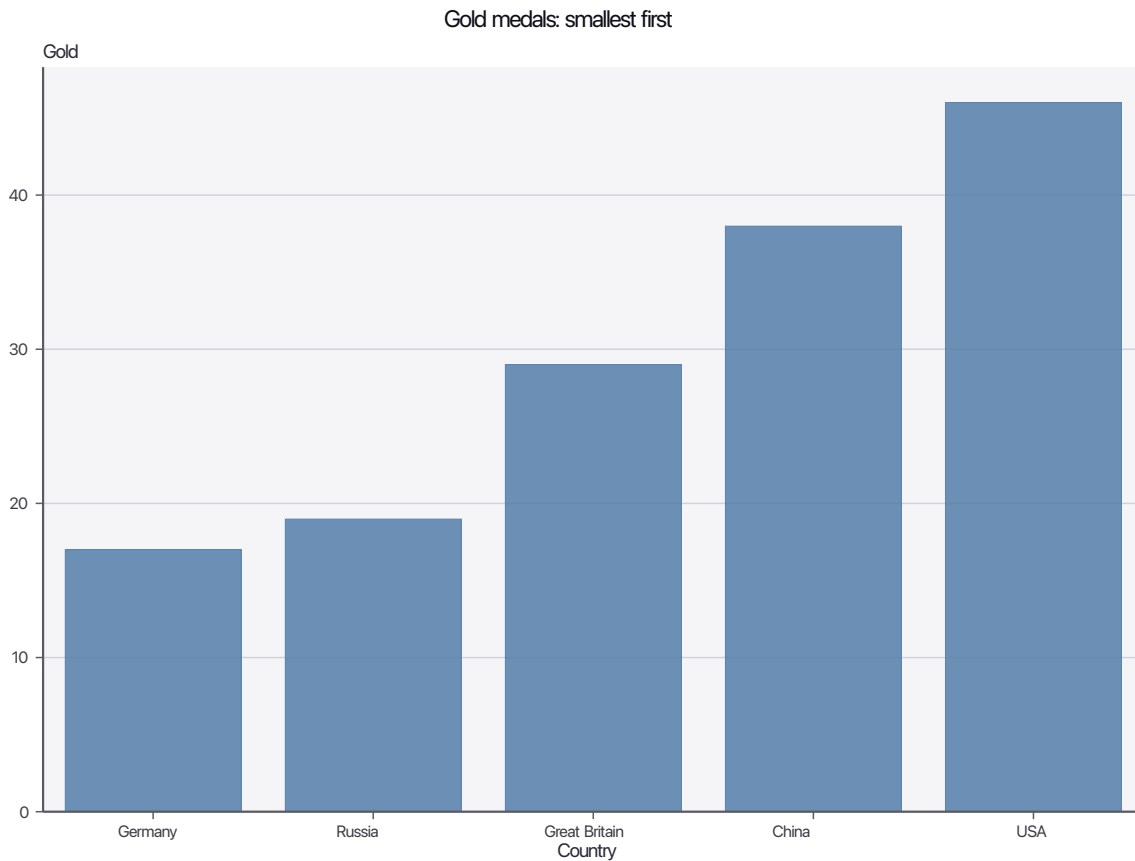
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  order(col.gold) +  
  title("Gold medals: smallest first"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) + order(:gold) +
  title("Gold medals: smallest first")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold), order(col.gold),
  title("Gold medals: smallest first"))
```



By category name, $A \rightarrow Z$, natural for names and labels:

R

```
data(medals) + bar + x(country) + y(gold) +
  order(country) +
  title("Gold medals: A to Z")
```

Python

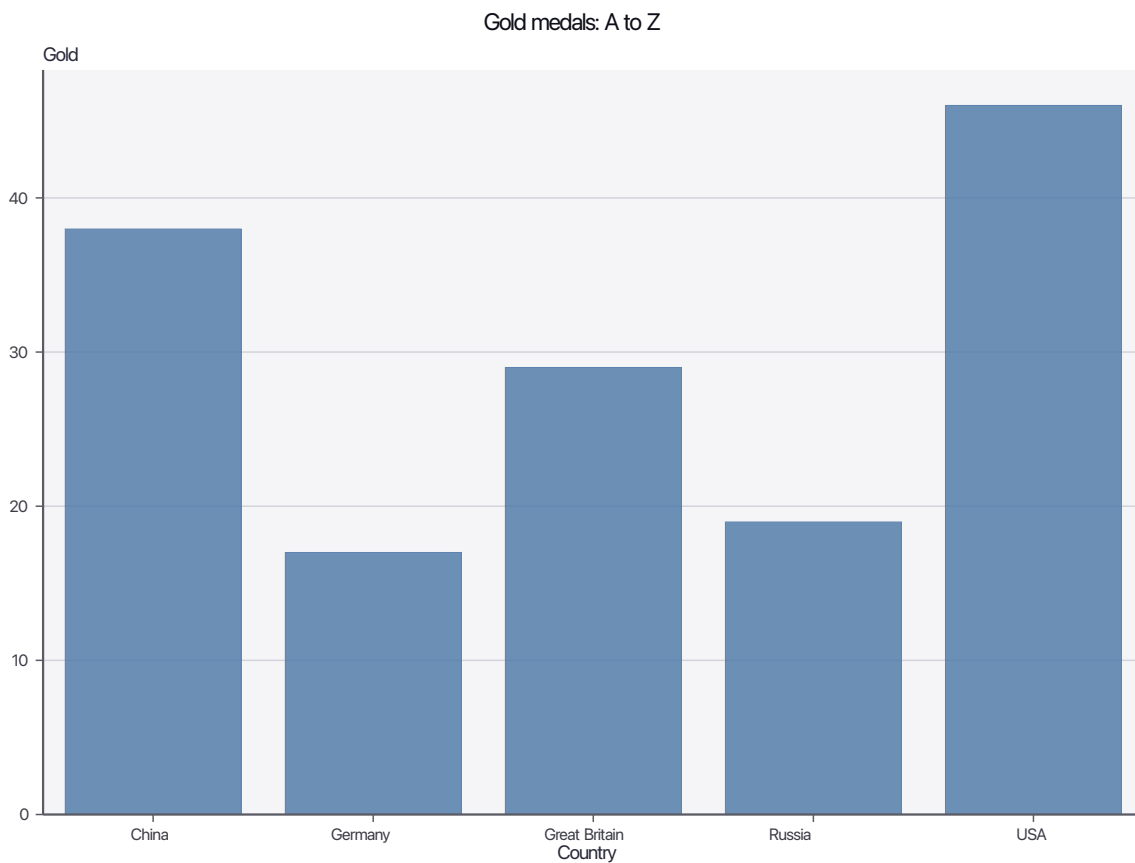
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  order(col.country) +  
  title("Gold medals: A to Z"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) + order(:country) +  
  title("Gold medals: A to Z")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold), order(col.country),  
  title("Gold medals: A to Z"))
```



By category name, $Z \rightarrow A$, the reverse:

R

```
data(medals) + bar + x(country) + y(gold) +  
  order(country, desc = TRUE) +  
  title("Gold medals: Z to A")
```

Python

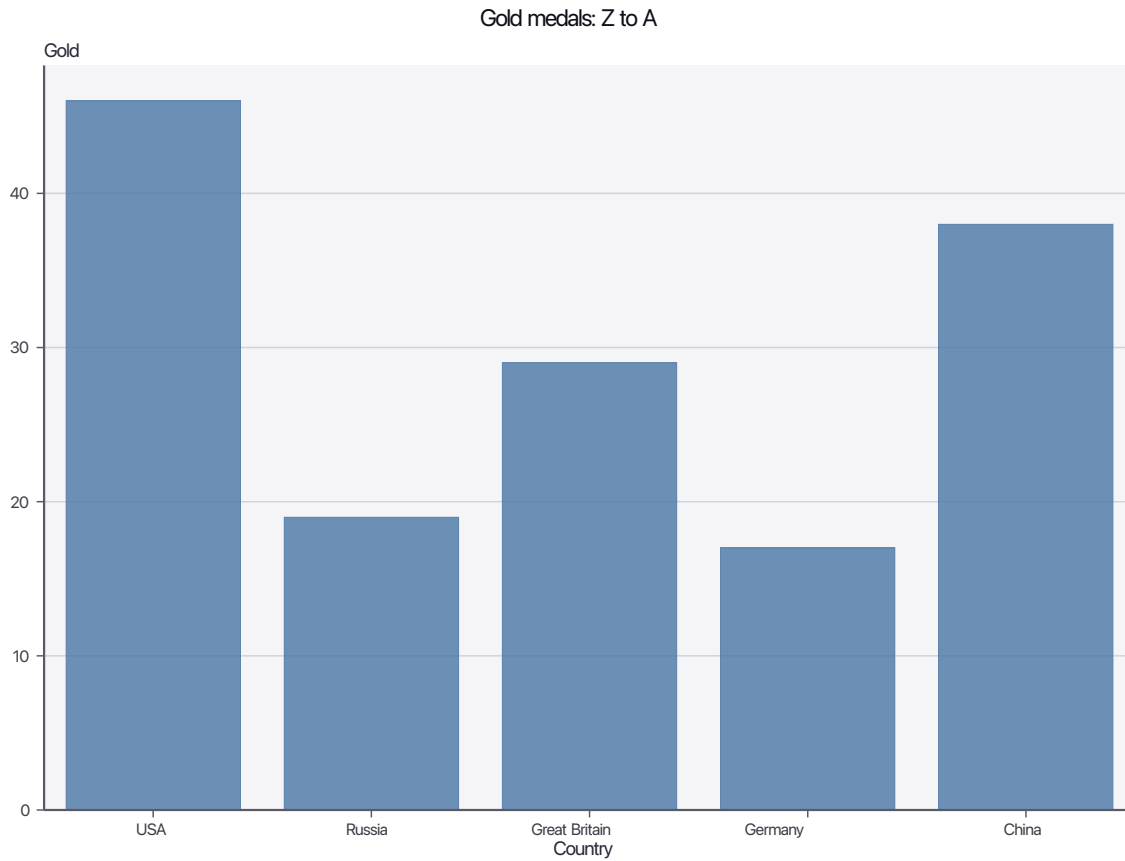
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  order(col.country, desc = True) +  
  title("Gold medals: Z to A"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) +  
  order(:country, desc = true) + title("Gold medals: Z to A")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),  
  order(col.country, { desc: true }), title("Gold medals: Z to A"))
```



By a **third column**, order countries by their silver medal count, while plotting gold:

R

```
data(medals) + bar + x(country) + y(gold) +
  order(silver, desc = TRUE) +
  title("Gold medals: ordered by silver")
```

Python

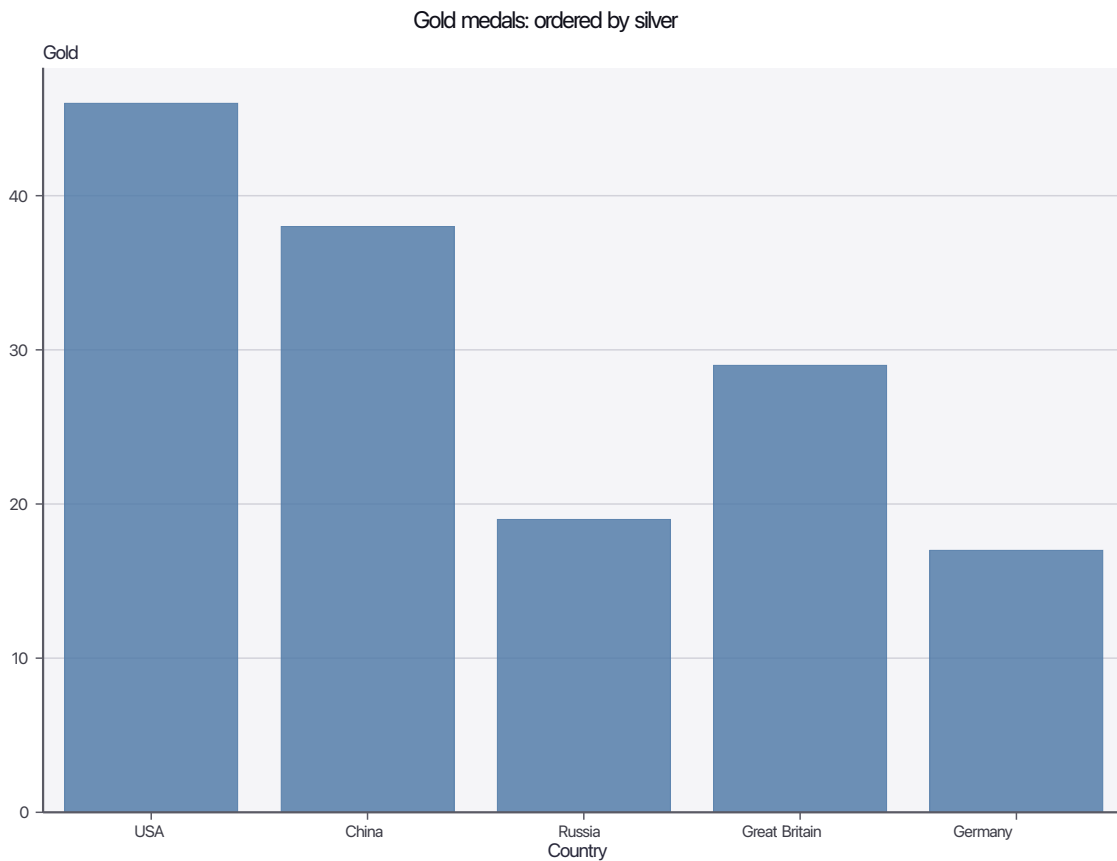
```
(data(medals) + bar + x(col.country) + y(col.gold) +
  order(col.silver, desc = True) +
  title("Gold medals: ordered by silver"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) +
  order(:silver, desc = true) + title("Gold medals: ordered by silver")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),
  order(col.silver, { desc: true }),
  title("Gold medals: ordered by silver"))
```



Expression	Order
<i>(none)</i>	Data order (rows as they appear)
<code>order(gold)</code>	Ascending by gold (smallest first)
<code>order(gold, desc = TRUE)</code>	Descending by gold (largest first)
<code>order(country)</code>	A → Z by country name

Expression	Order
<code>order(country, desc = TRUE)</code>	Z → A by country name
<code>order(silver, desc = TRUE)</code>	By any other column in the table

10.4 Duplicate categories

If the same category appears in multiple rows, `bar` draws one bar per row; the last silently overlaps the earlier ones.

Use the `sum` transform to total the values per category first:

R

```
medals_dup <- data.frame(
  country = c("USA", "USA", "GBR", "GBR", "JPN"),
  gold    = c(10, 5, 8, 4, 6)
)
data(medals_dup) + bar * sum + x(country) + y(gold) +
  order(gold, desc = TRUE) + title("Totals: bar * sum")
```

Python

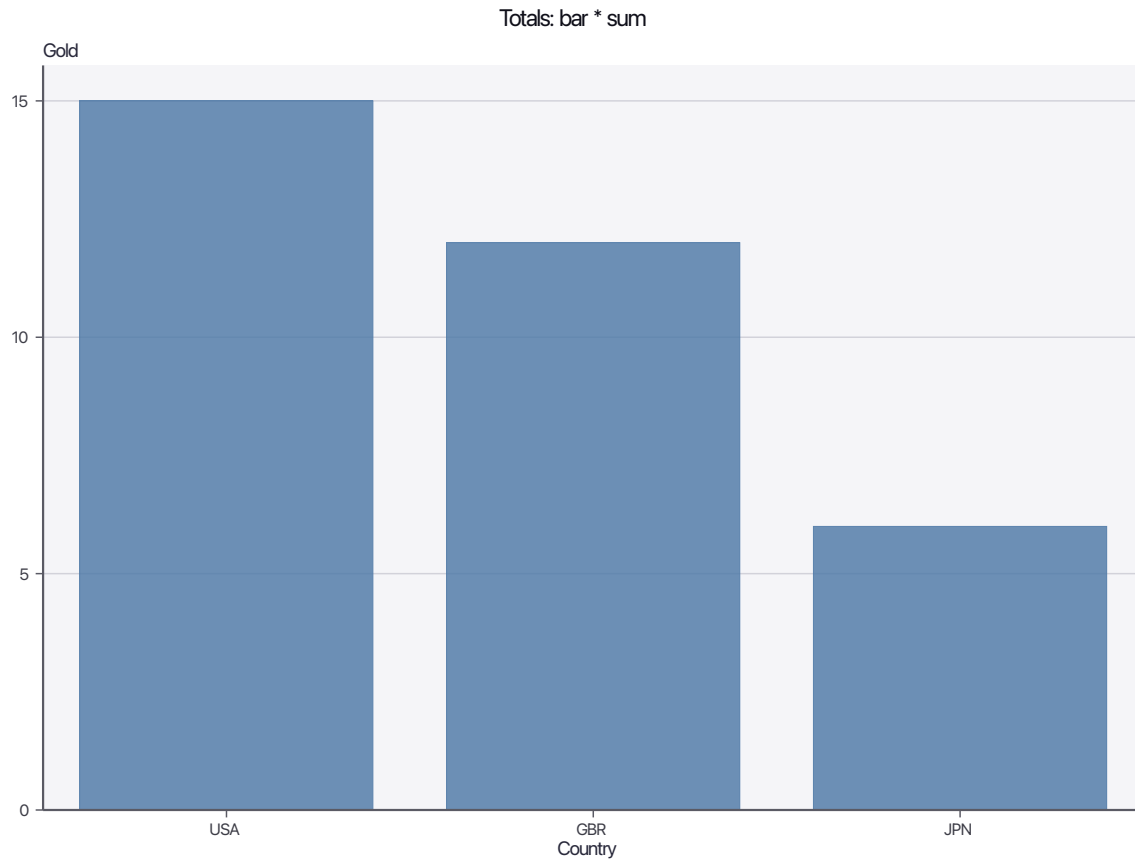
```
medals_dup = {"country": ["USA", "USA", "GBR", "GBR", "JPN"], "gold": [10, 5,
↪ 8, 4, 6]}
(data(medals_dup) + bar * sum + x(col.country) + y(col.gold) +
  order(col.gold, desc = True) + title("Totals: bar * sum"))
```

Julia

```
medals_dup = (country = ["USA", "USA", "GBR", "GBR", "JPN"], gold = [10, 5,
↪ 8, 4, 6],)
data(medals_dup) + bar * sum + x(:country) + y(:gold) +
  order(:gold, desc = true) + title("Totals: bar * sum")
```

JavaScript

```
const medals_dup = { country: ["USA", "USA", "GBR", "GBR", "JPN"], gold: [10,
↪ 5, 8, 4, 6] };
plot(data(medals_dup), layer(bar, sum), x(col.country), y(col.gold),
  order(col.gold, { desc: true }), title("Totals: bar * sum"))
```



“Given the medals dup table: bars derived by sum, x is country, y is gold, ordered by gold, largest first.”

`bar * sum` and `order()` compose freely, because they do different jobs. `sum` totals the values; `order()` sorts the categories those totals belong to.

`bar` handles negative values correctly: the bar grows away from zero in whichever direction the value sits.

R

```
balance <- data.frame(quarter = c("Q1", "Q2", "Q3", "Q4"),
                      profit = c(12.0, -5.0, 8.0, -2.0))
data(balance) + bar + x(quarter) + y(profit) +
  title("Negative values grow downward from zero")
```

Python

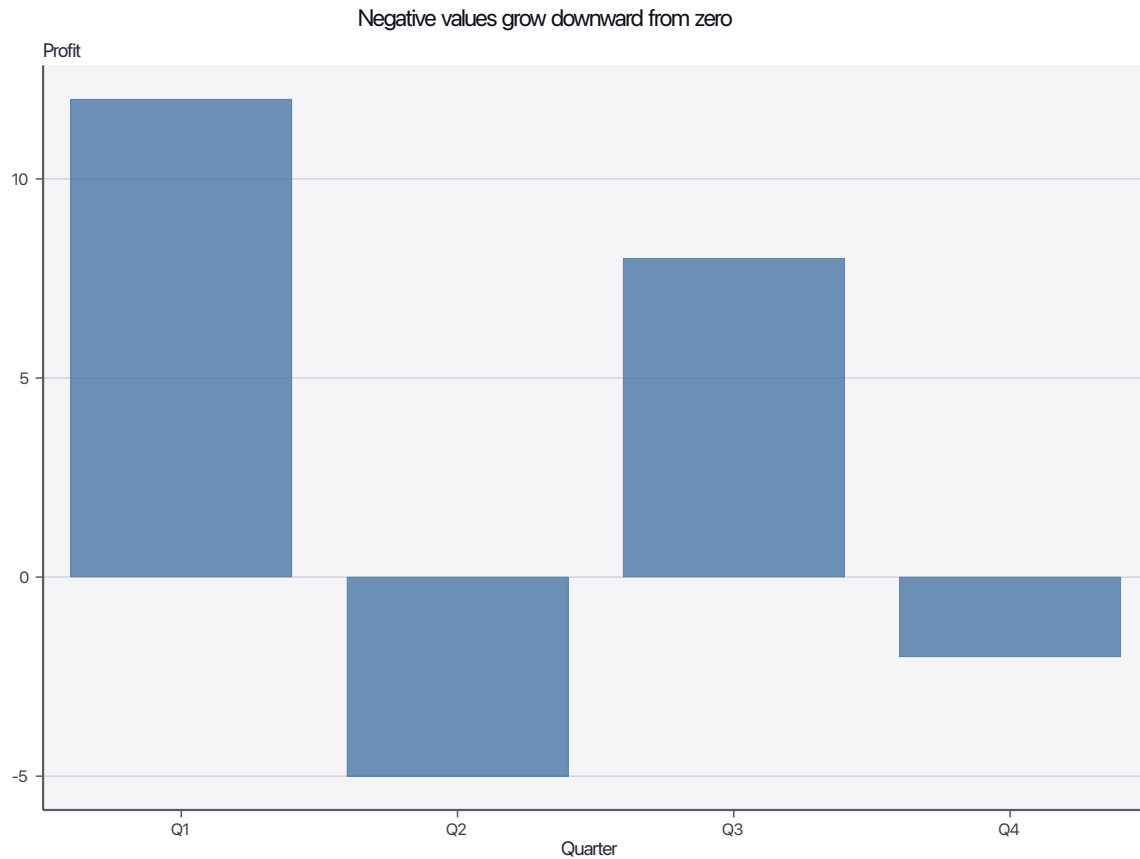
```
balance = {"quarter": ["Q1", "Q2", "Q3", "Q4"], "profit": [12.0, -5.0, 8.0,  
↪ -2.0]}  
(data(balance) + bar + x(col.quarter) + y(col.profit) +  
  title("Negative values grow downward from zero"))
```

Julia

```
balance = (quarter = ["Q1", "Q2", "Q3", "Q4"], profit = [12, -5, 8, -2],)  
data(balance) + bar + x(:quarter) + y(:profit) +  
  title("Negative values grow downward from zero")
```

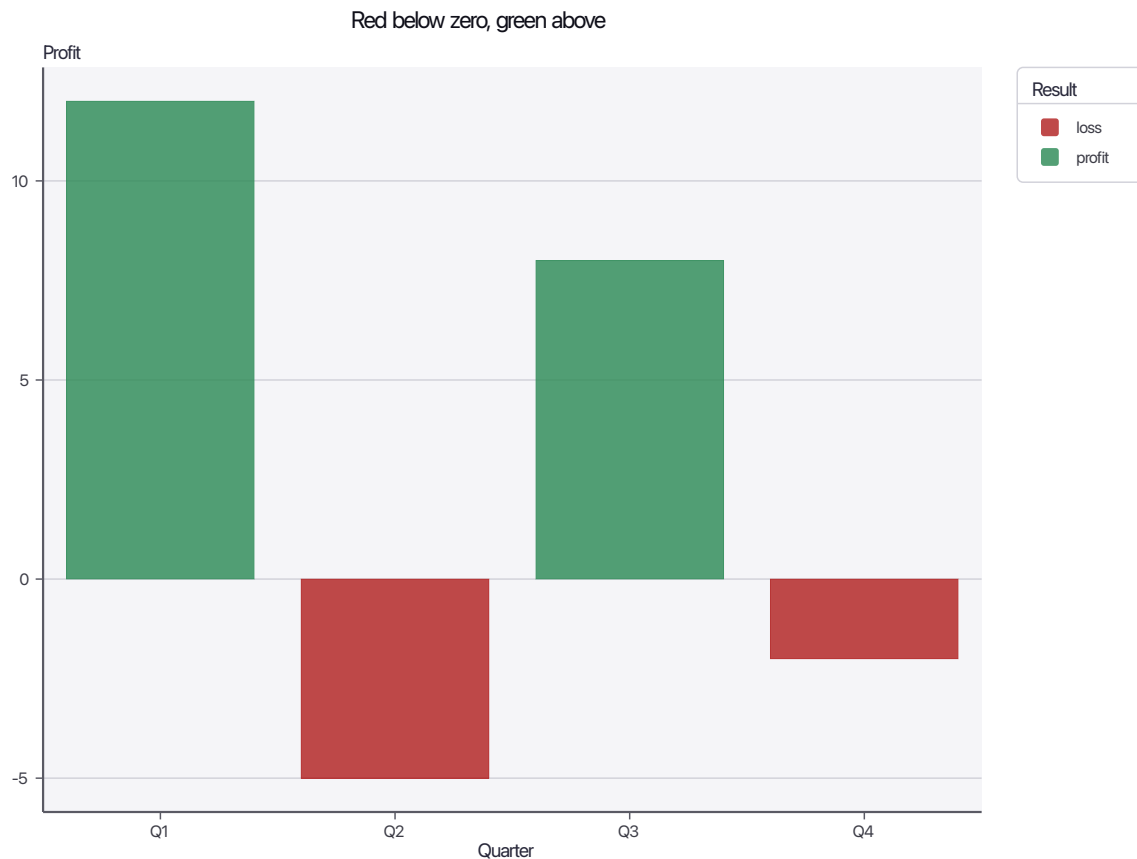
JavaScript

```
const balance = { quarter: ["Q1", "Q2", "Q3", "Q4"], profit: [12, -5, 8, -2]  
↪ };  
plot(data(balance), bar, x(col.quarter), y(col.profit),  
  title("Negative values grow downward from zero"))
```



To color the losses and the gains differently, look at what the request needs: `color` maps a column, so that column has to exist. “Is this negative” is a fact about the data, not about the rectangle. So compute it into a column, map that column, and let `palette()` say which colors the mapping uses:

```
balance$result <- factor(ifelse(balance$profit < 0, "loss", "profit"),
                          levels = c("loss", "profit"))
data(balance) + bar + x(quarter) + y(profit) + color(result) +
  palette(c("firebrick", "seagreen")) +
  title("Red below zero, green above")
```



“Given the balance table: bars, x is quarter, y is profit, color by result, with a palette of firebrick and seagreen.”

`palette()` hands its colors out in **category order**, and that is why `result` carries a declared order here instead of being plain text. A plain text column takes the order its values first appear in. The first row here is a profit, so the palette would give firebrick to the gains. Declaring `levels = c("loss", "profit")` pins the order, the legend reads in it too, and the plot cannot silently invert if next quarter’s numbers start with a loss. This is the same mechanism as [Declaring the order of categories](#).

Red for loss and green for gain is one common convention. It is also the hardest pair to separate for red-green color blindness, the most common kind. Texture solves that and keeps the colors, by giving each result its own hatch as well as its own color: see [Texture](#) below, and [Will it survive a photocopier?](#) in the cookbook.

10.5 Horizontal bars

Swap the bindings. There is no `flip` atom, because that would be a second way to say one thing. Put the categories on `y` and the measure on `x`:

R

```
data(medals) + bar + x(gold) + y(country) +  
  title("Categories on y: the bars lie down")
```

Python

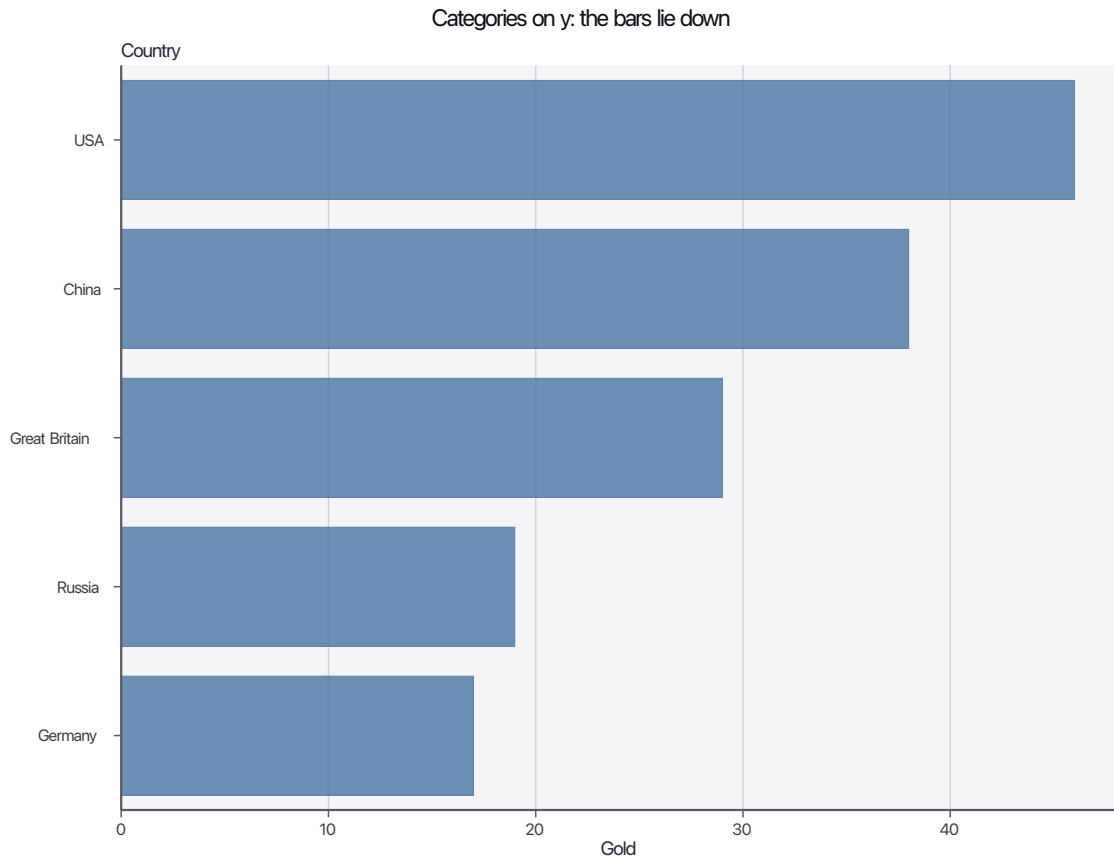
```
(data(medals) + bar + x(col.gold) + y(col.country) +  
  title("Categories on y: the bars lie down"))
```

Julia

```
data(medals) + bar + x(:gold) + y(:country) +  
  title("Categories on y: the bars lie down")
```

JavaScript

```
plot(data(medals), bar, x(col.gold), y(col.country),  
  title("Categories on y: the bars lie down"))
```



`gog` takes the orientation from the bindings: the axis carrying a number is the one the bars measure along, and the other is the one they sit on. Nothing else changes: `order`, transforms, color and `style()` all behave identically.

This is what you want whenever the category names are long, because a vertical axis gives each label a whole line instead of a cramped slot:

R

```
long <- data.frame(
  department = c("Research & Development", "Sales and Marketing",
                 "Customer Operations", "Finance", "People & Culture"),
  headcount  = c(128.0, 96.0, 74.0, 31.0, 22.0)
)
data(long) + bar + x(headcount) + y(department) +
  order(headcount, desc = TRUE) +
  x_label("Headcount") +
  title("Long labels fit on a vertical axis")
```

Python

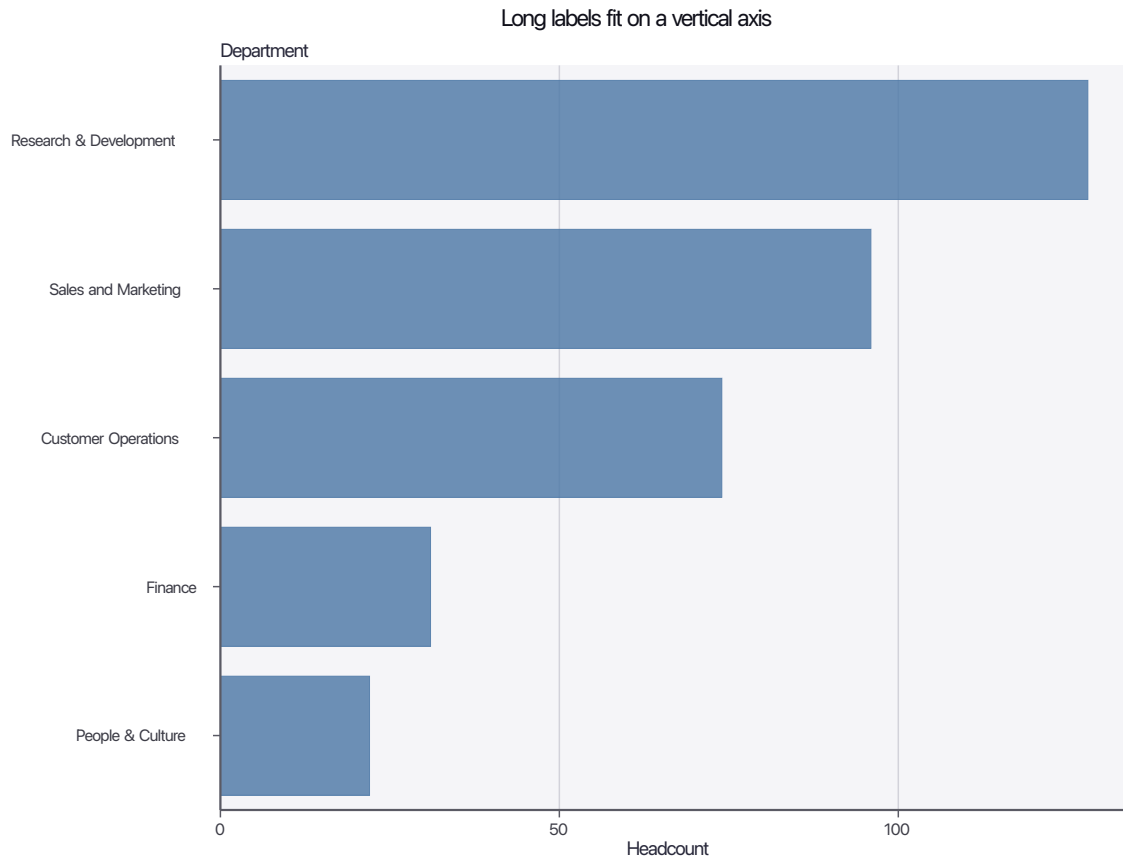
```
long = {"department": ["Research & Development", "Sales and Marketing",  
    "Customer Operations", "Finance", "People & Culture"],  
    ↪ "headcount": [128.0, 96.0, 74.0, 31.0, 22.0]}  
(data(long) + bar + x(col.headcount) + y(col.department) +  
    order(col.headcount, desc = True) +  
    x_label("Headcount") +  
    title("Long labels fit on a vertical axis"))
```

Julia

```
long = (department = ["Research & Development", "Sales and Marketing",  
    ↪ "Customer Operations", "Finance", "People & Culture"], headcount = [128,  
    ↪ 96, 74, 31, 22],)  
data(long) + bar + x(:headcount) + y(:department) +  
    order(:headcount, desc = true) + x_label("Headcount") +  
    title("Long labels fit on a vertical axis")
```

JavaScript

```
const long = { department: ["Research & Development", "Sales and Marketing",  
    ↪ "Customer Operations", "Finance", "People & Culture"], headcount: [128,  
    ↪ 96, 74, 31, 22] };  
plot(data(long), bar, x(col.headcount), y(col.department),  
    order(col.headcount, { desc: true }), x_label("Headcount"),  
    title("Long labels fit on a vertical axis"))
```



order means the same thing in both orientations: the first category in sort order is drawn first. Descending puts the largest bar leftmost on a vertical plot and topmost on a horizontal one.

Orientation is only ambiguous if neither axis is categorical. In that case **gog** draws vertically, so `x(year) + y(sales)` is a column chart. To lay that out horizontally, name the measure on **x**.

A bar needs something to measure, so two categorical axes are refused:

```
data(medals) + bar + x(country) + y(country)
```

Error:

```
! gog: `bar` has categorical columns on both axes - `x(country)` and
  ↳ `y(country)` - so there is nothing for it to measure. One axis must be a
  ↳ number: that is the length of the bar. To count rows per category
  ↳ instead, use `bar * count`.
```

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

10.6 Texture

A bar's fill can carry a **texture** as well as a color: a hatch instead of a solid block of paint, set with `style(pattern = )`. A texture reads without color, so a hatched bar chart stays legible in grayscale or a black-and-white print:

R

```
data(medals) + bar + x(country) + y(gold) +  
  style(pattern = "hatch") +  
  title("A hatched bar reads without color")
```

Python

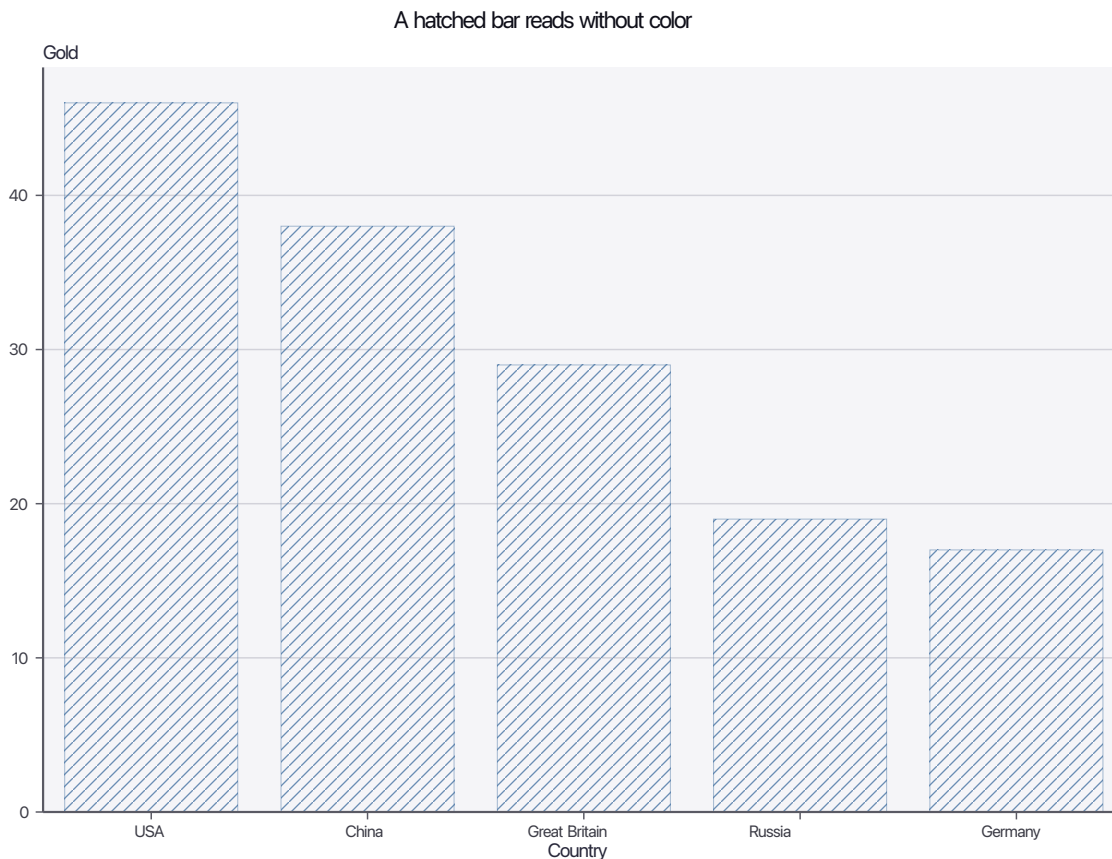
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  style(pattern = "hatch") +  
  title("A hatched bar reads without color"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) + style(pattern = "hatch") +  
  title("A hatched bar reads without color")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),  
  style({ pattern: "hatch" }), title("A hatched bar reads without color"))
```



“Given the medals: bars, x is country, y is gold, with pattern hatch.”

The five fill textures are "solid" (the default), "hatch", "crosshatch", "grid", and "dots"; the [style chapter](#) shows them beside the dashes a stroke takes, because `style(pattern = )` is one name with one meaning per shape. One texture covers the whole layer; to give each color category its *own* hatch, map a column with the [pattern\(\) channel](#), which does for texture what `shape()` does for glyphs.

10.7 Histogram

Combine `bar` with the `bin` transform to create a histogram. `bin` groups continuous x values into equal-width bins and counts the rows in each:

R

```
data(gapminder_2007) + bar * bin + x(life) + y(count) +  
  x_label("Life expectancy (years)") + y_label("Count") +  
  title("Histogram of life expectancy, 2007")
```

Python

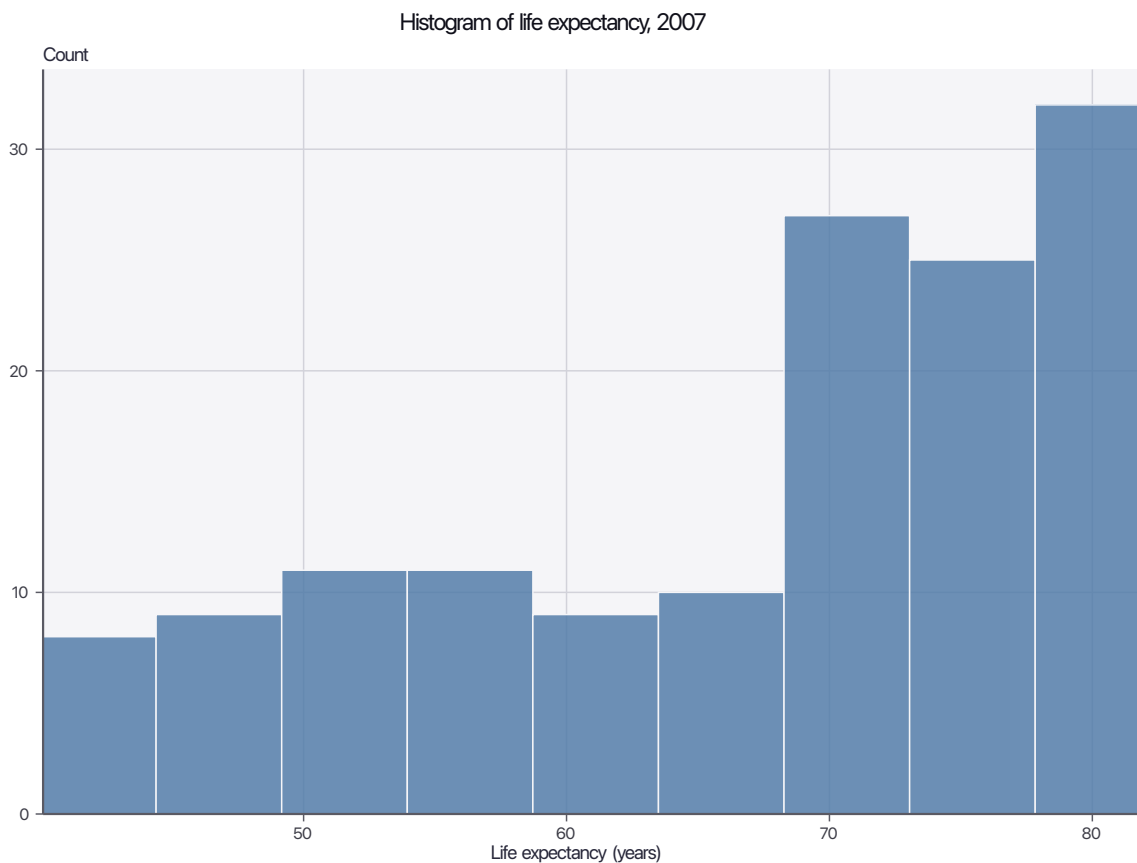
```
(data(gapminder_2007) + bar * bin + x(col.life) + y(col.count) +  
  x_label("Life expectancy (years)") + y_label("Count") +  
  title("Histogram of life expectancy, 2007"))
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) + y(:count) +  
  x_label("Life expectancy (years)") + y_label("Count") +  
  title("Histogram of life expectancy, 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life), y(col.count),  
  x_label("Life expectancy (years)"), y_label("Count"),  
  title("Histogram of life expectancy, 2007"))
```



“Given gapminder 2007: bars derived by bin, x is life, y is count.”

Thickness is automatic, and it is the clearest way a histogram and a bar chart differ on the page. A **categorical** bar fills 80% of the slot its category owns, and the empty fifth is what reads as “separate categories”. A histogram cuts a continuous axis into adjacent intervals, so nothing sits between one bin and the next and the bars **touch**, with a thin line in the panel’s background color to keep them readable. You set nothing in either plot. The bar chart at the top of this chapter and this histogram both take their thickness from the axis the bars sit on.

A histogram turns on its side by the same rule. Bind the measured column to y and the count is drawn along x:

R

```
data(gapminder_2007) + bar * bin + y(life) +
  y_label("Life expectancy (years)") +
  title("The same histogram, lying down")
```


Python

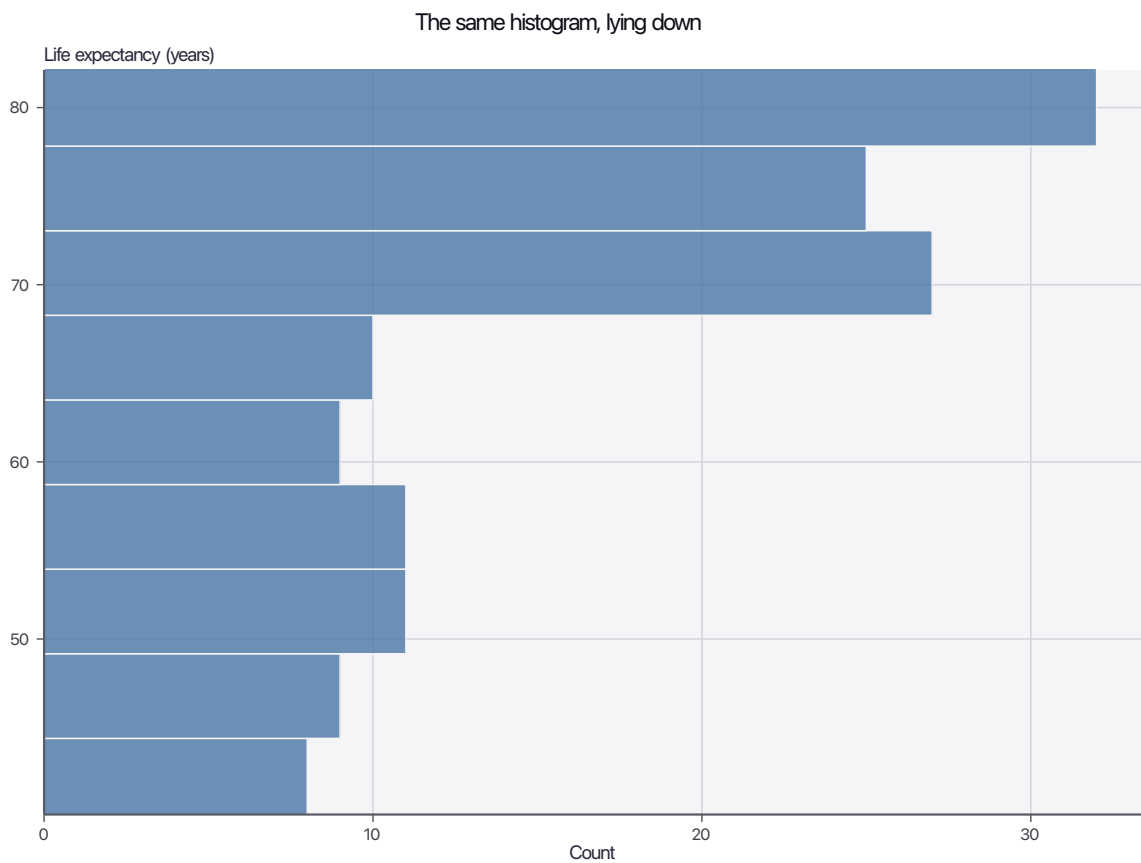
```
(data(gapminder_2007) + bar * bin + y(col.life) +  
  y_label("Life expectancy (years)") +  
  title("The same histogram, lying down"))
```

Julia

```
data(gapminder_2007) + bar * bin + y(:life) +  
  y_label("Life expectancy (years)") +  
  title("The same histogram, lying down")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), y(col.life),  
  y_label("Life expectancy (years)",  
  title("The same histogram, lying down"))
```



10.7.1 Overlaid histograms: one distribution per group

Add `color` and the histogram `splits`: one histogram per category, cut on the same bins and drawn in the same panel. This answers the question a single combined histogram cannot: not “how is petal length distributed?” but “how is it distributed *for each species?*”

R

```
data(iris_flowers) + bar * bin + x(petal_length) + color(species) +  
  x_label("Petal length (cm)") + title("Petal length by species")
```

Python

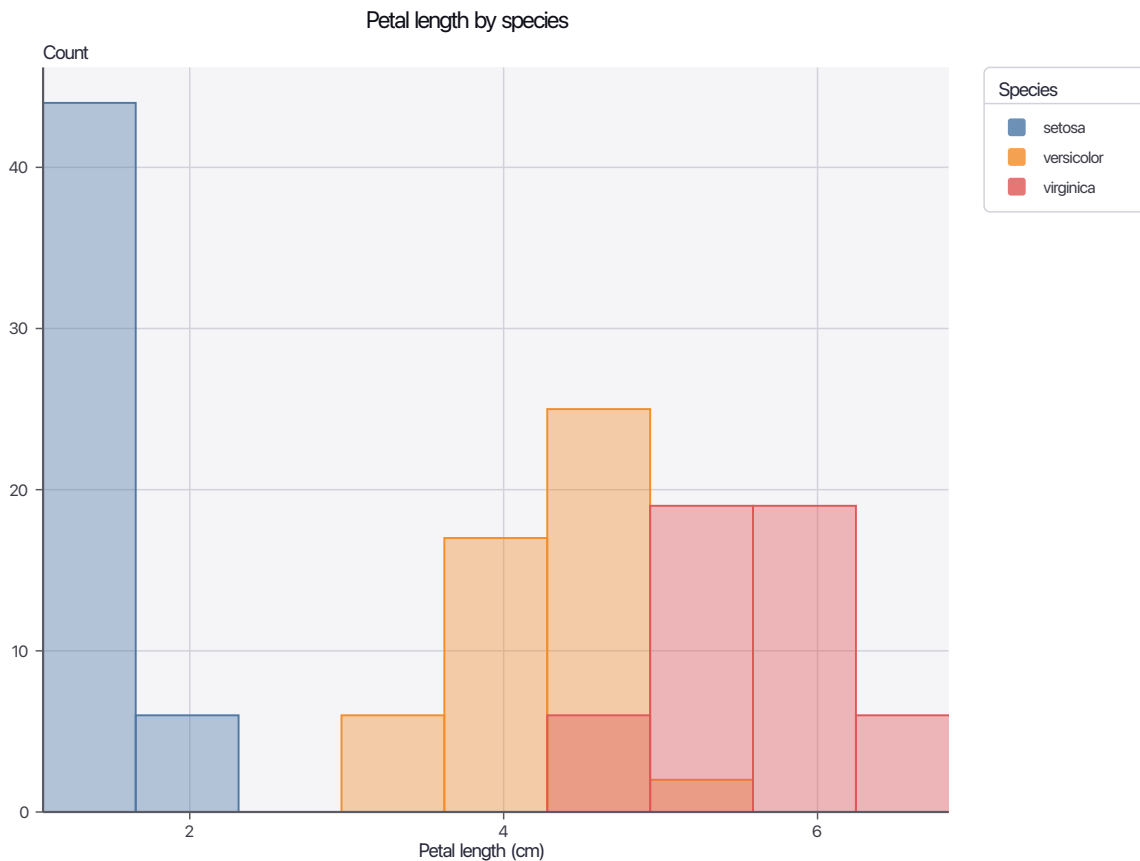
```
(data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species) +  
  x_label("Petal length (cm)") + title("Petal length by species"))
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length) + color(:species) +  
  x_label("Petal length (cm)") + title("Petal length by species")
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length),  
  color(col.species), x_label("Petal length (cm)",  
  title("Petal length by species"))
```



The three histograms **overlay**: each drawn in place, measured from the shared baseline. If the fills were opaque, the last species drawn would hide the others. So each bar is a translucent fill under a solid outline in its own color. The outline is what keeps the shape readable where the fills overlap. *setosa* sits alone on the left and reads clearly. *versicolor* and *virginica* overlap in the middle, and their outlines still separate them. The bins are shared across the groups, so the bars align and the overlap is real rather than an artifact of three different binnings.

The fill is faint by default so the series underneath show through. `style(opacity = )` sets how faint; the outline stays solid whatever you choose:

R

```
data(iris_flowers) + bar * bin + x(petal_length) + color(species) +
  style(opacity = 0.6) + x_label("Petal length (cm)") +
  title("A heavier fill: style(opacity = 0.6)")
```

Python

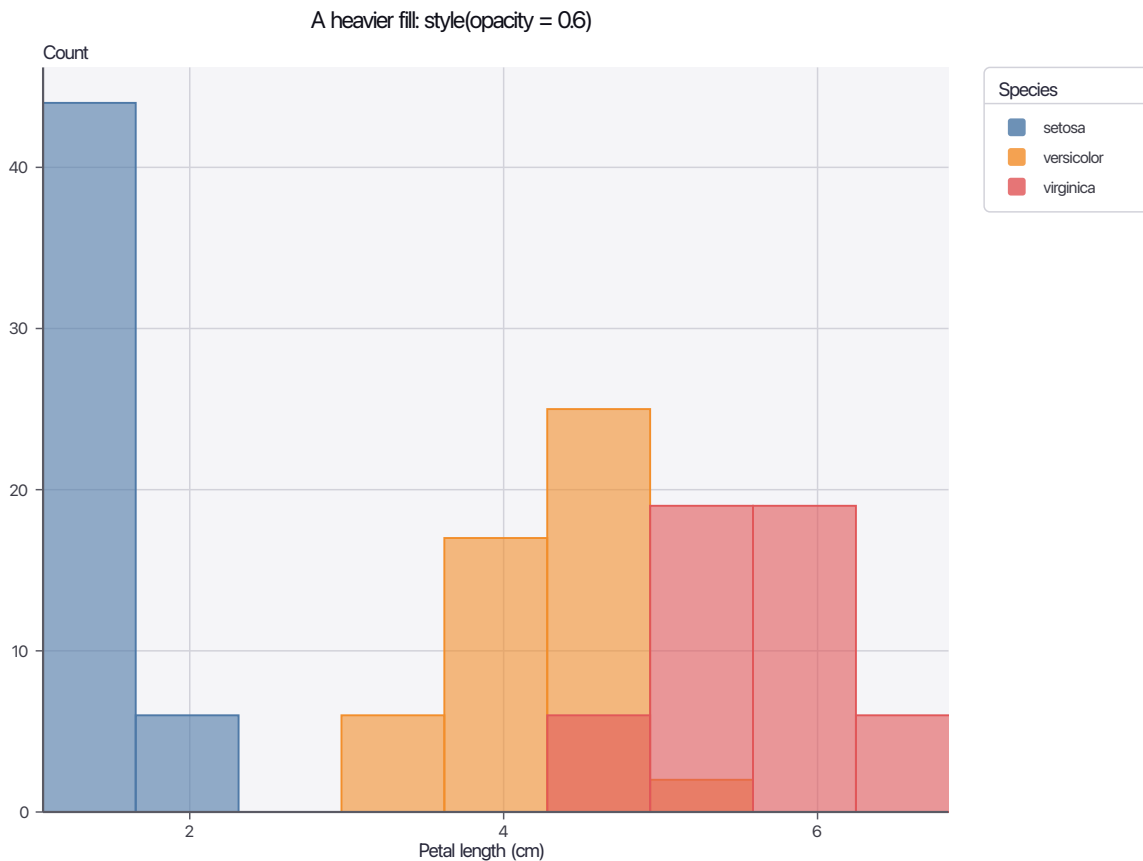
```
(data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species) +  
  style(opacity = 0.6) + x_label("Petal length (cm)") +  
  title("A heavier fill: style(opacity = 0.6)"))
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length) + color(:species) +  
  style(opacity = 0.6) + x_label("Petal length (cm)") +  
  title("A heavier fill: style(opacity = 0.6)")
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length),  
  color(col.species), style({ opacity: 0.6 } ),  
  x_label("Petal length (cm)"),  
  title("A heavier fill: style(opacity = 0.6)"))
```



“Given the iris flowers: bars derived by bin, x is petal length, color by species, with opacity 0.6.”

The outline is the series color by default, but you can recolor it: `style(border_color = )` sets the border, `style(border_size = )` its width. A white border cleanly parts the overlapping fills, the way many overlaid histograms are drawn:

R

```
data(iris_flowers) + bar * bin + x(petal_length) + color(species) +
  style(border_color = "white", border_size = 1.5) +
  x_label("Petal length (cm)") + title("White borders between the fills")
```

Python

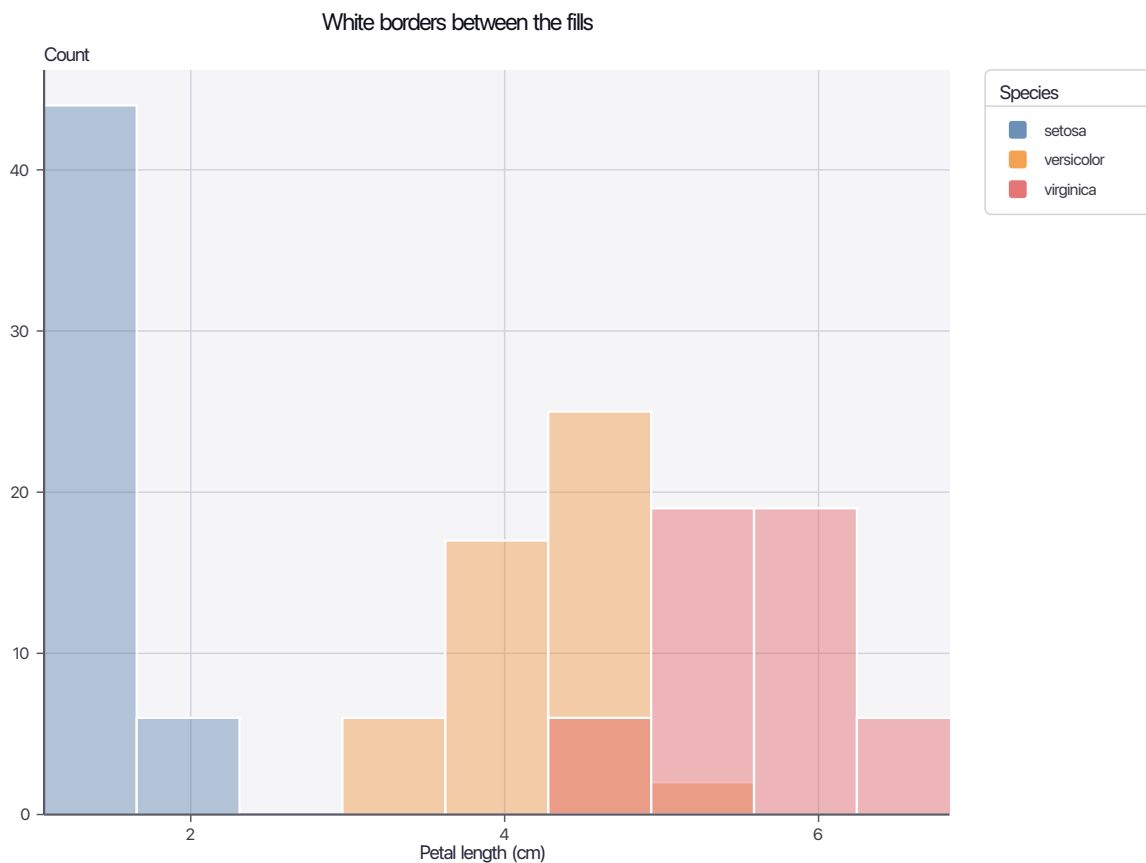
```
(data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species) +
  style(border_color = "white", border_size = 1.5) +
  x_label("Petal length (cm)") + title("White borders between the fills"))
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length) + color(:species) +
  style(border_color = "white", border_size = 1.5) +
  x_label("Petal length (cm)") + title("White borders between the fills")
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length),
  color(col.species), style({ border_color: "white", border_size: 1.5 }),
  x_label("Petal length (cm)", title("White borders between the fills"))
```



“Given the iris flowers: bars derived by bin, x is petal length, color by species, with border color white and border size 1.5.”

The fill and the outline are independent: a bar can be one color inside and another at its edge, whether the fill was mapped (as here) or set. See [Setting vs mapping](#).

Or remove the outline entirely. `style(border_size = 0)` draws no border, so the translucent fills overlap with nothing between them. This is the plainest overlaid histogram:

R

```
data(iris_flowers) + bar * bin + x(petal_length) + color(species) +  
  style(border_size = 0) +  
  x_label("Petal length (cm)") + title("No border: just the fills")
```

Python

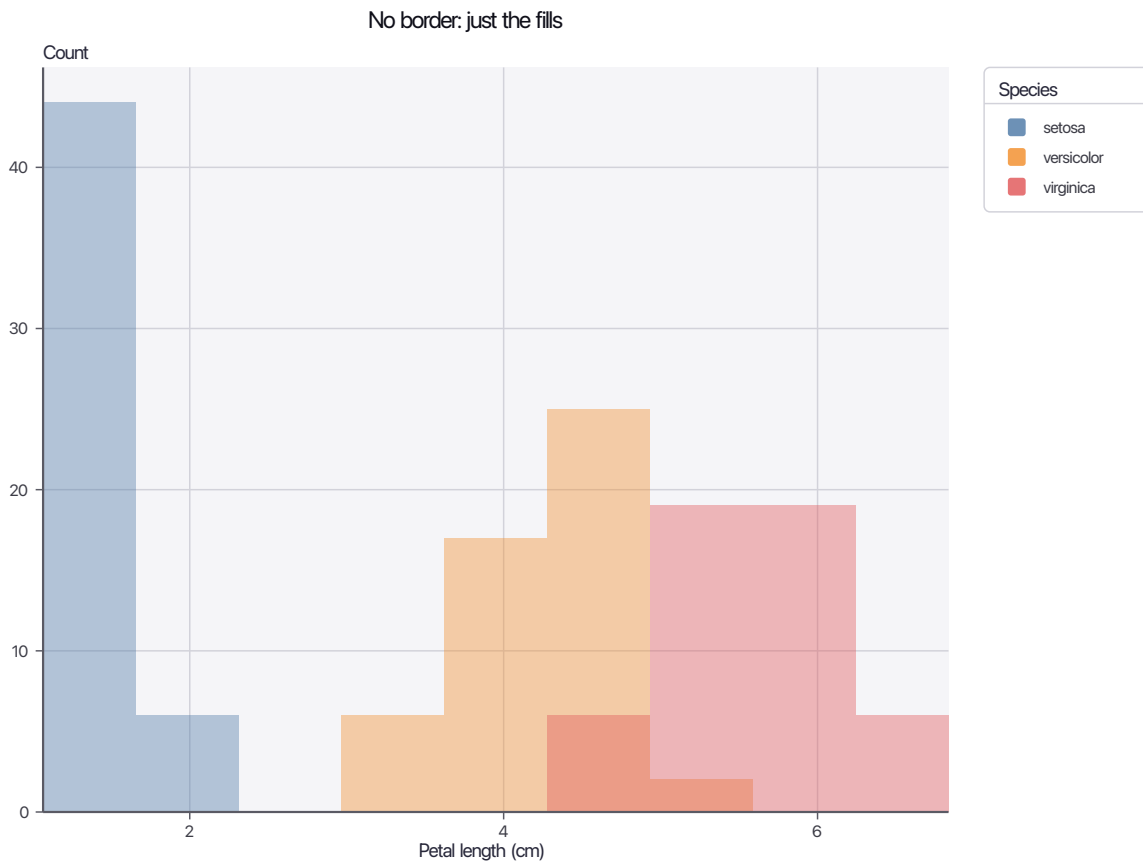
```
(data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species) +  
  style(border_size = 0) +  
  x_label("Petal length (cm)") + title("No border: just the fills"))
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length) + color(:species) +  
  style(border_size = 0) + x_label("Petal length (cm)") +  
  title("No border: just the fills")
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length),  
  color(col.species), style({ border_size: 0 }),  
  x_label("Petal length (cm)", title("No border: just the fills"))
```



The split reads the orientation off the bindings, with one addition: put `petal_length` on `y`, name the other axis with `+ x(count)`, and the overlaid histograms lie down. When the groups overlap so heavily that the outlines run together, give each one its own panel. ... `|facet(species)` draws the same three histograms side by side on a shared scale.

See [Transforms](#) for details on `bin` and other transforms.

10.8 Grouped bars: side by side

A color split draws the groups at one position, each one covering the last. To set them **side by side** as a grouped bar chart, add `dodge`. Each group's bar narrows to share the slot, and the split reads as a comparison within each category. The `gm_eras` table holds two years, 1957 and 2007, with one row per country in each. `mean` averages those rows down to one value per continent, and `era` is the column that splits them.

Table 10.2: `gm_eras`: first 5 of 284 rows

country	continent	year	life	population	gdp	era
Afghanistan	Asia	1957	30.332	9240934	820.8530	1957
Afghanistan	Asia	2007	43.828	31889923	974.5803	2007
Albania	Europe	1957	59.280	1476505	1942.2842	1957
Albania	Europe	2007	76.423	3600523	5937.0295	2007
Algeria	Africa	1957	45.685	10270856	3013.9760	1957

R

```
data(gm_eras) + bar * mean * dodge + x(continent) + y(life) + color(era) +
  y_label("Mean life expectancy") +
  title("Mean life expectancy by continent, 1957 vs 2007")
```

Python

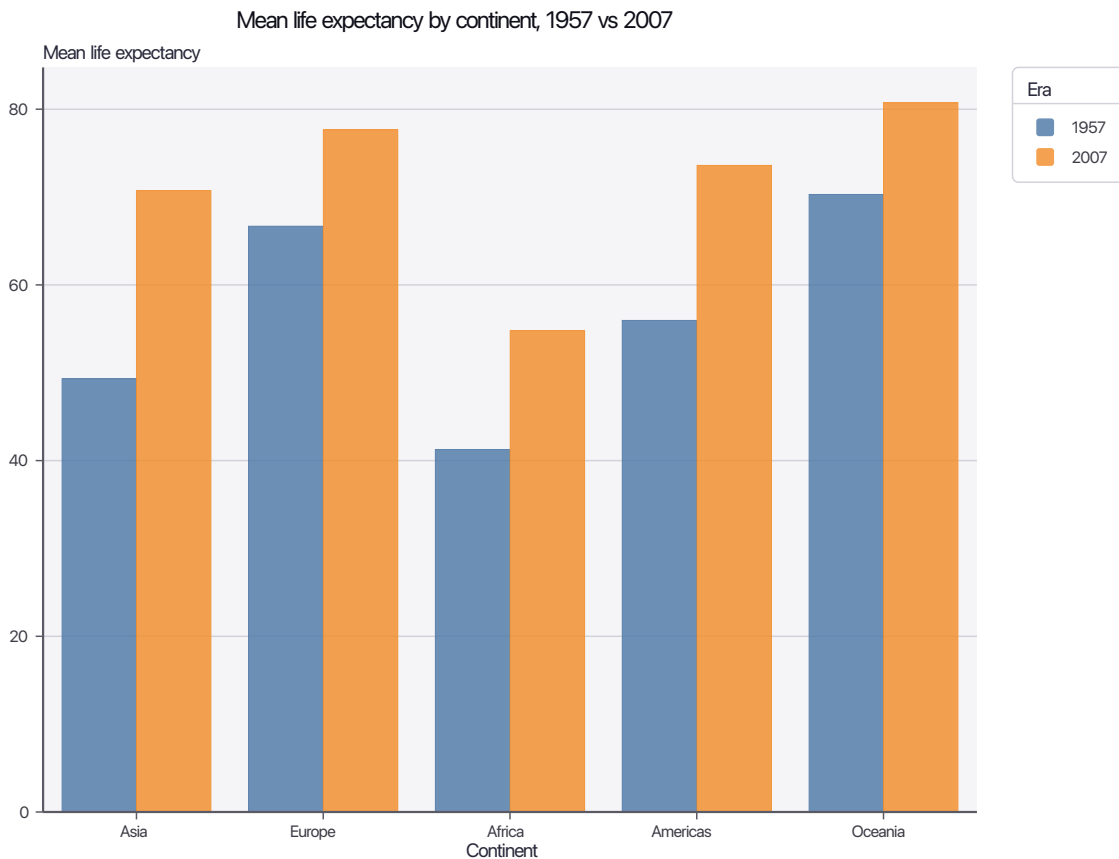
```
(data(gm_eras) + bar * mean * dodge + x(col.continent) + y(col.life) +
  ↪ color(col.era) +
  y_label("Mean life expectancy") +
  title("Mean life expectancy by continent, 1957 vs 2007"))
```

Julia

```
data(gm_eras) + bar * mean * dodge + x(:continent) + y(:life) +
  color(:era) + y_label("Mean life expectancy") +
  title("Mean life expectancy by continent, 1957 vs 2007")
```

JavaScript

```
plot(data(gm_eras), layer(bar, mean, dodge), x(col.continent),
  y(col.life), color(col.era), y_label("Mean life expectancy"),
  title("Mean life expectancy by continent, 1957 vs 2007"))
```



“Given the `gapminder` eras: bars derived by mean and dodge, x is continent, y is life, color by era.”

`dodge` is a collision modifier: it moves the bars rather than summarizing them, so it composes with the statistic (`bar * mean * dodge` here, `bar * count * dodge` for tallies). See [Transforms](#) for the whole family.

10.9 Stacked bars: piled up

The other way to place a color split: instead of side by side, pile the groups **on top of each other** with `stack`. Each bar’s height becomes the category’s total across the groups, and the segments read as parts of a whole:

R

```
data(gm_eras) + bar * sum * stack + x(continent) + y(population) + color(era)
↪ +
  y_label("Population") +
  title("Population by continent, 1957 and 2007 stacked")
```

Python

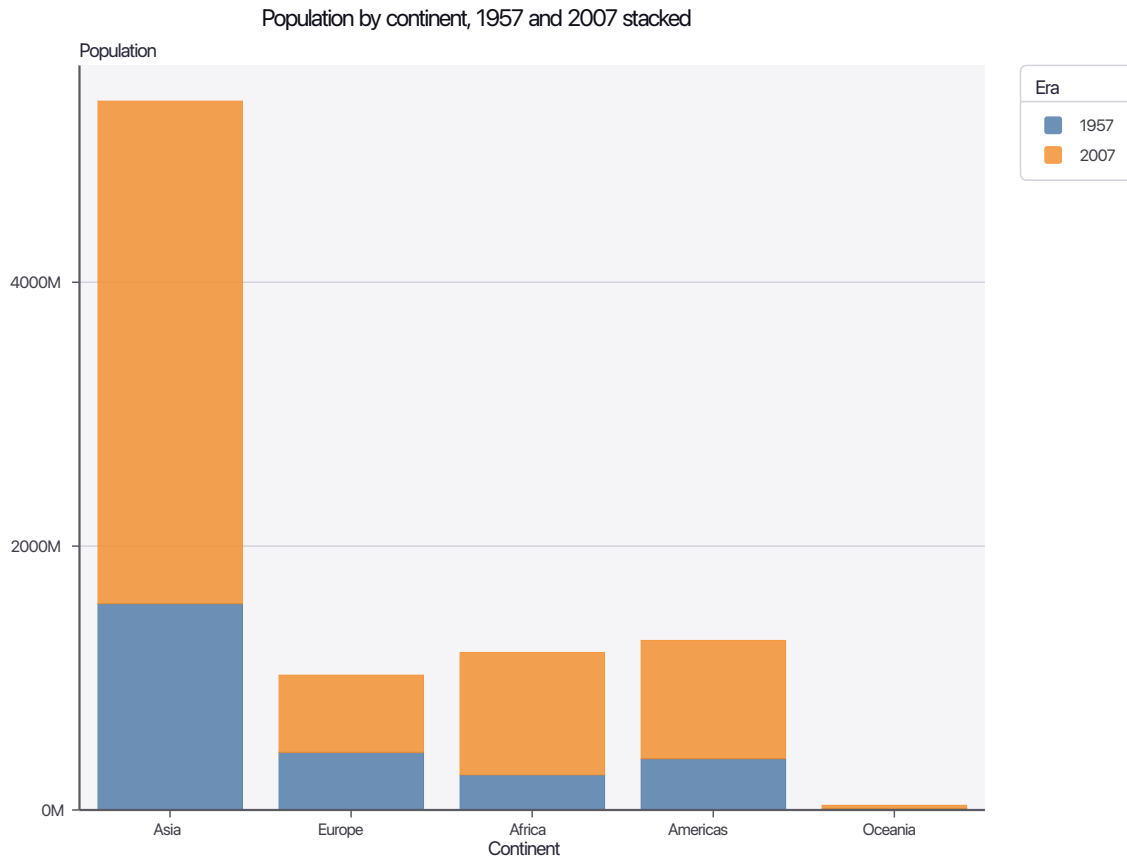
```
(data(gm_eras) + bar * sum * stack + x(col.continent) + y(col.population) +
↪ color(col.era) +
  y_label("Population") +
  title("Population by continent, 1957 and 2007 stacked"))
```

Julia

```
data(gm_eras) + bar * sum * stack + x(:continent) + y(:population) +
  color(:era) + y_label("Population") +
  title("Population by continent, 1957 and 2007 stacked")
```

JavaScript

```
plot(data(gm_eras), layer(bar, sum, stack), x(col.continent),
  y(col.population), color(col.era), y_label("Population"),
  title("Population by continent, 1957 and 2007 stacked"))
```



“Given the gapminder eras: bars derived by sum and stack, x is continent, y is population, color by era.”

dodge compares the groups (equal baselines, side by side); **stack** sums them (one bar, segments piled). Use **stack** when the parts add to a meaningful total (populations, counts, revenue), and **dodge** when you are comparing the groups. Note the measure: you stack **sum** of population, because quantities add, while the dodge example compared **mean** of life, because averages do not.

10.9.1 The 100% stacked bar

The plot above answers two questions at once, and the larger answer covers the smaller one. Asia’s two eras total 5.4 billion and Oceania’s total 0.04 billion, so the axis is scaled for Asia and the *split* inside the small continents cannot be read. When the composition is the question and the total is not, **stack(share = TRUE)** divides each pile by its own bar’s total, so every bar reaches 1 and the segments are the only thing left to read:

R

```
data(gm_eras) + bar * sum * stack(share = TRUE) + x(continent) +  
  ↪ y(population) +  
  color(era) + title("Each continent's 1957/2007 split, filled to one")
```

Python

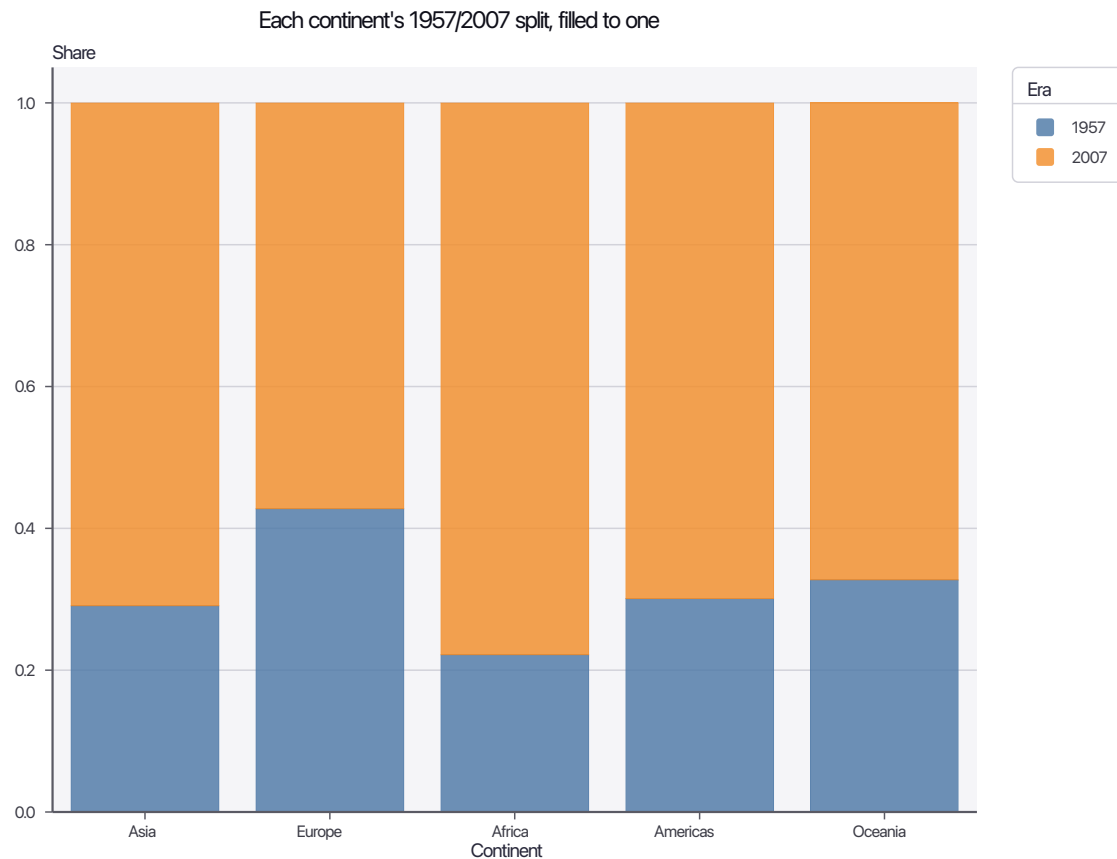
```
(data(gm_eras) + bar * sum * stack(share = True) + x(col.continent) +  
  ↪ y(col.population) +  
  color(col.era) + title("Each continent's 1957/2007 split, filled to one"))
```

Julia

```
data(gm_eras) + bar * sum * stack(share = true) + x(:continent) +  
  y(:population) + color(:era) +  
  title("Each continent's 1957/2007 split, filled to one")
```

JavaScript

```
plot(data(gm_eras), layer(bar, sum, stack({ share: true })),  
  x(col.continent), y(col.population), color(col.era),  
  title("Each continent's 1957/2007 split, filled to one"))
```



“Given the gapminder eras: bars derived by sum and stack as shares, x is continent, y is population, color by era.”

Now the comparison is between continents rather than inside one, and two continents show the range. **Africa** is the most uneven: 78% of its two-era total falls in 2007, so its population more than tripled. **Europe** is the most even at 57%, the closest any continent comes to a half-and-half split. Neither split can be read in the stacked plot above this section, at any size. What you lose is the totals: the plot no longer shows them at all. That is the cost of the choice, not a fault. Oceania’s bar is now as tall as Asia’s.

The axis says **Share**, because the numbers are fractions of one now, whatever they were before. That is true even though `y(population)` names a column: `stack(share = TRUE)` rescales an axis you already named. It is not **proportion** under another name. The two divide by different totals. **proportion** divides by the whole plot’s total, so its bars still show how large each category is. This one divides by the bar’s own total. And only this one works on a **sum** of a column, which **proportion** has no way to say.

10.10 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(pattern =)</code>	<code>solid</code> , <code>hatch</code> , <code>crosshatch</code> , <code>grid</code> , <code>dots</code>
<code>style(border_color =)</code>	any CSS color name or hex
<code>style(border_size =)</code>	pixels

And these vary per row if you map them to a column instead: `color()` (categories), `pattern()` (categories), `opacity()` (numbers), `play()` (either).

A bar has no `size`, and that is deliberate. Its position and its value already fix how large it is, so a width setting would be a second, conflicting answer. What it does have, being a shape with an inside and an outline, is a border.

R

```
data(medals) + bar * sum + x(country) + y(gold) +
  style(color = "goldenrod", pattern = "crosshatch",
        border_color = "black", border_size = 1) +
  y_label("Gold medals") + title("A fill, a texture, and a border")
```

Python

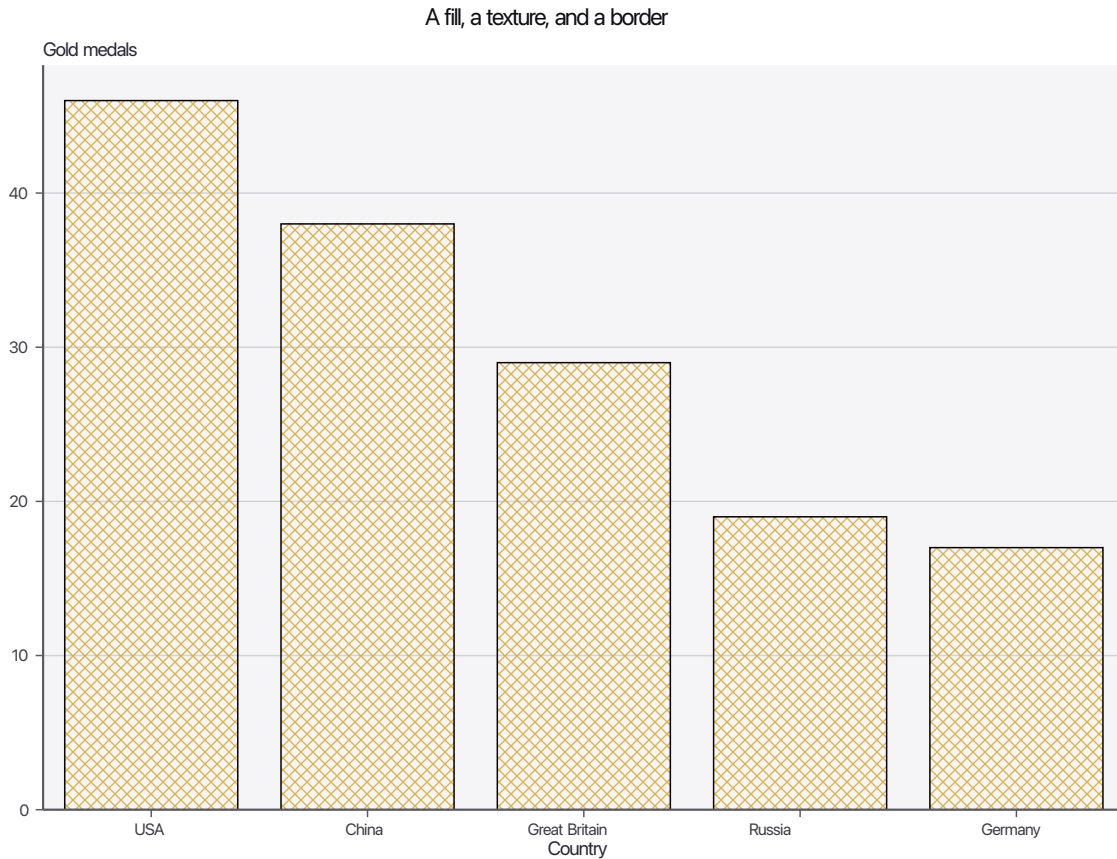
```
(data(medals) + bar * sum + x(col.country) + y(col.gold) +
  style(color = "goldenrod", pattern = "crosshatch",
        border_color = "black", border_size = 1) +
  y_label("Gold medals") + title("A fill, a texture, and a border"))
```

Julia

```
data(medals) + bar * sum + x(:country) + y(:gold) +
  style(color = "goldenrod", pattern = "crosshatch", border_color = "black",
  ↪ border_size = 1) +
  y_label("Gold medals") + title("A fill, a texture, and a border")
```

JavaScript

```
plot(data(medals), layer(bar, sum), x(col.country), y(col.gold),
     style({ color: "goldenrod", pattern: "crosshatch",
             border_color: "black", border_size: 1 }), y_label("Gold medals"),
     title("A fill, a texture, and a border"))
```



“Given the medals: bars derived by sum, x is country, y is gold, colored goldenrod, with pattern crosshatch, border color black and border size 1.”

The **pattern** values here are the five **fill textures**, not the three dashes a **line** takes. One setting name, and each shape realizes it in its own way, so a plot that has to survive grayscale printing can separate its bars by texture instead of color.

The one thing left to adjust is the histogram’s bins, and that is not a setting. **bin(bins = 20)** and **bin(width = 5)** control the histogram **bar * bin** draws. They belong to the transform, because binning is what has a parameter, not the rectangle.

10.11 What it refuses

A bar measures a length from a baseline, and both refusals below come from that one fact.

Binning cuts a continuous axis into intervals, so there is nothing for it to cut on a categorical one:

```
data(gapminder_2007) + bar * bin + x(continent)
```

Error:

```
! gog: `bin` cuts a continuous axis into intervals, and `x(continent)` is
  ↳ categorical - a category is one slot, with no width to cut. To tally rows
  ↳ per category, `count` is the transform that does it: `bar * count`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A category is already one slot per value, which is what count is for. The refusal says so rather than drawing empty bins.

Texture is the second refusal. A bar is a fill, so it takes a fill's textures, and a dash is a stroke's:

```
data(medals) + bar + x(country) + y(gold) + style(pattern = "dashed")
```

Error:

```
! gog: `style(pattern = )` on a `bar`: `"dashed"` is a stroke's dash, not a
  ↳ fill texture. Use "solid" (the default), "hatch", "crosshatch", "grid",
  ↳ or "dots".
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A bar's outline takes a color and a width, never a dash. `border_color` and `border_size` are the only two settings the outline has. And a bar given two categorical positions has nothing left to measure, which is the refusal shown above under horizontal bars.

11 Area

How much was there at each point in time, for each group? `area` fills the region between the data and a baseline at zero. It is `line` plus everything between it and the baseline: the same boundary, drawn as a quantity rather than as a path.

11.1 Basic usage

R

```
data(gapminder_asia) + area + x(year) + y(life) +  
  group(country) + style(opacity = 0.4) +  
  title("Life expectancy, five Asian countries")
```

Python

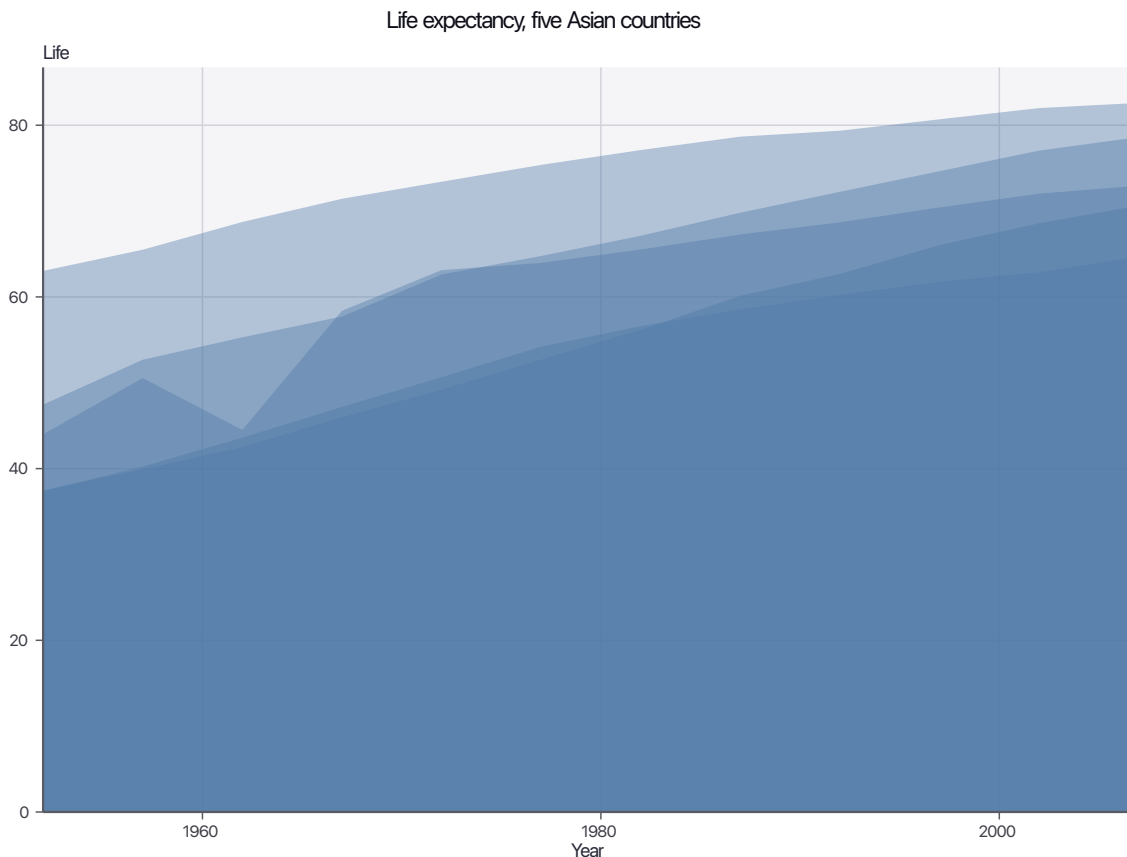
```
(data(gapminder_asia) + area + x(col.year) + y(col.life) +  
  group(col.country) + style(opacity = 0.4) +  
  title("Life expectancy, five Asian countries"))
```

Julia

```
data(gapminder_asia) + area + x(:year) + y(:life) + group(:country) +  
  style(opacity = 0.4) + title("Life expectancy, five Asian countries")
```

JavaScript

```
plot(data(gapminder_asia), area, x(col.year), y(col.life),  
  group(col.country), style({ opacity: 0.4 } ),  
  title("Life expectancy, five Asian countries"))
```



“Given gapminder Asia: areas, x is year, y is life, grouped by country.”

An area’s two axes are different kinds of thing, and the rest of this chapter follows from that difference. **x** is the **domain**, the axis the boundary is read along; **y** is the **measure**, the quantity the region fills to. That is why an area has no horizontal form the way a **bar** does. A bar takes its measure axis from the columns you bind. An area’s measure is always **y**.

Notice the fill reaches the left and right edges of the panel. A filled region has no glyph to keep clear of the panel edge. So the domain axis stops at the data, instead of leaving the margin a scatter gets at each end. [Scales](#) explains why.

11.2 The baseline has to be zero

A **line** says *where*; an **area** says *how much*. The fill is the quantity, so it has to start from zero. An area drawn from any other baseline would show a height that measures nothing.

That makes the mark a real choice rather than decoration. Life expectancy has no meaningful zero, and the plot above fills every year from zero up to each curve. Sales do have a meaningful zero, and the same mark reads correctly:

R

```
data(actuals) + area + x(year) + y(sales) +  
  y_label("Sales") + title("Sales: zero means zero")
```

Python

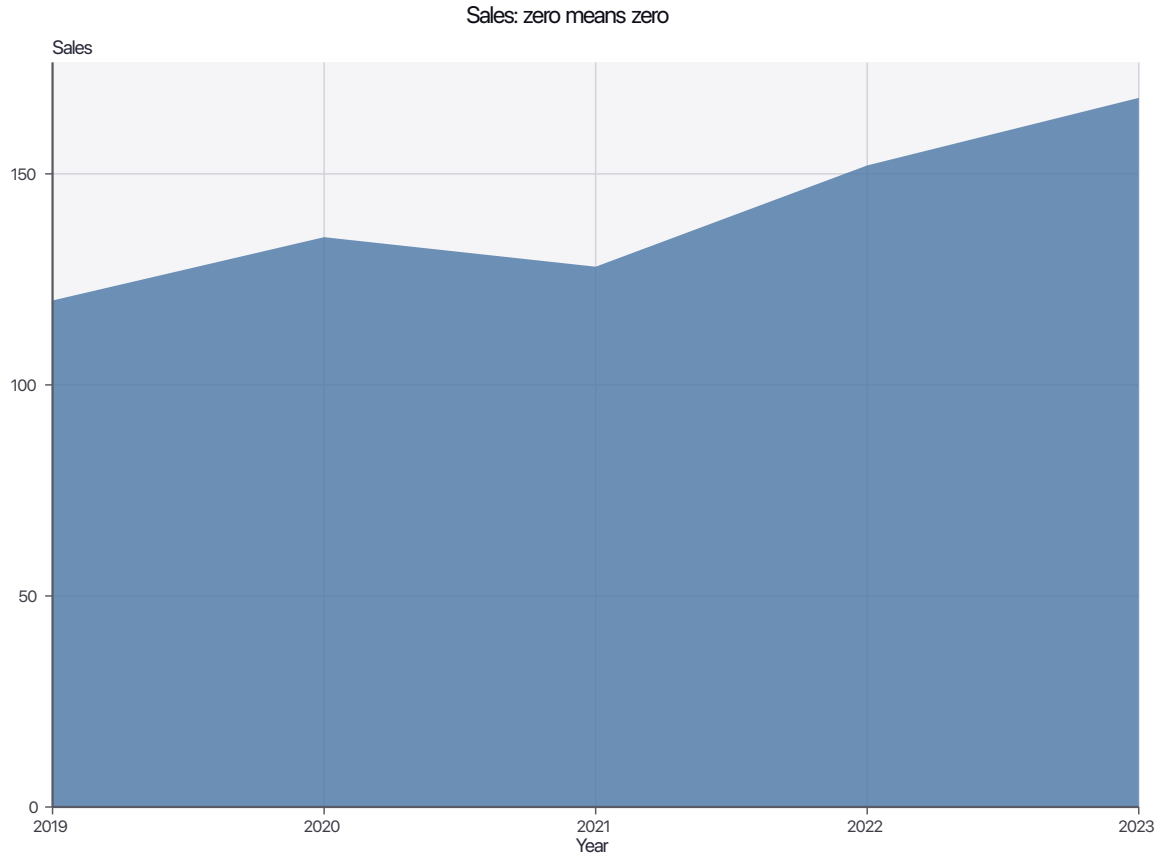
```
(data(actuals) + area + x(col.year) + y(col.sales) +  
  y_label("Sales") + title("Sales: zero means zero"))
```

Julia

```
data(actuals) + area + x(:year) + y(:sales) + y_label("Sales") +  
  title("Sales: zero means zero")
```

JavaScript

```
plot(data(actuals), area, x(col.year), y(col.sales), y_label("Sales"),  
  title("Sales: zero means zero"))
```



When the quantity you are drawing has no zero, `line` is the mark to use:

R

```
data(gapminder_asia) + line + x(year) + y(life) + color(country) +
  title("The same data, as paths rather than quantities")
```

Python

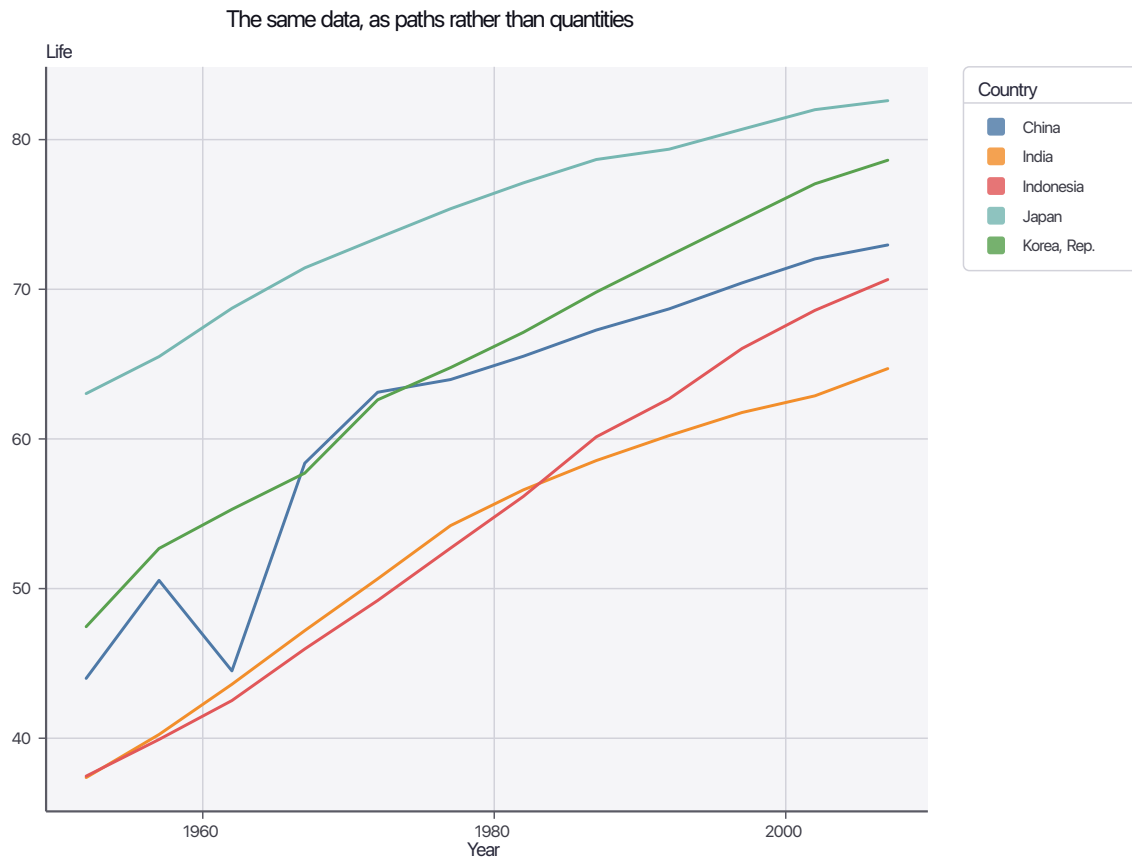
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + color(col.country)
↪ +
  title("The same data, as paths rather than quantities"))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + color(:country) +
  title("The same data, as paths rather than quantities")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),
  color(col.country),
  title("The same data, as paths rather than quantities"))
```



“Given gapminder Asia: lines, x is year, y is life, color by country.”

11.3 Multiple series

A categorical column *splits* a mark, one region per category, exactly as `color` splits `line` into one polyline each:

R

```
data(gapminder_asia) + area + x(year) + y(life) + color(country) +  
  style(opacity = 0.45) +  
  title("One region per country")
```

Python

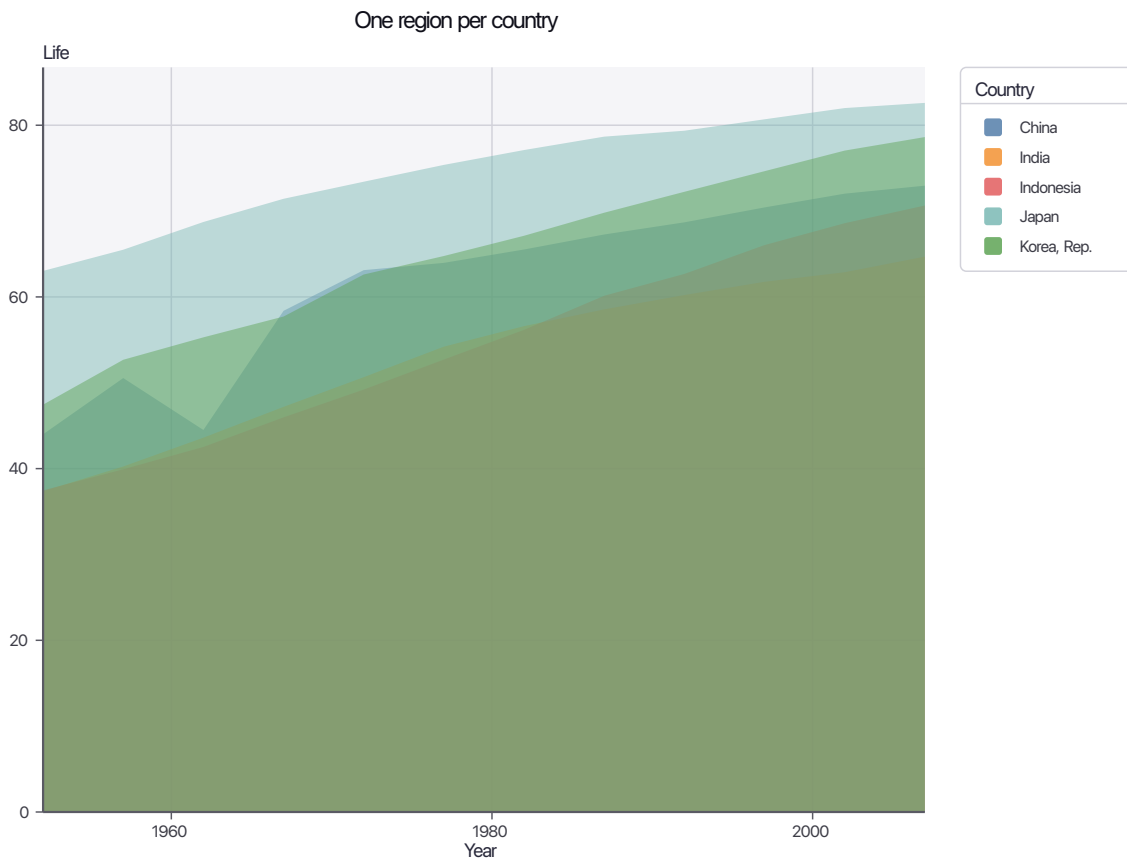
```
(data(gapminder_asia) + area + x(col.year) + y(col.life) + color(col.country)  
↪ +  
  style(opacity = 0.45) +  
  title("One region per country"))
```

Julia

```
data(gapminder_asia) + area + x(:year) + y(:life) + color(:country) +  
  style(opacity = 0.45) + title("One region per country")
```

JavaScript

```
plot(data(gapminder_asia), area, x(col.year), y(col.life),  
  color(col.country), style({ opacity: 0.45 }),  
  title("One region per country"))
```



Regions are filled, so unlike polylines they hide one another. Write that plot without an opacity and `gog` warns you, rather than letting you read a plot with a region hidden underneath:

R

```
data(gapminder_asia) + area + x(year) + y(life) + color(country) +
  title("Opaque regions: the last one drawn covers the rest")
```

Python

```
(data(gapminder_asia) + area + x(col.year) + y(col.life) + color(col.country)
  +
  title("Opaque regions: the last one drawn covers the rest"))
```

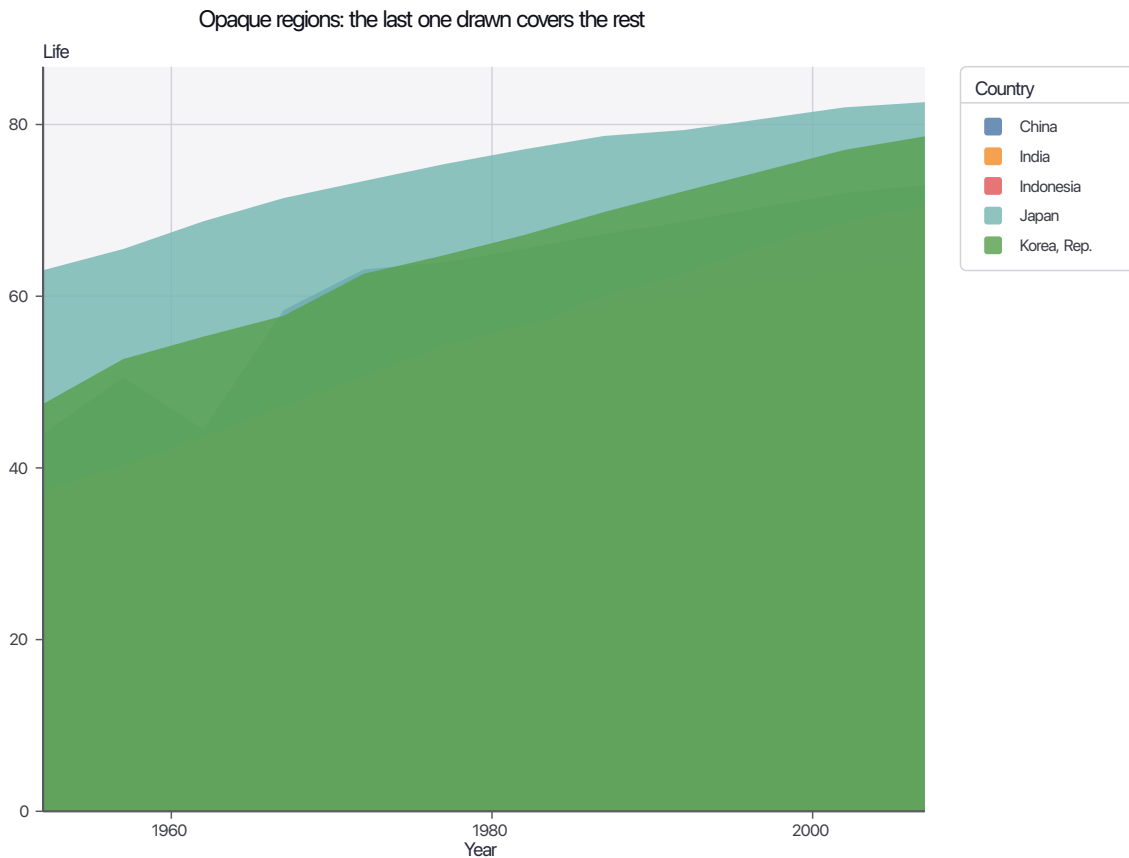
Julia


```
data(gapminder_asia) + area + x(:year) + y(:life) + color(:country) +
  title("Opaque regions: the last one drawn covers the rest")
```

JavaScript

```
plot(data(gapminder_asia), area, x(col.year), y(col.life),
  color(col.country),
  title("Opaque regions: the last one drawn covers the rest"))
```

gog: `area` split by `country` draws 5 regions on top of one another, and the
 ↪ last one drawn can hide the rest. Add `style(opacity = 0.5)` to see
 ↪ through them, or use `line` to compare shapes without filling.



Compare the legend with the plot: every country is named in the legend, and the lowest regions are painted over by the ones drawn after them. The note above the plot is advice rather than

a refusal. The grammar has no objection to this sentence, so `gog` guides your taste and still draws it. Stacking the regions instead of overlaying them is the other answer, and it is written `area * stack`, an area's collision modifier. Its sibling `dodge` subdivides a *width*, and an area has no width. An area is offset by *accumulating*, and that is `stack`. Each region sits on the cumulative height of the ones below, and they meet edge to edge in one band instead of hiding each other:

R

```
data(gapminder_asia) + area * stack + x(year) + y(population) +  
  ↪ color(country) +  
  y_label("Population") +  
  title("Five Asian countries, population stacked")
```

Python

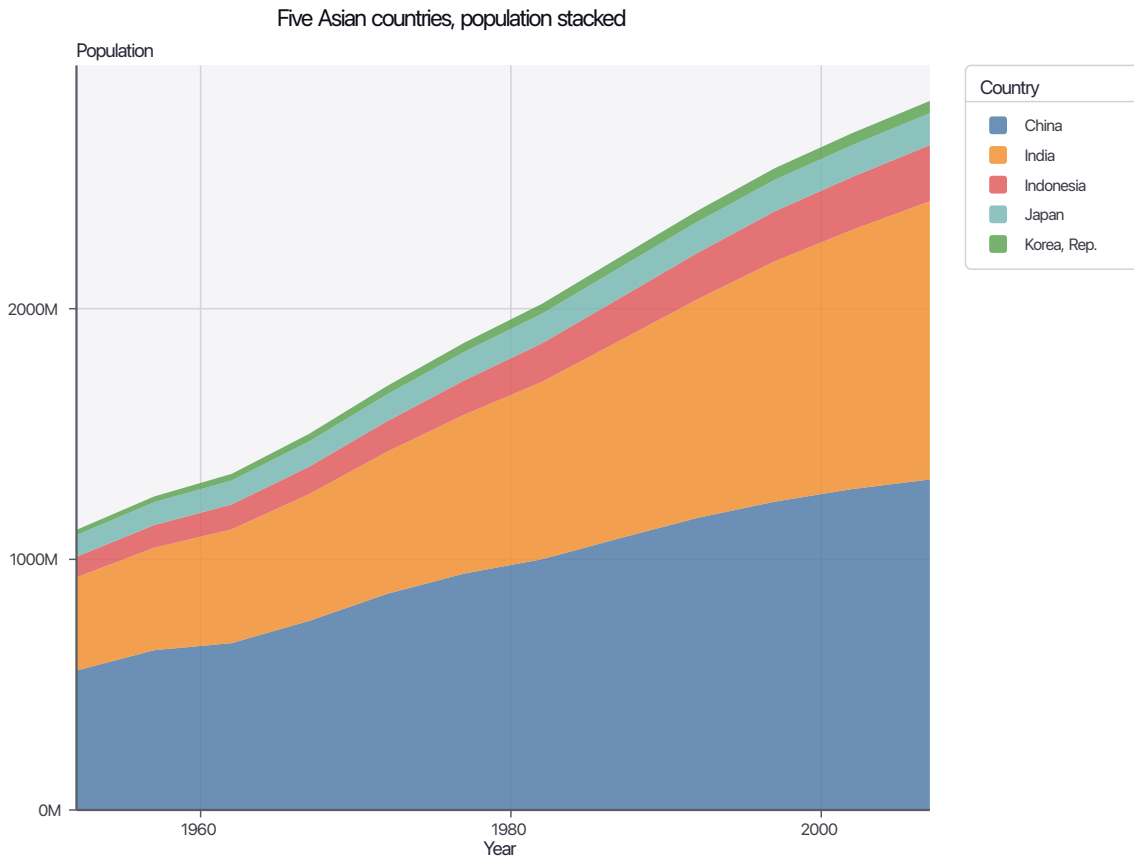
```
(data(gapminder_asia) + area * stack + x(col.year) + y(col.population) +  
  ↪ color(col.country) +  
  y_label("Population") +  
  title("Five Asian countries, population stacked"))
```

Julia

```
data(gapminder_asia) + area * stack + x(:year) + y(:population) +  
  color(:country) + y_label("Population") +  
  title("Five Asian countries, population stacked")
```

JavaScript

```
plot(data(gapminder_asia), layer(area, stack), x(col.year),  
  y(col.population), color(col.country), y_label("Population"),  
  title("Five Asian countries, population stacked"))
```



“Given gapminder Asia: areas derived by stack, x is year, y is population, color by country.”

The thickness of each color is that country’s population, and the band’s top edge is the five-country total. The measure moved from **life** to **population** on purpose: you stack quantities that *add* (populations sum to a real total), never averages (a stack of life expectancies means nothing). The [Transforms](#) chapter covers **stack**, and stacked bars, in full.

11.4 Giving an area an edge

An area takes no border of its own. The edge on top of a filled region is a **line**, and layering one over the other is how you ask for it:

R

```
data(actuals) + x(year) + y(sales) +  
  area + style(color = "steelblue", opacity = 0.25) +  
  line + style(color = "steelblue", size = 2.5) +  
  title("area + line: the fill and its edge, composed")
```

Python

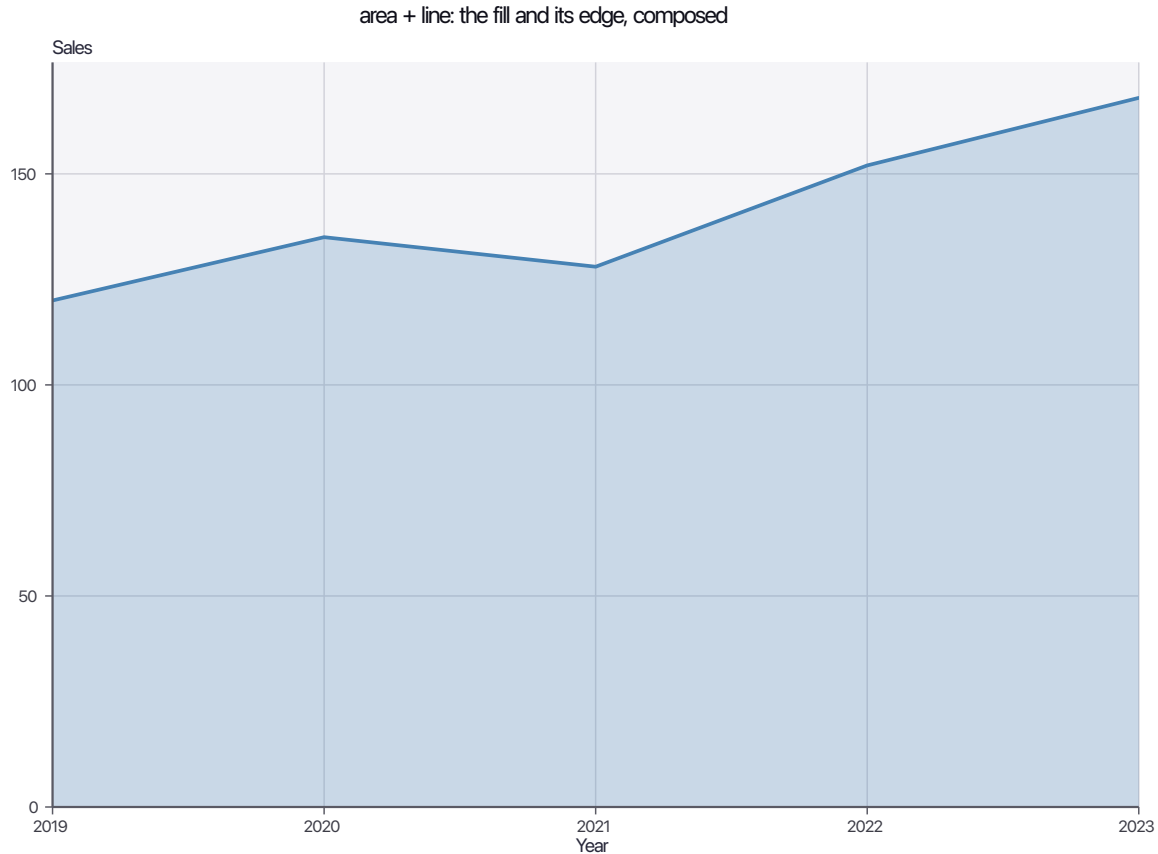
```
(data(actuals) + x(col.year) + y(col.sales) +  
  area + style(color = "steelblue", opacity = 0.25) +  
  line + style(color = "steelblue", size = 2.5) +  
  title("area + line: the fill and its edge, composed"))
```

Julia

```
data(actuals) + x(:year) + y(:sales) + area +  
  style(color = "steelblue", opacity = 0.25) + line +  
  style(color = "steelblue", size = 2.5) +  
  title("area + line: the fill and its edge, composed")
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), area,  
  style({ color: "steelblue", opacity: 0.25 }), line,  
  style({ color: "steelblue", size: 2.5 }),  
  title("area + line: the fill and its edge, composed"))
```



“Given the actuals: x is year, y is sales, an area and also a line.”

There is deliberately no border setting on `area` to use instead. Two marks that already exist, combined with `+`, are better than a new setting on one of them. The composed version is also more capable: the edge takes its own color, width and opacity as a `line`.

11.5 One region, one fill

An area is a single filled region, so `opacity` is a *setting* here and not a channel. A row is a vertex of the boundary, not a region of its own, so a per-row opacity has nothing to vary along. The same reasoning makes `opacity` a setting on `line`, one dimension down:

```
render_svg(data(gapminder_asia) + area + x(year) + y(life) + opacity(gdp))
```

Error:

```
! gog: `opacity` cannot be bound to `area` - an area has no opacity feature.  
  ↳ Remove the `gdp` mapping from `opacity`, or use a mark that has one.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

size is refused entirely: on area it cannot even be set. A stroke has a width to choose, so line lets you set it. An area's extent comes from its x, its y and its baseline, so there is no size to choose:

```
render_svg(data(actuals) + area + x(year) + y(sales) + style(size = 4))
```

Error:

```
! gog: `style(size = 4)` - an area has no size to set. Remove it, or use a  
  ↳ mark that has one.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Two marks, two different answers, both derived from what the mark *is* rather than listed in a table of options. `style(color = )` and `style(opacity = )` work on both.

11.6 With transforms

`bin`, `density` and the aggregations mean the same thing on an area as on a bar. Binned, it is a filled histogram:

R

```
data(gapminder_2007) + area * bin + x(life) +  
  x_label("Life expectancy (years)") +  
  title("area * bin: a filled histogram")
```

Python

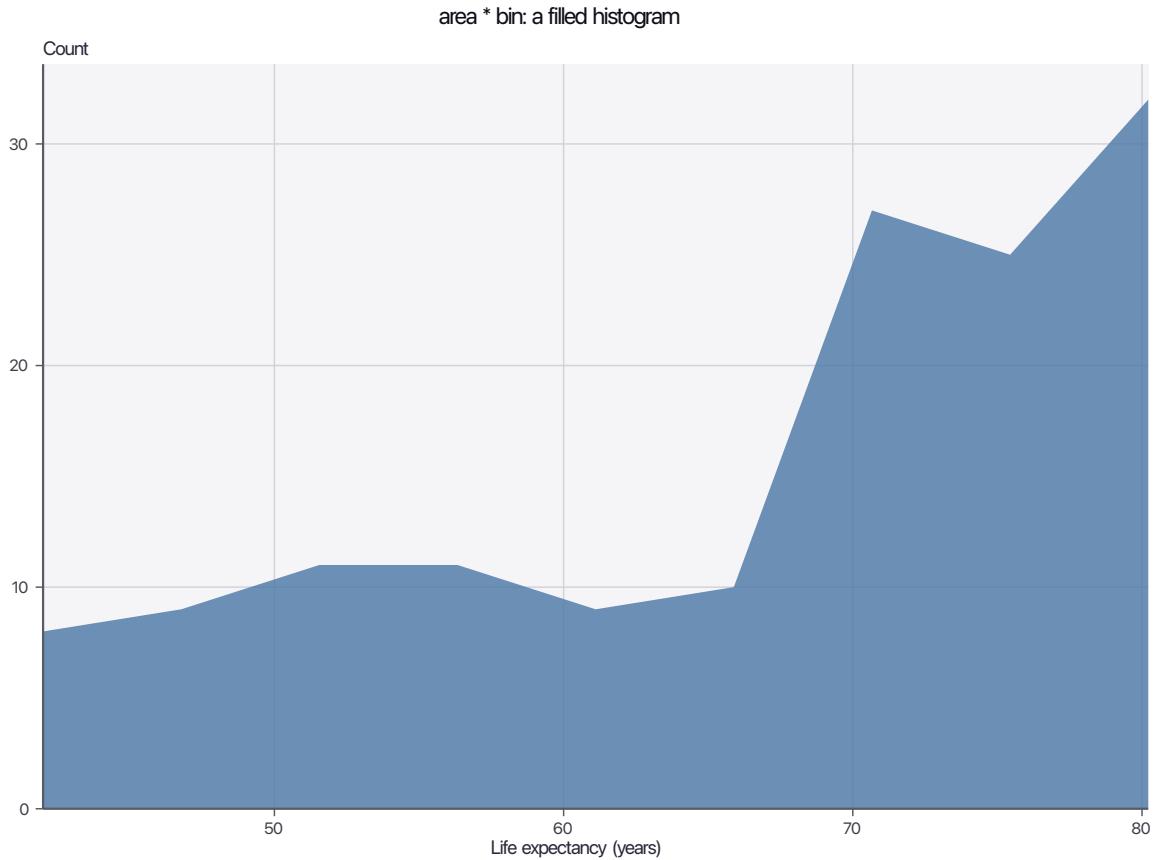
```
(data(gapminder_2007) + area * bin + x(col.life) +  
  x_label("Life expectancy (years)") +  
  title("area * bin: a filled histogram"))
```

Julia

```
data(gapminder_2007) + area * bin + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("area * bin: a filled histogram")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, bin), x(col.life),  
  x_label("Life expectancy (years)"),  
  title("area * bin: a filled histogram"))
```



“Given gapminder 2007: an area derived by bin, x is life.”

Smoothed into a density, it is the distribution as a shape:

R

```
data(gapminder_2007) + area * density + x(life) +  
  x_label("Life expectancy (years)") +  
  title("area * density")
```

Python

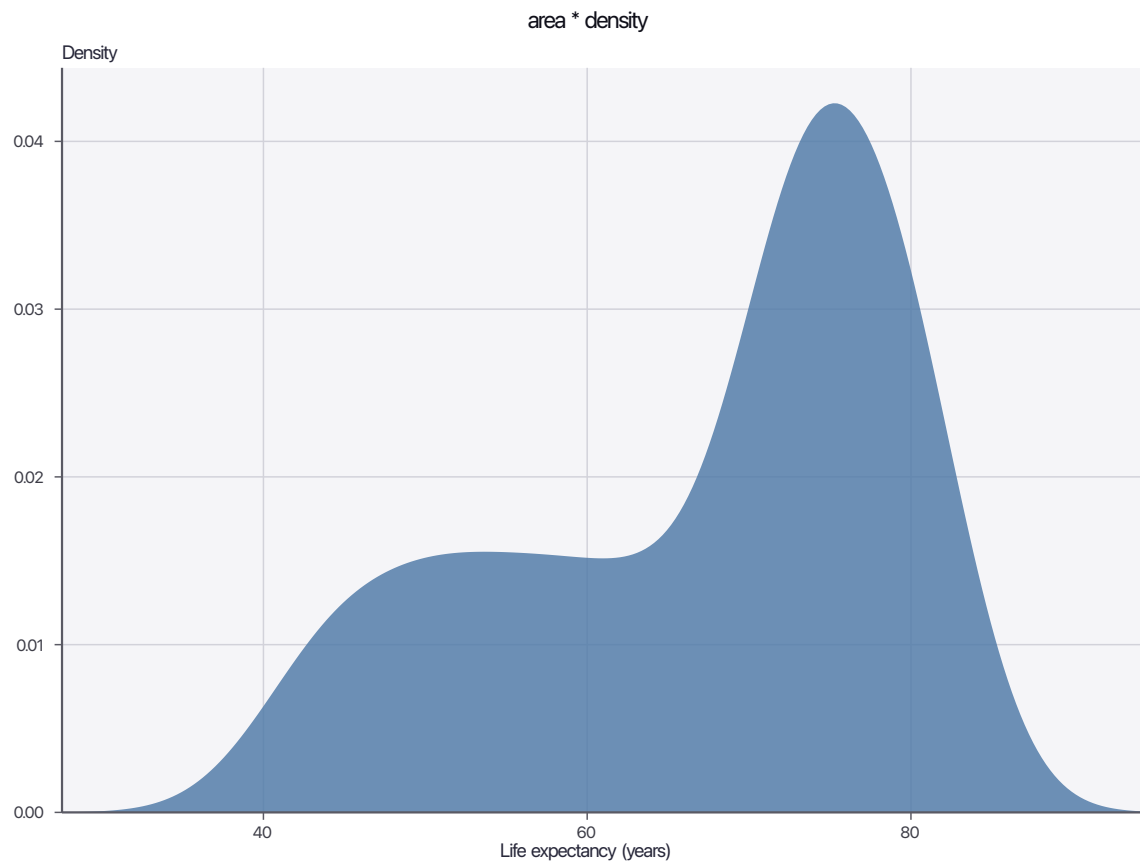
```
(data(gapminder_2007) + area * density + x(col.life) +  
  x_label("Life expectancy (years)") +  
  title("area * density"))
```

Julia

```
data(gapminder_2007) + area * density + x(:life) +  
  x_label("Life expectancy (years)") + title("area * density")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, density), x(col.life),  
  x_label("Life expectancy (years)", title("area * density"))
```



“Given gapminder 2007: an area derived by density, x is life.”

Give that same sentence a category on **x** and a number on **y**, and the estimate changes axis. It is drawn *across each category’s slot* instead of along the whole axis, one distribution per group. It is the half violin. `ribbon * density` draws the mirrored form, and explains what the width of a violin can mean:

R

```
data(gapminder_2007) + area * density + x(continent) + y(life) +  
  y_label("Life expectancy") +  
  title("area * density + x(continent): the half violin")
```

Python

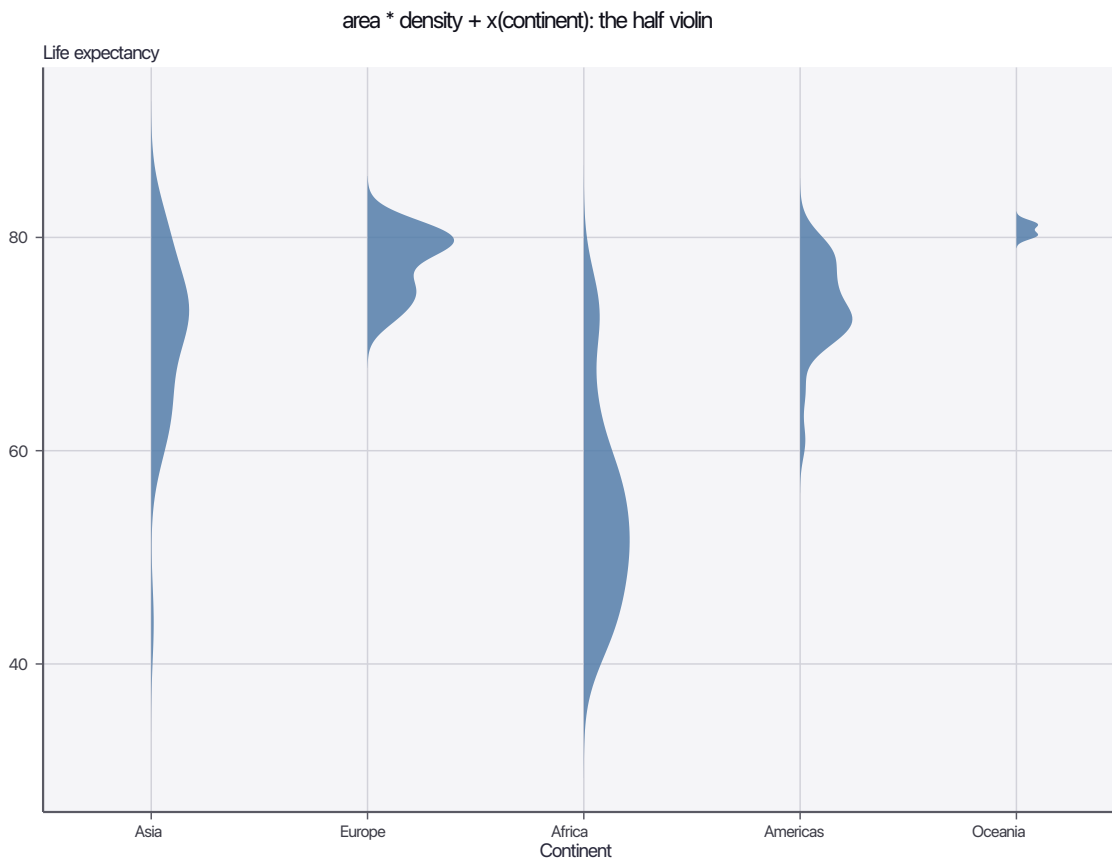
```
(data(gapminder_2007) + area * density + x(col.continent) + y(col.life) +  
  y_label("Life expectancy") +  
  title("area * density + x(continent): the half violin"))
```

Julia

```
data(gapminder_2007) + area * density + x(:continent) + y(:life) +  
  y_label("Life expectancy") +  
  title("area * density + x(continent): the half violin")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, density), x(col.continent),  
  y(col.life), y_label("Life expectancy"),  
  title("area * density + x(continent): the half violin"))
```



The two sentences differ by one binding, and that difference is exactly what the extra binding changes. With only `x(life)`, the y axis is unused, so the estimate is measured along it. With a category there is no unused axis, so it is measured across the slot instead.

And an aggregation summarizes before filling, exactly as it does for `bar`:

R

```
data(gapminder_asia) + area * mean + x(year) + y(life) +
  y_label("Mean life expectancy") +
  title("area * mean: five countries averaged per year")
```

Python

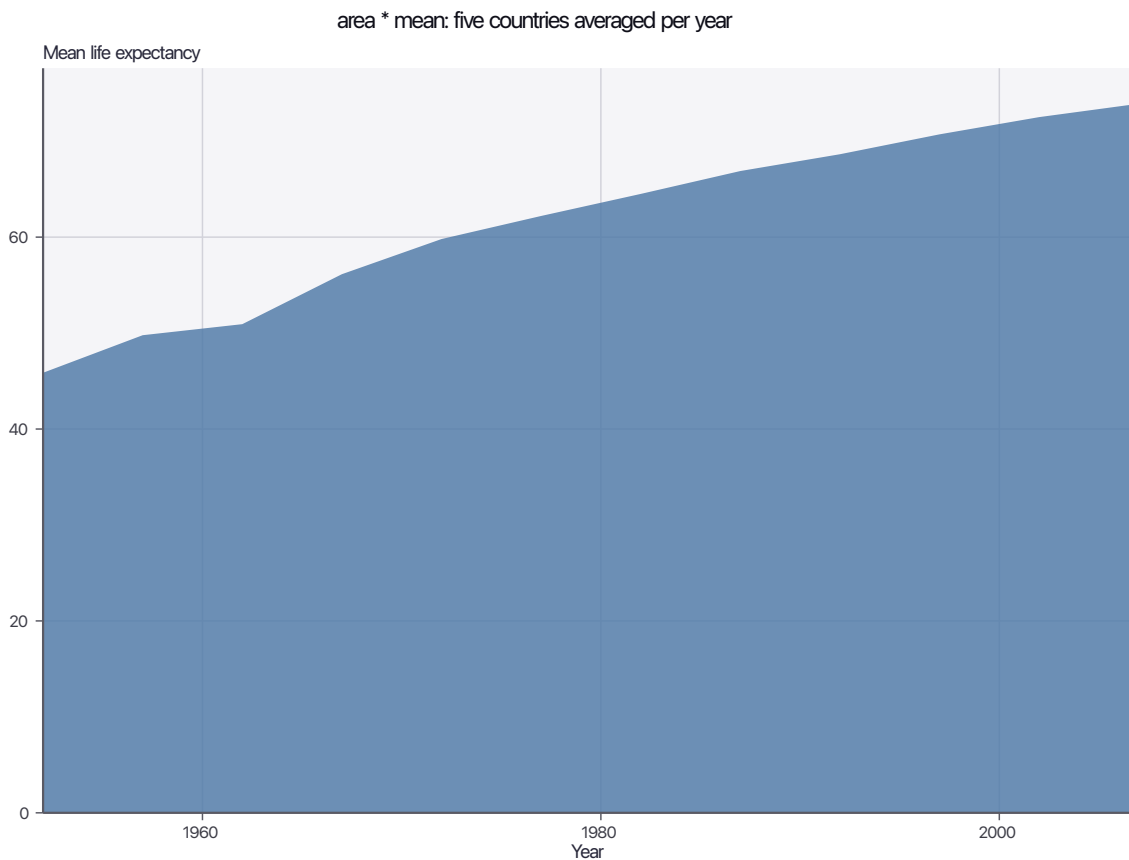
```
(data(gapminder_asia) + area * mean + x(col.year) + y(col.life) +
  y_label("Mean life expectancy") +
  title("area * mean: five countries averaged per year"))
```

Julia

```
data(gapminder_asia) + area * mean + x(:year) + y(:life) +  
  y_label("Mean life expectancy") +  
  title("area * mean: five countries averaged per year")
```

JavaScript

```
plot(data(gapminder_asia), layer(area, mean), x(col.year), y(col.life),  
  y_label("Mean life expectancy"),  
  title("area * mean: five countries averaged per year"))
```



“Given gapminder Asia: an area derived by mean, x is year, y is life.”

11.7 A category on the domain

The domain takes a category as readily as a number. Each category gets a slot, the boundary runs across them in axis order, and the region fills to the same baseline it always fills to:

R

```
data(gapminder_2007) + area * mean + x(continent) + y(life) +  
  y_label("Mean life expectancy") +  
  title("area * mean + x(continent): the filled profile")
```

Python

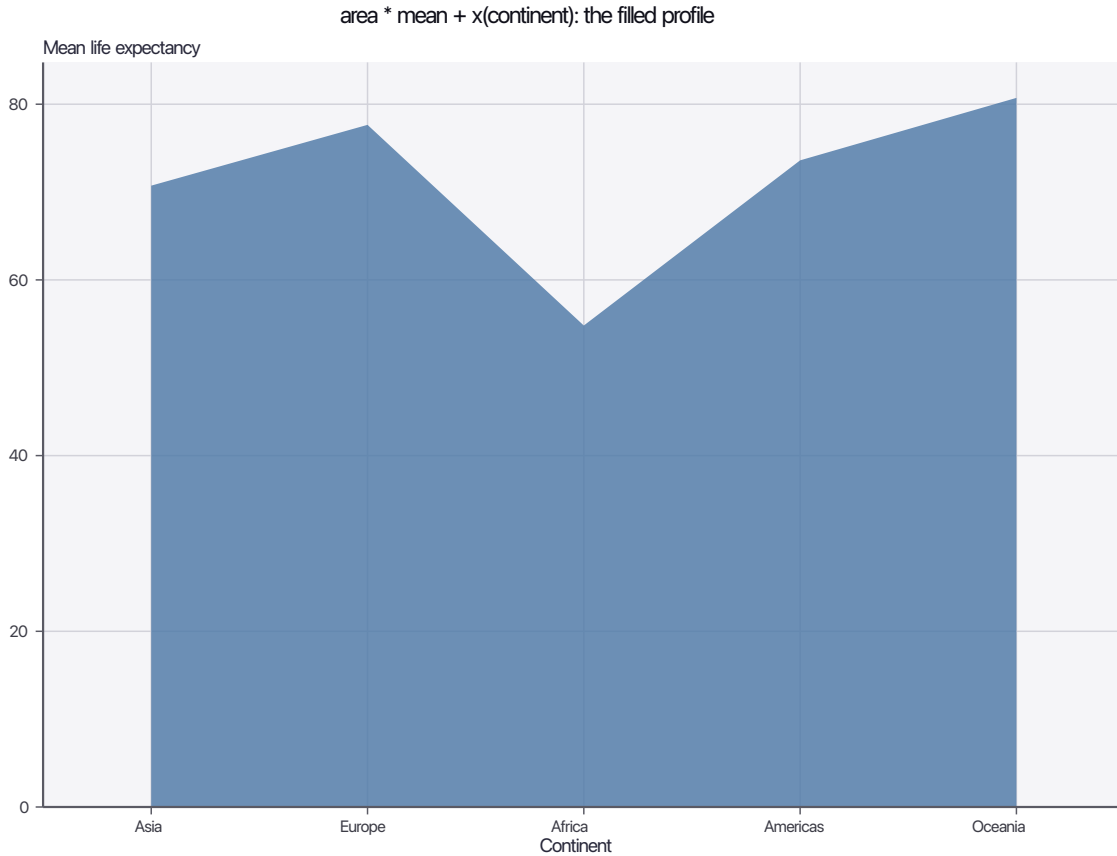
```
(data(gapminder_2007) + area * mean + x(col.continent) + y(col.life) +  
  y_label("Mean life expectancy") +  
  title("area * mean + x(continent): the filled profile"))
```

Julia

```
data(gapminder_2007) + area * mean + x(:continent) + y(:life) +  
  y_label("Mean life expectancy") +  
  title("area * mean + x(continent): the filled profile")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, mean), x(col.continent),  
  y(col.life), y_label("Mean life expectancy"),  
  title("area * mean + x(continent): the filled profile"))
```



Whether that is the right picture is your decision, not the grammar's. The straight segments pass through space where there is no data, and that is a real objection: no continent sits between Africa and the Americas. If you read only the vertices, each segment only joins one value to the next. If you read the segments themselves, they claim a change that was never measured. When the comparison is what matters, the bar chart of the same sentence is the safer reading, and it is one atom away.

What the fill gives you is a *shape*, and a shape can also be read in a circle. Add `polar()` and the same sentence draws the filled radar:

R

```
data(gapminder_2007) + area * mean + x(continent) + y(life) + polar() +
  title("area * mean + polar(): the filled radar")
```

Python

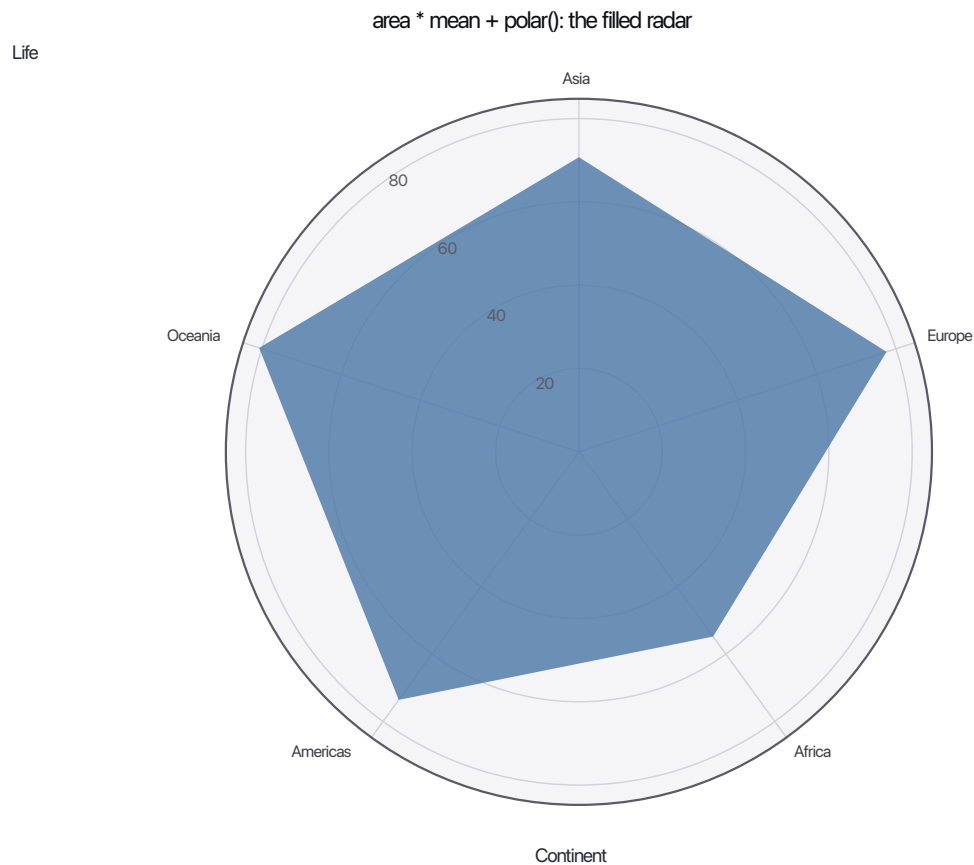
```
(data(gapminder_2007) + area * mean + x(col.continent) + y(col.life) +
  ↪ polar() +
  title("area * mean + polar(): the filled radar"))
```

Julia

```
data(gapminder_2007) + area * mean + x(:continent) + y(:life) + polar() +
  title("area * mean + polar(): the filled radar")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, mean), x(col.continent),
  y(col.life), polar(), title("area * mean + polar(): the filled radar"))
```



“Given gapminder 2007: an area derived by mean, x is continent, y is life, in polar.”

11.8 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(pattern =)</code>	<code>solid</code> , <code>hatch</code> , <code>crosshatch</code> , <code>grid</code> , <code>dots</code>

And these vary per row if you map them to a column instead: `color()` (categories), `pattern()` (categories), `group()` (categories), `play()` (either).

That is the shortest settings list in the book, shared with `ribbon`. `line` has one setting more and `bar` has two, and each gap has a reason. There is no `size`, because the region's extent comes from the data and the baseline. And there is no border, because the edge you would want is a *data curve*, which is already a line:

R

```
data(actuals) + x(year) + y(sales) +  
  area + style(color = "seagreen", pattern = "hatch", opacity = 0.5) +  
  line + style(color = "seagreen", size = 2) +  
  y_label("Sales") + title("A textured fill, and a line for the edge")
```

Python

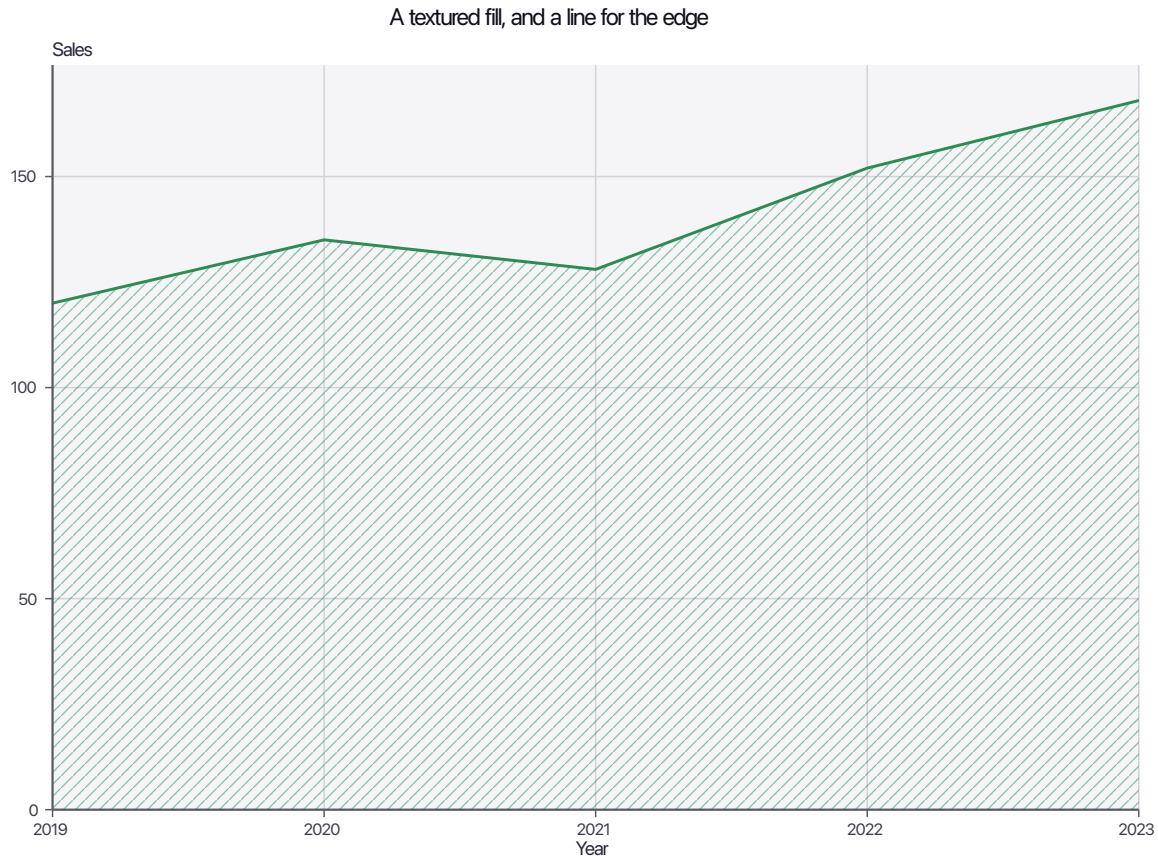
```
(data(actuals) + x(col.year) + y(col.sales) +  
  area + style(color = "seagreen", pattern = "hatch", opacity = 0.5) +  
  line + style(color = "seagreen", size = 2) +  
  y_label("Sales") + title("A textured fill, and a line for the edge"))
```

Julia

```
data(actuals) + x(:year) + y(:sales) + area +  
  style(color = "seagreen", pattern = "hatch", opacity = 0.5) + line +  
  style(color = "seagreen", size = 2) + y_label("Sales") +  
  title("A textured fill, and a line for the edge")
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), area,
     style({ color: "seagreen", pattern: "hatch", opacity: 0.5 }), line,
     style({ color: "seagreen", size: 2 }), y_label("Sales"),
     title("A textured fill, and a line for the edge"))
```



“Given the actuals: x is year, y is sales, an area colored seagreen, with pattern hatch and opacity 0.5, and also a line colored seagreen, with size 2.”

That layered line can be dashed, colored differently, or smoothed, and it traces only the data boundary. A border would also stroke the two sides and the baseline.

11.9 What it refuses

The first refusal comes from the measure axis. A region fills to a baseline at zero, and a category has none. So a categorical y is refused, and the message asks for the category on x instead:


```
render_svg(data(gapminder_2007) + area * mean + x(life) + y(continent))
```

Error:

```
! gog: `y(continent)` maps a categorical (text) column, but `y` on `area`  
  ↳ needs a continuous (numeric) column. On these marks `x` is the domain and  
  ↳ `y` the measure, and a category is not a quantity to measure: a mean of  
  ↳ category names is not a number, and a region has no categorical baseline  
  ↳ to close on. Put the category on `x` instead - `line * mean +  
  ↳ x(<category>) + y(<number>)` is the profile plot, and `area * mean` fills  
  ↳ it. Unlike `bar`/`box`/`interval`, these marks do not read their  
  ↳ orientation off the bindings, because their two axes do not have the same  
  ↳ role.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

The second refusal comes from the domain. `bin` cuts a continuous axis into intervals, and a category is one slot with no width to cut. `density` also needs a continuous axis, so it refuses only when the sentence leaves it no numeric axis, which is why the half violin above draws. Each message names the transform for categories: `count` after `bin`, `proportion` after `density`.

```
render_svg(data(gapminder_2007) + area * bin + x(continent))
```

Error:

```
! gog: `bin` cuts a continuous axis into intervals, and `x(continent)` is  
  ↳ categorical - a category is one slot, with no width to cut. To tally rows  
  ↳ per category, `count` is the transform that does it: `bar * count`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

12 Step

How do you draw a value that holds until it jumps? **step** draws it as a staircase. Where **line** slopes straight from one point to the next, **step** **holds** each value until it changes, then jumps. That is the right shape for anything that does not change between observations: a cumulative count, a survival probability, an interest rate. Each holds one value until the day it changes. It takes the same channels as **line**; only the path between the points differs.

Several fields have their own name for this shape, and one mark draws them all. Statisticians draw the **empirical cumulative distribution function** (the empirical CDF) with it. Medicine draws the **survival curve**, the Kaplan–Meier estimate (Kaplan & Meier, 1958), which steps down at each observed death. Reliability engineering draws the same curve and calls it a hazard or failure plot. Finance draws the **step chart** of a rate that holds until someone changes it. A cumulative count climbs and a survival probability falls; both are this one staircase.

12.1 Histogram outline

step * bin draws a histogram as an **outline** rather than as bars: one stepped line tracing the top of each bin. Split it by **color** for the overlaid outline histogram, one staircase per group, each drawn without a fill:

R

```
data(iris_flowers) + step * bin + x(petal_length) + color(species) +  
  x_label("Petal length (cm)") + title("Petal length by species: step  
  ↪ outline")
```

Python

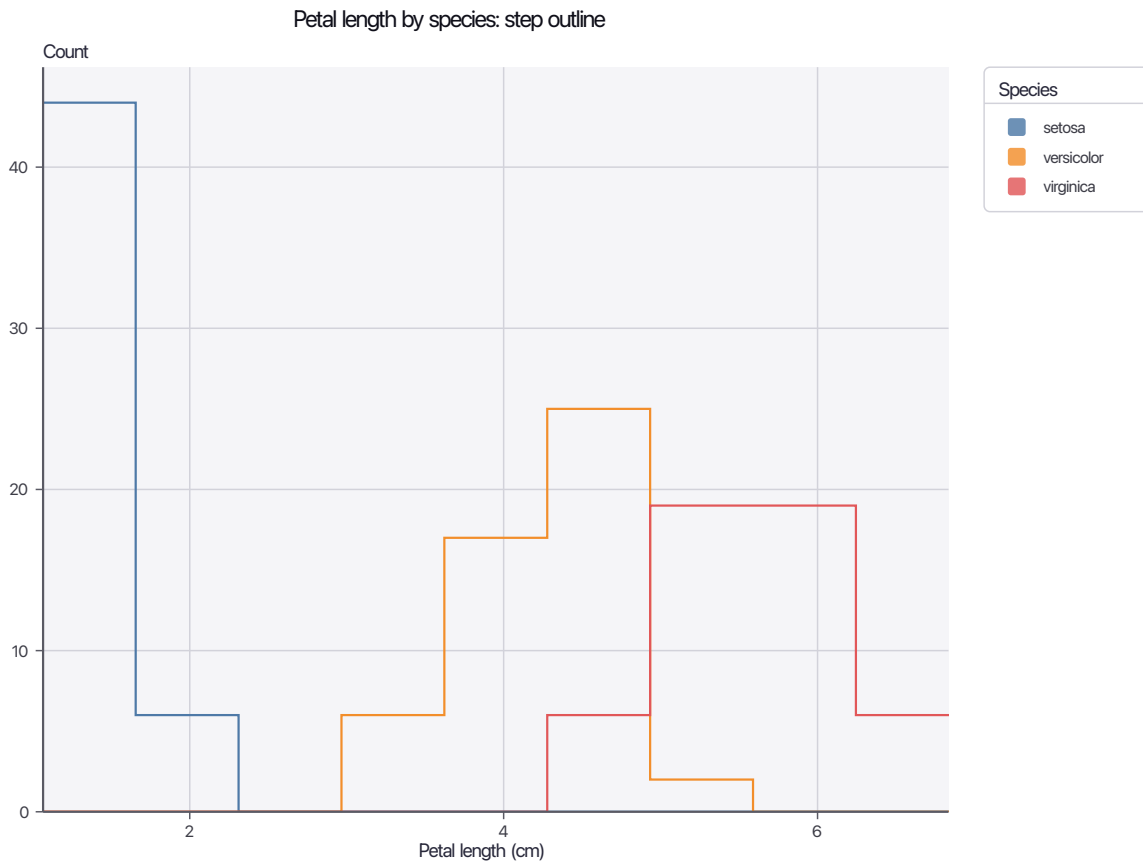
```
(data(iris_flowers) + step * bin + x(col.petal_length) + color(col.species) +  
  x_label("Petal length (cm)") + title("Petal length by species: step  
  ↪ outline"))
```

Julia

```
data(iris_flowers) + step * bin + x(:petal_length) + color(:species) +
  x_label("Petal length (cm)") +
  title("Petal length by species: step outline")
```

JavaScript

```
plot(data(iris_flowers), layer(step, bin), x(col.petal_length),
  color(col.species), x_label("Petal length (cm)",
  title("Petal length by species: step outline"))
```



“Given the iris flowers: step outlines derived by bin, x is petal length, color by species.”

Only the mark changes when you draw a binned distribution. These three are the ones you will use most:

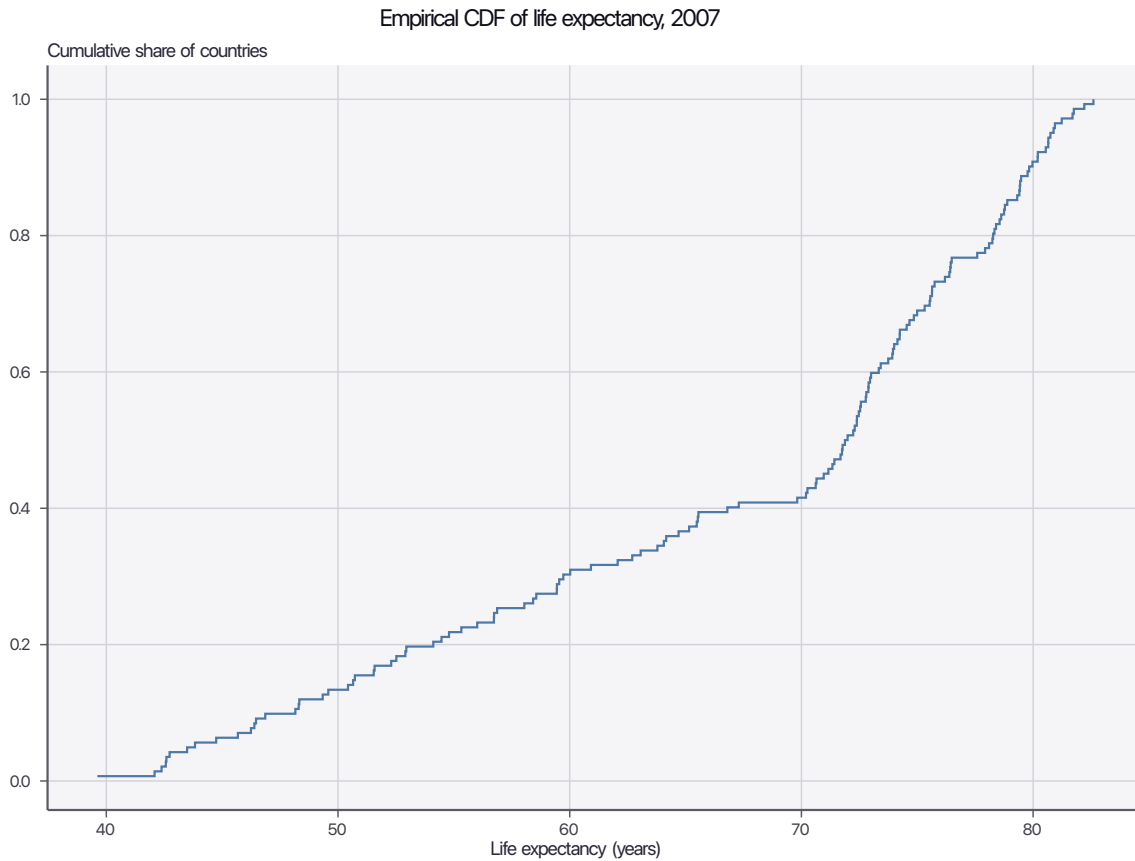
Mark derived by <code>bin</code>	Shape
<code>bar * bin</code>	filled bars
<code>line * bin</code>	frequency polygon: straight lines through the bin centers
<code>step * bin</code>	the staircase outline above

All three species are cut on the same bins, so their outlines share the same edges. Each drops to the axis where its species has no data. Because a step is a single unfilled stroke, each group's shape stays readable through the others, and no opacity has to be set.

12.2 A step function

Give `step` an `x` and a `y` directly and it draws a step function. An empirical CDF is the best-known example: with n observations it climbs by $1/n$ at each observation, and stays flat between them:

```
v <- sort(gapminder_2007$life)
ecdf_df <- data.frame(life = v, share = seq_along(v) / length(v))
data(ecdf_df) + step + x(life) + y(share) +
  x_label("Life expectancy (years)") + y_label("Cumulative share of
  ↪ countries") +
  title("Empirical CDF of life expectancy, 2007")
```



“Given the ecdf table: a step outline, x is life, y is share.”

A line on this same table would slope between points, implying cumulative shares at life expectancies no country has. The step is exact: the share stayed at that value until the next observation.

12.3 A value that holds until it changes

The same exactness matters for anything that jumps rather than changing gradually: a policy rate, a price, an inventory level. Each holds its value, then jumps on a single day; a straight line would draw a gradual change that never happened:

R

```

rates <- data.frame(
  year = c(2018, 2019, 2020, 2021, 2022, 2023),
  rate = c(1.50, 2.25, 0.25, 0.25, 1.75, 4.50)
)
data(rates) + step + x(year) + y(rate) +
  x_label("Year") + y_label("Policy rate (%)") +
  title("A rate holds, then jumps")

```

Python

```

rates = {"year": [2018, 2019, 2020, 2021, 2022, 2023], "rate": [1.50, 2.25,
↪ 0.25, 0.25, 1.75, 4.50]}
(data(rates) + step + x(col.year) + y(col.rate) +
  x_label("Year") + y_label("Policy rate (%)") +
  title("A rate holds, then jumps"))

```

Julia

```

rates = (year = [2018, 2019, 2020, 2021, 2022, 2023], rate = [1.5, 2.25,
↪ 0.25, 0.25, 1.75, 4.5],)
data(rates) + step + x(:year) + y(:rate) + x_label("Year") +
  y_label("Policy rate (%)") + title("A rate holds, then jumps")

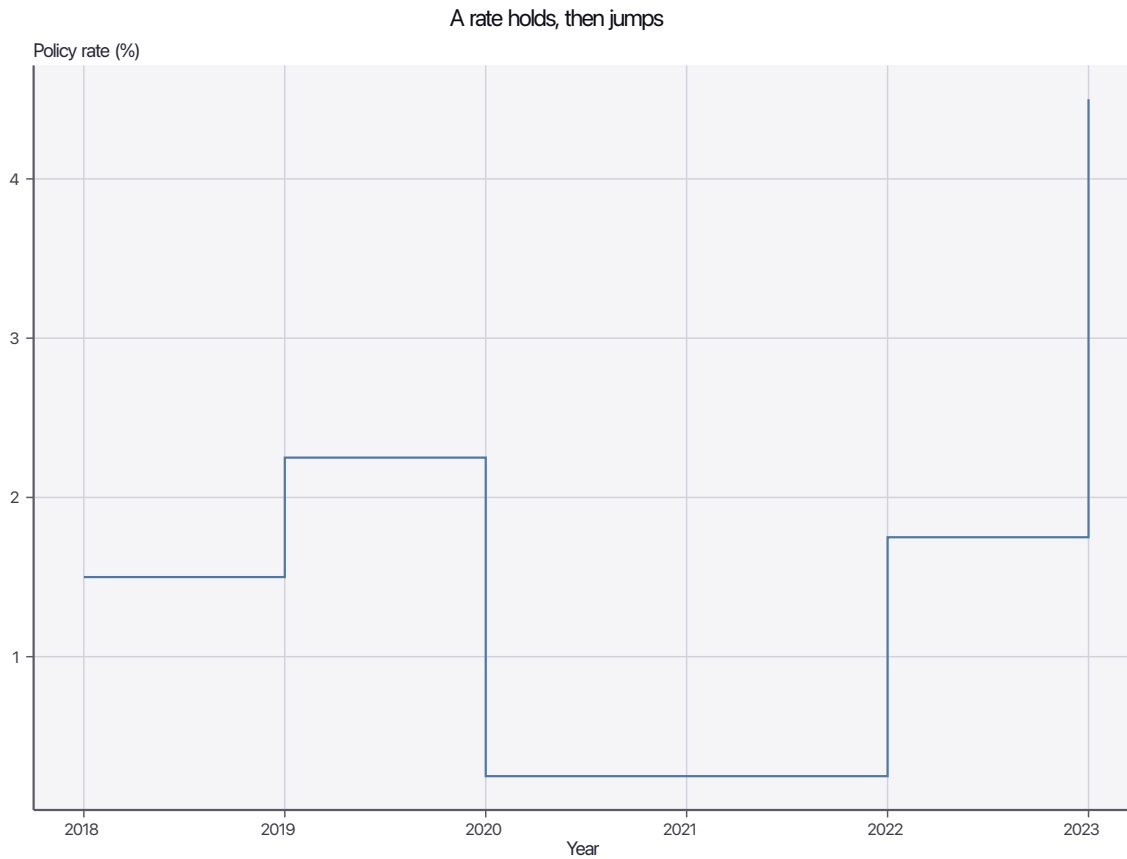
```

JavaScript

```

const rates = { year: [2018, 2019, 2020, 2021, 2022, 2023], rate: [1.5, 2.25,
↪ 0.25, 0.25, 1.75, 4.5] };
plot(data(rates), step, x(col.year), y(col.rate), x_label("Year"),
  y_label("Policy rate (%)"), title("A rate holds, then jumps"))

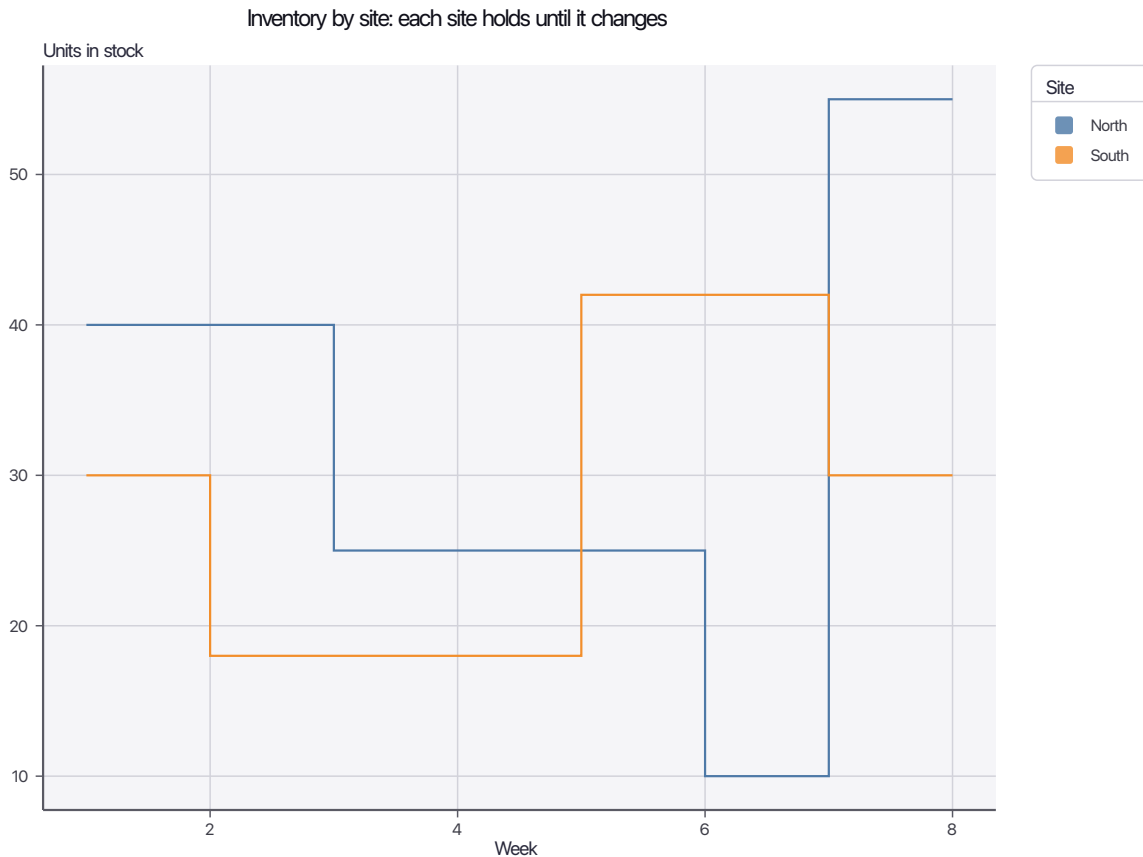
```



12.4 Multiple series

color and group split a step exactly as they split a line: one staircase per category. Here two series hold and jump independently:

```
inventory <- data.frame(
  week = rep(1:8, 2),
  units = c(40, 40, 25, 25, 25, 10, 55, 55,
            30, 18, 18, 18, 42, 42, 30, 30),
  site = rep(c("North", "South"), each = 8)
)
data(inventory) + step + x(week) + y(units) + color(site) +
  x_label("Week") + y_label("Units in stock") +
  title("Inventory by site: each site holds until it changes")
```



12.5 A category on the domain

`step` and `line` share one channel row: whatever one requires, accepts or refuses, the other does too. They differ in how the path is drawn, never in what you may bind. So a category on the domain works here as it does in [Line](#), and the staircase holds each category's value across its own slot:

R

```
data(gapminder_2007) + step * mean + x(continent) + y(life) +
  y_label("Mean life expectancy") +
  title("step * mean + x(continent): one value per category, held")
```

Python

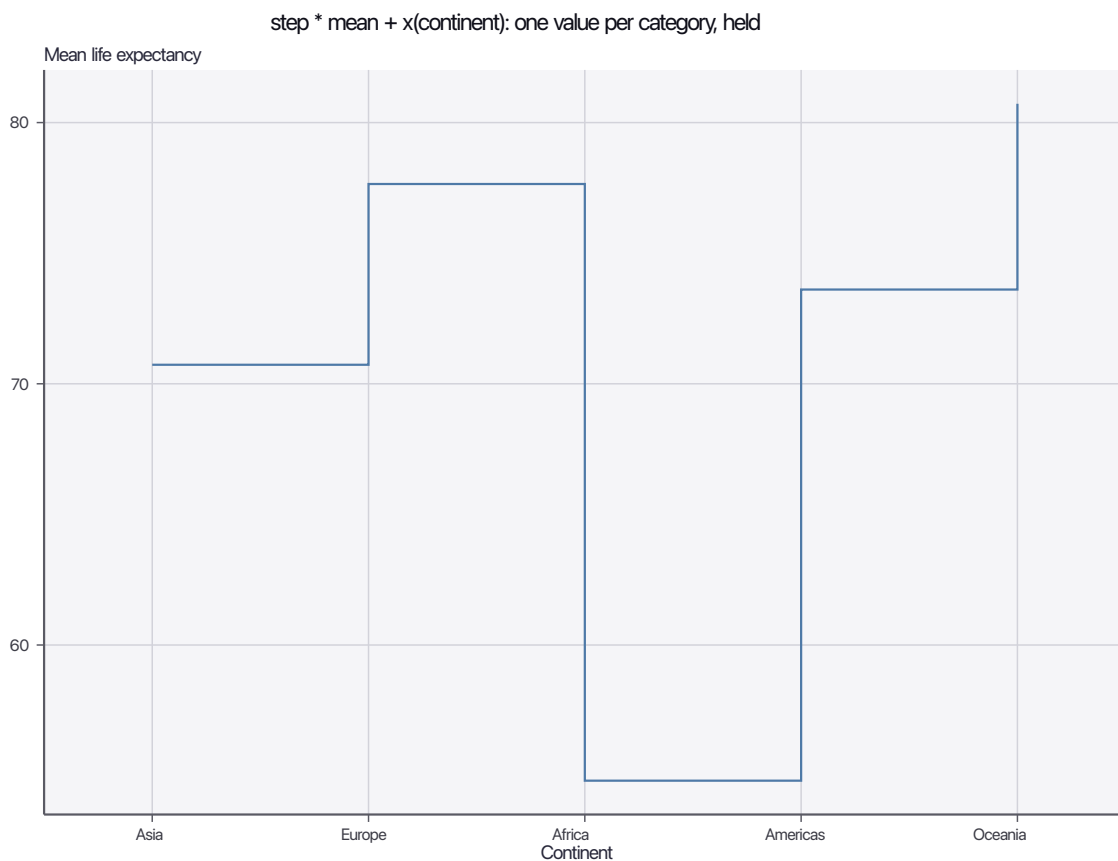

```
(data(gapminder_2007) + step * mean + x(col.continent) + y(col.life) +
  y_label("Mean life expectancy") +
  title("step * mean + x(continent): one value per category, held"))
```

Julia

```
data(gapminder_2007) + step * mean + x(:continent) + y(:life) +
  y_label("Mean life expectancy") +
  title("step * mean + x(continent): one value per category, held")
```

JavaScript

```
plot(data(gapminder_2007), layer(step, mean), x(col.continent),
  y(col.life), y_label("Mean life expectancy"),
  title("step * mean + x(continent): one value per category, held"))
```



“Given gapminder 2007: a step outline derived by mean, x is continent, y is life.”

Of **line** and **step**, the **step** claims less about what was measured. A **line** slopes between two categories, so it draws a change through values nothing recorded. A **step** holds flat and then jumps, which fits separate groups better. The [value that holds](#) section made this argument about time. It applies to categories in the same way.

12.6 Stroke width and opacity

Like **line**, a **step** is a single stroke per series, so **size** and **opacity** are not channels; they are settings on the whole stroke:

R

```
data(iris_flowers) + step * bin + x(petal_length) + color(species) +  
  style(size = 2.5, opacity = 0.7) +  
  x_label("Petal length (cm)") + title("A thicker, more transparent outline")
```

Python

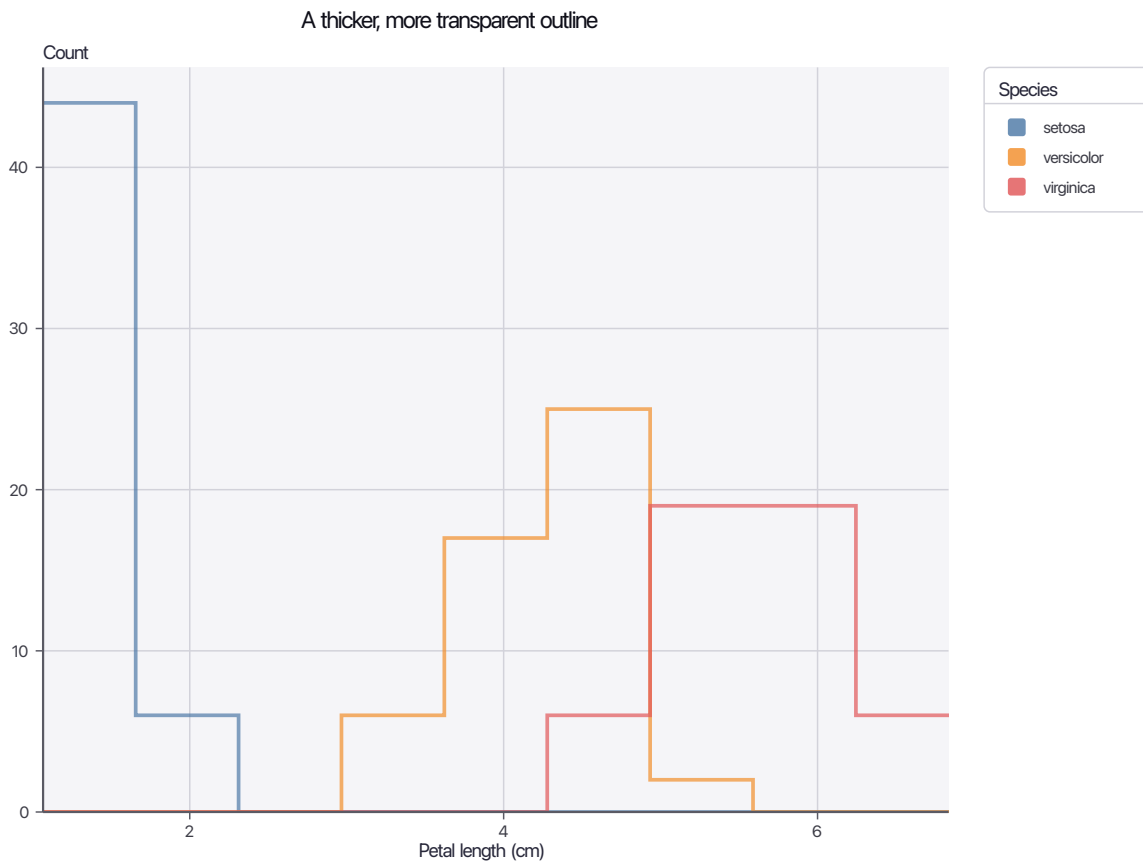
```
(data(iris_flowers) + step * bin + x(col.petal_length) + color(col.species) +  
  style(size = 2.5, opacity = 0.7) +  
  x_label("Petal length (cm)") + title("A thicker, more transparent  
↪ outline"))
```

Julia

```
data(iris_flowers) + step * bin + x(:petal_length) + color(:species) +  
  style(size = 2.5, opacity = 0.7) + x_label("Petal length (cm)") +  
  title("A thicker, more transparent outline")
```

JavaScript

```
plot(data(iris_flowers), layer(step, bin), x(col.petal_length),  
  color(col.species), style({ size: 2.5, opacity: 0.7 }),  
  x_label("Petal length (cm)"),  
  title("A thicker, more transparent outline"))
```



“Given the iris flowers: step outlines derived by bin, x is petal length, color by species, with size 2.5 and opacity 0.7.”

size sets the stroke width; **opacity** fades the whole staircase. Both follow the same *setting vs mapping* rule as **line**; see [Setting vs mapping](#).

12.7 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(pattern =)</code>	solid, dashed, dotted

And these vary per row if you map them to a column instead: `color()` (either), `pattern()` (categories), `group()` (categories), `play()` (either).

A step is a stroke, so it takes exactly what `line` takes: one color, one width, one opacity and one dash for the whole staircase. The six weeks table holds 42 consecutive days of `orders`, with `day` a real date:

Table 12.3: `six_weeks`: first 5 of 42 rows

day	orders	weekday	week
2024-03-01	20	Fri	Week 1
2024-03-02	23	Sat	Week 1
2024-03-03	25	Sun	Week 1
2024-03-04	28	Mon	Week 1
2024-03-05	30	Tue	Week 1

R

```
data(six_weeks) + step + x(day) + y(orders) +  
  style(color = "purple", size = 2.5, pattern = "dotted") +  
  y_label("Orders") + title("A dotted staircase")
```

Python

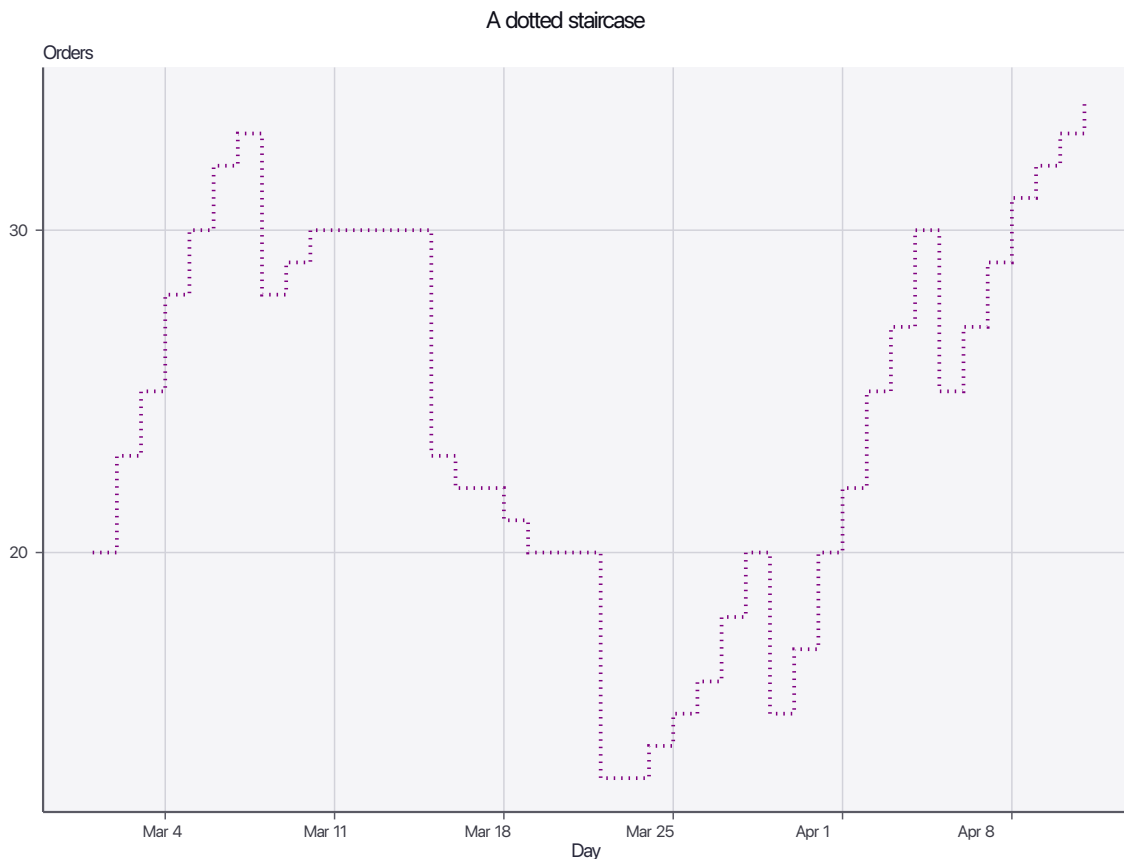
```
(data(six_weeks) + step + x(col.day) + y(col.orders) +  
  style(color = "purple", size = 2.5, pattern = "dotted") +  
  y_label("Orders") + title("A dotted staircase"))
```

Julia

```
data(six_weeks) + step + x(:day) + y(:orders) +  
  style(color = "purple", size = 2.5, pattern = "dotted") +  
  y_label("Orders") + title("A dotted staircase")
```

JavaScript

```
plot(data(six_weeks), step, x(col.day), y(col.orders),  
  style({ color: "purple", size: 2.5, pattern: "dotted" }),  
  y_label("Orders"), title("A dotted staircase"))
```



“Given the six weeks table: a step outline, x is day, y is orders, colored purple, with size 2.5 and pattern dotted.”

The staircase and the dash change different things. The staircase is *geometry*: it decides where the path goes, and puts a corner where the value changes. The dash is *paint*: it changes how the path looks, never where it goes. That is why **step** is a mark and **pattern** is a setting.

12.8 What it refuses

A step is one polyline (a single connected stroke), and that explains the two refusals below. The third is **line's**: a category on **y** is refused, because **y** carries a quantity.

One stroke has one width (there is no answer to “how wide is the staircase *here*”), so a width cannot vary along it:

```
data(gapminder_asia) + step + x(year) + y(life) + size(gdp)
```

Error:

```
! gog: `size` cannot be bound to `step` - a step has no size feature. Remove
  ↳ the `gdp` mapping from `size`, or use a mark that has one.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The stroke still *has* a width, which is why the setting above works and the mapping does not. That distinction is the subject of [Setting vs mapping](#).

The second refusal is about which patterns a stroke accepts. A hatch is a fill's texture, and a staircase is line-work with nothing inside it:

```
data(gapminder_asia) + step + x(year) + y(life) + style(pattern = "hatch")
```

Error:

```
! gog: `style(pattern = "hatch")` is not a stroke pattern. Use "solid" (the
  ↳ default), "dashed", or "dotted".
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A color is different. A **tread** is a flat section held at one value, and a **riser** is the jump between two values. Point anywhere on the staircase and that place has a color, so a measure can vary along it. Each tread takes its row's color, and each riser blends the colors of the two rows it joins:

R

```
data(gapminder_asia) + step + x(year) + y(life) + group(country) +
  color(gdp, scale = "log") + palette("viridis") +
  y_label("Life expectancy") + title("Each country's staircase, colored by
  ↳ its income")
```

Python

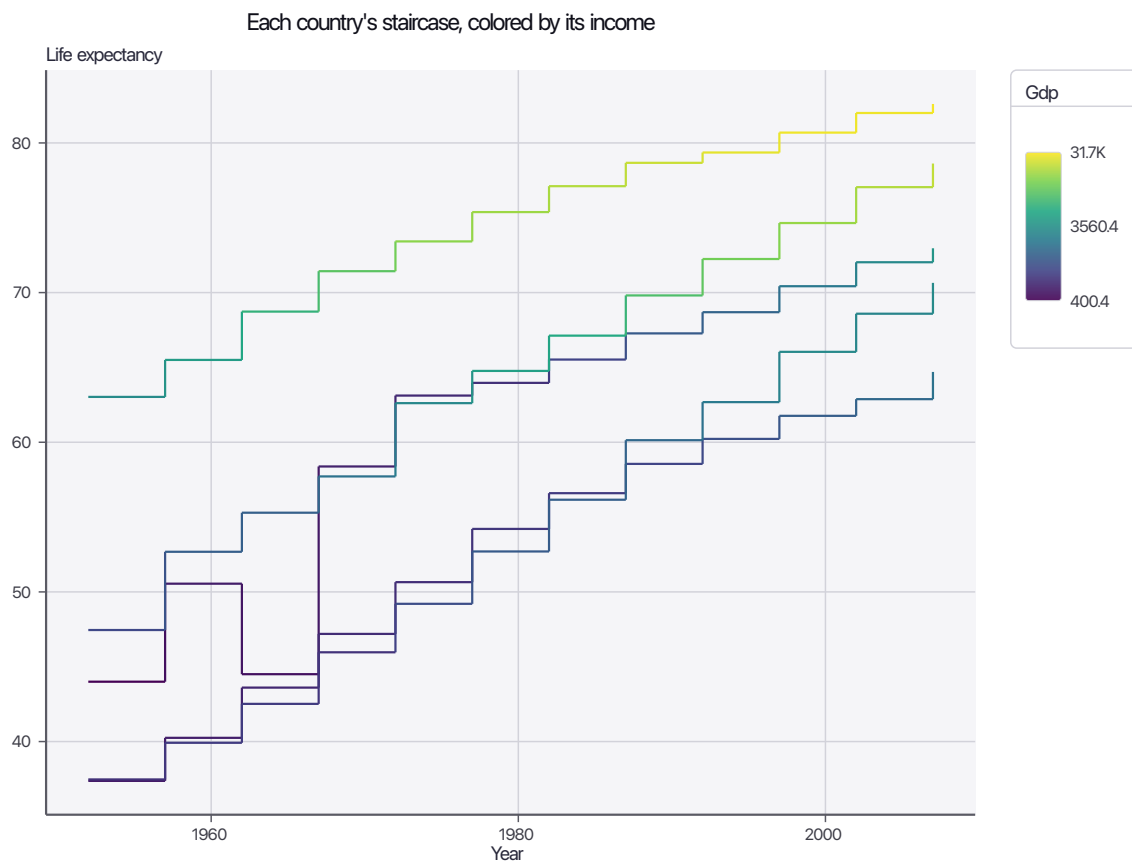
```
(data(gapminder_asia) + step + x(col.year) + y(col.life) + group(col.country)
  ↳ +
  color(col.gdp, scale = "log") + palette("viridis") +
  y_label("Life expectancy") + title("Each country's staircase, colored by
  ↳ its income"))
```

Julia

```
data(gapminder_asia) + step + x(:year) + y(:life) + group(:country) +
  color(:gdp, scale = "log") + palette("viridis") +
  y_label("Life expectancy") +
  title("Each country's staircase, colored by its income")
```

JavaScript

```
plot(data(gapminder_asia), step, x(col.year), y(col.life),
  group(col.country), color(col.gdp, { scale: "log" }),
  palette("viridis"), y_label("Life expectancy"),
  title("Each country's staircase, colored by its income"))
```



“Given gapminder Asia: step outlines, x is year, y is life, grouped by country, color by gdp on a log scale.”

13 Interval

How far does each group reach, from its lowest value to its highest? `interval` draws a span from a low value to a high one at each position: a whisker, capped at both ends. It is the mark behind error bars and min-max ranges. A `bar` measures from a baseline up to a value. An `interval` spans a low value and a high value instead: it has no baseline, and it needs both before it can draw. Whiskers stand up by default and lie down when the categories are on y; see [Whiskers on their side](#).

Almost everyone calls this an **error bar**, and the name hides a choice no drawing can show. Three different quantities get drawn as the same picture: a standard deviation, a standard error, and a confidence interval. On the same data, one can be several times longer than another. Nothing in the drawing says which one it is, so a reader without the caption is guessing at a number's meaning.

`gog` makes the sentence name the statistic instead. `interval * range` spans the observed extremes, `interval * confidence(0.95)` computes the interval and names its level, and `interval * bounds(lower, upper)` takes two columns you computed yourself. The mark is the same whisker in all three; what differs is the transform, and the transform is written down. That is the law of [Explicit Over Implicit](#), shown in one mark: the name *error bar* can hide the choice, and a written sentence cannot.

13.1 A range per group

The low value and the high value come from a transform, not from two extra channels. `range` reduces y to its minimum and maximum within each x group, and `interval` spans between them:

R

```
data(iris_flowers) + interval * range + x(species) + y(petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Range of petal length per species")
```

Python

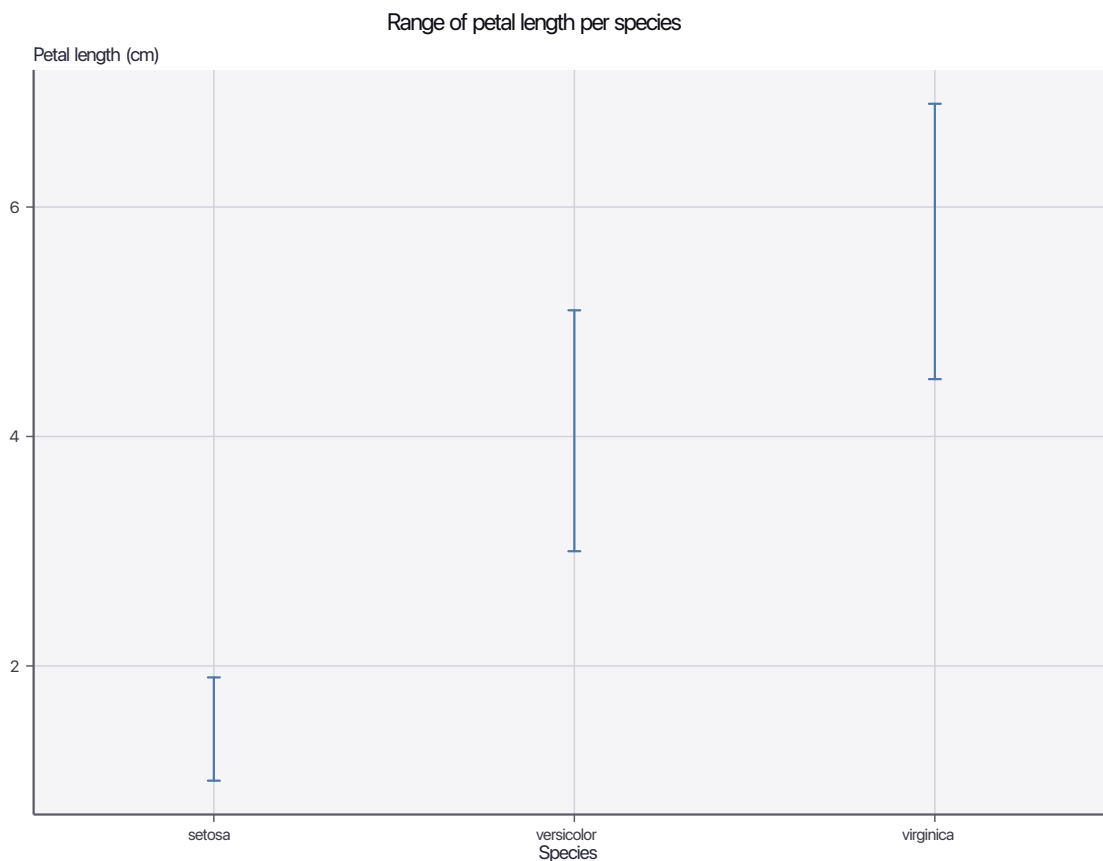

```
(data(iris_flowers) + interval * range + x(col.species) + y(col.petal_length)
↪ +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Range of petal length per species"))
```

Julia

```
data(iris_flowers) + interval * range + x(:species) + y(:petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Range of petal length per species")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, range), x(col.species),
  y(col.petal_length), x_label("Species"), y_label("Petal length (cm)"),
  title("Range of petal length per species"))
```



“Given the iris flowers: intervals derived by range, x is species, y is petal length.”

Read it exactly like `bar * mean`. Here the transform gives a range, while `mean` gives one number. You name the one variable whose spread you are asking about; the transform invents the two extents and the mark spans them.

13.2 The middle half of each group

A minimum and a maximum each come from a single row, so one unusual flower can set the whole whisker. To show the middle of a group instead of its extremes, give `range` two quantiles:

R

```
data(iris_flowers) + interval * range(0.25, 0.75) + x(species) +  
  ↪ y(petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Middle half of petal length per species")
```

Python

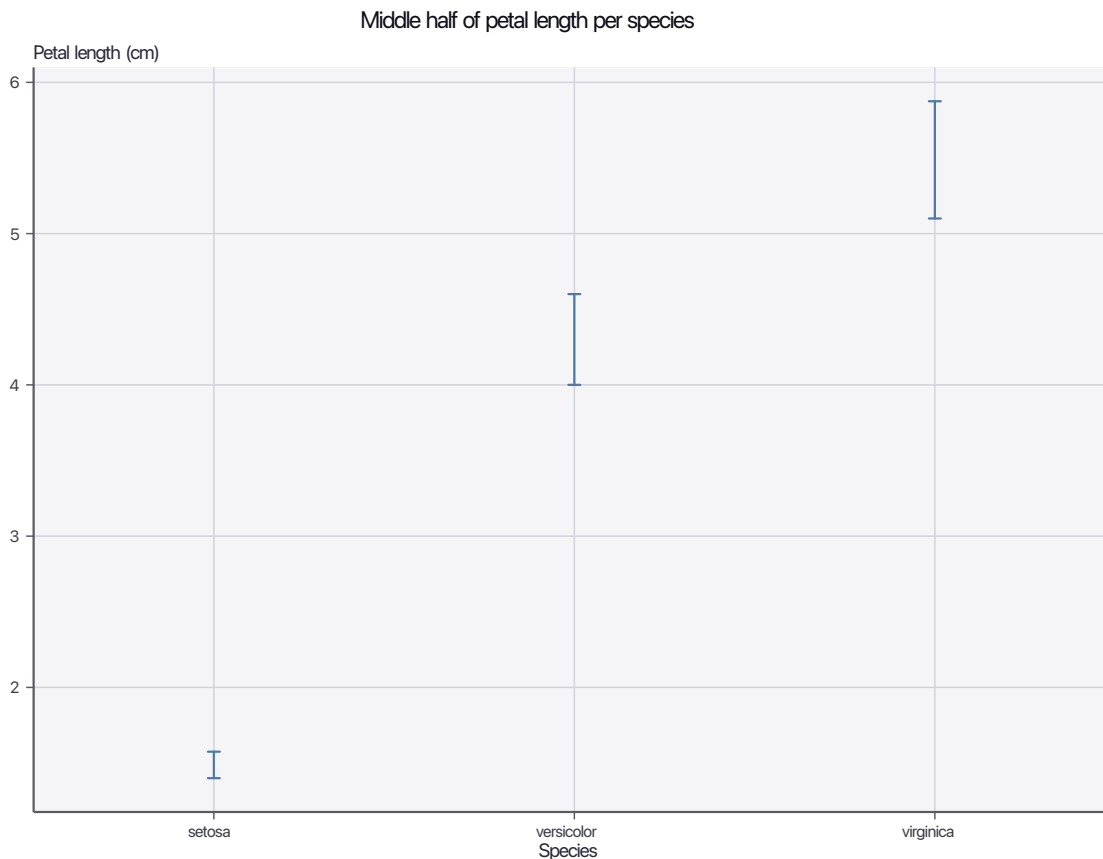
```
(data(iris_flowers) + interval * range(0.25, 0.75) + x(col.species) +  
  ↪ y(col.petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Middle half of petal length per species"))
```

Julia

```
data(iris_flowers) + interval * range(0.25, 0.75) + x(:species) +  
  y(:petal_length) + x_label("Species") + y_label("Petal length (cm)") +  
  title("Middle half of petal length per species")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, range(0.25, 0.75)),  
  x(col.species), y(col.petal_length), x_label("Species"),  
  y_label("Petal length (cm)"),  
  title("Middle half of petal length per species"))
```



“Given the iris flowers: intervals derived by range from 0.25 to 0.75, x is species, y is petal length.”

This spans the first quartile to the third, which is the **interquartile range**. `range(0.1, 0.9)` keeps the middle 80%, and any other pair works the same way. Bare `range` is `range(0, 1)`, so the plain sentence above is unchanged. [Transforms](#) gives the full account of the two arguments.

13.3 Three layers instead of one mark

A [box](#) draws five numbers in one mark, and you take all five or none. Draw the band and the full range as two layers, and the median as a third. You can restyle any of the three layers, drop one, or replace one:

R

```

data(iris_flowers) +
  interval * range + style(color = "lightgray") +
  interval * range(0.25, 0.75) + style(color = "steelblue", size = 8) +
  point * median + style(color = "white", size = 3) +
  x(species) + y(petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Petal length per species, drawn in three layers")

```

Python

```

(data(iris_flowers) +
  interval * range + style(color = "lightgray") +
  interval * range(0.25, 0.75) + style(color = "steelblue", size = 8) +
  point * median + style(color = "white", size = 3) +
  x(col.species) + y(col.petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Petal length per species, drawn in three layers"))

```

Julia

```

data(iris_flowers) + interval * range + style(color = "lightgray") +
  interval * range(0.25, 0.75) + style(color = "steelblue", size = 8) +
  point * median + style(color = "white", size = 3) + x(:species) +
  y(:petal_length) + x_label("Species") + y_label("Petal length (cm)") +
  title("Petal length per species, drawn in three layers")

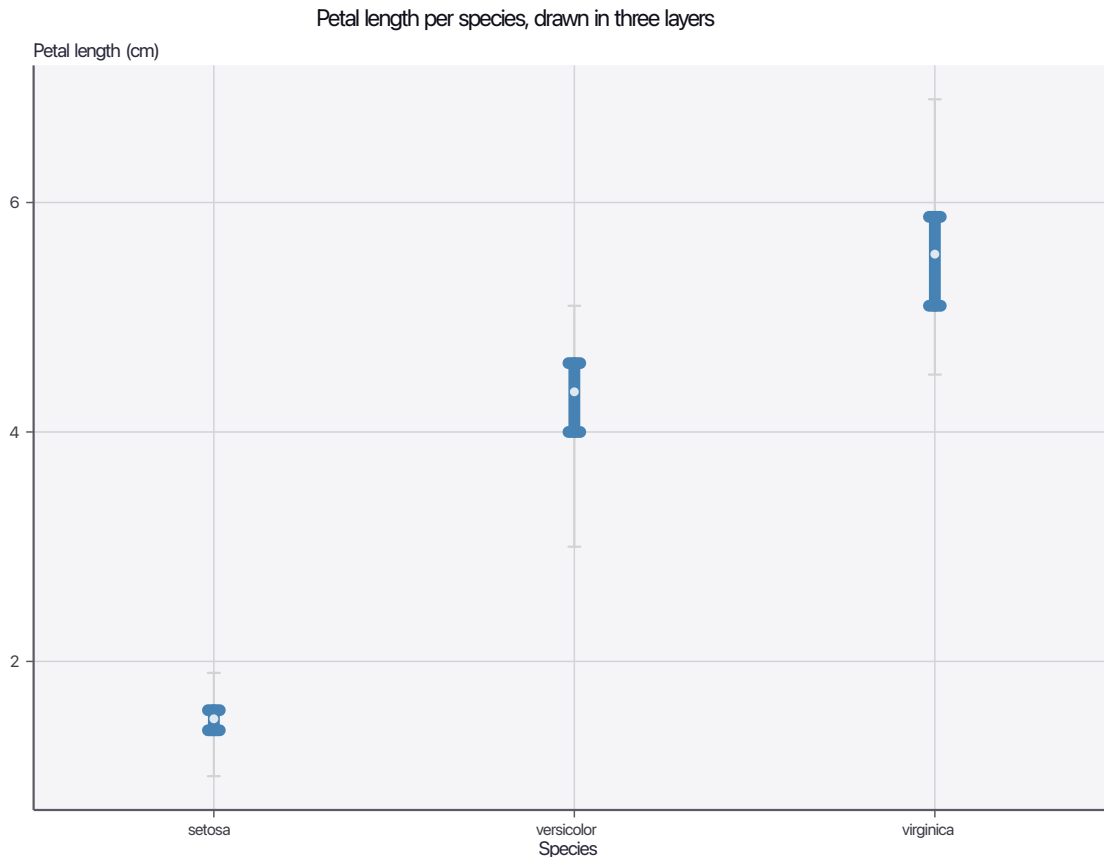
```

JavaScript

```

plot(data(iris_flowers), layer(interval, range),
  style({ color: "lightgray" }), layer(interval, range(0.25, 0.75)),
  style({ color: "steelblue", size: 8 }), layer(point, median),
  style({ color: "white", size: 3 }), x(col.species), y(col.petal_length),
  x_label("Species"), y_label("Petal length (cm)"),
  title("Petal length per species, drawn in three layers"))

```



“Given the iris flowers: intervals derived by range, and also intervals derived by range from 0.25 to 0.75, and also points derived by median, x is species, y is petal length.”

The gray whisker is the full range, the thick blue band is the middle half, and the white dot is the median. The three layers name the three statistics. A box plot draws the same three and names none of them.

This is not a **box** redrawn, and the difference is at the ends. A **box** stops its whiskers at a cutoff computed from the quartiles, and draws every point beyond that cutoff as an outlier. Drawn as a box, setosa shows four outlier dots that are absent here, because these whiskers run to the true minimum and maximum. What the layers give you instead is a plot you can change. Widen the band to `range(0.1, 0.9)` and the plot reports a different statistic, with the same marks. Drop the gray layer and the y axis covers the band rather than the extremes. Add a third layer with another pair of quantiles, and you have the [fan chart](#) drawn over categories instead of along a line. A **box** is still the shorter sentence when you want the five numbers and the outlier rule together.

13.4 A confidence interval, with a center

`range` shows the full spread. For the **uncertainty of the mean**, use `confidence`. It computes a t-interval for each group's mean (a t-interval widens when the group is small) and keeps that mean as the center. The interval then draws a dot at that center: a pointrange.

R

```
data(iris_flowers) + interval * confidence(0.95) + x(species) +  
  ↪ y(petal_length) +  
    x_label("Species") + y_label("Petal length (cm)") +  
    title("95% CI of mean petal length")
```

Python

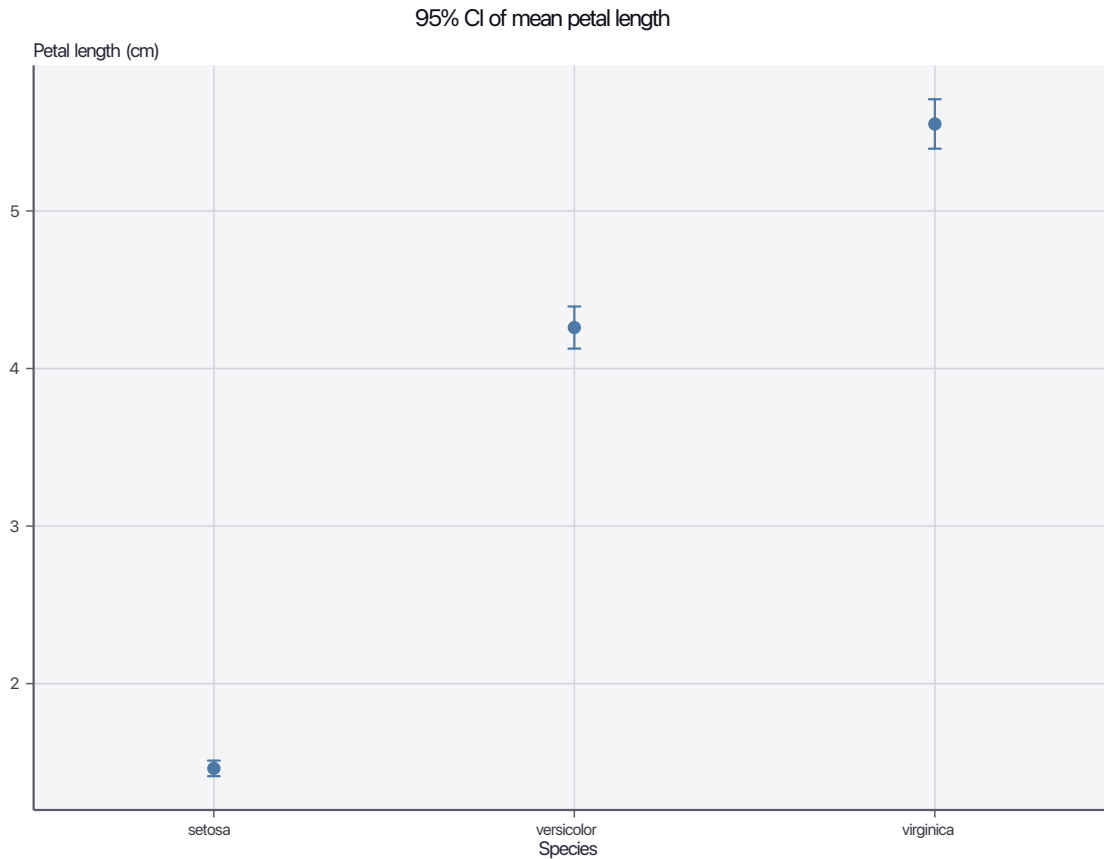
```
(data(iris_flowers) + interval * confidence(0.95) + x(col.species) +  
  ↪ y(col.petal_length) +  
    x_label("Species") + y_label("Petal length (cm)") +  
    title("95% CI of mean petal length"))
```

Julia

```
data(iris_flowers) + interval * confidence(0.95) + x(:species) +  
  y(:petal_length) + x_label("Species") + y_label("Petal length (cm)") +  
  title("95% CI of mean petal length")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, confidence(0.95)),  
  x(col.species), y(col.petal_length), x_label("Species"),  
  y_label("Petal length (cm)", title("95% CI of mean petal length"))
```



“Given the iris flowers: intervals derived by confidence at 95%, x is species, y is petal length.”

The level is a parameter, so `confidence(0.99)` draws wider than `confidence(0.95)`.

13.5 The spread of the data

The opening of this chapter named a standard deviation, a standard error and a confidence interval. `confidence` draws the third. The first is `deviation`:

R

```
data(iris_flowers) + interval * deviation + x(species) + y(petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Mean petal length, with one standard deviation")
```

Python

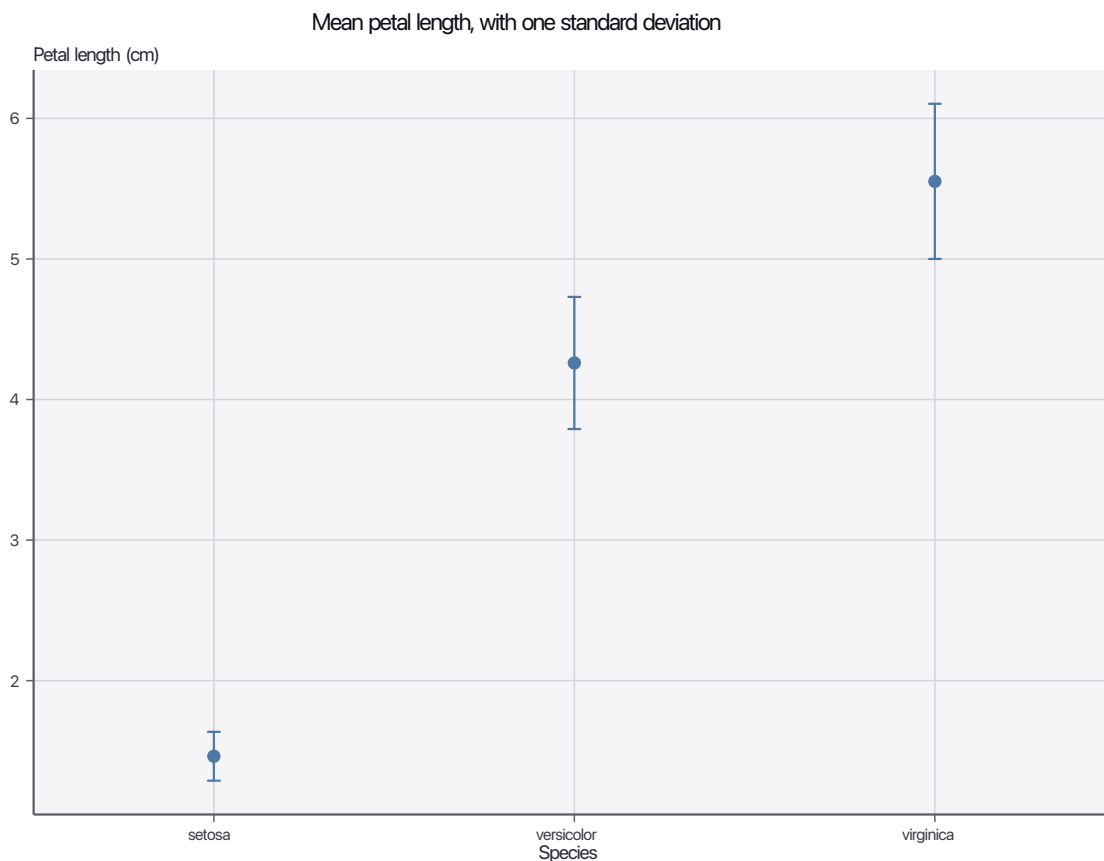
```
(data(iris_flowers) + interval * deviation + x(col.species) +
  ↪ y(col.petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Mean petal length, with one standard deviation"))
```

Julia

```
data(iris_flowers) + interval * deviation + x(:species) +
  y(:petal_length) + x_label("Species") + y_label("Petal length (cm)") +
  title("Mean petal length, with one standard deviation")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, deviation), x(col.species),
  y(col.petal_length), x_label("Species"), y_label("Petal length (cm)"),
  title("Mean petal length, with one standard deviation"))
```



“Given the iris flowers: intervals derived by deviation, x is species, y is petal length.”

`deviation(2)` reaches two standard deviations on each side of the mean, so that band is four standard deviations wide. The default is one each side. Like `confidence` it carries a center, so it draws a pointrange rather than a bare whisker.

The standard error has no transform of its own, and that is deliberate. `confidence` computes it internally and reports the interval, which is what a reader needs from it.

`deviation` and `confidence` look alike and answer different questions, so here they are on the same fifty flowers per species, sharing one y scale:

R

```
(data(iris_flowers) + interval * deviation + x(species) + y(petal_length) +  
  y_label("Petal length (cm)") + title("Spread of the data")) |  
  (data(iris_flowers) + interval * confidence + x(species) + y(petal_length)  
↪ +  
  y_label("") + title("Uncertainty of the mean"))
```

Python

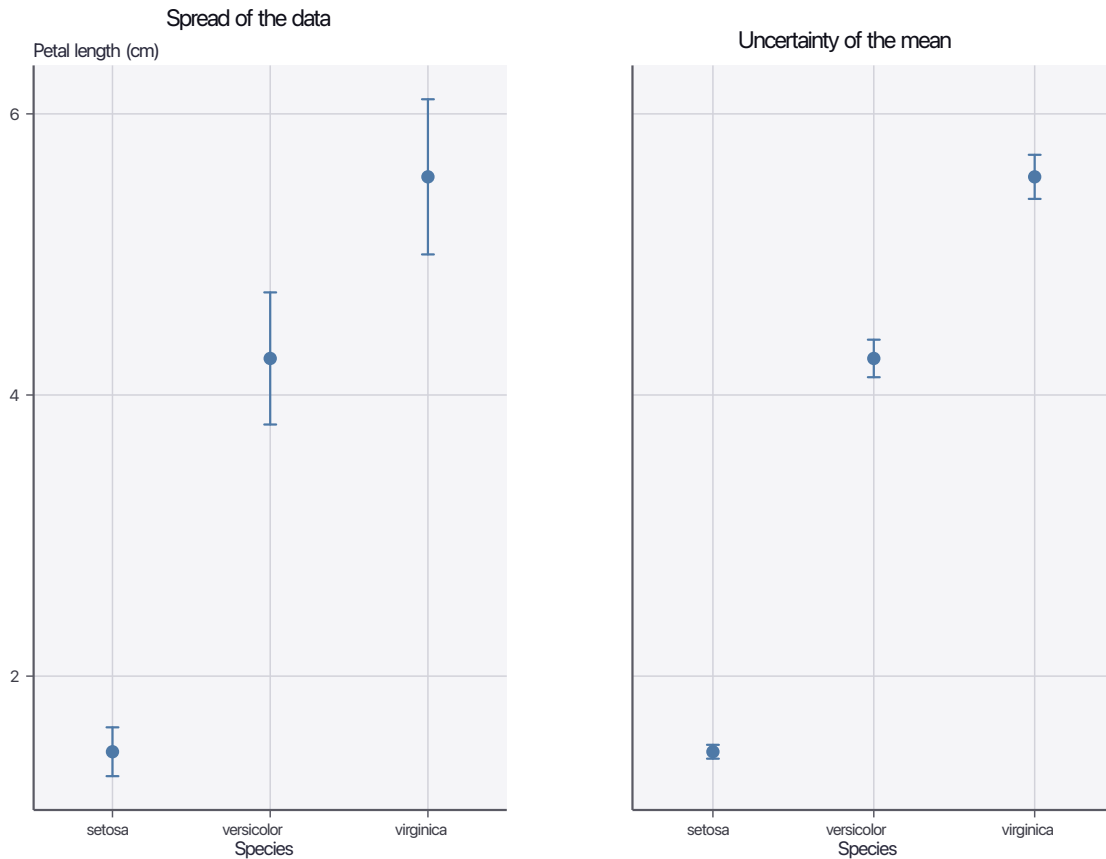
```
((data(iris_flowers) + interval * deviation + x(col.species) +  
↪ y(col.petal_length) +  
  y_label("Petal length (cm)") + title("Spread of the data")) |  
  (data(iris_flowers) + interval * confidence + x(col.species) +  
↪ y(col.petal_length) +  
  y_label("") + title("Uncertainty of the mean")))
```

Julia

```
(data(iris_flowers) + interval * deviation + x(:species) +  
  y(:petal_length) + y_label("Petal length (cm)") +  
  title("Spread of the data")) |  
  (data(iris_flowers) + interval * confidence + x(:species) +  
  y(:petal_length) + y_label("") + title("Uncertainty of the mean"))
```

JavaScript

```
beside(plot(data(iris_flowers), layer(interval, deviation),  
  x(col.species), y(col.petal_length), y_label("Petal length (cm)"),  
  title("Spread of the data")),  
  plot(data(iris_flowers), layer(interval, confidence), x(col.species),  
  y(col.petal_length), y_label(""), title("Uncertainty of the mean")))
```



“Given the iris flowers: intervals derived by deviation, x is species, y is petal length, beside intervals derived by confidence, x is species, y is petal length.”

The bands on the right are less than a third as long, on identical data, and the ratio is the same in all three species. The two numbers do not contradict each other. **deviation** describes how far the flowers are from their average, and measuring more of them does not make that band shorter. **confidence** describes how precisely the average itself is known. More flowers make it shorter: four times as many flowers make it half as long. Fifty flowers per species is what sets that ratio here.

So choose by the sentence you would write under the plot. “A *versicolor* petal is usually within half a centimeter of its species average” describes the left panel. “The average *versicolor* petal is 4.26 cm, and the interval reaches 0.13 cm on either side” describes the right one. Reporting one and meaning the other is the mistake the name *error bar* allows, and the reason **gog** asks you to name the statistic.

Which band is wider is not fixed, either. **confidence** narrows as the group gets larger. On fifty flowers it is under a third of the spread; on three flowers it is the wider of the two.

Whether a center exists is decided by the statistic, not by a setting. `confidence` and `deviation` have a mean and `range` does not, so the statistic chooses the geometry: a bare error bar or a pointrange. A setting can *hide* an existing center (`below`), but no setting adds one to `range`. See [Transforms](#) for the whole range family.

13.6 Why interval needs a transform

`bar` needs no transform, `bar + x + y`, because a bar measures from a baseline the grammar already supplies: zero. An `interval` has no such default. A low *and* a high are two numbers, and a plain `y()` supplies one. So the shortest interval that draws, its minimum syllable, includes a transform that produces a range. Ask for an interval without one and the engine says so, rather than drawing an empty panel:

```
data(iris_flowers) + interval + x(species) + y(petal_length)
```

Error:

```
! gog: `interval` draws a span from a low value to a high one, but nothing
  ↳ here produces those extents. Add a range transform - `interval * range +
  ↳ x(group) + y(value)` draws the min-max range per group, or `interval *
  ↳ bounds(lo, hi)` a pre-computed one.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

This is the design choice that keeps the [channel vocabulary](#) small. The obvious alternative would add `ymin` and `ymax` as two more channels meaningful to a single mark, and a box plot would need five channels. Instead a transform invents the extents, exactly as `bin` invents a count, so the channel set never grows. The same low and high rows are what [box](#) and [ribbon](#) draw from. When you already have the extents, from a fitted model or a published standard error, `bounds(lower, upper)` supplies them without computing anything: `interval * bounds(lower, upper)` draws one whisker per row.

13.7 Whiskers on their side

An interval reads its orientation off the bindings, the way a [bar](#) and a [box](#) do. Put the categories on `y` and the measured column on `x`, and the whiskers lie down:

R

```
data(iris_flowers) + interval * confidence(0.95) + x(petal_length) +
  ↳ y(species) +
  x_label("Petal length (cm)") + y_label("Species") +
  title("The same 95% CI, lying down")
```

Python

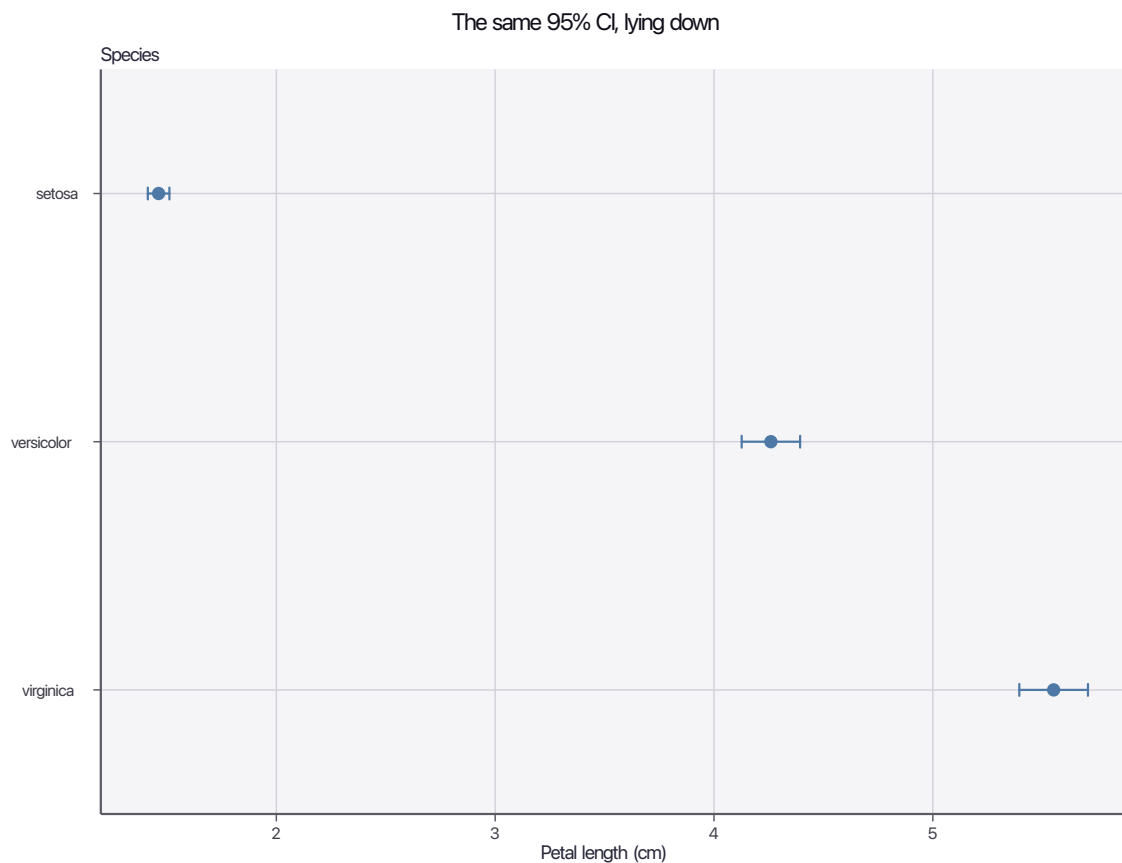
```
(data(iris_flowers) + interval * confidence(0.95) + x(col.petal_length) +  
  ↪ y(col.species) +  
    x_label("Petal length (cm)") + y_label("Species") +  
    title("The same 95% CI, lying down"))
```

Julia

```
data(iris_flowers) + interval * confidence(0.95) + x(:petal_length) +  
  y(:species) + x_label("Petal length (cm)") + y_label("Species") +  
  title("The same 95% CI, lying down")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, confidence(0.95)),  
  x(col.petal_length), y(col.species), x_label("Petal length (cm)",  
  y_label("Species"), title("The same 95% CI, lying down"))
```



The caps turn with the whisker, since a cap always crosses the span at its end, and the center dot stays at the center. Everything else in this chapter is unchanged: `pattern`, `color` and `dodge` behave the same whichever axis carries the categories, and so do the two settings the later sections introduce.

This is the usual shape for a **coefficient plot**, where each row is a named estimate and the reader scans down a column of intervals. Estimates like these normally arrive already computed, from a model rather than from raw observations. That is what `bounds` is for: it reads two columns you already have as the low and the high, and computes nothing. With the terms on y, the measured axis is the one `bounds` invents, so there is no `x()` to write:

R

```
coefs <- data.frame(
  term = c("Age", "Education", "Experience", "Region: North", "Region:
↵ South"),
  lo    = c(0.02, 0.31, 0.11, -0.24, -0.05),
```

```

    hi    = c(0.18, 0.55, 0.29, 0.06, 0.21)
  )
data(coefs) + interval * bounds(lo, hi) + y(term) +
  x_label("Effect on log wage") +
  title("A coefficient plot, from bounds you already have")

```

Python

```

coefs = {"term": ["Age", "Education", "Experience", "Region: North", "Region:
↪ South"], "lo": [0.02, 0.31, 0.11, -0.24, -0.05], "hi": [0.18, 0.55, 0.29,
↪ 0.06, 0.21]}
(data(coefs) + interval * bounds(col.lo, col.hi) + y(col.term) +
  x_label("Effect on log wage") +
  title("A coefficient plot, from bounds you already have"))

```

Julia

```

coefs = (term = ["Age", "Education", "Experience", "Region: North", "Region:
↪ South"], lo = [0.02, 0.31, 0.11, -0.24, -0.05], hi = [0.18, 0.55, 0.29,
↪ 0.06, 0.21],)
data(coefs) + interval * bounds(:lo, :hi) + y(:term) +
  x_label("Effect on log wage") +
  title("A coefficient plot, from bounds you already have")

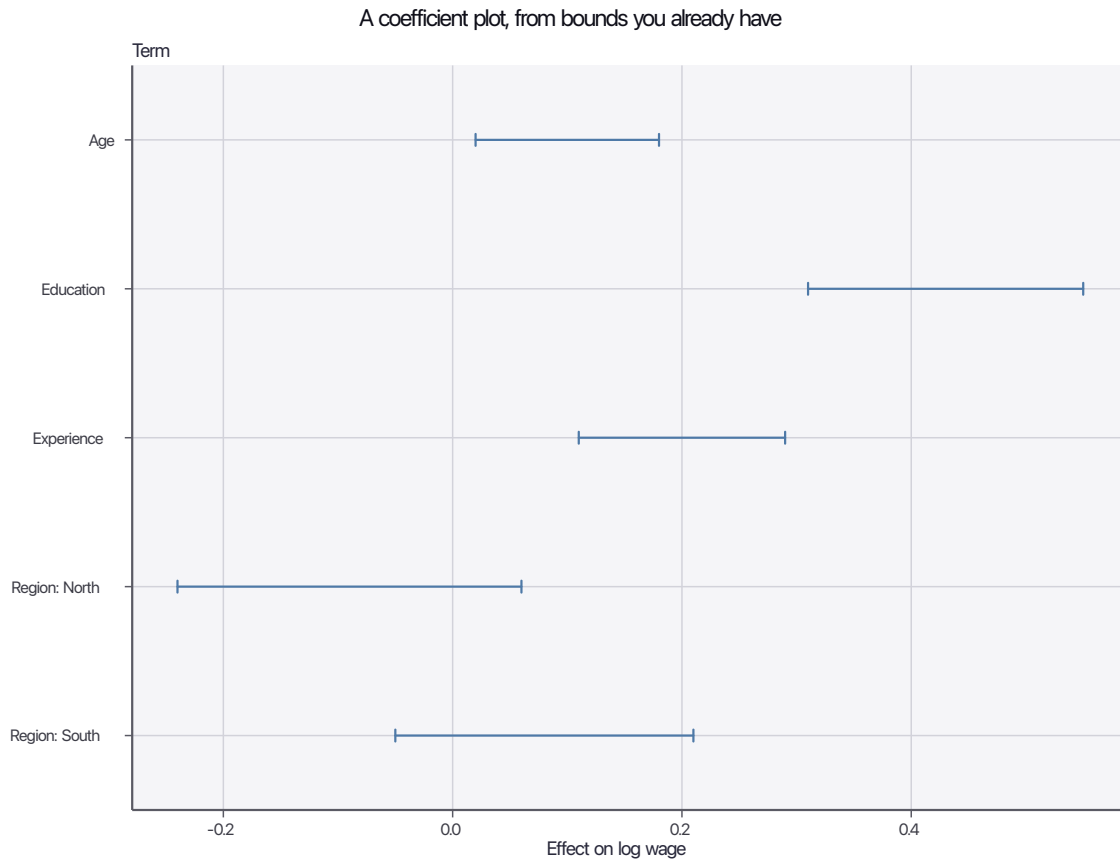
```

JavaScript

```

const coefs = { term: ["Age", "Education", "Experience", "Region: North",
↪ "Region: South"], lo: [0.02, 0.31, 0.11, -0.24, -0.05], hi: [0.18, 0.55,
↪ 0.29, 0.06, 0.21] };
plot(data(coefs), layer(interval, bounds(col.lo, col.hi)), y(col.term),
  x_label("Effect on log wage"),
  title("A coefficient plot, from bounds you already have"))

```



“Given the coefs table: intervals from lo to hi, y is term.”

The two intervals that cross zero cover positive and negative values alike, so the data does not settle whether those two effects are positive or negative.

An interval needs a number to span, so two categorical axes are refused:

```
data(gapminder_2007) + interval * range + x(continent) + y(continent)
```

Error:

```
! gog: `interval` has categorical columns on both axes - `x(continent)` and
  ↳ `y(continent)` - so there is nothing for it to span. One axis must be a
  ↳ number: that is the column the low and high extents come from.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

13.8 Splitting by color

color splits an interval into one whisker per group, the same discrete split line and bar make. The statistic is computed inside each group, so each gets its own interval, its own hue, and a legend:

R

```
data(iris_flowers) + interval * confidence(0.95) + x(species) +  
  ↪ y(petal_length) +  
    color(species) +  
    x_label("Species") + y_label("Petal length (cm)") +  
    title("95% CI of mean petal length, by species")
```

Python

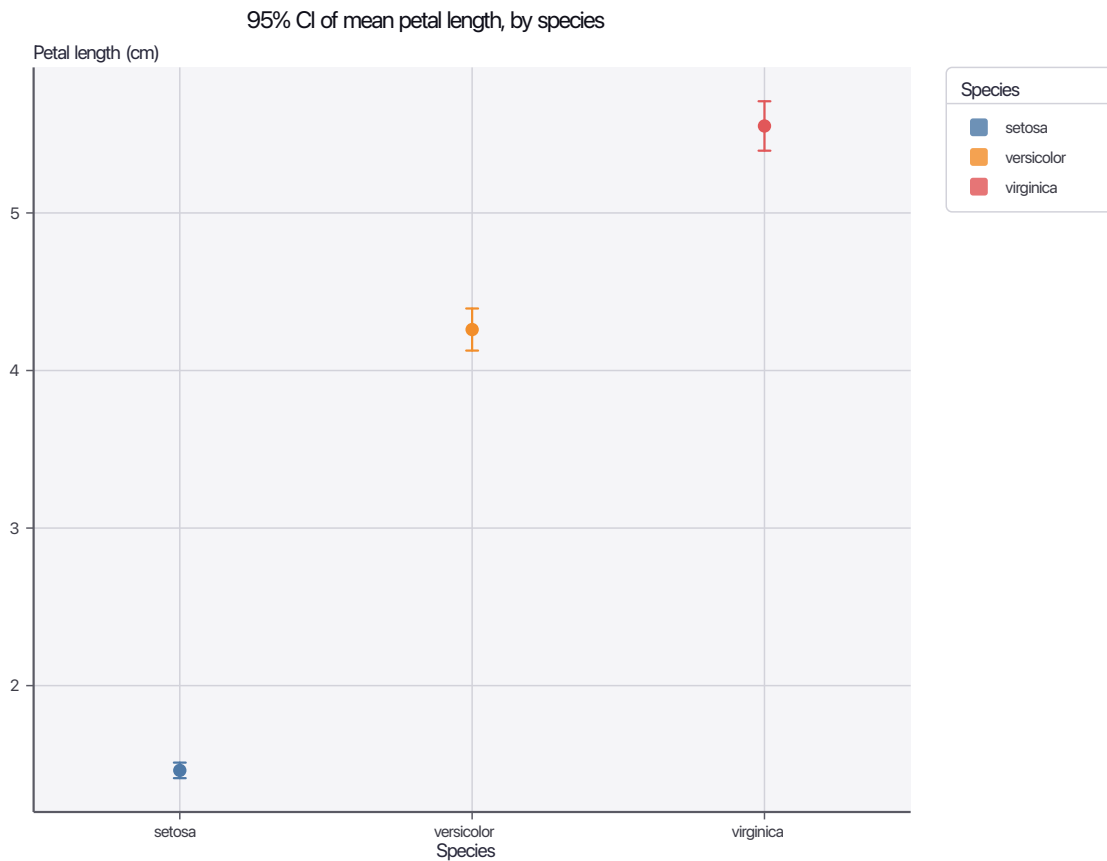
```
(data(iris_flowers) + interval * confidence(0.95) + x(col.species) +  
  ↪ y(col.petal_length) +  
    color(col.species) +  
    x_label("Species") + y_label("Petal length (cm)") +  
    title("95% CI of mean petal length, by species"))
```

Julia

```
data(iris_flowers) + interval * confidence(0.95) + x(:species) +  
  y(:petal_length) + color(:species) + x_label("Species") +  
  y_label("Petal length (cm)") +  
  title("95% CI of mean petal length, by species")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, confidence(0.95)),  
  x(col.species), y(col.petal_length), color(col.species),  
  x_label("Species"), y_label("Petal length (cm)"),  
  title("95% CI of mean petal length, by species"))
```

“Given the iris flowers: intervals derived by confidence at 95%, x is species, y is petal length, color by species.”

When the split is a *second* category, so several whiskers share one x , they overlap, and `dodge` sets them side by side, the same collision modifier that separates grouped bars and boxes:

R

```
data(gm_eras) + interval * range * dodge + x(continent) + y(life) +
  ↪ color(era) +
  y_label("Life expectancy range") +
  title("Life expectancy range per continent, 1957 vs 2007")
```

Python

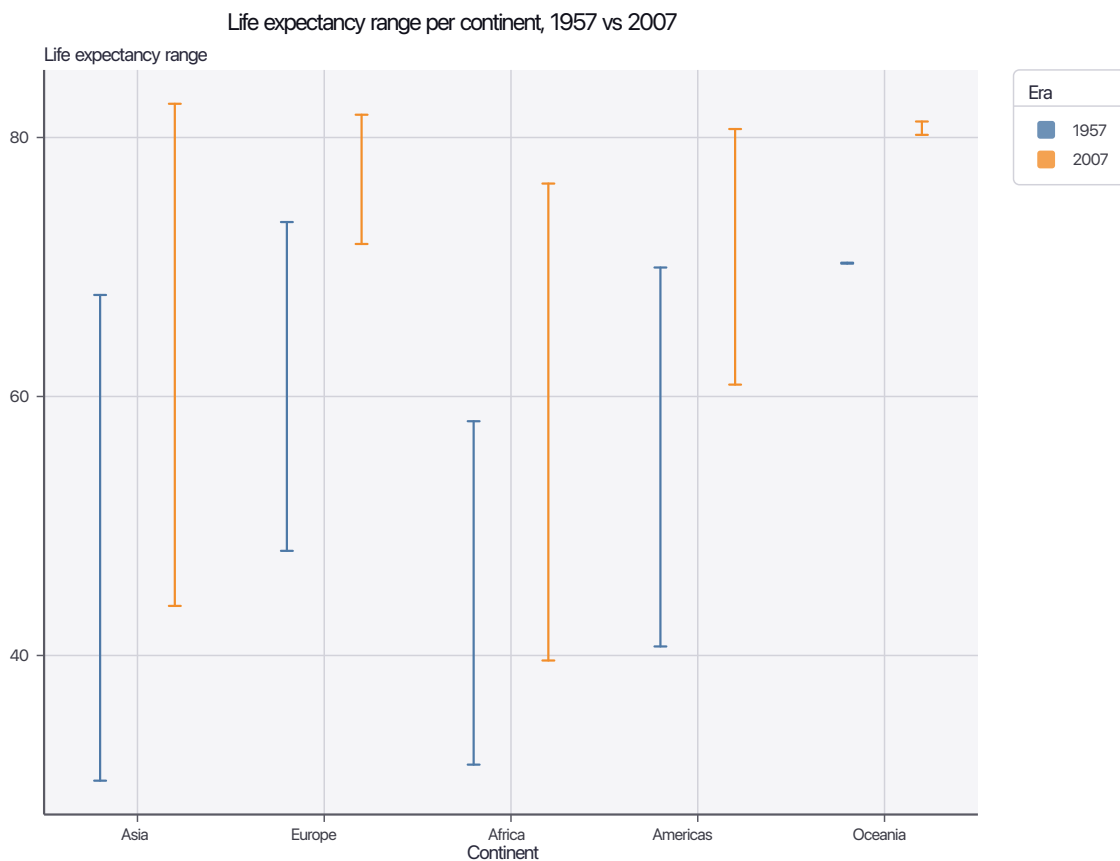
```
(data(gm_eras) + interval * range * dodge + x(col.continent) + y(col.life) +
  ↪ color(col.era) +
  y_label("Life expectancy range") +
  title("Life expectancy range per continent, 1957 vs 2007"))
```

Julia

```
data(gm_eras) + interval * range * dodge + x(:continent) + y(:life) +
  color(:era) + y_label("Life expectancy range") +
  title("Life expectancy range per continent, 1957 vs 2007")
```

JavaScript

```
plot(data(gm_eras), layer(interval, range, dodge), x(col.continent),
  y(col.life), color(col.era), y_label("Life expectancy range"),
  title("Life expectancy range per continent, 1957 vs 2007"))
```



“Given the *gapminder* eras: intervals derived by range and dodge, *x* is continent, *y* is life, color by era.”

Width and opacity stay *settings*. One whisker is a single stroke, so there is no row for **size** or opacity to vary over, and `style()` sets them for the layer:

R

```
data(iris_flowers) + interval * range + x(species) + y(petal_length) +  
  style(color = "tomato", size = 2.5) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("A heavier, one-color range")
```

Python

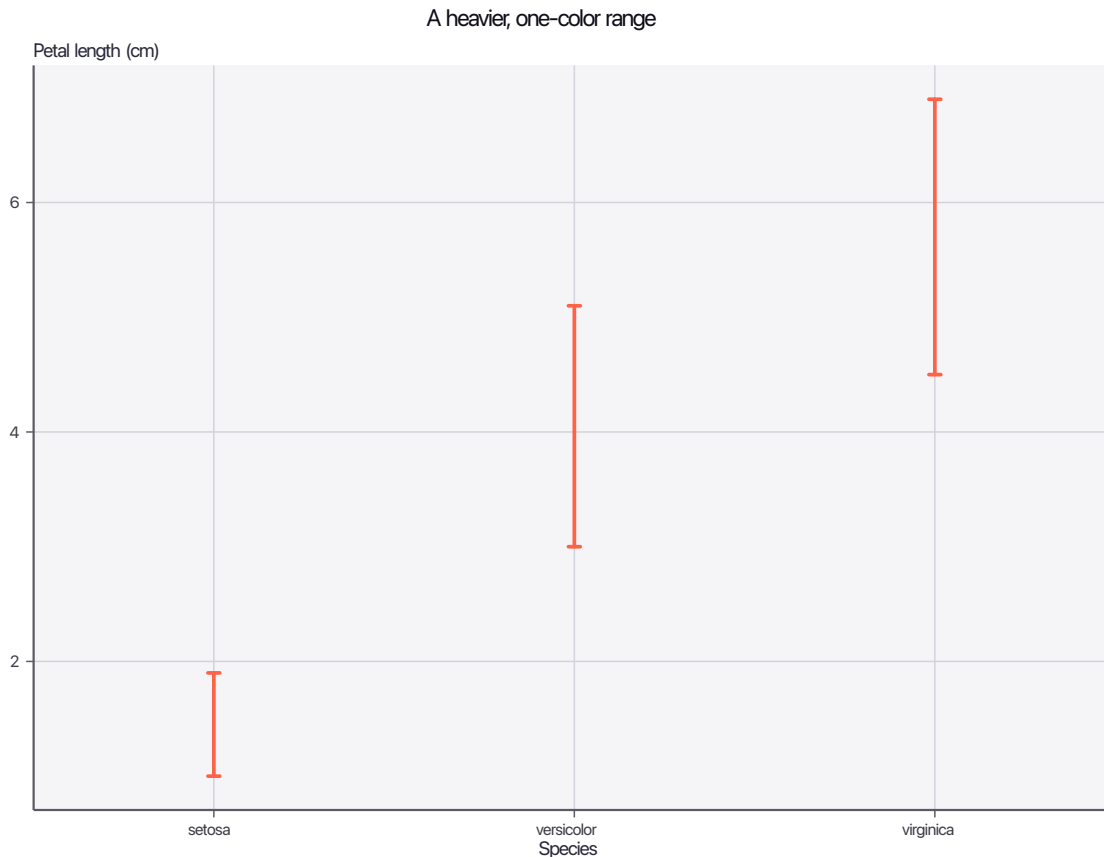
```
(data(iris_flowers) + interval * range + x(col.species) + y(col.petal_length)  
↪ +  
  style(color = "tomato", size = 2.5) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("A heavier, one-color range"))
```

Julia

```
data(iris_flowers) + interval * range + x(:species) + y(:petal_length) +  
  style(color = "tomato", size = 2.5) + x_label("Species") +  
  y_label("Petal length (cm)") + title("A heavier, one-color range")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, range), x(col.species),  
  y(col.petal_length), style({ color: "tomato", size: 2.5 }),  
  x_label("Species"), y_label("Petal length (cm)"),  
  title("A heavier, one-color range"))
```



“Given the iris flowers: intervals derived by range, x is species, y is petal length, colored tomato, with size 2.5.”

13.9 Caps, or a bare linerange

The end caps, the short crossbars, are decoration, not data; they only emphasize where a whisker stops. So whether to draw them is a *setting*: `style(caps = FALSE)` drops them for a bare linerange, and `caps = TRUE` (the default) keeps the capped error bar.

R

```
data(iris_flowers) + interval * confidence(0.95) + x(species) +
  ↪ y(petal_length) +
  color(species) + style(caps = FALSE) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("The same CI, drawn as a linerange")
```

Python

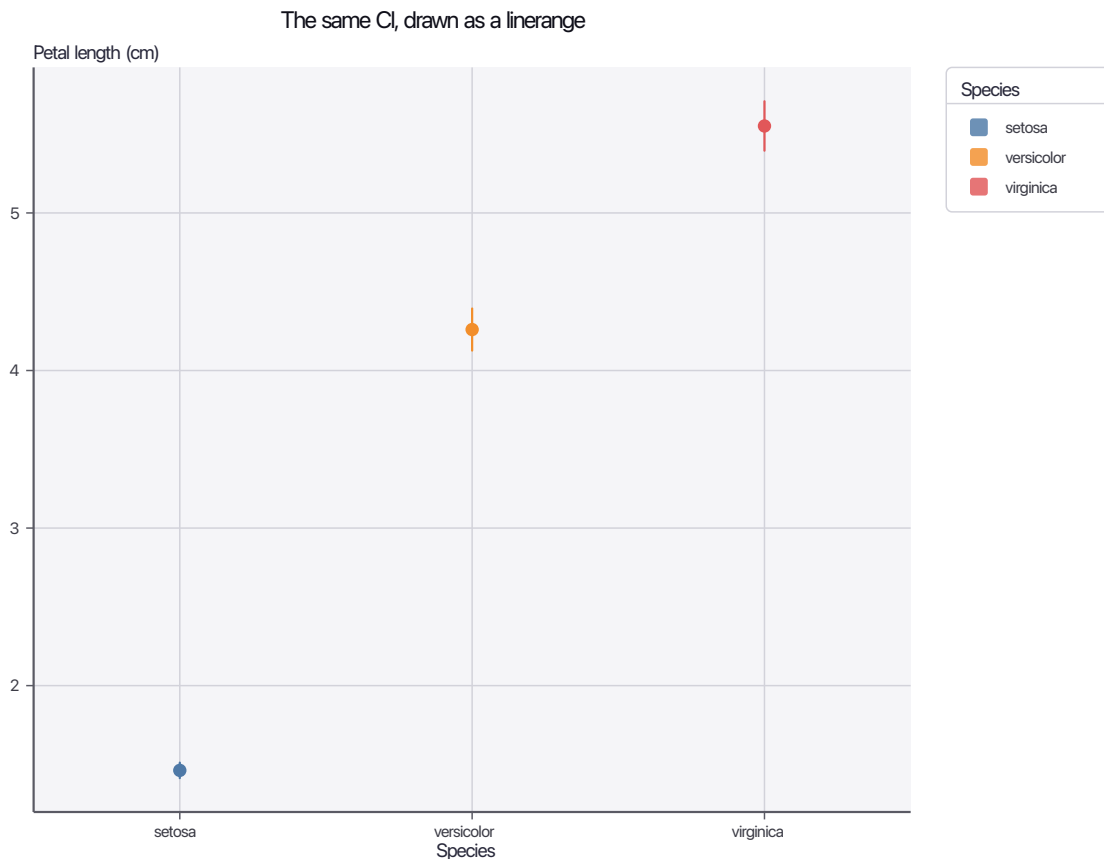
```
(data(iris_flowers) + interval * confidence(0.95) + x(col.species) +  
↪ y(col.petal_length) +  
  color(col.species) + style(caps = False) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("The same CI, drawn as a linerange"))
```

Julia

```
data(iris_flowers) + interval * confidence(0.95) + x(:species) +  
y(:petal_length) + color(:species) + style(caps = false) +  
x_label("Species") + y_label("Petal length (cm)") +  
title("The same CI, drawn as a linerange")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, confidence(0.95)),  
  x(col.species), y(col.petal_length), color(col.species),  
  style({ caps: false }), x_label("Species"),  
  y_label("Petal length (cm)"),  
  title("The same CI, drawn as a linerange"))
```



“Given the iris flowers: intervals derived by confidence at 95%, x is species, y is petal length, color by species, with caps off.”

Because caps are geometry only `interval` has, `style(caps = )` on any other mark is refused with direction, the same way a bar’s `border_size` is refused on a `line`. `caps` combines with `range`, `confidence` and `deviation`, and with the color split. So error bar, linerange, pointrange and their colored versions are one mark, one transform and one setting.

13.10 Hiding the center dot

A pointrange’s dot is the mean; sometimes you want the confidence whiskers without it, a plain error bar. `style(center = FALSE)` drops the dot, and `center = TRUE` (the default) keeps it. Like `caps`, it only *hides*: it cannot add a center to `range`, which has none to draw.

R

```
data(iris_flowers) + interval * confidence(0.95) + x(species) +
  ↪ y(petal_length) +
  color(species) + style(center = FALSE) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("The same CI, without its center dots")
```

Python

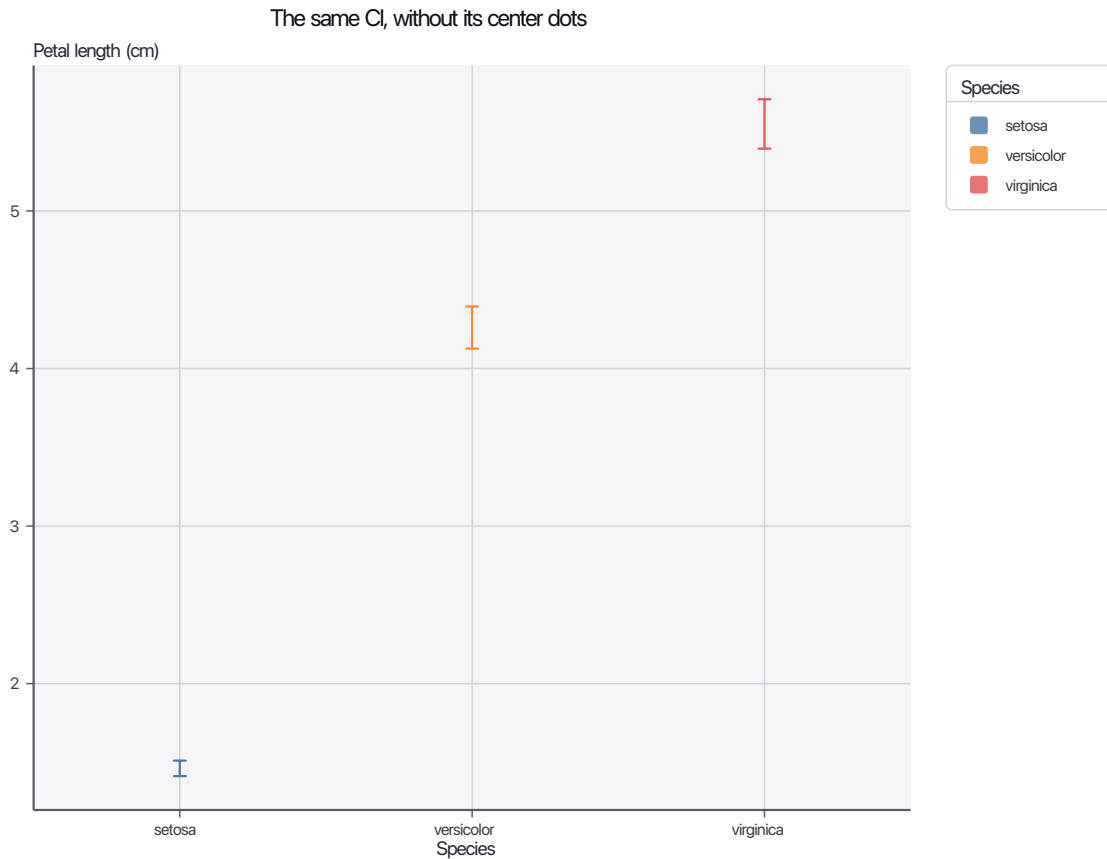
```
(data(iris_flowers) + interval * confidence(0.95) + x(col.species) +
  ↪ y(col.petal_length) +
  color(col.species) + style(center = False) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("The same CI, without its center dots"))
```

Julia

```
data(iris_flowers) + interval * confidence(0.95) + x(:species) +
  y(:petal_length) + color(:species) + style(center = false) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("The same CI, without its center dots")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, confidence(0.95)),
  x(col.species), y(col.petal_length), color(col.species),
  style({ center: false }), x_label("Species"),
  y_label("Petal length (cm)"),
  title("The same CI, without its center dots"))
```



“Given the iris flowers: intervals derived by confidence at 95%, x is species, y is petal length, color by species, with center off.”

`center` and `caps` are the interval’s two settings, one for its middle and one for its ends, so a confidence interval draws four ways. A capped pointrange is the default. A capped bare error bar is `center = FALSE`. A linerange with a dot is `caps = FALSE`, and a plain linerange is both `FALSE`. Like `style(caps = )`, `style(center = )` belongs to `interval` alone; on any other mark it is refused with `direction`.

13.11 A dashed whisker

A whisker is a **stroke**, so it takes `style(pattern = )` the same way a **line** does. This is the **settable rule**: a dash patterns every path-stroke mark (`line`, `step`, `interval`, `path`, `rule`). The dash is drawn on the whisker span only. The caps and the center dot stay solid, since a dashed crossbar is hard to read as a cap:

R

```
data(iris_flowers) + interval * confidence(0.95) + x(species) +  
  ↪ y(petal_length) +  
    color(species) + style(pattern = "dashed") +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("A dashed 95% CI")
```

Python

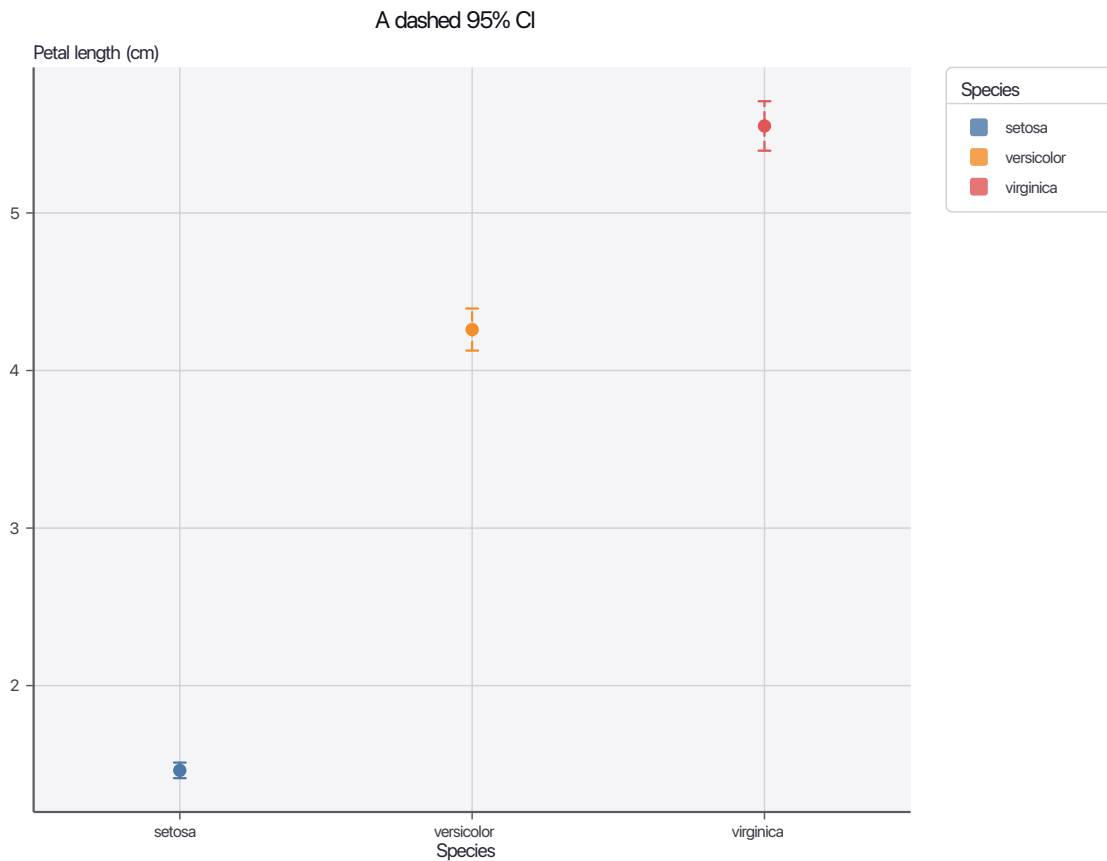
```
(data(iris_flowers) + interval * confidence(0.95) + x(col.species) +  
  ↪ y(col.petal_length) +  
    color(col.species) + style(pattern = "dashed") +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("A dashed 95% CI"))
```

Julia

```
data(iris_flowers) + interval * confidence(0.95) + x(:species) +  
  y(:petal_length) + color(:species) + style(pattern = "dashed") +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("A dashed 95% CI")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, confidence(0.95)),  
  x(col.species), y(col.petal_length), color(col.species),  
  style({ pattern: "dashed" }), x_label("Species"),  
  y_label("Petal length (cm)"), title("A dashed 95% CI"))
```



“Given the iris flowers: intervals derived by confidence at 95%, x is species, y is petal length, color by species, with pattern dashed.”

The five-number summary has its own mark, `box`, and the same low and high pair drawn as a continuous fill is the `ribbon` band. See [Transforms](#) for where `range`, `confidence` and `deviation` sit among the others.

13.12 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(pattern =)</code>	solid, dashed, dotted

Setting	Value
<code>style(caps =)</code>	TRUE, FALSE
<code>style(center =)</code>	TRUE, FALSE

And these vary per row if you map them to a column instead: `color()` (categories), `pattern()` (categories), `group()` (categories), `play()` (either).

`caps` and `center` are the interval's own settings, and both only ever *hide*. `caps` turns the end crossbars off, leaving a bare linerange; `center` hides the dot that `confidence` supplies. Neither can add geometry the statistic did not produce, so `style(center = TRUE)` on `interval * range` changes nothing: a range has no center. The plot below reads `gm_continents`, three continents across every year from 1952 to 2007, so each interval pools twelve years of countries.

Table 13.2: `gm_continents`: first 5 of 1056 rows

country	continent	year	life	population	gdp
Afghanistan	Asia	1952	28.801	8425333	779.4453
Afghanistan	Asia	1957	30.332	9240934	820.8530
Afghanistan	Asia	1962	31.997	10267083	853.1007
Afghanistan	Asia	1967	34.020	11537966	836.1971
Afghanistan	Asia	1972	36.088	13079460	739.9811

R

```
data(gm_continents) + interval * confidence(0.99) + x(continent) + y(life) +
  style(caps = FALSE, size = 3, color = "darkslateblue", pattern = "dotted")
↪ +
  y_label("Life expectancy") + title("A bare, thick, dotted 99% interval")
```

Python

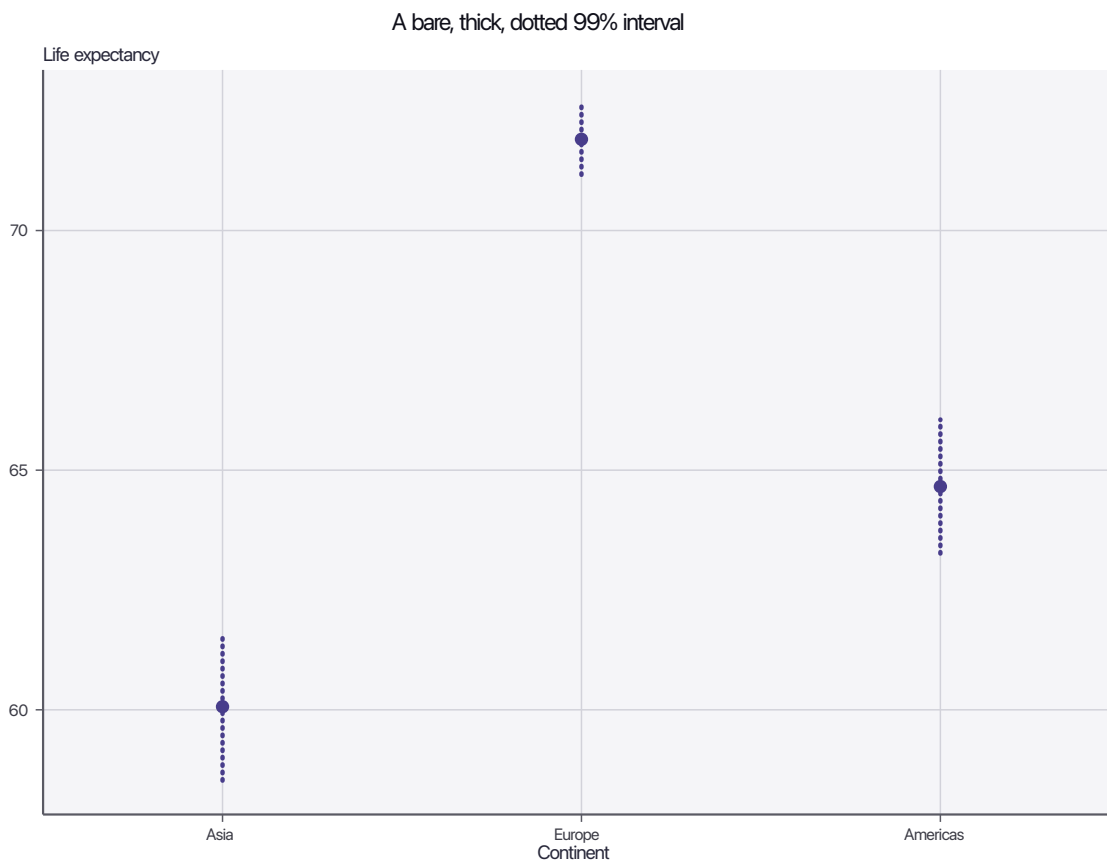
```
(data(gm_continents) + interval * confidence(0.99) + x(col.continent) +
  ↪ y(col.life) +
  style(caps = False, size = 3, color = "darkslateblue", pattern = "dotted")
  ↪ +
  y_label("Life expectancy") + title("A bare, thick, dotted 99% interval"))
```

Julia

```
data(gm_continents) + interval * confidence(0.99) + x(:continent) +
  y(:life) +
  style(caps = false, size = 3, color = "darkslateblue", pattern = "dotted")
↪ +
  y_label("Life expectancy") + title("A bare, thick, dotted 99% interval")
```

JavaScript

```
plot(data(gm_continents), layer(interval, confidence(0.99)),
  x(col.continent), y(col.life),
  style({ caps: false, size: 3, color: "darkslateblue",
  pattern: "dotted" }), y_label("Life expectancy"),
  title("A bare, thick, dotted 99% interval"))
```



One number is missing from that table, and the transform carries it: `confidence(0.99)` widens the interval from the default 0.95. That belongs to the transform rather than to `style()`,

because it changes *what is computed*, not how it is drawn. The test is whether the numbers change: they do here, so it is not a setting.

13.13 What it refuses

An interval is a stroke, however much it looks like a bar, so its pattern is a dash and a hatch is refused:

```
data(gapminder_2007) + interval * range + x(continent) + y(life) +  
  style(pattern = "hatch")
```

Error:

```
! gog: `style(pattern = "hatch")` is not a stroke pattern. Use "solid" (the  
  ↪ default), "dashed", or "dotted".
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

That refusal is worth explaining, because a capped range looks like a shape with an inside. The engine draws it as a stroke, which is why the whisker takes a dash and not a hatch. Two more refusals appear earlier in this chapter. A bare `interval` has no extents to span, under [Why interval needs a transform](#), and an interval with two categorical axes has nothing to measure, under [Whiskers on their side](#).

14 Box

How is each group spread out? **box** draws the **box-and-whisker** of a distribution, one per group. In each category it summarizes the measured column. A box spans the lower quartile to the upper, with a line at the median. Whiskers reach to the most extreme values *within 1.5 times the interquartile range (IQR) of the box*. Any points beyond that are outliers, drawn individually as dots. **bar * mean** gives one number per group. **box** shows how the whole group is spread, and draws the few values far from the rest as separate dots. Boxes stand up by default and lie down when the categories are on y; see [Boxes on their side](#).

The construction is John Tukey's, from *Exploratory Data Analysis* (Tukey, 1977), where he called it a **schematic plot**. The name readers kept was his informal one, box-and-whisker, now usually shortened to **box plot**. Mary Eleanor Spear's **range bar** of 1952 (Spear, 1952) had drawn the box and the extremes a quarter century earlier. The range bar had no rule for the **fence**, the cutoff that decides which values are outliers. That rule, 1.5 times the IQR, is Tukey's, and so is the choice of where ordinary values stop and unusual ones begin.

One difference is worth knowing if you know his book. Tukey's box spans **hinges**. Sort the values, cut the sorted list in half, and take the median of each half. **gog** spans the lower and upper **quartiles** instead, as most software does. The two agree on most samples and differ slightly on small ones. A box drawn by **gog** is therefore not always the box Tukey's own rule gives.

14.1 A summary per group

Put the category that splits the data on x, and put the measured column on y. The mark computes the summary itself:

R

```
data(iris_flowers) + box + x(species) + y(petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Distribution of petal length per species")
```

Python

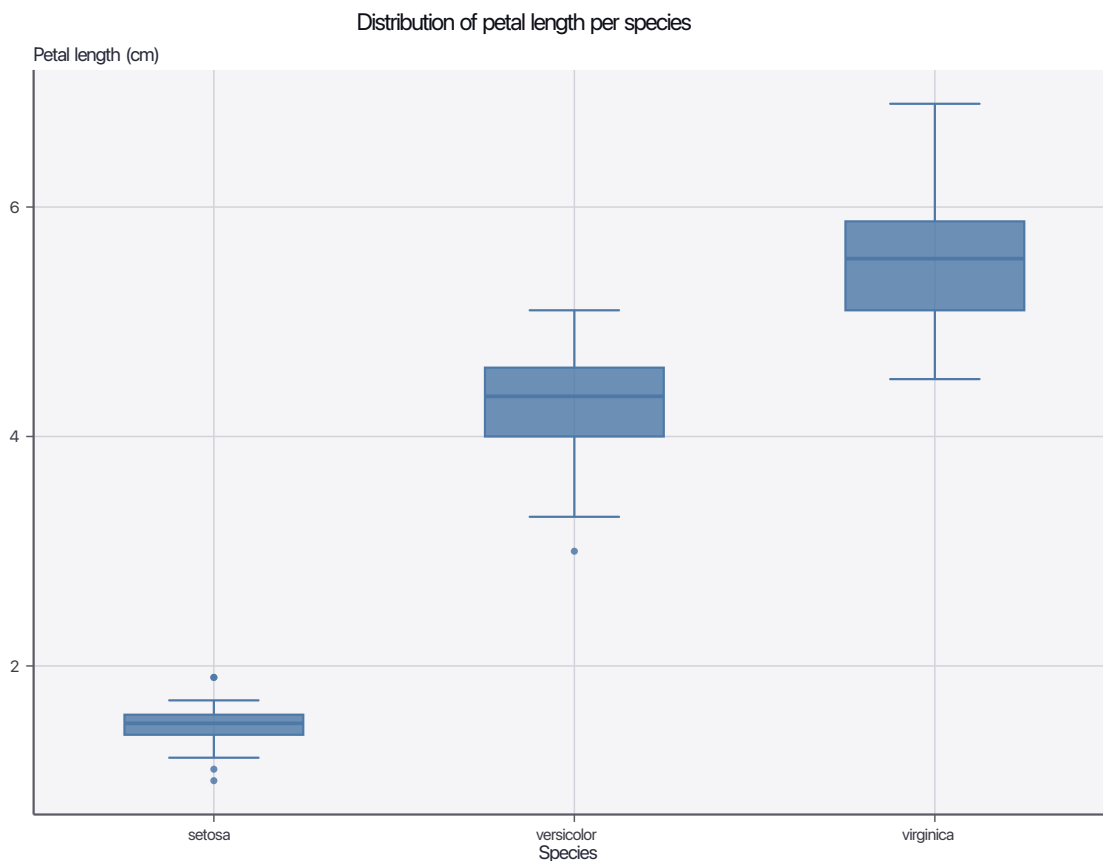
```
(data(iris_flowers) + box + x(col.species) + y(col.petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Distribution of petal length per species"))
```

Julia

```
data(iris_flowers) + box + x(:species) + y(:petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Distribution of petal length per species")
```

JavaScript

```
plot(data(iris_flowers), box, x(col.species), y(col.petal_length),  
  x_label("Species"), y_label("Petal length (cm)" ),  
  title("Distribution of petal length per species"))
```



“Given the iris flowers: boxes, x is species, y is petal length.”

Every box reads the same way. Half the flowers of a species fall inside the box (the middle 50%, from the lower quartile to the upper); the line across it is the median; the whiskers run to the rest of the values inside the fence. The separate dots are the outliers: flowers more than 1.5 IQR from the box. The whisker stops before them, and each one is drawn on its own.

14.2 Distribution across many groups

The comparison still works with more groups, and here the outliers name particular countries. This is life expectancy across the continents in 2007, five distributions in one panel, with two countries far below the rest of their continent:

R

```
data(gapminder_2007) + box + x(continent) + y(life) +  
  x_label("Continent") + y_label("Life expectancy (years)") +  
  title("Life expectancy by continent, 2007")
```

Python

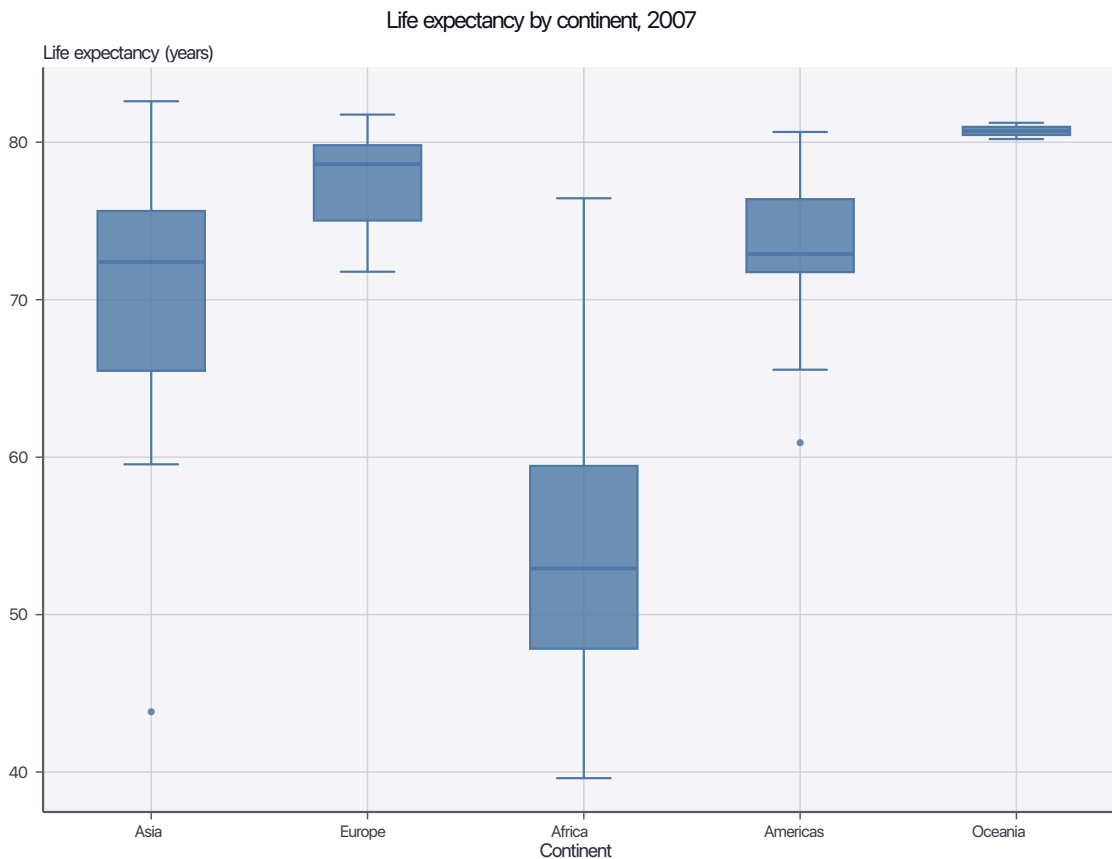
```
(data(gapminder_2007) + box + x(col.continent) + y(col.life) +  
  x_label("Continent") + y_label("Life expectancy (years)") +  
  title("Life expectancy by continent, 2007"))
```

Julia

```
data(gapminder_2007) + box + x(:continent) + y(:life) +  
  x_label("Continent") + y_label("Life expectancy (years)") +  
  title("Life expectancy by continent, 2007")
```

JavaScript

```
plot(data(gapminder_2007), box, x(col.continent), y(col.life),  
  x_label("Continent"), y_label("Life expectancy (years)"),  
  title("Life expectancy by continent, 2007"))
```

The low dot below Asia is Afghanistan, and the one below the Americas is Haiti. Each sits far enough below its continent's box that no whisker reaches it. That is what the Tukey construction is for: the whiskers cover the values near the middle, and the dots are the values far from it. When every value in a group's middle 50% is the same, the box has no height and draws as a flat line. `gog` does not give it a minimum height to make the spread look larger.

14.3 Boxes on their side

Swap the two positions and the same sentence draws the same plot, turned. There is no `flip` atom, for the reason there is none on `bar`. `gog` reads the two bindings and decides from them which axis carries the numbers. A second way to say one thing is a second thing to learn.

R

```
data(gapminder_2007) + box + x(life) + y(continent) +  
  x_label("Life expectancy (years)") + y_label("Continent") +  
  title("The same five distributions, lying down")
```

Python

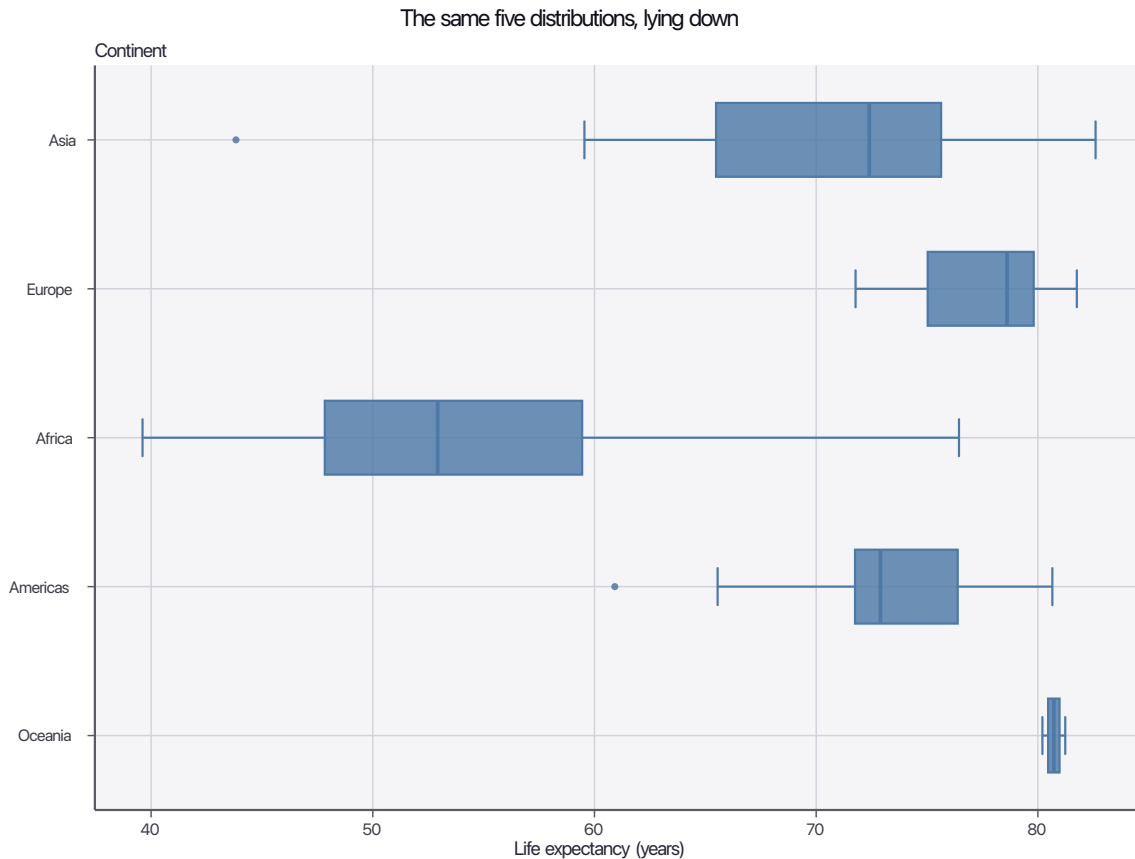
```
(data(gapminder_2007) + box + x(col.life) + y(col.continent) +  
  x_label("Life expectancy (years)") + y_label("Continent") +  
  title("The same five distributions, lying down"))
```

Julia

```
data(gapminder_2007) + box + x(:life) + y(:continent) +  
  x_label("Life expectancy (years)") + y_label("Continent") +  
  title("The same five distributions, lying down")
```

JavaScript

```
plot(data(gapminder_2007), box, x(col.life), y(col.continent),  
  x_label("Life expectancy (years)"), y_label("Continent"),  
  title("The same five distributions, lying down"))
```

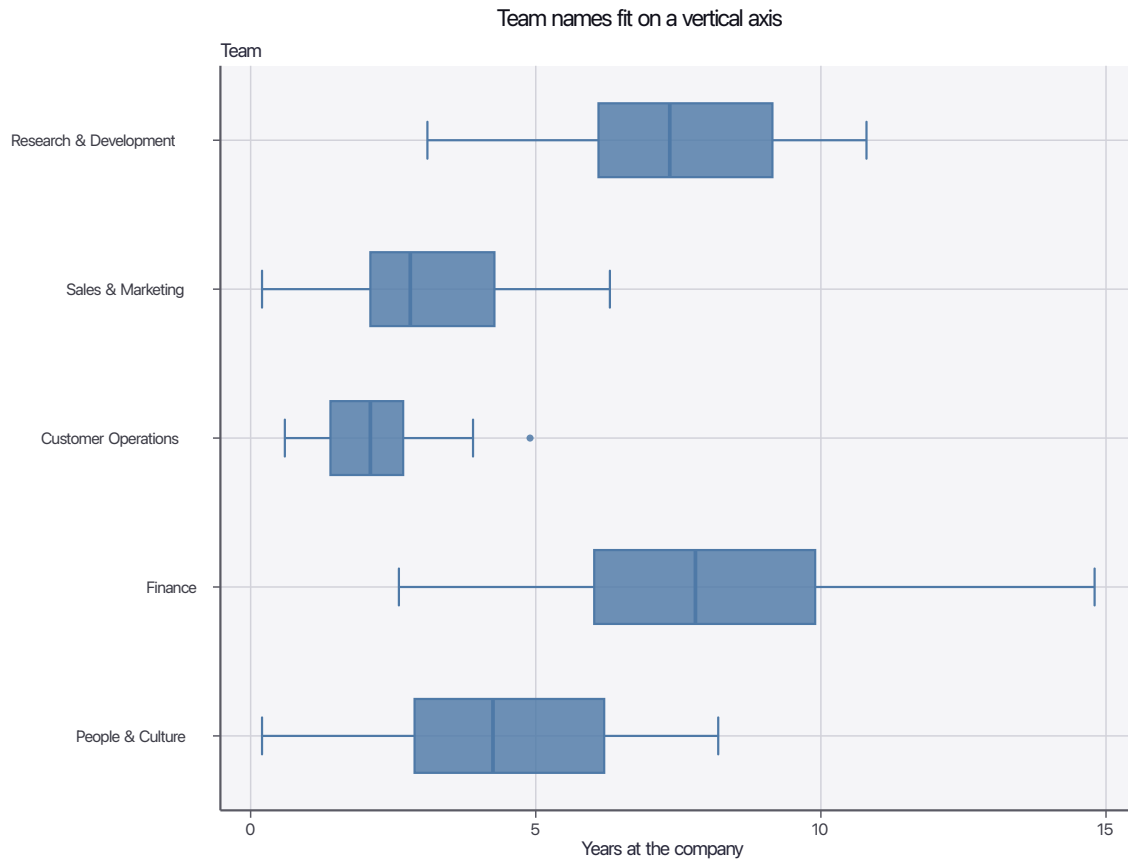


Compare it with the upright version above. Every box, whisker and outlier dot keeps its place relative to its neighbors, rotated a quarter turn. Afghanistan and Haiti are still the two dots beyond their continents' whiskers. Everything the rest of this chapter adds behaves the same whichever axis holds the categories: the color split, boxes set side by side, the whisker rule and `style()`.

Use it when the category names are long. The table below is invented: five teams, thirty people in each, and the years each person has been at the company. A vertical axis gives each name a whole line, while an upright plot has to fit it into a slot one box wide:

```
set.seed(4)
.teams <- c("Research & Development", "Sales & Marketing",
            "Customer Operations", "Finance", "People & Culture")
tenure <- data.frame(
  team = rep(.teams, each = 30),
  years = round(pmax(rnorm(150, rep(c(6.2, 3.1, 2.4, 7.8, 4.5), each = 30),
                                rep(c(2.4, 1.6, 1.2, 3.0, 2.0), each = 30)), 0.2),
  1)
```

```
)
data(tenure) + box + x(years) + y(team) +
  x_label("Years at the company") +
  title("Team names fit on a vertical axis")
```



A box needs a number to summarize, so two categorical axes are refused, exactly as they are on a bar:

```
data(gapminder_2007) + box + x(continent) + y(continent)
```

Error:

```
! gog: `box` has categorical columns on both axes - `x(continent)` and
  ↳ `y(continent)` - so there is nothing for it to summarize. One axis must
  ↳ be a number: that is the column the five-number summary reduces.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

14.4 Whiskers to the extremes

Sometimes you do not want outliers split off: you want the whiskers to run to the true minimum and maximum, the plain five-number summary. That is the one parameter the box mark carries, `box(whiskers = "range")`:

R

```
data(gapminder_2007) + box(whiskers = "range") + x(continent) + y(life) +  
  x_label("Continent") + y_label("Life expectancy (years)") +  
  title("The same, with whiskers to the full range")
```

Python

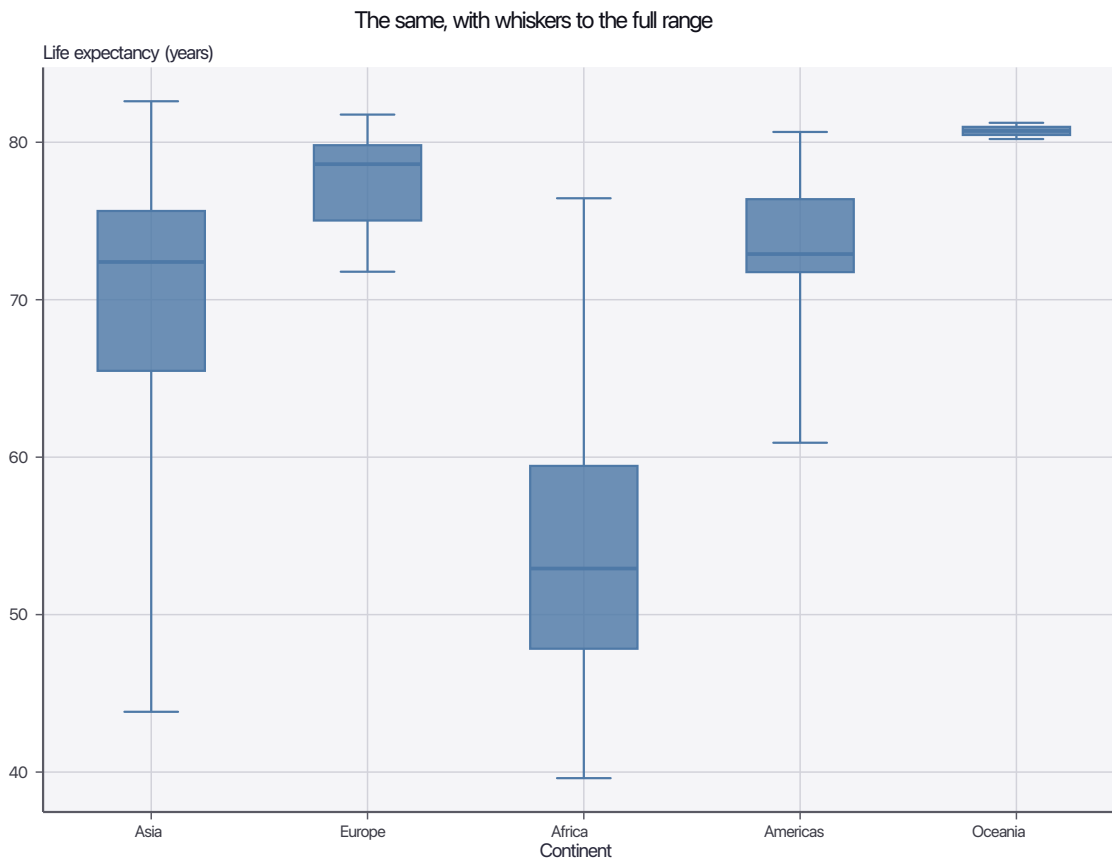
```
(data(gapminder_2007) + box(whiskers = "range") + x(col.continent) +  
↪ y(col.life) +  
  x_label("Continent") + y_label("Life expectancy (years)") +  
  title("The same, with whiskers to the full range"))
```

Julia

```
data(gapminder_2007) + box(whiskers = "range") + x(:continent) +  
  y(:life) + x_label("Continent") + y_label("Life expectancy (years)") +  
  title("The same, with whiskers to the full range")
```

JavaScript

```
plot(data(gapminder_2007), box({ whiskers: "range" }), x(col.continent),  
  y(col.life), x_label("Continent"), y_label("Life expectancy (years)"),  
  title("The same, with whiskers to the full range"))
```



“Given gapminder 2007: boxes with whiskers to the range, x is continent, y is life.”

Now Afghanistan and Haiti are the ends of their whiskers rather than dots. No point is called an outlier. The default is `"tukey"` (fences and outliers, the standard box plot); `"range"` is the whole five-number summary with nothing set apart. `whiskers` is the only choice a box offers.

14.5 Why box needs no transform

An `interval` cannot draw without a `range` transform to give it a low and a high; a `box` never needs one. `interval` is pure geometry (“span from here to there”) and *which* low and high it spans is a genuine choice (a min–max range, a confidence interval, a precomputed pair). A `box` is a named statistical glyph: the summary *is* what a box plot means. A statistic that draws as exactly one mark belongs *to* that mark, not beside it as a transform you compose. So the whole box is `box + x + y`, and adding a transform to it is refused with direction:

```
data(iris_flowers) + box * mean + x(species) + y(petal_length)
```

Error:

```
! gog: `box` already summarizes each group into a box - quartiles, median,
  ↳ whiskers - so it takes no transform, but `mean` was added. Drop it: `box
  ↳ + x(group) + y(value)` draws the box-and-whisker on its own.
gog: `box * mean * box` measures each cell twice - `mean` and `box` both
  ↳ reduce the column you named, and a cell holds one answer, so drawing both
  ↳ would mean drawing one of them and discarding the other. Keep whichever
  ↳ you meant: `box * mean` or `box * box`. To show both, draw them as two
  ↳ layers: `box * mean + box * box`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

This is the principle that keeps the [channel vocabulary](#) small, applied to a mark instead of a channel. Borrowing `ymin/ymax/lower/middle/upper` as channels would add *five* channels meaningful to a single mark; instead the summary is invented inside the mark, and the channel set never grows.

14.6 Coloring the boxes

`color` gives each box its own hue and a legend, the same discrete `split bar` and `interval` make. Mapped to the same column that sits on `x`, it colors one box per group:

R

```
data(iris_flowers) + box + x(species) + y(petal_length) + color(species) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("The same distributions, colored by species")
```

Python

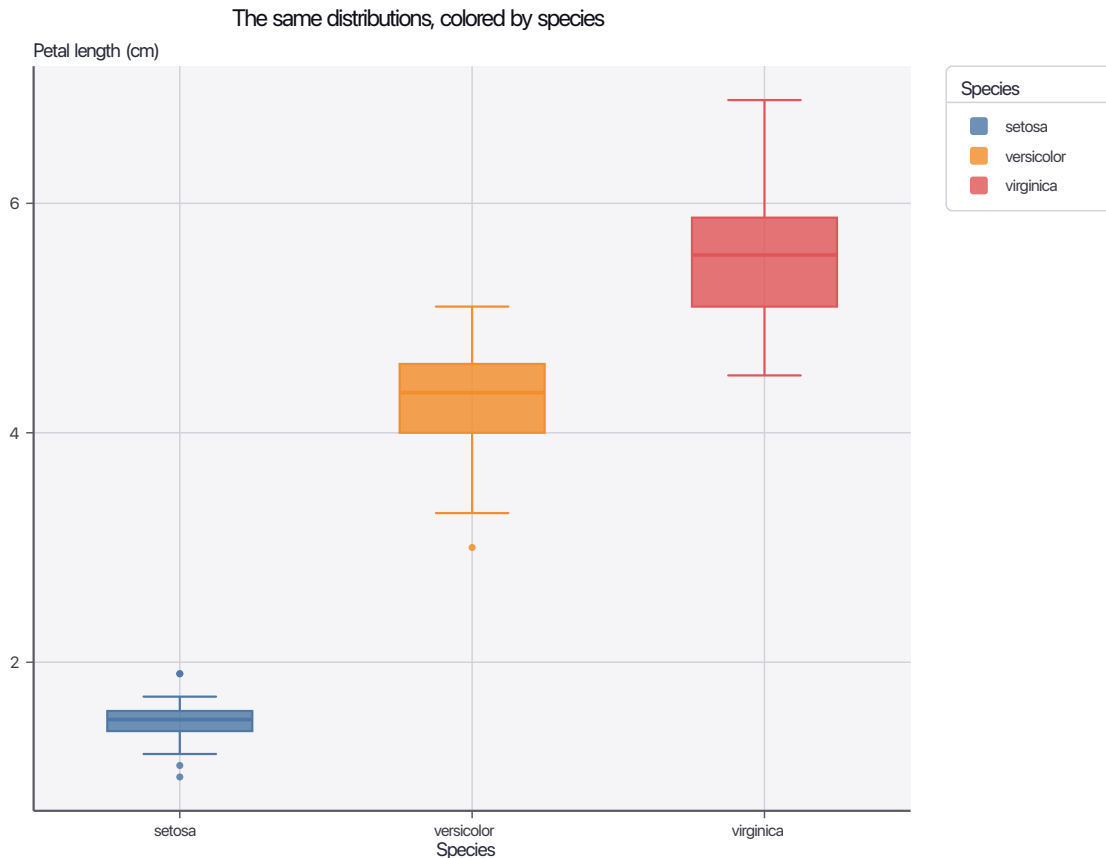
```
(data(iris_flowers) + box + x(col.species) + y(col.petal_length) +
  ↳ color(col.species) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("The same distributions, colored by species"))
```

Julia

```
data(iris_flowers) + box + x(:species) + y(:petal_length) +
  color(:species) + x_label("Species") + y_label("Petal length (cm)") +
  title("The same distributions, colored by species")
```

JavaScript

```
plot(data(iris_flowers), box, x(col.species), y(col.petal_length),  
     color(col.species), x_label("Species"), y_label("Petal length (cm)"),  
     title("The same distributions, colored by species"))
```



“Given the iris flowers: boxes, x is species, y is petal length, color by species.”

Mapping `color` to a *second* category splits each x-group into one box per color. Without `dodge` those boxes share one x and overlap, and their fills turn translucent so each stays readable. `dodge` sets them side by side, each continent’s two eras next to each other:

R

```
data(gm_eras) + box * dodge + x(continent) + y(life) + color(era) +  
  y_label("Life expectancy") +  
  title("Life expectancy per continent, 1957 vs 2007")
```


Python

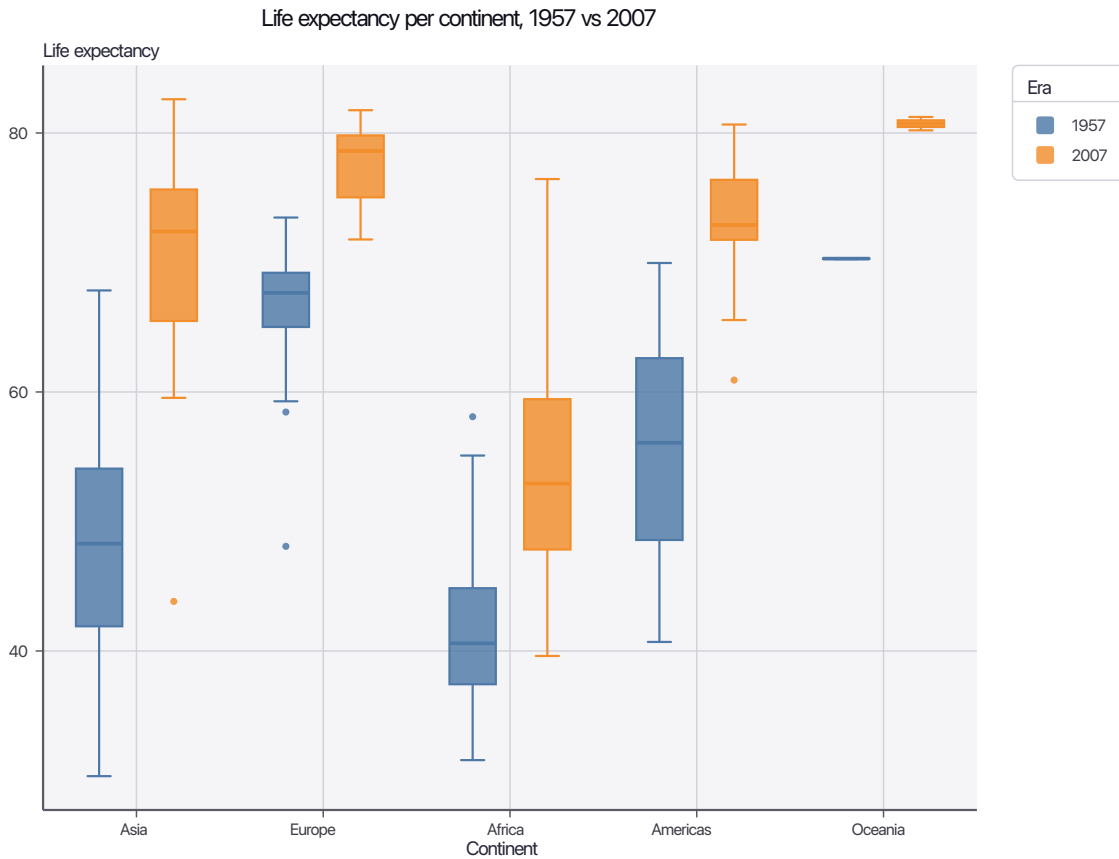
```
(data(gm_eras) + box * dodge + x(col.continent) + y(col.life) +  
  ↪ color(col.era) +  
  y_label("Life expectancy") +  
  title("Life expectancy per continent, 1957 vs 2007"))
```

Julia

```
data(gm_eras) + box * dodge + x(:continent) + y(:life) + color(:era) +  
  y_label("Life expectancy") +  
  title("Life expectancy per continent, 1957 vs 2007")
```

JavaScript

```
plot(data(gm_eras), layer(box, dodge), x(col.continent), y(col.life),  
  color(col.era), y_label("Life expectancy"),  
  title("Life expectancy per continent, 1957 vs 2007"))
```



“Given the gapminder eras: boxes derived by `dodge`, x is continent, y is life, color by era.”

Stroke width and opacity stay *settings*. `color` and `pattern` can split one group into several boxes. `size` and `opacity` cannot: they would have to change inside a single box, and a box is one shape. `style(size = )` sets the width of every line a box draws, for the whole layer: the outline, the whiskers, the median and the end caps.

`box` joins the other distribution marks. `bar * bin` draws one variable’s histogram, and `interval` draws a range or an error bar. `box` gives the summary per group, and the violin (`ribbon * density`) gives the whole shape behind that summary. See [Transforms](#) for `dodge`, and for how `mean`, `median` and `range` differ from the box’s own summary.

14.7 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(pattern =)</code>	solid, hatch, crosshatch, grid, dots
<code>style(border_color =)</code>	any CSS color name or hex
<code>style(border_size =)</code>	pixels

And these vary per row if you map them to a column instead: `color()` (categories), `pattern()` (categories), `group()` (categories), `play()` (either).

A box is the one mark in the kernel that takes a **parameter**. The other parameters you have met belong to a transform (`bin(bins = )`), to a position (`x(gdp, scale = "log")`), or to `style()`. A box carries its summary inside itself, so the one choice that summary offers belongs to the mark. The plot below reads `gm_continents`: three continents, every year from 1952 to 2007. Each box therefore covers many more countries than the 2007 plots above:

R

```
data(gm_continents) + box(whiskers = "range") + x(continent) + y(life) +
  style(color = "lightsteelblue", border_color = "navy", border_size = 1.2) +
  y_label("Life expectancy") + title("Whiskers to the true extremes, and a
  ↪ border")
```

Python

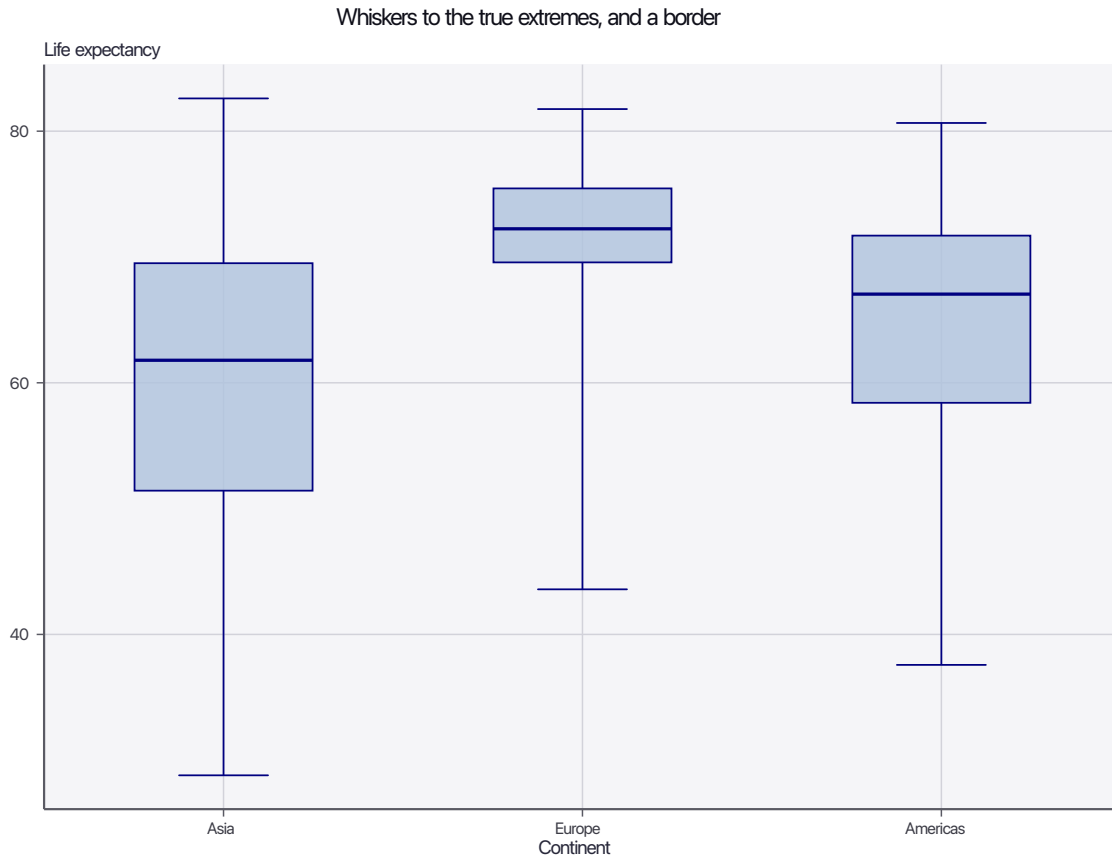
```
(data(gm_continents) + box(whiskers = "range") + x(col.continent) +
  ↪ y(col.life) +
  style(color = "lightsteelblue", border_color = "navy", border_size = 1.2) +
  y_label("Life expectancy") + title("Whiskers to the true extremes, and a
  ↪ border"))
```

Julia

```
data(gm_continents) + box(whiskers = "range") + x(:continent) + y(:life) +
  style(color = "lightsteelblue", border_color = "navy", border_size = 1.2) +
  y_label("Life expectancy") +
  title("Whiskers to the true extremes, and a border")
```

JavaScript

```
plot(data(gm_continents), box({ whiskers: "range" }), x(col.continent),
     y(col.life),
     style({ color: "lightsteelblue", border_color: "navy",
            border_size: 1.2 }), y_label("Life expectancy"),
     title("Whiskers to the true extremes, and a border"))
```



Both values of **whiskers** still draw a box, which is why it is a parameter and not a transform. It changes the rule the summary follows, not the shape the summary is drawn as.

border_color and **border_size** are worth using on a box specifically, because a box draws lines *and* a fill. Both settings apply to every line the box draws: the outline, the whiskers, the median and the caps. A pale fill with a dark edge stays legible at small sizes, and a solid fill does not.

14.8 What it refuses

A box draws lines and a fill, so its `pattern` takes a fill's texture and not a stroke's dash. "hatch", "crosshatch", "grid" and "dots" are accepted; "dashed" is refused.

```
data(gapminder_2007) + box + x(continent) + y(life) + style(pattern =  
  ↪ "dashed")
```

Error:

```
! gog: `style(pattern = )` on a `box`: `"dashed"` is a stroke's dash, not a  
  ↪ fill texture. Use "solid" (the default), "hatch", "crosshatch", "grid",  
  ↪ or "dots".
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A mapped `size` is the second. No row can vary a box's size: the box's width is the slot it sits in, and the line-work's width is a layer setting:

```
data(gapminder_2007) + box + x(continent) + y(life) + size(life)
```

Error:

```
! gog: `size` cannot be bound to `box` - a box has no size feature. Remove  
  ↪ the `life` mapping from `size`, or use a mark that has one.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

The third refusal is above, under Why box needs no transform: a box already summarizes, so it takes none.

15 Ribbon

How wide is the band of values at each point along x? `ribbon` draws a filled band from a low boundary to a high one across x: a confidence band, a spread band, the shaded region around a trend. It takes `area`'s single filled region and gives it `interval`'s low and high pair. The region then closes on a *second data boundary* instead of on a baseline at zero. Where an area fills from the data down to zero, a ribbon fills between two boundaries the data supplies.

15.1 A band from a range

The two boundaries come from a transform, exactly as an interval's two extents do. `range` reduces y to its minimum and maximum within each x, and the ribbon fills between them. Here, the spread of life expectancy across five Asian countries, year by year:

R

```
data(gapminder_asia) + ribbon * range + x(year) + y(life) +  
  y_label("Life expectancy") +  
  title("The min-max spread across five Asian countries, per year")
```

Python

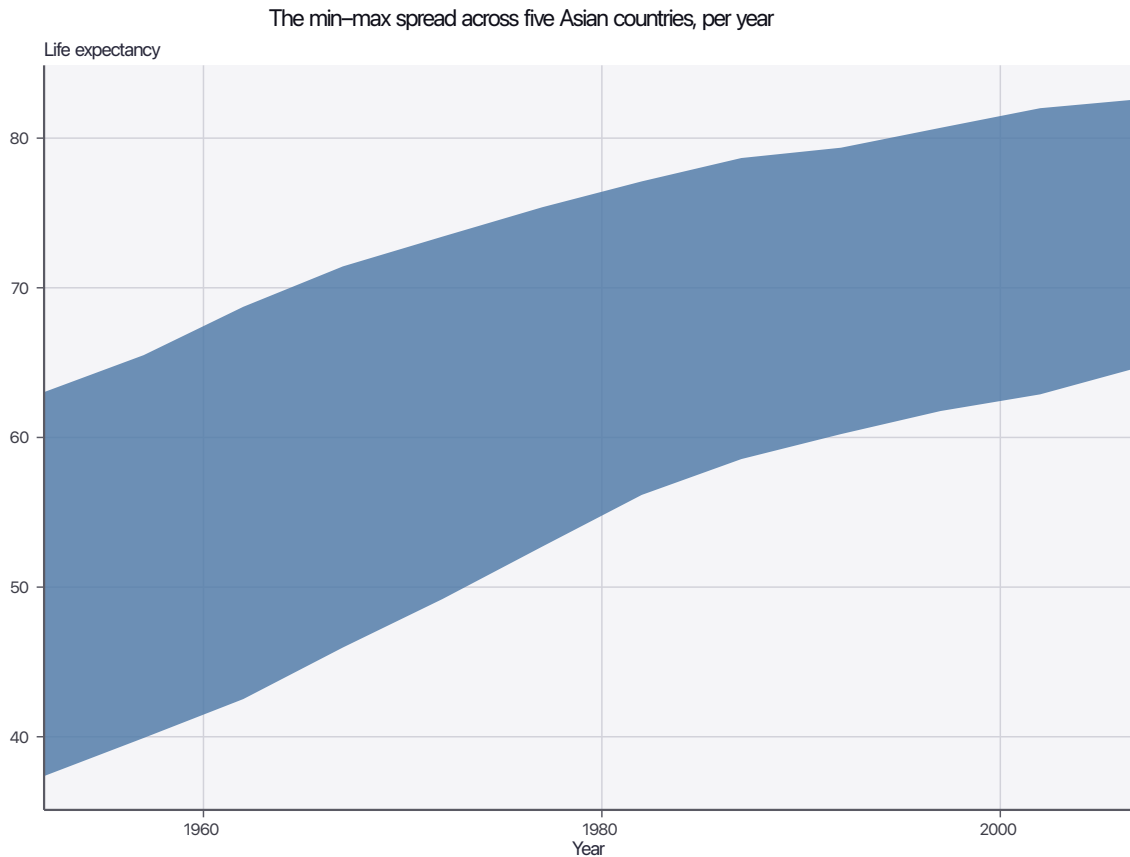
```
(data(gapminder_asia) + ribbon * range + x(col.year) + y(col.life) +  
  y_label("Life expectancy") +  
  title("The min-max spread across five Asian countries, per year"))
```

Julia

```
data(gapminder_asia) + ribbon * range + x(:year) + y(:life) +  
  y_label("Life expectancy") +  
  title("The min-max spread across five Asian countries, per year")
```

JavaScript

```
plot(data(gapminder_asia), layer(ribbon, range), x(col.year), y(col.life),
     y_label("Life expectancy"),
     title("The min-max spread across five Asian countries, per year"))
```



“Given gapminder Asia: a ribbon derived by range, x is year, y is life.”

Every year, the band’s lower edge is the shortest-lived country and its upper edge the longest-lived; the band is the region that contains all five. Read it as `interval * range` drawn as a *continuous* fill rather than a whisker at each x. The pair is the same; only the geometry differs.

15.2 The mean and its band

On its own a band shows spread but not center. A band is usually drawn with a line through the middle. You compose the two by layering, not by adding an option to either mark. A

ribbon * range for the spread, a line * mean for the trend, sharing one x and y:

R

```
data(gapminder_asia) + x(year) + y(life) +  
  ribbon * range + style(color = "steelblue", opacity = 0.25) +  
  line * mean + style(color = "steelblue", size = 2) +  
  y_label("Life expectancy") +  
  title("Mean life expectancy, with its min-max band")
```

Python

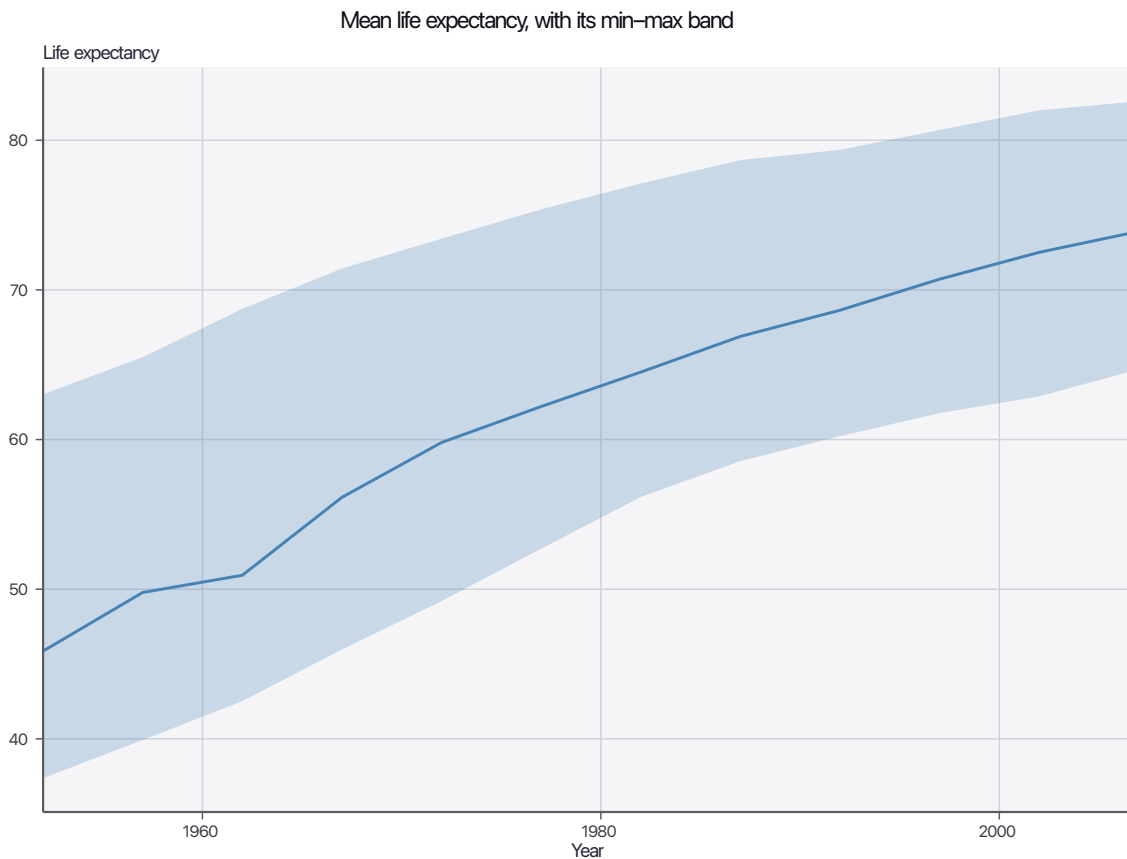
```
(data(gapminder_asia) + x(col.year) + y(col.life) +  
  ribbon * range + style(color = "steelblue", opacity = 0.25) +  
  line * mean + style(color = "steelblue", size = 2) +  
  y_label("Life expectancy") +  
  title("Mean life expectancy, with its min-max band"))
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + ribbon * range +  
  style(color = "steelblue", opacity = 0.25) + line * mean +  
  style(color = "steelblue", size = 2) + y_label("Life expectancy") +  
  title("Mean life expectancy, with its min-max band")
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), layer(ribbon, range),  
  style({ color: "steelblue", opacity: 0.25 }), layer(line, mean),  
  style({ color: "steelblue", size: 2 }), y_label("Life expectancy"),  
  title("Mean life expectancy, with its min-max band"))
```

“Given gapminder Asia: x is year, y is life, a ribbon derived by range and also a line derived by mean.”

Each layer runs its own transform on the shared data: `mean` collapses each year to one point for the line, `range` to a low/high pair for the band. This is the most common ribbon plot: a trend with its uncertainty or spread drawn around it. It is two marks the grammar already has, added with `+`.

15.3 A fan chart of nested bands

`gm_all` is the whole gapminder table: every country, in every year, 142 countries in all. A min-max band over it shows what every country did, but only the two extreme countries set its edges. Naming two quantiles instead gives a band that contains most of the countries. Layer two bands of different widths and you have a **fan chart**:

R

```
data(gm_all) + x(year) + y(life) +
  ribbon * range(0.1, 0.9) + style(color = "steelblue", opacity = 0.2) +
  ribbon * range(0.25, 0.75) + style(color = "steelblue", opacity = 0.35) +
  line * median + style(color = "steelblue", size = 2) +
  y_label("Life expectancy") +
  title("Life expectancy across 142 countries, 1952 to 2007")
```

Python

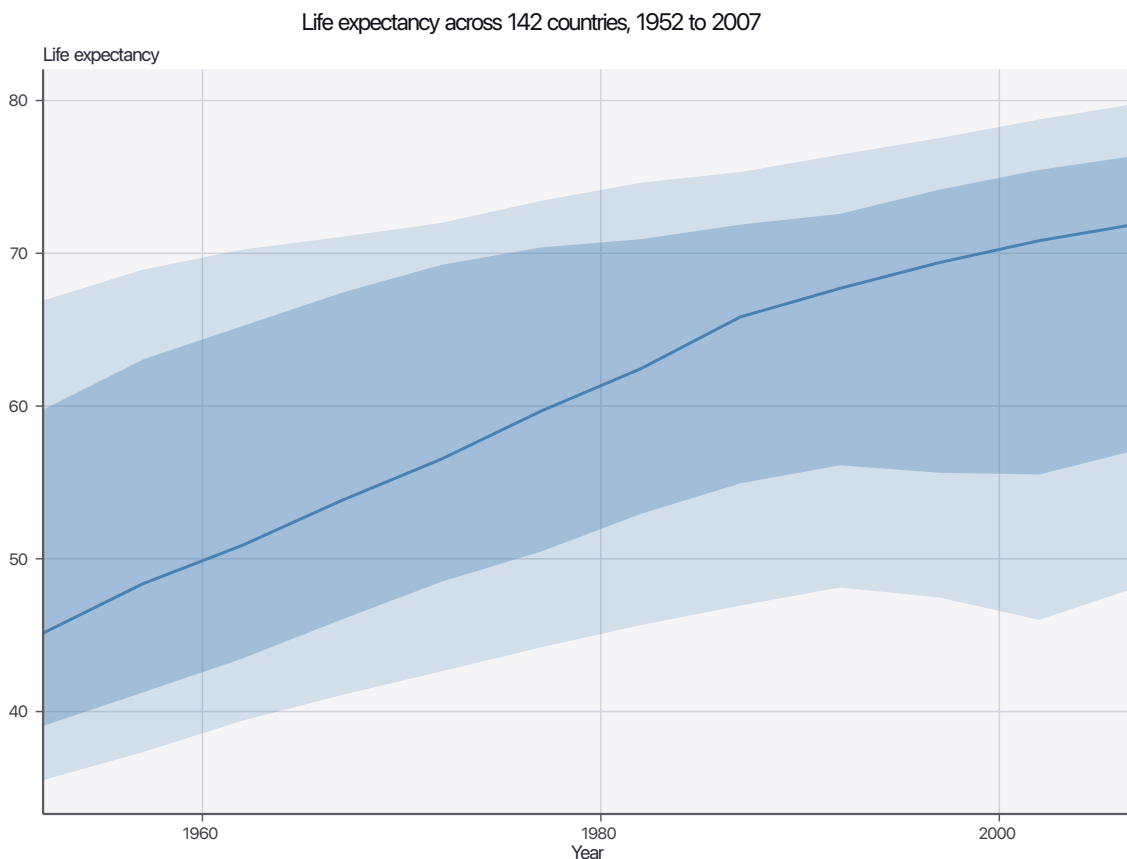
```
(data(gm_all) + x(col.year) + y(col.life) +
  ribbon * range(0.1, 0.9) + style(color = "steelblue", opacity = 0.2) +
  ribbon * range(0.25, 0.75) + style(color = "steelblue", opacity = 0.35) +
  line * median + style(color = "steelblue", size = 2) +
  y_label("Life expectancy") +
  title("Life expectancy across 142 countries, 1952 to 2007"))
```

Julia

```
data(gm_all) + x(:year) + y(:life) + ribbon * range(0.1, 0.9) +
  style(color = "steelblue", opacity = 0.2) + ribbon * range(0.25, 0.75) +
  style(color = "steelblue", opacity = 0.35) + line * median +
  style(color = "steelblue", size = 2) + y_label("Life expectancy") +
  title("Life expectancy across 142 countries, 1952 to 2007")
```

JavaScript

```
plot(data(gm_all), x(col.year), y(col.life),
  layer(ribbon, range(0.1, 0.9)),
  style({ color: "steelblue", opacity: 0.2 }),
  layer(ribbon, range(0.25, 0.75)),
  style({ color: "steelblue", opacity: 0.35 }), layer(line, median),
  style({ color: "steelblue", size: 2 }), y_label("Life expectancy"),
  title("Life expectancy across 142 countries, 1952 to 2007"))
```



“Given all the gapminder years: x is year, y is life, a ribbon derived by range from 0.1 to 0.9, and also a ribbon derived by range from 0.25 to 0.75, and also a line derived by median.”

The pale band contains the middle 80% of countries. The darker one contains the middle half, and the line is the median. The plot then says something the min-max version cannot. The median rose from 45 years to 72, and the 90th percentile from 67 to 80, so the distance between them fell from 22 years to 8.

The lower edges of the two bands do not rise steadily. Both lower edges, the 10th percentile and the 25th, decline between 1992 and 2002 while the median rises throughout. A min-max band cannot show this, because its lower edge is one country.

Each band is one more layer, so a third pair adds a third. Write the widest band first, because a later layer is painted over an earlier one.

15.4 Why a ribbon needs a transform

A low *and* a high are two numbers, and a plain `y()` supplies one. An **area** closes on zero, which the grammar supplies. A ribbon has no such second number, so like **interval** its minimum syllable includes a range-producing transform. Ask for one without it and the engine says so rather than drawing an empty panel:

```
data(gapminder_asia) + ribbon + x(year) + y(life)
```

Error:

```
! gog: `ribbon` draws a span from a low value to a high one, but nothing here
  ↳ produces those extents. Add a range transform - `ribbon * range + x(t) +
  ↳ y(value)` draws a band between the min and max at each x, or `ribbon *
  ↳ bounds(lo, hi)` a pre-computed one.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

This is the same design choice that keeps the **channel vocabulary** small. The alternative would add `ymin` and `ymax` as two more channels meaningful to a single mark. Instead the boundaries are invented by a transform, exactly as **bin** invents a count. The same low and high rows then serve the whisker, the box and the band alike. See **Transforms** for the **range**, **confidence** and **deviation** statistics.

Read the requirement as *where do the two boundaries come from?* rather than as a list of transforms. One answer looks nothing like the others: **density** over a category gives a ribbon its boundaries by **reflection**. That is **the violin**, at the end of this chapter.

15.5 A confidence band

range shows the full spread. For the **uncertainty of the mean**, use **confidence**. It computes a t-interval for the mean at each `x`, and the ribbon fills between its two bounds. The interval widens when the group is small:

R

```
data(gapminder_asia) + ribbon * confidence(0.95) + x(year) + y(life) +
  y_label("Life expectancy") +
  title("95% confidence band for mean life expectancy")
```

Python

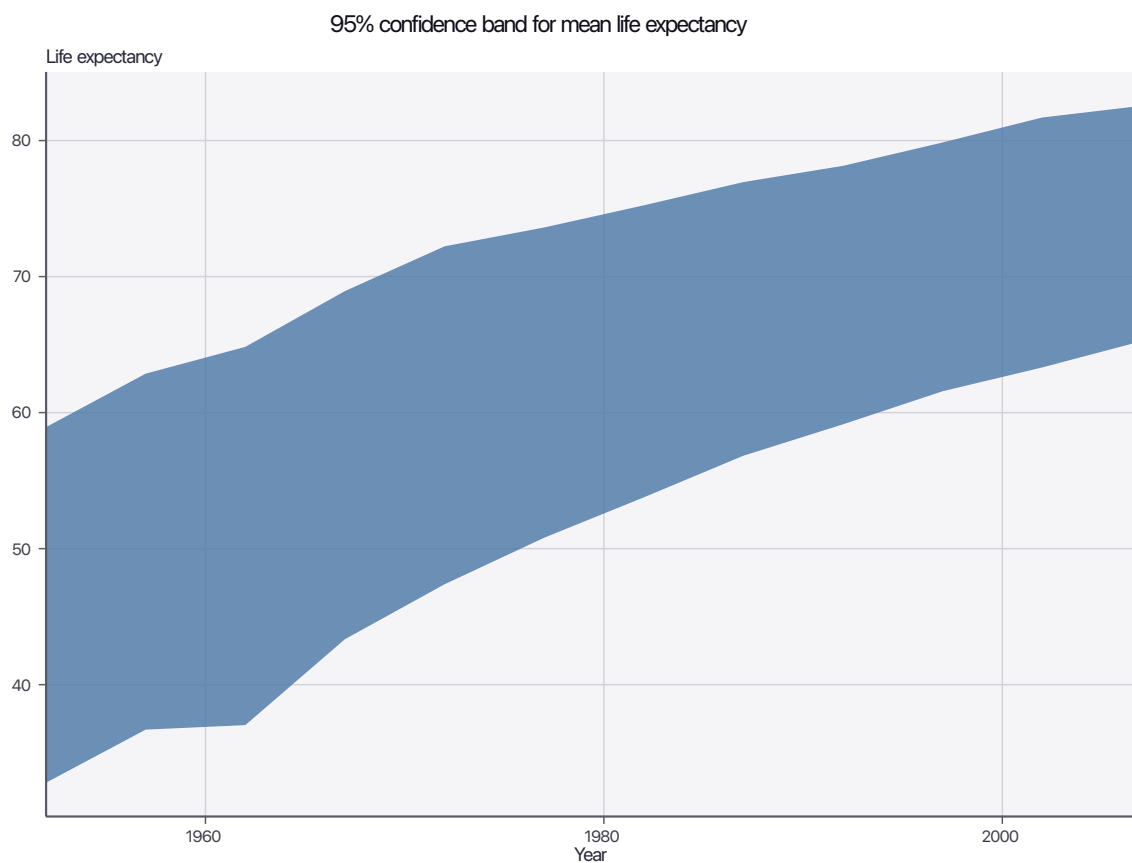
```
(data(gapminder_asia) + ribbon * confidence(0.95) + x(col.year) + y(col.life)
  ↪ +
  y_label("Life expectancy") +
  title("95% confidence band for mean life expectancy"))
```

Julia

```
data(gapminder_asia) + ribbon * confidence(0.95) + x(:year) + y(:life) +
  y_label("Life expectancy") +
  title("95% confidence band for mean life expectancy")
```

JavaScript

```
plot(data(gapminder_asia), layer(ribbon, confidence(0.95)), x(col.year),
  y(col.life), y_label("Life expectancy"),
  title("95% confidence band for mean life expectancy"))
```



“Given gapminder Asia: a ribbon derived by confidence at 95%, x is year, y is life.”

The band is wide because each mean comes from only five countries. Those five differ by more than fifteen years, so the interval around their mean is wide. A higher level draws a wider band still. The statistic you ask for chooses the band’s meaning: the observed spread with **range**, the uncertainty of the mean with **confidence**. There is no new mark to learn, which is the rule the **interval** family follows.

15.6 A pre-computed band

range, **confidence** and **deviation** compute the band *from raw values in the plot*. But most confidence bands are not computed in the plot at all. A model’s standard error, a standard error of measurement, a bootstrap interval: each is computed **before the plot**, and its bounds arrive as two columns. **gog** draws what you computed; it does not re-fit the model. The **score_band** table holds one row per raw test score, with the score a model predicts for it and the two edges of the band around that prediction:

Table 15.1: **score_band**: first 5 of 21 rows

score	expected	lower	upper
0	0	-1.610304	1.610304
1	1	-1.865557	3.865557
2	2	-1.944424	5.944424
3	3	-1.694803	7.694803
4	4	-1.259232	9.259232

The ribbon reads those edge columns directly, through **bounds(lower, upper)**:

R

```
data(score_band) + ribbon * bounds(lower, upper) + x(score) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("A pre-computed measurement band")
```

Python

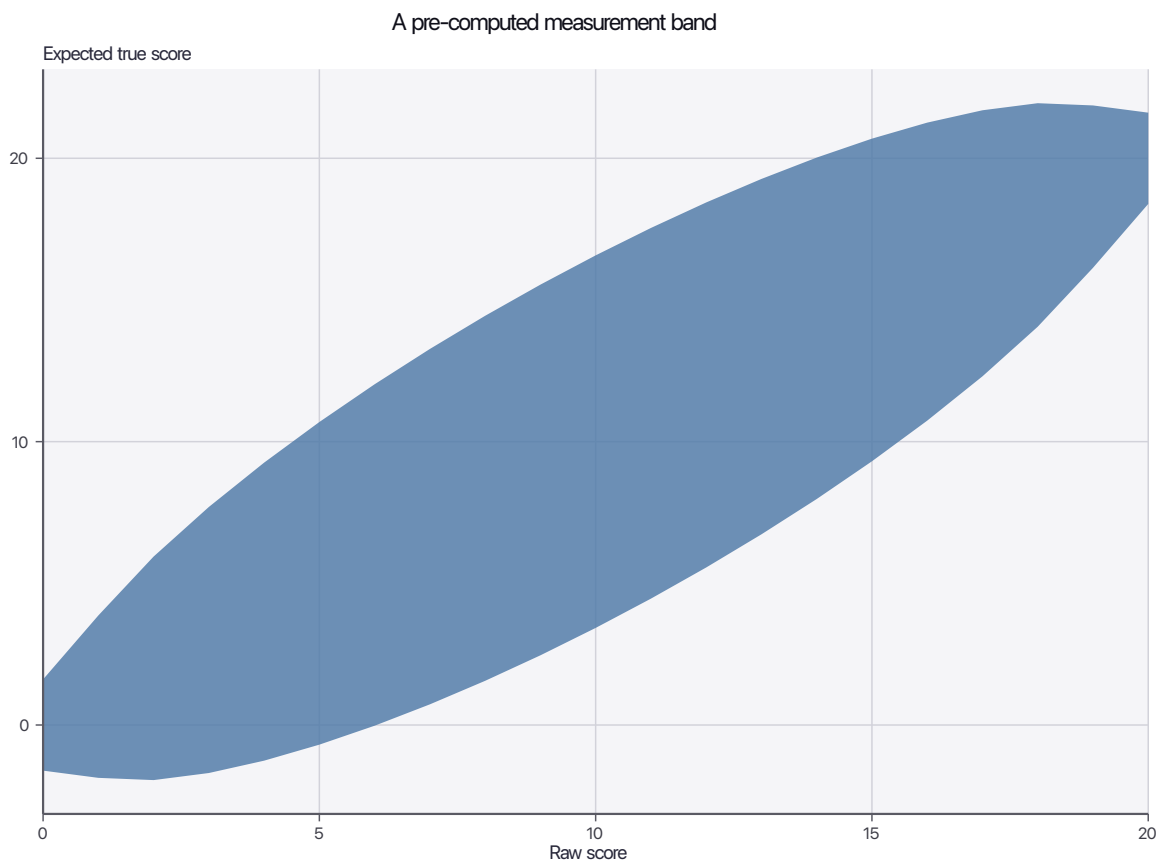
```
(data(score_band) + ribbon * bounds(col.lower, col.upper) + x(col.score) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("A pre-computed measurement band"))
```

Julia

```
data(score_band) + ribbon * bounds(:lower, :upper) + x(:score) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("A pre-computed measurement band")
```

JavaScript

```
plot(data(score_band), layer(ribbon, bounds(col.lower, col.upper)),  
  x(col.score), y_label("Expected true score"), x_label("Raw score"),  
  title("A pre-computed measurement band"))
```



“Given the score band: a ribbon from lower to upper, x is score.”

bounds is **range**’s counterpart, and it computes nothing. It reshapes the two columns you give it into the same low and high pair **range** produces. The band needs no **y()**, because the two

columns *are* the boundaries. It serves the case `range`, `confidence` and `deviation` cannot: a band computed before the data reached the plot.

It is usually drawn with the curve the band surrounds. Here that curve is the **expected true score**, the score the model predicts for each raw score. A dashed `line` draws it, with a point at every raw score. The band, the line and the points are three layers composed with `+`:

R

```
data(score_band) + x(score) + y(expected) +
  ribbon * bounds(lower, upper) + style(color = "steelblue", opacity = 0.18)
↪ +
  line + style(color = "steelblue", size = 1.4, pattern = "dashed") +
  point + style(color = "steelblue", size = 2.4) +
  y_label("Expected true score") + x_label("Raw score") +
  title("Expected score with its measurement error band")
```

Python

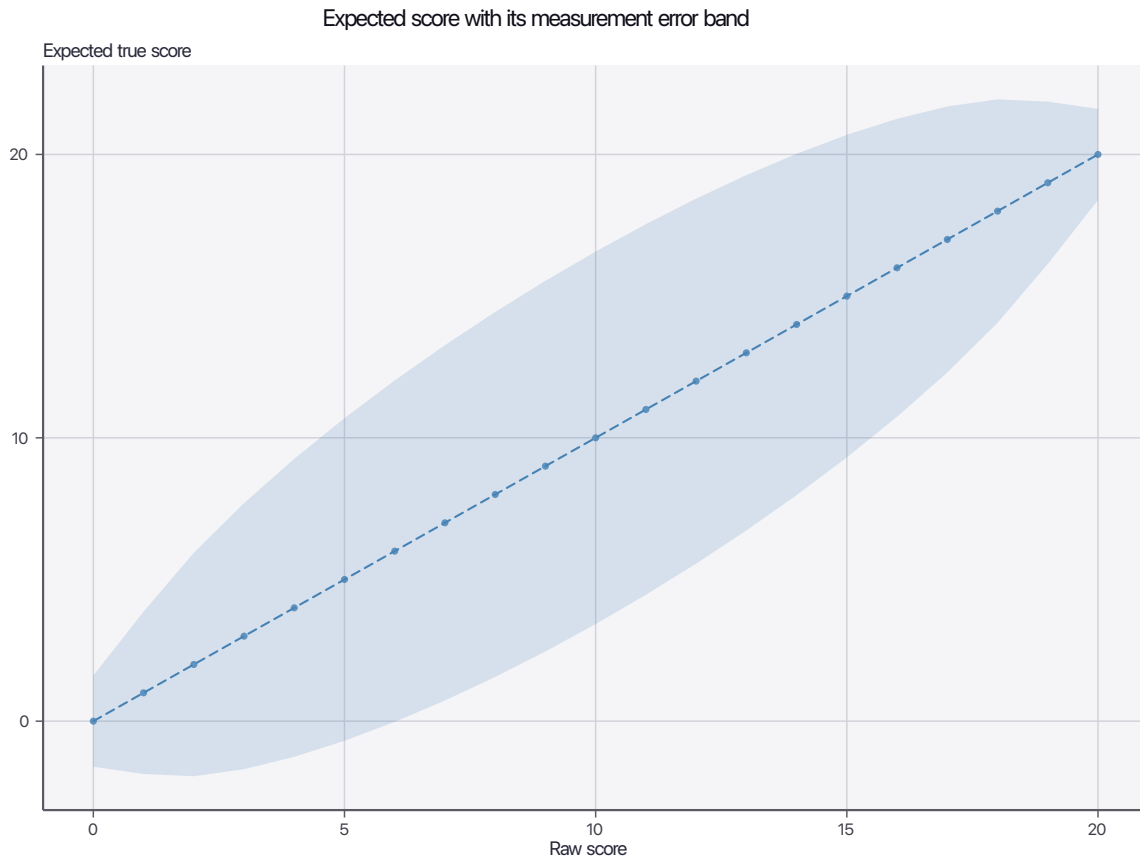
```
(data(score_band) + x(col.score) + y(col.expected) +
  ribbon * bounds(col.lower, col.upper) + style(color = "steelblue", opacity
↪ = 0.18) +
  line + style(color = "steelblue", size = 1.4, pattern = "dashed") +
  point + style(color = "steelblue", size = 2.4) +
  y_label("Expected true score") + x_label("Raw score") +
  title("Expected score with its measurement error band"))
```

Julia

```
data(score_band) + x(:score) + y(:expected) +
  ribbon * bounds(:lower, :upper) +
  style(color = "steelblue", opacity = 0.18) + line +
  style(color = "steelblue", size = 1.4, pattern = "dashed") + point +
  style(color = "steelblue", size = 2.4) +
  y_label("Expected true score") + x_label("Raw score") +
  title("Expected score with its measurement error band")
```

JavaScript


```
plot(data(score_band), x(col.score), y(col.expected),
     layer(ribbon, bounds(col.lower, col.upper)),
     style({ color: "steelblue", opacity: 0.18 }), line,
     style({ color: "steelblue", size: 1.4, pattern: "dashed" }), point,
     style({ color: "steelblue", size: 2.4 }),
     y_label("Expected true score"), x_label("Raw score"),
     title("Expected score with its measurement error band"))
```



“Given the score band: x is score, y is expected, a ribbon from lower to upper, and also a line and points.”

The dashed line is `style(pattern = )`: on a stroke the texture is a **dash**, so it applies to the **line** here, not to the band. The band takes the same setting, with different values. On a fill the texture is a **hatch**: "hatch", "crosshatch", "grid" or "dots", never "dashed".

15.7 Filled, or two dashed edges

A band is a low/high pair, and the *mark* chooses how to draw it: the same [No Exceptions](#) rule that gives one bin a bar, a line, and a step histogram. `ribbon` fills the pair; a `line` traces its **two boundaries**. That is the unfilled band many journals prefer, and with `style(pattern = )` it becomes the dashed pair of curves they often draw:

R

```
data(score_band) + x(score) + y(expected) +  
  line * bounds(lower, upper) + style(color = "gray", pattern = "dashed",  
  ↪ size = 1.2) +  
  line + style(color = "steelblue", size = 1.5) +  
  point + style(color = "steelblue", size = 2.4) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("The same band, unfilled: two dashed edges")
```

Python

```
(data(score_band) + x(col.score) + y(col.expected) +  
  line * bounds(col.lower, col.upper) + style(color = "gray", pattern =  
  ↪ "dashed", size = 1.2) +  
  line + style(color = "steelblue", size = 1.5) +  
  point + style(color = "steelblue", size = 2.4) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("The same band, unfilled: two dashed edges"))
```

Julia

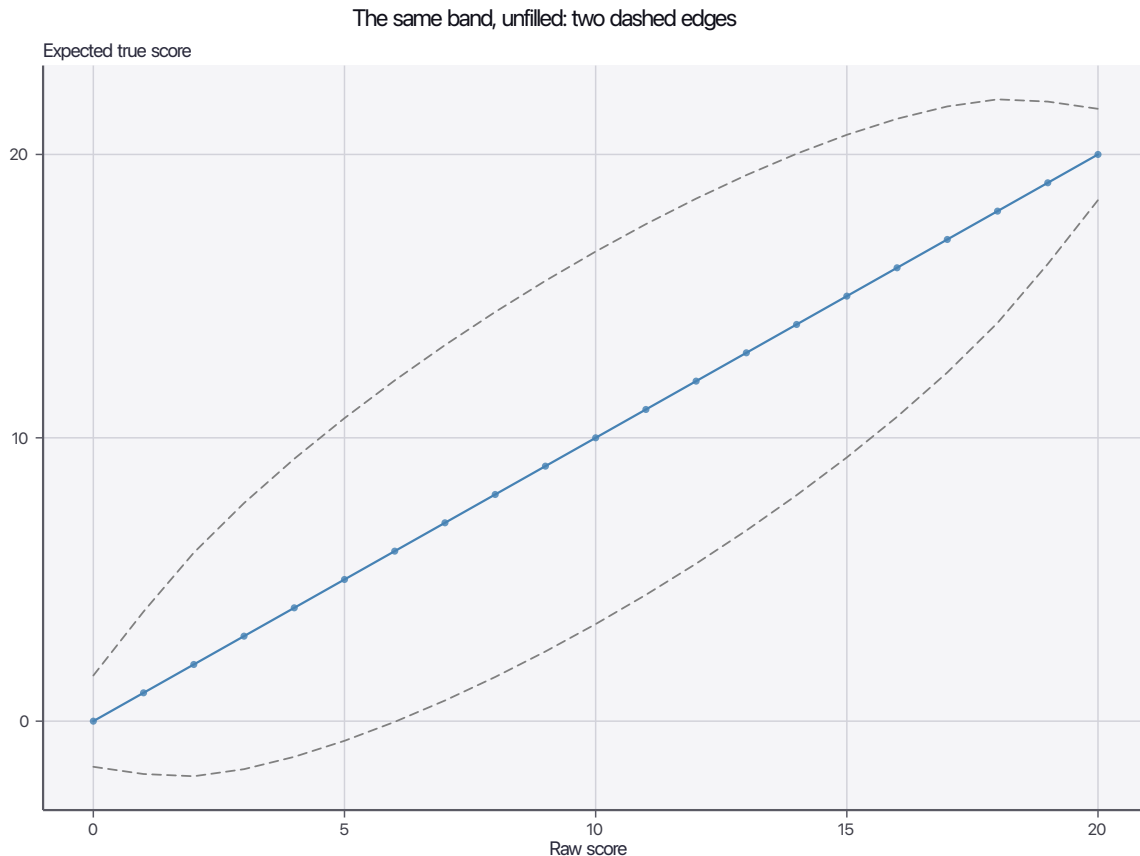
```
data(score_band) + x(:score) + y(:expected) +  
  line * bounds(:lower, :upper) +  
  style(color = "gray", pattern = "dashed", size = 1.2) + line +  
  style(color = "steelblue", size = 1.5) + point +  
  style(color = "steelblue", size = 2.4) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("The same band, unfilled: two dashed edges")
```

JavaScript

```

plot(data(score_band), x(col.score), y(col.expected),
     layer(line, bounds(col.lower, col.upper)),
     style({ color: "gray", pattern: "dashed", size: 1.2 }), line,
     style({ color: "steelblue", size: 1.5 }), point,
     style({ color: "steelblue", size: 2.4 }),
     y_label("Expected true score"), x_label("Raw score"),
     title("The same band, unfilled: two dashed edges"))

```



“Given the score band: x is score, y is expected, lines from lower to upper, and also a line and points.”

`line * bounds` is one layer that draws *both* edges, splitting the pair into a low curve and a high one. It therefore needs no second `y`. Two plain `line` layers can draw two columns you already have. They cannot split a pair that `range` computes, because that pair never becomes a column you can name. `line * bounds` reads any pair the same way: `line * range` traces a min-max envelope, `line * confidence` the two edges of a confidence interval, and `step *`

`bounds` draws them as staircases. Filled or unfilled, solid or dashed, it is one pair and a choice of mark, never a new option added to the band.

15.8 One band per group

A categorical column *splits* a ribbon into one band per group, the same split `area` and `line` make. Here each continent gets its own band, the range of its countries' life expectancy over time:

R

```
data(gm_continents) + ribbon * range + x(year) + y(life) + color(continent) +  
  y_label("Life expectancy") +  
  title("Life expectancy spread, three continents")
```

Python

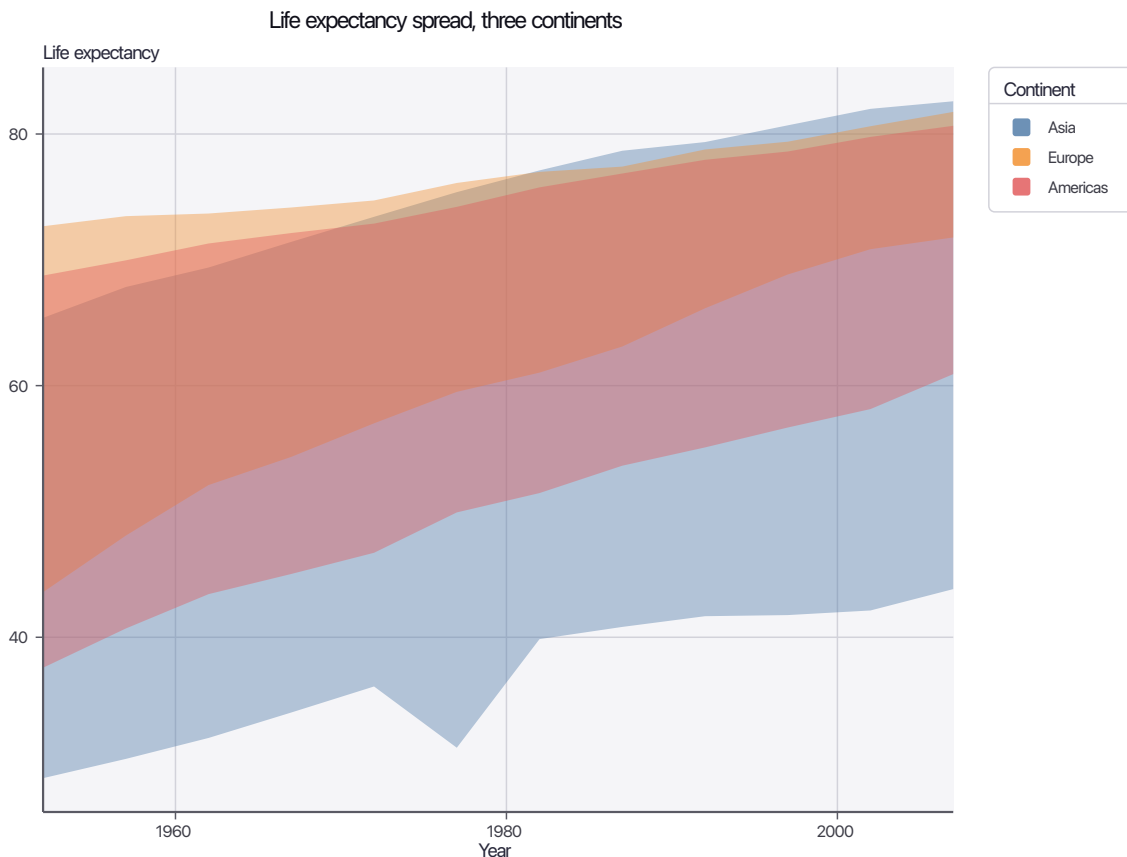
```
(data(gm_continents) + ribbon * range + x(col.year) + y(col.life) +  
  ↪ color(col.continent) +  
  y_label("Life expectancy") +  
  title("Life expectancy spread, three continents"))
```

Julia

```
data(gm_continents) + ribbon * range + x(:year) + y(:life) +  
  color(:continent) + y_label("Life expectancy") +  
  title("Life expectancy spread, three continents")
```

JavaScript

```
plot(data(gm_continents), layer(ribbon, range), x(col.year), y(col.life),  
  color(col.continent), y_label("Life expectancy"),  
  title("Life expectancy spread, three continents"))
```



“Given the gapminder continents: ribbons derived by range, x is year, y is life, color by continent.”

The bands overlap where two continents cover the same range of life expectancy. A split **area** stays opaque, and **stack** is its answer. A ribbon instead draws its split **translucent** by default, so a band behind another still shows through. That difference is principled: an area measures a height from the baseline, so its groups can be piled into a total; a ribbon already spans a low to a high and measures no such height, so there is nothing to stack. Overlapping bands are read through transparency, never stacked, which is why **ribbon * stack** is refused and the refusal names `style(opacity = )`.

15.9 One region, one fill

A ribbon is a single filled region, so **opacity** is a *setting* here, not a channel, exactly as on **area**. A row is a vertex of a boundary, not a region of its own, so a per-row opacity has nothing to vary along.

```
render_svg(data(gapminder_asia) + ribbon * range + x(year) + y(life) +
  ↪ opacity(gdp))
```

Error:

```
! gog: `opacity` cannot be bound to `ribbon` - a ribbon has no opacity
  ↪ feature. Remove the `gdp` mapping from `opacity`, or use a mark that has
  ↪ one.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

And `size` cannot even be set: a band's extent comes from its `x` and its two boundaries, so there is no width to choose. Its edge, if you want one, is a line layered along a boundary (`ribbon + line`, the composition the pre-computed band drew above), never an outline option on the mark.

15.10 A category on the domain

A ribbon takes a category on `x`, the same way an `area` does, and fills the spread across the categories in axis order:

R

```
data(gapminder_2007) + ribbon * range + line * mean + x(continent) + y(life)
  ↪ +
  y_label("Life expectancy") +
  title("ribbon * range + line * mean: spread and center, per continent")
```

Python

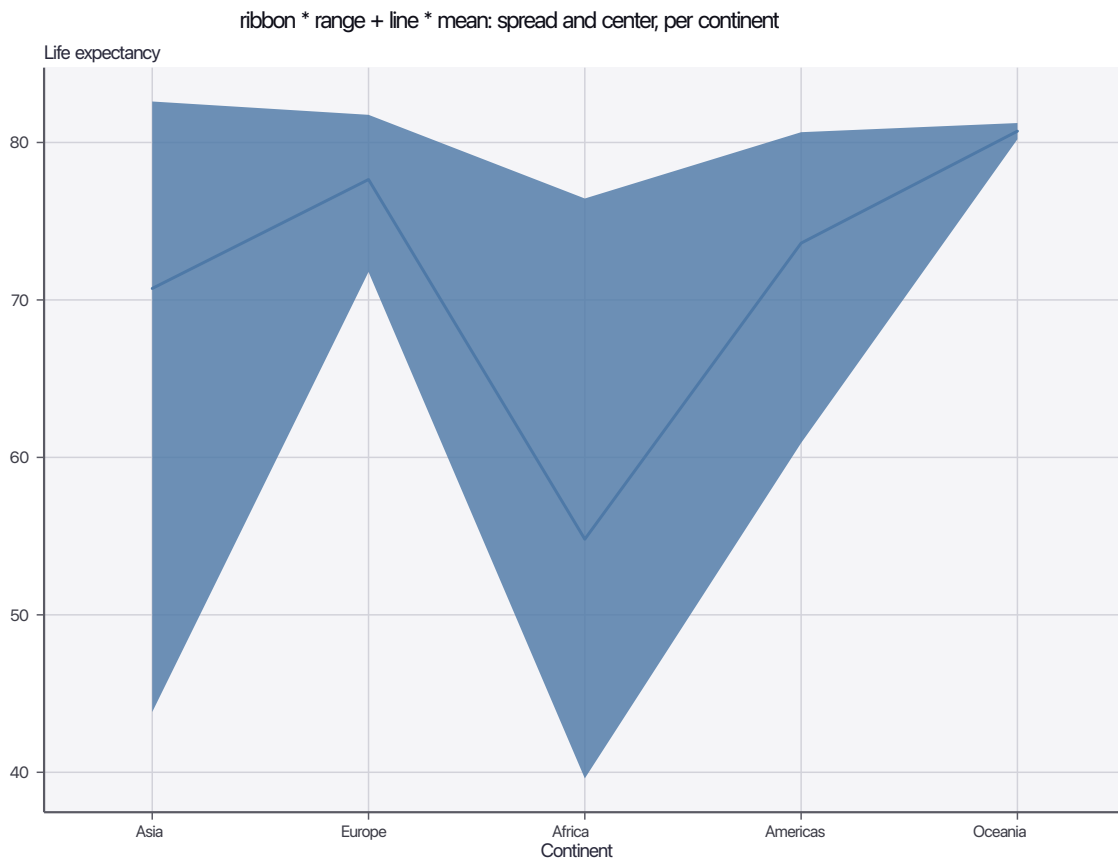
```
(data(gapminder_2007) + ribbon * range + line * mean + x(col.continent) +
  ↪ y(col.life) +
  y_label("Life expectancy") +
  title("ribbon * range + line * mean: spread and center, per continent"))
```

Julia

```
data(gapminder_2007) + ribbon * range + line * mean + x(:continent) +
  y(:life) + y_label("Life expectancy") +
  title("ribbon * range + line * mean: spread and center, per continent")
```

JavaScript

```
plot(data(gapminder_2007), layer(ribbon, range), layer(line, mean),
     x(col.continent), y(col.life), y_label("Life expectancy"),
     title("ribbon * range + line * mean: spread and center, per continent"))
```



For most such data an `interval` is the better sentence. It draws a separate whisker at each category and claims nothing about the space between them. A ribbon instead slopes from one category to the next, across positions where no rows exist. The grammar does not decide that for you: both sentences are well formed, and which one fits *your* data is a question about the data.

R

```
data(gapminder_2007) + interval * range + x(continent) + y(life) +
  y_label("Life expectancy") +
  title("interval * range: the same numbers, claiming less")
```

Python

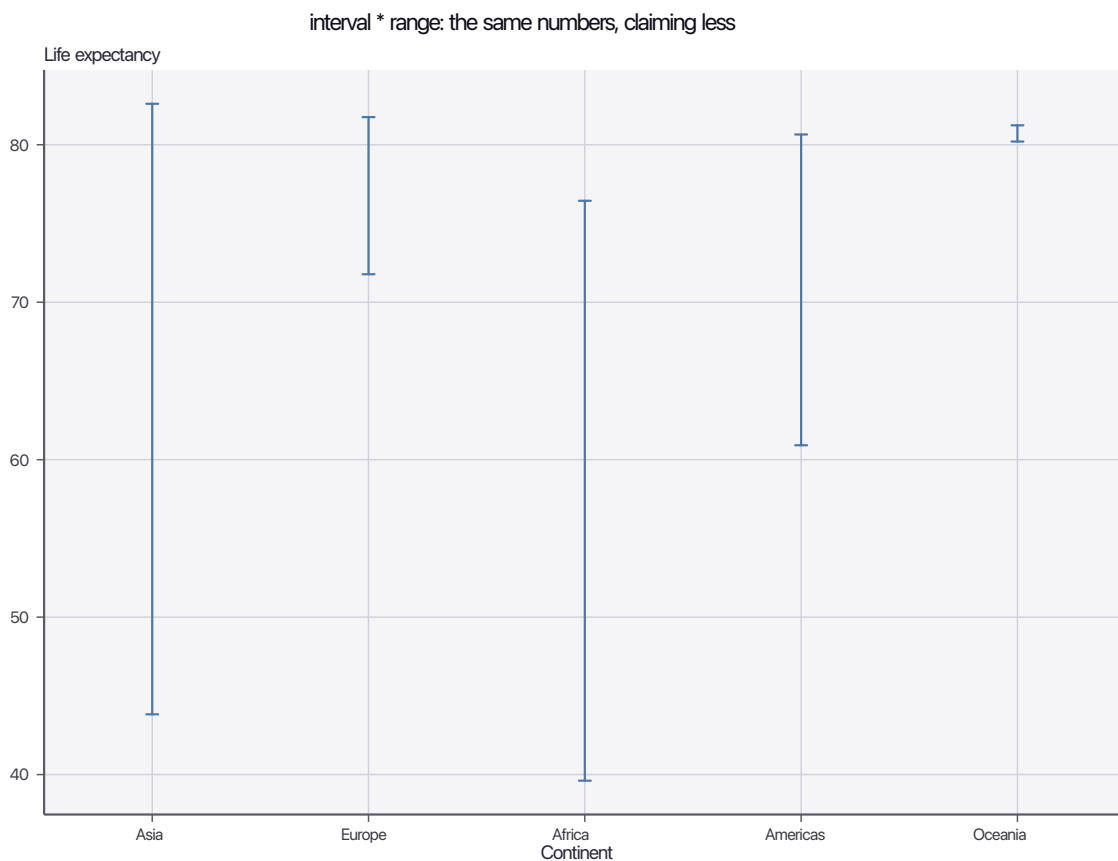
```
(data(gapminder_2007) + interval * range + x(col.continent) + y(col.life) +  
  y_label("Life expectancy") +  
  title("interval * range: the same numbers, claiming less"))
```

Julia

```
data(gapminder_2007) + interval * range + x(:continent) + y(:life) +  
  y_label("Life expectancy") +  
  title("interval * range: the same numbers, claiming less")
```

JavaScript

```
plot(data(gapminder_2007), layer(interval, range), x(col.continent),  
  y(col.life), y_label("Life expectancy"),  
  title("interval * range: the same numbers, claiming less"))
```



“Given gapminder 2007: intervals derived by range, x is continent, y is life.”

15.11 The violin: a distribution per category

A **range** on a category gives two numbers per group, the lowest and the highest. **density** gives a whole *shape* per group, and a ribbon has room for that shape: the estimate spreads along the measure and is drawn across the category’s slot, mirrored. That is the violin plot (Hintze & Nelson, 1998), and it is this chapter’s sentence with one word changed:

R

```
data(gapminder_2007) + ribbon * density + x(continent) + y(life) +  
  y_label("Life expectancy") +  
  title("ribbon * density: one distribution per continent")
```

Python

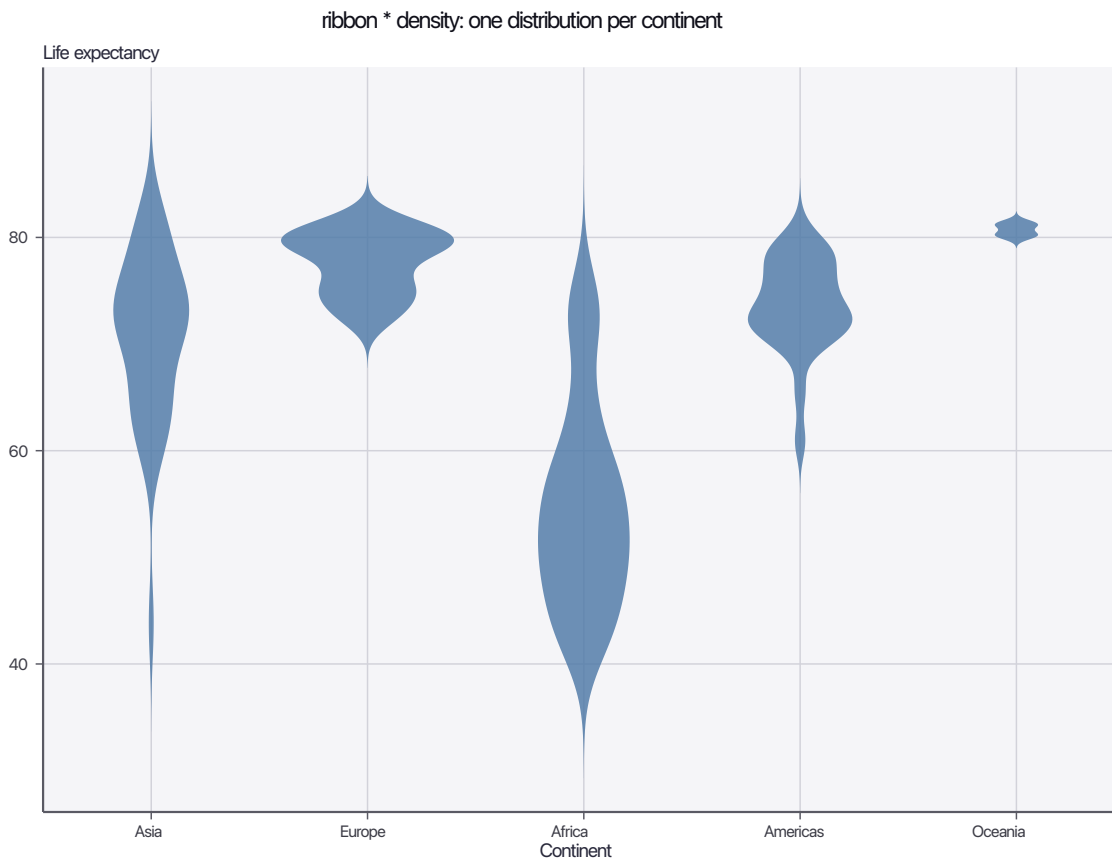
```
(data(gapminder_2007) + ribbon * density + x(col.continent) + y(col.life) +  
  y_label("Life expectancy") +  
  title("ribbon * density: one distribution per continent"))
```

Julia

```
data(gapminder_2007) + ribbon * density + x(:continent) + y(:life) +  
  y_label("Life expectancy") +  
  title("ribbon * density: one distribution per continent")
```

JavaScript

```
plot(data(gapminder_2007), layer(ribbon, density), x(col.continent),  
  y(col.life), y_label("Life expectancy"),  
  title("ribbon * density: one distribution per continent"))
```



“Given gapminder 2007: ribbons derived by density, x is continent, y is life.”

Nothing new was added to the grammar to draw it. A ribbon closes on a second boundary the data supplies, and a violin closes on the mirror image of its own curve. **density** is the transform that estimates a distribution, and this plot is that estimate read per group. There is no **violin** word to learn, because a word for it would be four things at once (the mark, the statistic, the mirror, and the slot), and each of those is already said by something you know.

Read it against the **box** of the same two columns. A box gives you five numbers you can name; the violin gives you the shape those five numbers summarize. Africa’s violin has two peaks, which its median hides.

R

```
data(gapminder_2007) + box + x(continent) + y(life) +
  y_label("Life expectancy") + title("box + x + y: the same distributions,
  ↪ summarized")
```

Python

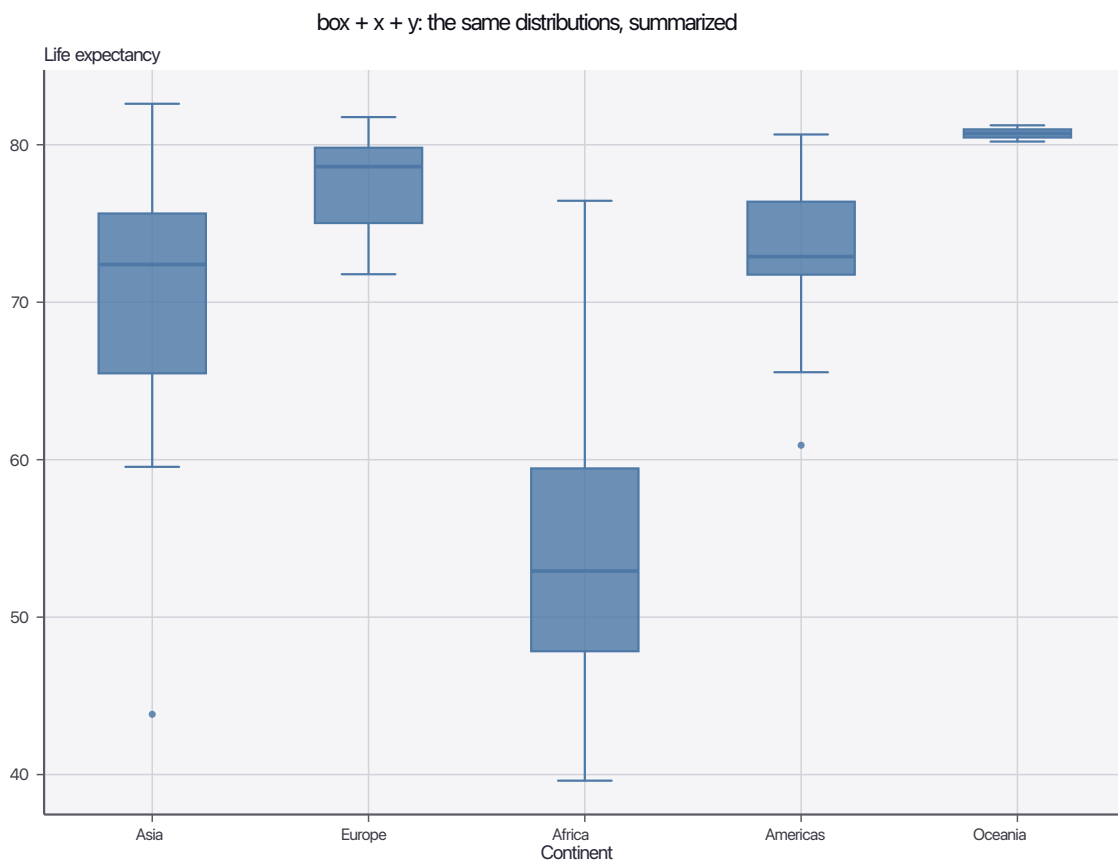
```
(data(gapminder_2007) + box + x(col.continent) + y(col.life) +  
  y_label("Life expectancy") + title("box + x + y: the same distributions,  
  ↪ summarized"))
```

Julia

```
data(gapminder_2007) + box + x(:continent) + y(:life) +  
  y_label("Life expectancy") +  
  title("box + x + y: the same distributions, summarized")
```

JavaScript

```
plot(data(gapminder_2007), box, x(col.continent), y(col.life),  
  y_label("Life expectancy"),  
  title("box + x + y: the same distributions, summarized"))
```



“Given gapminder 2007: boxes, x is continent, y is life.”

15.11.1 What the width means

Every group’s density integrates to 1 on its own. Left at that, the widths would say nothing about how many rows each group has. A continent of two countries would be drawn as wide as a continent of fifty-two. So by default the estimate is weighted by the group’s row count, and width means the same thing across the whole panel. Oceania above holds two countries, so its violin is drawn narrow.

`compare = "shape"` removes that weighting and draws every violin to the same area. It is the right reading when the question is about the shapes alone. It is the wrong one when a small group’s narrow, tall estimate sets the width for every other violin:

R

```
data(gapminder_2007) + ribbon * density(compare = "shape") + x(continent) +  
  ↪ y(life) +  
  y_label("Life expectancy") +  
  title('density(compare = "shape"): every violin the same area')
```

Python

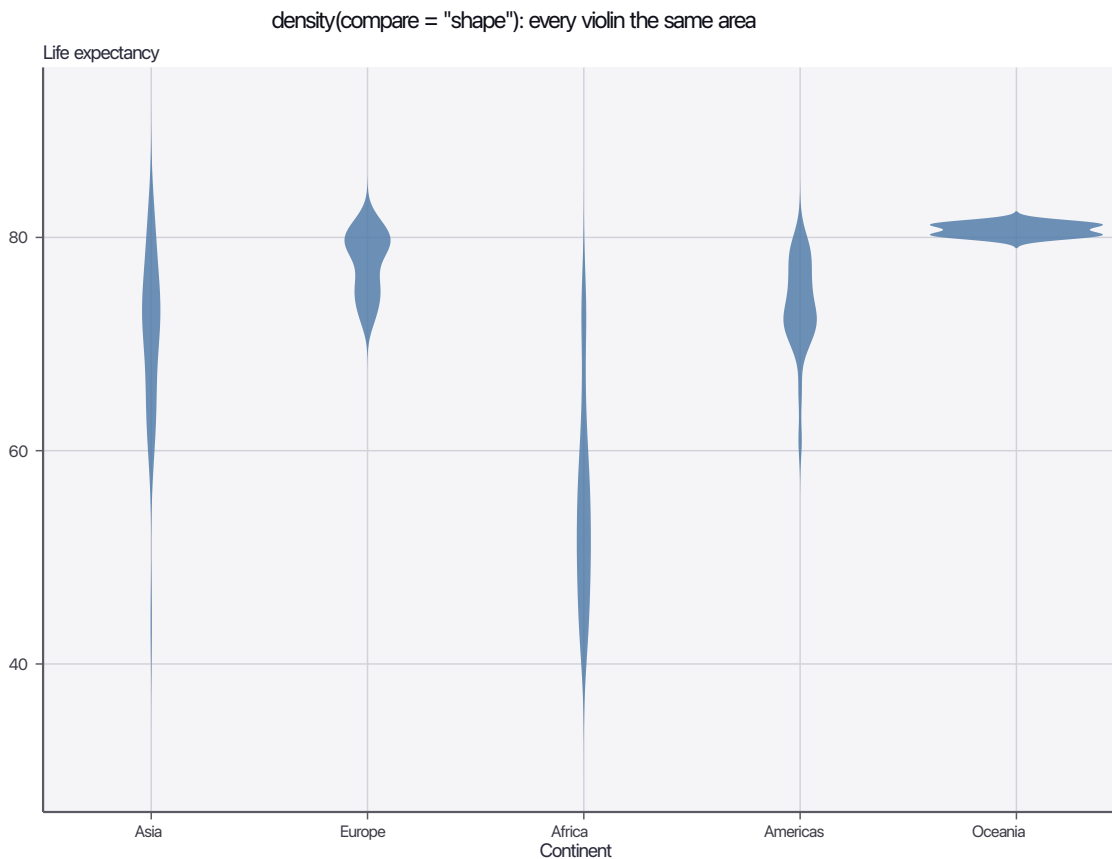
```
(data(gapminder_2007) + ribbon * density(compare = "shape") +  
  ↪ x(col.continent) + y(col.life) +  
  y_label("Life expectancy") +  
  title('density(compare = "shape"): every violin the same area'))
```

Julia

```
data(gapminder_2007) + ribbon * density(compare = "shape") +  
  x(:continent) + y(:life) + y_label("Life expectancy") +  
  title("density(compare = \"shape\"): every violin the same area")
```

JavaScript

```
plot(data(gapminder_2007), layer(ribbon, density({ compare: "shape" })),  
  x(col.continent), y(col.life), y_label("Life expectancy"),  
  title("density(compare = \"shape\"): every violin the same area"))
```



“Given gapminder 2007: ribbons derived by density compared by shape, x is continent, y is life.”

Two countries now fill as much space as fifty-two. Two nearby values make a tall narrow estimate, so Oceania sets the width and every other violin is drawn thinner. Both plots are honest about something different, which is why the choice is a word you write rather than a default nobody stated.

15.11.2 Sideways, and halved

The category can go on either axis, and the violins turn to follow it. The orientation is read off the bindings, exactly as a **bar**’s or a **box**’s is. A category on y is the form with room for long category names:

R

```
data(gapminder_2007) + ribbon * density + x(life) + y(continent) +  
  x_label("Life expectancy") + title("the same violins, lying down")
```

Python

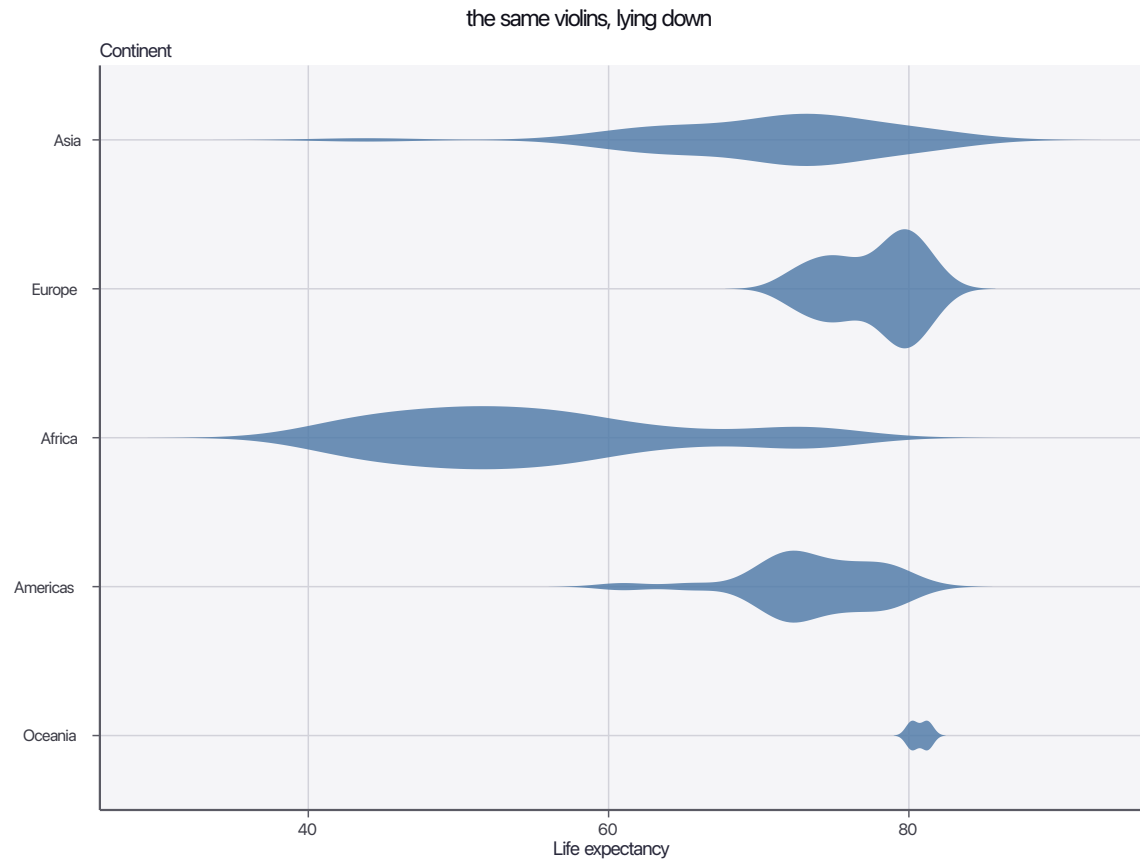
```
(data(gapminder_2007) + ribbon * density + x(col.life) + y(col.continent) +  
  x_label("Life expectancy") + title("the same violins, lying down"))
```

Julia

```
data(gapminder_2007) + ribbon * density + x(:life) + y(:continent) +  
  x_label("Life expectancy") + title("the same violins, lying down")
```

JavaScript

```
plot(data(gapminder_2007), layer(ribbon, density), x(col.life),  
  y(col.continent), x_label("Life expectancy"),  
  title("the same violins, lying down"))
```



An `area` draws the same estimate closed on the slot's center line instead of on its own reflection. One side rather than two is the difference between the two marks everywhere else in the book:

R

```
data(gapminder_2007) + area * density + x(continent) + y(life) +
  y_label("Life expectancy") + title("area * density: the half violin")
```

Python

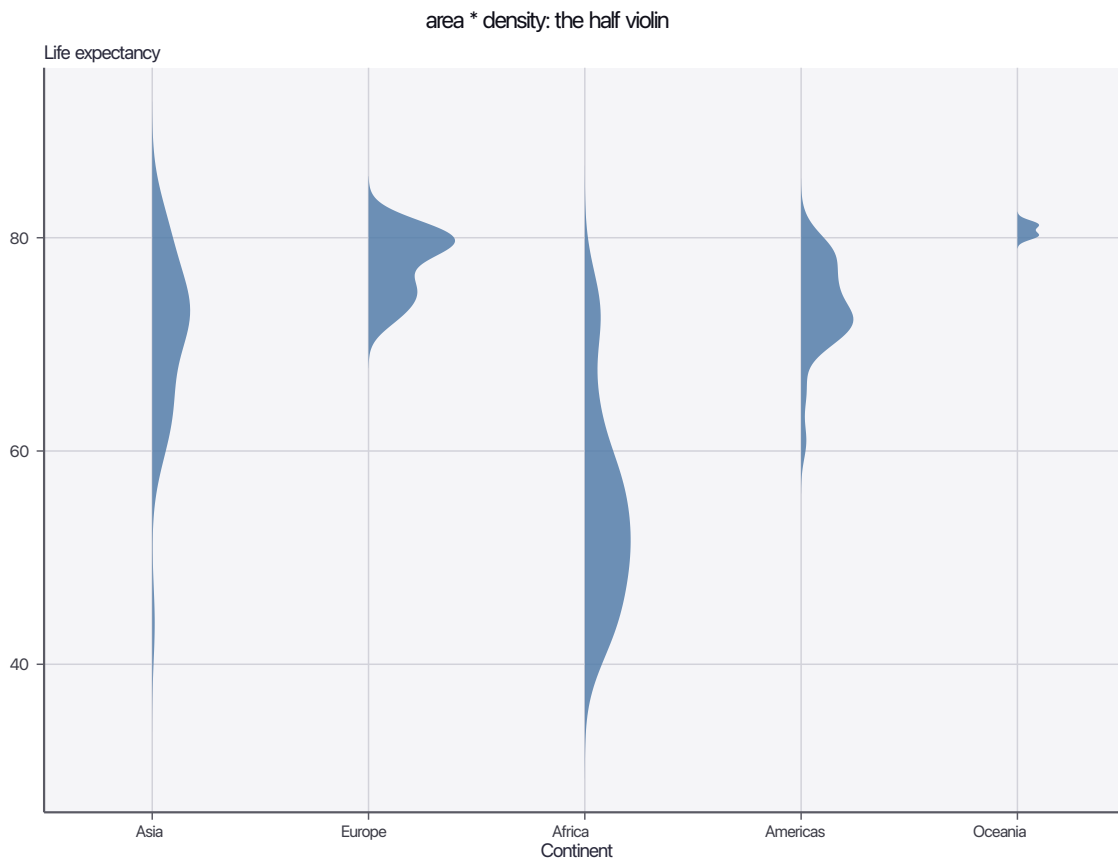
```
(data(gapminder_2007) + area * density + x(col.continent) + y(col.life) +
  y_label("Life expectancy") + title("area * density: the half violin"))
```

Julia

```
data(gapminder_2007) + area * density + x(:continent) + y(:life) +
  y_label("Life expectancy") + title("area * density: the half violin")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, density), x(col.continent),
  y(col.life), y_label("Life expectancy"),
  title("area * density: the half violin"))
```



“Given gapminder 2007: areas derived by density, x is continent, y is life.”

Two choices, made independently: which mark closes the region, and which axis carries the category. That is four plots, and the fourth one has a name of its own. Halve the violin *and* lay it down, and each category gets a filled curve on its own line above a shared measure axis, which is the **ridgeline plot**:

R

```
data(gapminder_2007) + area * density + x(life) + y(continent) +  
  x_label("Life expectancy") + title("area * density + x(life): the  
  ↪  ridgeline")
```

Python

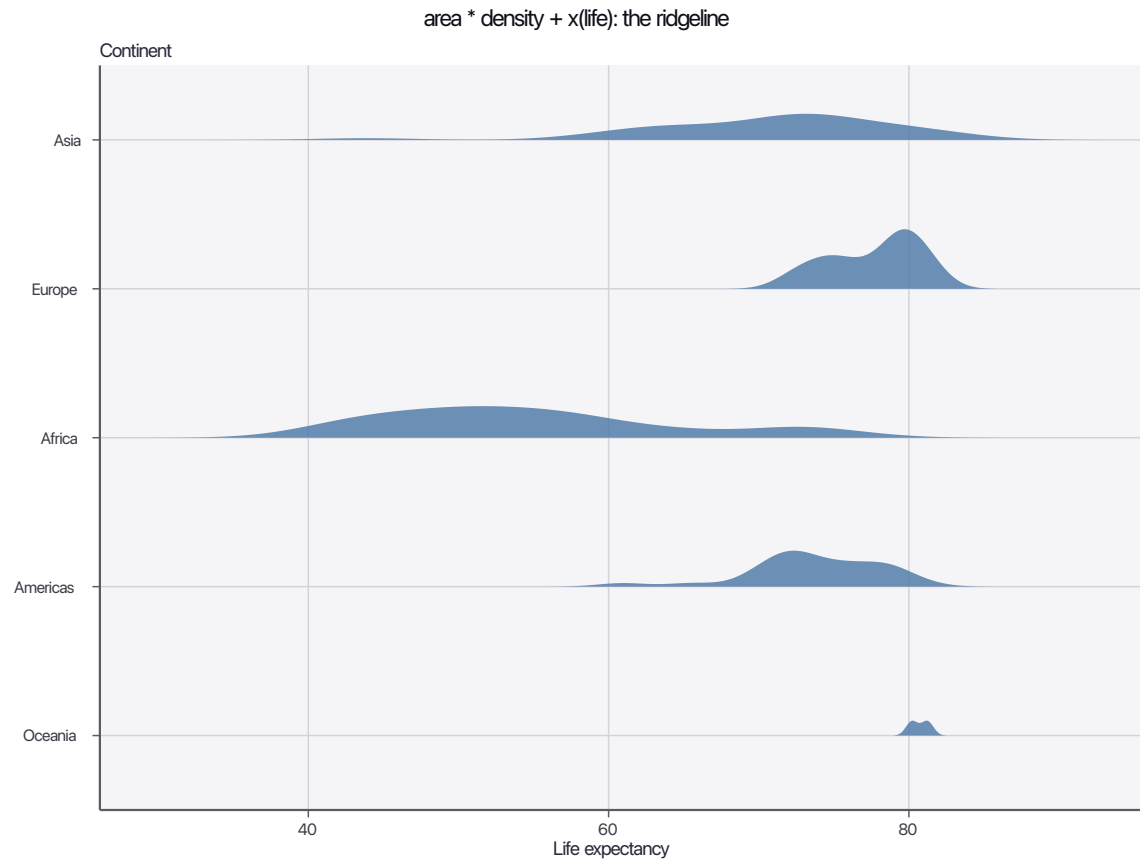
```
(data(gapminder_2007) + area * density + x(col.life) + y(col.continent) +  
  x_label("Life expectancy") + title("area * density + x(life): the  
  ↪  ridgeline"))
```

Julia

```
data(gapminder_2007) + area * density + x(:life) + y(:continent) +  
  x_label("Life expectancy") +  
  title("area * density + x(life): the ridgeline")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, density), x(col.life),  
  y(col.continent), x_label("Life expectancy"),  
  title("area * density + x(life): the ridgeline"))
```



Nothing was added for it either. It is the half violin's sentence with x and y exchanged. It arrives because the two choices are genuinely independent: nothing in the grammar ties *where a region closes* to *which axis holds the category*. All four combinations are sentences, and the named chart is simply one of them.

Read the four plots together and you can see what [orthogonality](#) buys. A word for the violin and a word for the ridgeline would be two names for one idea, and the other two combinations would have no word at all.

15.11.3 Letting the ridges overlap

Each shape reaches four tenths of a slot by default, so two violins face to face fill four fifths of the space between their categories, and the remaining fifth stays empty. That is a [bar](#)'s rule, and for a bar's reason: the gap is what says the categories are separate rather than a divided continuum.

The ridgeline as it is usually drawn (the form some call a joyplot) does not stay inside its slot. Its ridges are tall enough to reach into the row above, and that overlap is part of the design. `density(reach = )` says how far, **in slots**:

R

```
data(gapminder_2007) + area * density(reach = 2.5) + x(life) + y(continent) +  
  x_label("Life expectancy") + title("reach = 2.5: each ridge two and a half  
  ↪ slots tall")
```

Python

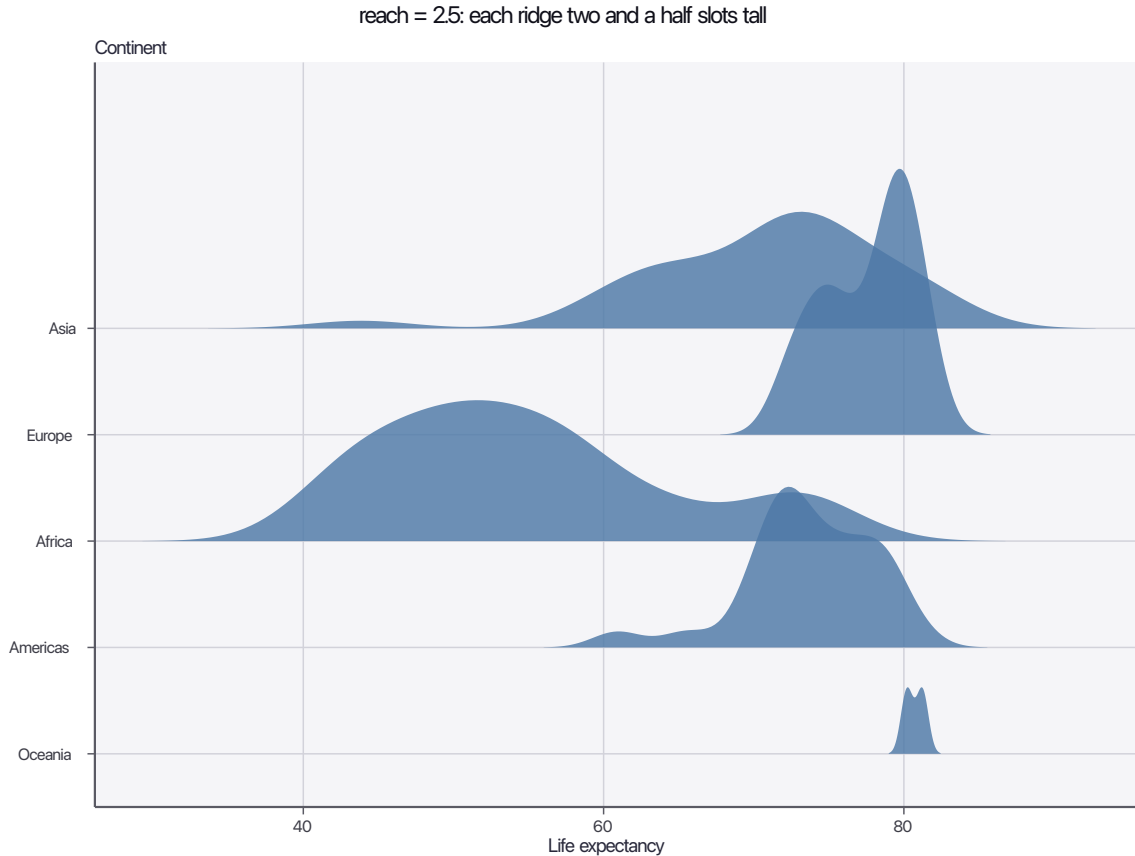
```
(data(gapminder_2007) + area * density(reach = 2.5) + x(col.life) +  
  ↪ y(col.continent) +  
  x_label("Life expectancy") + title("reach = 2.5: each ridge two and a half  
  ↪ slots tall"))
```

Julia

```
data(gapminder_2007) + area * density(reach = 2.5) + x(:life) +  
  y(:continent) + x_label("Life expectancy") +  
  title("reach = 2.5: each ridge two and a half slots tall")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, density({ reach: 2.5 })),  
  x(col.life), y(col.continent), x_label("Life expectancy"),  
  title("reach = 2.5: each ridge two and a half slots tall"))
```



“Given gapminder 2007: areas derived by density reaching 2.5 slots, x is life, y is continent.”

The number is measured from the line a category sits on to the shape’s furthest point. That is why the number means the same thing to both marks. A **ribbon** reaches that far on each side, an **area** on one side only. The half violin therefore stays exactly half of the violin, at any reach. Past 0.5 the shapes leave their own slots. The grammar does not guard against that. It is the plot being asked for, and the later category simply draws over the earlier one.

The category axis grows to hold the ridges. Half a slot of margin is right for a **bar** or a **box**, which stand *in* their slots. A ridge 2.5 slots tall does not stand in its slot, so the axis adds the room it needs. Without that room the panel edge would cut the top ridge off, and you would see the cut.

The edge is a layer here, as it is for every filled band in this chapter. A **ribbon** takes no border setting, and neither does the violin. A **line** given the same transform traces the outline of the fill. That is the *filled, or two dashed edges* rule again, read against a slot:

R

```
data(gapminder_2007) + x(life) + y(continent) +
  area * density(reach = 2.5) + style(color = "steelblue", opacity = 0.9) +
  line * density(reach = 2.5) + style(color = "black", size = 1) +
  x_label("Life expectancy") + title("The same ridges, with their edges
↪ drawn")
```

Python

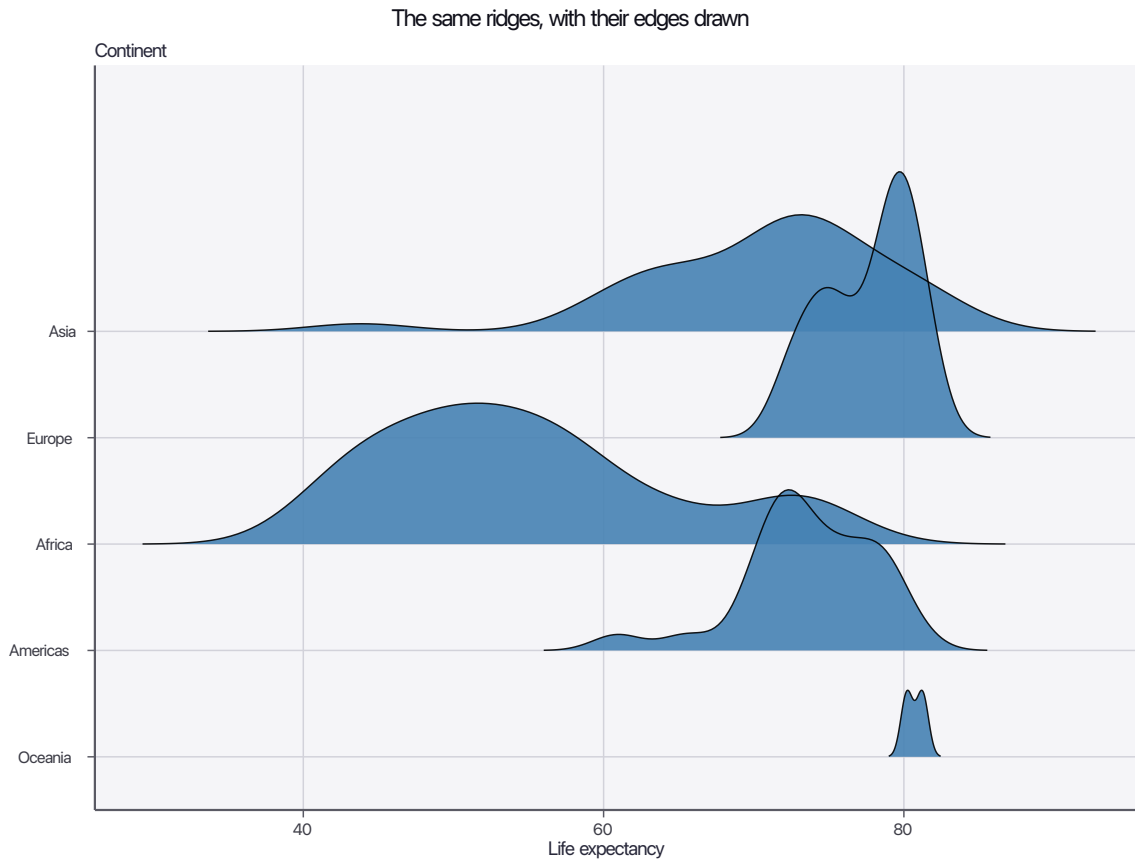
```
(data(gapminder_2007) + x(col.life) + y(col.continent) +
  area * density(reach = 2.5) + style(color = "steelblue", opacity = 0.9) +
  line * density(reach = 2.5) + style(color = "black", size = 1) +
  x_label("Life expectancy") + title("The same ridges, with their edges
↪ drawn"))
```

Julia

```
data(gapminder_2007) + x(:life) + y(:continent) +
  area * density(reach = 2.5) +
  style(color = "steelblue", opacity = 0.9) +
  line * density(reach = 2.5) + style(color = "black", size = 1) +
  x_label("Life expectancy") +
  title("The same ridges, with their edges drawn")
```

JavaScript

```
plot(data(gapminder_2007), x(col.life), y(col.continent),
  layer(area, density({ reach: 2.5 })),
  style({ color: "steelblue", opacity: 0.9 }),
  layer(line, density({ reach: 2.5 })),
  style({ color: "black", size: 1 }), x_label("Life expectancy"),
  title("The same ridges, with their edges drawn"))
```



Both layers read the same `density(reach = 2.5)`, so the stroke cannot drift off the fill it belongs to, the reason an edge is a layer rather than a setting. Color each ridge by its own category and the plot is complete:

R

```
data(gapminder_2007) + x(life) + y(continent) +
  area * density(reach = 2.2) + color(continent) +
  line * density(reach = 2.2) + style(color = "black", size = 0.9) +
  x_label("Life expectancy") + title("One hue per ridge")
```

Python

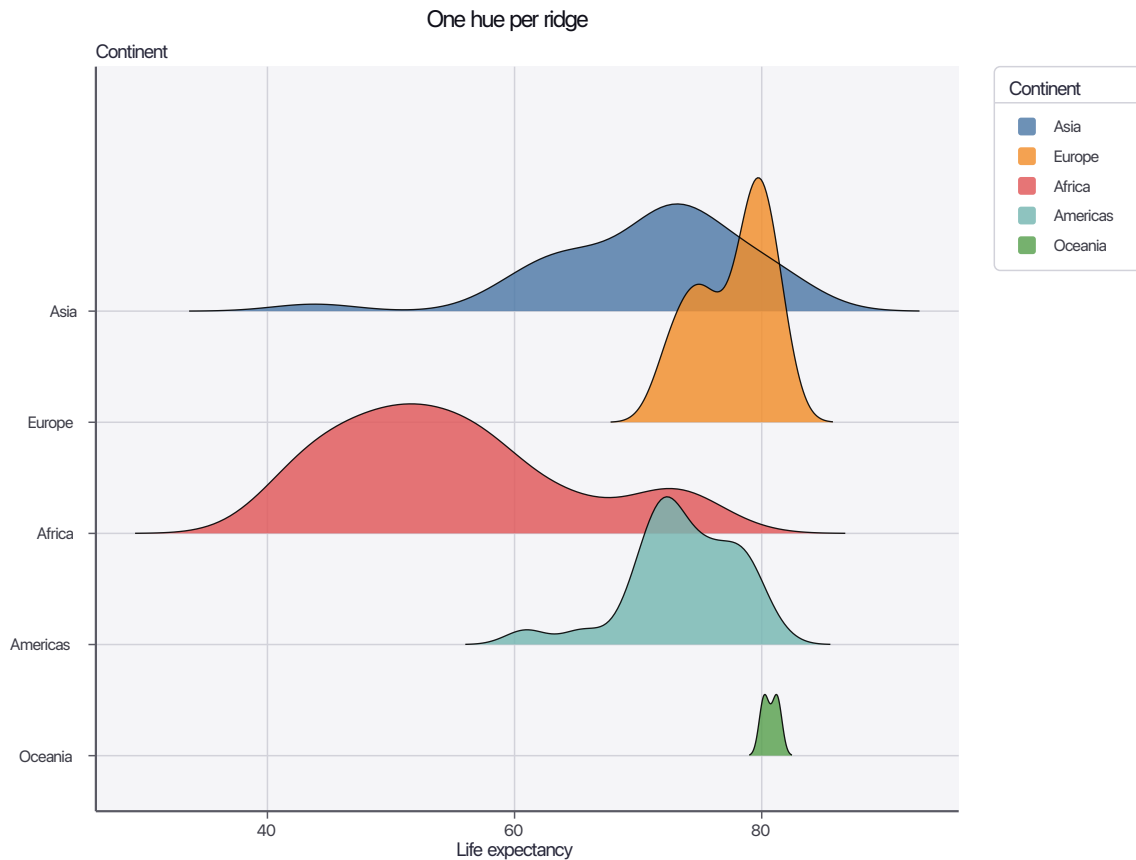
```
(data(gapminder_2007) + x(col.life) + y(col.continent) +
  area * density(reach = 2.2) + color(col.continent) +
  line * density(reach = 2.2) + style(color = "black", size = 0.9) +
  x_label("Life expectancy") + title("One hue per ridge"))
```

Julia

```
data(gapminder_2007) + x(:life) + y(:continent) +  
  area * density(reach = 2.2) + color(:continent) +  
  line * density(reach = 2.2) + style(color = "black", size = 0.9) +  
  x_label("Life expectancy") + title("One hue per ridge")
```

JavaScript

```
plot(data(gapminder_2007), x(col.life), y(col.continent),  
  layer(area, density({ reach: 2.2 })), color(col.continent),  
  layer(line, density({ reach: 2.2 })),  
  style({ color: "black", size: 0.9 }, x_label("Life expectancy"),  
  title("One hue per ridge"))
```



The `color(continent)` is written *after* the area, so it belongs to that layer alone: a plot-scoped one would color the line too, and a layer cannot both map color and set it. [Scope is position](#), and this is the sentence where it matters.

`step * density` traces the same estimate as a staircase, for the same reason `step * bin` is the outline histogram: [No Exceptions](#) means a mark draws whatever a transform hands it, in the mark's own way.

15.12 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(pattern =)</code>	<code>solid</code> , <code>hatch</code> , <code>crosshatch</code> , <code>grid</code> , <code>dots</code>

And these vary per row if you map them to a column instead: `color()` (categories), `pattern()` (categories), `group()` (categories), `play()` (either).

A ribbon takes `area`'s list, and for `area`'s reasons. There is no `size`, because the band's extent comes from the two boundaries its transform supplies. There is no border, because the edge is a line traced along the same pair.

R

```
data(score_band) + x(score) +
  ribbon * bounds(lower, upper) +
    style(color = "slateblue", pattern = "grid", opacity = 0.45) +
  line * bounds(lower, upper) + style(color = "slateblue", pattern =
↪ "dashed") +
  y_label("Expected true score") + title("A textured band, and its two
↪ edges")
```

Python

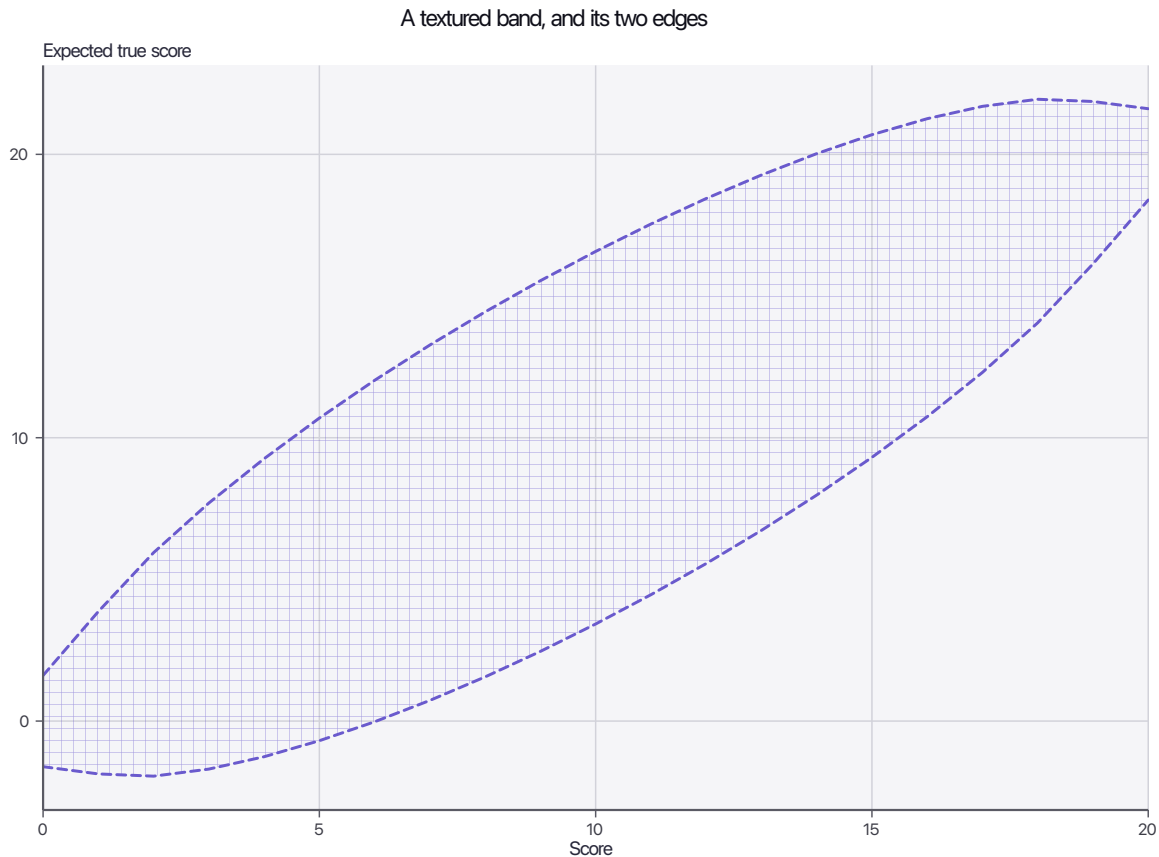
```
(data(score_band) + x(col.score) +
  ribbon * bounds(col.lower, col.upper) +
    style(color = "slateblue", pattern = "grid", opacity = 0.45) +
  line * bounds(col.lower, col.upper) + style(color = "slateblue", pattern =
↪ "dashed") +
  y_label("Expected true score") + title("A textured band, and its two
↪ edges"))
```


Julia

```
data(score_band) + x(:score) + ribbon * bounds(:lower, :upper) +  
  style(color = "slateblue", pattern = "grid", opacity = 0.45) +  
  line * bounds(:lower, :upper) +  
  style(color = "slateblue", pattern = "dashed") +  
  y_label("Expected true score") +  
  title("A textured band, and its two edges")
```

JavaScript

```
plot(data(score_band), x(col.score),  
  layer(ribbon, bounds(col.lower, col.upper)),  
  style({ color: "slateblue", pattern: "grid", opacity: 0.45 } ),  
  layer(line, bounds(col.lower, col.upper)),  
  style({ color: "slateblue", pattern: "dashed" } ),  
  y_label("Expected true score"),  
  title("A textured band, and its two edges"))
```



“Given the score band: x is score, a ribbon from lower to upper, with pattern grid, and also lines from lower to upper, with pattern dashed.”

Both marks read the same `bounds(lower, upper)` there, exactly as in the ridge section, so the edges stay on the band. Note the two `pattern` values in one sentence: `"grid"` is a fill texture on the ribbon and `"dashed"` a stroke dash on the line, the same setting realized once per geometry.

15.13 What it refuses

The measure stays numeric. A band from `range` spans a low value to a high one, and both are quantities, so a category on `y` is refused. The violins above put a category on `y`, but there `density` makes it a slot rather than the measure:

```
render_svg(data(gapminder_2007) + ribbon * range + x(life) + y(continent))
```

Error:

```
! gog: `y(continent)` maps a categorical (text) column, but `y` on `ribbon`  
  ↳ needs a continuous (numeric) column. On these marks `x` is the domain and  
  ↳ `y` the measure, and a category is not a quantity to measure: a mean of  
  ↳ category names is not a number, and a region has no categorical baseline  
  ↳ to close on. Put the category on `x` instead - `line * mean +  
  ↳ x(<category>) + y(<number>)` is the profile plot, and `area * mean` fills  
  ↳ it. Unlike `bar`/`box`/`interval`, these marks do not read their  
  ↳ orientation off the bindings, because their two axes do not have the same  
  ↳ role.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

ribbon completes the low/high family: [interval](#) draws the pair as a whisker at each x, [line](#) and [step](#) trace its two boundaries, and [ribbon](#) fills between them. One [range](#), four marks: [No Exceptions](#) made visible. See [Transforms](#) for the [range](#), [confidence](#) and [deviation](#) statistics all four share.

16 Text

Which point is which? `text` draws a string at each (x, y) position. It is `point`'s sibling: both place one glyph per row. A point's glyph is a dot. A text mark's glyph is a **string**, and that string comes from a column, through the `label` channel.

16.1 The mark and its content

A point tells you *where* a row sits; it cannot tell you *which* row. Five medal counts drawn as dots are five anonymous points:

R

```
data(medals) + point + x(gold) + y(silver) +  
  x_label("Gold") + y_label("Silver") +  
  title("Five countries, and no names")
```

Python

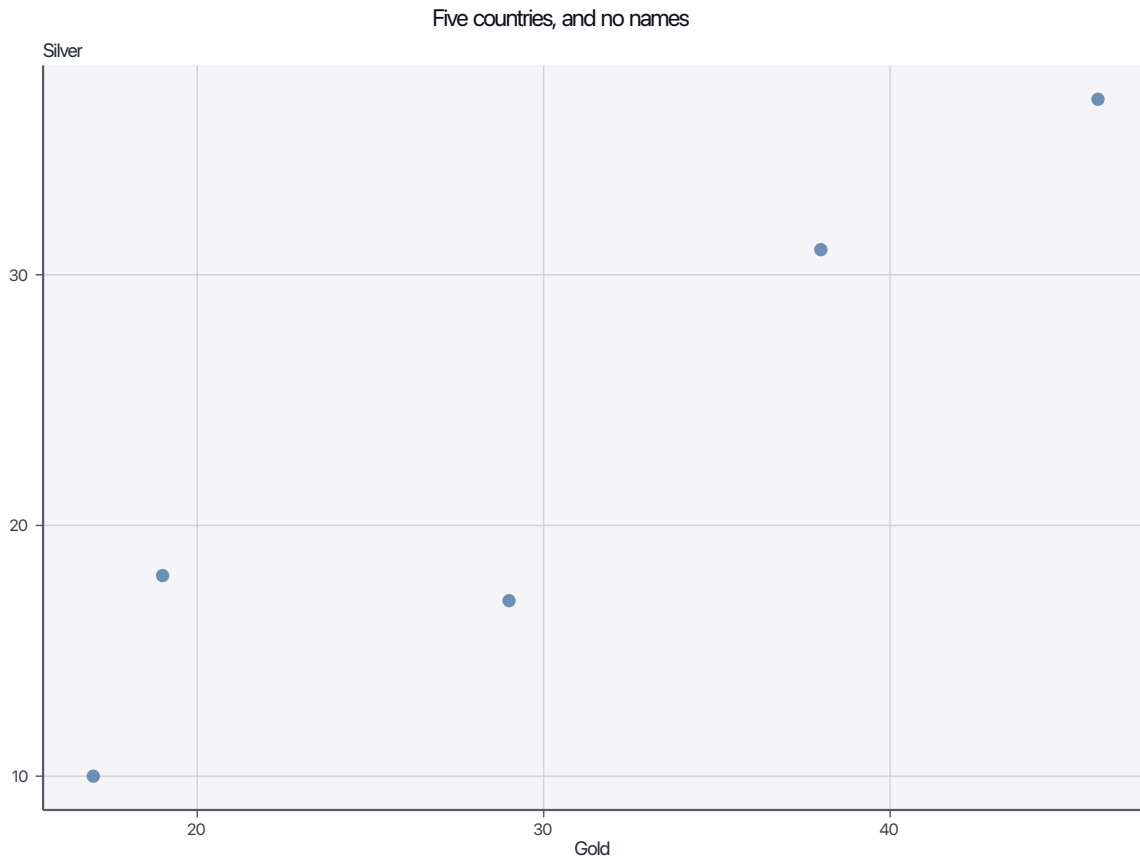
```
(data(medals) + point + x(col.gold) + y(col.silver) +  
  x_label("Gold") + y_label("Silver") +  
  title("Five countries, and no names"))
```

Julia

```
data(medals) + point + x(:gold) + y(:silver) + x_label("Gold") +  
  y_label("Silver") + title("Five countries, and no names")
```

JavaScript

```
plot(data(medals), point, x(col.gold), y(col.silver), x_label("Gold"),  
  y_label("Silver"), title("Five countries, and no names"))
```



“Given the medals: points, x is gold, y is silver.”

Bind `label` and draw the names themselves at those positions, and each mark now shows which row it is:

R

```
data(medals) + text + x(gold) + y(silver) + label(country) +
  x_label("Gold") + y_label("Silver") +
  title("The same five, named")
```

Python

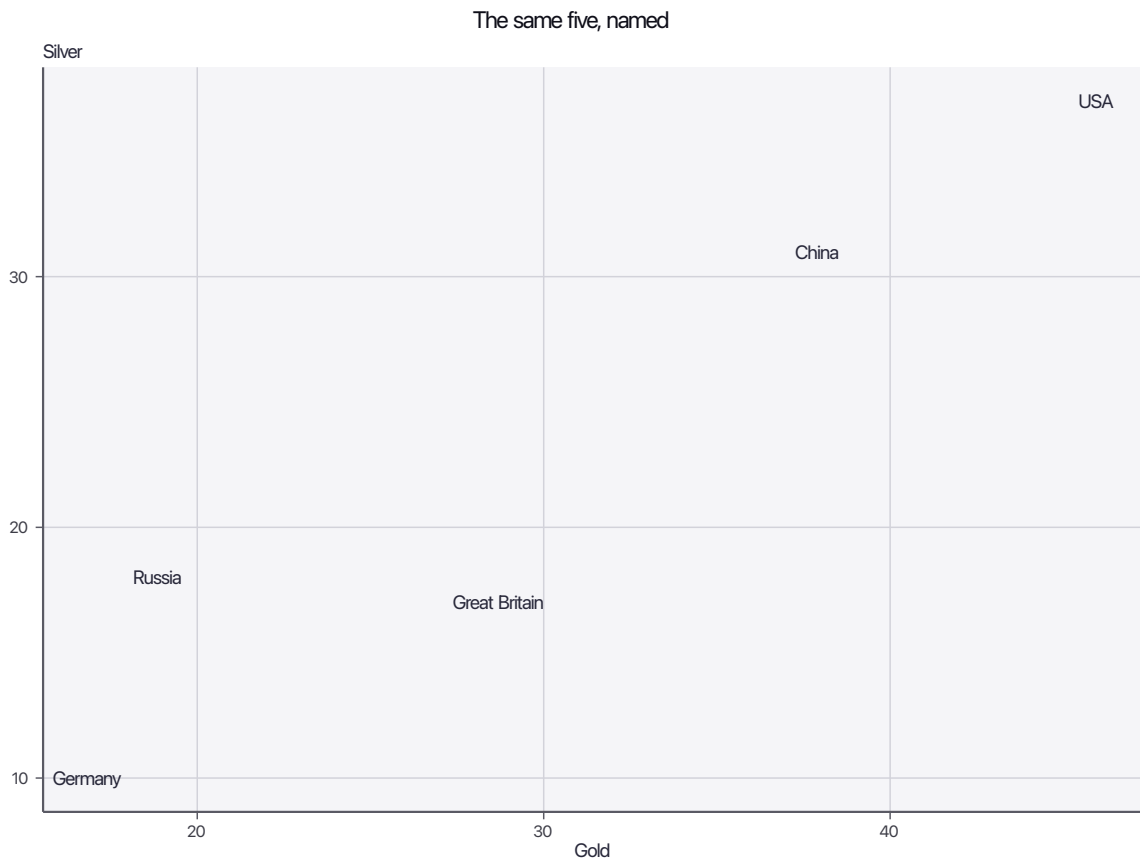
```
(data(medals) + text + x(col.gold) + y(col.silver) + label(col.country) +
  x_label("Gold") + y_label("Silver") +
  title("The same five, named"))
```

Julia

```
data(medals) + text + x(:gold) + y(:silver) + label(:country) +  
  x_label("Gold") + y_label("Silver") + title("The same five, named")
```

JavaScript

```
plot(data(medals), text, x(col.gold), y(col.silver), label(col.country),  
  x_label("Gold"), y_label("Silver"), title("The same five, named"))
```



“Given the medals: text, x is gold, y is silver, label by country.”

That is the whole difference between the two marks: `point` draws a glyph *at* the row, `text` draws the row’s own value *as* the glyph.

16.2 Both at once: the labeled scatter

They compose. Layer a `point` and a `text` over one `(x, y)`. The dot gives the exact position and the string gives the name. That is a labeled scatter, built from two marks rather than a third mark of its own. Each `style()` applies to the mark written before it, so the gray below belongs to the dots alone. A label would otherwise sit on top of its dot, so `style(nudge = "up")` moves it off:

R

```
data(medals) + point + style(size = 5, color = "#9e9e9e") +  
  text + x(gold) + y(silver) + label(country) + style(nudge = "up") +  
  x_label("Gold") + y_label("Silver") +  
  title("Located by the dot, named just above")
```

Python

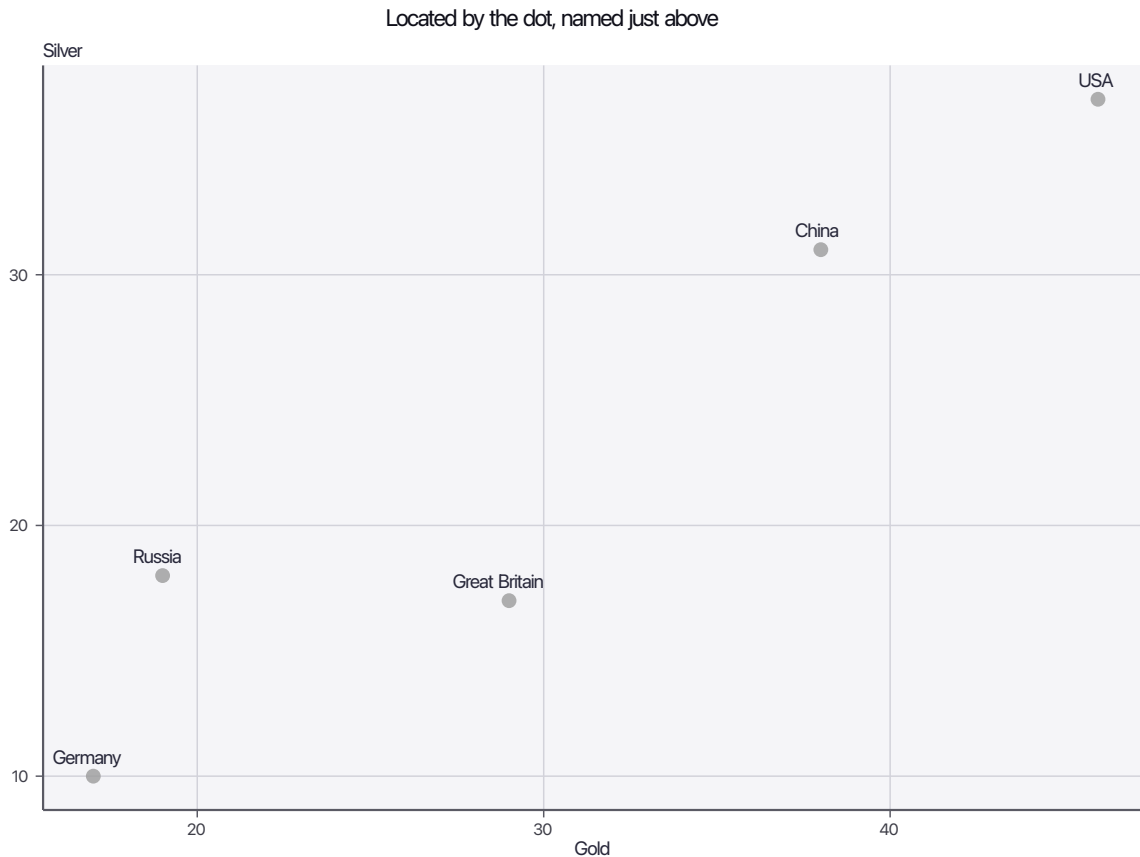
```
(data(medals) + point + style(size = 5, color = "#9e9e9e") +  
  text + x(col.gold) + y(col.silver) + label(col.country) + style(nudge =  
  ↪ "up") +  
  x_label("Gold") + y_label("Silver") +  
  title("Located by the dot, named just above"))
```

Julia

```
data(medals) + point + style(size = 5, color = "#9e9e9e") + text +  
  x(:gold) + y(:silver) + label(:country) + style(nudge = "up") +  
  x_label("Gold") + y_label("Silver") +  
  title("Located by the dot, named just above")
```

JavaScript

```
plot(data(medals), point, style({ size: 5, color: "#9e9e9e" }), text,  
  x(col.gold), y(col.silver), label(col.country), style({ nudge: "up" }),  
  x_label("Gold"), y_label("Silver"),  
  title("Located by the dot, named just above"))
```



“Given the medals: points and also text, x is gold, y is silver, label by country, with nudge up.”

There is no “labeled point” mark to learn: a labeled scatter is just `point + text`, layered with the `+` [operator](#).

Five names fit here. Fifty would not, and choosing which few to keep is the hardest part of a labeled scatter. In the web edition you can try them first. Clicking a point prints its values on the plot, which [Selection](#) calls a stamp. That shows how a few labels look before you write any. A stamp disappears when the page reloads; a `text` label is part of the specification, so it is drawn every time.

16.2.1 Nudging the label off its point

`style(nudge = )` shifts a text label off its (x, y). It takes one direction: "up", "down", "left", or "right":

R


```
data(medals) + point + style(size = 5, color = "#9e9e9e") +
  text + x(gold) + y(silver) + label(country) + style(nudge = "right") +
  x_label("Gold") + y_label("Silver") +
  title("The same, nudged right")
```

Python

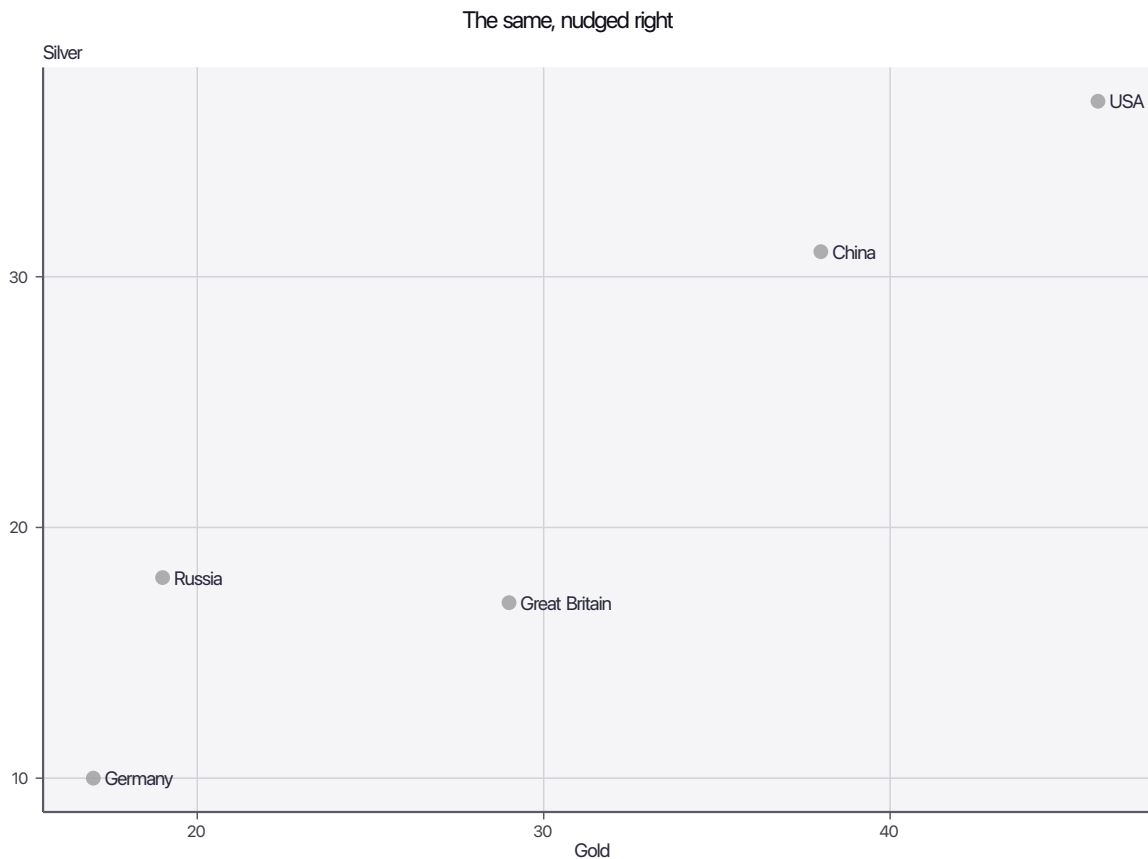
```
(data(medals) + point + style(size = 5, color = "#9e9e9e") +
  text + x(col.gold) + y(col.silver) + label(col.country) + style(nudge =
  ↪ "right") +
  x_label("Gold") + y_label("Silver") +
  title("The same, nudged right"))
```

Julia

```
data(medals) + point + style(size = 5, color = "#9e9e9e") + text +
  x(:gold) + y(:silver) + label(:country) + style(nudge = "right") +
  x_label("Gold") + y_label("Silver") + title("The same, nudged right")
```

JavaScript

```
plot(data(medals), point, style({ size: 5, color: "#9e9e9e" }), text,
  x(col.gold), y(col.silver), label(col.country),
  style({ nudge: "right" }), x_label("Gold"), y_label("Silver"),
  title("The same, nudged right"))
```



The distance comes from the font size, just enough to sit outside a default dot, so there is nothing to specify; **nudge** takes only the direction. It is a *setting*, not a collision modifier, and the two differ in where the offset comes from. **jitter** and **dodge** compute their offsets from the data, so every mark moves by its own amount. A nudge is one *fixed* shift, the same for every label, derived from nothing in the data. It is **text-only**. Nudging a **point** or **bar** would move the data itself, so the engine refuses it there and says why.

16.2.2 **repel**: moving labels off each other

A nudge shifts every label the same way, so it moves each label off its own dot and does nothing else. When the labels collide with *each other*, one fixed shift moves them all together and the words still overlap. Thirty European countries are enough to show it:

Table 16.1: gm_europe: first 5 of 30 rows

country	continent	year	life	population	gdp
Albania	Europe	2007	76.423	3600523	5937.030
Austria	Europe	2007	79.829	8199783	36126.493
Belgium	Europe	2007	79.441	10392226	33692.605
Bosnia and Herzegovina	Europe	2007	74.852	4552198	7446.299
Bulgaria	Europe	2007	73.005	7322858	10680.793

R

```
data(gm_europe) + point + style(color = "#9e9e9e") +
  text + x(gdp) + y(life) + label(country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Thirty countries, named")
```

Python

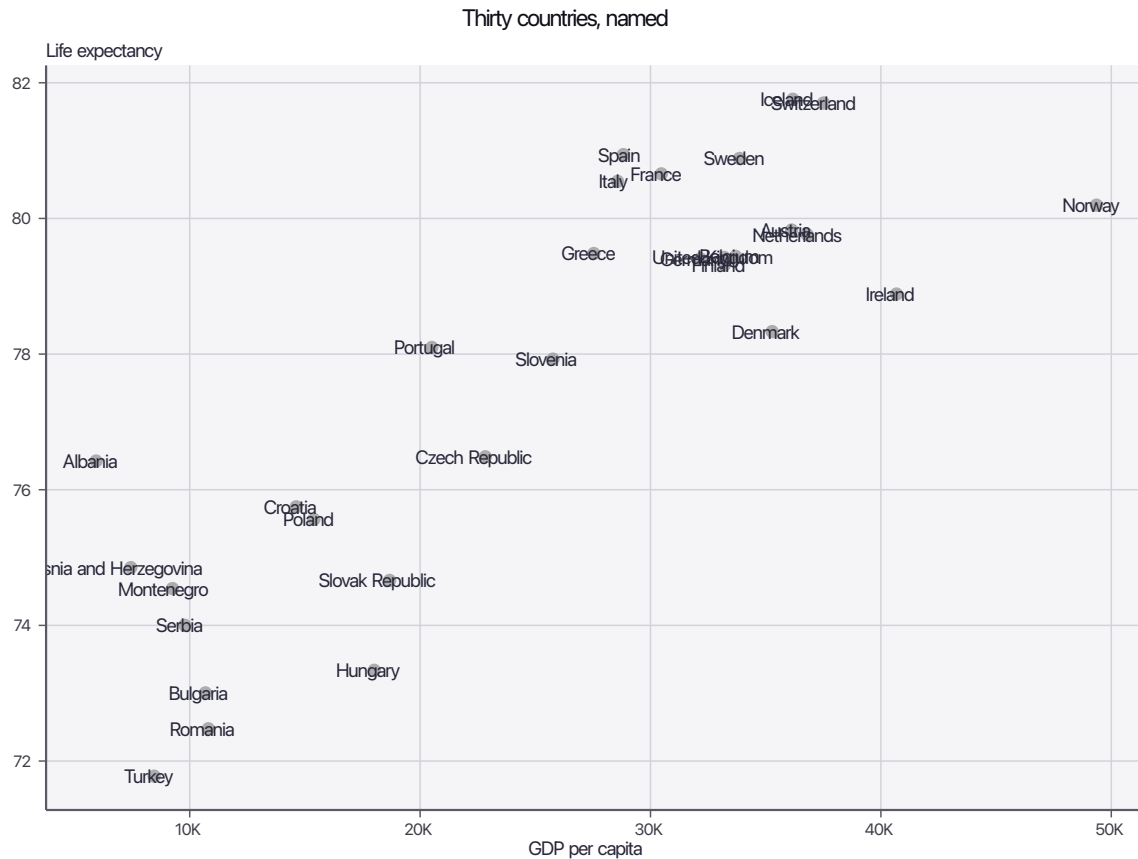
```
(data(gm_europe) + point + style(color = "#9e9e9e") +
  text + x(col.gdp) + y(col.life) + label(col.country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Thirty countries, named"))
```

Julia

```
data(gm_europe) + point + style(color = "#9e9e9e") + text + x(:gdp) +
  y(:life) + label(:country) + x_label("GDP per capita") +
  y_label("Life expectancy") + title("Thirty countries, named")
```

JavaScript

```
plot(data(gm_europe), point, style({ color: "#9e9e9e" }), text,
  x(col.gdp), y(col.life), label(col.country), x_label("GDP per capita"),
  y_label("Life expectancy"), title("Thirty countries, named"))
```



`repel` solves that, and it is a *collision modifier* rather than a setting. It measures where the labels and the dots actually are, then moves each label off the words around it:

R

```
data(gm_europe) + point + style(color = "#9e9e9e") +
  text * repel + x(gdp) + y(life) + label(country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The same thirty, moved apart")
```

Python

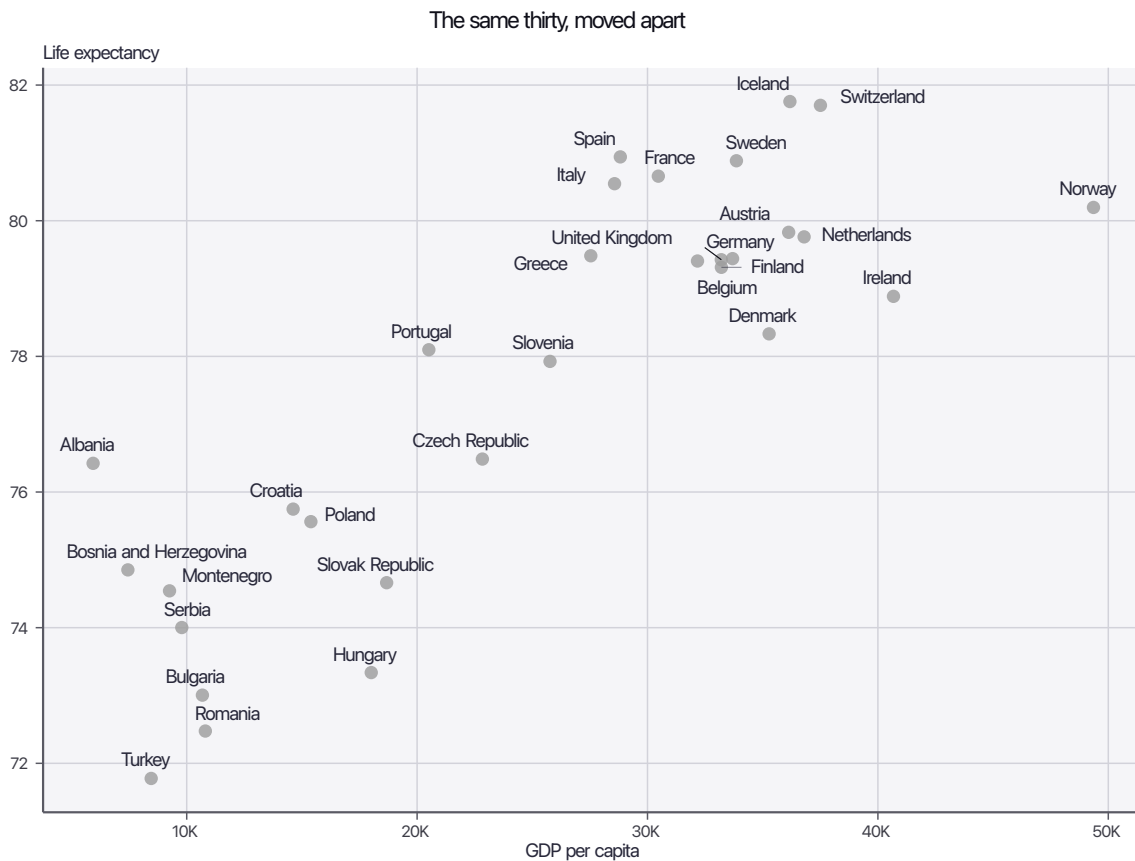
```
(data(gm_europe) + point + style(color = "#9e9e9e") +
  text * repel + x(col.gdp) + y(col.life) + label(col.country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The same thirty, moved apart"))
```

Julia

```
data(gm_europe) + point + style(color = "#9e9e9e") + text * repel +  
  x(:gdp) + y(:life) + label(:country) + x_label("GDP per capita") +  
  y_label("Life expectancy") + title("The same thirty, moved apart")
```

JavaScript

```
plot(data(gm_europe), point, style({ color: "#9e9e9e" }),  
  layer(text, repel), x(col.gdp), y(col.life), label(col.country),  
  x_label("GDP per capita"), y_label("Life expectancy"),  
  title("The same thirty, moved apart"))
```



“Given gapminder Europe: points and also text derived by repel, x is gdp, y is life, label by country.”

The modifier does three things in that picture. Almost every label comes to rest just outside its dot, so the word does not cover the point it names. A label that moved far keeps a thin line back to its point, so you can still see which point it names. And each label rests beside its dot, on whichever side has room. **repel** pulls a label toward its dot and pushes it away from every word and every other dot. A label with no neighbors comes to rest just above its dot.

The placement is **deterministic**. It uses no random-number generator and no system time, so the same specification draws the same picture every time.

When no arrangement fits, **repel** still draws every label and reports how many still overlap. It never drops a label in silence. A plot missing the names it could not fit would look like a plot of fewer countries, so **gog** reports the count instead. A larger plot, a smaller **style(size =)**, or fewer rows separate the labels.

The setting and the collision modifier compose, and each does one thing. **style(nudge =)** says which side a label prefers, and **repel** resolves the overlaps that remain:

R

```
data(gm_europe) + point + style(color = "#9e9e9e") +
  text * repel + x(gdp) + y(life) + label(country) + style(nudge = "right") +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Nudged right first, then moved apart")
```

Python

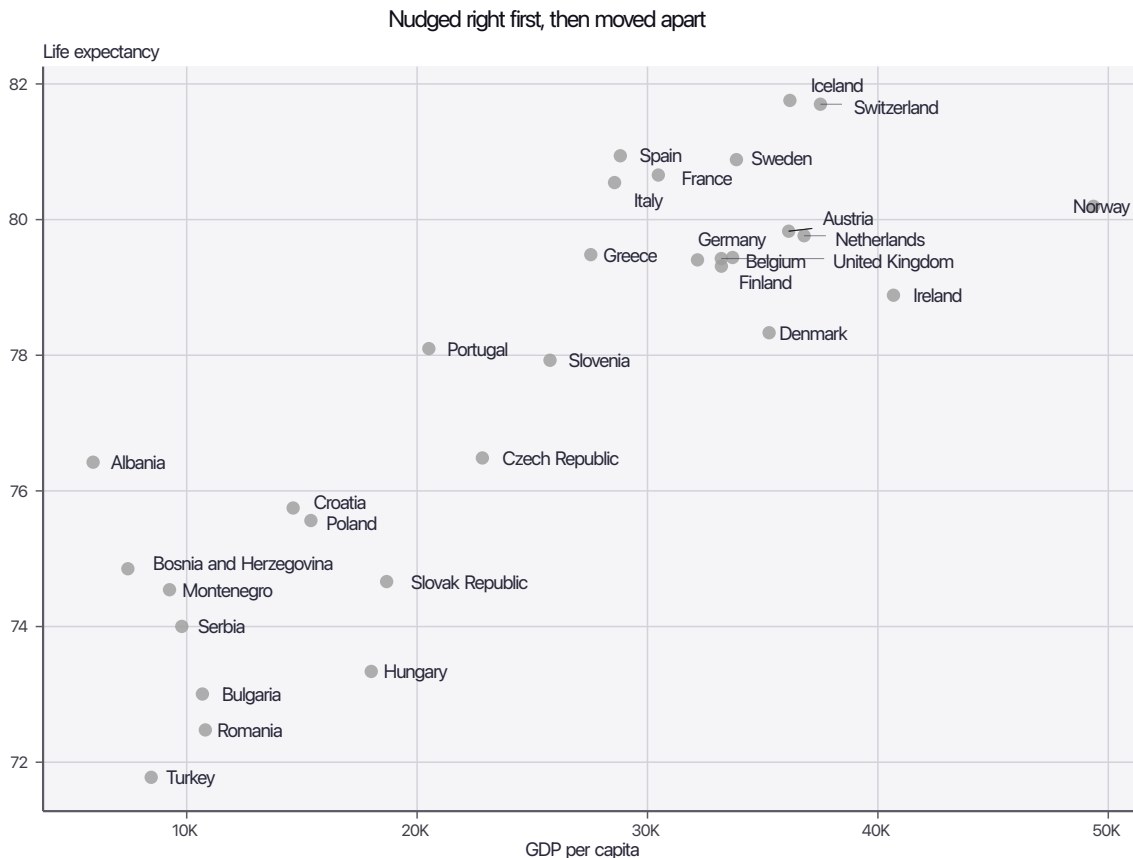
```
(data(gm_europe) + point + style(color = "#9e9e9e") +
  text * repel + x(col.gdp) + y(col.life) + label(col.country) + style(nudge
↵ = "right") +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Nudged right first, then moved apart"))
```

Julia

```
data(gm_europe) + point + style(color = "#9e9e9e") + text * repel +
  x(:gdp) + y(:life) + label(:country) + style(nudge = "right") +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("Nudged right first, then moved apart")
```

JavaScript

```
plot(data(gm_europe), point, style({ color: "#9e9e9e" }),
     layer(text, repel), x(col.gdp), y(col.life), label(col.country),
     style({ nudge: "right" }), x_label("GDP per capita"),
     y_label("Life expectancy"),
     title("Nudged right first, then moved apart"))
```



16.3 Numeric labels: the value on each mark

A `label` need not be a name. Bind a numeric column and each row's value is drawn. A whole number is printed without a decimal point. Layer `text` over `bar` to print each bar's height at its top:

R

```
data(medals) + bar + style(color = "steelblue") +
  text + x(country) + y(gold) + label(gold) +
  y_label("Gold medals") +
  title("Each bar labeled with its height")
```

Python

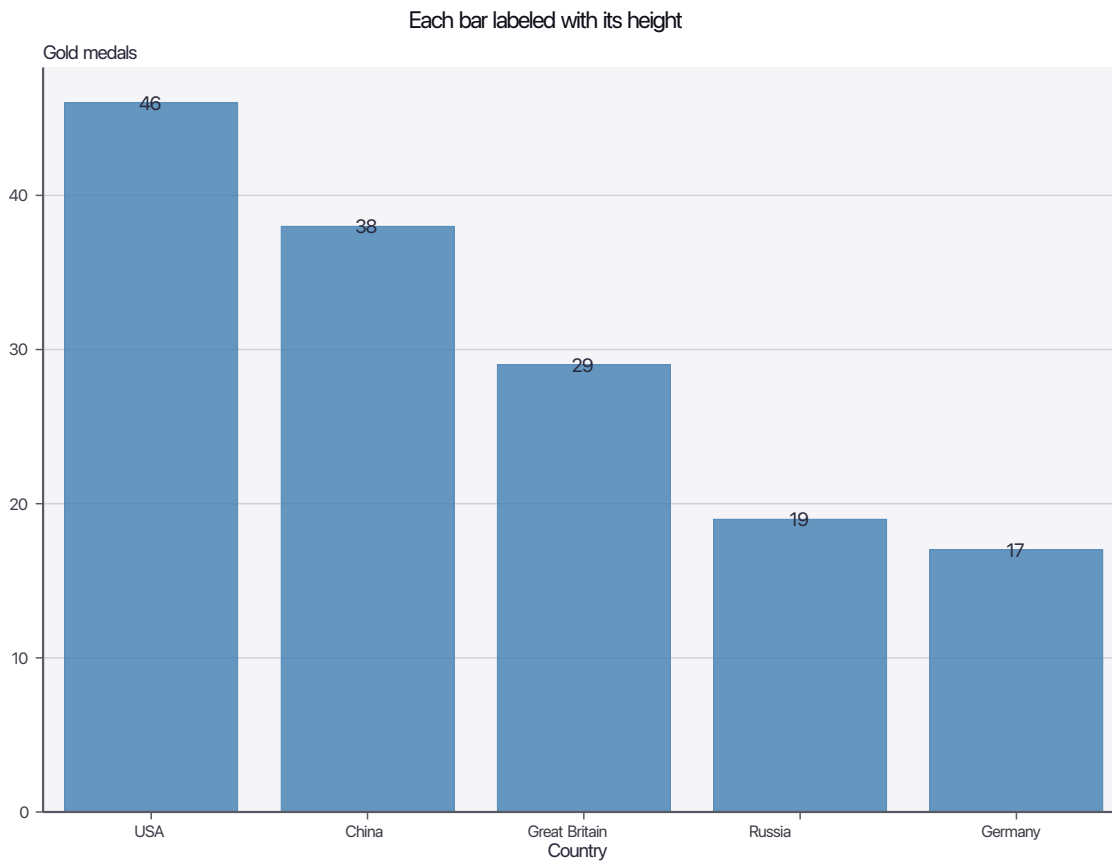
```
(data(medals) + bar + style(color = "steelblue") +
  text + x(col.country) + y(col.gold) + label(col.gold) +
  y_label("Gold medals") +
  title("Each bar labeled with its height"))
```

Julia

```
data(medals) + bar + style(color = "steelblue") + text + x(:country) +
  y(:gold) + label(:gold) + y_label("Gold medals") +
  title("Each bar labeled with its height")
```

JavaScript

```
plot(data(medals), bar, style({ color: "steelblue" }), text,
  x(col.country), y(col.gold), label(col.gold), y_label("Gold medals"),
  title("Each bar labeled with its height"))
```

“Given the medals: bars and also text, x is country, y is gold, label by gold.”

16.4 Color

`color` splits the labels by group and earns a legend, the same discrete split `bar` makes. Here each country’s name takes its own hue:

R

```
data(medals) + text + x(gold) + y(silver) + label(country) + color(country) +  
  x_label("Gold") + y_label("Silver") +  
  title("One hue per country")
```

Python

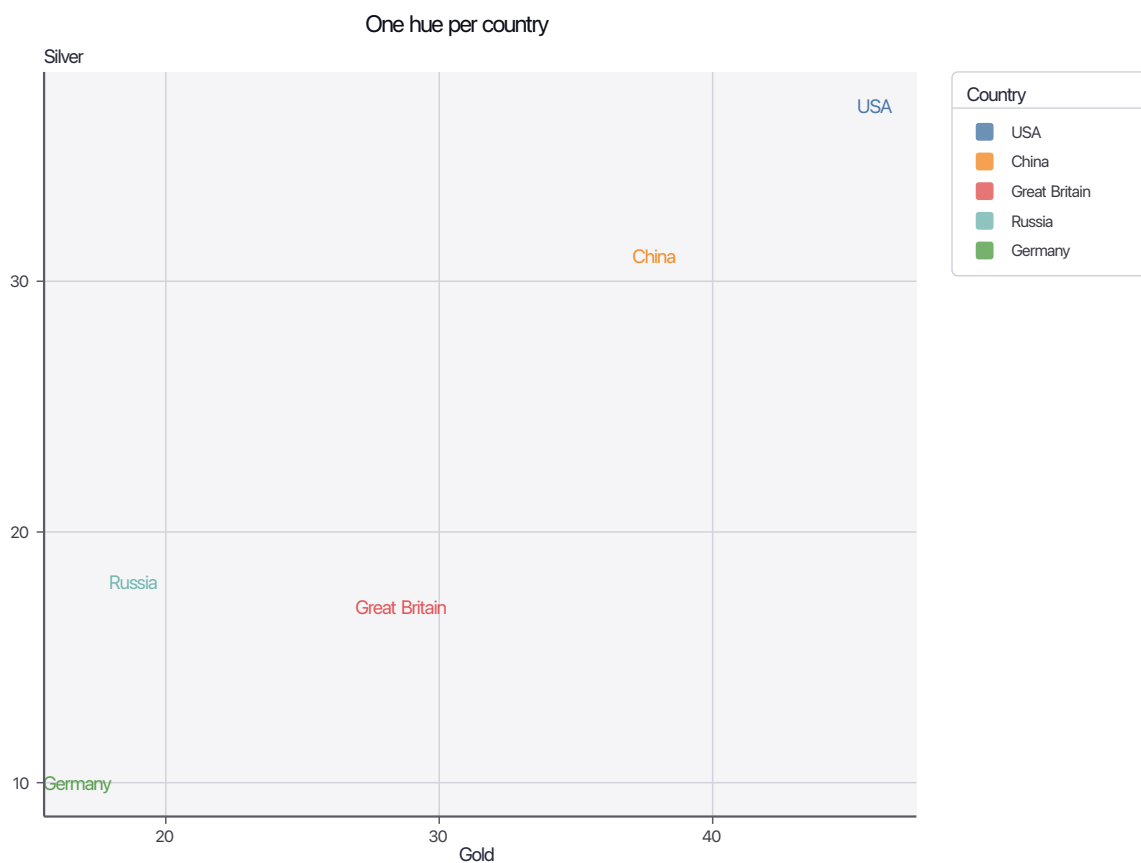
```
(data(medals) + text + x(col.gold) + y(col.silver) + label(col.country) +
  ↪ color(col.country) +
  x_label("Gold") + y_label("Silver") +
  title("One hue per country"))
```

Julia

```
data(medals) + text + x(:gold) + y(:silver) + label(:country) +
  color(:country) + x_label("Gold") + y_label("Silver") +
  title("One hue per country")
```

JavaScript

```
plot(data(medals), text, x(col.gold), y(col.silver), label(col.country),
  color(col.country), x_label("Gold"), y_label("Silver"),
  title("One hue per country"))
```



“Given the medals: text, x is gold, y is silver, label by country, color by country.”

16.5 Styling: one color, one size

`style(color = )` sets the color of the words. `style(size = )` sets the font size in pixels. Both cover the whole layer:

R

```
data(medals) + text + x(gold) + y(silver) + label(country) +  
  style(color = "seagreen", size = 17) +  
  x_label("Gold") + y_label("Silver") +  
  title("Bigger, greener labels")
```

Python

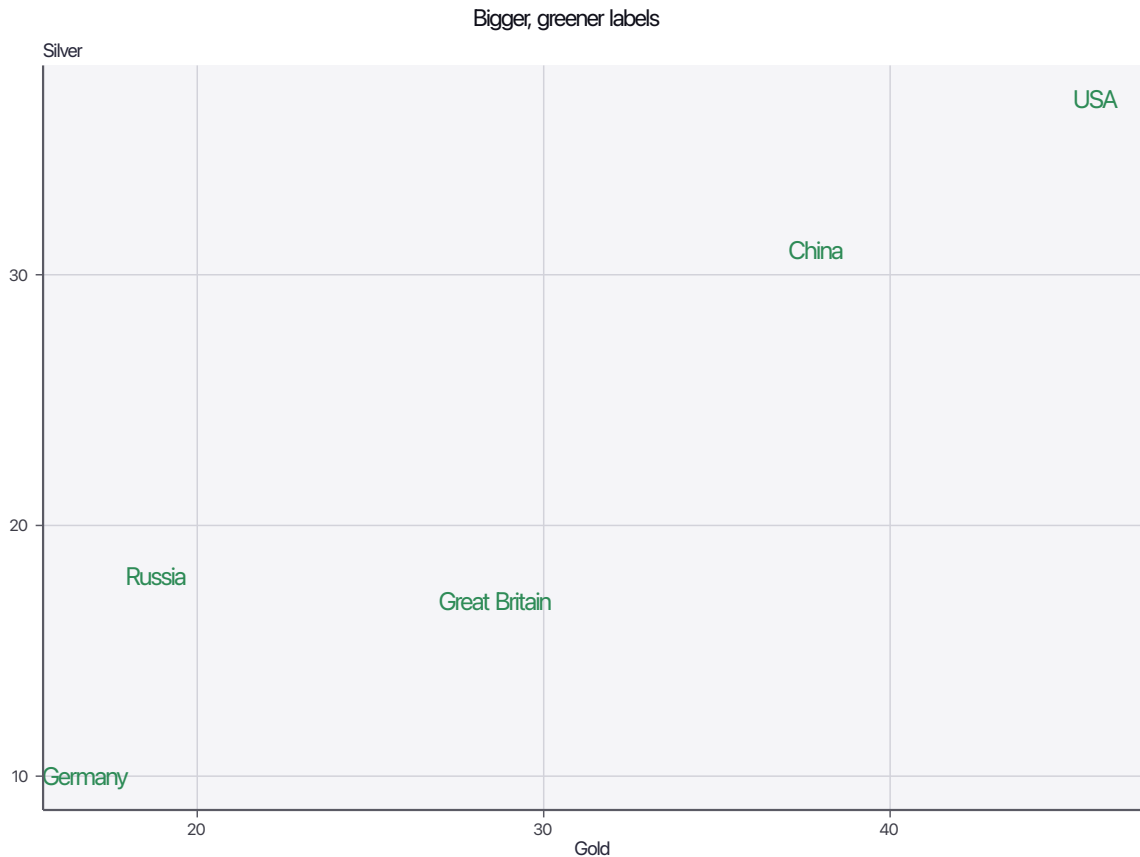
```
(data(medals) + text + x(col.gold) + y(col.silver) + label(col.country) +  
  style(color = "seagreen", size = 17) +  
  x_label("Gold") + y_label("Silver") +  
  title("Bigger, greener labels"))
```

Julia

```
data(medals) + text + x(:gold) + y(:silver) + label(:country) +  
  style(color = "seagreen", size = 17) + x_label("Gold") +  
  y_label("Silver") + title("Bigger, greener labels")
```

JavaScript

```
plot(data(medals), text, x(col.gold), y(col.silver), label(col.country),  
  style({ color: "seagreen", size: 17 }), x_label("Gold"),  
  y_label("Silver"), title("Bigger, greener labels"))
```



“Given the medals: text, x is gold, y is silver, label by country, colored seagreen, with size 17.”

On `text`, `size` is a layer setting, not a mapped channel, so one font size covers every label. A word cloud sizes each word by its frequency, which needs a size per row.

16.6 Why label is a channel

`text` *requires* a label: it is the mark’s minimum syllable, the way `x` and `y` are. Ask for a `text` mark with no label and the engine refuses, and says what to write instead:

```
data(medals) + text + x(gold) + y(silver)
```

Error:

```
! gog: `text` needs `label()` but none is set. Add `label(<column>)` - a text
  ↪ cannot be drawn without it.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The grammar's own rule decides this. A label's string is *data*, and data reaches a mark only through a channel. So `label` is a channel, and a mark takes no `text("USA")` argument. So a fixed note comes from a one-row table (`data`), not a setting. Only `text` accepts `label`; every other mark refuses it, which is why a labeled scatter is `point + text` and never a `label` on `point`. `text` is the mark; `label` is the column it draws.

16.7 A note is a layer

That one-row table is worth seeing drawn. In `ggplot2` a note is a separate function, `annotate()`. In `gog` it is a table with one row. A second `data()` opens a second layer over its own table (the [Data](#) chapter's rule), and a table with one row in it puts one string on the plot:

R

```
notes <- data.frame(gdp = 35000, life = 66, what = "high income, short
↪ lives")
data(gapminder_2007) + point + x(gdp) + y(life) +
  data(notes) + text + label(what) +
  title("A note is a layer, not a function")
```

Python

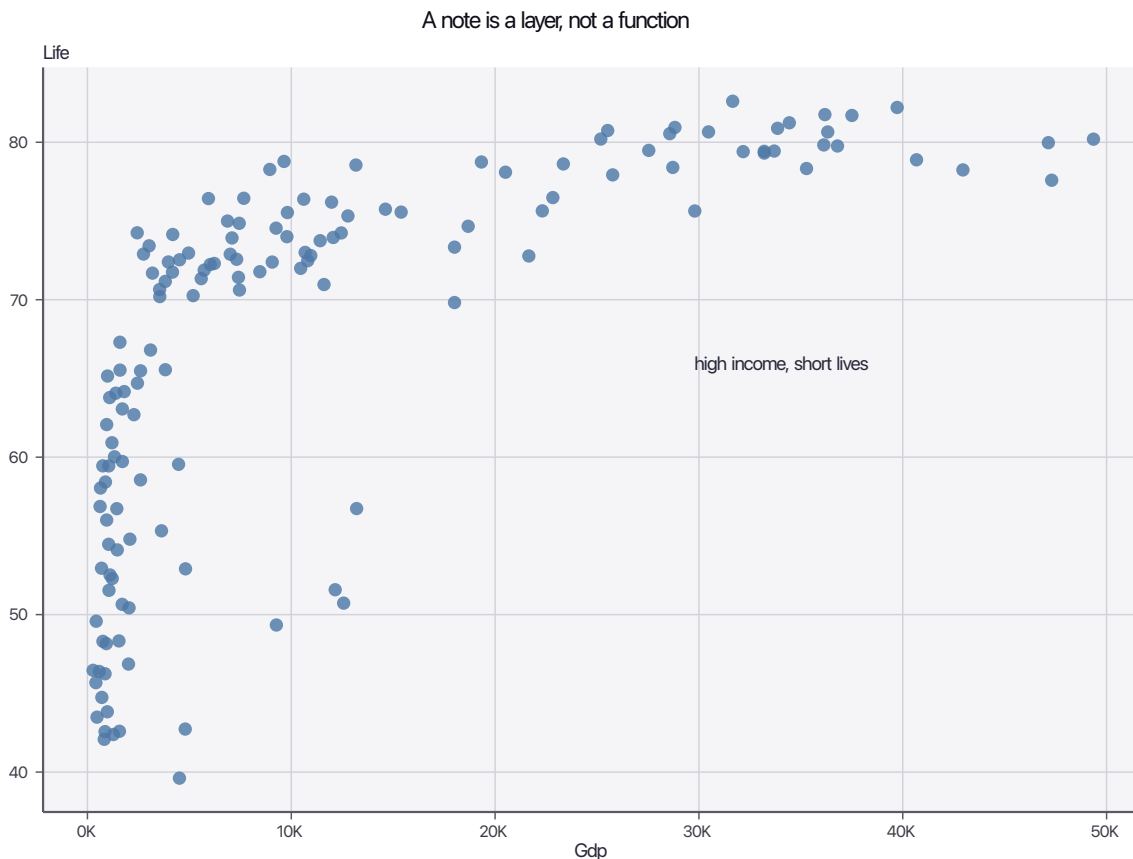
```
notes = {"gdp": [35000], "life": [66], "what": ["high income, short lives"]}
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  data(notes) + text + label(col.what) +
  title("A note is a layer, not a function"))
```

Julia

```
notes = (gdp = [35000], life = [66], what = ["high income, short lives"],)
data(gapminder_2007) + point + x(:gdp) + y(:life) + data(notes) + text +
  label(:what) + title("A note is a layer, not a function")
```

JavaScript

```
const notes = { gdp: [35000], life: [66], what: ["high income, short lives"]
↪ };
plot(data(gapminder_2007), point, x(col.gdp), y(col.life), data(notes),
  text, label(col.what), title("A note is a layer, not a function"))
```



“Given gapminder 2007: points, x is gdp, y is life, and also text from the notes, label by what.”

Nothing was added to the grammar to reach that. `data()` already binds the nearest table, `text` already draws one string per row, and a note is a table that happens to have one row. The note is data like any other, so it is scaled like any other. Place it far from the points and the axes widen to include it.

One thing to know while writing a note. `x` and `y` belong to the whole plot, so the note’s table needs a column for each. The note above reuses the base names, `gdp` and `life`. A note with its own names writes its positions after `text`: `+ text + x(at) + y(value) + label(what)`. Everything else `annotate()` does in the data’s own units is already sayable. A dot on the plot is a `point` over the same one-row table, and several notes are several rows.

16.8 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(nudge =)</code>	up, down, left, right

And these vary per row if you map them to a column instead: `color()` (categories), `label()` (either), `play()` (either).

`nudge` is the one setting only `text` has. It exists so a layered `point + text` does not draw the label on top of the dot:

R

```
data(medals) + x(country) + y(gold) +
  point + style(color = "gray", size = 6) +
  text + label(gold) + style(nudge = "up", size = 11, color = "black") +
  y_label("Gold medals") + title("Labels nudged off their points")
```

Python

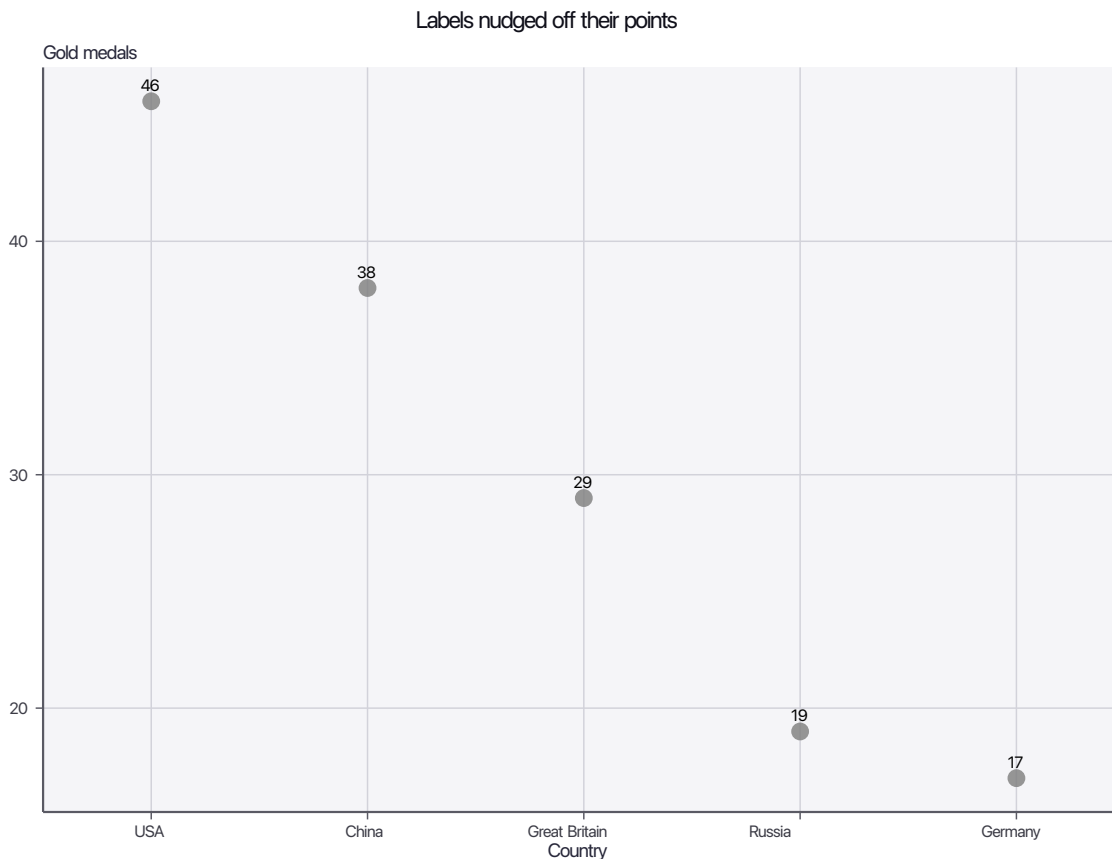
```
(data(medals) + x(col.country) + y(col.gold) +
  point + style(color = "gray", size = 6) +
  text + label(col.gold) + style(nudge = "up", size = 11, color = "black") +
  y_label("Gold medals") + title("Labels nudged off their points"))
```

Julia

```
data(medals) + x(:country) + y(:gold) + point +
  style(color = "gray", size = 6) + text + label(:gold) +
  style(nudge = "up", size = 11, color = "black") +
  y_label("Gold medals") + title("Labels nudged off their points")
```

JavaScript

```
plot(data(medals), x(col.country), y(col.gold), point,
  style({ color: "gray", size: 6 }), text, label(col.gold),
  style({ nudge: "up", size: 11, color: "black" }),
  y_label("Gold medals"), title("Labels nudged off their points"))
```



The four directions and the fixed offset are under [Nudging the label off its point](#), with `repel` for labels that collide with each other.

A mapped `size` on `text` is valid grammar this engine does not draw, so `size(population)` is refused as unsupported rather than ignored. Set the font size with `style(size = )` instead.

16.9 What it refuses

A text mark's glyph *is* its content, so it has nothing left for `shape` to vary:

```
data(medals) + text + x(gold) + y(silver) + label(country) + shape(country)
```

Error:

```
! gog: `shape` cannot be bound to `text` - a text has no shape feature.
  ↳ Remove the `country` mapping from `shape`, or use a mark that has one.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```


A text mark's form comes from `label`, for the reason [Why label is a channel](#) gave. For the same reason a string takes no texture, being neither a stroke to dash nor a region to hatch:

```
data(medals) + text + x(gold) + y(silver) + label(country) +  
  style(pattern = "hatch")
```

Error:

```
! gog: `style(pattern = )` is a stroke's dash or a fill's texture, and a  
  ↳ `text` is a glyph, not either - a point's form is set by `shape`, a  
  ↳ text's by its content. The strokes  
  ↳ (`line`/`step`/`interval`/`path`/`rule`/`edge`) take a dash; the fills  
  ↳ (`area`/`bar`/`box`/`ribbon`/`zone`) a texture.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

17 Path

How did income and life expectancy change together in each country, year by year? `path` strokes the rows *in the order the table gives them*. That is all of it, and it is the one thing `line` does not do. A line sorts its vertices by `x` first, because a line draws a *function*: one `y` for each `x`, read left to right along a domain. A path draws a *route*. It visits row 1, then row 2, and it may turn back, cross itself, or return to where it started.

Everything the two marks share, they share exactly: same stroke, same `color` and `group` splits, same `style(size = )`, same `style(pattern = )` dash. One question separates them: does the order of the rows mean something? Because a path keeps that order, it can carry three things a `line` cannot in general: an arrowhead, a spiral, and the third dimension. All three follow from the row order, so none of them is a special case.

17.1 The connected scatterplot

Here are five Asian countries, each measured twelve times between 1952 and 2007. Put income on one axis and life expectancy on the other, and connect each country's measurements in the order they were taken:

R

```
data(gapminder_asia) + path + x(gdp) + y(life) + color(country) +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Fifty-five years, five countries, one plane")
```

Python

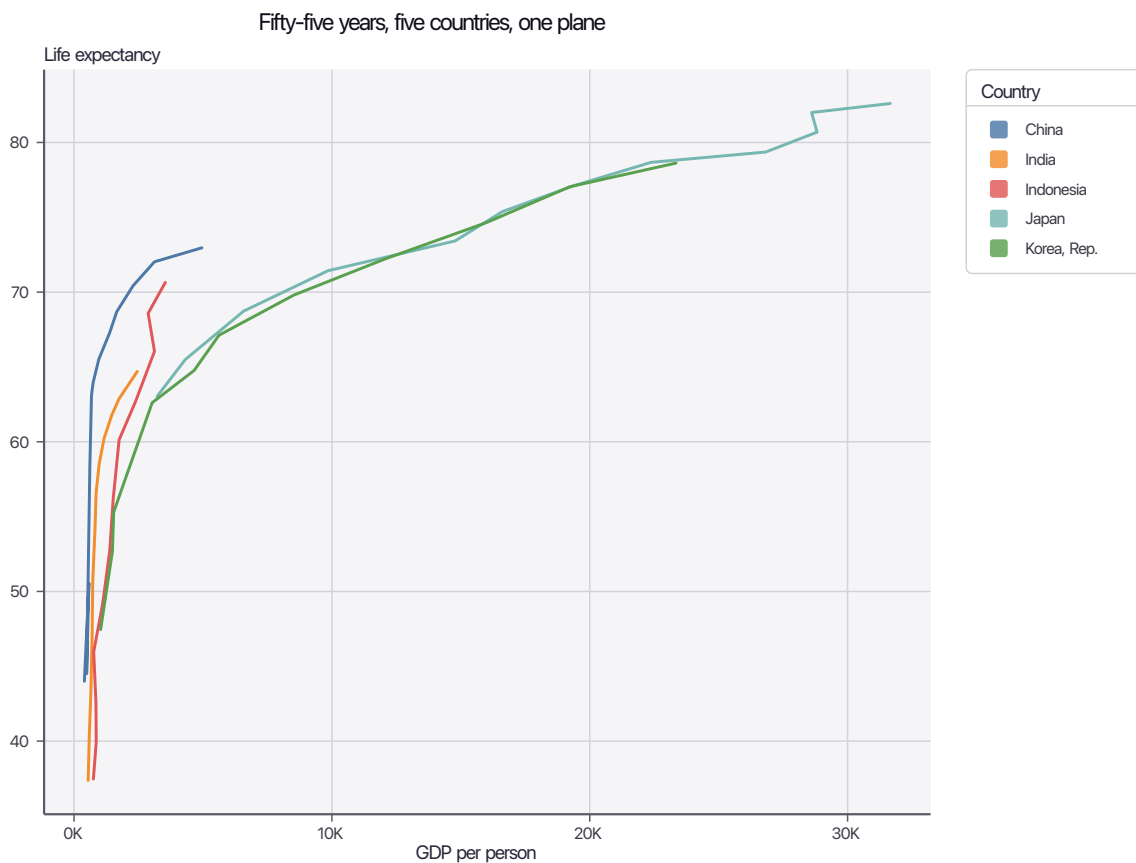
```
(data(gapminder_asia) + path + x(col.gdp) + y(col.life) + color(col.country)  
  ↪ +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Fifty-five years, five countries, one plane"))
```

Julia

```
data(gapminder_asia) + path + x(:gdp) + y(:life) + color(:country) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("Fifty-five years, five countries, one plane")
```

JavaScript

```
plot(data(gapminder_asia), path, x(col.gdp), y(col.life),
  color(col.country), x_label("GDP per person"),
  y_label("Life expectancy"),
  title("Fifty-five years, five countries, one plane"))
```



“Given gapminder Asia: paths, x is gdp, y is life, color by country.”

Each route is one country’s history. Time is nowhere on the plot, and yet the whole plot is about time: it is the order the points are joined in. Japan climbs steeply, then turns right

and flattens: past that corner, income keeps rising and life expectancy barely moves. Korea's route bends in the same place two decades later. China's route goes almost straight up at very low income. That shape belongs to a country that got healthier long before it got rich, and Indonesia's route bends the same way. This is a **connected scatterplot**, and no other arrangement of these atoms draws it.

17.2 Why line cannot draw it

Change one word and the same sentence draws the wrong picture:

R

```
data(gapminder_asia) + line + x(gdp) + y(life) + color(country) +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("the same data, sorted by x")
```

Python

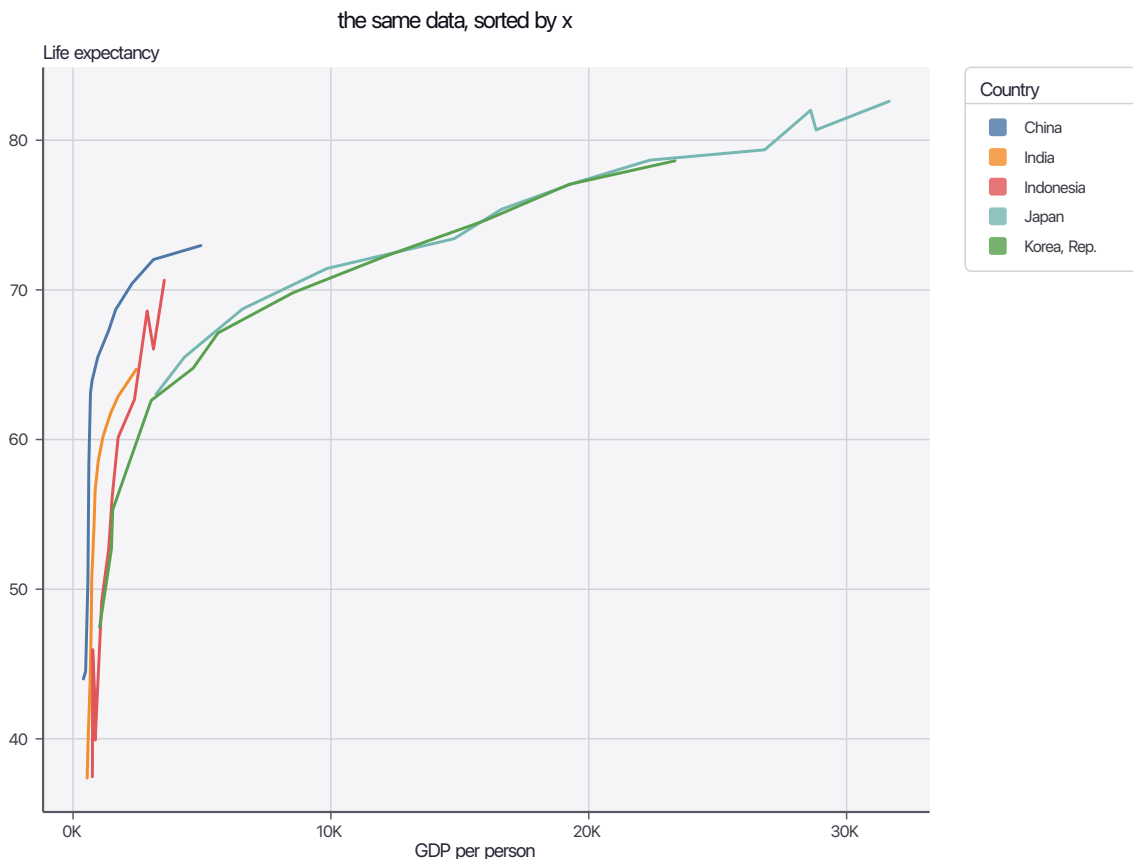
```
(data(gapminder_asia) + line + x(col.gdp) + y(col.life) + color(col.country)  
↪ +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("the same data, sorted by x"))
```

Julia

```
data(gapminder_asia) + line + x(:gdp) + y(:life) + color(:country) +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("the same data, sorted by x")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.gdp), y(col.life),  
  color(col.country), x_label("GDP per person"),  
  y_label("Life expectancy"), title("the same data, sorted by x"))
```



“Given gapminder Asia: lines, x is gdp, y is life, color by country.”

Look at China’s route. Its income did not rise steadily: it fell and recovered, so the year order and the income order are different sequences, and `line` draws the income one. Every point is in the right place, and every route between them is wrong. That is not a bug in `line`; it is what a line *is*. A function has one value per `x`, so sorting changes nothing. For a time series read along a date axis, the sort puts the rows in date order.

The two marks answer different questions, so a specification that names the wrong one gets a plausible picture rather than an error. The rule is this. If the reader should follow the marks in a particular order, that order has to come from the table. Only `path` reads it there.

17.3 Which end is the last row?

A route has a direction, and the plot above does not show it. Japan’s route could be read from either end. So `path` takes a setting no other mark can:

R

```
data(gapminder_asia) + path + x(gdp) + y(life) + color(country) +  
  style(arrow = "end") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("style(arrow = \"end\"): the head sits at 2007")
```

Python

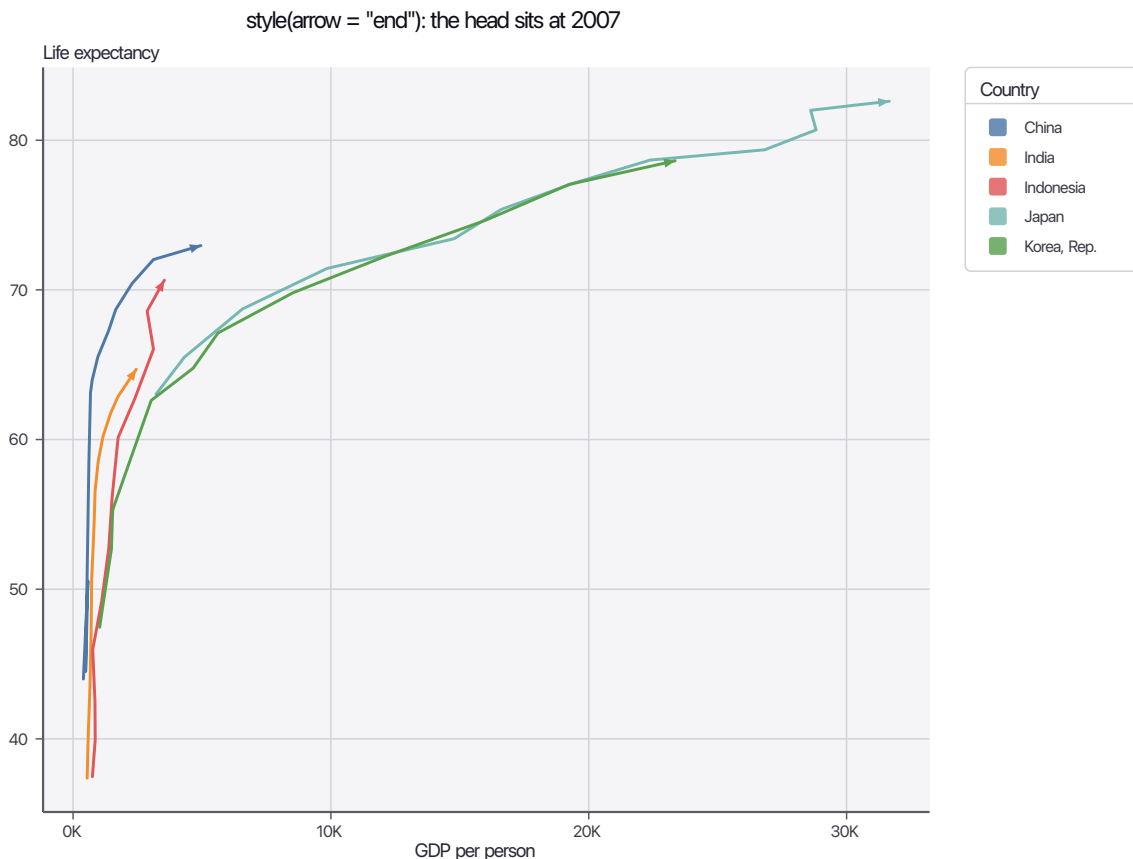
```
(data(gapminder_asia) + path + x(col.gdp) + y(col.life) + color(col.country)  
↪ +  
  style(arrow = "end") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("style(arrow = \"end\"): the head sits at 2007"))
```

Julia

```
data(gapminder_asia) + path + x(:gdp) + y(:life) + color(:country) +  
  style(arrow = "end") + x_label("GDP per person") +  
  y_label("Life expectancy") +  
  title("style(arrow = \"end\"): the head sits at 2007")
```

JavaScript

```
plot(data(gapminder_asia), path, x(col.gdp), y(col.life),  
  color(col.country), style({ arrow: "end" }), x_label("GDP per person"),  
  y_label("Life expectancy"),  
  title("style(arrow = \"end\"): the head sits at 2007"))
```



“Given gapminder Asia: paths, x is gdp, y is life, color by country, with arrow end.”

Now each route points at its last row. `arrow` takes "end", "start", or "both": a head at the last row, at the first row, or at both ends.

It belongs to `path` alone, and every other mark refuses it. The refusal is worth reading, because it states why `path` is a separate mark:

```
render_svg(data(gapminder_asia) + line + x(year) + y(life) + style(arrow =
  ↪ "end"))
```

Error:

```
! gog: `style(arrow = )` is a `path` setting, and a `line` cannot carry it -
  ↪ a line sorts its vertices by `x`, so its last point is wherever the
  ↪ domain ends rather than where the data stopped - a head there would point
  ↪ at the sort. Use `path`, which strokes the rows in the data's own order
  ↪ and so has an end the data chose.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A line's last vertex is wherever the domain happens to end, so a head there marks the sort rather than anything the data did. Only a path has an end that comes from the data.

17.4 An arrow is a layer

The [Text](#) chapter drew the note; `path` draws the arrow beside it. A note is a one-row table; the arrow that points from the note to the thing is a *two-row* table, one row for each end:

R

```
seg <- data.frame(gdp = c(20000, 13100), life = c(45.5, 50.2))
note <- data.frame(gdp = 20500, life = 44.8, what = "Botswana")

data(gapminder_2007) + point + x(gdp) + y(life) +
  data(seg) + path + style(arrow = "end", color = "#c0392b") +
  data(note) + text + label(what) +
  title("an annotation, in the grammar you already have")
```

Python

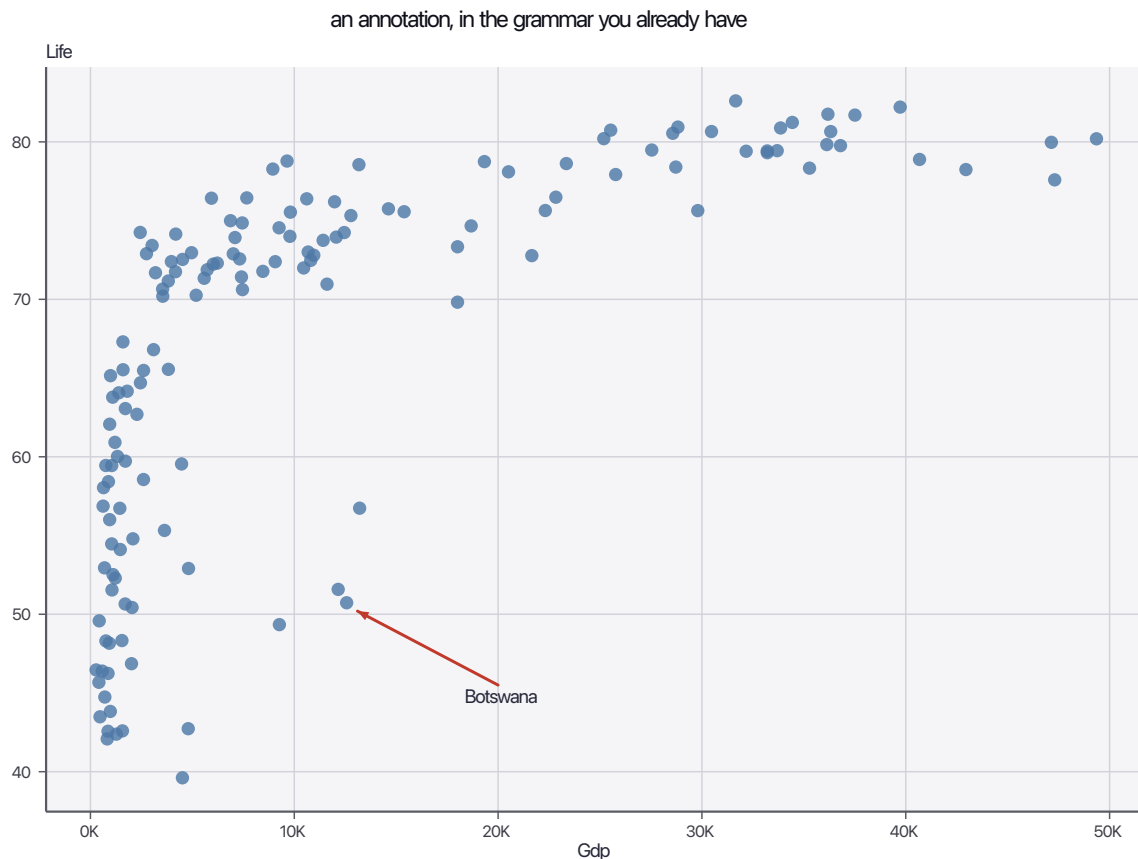
```
seg = {"gdp": [20000, 13100], "life": [45.5, 50.2]}
note = {"gdp": [20500], "life": [44.8], "what": ["Botswana"]}
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  data(seg) + path + style(arrow = "end", color = "#c0392b") +
  data(note) + text + label(col.what) +
  title("an annotation, in the grammar you already have"))
```

Julia

```
seg = (gdp = [20000, 13100], life = [45.5, 50.2],)
note = (gdp = [20500], life = [44.8], what = ["Botswana"],)
data(gapminder_2007) + point + x(:gdp) + y(:life) + data(seg) + path +
  style(arrow = "end", color = "#c0392b") + data(note) + text +
  label(:what) + title("an annotation, in the grammar you already have")
```

JavaScript


```
const seg = { gdp: [20000, 13100], life: [45.5, 50.2] };
const note = { gdp: [20500], life: [44.8], what: ["Botswana"] };
plot(data(gapminder_2007), point, x(col.gdp), y(col.life), data(seg),
  path, style({ arrow: "end", color: "#c0392b" }), data(note), text,
  label(col.what),
  title("an annotation, in the grammar you already have"))
```



“Given gapminder 2007: points, x is gdp, y is life, and also a path from the seg table and text from the note table, label by what.”

This grammar has no `annotate()` function, and it needs none. An annotation is a layer over a small table, and the marks it uses are the marks everything else uses: `text` for the words, `path` for the arrow, `point` for a marker. What the table holds is data, so it is scaled like data, which is why the arrow’s ends are written in the data’s own units rather than in pixels.

17.5 Two transforms, and the rest refused

The other stroke and band marks compose with many transforms. `path` composes with two, `density` and `cluster`, and the refusals say why:

```
render_svg(data(gapminder_asia) + path * mean + x(country) + y(life))
```

Error:

```
! gog: `path` strokes the rows in the order the table gives them, and `mean`  
  ↳ replaces those rows with one summary per key - after which the order is  
  ↳ the keys' and the path is a `line`. Use `line * mean`, which sorts by `x`  
  ↳ and is the mark a statistic is drawn on.
```

```
gog: `path * mean` reduces a column within each cell, but nothing says which  
  ↳ column - `path` measures by `color`, and no `color()` is bound. Name it:  
  ↳ `path * mean + x(<a>) + y(<b>) + color(<column>)`. To count the rows in  
  ↳ each cell instead of reducing a column, `count` needs no such binding:  
  ↳ `path * count + x(<a>) + y(<b>)`.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A summary replaces the rows. `mean` reduces each country to one number, and what is left to connect is one point per country, in whatever order the summary produced them. That is a **line**, written the long way around. The order a *path* *is* does not survive being summarized, and that one fact empties almost the whole row this mark has on the [Combinations](#) grid.

`density` is the one exception, and it does not break the rule above. A summary reduces each group to one value. A density estimated over the plane groups nothing and reduces nothing. It produces vertices *in the order they were traced*, which is the one thing a path is for.

17.6 The contour: path * density

Bind both positions and compose `density`. `gog` estimates how thick the cloud of points is everywhere on the plane, then traces the rings where it is equally thick:

R

```
data(iris_flowers) + path * density +  
  x(sepal_length) + y(petal_length) +  
  title("where the flowers cluster")
```

Python

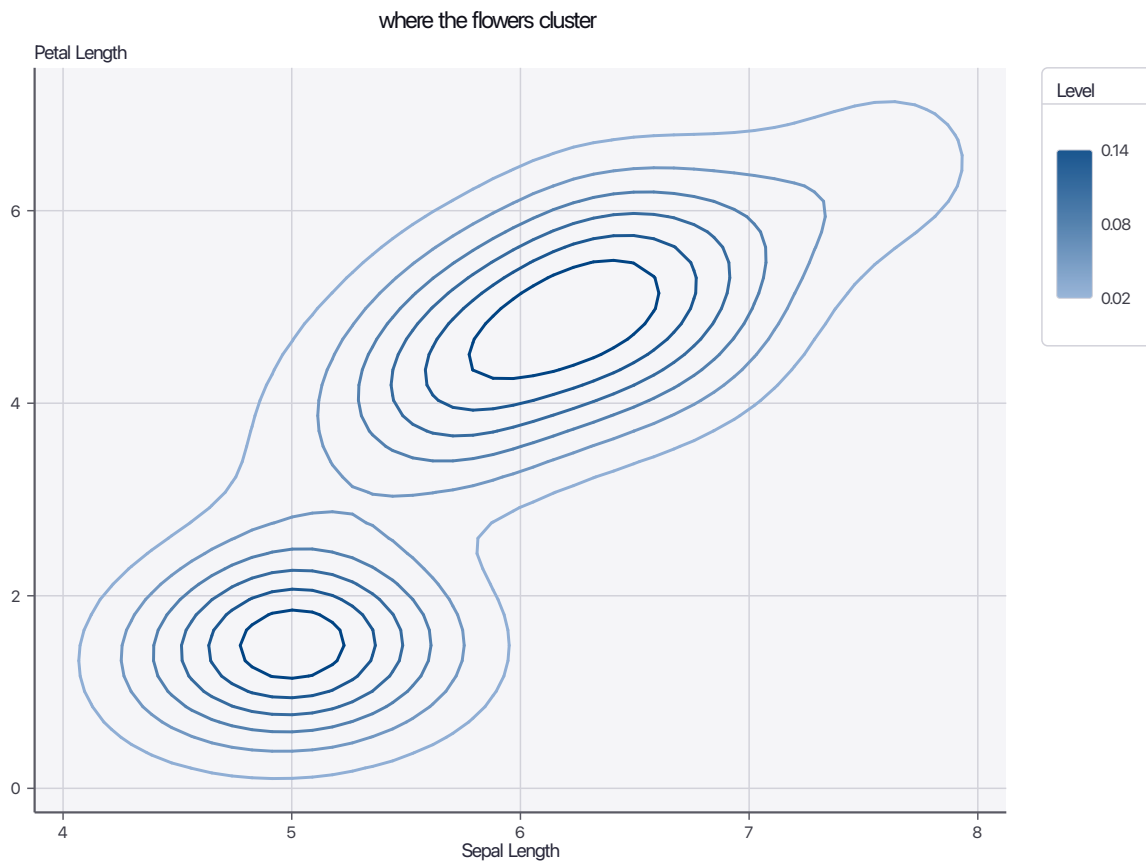
```
(data(iris_flowers) + path * density +
  x(col.sepal_length) + y(col.petal_length) +
  title("where the flowers cluster"))
```

Julia

```
data(iris_flowers) + path * density + x(:sepal_length) +
  y(:petal_length) + title("where the flowers cluster")
```

JavaScript

```
plot(data(iris_flowers), layer(path, density), x(col.sepal_length),
  y(col.petal_length), title("where the flowers cluster"))
```



“Given the iris flowers: paths derived by density, x is sepal length, y is petal length.”

Nothing new was added to the grammar to draw that. One rule decides this, and the same rule turns `zone * bin` into a `heatmap`. The mark decides how many axes a transform cuts, and you never state it. A `bar` leaves an axis free to measure the density along, so `bar * density` spreads it along one axis and draws a curve. A `path` is a route through two columns and treats them equally, so it has no axis left over. `density` therefore cuts both. There is no `contour` atom for the same reason there is no `pie()` (`Polar` makes that argument): the sentence already exists.

Read the outermost ring, then the ones inside it. The outermost is a *single* ring around both clusters, and the next level inward splits into two separate rings. So a level alone cannot tell `gog` where one stroke ends. Each ring is its own stroke, and the level only decides its color, so the two inner rings are never joined by a stroke across the gap between them.

The color is the level, and nothing had to bind it, exactly as `bar * bin` binds `y` for you. Say it out loud if you prefer, and change how many rings are traced:

R

```
data(iris_flowers) + path * density(levels = 10) +  
  x(sepal_length) + y(petal_length) + color(level) +  
  title("ten levels, named out loud")
```

Python

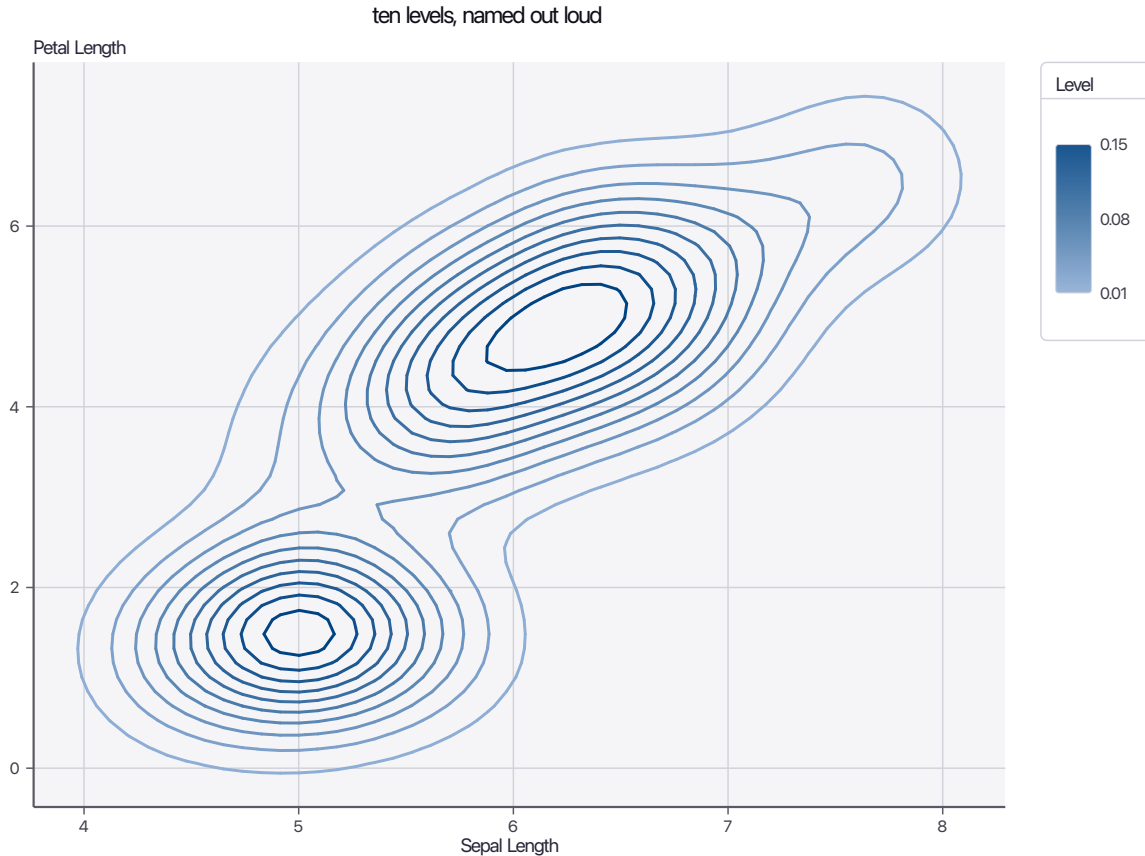
```
(data(iris_flowers) + path * density(levels = 10) +  
  x(col.sepal_length) + y(col.petal_length) + color(col.level) +  
  title("ten levels, named out loud"))
```

Julia

```
data(iris_flowers) + path * density(levels = 10) + x(:sepal_length) +  
  y(:petal_length) + color(:level) + title("ten levels, named out loud")
```

JavaScript

```
plot(data(iris_flowers), layer(path, density({ levels: 10 })),  
  x(col.sepal_length), y(col.petal_length), color(col.level),  
  title("ten levels, named out loud"))
```



“Given the iris flowers: paths derived by density at 10 levels, x is sepal length, y is petal length, color by level.”

density is one transform read three ways. A **line** draws a curve, a **zone** paints the whole field, and a **path** strokes the level sets that **levels** cuts the field into. Each parameter belongs to one reading and is refused in the other two. **levels** cuts the **field**, the estimated thickness everywhere on the plane, into that many steps. A one-dimensional density is a single curve, and a curve has no field to cut.

```
render_svg(data(iris_flowers) + line * density(levels = 8) + x(sepal_length))
```

Error:

```
! gog: `density(levels = )` cuts a field into levels, and a `line` estimates
  ↳ *one curve* - a curve has no levels to cut. Drop it for a density curve,
  ↳ or read the density over both axes: `path * density(levels = )` traces
  ↳ the contours, `zone * density(levels = )` fills the bands between them.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

`bandwidth` is refused for the mirror reason: it needs one axis, and a contour has two. It is a width in the data's own units, and a contour spreads over two columns that measure different things. Here the two agree, since sepal length and petal length are both in centimeters, but two axes usually do not. One number cannot be a width in both:

```
render_svg(data(iris_flowers) + path * density(bandwidth = 0.5) +
  x(sepal_length) + y(petal_length))
```

Error:

```
! gog: `density(bandwidth = )` is a width in one column's own units, and
  ↳ `path * density` spreads over *two* columns measuring different
  ↳ quantities - one number cannot be a width in both. Use `density(adjust =
  ↳ )`, which scales the automatic bandwidth on each axis by the same
  ↳ dimensionless factor.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

`adjust` is a multiplier rather than a length, so it means the same thing on either axis. It is the one parameter all three readings accept:

R

```
data(iris_flowers) + path * density(adjust = 1.6) +
  x(sepal_length) + y(petal_length) +
  title("the same cloud, smoothed harder")
```

Python

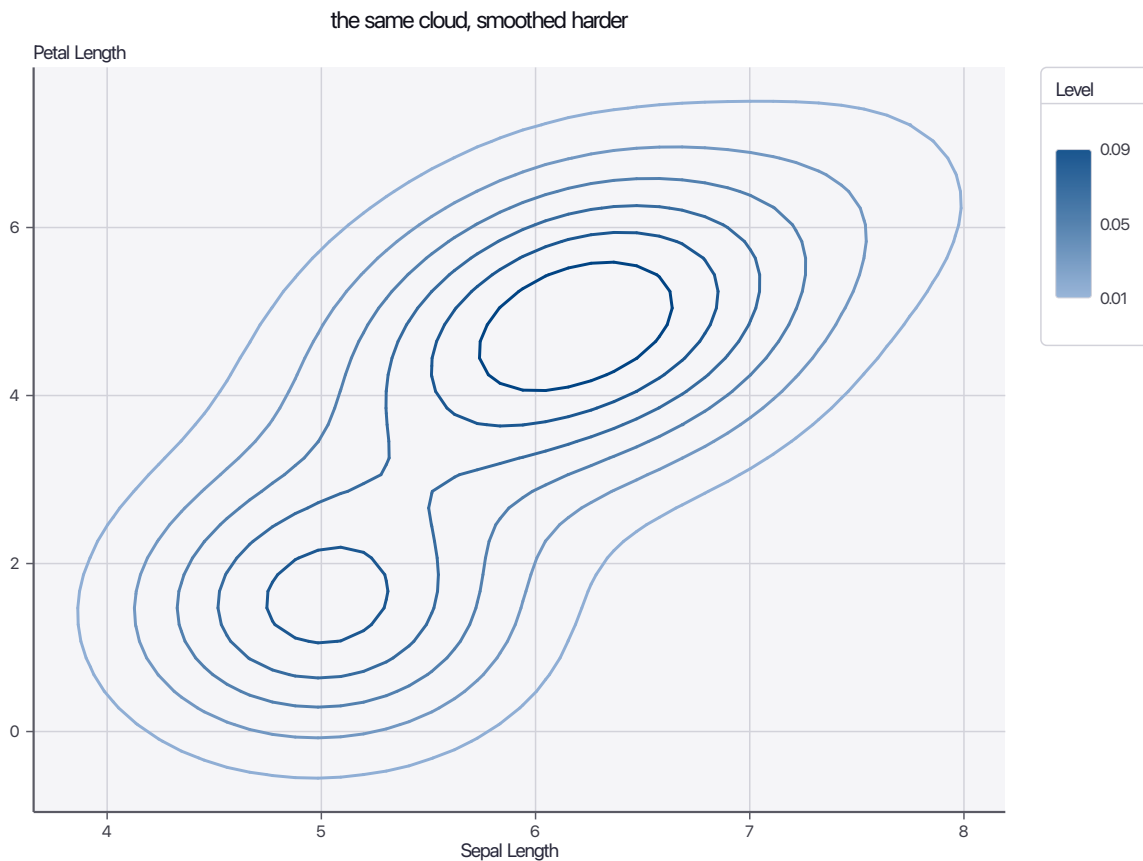
```
(data(iris_flowers) + path * density(adjust = 1.6) +
  x(col.sepal_length) + y(col.petal_length) +
  title("the same cloud, smoothed harder"))
```

Julia

```
data(iris_flowers) + path * density(adjust = 1.6) + x(:sepal_length) +
  y(:petal_length) + title("the same cloud, smoothed harder")
```

JavaScript

```
plot(data(iris_flowers), layer(path, density({ adjust: 1.6 })),
  x(col.sepal_length), y(col.petal_length),
  title("the same cloud, smoothed harder"))
```

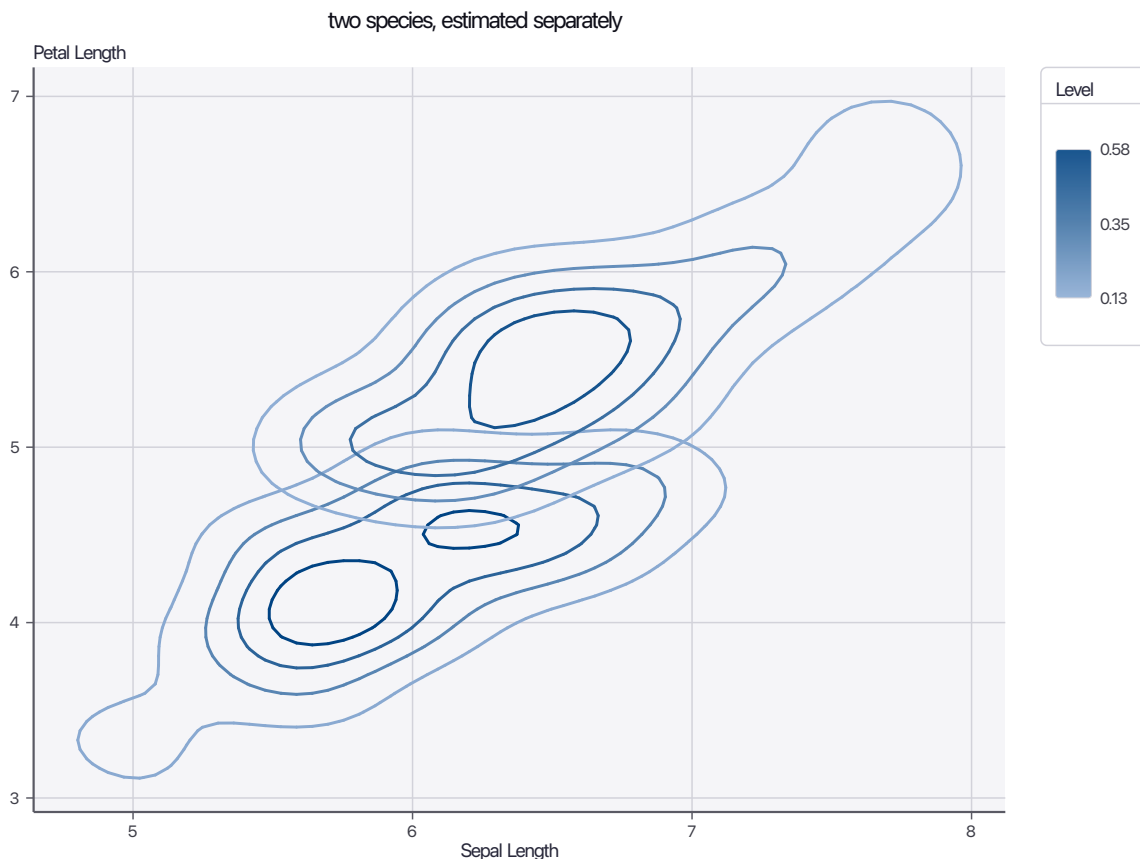


“Given the iris flowers: paths derived by density adjusted by 1.6, x is sepal length, y is petal length.”

To estimate one field per category, split the rows with **group**. The level is already on **color**, which is why the split has to be **group**:

```
petals <- iris_flowers[iris_flowers$species != "setosa", ]

data(petals) + path * density(levels = 4) +
  x(sepal_length) + y(petal_length) + group(species) +
  title("two species, estimated separately")
```



“Given the petals: paths derived by density at 4 levels, x is sepal length, y is petal length, grouped by species.”

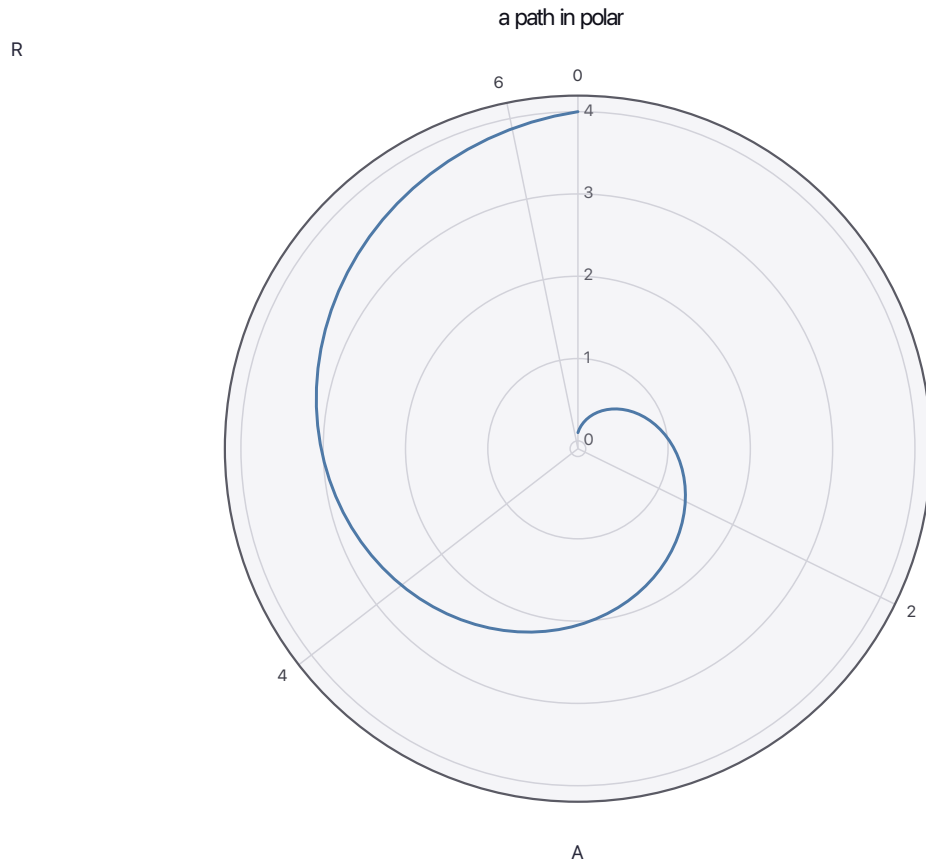
A **zone** has no measure axis either, so it reads **density** in two dimensions too, and it *fills* what this mark strokes. Give both marks the same **levels** and you get the same curves twice: once as strokes, once as the edges of filled bands. That pair is the plot usually meant by a *filled* density. That plot belongs to the mark that fills it, **zone * density**, and the zone chapter covers the color ramp it uses. One field, two marks. It is **the same split** as **bar * bin** and **line * bin**: the mark chooses the geometry, and the transform stays the same.

17.7 In polar, a route is a spiral

path bends into polar like every other mark. Give it an angle and a radius that both grow, and the route winds outward:


```
spiral <- data.frame(a = seq(0, 6.2, length.out = 90))
spiral$r <- seq(0.1, 4, length.out = 90)

data(spiral) + path + x(a) + y(r) + polar() +
  title("a path in polar")
```



“Given the spiral: a path, x is a , y is r , in polar.”

There is one thing **path** does not copy from **line**: the segment that closes a radar chart (**Polar** draws one). A categorical **line** in polar joins its last vertex back to its first, because the categories fill the whole turn, and a gap there would leave one sector blank. A path does not read the angular axis as a set of categories. Its last vertex is where the data stopped, so closing the route would add a segment nobody asked for. If a route does return to its start, say so by repeating the first row.

17.8 In space, the route is unchanged

The `thermals` table holds two gliders circling the same thermal, a rising column of warm air, sampled as they climb:

Table 17.1: `thermals`: first 5 of 340 rows

east	north	altitude	glider	second
330.0000	0.00000	900.0000	Alpha	0
327.5939	39.77710	907.0082	Alpha	2
320.4108	78.97417	914.0163	Alpha	4
308.5554	117.01961	921.0245	Alpha	6
292.2005	153.35865	928.0327	Alpha	8

Bind `z` and the route runs through the cube instead of across the page. Of the marks that join their rows into one stroke, `path` is the only one that takes the third position. The reason is the row order this whole chapter is about:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) + color(glider) +  
  title("Two gliders, one thermal")
```

Python

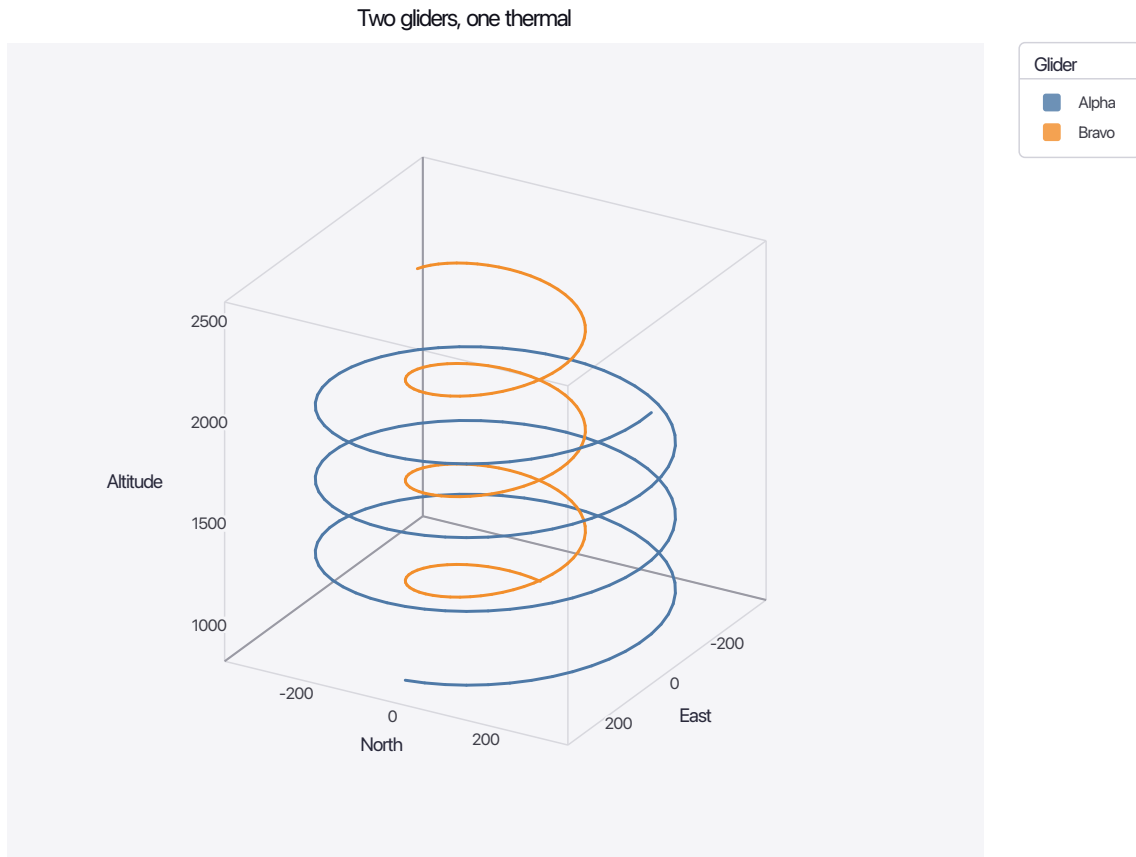
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +  
  color(col.glider) +  
  title("Two gliders, one thermal"))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +  
  color(:glider) + title("Two gliders, one thermal")
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),  
  color(col.glider), title("Two gliders, one thermal"))
```



“Given the thermals: paths, x is east, y is north, z is altitude, color by glider.”

An order belongs to no axis, so it survives the third dimension untouched: the route is the same route however the scene is turned. A `line` sorts by `x`, and the cube has no left to right. There `x` is one of three equal positions. At some viewing angles it runs straight into the page and becomes depth, which would leave a line sorted by an axis the reader cannot see. So the sort that makes a `line` a line cannot follow it into the cube. `line`, `step`, `area` and `ribbon` refuse `z` rather than half-drawing it:

```
data(thermals) + line + x(east) + y(north) + z(altitude)
```

Error:

! gog: `line` reads a **domain** left to right - it sorts by `x` and draws one
 ↪ value for each - and a cube has no left to right: `x` is one of three
 ↪ equal positions, and at some viewing angles it runs into the page and
 ↪ becomes depth. A `line` in space would be sorted by an axis the reader
 ↪ cannot see, so this is refused rather than drawn. For a route through
 ↪ three dimensions use `path`, which is `line` with that sort removed:
 ↪ `path + x(<a>) + y(<b>) + z(altitude)`.
 gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

Watch the two routes where they cross: whichever is nearer to the eye covers the other, and which one that is changes as they climb. A glyph sits at one distance from the eye and can be sorted whole. A stroke running through the cube has a far end and a near end, so the unit that gets sorted is the **segment**. The segments of both routes are sorted together. If each stroke were sorted as a whole, one glider would sit entirely in front of the other: two coils that never meet, rather than two that thread through each other.

Cutting the route into segments is also what lets **color** carry a **measure** along it. Bind the altitude to the color as well as to the height, and the climb is said twice: once by where the route is, once by what color it is:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) +
  color(altitude) + group(glider) + palette("viridis") +
  space(turn = -25, tilt = 20) +
  title("Altitude, said twice")
```

Python

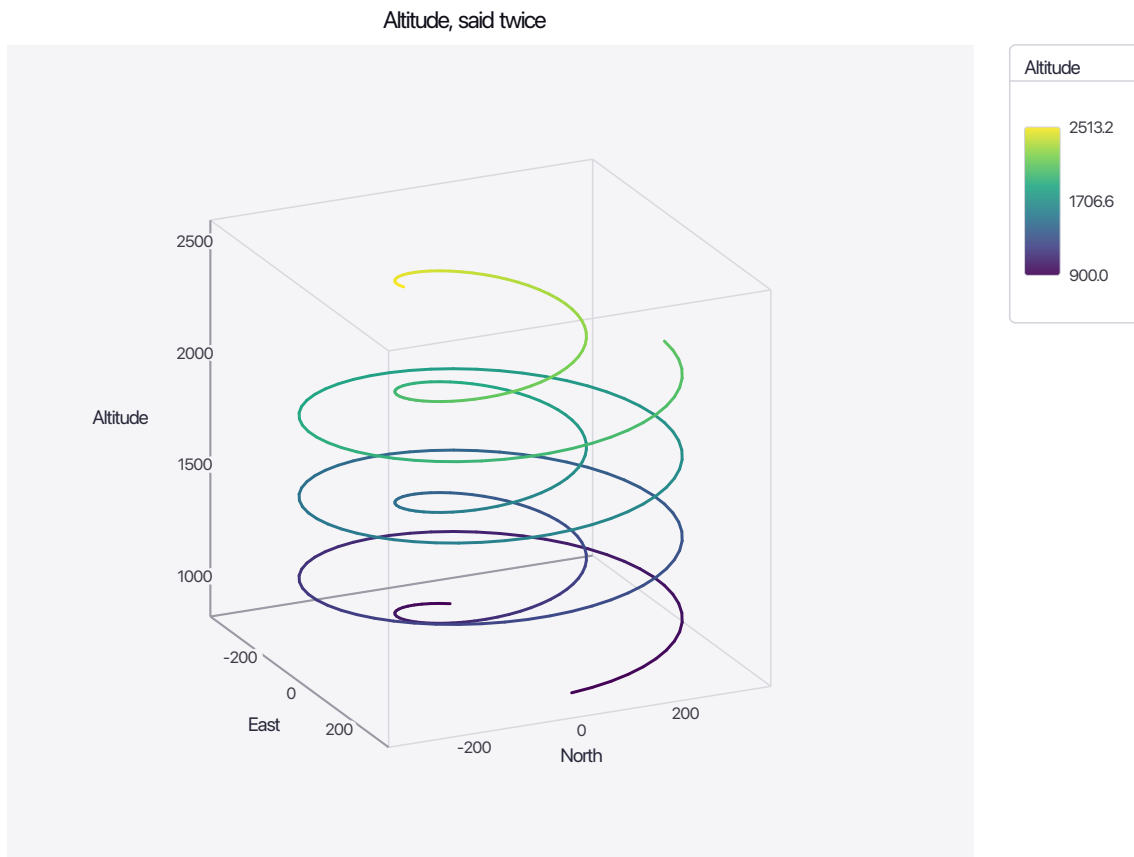
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +
  color(col.altitude) + group(col.glider) + palette("viridis") +
  space(turn = -25, tilt = 20) +
  title("Altitude, said twice"))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +
  color(:altitude) + group(:glider) + palette("viridis") +
  space(turn = -25, tilt = 20) + title("Altitude, said twice")
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),
     color(col.altitude), group(col.glider), palette("viridis"),
     space({ turn: -25, tilt: 20 }), title("Altitude, said twice"))
```



“Given the thermals: paths, x is east, y is north, z is altitude, color by altitude, grouped by glider, with the viridis palette.”

`palette("viridis")` picks the ramp; `group(glider)` keeps the two routes apart. Drop that last atom and the plot is wrong. A *category* on `color` splits the mark into one stroke per group, but a *measure* has no categories to split on. Without `group` the two gliders would be joined into a single route, and a straight line would run from where one glider's rows stop to where the other's begin. Color answers what the route was carrying; `group` answers which route it is. Two questions, two atoms.

A dash survives the segmentation too. Each segment is drawn as its own piece, and a dash would normally restart at the start of each piece:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) + color(glider) +  
  style(pattern = "dashed") +  
  space(turn = -25, tilt = 20)
```

Python

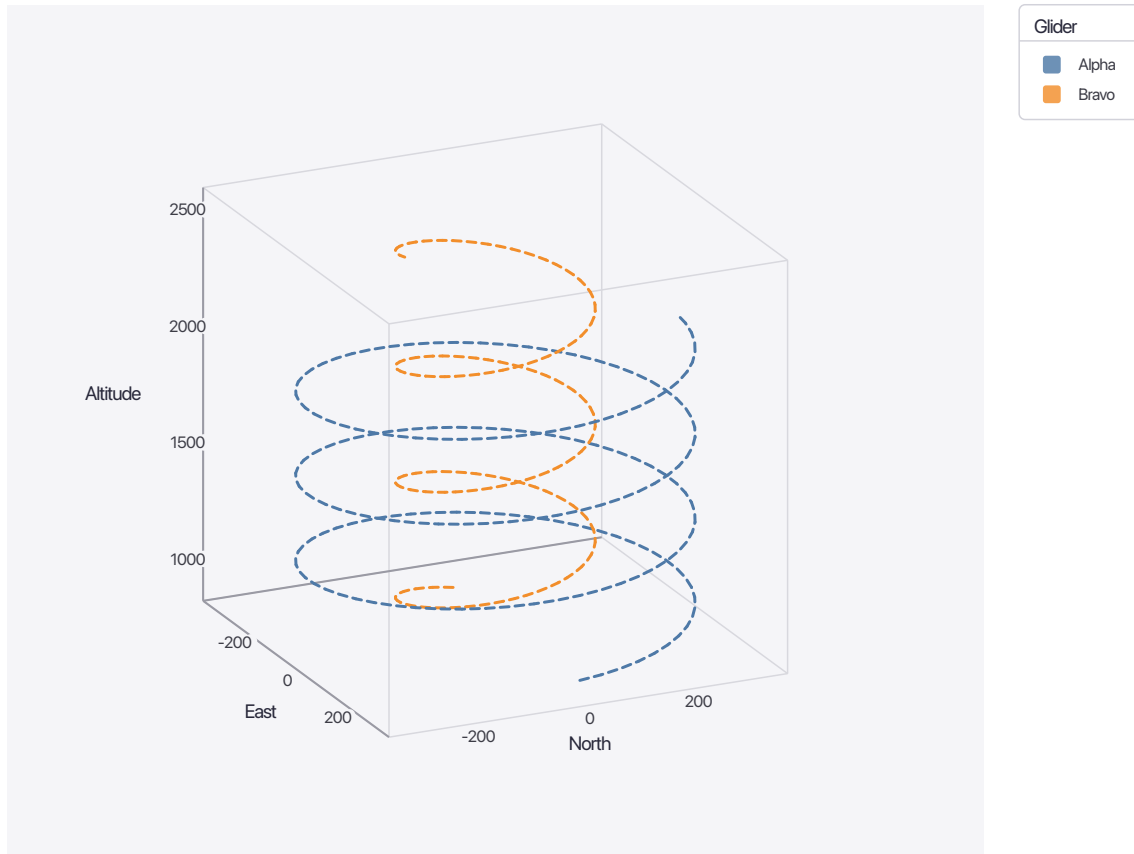
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +  
  ↪ color(col.glider) +  
  style(pattern = "dashed") +  
  space(turn = -25, tilt = 20))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +  
  color(:glider) + style(pattern = "dashed") +  
  space(turn = -25, tilt = 20)
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),  
  color(col.glider), style({ pattern: "dashed" }),  
  space({ turn: -25, tilt: 20 }))
```



“Given the thermals: paths, x is east, y is north, z is altitude, color by glider, with pattern dashed.”

The dashes run along the route rather than resetting at every sample. The dash belongs to the route, not to the segments the depth sort needs. An arrowhead is sorted the same way, at the depth of its own tip. A head that sits deep in the cube is covered by whatever passes in front of it.

[Space](#) owns the third dimension and the viewing angle.

17.9 What a path is for

The connected scatterplot is the best known use, but the mark is more general than that. Any picture whose reading order is the table’s order is a path. A trajectory through two measurements over time is one. So is a route through space, and a cycle that returns to where it began. So is a hysteresis loop, which is a cycle whose forward and return paths differ, and so

is an arrow. The test is always the same question. Does the order of the rows mean something? If it does, the mark is **path**; if the reader should read left to right, it is **line**.

17.10 What you can set

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(pattern =)</code>	solid, dashed, dotted
<code>style(arrow =)</code>	end, start, both

And these vary per row if you map them to a column instead: `color()` (either), `pattern()` (categories), `group()` (categories), `play()` (either).

arrow belongs to **path** alone, and **geometry** decides that rather than **taste**. A **head** marks a direction, and a **path** is the only mark that has one. **line**, **step** and **area** sort their vertices by **x**, so their last point is wherever the domain ends rather than where the data stopped.

R

```
data(gapminder_asia) + path + x(gdp) + y(life) + color(country) +  
  style(arrow = "both", size = 2.5, pattern = "dotted") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Both ends marked, on a dotted route")
```

Python

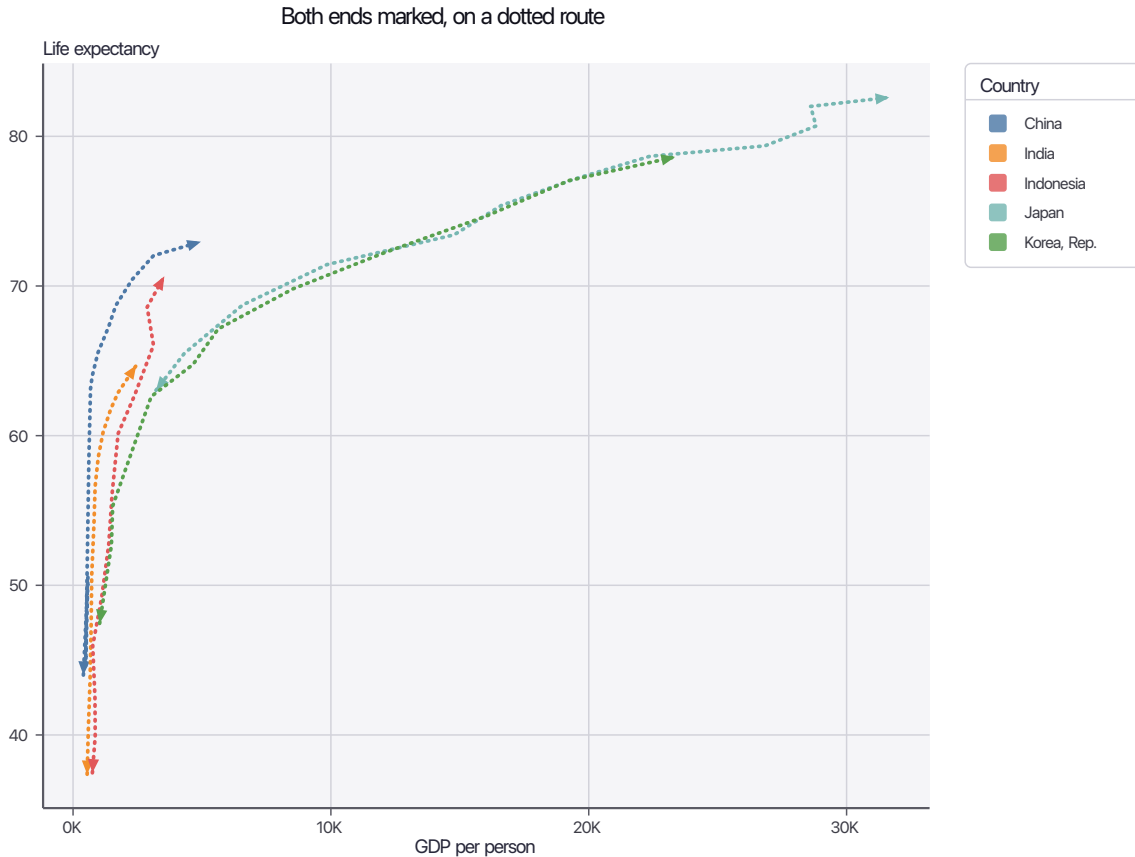
```
(data(gapminder_asia) + path + x(col.gdp) + y(col.life) + color(col.country)  
↪ +  
  style(arrow = "both", size = 2.5, pattern = "dotted") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Both ends marked, on a dotted route"))
```

Julia

```
data(gapminder_asia) + path + x(:gdp) + y(:life) + color(:country) +  
  style(arrow = "both", size = 2.5, pattern = "dotted") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Both ends marked, on a dotted route")
```


JavaScript

```
plot(data(gapminder_asia), path, x(col.gdp), y(col.life),
     color(col.country),
     style({ arrow: "both", size: 2.5, pattern: "dotted" }),
     x_label("GDP per person"), y_label("Life expectancy"),
     title("Both ends marked, on a dotted route"))
```



“Given *gapminder Asia*: *paths*, *x* is *gdp*, *y* is *life*, color by country, with arrow both, size 2.5 and pattern dotted.”

It takes a *value* rather than a true-or-false flag, because “both” is an ordinary thing to ask for, and a flag would have needed a second setting to say it. The heads draw solid on a dotted path. The dash is the route’s texture, and a head cut into dashes no longer reads as an arrow.

The transform whose parameters this chapter covers is **density**, and **levels** and **adjust** are above. **cluster** takes its own, in [Cluster](#). Everything else that varies is either mapped or set.

17.11 What it refuses

A path is one stroke, and a stroke has one width. There is no answer to how thick the route is at a single point, so width cannot vary along it:

```
data(gapminder_asia) + path + x(gdp) + y(life) + size(gdp)
```

Error:

```
! gog: `size` cannot be bound to `path` - a path has no size feature. Remove  
  ↳ the `gdp` mapping from `size`, or use a mark that has one.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Set it instead and every segment takes the same width, which is the [Setting vs mapping](#) distinction again. The refusals that belong to `path` alone are stated above rather than here. One is the summaries it cannot take, because a summary replaces the very rows whose order *is* the path. One is `density(bandwidth = )`, which is a width in one column's units and cannot be a width in two. The last is an arrow asked of `line`, which has no end for a head to sit on.

18 Rule

Where is the line you want to compare every value against? **rule** draws a line at one position and lets the other axis decide how far it reaches. Every other mark reads two positions; a rule reads one and takes the rest from the panel. That gives two familiar plots: a reference line at a threshold, and a rug of ticks showing where the observations sit.

Those are often three separate marks: a vertical reference line, a horizontal one, and a rug. **gog** has one. A vertical and a horizontal reference line differ only in which axis carries the position. The grammar reads that off the bindings, exactly as it does for a bar.

18.1 A line at a threshold

Here is the 2007 gapminder scatter, income against life expectancy, with lines drawn at three life-expectancy thresholds:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  data(life_bands) + rule +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Three thresholds, one table")
```

Python

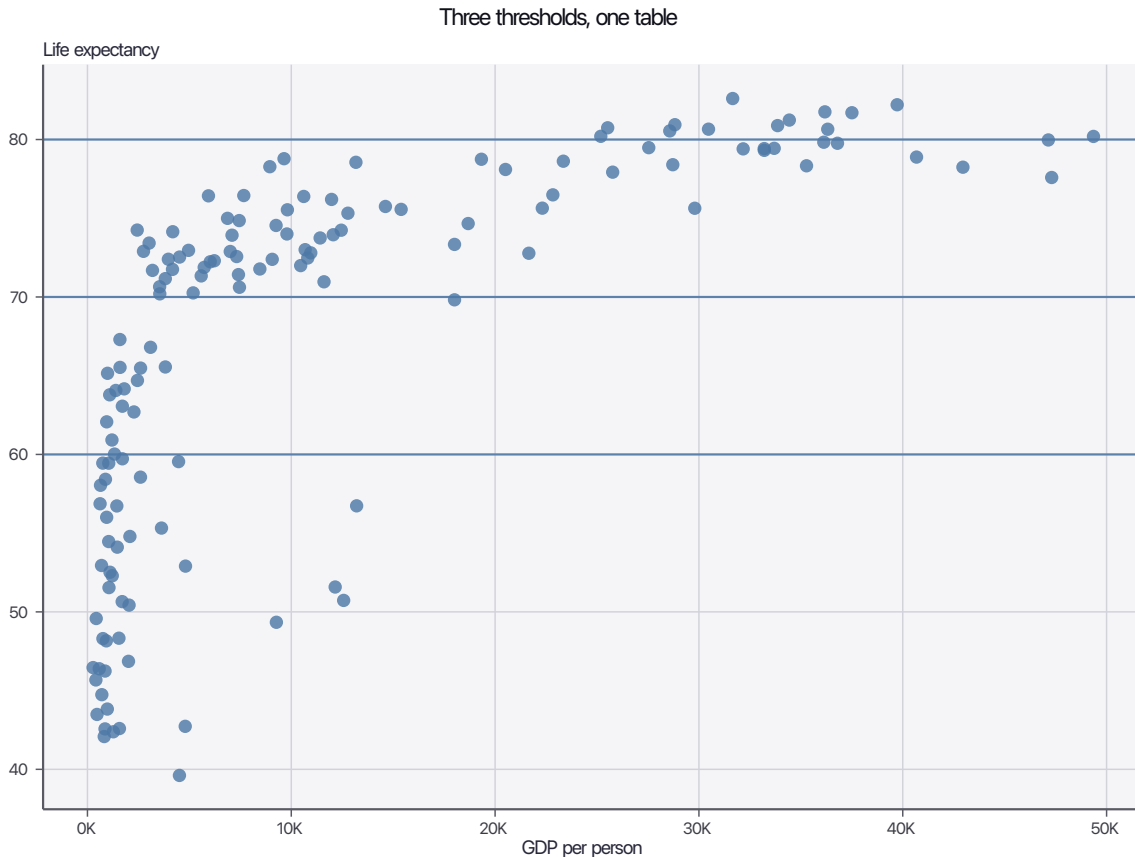
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  data(life_bands) + rule +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Three thresholds, one table"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + data(life_bands) +  
  rule + x_label("GDP per person") + y_label("Life expectancy") +  
  title("Three thresholds, one table")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
     data(life_bands), rule, x_label("GDP per person"),  
     y_label("Life expectancy"), title("Three thresholds, one table"))
```



“Given gapminder 2007: points, x is gdp, y is life, and also rules from the life bands.”

Three lines, and only one **rule** in the sentence. That is the mark’s most useful property: its position is a **column**, not a number. **life_bands** holds three values, so it draws three lines. Add a fourth row and a fourth line appears with no change to the plot’s sentence at all.

Every atom that takes a column works this way (**color(species)** names a column, not a color), and that is why there is no **rule(70)**. A number written into the plot would be a value the grammar could not scale, facet, or bend into another coordinate space; a column is data, so it scales, facets and bends with everything else.

18.2 Which axis it lands on

Nothing in that sentence said “horizontal”. The mark decided it, and this is how: `life_bands` has a `life` column and no `gdp` column, and the plot’s `y` is `life`. One axis answers, the other does not, so the rule sits on the one that does and spans the one that does not.

Change which column the table holds and the same word draws the line vertically:

Table 18.1: `gdp_threshold`: all 1 rows

gdp
10000

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  data(gdp_threshold) + rule + style(color = "firebrick", pattern = "dashed")  
↪ +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("The same mark, the other axis")
```

Python

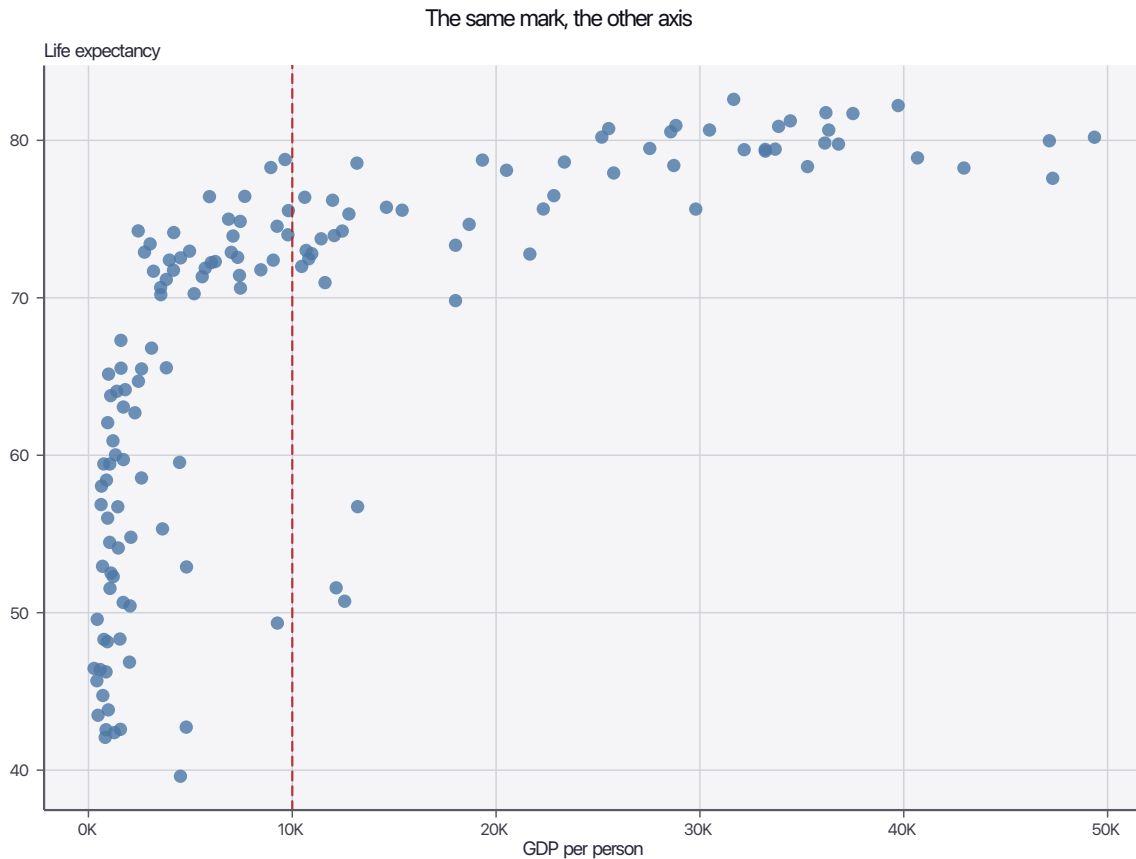
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  data(gdp_threshold) + rule + style(color = "firebrick", pattern = "dashed")  
↪ +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("The same mark, the other axis"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + data(gdp_threshold) +  
  rule + style(color = "firebrick", pattern = "dashed") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("The same mark, the other axis")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
     data(gdp_threshold), rule,
     style({ color: "firebrick", pattern: "dashed" }),
     x_label("GDP per person"), y_label("Life expectancy"),
     title("The same mark, the other axis"))
```



`gdp_threshold` holds `gdp`, the plot's `x`, so the line stands up. The mark did not change, the sentence did not change, and there is no `flip` anywhere: the column the table holds is what chose the axis.

18.3 A rug is the same mark

A rug tick has no second position either: it sits at one value and reaches a little way in from the edge. That is not a different geometry, only a different distance, so it is a setting rather than a second mark:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + style(opacity = 0.45) +  
  rule + x(gdp) + style(reach = "edge") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Where the countries actually sit on the income axis")
```

Python

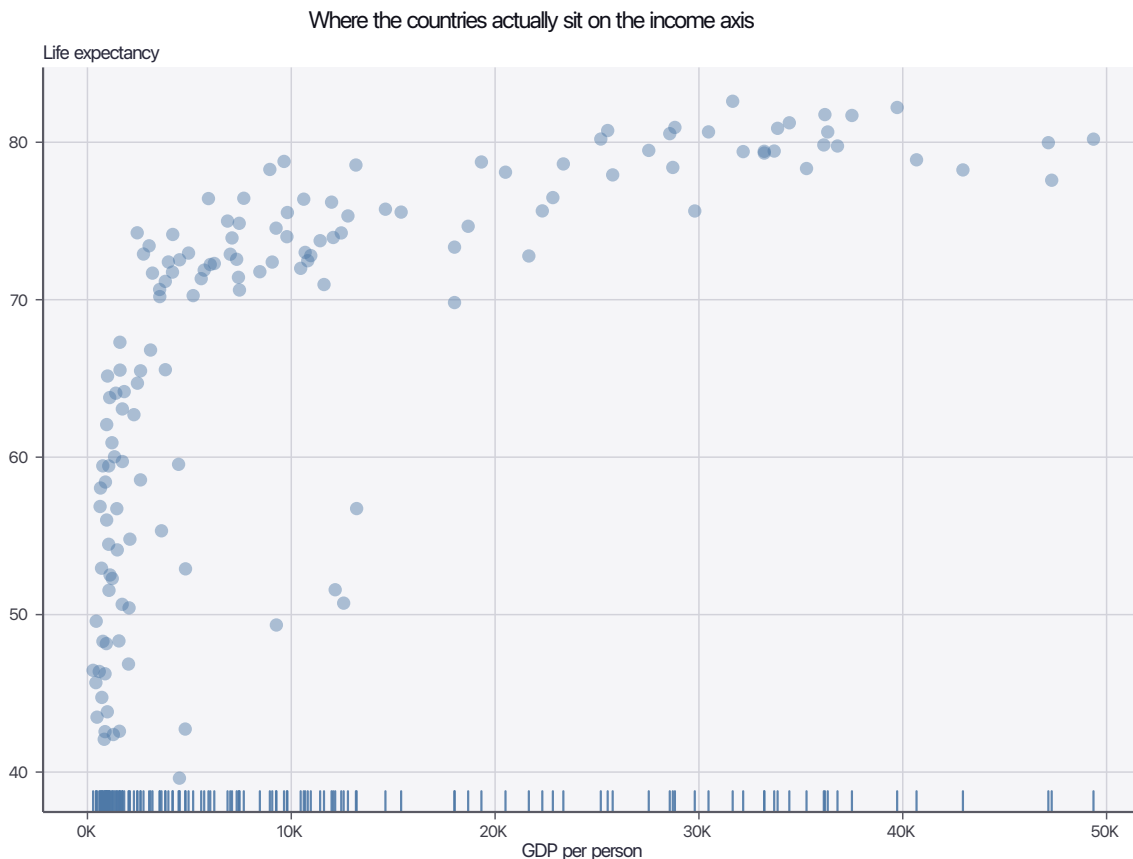
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + style(opacity =  
↪ 0.45) +  
  rule + x(col.gdp) + style(reach = "edge") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Where the countries actually sit on the income axis"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  style(opacity = 0.45) + rule + x(:gdp) + style(reach = "edge") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Where the countries actually sit on the income axis")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  style({ opacity: 0.45 }), rule, x(col.gdp), style({ reach: "edge" } ),  
  x_label("GDP per person"), y_label("Life expectancy"),  
  title("Where the countries actually sit on the income axis"))
```



“Given gapminder 2007: points, x is gdp, y is life, and also rules, x is gdp, with reach edge.”

The rule names `x(gdp)` even though the points already named it. This layer has no table of its own, so it reads the scatter’s, which holds both `gdp` and `life`. Both axes answer, so the rule has to say which one it sits on. `life_bands` above held only one, which is why those rules named nothing.

Look along the bottom. Each tick is one country, drawn at its own income. Together they answer a question the points above cannot. How many countries sit at each income, counting the ones hidden under other points? Points overlap, so the 66 countries below \$5,000 look like far fewer dots than that. The ticks share only one coordinate instead of two, so they overlap less, and the crowding at the left stays visible.

That is what a rug is for in general. It puts the one-dimensional distribution of a variable in the margin of a plot that is already showing something else. You can read the spread of `gdp` without losing the scatter, and without a second plot whose axis you would have to match.

Everything else about the mark is unchanged. The only new setting is `style(reach = "edge")`, and there is no distance to set. A tick’s length comes from the panel, the way a label’s nudge

distance comes from the font size. A number there would be a pixel count that meant something different in every facet and at every plot size.

18.3.1 The other axis

A rug is not tied to the horizontal. Which axis it lands on is decided by the position you name on the layer. Write `y(life)` instead of `x(gdp)` and the ticks run up the left edge. Write both, as two layers, and the scatter gets a rug along the bottom and another up the left at once:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + style(opacity = 0.45) +  
  rule + x(gdp) + style(reach = "edge") +  
  rule + y(life) + style(reach = "edge") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Both margins, from two layers")
```

Python

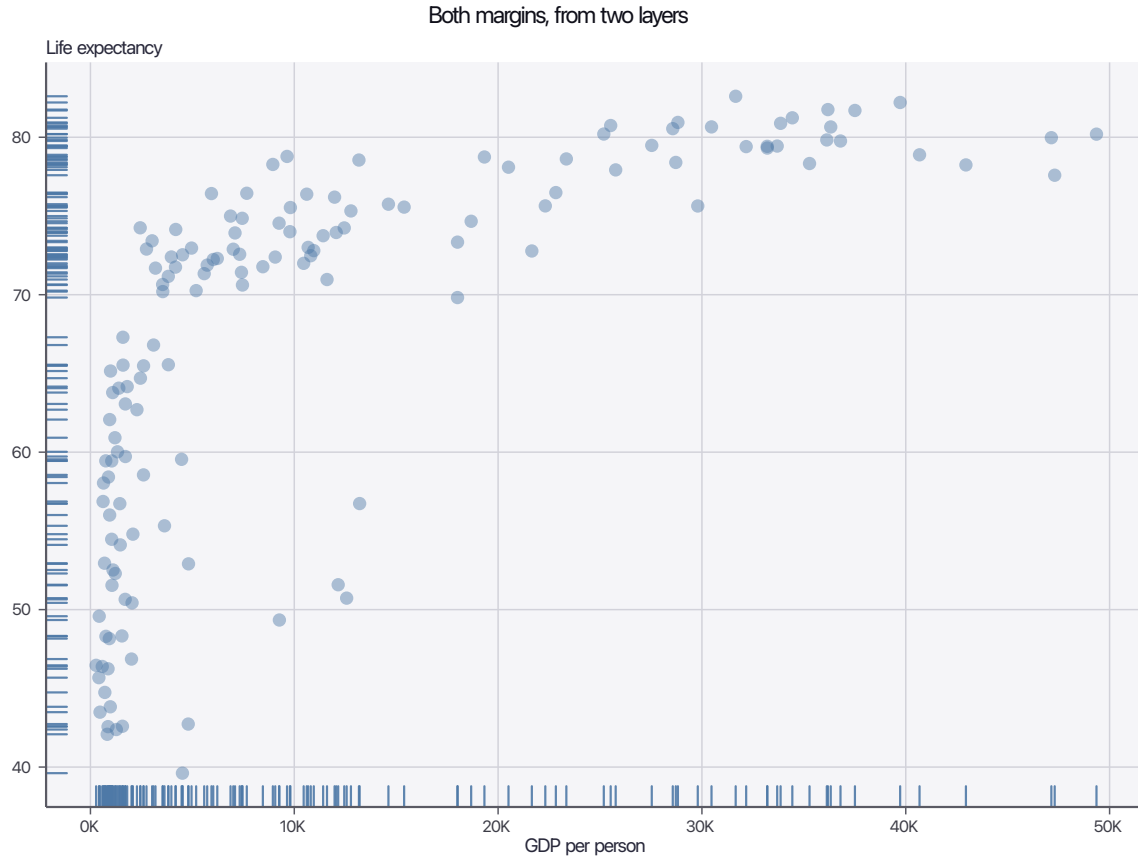
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + style(opacity =  
↪ 0.45) +  
  rule + x(col.gdp) + style(reach = "edge") +  
  rule + y(col.life) + style(reach = "edge") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Both margins, from two layers"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  style(opacity = 0.45) + rule + x(:gdp) + style(reach = "edge") + rule +  
  y(:life) + style(reach = "edge") + x_label("GDP per person") +  
  y_label("Life expectancy") + title("Both margins, from two layers")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  style({ opacity: 0.45 }), rule, x(col.gdp), style({ reach: "edge" }),  
  rule, y(col.life), style({ reach: "edge" }), x_label("GDP per person"),  
  y_label("Life expectancy"), title("Both margins, from two layers"))
```



Two layers rather than one, because each rug names one axis, and both read the scatter's table.

Read the left edge and the distribution looks different. The ticks crowd together above 70, thin out between 60 and 70, then run down to 40 in a looser second band. That thin band in the middle is what to notice: of the 142 countries, 83 sit above 70 and 43 below 60, with only 16 between. It is a distribution with two groups and a gap, not one spread, and neither the scatter nor the bottom rug shows it.

18.4 Coloring the lines

Each row of a rule's table is its own segment, so `color` maps per row. The `band` column in `life_bands`, which names each threshold, is for this:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + style(color = "lightgray")
↪ +
  data(life_bands) + rule + color(band) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("One line per band, keyed by the legend")
```

Python

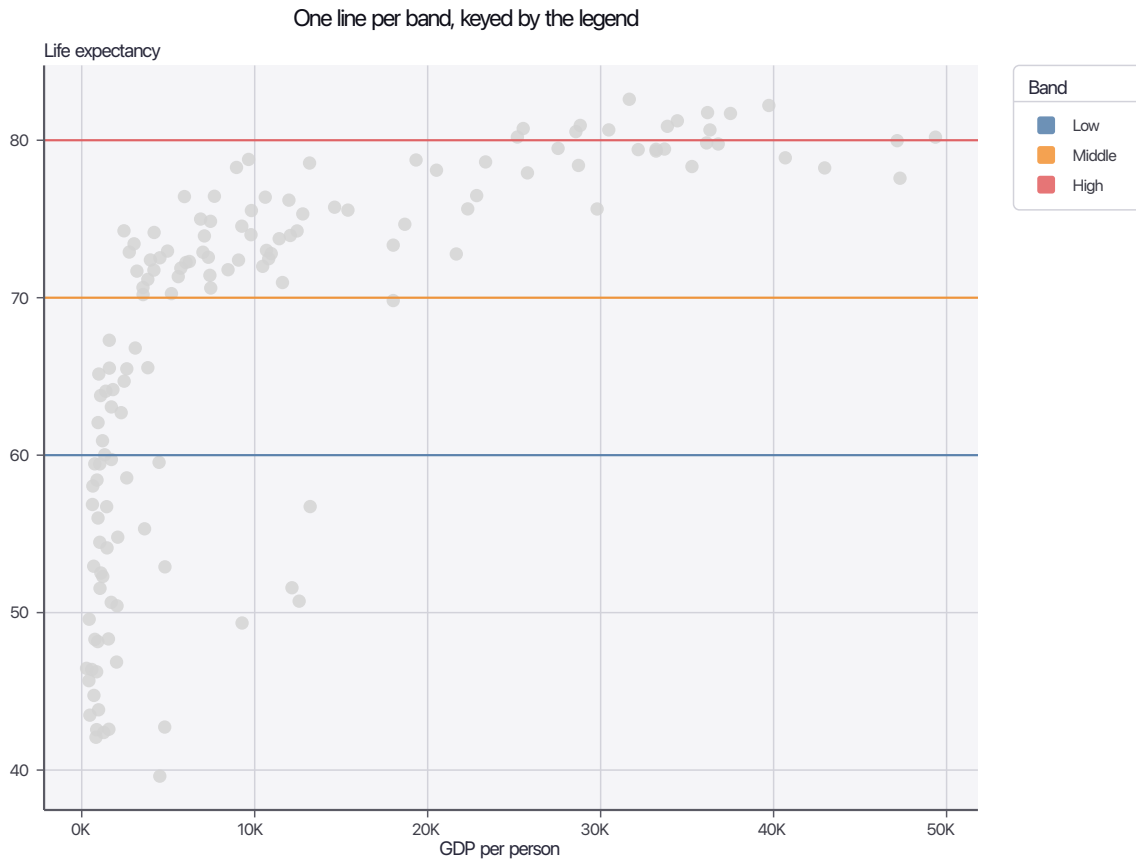
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + style(color =
↪ "lightgray") +
  data(life_bands) + rule + color(col.band) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("One line per band, keyed by the legend"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  style(color = "lightgray") + data(life_bands) + rule + color(:band) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("One line per band, keyed by the legend")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  style({ color: "lightgray" }), data(life_bands), rule, color(col.band),
  x_label("GDP per person"), y_label("Life expectancy"),
  title("One line per band, keyed by the legend"))
```



“Given gapminder 2007: points, x is gdp, y is life, and also rules from the life bands, color by band.”

A thin line gives a reader no area to compare shades in, so `color` here takes categories, not a continuous ramp. `line` and `path` answer the same way.

18.5 Thickening a rule

Width is the rule’s other visible setting. `style(size = )` sets it, and a wide, faint rule reads as a highlighted band:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + style(opacity = 0.55) +
  data(life_bands) + rule + style(color = "seagreen", size = 26, opacity =
    ↪ 0.20) +
```

```
x_label("GDP per person") + y_label("Life expectancy") +
title("A thick rule: emphasis, measured in pixels")
```

Python

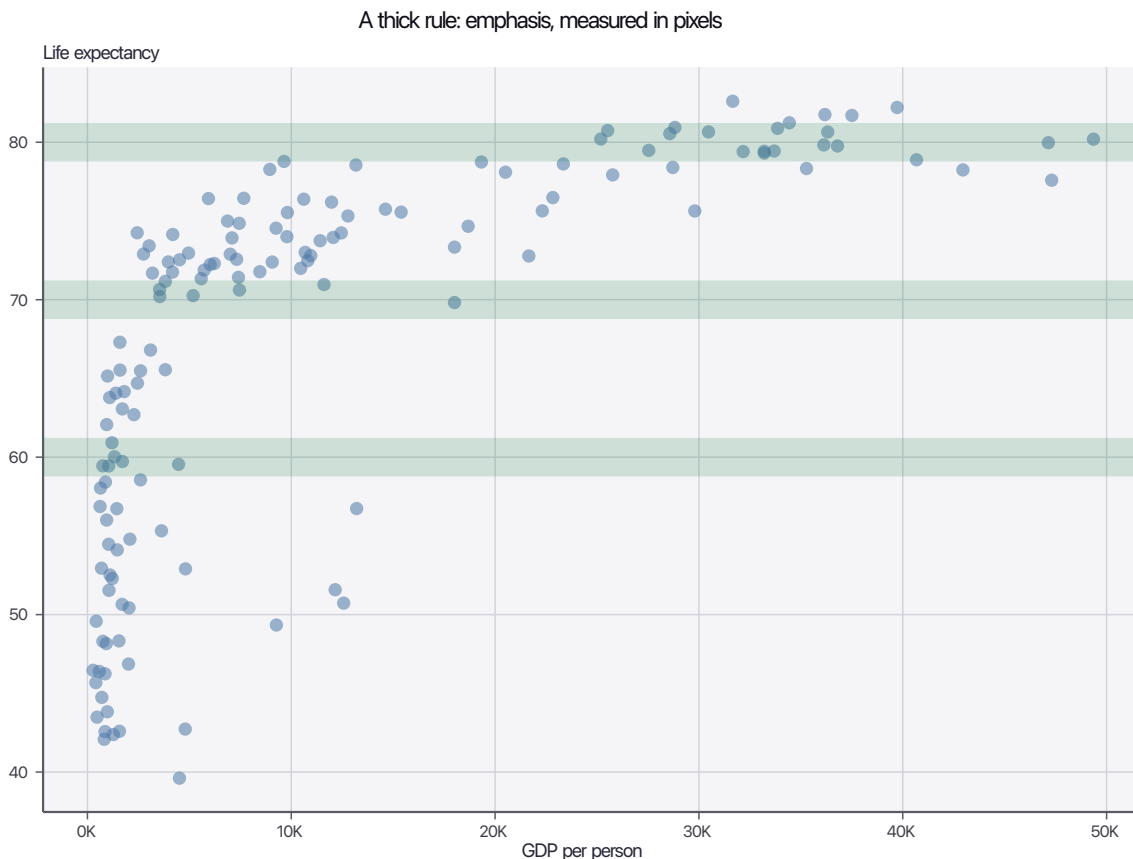
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + style(opacity =
↪ 0.55) +
  data(life_bands) + rule + style(color = "seagreen", size = 26, opacity =
↪ 0.20) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("A thick rule: emphasis, measured in pixels"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  style(opacity = 0.55) + data(life_bands) + rule +
  style(color = "seagreen", size = 26, opacity = 0.2) +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("A thick rule: emphasis, measured in pixels")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  style({ opacity: 0.55 }), data(life_bands), rule,
  style({ color: "seagreen", size: 26, opacity: 0.2 }),
  x_label("GDP per person"), y_label("Life expectancy"),
  title("A thick rule: emphasis, measured in pixels"))
```



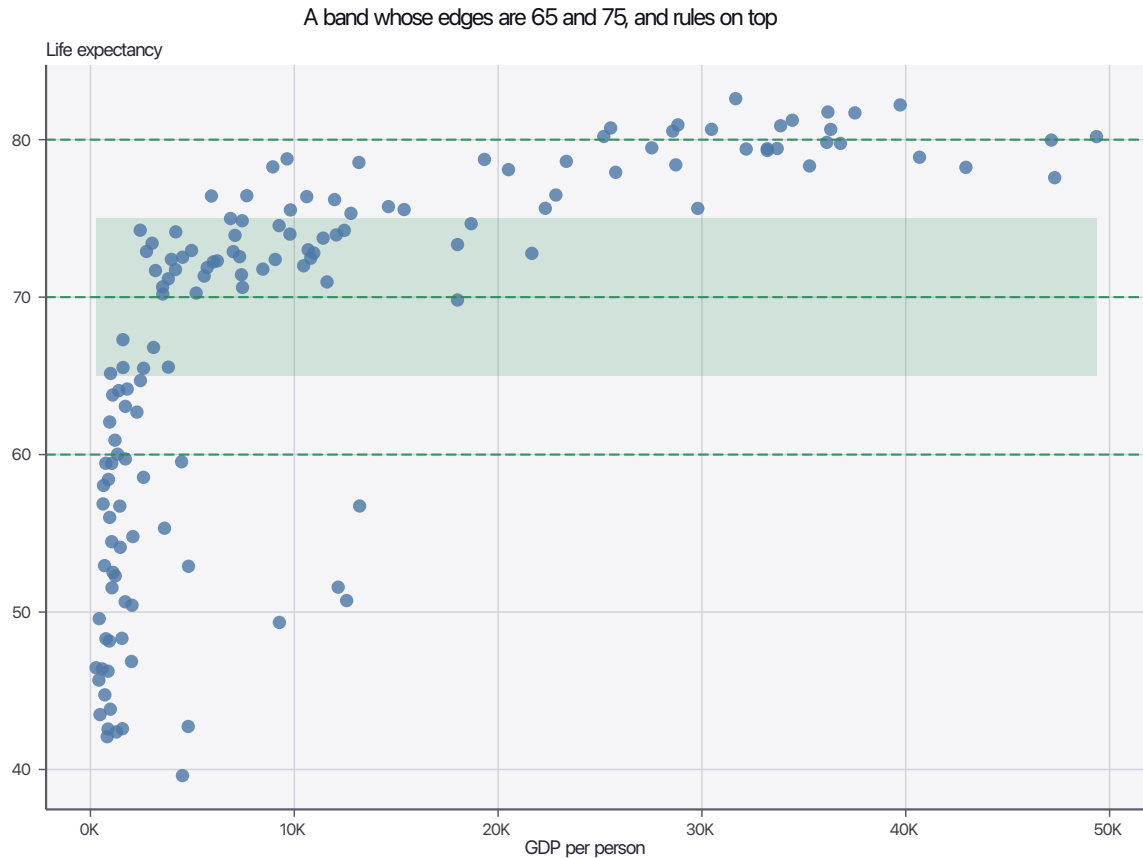
“Given gapminder 2007: points, x is gdp, y is life, and also rules from the life bands, colored seagreen, with size 26 and opacity 0.20.”

That is a good way to mark a value, as long as you remember what the width is: pixels, not data. The band above covers about two and a half years of life expectancy. It covers that much only because this panel holds the range from 40 to 80. Facet it, resize it, or change the data’s range and the same `size = 26` will cover a different number of years while looking identical. Nothing is wrong with that as long as the width is decoration; it is wrong the moment the width is supposed to *mean* something.

When it should mean something (“the healthy range is 65 to 75”, not “look here”), the thickness belongs to the data, and then you want a region with real edges. That is `ribbon`, given a low column and a high column you computed. The table has two rows, one at each end of the income axis, so the band reaches from side to side:

```
healthy <- data.frame(gdp = base::range(gapminder_2007$gdp), lo = 65.0, hi =
  ↪ 75.0)
```

```
data(healthy) + ribbon * bounds(lo, hi) + style(color = "seagreen", opacity =
↪ 0.18) +
  data(gapminder_2007) + point + x(gdp) + y(life) +
  data(life_bands) + rule + style(color = "seagreen", pattern = "dashed") +
  x_label("GDP per person") + y_label("Life expectancy") +
  title("A band whose edges are 65 and 75, and rules on top")
```



“Given the healthy table: a ribbon from lo to hi; then given gapminder 2007: points, x is gdp, y is life, and also rules from the life bands.”

Now 65 and 75 are positions. They rescale with the axis and keep the same values in every facet. On a log axis they land where the data puts them, not at a fixed number of pixels. The highlight is written *first* so the points draw over it.

Give that same table two different **gdp** values instead of the data's range, and the band becomes a box bounded on both axes. That is how you shade a corner of a scatter, or one span of a time series.

Neither the band nor the box stretches to the edge of the panel. A ribbon is bounded by its data, so it stops exactly at the numbers you gave it, and you cannot pad them outward without widening the scale itself. For a rectangle that takes one side from the data and the other from the panel, use **zone**. It is **rule** with one more dimension.

18.6 In polar: the ring and the spoke

The mark does one thing: it spans the whole of the axis it does not name. In polar that is unchanged, and it draws two shapes rather than one, because a circle has two axes to span. **Polar** builds the circle itself; what matters here is that a rule spans whichever axis it does not name.

winds is the table [The book's data](#) introduced, and **speed_target** holds the one target speed:

Table 18.2: **speed_target**: all 1 rows

speed
15

The angular axis runs all the way around the circle, and that whole sweep is the **turn**. Span the turn whole and you get a ring. Here it marks a target wind speed on a circular wind plot, and **speed** is the radial axis:

R

```
data(winds) + line * mean + x(direction) + y(speed) + polar() +
  data(speed_target) + rule + style(color = "firebrick", pattern = "dashed")
↪ +
  title("A target speed, ringed")
```

Python

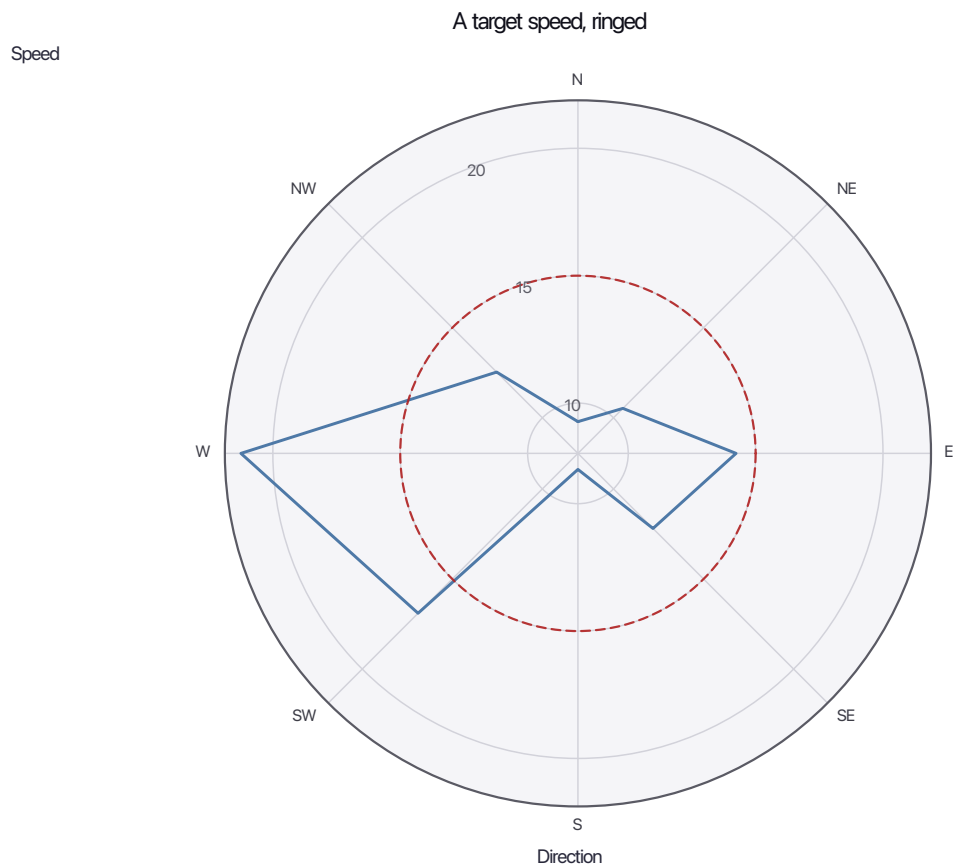
```
(data(winds) + line * mean + x(col.direction) + y(col.speed) + polar() +
  data(speed_target) + rule + style(color = "firebrick", pattern = "dashed")
↪ +
  title("A target speed, ringed"))
```


Julia

```
data(winds) + line * mean + x(:direction) + y(:speed) + polar() +  
data(speed_target) + rule +  
style(color = "firebrick", pattern = "dashed") +  
title("A target speed, ringed")
```

JavaScript

```
plot(data(winds), layer(line, mean), x(col.direction), y(col.speed),  
polar(), data(speed_target), rule,  
style({ color: "firebrick", pattern: "dashed" }),  
title("A target speed, ringed"))
```



“Given the winds: a line derived by mean, x is direction, y is speed, in polar, and also rules from the speed target.”

Span the *radius* whole and you get a spoke, a line from the center to the rim. Give the rule a table holding **direction** instead of **speed**, and the line stands at that compass point. Neither shape is a special case in the mark. Spanning the axis a rule does not name is what it always did, and in a circle the radial axis runs from the center outward while the angular axis runs all the way around.

That is what a mark must show before polar counts as the same mark: stated once, read off the bindings, meaning the same thing in every space. A rule that needed a special case in polar would not be one mark. It would be three: a flat line, a ring, and a spoke.

18.7 What you can set

Everything a rule can be told to look like, generated from the engine's own legality table, so this page cannot differ from what `style()` accepts:

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(pattern =)</code>	solid, dashed, dotted
<code>style(reach =)</code>	panel, edge

And these vary per row if you map them to a column instead: `color()` (categories), `pattern()` (categories), `play()` (either).

Four of those are a stroke's ordinary controls, and one is the mark's own. `style(reach = )` decides how far the rule crosses the axis it does not name, and it is what makes the rug the same mark rather than a second one. The other four behave here as on `line` or `path`:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + style(color = "lightgray")
↪ +
  data(data.frame(life = 50.0)) + rule + style(color = "firebrick", size = 4)
↪ +
  data(data.frame(life = 60.0)) + rule + style(color = "seagreen", pattern =
↪ "dashed") +
  data(data.frame(life = 70.0)) + rule +
    style(color = "steelblue", pattern = "dotted", size = 2.5) +
  data(data.frame(life = 80.0)) + rule +
    style(color = "black", size = 6, opacity = 0.25) +
```

```
x_label("GDP per person") + y_label("Life expectancy") +
title("Bottom to top: size, dashed, dotted, a faded thick rule")
```

Python

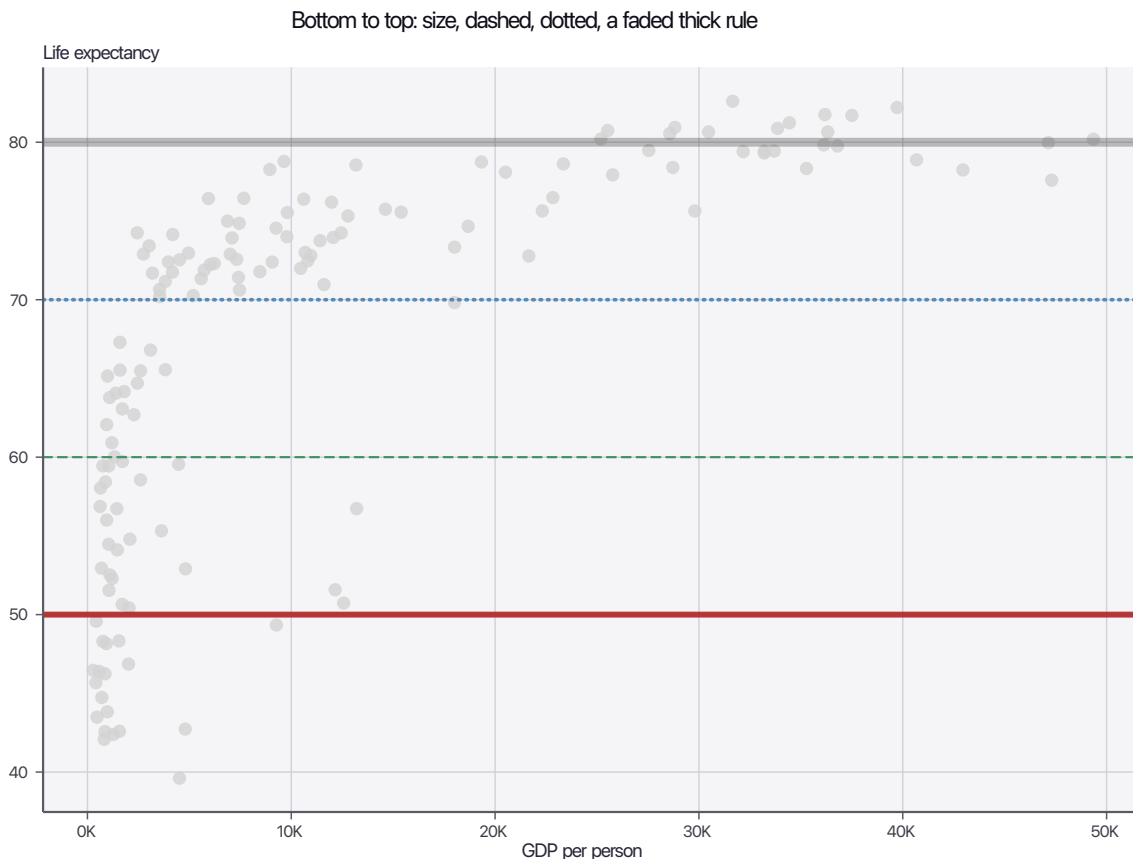
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + style(color =
↪ "lightgray") +
  data({"life": [50.0]}, name='data.frame(life = 50.0)') + rule + style(color
↪ = "firebrick", size = 4) +
  data({"life": [60.0]}, name='data.frame(life = 60.0)') + rule + style(color
↪ = "seagreen", pattern = "dashed") +
  data({"life": [70.0]}, name='data.frame(life = 70.0)') + rule +
  style(color = "steelblue", pattern = "dotted", size = 2.5) +
  data({"life": [80.0]}, name='data.frame(life = 80.0)') + rule +
  style(color = "black", size = 6, opacity = 0.25) +
x_label("GDP per person") + y_label("Life expectancy") +
title("Bottom to top: size, dashed, dotted, a faded thick rule"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  style(color = "lightgray") + data((life = [50],)) + rule +
  style(color = "firebrick", size = 4) + data((life = [60],)) + rule +
  style(color = "seagreen", pattern = "dashed") + data((life = [70],)) +
  rule + style(color = "steelblue", pattern = "dotted", size = 2.5) +
  data((life = [80],)) + rule +
  style(color = "black", size = 6, opacity = 0.25) +
x_label("GDP per person") + y_label("Life expectancy") +
title("Bottom to top: size, dashed, dotted, a faded thick rule")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  style({ color: "lightgray" }), data({ life: [50] }), rule,
  style({ color: "firebrick", size: 4 }), data({ life: [60] }), rule,
  style({ color: "seagreen", pattern: "dashed" }), data({ life: [70] }),
  rule, style({ color: "steelblue", pattern: "dotted", size: 2.5 }),
  data({ life: [80] }), rule,
  style({ color: "black", size: 6, opacity: 0.25 }),
x_label("GDP per person"), y_label("Life expectancy"),
title("Bottom to top: size, dashed, dotted, a faded thick rule"))
```



“Given gapminder 2007: points, x is gdp, y is life, colored lightgray; then given one row with life 50: a rule colored firebrick, with size 4; then given one row with life 60: a rule colored seagreen, with pattern dashed; then given one row with life 70: a rule colored steelblue, with pattern dotted and size 2.5; then given one row with life 80: a rule colored black, with size 6 and opacity 0.25.”

The **pattern** values are the three **dashes**, because a rule is a stroke. `style(pattern = "hatch")` is refused, pointing at the dashes: a hatch is a *fill*'s texture, and it is what **zone** takes instead. One setting name, one realization per geometry.

What a rule refuses follows from the same geometry. There is no **shape**, because a stroke has no glyph to choose. And there is no border:

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  data(life_bands) + rule + style(border_color = "black")
```

Error:

! gog: a `rule` is drawn with a stroke, not a filled shape - it has no
↪ separate border. `style(color = )` sets its color and `style(size = )`
↪ its width.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A border belongs to a filled shape, and a rule is the outline already, so `style(color = )` is its color and `style(size = )` its width. That is the same answer line, step and path give, because it is the same geometry.

18.8 What it refuses

Give a rule the plot's own table and both position columns answer at once. There is then nothing to say which axis is meant, so gog refuses rather than picking one:

```
data(gapminder_2007) + point + x(gdp) + y(life) + rule
```

Error:

! gog: `rule` marks a value on one axis and spans the other, but this table
↪ has a column for both - `x(gdp)` and `y(life)` - so there is nothing to
↪ say which axis is meant. Say it on the layer: `rule + x(gdp)` places
↪ every line by `gdp`, `rule + y(life)` by `life`. Giving the rule its own
↪ table holding just the one column works too.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

That refusal is why the rugs above name their axis, `rule + x(gdp)` and `rule + y(life)`. A threshold with a table of its own needs no position. That table holds only one of the two columns, so the axis is read off it. A bar with categories on both axes is refused for a related reason. Neither axis measures anything, so there is nothing to read the orientation from, and gog will not guess.

A rule takes no transform either. It has handed one whole axis to the panel, so there is no measure for a statistic to compute and nowhere to put the answer:

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  data(life_bands) + rule * mean
```

Error:

! gog: `rule` is placed by one column and spans the other axis, so it has no
↪ measure for `mean` to compute and nowhere to put the answer. Compute the
↪ value where your data lives and give the rule a table of the results -
↪ one row per line - which is what a rule's position always is: a column.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

The message points back at what a rule is. A line at the mean is a mean you computed yourself, passed in as a column. A rule's position always comes from a column.

19 Zone

Which region of the plot should the values stay inside? **zone** shades a rectangle. It is **rule**'s sibling one dimension up, and it works the same way. A rule takes one *position* and spans the axis it was not given. A zone takes a *pair* and spans the axis it was given no pair for. Give it pairs on both axes and it is a rectangle you placed.

That “spans the axis it was not given” is the whole reason the mark exists, because everything else about a rectangle can already be drawn. What could not be drawn is a rectangle that reaches the **panel**.

19.1 A band across the panel

The **quarterly** table holds one sales figure for each year, and **target_band** holds the two edges of a target band:

Table 19.1: **quarterly**: first 5 of 19 rows

year	sales
2005	95
2006	104
2007	118
2008	112
2009	88

Table 19.2: **target_band**: all 1 rows

lower	upper
150	175

Here are the sales with the target band behind them, 150 to 175, whatever the years happen to be:

R

```
data(quarterly) + x(year) + y(sales) +
  data(target_band) + zone * bounds(lower, upper) + style(color = "seagreen")
↪ +
data(quarterly) + line + point +
y_label("Sales") + title("A target band, 150 to 175")
```

Python

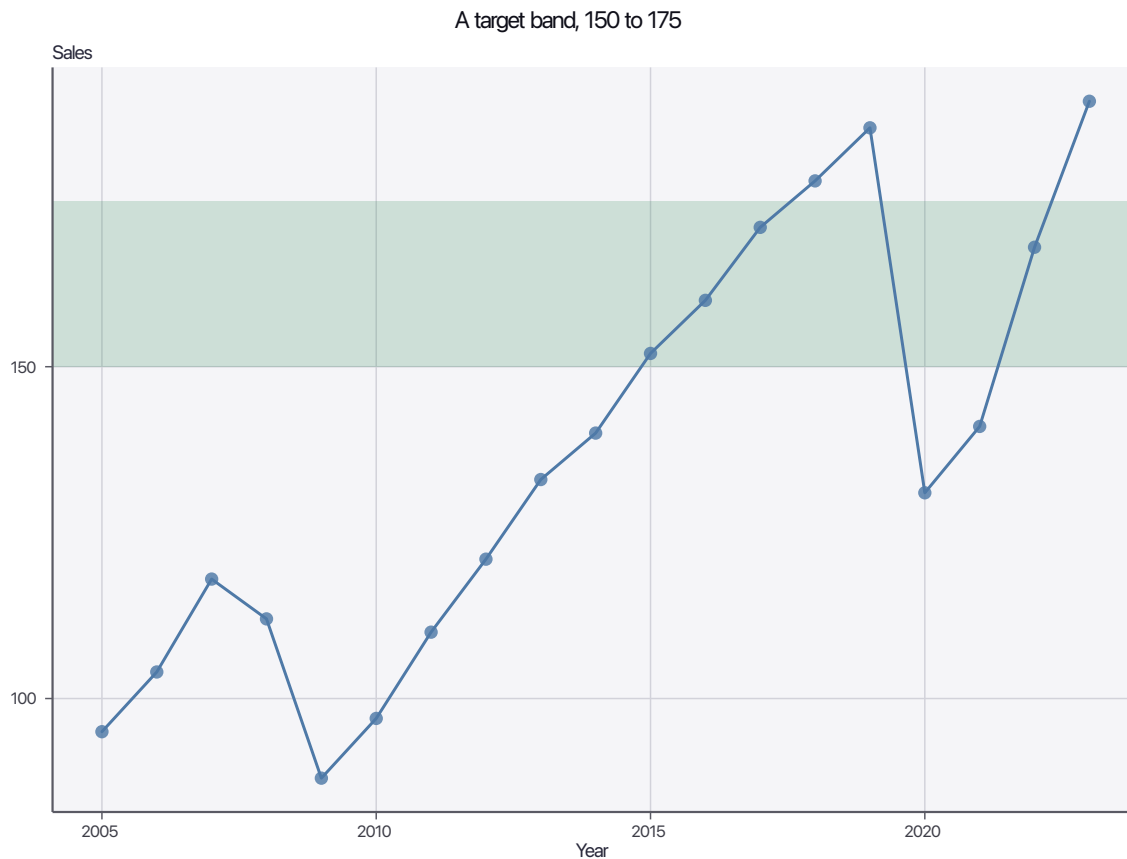
```
(data(quarterly) + x(col.year) + y(col.sales) +
  data(target_band) + zone * bounds(col.lower, col.upper) + style(color =
↪ "seagreen") +
  data(quarterly) + line + point +
  y_label("Sales") + title("A target band, 150 to 175"))
```

Julia

```
data(quarterly) + x(:year) + y(:sales) + data(target_band) +
  zone * bounds(:lower, :upper) + style(color = "seagreen") +
  data(quarterly) + line + point + y_label("Sales") +
  title("A target band, 150 to 175")
```

JavaScript

```
plot(data(quarterly), x(col.year), y(col.sales), data(target_band),
  layer(zone, bounds(col.lower, col.upper)), style({ color: "seagreen" }),
  data(quarterly), line, point, y_label("Sales"),
  title("A target band, 150 to 175"))
```

“Given quarterly: x is year, y is sales, a target zone from lower to upper, and also a line and points.”

`bounds(lower, upper)` names the two columns holding the bottom and top. Nothing names a left or a right, so the zone takes those from the panel and runs edge to edge. The zone is written *before* the line so the data draws on top of it; a highlight belongs behind what it highlights.

19.2 A band up the panel

Swap which pair you give it and the rectangle stands up instead. The `recessions` table holds the two slumps, each with a start year and an end year:

Table 19.3: recessions: all 2 rows

start	end	slump
2007.9	2009.5	Financial crisis
2020.0	2020.9	Pandemic

R

```
data(quarterly) + x(year) + y(sales) +
  data(recessions) + zone * bounds(start = start, end = end) +
  style(color = "indianred") +
  data(quarterly) + line + point +
  y_label("Sales") + title("Two slumps, from one table")
```

Python

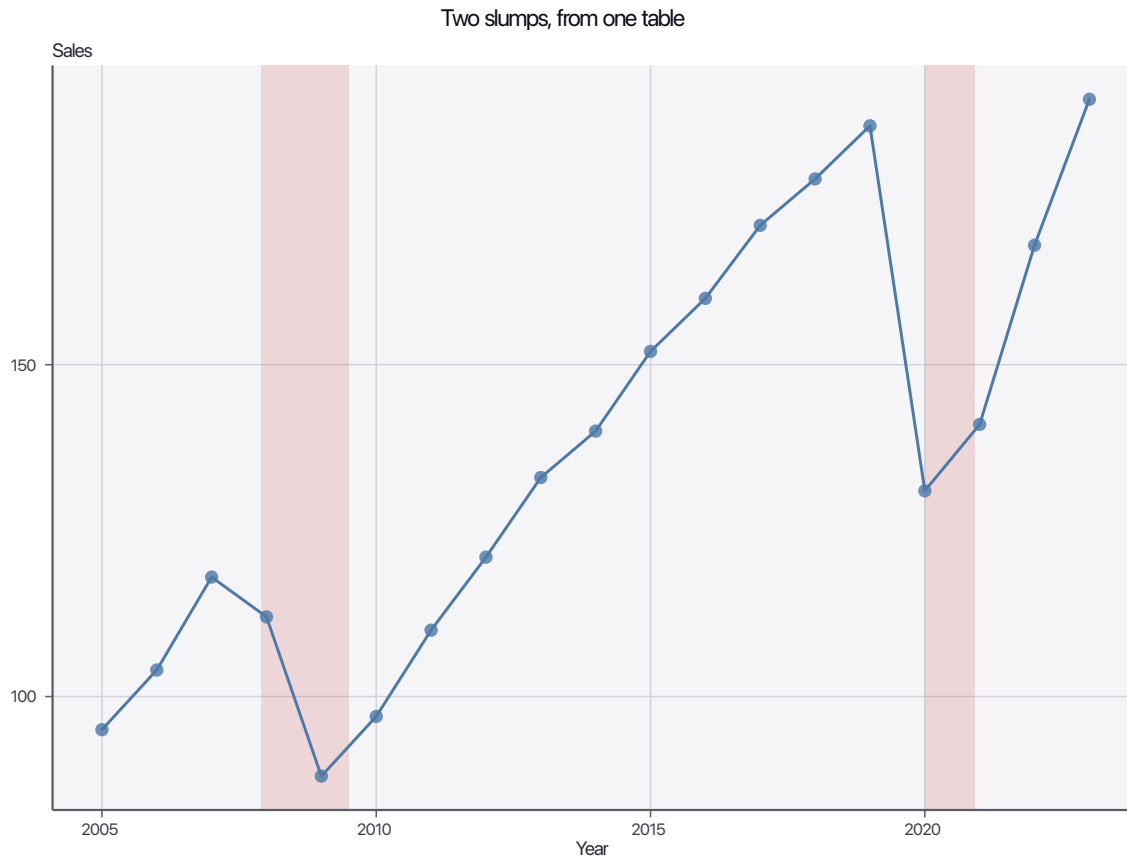
```
(data(quarterly) + x(col.year) + y(col.sales) +
  data(recessions) + zone * bounds(start=col.start, end=col.end) +
  style(color = "indianred") +
  data(quarterly) + line + point +
  y_label("Sales") + title("Two slumps, from one table"))
```

Julia

```
data(quarterly) + x(:year) + y(:sales) + data(recessions) +
  zone * bounds(start = :start, var"end" = :end) +
  style(color = "indianred") + data(quarterly) + line + point +
  y_label("Sales") + title("Two slumps, from one table")
```

JavaScript

```
plot(data(quarterly), x(col.year), y(col.sales), data(recessions),
  layer(zone, bounds({ start: col.start, end: col.end })),
  style({ color: "indianred" }), data(quarterly), line, point,
  y_label("Sales"), title("Two slumps, from one table"))
```



“Given quarterly: x is year, y is sales, recession zones from start to end, and also a line and points.”

Two shaded spans, and only one **zone** in the sentence. **One row is one rectangle**, so a two-row table draws two bands, the same property that lets one **rule** table draw three thresholds. Add a row and a third band appears with no change to the plot.

Look at where the red reaches: the top and bottom of the panel exactly. That is the thing a **ribbon** cannot do. A ribbon is bounded by its data, so it stops at whatever numbers you hand it, and any number big enough to reach the top would *widen the axis to include it*, changing the plot in order to decorate it.

The pair is called **start/end** rather than **left/right** on purpose. Left and right are facts about a screen. This grammar reads orientation off the bindings, so the same words keep working when the category axis becomes an angle.

19.3 Both pairs: a rectangle

Name all four and neither axis comes from the panel:

R

```
corner <- data.frame(start = 2015.0, end = 2019.0, lower = 145.0, upper =  
  ↪ 190.0)  
  
data(quarterly) + x(year) + y(sales) +  
  data(corner) + zone * bounds(lower, upper, start = start, end = end) +  
    style(color = "goldenrod", opacity = 0.3) +  
  data(quarterly) + line + point +  
  y_label("Sales") + title("A box over four years of sales")
```

Python

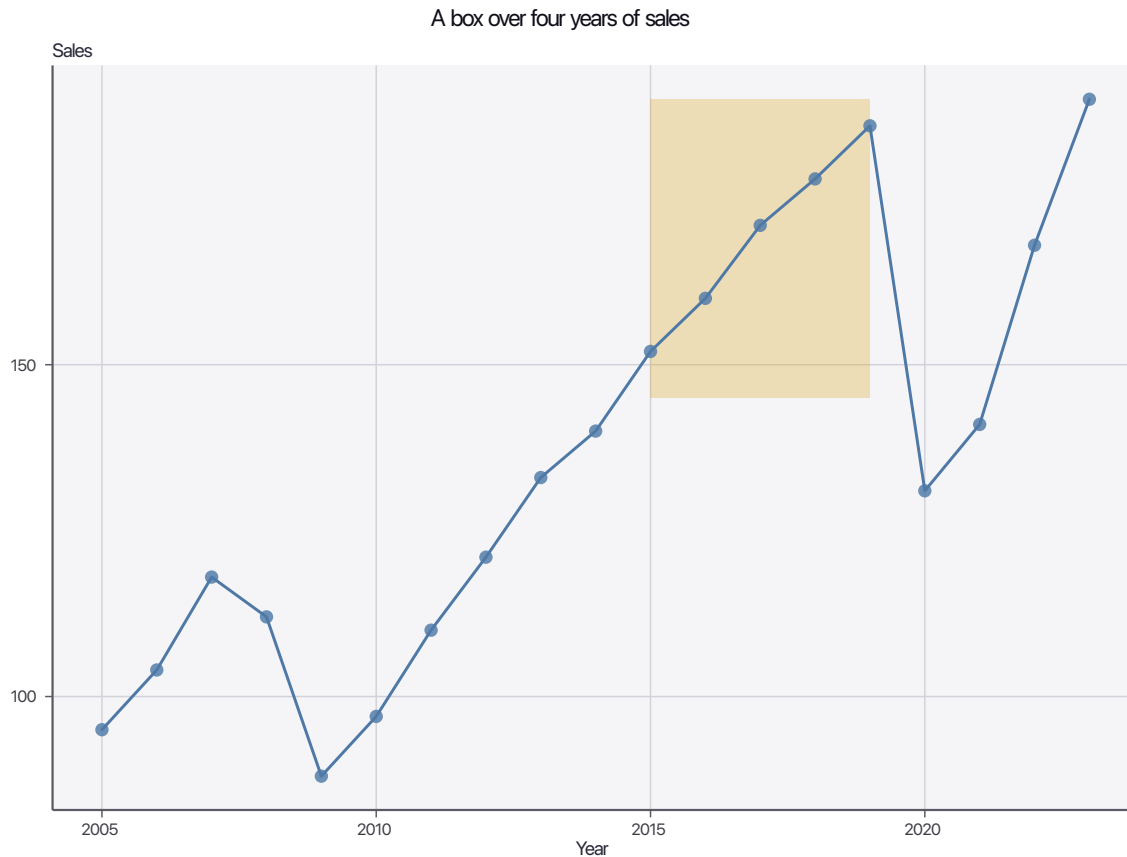
```
corner = {"start": [2015.0], "end": [2019.0], "lower": [145.0], "upper":  
  ↪ [190.0]}  
(data(quarterly) + x(col.year) + y(col.sales) +  
  data(corner) + zone * bounds(col.lower, col.upper, start=col.start,  
  ↪ end=col.end) +  
  style(color = "goldenrod", opacity = 0.3) +  
  data(quarterly) + line + point +  
  y_label("Sales") + title("A box over four years of sales"))
```

Julia

```
corner = (start = [2015], end = [2019], lower = [145], upper = [190],)  
data(quarterly) + x(:year) + y(:sales) + data(corner) +  
  zone * bounds(:lower, :upper, start = :start, var"end" = :end) +  
  style(color = "goldenrod", opacity = 0.3) + data(quarterly) + line +  
  point + y_label("Sales") + title("A box over four years of sales")
```

JavaScript

```
const corner = { start: [2015], end: [2019], lower: [145], upper: [190] };  
plot(data(quarterly), x(col.year), y(col.sales), data(corner),  
  layer(zone,  
    bounds(col.lower, col.upper, { start: col.start, end: col.end })),  
  style({ color: "goldenrod", opacity: 0.3 }), data(quarterly), line,  
  point, y_label("Sales"), title("A box over four years of sales"))
```



“Given quarterly: x is year, y is sales, a corner zone from lower to upper and from start to end, and also a line and points.”

So one atom covers the three shapes people actually want (a horizontal band, a vertical band, and a box), and which one you get is decided by which pairs you name. There is no `hband`, no `vband`, no `rect`.

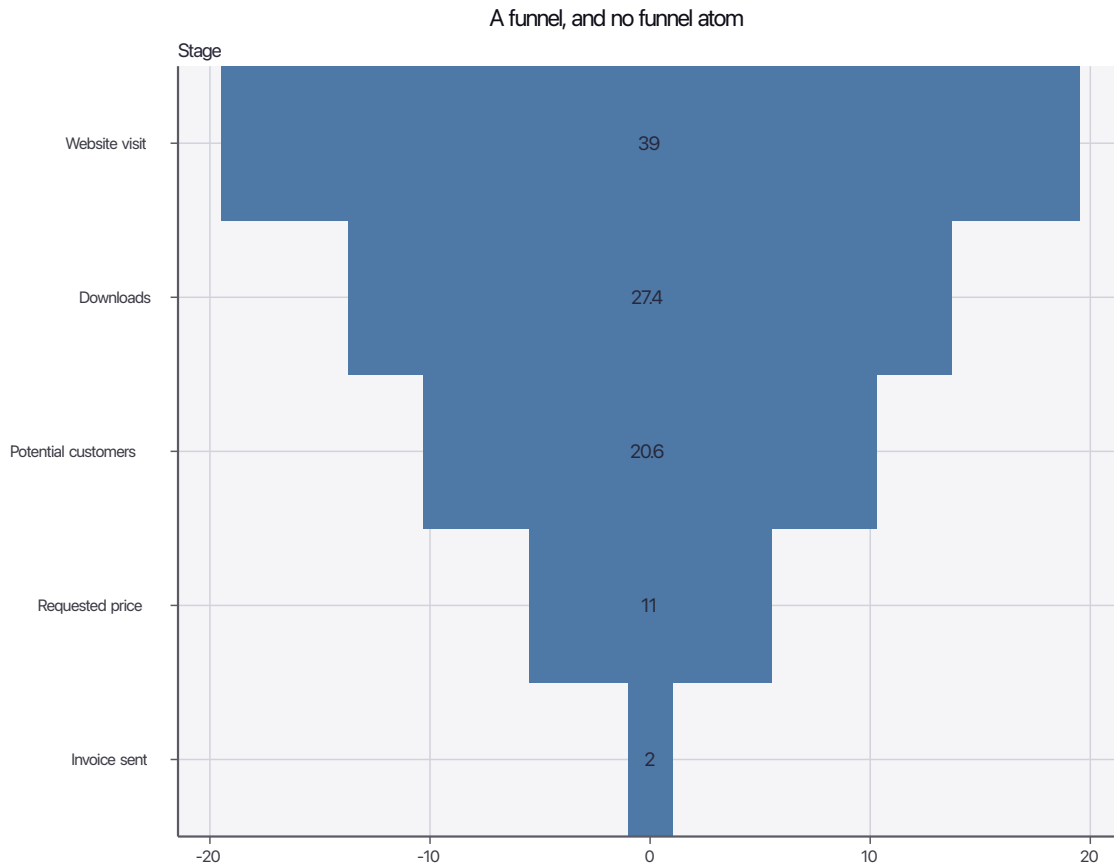
19.4 Which axis the pair bounds

`lower` and `upper` bound the *measure* axis. Every plot so far has measured along y , so every `lower/upper` band so far has run across the panel. Now put a category on y instead. The measure moves to x , and the same two columns draw the plot on its side:

```
stage <- c("Website visit", "Downloads", "Potential customers",
           "Requested price", "Invoice sent")
```

```
n      <- c(39, 27.4, 20.6, 11, 2)
funnel <- data.frame(stage = factor(stage, levels = stage),
                     n = n, lo = -n / 2, hi = n / 2, mid = 0)

data(funnel) + zone * bounds(lo, hi) + y(stage) + style(opacity = 1) +
  text + label(n) + x(mid) +
  x_label("") + y_label("Stage") + title("A funnel, and no funnel atom")
```



“Given the funnel: zones from lo to hi, y is stage, and also text, x is mid, label by n.”

That is a funnel chart. No atom in the sentence is a funnel.

The blocks are the **zone**, one row each. The columns **lo** and **hi** hold $-n/2$ and $n/2$, computed in R before the plot. The waterfall below does the same, computing its running total in the language you write the plot in. The numbers inside the blocks are a second layer, drawn by **text**.

A funnel is a bar chart whose blocks are centered on zero. Centering is arithmetic on two columns. It is not a chart type.

The plot works because of how the pair is named. `lower` and `upper` describe the *measure* axis. They do not mean the bottom and the top of a screen. `gog` never asks which way the screen runs; it asks what each axis carries. Here `y` carries a category, so `y` holds slots, and the measurement can only go on `x`. This is the rule `bar` already follows: one categorical position, one measured position, and their types decide the orientation. It is why there is no `flip`.

19.5 Coloring the zones

Each row is its own rectangle, so `color` maps per row. `recessions` carries a name for each slump:

R

```
data(quarterly) + x(year) + y(sales) +  
  data(recessions) + zone * bounds(start = start, end = end) + color(slump) +  
  data(quarterly) + line + point +  
  y_label("Sales") + title("Named, and keyed by the legend")
```

Python

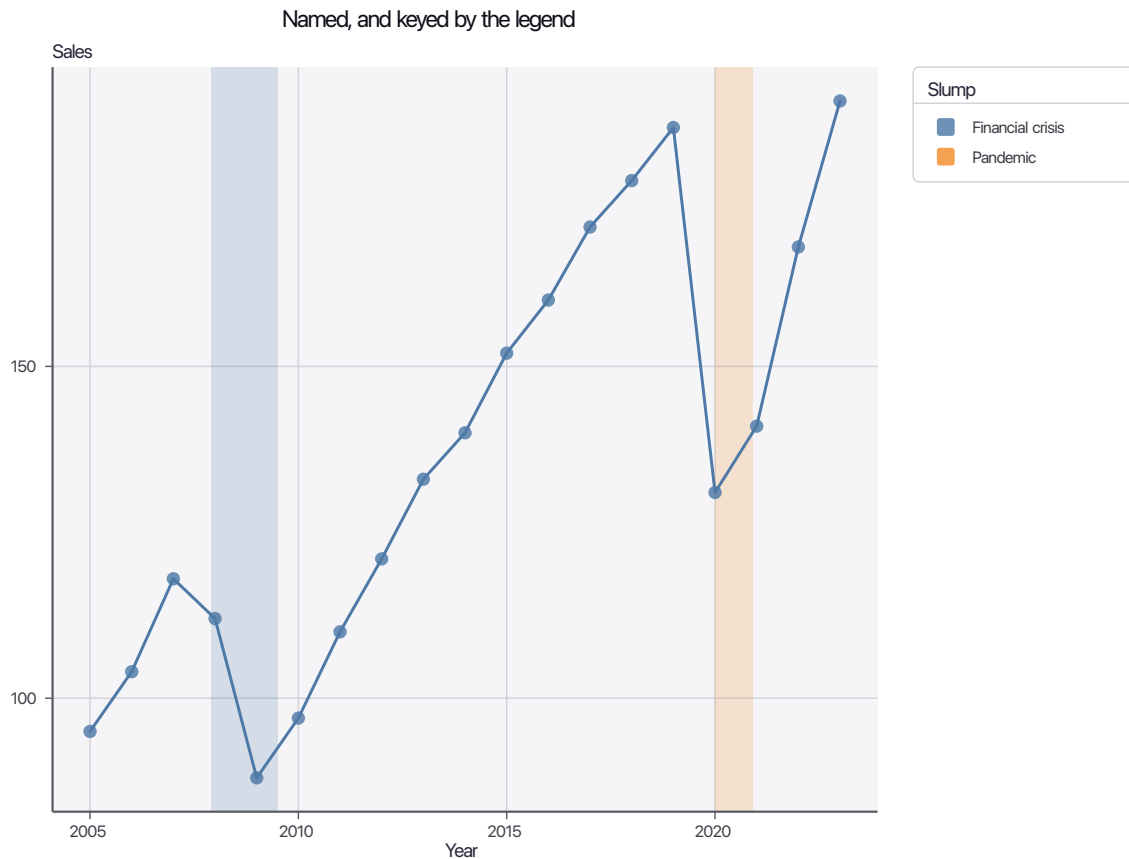
```
(data(quarterly) + x(col.year) + y(col.sales) +  
  data(recessions) + zone * bounds(start=col.start, end=col.end) +  
  ↪ color(col.slump) +  
  data(quarterly) + line + point +  
  y_label("Sales") + title("Named, and keyed by the legend"))
```

Julia

```
data(quarterly) + x(:year) + y(:sales) + data(recessions) +  
  zone * bounds(start = :start, var"end" = :end) + color(:slump) +  
  data(quarterly) + line + point + y_label("Sales") +  
  title("Named, and keyed by the legend")
```

JavaScript

```
plot(data(quarterly), x(col.year), y(col.sales), data(recessions),
     layer(zone, bounds({ start: col.start, end: col.end })),
     color(col.slump), data(quarterly), line, point, y_label("Sales"),
     title("Named, and keyed by the legend"))
```



“Given quarterly: x is year, y is sales, recession zones from start to end, color by slump, and also a line and points.”

A zone’s **color** also takes a **number**, reading the sequential ramp instead of a palette, where **bar**, **area** and **ribbon** keep to categories. A rectangle is large enough to read a ramp against. That is what a heatmap needs, so a heatmap cell is this mark rather than a new one. The heatmap section below is that plot.

19.6 Rectangles that *are* the data

Every zone so far has been a highlight: a rectangle behind a line, saying where to look. Nothing says it has to sit behind anything. Give a zone a pair on the measure axis and a categorical position. Each category's rectangle then runs from one number to another, with no plot underneath. Two common finance charts are made of exactly that.

A **waterfall** is a running total, broken into the steps that made it. `gog` has no transform for that and the omission is deliberate: **stack** accumulates *within* a position, one pile per slot, and a waterfall accumulates *across* positions. So the running total is arithmetic, and arithmetic belongs with the data, in the host language. The `cashflow` table holds one row per step of a cash statement: a **delta** for the step, and a **total** flag marking the subtotals:

Table 19.4: `cashflow`: first 5 of 6 rows

step	delta	total
Opening	120	TRUE
Sales	45	FALSE
Refunds	-18	FALSE
Costs	-32	FALSE
Tax	-14	FALSE

```
w <- cashflow
w$top <- cumsum(w$delta)           # where each step ends
w$base <- c(0, head(w$top, -1))    # and where it started: the step before
w$base[w$total] <- 0               # except the subtotals, which start at
  ↪ zero
w$dir <- ifelse(w$total, "total", ifelse(w$delta >= 0, "gain", "loss"))

slot <- seq_len(nrow(w)) - 1       # category k sits at k on a categorical
  ↪ axis
w$left <- slot - 0.35              # so a side named at k ± 0.35 is inside
  ↪ its slot
w$right <- slot + 0.35
```

That is R because it is R's job, and the other three answer it the same way: a running total belongs to the host language. What follows is the sentence, and it has four spellings like every other:

R

```
data(w) + zone * bounds(base, top, start = left, end = right) +
  x(step) + color(dir) + style(opacity = 1) +
  palette(c("steelblue", "seagreen", "indianred")) +
  y_label("Cash (thousands)") + title("Where the money went")
```

Python

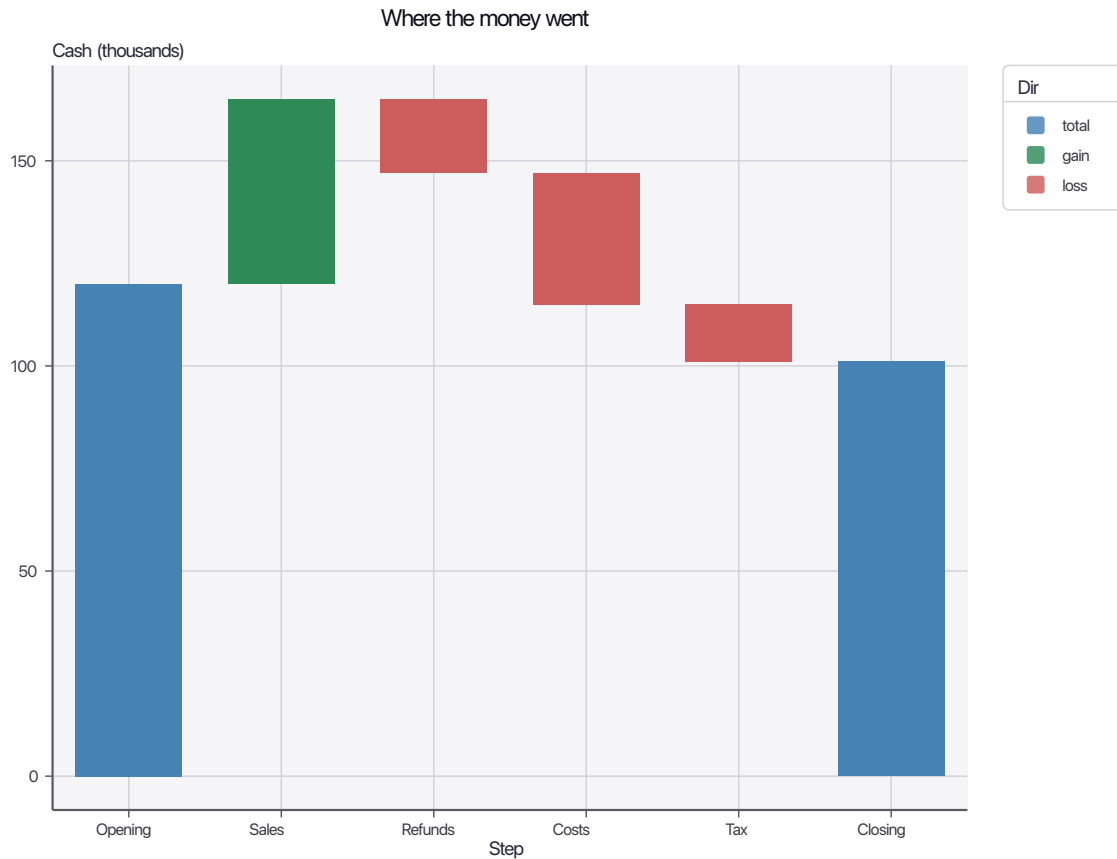
```
(data(w) + zone * bounds(col.base, col.top, start=col.left, end=col.right) +
  x(col.step) + color(col.dir) + style(opacity = 1) +
  palette(["steelblue", "seagreen", "indianred"])) +
  y_label("Cash (thousands)") + title("Where the money went"))
```

Julia

```
data(w) + zone * bounds(:base, :top, start = :left, var"end" = :right) +
  x(:step) + color(:dir) + style(opacity = 1) +
  palette(["steelblue", "seagreen", "indianred"]) +
  y_label("Cash (thousands)") + title("Where the money went")
```

JavaScript

```
plot(data(w),
  layer(zone,
    bounds(col.base, col.top, { start: col.left, end: col.right })),
  x(col.step), color(col.dir), style({ opacity: 1 }),
  palette(["steelblue", "seagreen", "indianred"]),
  y_label("Cash (thousands)", title("Where the money went"))
```



“Given the `w` table: zones from base to top and from left to right, `x` is step, color by `dir`, with `opacity 1`.”

A few lines of host arithmetic and one sentence. The bars that start at zero are not a special case. They are rows whose `base` is 0, a number the accounts supplied like every other.

Two parts of that sentence are worth pausing on, because both are this mark’s defaults being overruled on purpose.

`style(opacity = 1)` is there because a zone whose sides you named is drawn translucent, at 20%. A rectangle someone chose is nearly always a highlight, with something drawn over it. Here nothing is: the rectangle *is* the measurement. A cut zone (a [heatmap](#) cell) is already opaque for the same reason, and this is how you say the named one is data too.

The `left/right` pair makes the gap between bars. Given no sides, a zone fills its slot whole, where a [bar](#) takes four fifths of it and leaves one fifth empty, which is what separates the categories. A zone’s width normally means something: it is the region the category owns. A waterfall bar’s width means nothing, so it should behave like a bar, and naming the sides is

how to ask. The arithmetic works because a categorical axis puts category k at k , so the slot it owns is exactly $[k-\frac{1}{2}, k+\frac{1}{2}]$ and $k \pm 0.35$ sits inside it. Drop those two columns and the rectangles touch, which reads as one connected shape rather than six steps.

A **candlestick** is the same idea with the rectangle floating. Each session has four numbers: open, close, high and low. `gog` needs no new channels for them, because a rectangle's sides read the pair a [ribbon](#) and an [interval](#) already read. Low to high is a whisker; open to close is a rectangle; that is two layers over one table:

Table 19.5: `sessions`: first 5 of 14 rows

session	open	close	high	low
01	100.39	100.45	100.66	99.98
02	99.41	99.72	99.93	98.66
03	101.02	103.11	104.19	100.53
04	102.09	102.01	102.98	101.60
05	105.03	104.85	105.71	103.86

```
s <- sessions
s$body_lo <- pmin(s$open, s$close)
s$body_hi <- pmax(s$open, s$close)
s$dir      <- ifelse(s$close >= s$open, "up", "down")

slot <- seq_len(nrow(s)) - 1      # the same slot arithmetic the waterfall
  ↪ used
s$left  <- slot - 0.3             # a candle body is narrower still
s$right <- slot + 0.3
```

R

```
data(s) + x(session) +
  interval * bounds(low, high) + color(dir) +
  zone * bounds(body_lo, body_hi, start = left, end = right) +
  color(dir) + style(opacity = 1) +
  palette(c("seagreen", "indianred")) +
  y_label("Price") + title("Fourteen sessions")
```

Python

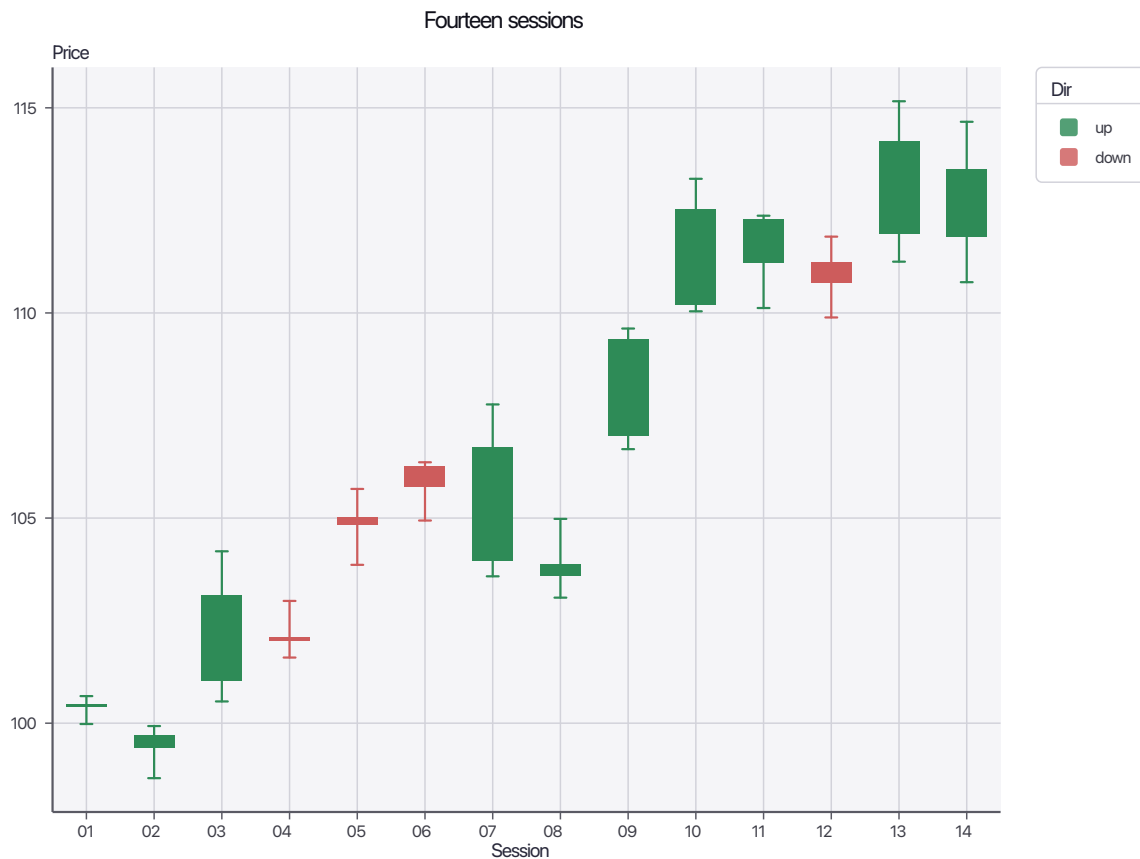
```
(data(s) + x(col.session) +
  interval * bounds(col.low, col.high) + color(col.dir) +
  zone * bounds(col.body_lo, col.body_hi, start=col.left, end=col.right) +
  color(col.dir) + style(opacity = 1) +
  palette(["seagreen", "indianred"]) +
  y_label("Price") + title("Fourteen sessions"))
```

Julia

```
data(s) + x(:session) + interval * bounds(:low, :high) + color(:dir) +
  zone * bounds(:body_lo, :body_hi, start = :left, var"end" = :right) +
  color(:dir) + style(opacity = 1) + palette(["seagreen", "indianred"]) +
  y_label("Price") + title("Fourteen sessions")
```

JavaScript

```
plot(data(s), x(col.session), layer(interval, bounds(col.low, col.high)),
  color(col.dir),
  layer(zone,
    bounds(col.body_lo, col.body_hi, { start: col.left, end: col.right })),
  color(col.dir), style({ opacity: 1 }),
  palette(["seagreen", "indianred"]), y_label("Price"),
  title("Fourteen sessions"))
```



“Given the `s` table: `x` is session, intervals from low to high, color by dir, and also zones from body lo to body hi and from left to right, color by dir.”

`color(dir)` is said twice, once per layer, and that is [scope](#) working rather than repetition: a channel written after a mark belongs to that mark, so the whisker takes its session’s color along with the body. Both readings of the same column earn *one* legend, because it is one column.

There is no `candlestick` atom, and the grammar does not need one. The chart is a composition of things that already exist, which is the whole claim of the grammar: [Chart names](#) lists it beside the pie and the rose, which are also sentences rather than words. What the two charts share is the shape of the answer. Four numbers a session: two of them a *span*, two of them a *rectangle’s sides*. The pair a band mark already reads turns out to be exactly the pair a rectangle needs. That is why neither chart needed a new channel.

19.7 Cells cut from the data: the heatmap

So far every zone has been a rectangle you *chose*, its sides named by columns you already had. There is a second way to get sides, and it needs no new mark: let `bin` cut them.

Gapminder has 1704 country-years. Draw them as points and the crowded middle becomes one dark mass, because a thousand dots in one place look exactly like two hundred:

R

```
data(gm_all) + zone * bin(24) + x(gdp, scale = "log") + y(life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Where the country-years actually are")
```

Python

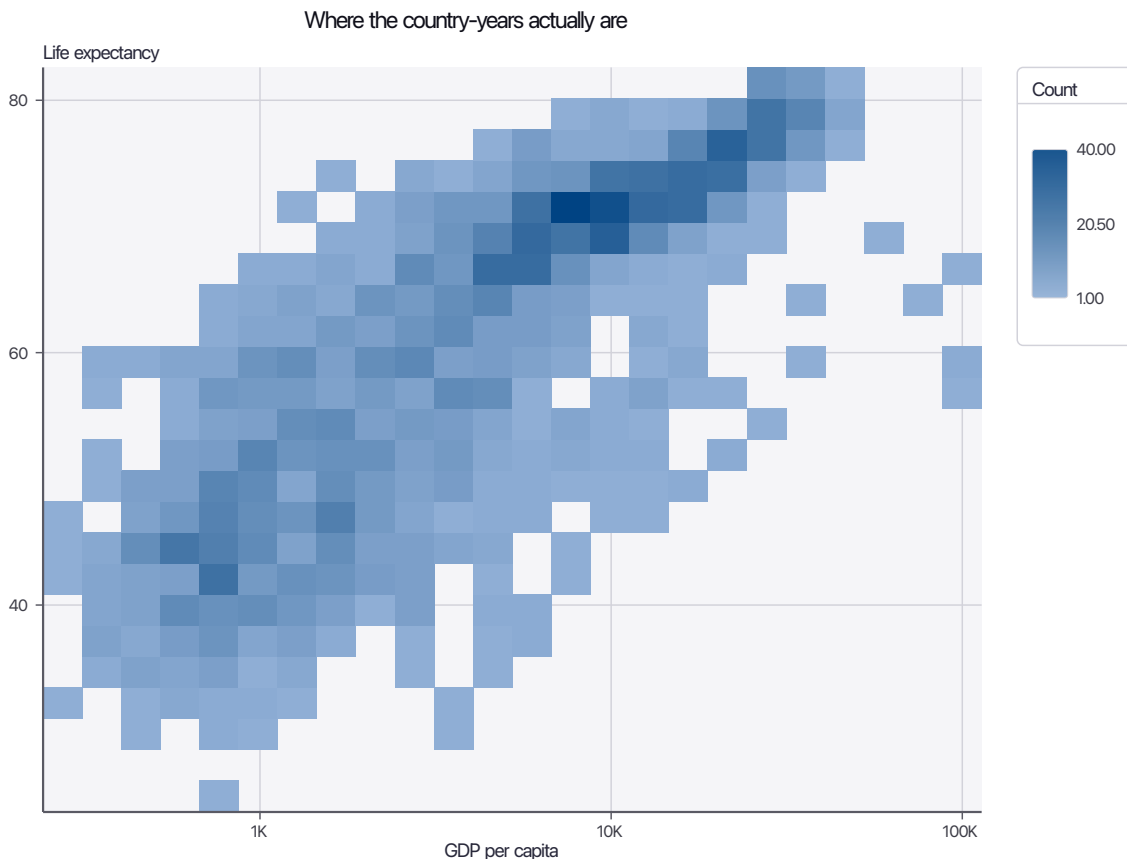
```
(data(gm_all) + zone * bin(24) + x(col.gdp, scale = "log") + y(col.life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Where the country-years actually are"))
```

Julia

```
data(gm_all) + zone * bin(24) + x(:gdp, scale = "log") + y(:life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Where the country-years actually are")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(24)), x(col.gdp, { scale: "log" }),  
  y(col.life), x_label("GDP per capita"), y_label("Life expectancy"),  
  title("Where the country-years actually are"))
```



“Given all the gapminder years: zones derived by bin into 24, x is gdp on a log scale, y is life.”

Read the band of dark cells running up the middle: wealth and life expectancy rise together. The darkest cell says how many country-years sit at that combination, which overlapping dots cannot.

Three things in that sentence are worth slowing down on.

bin cut both axes, and nothing asked it to. It is the same transform that makes a histogram, and how many axes it cuts is read off the mark rather than requested. A **bar** leaves one axis free to measure a height along, so **bar * bin** cuts the other one. A zone measures nothing by height, and needs an extent on *both* axes before it can be a rectangle at all, so **zone * bin** cuts every axis that has a width to cut, which here is both of them. One transform, one word, readings decided by what the mark needs. There is no **bin2d**.

Note the words *has a width to cut*. The mixed mesh below is where they matter: an axis whose values are categories arrived already cut, and **bin** leaves it alone.

The count became the color, and nothing bound it. Both positions were taken, so the measurement went to the one channel left that can carry a number, and the legend named itself. This is what `bar * bin` already does for the y axis: a transform that invents a column also says where it goes.

The column it invents is called `count`, and you may name it out loud. That is not a different plot, it is the same sentence with nothing left implied:

```
short <- data(gm_all) + zone * bin(24) + x(gdp, scale = "log") + y(life)
long  <- data(gm_all) + zone * bin(24) + x(gdp, scale = "log") + y(life) +
      color(count)

identical(render_svg(short), render_svg(long))
```

[1] TRUE

So why write it? Because *a binding is where a scale goes*, and the short form has nowhere to put one. Counts are badly skewed here: the fullest cells hold about forty country-years while most hold one or two. On a linear ramp nearly every cell is the same pale blue. Name the channel and you can say `scale = "log"` on it, the same way you already did on `x`:

R

```
data(gm_all) + zone * bin(24) + x(gdp, scale = "log") + y(life) +
  color(count, scale = "log") +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The same cells, counted on a log ramp")
```

Python

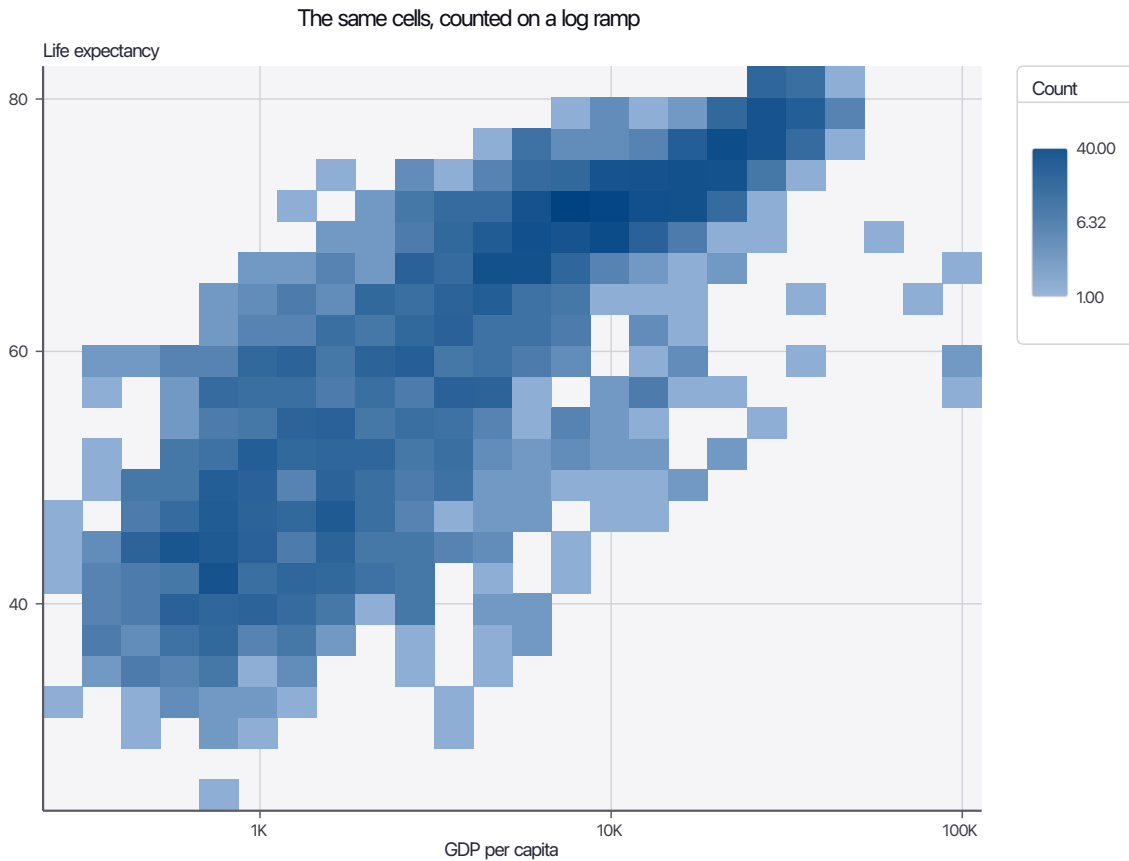
```
(data(gm_all) + zone * bin(24) + x(col.gdp, scale = "log") + y(col.life) +
  color(col.count, scale = "log") +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The same cells, counted on a log ramp"))
```

Julia

```
data(gm_all) + zone * bin(24) + x(:gdp, scale = "log") + y(:life) +
  color(:count, scale = "log") + x_label("GDP per capita") +
  y_label("Life expectancy") +
  title("The same cells, counted on a log ramp")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(24)), x(col.gdp, { scale: "log" }),
     y(col.life), color(col.count, { scale: "log" }),
     x_label("GDP per capita"), y_label("Life expectancy"),
     title("The same cells, counted on a log ramp"))
```



“Given all the gapminder years: zones derived by bin into 24, x is gdp on a log scale, y is life, color by count on a log scale.”

Now the sparse tail separates and the ridge is a gradient rather than a blot. Look at the legend’s middle label: 6.32, not 20.5, because the midpoint of a log ramp is the geometric mean, so the strip names the color it actually paints there.

The white gaps are not zeroes. A cell with no country-years in it is left as panel rather than painted the pale end of the ramp, because painting it would claim a measurement nobody made.

The ragged edge you get instead *is* the region the data actually covers, which is information rather than an unfinished job.

`bin(24)` sets the mesh, exactly as it does on a histogram, and one number cuts both axes into that many bins. A coarser mesh shows less detail, and claims less precision:

R

```
data(gm_all) + zone * bin(8) + x(gdp, scale = "log") + y(life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("The same data, eight bins a side")
```

Python

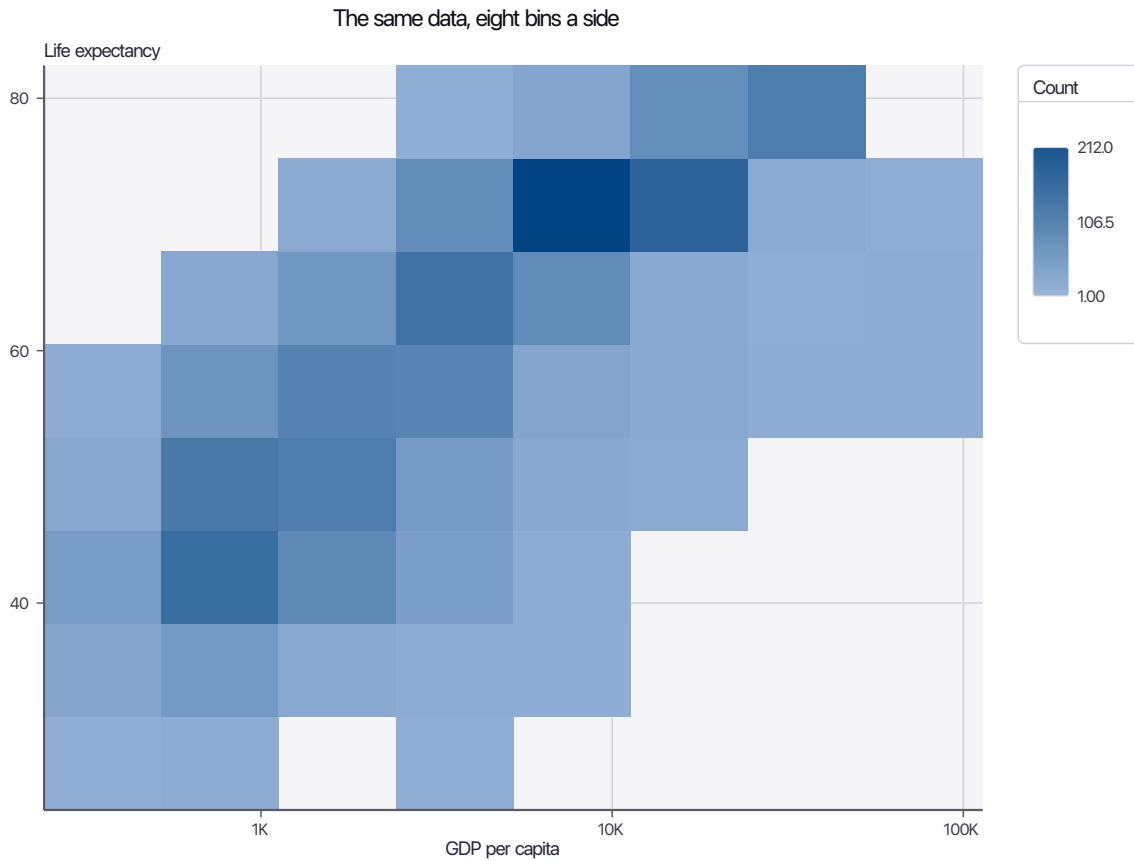
```
(data(gm_all) + zone * bin(8) + x(col.gdp, scale = "log") + y(col.life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("The same data, eight bins a side"))
```

Julia

```
data(gm_all) + zone * bin(8) + x(:gdp, scale = "log") + y(:life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("The same data, eight bins a side")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(8)), x(col.gdp, { scale: "log" }),  
  y(col.life), x_label("GDP per capita"), y_label("Life expectancy"),  
  title("The same data, eight bins a side"))
```



Notice the log scale carried through untouched. The cells are cut in log space, so they are even on the page rather than even in dollars, which is what you want when the variable spans from a few hundred dollars to a hundred thousand.

A binned zone also drops the translucency: here the zone *is* the data, with nothing behind it to show through.

19.8 Hexagons: the other mesh

The cells above are rectangles, and that is a choice rather than a fact about binning. Say `tiling = "hex"` and the plane is cut a different way:

R

```
data(gm_all) + zone * bin(20, tiling = "hex") + x(gdp, scale = "log") +
  ↪ y(life) +
  palette("viridis") +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The same country-years, hexagonal cells")
```

Python

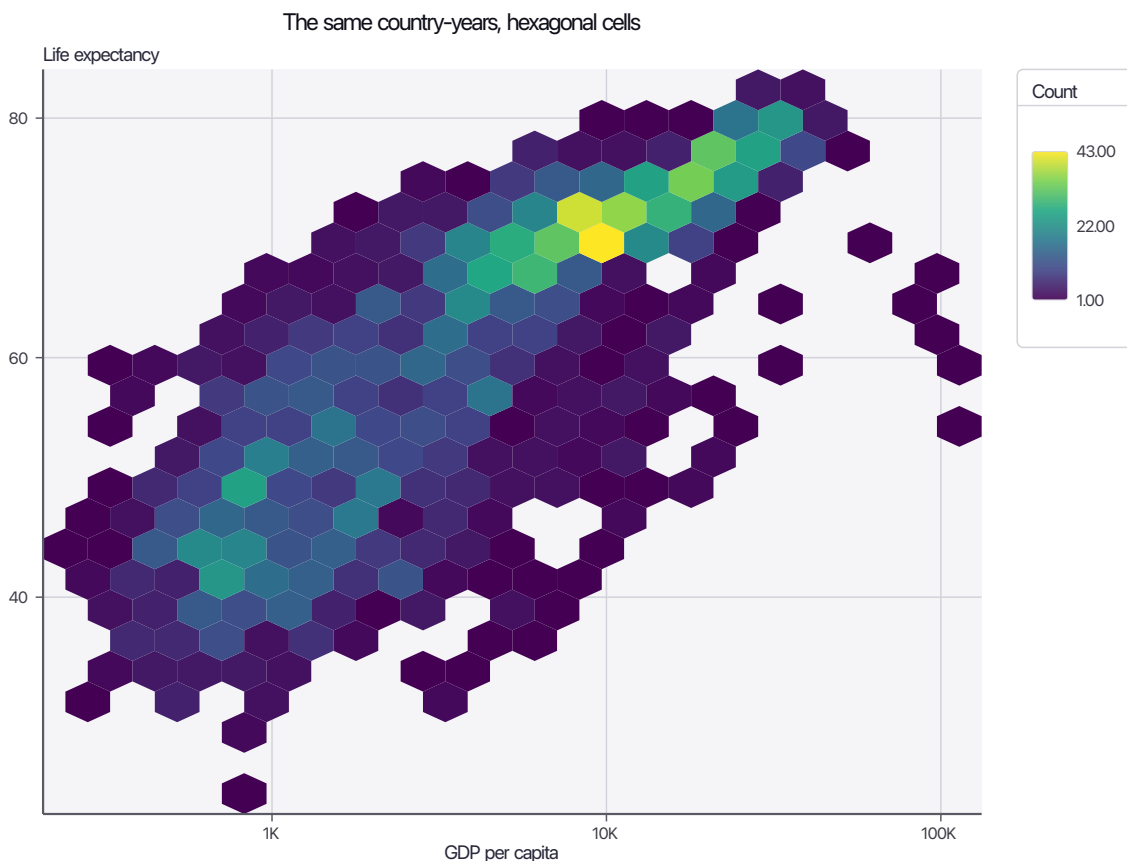
```
(data(gm_all) + zone * bin(20, tiling = "hex") + x(col.gdp, scale = "log") +
  ↪ y(col.life) +
  palette("viridis") +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The same country-years, hexagonal cells"))
```

Julia

```
data(gm_all) + zone * bin(20, tiling = "hex") + x(:gdp, scale = "log") +
  y(:life) + palette("viridis") + x_label("GDP per capita") +
  y_label("Life expectancy") +
  title("The same country-years, hexagonal cells")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20, { tiling: "hex" })),
  x(col.gdp, { scale: "log" }), y(col.life), palette("viridis"),
  x_label("GDP per capita"), y_label("Life expectancy"),
  title("The same country-years, hexagonal cells"))
```



“Given all the gapminder years: zones derived by bin into 20 on hexagons, x is gdp on a log scale, y is life, with the viridis palette.”

Two things changed there, and only one of them is the mesh. `palette("viridis")` replaces the default sequential ramp, and it is the right choice for a count. Viridis is built so equal steps in the number are equal steps in perceived brightness. On a plot whose entire message is *how many*, that is the difference between reading the ridge and guessing at it. The palette is a free choice on either mesh, though, so do not read it as something hexagons require.

The mesh itself is not decoration, and the reason to prefer it is a defect in the rectangular one rather than a preference about shapes. A square grid lines its cell centers up in rows *and* columns, and the eye is very good at seeing rows and columns: it reads that alignment as if it were structure in the data. Wilkinson states it directly: rectangular bins “lead the eye to align bin centers and to see regularity where there is none” (Wilkinson, 2005). That is why Carr devised hexagonal binning in 1987 (Carr et al., 1987). A hexagonal mesh staggers alternate rows, so there is no aligned `lattice` left to mistake for a finding.

To see that on its own, hold the palette still and change only the mesh:

R

```
data(gm_all) + zone * bin(20) + x(gdp, scale = "log") + y(life) +  
  palette("viridis") +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Rectangular cells, same palette")
```

Python

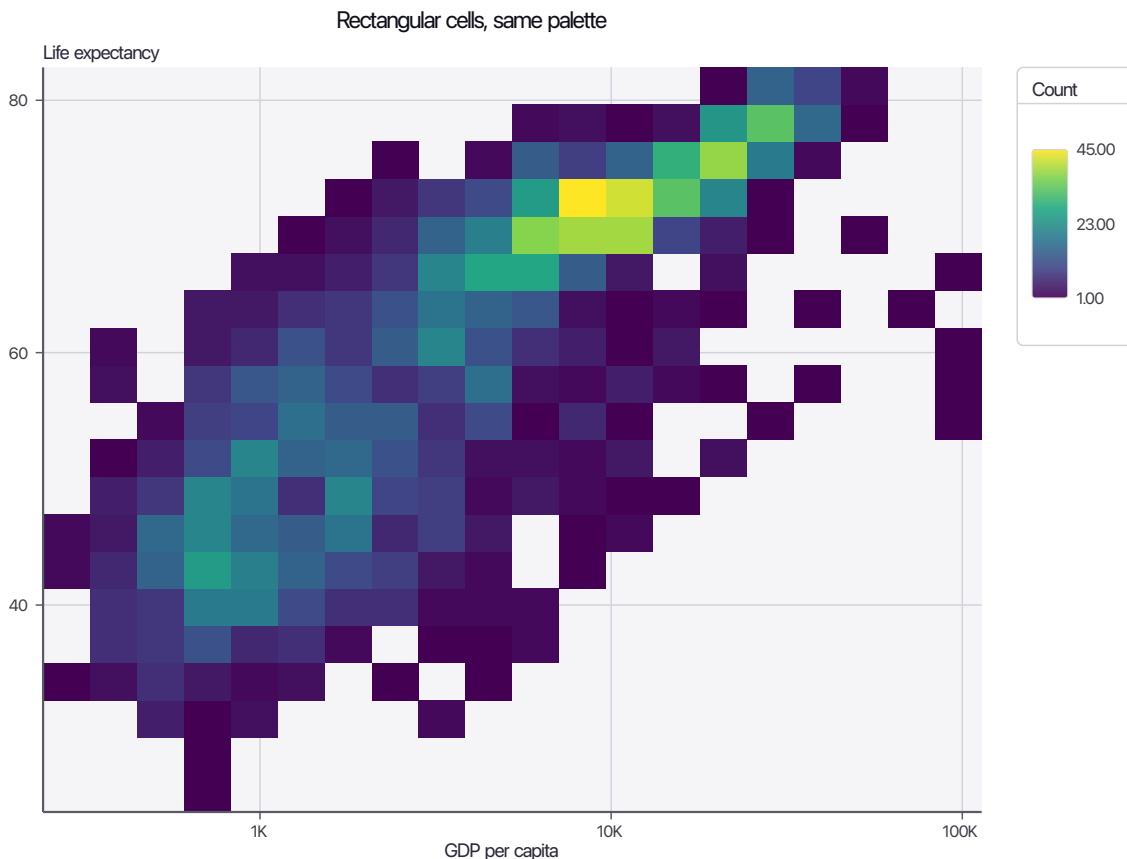
```
(data(gm_all) + zone * bin(20) + x(col.gdp, scale = "log") + y(col.life) +  
  palette("viridis") +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Rectangular cells, same palette"))
```

Julia

```
data(gm_all) + zone * bin(20) + x(:gdp, scale = "log") + y(:life) +  
  palette("viridis") + x_label("GDP per capita") +  
  y_label("Life expectancy") + title("Rectangular cells, same palette")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20)), x(col.gdp, { scale: "log" }),  
  y(col.life), palette("viridis"), x_label("GDP per capita"),  
  y_label("Life expectancy"), title("Rectangular cells, same palette"))
```



Look along the sparse edges of the two. The rectangular cells line up into faint rows and columns that continue past the edge of the data. The hexagons do not, and what is left at the edge is the shape of the data rather than the shape of the mesh.

19.8.1 Why the mesh belongs to bin

Everywhere else in this book, a transform computes and the **mark** decides how the result is drawn: `bar * bin`, `line * bin` and `step * bin` are one histogram painted three ways. So it would be reasonable to expect the hexagon to be a mark, or a setting on one. It is neither, and the reason matters.

Those three marks all receive **the same numbers**. A hexagonal mesh staggers its rows, which changes *which rows land in which cell*, so the counts themselves come out different. You cannot draw a hexagonal plot from rectangularly binned data, whatever the mark does with it. The mesh is decided before anything is drawn, so it belongs to the transform that does the cutting.

That is also how Wilkinson organizes it. His `bin` is not one method but a family of ways to partition a plane, with `rect` and `hex` among the members. The mesh both shapes the cells *and* tags each row with the cell it fell in. One operation, both effects.

The consequence you can see: a tiling means nothing to a bin that cuts one axis, because there the cells are intervals and an interval has no shape.

```
data(gm_all) + bar * bin(tiling = "hex") + x(life)
```

Error:

```
! gog: `bin(tiling = )` says how to divide a *plane*, and a `bar` bins one
  ↳ axis - its cells are intervals, and an interval has no shape. Drop the
  ↳ tiling for a histogram, or use `zone * bin(tiling = "hex")` to cut both
  ↳ axes into cells and color each by its count.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

19.8.2 One caveat

The hexagons are regular in the space the mesh is cut in, which normalizes both axes to the same number of steps. The panel then stretches that space by whatever its own proportions are, so on a wide panel the cells come out slightly wide. Look closely at the plot above and you can see it.

This is what the original `hexbin` package exposes as its `shape` argument. The fix would be a panel told to keep equal proportions, which is what a `polar` plot needs for round circles. `gog` has no setting for that, so a roughly square panel gives roughly regular hexagons.

19.9 Cells the data estimated: zone * density

`bin` counts what landed in each cell. There is a second thing worth measuring at a cell, and it needs no new mark either: how *thick* the cloud is there. That is **density**, the transform that draws a smooth curve on a [line](#), read in two dimensions for exactly the reason `bin` was.

R

```
data(iris_flowers) + zone * density +
  x(sepal_length) + y(petal_length) +
  title("Where the flowers cluster")
```

Python

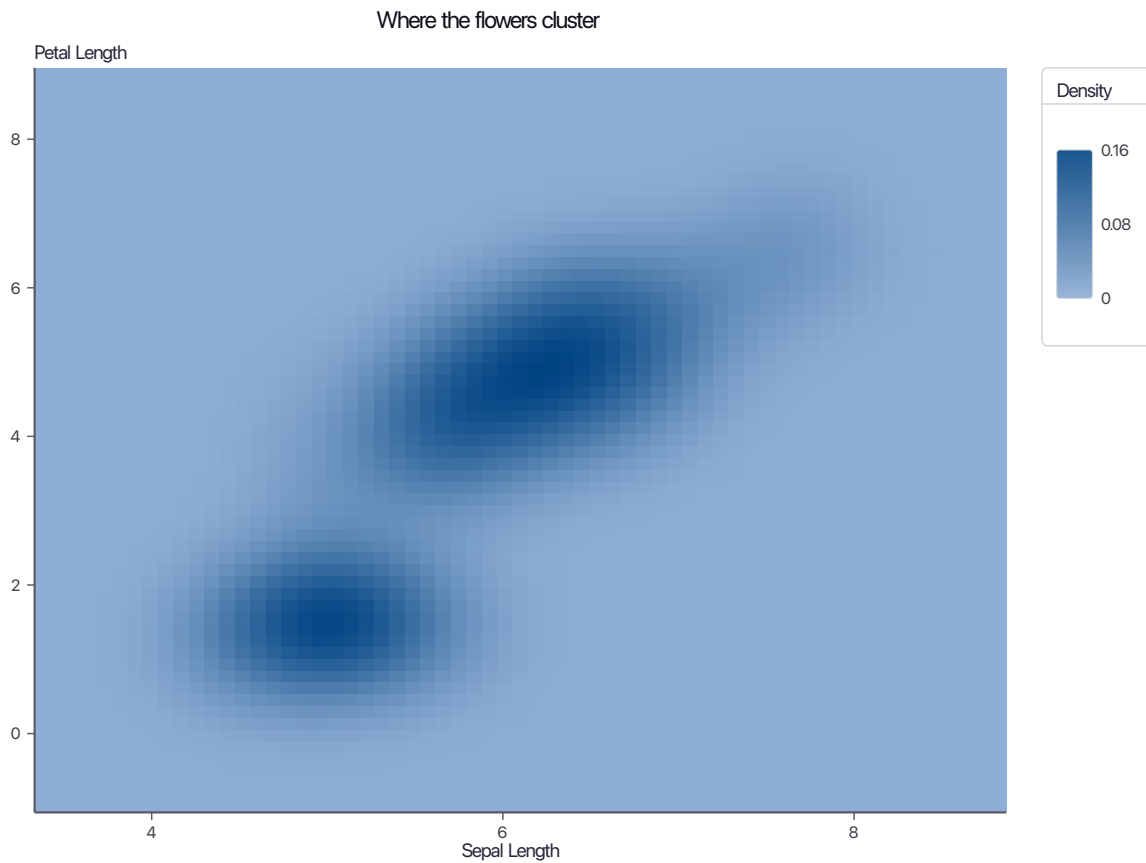
```
(data(iris_flowers) + zone * density +
  x(col.sepal_length) + y(col.petal_length) +
  title("Where the flowers cluster"))
```

Julia

```
data(iris_flowers) + zone * density + x(:sepal_length) +
  y(:petal_length) + title("Where the flowers cluster")
```

JavaScript

```
plot(data(iris_flowers), layer(zone, density), x(col.sepal_length),
  y(col.petal_length), title("Where the flowers cluster"))
```



“Given the iris flowers: zones derived by density, x is sepal length, y is petal length.”

Every sentence from the heatmap above is true here word for word. Both axes were cut and nothing asked for it, because a zone measures nothing by height and needs an extent on both before it can be a rectangle at all. The measurement went to `color` with nothing bound, and the column it invents is called `density`, so `color(density)` names it out loud and draws the identical plot. `zone * bin` and `zone * density` are one plot measured two ways, counted and estimated.

The white gaps are the difference, and here there are none. A count of zero says the data did not go to that cell, so `bin` leaves it unpainted. An estimate exists at every point of the plane. The estimator returns a value for every cell, so the mesh is painted edge to edge. What it paints is a real density rather than an arbitrary color value. The cell values times the cell areas sum to one, the same property that makes the curve’s y a density.

`palette()` chooses the ramp, exactly as it does for a binned heatmap, and the argument for `viridis` on a count is the argument for it here:

R

```
data(iris_flowers) + zone * density +  
  x(sepal_length) + y(petal_length) + palette("viridis") +  
  title("Filled density, viridis")
```

Python

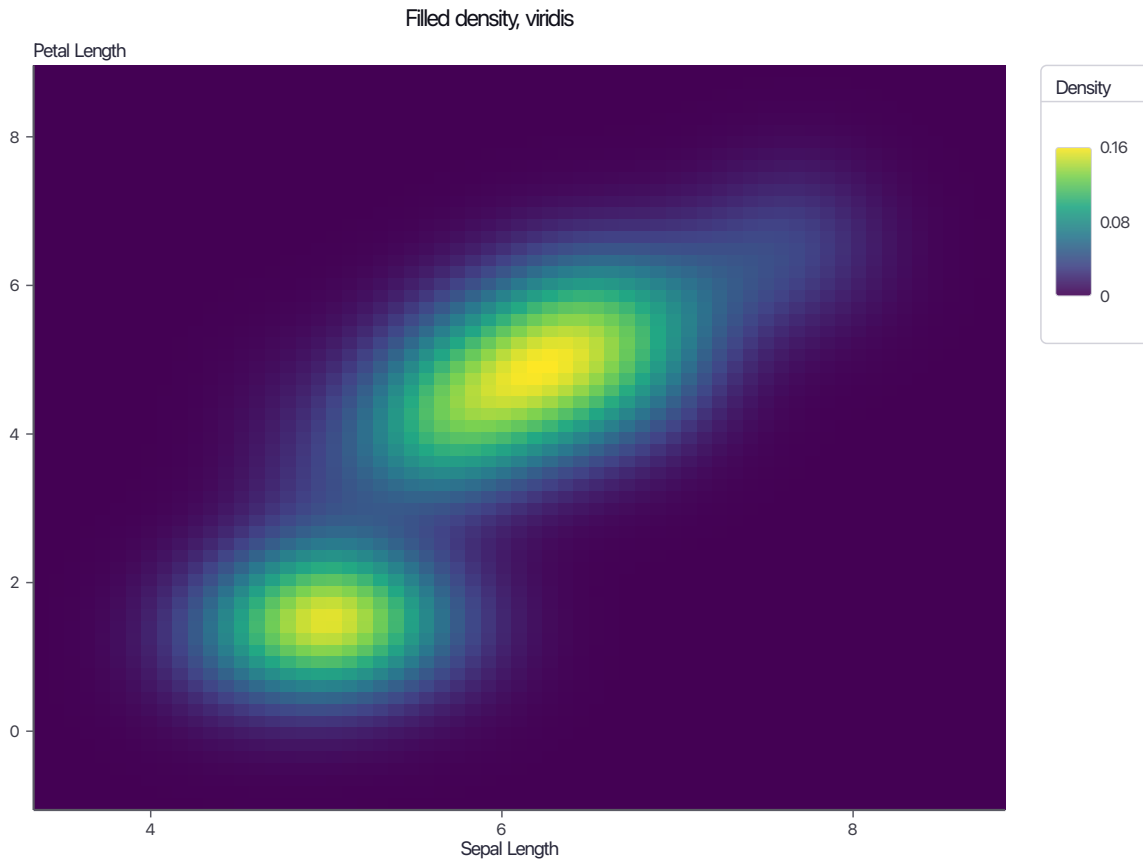
```
(data(iris_flowers) + zone * density +  
  x(col.sepal_length) + y(col.petal_length) + palette("viridis") +  
  title("Filled density, viridis"))
```

Julia

```
data(iris_flowers) + zone * density + x(:sepal_length) +  
  y(:petal_length) + palette("viridis") + title("Filled density, viridis")
```

JavaScript

```
plot(data(iris_flowers), layer(zone, density), x(col.sepal_length),  
  y(col.petal_length), palette("viridis"),  
  title("Filled density, viridis"))
```



Look at the dark area away from the two clusters: it is even, and the cells meet cleanly. Blending each cell's edge into the background would leave a few percent of the panel showing at every seam. A pale ramp hides that; this one would draw it as a fine `lattice` over the whole field, the same false structure the hexagon section describes, arriving from the renderer rather than from the mesh. So a cut cell's edges land on whole pixels instead.

The parameter a painted field takes is `adjust`, the multiplier on the bandwidth the estimator chose for itself. It is dimensionless, which is why it is the one parameter that means the same thing on both axes:

R

```
data(iris_flowers) + zone * density(adjust = 1.6) +  
  x(sepal_length) + y(petal_length) + palette("viridis") +  
  title("The same cloud, smoothed harder")
```

Python

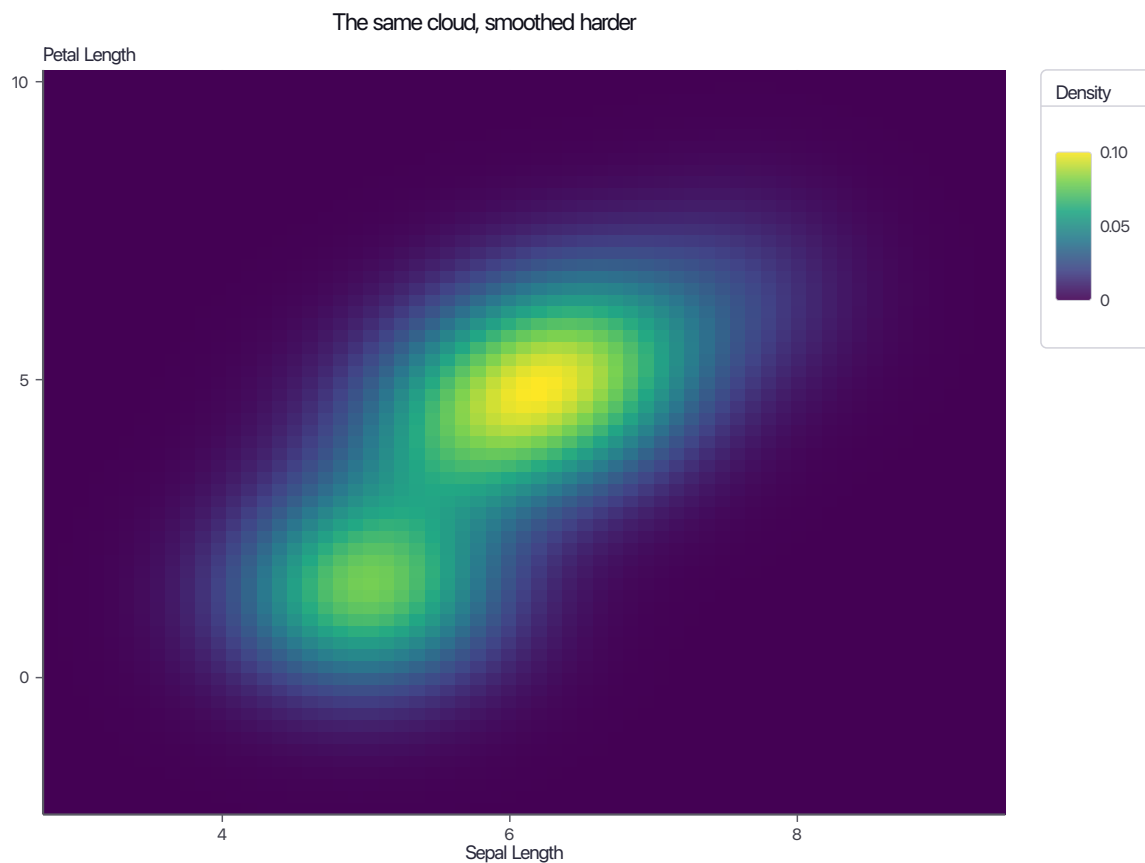
```
(data(iris_flowers) + zone * density(adjust = 1.6) +
  x(col.sepal_length) + y(col.petal_length) + palette("viridis") +
  title("The same cloud, smoothed harder"))
```

Julia

```
data(iris_flowers) + zone * density(adjust = 1.6) + x(:sepal_length) +
  y(:petal_length) + palette("viridis") +
  title("The same cloud, smoothed harder")
```

JavaScript

```
plot(data(iris_flowers), layer(zone, density({ adjust: 1.6 })),
  x(col.sepal_length), y(col.petal_length), palette("viridis"),
  title("The same cloud, smoothed harder"))
```



“Given the iris flowers: zones derived by density adjusted by 1.6, x is sepal length, y is petal length, with the viridis palette.”

The same field *traced* rather than painted is `path * density`, the contour, and the division between them is the one `bar * bin` and `line * bin` already made: the mark chooses the geometry, the transform stays constant.

19.10 Bands the data cut: `zone * density(levels = )`

Ask for levels and the field stops being continuous. It is cut into that many steps, and a zone fills between them:

R

```
data(iris_flowers) + zone * density(levels = 8) +  
  x(sepal_length) + y(petal_length) + palette("viridis") +  
  title("Filled density bands")
```

Python

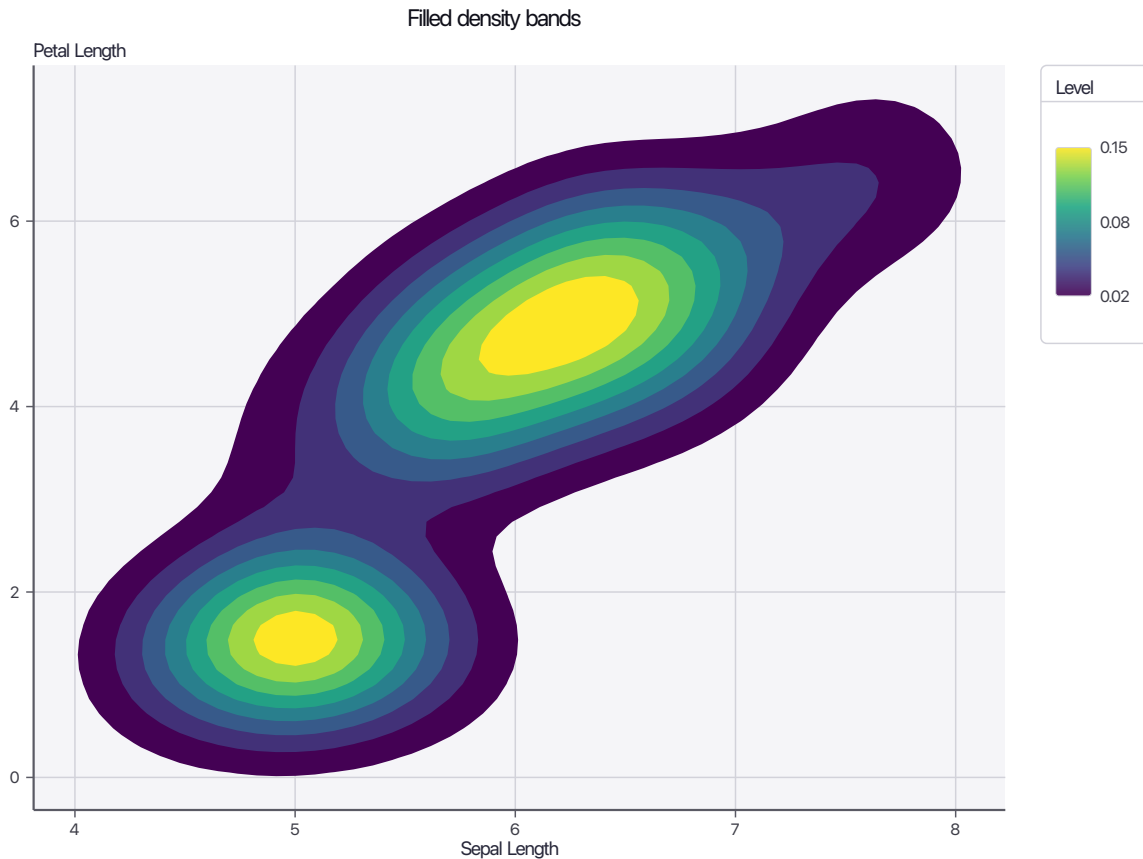
```
(data(iris_flowers) + zone * density(levels = 8) +  
  x(col.sepal_length) + y(col.petal_length) + palette("viridis") +  
  title("Filled density bands"))
```

Julia

```
data(iris_flowers) + zone * density(levels = 8) + x(:sepal_length) +  
  y(:petal_length) + palette("viridis") + title("Filled density bands")
```

JavaScript

```
plot(data(iris_flowers), layer(zone, density({ levels: 8 })),  
  x(col.sepal_length), y(col.petal_length), palette("viridis"),  
  title("Filled density bands"))
```



“Given the iris flowers: zones derived by density at 8 levels, x is sepal length, y is petal length.”

That is the **filled contour**, and it is the same parameter `path` takes. `levels` means one thing (cut the field into this many) and then each mark does what it does: a path traces the boundaries, a zone fills between them. So the two are not two features but one sentence read by two marks, and a band’s edge falls exactly on the contour line:

R

```
data(iris_flowers) + path * density(levels = 8) +
  x(sepal_length) + y(petal_length) + palette("viridis") +
  title("The same eight levels, traced")
```

Python

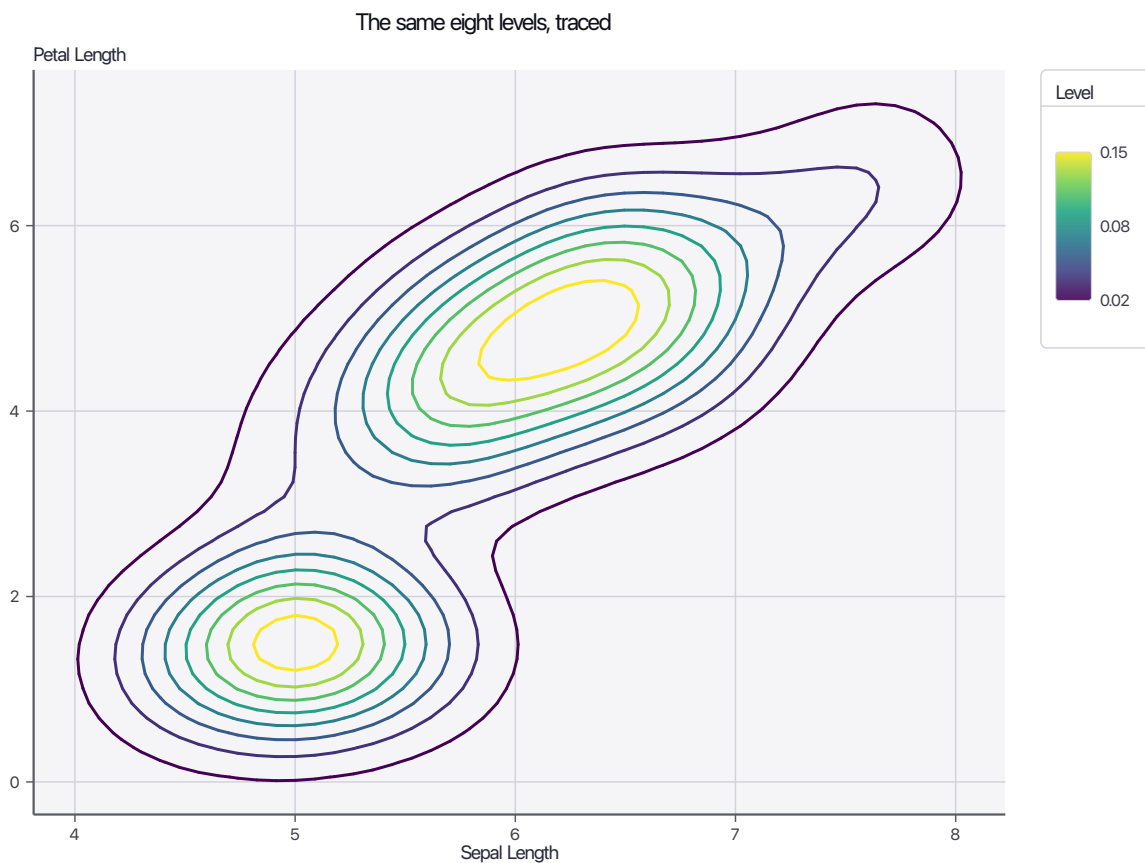
```
(data(iris_flowers) + path * density(levels = 8) +
  x(col.sepal_length) + y(col.petal_length) + palette("viridis") +
  title("The same eight levels, traced"))
```

Julia

```
data(iris_flowers) + path * density(levels = 8) + x(:sepal_length) +
  y(:petal_length) + palette("viridis") +
  title("The same eight levels, traced")
```

JavaScript

```
plot(data(iris_flowers), layer(path, density({ levels: 8 })),
  x(col.sepal_length), y(col.petal_length), palette("viridis"),
  title("The same eight levels, traced"))
```



“Given the iris flowers: paths derived by density at 8 levels, x is sepal length, y is petal length.”

That makes three readings of one transform, and the mark decides every time: `line * density` is a curve, `zone * density` a painted field, and `levels` turns the field into level sets that `path` strokes and `zone` fills.

The bands nest, and that is why plain filled shapes are enough to draw them. A denser region is always *inside* a less dense one, so `gog` paints the outermost band first and lets each inner one cover it; nothing has to be cut out. The exception is a crater: a ring-shaped cluster with a hollow middle, points scattered around a circle. There the hollow really is a hole, and painting the band over it fills it in. `gog` has no mark for a shape with a hole, so a crater is the one field `zone * density` draws wrongly. `path * density` traces it correctly.

19.11 Cells the axes already made: the tile plot

Three sources of sides so far, and every one of them put the sides in a column: `bounds` named them, `bin` cut them, `density(levels = )` traced them. There is a fourth, and it needs no column at all.

A categorical axis is already divided. Six weeks on the x axis is six slots. The slot “Week 3” owns is a place on the panel with a left edge and a right edge, both fixed by the axis when it laid the categories out. So a category bounds its own axis, and two categorical positions bound a zone completely, with no transform in the sentence:

R

```
data(six_weeks) + zone + x(week) + y(weekday) + color(orders) +  
  title("Six weeks of orders, a day at a time")
```

Python

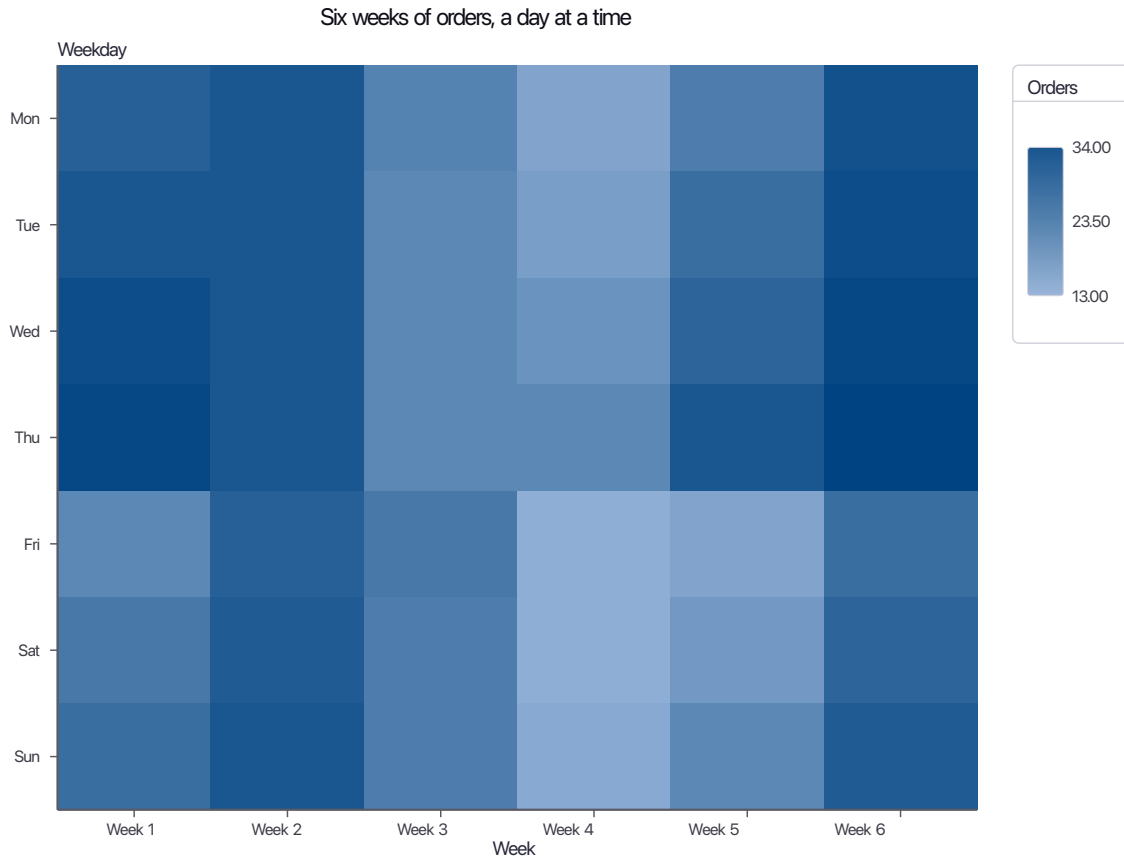
```
(data(six_weeks) + zone + x(col.week) + y(col.weekday) + color(col.orders) +  
  title("Six weeks of orders, a day at a time"))
```

Julia

```
data(six_weeks) + zone + x(:week) + y(:weekday) + color(:orders) +  
  title("Six weeks of orders, a day at a time")
```

JavaScript

```
plot(data(six_weeks), zone, x(col.week), y(col.weekday),
     color(col.orders), title("Six weeks of orders, a day at a time"))
```



“Given the six weeks table: zones, x is week, y is weekday, color by orders.”

That is a calendar heatmap, and the whole of it is **zone** + **x** + **y** + **color**. What makes it worth drawing is that 42 daily counts carry two rhythms at once. A column of numbers shows neither, while a grid separates them by direction. Read *down* a column for the week (four is the pale one, the lowest week) and *across* a row for the weekday, where Thursdays are consistently darkest and Fridays lightest. The extremes are the cells where the two rhythms agree: the darkest cell is Thursday of week six, the palest the Friday in week four.

The rule underneath is one sentence: a category owns a slot, a number is a point. A point has no width, which is why two continuous positions still leave a zone with nothing to bound it, and why **bin** had to be asked to cut them into cells first. Nothing else changed, and nothing was added to the kernel: no mark, no transform, no channel.

19.11.1 bin cuts, count tallies

The calendar table had one row per cell already. When it does not, when you hold the raw observations and want to know how many fell in each cell, the transform that tallies is `count`, exactly as it is for a bar chart:

R

```
data(winds) + zone * count + x(direction) + y(season) +  
  title("Wind observations by direction and season")
```

Python

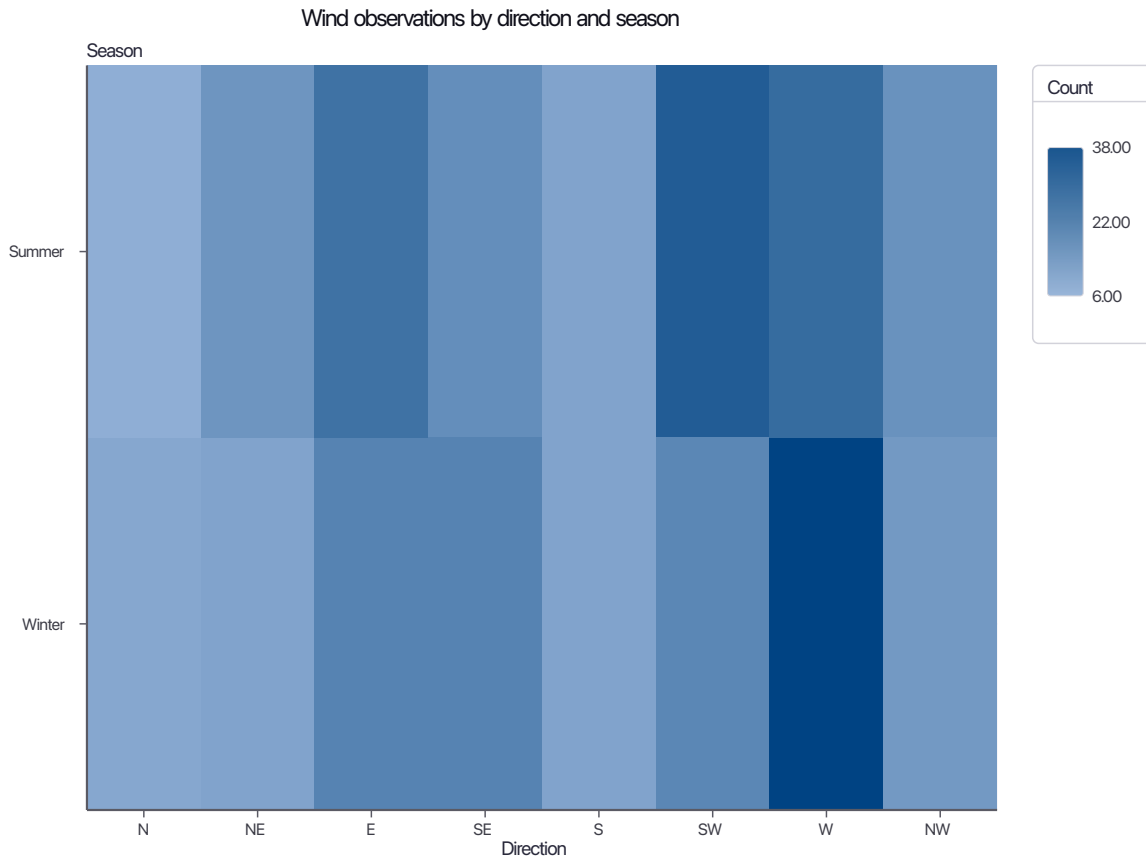
```
(data(winds) + zone * count + x(col.direction) + y(col.season) +  
  title("Wind observations by direction and season"))
```

Julia

```
data(winds) + zone * count + x(:direction) + y(:season) +  
  title("Wind observations by direction and season")
```

JavaScript

```
plot(data(winds), layer(zone, count), x(col.direction), y(col.season),  
  title("Wind observations by direction and season"))
```



“Given the winds: zones derived by count, x is direction, y is season.”

That is the shape of a confusion matrix, the table that counts predicted classes against actual ones, and a confusion matrix is the same sentence with different columns: `zone * count + x(actual) + y(predicted)`.

Notice what was *not* needed there. `bin` cuts a continuous axis into cells before anything can be counted in them; a categorical axis arrives already cut, so there is nothing left to do but tally. That is the same division `bar * bin` and `bar * count` have always had, now with both axes in it, and it is why each one’s refusal names the other. Ask a tile plot for a continuous axis and it names `bin`. Ask a heatmap for two categorical ones, which leave it nothing to cut, and it names `count`.

The second dimension changes only where the answer goes. A bar has a length to put it on, so `bar * count` writes its tally to the y axis; a zone measures nothing by length, so `zone * count` writes it to `color` and the legend names itself. Both positions were taken, so there was nowhere else for it to go.

proportion is the same tally as a share of the whole, which is worth having when the absolute numbers are not the point:

R

```
data(winds) + zone * proportion + x(direction) + y(season) +  
  palette("viridis") + title("The same observations, as shares")
```

Python

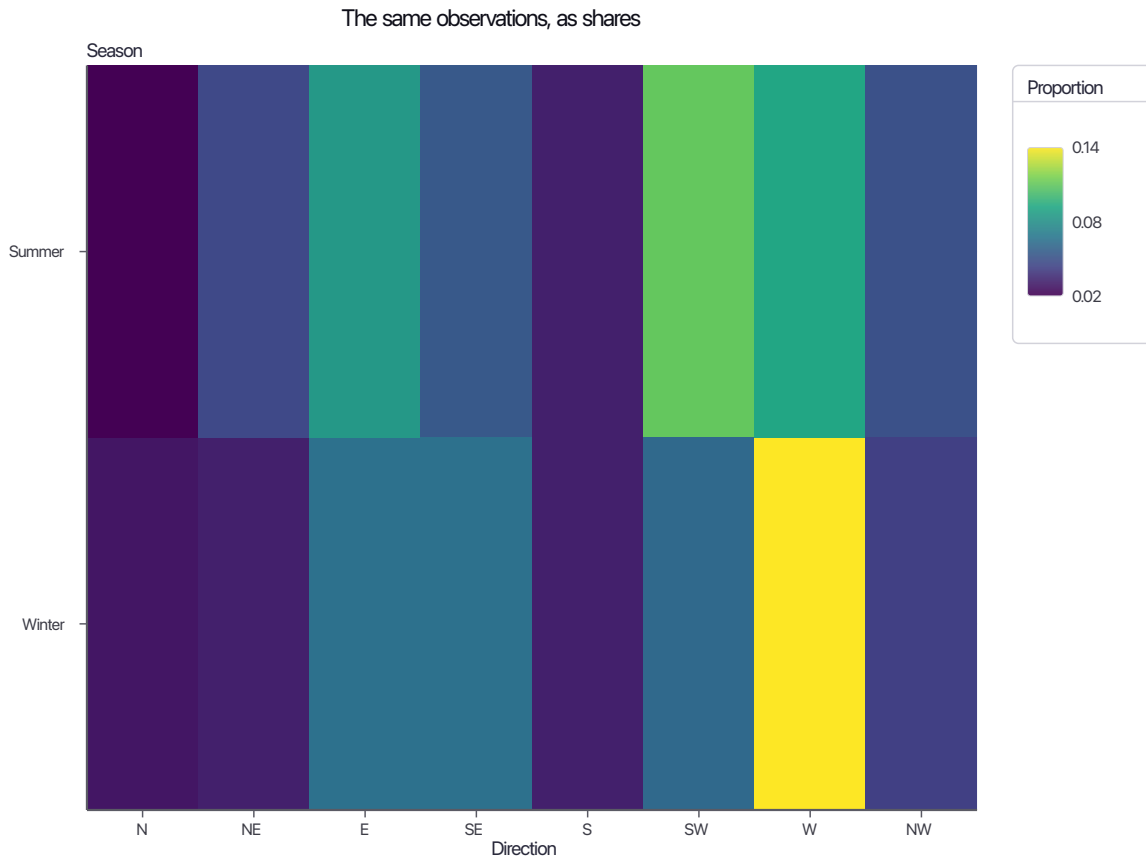
```
(data(winds) + zone * proportion + x(col.direction) + y(col.season) +  
  palette("viridis") + title("The same observations, as shares"))
```

Julia

```
data(winds) + zone * proportion + x(:direction) + y(:season) +  
  palette("viridis") + title("The same observations, as shares")
```

JavaScript

```
plot(data(winds), layer(zone, proportion), x(col.direction),  
  y(col.season), palette("viridis"),  
  title("The same observations, as shares"))
```



“Given the winds: zones derived by proportion, x is direction, y is season.”

That share is of *every* row counted, not of a row or a column of the grid. A confusion matrix divided by the total in each true class asks a different question, *which margin normalizes*, and `gog` does not answer it by giving one word two meanings.

It reads a cut mesh the same way: `proportion` divides whatever measurement the cells carry, whichever transform put it there. Compose it with `bin` and the heatmap’s counts come back as fractions of the whole, the [relative-frequency histogram](#) with a second axis:

R

```
data(gm_all) + zone * bin(20) * proportion + x(gdp, scale = "log") + y(life)
  ↪ +
  palette("viridis") + title("The heatmap, as shares of all 1704 rows")
```

Python

```
(data(gm_all) + zone * bin(20) * proportion + x(col.gdp, scale = "log") +
  ↪ y(col.life) +
  palette("viridis") + title("The heatmap, as shares of all 1704 rows"))
```

Julia

```
data(gm_all) + zone * bin(20) * proportion + x(:gdp, scale = "log") +
  y(:life) + palette("viridis") +
  title("The heatmap, as shares of all 1704 rows")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20), proportion),
  x(col.gdp, { scale: "log" }), y(col.life), palette("viridis"),
  title("The heatmap, as shares of all 1704 rows"))
```



19.11.2 count tallies rows, mean reduces a column

The wind table holds more than the cell each observation fell in. Each row also carries a **speed**, and the question “how fast, on average, in each cell?” is not a tally at all; it is the other kind of statistic, the kind you point at a column:

R

```
data(winds) + zone * mean + x(direction) + y(season) + color(speed) +  
  title("Mean wind speed, by direction and season")
```

Python

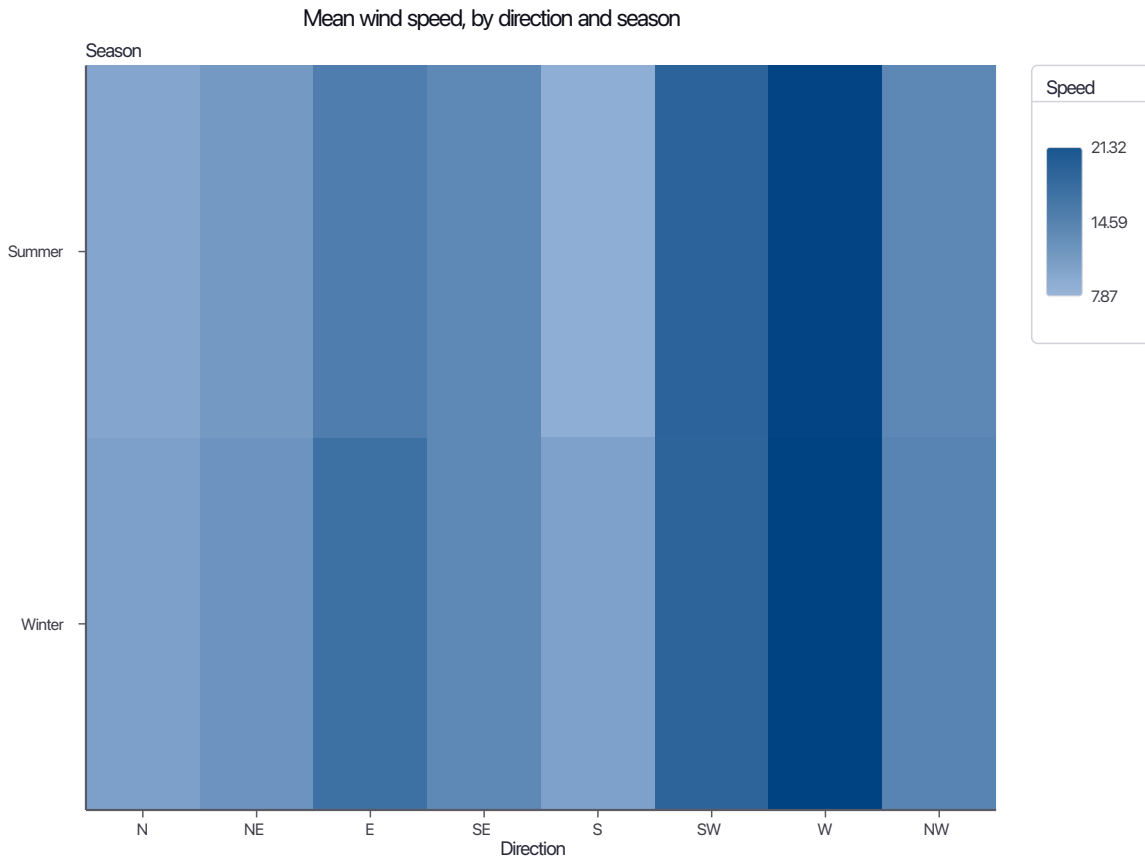
```
(data(winds) + zone * mean + x(col.direction) + y(col.season) +  
  ↪ color(col.speed) +  
  title("Mean wind speed, by direction and season"))
```

Julia

```
data(winds) + zone * mean + x(:direction) + y(:season) + color(:speed) +  
  title("Mean wind speed, by direction and season")
```

JavaScript

```
plot(data(winds), layer(zone, mean), x(col.direction), y(col.season),  
  color(col.speed), title("Mean wind speed, by direction and season"))
```

“Given the winds: zones derived by mean, x is direction, y is season, color by speed.”

Compare that with `zone * count` two plots up. The sentence is the same shape with one more binding, and that binding is the whole difference: `count` was handed no column, so it counts rows and names its own answer; `mean` was handed one, so `color(speed)` says which. All six summaries read a mesh this way (`sum`, `mean`, `median`, `max`, `min` and `quantile`), and each writes its answer back into the column it reduced.

This is where the rule stated under the first heatmap gets its second half. Color is where the answer went, and color is therefore the channel that names the column too, because a summary reduces *in place*. On a flat bar, `bar * mean + x(continent) + y(life)` names `life` with `y` and puts the mean back on `y`; the source and the destination are one binding. A zone measures by color, so `color` plays both parts.

Which gives the rule in one sentence, and it is the same rule the heatmap’s dimensions come from: a summary groups by every position the mark does not measure with, and reduces the column named on the position it measures with. A bar measures with `y`, so it groups by `x`. A

zone measures with nothing positional, so it groups by both. In [the cube](#) a bar measures with z, so it groups by the pair as well.

Say `mean` with nothing for it to reduce and the engine asks for the column rather than guessing:

```
data(winds) + zone * mean + x(direction) + y(season)
```

Error:

```
! gog: `zone * mean` reduces a column within each cell, but nothing says
  ↳ which column - `zone` measures by `color`, and no `color()` is bound.
  ↳ Name it: `zone * mean + x(<a>) + y(<b>) + color(<column>)` . To count the
  ↳ rows in each cell instead of reducing a column, `count` needs no such
  ↳ binding: `zone * count + x(<a>) + y(<b>)` .
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

And a summary still needs a cell to summarize into. These six *measure* without cutting, so they need axes that arrive already divided: a number is a point, and a point owns no cell to summarize into.

```
data(winds) + zone * mean + x(speed) + y(season) + color(bearing)
```

Error:

```
! gog: `zone * mean` summarizes `bearing` inside the cell each pair of
  ↳ categories owns, and `x(speed)` carries numbers - a number is a point, so
  ↳ it owns no cell to summarize into. Either put a category on both
  ↳ positions, or cut the numeric axis into cells first: `zone * bin * mean +
  ↳ x(<a>) + y(<b>) + color(bearing)` bins where your data lives and means
  ↳ `bearing` inside each cell.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Read the refusal rather than only the fact of it. It gives you both ways out: put a category on each position, or cut the numeric axis first with `bin`.

19.11.3 Cells cut, then summarized: the summary heatmap

A category owns a slot, and that is one way an axis can arrive already divided. Cutting a continuous axis is the other, and `bin` is what cuts. Compose the two and each supplies half of what a cell needs:

R

```
data(gm_all) + zone * bin(15) * mean + x(year) + y(gdp, scale = "log") +
  color(life) + title("Mean life expectancy, by era and income")
```

Python

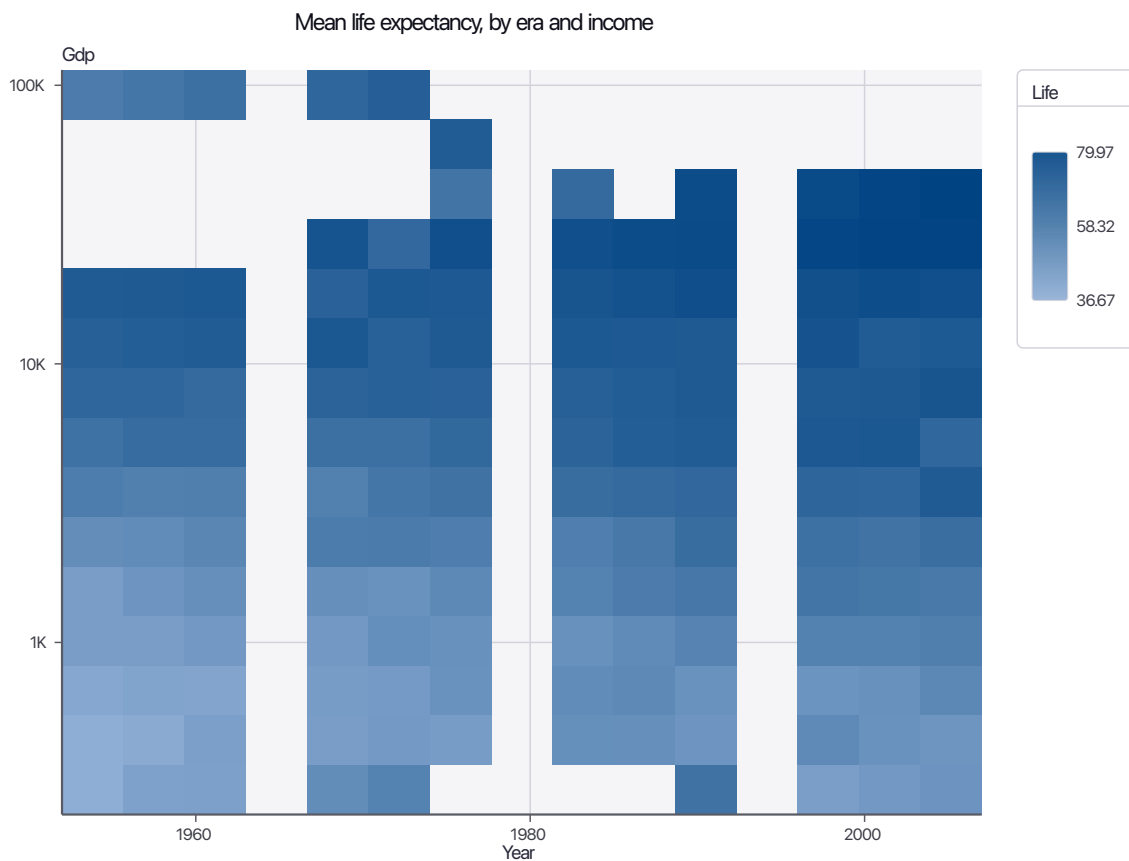
```
(data(gm_all) + zone * bin(15) * mean + x(col.year) + y(col.gdp, scale =  
↪ "log") +  
color(col.life) + title("Mean life expectancy, by era and income"))
```

Julia

```
data(gm_all) + zone * bin(15) * mean + x(:year) + y(:gdp, scale = "log") +  
color(:life) + title("Mean life expectancy, by era and income")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(15), mean), x(col.year),  
y(col.gdp, { scale: "log" })), color(col.life),  
title("Mean life expectancy, by era and income"))
```



`bin(15)` cuts the plane into cells, `mean` reduces `life` inside each, and `color` both names that column and carries the answer, exactly as it does on the tile plot above. Read the panel from bottom left to top right and the two patterns separate. Life expectancy rises with income up the panel, and rises again with time across it.

This is the same mesh a histogram of those two columns would cut. Nothing about `bin` changes when it is composed. It only stops reporting how many rows fell in each cell, because `mean` now supplies the measurement. So the plot above and `zone * bin(15) + x(year) + y(gdp, scale = "log")` tile the panel identically, and can be read against each other: one says how many observations there are in a cell, the other says what they were like.

The mixed case follows without a new rule, because the two halves are still stated one axis at a time. Cut the income axis, leave the continents in their slots:

R

```
data(gm_all) + zone * bin(12) * mean + x(gdp, scale = "log") + y(continent) +
  color(life) + title("Mean life expectancy, by income band and continent")
```

Python

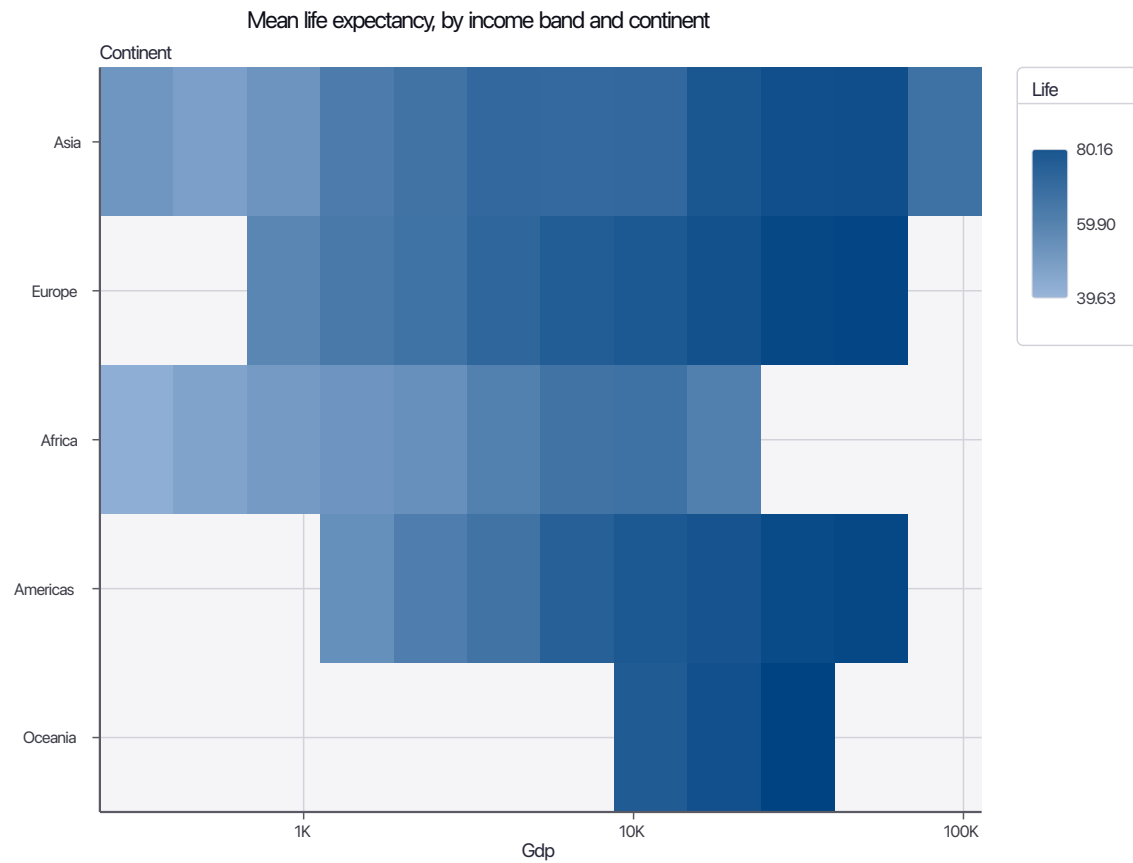
```
(data(gm_all) + zone * bin(12) * mean + x(col.gdp, scale = "log") +
  ↪ y(col.continent) +
  color(col.life) + title("Mean life expectancy, by income band and
  ↪ continent"))
```

Julia

```
data(gm_all) + zone * bin(12) * mean + x(:gdp, scale = "log") +
  y(:continent) + color(:life) +
  title("Mean life expectancy, by income band and continent")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(12), mean),
  x(col.gdp, { scale: "log" }), y(col.continent), color(col.life),
  title("Mean life expectancy, by income band and continent"))
```



19.11.4 One of each: the mixed mesh

The pair above was stated one axis at a time, which leaves a case in the middle: one axis continuous, one categorical. Nothing new has to be decided for it. `bin` cuts the axis that has a width to cut and leaves alone the one that arrived already cut, so what comes out is a row of cells per category:

R

```
data(gm_all) + zone * bin(20) + x(life) + y(continent) +
  palette("viridis") + title("Life expectancy, a distribution per continent")
```

Python

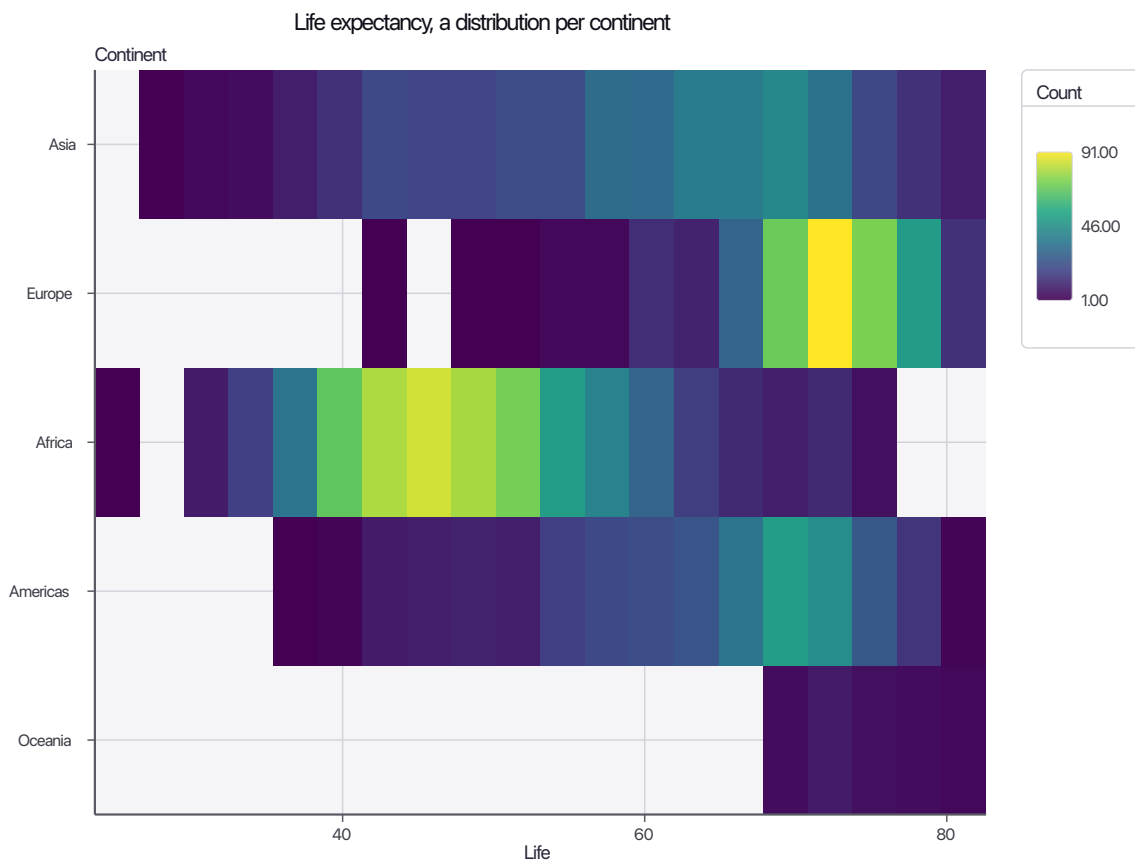
```
(data(gm_all) + zone * bin(20) + x(col.life) + y(col.continent) +
  palette("viridis") + title("Life expectancy, a distribution per
  ↪ continent"))
```

Julia

```
data(gm_all) + zone * bin(20) + x(:life) + y(:continent) +
  palette("viridis") +
  title("Life expectancy, a distribution per continent")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20)), x(col.life), y(col.continent),
  palette("viridis"),
  title("Life expectancy, a distribution per continent"))
```



That is a distribution shown as shade rather than as height, which is what makes five of them fit in the space one histogram would take. Read a row and it is a histogram lying on its side. Read the plot and the shape of each continent's spread is what stands out: Africa's mass sits far left and Europe's far right, Asia and the Americas are wide where Africa and Europe are concentrated, and Oceania is two countries and says so.

Two things about it are worth naming, because they are what make it a *mesh* and not five plots side by side. Every row is cut on **the same edges**, and the layout is computed once from the whole column, never per category, so a column of cells is a straight comparison and the boundaries line up all the way down. And an empty cell stays empty. Oceania's row starts where its data starts, rather than being painted the palest end of the ramp. The ragged left edge shows where each distribution reaches, not a claim that anyone lived to 25 there.

The mirror is the same sentence with the axes swapped, and it is the reading that stacks the distributions upright:

R

```
data(gm_all) + zone * bin(20) + y(life) + x(continent) + palette("viridis")
```

Python

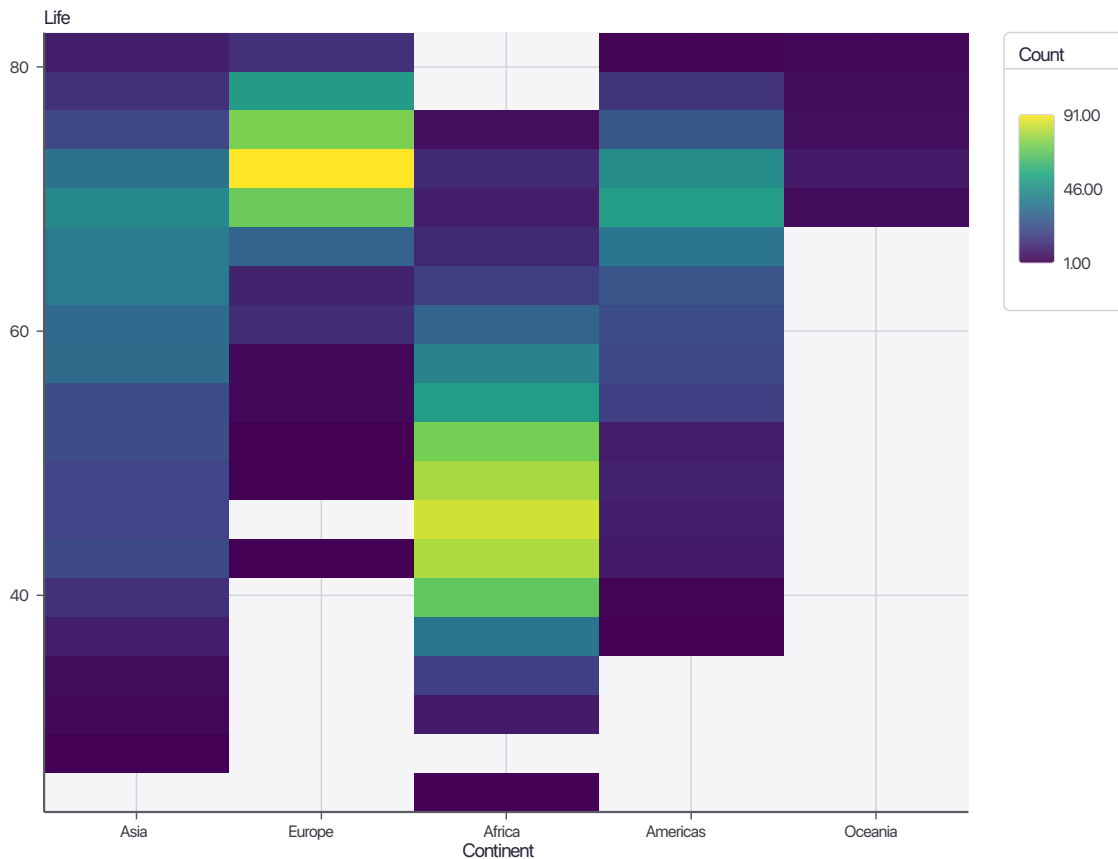
```
data(gm_all) + zone * bin(20) + y(col.life) + x(col.continent) +  
↪ palette("viridis")
```

Julia

```
data(gm_all) + zone * bin(20) + y(:life) + x(:continent) +  
palette("viridis")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20)), y(col.life), x(col.continent),  
palette("viridis"))
```



Nothing was added for either. Three cases, one rule read one axis at a time: cut both axes and it is the heatmap, cut neither and it is the tile plot, cut one and it is this.

19.11.5 Half a mesh: bounded by one slot

The rule was stated one axis at a time, so read it that way and a zone bounded on *one* categorical axis follows. It takes that slot and spans the panel on the other, which is what this mark has done from the beginning. One row is still one rectangle, so the table you give it is the list of slots to shade:

R

```
prevailing <- data.frame(direction = c("SW", "W"))

data(winds) + point * jitter + x(direction) + y(speed) +
  data(prevailing) + zone + x(direction) + style(color = "goldenrod") +
```



```
data(winds) + point * jitter + x(direction) + y(speed) +
y_label("Speed") + title("The two directions the wind actually comes from")
```

Python

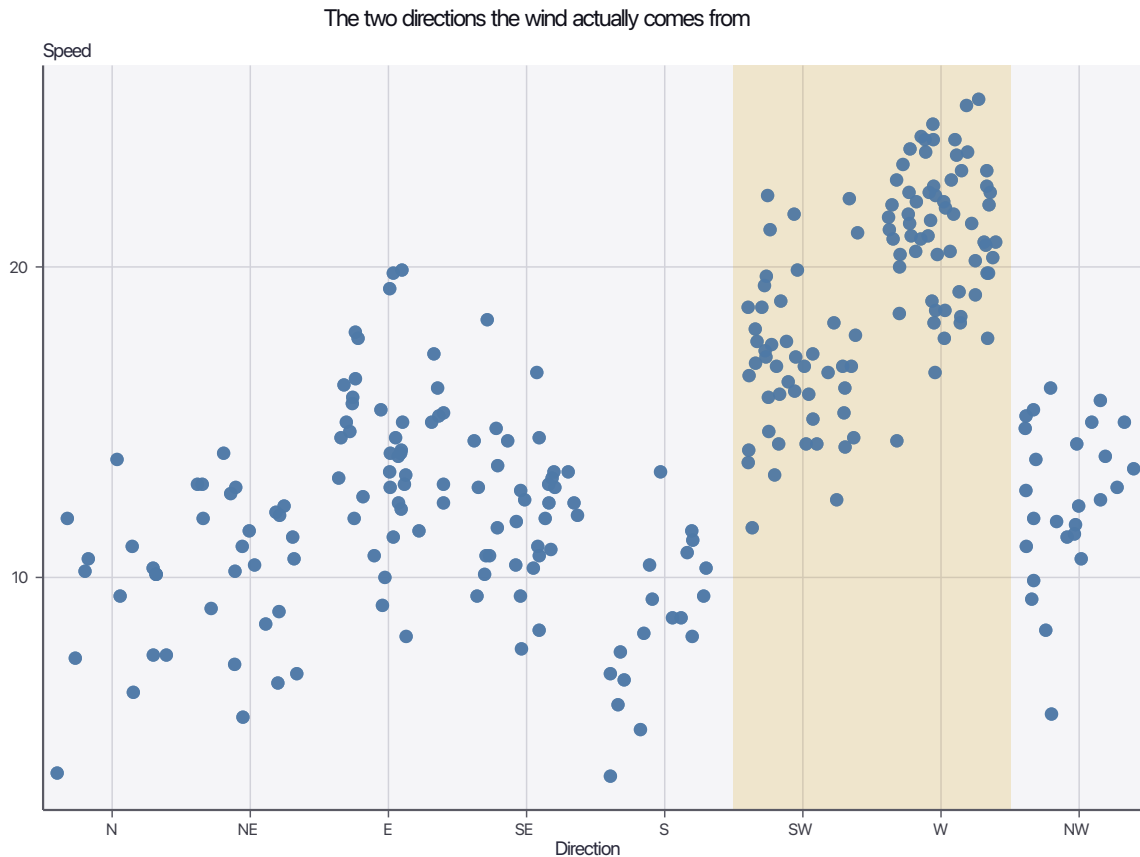
```
prevailing = {"direction": ["SW", "W"]}
(data(winds) + point * jitter + x(col.direction) + y(col.speed) +
 data(prevailing) + zone + x(col.direction) + style(color = "goldenrod") +
 data(winds) + point * jitter + x(col.direction) + y(col.speed) +
 y_label("Speed") + title("The two directions the wind actually comes
↪ from"))
```

Julia

```
prevailing = (direction = ["SW", "W"],)
data(winds) + point * jitter + x(:direction) + y(:speed) +
 data(prevailing) + zone + x(:direction) + style(color = "goldenrod") +
 data(winds) + point * jitter + x(:direction) + y(:speed) +
 y_label("Speed") +
 title("The two directions the wind actually comes from")
```

JavaScript

```
const prevailing = { direction: ["SW", "W"] };
plot(data(winds), layer(point, jitter), x(col.direction), y(col.speed),
 data(prevailing), zone, x(col.direction), style({ color: "goldenrod" }),
 data(winds), layer(point, jitter), x(col.direction), y(col.speed),
 y_label("Speed"),
 title("The two directions the wind actually comes from"))
```



“Given the winds: points derived by jitter, x is direction, y is speed; then given the prevailing table: zones, x is direction; then given the winds: points derived by jitter, x is direction, y is speed.”

Nothing was built for that. It is the same sentence as the tile plot with one axis left unsaid, which is the test that the fourth source is a rule rather than a special case for one chart.

The two-row table is doing the work [rule](#)’s table does, and for the same reason: **a zone’s position is a column**, so one table shades as many slots as it has rows. Hand it the whole **winds** table instead and you get one rectangle per observation, hundreds of translucent golds stacked into eight solid columns. That is the rule behaving exactly as stated, and not the plot you wanted.

One difference to see rather than be told: the column highlight is translucent and the calendar heatmap is not, for the reason the settings section states in full. `style(opacity = )` overrides either.

19.12 Sides the data drew: the choropleth

Every section above finds a zone's sides in the same place: **the row being drawn**. **bounds** names them, **bin** and **density** cut them, and a category owns a slot. There is a fifth source, and it is the only one that spans many rows.

A boundary is a list of points around the edge of a region: a coastline, a border, the outline of a sales territory. It arrives as one row per point, so a single shape needs hundreds of rows, and the sentence needs a word for which rows belong together. That word is **group**:

Table 19.6: `world_borders`: first 5 of 4150 rows

lon	lat	country	continent	piece
66.519	37.363	Afghanistan	Asia	p001
69.196	37.151	Afghanistan	Asia	p001
70.807	38.486	Afghanistan	Asia	p001
71.845	36.738	Afghanistan	Asia	p001
73.260	37.495	Afghanistan	Asia	p001

R

```
data(world_borders) + zone + x(lon) + y(lat) + group(country) +  
  color(continent) + map()
```

Python

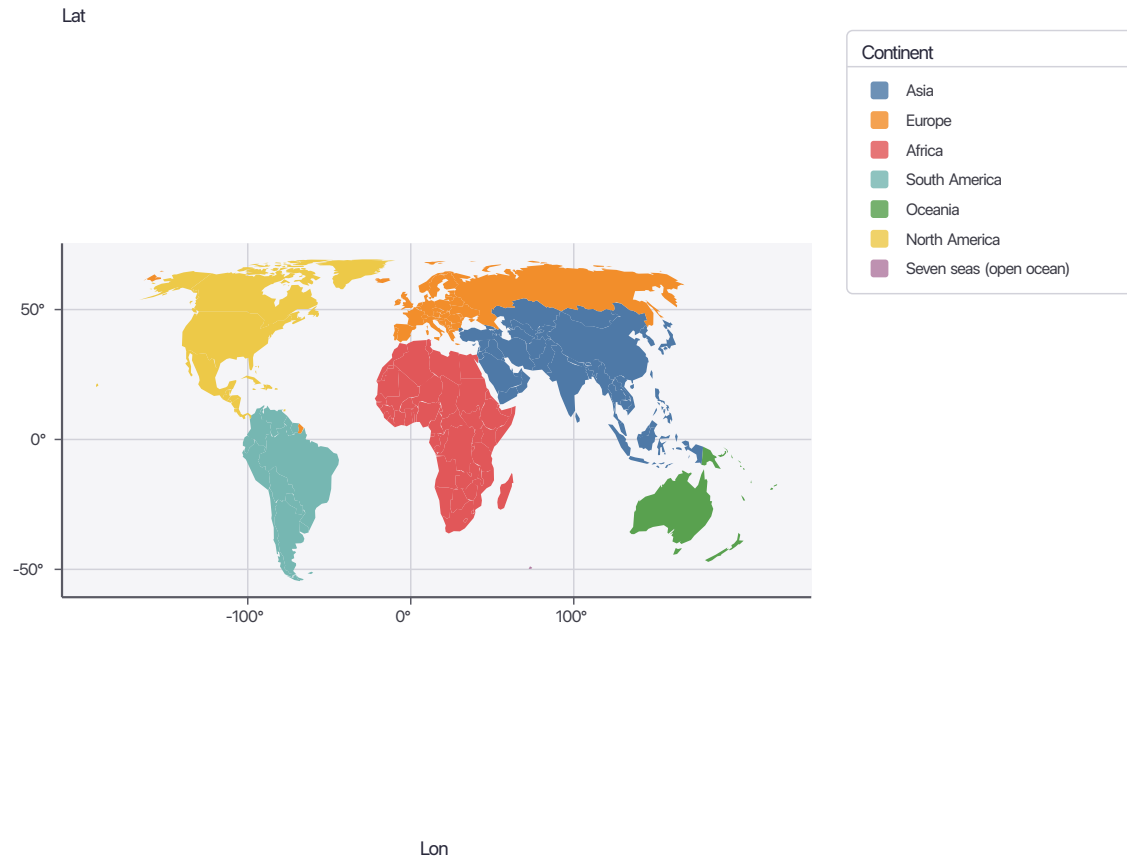
```
(data(world_borders) + zone + x(col.lon) + y(col.lat) + group(col.country) +  
  color(col.continent) + map())
```

Julia

```
data(world_borders) + zone + x(:lon) + y(:lat) + group(:country) +  
  color(:continent) + map()
```

JavaScript

```
plot(data(world_borders), zone, x(col.lon), y(col.lat),  
  group(col.country), color(col.continent), map())
```



“Given the world borders: zones, x is lon, y is lat, grouped by country, color by continent, in the map space.”

That is the **choropleth**, and it lives in the **map** space, where the two positions are longitude and latitude. The mark did not change and no atom was added: this mark was never about rectangles. It has always been about naming an extent.

19.13 What you can set

Everything a zone can be told to look like, generated from the engine’s own legality table, so this page cannot differ from what **style()** accepts:

Setting	Value
style(color =)	any CSS color name or hex
style(opacity =)	0 to 1

Setting	Value
<code>style(pattern =)</code>	<code>solid</code> , <code>hatch</code> , <code>crosshatch</code> , <code>grid</code> , <code>dots</code>
<code>style(border_color =)</code>	any CSS color name or hex
<code>style(border_size =)</code>	pixels

And these vary per row if you map them to a column instead: `color()` (either), `pattern()` (categories), `group()` (categories), `play()` (either).

A zone is a filled region, so those five are exactly what a fill takes, and no more. One of them has a default worth knowing: `opacity` is 0.2 for a zone you placed and 1 for one whose cells a mesh made. That covers cells `bin` cut, cells `density` estimated, and the slots two categories own. A placed zone is drawn *under* your data and has to let it show through; a cut zone *is* the data, with nothing behind it to reveal.

All three at once, one per band:

R

```
early <- data.frame(start = 2006.0, end = 2009.0)
middle <- data.frame(start = 2011.0, end = 2014.0)
late <- data.frame(start = 2017.0, end = 2020.0)

data(quarterly) + x(year) + y(sales) +
  data(early) + zone * bounds(start = start, end = end) +
    style(color = "seagreen") +
  data(middle) + zone * bounds(start = start, end = end) +
    style(color = "indianred", opacity = 0.55) +
  data(late) + zone * bounds(start = start, end = end) +
    style(color = "steelblue", pattern = "hatch", opacity = 0.7) +
  data(quarterly) + line + point +
  y_label("Sales") + title("Color, then opacity, then a hatch")
```

Python

```
early = {"start": [2006.0], "end": [2009.0]}
middle = {"start": [2011.0], "end": [2014.0]}
late = {"start": [2017.0], "end": [2020.0]}
(data(quarterly) + x(col.year) + y(col.sales) +
  data(early) + zone * bounds(start=col.start, end=col.end) +
    style(color = "seagreen") +
  data(middle) + zone * bounds(start=col.start, end=col.end) +
    style(color = "indianred", opacity = 0.55) +
```

```

data(late) + zone * bounds(start=col.start, end=col.end) +
  style(color = "steelblue", pattern = "hatch", opacity = 0.7) +
data(quarterly) + line + point +
y_label("Sales") + title("Color, then opacity, then a hatch"))

```

Julia

```

early = (start = [2006], end = [2009],)
middle = (start = [2011], end = [2014],)
late = (start = [2017], end = [2020],)
data(quarterly) + x(:year) + y(:sales) + data(early) +
  zone * bounds(start = :start, var"end" = :end) +
  style(color = "seagreen") + data(middle) +
  zone * bounds(start = :start, var"end" = :end) +
  style(color = "indianred", opacity = 0.55) + data(late) +
  zone * bounds(start = :start, var"end" = :end) +
  style(color = "steelblue", pattern = "hatch", opacity = 0.7) +
data(quarterly) + line + point + y_label("Sales") +
title("Color, then opacity, then a hatch")

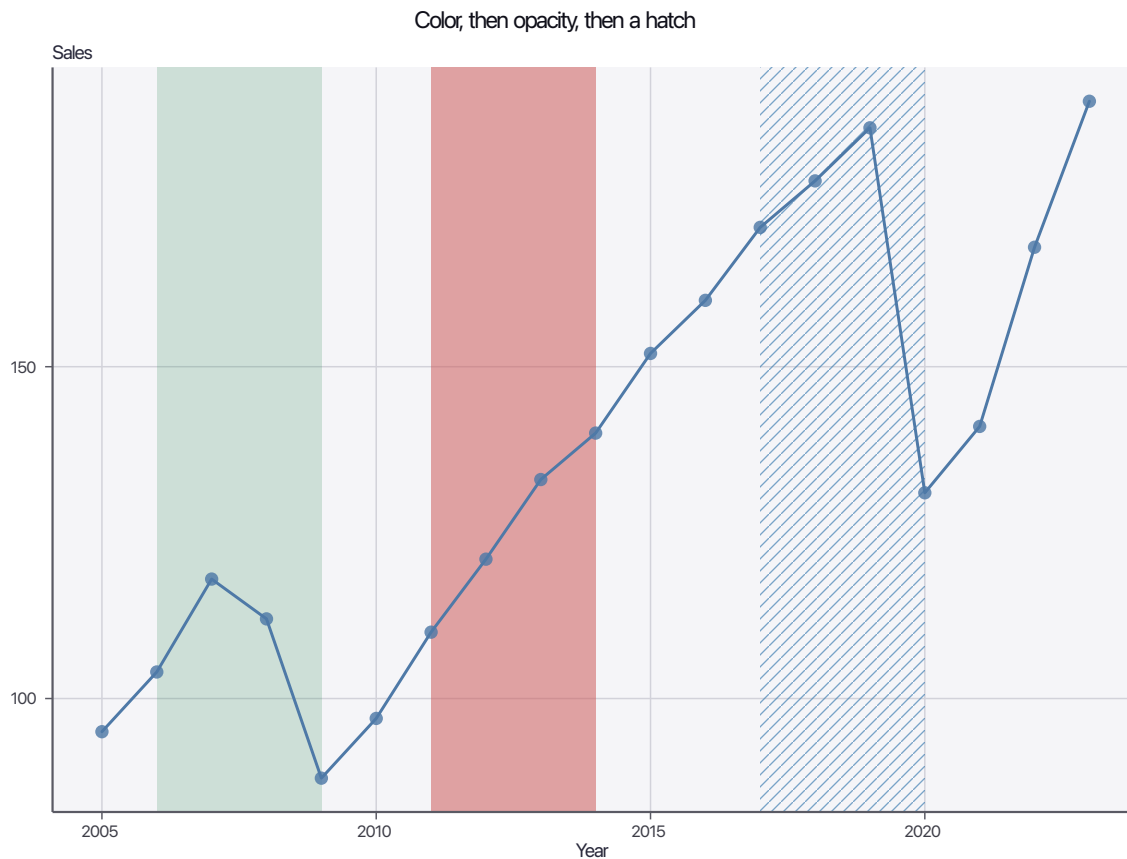
```

JavaScript

```

const early = { start: [2006], end: [2009] };
const middle = { start: [2011], end: [2014] };
const late = { start: [2017], end: [2020] };
plot(data(quarterly), x(col.year), y(col.sales), data(early),
  layer(zone, bounds({ start: col.start, end: col.end })),
  style({ color: "seagreen" }), data(middle),
  layer(zone, bounds({ start: col.start, end: col.end })),
  style({ color: "indianred", opacity: 0.55 }), data(late),
  layer(zone, bounds({ start: col.start, end: col.end })),
  style({ color: "steelblue", pattern: "hatch", opacity: 0.7 }),
data(quarterly), line, point, y_label("Sales"),
title("Color, then opacity, then a hatch"))

```



“Given quarterly: x is year, y is sales, an early zone from start to end colored seagreen, and also a middle zone from start to end colored indianred, with opacity 0.55, and also a late zone from start to end colored steelblue, with pattern hatch and opacity 0.7, and also a line and points.”

The default opacity is what makes the first band readable without your saying anything: a highlight that hides the data is not a highlight. Raise it when the zone is the subject of the plot rather than its background, as the red band does here.

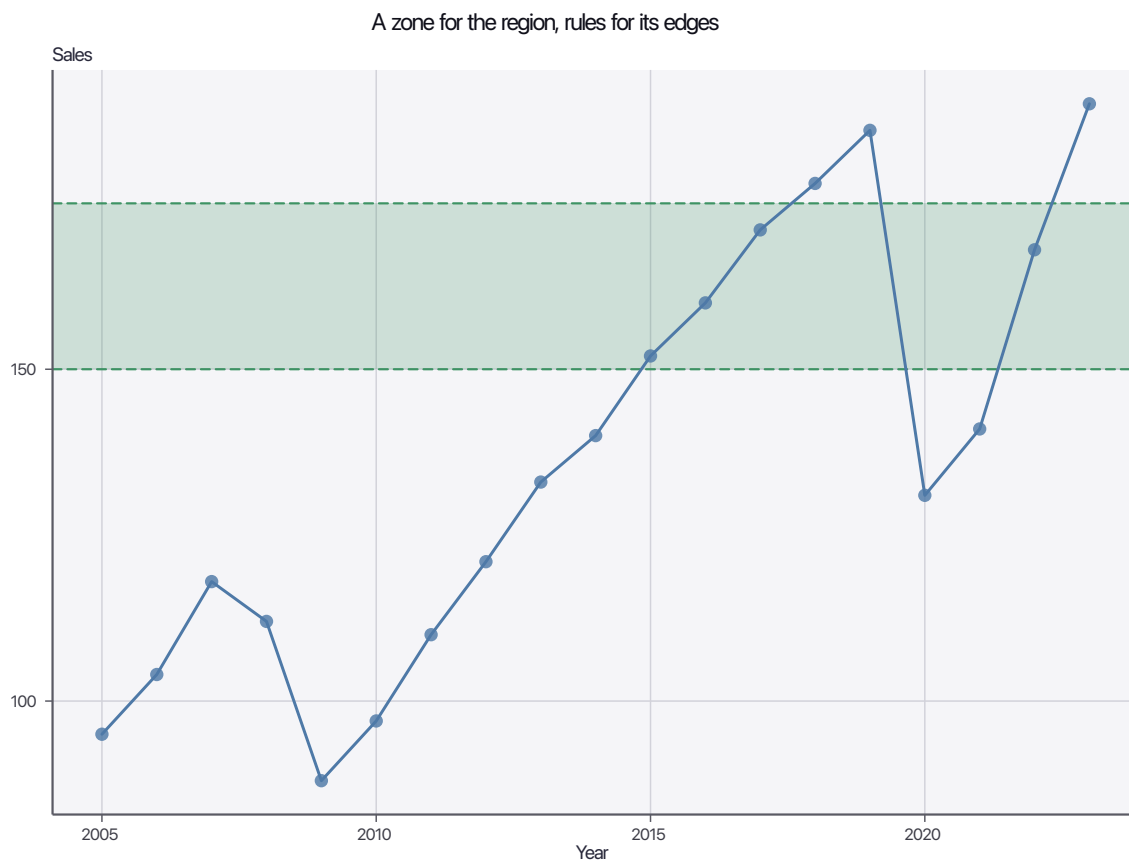
The hatch is worth using when a plot will be printed in gray, or when two overlapping zones need more than color to separate them. Give it more opacity than a flat fill wants, as the blue band does: a hatch is line-work with gaps between the lines, so fading it to 0.2 leaves very little to see. It takes the *fill* textures, not a stroke’s dashes; `style(pattern = "dashed")` on a zone is refused, pointing at the four textures instead.

One setting you might expect is refused, and the refusal says something about the mark: `style(size = )`, because a zone’s extent *is* its bounds and a size would be a second, contradictory answer to how big it is.

A **border** has two good answers, and both are available. A zone highlighting a region behind a line reads better with no border around it. Two rules draw better edges: dashed, colored per row, and real positions rather than decoration:

```
edges <- data.frame(sales = c(target_band$lower, target_band$upper))

data(quarterly) + x(year) + y(sales) +
  data(target_band) + zone * bounds(lower, upper) + style(color = "seagreen")
↪ +
  data(edges) + rule + style(color = "seagreen", pattern = "dashed") +
  data(quarterly) + line + point +
  y_label("Sales") + title("A zone for the region, rules for its edges")
```



“Given quarterly: x is year, y is sales, a target zone from lower to upper, and also rules from the edges, and also a line and points.”

The two marks read their positions differently, which is why the edge table is derived rather than reused. A zone is told which columns hold its sides (`bounds(lower, upper)`), while a rule reads whichever of the plot's own position columns its table carries, so the edges have to arrive as a `sales` column. Building that column *from* `target_band` is what keeps the two from drifting apart when the band moves.

But the mark is not only a highlight. It draws the waterfall, the heatmap's cells, [the icicle](#) and the [mosaic](#), and in every one of those the zone *is* the data rather than a background behind it. There the composition above stops being an answer: a region has four sides, a mosaic has as many regions as the table has rows, and two rules per cell hand-placed is not a sentence anyone writes. So a zone takes `style(border_color = , border_size = )` like every other closed-glyph fill:

R

```
data(quarterly) + x(year) + y(sales) +  
  data(recessions) + zone * bounds(start = start, end = end) +  
    style(border_color = "black") +  
  data(quarterly) + line +  
  y_label("Sales") + title("A zone with a border of its own")
```

Python

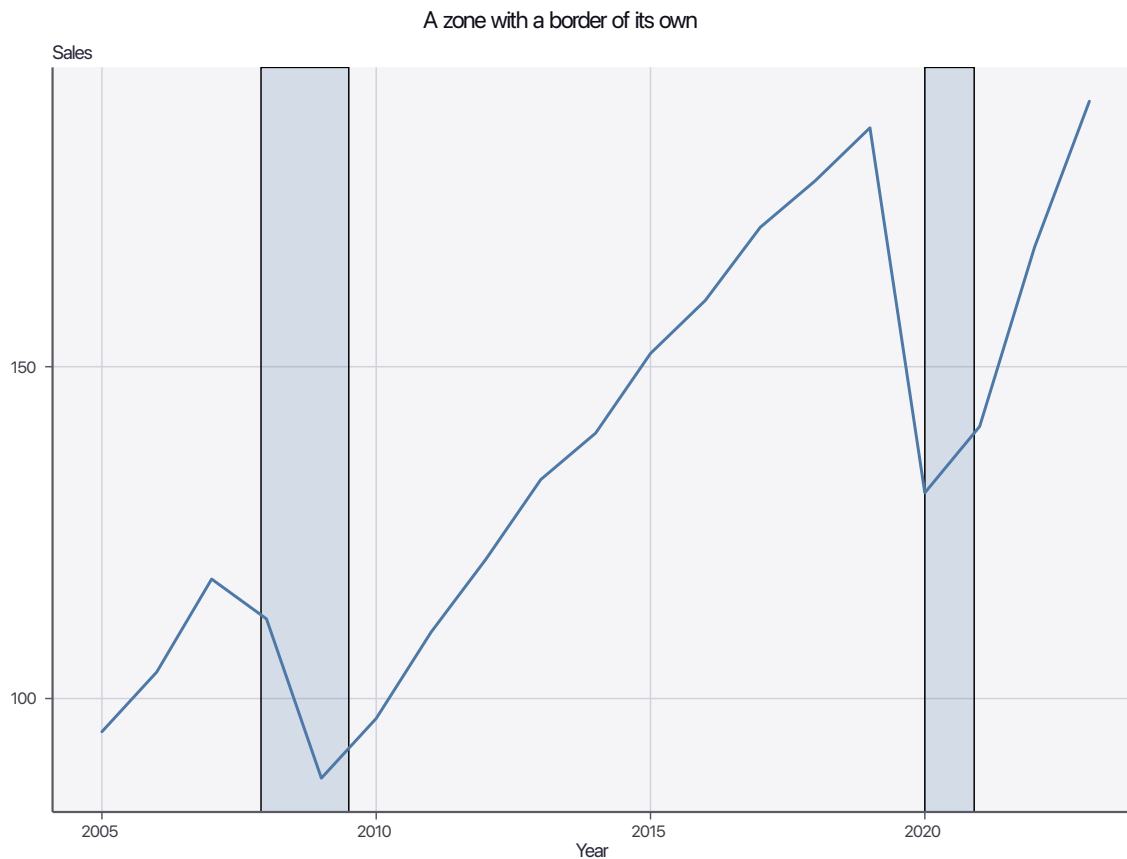
```
(data(quarterly) + x(col.year) + y(col.sales) +  
  data(recessions) + zone * bounds(start=col.start, end=col.end) +  
    style(border_color = "black") +  
  data(quarterly) + line +  
  y_label("Sales") + title("A zone with a border of its own"))
```

Julia

```
data(quarterly) + x(:year) + y(:sales) + data(recessions) +  
  zone * bounds(start = :start, var"end" = :end) +  
  style(border_color = "black") + data(quarterly) + line +  
  y_label("Sales") + title("A zone with a border of its own")
```

JavaScript

```
plot(data(quarterly), x(col.year), y(col.sales), data(recessions),  
  layer(zone, bounds({ start: col.start, end: col.end })),  
  style({ border_color: "black" }), data(quarterly), line,  
  y_label("Sales"), title("A zone with a border of its own"))
```



“Given quarterly: x is year, y is sales, recession zones from start to end, with border color black, and also a line.”

Both readings stay available, and which one is right depends on whether the zone is the background or the subject. A zone you do not ask for a border draws no stroke: with no `border_color` and no `border_size` there is none at all.

19.14 What it refuses

A zone with nothing to bound it has no sides:

```
data(quarterly) + x(year) + y(sales) + zone + line
```

Error:

! gog: `zone` shades a rectangle, but nothing here says where its sides are.

- ↪ Four things can: a categorical position, whose category owns a slot -
- ↪ `zone + x(method) + y(dataset) + color(score)` fills every cell where two
- ↪ categories cross; `bounds`, which names the sides from columns you hold -
- ↪ `zone \* bounds(lo, hi)` on the measure axis, `zone * bounds(start = a,
- ↪ end = b)` on the domain axis, all four for a box, and the axis you leave
- ↪ out spans the panel; `bin`, which cuts them out of two continuous axes
- ↪ and counts the rows in each cell; and `density`, which cuts the same
- ↪ cells and estimates a value at each. A continuous position on its own is
- ↪ none of them - a number is a point, and a point has no width.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

And half a pair is not a side. The refusal names the half you gave, because that is nearly always a typo rather than a misunderstanding:

```
data(quarterly) + x(year) + y(sales) +
  data(target_band) + zone * bounds(lower = lower) +
  line
```

Error:

! gog: `zone` was given `lower` but not the other half of any pair. A

- ↪ rectangle needs both ends of a side: `bounds(lower, upper)`,
- ↪ `bounds(start, end)`, or all four.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

The *start/end* pair belongs to a rectangle and to nothing else. A band spans the measure axis *at each position*, so it has no extent along the domain to bound:

```
slumps <- data.frame(year = 2008.0, sales = 100.0,
                     start = 2007.5, end = 2009.5)
data(slumps) + x(year) + y(sales) +
  ribbon * bounds(start = start, end = end)
```

Error:

! gog: `bounds(start, end)` bounds a rectangle along the domain axis, and a

- ↪ `ribbon` has no extent there - it spans the measure axis at each
- ↪ position. Keep `bounds(lower, upper)`, or use `zone` to shade a
- ↪ rectangle.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A zone says where its cells are and what is in them, and a transform can supply either or both. *bounds* names the sides and measures nothing. *count* tallies into the cells your categories already are, and says nothing about where those are. *density* estimates a field and samples

it. `bin` cuts cells out of a continuous axis and supplies both, which is why it is also the one that can supply just one of them. `proportion` answers neither question: it divides whatever measurement is there by its total, so it stands after any of them. Two transforms answering the same half is a contradiction, so a rectangle whose sides you named, one per row, has no mesh for a summary to group into:

```
data(quarterly) + x(year) + y(sales) +  
  data(target_band) + zone * bounds(lower, upper) * mean +  
  line
```

Error:

```
! gog: `zone * bounds * mean` says what this rectangle is twice - `bounds`  
  ↳ names its sides from columns you hold, one rectangle per row, and `mean`  
  ↳ summarizes a column within the cells your *positions* make. Keep  
  ↳ whichever you meant: `zone * bounds(...)` to shade a region you chose, or  
  ↳ `zone * mean + x(<a>) + y(<b>) + color(<column>)` to summarize one within  
  ↳ every cell two categories cross. To shade a band the data computed,  
  ↳ `ribbon * range` is the mark that spans a statistic.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

To shade a band the data *computed* rather than one you chose, the mark that spans a statistic is `ribbon`: `ribbon * range` fills between the minimum and maximum at each x. The division is the same one `rule` draws: a zone shades where you say, a ribbon shades what the data says.

Two measurements is the same contradiction on the other half. `count` invents one and `mean` reduces the column you named, and a cell holds a single number. `count` cuts nothing, so with `mean` measuring the cells it has no work left:

```
data(winds) + zone * count * mean + x(direction) + y(season) + color(speed)
```

Error:

```
! gog: `zone * count * mean` measures each cell twice - `count` supplies only  
  ↳ a measurement: its cells are the slots the positions already own, so with  
  ↳ `mean` measuring them too the cell is measured twice and `count` has  
  ↳ nothing left to contribute. Keep whichever you meant: `zone * count` to  
  ↳ measure what `count` computes, or `zone * mean + color(<column>)` to  
  ↳ reduce the column you name. To cut a continuous axis into cells and  
  ↳ reduce a column inside each, `bin` is the transform that cuts without  
  ↳ keeping the measurement: `zone * bin * mean + x(<number>)`.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

`bin * mean` [above](#) has the same shape and is not a contradiction: `bin` supplies the cells, and `mean` supplies the measurement inside them.

Asking for both extents at once is refused rather than resolved by picking one, since the two would disagree the moment the mesh moved:

```
banded <- data.frame(gdp = 1000.0, life = 50.0, lower = 45.0, upper = 55.0)
data(banded) + x(gdp) + y(life) +
  zone * bounds(lower, upper) * bin
```

Error:

```
! gog: `zone * bounds * bin` says where the sides are twice - `bounds` names
  ↳ them from columns you have, `bin` cuts them from the data. Keep whichever
  ↳ you meant: `zone * bounds(...)` to shade a rectangle you chose, `zone *
  ↳ bin` to tile the panel with measured cells.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A `bin` needs *something to cut*, and two categorical axes leave it nothing: a category is one slot, with no width. The refusal does not stop there, though: a cell per pair of *categories* is the tile plot, which does no binning at all, so it names the transform that does tally into cells you already have:

```
data(six_weeks) + zone * bin + x(weekday) + y(week)
```

Error:

```
! gog: `zone * bin` cuts an axis into cells, and there is no axis here it can
  ↳ cut - `x(weekday)` is categorical, and so is the other one. A category is
  ↳ one slot, with no width to cut. To tally rows into the cells two
  ↳ categorical axes already make, `count` is the transform that does it:
  ↳ `zone * count + x(<a>) + y(<b>)` draws a cell per pair, colored by how
  ↳ many rows fell there. (With *one* categorical axis `zone * bin` draws the
  ↳ mixed mesh - the continuous axis cut into cells, one row of them per
  ↳ category.)
gog: `zone * bin` cuts an axis into cells, and there is no axis here it can
  ↳ cut - `y(week)` is categorical, and so is the other one. A category is
  ↳ one slot, with no width to cut. To tally rows into the cells two
  ↳ categorical axes already make, `count` is the transform that does it:
  ↳ `zone * count + x(<a>) + y(<b>)` draws a cell per pair, colored by how
  ↳ many rows fell there. (With *one* categorical axis `zone * bin` draws the
  ↳ mixed mesh - the continuous axis cut into cells, one row of them per
  ↳ category.)
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

One categorical axis is allowed, and it draws. That is the mixed mesh above. `density` is where the two differ, and it is not an oversight. A field is estimated *between* the data points, so it needs somewhere to spread on both axes. A density per category would be normalized inside each slot on its own, which makes its cells incomparable across slots while the color ramp claims otherwise. It is the *which margin normalizes* question again, and the answer here is the one `proportion` gives everywhere: a share is of the whole table, and a per-slot denominator is a different plot that has to be asked for. So a field asks for two continuous axes and names what to use instead:

```
data(gm_all) + zone * density + x(life) + y(continent)
```

Error:

```
! gog: `zone * density` estimates a density over the *plane*, and
  ↳ `y(continent)` is categorical - a category is one slot, with no interval
  ↳ for the estimate to spread along. Both axes must be continuous. To
  ↳ compare one continuous distribution across categories, `line * density +
  ↳ color(continent)` draws a curve per group.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A tiling is refused for a reason of its own, and that reason is distance. `hex` interleaves two lattices, weighing a step up against a step across, so it needs a *distance* on both axes; a category's slots are an order and no more. A mixed mesh has two axes and still no plane, and its cells are rectangles by construction:

```
data(gm_all) + zone * bin(tiling = "hex") + x(life) + y(continent)
```

Error:

```
! gog: `bin(tiling = "hex")` partitions a *plane*, and `y` here is
  ↳ categorical - its slots are an order, not a distance, so there is nothing
  ↳ for a hexagon to be regular against. This plot is the mixed mesh: one
  ↳ axis cut into cells, one row of them per category, and its cells are
  ↳ rectangles by construction. Drop the tiling, or bind both axes to numbers
  ↳ for `bin(tiling = "hex")` to have a plane to cut.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

And the mirror of the first refusal, which is the same rule read the other way. A tally needs cells that exist; a number is a point and owns none, so `count` sends you back to `bin`:

```
data(gm_all) + zone * count + x(gdp) + y(life)
```

Error:

! gog: `zone \* count` tallies rows into the cells that two **categorical** axes
 ↪ already make, and `x(gdp)` is continuous - a number is a point, and a
 ↪ point owns no cell. To cut a continuous axis into cells first, `bin` is
 ↪ the transform that does it: `zone \* bin` counts the rows in each, which
 ↪ is the heatmap.

gog: `zone \* count` tallies rows into the cells that two **categorical** axes
 ↪ already make, and `y(life)` is continuous - a number is a point, and a
 ↪ point owns no cell. To cut a continuous axis into cells first, `bin` is
 ↪ the transform that does it: `zone \* bin` counts the rows in each, which
 ↪ is the heatmap.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

And the cells' measure is already the count, so coloring by anything else asks for a column that binning has replaced:

```
data(gm_all) + zone * bin + x(gdp) + y(life) + color(continent)
```

Error:

! gog: `zone \* bin` already measures each cell by how many rows fell in it,
 ↪ and color is where that measurement goes - so `color(continent)` has
 ↪ nothing to read: the transform replaced those rows with its own. Drop the
 ↪ binding and the measurement colors the cells, or say `color(count)` to
 ↪ name it out loud. To compare across a category, facet on it: `|`
 ↪ continent`.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A mesh gog does not cut is refused with the list of the ones it does, rather than quietly drawing rectangles instead:

```
data(gm_all) + zone * bin(tiling = "octagon") + x(gdp) + y(life)
```

Error:

! gog: `octagon` is not a tiling. `bin(tiling = )` takes `"rect"` or `"hex"`.
 ↪ `"rect"` cuts equal-interval cells on each axis; `"hex"` staggers
 ↪ alternate rows, which stops the eye reading the mesh's own alignment as
 ↪ structure in the data.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

density's bandwidth is refused here for a reason of its own. It is a width in one column's own units, and a field spreads over two columns measuring different quantities, so one number cannot be a width in both. That is why `adjust` is the parameter the section above uses: a multiplier means the same thing on either axis.

```
data(iris_flowers) + zone * density(bandwidth = 0.5) +  
  x(sepal_length) + y(petal_length)
```

Error:

```
! gog: `density(bandwidth = )` is a width in one column's own units, and  
  ↳ `zone * density` spreads over *two* columns measuring different  
  ↳ quantities - one number cannot be a width in both. Use `density(adjust =  
  ↳ )`, which scales the automatic bandwidth on each axis by the same  
  ↳ dimensionless factor.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```


20 Surface

What shape does a landscape have, sampled on a grid? **surface** draws a sheet through your samples. It is the only mark that can be drawn in the cube by itself. Every other mark draws in the plane, and several of them also draw in space. A surface is a sheet through *three* positions, and there is no such thing without the third.

It is not **area** with a **z**. An area fills the space between a curve and a baseline. A surface has no baseline. Both are fills, and that is all they share.

20.1 A sheet through the samples

Here is Maunga Whau, a volcano in Auckland. The book's copy keeps every second row and column, a 20 m grid. Ross Ihaka digitized the elevations from a topographic map, and they reach this page through R's **datasets** package (R Core Team, 2026):

R

```
data(maunga_whau) + surface +  
  x(east) + y(north) + z(elevation) +  
  z_label("Elevation (m)") + title("Maunga Whau, at 20 m")
```

Python

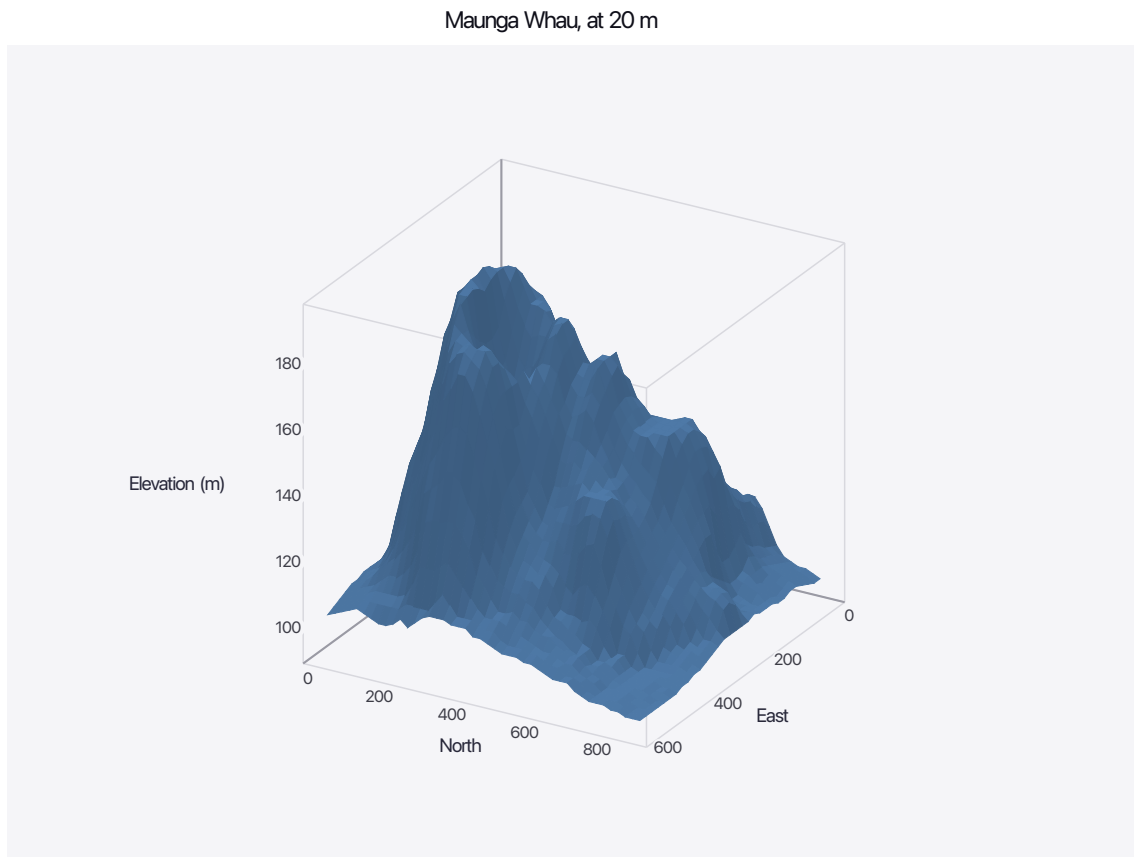
```
(data(maunga_whau) + surface +  
  x(col.east) + y(col.north) + z(col.elevation) +  
  z_label("Elevation (m)") + title("Maunga Whau, at 20 m"))
```

Julia

```
data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +  
  z_label("Elevation (m)") + title("Maunga Whau, at 20 m")
```

JavaScript

```
plot(data(maunga_whau), surface, x(col.east), y(col.north),
     z(col.elevation), z_label("Elevation (m)",
     title("Maunga Whau, at 20 m"))
```



“Given Maunga Whau: a surface, x is east, y is north, z is elevation.”

Three positions and a mark. There is no `space()` in that sentence and none is needed: binding z is what puts a plot in the cube, exactly as it is for a [3-D scatter](#). `space(turn =, tilt =)` still sets the angle when you want a different one.

20.2 One row per crossing

Given two position columns and no transform, a surface reads its rows as **nodes**, and the sheet is the faces between them. You never declare which nodes are neighbors. `gog` recovers that

from your two position columns: the distinct `east` values become the mesh's vertical grid lines and the distinct `north` values its horizontal ones. A face is drawn wherever all four corners of a cell are present.

So the table this reading wants is one row per (x, y) crossing: every `x` value paired with every `y` value.

```
head(maunga_whau, 4)
```

	east	north	elevation
1	20	20	100
2	20	40	102
3	20	60	104
4	20	80	105

```
nrow(maunga_whau)
```

```
[1] 1364
```

1364 rows, 31 distinct `east` values and 44 distinct `north` values: $31 \times 44 = 1364$, so the grid is full, and $30 \times 43 = 1290$ faces get drawn. That arithmetic is the whole rule, and it is why a scatter cannot be a surface. Each of the 142 countries has its own `gdp` and its own `life`. That makes a grid 142 wide and 142 deep holding only 142 nodes, so no cell has all four corners. The sheet would be empty:

```
data(gapminder_2007) + surface + x(gdp) + y(life) + z(population)
```

Error:

```
! gog: `surface` found no complete cell to draw a face on - 142 rows over 142
  ↳ distinct `gdp` and 142 distinct `life` values, which is a scatter rather
  ↳ than a grid. A surface needs one row per (x, y) crossing, the shape
  ↳ `expand.grid()` makes. For a cloud of scattered points use `point` with
  ↳ `z(population)`; to estimate a field *from* them use `surface * density +
  ↳ x(gdp) + y(life) + space()`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

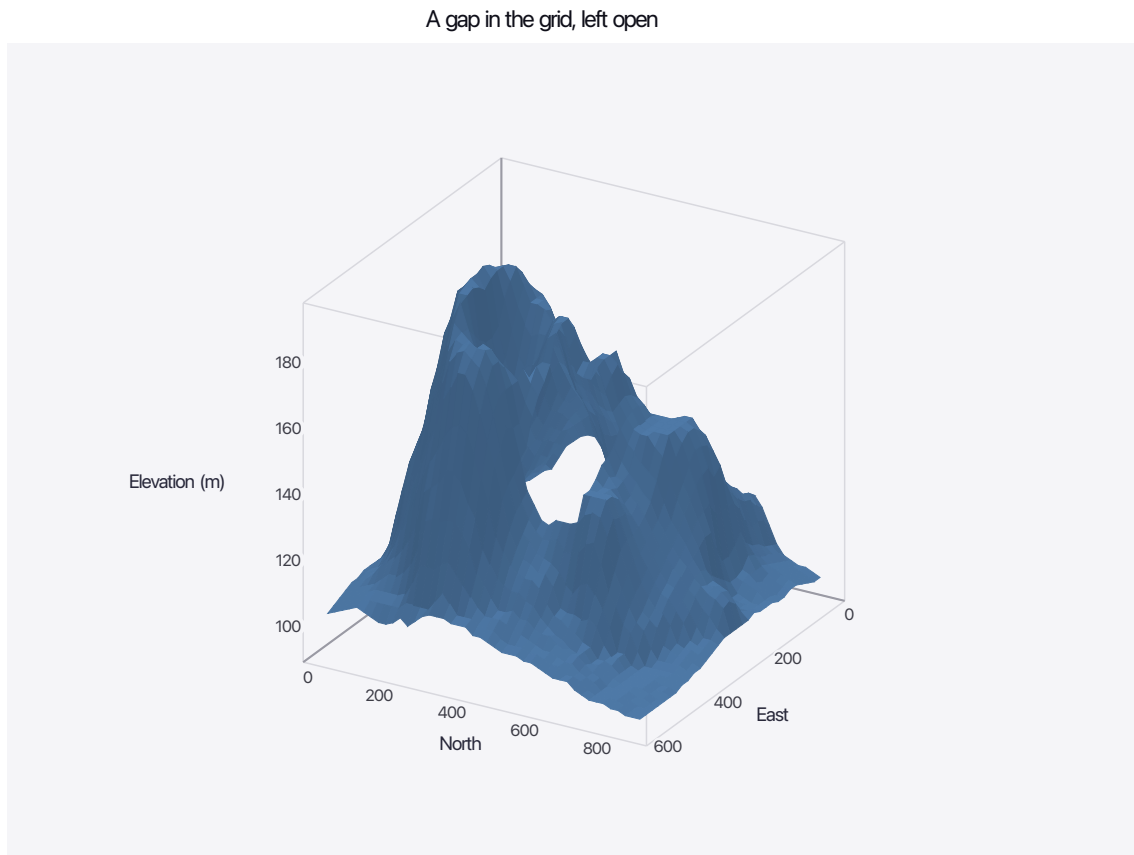
That refusal names the two sentences that *do* draw. The section on the third geometry of one field draws the second one.

Because a face needs all four of its corners, a gap in the grid is an **opening** in the sheet rather than a face drawn across it. Drop the crossings around the summit and the sheet opens there rather than covering the gap:

```
peak <- maunga_whau[!(maunga_whau$east > 300 & maunga_whau$east < 440 &
                      maunga_whau$north > 400 & maunga_whau$north < 560), ]

data(peak) + surface + x(east) + y(north) + z(elevation) +
  z_label("Elevation (m)") + title("A gap in the grid, left open")
```

gog: `surface` has 1322 of 1364 crossings filled, so the sheet is drawn with
 ↪ 42 open - a face needs all four of its corners. That is a true picture of
 ↪ a grid with gaps; if you expected a whole sheet, check that `east` and
 ↪ `north` repeat exactly across rows rather than to a rounded value.



The note above the plot counts the crossings gog did find: 1322 of 1364, so 42 are open. It is advice rather than a refusal, because a grid with gaps is a legitimate thing to have and a torn sheet is a true picture of one. What would not be legitimate is quietly spanning a gap, which asserts a height nobody measured. The note also tells you when the gaps were an accident. If

the two position columns hold rounded values instead of exact repeats, the `lattice` fills with holes nobody designed.

20.3 A continuous floor

Both position columns have to be numbers. A face spans the gap between two samples and so asserts every height *inside* that gap, and between one category and the next there is no inside to assert anything about:

```
data(gm_eras) + surface + x(continent) + y(year) + z(life)
```

Error:

```
! gog: `x(continent)` maps a categorical (text) column, but `x` on `surface`  
  ↳ needs a continuous (numeric) column. A face spans the gap between two  
  ↳ samples, and between two categories there is nothing to span. For a mesh  
  ↳ over categories use `bar * count + x(<a>) + y(<b>) + space()` - a column  
  ↳ stands in its own cell and claims nothing in between.
```

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

The direction there points at the other mark. A category owns a slot with air on either side of it, and a 3-D bar stands *in* its slot, claiming nothing about the space between cells. So `bar * count + x(a) + y(b) + space()` stands one bar over each pair of categories, and a surface cannot: tiles that float apart are not a sheet.

Numbers cut into cells are a different matter, and a surface takes those: the section on one value per cell, below. So the two marks differ over the **slot**, not over the cut. That is the same distinction the flat marks make between a category and a cut number, one dimension higher.

20.4 The third geometry of one field

A surface takes two transforms, and `density` is the one that makes it the third way `gog` draws a field. The same estimate over the same two columns, three times:

R

```
data(iris_flowers) + zone * density +  
  x(petal_length) + y(sepal_length) + title("As cells: zone")
```

Python

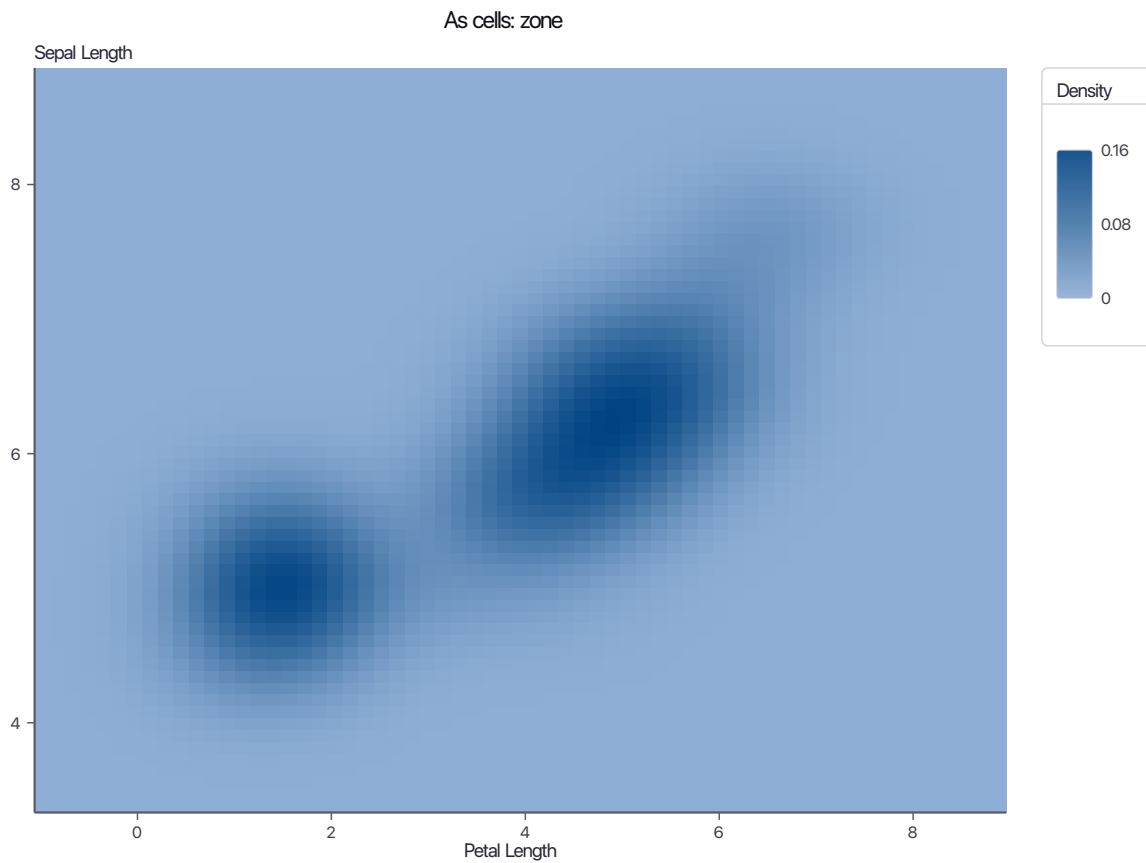
```
(data(iris_flowers) + zone * density +  
  x(col.petal_length) + y(col.sepal_length) + title("As cells: zone"))
```

Julia

```
data(iris_flowers) + zone * density + x(:petal_length) +  
  y(:sepal_length) + title("As cells: zone")
```

JavaScript

```
plot(data(iris_flowers), layer(zone, density), x(col.petal_length),  
  y(col.sepal_length), title("As cells: zone"))
```



“Given the iris flowers: zones derived by density, x is petal length, y is sepal length.”

R

```
data(iris_flowers) + path * density +  
  x(petal_length) + y(sepal_length) + title("As contours: path")
```

Python

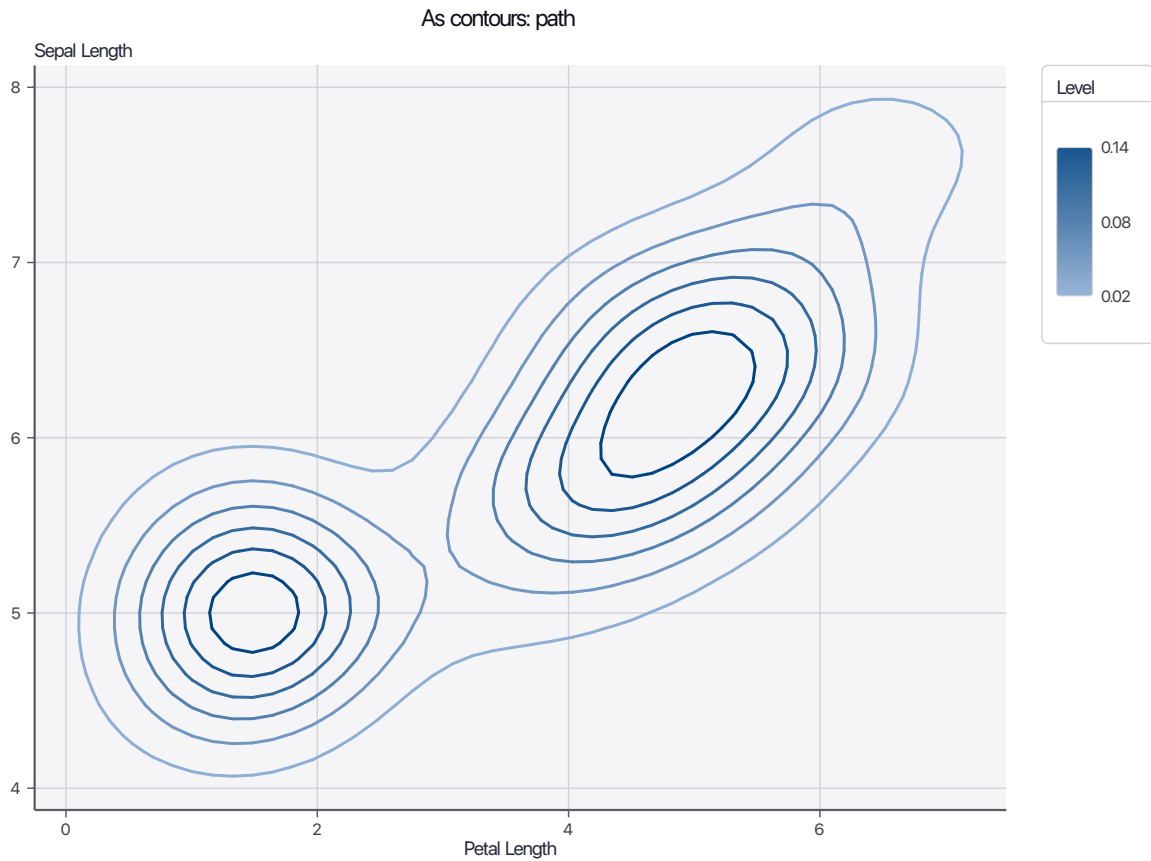
```
(data(iris_flowers) + path * density +  
  x(col.petal_length) + y(col.sepal_length) + title("As contours: path"))
```

Julia

```
data(iris_flowers) + path * density + x(:petal_length) +  
  y(:sepal_length) + title("As contours: path")
```

JavaScript

```
plot(data(iris_flowers), layer(path, density), x(col.petal_length),  
  y(col.sepal_length), title("As contours: path"))
```



“Given the iris flowers: paths derived by density, x is petal length, y is sepal length.”

R

```
data(iris_flowers) + surface * density +
  x(petal_length) + y(sepal_length) + space() +
  title("As a sheet: surface")
```

Python

```
(data(iris_flowers) + surface * density +
  x(col.petal_length) + y(col.sepal_length) + space() +
  title("As a sheet: surface"))
```

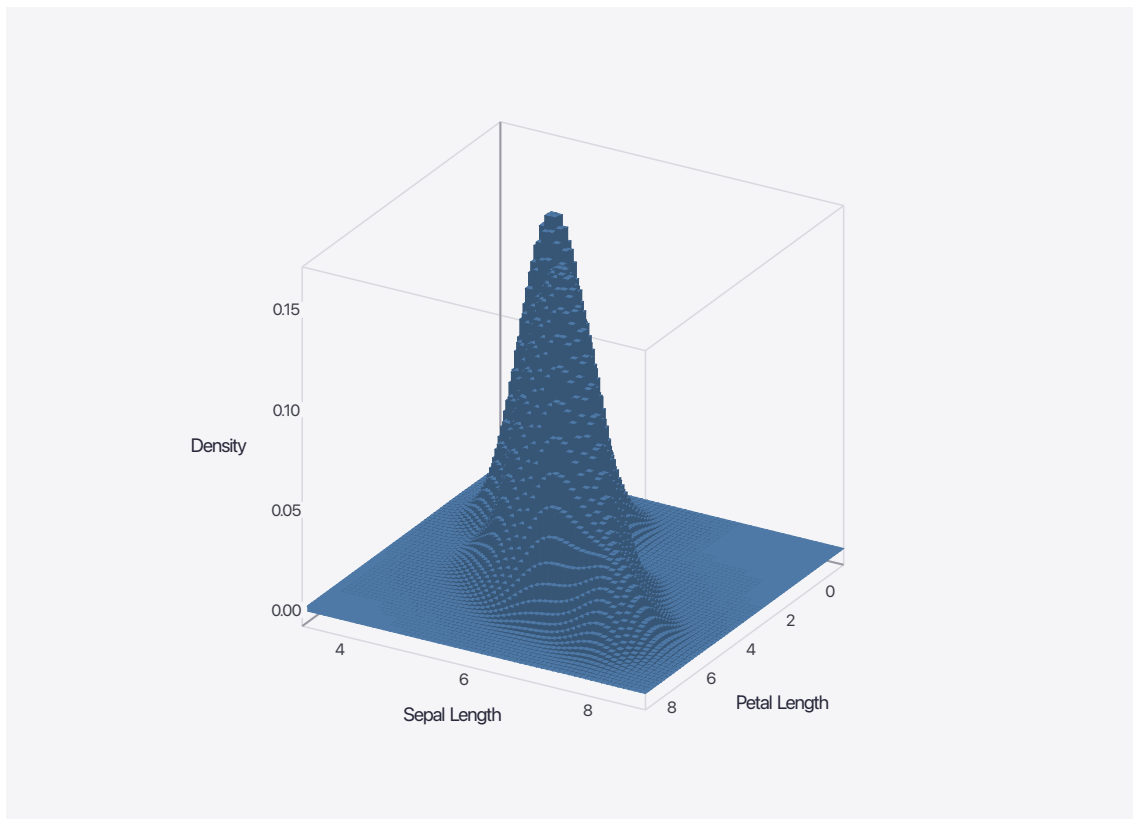
Julia


```
data(iris_flowers) + surface * density + x(:petal_length) +
  y(:sepal_length) + space() + title("As a sheet: surface")
```

JavaScript

```
plot(data(iris_flowers), layer(surface, density), x(col.petal_length),
  y(col.sepal_length), space(), title("As a sheet: surface"))
```

As a sheet: surface



“Given the iris flowers: a surface derived by density, x is petal length, y is sepal length, in the cube.”

Look at what the third one shows that the other two cannot: *how much* higher the tight cluster is than the broad one. Cells and contours show *where* the field is, and a height shows *how much*. The two clusters look equally large in the first two plots and very unequal in the third. Which of the three you want depends on what you are asking, not on which mark is better.

One transform, three marks, and the estimate itself is the same in all three. What differs is where the measurement goes: each mark puts it where its own geometry measures. A **zone** measures by color. A **path** measures by the level the contour was cut at. A surface measures by **height**. That is why the third sentence never binds `z()` and still gets a third axis, labeled Density: the transform supplies the measurement, and the mark puts it on the height. A flat `bar * bin` does the same for `y`.

Here the transform stays the same and the mark chooses the geometry. That is the rule that already made the contour and the estimated heatmap one feature instead of two.

`space()` is in that sentence and not in the volcano's, and the reason matters. A bound `z` says *there are three columns here*, which nothing else can mean. An invented height says only *this layer has a measure of its own*. `density` invents one in the plane too, so `space()` is what tells gog to put the answer on a third axis.

20.5 When each cell holds one value

`bin` is the other transform, and it draws a different sheet: flat across each cell, stepping at the boundary. Here is the same volcano, read as a coarse grid rather than a fine one:

R

```
data(maunga_whau) + surface * bin(12) * mean +  
  x(east) + y(north) + z(elevation) +  
  z_label("Elevation (m)") + title("Maunga Whau, one elevation per cell")
```

Python

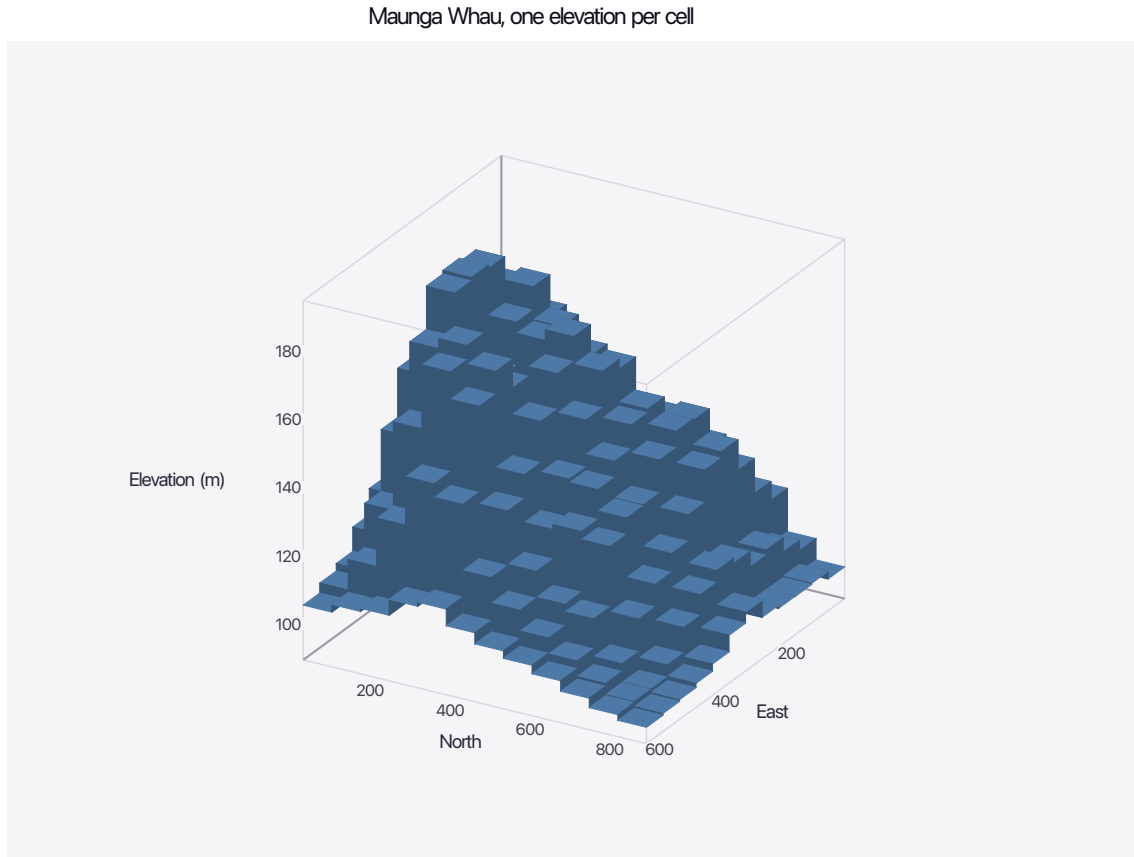
```
(data(maunga_whau) + surface * bin(12) * mean +  
  x(col.east) + y(col.north) + z(col.elevation) +  
  z_label("Elevation (m)") + title("Maunga Whau, one elevation per cell"))
```

Julia

```
data(maunga_whau) + surface * bin(12) * mean + x(:east) + y(:north) +  
  z(:elevation) + z_label("Elevation (m)") +  
  title("Maunga Whau, one elevation per cell")
```

JavaScript

```
plot(data(maunga_whau), layer(surface, bin(12), mean), x(col.east),
     y(col.north), z(col.elevation), z_label("Elevation (m)",
     title("Maunga Whau, one elevation per cell"))
```



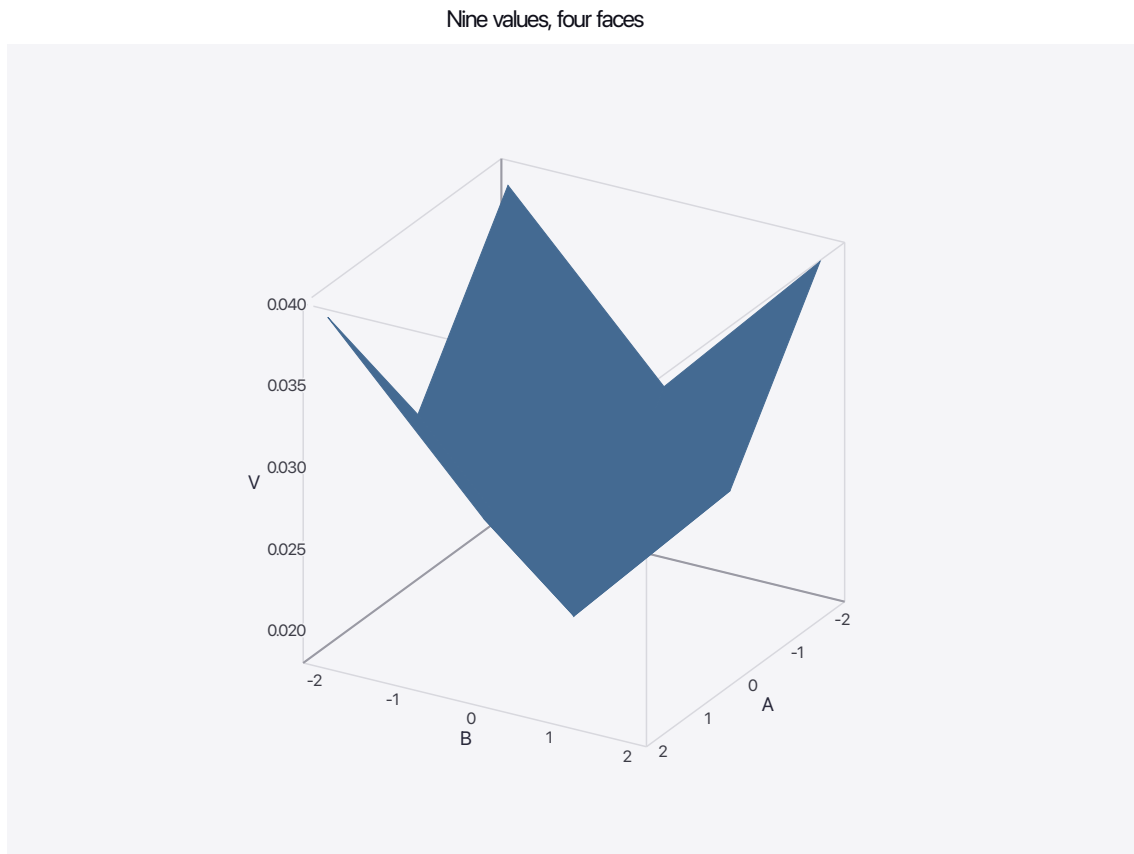
“Given Maunga Whau: a surface derived by bin into 12 and mean, x is east, y is north, z is elevation.”

Nothing about the mountain changed. What changed is the **claim**. The sheet at the top of this chapter says the ground rises smoothly between its samples; this one says each cell has a single elevation and nothing is asserted about the ground inside it. Both are honest about different data, and which you want is a question about how yours was measured.

That question has a clear answer whenever a design produces exactly one value per cell, which is the ordinary case in an experiment or a simulation: a grid of conditions, one number in each. Read as nodes, such a table draws a sheet smaller than the grid it describes. Nine values on a 3×3 grid make 2×2 blocks of four corners, so the sheet is four faces, and it stops at the outer samples instead of covering their cells:

```
cells <- expand.grid(a = c(-2, 0, 2), b = c(-2, 0, 2))
cells$v <- 0.019 + 0.0025 * (cells$a^2 + cells$b^2)

data(cells) + surface + x(a) + y(b) + z(v) + title("Nine values, four faces")
```



Nine numbers became four faces, and each face is a sheet stretched over the four corners around it. Cut the x-y plane into cells instead, and each cell gets a flat top of its own, a **plateau**:

R

```
data(cells) + surface * bin(3) * mean + x(a) + y(b) + z(v) +
  color(v) + palette("plasma") + title("Nine values, nine plateaus")
```

Python

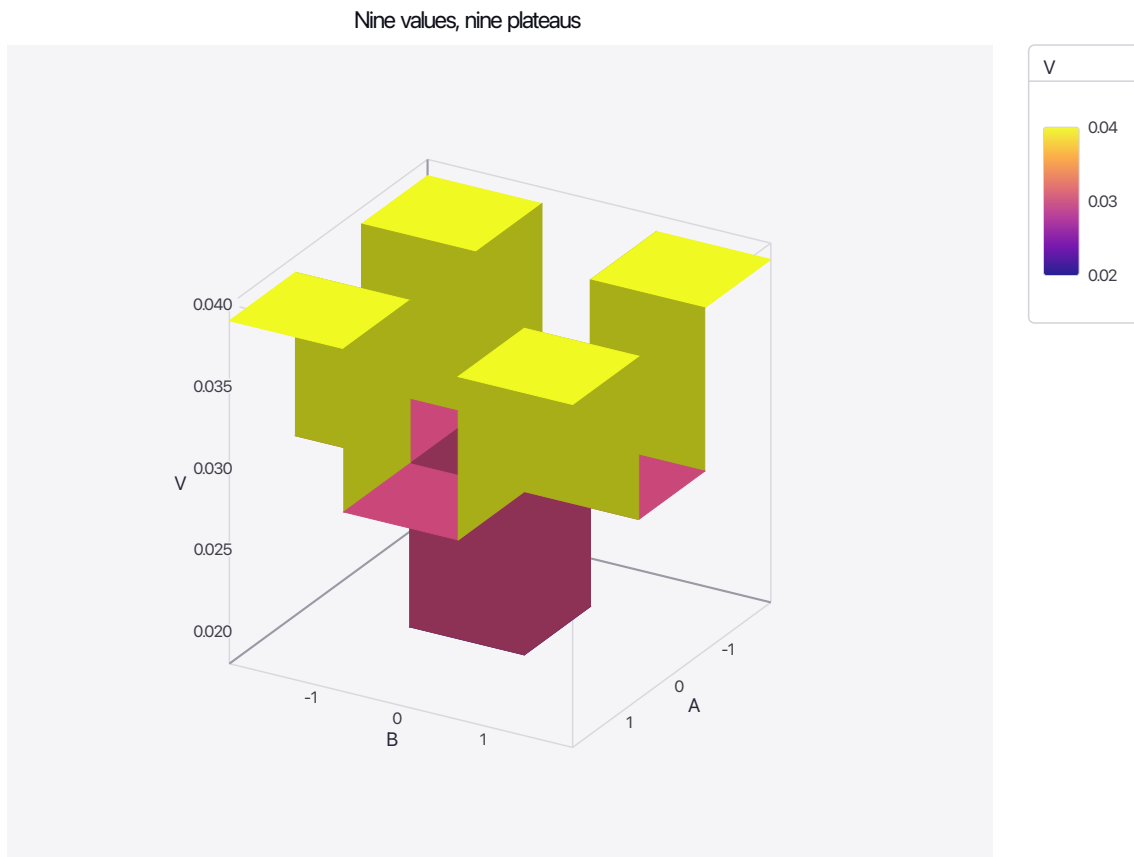
```
(data(cells) + surface * bin(3) * mean + x(col.a) + y(col.b) + z(col.v) +
  color(col.v) + palette("plasma") + title("Nine values, nine plateaus"))
```

Julia

```
data(cells) + surface * bin(3) * mean + x(:a) + y(:b) + z(:v) +
  color(:v) + palette("plasma") + title("Nine values, nine plateaus")
```

JavaScript

```
plot(data(cells), layer(surface, bin(3), mean), x(col.a), y(col.b),
  z(col.v), color(col.v), palette("plasma"),
  title("Nine values, nine plateaus"))
```



“Given the cells: a surface derived by bin into 3 and mean, x is a, y is b, z is v, color by v, with the plasma palette.”

Nine plateaus, and three colors. That formula makes a symmetric bowl, so the four corner cells share one height and the four edge cells share another. The ramp is what makes that symmetry visible, where the sheet above left it to be read off the axis.

bin cuts the **x** and **y** axes into cells, and **mean** averages the column named on **z** inside each one. That is the same sentence a 3-D bar uses. The transforms mean the same thing on both marks, and only the geometry laid on the cell differs. Where a table holds one row per cell, as this one does, the mean of a single value is that value, and the transform is doing nothing but naming the column.

The step between two plateaus is drawn, as a **riser** standing on the line the two cells share. That line has no width, so the riser asserts nothing about any area: it is simply what a jump looks like. Without it the mark would draw disconnected tiles floating at their own heights, and a reader could not tell a step from a missing cell.

A riser spans the *difference* between two neighbors, and that is what separates this from standing a 3-D bar on every cell. A bar runs the whole way down to the baseline, so a field of bars hides its own back rows as soon as you turn it. A riser is only as tall as its own step. Both are turned by 60 degrees below, so you can see which one is still legible. Both are set to one color, so the only thing that differs between them is the shape:

R

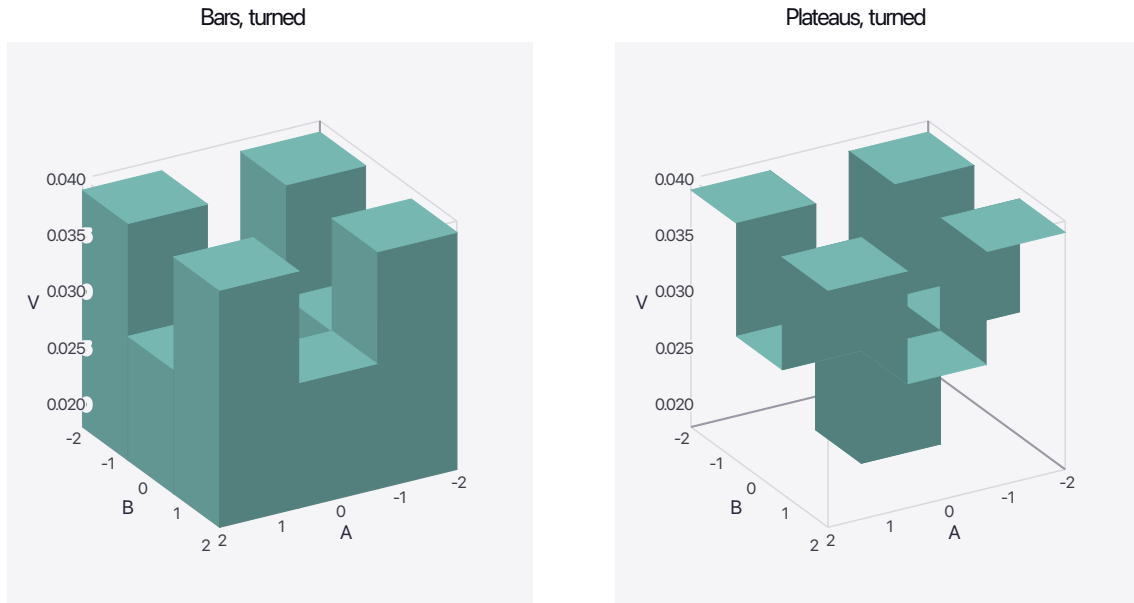
```
((data(cells) + bar * bin(3) * mean + x(a) + y(b) + z(v) +  
  space(turn = 60) + style(color = "#76b7b2") + title("Bars, turned")) |  
  (data(cells) + surface * bin(3) * mean + x(a) + y(b) + z(v) +  
    space(turn = 60) + style(color = "#76b7b2") + title("Plateaus,  
↪ turned")))) +  
  theme(height = 430)
```

Python

```
((data(cells) + bar * bin(3) * mean + x(col.a) + y(col.b) + z(col.v) +  
  space(turn = 60) + style(color = "#76b7b2") + title("Bars, turned")) |  
  (data(cells) + surface * bin(3) * mean + x(col.a) + y(col.b) + z(col.v) +  
    space(turn = 60) + style(color = "#76b7b2") + title("Plateaus,  
↪ turned")))) +  
  theme(height = 430))
```

Julia

```
((data(cells) + bar * bin(3) * mean + x(:a) + y(:b) + z(:v) +
  space(turn = 60) + style(color = "#76b7b2") + title("Bars, turned")) |
  (data(cells) + surface * bin(3) * mean + x(:a) + y(:b) + z(:v) +
  space(turn = 60) + style(color = "#76b7b2") +
  title("Plateaus, turned"))) + theme(height = 430)
```



“Given the cells: bars derived by bin into 3 and mean, x is a , y is b , z is v , beside a surface derived by bin into 3 and mean, x is a , y is b , z is v .”

| composes them ([Composition](#)), so the two are one figure and turn together. That is what makes this a fair comparison, rather than two pictures the reader has to match angle by angle.

The `theme(height = )` above sets the whole figure’s height in pixels, and two cubes side by side are why it is set. They share the page’s width, and each one keeps the full height. A cube is fitted with one scale on both axes, so it never stretches to fill a tall panel. Without the setting, blank space would be left above and below each cube.

Empty cells need no special rule and get none. `bin` leaves them out, because a cell with no rows is not a measurement of zero. A plateau claims only its own cell, so a cell with nothing in it is simply a cell with no top. It is the same opening a missing crossing makes in a node mesh, for a different reason.

What a surface refuses is a grid of **slots**. `count` and `proportion` count rows into the cells two categories make, and categories have gaps between them:

```
data(gm_eras) + surface * count + x(continent) + y(era) + space()
```

Error:

```
! gog: a `surface` is a sheet over a floor whose cells tile without gaps, and
  ↳ `count` does not give it one - `smooth` fits a curve along a domain a
  ↳ cell has none of, `range`/`confidence`/`bounds` give a low and a high
  ↳ where a cell holds one height, and `count`/`proportion` tally into the
  ↳ cells two *categories* make, which a surface refuses because slots leave
  ↳ air between them and disconnected tiles are not a sheet. A surface takes
  ↳ the two transforms that do tile: `bin` cuts the floor into adjacent cells
  ↳ and lays a flat lid on each - `surface * bin * mean + x(<a>) + y(<b>) +
  ↳ z(<column>)` reduces the column you name inside every cell - and
  ↳ `density` estimates a value at every node, which the sheet then
  ↳ interpolates between: `surface * density + x(<a>) + y(<b>) + space()`.
  ↳ Over categories, `bar` is the mark, where the column under each tile says
  ↳ which cell it belongs to.
gog: `x(continent)` maps a categorical (text) column, but `x` on `surface`
  ↳ needs a continuous (numeric) column. A face spans the gap between two
  ↳ samples, and between two categories there is nothing to span. For a mesh
  ↳ over categories use `bar * count + x(<a>) + y(<b>) + space()` - a column
  ↳ stands in its own cell and claims nothing in between.
gog: `y(era)` maps a categorical (text) column, but `y` on `surface` needs a
  ↳ continuous (numeric) column. A face spans the gap between two samples,
  ↳ and between two categories there is nothing to span. For a mesh over
  ↳ categories use `bar * count + x(<a>) + y(<b>) + space()` - a column
  ↳ stands in its own cell and claims nothing in between.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

20.6 The mesh, and what the shading means

`border_color` and `border_size` draw the **mesh lines**. A face is a closed fill like a bar's or a box's, so the [settable rule](#) gives it a border. On a mesh those borders draw the grid itself:

R

```
data(maunga_whau) + surface +
  x(east) + y(north) + z(elevation) +
  style(border_color = "white", border_size = 0.4) +
  z_label("Elevation (m)") + title("The mesh, drawn")
```

Python


```
(data(maunga_whau) + surface +
  x(col.east) + y(col.north) + z(col.elevation) +
  style(border_color = "white", border_size = 0.4) +
  z_label("Elevation (m)") + title("The mesh, drawn"))
```

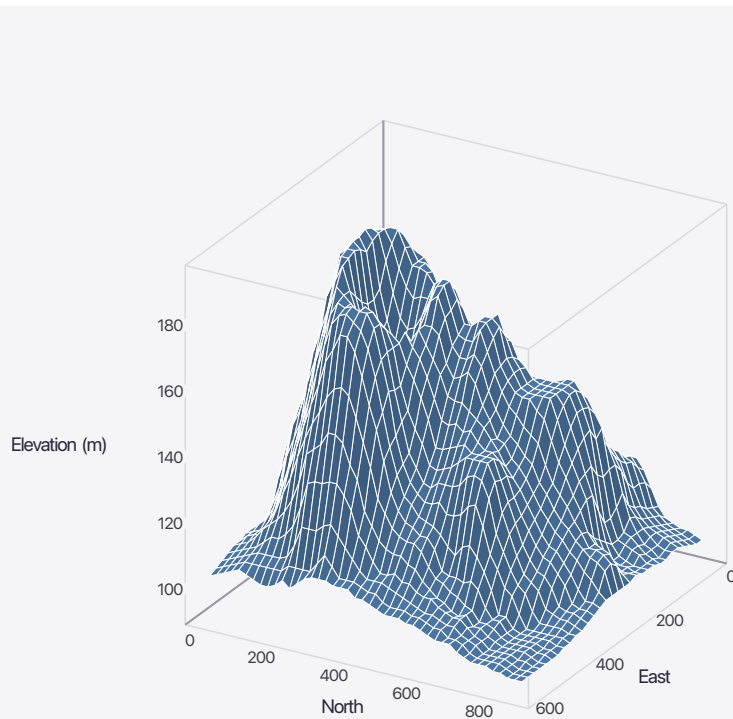
Julia

```
data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
  style(border_color = "white", border_size = 0.4) +
  z_label("Elevation (m)") + title("The mesh, drawn")
```

JavaScript

```
plot(data(maunga_whau), surface, x(col.east), y(col.north),
  z(col.elevation), style({ border_color: "white", border_size: 0.4 }),
  z_label("Elevation (m)", title("The mesh, drawn"))
```

The mesh, drawn



“Given Maunga Whau: a surface, x is east, y is north, z is elevation, with border color white and border size 0.4.”

The mesh adds no shapes to the picture, because every face is already stroked in its own shade. That thin stroke covers the pale seam that would otherwise show between neighboring faces. `border_color` replaces the stroke’s color. Set `border_size = 0` for a seamless sheet.

Notice what the shading is doing in both plots, and what it is *not*. The faces are shaded by their **slope**: a steep one is darker, a level one keeps its full color. That is a fact about the terrain, not about where a light is, so turning the scene rearranges the faces on the page and repaints none of them:

R

```
data(maunga_whau) + surface +  
  x(east) + y(north) + z(elevation) +  
  space(turn = 130, tilt = 35) +  
  z_label("Elevation (m)") + title("Turned, not relit")
```

Python

```
(data(maunga_whau) + surface +  
  x(col.east) + y(col.north) + z(col.elevation) +  
  space(turn = 130, tilt = 35) +  
  z_label("Elevation (m)") + title("Turned, not relit"))
```

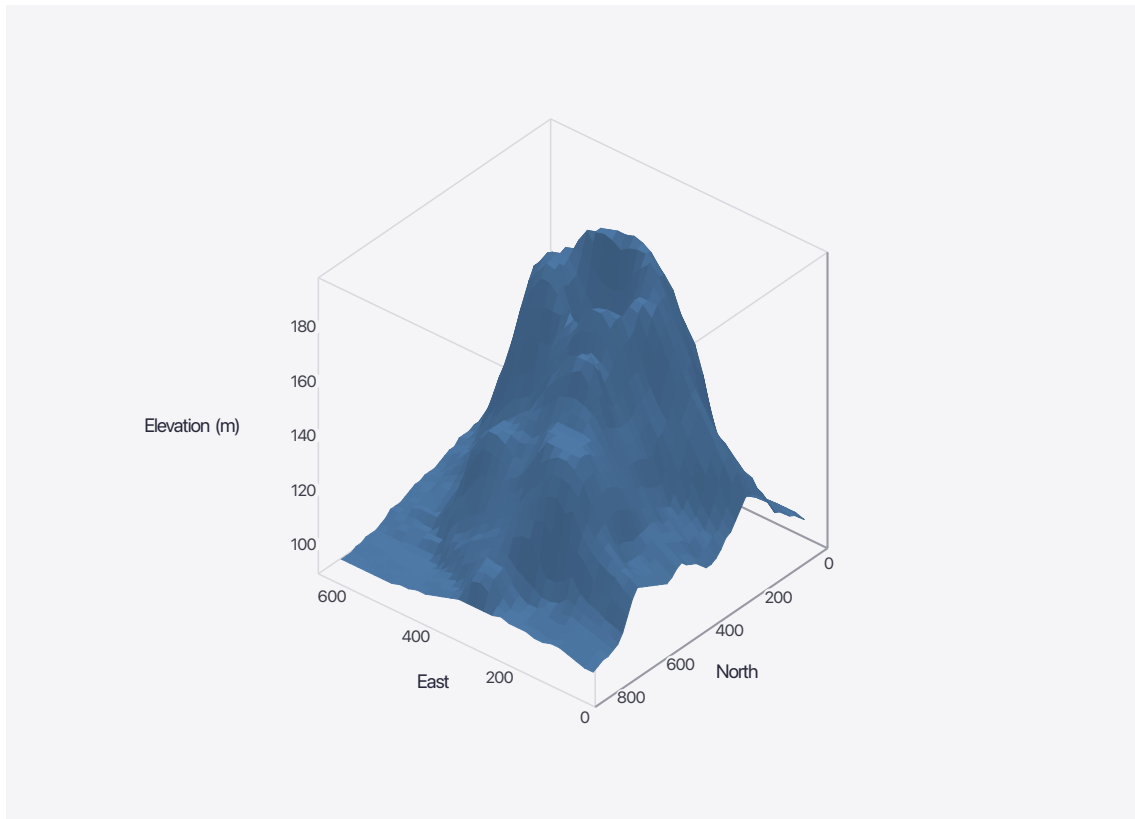
Julia

```
data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +  
  space(turn = 130, tilt = 35) + z_label("Elevation (m)") +  
  title("Turned, not relit")
```

JavaScript

```
plot(data(maunga_whau), surface, x(col.east), y(col.north),  
  z(col.elevation), space({ turn: 130, tilt: 35 } ),  
  z_label("Elevation (m)", title("Turned, not relit"))
```

Turned, not relit



“Given Maunga Whau: a surface, x is east, y is north, z is elevation, turned 130, tilted 35.”

Compare the crater walls here with the first plot. Same faces, same shades, seen from the other side. A lamp fixed to the page would have repainted every one of them, and a color that changed while the data did not would mislead a reader with no way to notice.

20.7 Color by height, per face

`color` maps on a surface, and a **measured** column colors it face by face along a ramp. `palette()` chooses which ramp. "viridis" suits a topographic sheet, because it gets lighter the whole way, so you can judge the height from the color even where a face is turned away:

R

```
data(maunga_whau) + surface +
  x(east) + y(north) + z(elevation) + color(elevation) +
  palette("viridis") +
  z_label("Elevation (m)") + title("Height twice: as a position and as a
↪ ramp")
```

Python

```
(data(maunga_whau) + surface +
  x(col.east) + y(col.north) + z(col.elevation) + color(col.elevation) +
  palette("viridis") +
  z_label("Elevation (m)") + title("Height twice: as a position and as a
↪ ramp"))
```

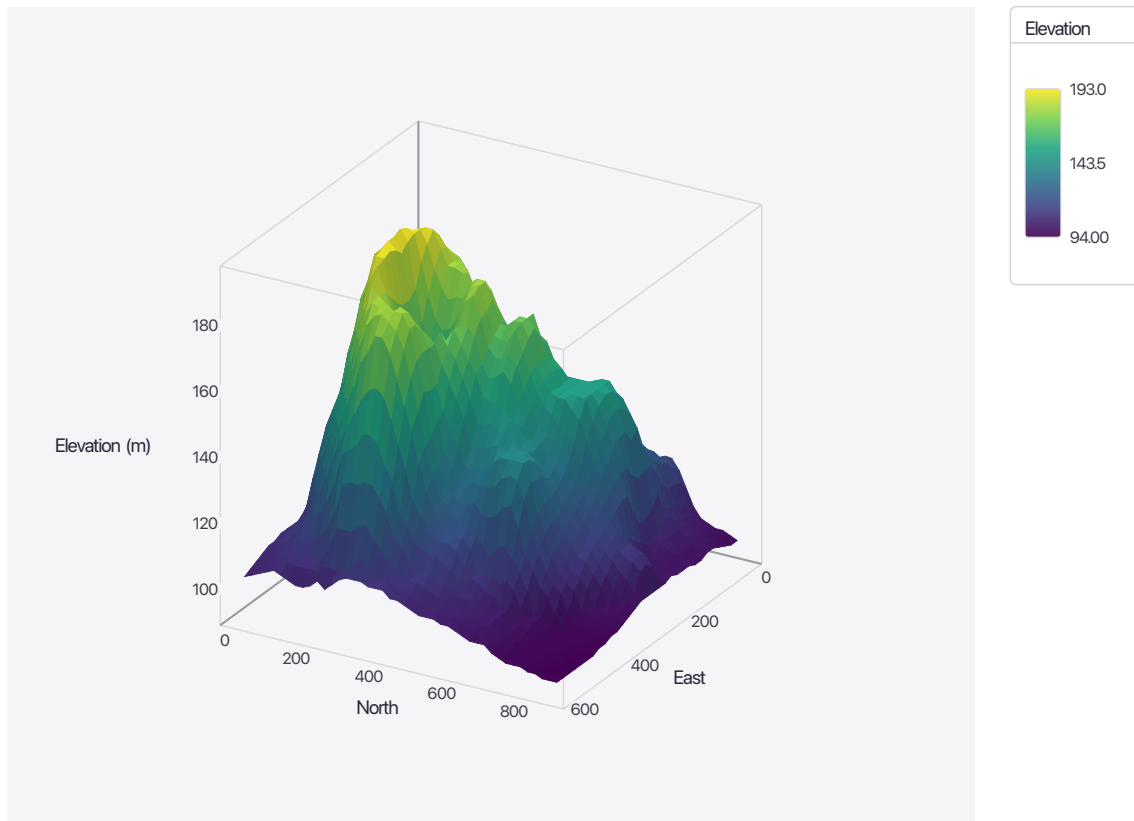
Julia

```
data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
  color(:elevation) + palette("viridis") + z_label("Elevation (m)") +
  title("Height twice: as a position and as a ramp")
```

JavaScript

```
plot(data(maunga_whau), surface, x(col.east), y(col.north),
  z(col.elevation), color(col.elevation), palette("viridis"),
  z_label("Elevation (m)",
  title("Height twice: as a position and as a ramp"))
```

Height twice: as a position and as a ramp



Compare that with the first plot in this chapter. The summit and the crater rim are in that plot too. With slope shading alone you have to work them out from which faces are dark. The ramp states the heights, and the legend decodes them.

Mapping the height to color when it is already the height is redundant on purpose, and it is what a reader of a topographic map expects, so the ramp is worth drawing. It also marks a real boundary. An [area](#) refuses a measured color, because a region has one interior and coloring an interior by a measure is a gradient fill, which is a different and much larger job. A mesh has *faces*, and a face is already small enough to hold one value. So the rule is not about regions and strokes. It is whether a mark has parts small enough to each hold one value, and a stroke has segments where a surface has faces.

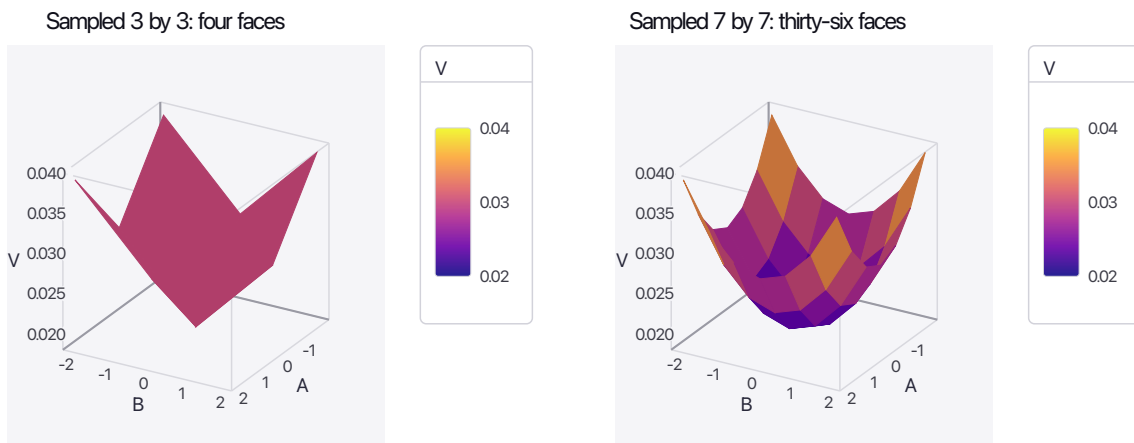
A face has four corners and one color. What it reads is the field at its center, which is the mean of those four, so the ramp describes the mesh you have and not only the field you sampled from. Sample the same bowl twice and see where the shape appears:

```

bowl <- function(n) {
  g <- expand.grid(a = seq(-2, 2, length.out = n), b = seq(-2, 2, length.out
↪ = n))
  g$v <- 0.019 + 0.0025 * (g$a^2 + g$b^2)
  g
}
coarse <- bowl(3)
fine <- bowl(7)

((data(coarse, name = "coarse") + surface + x(a) + y(b) + z(v) +
  color(v) + palette("plasma") + title("Sampled 3 by 3: four faces")) |
  (data(fine, name = "fine") + surface + x(a) + y(b) + z(v) +
    color(v) + palette("plasma") + title("Sampled 7 by 7: thirty-six
↪ faces")))) +
  theme(height = 320)

```



“Given coarse: a surface, x is a, y is b, z is v, color by v, with the plasma palette, beside the same given fine.”

Two tables on one page, so each is named. A layer finds its columns in the nearest table by that name, and two tables with the same name would leave the second one unreachable ([Data](#)).

The left sheet is one flat color, and that is honest rather than broken. Every face has one high corner, one low corner and two middle ones, so all four average to the same number. The mesh is too coarse to hold the shape, and the ramp says so instead of inventing a gradient across it. The right sheet has the same field under it and enough faces to resolve it, so the bowl appears.

This is the reading that makes the legend trustworthy. A color is the height at one named place on the sheet, so two faces of one color really are at one height, and a sheet of one color really is flat on average.

The slope shading is still underneath the ramp, and the two combine rather than cancel: a face takes its color from the ramp and is then darkened by how steep it is. Viridis stays readable because only the *low* ground reaches its dark colors, and on a terrain the low ground is also the flattest. The two darkenings therefore do not add together, and the darkest face on this sheet is lighter than a fully-shaded viridis floor would have been.

An **estimated** height takes a ramp the same way, and you name it the same way. `density` writes its estimate into a column called `density`, so that is the word `color` takes, exactly as it does on a `zone` or a `path`:

R

```
data(iris_flowers) + surface * density +  
  x(petal_length) + y(sepal_length) + space() + color(density) +  
  palette("viridis") + title("The estimate, as a height and as a ramp")
```

Python

```
(data(iris_flowers) + surface * density +  
  x(col.petal_length) + y(col.sepal_length) + space() + color(col.density) +  
  palette("viridis") + title("The estimate, as a height and as a ramp"))
```

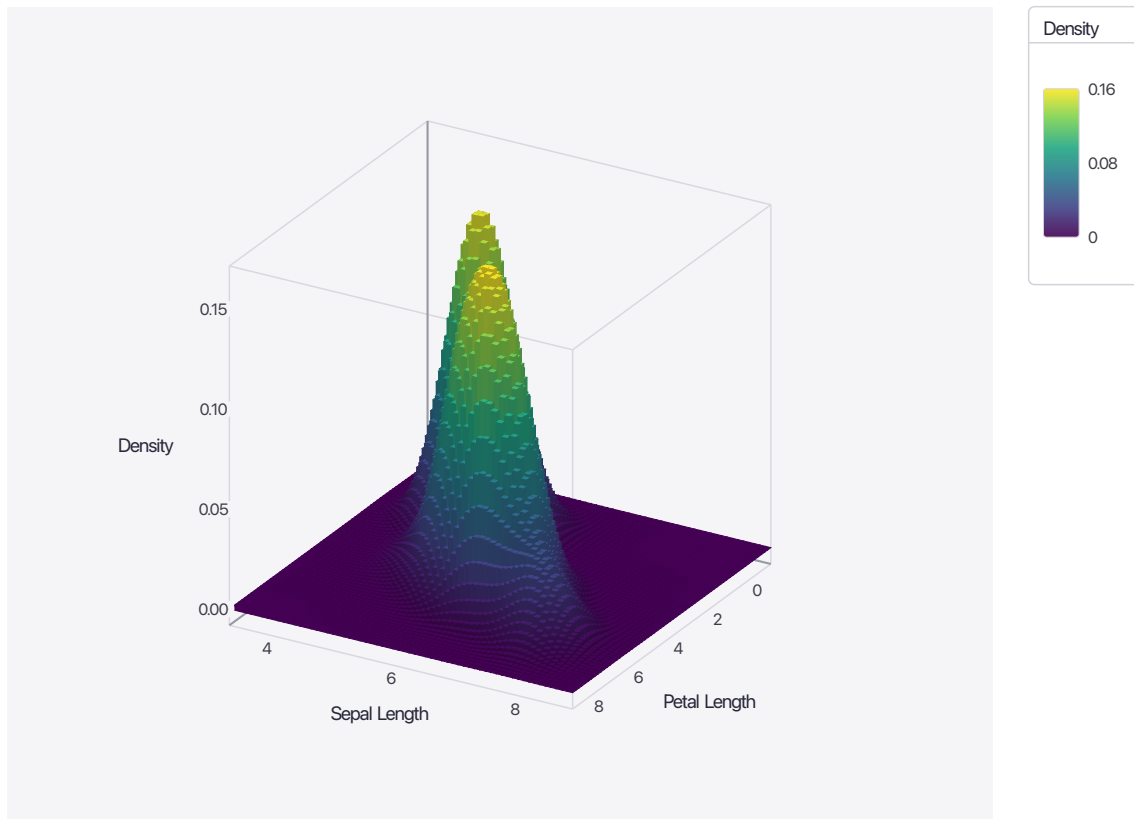
Julia

```
data(iris_flowers) + surface * density + x(:petal_length) +  
  y(:sepal_length) + space() + color(:density) + palette("viridis") +  
  title("The estimate, as a height and as a ramp")
```

JavaScript

```
plot(data(iris_flowers), layer(surface, density), x(col.petal_length),  
  y(col.sepal_length), space(), color(col.density), palette("viridis"),  
  title("The estimate, as a height and as a ramp"))
```

The estimate, as a height and as a ramp



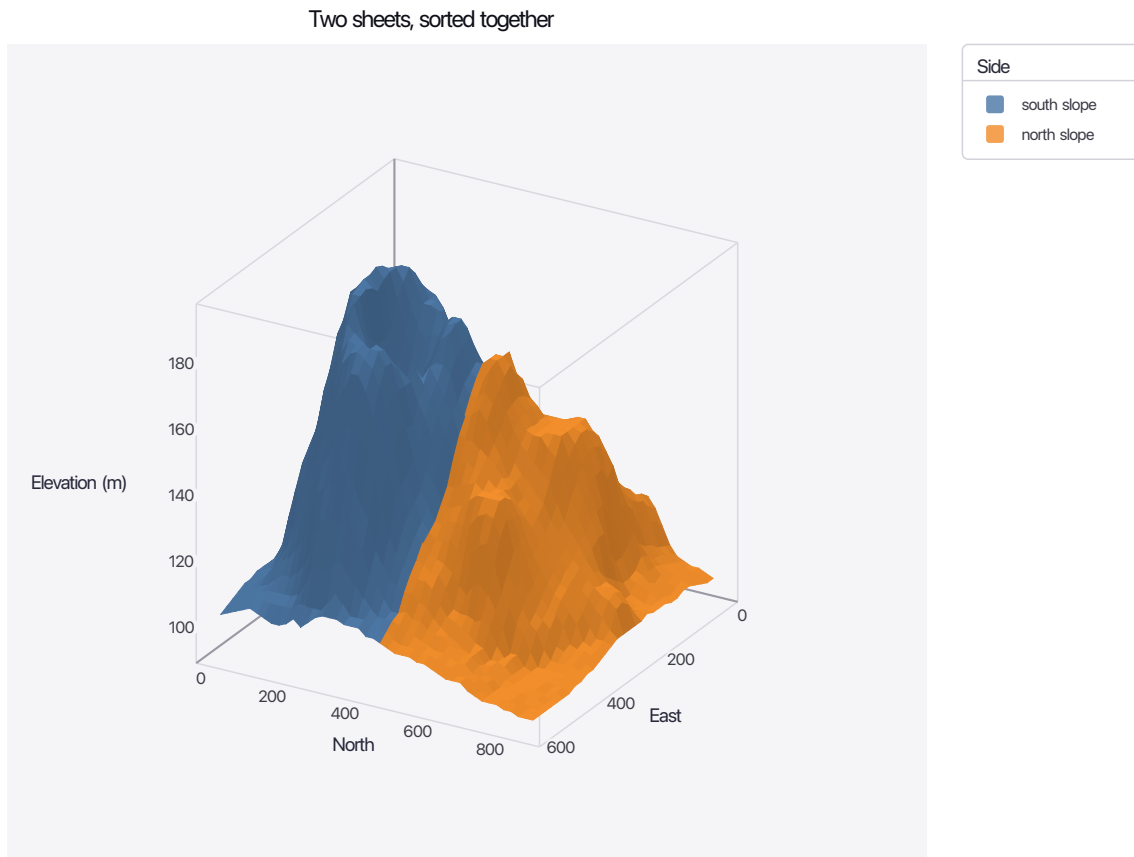
Nothing there is a special case. The transform makes a column, the column has a name, and a name can be bound. Whether the height came from your table or from an estimate changes where the number came from. It does not change what you may say about it.

Whether you *want* the color as well as the height is a separate question, and the plain sheet earlier in this chapter is the one that makes the argument about height. Use the ramp when the low ground matters: slope shading darkens a steep face and leaves a level one alone, so a broad, shallow mound and the flat floor around it are the same color, and the ramp is what separates them.

A **categorical** color splits the mark instead, one sheet per group. `gog` sorts every face of both sheets by depth together, so the two sheets cross rather than one sitting wholly in front:

```
band <- maunga_whau
band$side <- ifelse(band$north > 440, "north slope", "south slope")

data(band) + surface + x(east) + y(north) + z(elevation) + color(side) +
  z_label("Elevation (m)") + title("Two sheets, sorted together")
```

20.8 Opacity by a measure, per face

`opacity` maps on a surface, and it maps for the reason `color` does. A face is small enough to hold one value, so it can hold an opacity too. Every face takes its own opacity, and a face fades where its measure is small:

R

```
data(maunga_whau) + surface +  
  x(east) + y(north) + z(elevation) + opacity(elevation) +  
  z_label("Elevation (m)") + title("Faint where the ground is low")
```

Python

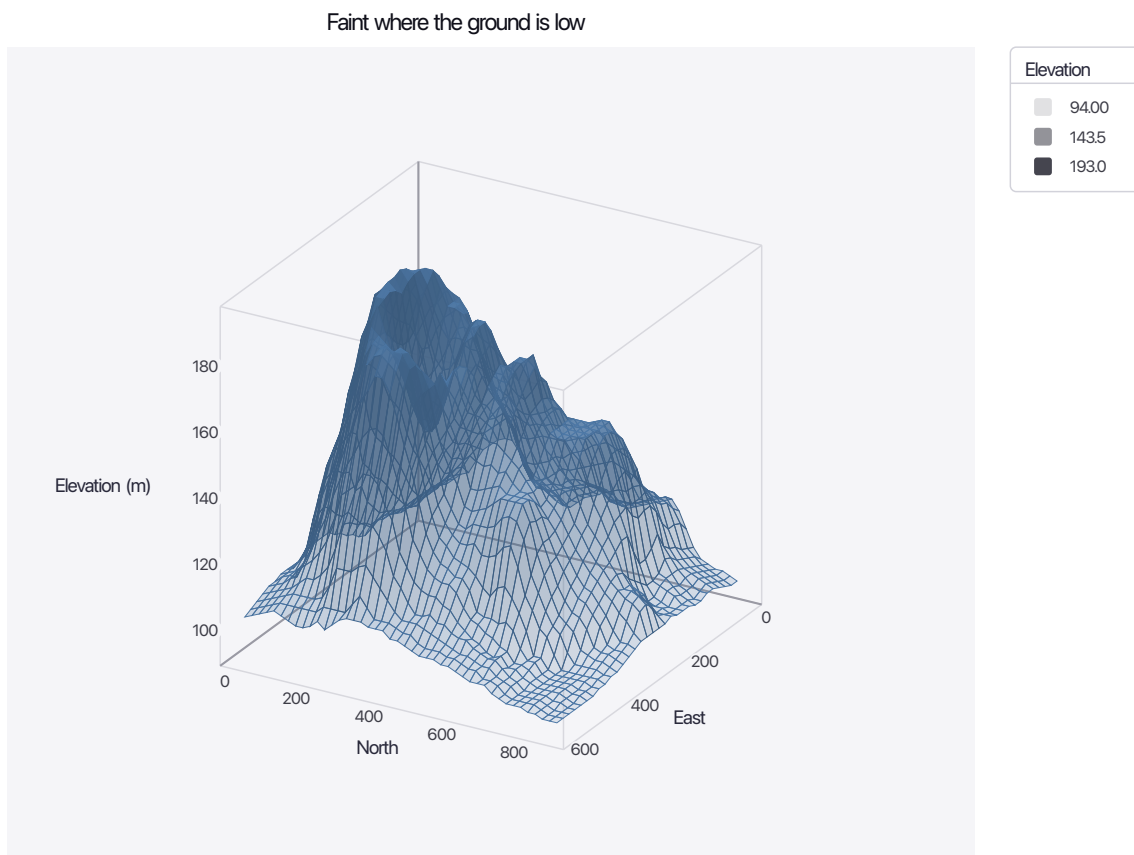
```
(data(maunga_whau) + surface +
  x(col.east) + y(col.north) + z(col.elevation) + opacity(col.elevation) +
  z_label("Elevation (m)") + title("Faint where the ground is low"))
```

Julia

```
data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
  opacity(:elevation) + z_label("Elevation (m)") +
  title("Faint where the ground is low")
```

JavaScript

```
plot(data(maunga_whau), surface, x(col.east), y(col.north),
  z(col.elevation), opacity(col.elevation), z_label("Elevation (m)",
  title("Faint where the ground is low")))
```



“Given Maunga Whau: a surface, x is east, y is north, z is elevation, opacity by elevation.”

Over the low ground the fill fades to almost nothing and each face keeps its outline, so that part of the sheet shows as mesh lines alone. A ramp cannot do that; an opacity can. A color changes what a face says. An opacity changes whether the face hides what is behind it. The legend beside the plot reads the same way `color`’s does, with the smallest, middle and largest values of the column.

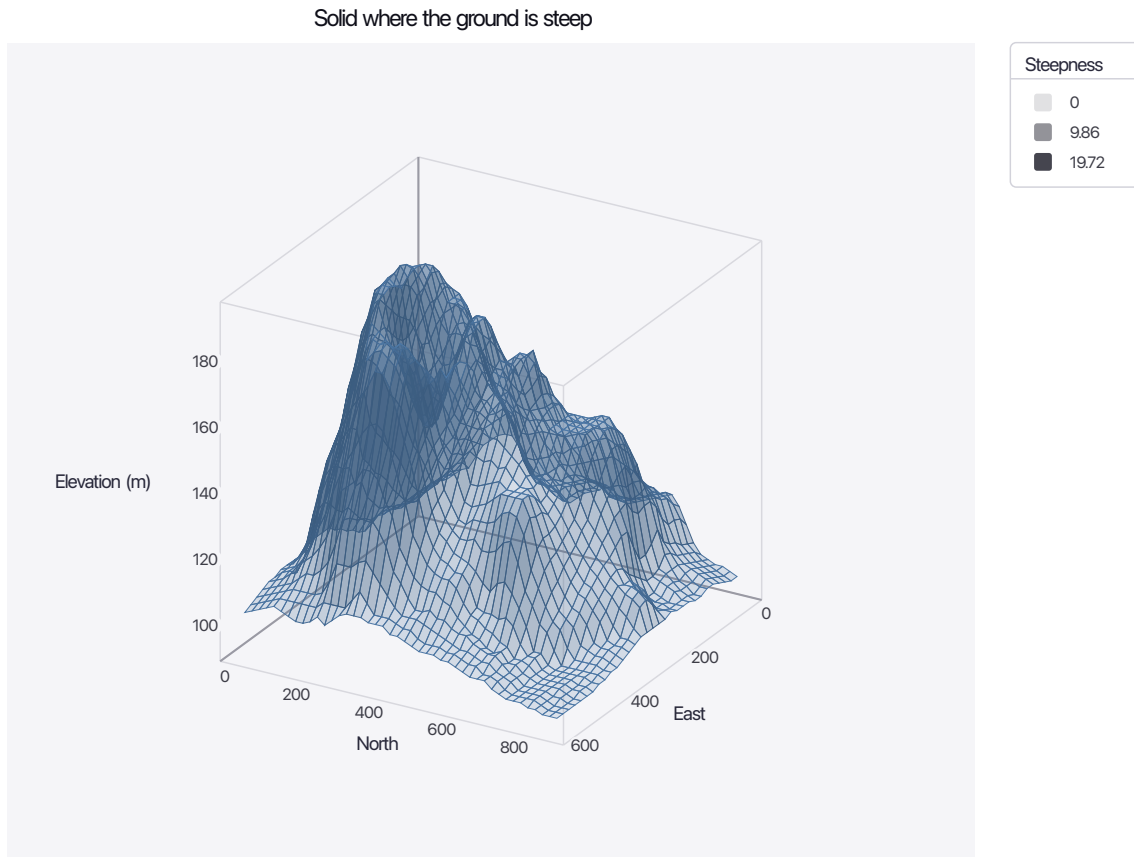
No face disappears completely. A mapped `opacity` runs from 0.15 to 0.95, so the faintest face here is still drawn, which is the rule every mapped `opacity` follows ([Channels](#)).

A face reads its opacity where it reads its color, at its own center, which is the mean of its four corners. A plateau and a riser each own one cell, so each reads its own row. Either way one face has one reading, whichever channel asks for it.

In the plot above, the height is said twice: once as a position and once as an opacity. A second column is the more useful case: the shape stays in `z`, and another measure decides which faces fade.

```
relief <- maunga_whau
h <- matrix(relief$elevation, nrow = length(unique(relief$north)))
up  <- rbind(h[-1, ], h[nrow(h), ])
side <- cbind(h[, -1], h[, ncol(h)])
relief$steepness <- as.vector(sqrt((up - h)^2 + (side - h)^2))

data(relief) + surface + x(east) + y(north) + z(elevation) +
  ↪ opacity(steepness) +
  z_label("Elevation (m)") + title("Solid where the ground is steep")
```



steepness combines the two elevation changes at each crossing, one toward the next east value and one toward the next north value. The steep sides of the hill stay solid, and the flat ground around them fades away. The sheet is the same shape as the one above it, and a different part of it is solid.

An [area](#) refuses a mapped opacity for the reason it refuses a measured color. A region has one interior and one fill, so a row there is a corner of the outline rather than a part that can carry a value. A mesh has faces, so it has parts. Both channels ask the same question of a mark, so a mark that refuses one refuses the other.

One value for the whole sheet is still `style(opacity = )`. Set it when you want to see one sheet through another, and map it when you want each face to fade by its own value.

20.9 One sheet, four angles

A still 3-D plot hides something at every angle, and which thing it hides is the angle's choice. So it is worth turning the same sheet rather than trusting one view of it. Each of these changes exactly *one* of turn/tilt from the first:

R

```
((data(maunga_whau) + surface + x(east) + y(north) + z(elevation) +
  color(elevation) + palette("viridis") + space(turn = 30, tilt = 25) +
  z_label("Elevation (m)") + title("The reference three-quarter view")) |
 (data(maunga_whau) + surface + x(east) + y(north) + z(elevation) +
  color(elevation) + palette("viridis") + space(turn = 140, tilt = 25) +
  z_label("Elevation (m)") + title("Turned: the same sheet from the
↪ west")))/
((data(maunga_whau) + surface + x(east) + y(north) + z(elevation) +
  color(elevation) + palette("viridis") + space(turn = 30, tilt = 70) +
  z_label("Elevation (m)") + title("Tilted up: the crater appears")) |
 (data(maunga_whau) + surface + x(east) + y(north) + z(elevation) +
  color(elevation) + palette("viridis") + space(turn = 30, tilt = 5) +
  z_label("Elevation (m)") + title("Tilted down: the skyline")))
```

Python

```
((data(maunga_whau) + surface + x(col.east) + y(col.north) +
↪ z(col.elevation) +
  color(col.elevation) + palette("viridis") + space(turn = 30, tilt = 25) +
  z_label("Elevation (m)") + title("The reference three-quarter view")) |
 (data(maunga_whau) + surface + x(col.east) + y(col.north) + z(col.elevation)
↪ +
  color(col.elevation) + palette("viridis") + space(turn = 140, tilt = 25)
↪ +
  z_label("Elevation (m)") + title("Turned: the same sheet from the
↪ west")))/
((data(maunga_whau) + surface + x(col.east) + y(col.north) + z(col.elevation)
↪ +
  color(col.elevation) + palette("viridis") + space(turn = 30, tilt = 70) +
  z_label("Elevation (m)") + title("Tilted up: the crater appears")) |
 (data(maunga_whau) + surface + x(col.east) + y(col.north) + z(col.elevation)
↪ +
  color(col.elevation) + palette("viridis") + space(turn = 30, tilt = 5) +
  z_label("Elevation (m)") + title("Tilted down: the skyline")))
```

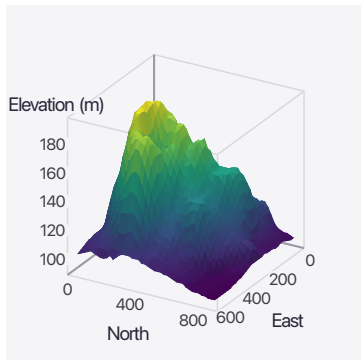
Julia

```
((data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
color(:elevation) + palette("viridis") + space(turn = 30, tilt = 25) +
z_label("Elevation (m)") + title("The reference three-quarter view")) |
(data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
color(:elevation) + palette("viridis") + space(turn = 140, tilt = 25) +
z_label("Elevation (m)") +
title("Turned: the same sheet from the west")))) /
((data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
color(:elevation) + palette("viridis") + space(turn = 30, tilt = 70) +
z_label("Elevation (m)") + title("Tilted up: the crater appears")) |
(data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
color(:elevation) + palette("viridis") + space(turn = 30, tilt = 5) +
z_label("Elevation (m)") + title("Tilted down: the skyline")))
```

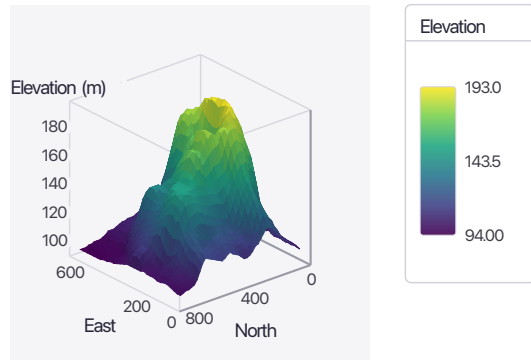
JavaScript

```
below(beside(plot(data(maunga_whau), surface, x(col.east), y(col.north),
z(col.elevation), color(col.elevation), palette("viridis"),
space({ turn: 30, tilt: 25 }), z_label("Elevation (m)"),
title("The reference three-quarter view")),
plot(data(maunga_whau), surface, x(col.east), y(col.north),
z(col.elevation), color(col.elevation), palette("viridis"),
space({ turn: 140, tilt: 25 }), z_label("Elevation (m)"),
title("Turned: the same sheet from the west"))),
beside(plot(data(maunga_whau), surface, x(col.east), y(col.north),
z(col.elevation), color(col.elevation), palette("viridis"),
space({ turn: 30, tilt: 70 }), z_label("Elevation (m)"),
title("Tilted up: the crater appears")),
plot(data(maunga_whau), surface, x(col.east), y(col.north),
z(col.elevation), color(col.elevation), palette("viridis"),
space({ turn: 30, tilt: 5 }), z_label("Elevation (m)"),
title("Tilted down: the skyline"))))
```

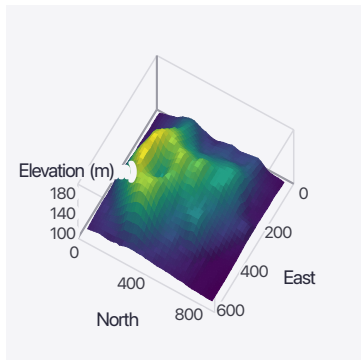
The reference three-quarter view



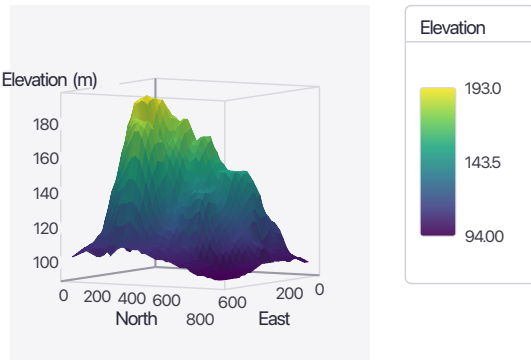
Turned: the same sheet from the west



Tilted up: the crater appears



Tilted down: the skyline



“Given Maunga Whau: a surface, x is east, y is north, z is elevation, color by elevation, with the viridis palette, turned 30, tilted 25, beside the same turned 140; above the same tilted 70, beside the same tilted 5.”

Read them as a set and you can see what each angle hides. The crater is the most interesting thing about this hill, and the first two plots do not show it. The summit stands in front of the hollow, so no amount of turning at a low tilt reveals it. Tilt up and the hollow opens, because you are now looking down on the hill from above. The same tilt flattens the heights, until the sheet is a colored map of itself. Tilt down instead and you get the opposite: the side view is exact, the view from above is gone, and the sheet shows as an outline against the sky. One or the other, never both.

That is the argument for a plot you can turn, made rather than asserted. Here you need both the view from above and the view from the side, and no single still view shows this hill whole. So turn the four with the mouse. Lift your eye until the crater opens, and watch the heights flatten as it does. The four still views make the argument; dragging them is how you check it.

All four turn together, because `|` and `/` composed them into one figure, and a composition turns as one. Each panel keeps the angle its own sentence asked for, and the drag adds the same change to every one. They turn as a set, and *reset* returns each to where it began. To turn one alone, write it alone.

That is composition rather than faceting, and the two share the same pair of operators. A facet names a **column** ([Faceting](#)), and a viewing angle is not a column: it is a property of the coordinate space, the way a `polar()` start angle is. `gog` does build small multiples of cubes, and these four panels are not one ([Space](#) has that example).

One limit shows in these four plots, and it belongs to the cube rather than to the mark. Notice the vertical axis in the tilted-up panel. Its numbers sit tight against each other, because that axis is projecting at about a third of its length and they are still all being drawn. They do clear each other, and tilting further makes `gog` drop some numbers rather than overlap them. Numbers packed close together are simply what a foreshortened axis looks like. [Space](#) says which numbers a crowded axis drops, and why the scale itself never changes.

20.10 What a surface is for

Anything that is a *height over a plane*. Terrain, and the sea floor. A response surface from a designed experiment. A model's fit measured over two of its parameters. A model's predictions over a grid of two predictors. And, through **density**, the shape of a two-dimensional distribution.

What it is not for is data that merely *has* three numbers per row. That is a [3-D scatter](#), and the refusal above names it for you. The question to ask is whether the space between two of your samples means anything. On a grid it does; in a cloud it does not.

20.11 What you can set

Everything a surface can be told to look like, generated from the engine's own legality table, so this page cannot differ from what `style()` accepts:

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(border_color =)</code>	any CSS color name or hex
<code>style(border_size =)</code>	pixels

And these vary per row if you map them to a column instead: `color()` (either), `opacity()` (numbers), `group()` (categories), `play()` (either).

A surface is a fill, so it takes a fill's settings, with two absences the next section explains: no `size`, and no `pattern`.

20.12 What it refuses

A surface with no height is the same mistake as a surface in the plane, so it is one refusal rather than two, and it names both routes into the cube along with the mark that draws a field in the plane:

```
data(maunga_whau) + surface + x(east) + y(north)
```

Error:

```
! gog: a `surface` is a sheet through three positions, so it needs the cube.
  ↳ Bind the height - `surface + x(a) + y(b) + z(h)` - or let a transform
  ↳ invent it: `surface * density + x(a) + y(b) + space()`. For the same
  ↳ field drawn in the plane, `zone` paints it as cells and `path` traces its
  ↳ contours.
```

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

`density(levels = )` cuts a field into level *sets*, and a level set is a region in the plane. A surface has already given the third axis to the measurement, so a band drawn on it could only be a color, which is `zone`'s reading of that same request:

```
data(iris_flowers) + surface * density(levels = 5) +
  x(petal_length) + y(sepal_length) + space()
```

Error:

```
! gog: `density(levels = )` cuts a field into level sets, which are regions
  ↳ in the plane - a `surface` draws the field itself, with the estimate as
  ↳ its height, so it has no axis left to put a band on. Drop `levels` for
  ↳ the sheet, or `zone * density(levels = )` to fill the bands and `path *
  ↳ density(levels = )` to trace their boundaries. For a sheet in steps, cut
  ↳ the floor rather than the height: `surface * bin * mean + x(<a>) + y(<b>)
  ↳ + z(<column>)` lays a flat plateau on every cell.
```

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

A stepped sheet can be cut on either of two axes, and only one of the two draws. Cutting the **floor** into cells is `surface * bin`, the section on one value per cell above, and it draws. Cutting the **height** into bands is what `levels` asks for, and that one is refused. It would cut

the measurement rather than the grid, so the plateaus would be contours of equal height rather than one per cell.

bandwidth is refused for the reason it is refused on every two-dimensional field: it is a length in one column's own units, and a field spreads over two columns measuring different quantities, so one number cannot be a width in both. Use `density(adjust = )`, which is dimensionless and scales both.

There is no **size** on a surface. Its extent is the grid it is drawn on, so a size would be a second and contradictory answer to how big it is. **area** refuses a size for the same reason. And **pattern** is refused, which is the one place a fill does not take a texture:

```
data(maunga_whau) + surface + x(east) + y(north) + z(elevation) +  
  style(pattern = "hatch")
```

Error:

```
! gog: `style(pattern = )` textures a fill, and a `surface` is a mesh of  
  ↳ fills - but a hatch tile is a texture in *screen* space, and a projected  
  ↳ mesh foreshortens every face differently, so one tile would read as a  
  ↳ different density on every face. That is a texture that changes with the  
  ↳ viewing angle instead of with the data. A surface says its shape with  
  ↳ slope shading; `style(border_color = )` draws its mesh lines, and `color`  
  ↳ ramps it.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A hatch tile is a texture in *screen* space, and every face of a projected mesh is foreshortened differently, so one tile size would read as a different density on every face. The texture would then follow the viewing angle rather than the data, which is the one thing this mark's shading is built to avoid.

21 Edge

Which things in your table are connected to which? **edge** draws one stroke per row of the table, between the two things that row names. The other strokes draw between positions taken from columns. A **line** runs along its domain, a **path** follows the table's order, an **interval** spans two values. An **edge** is different, because the endpoints of an edge are names, not numbers, so no axis can measure them. The picture an **edge** layer makes is the **network diagram**.

So an edge takes no **x** and no **y**. Its two endpoints come from a **layout**, a transform that reads the two name columns and computes a position for every name. Write the source column first and the destination second.

trade_partners holds one row per trade relation: who exports, who imports, and the value of the goods, in billions of dollars, rounded:

Table 21.1: **trade_partners**: first 5 of 22 rows

exporter	importer	billions
China	United States	500
Mexico	United States	455
Canada	United States	420
United States	Mexico	320
United States	Canada	355

Name the table, then write three things: the mark, its layout, and the space **network()**:

R

```
data(trade_partners) + edge * layout(exporter, importer) + network()
```

Python

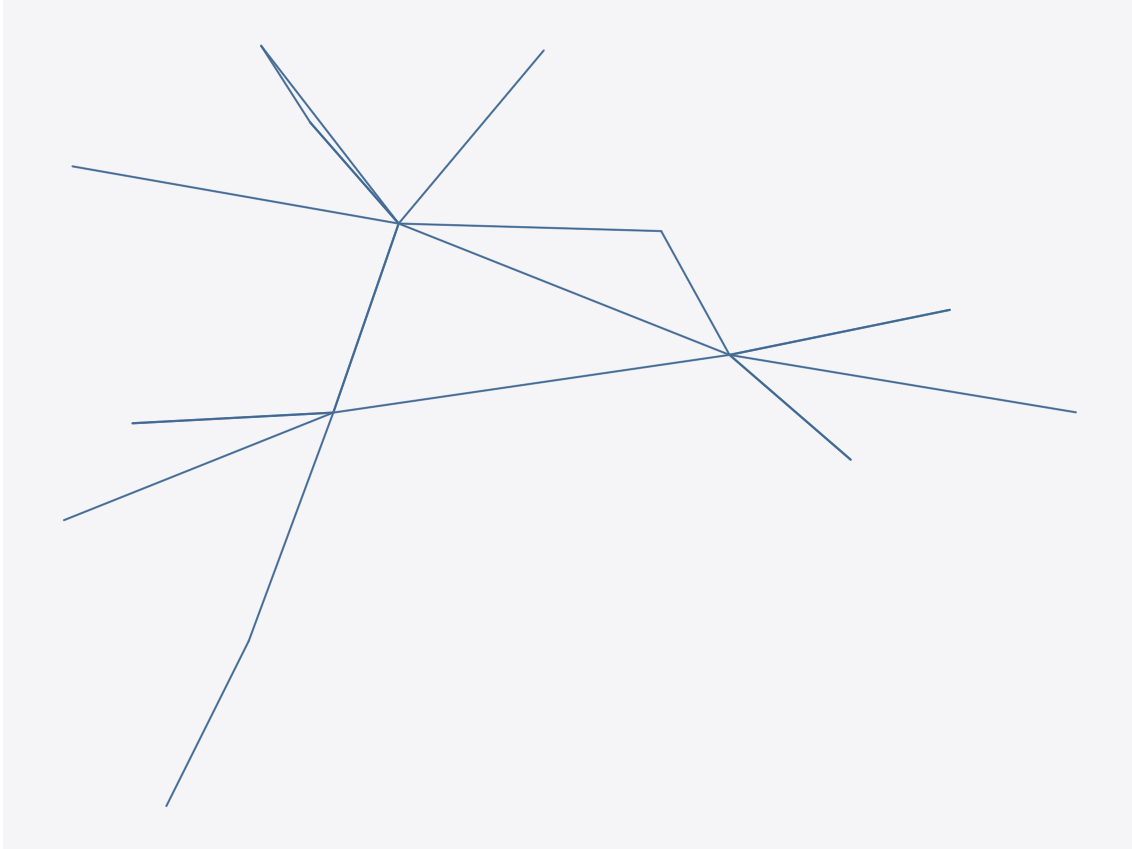
```
data(trade_partners) + edge * layout(col.exporter, col.importer) + network()
```

Julia

```
data(trade_partners) + edge * layout(:exporter, :importer) + network()
```

JavaScript

```
plot(data(trade_partners),  
  layer(edge, layout(col.exporter, col.importer)), network())
```



“Given trade partners: edges derived by layout from exporter to importer, in the network.”

An **edge** draws only inside **network()**. Every other space labels its axes with numbers, and an edge’s two endpoints are names. **gog** refuses the sentence rather than drawing it.

21.1 Color and opacity, one per row

An edge is one row, so its color and its opacity can follow a column of the table. `color` maps a category, and `opacity` maps a number such as `billions`. A second layer, `point * layout(exporter, importer)`, reads the same layout and draws a dot at each name:

R

```
data(trade_partners) +  
  edge * layout(exporter, importer) + opacity(billions) +  
  point * layout(exporter, importer) +  
  network() +  
  title("Heavier trade, darker stroke")
```

Python

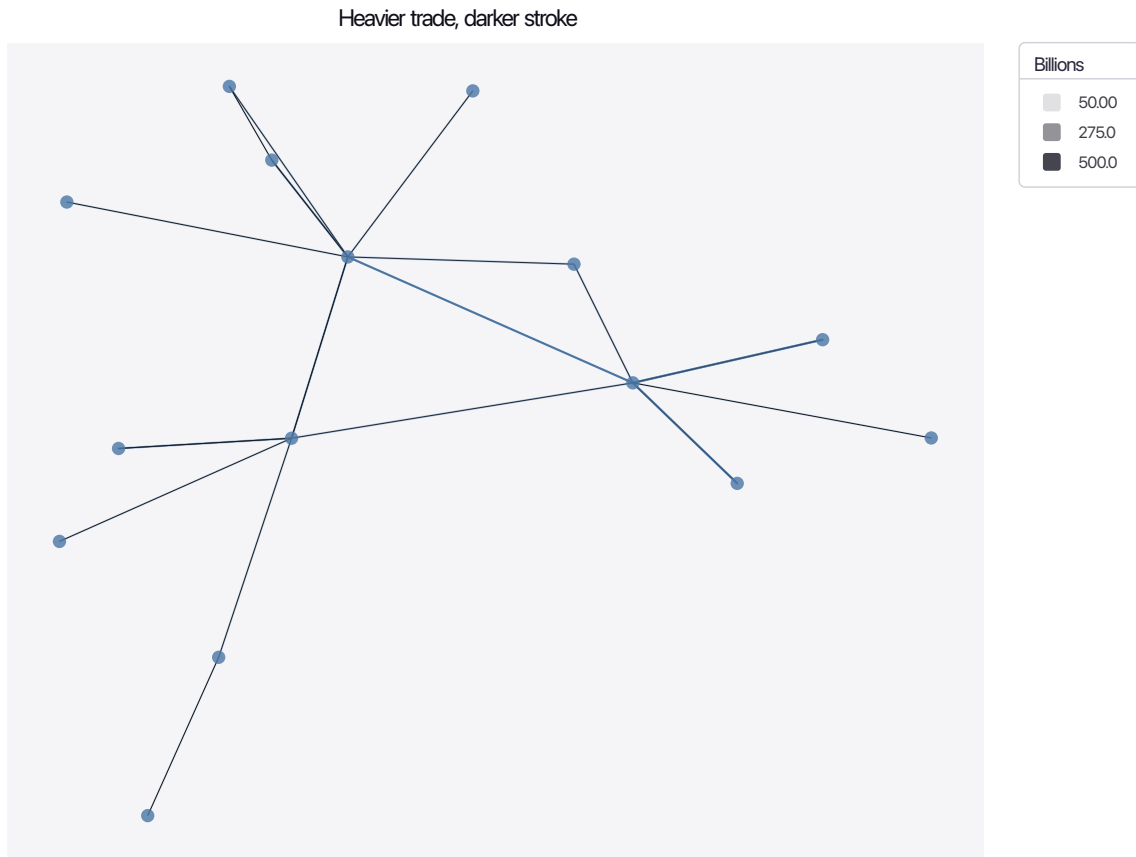
```
(data(trade_partners) +  
  edge * layout(col.exporter, col.importer) + opacity(col.billions) +  
  point * layout(col.exporter, col.importer) +  
  network() +  
  title("Heavier trade, darker stroke"))
```

Julia

```
data(trade_partners) + edge * layout(:exporter, :importer) +  
  opacity(:billions) + point * layout(:exporter, :importer) + network() +  
  title("Heavier trade, darker stroke")
```

JavaScript

```
plot(data(trade_partners),  
  layer(edge, layout(col.exporter, col.importer)), opacity(col.billions),  
  layer(point, layout(col.exporter, col.importer)), network(),  
  title("Heavier trade, darker stroke"))
```



“Given trade partners: edges derived by layout from exporter to importer, opacity by billions, and also points derived by layout from exporter to importer, in the network.”

On line, `opacity` is only a setting, because a line is one stroke with one value. An edge layer is many strokes, and each row gives its stroke an opacity of its own.

`color` works the same way, one value per row. Each stroke takes the color of its exporter, so Germany’s five strokes share one color:

R

```
data(trade_partners) +
  edge * layout exporter, importer) + color exporter) +
  point * layout exporter, importer) +
  network() +
  title("Each stroke, colored by its exporter")
```

Python

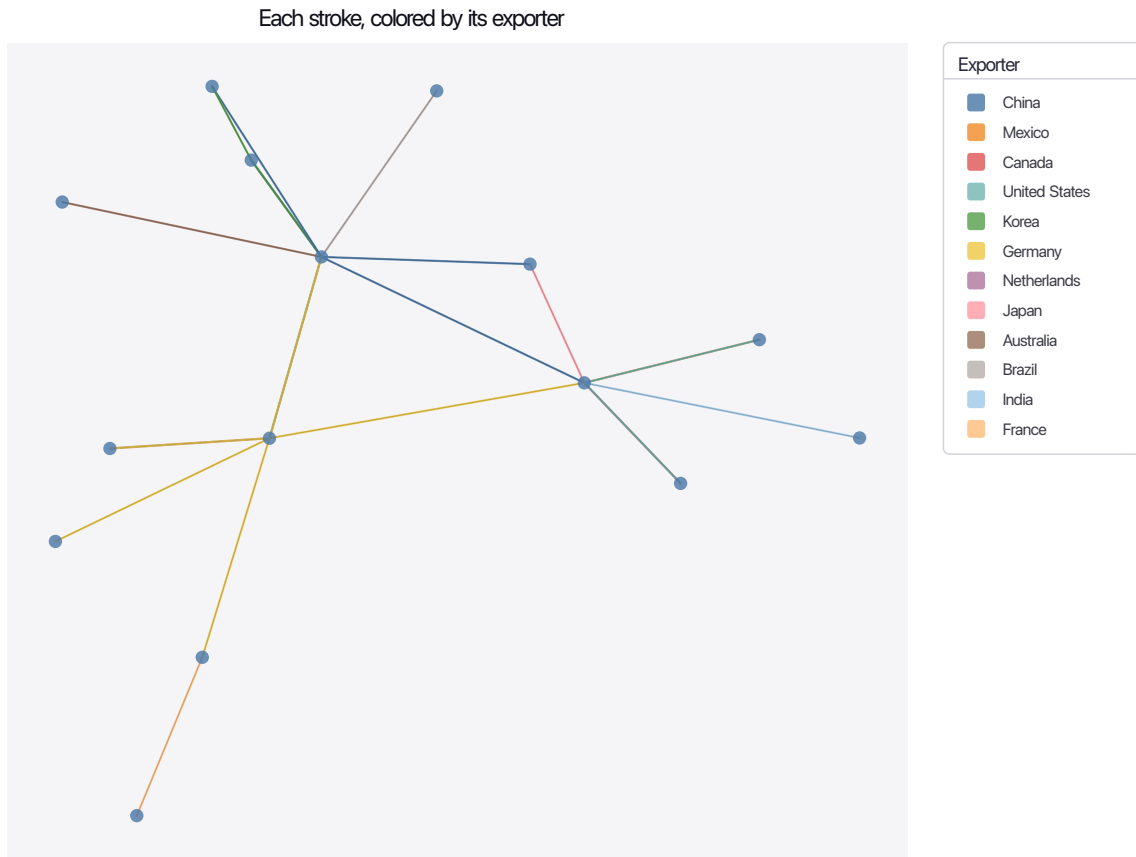
```
(data(trade_partners) +
  edge * layout(col.exporter, col.importer) + color(col.exporter) +
  point * layout(col.exporter, col.importer) +
  network() +
  title("Each stroke, colored by its exporter"))
```

Julia

```
data(trade_partners) + edge * layout(:exporter, :importer) +
  color(:exporter) + point * layout(:exporter, :importer) + network() +
  title("Each stroke, colored by its exporter")
```

JavaScript

```
plot(data(trade_partners),
  layer(edge, layout(col.exporter, col.importer)), color(col.exporter),
  layer(point, layout(col.exporter, col.importer)), network(),
  title("Each stroke, colored by its exporter"))
```



“Given trade partners: edges derived by layout from exporter to importer, color by exporter, and also points derived by layout from exporter to importer, in the network.”

21.2 What you can set

Everything an edge can be told to look like, generated from the engine’s own legality table, so this page cannot differ from what `style()` accepts:

Setting	Value
<code>style(color =)</code>	any CSS color name or hex
<code>style(opacity =)</code>	0 to 1
<code>style(size =)</code>	pixels
<code>style(pattern =)</code>	solid, dashed, dotted

And these vary per row if you map them to a column instead: `color()` (categories), `opacity()` (numbers), `play()` (either).

`style(size = )` sets the stroke width for the whole layer. `style(color = )` and `style(opacity = )` give every stroke one value, for the case where no column maps. `style(pattern = )` chooses the dash: `solid`, `dashed` or `dotted`.

21.3 What it refuses

An edge with no layout has no endpoints to draw. `x` and `y` cannot supply them, so `gog` refuses each binding, then refuses the empty `network()`:

```
data(trade_partners) + edge + x(billions) + y(exporter) + network()
```

Error:

```
! gog: `x` cannot be bound to `edge` - an edge has no x feature. Remove the
  ↳ `billions` mapping from `x`, or use a mark that has one.
gog: `y` cannot be bound to `edge` - an edge has no y feature. Remove the
  ↳ `exporter` mapping from `y`, or use a mark that has one.
gog: `network()` draws what a `layout` places, and nothing here carries one.
  ↳ Compose the syllable: `edge * layout(from, to) + network()`, with `point
  ↳ * layout(...)` for the nodes and `text * layout(...) + label(name)` for
  ↳ their names.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A layout is read only inside `network()`, so the space cannot be left out:

```
data(trade_partners) + edge * layout(exporter, importer)
```

Error:

```
! gog: a `layout`'s positions are the graph's, not the data's - an axis under
  ↳ them would number something no reader can mean. Draw it in the network:
  ↳ `+ network()` flat, or `+ network(turn = 30, tilt = 25)` for the cube.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The [Network](#) chapter draws the rest: a dot for every name, a label beside each dot, the same graph in a cube, and the other refusals.

22 Scales

What do you do when most of your data is crowded into one corner? You change the **scale**, which decides how a number becomes a position. It is written on the binding, not as an atom of its own:

```
x(gdp, scale = "log")
```

Scales are a property of the axis, so they live where the axis is bound. There is no `scale()` atom to write separately. That would make you name the channel twice, once to bind it and once to scale it, and the two could then disagree.

Writing the scale on the binding helps a second way, and that way is about remembering rather than being correct. The scale sits inside the thing it measures, so there is only one place to look. A reader who wants to know how `gdp` is being read finds the answer in `x(gdp, scale = "log")` itself. Nothing else in the sentence can change it, and there is no second function to go and find.

22.1 A scale is not a transform

`bin` and `mean` are **transforms**: they read your data and produce different data. A scale produces no new values at all. It changes how far along the axis a number lands and what the ticks say, and nothing else.

The difference is visible in the finished plot. Here is `gdp` plotted with a log **scale**:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(continent) +  
  x_label("GDP per capita") +  
  title("Income and life expectancy: log scale")
```

Python

```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +
  color(col.continent) +
  x_label("GDP per capita") +
  title("Income and life expectancy: log scale"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +
  color(:continent) + x_label("GDP per capita") +
  title("Income and life expectancy: log scale")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),
  y(col.life), color(col.continent), x_label("GDP per capita"),
  title("Income and life expectancy: log scale"))
```



“Given *gapminder 2007*: points, *x* is *gdp* on a *log* scale, *y* is *life*, color by *continent*.”

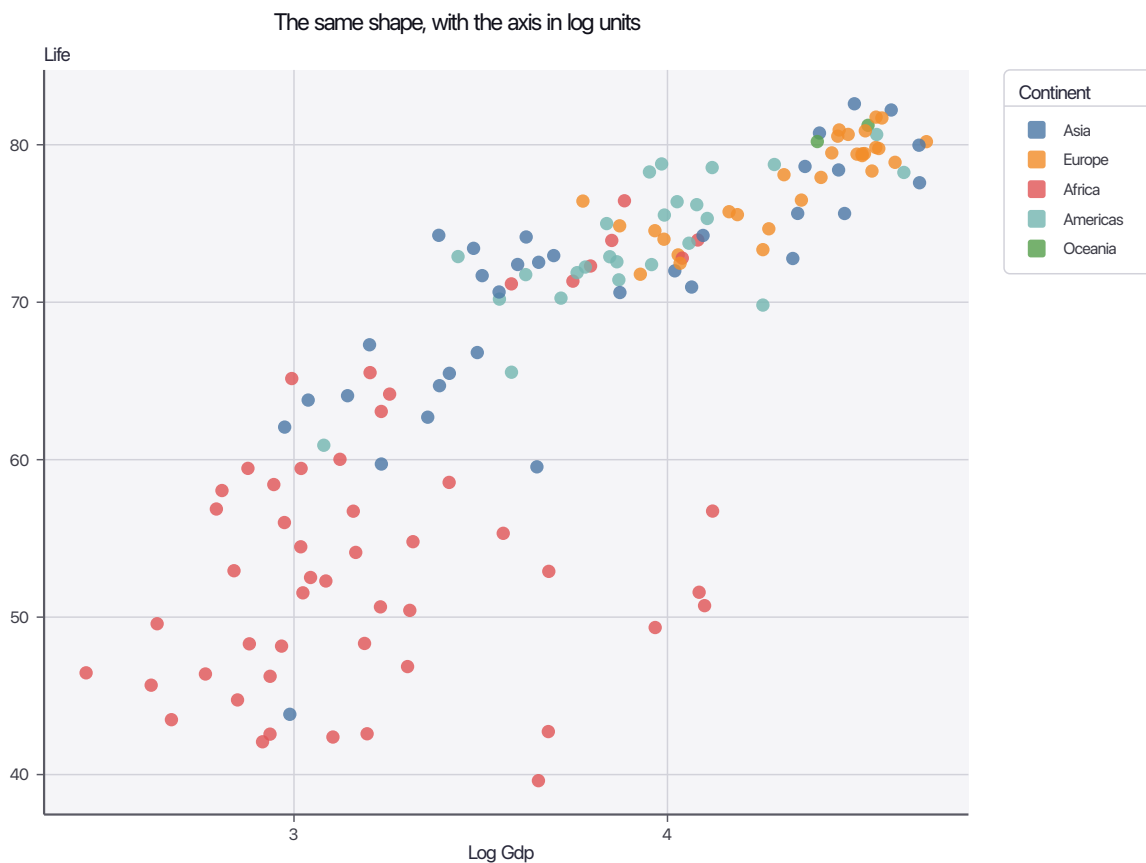
The axis reads 1K and 10K. Those are dollars, the units you supplied.

The word *log* appears nowhere on the plot, and that is deliberate. The units did not change, so the axis name does not claim they did. The ticks are the only sign: each is a whole power of ten, so equal steps along the axis mean equal ratios. Write `x_label("GDP per capita (log scale)")` when a reader needs it in words. The axis name is yours, and `gog` never writes the scale into it.

Now the same picture drawn by logging the column yourself:

```
gm_logged <- gapminder_2007
gm_logged$log_gdp <- log10(gm_logged$gdp)

data(gm_logged) + point + x(log_gdp) + y(life) +
  color(continent) +
  title("The same shape, with the axis in log units")
```



Identical shape, and the axis now reads 3 and 4. To know a country's income, the reader has to raise ten to that power. That arithmetic is the work a scale exists to do for them.

So the choice between the two is about what the quantity *is*. Log the column when the logarithm is itself the quantity of interest: a log-odds, a decibel, a pH. Use a log scale when the quantity is still dollars or people, and you only want to see it across orders of magnitude.

22.2 The order of a scale and a transform

Once a transform is in the plot, a scale is no longer only about display, because a transform has to choose which numbers to work on. `gog` follows one rule, and it has no per-transform exceptions:

A transform groups by one axis and measures on the other. The scale is applied before the transform on the grouping axis, and after it on the measured axis.

22.2.1 Scaling the grouping axis

`bin` groups by `x`, so a `log x` is applied first. The bins are cut on the logarithms, not on the dollars. Each bin then spans one constant *ratio*, so the bars land at a constant spacing and cover the axis:

R

```
data(gapminder_2007) + bar * bin + x(gdp, scale = "log") +  
  x_label("GDP per capita") +  
  title("Bins cut in log space are evenly spaced")
```

Python

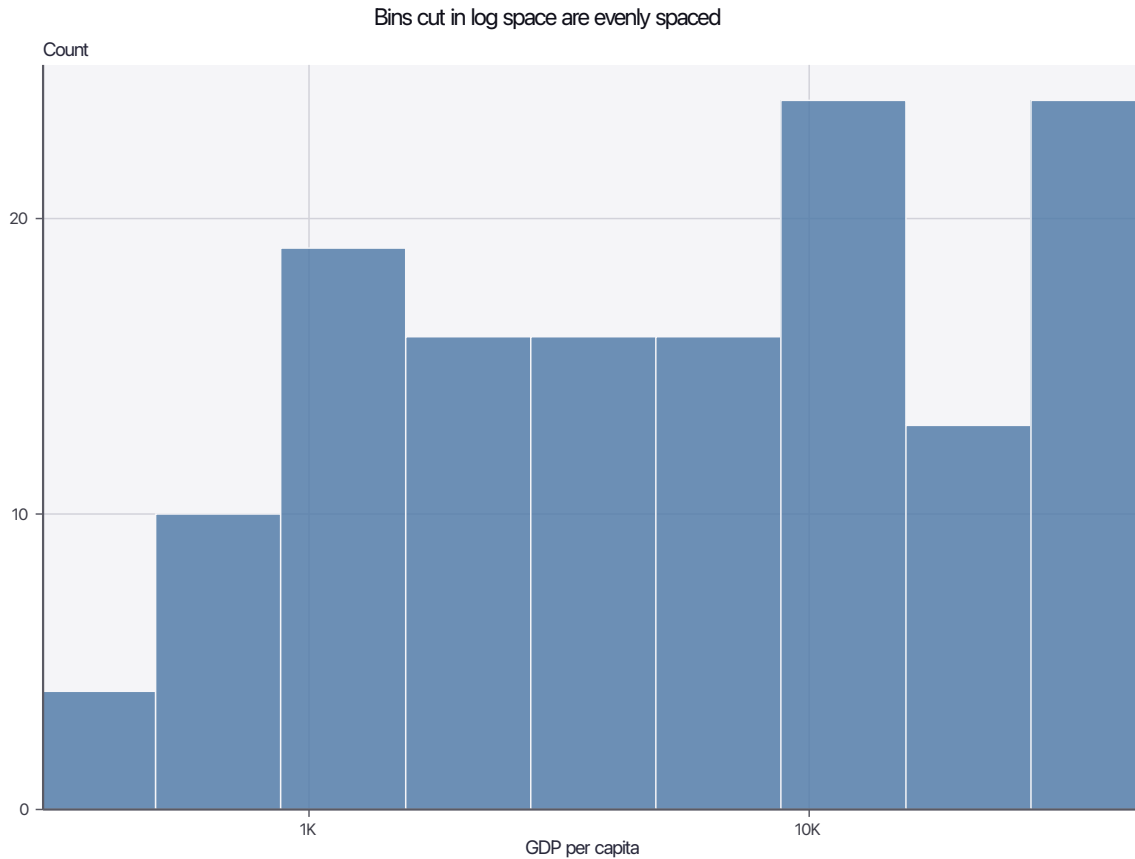
```
(data(gapminder_2007) + bar * bin + x(col.gdp, scale = "log") +  
  x_label("GDP per capita") +  
  title("Bins cut in log space are evenly spaced"))
```

Julia

```
data(gapminder_2007) + bar * bin + x(:gdp, scale = "log") +  
  x_label("GDP per capita") +  
  title("Bins cut in log space are evenly spaced")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.gdp, { scale: "log" }),  
  x_label("GDP per capita"),  
  title("Bins cut in log space are evenly spaced"))
```



“Given gapminder 2007: bars derived by bin, x is gdp on a log scale.”

Suppose the bins had been cut in dollars and only the *drawing* had been logarithmic. The first bin alone would cover most of the axis, and the rest would crowd into the right edge. That reads as a broken picture rather than as a modeling choice, which is why the grouping axis is scaled first.

22.2.2 The measured value keeps its own units

`sum` writes to `y`, so a `log y` is applied *after* the sum. The two bars below stand for 100 and 10:

R

```
receipts <- data.frame(  
  store = c("North", "North", "South", "South"),  
  sales = c(10.0, 90.0, 1.0, 9.0)  
)  
  
data(receipts) + bar * sum + x(store) + y(sales, scale = "log") +  
  title("A sum is still a sum")
```

Python

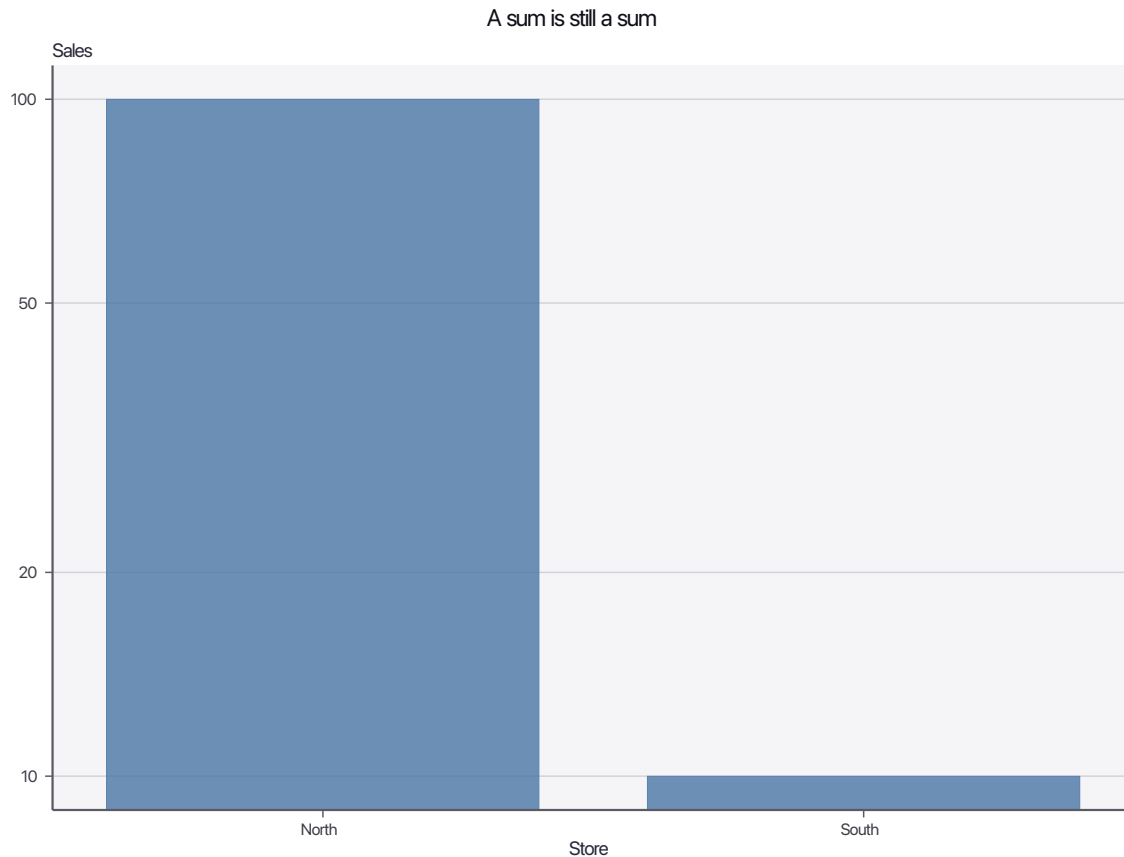
```
receipts = {"store": ["North", "North", "South", "South"], "sales": [10.0,  
↪ 90.0, 1.0, 9.0]}  
(data(receipts) + bar * sum + x(col.store) + y(col.sales, scale = "log") +  
  title("A sum is still a sum"))
```

Julia

```
receipts = (store = ["North", "North", "South", "South"], sales = [10, 90, 1,  
↪ 9],)  
data(receipts) + bar * sum + x(:store) + y(:sales, scale = "log") +  
  title("A sum is still a sum")
```

JavaScript

```
const receipts = { store: ["North", "North", "South", "South"], sales: [10,  
↪ 90, 1, 9] };  
plot(data(receipts), layer(bar, sum), x(col.store),  
  y(col.sales, { scale: "log" }), title("A sum is still a sum"))
```



“Given the receipts: bars derived by sum, x is store, y is sales on a log scale.”

If the scale had been applied first, `gog` would have added the *logarithms*, giving the log of a product, which is not a quantity anyone asked for. Summing first and displaying second keeps `sum` meaning what its name says.

The order is worth knowing if you have used `ggplot2`, which does the opposite. Under `scale_y_log10()`, a `stat_summary(fun = mean)` averages the logged values. What comes back is a geometric mean, the average computed on the logs. `gog` computes the summary in your data’s units and scales the result, so the number the bar stands for is the number you would get from `sum()` or `mean()` yourself.

22.2.3 `smooth` fits inside its groups

`smooth` groups by x , so a log x fits the curve against log income. That is what straightens a relationship otherwise crowded into the left of the panel:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  style(color = "lightgray") +  
  line * smooth + x(gdp, scale = "log") + y(life) +  
  x_label("GDP per capita") +  
  title("A trend fitted against log income")
```

Python

```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  style(color = "lightgray") +  
  line * smooth + x(col.gdp, scale = "log") + y(col.life) +  
  x_label("GDP per capita") +  
  title("A trend fitted against log income"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  style(color = "lightgray") + line * smooth + x(:gdp, scale = "log") +  
  y(:life) + x_label("GDP per capita") +  
  title("A trend fitted against log income")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), style({ color: "lightgray" }), layer(line, smooth),  
  x(col.gdp, { scale: "log" }), y(col.life), x_label("GDP per capita"),  
  title("A trend fitted against log income"))
```



“Given gapminder 2007: points, x is gdp on a log scale, y is life, and also a line derived by smooth, x is gdp on a log scale, y is life.”

22.3 What a log scale refuses

A logarithm is undefined at zero and below, so those rows have no position on the axis. `gog` says so rather than dropping them quietly:

```
readings <- data.frame(  
  depth = c(1.0, 0.0, -4.0, 100.0),  
  temp  = c(12.0, 14.0, 15.0, 9.0)  
)  
  
data(readings) + point + x(depth, scale = "log") + y(temp)
```

Error:

```
! gog: `x(depth, scale = "log")` has no place for 2 of 4 rows - a logarithm
  ↳ is undefined at zero and below, and `depth` reaches -4. Filter those rows
  ↳ before plotting, or use a linear scale.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The message names how many rows are affected and the smallest value in the column, because “some of your data cannot be drawn” is not enough to act on.

Text has no logarithm either, and a text column already gets a categorical axis without being asked:

```
data(gapminder_2007) + point + x(continent, scale = "log") + y(life)
```

Error:

```
! gog: `x(continent, scale = "log")` needs a number to take the logarithm of,
  ↳ but `continent` is text. A text column already gets a categorical axis -
  ↳ remove `scale = "log"`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

22.4 Choosing the base

`scale = "log"` means base 10 unless you say otherwise:

```
x(freq, scale = "log", base = 2)
```

The base is a **number**, not a name. There is no `"log2"` or `"ln"` scale, because that would enumerate what one parameter already derives, the same objection that rules out `ggplot2`'s `scale_x_log10` family.

22.4.1 The base changes the labels, not the picture

One result here surprises most people, and it is worth knowing before you set a base. Any two logarithms of the same column differ by a constant factor, and `gog` normalizes every axis by its own range. The factor cancels, so the points land in *the same places in every base*. The transforms are unaffected too: bins of equal width in $\log_2$ space are equal width in $\log_1$ space, so `bar * bin` cuts at the same values whichever base you name.

What the base actually chooses is where the gridlines fall and how they read. Here are six frequencies, each one an octave above the last, on base 10 and then base 2:

R

```

octaves <- data.frame(
  freq = c(55.0, 110.0, 220.0, 440.0, 880.0, 1760.0),
  level = c(3.0, 6.0, 9.0, 7.0, 4.0, 2.0)
)

data(octaves) + point + x(freq, scale = "log") + y(level) +
  x_label("Frequency (Hz)") + title("Base 10: gridlines at 100, 1K")

```

Python

```

octaves = {"freq": [55.0, 110.0, 220.0, 440.0, 880.0, 1760.0], "level": [3.0,
↪ 6.0, 9.0, 7.0, 4.0, 2.0]}
(data(octaves) + point + x(col.freq, scale = "log") + y(col.level) +
  x_label("Frequency (Hz)") + title("Base 10: gridlines at 100, 1K"))

```

Julia

```

octaves = (freq = [55, 110, 220, 440, 880, 1760], level = [3, 6, 9, 7, 4,
↪ 2],)
data(octaves) + point + x(:freq, scale = "log") + y(:level) +
  x_label("Frequency (Hz)") + title("Base 10: gridlines at 100, 1K")

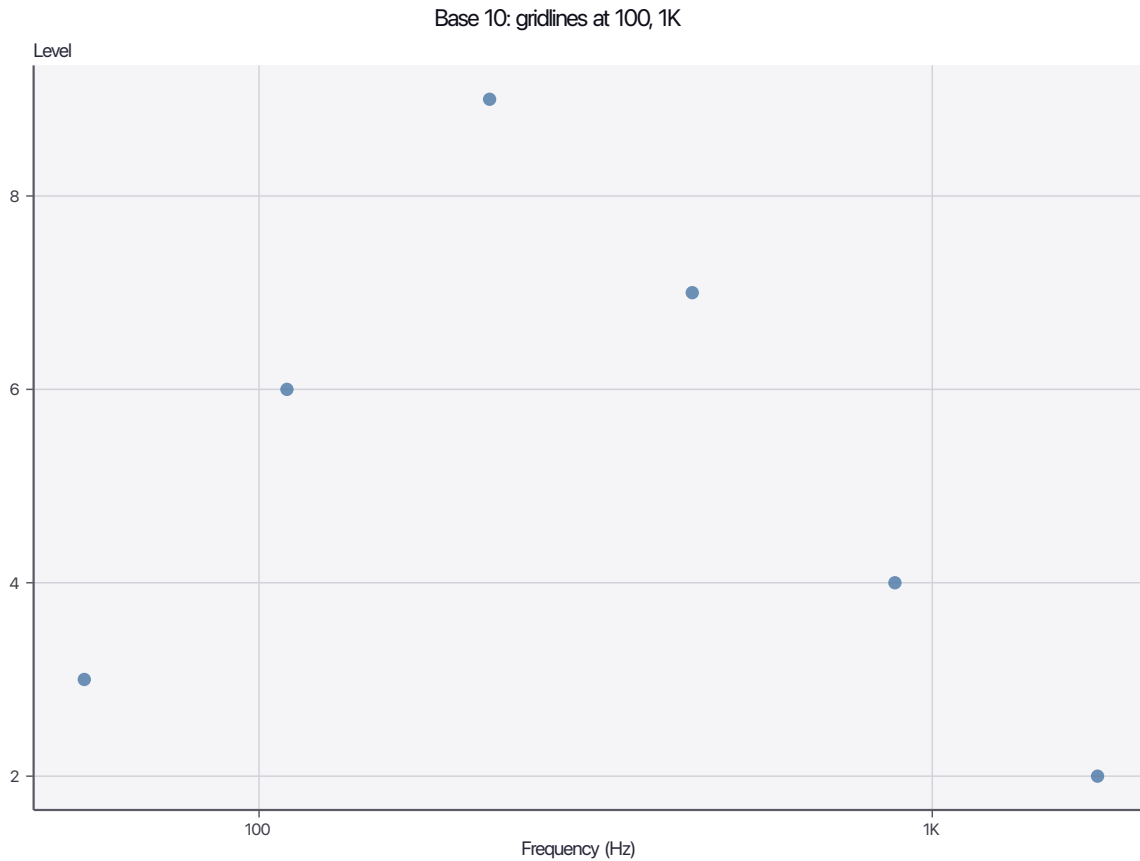
```

JavaScript

```

const octaves = { freq: [55, 110, 220, 440, 880, 1760], level: [3, 6, 9, 7,
↪ 4, 2] };
plot(data(octaves), point, x(col.freq, { scale: "log" }), y(col.level),
  x_label("Frequency (Hz)", title("Base 10: gridlines at 100, 1K"))

```



R

```
data(octaves) + point + x(freq, scale = "log", base = 2) + y(level) +
  x_label("Frequency (Hz)") + title("Base 2: gridlines on every octave")
```

Python

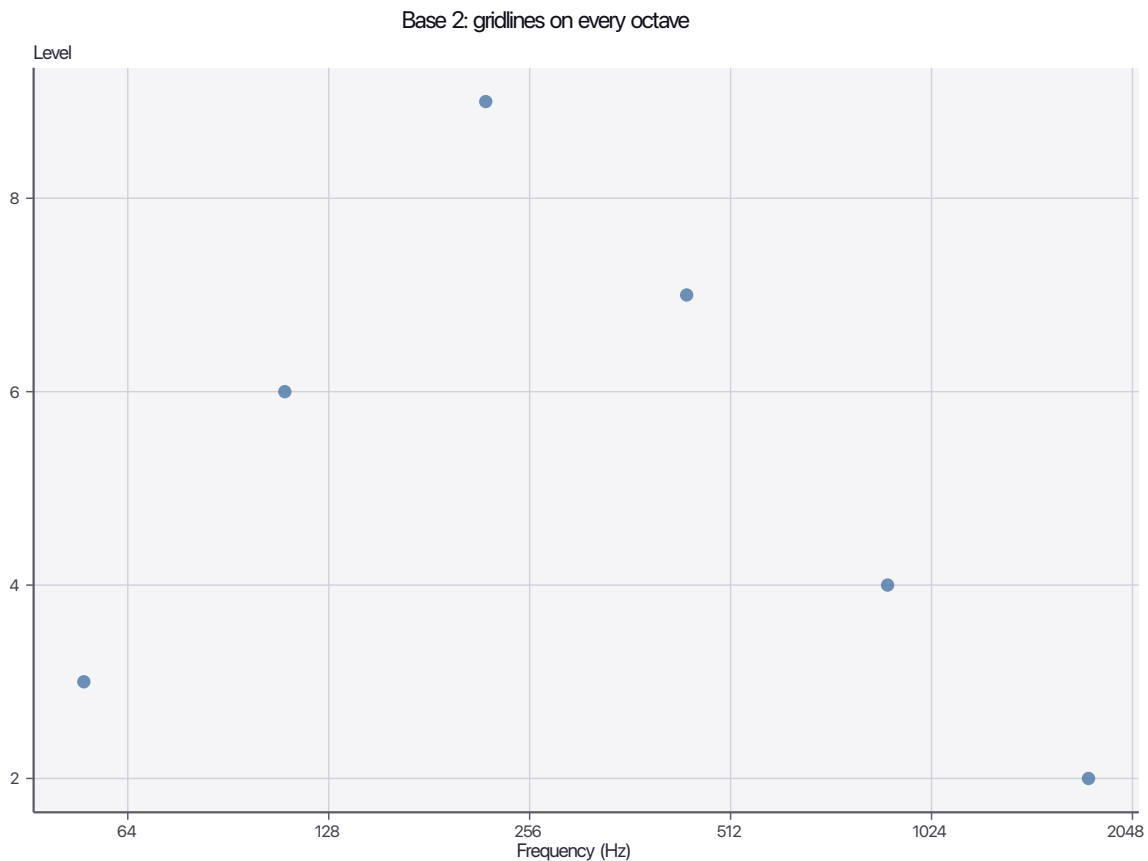
```
(data(octaves) + point + x(col.freq, scale = "log", base = 2) + y(col.level)
↪ +
  x_label("Frequency (Hz)") + title("Base 2: gridlines on every octave"))
```

Julia

```
data(octaves) + point + x(:freq, scale = "log", base = 2) + y(:level) +  
  x_label("Frequency (Hz)") + title("Base 2: gridlines on every octave")
```

JavaScript

```
plot(data(octaves), point, x(col.freq, { scale: "log", base: 2 } ),  
  y(col.level), x_label("Frequency (Hz)",  
  title("Base 2: gridlines on every octave"))
```



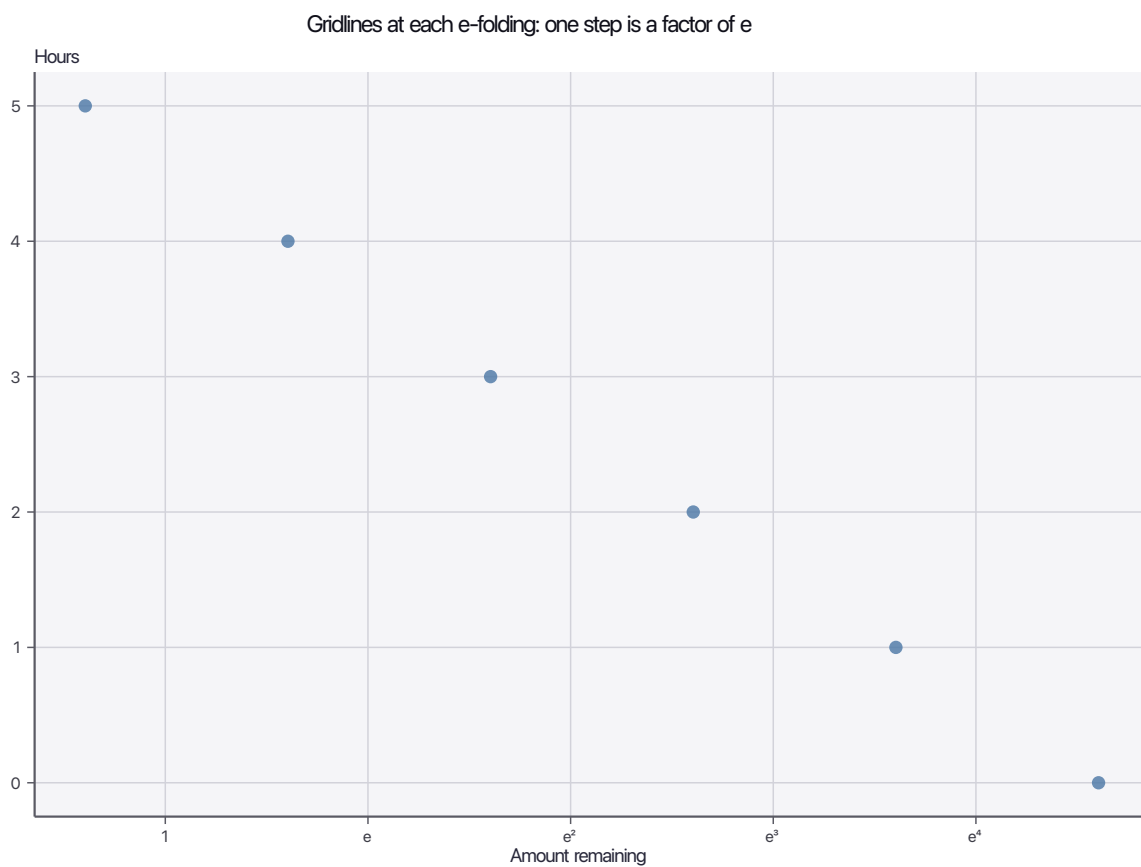
“Given the octaves: points, x is freq on a log scale in base 2, y is level.”

The points have not moved. The second chart puts a gridline on each doubling, which is what “an octave” means, so for pitch, bits, or doubling times, base 2 is the one whose labels name what you are counting.

22.4.2 Natural log

R has no `e` constant, so it is `base = exp(1)`:

```
decay <- data.frame(  
  hours = 0:5 + 0.0,  
  amount = exp(-(0:5)) * 100  
)  
  
data(decay) + point + x(amount, scale = "log", base = exp(1)) + y(hours) +  
  x_label("Amount remaining") +  
  title("Gridlines at each e-folding: one step is a factor of e")
```



Notice the labels: e , e^2 , e^3 , not 2.718, 7.389, 20.09. That follows from one rule `gog` applies to every base:

Label the quantity when it is a short number; otherwise label the power.

Base 10 gives 1, 10, 100, 1K, always clean. Base 2 gives 1, 2, 4 ... 1048576, clean until it grows too wide to sit under a tick, and then $2^2 \cdot 4$. Base e has no clean quantity above 1, so every tick past the first reads as a power, which is precisely what somebody counting e-foldings wanted. If instead you want the axis to read 0, 1, 2, 3 in ln units, you do not want a log scale at all. You want `log()` applied to the column, and an axis honestly labeled in log units, as at the top of this chapter. A scale keeps the reader's units; a logged column changes them.

A base only means something as the base *of* a logarithm, so `gog` refuses one that has no log scale to belong to:

```
data(octaves) + point + x(freq, base = 2) + y(level)
```

Error:

```
! gog: `x(freq, base = ...)` has no scale to be the base of. A base belongs to
↳ a logarithm - add `scale = "log"`, or remove the base.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

22.5 Time: a date column gets a calendar axis

The time scale is one you never have to write. A `Date` or `POSIXct` column *is* temporal, the type already says so, and `gog` reads the calendar from the type, the same way a text column already gets a categorical axis:

```
revenue <- data.frame(
  day   = as.Date(paste0(1994:2019, "-06-30")),
  sales = c(52, 57, 55, 61, 64, 60, 68, 66, 71, 75, 74, 80, 78, 85,
            72, 72, 79, 84, 88, 87, 93, 96, 94, 101, 105, 103)
)

data(revenue) + line + x(day) + y(sales) +
  title("Twenty-six years, ticked in years")
```




No `scale = "time"` anywhere: writing it is allowed and means nothing extra, exactly like writing `scale = "linear"` on a number.

The ticks land on **calendar boundaries**, because the calendar is not decimal. An axis of years steps 1, 2 or 5 like any number line. Below the year those steps mean nothing: nobody reads an axis cut in fifths of a year. Months step by 1, 2, 3 or 6, so January stays a gridline and 3-month steps read as quarters. Weeks land on **Mondays**, and days, hours and minutes take the steps a clock face suggests.

R

```
# six_weeks is the cast's daily orders table, see the Preface
data(six_weeks) + line + x(day) + y/orders) +
  title("Six weeks, ticked on Mondays")
```

Python

```
(data(six_weeks) + line + x(col.day) + y(col.orders) +
  title("Six weeks, ticked on Mondays"))
```

Julia

```
data(six_weeks) + line + x(:day) + y(:orders) +
  title("Six weeks, ticked on Mondays")
```

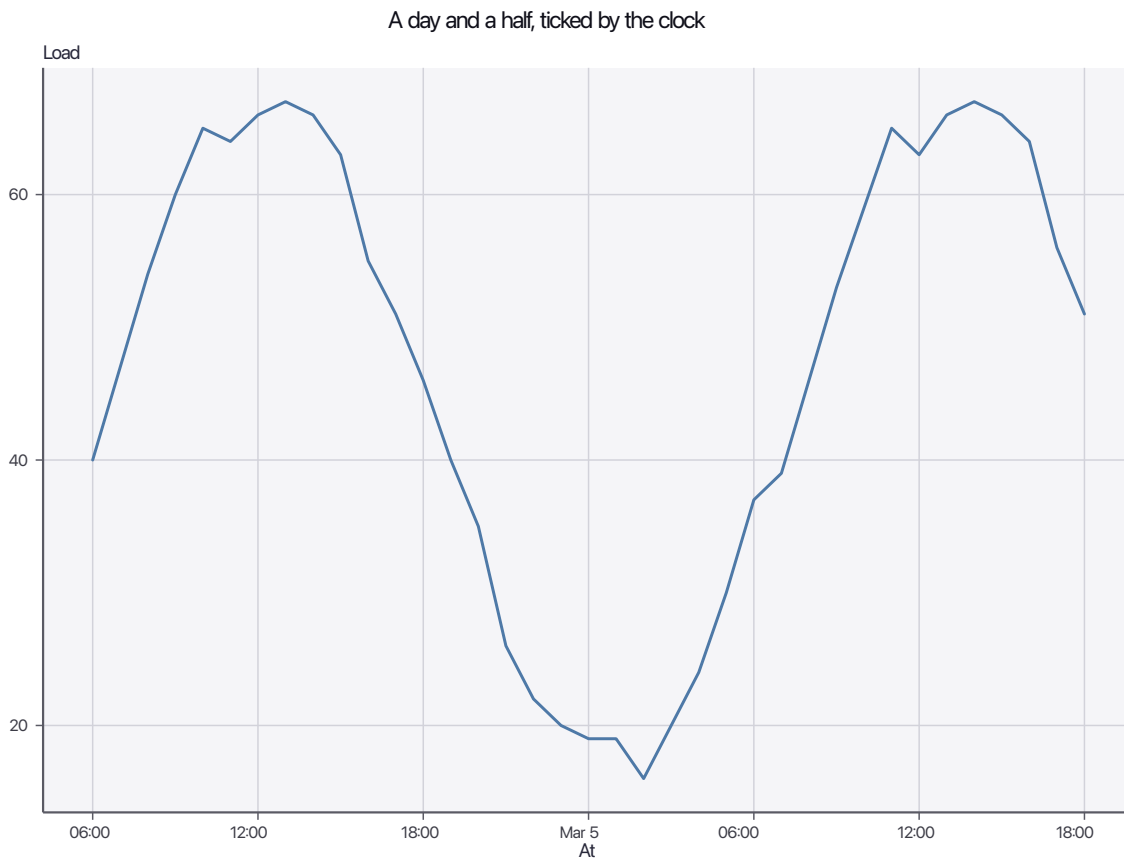
JavaScript

```
plot(data(six_weeks), line, x(col.day), y(col.orders),
  title("Six weeks, ticked on Mondays"))
```



A tick is labeled at its own resolution: 2010 for a year step, Jan 2024 for a month step, Mar 4 for a day step. A `Date` column never gets clock ticks, however narrow its range; a `POSIXct` may:

```
monitoring <- data.frame(  
  at = as.POSIXct("2024-03-04 06:00", tz = "UTC") + 3600 * 0:36,  
  load = round(40 + 25 * sin(0:36 / 4) + (0:36 %% 5))  
)  
  
data(monitoring) + line + x(at) + y(load) +  
  title("A day and a half, ticked by the clock")
```



00:00 happens every day, so it names no particular one. The midnight tick shows the day's name instead. One rule covers every label on a time axis: *a tick names the finest unit that distinguishes it from its neighbors.*

22.5.1 Bars sit on dates; they never measure one

Bars positioned at dates are an ordinary time series, and the ticks still come from the calendar rather than one per bar. Wilkinson's stock-price example ticks Sundays, not trades (Wilkinson, 2005), and the same rule here ticks the six weeks of orders at week boundaries:

R

```
data(six_weeks) + bar + x(day) + y(orders) +  
  title("Daily bars, weekly ticks")
```

Python

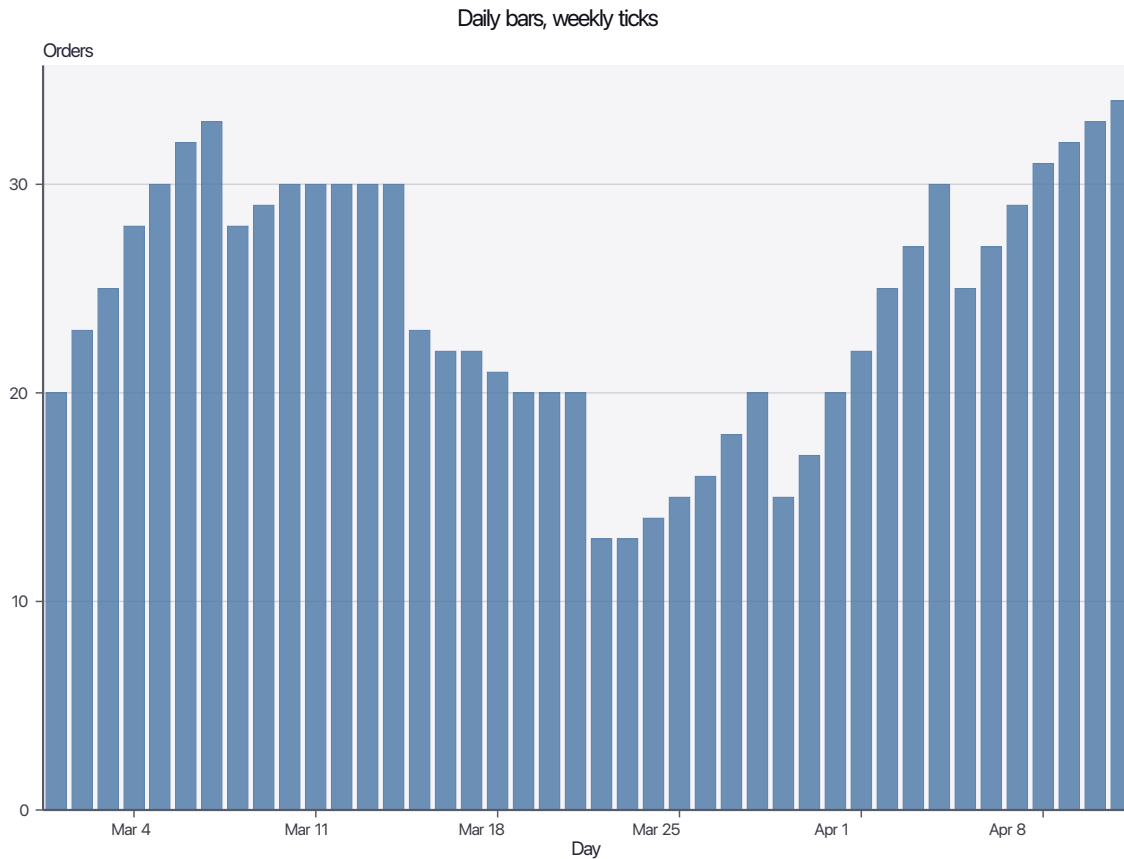
```
(data(six_weeks) + bar + x(col.day) + y(col.orders) +  
  title("Daily bars, weekly ticks"))
```

Julia

```
data(six_weeks) + bar + x(:day) + y(:orders) +  
  title("Daily bars, weekly ticks")
```

JavaScript

```
plot(data(six_weeks), bar, x(col.day), y(col.orders),  
  title("Daily bars, weekly ticks"))
```



The other direction is refused. A bar’s *length* is an amount, and a moment in time is not an amount. A bar “reaching” 2007 would be measured from 1970, an origin nobody chose:

```
data(revenue) + bar + x(sales) + y(day)
```

Error:

```
! gog: `bar` measures its length along `day`, but that is a date column - a
  ↳ bar's length is an amount, and a moment in time is not an amount. Put the
  ↳ date on `x()` and measure a number, or use `point`/`line` for values that
  ↳ are dates.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

22.5.2 What a time axis refuses

A moment has no logarithm. The calendar’s zero is an arbitrary convention, and a logarithm measured from an arbitrary zero measures nothing:

```
data(revenue) + line + x(day, scale = "log") + y(sales)
```

Error:

```
! gog: `x(day, scale = "log")` - `day` is a date column, and a moment in time
  ↳ has no logarithm: the calendar's zero is an arbitrary origin. Log the
  ↳ measured axis instead, or remove the scale.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The reading works in one direction only. A date column is temporal by type, so a plain number cannot be *declared* temporal at the binding. The engine cannot know whether a plain number like 20656 counts days, seconds, or years:

```
data(revenue) + point + x(sales, scale = "time") + y(sales)
```

Error:

```
! gog: `x(sales, scale = "time")` - `sales` is not a date column, and a
  ↳ number alone does not say what moment it is. Convert it with `as.Date()`
  ↳ (or `as.POSIXct()`); gog reads the calendar from the column's type.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The fix belongs in the data, where the answer is known: convert the column with `as.Date()` or `as.POSIXct()`.

A word on timezones. The engine never converts one; it draws the clock time you gave it. A `POSIXct` is formatted in its own timezone when it is drawn, and never converted, so the axis can never disagree with what `print()` showed you. If two series must share an axis, put them in one timezone before plotting; that decision belongs to the analysis, not the engine.

22.5.3 Time on the other channels

A scale answers *how far along?* on every channel that measures, and a date column is no exception: how recent a row is, shown as a color ramp, is a normal question. A legend row stands alone, with no neighboring ticks to give it context, so it carries the full date:

R

```
data(six_weeks) + point + x(day) + y(orders) + color(day) +
  title("Recent days, darker")
```

Python

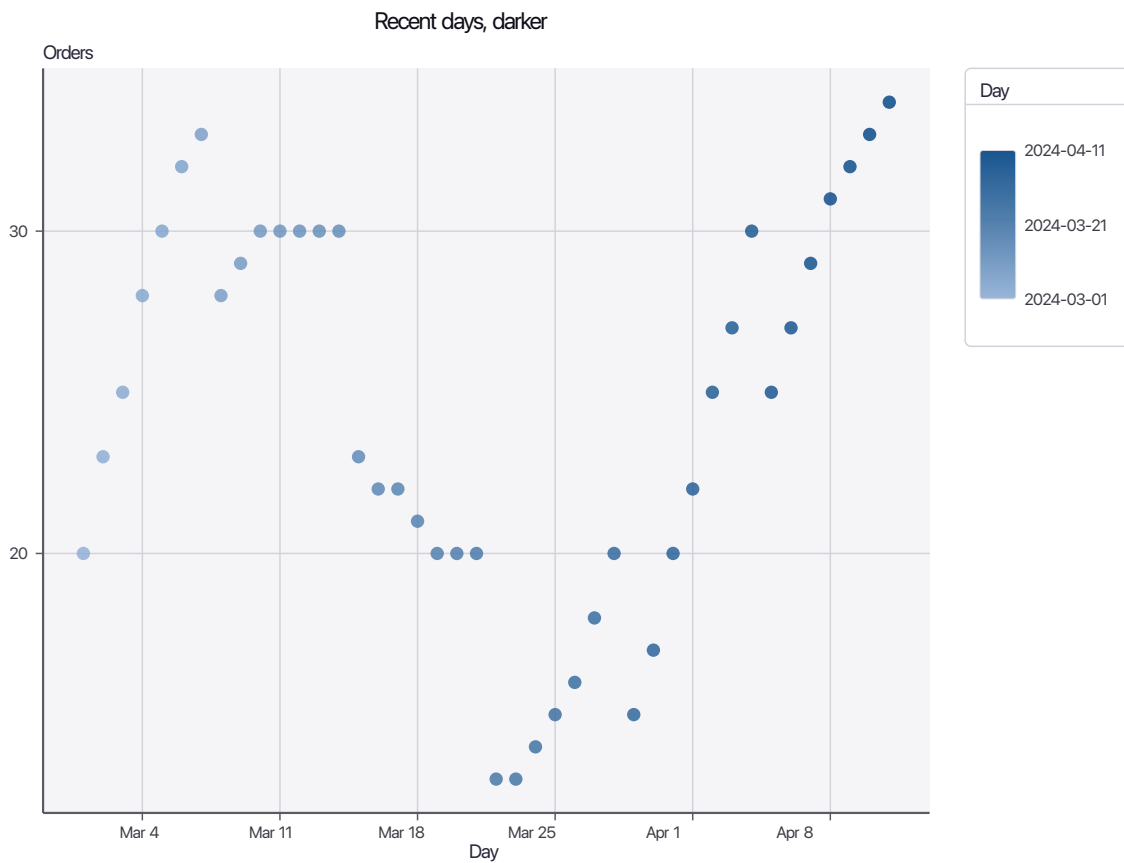
```
(data(six_weeks) + point + x(col.day) + y(col.orders) + color(col.day) +
  title("Recent days, darker"))
```

Julia

```
data(six_weeks) + point + x(:day) + y(:orders) + color(:day) +
  title("Recent days, darker")
```

JavaScript

```
plot(data(six_weeks), point, x(col.day), y(col.orders), color(col.day),
  title("Recent days, darker"))
```



22.6 The scales

Scale	What it does
linear	Equal steps for equal differences. The default
log	Equal steps for equal ratios, on <code>x</code> , <code>y</code> , <code>color</code> , <code>size</code> and <code>opacity</code> , with any <code>base</code> above 1
time	Positions dates and times, chosen from the column type: <code>Date</code> and <code>POSIXct</code> columns, on every channel that measures
category	One slot per distinct value, chosen from the column type: text columns, and columns with a declared order

22.7 The axis fits your data

An axis shows round numbers on its ticks, but the **range** follows your data, not the round numbers. The ticks are the nice values that fall *inside* the data, or at most a little past it into the margin; the axis does not stretch out to the next ten just to end on one. A span of 1952–2007 is drawn as 1952–2007, with ticks at 1960, 1980 and 2000, not padded to 1940–2020 with a third of the panel left blank:

R

```
data(gapminder_asia) + area + x(year) + y(life) +  
  group(country) + style(opacity = 0.4) +  
  title("The year axis fits 1952–2007; the ticks fall inside it")
```

Python

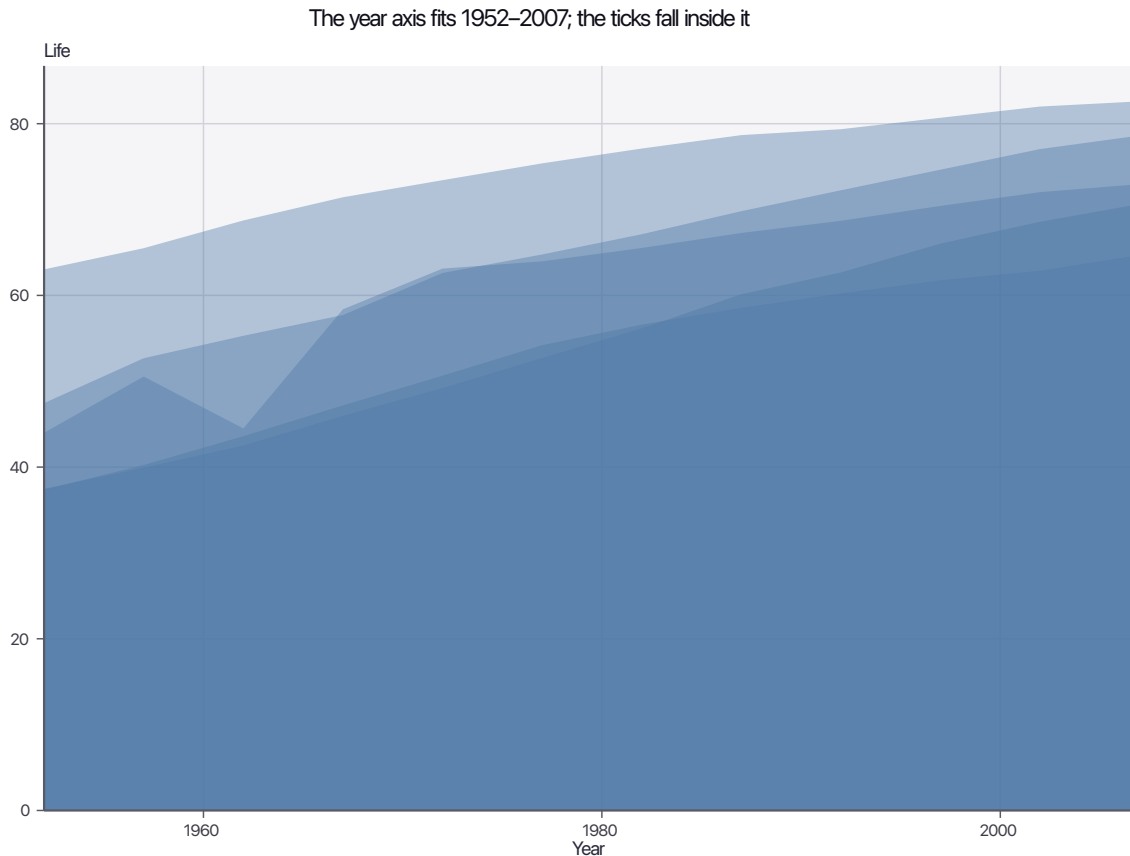
```
(data(gapminder_asia) + area + x(col.year) + y(col.life) +  
  group(col.country) + style(opacity = 0.4) +  
  title("The year axis fits 1952–2007; the ticks fall inside it"))
```

Julia

```
data(gapminder_asia) + area + x(:year) + y(:life) + group(:country) +  
  style(opacity = 0.4) +  
  title("The year axis fits 1952–2007; the ticks fall inside it")
```


JavaScript

```
plot(data(gapminder_asia), area, x(col.year), y(col.life),  
  group(col.country), style({ opacity: 0.4 } ),  
  title("The year axis fits 1952–2007; the ticks fall inside it"))
```



“Given gapminder Asia: areas, x is year, y is life, grouped by country.”

Two more rules refine that fit, and the plot shows the reason for each. A *free end* leaves a small margin, so a mark never touches the panel edge. A point at the exact maximum would otherwise be sliced in half by the panel edge:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent) +  
  x_label("GDP per capita") + title("Every point sits clear of the panel  
  ↪ edge")
```

Python

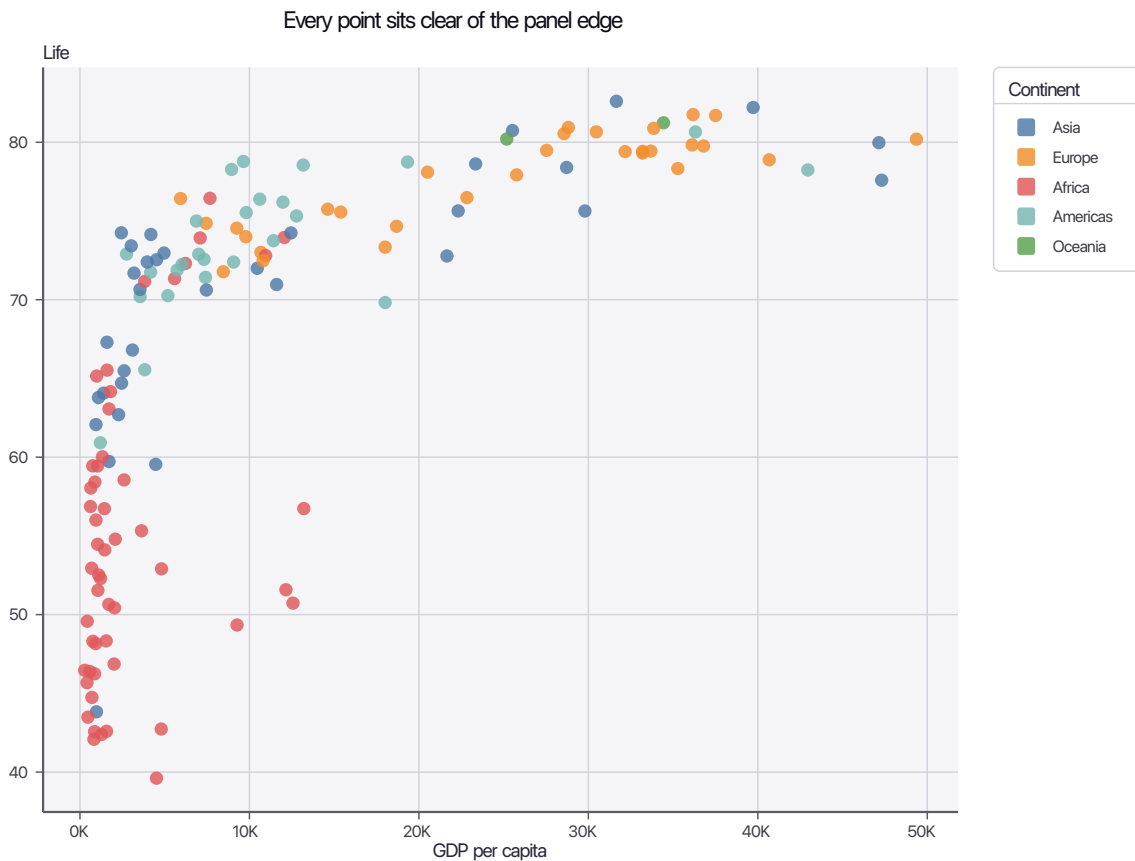
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ color(col.continent) +  
  x_label("GDP per capita") + title("Every point sits clear of the panel  
  ↪ edge"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  x_label("GDP per capita") +  
  title("Every point sits clear of the panel edge")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent), x_label("GDP per capita"),  
  title("Every point sits clear of the panel edge"))
```



A baseline gets no margin. When a bar or an area measures from zero, that zero is a real coordinate. A gap beneath it would draw the baseline in the wrong place, so the baseline sits on the panel edge. An `area` has no glyph to clip, so along the axis it fills it runs to both panel edges. That is why the year area spans the full width while the income scatter keeps its margins: a fill's edges *are* the shape, a point's are not.

22.8 A scale is not only for axes

`x` and `y` turn a number into a position, but so do `color`, `size` and `opacity`: a ramp position, a radius, a transparency. All five answer *how far along?*, so all five take a scale, and `z` takes one for the same reason.

This matters most for a column spread across orders of magnitude. `population` runs from about 200 thousand to 1.3 billion. On a linear ramp the median country sits in the bottom 2% of that range, so nearly every point gets the same color:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(population) +  
  x_label("GDP per capita") + title("Linear color: nearly one flat color")
```

Python

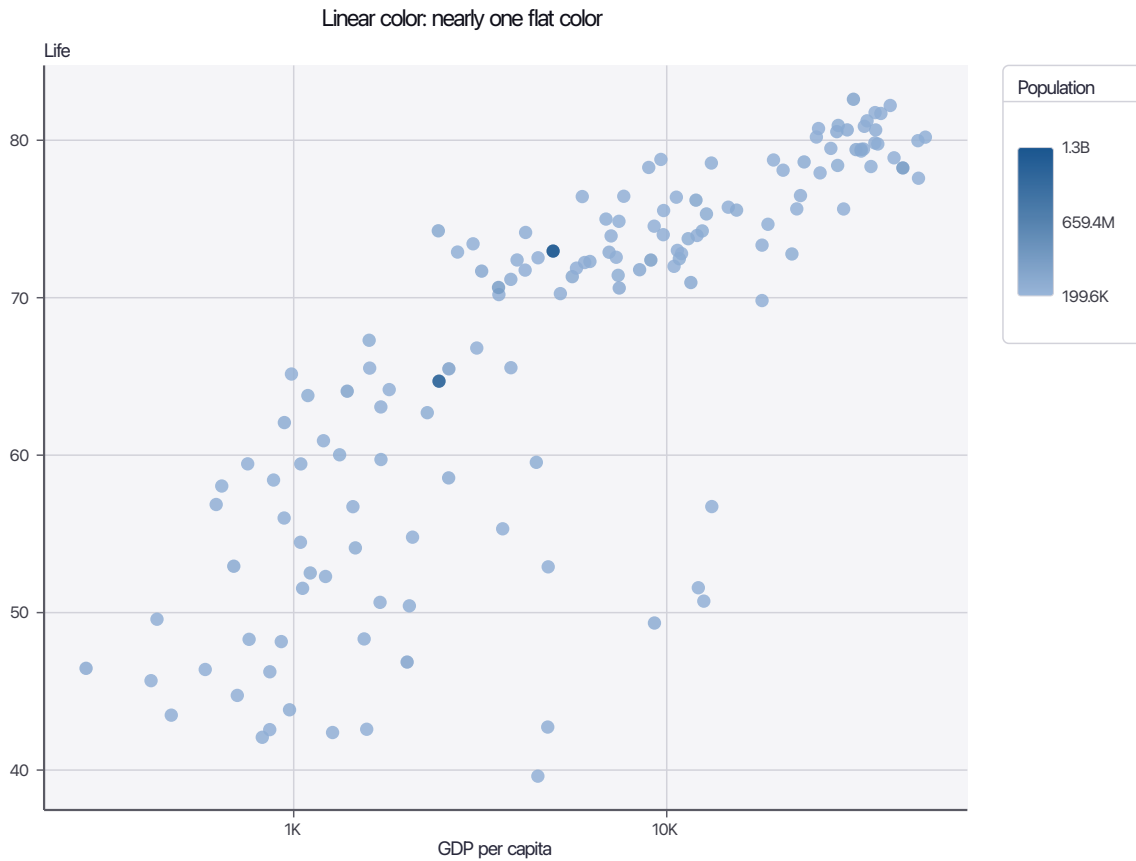
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.population) +  
  x_label("GDP per capita") + title("Linear color: nearly one flat color"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:population) + x_label("GDP per capita") +  
  title("Linear color: nearly one flat color")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.population), x_label("GDP per capita"),  
  title("Linear color: nearly one flat color"))
```



R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +
  color(population, scale = "log") +
  x_label("GDP per capita") + title("Log color: the ramp is actually used")
```

Python

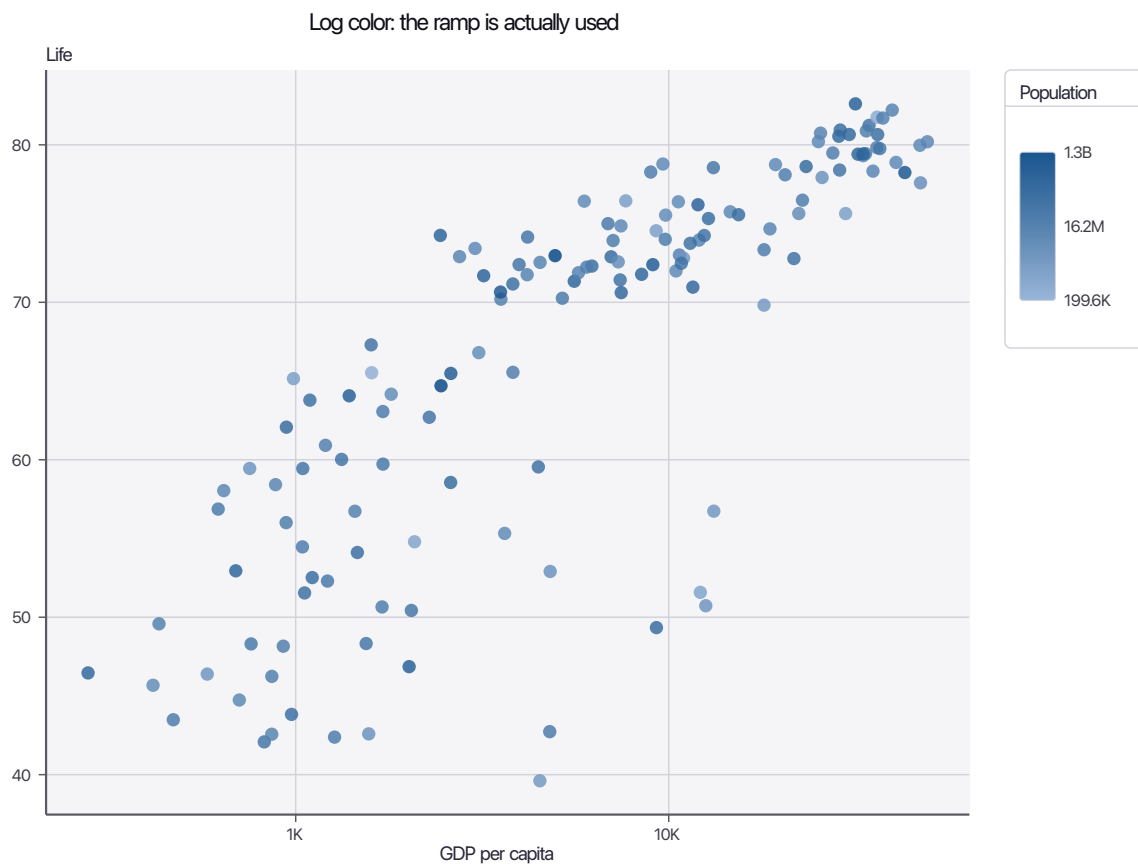
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +
  color(col.population, scale = "log") +
  x_label("GDP per capita") + title("Log color: the ramp is actually used"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +
  color(:population, scale = "log") + x_label("GDP per capita") +
  title("Log color: the ramp is actually used")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),
  y(col.life), color(col.population, { scale: "log" }),
  x_label("GDP per capita"),
  title("Log color: the ramp is actually used"))
```



“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by population on a log scale.”

The same is true of **size**, where a linear scale leaves almost every point at the minimum radius:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  size(population, scale = "log") + style(color = "steelblue") +  
  x_label("GDP per capita") + title("Log size")
```

Python

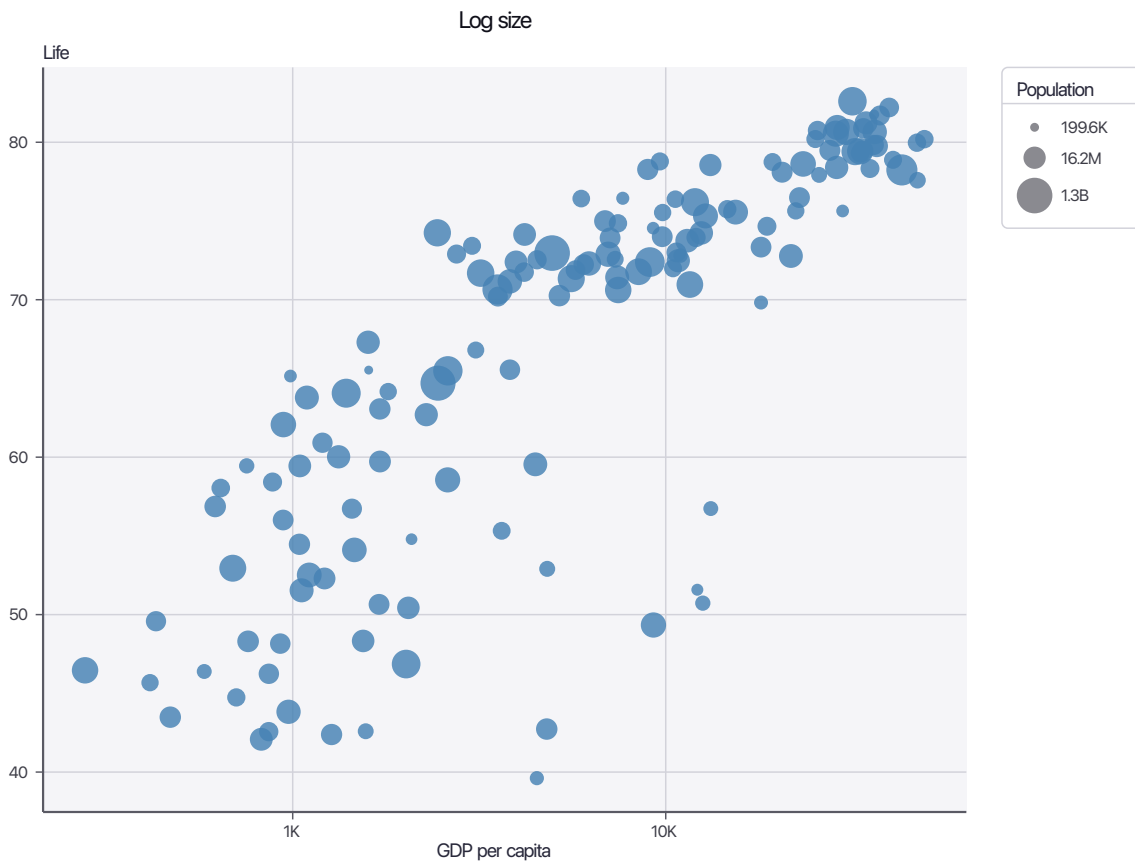
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  size(col.population, scale = "log") + style(color = "steelblue") +  
  x_label("GDP per capita") + title("Log size"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  size(:population, scale = "log") + style(color = "steelblue") +  
  x_label("GDP per capita") + title("Log size")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), size(col.population, { scale: "log" }),  
  style({ color: "steelblue" }), x_label("GDP per capita"),  
  title("Log size"))
```



“Given gapminder 2007: points, x is gdp on a log scale, y is life, size by population on a log scale.”

Look again at the middle row of the two color legends further up. On a linear ramp it is the arithmetic midpoint; on a log ramp it is the **geometric** one: $\sqrt{(\min \cdot \max)}$, about 16M rather than 660M. It has to be, because that label names the color painted at the middle of the color ramp.

`shape` and `group` take no `scale` argument. They answer *which one?*, and there is no distance between circle and square for a scale to run along.

22.9 limits: when your data is not the authority

Everything above derives the range from the data, which is right by default and wrong whenever the data does not cover the domain. A tide gauge reading every three hours never reaches midnight. An axis drawn as a circle cannot know the missing hours are still part of the day,

because nothing in a column of hours says the day wraps. The `tide` table holds a gauge's readings, one height every three hours. `polar()` draws the axis as a circle, and `Polar` teaches that space in full. `limits` states the domain:

Table 22.2: `tide`: first 5 of 8 rows

hour	height
1	2.1
4	4.4
7	3.6
10	1.2
13	0.9

R

```
data(tide) + line + x(hour, limits = c(0, 24)) + y(height) + polar() +
  title("The whole day, whatever hours the gauge recorded")
```

Python

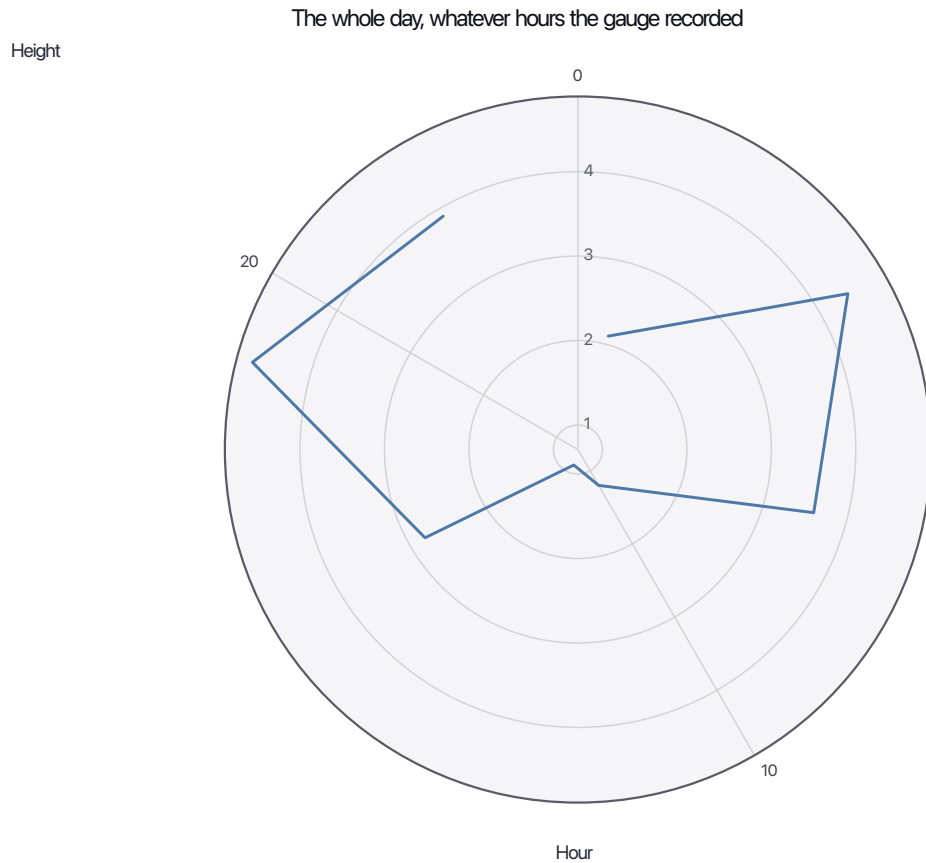
```
(data(tide) + line + x(col.hour, limits = [0, 24]) + y(col.height) + polar()
↪ +
  title("The whole day, whatever hours the gauge recorded"))
```

Julia

```
data(tide) + line + x(:hour, limits = [0, 24]) + y(:height) + polar() +
  title("The whole day, whatever hours the gauge recorded")
```

JavaScript

```
plot(data(tide), line, x(col.hour, { limits: [0, 24] }), y(col.height),
  polar(), title("The whole day, whatever hours the gauge recorded"))
```



“Given the tide: a line, x is hour from 0 to 24, y is height, in polar.”

Without that, one turn would cover 1 to 22 and the first and last readings would be drawn on the same spoke. With it, each sits where a clock would put it. The curve still does not close, and should not. The gauge took no reading at midnight, and a domain says what range the axis covers rather than inventing an observation. That gap is the data’s, drawn at its true width. [Polar](#) sets out the three ways a circle does and does not close.

`limits` is written on the binding, beside `scale` and `base`, for the same reason they are: a domain belongs to one channel, and an atom of its own would name that channel twice. A stated end is drawn exactly where you state it. The margin exists to keep a glyph off the panel edge, and an end you chose is not a data extreme that needs the room.

A domain can widen the axis as easily as it narrows it. Nothing was dropped above: the readings run 1 through 22 and all of them lie inside 0 through 24. Narrowing is the direction that removes rows, and it says so:

R

```
data(gapminder_2007) + point + x(gdp, limits = c(0, 30000)) + y(life) +
  color(continent) + x_label("GDP per capita") +
  title("Under $30,000, and the countries above it are reported")
```

Python

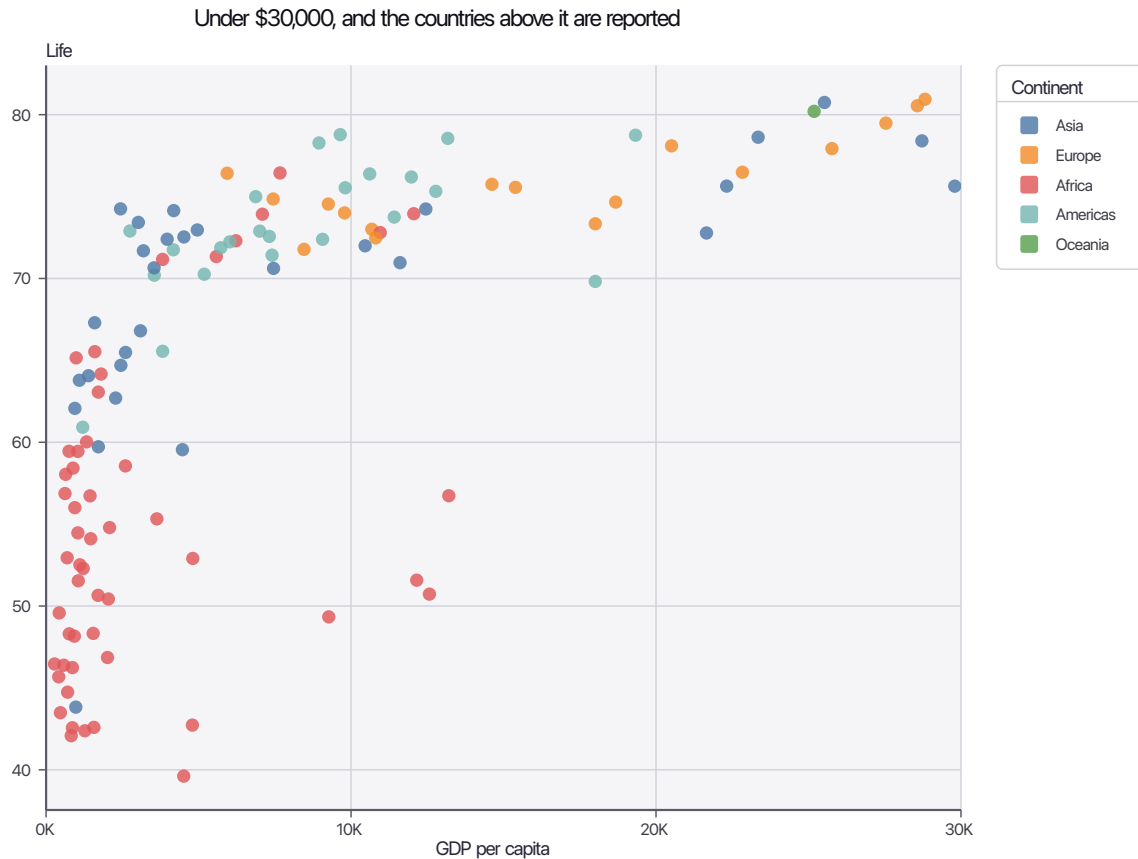
```
(data(gapminder_2007) + point + x(col.gdp, limits = [0, 30000]) + y(col.life)
↪ +
  color(col.continent) + x_label("GDP per capita") +
  title("Under $30,000, and the countries above it are reported"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, limits = [0, 30000]) + y(:life) +
  color(:continent) + x_label("GDP per capita") +
  title("Under \$30,000, and the countries above it are reported")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { limits: [0, 30000] }),
  y(col.life), color(col.continent), x_label("GDP per capita"),
  title("Under $30,000, and the countries above it are reported"))
```



It also prints a line on your console, naming how many rows fell outside and what the domain was. This page cannot show you that line, because only refusals appear in a rendered book. That is deliberate: removing them is what you asked for, so the plot draws, but `gog` will not remove data without saying so. Ask for a domain no row can satisfy and it refuses instead, because the alternative is an empty panel with axes drawn over nothing.

Keeping every country on the page and marking one group of them is a different operation, and it has a different word. `limits` runs before the transforms and removes rows. `brush` runs after the picture is composed and removes nothing. [Selection](#) sets the two side by side.

You can state one end alone. An empty end leaves that end to the data, which is how you fix a baseline and let the top follow the data:

R

```
data(gapminder_2007) + point * mean + x(continent) + y(life, limits = c(0,
↪ NA)) +
  title("y from zero, top still fitted to the data")
```

Python

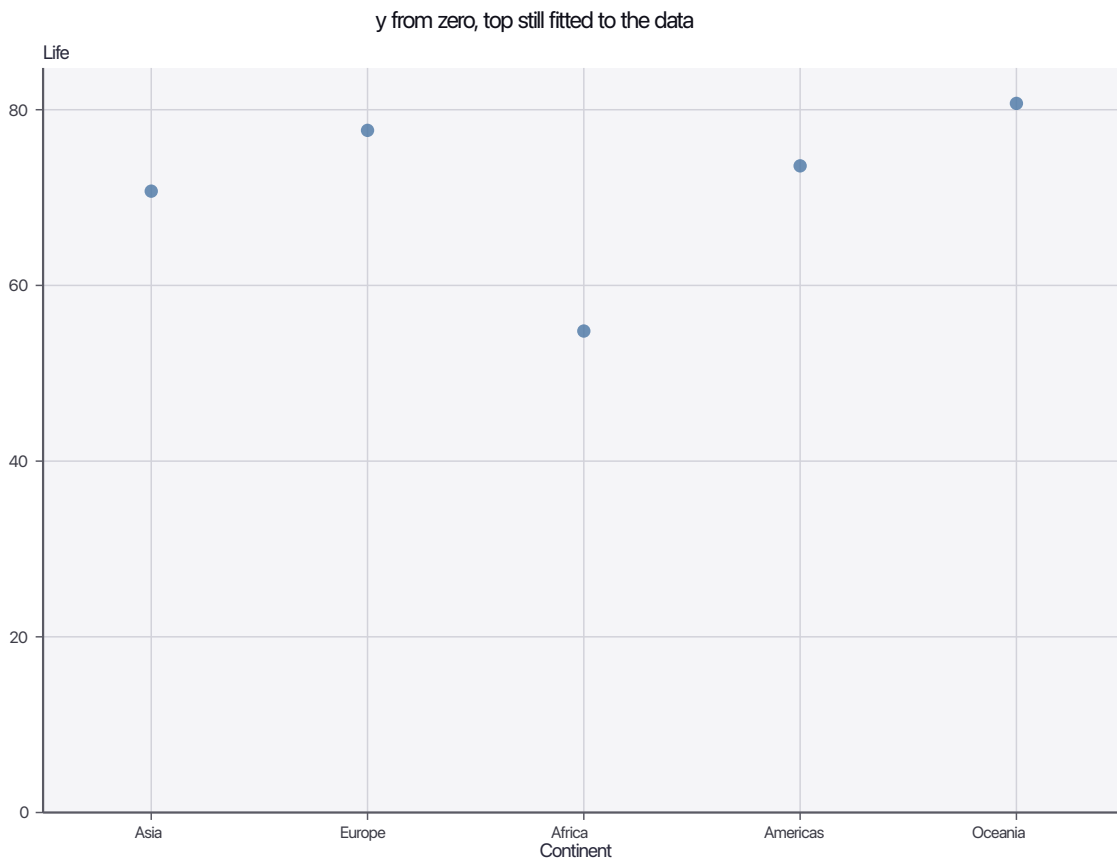
```
(data(gapminder_2007) + point * mean + x(col.continent) + y(col.life, limits  
↪ = [0, None]) +  
  title("y from zero, top still fitted to the data"))
```

Julia

```
data(gapminder_2007) + point * mean + x(:continent) +  
  y(:life, limits = [0, missing]) +  
  title("y from zero, top still fitted to the data")
```

JavaScript

```
plot(data(gapminder_2007), layer(point, mean), x(col.continent),  
  y(col.life, { limits: [0, null] }),  
  title("y from zero, top still fitted to the data"))
```



“Given gapminder 2007: points derived by mean, x is continent, y is life from 0.”

It is not only for axes. A domain is a property of the scale, so it reaches every channel that measures. That is how you fix one color ramp across plots built from different data:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(population, scale = "log", limits = c(1e5, 2e9)) +  
  x_label("GDP per capita") + title("A ramp fixed from 100K to 2B")
```

Python

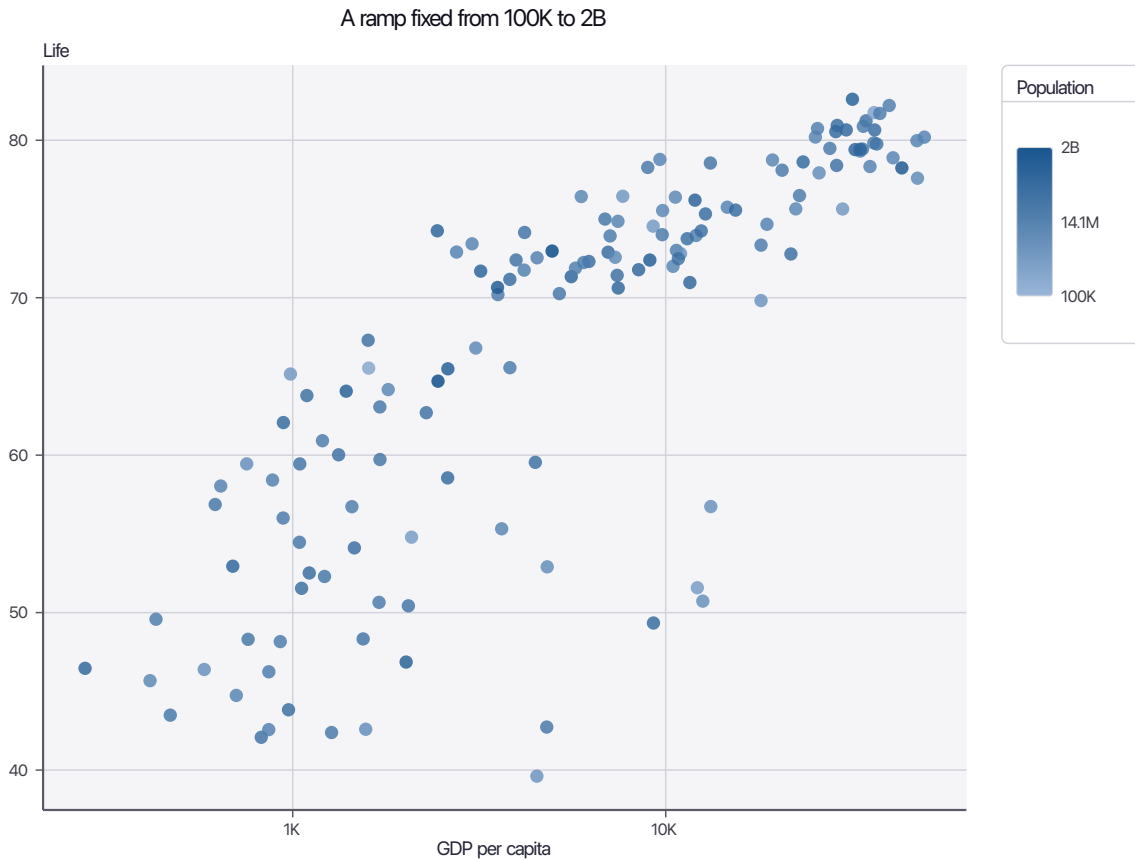
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.population, scale = "log", limits = [1e5, 2e9]) +  
  x_label("GDP per capita") + title("A ramp fixed from 100K to 2B"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:population, scale = "log", limits = [1e+05, 2e+09]) +  
  x_label("GDP per capita") + title("A ramp fixed from 100K to 2B")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life),  
  color(col.population, { scale: "log", limits: [1e+05, 2e+09] }),  
  x_label("GDP per capita"), title("A ramp fixed from 100K to 2B"))
```



On a categorical axis `limits` is refused, and the refusal points you at the two fixes that do the job you probably meant. A category has no range to lie inside. Choosing *which* categories appear is filtering your table, and choosing what order they appear in is `order()`.

```
data(gapminder_2007) + bar * mean + x(continent, limits = c(0, 3)) + y(life)
```

Error:

```
! gog: `x(continent, limits = ...)` - `continent` is text, and a category has
  ↳ no range to lie inside. To choose which categories appear, filter the
  ↳ table before plotting; to change the order they appear in, use
  ↳ `order(continent)`.
```

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

22.10 The column's type picks the axis

The scales in the table above are not all chosen the same way. You ask for `log`. You do not ask for `time` or `category`. `gog` reads those off the column: a `Date` column gets a calendar axis, and a text column gets one slot per distinct value.

You may still write them. Writing `scale = "category"` on a text column is allowed and changes nothing, in the same way `scale = "linear"` on a number changes nothing. The plot below is identical to the same sentence with the scale left off:

R

```
data(gapminder_2007) + bar * mean + x(continent, scale = "category") +  
  ↪ y(life) +  
  title("Writing it changes nothing")
```

Python

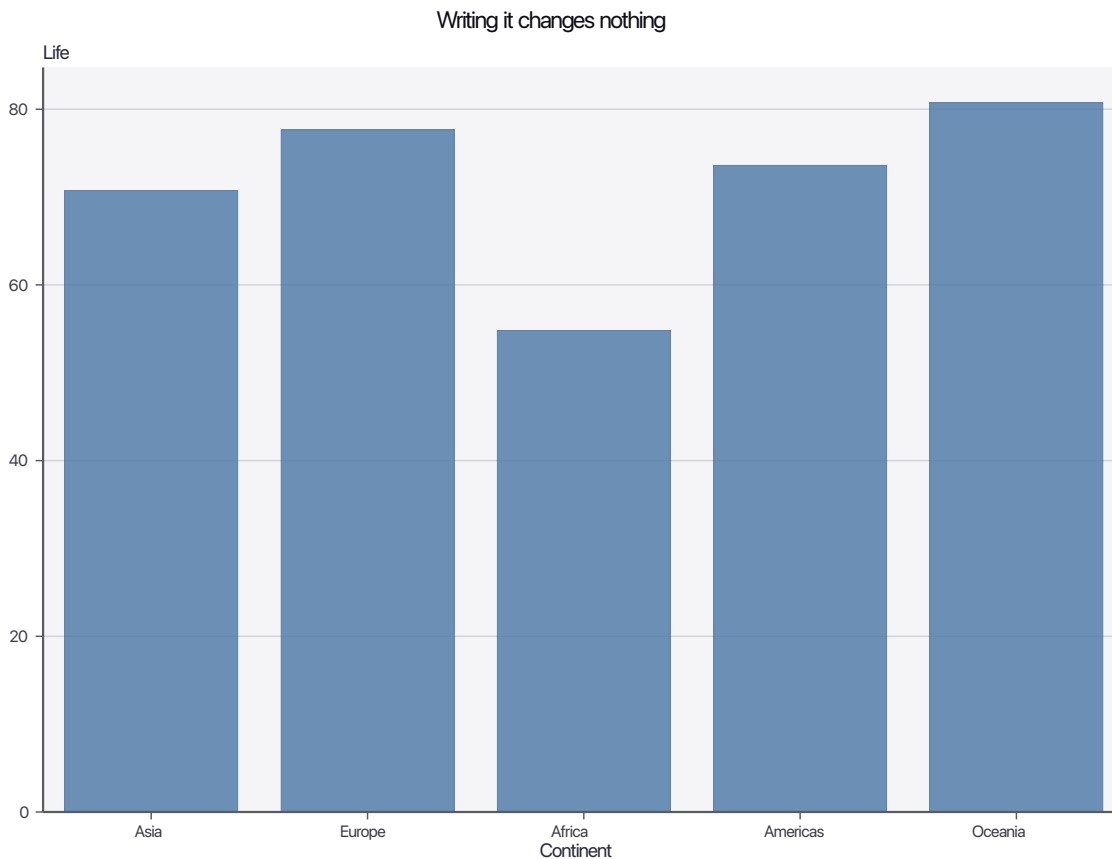
```
(data(gapminder_2007) + bar * mean + x(col.continent, scale = "category") +  
  ↪ y(col.life) +  
  title("Writing it changes nothing"))
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent, scale = "category") +  
  y(:life) + title("Writing it changes nothing")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean),  
  x(col.continent, { scale: "category" }), y(col.life),  
  title("Writing it changes nothing"))
```

“Given gapminder 2007: bars derived by mean, x is continent as a category, y is life.”

What you cannot do is use a scale to contradict the column. Ask for a categorical axis on a number and gog refuses:

```
data(gapminder_2007) + point + x(gdp, scale = "category") + y(life)
```

Error:

```
! gog: `x(gdp, scale = "category")` - a scale says how a measured column is
  ↳ placed; whether an axis measures at all is the column's type. Removing
  ↳ the scale alone would leave the continuous axis you were trying to
  ↳ escape. Make `gdp` text - in R, `factor(gdp)` - and drop the scale. To
  ↳ cut the numbers into ranges instead, use `bin`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The reason behind that refusal also explains the others in this chapter. A scale says *how* a measured column is placed. Whether the axis measures at all is the column's type. So log

on a text column is refused, `time` on a plain number is refused, and a date column cannot be turned back into raw numbers by asking for `linear`. Every one of those refusals tells you to change the column, because the column is where the answer lives.

There are two things you might mean by “draw this number as categories”, and `gog` has a word for each. For one slot per distinct value, declare the column’s categories: `factor()` in R and `ordered()` in the other three bindings, both shown in [Data](#). For ranges, use `bin`, which cuts the numbers into intervals and is covered in [Transforms](#).

22.11 `tick_count`: how many ticks the axis aims for

An axis picks five ticks or so by default, and the count is the one part of the fit you may want to choose. A crowded axis is easier to read with fewer; an axis someone will take numbers from is easier with more. `tick_count` is written on the binding, beside `scale`, `base` and `limits`, and for the same reason. How many ticks an axis gets is a property of the scale, so it belongs to the sentence that describes the scale rather than to `theme()`, which controls the page.

R

```
(data(gapminder_2007) + point + x(gdp, tick_count = 3) + y(life) +  
  x_label("GDP per capita") + title("A low count: the shape, not the  
↪ numbers")) |  
  (data(gapminder_2007) + point + x(gdp, tick_count = 12) + y(life) +  
    x_label("GDP per capita") + title("A high count: for reading values  
↪ off"))
```

Python

```
((data(gapminder_2007) + point + x(col.gdp, tick_count = 3) + y(col.life) +  
  x_label("GDP per capita") + title("A low count: the shape, not the  
↪ numbers")) |  
  (data(gapminder_2007) + point + x(col.gdp, tick_count = 12) + y(col.life) +  
    x_label("GDP per capita") + title("A high count: for reading values  
↪ off")))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp, tick_count = 3) + y(:life) +  
  x_label("GDP per capita") +  
  title("A low count: the shape, not the numbers")) |  
  (data(gapminder_2007) + point + x(:gdp, tick_count = 12) + y(:life) +
```

```
x_label("GDP per capita") +
title("A high count: for reading values off"))
```

JavaScript

```
beside(plot(data(gapminder_2007), point, x(col.gdp, { tick_count: 3 }),
  y(col.life), x_label("GDP per capita"),
  title("A low count: the shape, not the numbers")),
plot(data(gapminder_2007), point, x(col.gdp, { tick_count: 12 }),
  y(col.life), x_label("GDP per capita"),
  title("A high count: for reading values off")))
```



“Given gapminder 2007: points, x is gdp with 3 ticks, y is life, beside the same with 12 ticks.”

The count is a target rather than a promise, and the difference is worth understanding. The count chooses a *step*, and the step is then rounded to a number a person would pick, so asking

for eight on an axis running 0 to 100 gets a step of 10 and eleven ticks. What you can rely on is the direction: ask for more and you never get fewer, and every tick still falls on a round number.

Both panels above cover the same range. That is the rule the count obeys: it changes how densely an axis is *labeled*, never where the axis stops. Ask for two ticks and the axis does not shrink to the two values it draws.

R

```
data(gapminder_asia) + line + x(year, tick_count = 2) + y(life) +  
  group(country) + style(opacity = 0.4) +  
  title("Two ticks, and still 1952 to 2007")
```

Python

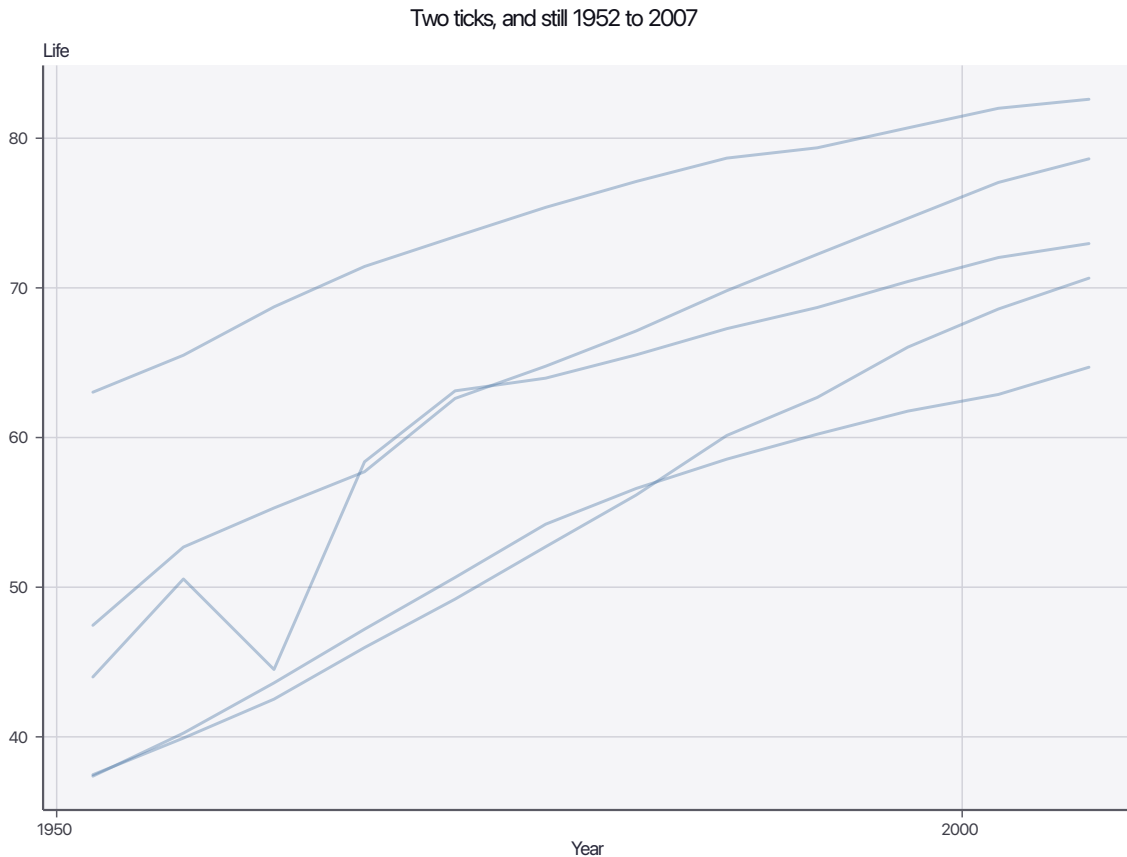
```
(data(gapminder_asia) + line + x(col.year, tick_count = 2) + y(col.life) +  
  group(col.country) + style(opacity = 0.4) +  
  title("Two ticks, and still 1952 to 2007"))
```

Julia

```
data(gapminder_asia) + line + x(:year, tick_count = 2) + y(:life) +  
  group(:country) + style(opacity = 0.4) +  
  title("Two ticks, and still 1952 to 2007")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year, { tick_count: 2 }),  
  y(col.life), group(col.country), style({ opacity: 0.4 }),  
  title("Two ticks, and still 1952 to 2007"))
```



A categorical axis has one tick per category, so the count there is the data's rather than yours, and the refusal points at the two fixes that do the jobs you might have meant:

```
data(gapminder_2007) + bar * mean + x(continent, tick_count = 3) + y(life)
```

Error:

```
! gog: `x(continent, tick_count = 3)` - `continent` is text, and a
  ↳ categorical axis has one tick per category, so the count is the data's
  ↳ rather than yours. To change which categories appear, filter the table
  ↳ before plotting; to change their order, use `order(continent)`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

And a legend is not a short axis. `limits` reaches every channel that measures, because every one of them has a domain; `tick_count` reaches only the three position channels that draw an axis: `x`, `y` and `z`. A color legend names three rows, both ends and the middle, and those come from the scale's own shape rather than from a count you could choose. On a log ramp the middle row is the geometric mean, the label for the color actually painted halfway along the

ramp. So `color()`, `size()` and `opacity()` have no tick count to set, the same way `shape()` has no `scale` and no `limits`: there is nothing for the count to describe.

22.11.1 Crowding in the plane and in the cube

A flat panel and a cube count their ticks differently, and neither answer is wrong. A flat axis draws *every* tick it chose, however tight they get. The panel's width is already fixed, and the labels sit in a margin reserved for them. An edge label that would overhang its panel is anchored inward instead. Nothing is dropped, so what you asked for is what you count.

A cube cannot make that promise. An axis in [space](#) is drawn at whatever length the viewing angle leaves it. Tilted far enough, it has no room for its numbers, so the cube draws as many as fit and leaves the rest out. Both are the same rule underneath, *do not print one number through another*. They differ only in how much room each has, because the plane can reserve room in advance and the cube cannot. When a count you stated has to be thinned that way, the engine says so rather than quietly drawing fewer.

23 Channels and legends

Which visual property should carry which column? A channel binds a data column to a visual property of a mark, and the channels do not all do the same work. `x`, `y` and `z` are the **vowels**: without one a mark does not render. The five below that earn a legend are the **final consonants**, the optional `□` (*m*) that turns `가` (*ga*) into `감` (*gam*): never required, and each one changes what the plot says. Every channel atom follows the same pattern:

```
channel(field)
```

No exceptions. `x` and `y` are channels too: the same rule applies everywhere.

The word for this is not settled outside `gog`, so you may know it as something else. Bertin wrote the vocabulary down first, in *Sémiologie graphique* (Bertin, 1967). He separated the plane's two dimensions from six **retinal variables**: size, value, texture, color, orientation, and shape. The eye does not read the two groups the same way. Wilkinson and `ggplot2` both say **aesthetic**, which is why you write `aes()` there and nothing like it here. The visualization literature settled instead on **channel**, paired with the mark it modifies (Munzner, 2014), and that is the word `gog` takes. A channel is a path a signal travels down, which is what a color or a size is here. `gog` takes the everyday word for the job, as its [Plain Names](#) law asks. **Encoding** is the same idea named as an act rather than a thing, and this book keeps both words. A channel is what you write; an encoding is what it does. [Encoding scope](#) is about the second.

Atom	Visual property	Variable type
<code>x(field)</code>	horizontal position	any
<code>y(field)</code>	vertical position	any
<code>z(field)</code>	depth: the third position, which makes a plot 3-D	continuous
<code>color(field)</code>	fill / stroke color	categorical <i>or</i> continuous
<code>size(field)</code>	point radius	continuous
<code>shape(field)</code>	glyph shape	categorical
<code>pattern(field)</code>	fill hatch / stroke dash	categorical
<code>opacity(field)</code>	transparency	continuous
<code>group(field)</code>	splits a mark into one series per category, without encoding it	categorical
<code>label(field)</code>	the string a <code>text</code> mark draws	any

Atom	Visual property	Variable type
<code>play(field)</code>	cuts the plot into frames and plays them in sequence	any

A legend is generated automatically whenever one of the five refining channels (`color`, `size`, `shape`, `pattern`, `opacity`) is bound to a column, with no configuration required. `x` and `y` earn an axis instead, and `group` earns neither: it splits a mark without encoding anything, so there is nothing for a key to decode.

23.1 color

`color(field)` maps a column to color, and a legend appears automatically. It takes *either* kind of column, and the kind decides what color means.

It is spelled `color`, and only `color`. `gog` writes American English throughout and accepts no second spelling, so there is no `colour` here as there is in `ggplot2`; writing one earns a refusal that names the fix (see [Law 3](#)).

A `text` column asks “*which one?*” Each category gets its own hue from the palette:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent)
```

Python

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  color(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent)
```

JavaScript


```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
     color(col.continent))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent.”

A **numeric** column asks “how much?” Its values run along a **ramp**. A ramp is a range of colors that a number is placed along, the way an axis is a range of positions. This one runs light to dark, and the legend is the ramp itself, labeled at its minimum, midpoint, and maximum:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(life) + style(size = 6)
```

Python

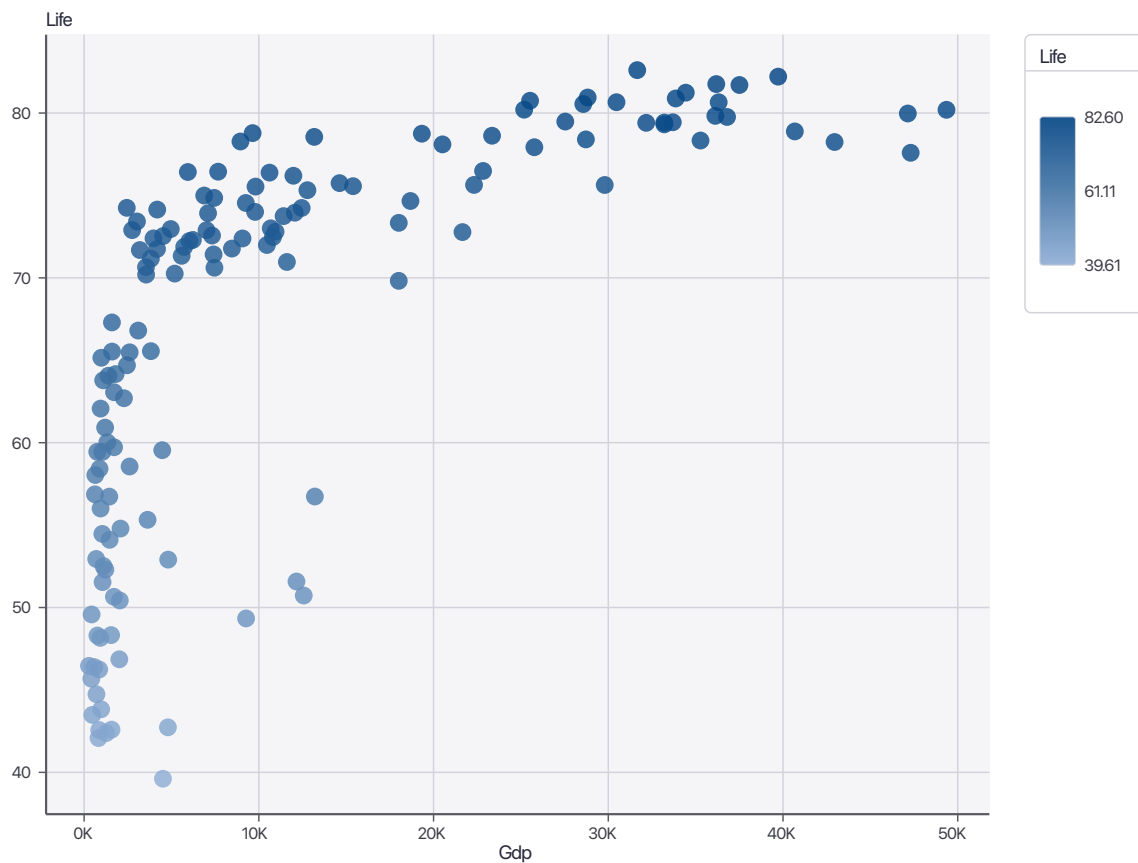
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.life) + style(size = 6))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:life) +  
  style(size = 6)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.life), style({ size: 6 })))
```



Notice the legend: color gets one continuous strip while **size** and **opacity** get three sampled examples. A legend should show the *scale*, and color is the only continuous channel whose

whole range fits in a fixed space; there is no way to draw a continuum of circles. The rule is the same for every channel; only the shape of the legend changes. The difference matters most for a multi-hue ramp. Three sampled colors from *viridis* say nothing about the colors between them, while three circles of growing size do.

The default ramp is one hue, not a rainbow, and that too is deliberate. A sequential ramp encodes magnitude, and **lightness** is what a reader perceives as *more*. One hue running light to dark is read quickly, stays readable in grayscale and under color blindness, and stays distinct from the categorical palette when both appear in one figure. A rainbow encodes magnitude as *hue*, which has no intrinsic order, so the reader has to consult the legend for every point. *gog*'s default is a single blue built on the same hue as the first categorical color, so the two scales read as one system. See [Ramps](#) below for the multi-hue alternatives, the diverging ramps, and custom stops.

color also implies grouping for line marks. Each unique value gets its own connected line:

R

```
data(gapminder_asia) + x(year) + y(life) + color(country) +  
  line + point
```

Python

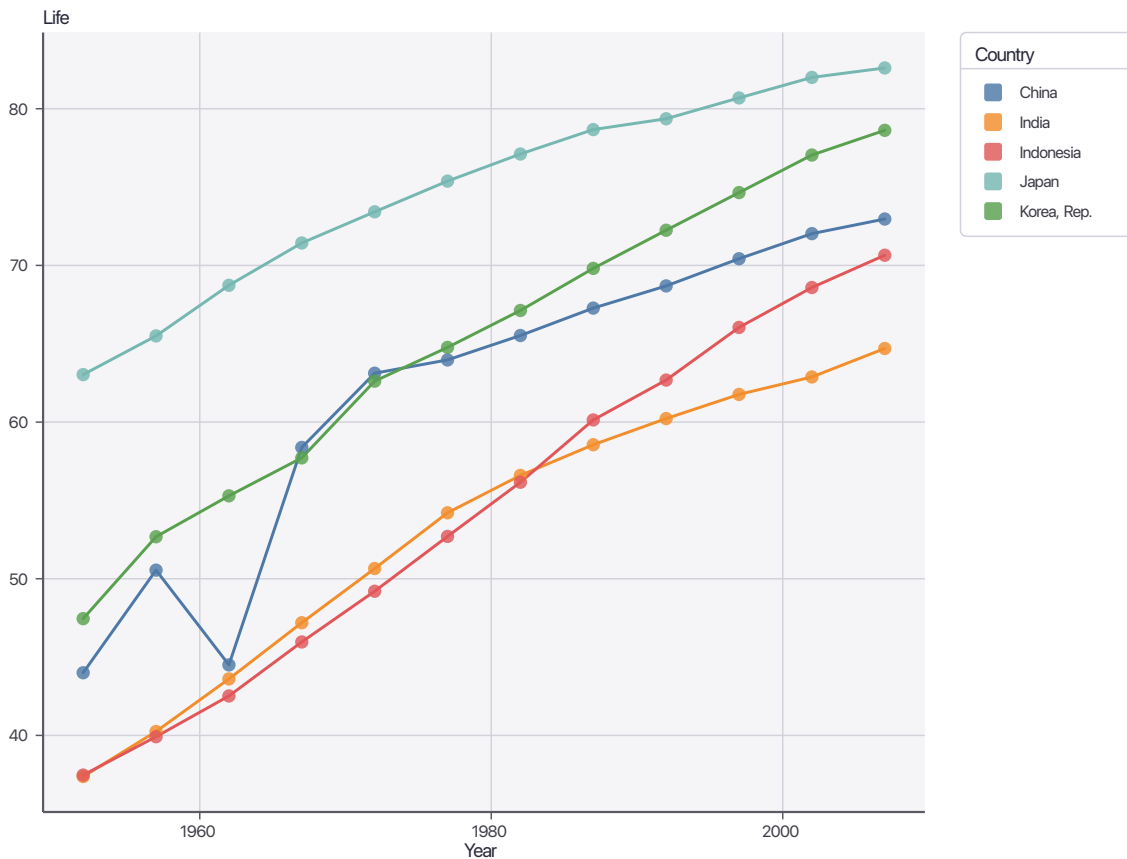
```
(data(gapminder_asia) + x(col.year) + y(col.life) + color(col.country) +  
  line + point)
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + color(:country) + line +  
  point
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), color(col.country),  
  line, point)
```



“Given `gapminder Asia`: *x* is year, *y* is life, color by country, lines and also points.”

Position decides which layers a channel reaches. `color(country)` is written *before* any mark above, so it belongs to the plot and colors both the lines and the points. Written *after* a mark it binds to that mark alone:

R

```
data(gapminder_asia) + x(year) + y(life) +
  line + color(country) +
  point + size(population)
```

Python

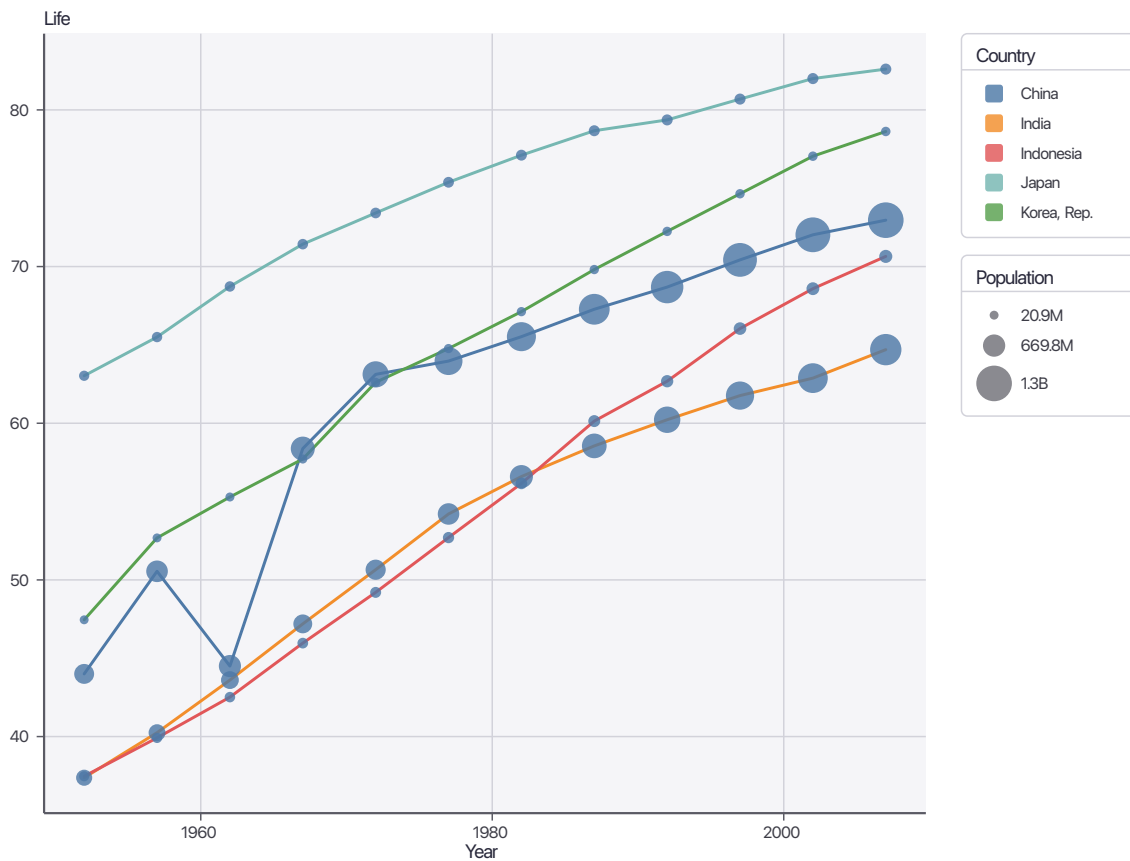
```
(data(gapminder_asia) + x(col.year) + y(col.life) +
  line + color(col.country) +
  point + size(col.population))
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + line + color(:country) +  
  point + size(:population)
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), line,  
  color(col.country), point, size(col.population))
```



“Given gapminder Asia: x is year, y is life, lines, color by country, and also points, size by population.”

Channels bind forward, never backward. See [Encoding scope](#).

23.2 size

`size(field)` scales point radius linearly from 3px (min) to 12px (max). A size legend shows three representative circles.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  size(population)
```

Python

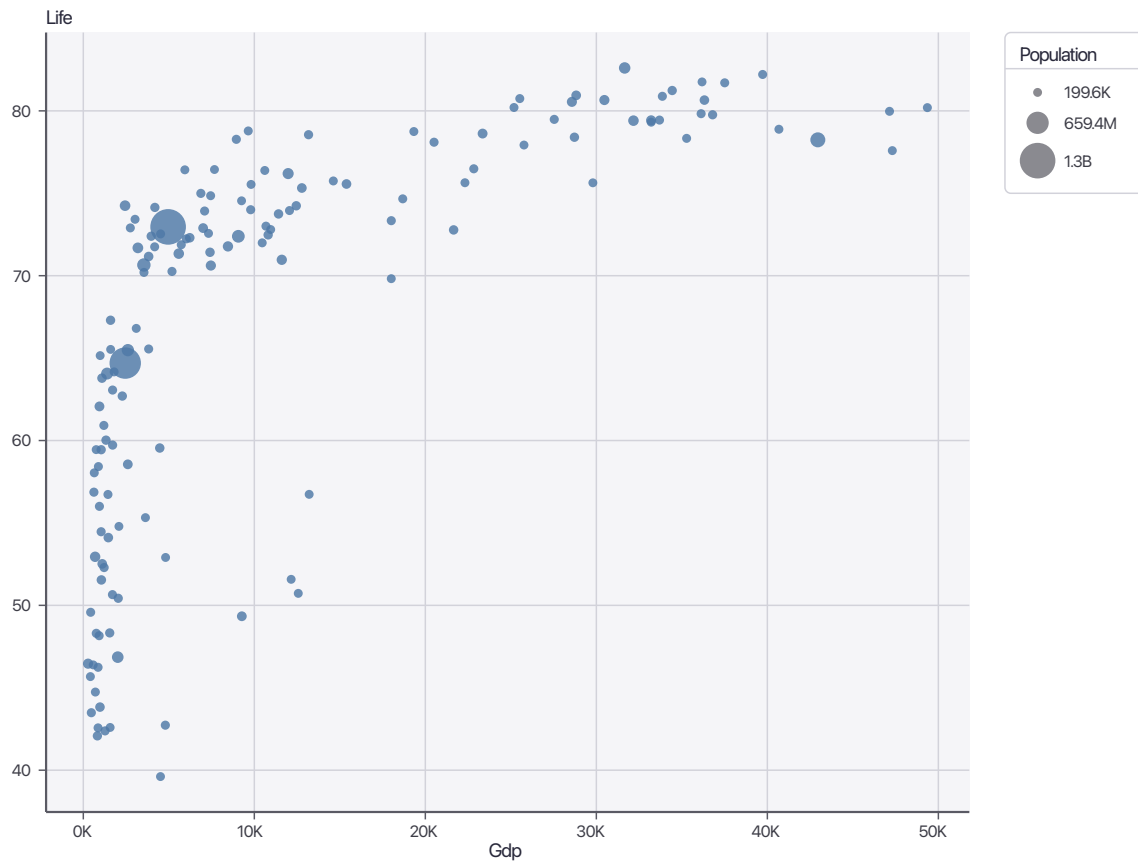
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  size(col.population))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + size(:population)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  size(col.population))
```



Combine color and **size** for a classic bubble chart:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + size(population)
```

Python

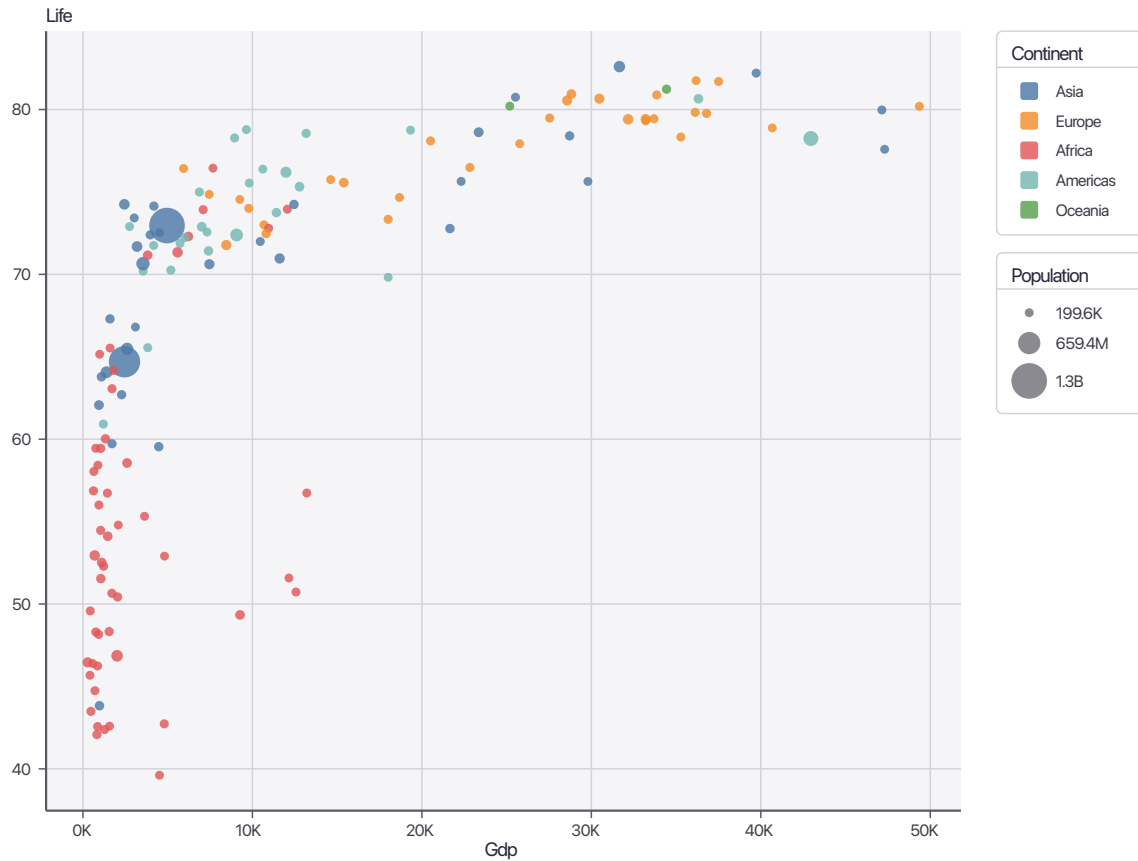
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  color(col.continent) + size(col.population))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +
  size(:population)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  color(col.continent), size(col.population))
```



23.3 shape

`shape(field)` maps a categorical column to one of five glyphs: circle, square, triangle, diamond, cross. This is what separates groups in a figure printed in black and white. Five is the whole

set. Do not map **shape** to a column with more categories: the sixth starts over at the circle, and two categories then share a glyph. The [Shape](#) section of the point chapter lists the ways past five.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  shape(continent)
```

Python

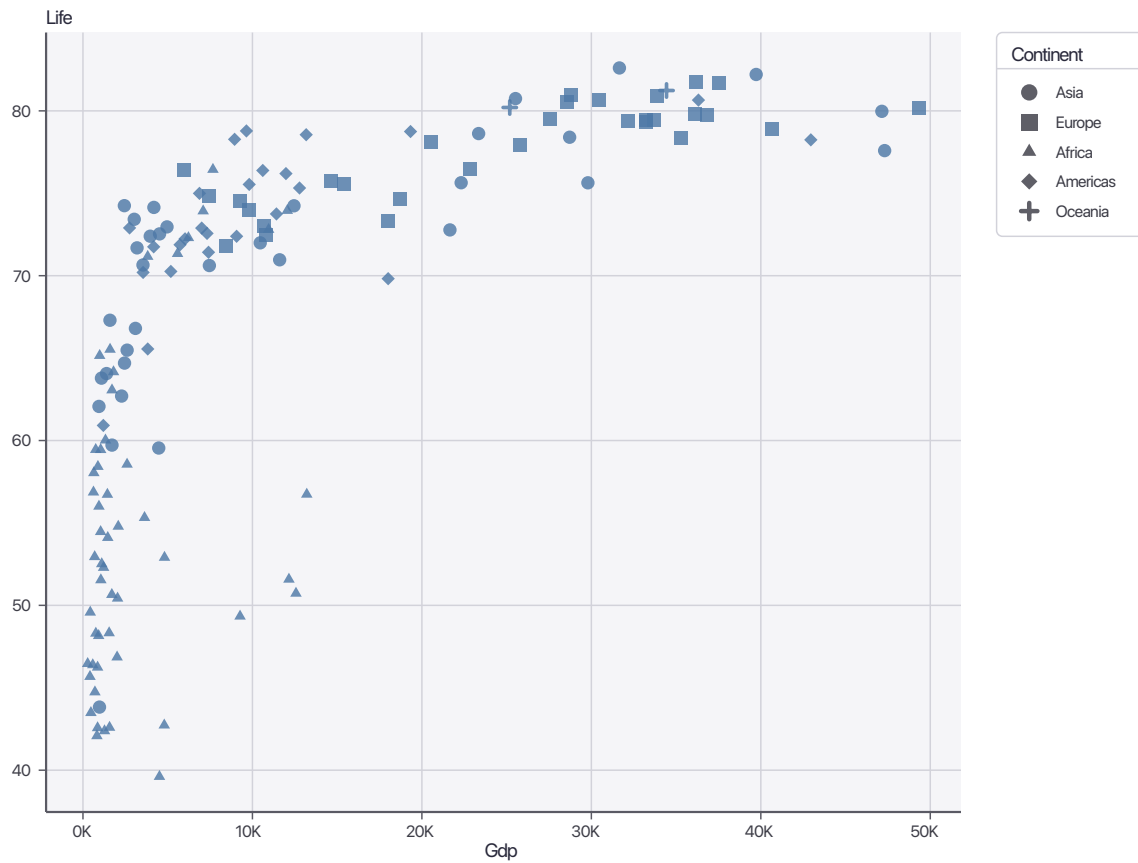
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  shape(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + shape(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  shape(col.continent))
```



“Given gapminder 2007: points, x is gdp, y is life, shape by continent.”

color and shape can be bound to the same column. Saying one column twice is **redundant encoding**, and it makes the plot easier to read:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + shape(continent)
```

Python

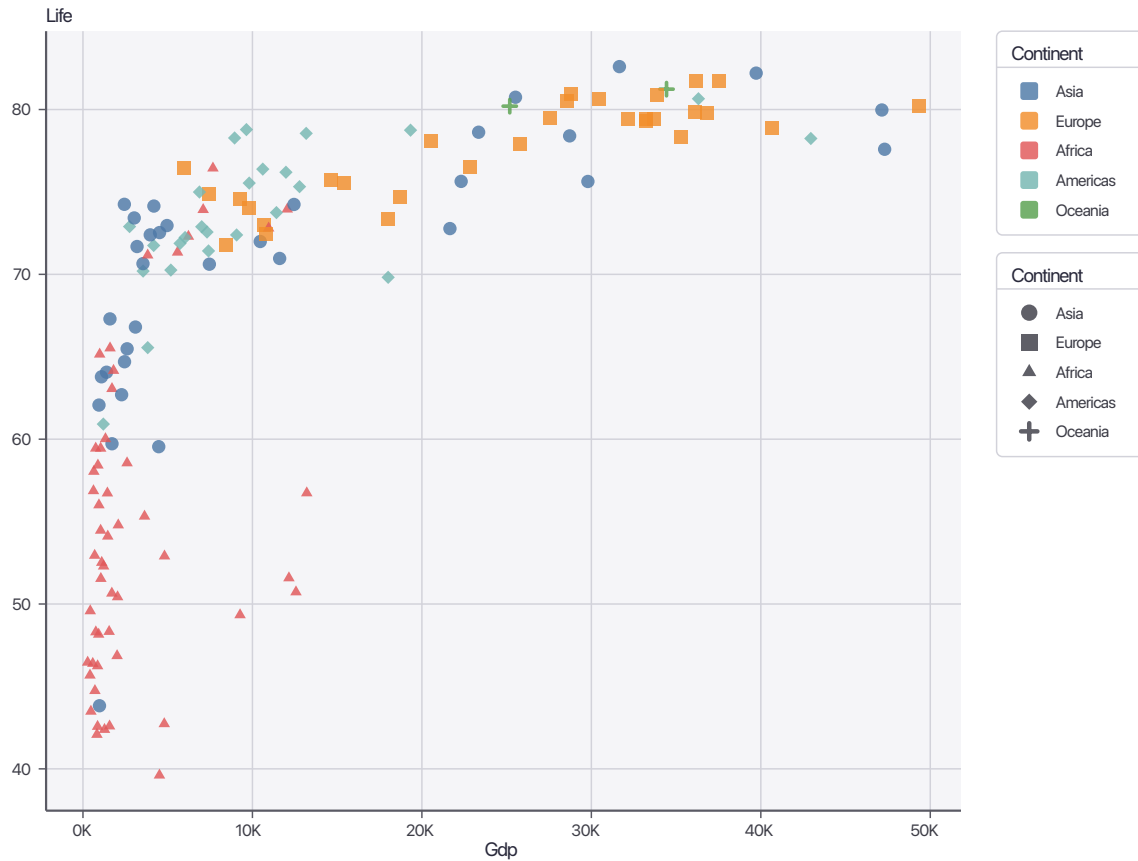
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  color(col.continent) + shape(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +
  shape(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  color(col.continent), shape(col.continent))
```



23.4 pattern

`pattern(field)` does for textures what `shape` does for glyphs: it maps a categorical column to a **texture**. The mark's geometry decides which one: a **hatch** on the fills (bar, box, area,

ribbon, zone), a **dash** on the strokes (line, step, interval, path, rule). Textures read without hue, so a plot mapped by **pattern** stays legible in grayscale or with color blindness. The first category is always the plain one (solid), the rest textured:

R

```
data(iris_flowers) + bar * mean + x(species) + y(petal_length) +  
  pattern(species) + y_label("Mean petal length (cm)")
```

Python

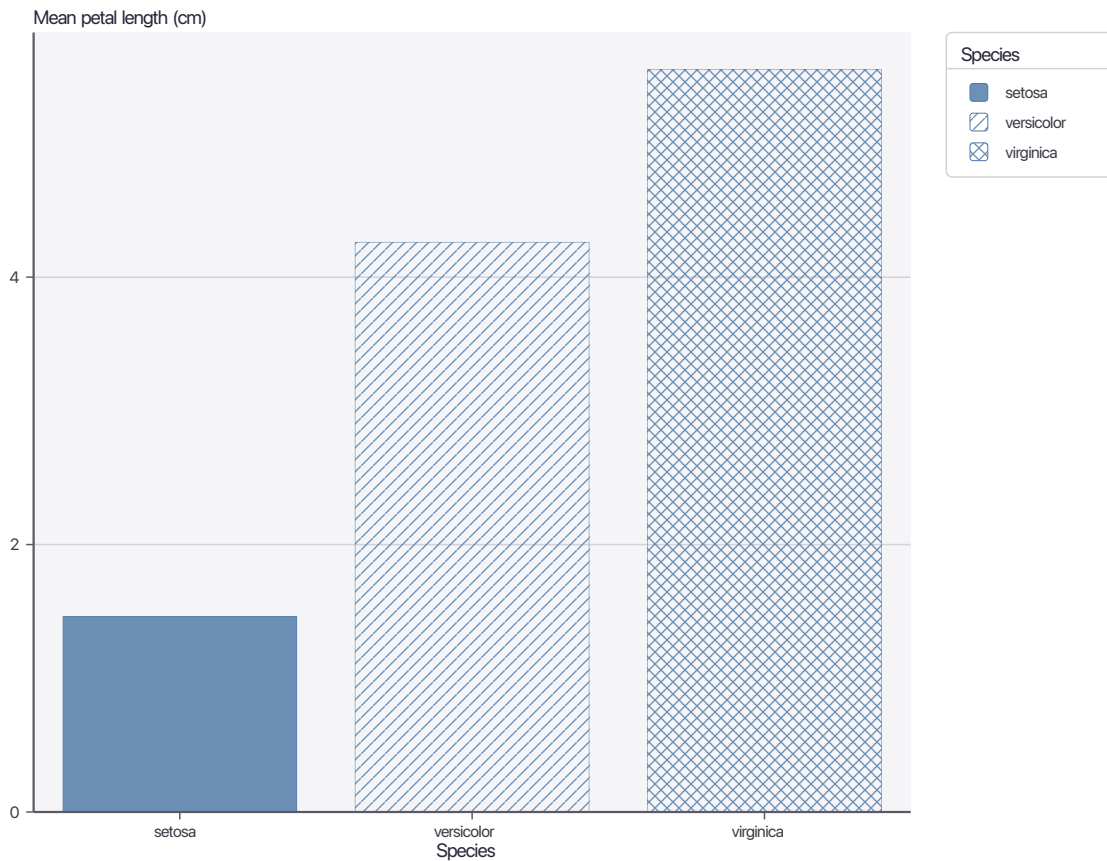
```
(data(iris_flowers) + bar * mean + x(col.species) + y(col.petal_length) +  
  pattern(col.species) + y_label("Mean petal length (cm)"))
```

Julia

```
data(iris_flowers) + bar * mean + x(:species) + y(:petal_length) +  
  pattern(:species) + y_label("Mean petal length (cm)")
```

JavaScript

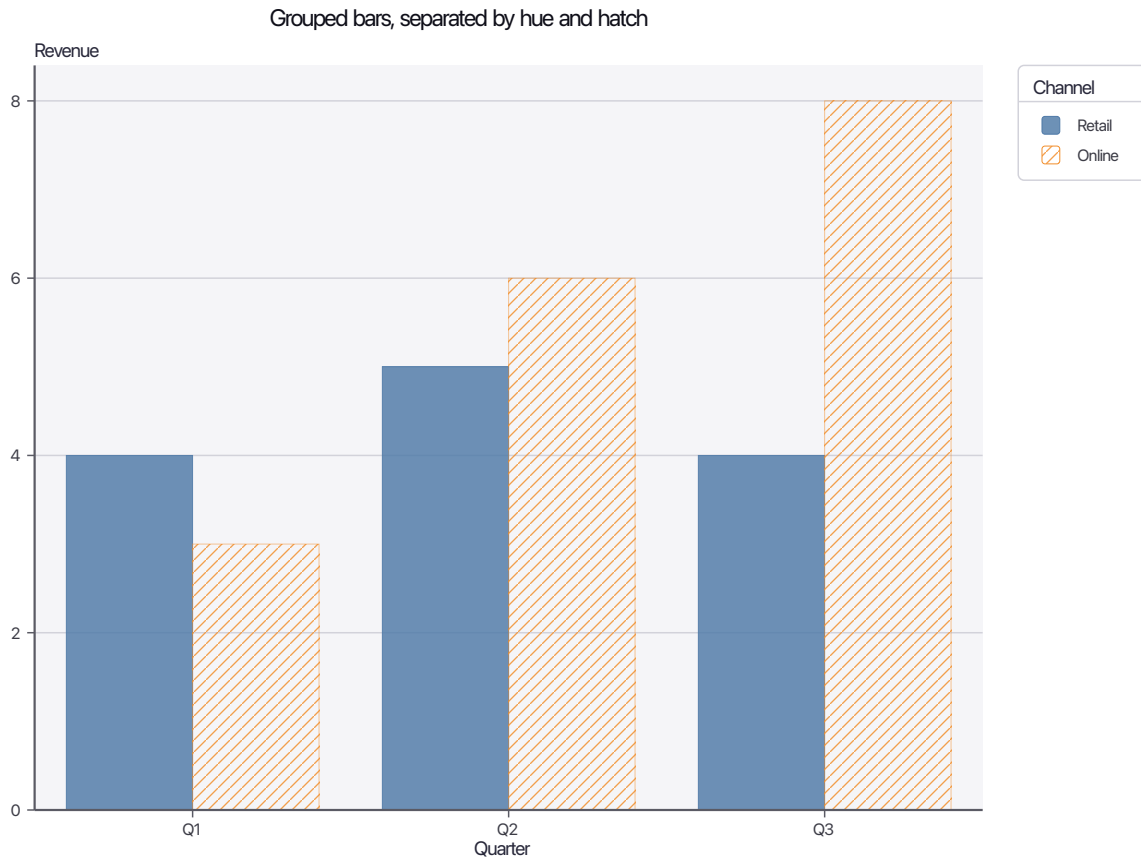
```
plot(data(iris_flowers), layer(bar, mean), x(col.species),  
  y(col.petal_length), pattern(col.species),  
  y_label("Mean petal length (cm)"))
```



“Given the iris flowers: bars derived by mean, x is species, y is petal length, pattern by species.”

The point of the channel is redundant encoding: the same column on **color** and **pattern**, so a reader separates the series by hue and by texture at once. **gog** draws these as **one merged legend**, not two:

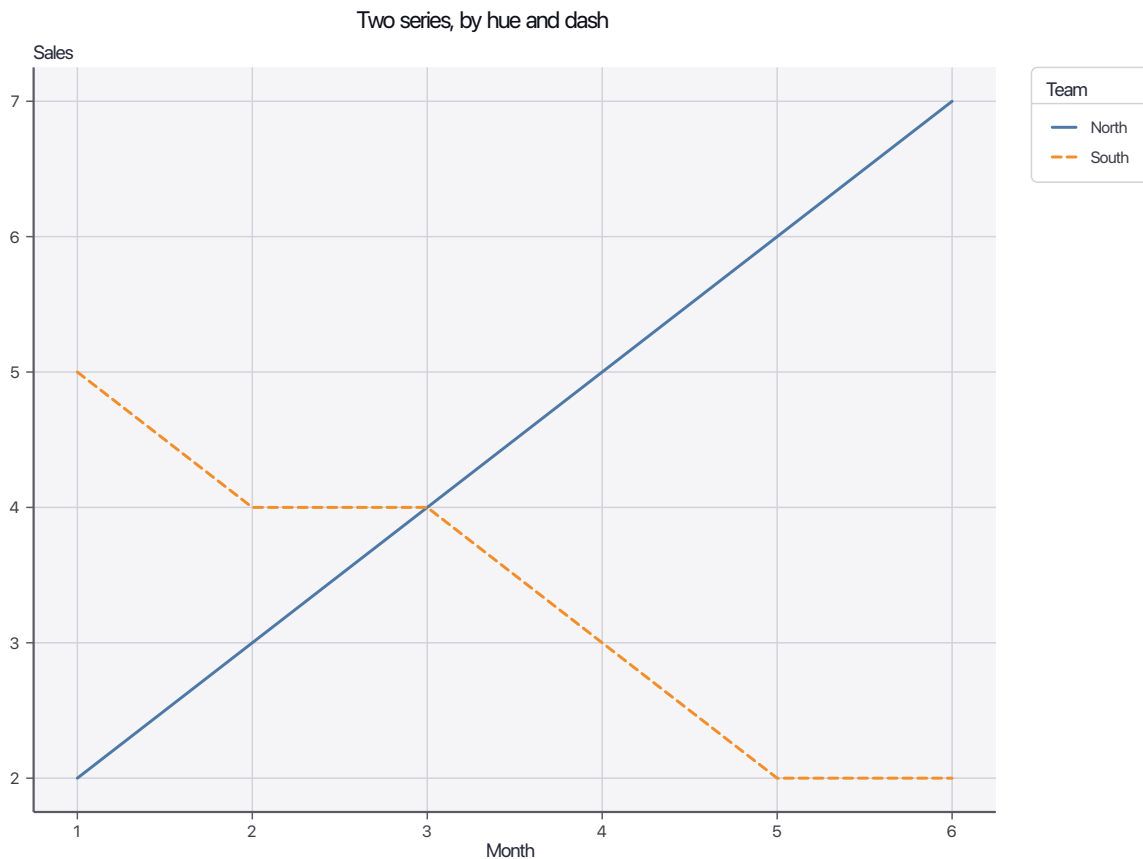
```
sales <- data.frame(
  quarter = rep(c("Q1", "Q2", "Q3"), each = 2),
  channel = rep(c("Retail", "Online"), 3),
  revenue = c(4, 3, 5, 6, 4, 8)
)
data(sales) + bar * sum * dodge + x(quarter) + y(revenue) + color(channel) +
  pattern(channel) + title("Grouped bars, separated by hue and hatch")
```



“Given the sales table: bars derived by sum and dodge, x is quarter, y is revenue, color by channel, pattern by channel.”

On a stroke the same channel dashes each series instead, so these two lines are separated by hue and by dash:

```
trend <- data.frame(
  month = rep(1:6, 2),
  team = rep(c("North", "South"), each = 6),
  sales = c(2, 3, 4, 5, 6, 7, 5, 4, 4, 3, 2, 2)
)
data(trend) + line + x(month) + y(sales) + color(team) + pattern(team) +
  title("Two series, by hue and dash")
```



The texture palette is deliberately small: four hatches beside solid on a fill, and two dashes beside solid on a stroke. So `pattern` suits a few categories rather than dozens, and past that count the textures repeat. To fix one texture for a whole layer instead of mapping a column, use `style(pattern = )`.

23.5 opacity

`opacity(field)` maps a continuous column to transparency. Use it when marks overlap, so the crowded regions look darker than the empty ones.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + opacity(population) +  
  title("Opacity: larger populations drawn more solidly")
```

Python

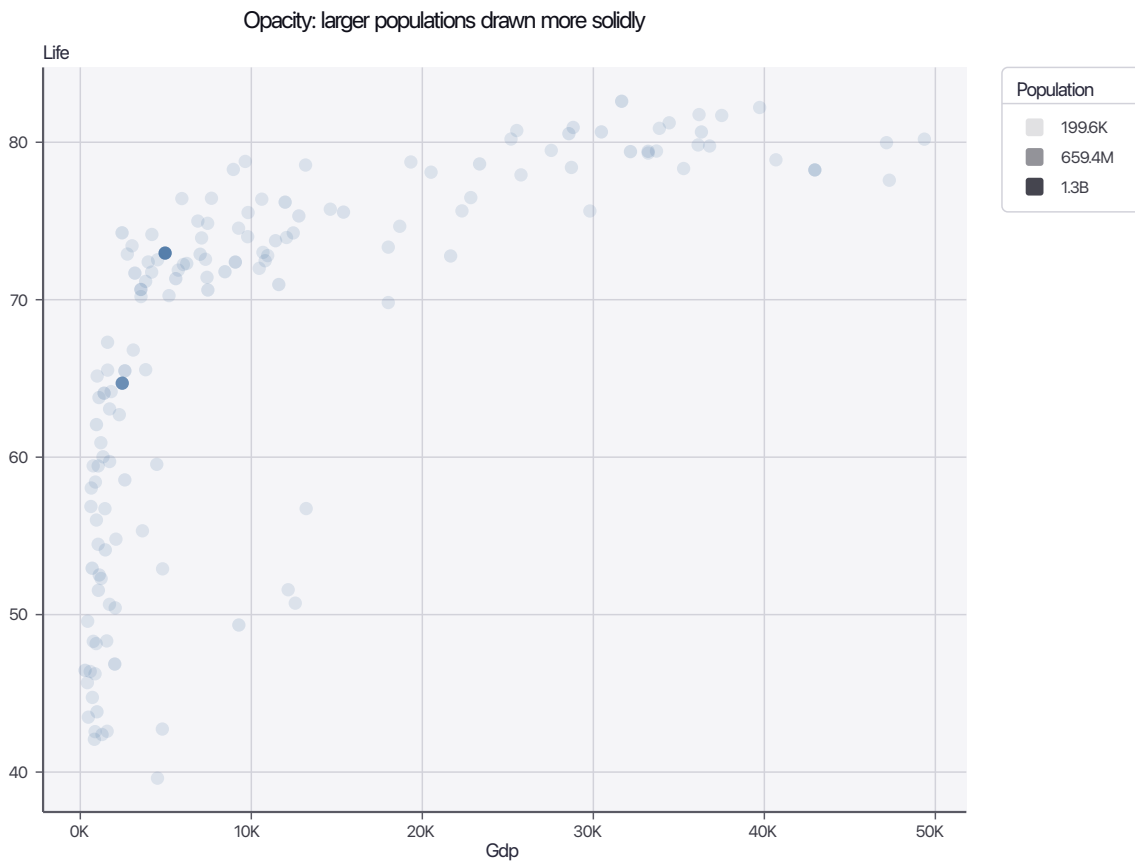
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ opacity(col.population) +  
  title("Opacity: larger populations drawn more solidly"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + opacity(:population) +  
  title("Opacity: larger populations drawn more solidly")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  opacity(col.population),  
  title("Opacity: larger populations drawn more solidly"))
```

“Given *gapminder 2007*: points, *x* is *gdp*, *y* is *life*, opacity by *population*.”

Mapped values run from 0.15 to 0.95 rather than 0 to 1: a fully transparent mark is invisible, which is never what a reader wants. Marks with no `opacity` binding are drawn at 0.82, unless `style(opacity = )` sets one.

Like `size`, `opacity` produces a legend showing the minimum, midpoint, and maximum of the column.

On line, `opacity` cannot be *mapped*: a stroke has *one* opacity, and one value cannot carry a column. This is the same rule that refuses a per-row `size` on a line, not a special case for `opacity`:

```
data(gapminder_asia) + line + x(year) + y(life) + opacity(population)
```

Error:

! gog: `opacity` cannot be bound to `line` - a line has no opacity feature.
↪ Remove the `population` mapping from `opacity`, or use a mark that has
↪ one.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

That one stroke still *has* an opacity, so you can *set* it for the whole line:

R

```
data(gapminder_asia) + line + x(year) + y(life) + group(country) +  
  style(opacity = 0.4)
```

Python

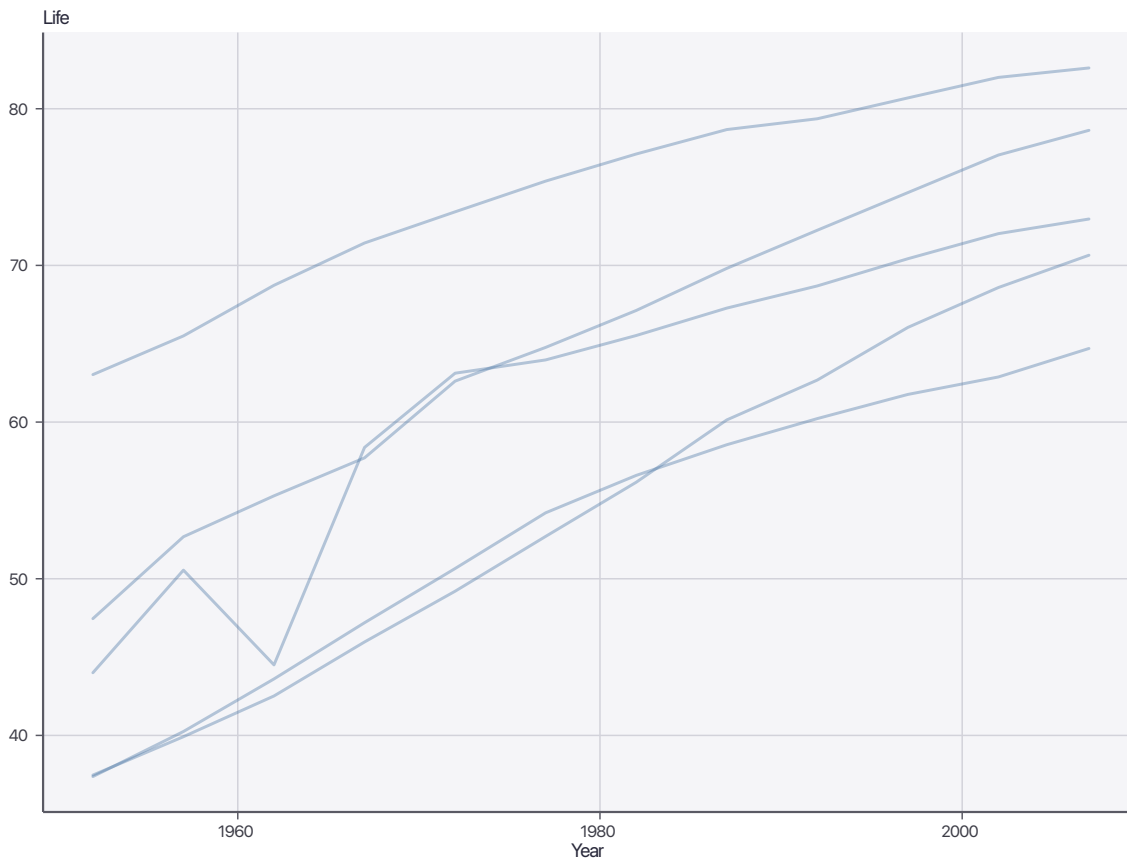
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + group(col.country)  
↪ +  
  style(opacity = 0.4))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country) +  
  style(opacity = 0.4)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
  group(col.country), style({ opacity: 0.4 }))
```



“Given gapminder Asia: lines, x is year, y is life, grouped by country, with opacity 0.4.”

To distinguish lines *by data*, use `color` or `group`.

`color` behaves differently from `size` and `opacity` here. The difference looks like an exception and is not, so it is worth stating exactly. A width, an opacity and a dash are properties of the **whole stroke**: ask where along the line the width is 3px and the question has no answer. A color is not like that. You can point at one place on a line and ask what color it is, so a measure can vary along it, and `color` on a stroke mark accepts either kind of column:

R

```
data(gapminder_asia) + line + x(year) + y(life) + group(country) +
  color(gdp, scale = "log") + palette("viridis") +
  y_label("Life expectancy") + title("One line per country, colored by
  ↪ income")
```

Python

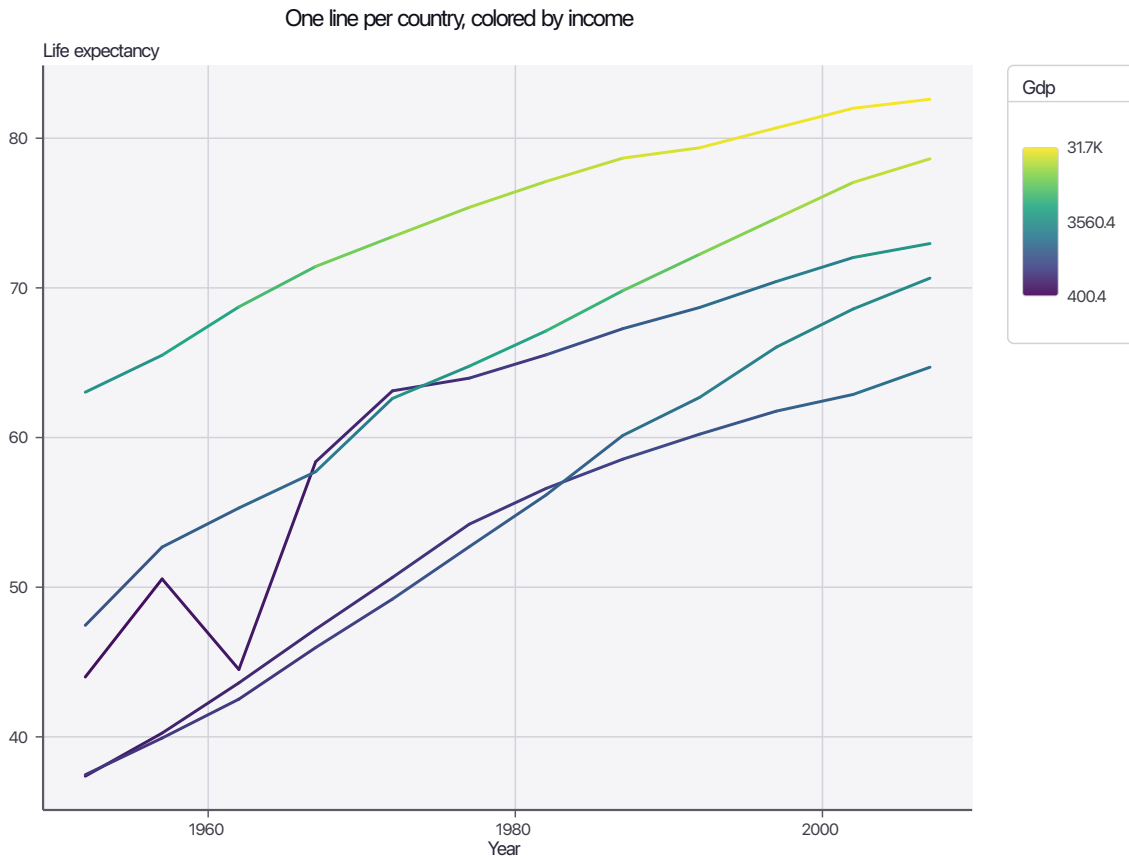
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + group(col.country)
↪ +
  color(col.gdp, scale = "log") + palette("viridis") +
  y_label("Life expectancy") + title("One line per country, colored by
↪ income"))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country) +
  color(:gdp, scale = "log") + palette("viridis") +
  y_label("Life expectancy") +
  title("One line per country, colored by income")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),
  group(col.country), color(col.gdp, { scale: "log" }),
  palette("viridis"), y_label("Life expectancy"),
  title("One line per country, colored by income"))
```



“Given gapminder Asia: lines, x is year, y is life, grouped by country, color by gdp on a log scale, with the viridis palette.”

A category on `color` splits the mark into one stroke per group. A measure varies the color *along* each stroke and earns the ramp legend instead of a swatch per series. One channel, one mark, two behaviors, and the column’s type decides which one you get. `group` still splits the mark either way, which is why the sentence above draws one line per country and runs the ramp along each line. See [Setting vs mapping](#).

23.6 group

`group(field)` connects line segments within the same group without adding a color distinction. Use it when you want a single color for all lines but still need them separated by category.

R

```
data(gapminder_asia) + line + x(year) + y(life) +  
  group(country)
```

Python

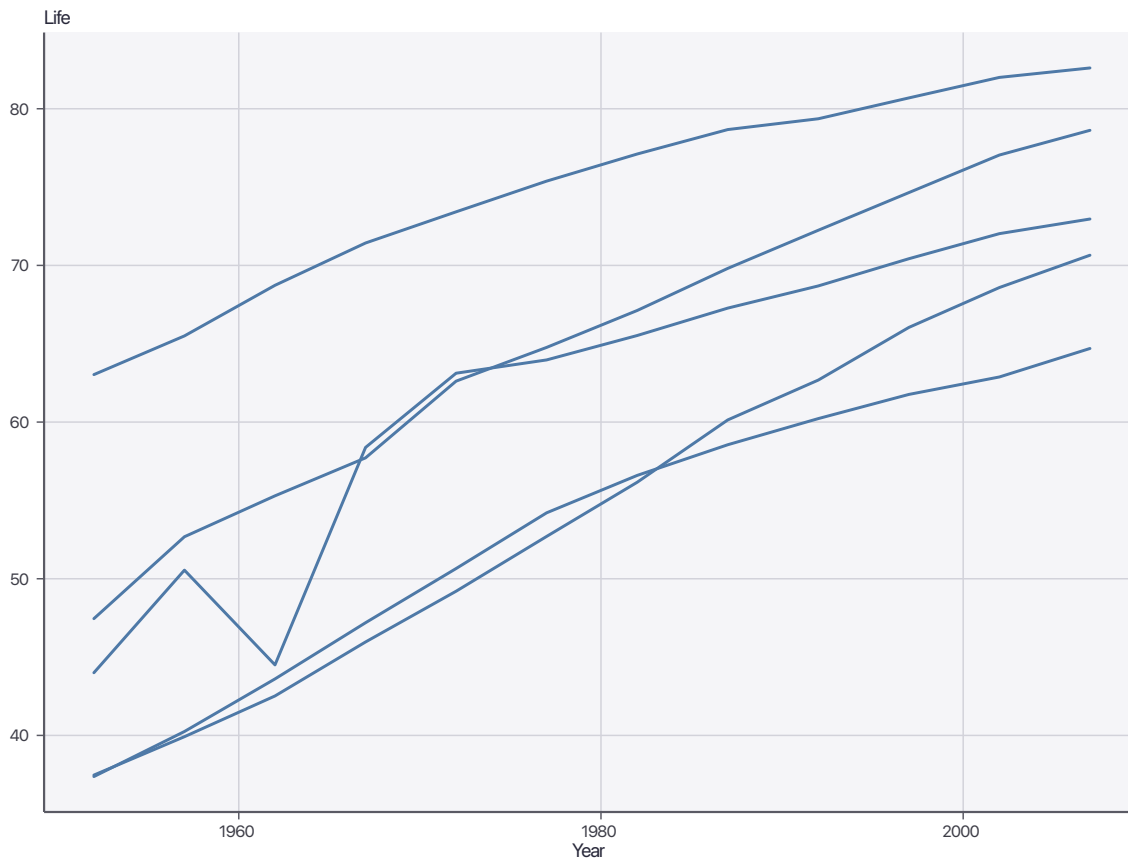
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) +  
  group(col.country))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
  group(col.country))
```



When you want color *and* grouping, use `color(field)` alone: it implies grouping automatically.

23.7 Multiple channels

Channels compose freely, so you can bind several of them on one mark:

R

```
# Color + size + shape on the same plot
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + size(population) + shape(continent) +
  title("GDP and life expectancy, 2007")
```

Python

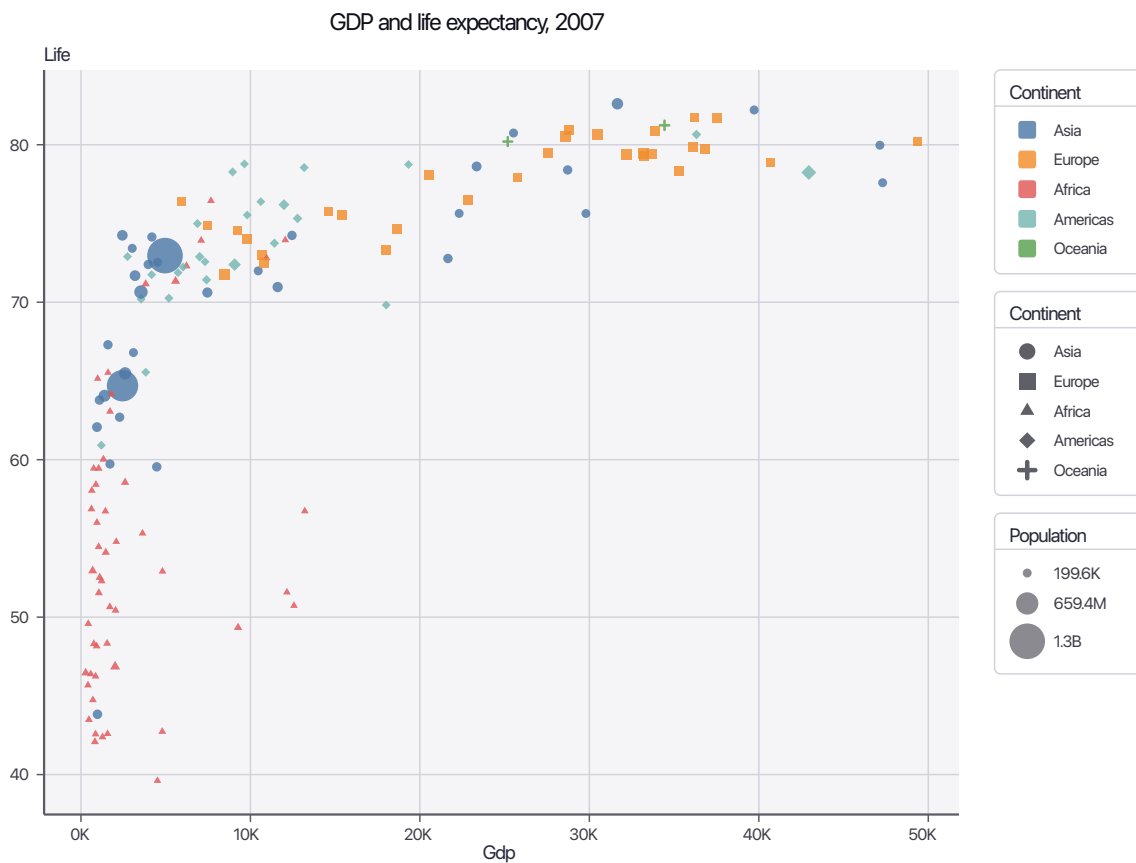
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.continent) + size(col.population) + shape(col.continent) +  
  title("GDP and life expectancy, 2007"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  size(:population) + shape(:continent) +  
  title("GDP and life expectancy, 2007")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent), size(col.population), shape(col.continent),  
  title("GDP and life expectancy, 2007"))
```



23.8 Palettes

`palette()` chooses the colors the `color` channel assigns. There is no separate atom for ramps, which `ggplot2` calls gradients. One word covers both, because the column decides. A text column reads a set of colors as *one color per category*. A numeric column reads the same set as **stops**, the fixed colors a ramp passes through.

There are three kinds of named palette, and they say three different things: a **categorical** palette gives each category its own color, a **sequential** ramp runs from low values to high, and a **diverging** ramp measures distance from a center in either direction. The type of the column decides which of the three kinds you may name, so the sections below follow that order.

`gog` uses a 20-color default palette, so most tables never use them all. When a column has more categories than that, `gog` builds the extra colors from the HSL color wheel, and no two categories share a color.

23.8.1 Categorical palettes

Name	Colors	Best for
"gog"	20 (default)	General use
"okabe"	8 base + 8 light variants	Colorblind accessibility
"soft"	8, muted	Filled marks: bars, areas, zones

"okabe" is Okabe and Ito's set (Okabe & Ito, 2008), chosen so that its eight colors stay distinct for readers with the common forms of color blindness:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  color(continent) + palette("okabe")
```

Python

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.continent) + palette("okabe"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  palette("okabe")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent), palette("okabe"))
```



The default is tuned for points and lines, where a color occupies a few pixels and must stay visible from across the page. A fill is large, so it is already easy to see, and the saturation that makes a small point findable is too strong on a bar. A row of saturated bars draws more attention than the numbers the bars report. **soft** is the same eight colors at lower saturation. Compare these two, which differ only in the palette:

R

```
data(gapminder_2007) + bar * count + x(continent) + color(continent) +  
  title("The default, over filled bars")
```

Python

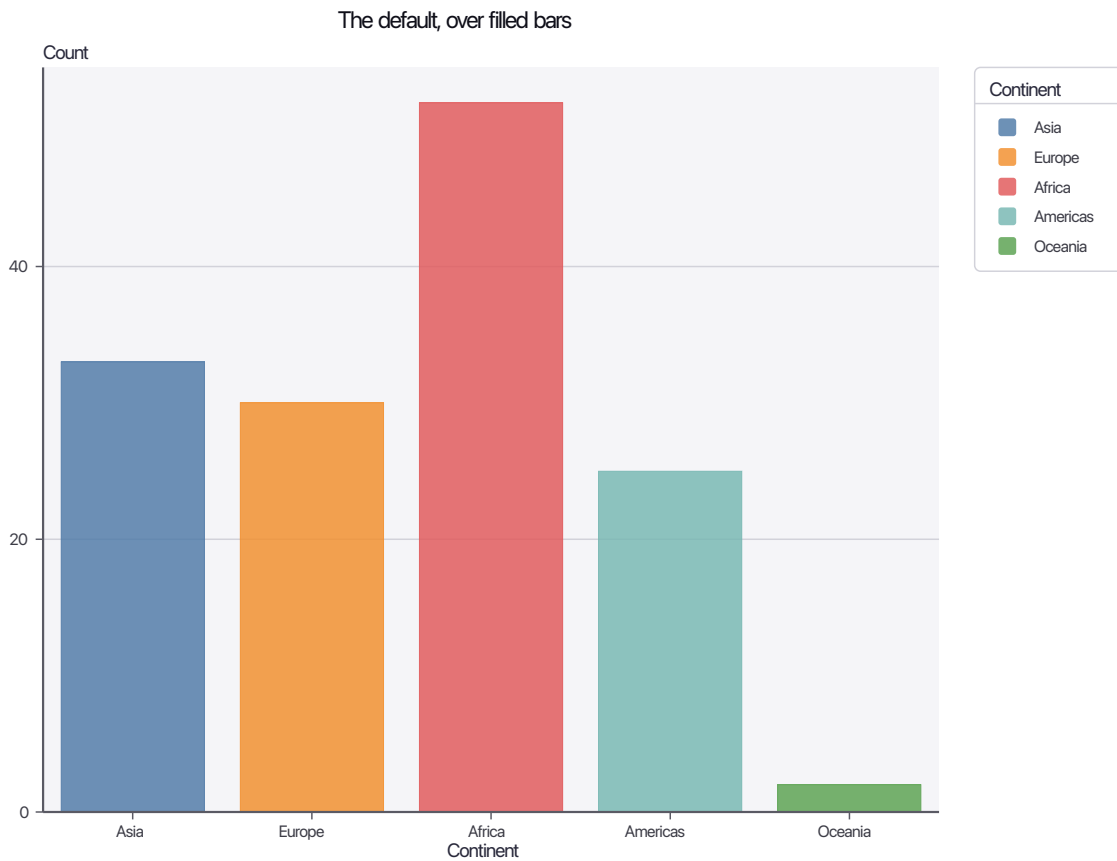
```
(data(gapminder_2007) + bar * count + x(col.continent) + color(col.continent)  
↪ +  
  title("The default, over filled bars"))
```

Julia

```
data(gapminder_2007) + bar * count + x(:continent) + color(:continent) +  
  title("The default, over filled bars")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, count), x(col.continent),  
  color(col.continent), title("The default, over filled bars"))
```



“Given gapminder 2007: bars derived by count, x is continent, color by continent.”

R

```
data(gapminder_2007) + bar * count + x(continent) + color(continent) +
  palette("soft") + title("palette(\"soft\")")
```

Python

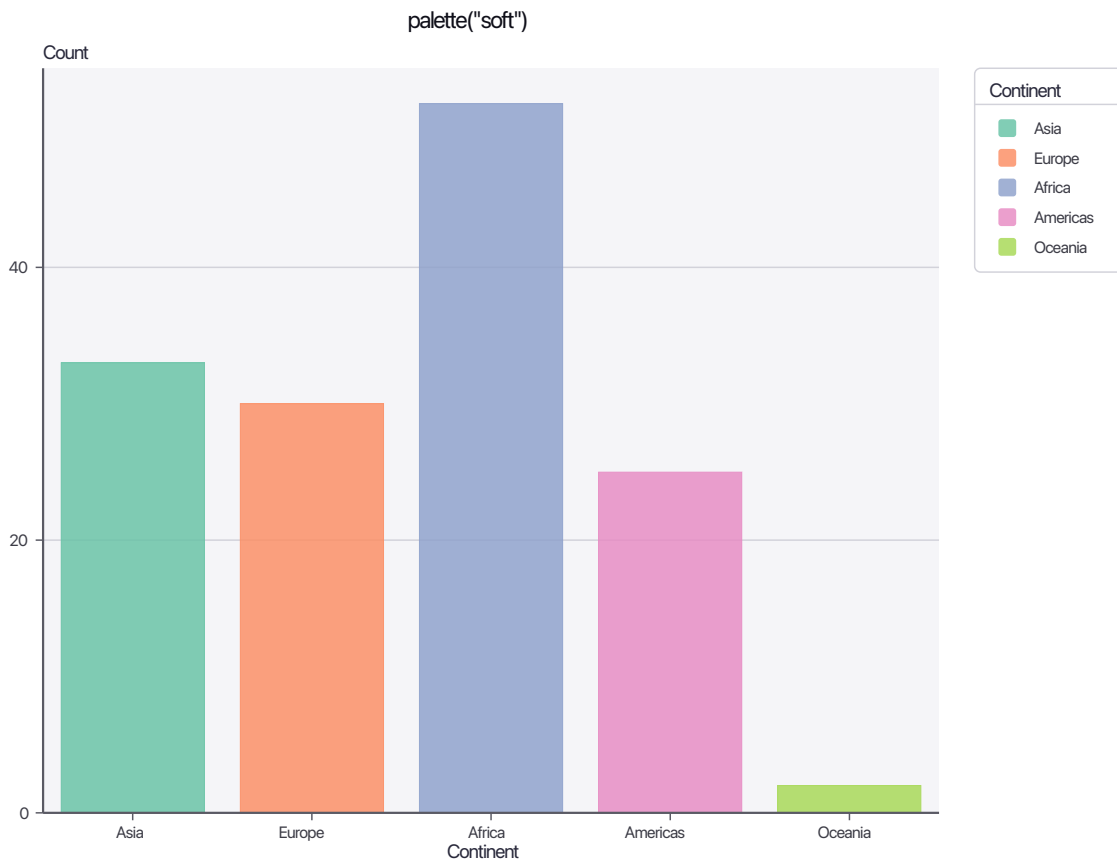
```
(data(gapminder_2007) + bar * count + x(col.continent) + color(col.continent)
↪ +
  palette("soft") + title("palette(\"soft\")"))
```

Julia

```
data(gapminder_2007) + bar * count + x(:continent) + color(:continent) +  
  palette("soft") + title("palette(\"soft\")")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, count), x(col.continent),  
  color(col.continent), palette("soft"), title("palette(\"soft\")"))
```



Nothing is wrong with the first one, and on a scatter of the same data it is the better choice. The difference is what the color is doing: a point has to be findable, while a bar is already visible and only has to differ from its neighbors.

23.8.2 Ramps

For a **numeric** color column, `palette()` names a ramp instead. These run one way, from low values to high, and lightness is what carries the ordering.

Name	What it is	Use it when
"blue"	the default: one hue, light→dark	almost always
"viridis"	many hues, with equal color steps for equal value steps	dense data where fine levels must stay distinct
"magma"	viridis's family: black through purple to cream	a dark plot with strong contrast
"inferno"	the same family, hotter	as above, warmer
"plasma"	the same family, blue through magenta to yellow	as above, cooler at the low end
"cividis"	the family's colorblind-optimized member	the ramp itself has to survive two-color vision (dichromacy)
"gray"	no hue at all: light gray to near-black	figures that will be printed in black and white

A table is a poor way to choose a ramp. Here are six of the seven, all but the default **blue**, on the Maunga Whau crater. The crater is cut into cells and colored by mean height, the dense continuous surface a ramp is picked for. `palette()` is a property of the plot, so six ramps means six plots, and | and / put them on one page:

R

```
((data(maunga_whau) + zone * bin(16) * mean + x(east) + y(north) +  
  color(elevation) + palette("viridis") + title("viridis")) |  
  (data(maunga_whau) + zone * bin(16) * mean + x(east) + y(north) +  
    color(elevation) + palette("magma") + title("magma"))) /  
((data(maunga_whau) + zone * bin(16) * mean + x(east) + y(north) +  
  color(elevation) + palette("inferno") + title("inferno")) |  
  (data(maunga_whau) + zone * bin(16) * mean + x(east) + y(north) +  
    color(elevation) + palette("plasma") + title("plasma"))) /  
((data(maunga_whau) + zone * bin(16) * mean + x(east) + y(north) +  
  color(elevation) + palette("cividis") + title("cividis")) |  
  (data(maunga_whau) + zone * bin(16) * mean + x(east) + y(north) +  
    color(elevation) + palette("gray") + title("gray")))
```

Python

```

(((data(maunga_whau) + zone * bin(16) * mean + x(col.east) + y(col.north) +
  color(col.elevation) + palette("viridis") + title("viridis")) |
  (data(maunga_whau) + zone * bin(16) * mean + x(col.east) + y(col.north) +
  color(col.elevation) + palette("magma") + title("magma")))) /
  ((data(maunga_whau) + zone * bin(16) * mean + x(col.east) + y(col.north) +
  color(col.elevation) + palette("inferno") + title("inferno")) |
  (data(maunga_whau) + zone * bin(16) * mean + x(col.east) + y(col.north)
↪ +
  color(col.elevation) + palette("plasma") + title("plasma")))) /
  ((data(maunga_whau) + zone * bin(16) * mean + x(col.east) + y(col.north) +
  color(col.elevation) + palette("cividis") + title("cividis")) |
  (data(maunga_whau) + zone * bin(16) * mean + x(col.east) + y(col.north)
↪ +
  color(col.elevation) + palette("gray") + title("gray"))))

```

Julia

```

(((data(maunga_whau) + zone * bin(16) * mean + x(:east) + y(:north) +
  color(:elevation) + palette("viridis") + title("viridis")) |
  (data(maunga_whau) + zone * bin(16) * mean + x(:east) + y(:north) +
  color(:elevation) + palette("magma") + title("magma")))) /
  ((data(maunga_whau) + zone * bin(16) * mean + x(:east) + y(:north) +
  color(:elevation) + palette("inferno") + title("inferno")) |
  (data(maunga_whau) + zone * bin(16) * mean + x(:east) + y(:north) +
  color(:elevation) + palette("plasma") + title("plasma")))) /
  ((data(maunga_whau) + zone * bin(16) * mean + x(:east) + y(:north) +
  color(:elevation) + palette("cividis") + title("cividis")) |
  (data(maunga_whau) + zone * bin(16) * mean + x(:east) + y(:north) +
  color(:elevation) + palette("gray") + title("gray")))

```

JavaScript

```

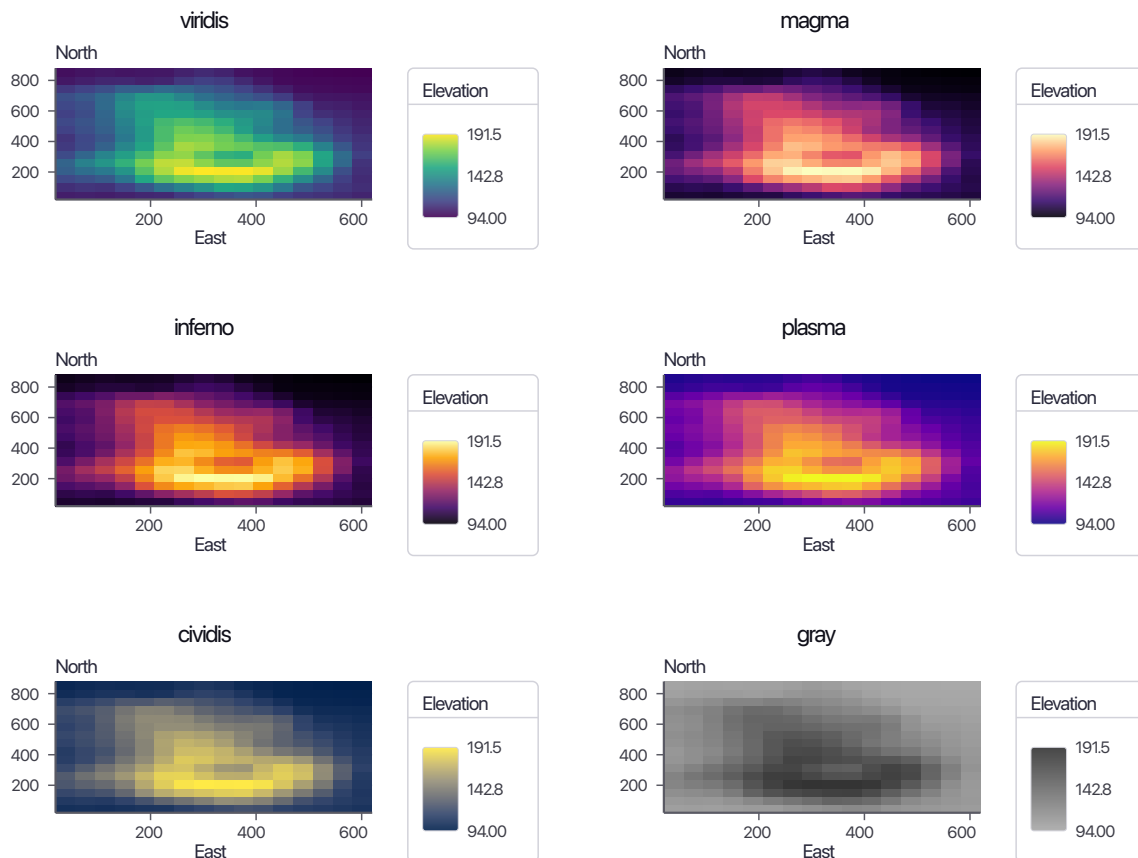
below(below(beside(plot(data(maunga_whau), layer(zone, bin(16), mean),
  x(col.east), y(col.north), color(col.elevation), palette("viridis"),
  title("viridis")),
  plot(data(maunga_whau), layer(zone, bin(16), mean), x(col.east),
  y(col.north), color(col.elevation), palette("magma"), title("magma"))),
  beside(plot(data(maunga_whau), layer(zone, bin(16), mean), x(col.east),
  y(col.north), color(col.elevation), palette("inferno"),
  title("inferno")),
  plot(data(maunga_whau), layer(zone, bin(16), mean), x(col.east),

```

```

y(col.north), color(col.elevation), palette("plasma"),
title("plasma")))),
beside(plot(data(maunga_whau), layer(zone, bin(16), mean), x(col.east),
y(col.north), color(col.elevation), palette("cividis"),
title("cividis")),
plot(data(maunga_whau), layer(zone, bin(16), mean), x(col.east),
y(col.north), color(col.elevation), palette("gray"), title("gray"))))

```



“Given Maunga Whau: zones derived by bin into 16 and mean, x is east, y is north, color by elevation, with the viridis palette, beside the same with magma; above the same pair with inferno and plasma; above the same pair with cividis and gray.”

That is the same sentence six times with one word changed. It is written out in full because a shortcut here would be R code, and every sentence in this book is drawn in four languages.

Four things are worth noticing in those six plots.

`gray` runs the other way. Its crater is dark at the summit while the other five are bright there. That is not a mistake in either: `gog`'s own ramps (`gray` and the default `blue`) run light to dark, because with one hue a darker color reads as a larger value, and that also holds after photocopying. The `viridis` family runs dark to light, because that is what those ramps are, and reversing them would give a ramp named `magma` that no longer looks like magma. The legend is what settles it in any given plot, and it is the reason a mapped channel always gets one: read the top label and you know which end holds the largest values.

Of the five imported ramps, `magma` and `inferno` end on a near-white. `magma`'s last stop holds barely any contrast against the panel, so the very top of the scale nearly disappears, which you can see in the pale flat summits above. A ramp with equal color steps ends pale by design, so that is not an oversight, and it is why `blue` is the default rather than one of these. `blue` and `gray` stop short of white, so their pale end stays visible.

`cividis` has the least contrast, and deliberately: it uses only the two hues that survive the common forms of color blindness. Less contrast is what that costs, and it is worth accepting whenever the ramp itself has to stay readable.

`gray` is for the figure that has to survive photocopying. It is a ramp only, and there is no categorical gray palette. Readers cannot tell more than about four grays apart, and the grammar has a better answer for categories in a print figure. That answer is `pattern`, which separates them without spending color at all, and asking for `palette("gray")` on a text column says so:

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + palette("gray")
```

Error:

```
! gog: `palette("gray")` is a sequential ramp for numbers, but `color` is
  ↳ bound to a text column. Use a categorical palette - `gog`, `okabe`, or
  ↳ `soft` - or list one color per category. For categories in a figure that
  ↳ has to print in black and white, reach for `pattern(<column>)` instead -
  ↳ it tells them apart without spending color.
```

`gog`: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

23.8.3 Diverging ramps

A sequential ramp means *more*. A **diverging** ramp means *away from a center, in two directions*, which is what a change, a residual, a correlation and an anomaly all are. `gog` provides two:

Name	What it is	Use it when
"blue_red"	blue → neutral → red	the usual choice

Name	What it is	Use it when
"brown_tea"	brown → neutral → teal	the sign of the value is what matters, and readers may be colorblind

The name reads low to high, so `blue_red` paints the smallest values blue and the largest red. Both turn through the same gray, and that gray marks the center of the scale.

Which value gets the neutral is not a property of the palette. It is the middle of the scale's domain, so `limits` is what puts zero in the middle. Without `limits`, the middle falls on the data's own midpoint. Here the cash flow runs from -32 to 120, so the midpoint is 44: that is what gets painted gray, and zero comes out blue.

R

```
data(cashflow) + point + x(step) + y(delta) +
  color(delta) + palette("blue_red") + style(size = 9) +
  title("The neutral lands at 44, not at 0")
```

Python

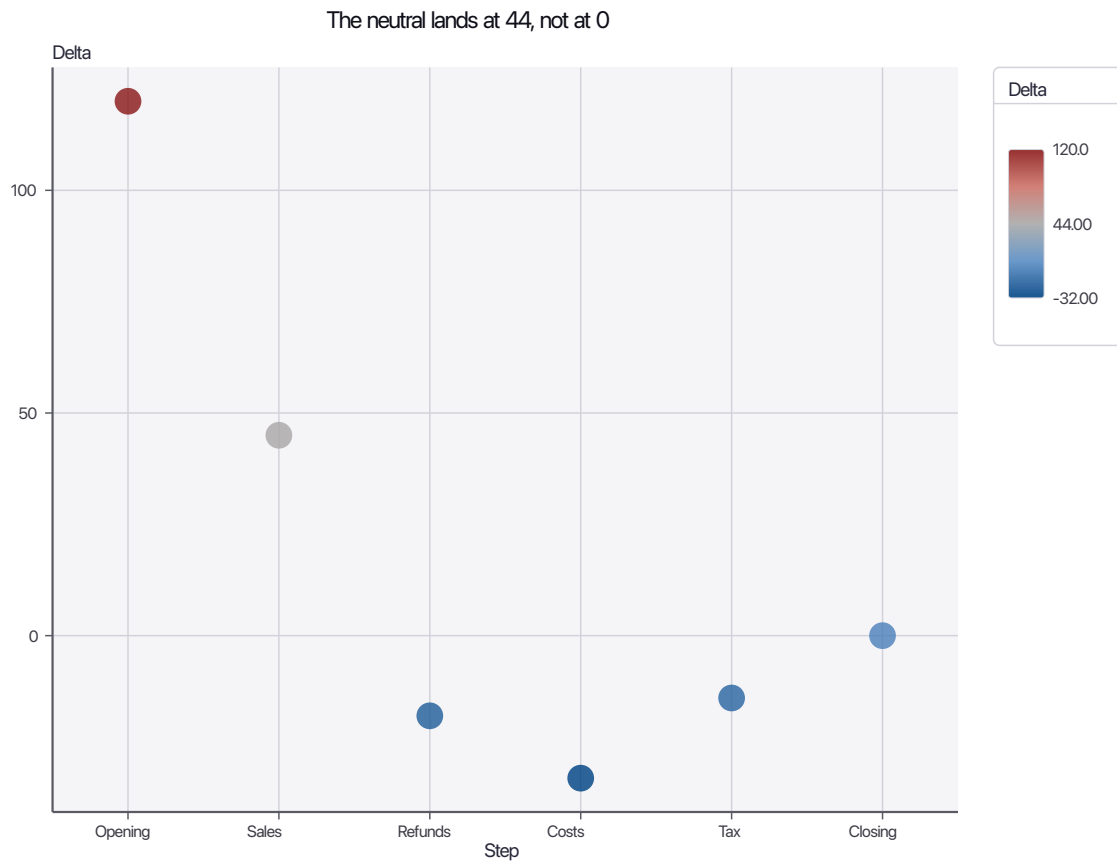
```
(data(cashflow) + point + x(col.step) + y(col.delta) +
  color(col.delta) + palette("blue_red") + style(size = 9) +
  title("The neutral lands at 44, not at 0"))
```

Julia

```
data(cashflow) + point + x(:step) + y(:delta) + color(:delta) +
  palette("blue_red") + style(size = 9) +
  title("The neutral lands at 44, not at 0")
```

JavaScript

```
plot(data(cashflow), point, x(col.step), y(col.delta), color(col.delta),
  palette("blue_red"), style({ size: 9 }),
  title("The neutral lands at 44, not at 0"))
```



State a symmetric domain and the neutral lands on zero. Read the “Closing” point, which is exactly zero:

R

```
data(cashflow) + point + x(step) + y(delta) +
  color(delta, limits = c(-120, 120)) + palette("blue_red") + style(size = 9)
↪ +
  title("limits = c(-120, 120) puts zero on the neutral")
```

Python

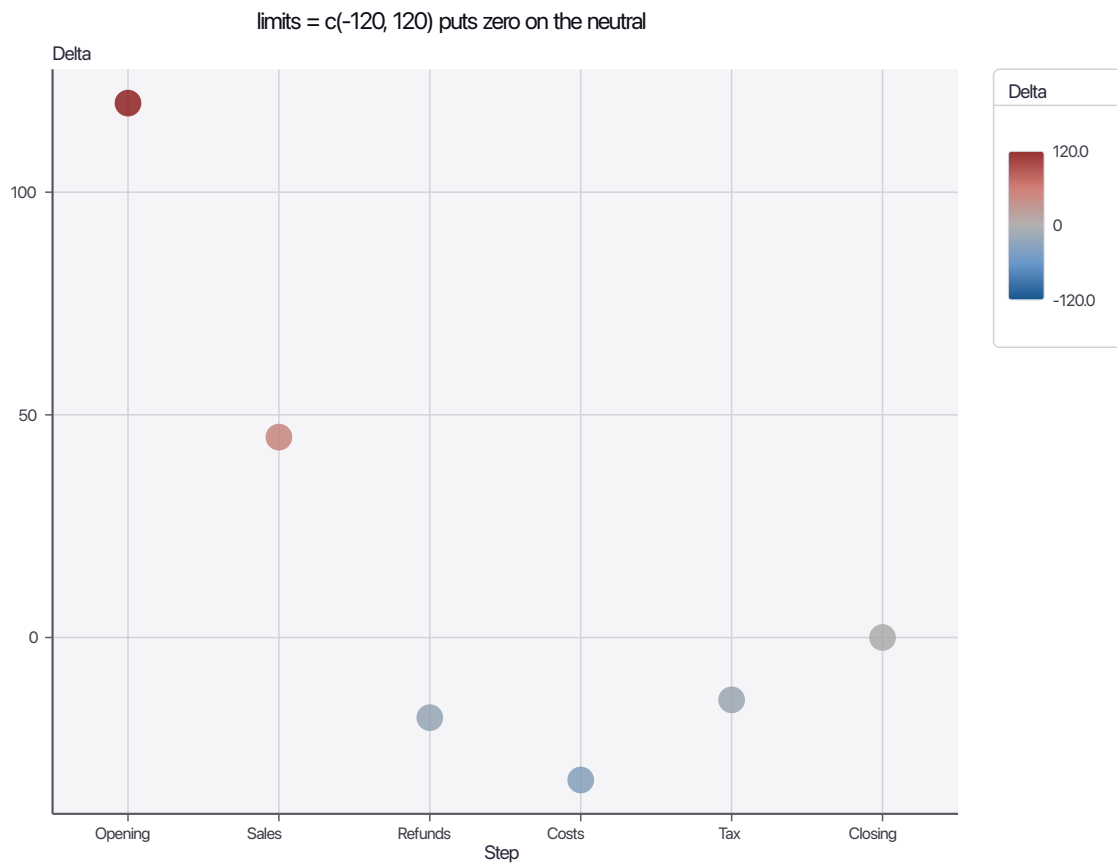
```
(data(cashflow) + point + x(col.step) + y(col.delta) +
  color(col.delta, limits = [-120, 120]) + palette("blue_red") + style(size =
↪ 9) +
  title("limits = c(-120, 120) puts zero on the neutral"))
```

Julia

```
data(cashflow) + point + x(:step) + y(:delta) +  
  color(:delta, limits = [-120, 120]) + palette("blue_red") +  
  style(size = 9) +  
  title("limits = c(-120, 120) puts zero on the neutral")
```

JavaScript

```
plot(data(cashflow), point, x(col.step), y(col.delta),  
  color(col.delta, { limits: [-120, 120] }), palette("blue_red"),  
  style({ size: 9 }),  
  title("limits = c(-120, 120) puts zero on the neutral"))
```



“Given the cashflow table: points, x is step, y is delta, color by delta from -120 to 120, with the blue red palette.”

Two things follow from doing it this way. The ramp stays **linear**: equal distance in color is equal distance in the data on both sides of the center. A scale that fixes a midpoint and then rescales each side to the data's own ends loses that. And `limits` removes rows outside it, so the domain has to be wide enough to hold the data: `c(-50, 50)` here would drop the opening balance and say so. Symmetric limits have one drawback, worth knowing before you set them. If a signed column runs from -2 to 100, almost none of the blue half is used, because the data reaches only a little way below zero.

The colors you pass become the stops, and `gog` interpolates between them:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  color(life) + palette(c("white", "navy")) + style(size = 6) +  
  title("Custom stops")
```

Python

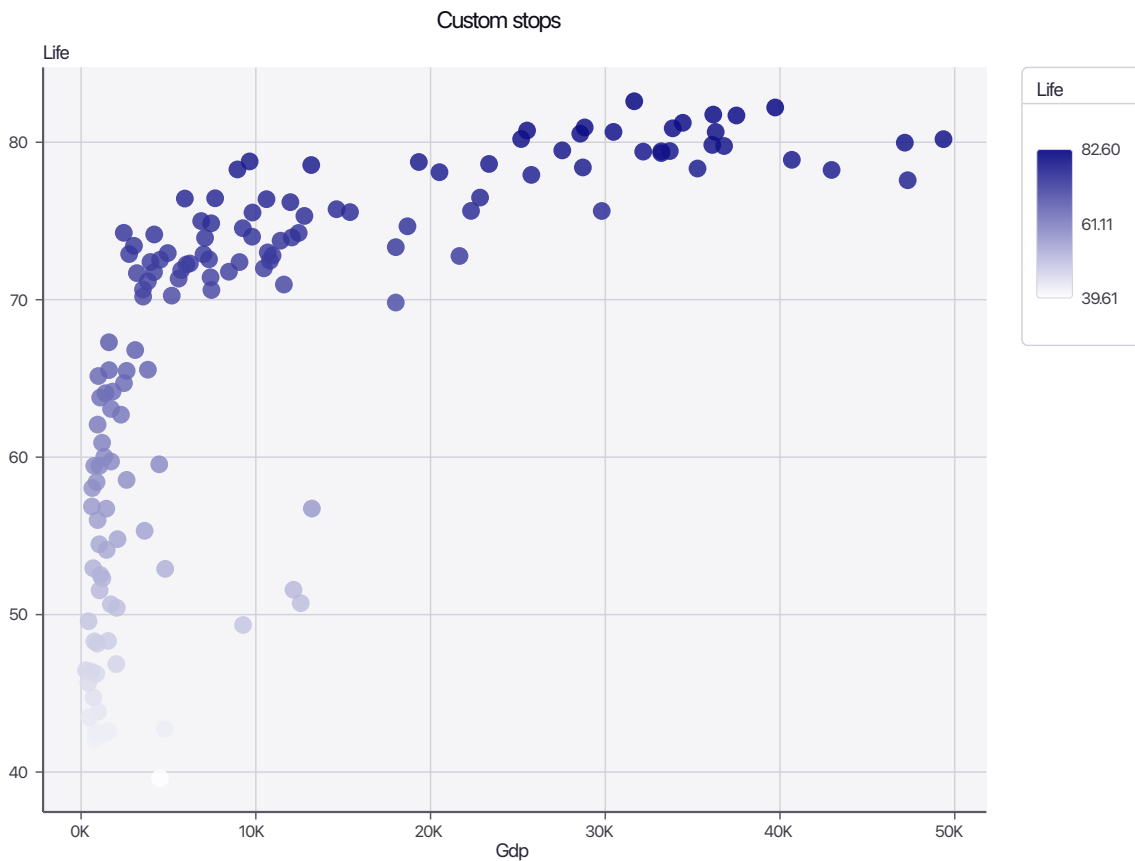
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.life) + palette(["white", "navy"]) + style(size = 6) +  
  title("Custom stops"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:life) +  
  palette(["white", "navy"]) + style(size = 6) + title("Custom stops")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.life), palette(["white", "navy"]), style({ size: 6 }),  
  title("Custom stops"))
```



“Given gapminder 2007: points, x is gdp, y is life, color by life, with a palette of white and navy.”

Asking for a palette of the wrong kind is refused rather than guessed at. A categorical palette cannot serve a numeric column:

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(life) + palette("okabe")
```

Error:

```
! gog: `palette("okabe")` hands out one color per category, but `color` is
  ↳ bound to a numeric column. Use a ramp - `blue`, `viridis`, `magma`,
  ↳ `inferno`, `plasma`, `cividis`, or `gray` run one way, `blue_red` or
  ↳ `brown_teal` diverge from a center - or give your own stops, e.g.
  ↳ `palette(c("white", "navy"))`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

No ramp survives a skewed column, and swapping palettes will not help. A linear ramp gives most of its colors to the range where most rows sit. Color `population` and nearly every country falls at the light end, because two countries are several times larger than any other. That is the data, not the palette, so the fix is a [log scale](#) on the channel:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(population, scale = "log") +  
  x_label("GDP per capita")
```

Python

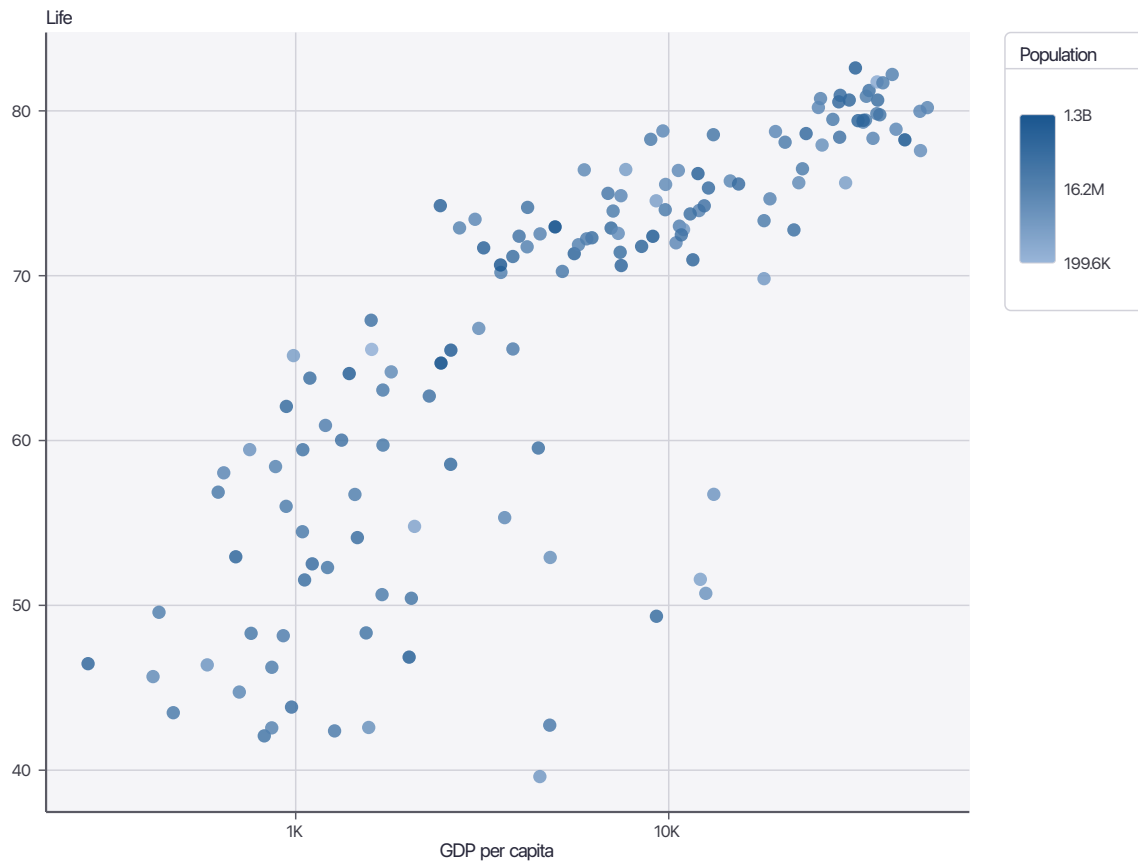
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.population, scale = "log") +  
  x_label("GDP per capita"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:population, scale = "log") + x_label("GDP per capita")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.population, { scale: "log" }),  
  x_label("GDP per capita"))
```



color, size and opacity all take `scale =`, because all three answer “how far along?”. `shape`, `pattern` and `group` do not: they answer “which one?”.

23.8.4 Custom colors

Pass the colors yourself: hex values, CSS color names such as `firebrick`, or a mix of the two:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) +
  palette(c("#e41a1c", "#377eb8", "#4daf4a", "#984ea3", "#ff7f00"))
```

Python

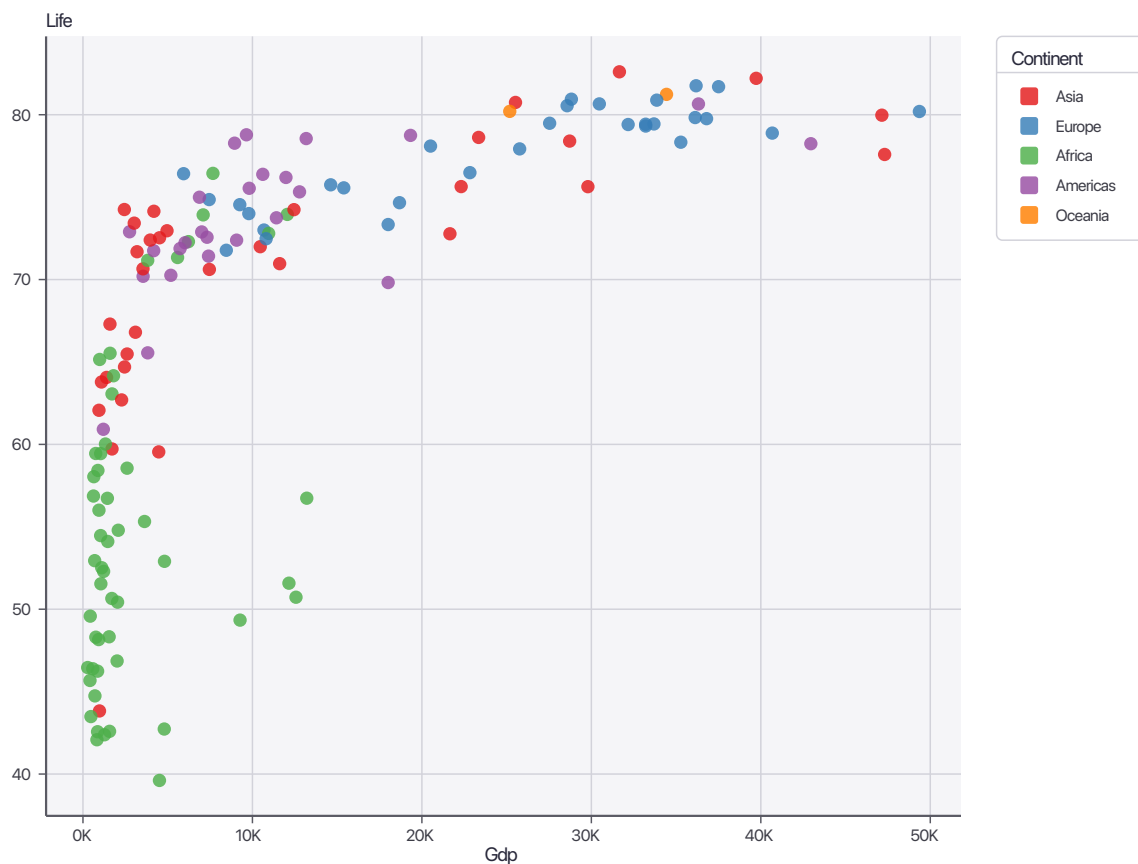

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  color(col.continent) +
  palette(["#e41a1c", "#377eb8", "#4daf4a", "#984ea3", "#ff7f00"]))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +
  palette(["#e41a1c", "#377eb8", "#4daf4a", "#984ea3", "#ff7f00"])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  color(col.continent),
  palette(["#e41a1c", "#377eb8", "#4daf4a", "#984ea3", "#ff7f00"]))
```



R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  color(continent) +  
  palette(c("firebrick", "steelblue", "seagreen", "rebeccapurple",  
    ↪ "darkorange"))
```

Python

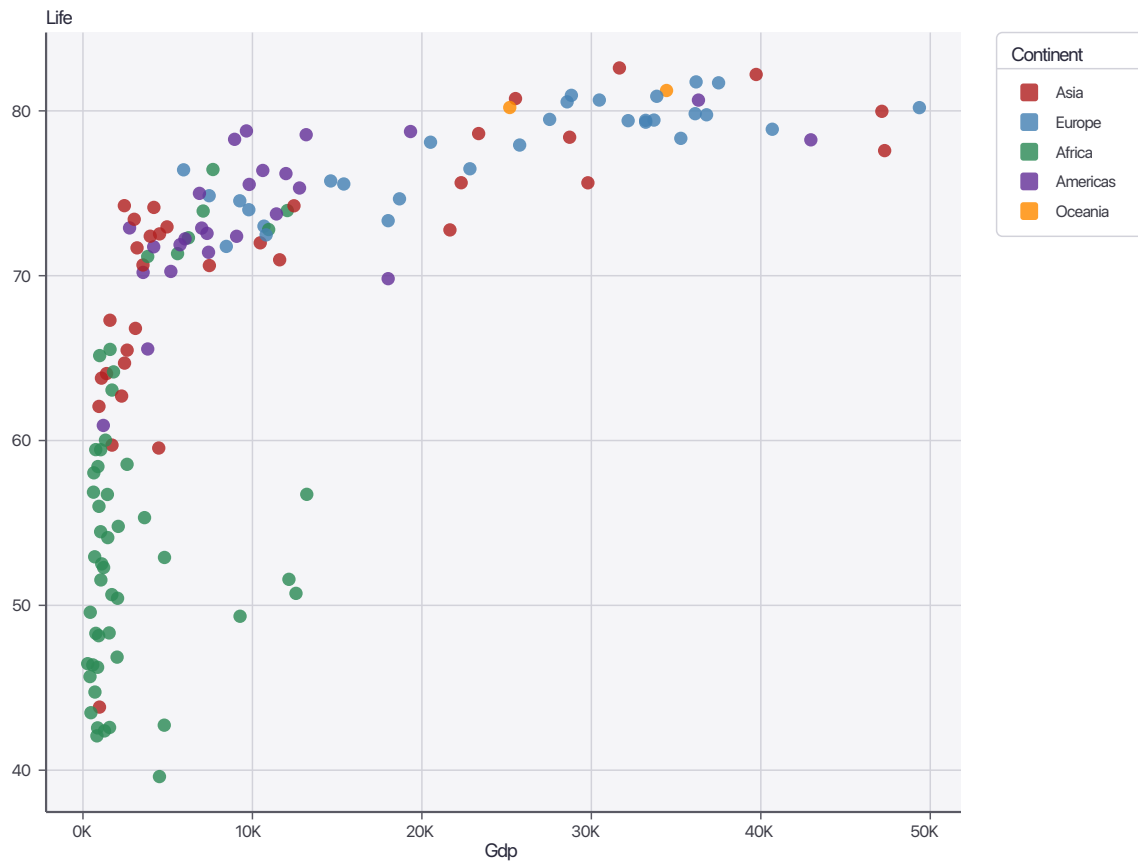
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  color(col.continent) +  
  palette(["firebrick", "steelblue", "seagreen", "rebeccapurple",  
    ↪ "darkorange"]))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  palette(["firebrick", "steelblue", "seagreen", "rebeccapurple",  
    ↪ "darkorange"])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent),  
  palette(["firebrick", "steelblue", "seagreen", "rebeccapurple",  
    "darkorange"]))
```



A color `gog` cannot recognize is refused with the nearest match offered, rather than handed to the renderer to be painted black:

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + palette(c("firebrick", "stelblue"))
```

Error:

```
! gog: `palette()` entry "stelblue" is not a color. Did you mean "steelblue"?
  ↳ Use a CSS color name, or a hex value like "#4e79a7".
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Colors are assigned in **first-appearance order** in the data (the first unique value gets the first palette color, the second unique value gets the second, and so on), so the same table always gets the same colors. (Unless the column [declares its order](#); then that order sets the sequence.)

One mistake to know about: a single color is not a palette. `palette()` takes a palette *name* or the *colors themselves*, and `palette("red")` is neither; it is an attempt to set one constant color, which is `style(color = "red")`:

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  color(continent) + palette("red")
```

Error:

```
! gog: `palette("red")` is not a known palette. Named palettes are gog,  
  ↳ okabe, soft for categories; blue, viridis, magma, inferno, plasma,  
  ↳ cividis, gray for numbers; blue_red, brown_tea for numbers that diverge  
  ↳ from a center. `"red"` is a color, not a palette. To paint every mark one  
  ↳ color use `style(color = "red")`; to give each category its own color  
  ↳ pass a vector: `palette(c("red", ...))`.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

23.9 Legend placement

gog places every legend to the right of the plot area. The plot area shrinks by itself to make room, so there is no margin to set, no position to choose, and no way to hide a legend.

24 Setting vs mapping

Should a mark's color come from your data, or from you? There are two ways to make something red, and `gog` keeps them apart on purpose.

Mapping lets the data decide a visual property. `color(continent)` says *let the continent decide the color*, so `gog` builds a scale, assigns a color per continent, and draws a legend to decode it.

Setting means you give the property a value yourself. `style(color = "tomato")` says *make them all tomato*. No column is read, no scale is built, and no legend appears, because there is nothing to decode.

R

```
# MAP: color answers "which continent?"
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent) +
  title("Mapped: a legend decodes the color")
```

Python

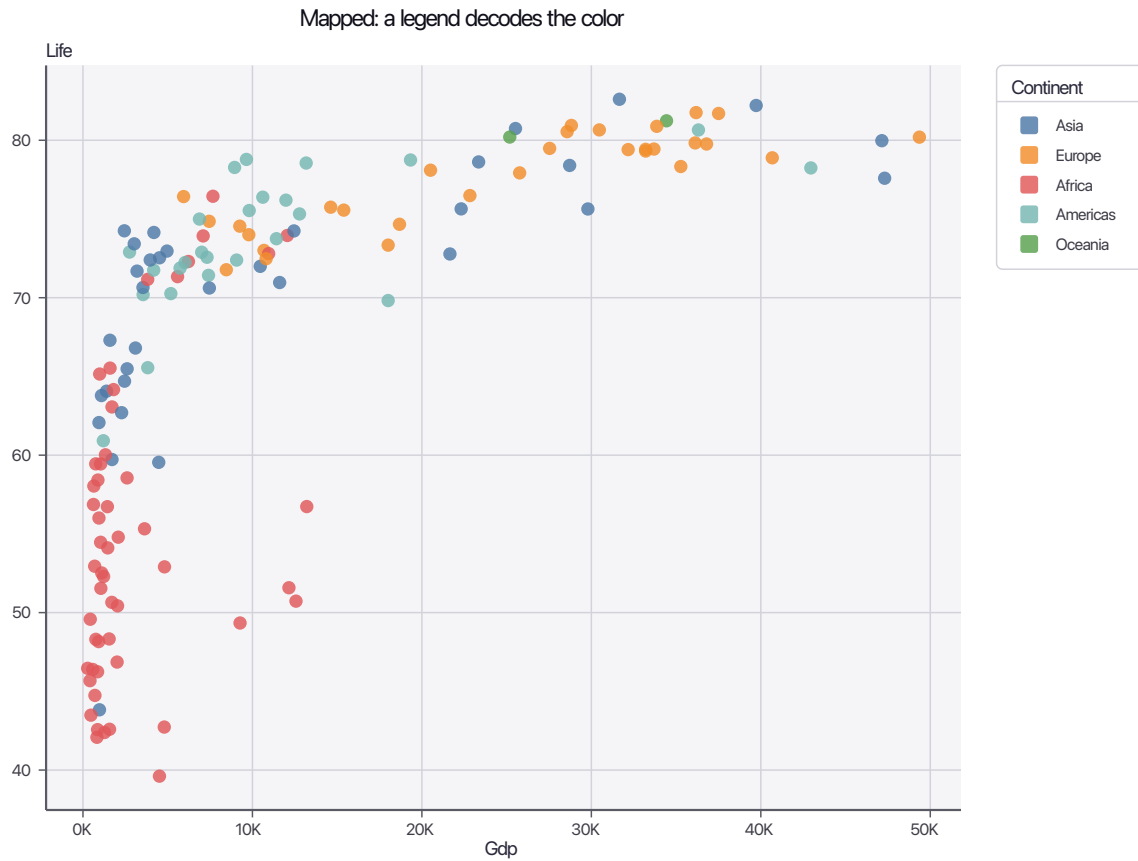
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  ↪ color(col.continent) +
  title("Mapped: a legend decodes the color"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +
  title("Mapped: a legend decodes the color")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  color(col.continent), title("Mapped: a legend decodes the color"))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent.”

R

```
# SET: color answers nothing; note the missing legend
data(gapminder_2007) + point + x(gdp) + y(life) +
  style(color = "tomato") +
  title("Set: no legend, because there is nothing to decode")
```

Python

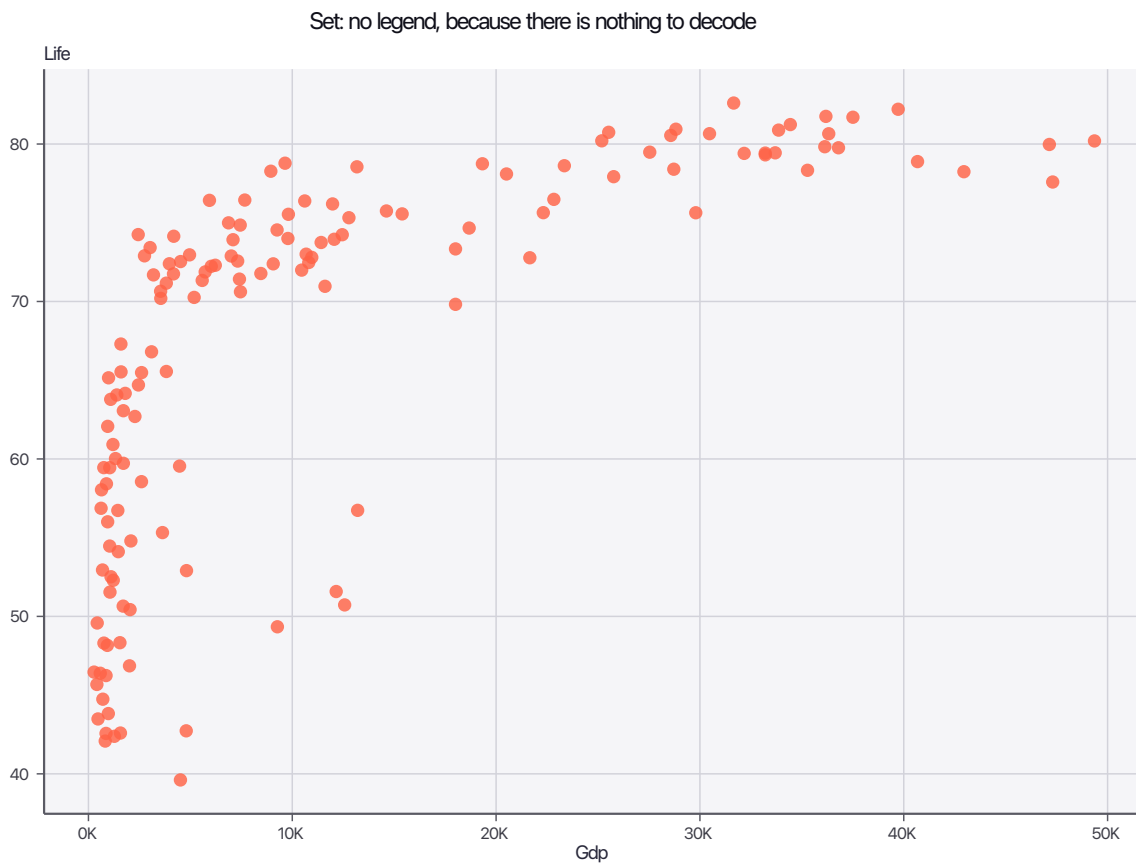
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  style(color = "tomato") +
  title("Set: no legend, because there is nothing to decode"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  style(color = "tomato") +  
  title("Set: no legend, because there is nothing to decode")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  style({ color: "tomato" }),  
  title("Set: no legend, because there is nothing to decode"))
```



“Given gapminder 2007: points, x is gdp, y is life, colored tomato.”

24.1 Why a constant is not a channel

Every channel in `gog` maps a data column to a visual property. A constant maps nothing, so it fails the definition: the same reason rotating a 3-D plot is a property of the coordinate space rather than a channel.

A practical test agrees: a mapping to color, size, shape, pattern or opacity draws a legend, and a setting draws none. If the plot needs a legend to be read, it was a mapping.

24.2 What you can set

Every setting is a row and every mark is a column. The grid shows what each mark takes and what it refuses, including combinations you would not have guessed were legal. Legal is not the same as useful. [Law 8](#) says the grammar guarantees a plot is legal, and you judge whether it is useful. The table shows the limit of what can be expressed. Its companion grid, what a mark lets you *map* rather than set, is [Combinations](#).

Set- ting	point	line	area	bar	step	interval	rule	ribbon	text	path	rule	zone	surface	edge
color														
opacity														
size			—	—				—				—	—	
shape	—		—	—	—	—	—	—	—	—	—	—	—	—
pattern									—				—	
border_color	—		—		—	—	—	—	—	—	—			—
border_size	—		—		—	—	—	—	—	—	—			—
caps	—	—	—	—	—		—	—	—	—	—	—	—	—
center	—	—	—	—	—		—	—	—	—	—	—	—	—
nudge	—	—	—	—	—	—	—	—		—	—	—	—	—
arrow	—	—	—	—	—	—	—	—	—		—	—	—	—
reach	—	—	—	—	—	—	—	—	—	—		—	—	—

(the setting is available · - the mark's geometry cannot carry it. Generated live from the engine, like the [Combinations](#) grid, so it cannot drift from what `style()` accepts.)

`color` (CSS name or hex) and `opacity` (0–1) are set on every mark. The other rows are not arbitrary permissions: each follows from the mark's geometry, and a setting is available on *every* mark of a kind, never one by accident. That is the [settable rule](#): a setting spans its *geometry class*, the group of marks built the same way.

size is a measurement in pixels: a radius, a stroke width, or a font size. It belongs to every glyph and stroke. The grid leaves it blank for **bar**, **area**, **ribbon**, **zone** and **surface**, whose positions already fix their size. **shape** (one of five glyphs) is **point**'s alone.

pattern is the **texture** of a mark's paint: a general aesthetic, realized by geometry the way **color** is a fill on a bar and a stroke on a line. On the six path strokes (**line**, **step**, **path**, **interval**, **rule**, **edge**) it is the **dash**: "solid"/"dashed"/"dotted", a dashed error bar included. On the five fills (**bar**, **box**, **area**, **ribbon**, **zone**) it is a **fill texture**: "solid"/"hatch"/"crosshatch"/"grid"/"dots", the hatchings a plot can be read by in grayscale or in print, where hue is gone. It is paint, never geometry: a dashed line and a solid one trace the same path. A **step**, by contrast, is a different mark because it traces a different path.

R

```
data(gapminder_asia) + line * mean + x(year) + y(life) +  
  style(pattern = "dashed", size = 1.5) + y_label("Mean life expectancy") +  
  title("A dashed trend line")
```

Python

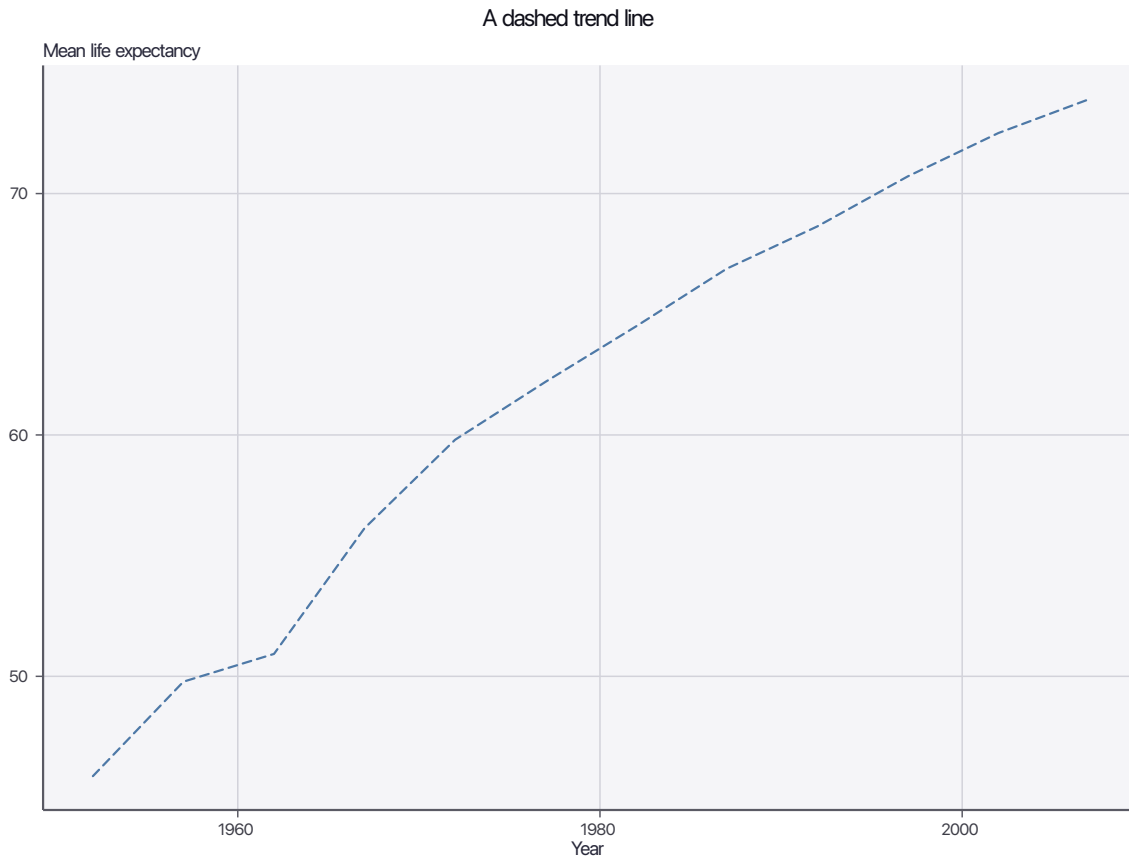
```
(data(gapminder_asia) + line * mean + x(col.year) + y(col.life) +  
  style(pattern = "dashed", size = 1.5) + y_label("Mean life expectancy") +  
  title("A dashed trend line"))
```

Julia

```
data(gapminder_asia) + line * mean + x(:year) + y(:life) +  
  style(pattern = "dashed", size = 1.5) +  
  y_label("Mean life expectancy") + title("A dashed trend line")
```

JavaScript

```
plot(data(gapminder_asia), layer(line, mean), x(col.year), y(col.life),  
  style({ pattern: "dashed", size: 1.5 } ),  
  y_label("Mean life expectancy"), title("A dashed trend line"))
```



“Given gapminder Asia: a line derived by mean, x is year, y is life, with pattern dashed and size 1.5.”

On a fill the same setting hatches the region, a print-ready look that survives a black-and-white photocopy:

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life) +
  style(pattern = "crosshatch") +
  y_label("Mean life expectancy") +
  title("A crosshatched bar")
```

Python

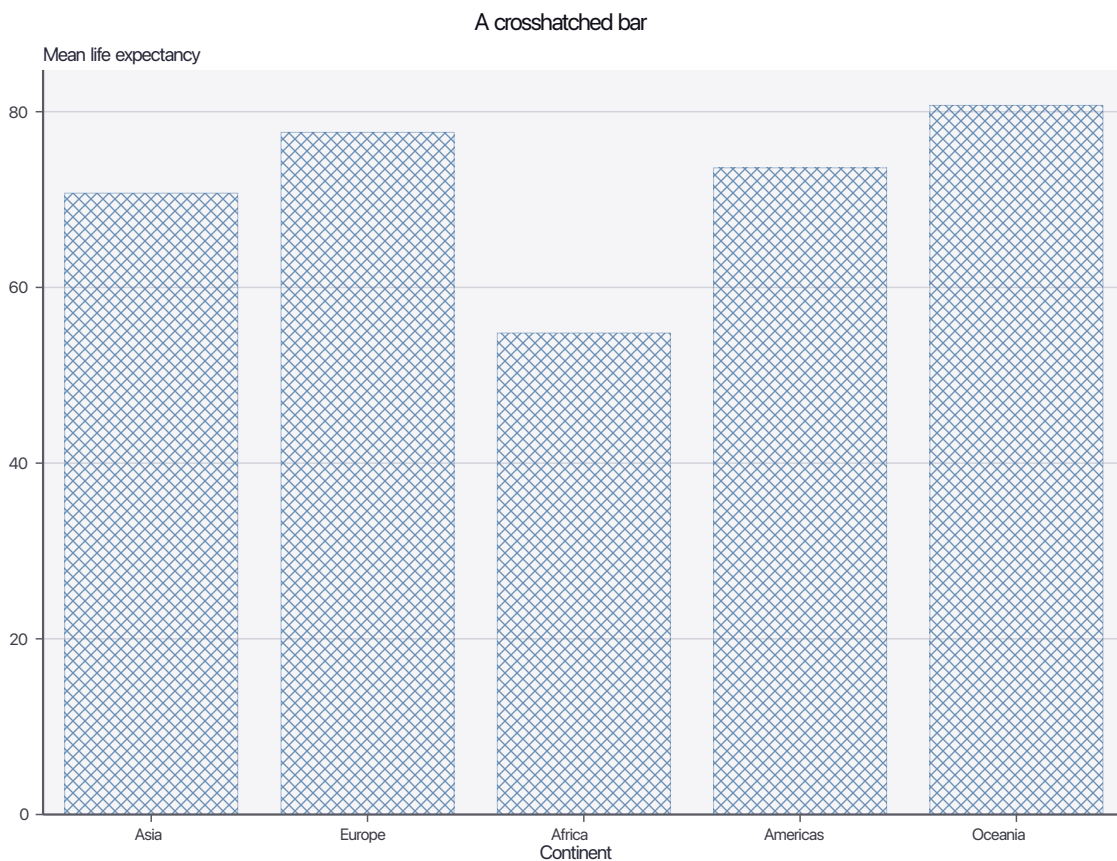
```
(data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life) +
  style(pattern = "crosshatch") +
  y_label("Mean life expectancy") +
  title("A crosshatched bar"))
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life) +
  style(pattern = "crosshatch") + y_label("Mean life expectancy") +
  title("A crosshatched bar")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),
  y(col.life), style({ pattern: "crosshatch" }),
  y_label("Mean life expectancy"), title("A crosshatched bar"))
```



“Given *gapminder 2007*: bars derived by mean, *x* is continent, *y* is life, with pattern crosshatch.”

The texture is one per layer: `style()` gives every bar in the layer the *same* hatch, so it cannot give each category of a color split its own. Doing that is a *mapping* (a category to a texture), which is the `pattern()` channel, the texture channel that matches `shape()`.

`border_color` and `border_size` draw an outline separate from the fill, and 0 means no border. They belong to the five **closed-glyph fills** (bar, box, point, zone and surface). On those marks no composition can trace the border, so a setting is the only way to draw it. `color` fills the shape; `border_color` outlines it; the two are independent, so a box can be pale inside and dark-edged:

R

```
data(gapminder_2007) + box + x(continent) + y(life) +  
  style(color = "lightsteelblue", border_color = "#26456e", border_size =  
↪ 1.6) +  
  y_label("Life expectancy") + title("A box: light fill, dark outline")
```

Python

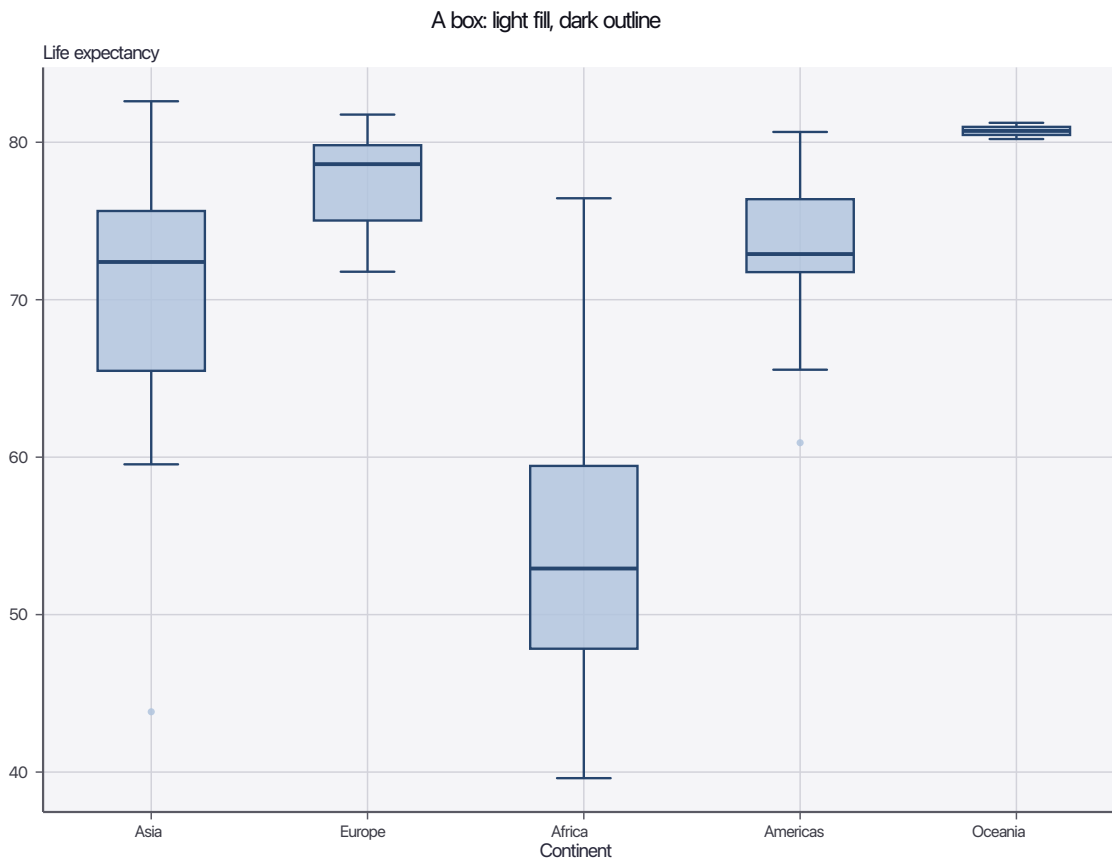
```
(data(gapminder_2007) + box + x(col.continent) + y(col.life) +  
  style(color = "lightsteelblue", border_color = "#26456e", border_size =  
↪ 1.6) +  
  y_label("Life expectancy") + title("A box: light fill, dark outline"))
```

Julia

```
data(gapminder_2007) + box + x(:continent) + y(:life) +  
  style(color = "lightsteelblue", border_color = "#26456e", border_size =  
↪ 1.6) +  
  y_label("Life expectancy") + title("A box: light fill, dark outline")
```

JavaScript

```
plot(data(gapminder_2007), box, x(col.continent), y(col.life),  
  style({ color: "lightsteelblue", border_color: "#26456e",  
    border_size: 1.6 }), y_label("Life expectancy"),  
  title("A box: light fill, dark outline"))
```



“Given *gapminder 2007*: boxes, x is continent, y is life, colored *lightsteelblue*, with border color *#26456e* and border size 1.6.”

The same two settings outline a **point**, so overlapping points stay separate, exactly as on **bar** and **box**:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  style(color = "steelblue", opacity = 0.7, size = 5, border_color = "white",
  ↪ border_size = 1) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("A point with a white border")
```

Python

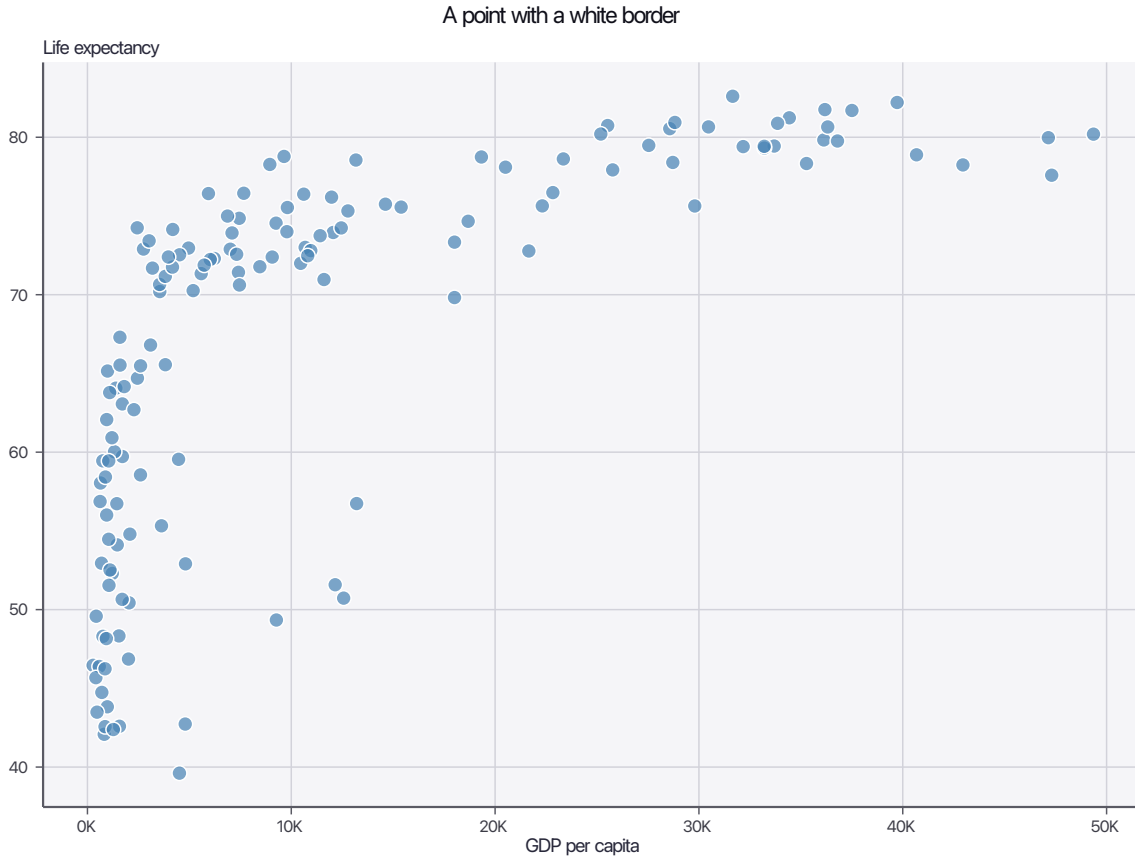
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  style(color = "steelblue", opacity = 0.7, size = 5, border_color = "white",
  ↪ border_size = 1) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("A point with a white border"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  style(color = "steelblue", opacity = 0.7, size = 5, border_color = "white",
  ↪ border_size = 1) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("A point with a white border")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  style({ color: "steelblue", opacity: 0.7, size: 5,
  border_color: "white", border_size: 1 }), x_label("GDP per capita"),
  y_label("Life expectancy"), title("A point with a white border"))
```



“Given gapminder 2007: points, x is gdp, y is life, colored steelblue, with opacity 0.7, size 5, border color white and border size 1.”

A **curve fill** (`area`, `ribbon`) takes no border. Its edge is a data curve, so you draw it as a **line** layer (`area + line`, `line * bounds`), which can do more than a border setting. And a **cross** point has no fill to outline, so a border on a **cross**-shaped glyph simply has nothing to draw on.

The last five rows are **display toggles**, each tied to one mark’s own geometry. **caps** and **center** show or hide an **interval**’s end caps and center dot. **nudge** moves a **text** label off its point. **arrow** puts a head on a **path**’s end. And **reach** says how far a **rule** crosses the axis it does not name. They are single-mark on purpose, because only a whisker has a cap. That is why their rows are nearly empty, and an empty row tells you the setting belongs to one mark.

arrow shows this best: only **path** is checked in its row, and the reason is geometry. A head marks a direction, and only a path has one: a **line** sorts its vertices by **x**, so its last point is the largest **x**, not the point the data ended on.

R

```
data(gapminder_asia) + path + x(gdp) + y(life) + color(country) +  
  style(arrow = "end") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Each path ends at 2007")
```

Python

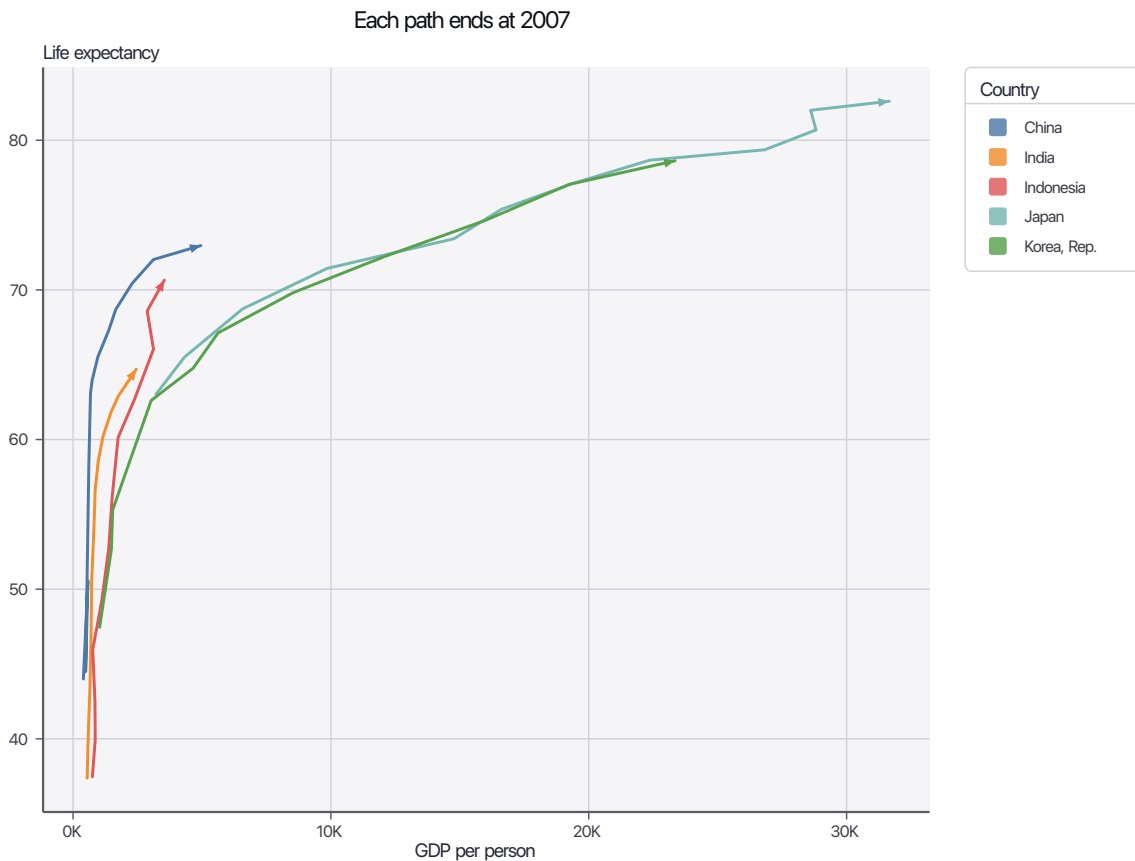
```
(data(gapminder_asia) + path + x(col.gdp) + y(col.life) + color(col.country)  
↪ +  
  style(arrow = "end") +  
  x_label("GDP per person") + y_label("Life expectancy") +  
  title("Each path ends at 2007"))
```

Julia

```
data(gapminder_asia) + path + x(:gdp) + y(:life) + color(:country) +  
  style(arrow = "end") + x_label("GDP per person") +  
  y_label("Life expectancy") + title("Each path ends at 2007")
```

JavaScript

```
plot(data(gapminder_asia), path, x(col.gdp), y(col.life),  
  color(col.country), style({ arrow: "end" }), x_label("GDP per person"),  
  y_label("Life expectancy"), title("Each path ends at 2007"))
```

“Given gapminder Asia: paths, x is gdp, y is life, color by country, with arrow end.”

It takes "end", "start", or "both": a value rather than a TRUE/FALSE, because a double-headed arrow is a common request, and TRUE/FALSE would have needed a second setting.

24.2.1 One color, any mark

`style(color = )` is the same instruction whichever mark it lands on. It fills points, strokes lines, and fills bars:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  style(color = "red") +
  title("point")
```

Python

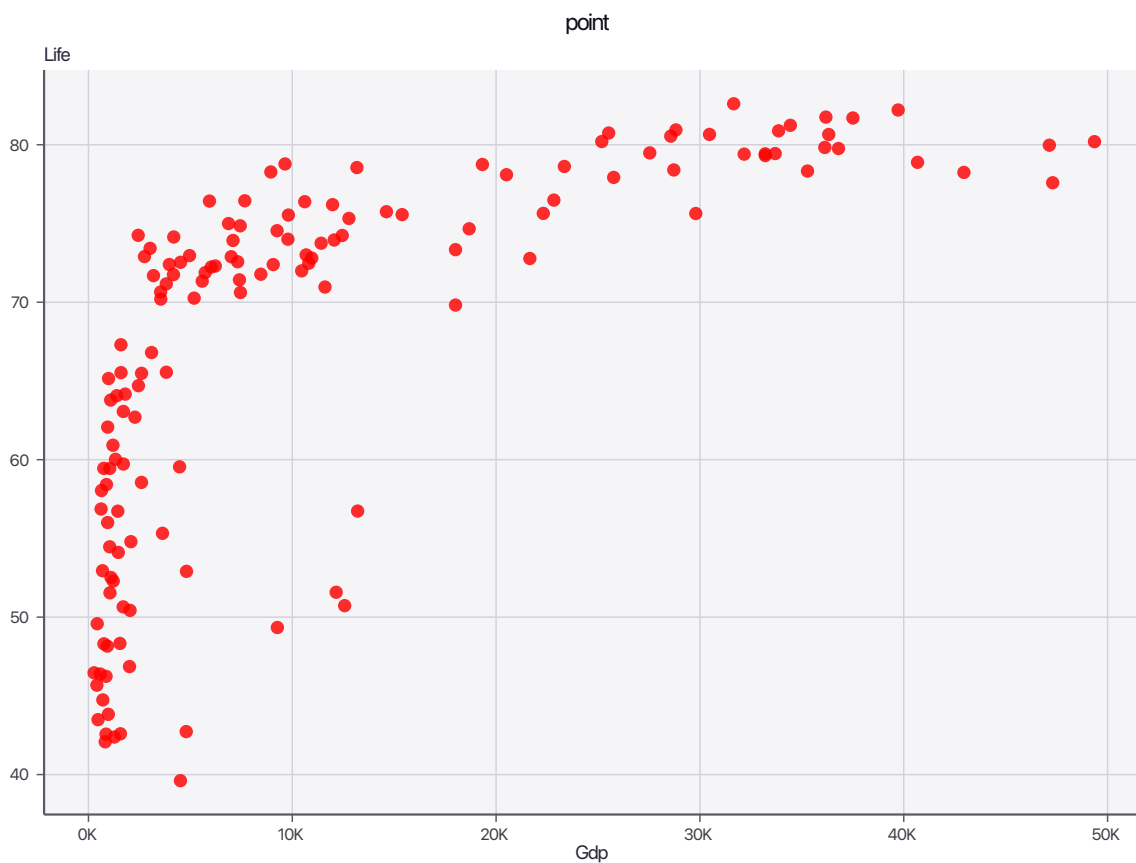
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  style(color = "red") +  
  title("point"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + style(color = "red") +  
  title("point")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  style({ color: "red" }), title("point"))
```



R

```
data(gapminder_asia) + line + x(year) + y(life) + group(country) +  
  style(color = "red") +  
  title("line")
```

Python

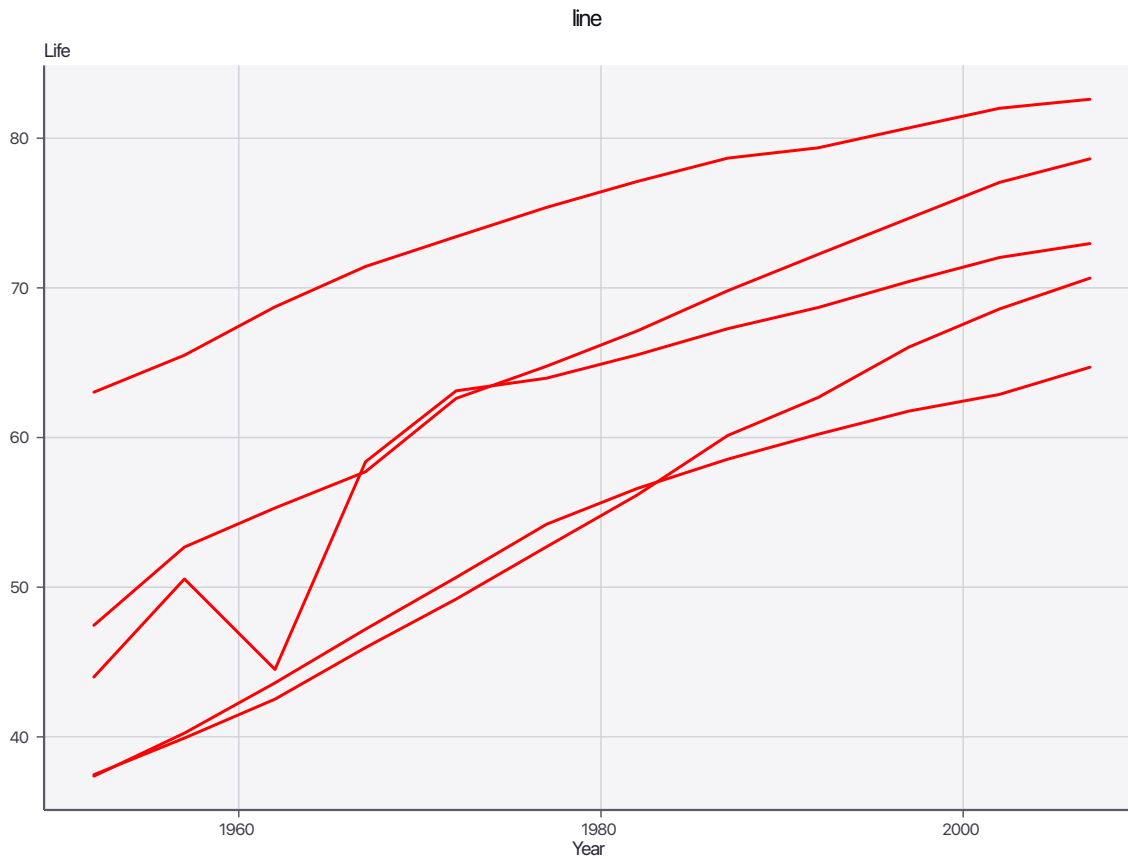
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + group(col.country)  
↪ +  
  style(color = "red") +  
  title("line"))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country) +  
  style(color = "red") + title("line")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
  group(col.country), style({ color: "red" }), title("line"))
```



“Given gapminder Asia: lines, x is year, y is life, grouped by country, colored red.”

R

```
data(medals) + bar + x(country) + y(gold) +
  style(color = "red") +
  title("bar")
```

Python

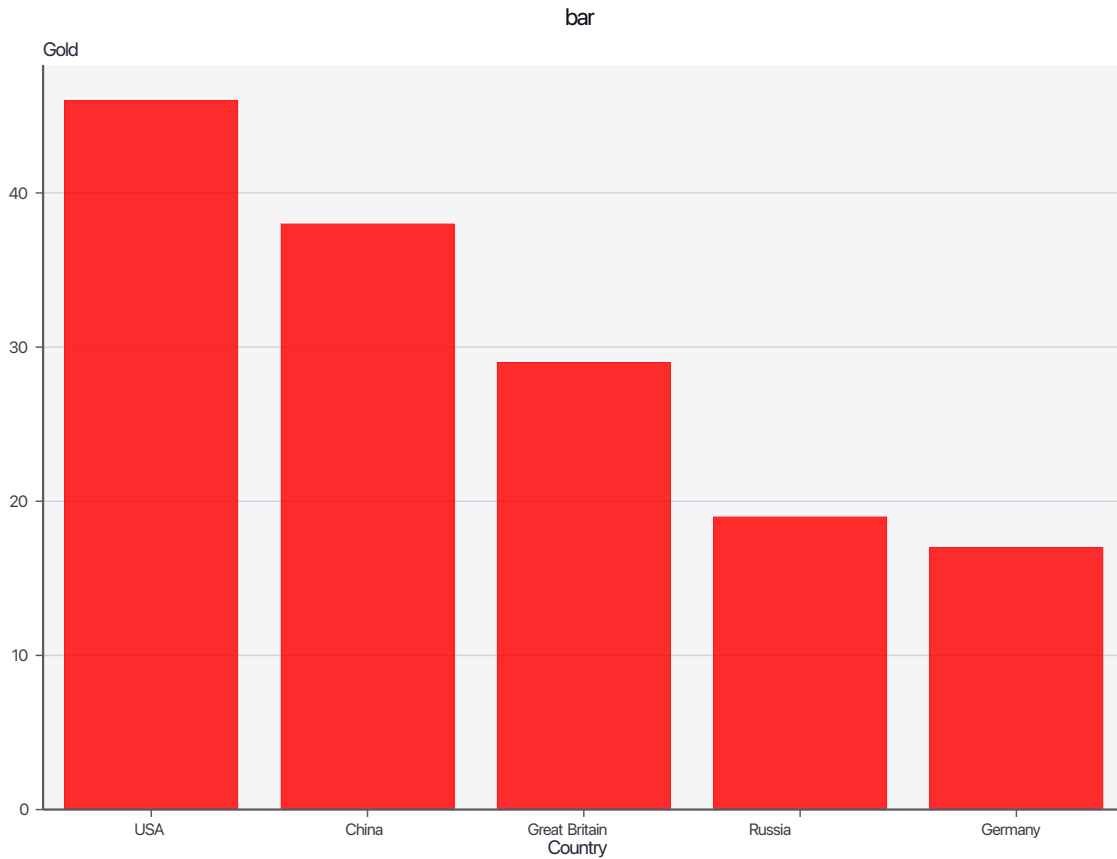
```
(data(medals) + bar + x(col.country) + y(col.gold) +
  style(color = "red") +
  title("bar"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) + style(color = "red") +
  title("bar")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),
  style({ color: "red" }), title("bar"))
```



This next sentence looks like the same request, and it is not:

```
data(gapminder_2007) + point + x(gdp) + y(life) + color("red")
```

Error:

! gog: `color("red")` binds a *value*, and a channel takes a *column*:
 ↳ `color(red)` maps the column called `red`.

To fix one value for the whole layer instead - no legend, nothing to decode
↪ - that is a setting: ``style(color = "red")``.

`color()` names a **column**, so `color("red")` asks for a column called `red`. Setting a value and mapping a column are different requests, and `ggg` makes you say which one you mean.

24.2.2 Several at once

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  style(color = "#2a6f97", opacity = 0.45, size = 7, shape = "diamond")
```

Python

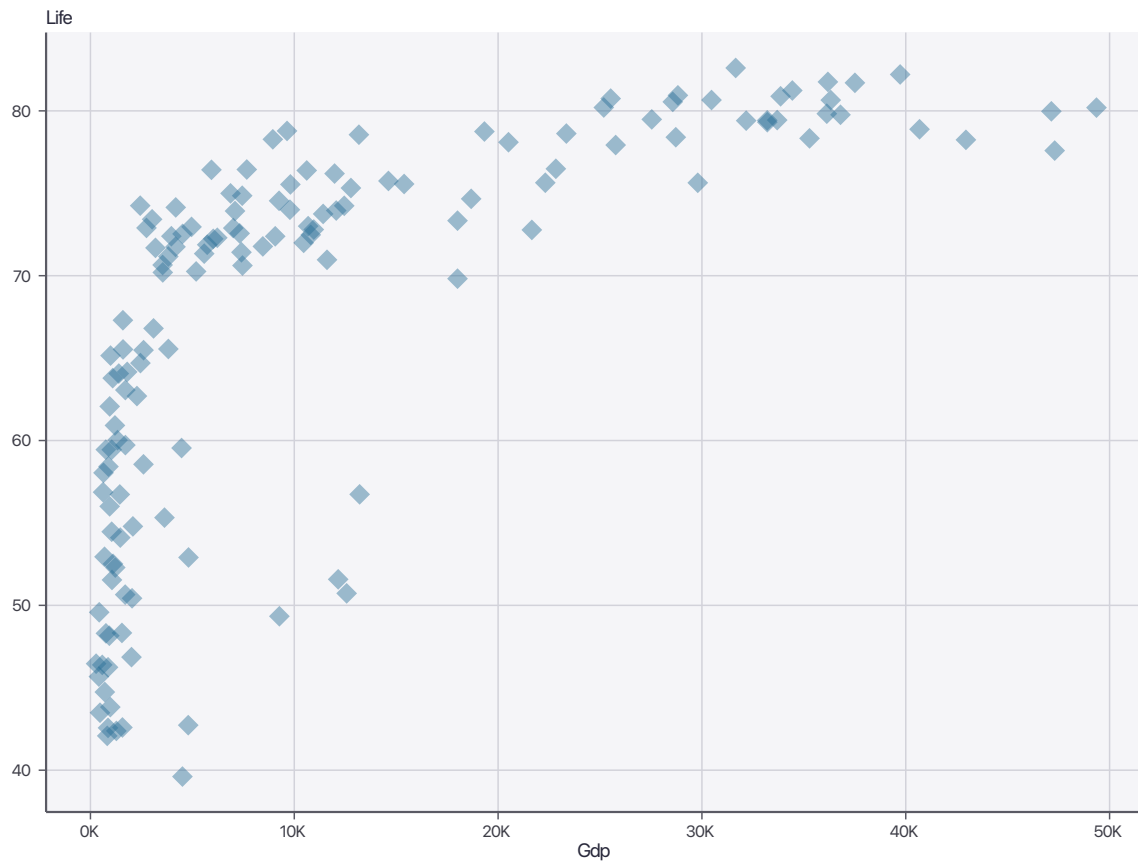
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  style(color = "#2a6f97", opacity = 0.45, size = 7, shape = "diamond"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  style(color = "#2a6f97", opacity = 0.45, size = 7, shape = "diamond")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  style({ color: "#2a6f97", opacity: 0.45, size: 7, shape: "diamond" })))
```



“Given gapminder 2007: points, x is gdp, y is life, colored #2a6f97, with opacity 0.45, size 7 and shape diamond.”

One `style()` call carries as many settings as the mark can take, and every row in the layer is drawn with all of them. The four here are independent: remove any one and the other three are unchanged. Each mark’s chapter lists what you can set on it, under [What you can set](#).

24.2.3 Colors are CSS colors

A CSS name and a hex value both work anywhere a color is accepted, in `style()` and in `palette()`:

R

```
data(medals) + bar + x(country) + y(gold) +
  style(color = "darkslateblue") +
  title("A CSS color name")
```

Python

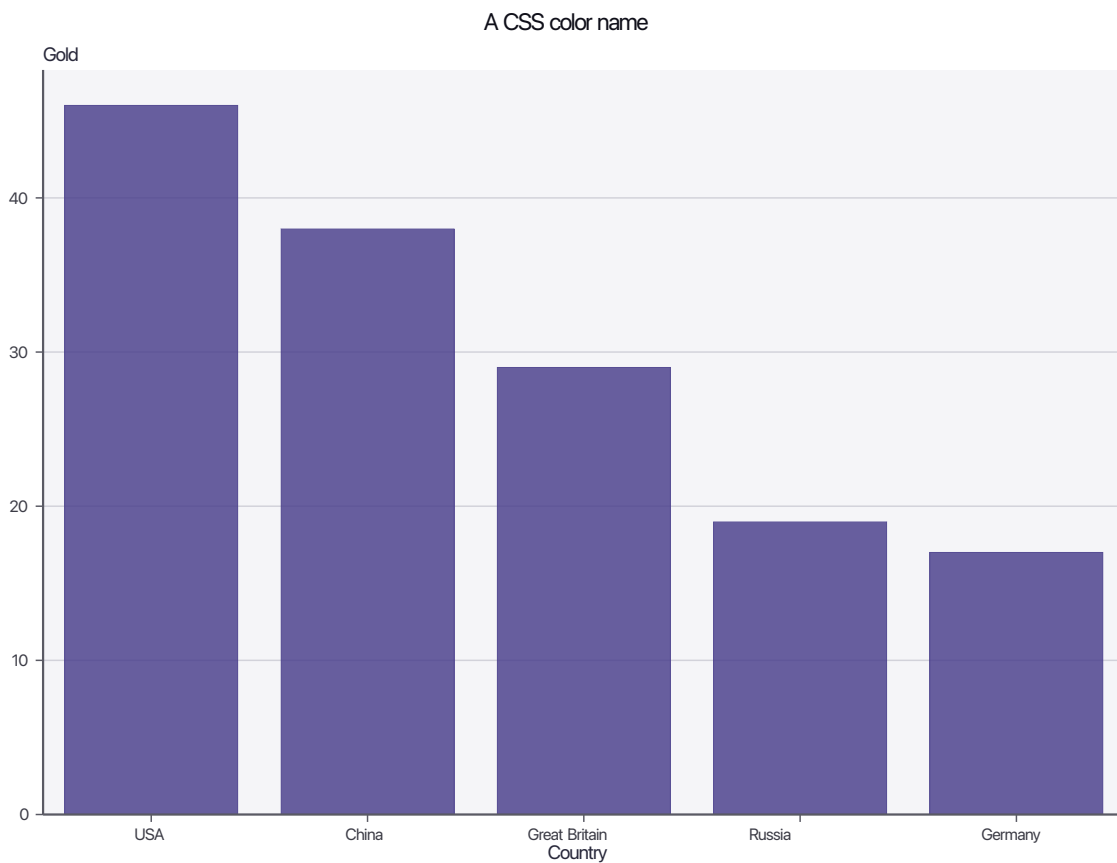
```
(data(medals) + bar + x(col.country) + y(col.gold) +  
  style(color = "darkslateblue") +  
  title("A CSS color name"))
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) +  
  style(color = "darkslateblue") + title("A CSS color name")
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),  
  style({ color: "darkslateblue" }), title("A CSS color name"))
```



A name `gog` does not recognize is refused, and the closest valid name is offered, rather than being passed to the renderer and painted black:

```
data(medals) + bar + x(country) + y(gold) + style(color = "stelblue")
```

Error:

```
! gog: `style(color = "stelblue")` is not a color. Did you mean "steelblue"?  
  ↳ Use a CSS color name, or a hex value like "#4e79a7".  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

R users often assume R color names are CSS color names. They are not. R's `colors()` includes numbered shades (`gray80`, `gray50`, `steelblue3`) that CSS does not have. `gog` uses CSS colors because that is what SVG accepts and what the other three bindings use too, so the vocabulary is the same in every language:

```
data(medals) + bar + x(country) + y(gold) + style(color = "gray80")
```

Error:

```
! gog: `style(color = "gray80")` is not a color. `gray80` is an R color name.  
  ↳ gog uses CSS colors, which have no numbered shades - use "lightgray",  
  ↳ "darkgray", or "gray", or a hex value like "#cccccc" for an exact shade.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

For an exact shade, give the hex value.

24.3 What settings are for

Two jobs occur constantly, and neither is a mapping.

Overplotting. With enough points, the marks cover one another and the shape of the cloud disappears. Lowering opacity is a statement about the *rendering*, not about the data:

R

```
data(gm_all) + point + x(gdp) + y(life) +  
  style(opacity = 0.2, size = 3) +  
  title("1704 points: transparency reveals the density")
```

Python

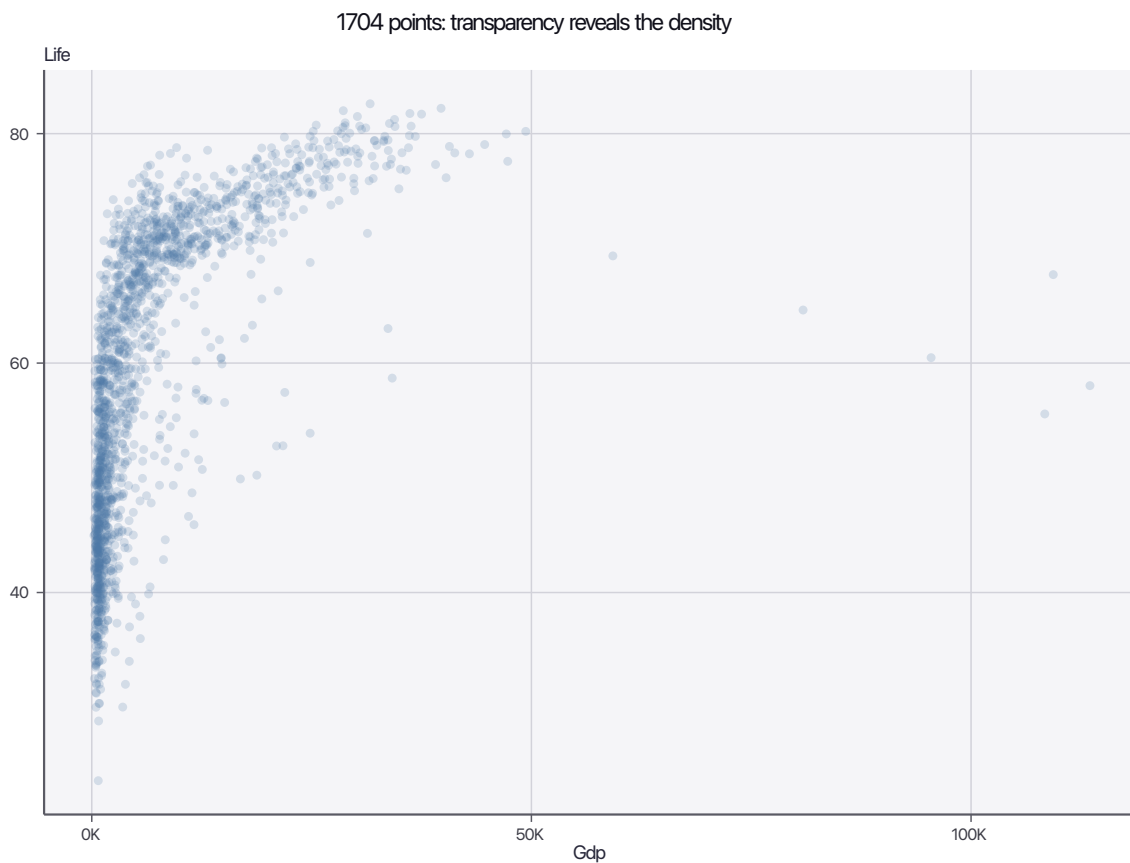
```
(data(gm_all) + point + x(col.gdp) + y(col.life) +
  style(opacity = 0.2, size = 3) +
  title("1704 points: transparency reveals the density"))
```

Julia

```
data(gm_all) + point + x(:gdp) + y(:life) +
  style(opacity = 0.2, size = 3) +
  title("1704 points: transparency reveals the density")
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp), y(col.life),
  style({ opacity: 0.2, size: 3 }),
  title("1704 points: transparency reveals the density"))
```



A reference layer. When one layer is the point of the plot, the others should be drawn faintly:

R

```
data(gapminder_asia) + x(year) + y(life) +  
  line + group(country) + style(color = "lightgray") +  
  line * smooth + style(color = "firebrick", size = 3) +  
  title("Five countries in gray; the trend in red")
```

Python

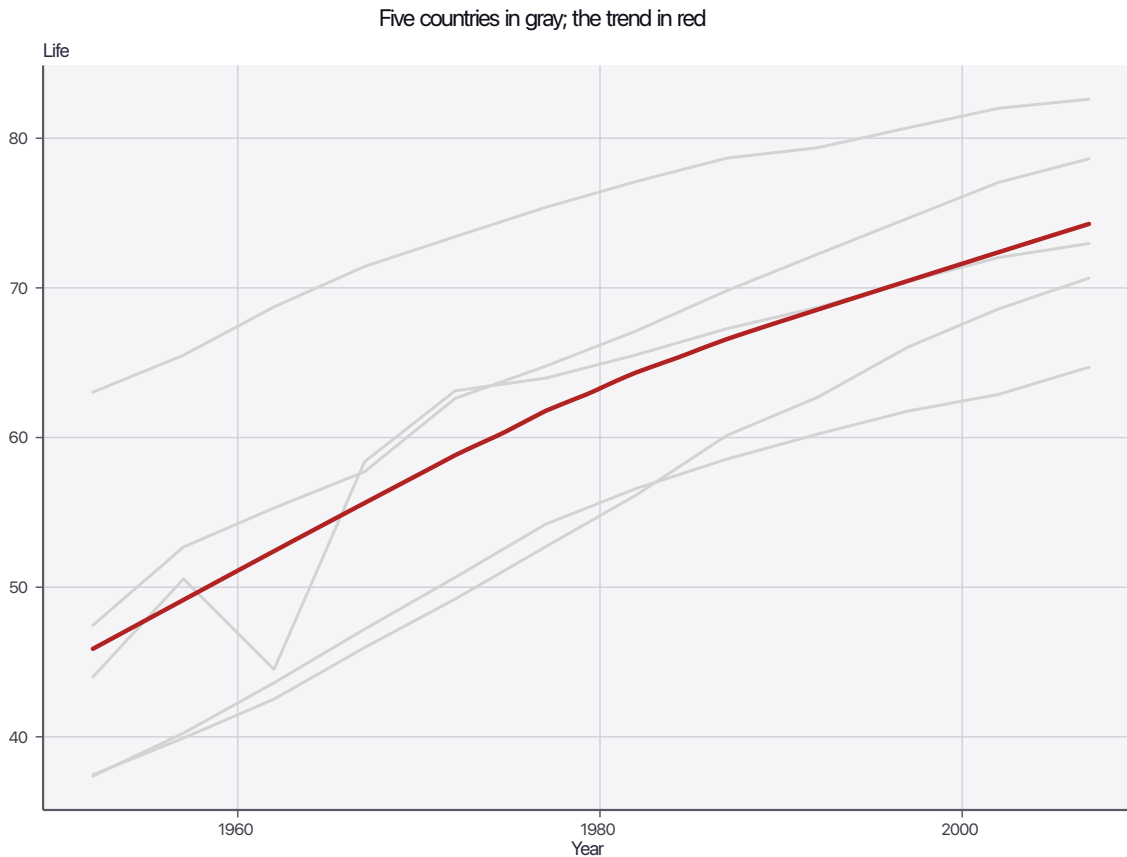
```
(data(gapminder_asia) + x(col.year) + y(col.life) +  
  line + group(col.country) + style(color = "lightgray") +  
  line * smooth + style(color = "firebrick", size = 3) +  
  title("Five countries in gray; the trend in red"))
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + line + group(:country) +  
  style(color = "lightgray") + line * smooth +  
  style(color = "firebrick", size = 3) +  
  title("Five countries in gray; the trend in red")
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), line,  
  group(col.country), style({ color: "lightgray" }), layer(line, smooth),  
  style({ color: "firebrick", size: 3 }),  
  title("Five countries in gray; the trend in red"))
```



“Given gapminder Asia: x is year, y is life, lines, grouped by country, colored lightgray, and also a line derived by smooth, colored firebrick, with size 3.”

24.4 line takes settings it refuses as channels

A polyline is drawn with *one* stroke. Nothing varies from row to row along that stroke, so **size** and **opacity** are not channels on line:

```
data(gapminder_asia) + x(year) + y(life) + line + group(country) +
  ↪ opacity(life)
```

Error:

```
! gog: `opacity` cannot be bound to `line` - a line has no opacity feature.
  ↪ Remove the `life` mapping from `opacity`, or use a mark that has one.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

But one stroke still *has* a width and an opacity. Those are settings:

R

```
data(gapminder_asia) + line + x(year) + y(life) + group(country) +  
  style(size = 4, opacity = 0.5) +  
  title("Set on a line: one width, one opacity")
```

Python

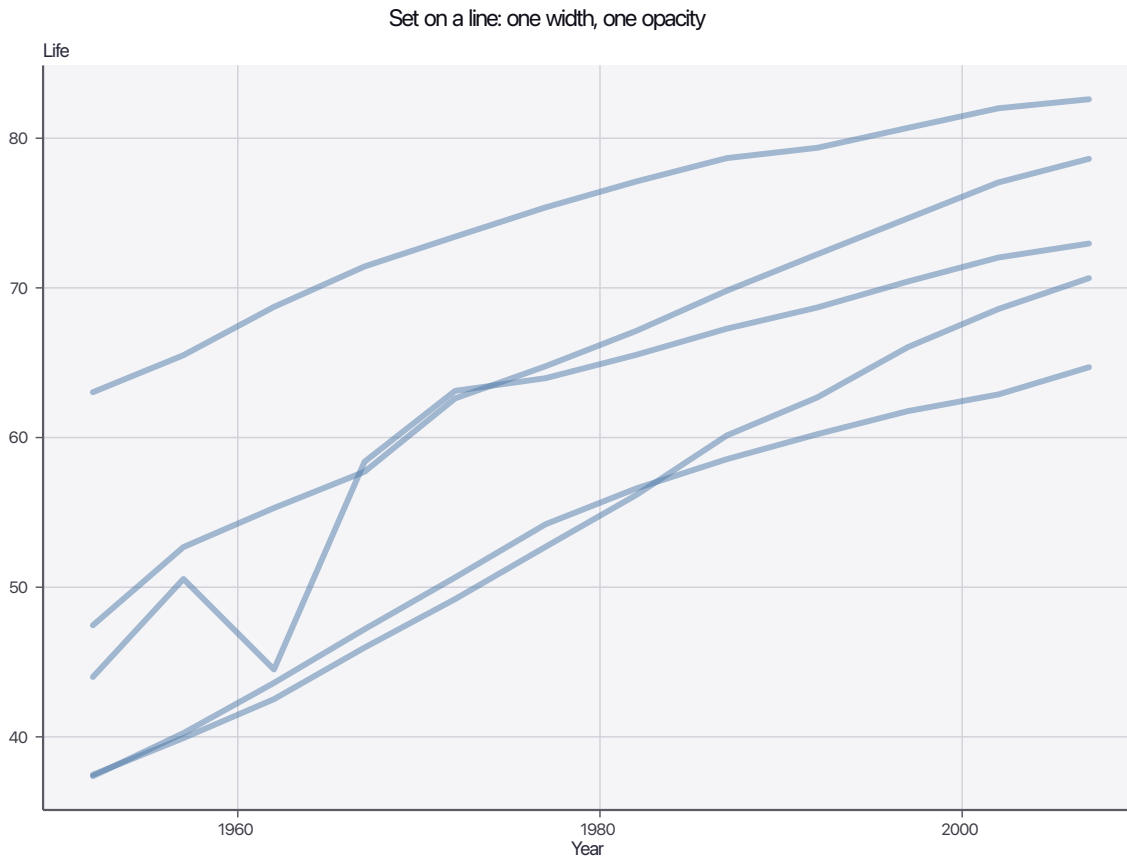
```
(data(gapminder_asia) + line + x(col.year) + y(col.life) + group(col.country)  
↪ +  
  style(size = 4, opacity = 0.5) +  
  title("Set on a line: one width, one opacity"))
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country) +  
  style(size = 4, opacity = 0.5) +  
  title("Set on a line: one width, one opacity")
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
  group(col.country), style({ size: 4, opacity: 0.5 }),  
  title("Set on a line: one width, one opacity"))
```



This is where the distinction matters most. “Can a column be mapped here?” and “Does this mark have this property?” are different questions, and `line` is where the answers differ.

24.5 You cannot map and set the same property

Doing both would mean one of the two instructions is silently discarded, which `gog` never does:

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + style(color = "tomato")
```

Error:

```
! gog: `color(continent)` maps color and `style(color = "tomato")` sets it -
  ↳ one layer cannot do both. Keep `color(continent)` to show the data, or
  ↳ `style(color = "tomato")` to fix every point at one value.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

Pick the one you meant.

24.6 style applies to one layer only

`color()` written before any mark applies to every layer that can take it. `style()` attaches to the nearest mark before it and goes no further, because a decoration written for one layer should not change another:

R

```
data(gapminder_2007) + x(gdp) + y(life) +  
  line * smooth +  
  point + style(color = "tomato", opacity = 0.4) +  
  title("Only the points are tomato")
```

Python

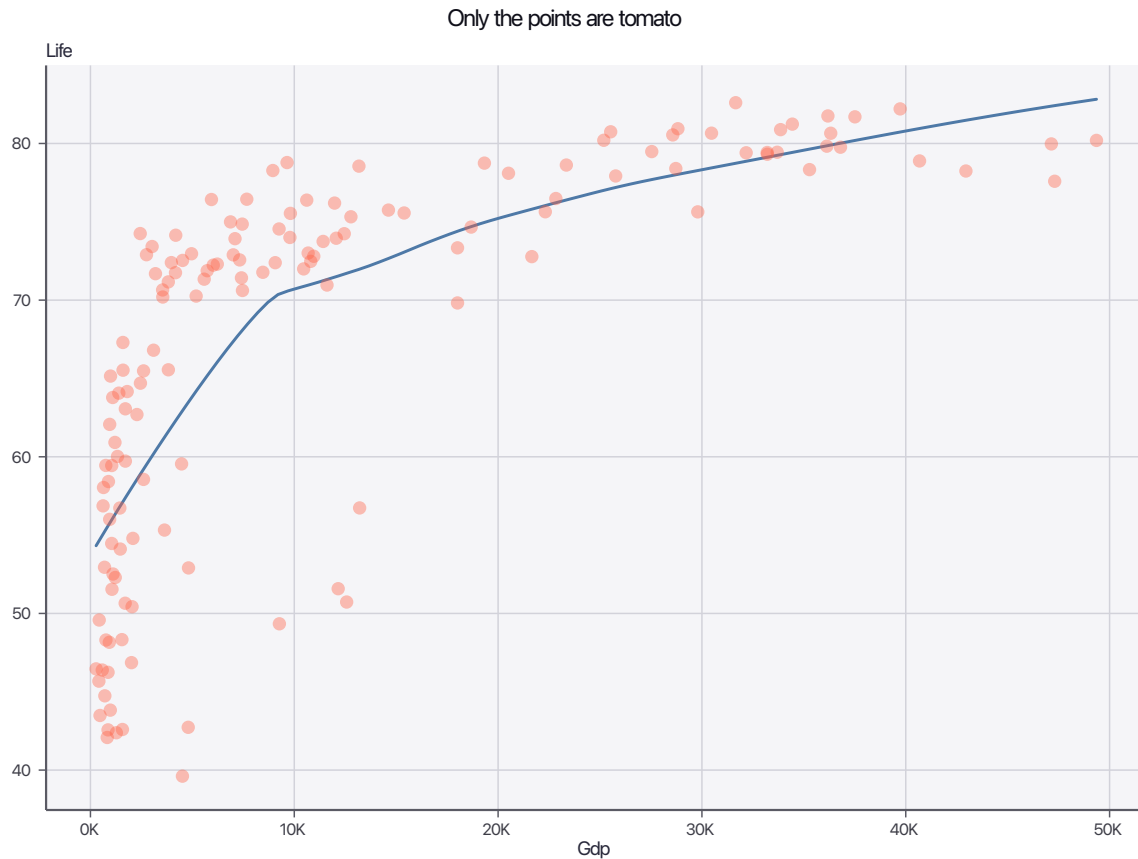
```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +  
  line * smooth +  
  point + style(color = "tomato", opacity = 0.4) +  
  title("Only the points are tomato"))
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + line * smooth + point +  
  style(color = "tomato", opacity = 0.4) +  
  title("Only the points are tomato")
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), layer(line, smooth),  
  point, style({ color: "tomato", opacity: 0.4 } ),  
  title("Only the points are tomato"))
```



To style both layers, say so on both.

24.7 palette: choosing the colors a mapping uses

`style(color = )` is written instead of a mapping. `palette()` keeps the mapping and chooses which colors the categories receive:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent) +  
  palette(c("tomato", "steelblue", "seagreen", "goldenrod", "orchid"))
```

Python

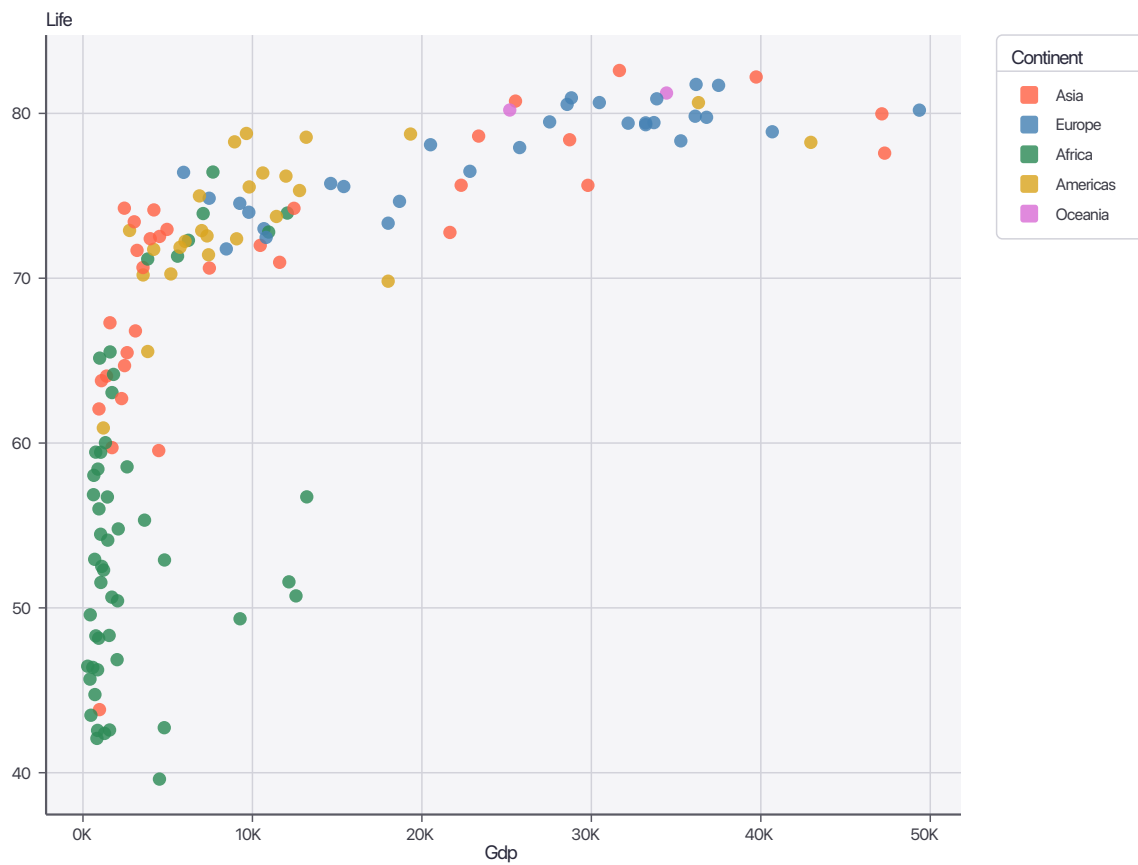

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  ↪ color(col.continent) +
  palette(["tomato", "steelblue", "seagreen", "goldenrod", "orchid"]))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +
  palette(["tomato", "steelblue", "seagreen", "goldenrod", "orchid"])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  color(col.continent),
  palette(["tomato", "steelblue", "seagreen", "goldenrod", "orchid"]))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent, with a palette of tomato, steelblue, seagreen, goldenrod and orchid.”

A palette needs a mapping to give its colors to. With no column bound to `color` it has nothing to color, so it is refused rather than accepted and then ignored:

```
data(gapminder_2007) + point + x(gdp) + y(life) + palette("okabe")
```

Error:

```
! gog: `palette()` chooses the colors a `color` mapping hands out, and
  ↳ nothing here maps `color`, so it would have no effect. Bind a column with
  ↳ `color(<column>)` to use it - or, to paint every mark one color, that is
  ↳ a setting rather than a palette: `style(color = "...")`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The message names two repairs, because there are two things you may have meant. To give each category its own color, bind the column with `color(continent)`. To paint every mark one color, that is a setting rather than a palette, and `style(color = "steelblue")` is where it belongs.

A mapping need not be written to be there. A `zone * density` colors its cells by a measure it computes itself ([Zone](#)), so `palette()` works on that plot.

One color, though, is not a palette. `palette()` takes either a **palette name** ("gog", "okabe") or the **colors themselves**; a single color name is neither, so it is refused rather than quietly using the default:

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent) +
  palette("red")
```

Error:

```
! gog: `palette("red")` is not a known palette. Named palettes are gog,
  ↳ okabe, soft for categories; blue, viridis, magma, inferno, plasma,
  ↳ cividis, gray for numbers; blue_red, brown_tea for numbers that diverge
  ↳ from a center. `"red"` is a color, not a palette. To paint every mark one
  ↳ color use `style(color = "red")`; to give each category its own color
  ↳ pass a vector: `palette(c("red", ...))`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Making everything one color is *setting*, not choosing a palette, so use `style(color = "red")`.

24.8 theme: settings that belong to no mark

`style()` sets a mark's own appearance. Some settings belong to no mark at all: the gridlines, the shape of the panel, the angle the tick labels are read at. Those belong to the whole plot rather than to any mark, and they have their own word.

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(continent) + theme(grid = "none")
```

Python

```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.continent) + theme(grid = "none"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + theme(grid = "none")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.continent), theme({ grid: "none" }))
```



“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by continent.”

`grid` names its sets by the axis whose ticks they mark, not by the direction they run: `"x"` keeps the lines standing up from the x axis and drops the rest. Naming them that way is what lets the setting survive a bent plane, where the y axis's gridlines are rings and nothing runs vertically at all:

R

```
data(winds) + bar * proportion * stack + color(season) + polar() +
  theme(grid = "x")
```

Python

```
(data(winds) + bar * proportion * stack + color(col.season) + polar() +
  theme(grid = "x"))
```

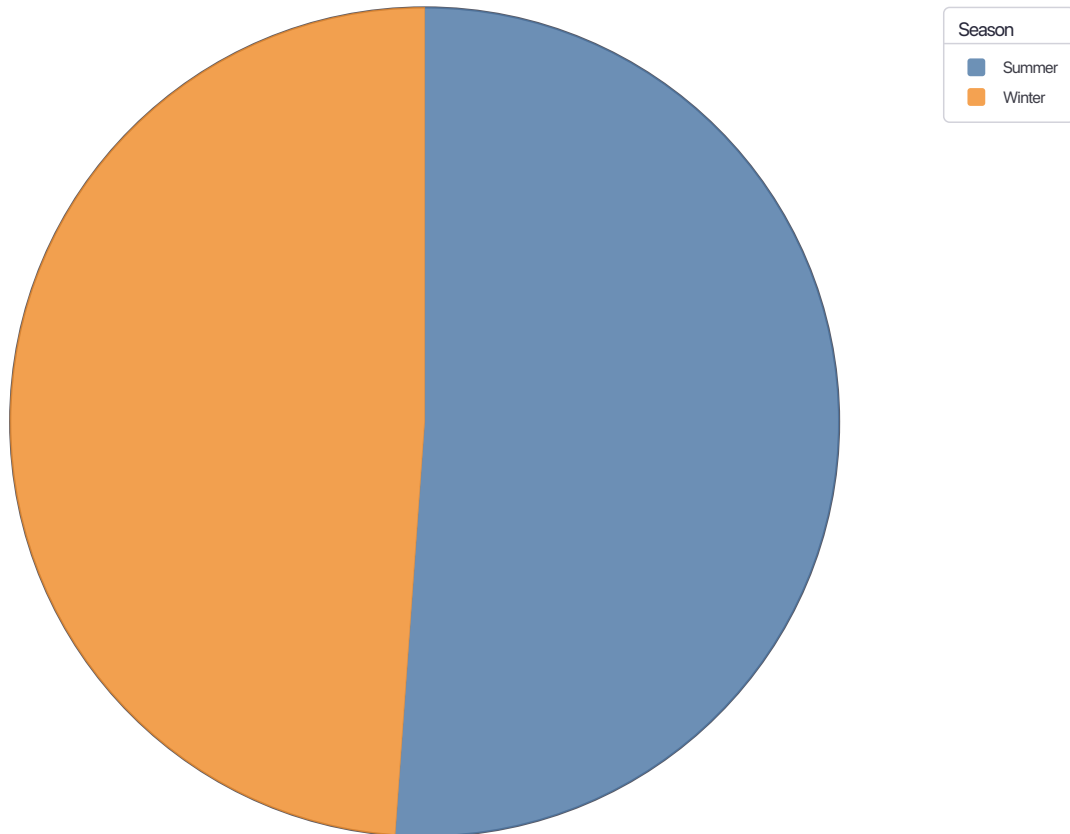
Julia

```
data(winds) + bar * proportion * stack + color(:season) + polar() +  
  theme(grid = "x")
```

JavaScript

```
plot(data(winds), layer(bar, proportion, stack), color(col.season),  
  polar(), theme({ grid: "x" })))
```

Proportion



“Given the winds: bars derived by proportion and stack, color by season, in polar.”

`ratio` sets the panel’s width divided by its height, which is how a circle comes out round. The image keeps the size it was given and the panel shrinks inside it, so the plot still takes the same room on a page:

R

```
data(winds) + bar * count + x(direction) + polar() + theme(ratio = 1)
```

Python

```
data(winds) + bar * count + x(col.direction) + polar() + theme(ratio = 1)
```

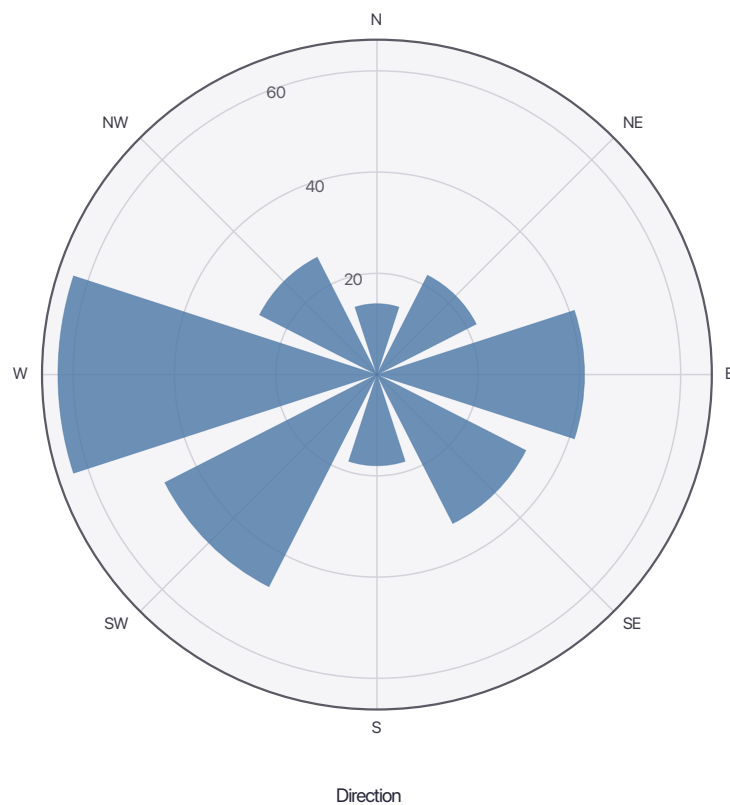
Julia

```
data(winds) + bar * count + x(:direction) + polar() + theme(ratio = 1)
```

JavaScript

```
plot(data(winds), layer(bar, count), x(col.direction), polar(),  
  theme({ ratio: 1 })))
```

Count



“Given the winds: bars derived by count, x is direction, in polar.”

And `tick_angle` turns the x labels, which is the answer to category names that overlap. The bottom margin grows to hold them:

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life) +  
  theme(tick_angle = 45)
```

Python

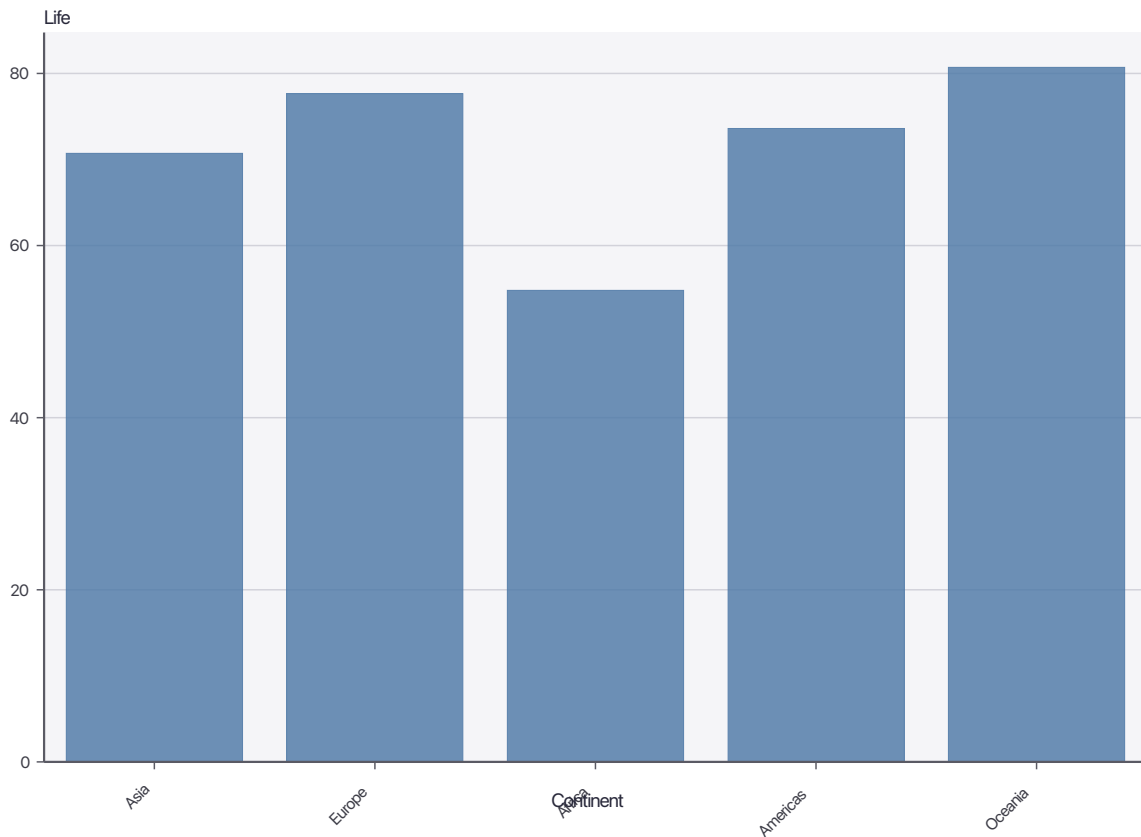
```
(data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life) +  
  theme(tick_angle = 45))
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life) +  
  theme(tick_angle = 45)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),  
  y(col.life), theme({ tick_angle: 45 })))
```



`font_size` sets how big the plot's text is. It is **one number, not three**. The number itself is the size of a tick label, in pixels. The axis names and the title are a fixed step larger. The default of 11 draws ticks at 11, axis names at 13 and the title at 16. Ask for 16 and those three become 16, 19 and 23:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +
  title("Bigger text, one number") + theme(font_size = 16)
```

Python

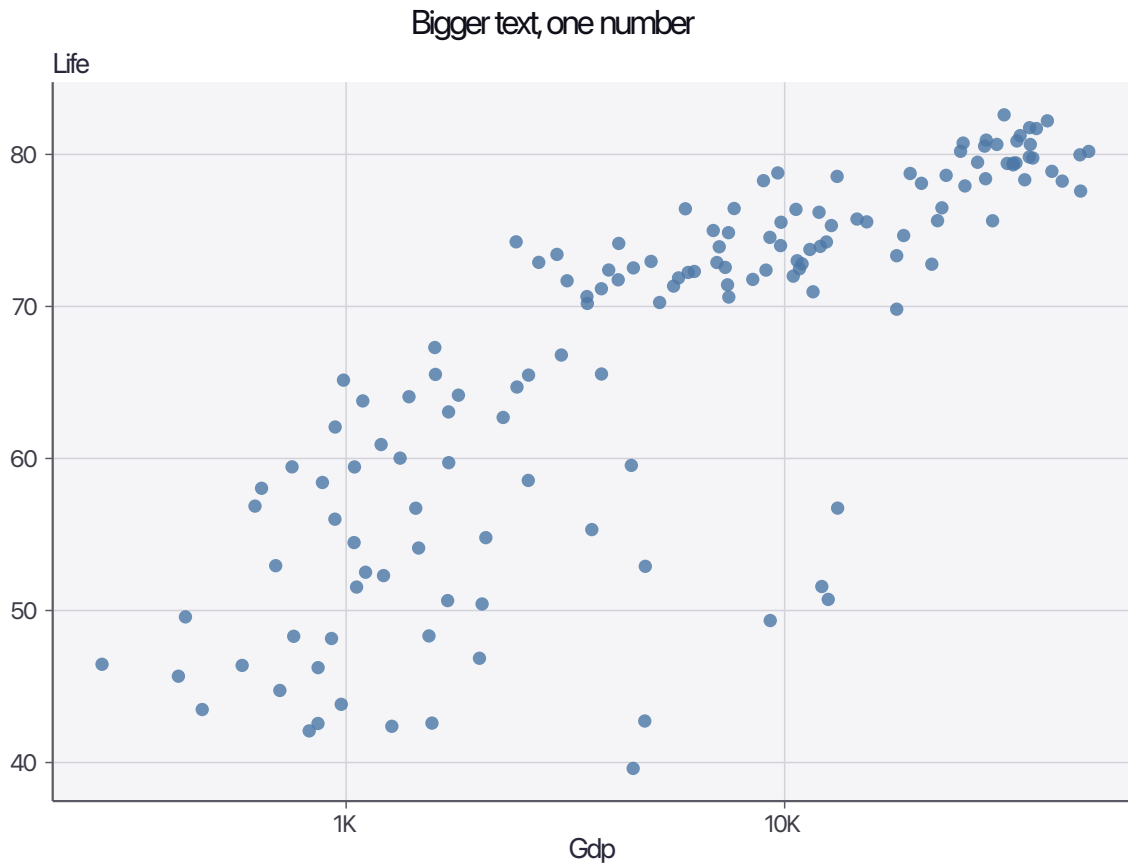
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +
  title("Bigger text, one number") + theme(font_size = 16))
```

Julia


```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +
  title("Bigger text, one number") + theme(font_size = 16)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),
  y(col.life), title("Bigger text, one number"), theme({ font_size: 16 }))
```



Look at the margins. They grew with the text, because the room an axis needs is measured from the labels that go in it. Text and the space it sits in are one decision, so larger text never covers the plot.

A smaller number keeps a plot readable when it has to be printed small:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  title("Smaller text, same plot") + theme(font_size = 8)
```

Python

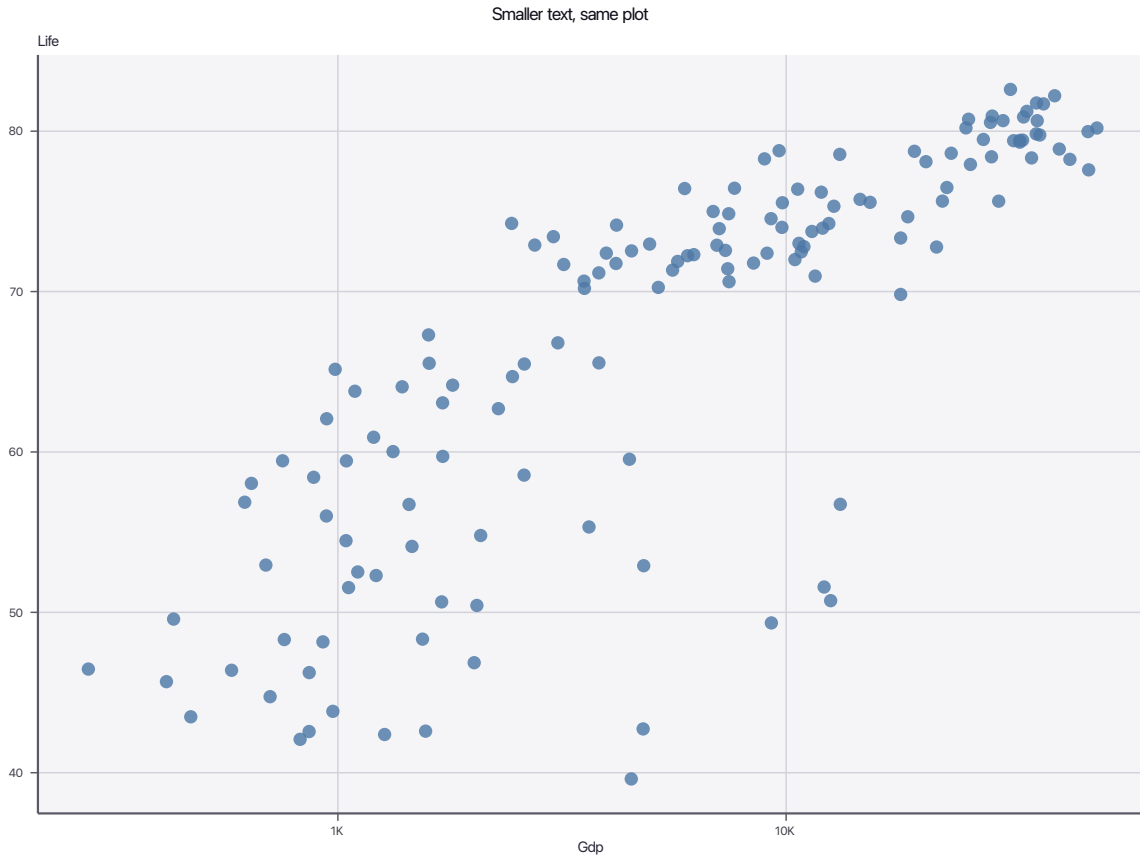
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  title("Smaller text, same plot") + theme(font_size = 8))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  title("Smaller text, same plot") + theme(font_size = 8)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), title("Smaller text, same plot"), theme({ font_size: 8 }))
```



One number covers all the text because the three sizes were never independent. If the three sizes were set separately, one plot could carry three unrelated text sizes. Asking for a big title alone is not possible here, and that is the design rather than a gap in it.

The number is a **measurement in pixels, not a multiplier**. `font_size = 1.5` is refused, because a number that small is almost always meant as a multiplier:

```
data(gapminder_2007) + point + x(gdp) + y(life) + theme(font_size = 1.5)
```

Error:

```
! gog: `theme(font_size = )` is how many pixels a tick label is, not a
  ↪ multiplier, so it needs one number of at least 4. The default is 11, and
  ↪ the axis names and the title are derived from it.
```

`font_size` does not name a typeface. The engine measures text with its own table of character widths, which is what lets it lay out a plot without a font installed, so there is no font for it to choose. The computer that opens the plot supplies the letters.

A theme can also be asked for by name. A named bundle of these properties is a **preset**, and you can still adjust it. A preset you could not adjust would send you back to setting every property one at a time:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  theme("minimal", ratio = 1)
```

Python

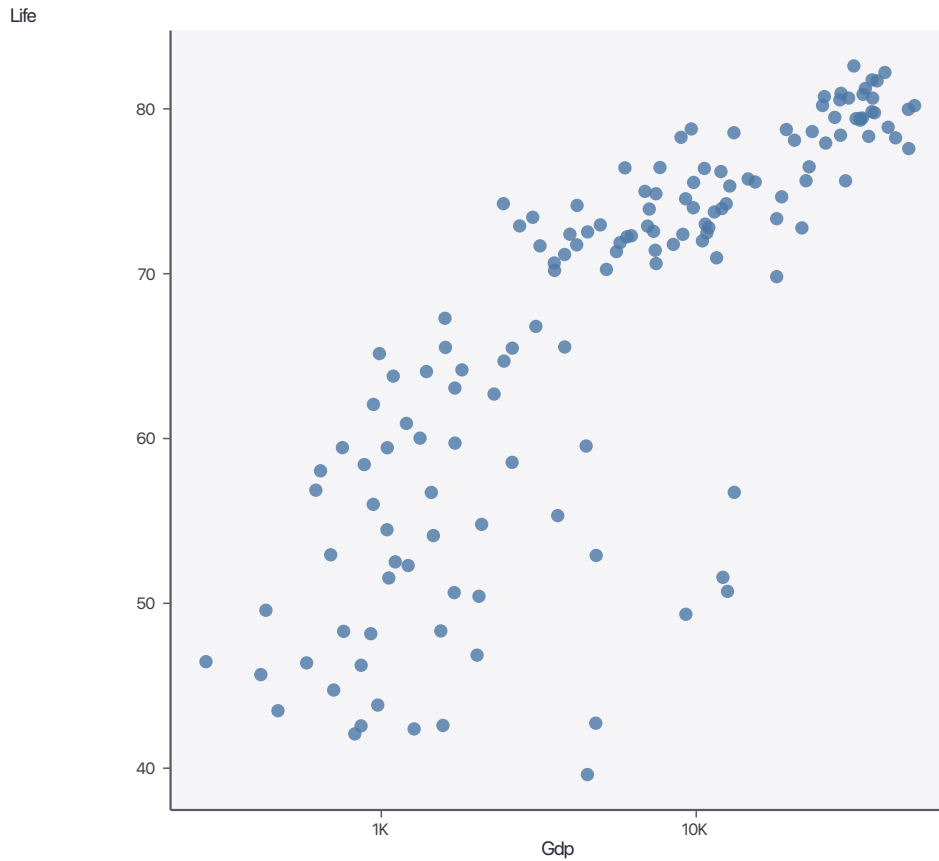
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  theme("minimal", ratio = 1))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  theme("minimal", ratio = 1)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), theme("minimal", { ratio: 1 }))
```



24.9 The journal look

`theme("bw")` is the one most often wanted for publication: a white panel inside a full rectangle.

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +
  color(continent) + theme("bw")
```

Python

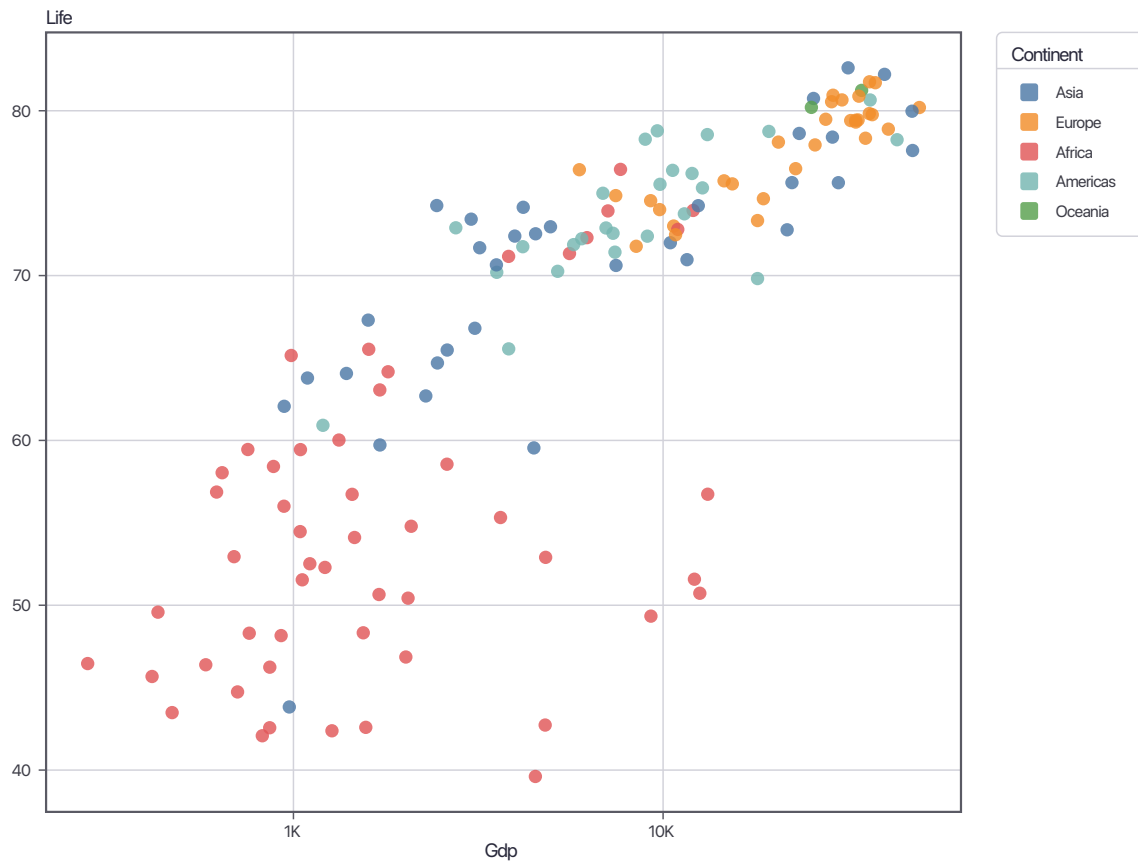
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +
  color(col.continent) + theme("bw"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + theme("bw")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.continent), theme("bw"))
```



The name is the misleading part, and it is worth being exact about it. What goes black and white is the page around the data, never the data: the continents above keep every color the palette gave them. In a “bw” plot only the data carries color; everything around it is black, white or gray.

Three settings carry the whole preset. This sentence draws the same picture as `theme("bw")` above:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(continent) + theme(background = "white", frame = "full", strip =  
  ↪ "white")
```

Python

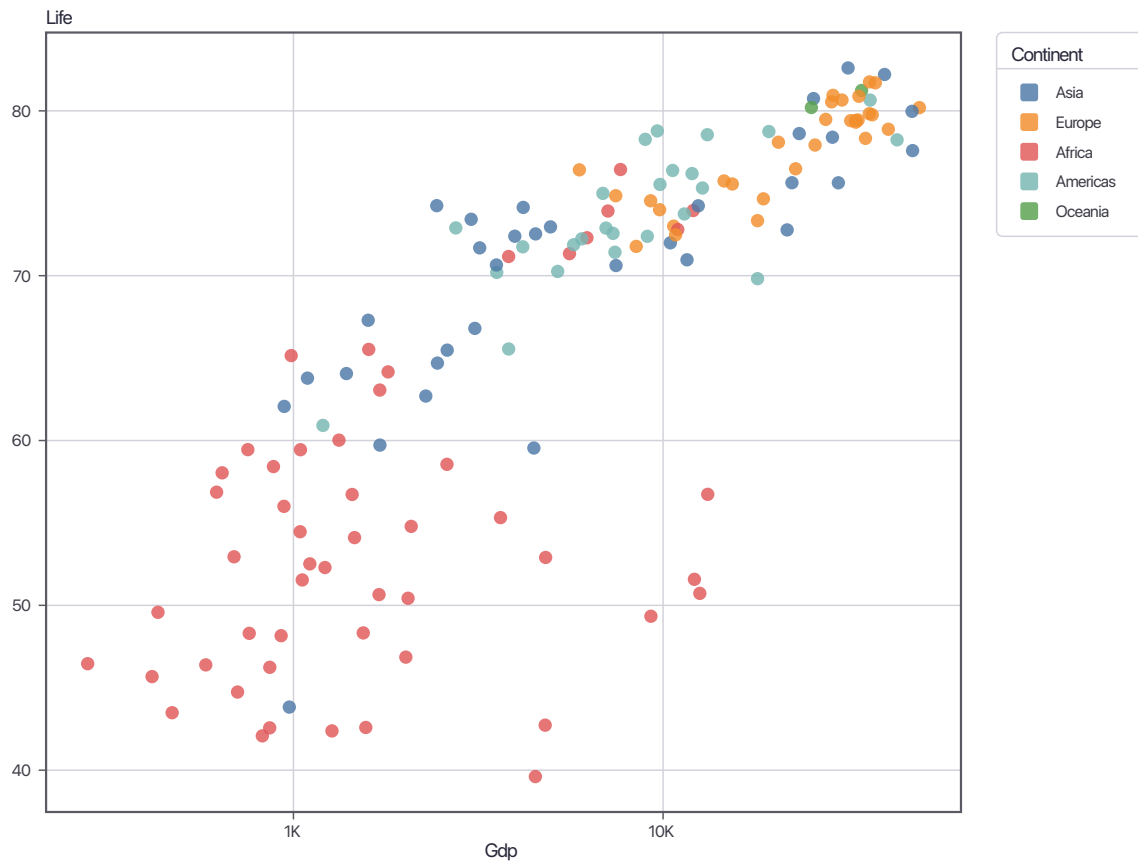
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.continent) + theme(background = "white", frame = "full", strip =  
  ↪ "white"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) +  
  theme(background = "white", frame = "full", strip = "white")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.continent),  
  theme({ background: "white", frame: "full", strip: "white" }))
```



That is the rule every preset in `gog` follows: a preset only names settings you could have written yourself. One that went further would be a second vocabulary inside the first.

24.10 The strip, and how it prints

The third property is `strip`. It colors the band above each panel, the band that names the category the panel holds. You only see it on a faceted plot, and its color matters more in print than on screen.

By default the band is a light gray. On a screen that is fine, and it ties each name to its own panel:

R

```
(data(gapminder_2007) + point + x(gdp, tick_count = 3) + y(life)) |
  facet(continent)
```


Python

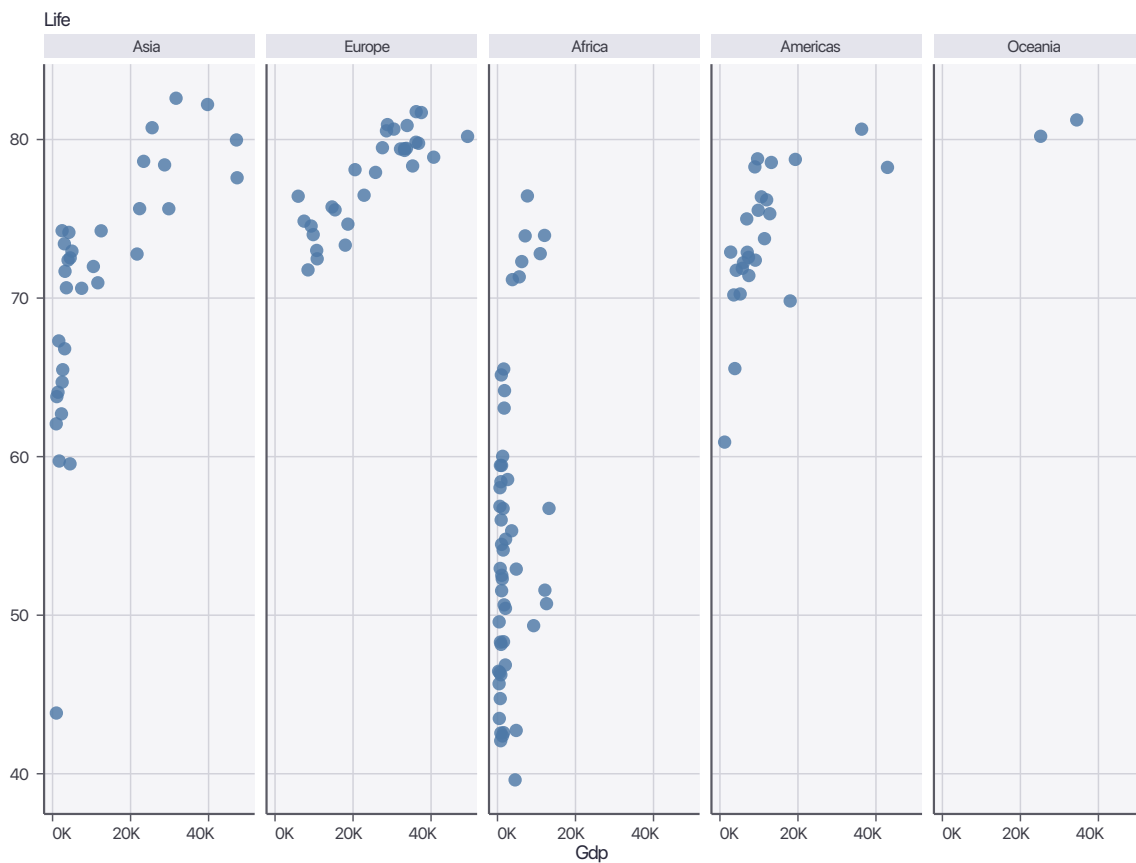
```
((data(gapminder_2007) + point + x(col.gdp, tick_count = 3) + y(col.life)) |  
  facet(col.continent))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp, tick_count = 3) + y(:life)) |  
  facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { tick_count: 3 }),  
     y(col.life), across(col.continent))
```



“Given gapminder 2007: points, x is gdp with 3 ticks, y is life, split into panel columns by continent.”

On paper it is not fine. A light gray is printed as a pattern of tiny dots, and that pattern sits directly under small text. The label gets harder to read, and a photocopy makes it worse. This is the same reason `pattern` exists for fills: what survives on a screen and what survives on a page are different questions.

So `theme("bw")` makes the band white as well. The framed panel already separates one panel from the next, so the tint has nothing left to do:

R

```
(data(gapminder_2007) + point + x(gdp, tick_count = 3) + y(life) +  
  ↪ theme("bw")) |  
  facet(continent)
```

Python

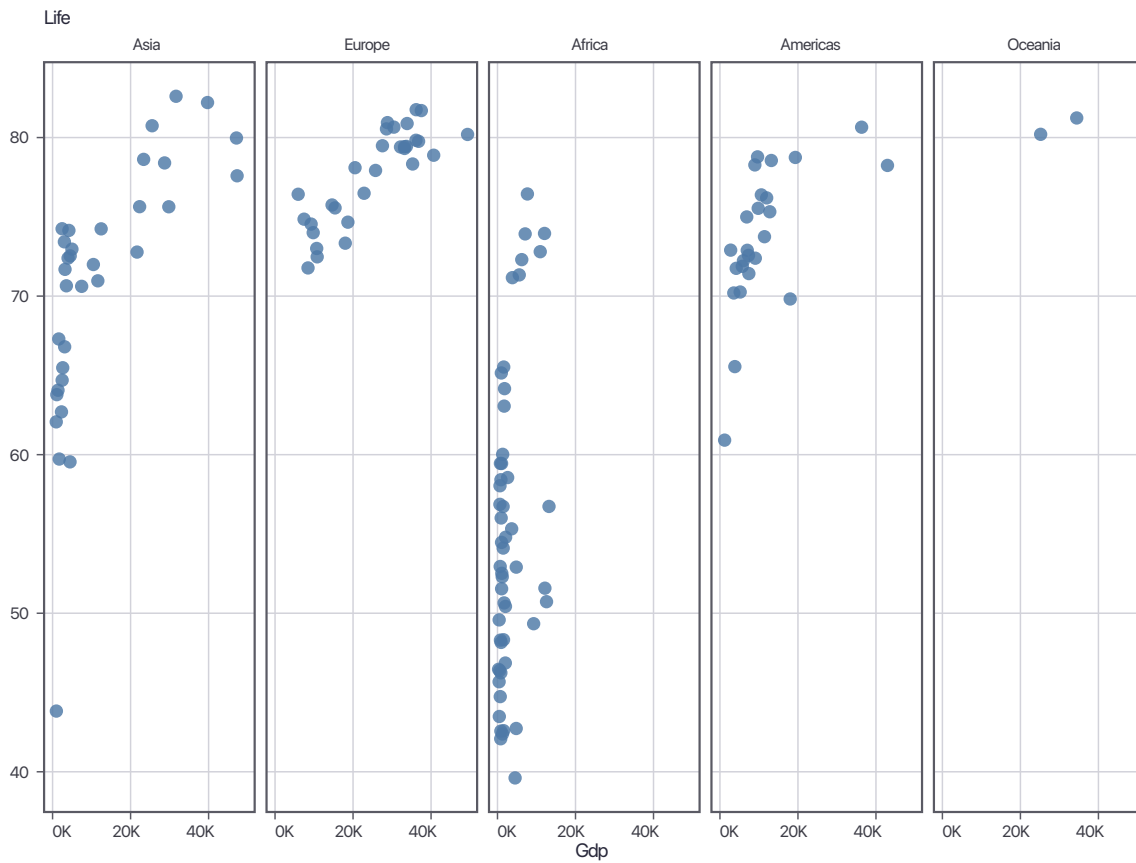
```
((data(gapminder_2007) + point + x(col.gdp, tick_count = 3) + y(col.life) +  
  ↪ theme("bw")) |  
  facet(col.continent))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp, tick_count = 3) + y(:life) +  
  theme("bw")) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { tick_count: 3 }),  
  y(col.life), theme("bw"), across(col.continent))
```



You can also name the color yourself, for a poster or a slide where a tint helps rather than hurts:

R

```
(data(gapminder_2007) + point + x(gdp, tick_count = 3) + y(life) +
  theme(strip = "#dce8f0", frame = "full")) |
facet(continent)
```

Python

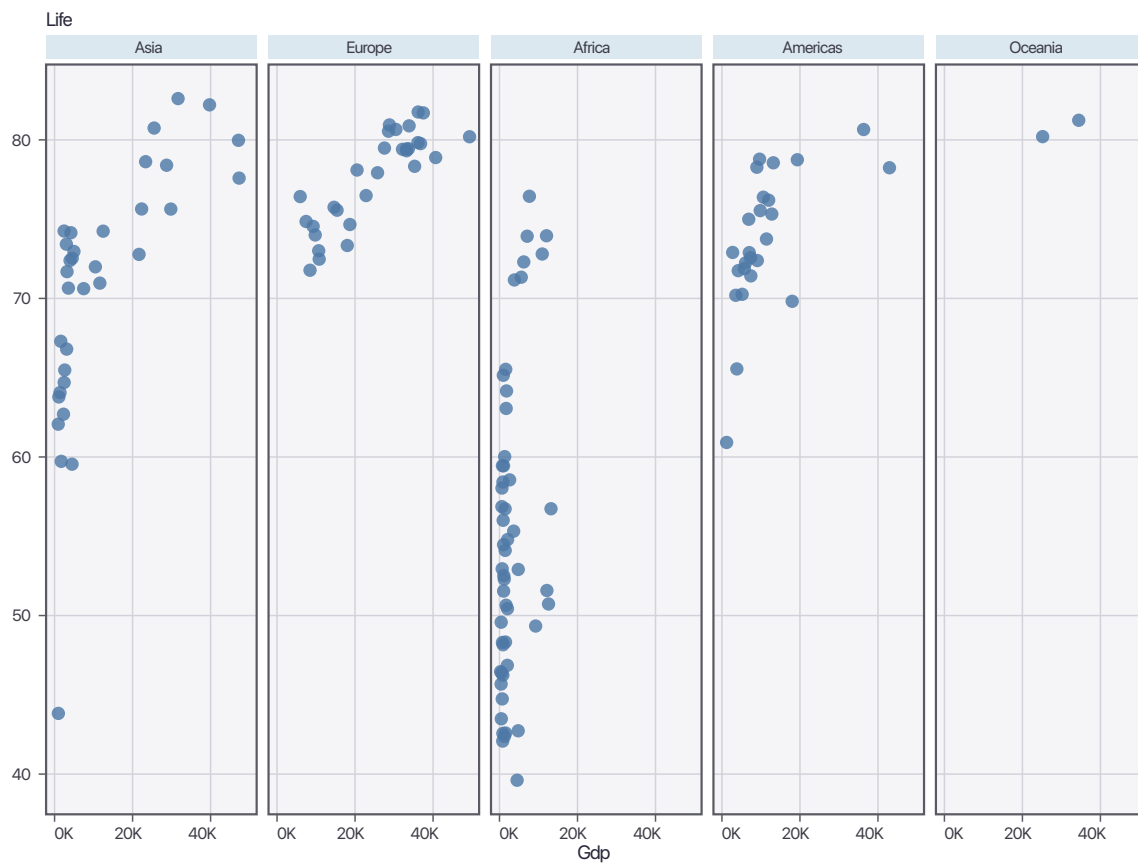
```
((data(gapminder_2007) + point + x(col.gdp, tick_count = 3) + y(col.life) +
  theme(strip = "#dce8f0", frame = "full")) |
facet(col.continent))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp, tick_count = 3) + y(:life) +
  theme(strip = "#dce8f0", frame = "full")) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { tick_count: 3 }),
  y(col.life), theme({ strip: "#dce8f0", frame: "full" }),
  across(col.continent))
```



An animated plot ([Play](#), later in the book) gets the same band, because it names the value it is showing the same way. `theme(strip = )` colors that one too. A property that moved one and left the other would be exactly the kind of exception [Law 2](#) forbids.

24.10.1 The label color follows the band

A dark strip with light text is a common look, and it takes one property:

R

```
(data(gapminder_2007) + point + x(gdp, tick_count = 3) + y(life) +  
  theme(strip = "#22223b", frame = "full")) |  
  facet(continent)
```

Python

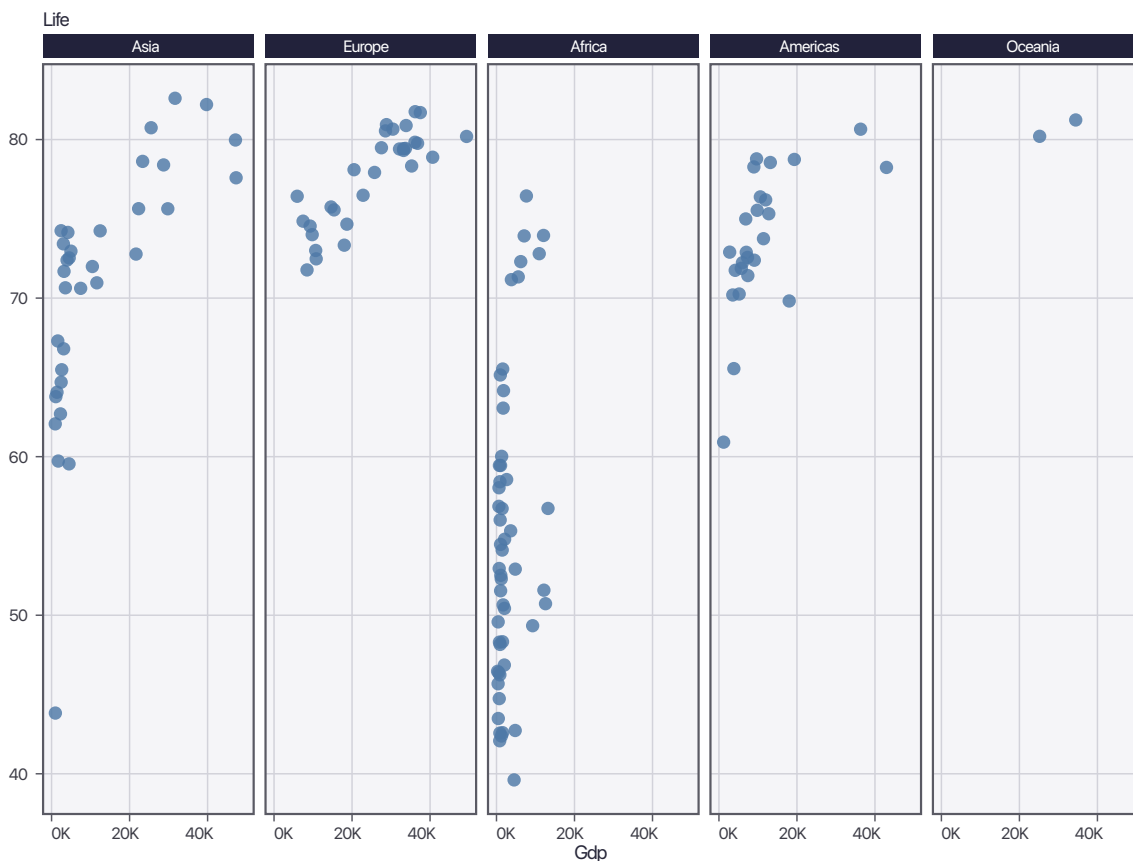
```
((data(gapminder_2007) + point + x(col.gdp, tick_count = 3) + y(col.life) +  
  theme(strip = "#22223b", frame = "full")) |  
  facet(col.continent))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp, tick_count = 3) + y(:life) +  
  theme(strip = "#22223b", frame = "full")) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { tick_count: 3 }),  
  y(col.life), theme({ strip: "#22223b", frame: "full" }),  
  across(col.continent))
```



You did not name a text color, and the text came out white. `gog` reads the band you asked for and picks whichever of its two default label colors contrasts more against it. A pale band keeps the dark label color, so the plots earlier in this section are unchanged.

`gog` works the label color out; it does not guess to save you typing. Without that rule, `theme(strip = "black")` would paint the near-black default label onto a near-black band. Every panel name would be unreadable while the plot still looked finished. `gog` will not draw a panel name that cannot be read, so it supplies the label color you did not write.

When the label color is a real choice rather than a legibility question, name it. A navy band with gold text is legible, so `gog` uses the color you name:

R

```
(data(gapminder_2007) + point + x(gdp, tick_count = 3) + y(life) +
  theme(strip = "navy", strip_text = "gold", frame = "full")) |
  facet(continent)
```

Python

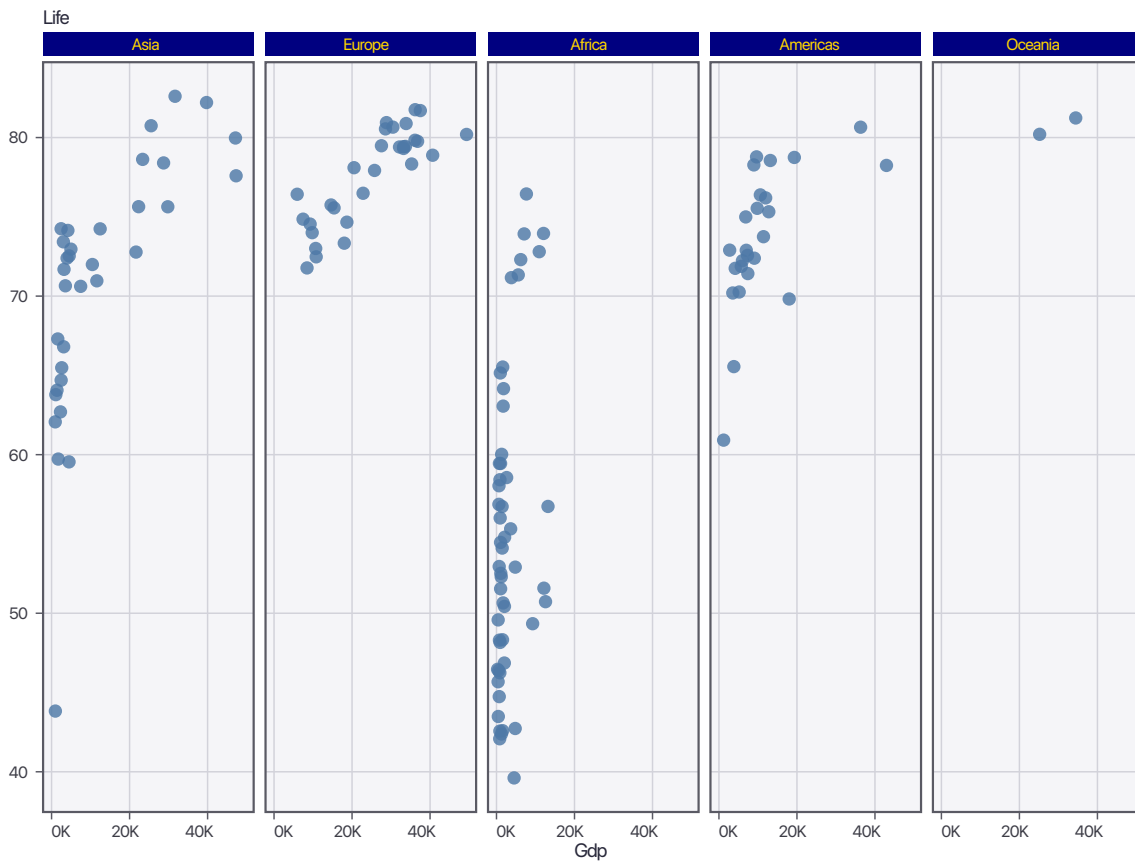
```
((data(gapminder_2007) + point + x(col.gdp, tick_count = 3) + y(col.life) +  
  theme(strip = "navy", strip_text = "gold", frame = "full")) |  
  facet(col.continent))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp, tick_count = 3) + y(:life) +  
  theme(strip = "navy", strip_text = "gold", frame = "full")) |  
  facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { tick_count: 3 }),  
  y(col.life),  
  theme({ strip: "navy", strip_text: "gold", frame: "full" }),  
  across(col.continent))
```



That is the shape of every derivation in **gog**: it decides what you did not say, and stops deciding as soon as you say it. The engine guarantees the plot is well-formed and leaves taste to you, which is [Law 8](#).

One band gives **gog** no color to read. **"transparent"** shows whatever is behind it, and **gog** cannot know what that is. So the label stays dark, which suits the white page a transparent band usually sits on.

frame says how the panel is bounded, and it takes three values. **"full"** closes the axis lines into a rectangle, **"axes"** is the default pair along the bottom and left, and **"none"** removes them:

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life) +
  theme(frame = "none", grid = "y")
```

Python

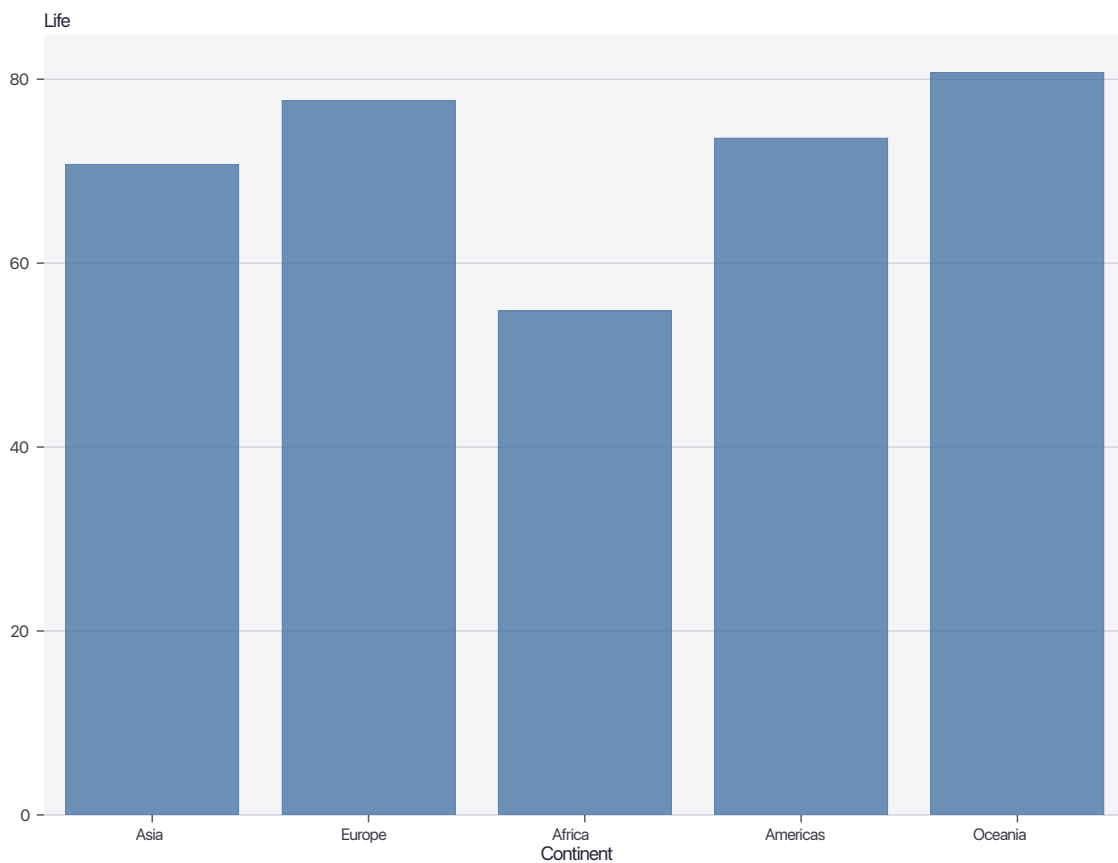

```
(data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life) +  
  theme(frame = "none", grid = "y"))
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life) +  
  theme(frame = "none", grid = "y")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),  
  y(col.life), theme({ frame: "none", grid: "y" })))
```



It is called `frame` and not `border` because `style(border_color = )` already uses that word, and it means something else: the border of a *mark*, a bar's or a point's. A frame surrounds the picture; a border is what a shape is drawn with. One word, one meaning.

`background` takes any color the rest of the grammar takes, `transparent` among them. A journal usually asks for that one, because the figure goes onto a page whose color `gog` does not know:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  theme(background = "transparent", frame = "full")
```

Python

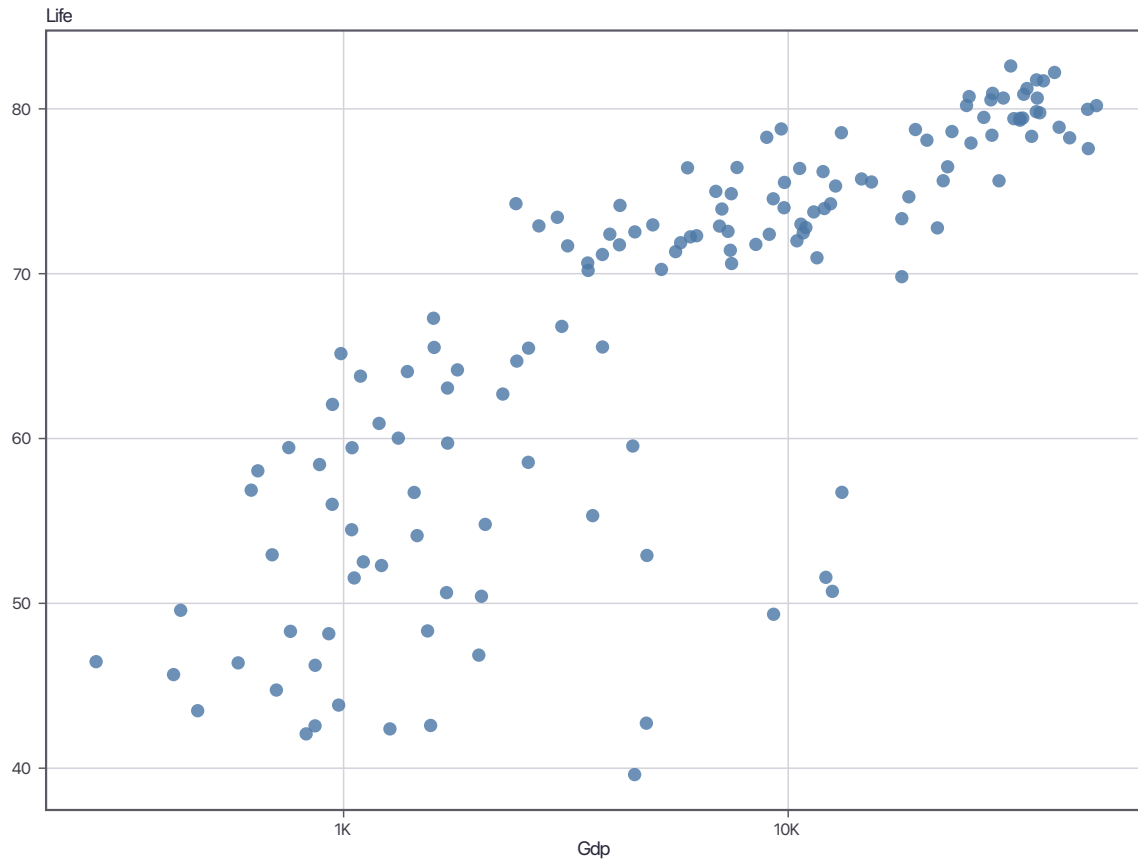
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  theme(background = "transparent", frame = "full"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  theme(background = "transparent", frame = "full")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), theme({ background: "transparent", frame: "full" }))
```



Everything `theme()` accepts is validated, and a name it does not know is refused rather than quietly ignored:

```
data(gapminder_2007) + point + x(gdp) + y(life) + theme("dark")
```

Error:

```
! gog: `theme("dark")` is not a theme. gog has `gog`, `minimal`, or `bw`. A
  ↳ theme is a named preset you can then adjust - `theme("minimal", ratio =
  ↳ 1)`.
```

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

```
data(gapminder_2007) + point + x(gdp) + y(life) + theme(grid = "vertical")
```

Error:

```
! gog: `theme(grid = )` is one of "both", "x", "y" or "none".
```

The line between the two settings atoms is worth stating once, because it is not about which is more important. `style()` is the ink: it belongs to a mark, and a plot with three layers can carry three different ones. `theme()` is the page: there is one of it, and no single layer has gridlines or a background of its own. The two never overlap, and that is why they are two words rather than one.

Text size is where that split is easiest to see, because both atoms can set a size. `style(size = 17)` on a `text` layer sets how big *those labels* are, and they are labels the data asked for. `theme(font_size = 16)` sets how big the tick labels, the axis names and the title are, and those belong to the page. One is a property of a mark and the other is a property of the plot, so neither can do the other's job.

25 Transforms

How is a measurement distributed across your rows? A transform derives a new mark from one that already exists. It modifies how data reaches the mark, preprocessing it statistically before any pixel is drawn.

The family has other names. `ggplot2` says **stat**, and the chapter Wilkinson devotes to it says **statistics** (Wilkinson, 2005); SQL says **aggregate**, `dplyr` says **summarize**, a spreadsheet says **pivot**. Each of those covers about half of what `gog` groups here. **mean**, **count** and **density** really are statistics, and they replace rows with a summary. But **bin** cuts an axis into cells, **bounds** reshapes two columns you already have and computes nothing at all, and **stack**, **dodge**, **jitter** and **repel** are about collisions between marks rather than about the values behind them. Calling all of that a statistic would misname most of it, so the word here is the widest one that stays true: a transform is anything that changes what reaches the mark.

Transforms are applied with the `*` operator:

```
mark * transform
```

The result is a derived mark type, not a new atom to memorize.

Transform	What it does
<code>bin</code>	cuts a continuous axis into equal-width cells, and counts the rows in each
<code>smooth</code>	fits a LOESS curve through the (x, y) pairs
<code>count</code>	counts the rows at each distinct x
<code>density</code>	estimates the distribution of x with a Gaussian kernel
<code>proportion</code>	divides the measurement by its total, so the shares sum to 1
<code>sum</code>	totals y within each x
<code>mean</code>	averages y within each x
<code>median</code>	the middle y within each x
<code>max</code>	the largest y within each x
<code>min</code>	the smallest y within each x

Transform	What it does
<code>quantile</code>	the <code>y</code> at one probability within each <code>x</code> , <code>quantile(0.9)</code>
<code>range</code>	a band of <code>y</code> per group, as a pair; the whole group, or two quantiles you name
<code>confidence</code>	the mean's confidence interval per group: low, high and center
<code>deviation</code>	the spread of <code>y</code> per group: the mean plus and minus a standard deviation

The middle six (`sum`, `mean`, `median`, `max`, `min`, `quantile`) form the **aggregation family**. All share the same pattern: group by `x`, reduce `y` to one value. `range`, `confidence` and `deviation` group the same way but write a low and a high, the extents an [interval](#) spans.

25.1 One variable, or two?

Fourteen of the transforms fall into two families by how many columns they *read*, and that number is exactly what decides whether you write `y()`. The rest of the kernel's twenty-three come later in this chapter: `bounds`, which reshapes rather than computes; `partition`, `flow`, `layout` and `cluster`, which each compute a whole picture; and the four collision modifiers.

Inventing transforms (`bin`, `count`, `density`) read a *single* variable and make the measured axis from it. `proportion` joins them whenever it stands alone, having then no measurement to rescale but the tally it makes itself. You name only the variable whose shape you are asking about, and the transform invents the `y` column and labels it ("Count", "Proportion", "Density"). `bar * bin + x(life)` and `line * density + x(life)` are the same grammatical shape: one name in, a new axis out. The engine is not overlooking a missing `y()`. There is exactly one sensible `y` here, the count or the density, so it fills that in.

Reading transforms (`smooth`, `sum`, `mean`, `median`, `max`, `min`, `quantile`, `range`, `confidence`, `deviation`) read *two* columns and reshape the second against the first. They fit or aggregate an existing `y`, so a `y()` must be in scope: `bar * mean + x(country) + y(gold)` has no way to guess which column to average. Leave `y()` off and the engine invents nothing: only you know which column was meant, so it stops and names what is missing (`add y(<column>)`). `range`, `confidence` and `deviation` are the three that read a `y` like the others but *write more than one*: `range` a low and a high, the other two a low, high and center, the extents an [interval](#) spans.

So `y()`'s presence is not a quirk to memorize per transform; it reads straight off the family. This is one instance of [Explicit Over Implicit](#): the short form is allowed exactly when it cannot be misread, and refused when it could.

A statistic runs *within each group* when `color` (or `group`) is bound. `bar * bin + color(species)` is one histogram *per species*, see [overlaid histograms](#), not one combined bar that has discarded the split; and the same rule gives a density curve, a count, or a mean per group. Each output keeps its group, so the mark colors it and the legend agrees.

25.2 bin: histogram

`bin` is the first inventing transform. It cuts `x` into equal-width bins and counts the rows in each. Combine it with `bar` to draw a histogram:

R

```
data(gapminder_2007) + bar * bin + x(life) +  
  x_label("Life expectancy (years)") +  
  title("Histogram of Life Expectancy, 2007")
```

Python

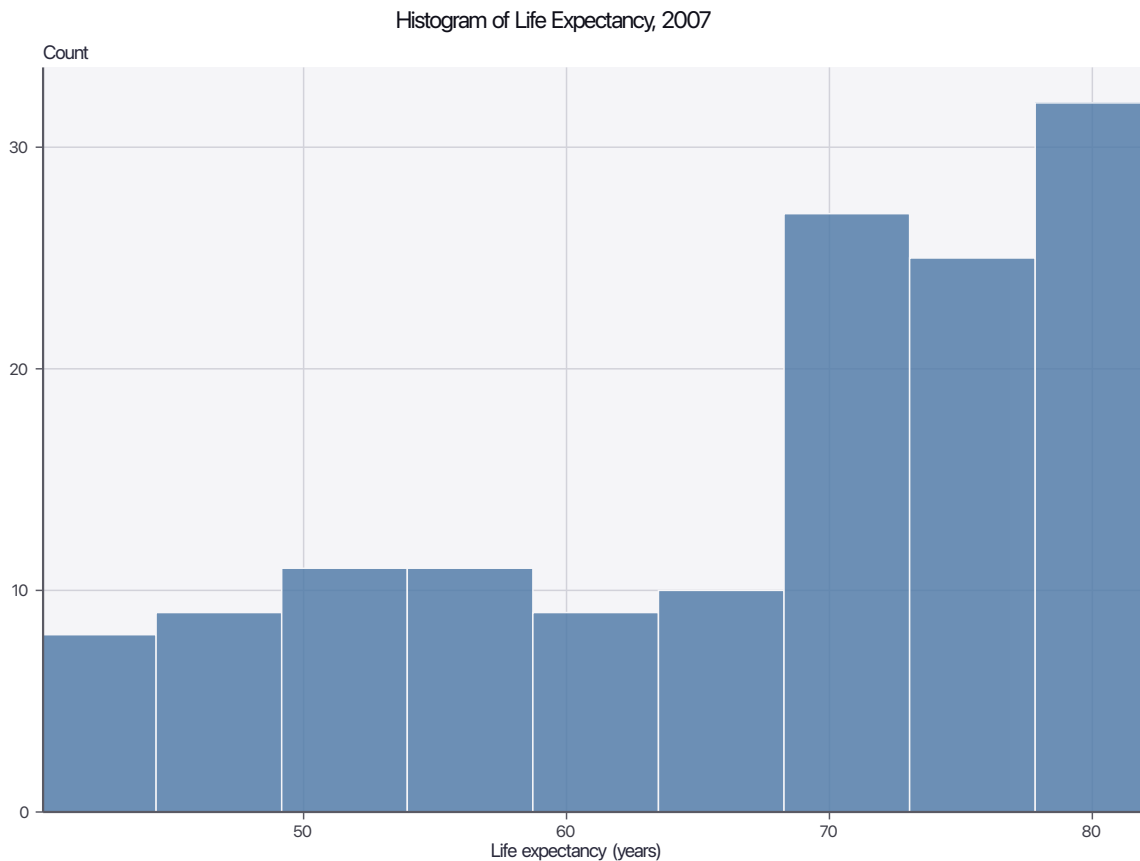
```
(data(gapminder_2007) + bar * bin + x(col.life) +  
  x_label("Life expectancy (years)") +  
  title("Histogram of Life Expectancy, 2007"))
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("Histogram of Life Expectancy, 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life),  
  x_label("Life expectancy (years)"),  
  title("Histogram of Life Expectancy, 2007"))
```



“Given gapminder 2007: bars derived by bin, x is life.”

Bin count defaults to Sturges’ rule (Sturges, 1926): $k = \log_2(n) + 1$. Notice there is no `y()` in the expression: `bin` invents the count column internally, and the y-axis is labeled “Count” automatically.

25.2.1 Choosing the bin count

Sturges’ rule is a sensible default, not a mandate. Pass a number to set the bin count, or name a `width` to fix the bin width in the data’s own units:

R

```
data(gapminder_2007) + bar * bin(30) + x(life) +
  x_label("Life expectancy (years)") + title("Thirty bins: a finer
  ↪ histogram")
```


Python

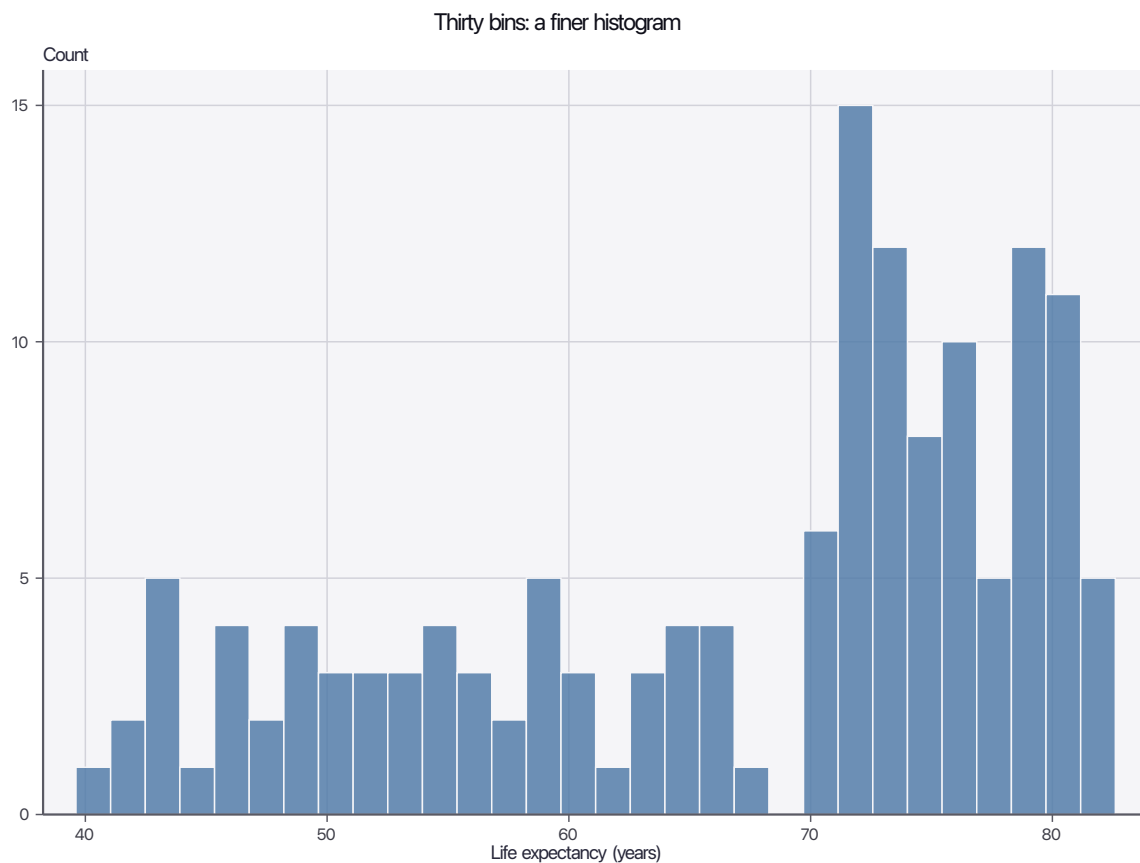
```
(data(gapminder_2007) + bar * bin(30) + x(col.life) +  
  x_label("Life expectancy (years)") + title("Thirty bins: a finer  
  ↪ histogram"))
```

Julia

```
data(gapminder_2007) + bar * bin(30) + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("Thirty bins: a finer histogram")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin(30)), x(col.life),  
  x_label("Life expectancy (years)",  
  title("Thirty bins: a finer histogram"))
```



“Given gapminder 2007: bars derived by bin into 30, x is life.”

R

```
data(gapminder_2007) + bar * bin(width = 5) + x(life) +  
  x_label("Life expectancy (years)") + title("Bins five years wide")
```

Python

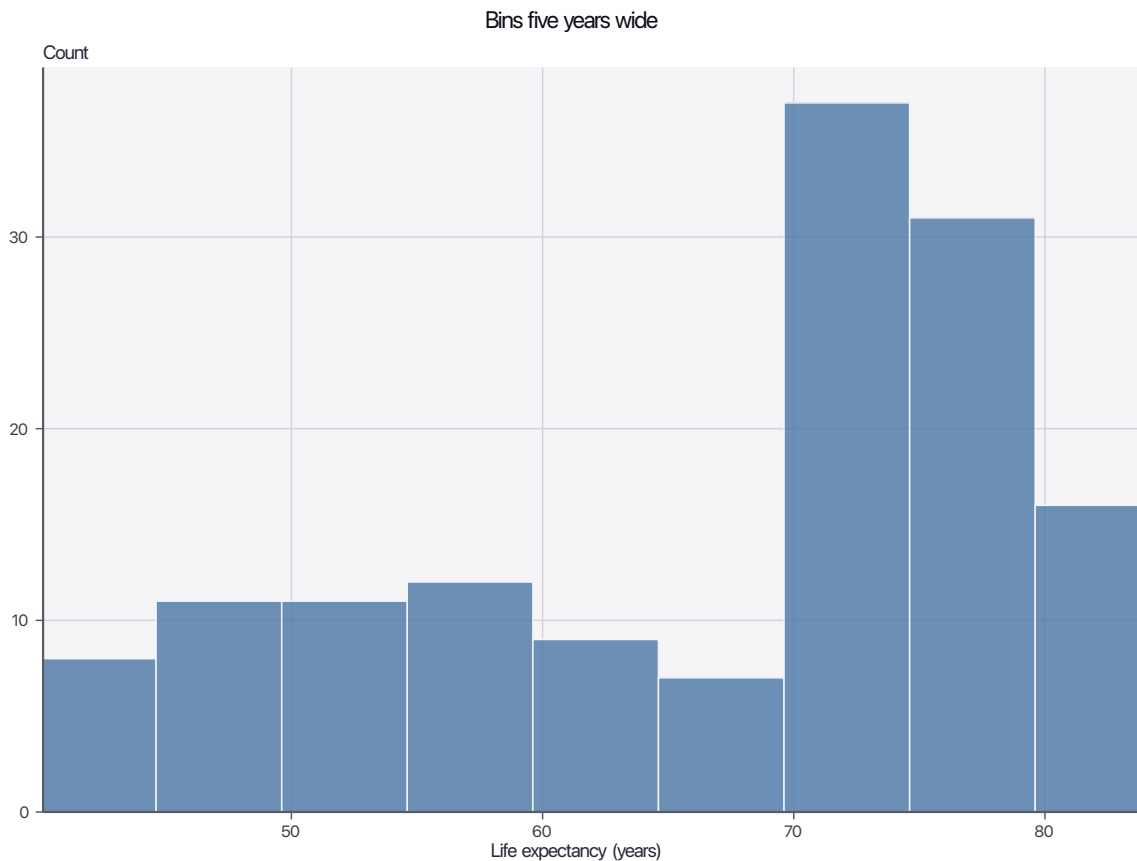
```
(data(gapminder_2007) + bar * bin(width = 5) + x(col.life) +  
  x_label("Life expectancy (years)") + title("Bins five years wide"))
```

Julia

```
data(gapminder_2007) + bar * bin(width = 5) + x(:life) +  
  x_label("Life expectancy (years)") + title("Bins five years wide")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin({ width: 5 })), x(col.life),  
  x_label("Life expectancy (years)", title("Bins five years wide"))
```



“Given gapminder 2007: bars derived by bin at width 5, x is life.”

The positional number is the *count*: `bin(30)` is thirty bins. A width is the less common request, so it is said out loud, `bin(width = 5)`; the two are mutually exclusive. Writing `bin(30, width = 5)` refuses with direction rather than guessing which you meant: the same reason `bin(30)` alone cannot mean a width, since on this data five bins and five-wide bins are opposite plots.

25.2.2 Why `bar * bin`, not a `histogram()` shortcut?

Because `bin` does the same computation regardless of the mark, and the *mark* chooses how the bins are drawn. `bar * bin` fills bars. `line * bin` connects the bin centers into a [frequency polygon](#), and `step * bin` traces the histogram as a staircase outline. Add `stack` and a fourth reading appears, the dot plot further down this chapter: one dot per observation rather than one mark per bin. Adding a `histogram()` shortcut would violate Law 2 (No Exceptions) by special-casing a particular mark+transform pair.

25.2.3 The same bin, cutting two axes

The mark decides something else too: *how many axes get cut*. Each mark above leaves one axis free to measure the count along, so `bin` cuts the other one. A `zone` measures nothing by length, and it needs an extent on both axes to be a rectangle. So the same word cuts both, and the count goes to `color`, the only channel left holding a number:

R

```
data(gm_all) + zone * bin(24) + x(gdp, scale = "log") + y(life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("One bin, two axes: the heatmap")
```

Python

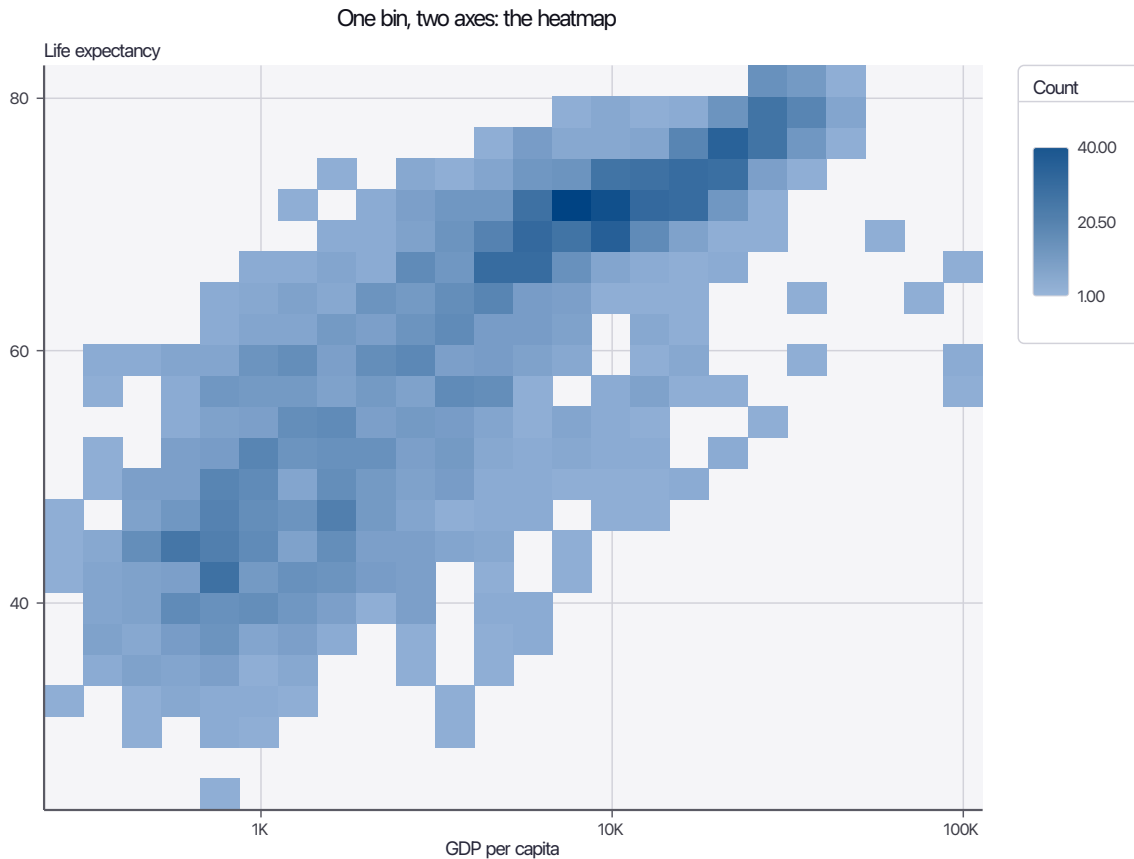
```
(data(gm_all) + zone * bin(24) + x(col.gdp, scale = "log") + y(col.life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("One bin, two axes: the heatmap"))
```

Julia

```
data(gm_all) + zone * bin(24) + x(:gdp, scale = "log") + y(:life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("One bin, two axes: the heatmap")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(24)), x(col.gdp, { scale: "log" }),  
  y(col.life), x_label("GDP per capita"), y_label("Life expectancy"),  
  title("One bin, two axes: the heatmap"))
```



“Given all the gapminder years: zones derived by bin into 24, x is gdp on a log scale, y is life.”

That is a heatmap, with no atom added to the kernel.

25.2.4 One of each: the mixed mesh

“Cuts both” needs one qualification. What `bin` cuts is every axis that has a width to cut, and a categorical axis has none, because it arrived already cut: a category *is* a cell. Give a zone one of each and the transform does the only coherent thing, which is to cut the continuous axis and leave the other alone:

R

```
data(gm_all) + zone * bin(20) + x(life) + y(continent) +
  palette("viridis") + title("One axis cut, one already cut")
```

Python

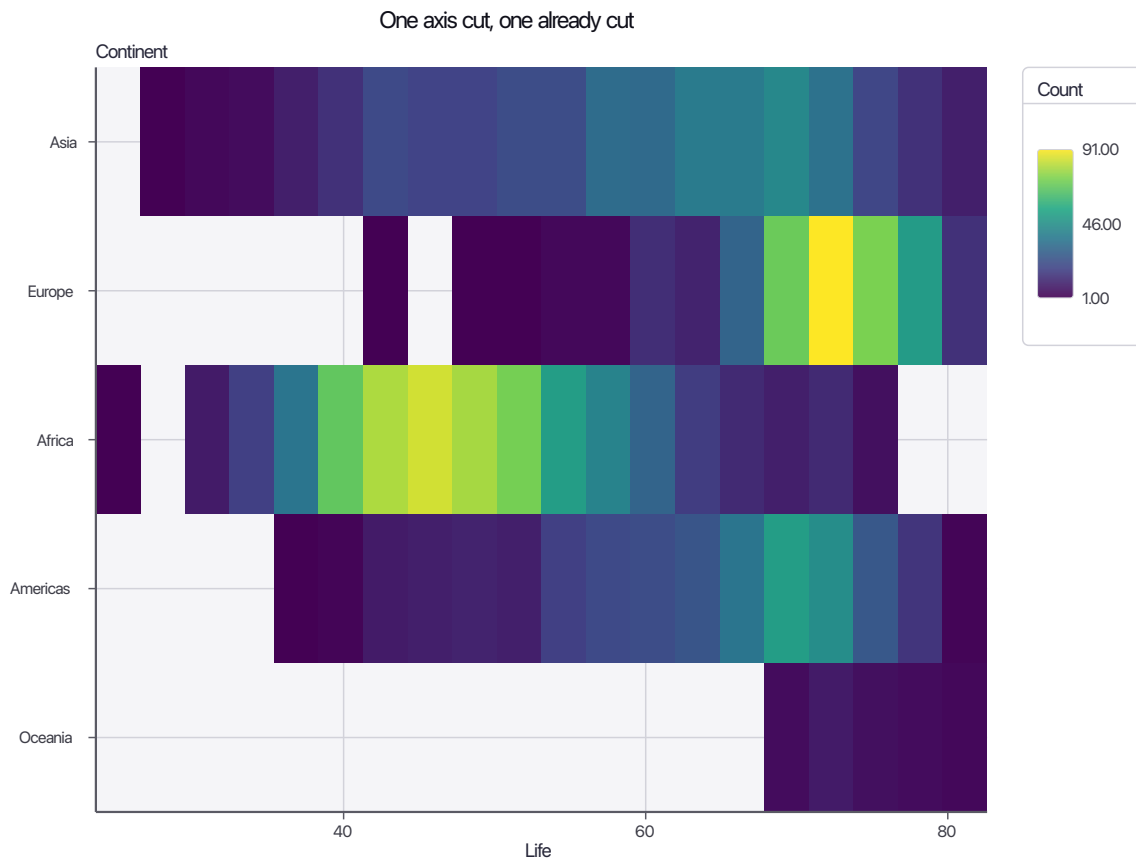
```
(data(gm_all) + zone * bin(20) + x(col.life) + y(col.continent) +  
  palette("viridis") + title("One axis cut, one already cut"))
```

Julia

```
data(gm_all) + zone * bin(20) + x(:life) + y(:continent) +  
  palette("viridis") + title("One axis cut, one already cut")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20)), x(col.life), y(col.continent),  
  palette("viridis"), title("One axis cut, one already cut"))
```



“Given all the gapminder years: zones derived by bin into 20, x is life, y is continent, with the viridis palette.”

A row of cells per continent, every row cut on the same edges, so a column compares straight down. It is a distribution per category shown as shade rather than as height, which is why five of them fit where one histogram would go. The [Zone chapter](#) reads all three cases as one rule.

25.2.5 Which mesh: tiling

`bin` also decides the *shape* of the cells it cuts, because a different mesh puts different rows in different cells and so changes the counts. Rectangles are the default; hexagons stagger alternate rows, which stops the eye reading the mesh’s own alignment as structure in the data (Carr et al., 1987):

R

```
data(gm_all) + zone * bin(20, tiling = "hex") + x(gdp, scale = "log") +  
  ↪ y(life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Carr's staggered mesh: tiling = hex")
```

Python

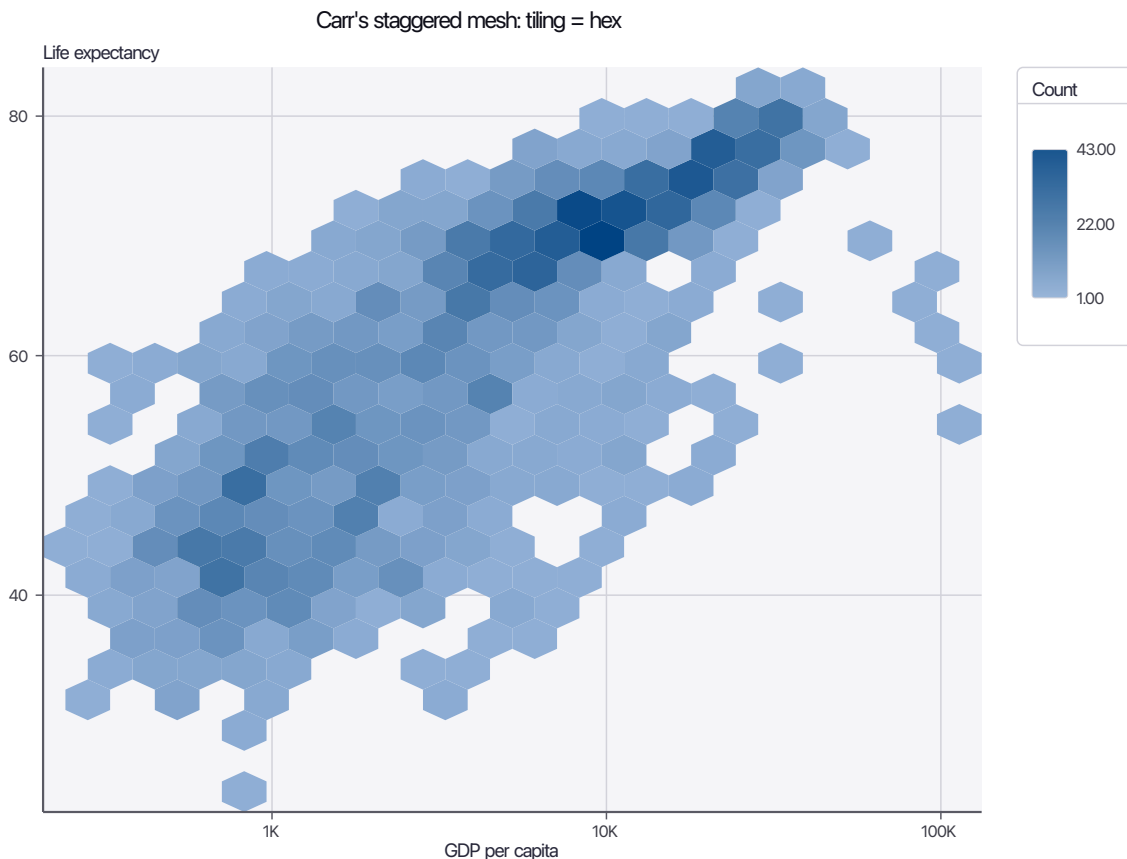
```
(data(gm_all) + zone * bin(20, tiling = "hex") + x(col.gdp, scale = "log") +  
  ↪ y(col.life) +  
  x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Carr's staggered mesh: tiling = hex"))
```

Julia

```
data(gm_all) + zone * bin(20, tiling = "hex") + x(:gdp, scale = "log") +  
  y(:life) + x_label("GDP per capita") + y_label("Life expectancy") +  
  title("Carr's staggered mesh: tiling = hex")
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20, { tiling: "hex" })),  
  x(col.gdp, { scale: "log" }), y(col.life), x_label("GDP per capita"),  
  y_label("Life expectancy"),  
  title("Carr's staggered mesh: tiling = hex"))
```



“Given all the gapminder years: zones derived by bin into 20 on hexagons, x is gdp on a log scale, y is life.”

A tiling means nothing to a bin that cuts one axis, since there the cells are intervals and an interval has no shape, so `bar * bin(tiling = )` is refused pointing at `zone`. The [Zone chapter](#) works through both meshes, including why the empty cells are left as panel.

25.3 count: frequency aggregation

`count` counts the number of rows for each unique value of x , and it works on both string and numeric columns. It is the categorical counterpart to `bin`: `bin` groups a continuous variable; `count` groups a discrete one.

R


```
# How many countries are in each continent?  
data(gapminder_2007) + bar * count + x(continent) +  
  title("Countries per Continent, 2007")
```

Python

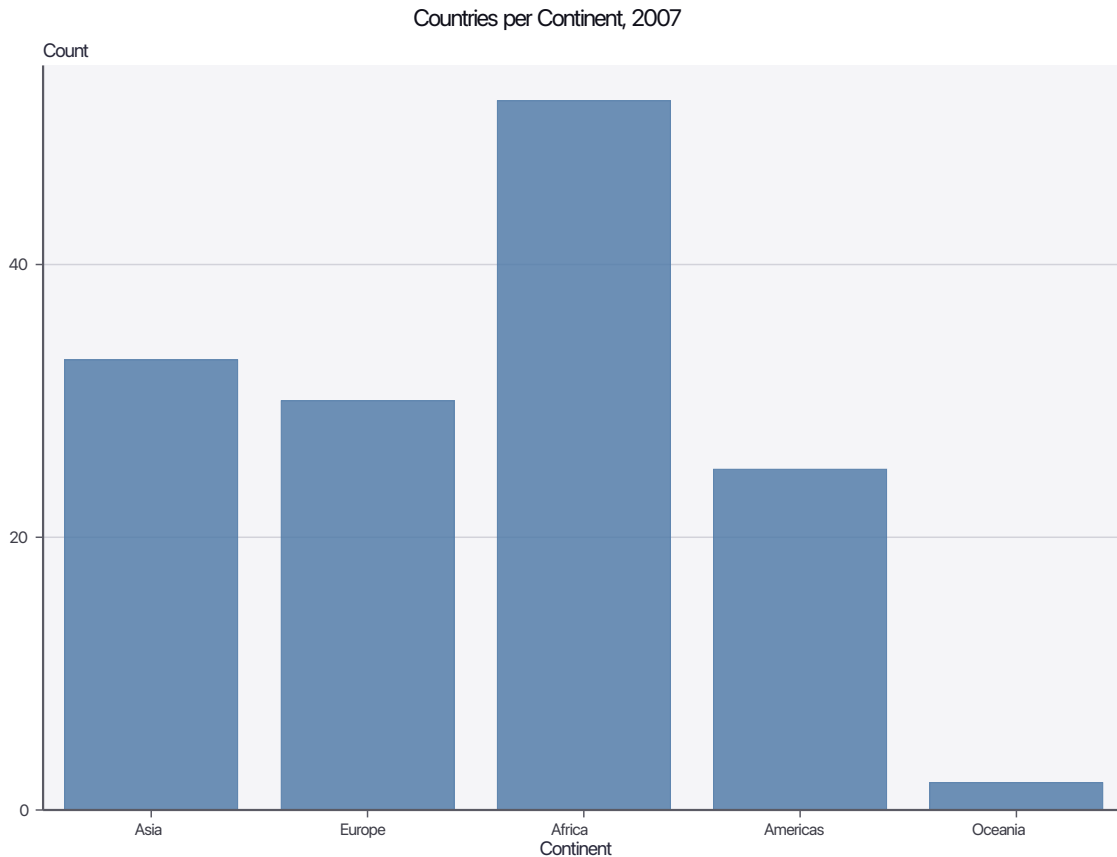
```
(data(gapminder_2007) + bar * count + x(col.continent) +  
  title("Countries per Continent, 2007"))
```

Julia

```
data(gapminder_2007) + bar * count + x(:continent) +  
  title("Countries per Continent, 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, count), x(col.continent),  
  title("Countries per Continent, 2007"))
```



“Given gapminder 2007: bars derived by count, x is continent.”

The bars show it. Histogram bars touch, because the slices adjoin on one continuum. Bar chart bars stand apart, because the categories are separate.

`count`, `bin`, and `density` are the three **inventing** transforms, and `proportion` behaves as one when it stands alone, so `y()` is optional for all four and the y-axis labels itself automatically (“Count”, “Proportion”, “Density”). Bind `y(my_name)` only when you want a custom axis label.

25.3.1 The same count, tallying two axes

`bin` cuts one axis on a bar and two on a zone, and `count` divides the same way for the same reason. A bar has a length to show the tally along, so it groups by one axis; a **zone** measures nothing by length, so it tallies rows into the cell where *two* categories cross and the count goes to `color`:

R

```
data(winds) + zone * count + x(direction) + y(season) +  
  title("One count, two axes: the tile plot")
```

Python

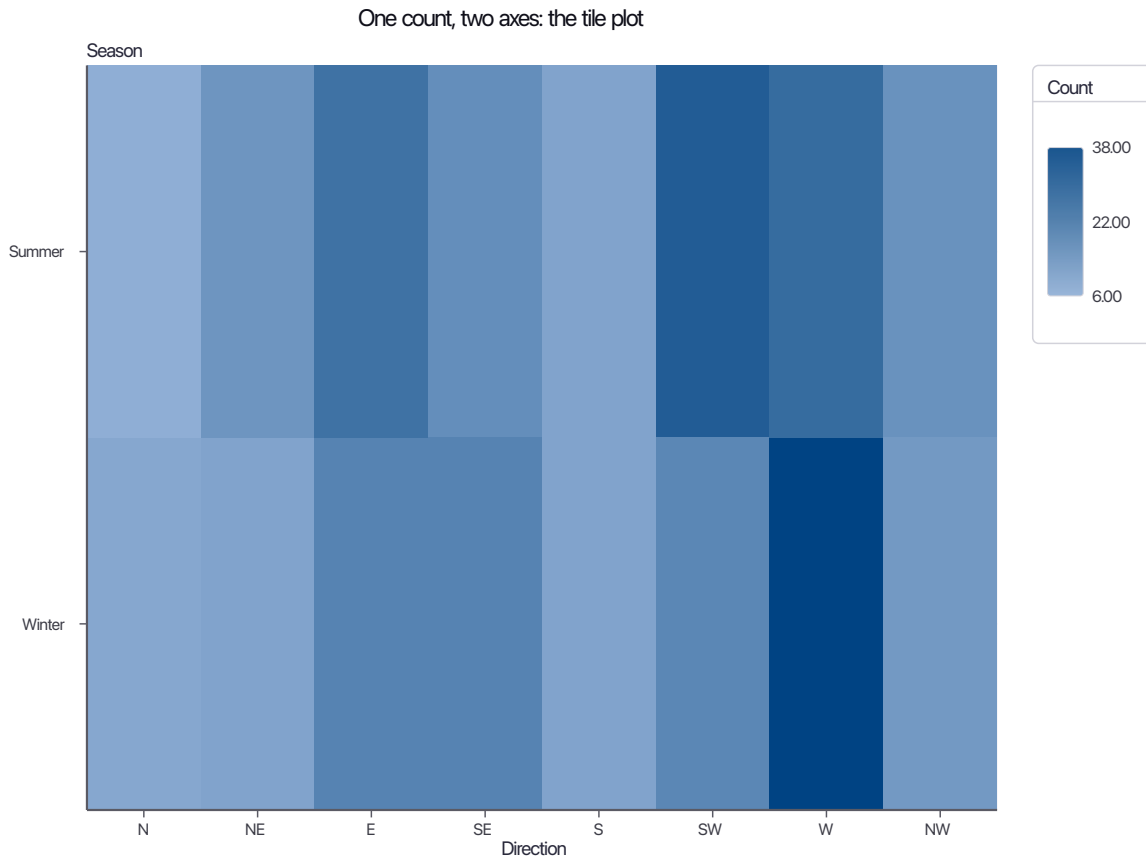
```
(data(winds) + zone * count + x(col.direction) + y(col.season) +  
  title("One count, two axes: the tile plot"))
```

Julia

```
data(winds) + zone * count + x(:direction) + y(:season) +  
  title("One count, two axes: the tile plot")
```

JavaScript

```
plot(data(winds), layer(zone, count), x(col.direction), y(col.season),  
  title("One count, two axes: the tile plot"))
```



That is the tile plot (a confusion matrix is the same sentence with different columns), and like the heatmap, no atom was added to the kernel.

The pair of them states the whole rule: **bin** cuts, **count** tallies. A continuous axis has to be cut into cells before there is anything to count in; a categorical axis arrives already cut, because a category *is* a cell. So each is refused where the other belongs, and each refusal names the other.

Counting the axes one at a time gives three plots, not two. Cut both axes and it is the heatmap; cut neither and it is the tile plot; cut one of two and it is the mixed mesh above. A **bin** is refused only where there is nothing left for it to cut at all.

25.4 proportion: relative frequency

`proportion` divides a measurement by its total, producing shares in $[0, 1]$ that sum to 1. Left to itself it has nothing to divide yet, so it tallies the rows first exactly as `count` does:

R

```
# What share of countries belongs to each continent?  
data(gapminder_2007) + bar * proportion + x(continent) +  
  title("Continent share, 2007")
```

Python

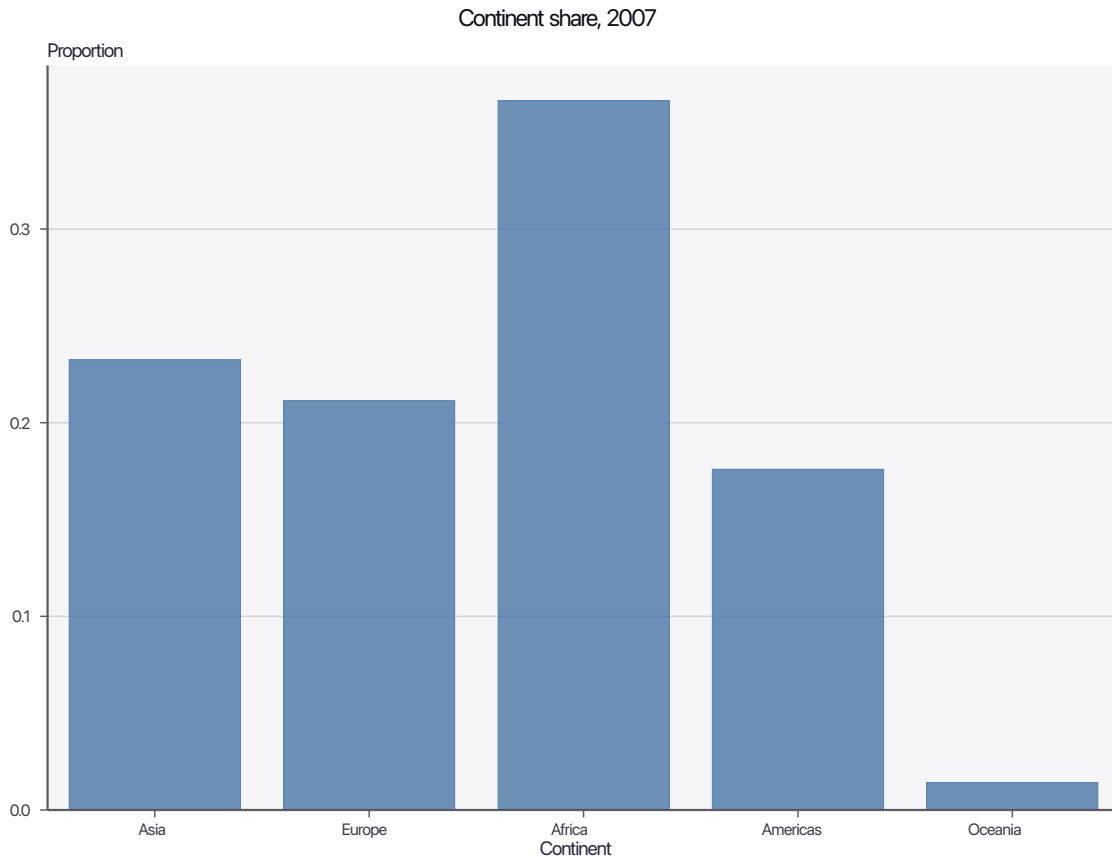
```
(data(gapminder_2007) + bar * proportion + x(col.continent) +  
  title("Continent share, 2007"))
```

Julia

```
data(gapminder_2007) + bar * proportion + x(:continent) +  
  title("Continent share, 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, proportion), x(col.continent),  
  title("Continent share, 2007"))
```



“Given gapminder 2007: bars derived by proportion, x is continent.”

Use **proportion** when you care about relative composition rather than raw counts, for example “what fraction of respondents chose each option?” Because **count** and **proportion** differ only by a constant factor (total n), **order** works identically for both: sorting by the y column produces the same order.

It follows **count** into two dimensions as well, so **zone * proportion** reads a tile plot as shares (see [Zone](#)).

25.4.1 The total is the whole plot, always

The denominator is every row the plot draws, in every context. That is the one thing the word is allowed to mean. Split the bars by a second column and they still sum to 1 between them, so each bar reads as *this* pair’s share of everything:

R

```
data(winds) + bar * proportion + x(direction) + color(season) +  
  title("Every bar is a share of all 264 observations")
```

Python

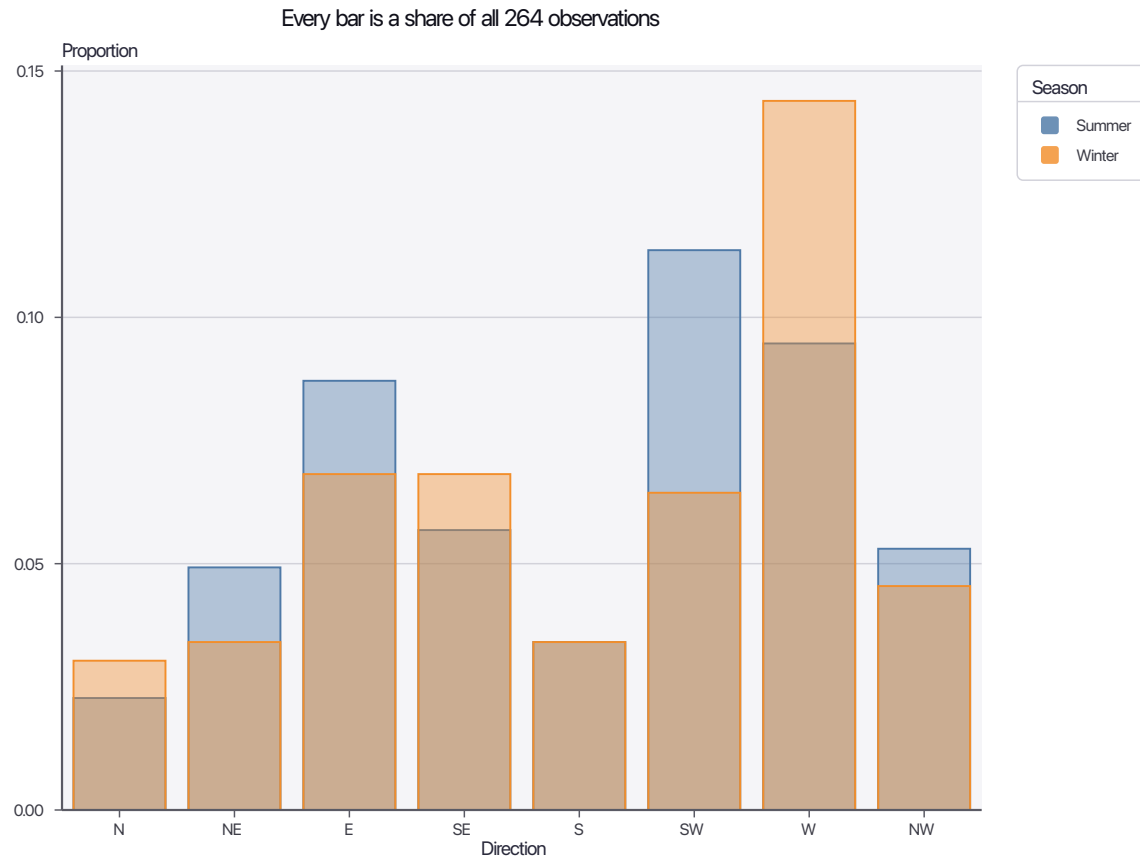
```
(data(winds) + bar * proportion + x(col.direction) + color(col.season) +  
  title("Every bar is a share of all 264 observations"))
```

Julia

```
data(winds) + bar * proportion + x(:direction) + color(:season) +  
  title("Every bar is a share of all 264 observations")
```

JavaScript

```
plot(data(winds), layer(bar, proportion), x(col.direction),  
  color(col.season),  
  title("Every bar is a share of all 264 observations"))
```



“Given the winds: bars derived by proportion, x is direction, color by season.”

Add up the sixteen segments there and you get 1, not 2. What you cannot get that way is the *conditional* reading: “of the summer winds, what share came from the west”. That reading is the facet. It cuts the data into panels before any of this runs, so each panel normalizes inside itself:

R

```
data(winds) + bar * proportion + x(direction) | facet(season)
```

Python

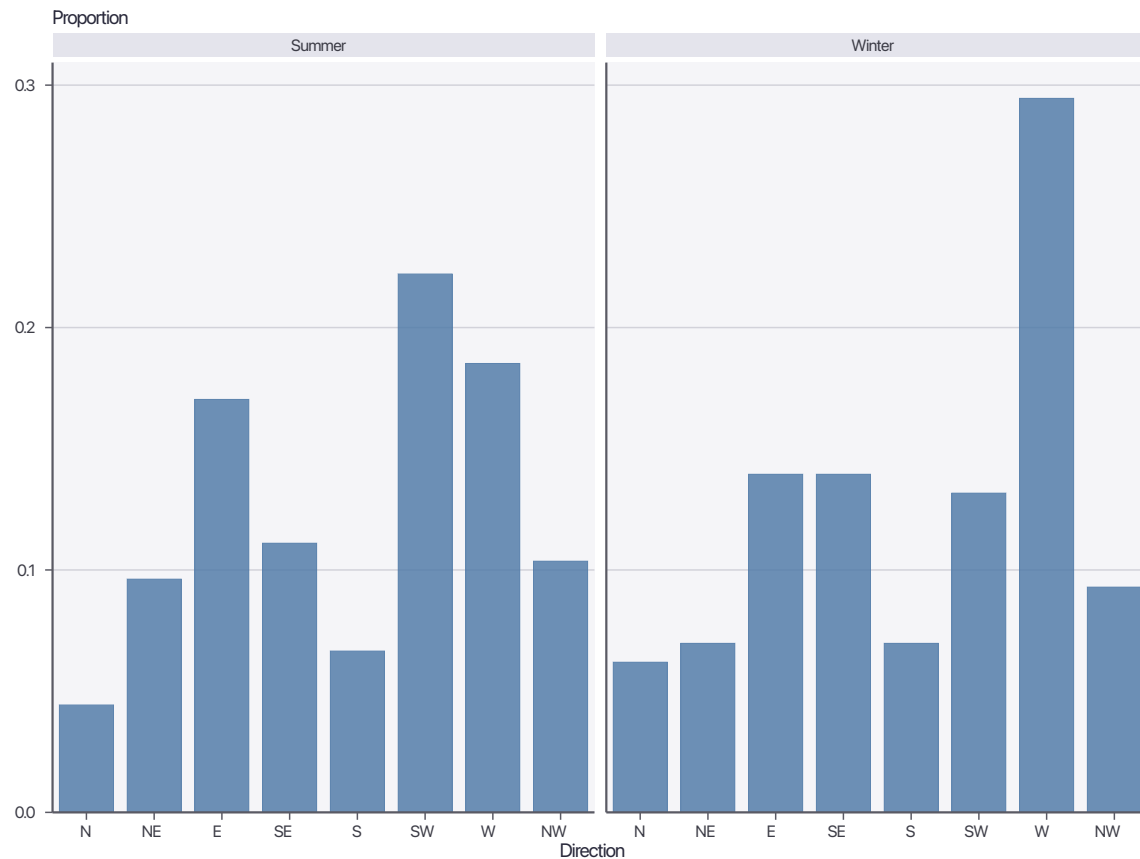
```
data(winds) + bar * proportion + x(col.direction) | facet(col.season)
```

Julia


```
data(winds) + bar * proportion + x(:direction) | facet(:season)
```

JavaScript

```
plot(data(winds), layer(bar, proportion), x(col.direction),  
  across(col.season))
```



“Given the winds: bars derived by proportion, x is direction, split into panel columns by season.”

Read the two plots against each other and the difference is the whole distinction: in the first, summer’s bars and winter’s bars divide one total between them; in the second each season gets its own.

25.4.2 Normalizing something other than a tally

Because `proportion` divides whatever measurement it finds, it composes with any transform that leaves one number per cell. Put it after a `bin` and the histogram's own counts come back as fractions, the **relative-frequency histogram**, on exactly the mesh a plain `bar * bin` cuts:

R

```
data(gm_all) + bar * bin(20) * proportion + x(life) +  
  title("Relative frequency, not counts")
```

Python

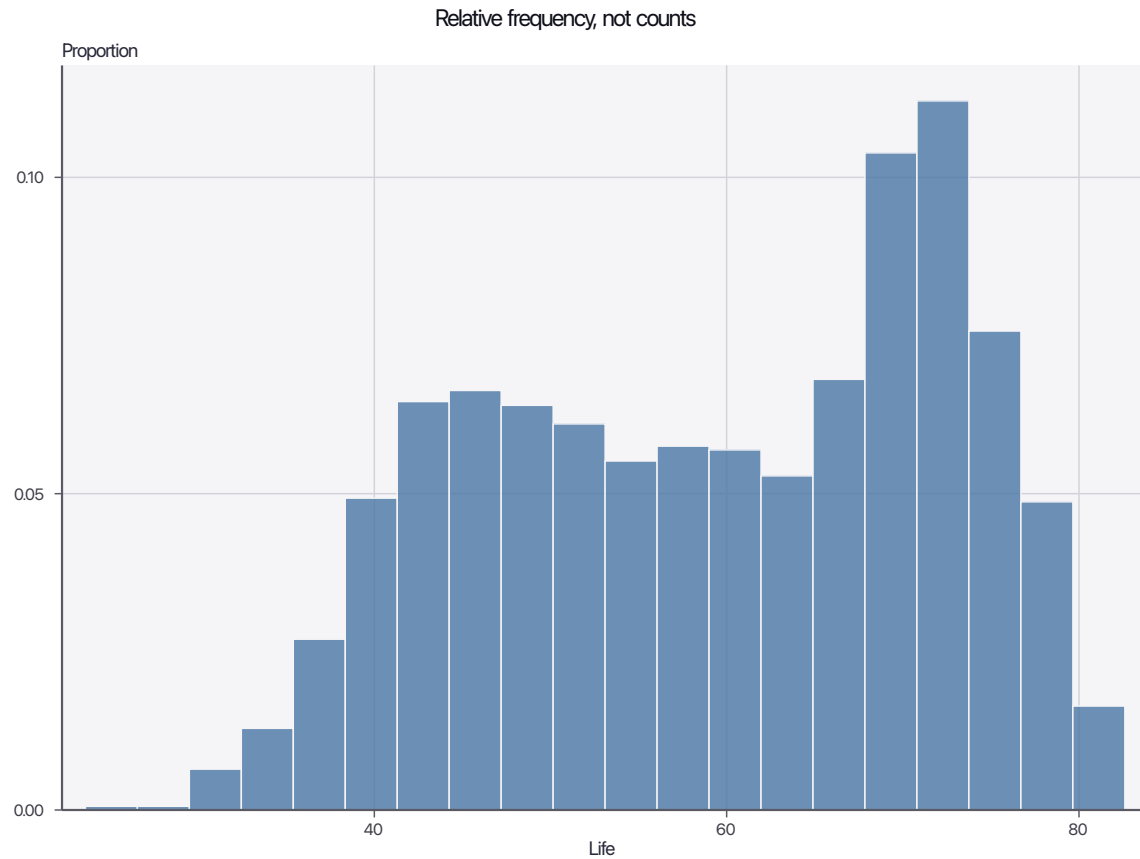
```
(data(gm_all) + bar * bin(20) * proportion + x(col.life) +  
  title("Relative frequency, not counts"))
```

Julia

```
data(gm_all) + bar * bin(20) * proportion + x(:life) +  
  title("Relative frequency, not counts")
```

JavaScript

```
plot(data(gm_all), layer(bar, bin(20), proportion), x(col.life),  
  title("Relative frequency, not counts"))
```



“Given all the gapminder years: bars derived by bin into 20 and proportion, x is life.”

Put it after a summary and the column you named is the thing rescaled, so each slot reads as its share of the grand total:

R

```
data(gapminder_2007) + bar * sum * proportion + x(continent) + y(population)
↪ +
  title("Each continent's share of world population, 2007")
```

Python

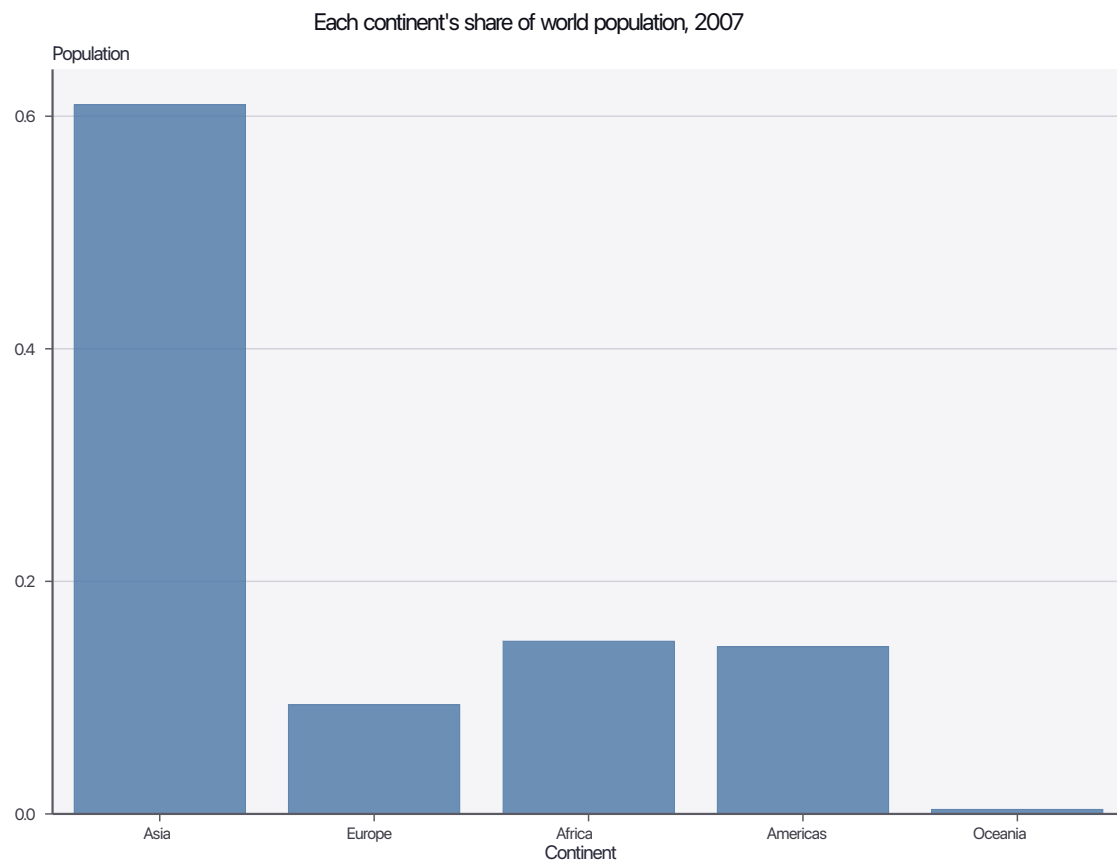
```
(data(gapminder_2007) + bar * sum * proportion + x(col.continent) +
↪ y(col.population) +
  title("Each continent's share of world population, 2007"))
```

Julia

```
data(gapminder_2007) + bar * sum * proportion + x(:continent) +  
y(:population) +  
title("Each continent's share of world population, 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, sum, proportion), x(col.continent),  
y(col.population),  
title("Each continent's share of world population, 2007"))
```



“Given gapminder 2007: bars derived by sum and proportion, x is continent, y is population.”

What it refuses is what leaves it nothing to divide by. A **density** already integrates to 1, and its cells are sample points rather than parts of a whole. A **smooth** gives a value *at* each x, so

adding those values up answers nothing. A transform that writes a low and a high leaves two numbers per cell, and a span is not part of any total.

```
data(gm_all) + line * density * proportion + x(life)
```

Error:

```
! gog: `line * density * proportion` has nothing to normalize - a density
  ↳ already integrates to 1, and its cells are points where the estimate was
  ↳ sampled rather than parts of a whole, so the sum of its heights is not a
  ↳ quantity anything is a share of. For the estimated shape, `line * density
  ↳ + x(<number>)`; for shares of the rows themselves, cut them into cells
  ↳ first: `bar * bin * proportion + x(<number>)` is the same curve's
  ↳ histogram, read as fractions.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

25.5 density: kernel density estimate

`density` is the third inventing transform: it estimates the distribution `bin` counts, as a smooth curve. The curve is the probability density function of `x`, estimated with a Gaussian kernel and evaluated at 256 evenly spaced points. By default the bandwidth (how wide each kernel is, and so how smooth the curve) is chosen automatically by Silverman's rule-of-thumb (Silverman, 1986), IQR-adjusted for robustness.

Combine it with `line` to draw a smooth density curve:

R

```
data(gapminder_2007) + line * density + x(life) +
  x_label("Life expectancy (years)") +
  title("Kernel Density: Life Expectancy 2007")
```

Python

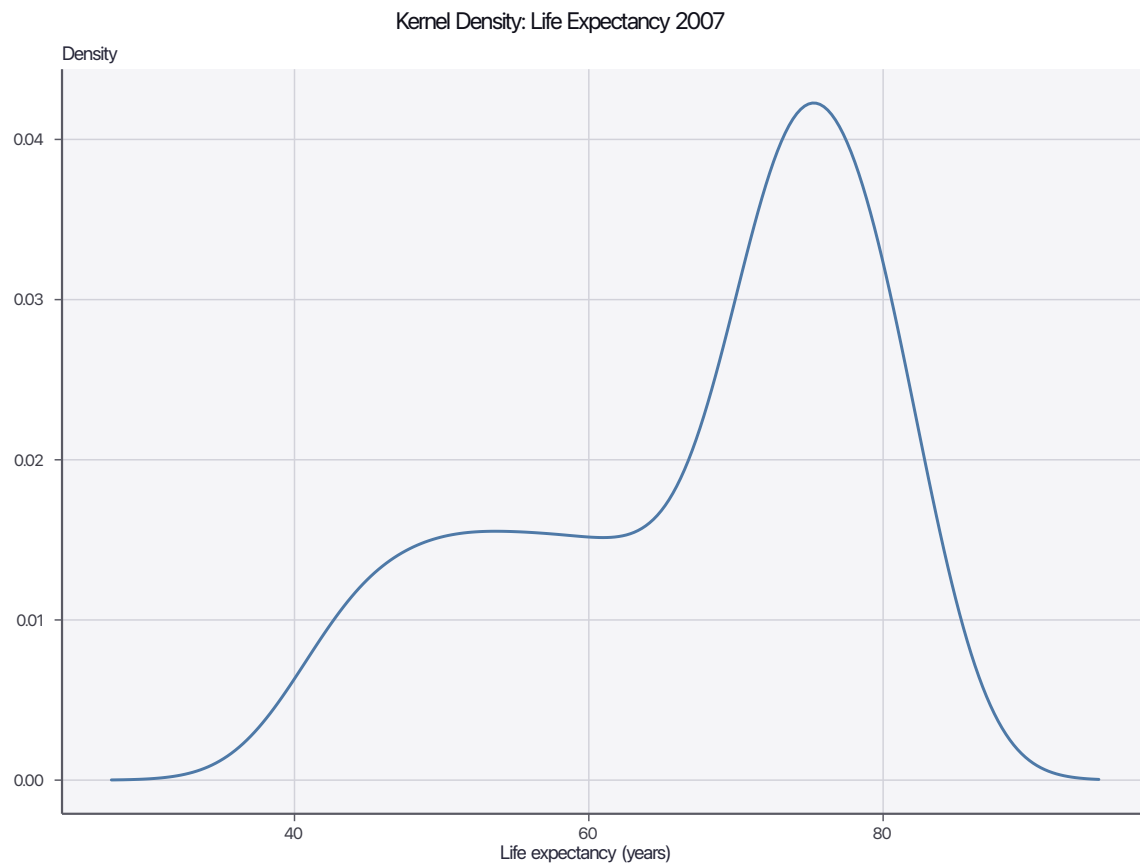
```
(data(gapminder_2007) + line * density + x(col.life) +
  x_label("Life expectancy (years)") +
  title("Kernel Density: Life Expectancy 2007"))
```

Julia

```
data(gapminder_2007) + line * density + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("Kernel Density: Life Expectancy 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density), x(col.life),  
  x_label("Life expectancy (years)",  
  title("Kernel Density: Life Expectancy 2007"))
```



“Given gapminder 2007: a line derived by density, x is life.”

As with the other inventing transforms, there is no `y()` to write: the density column and its axis label are both made for you.

25.5.1 Choosing the bandwidth

The automatic bandwidth is a sensible default, not a mandate, and it is `density`'s one parameter, just as the bin count is `bin`'s. It is set the same two ways. A positional number is an *adjust* multiplier on the automatic bandwidth: `density(2)` is twice as smooth, `density(0.5)` half as smooth and so more jagged.

R

```
data(gapminder_2007) + line * density(2) + x(life) +  
  x_label("Life expectancy (years)") + title("density(2): twice as smooth")
```

Python

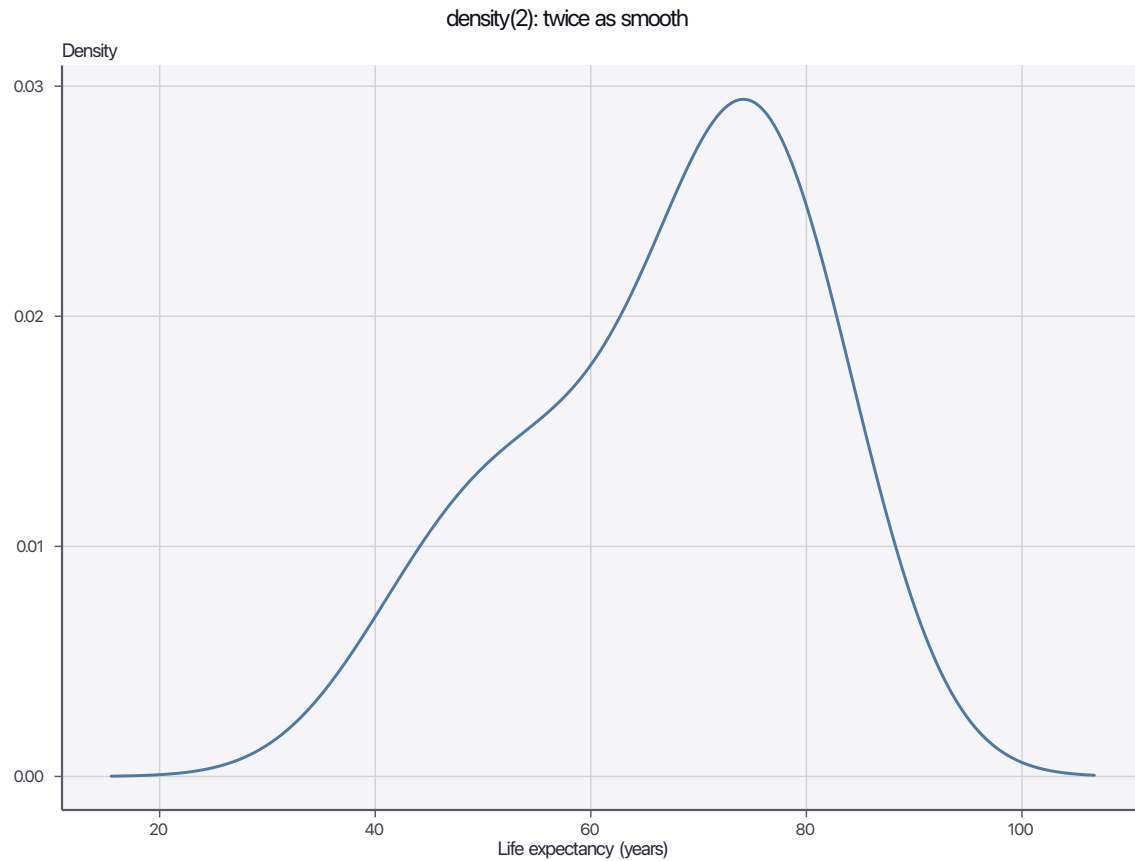
```
(data(gapminder_2007) + line * density(2) + x(col.life) +  
  x_label("Life expectancy (years)") + title("density(2): twice as smooth"))
```

Julia

```
data(gapminder_2007) + line * density(2) + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("density(2): twice as smooth")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density(2)), x(col.life),  
  x_label("Life expectancy (years)'),  
  title("density(2): twice as smooth"))
```



“Given gapminder 2007: a line derived by density adjusted by 2, x is life.”

To fix the bandwidth in the data’s own units instead, name it. Here `bandwidth = 1` sets each kernel one year wide, narrower than the default, so the curve keeps more of the distribution’s fine structure:

R

```
data(gapminder_2007) + line * density(bandwidth = 1) + x(life) +
  x_label("Life expectancy (years)") + title("bandwidth = 1 year: an absolute
↪ width")
```

Python

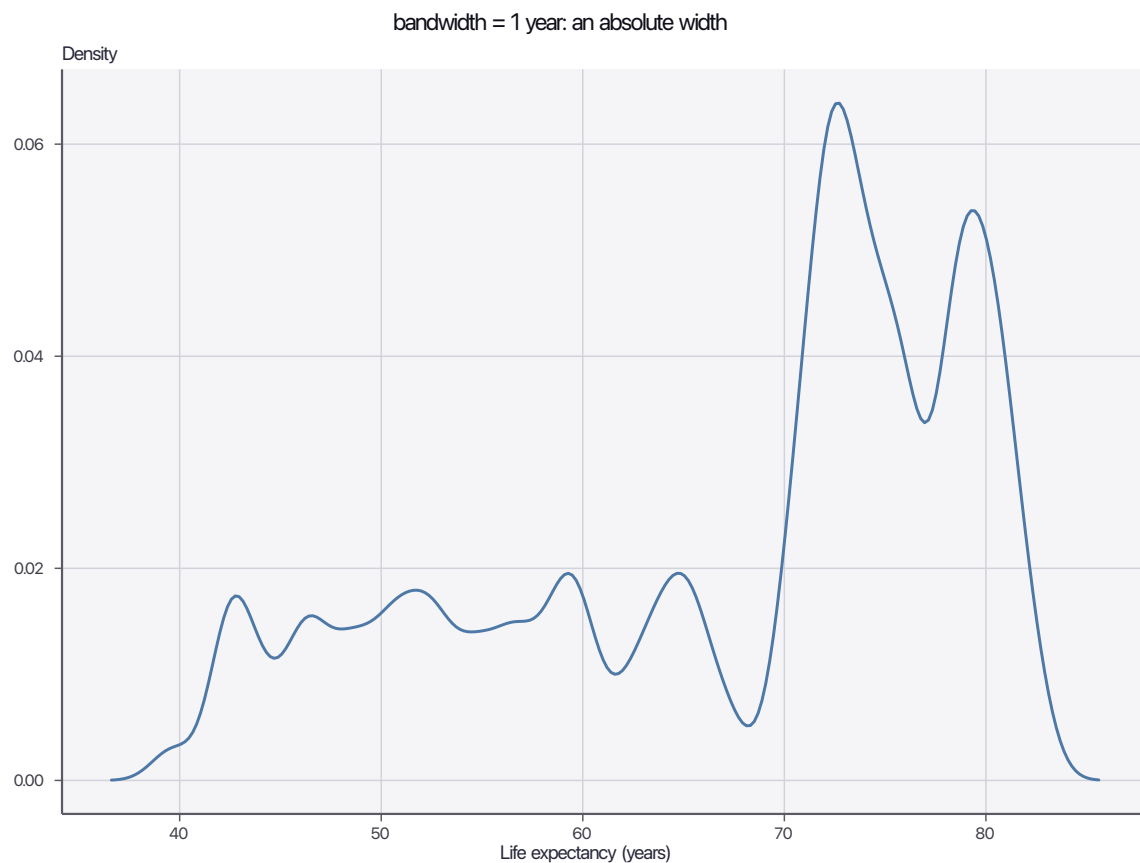
```
(data(gapminder_2007) + line * density(bandwidth = 1) + x(col.life) +
  x_label("Life expectancy (years)") + title("bandwidth = 1 year: an absolute
↪ width"))
```


Julia

```
data(gapminder_2007) + line * density(bandwidth = 1) + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("bandwidth = 1 year: an absolute width")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density({ bandwidth: 1 })),  
  x(col.life), x_label("Life expectancy (years)",  
  title("bandwidth = 1 year: an absolute width"))
```



“Given gapminder 2007: a line derived by density at bandwidth 1, x is life.”

This mirrors `bin` exactly. The positional number is the *adjust* multiplier: the common request, since you rarely know the bandwidth you want in advance and only need the automatic one

a little smoother or rougher. An absolute width is the less common request, so it is said out loud, `density(bandwidth = 1)`; the two are mutually exclusive, and `density(2, bandwidth = 1)` refuses with direction rather than guessing which you meant.

25.5.2 One transform, four readings

`density` estimates a distribution; what it is *drawn as* depends on how much room the bindings leave it, and the answer is different in each case rather than being four transforms wearing one name.

Bind one number and it is a **curve** along the free axis, which is everything above. Bind two numbers to a mark with no measure axis and it is a **field** over the plane: `path * density` traces its contours and `zone * density` paints it cell by cell, with `surface * density` raising it into the cube. Bind a **category and a number** and there is no free axis left, so the estimate is drawn across the category's slot, one distribution per group, which is [the violin](#):

R

```
data(gapminder_2007) + ribbon * density + x(continent) + y(life) +  
  y_label("Life expectancy") + title("ribbon * density: the slot reading")
```

Python

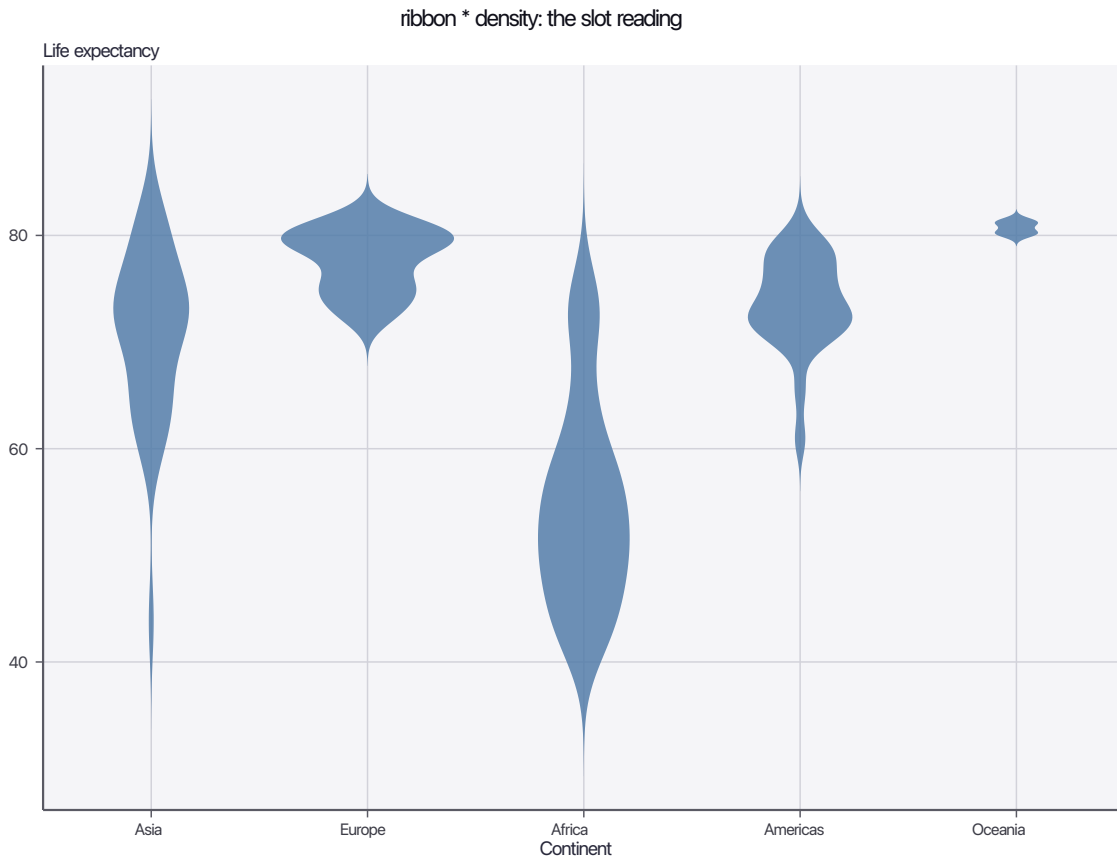
```
(data(gapminder_2007) + ribbon * density + x(col.continent) + y(col.life) +  
  y_label("Life expectancy") + title("ribbon * density: the slot reading"))
```

Julia

```
data(gapminder_2007) + ribbon * density + x(:continent) + y(:life) +  
  y_label("Life expectancy") + title("ribbon * density: the slot reading")
```

JavaScript

```
plot(data(gapminder_2007), layer(ribbon, density), x(col.continent),  
  y(col.life), y_label("Life expectancy"),  
  title("ribbon * density: the slot reading"))
```



“Given gapminder 2007: ribbons derived by density, x is continent, y is life.”

Each of `density`'s parameters belongs to one of those readings, and asking for one in a reading it cannot mean is refused rather than ignored. `bandwidth` is a length in one column's units, so it means nothing to a field, whose two axes measure different quantities. `levels` counts lines of equal density, and a curve has none to count. The slot reading has two parameters of its own. `compare` says what a violin's width *means* from one slot to the next. `reach` says how far the shape extends, measured in slots, and past half a slot the shapes reach into their neighbors: the ridgeline. Neither means anything to a plot with no slots:

```
data(gapminder_2007) + line * density(compare = "count") + x(life)
```

Error:

```
! gog: `density(compare = )` says what a violin's width means from one slot
  ↳ to the next, and `line * density` here has no slots - it estimates one
  ↳ curve along one axis. Bind a category to give it slots to compare:
  ↳ `ribbon * density(compare = "count") + x(<category>) + y(<number>)`.
```

Otherwise drop `compare`.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

25.6 What bin, density and smooth refuse

The two families above sort the transforms by how many columns they read. A second property cuts across that sort: these three all describe how values are spread *along* an axis, so the axis they read has to carry a number. A category is one slot rather than a stretch of line, so there is nothing to cut into intervals, nothing for a curve to spread over, and nothing to fit through.

The violin above looks like an exception, and it is worth being precise about why it is not one. There the category is not the axis being estimated along: `y(life)` is, and the category only says *which rows* each estimate is made from. Take the number away and there is nothing left to spread over, so the refusal returns, which is why every chunk below binds a category and nothing else.

Each refusal names the atom that asks the same question of categories. Binning one is really counting it:

```
data(gapminder_2007) + bar * bin + x(continent)
```

Error:

```
! gog: `bin` cuts a continuous axis into intervals, and `x(continent)` is
  ↳ categorical - a category is one slot, with no width to cut. To tally rows
  ↳ per category, `count` is the transform that does it: `bar * count`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A density over categories is really a share of the total:

```
data(gapminder_2007) + bar * density + x(continent)
```

Error:

```
! gog: `density` estimates a continuous distribution, and `x(continent)` is
  ↳ categorical - there is no number line for the curve to spread along. For
  ↳ the share of rows in each category, that is `bar * proportion`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

And a curve fitted across categories is really a summary within each one:

```
data(gapminder_2007) + point * smooth + x(continent) + y(life)
```

Error:

```
! gog: `smooth` fits a curve of `y` against `x`, and `x(continent)` is
  ↳ categorical - a fit needs a number line to run along. For a typical value
  ↳ per category, `bar * mean` (or `point * mean`) says it directly.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Which axis must be numeric is read off the bindings rather than fixed to x. The [horizontal histogram](#) is `bar * bin + y(life)`: it cuts y, because y is the axis it was handed and x is the one it invents. `smooth` is the strict one of the three, since it fits y against x and so needs a number on both.

25.6.1 A fit needs rows to fit through

Having numbers is not the same as having enough of them. `smooth` fits a curve, and two points are a straight line while one is a point, so it asks for at least three:

```
two_readings <- subset(gm_all, country == "France" & year >= 2002)
data(two_readings) + line * smooth + x(year) + y(life)
```

Error:

```
! gog: `smooth` fits a curve through the rows and needs at least 3 of them.
  ↳ This data has 2. Plot what you have with `point` and no transform, which
  ↳ draws the rows as rows rather than as a curve through them.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The count that matters is the count *inside each group*, not in the table. A statistic runs within every group you split by, and faceting splits the rows before any of that. So a table with plenty of rows can still be asked for a fit through two:

```
recent <- subset(gm_all, year >= 2002)
data(recent) + line * smooth + x(year) + y(life) + group(country) |
  facet(continent)
```

Error:

```
! gog: `smooth` fits a curve through the rows and needs at least 3 of them.
  ↳ Split by `continent` or `country`, the smallest group has 2. Drop the
  ↳ split so the fit reads the rows together, or plot what you have with
  ↳ `point` and no transform, which draws the rows as rows rather than as a
  ↳ curve through them.
```

`gog`: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

There are 284 rows there and the refusal still stands, because each country in each panel is its own fit and each has two readings. The message names both splits, so you know which one to give up. Drop `group(country)` and each panel fits one curve through a continent's rows, which is plenty.

This matters more than a thin plot usually would, because of what the picture would otherwise show. Below three rows the curve cannot be computed, so what would be drawn is the rows themselves, connected in order. A two-point group would draw a straight segment beside a hundred-point curve, with nothing to say that one is a fit and the other is the data. A plot that looks finished and is not is worse than one that refuses.

25.7 Chaining transforms

Two transforms on one mark have to divide the work, and there are only four jobs to divide. A transform can say *where the cells are*, it can say *what is in them*, it can say *what scale the answer is read on*, and it can say *where the marks sit*. `bounds` names the sides of a rectangle. `count` tallies into the cells your categories already own, and says nothing about where those cells are. `mean` and its five siblings reduce a column you name, and say nothing about where the cells are either. `proportion` does the third job and only the third: it divides by a total, which is why it can stand after almost anything. `dodge`, `stack`, `jitter` and `repel` do the fourth, moving marks that would otherwise land on top of one another.

Two transforms compose when they do different jobs, and contradict when they do the same one. A cell holds one extent, one measurement, one scale and one arrangement. A second transform doing a job the first already did therefore leaves no reading that keeps both. The engine refuses, rather than drawing one and quietly throwing the other away. [Combinations](#) has the whole grid, generated from the same rule.

Because each job is filled once, the order you write them in does not change the picture. Write them in the order they happen and the sentence reads the way it runs.

`bin` is the one transform that does two of the four jobs, and the only one that can drop one of them. Cutting an axis is what makes it a `bin`; the tally it hands back is a by-product of the cut. So when you compose it with a statistic, `bin` keeps the cut and the statistic keeps the measurement:

R

```
data(gm_all) + bar * bin(20) * mean + x(gdp, scale = "log") + y(life) +  
  title("Mean life expectancy by income band")
```

Python

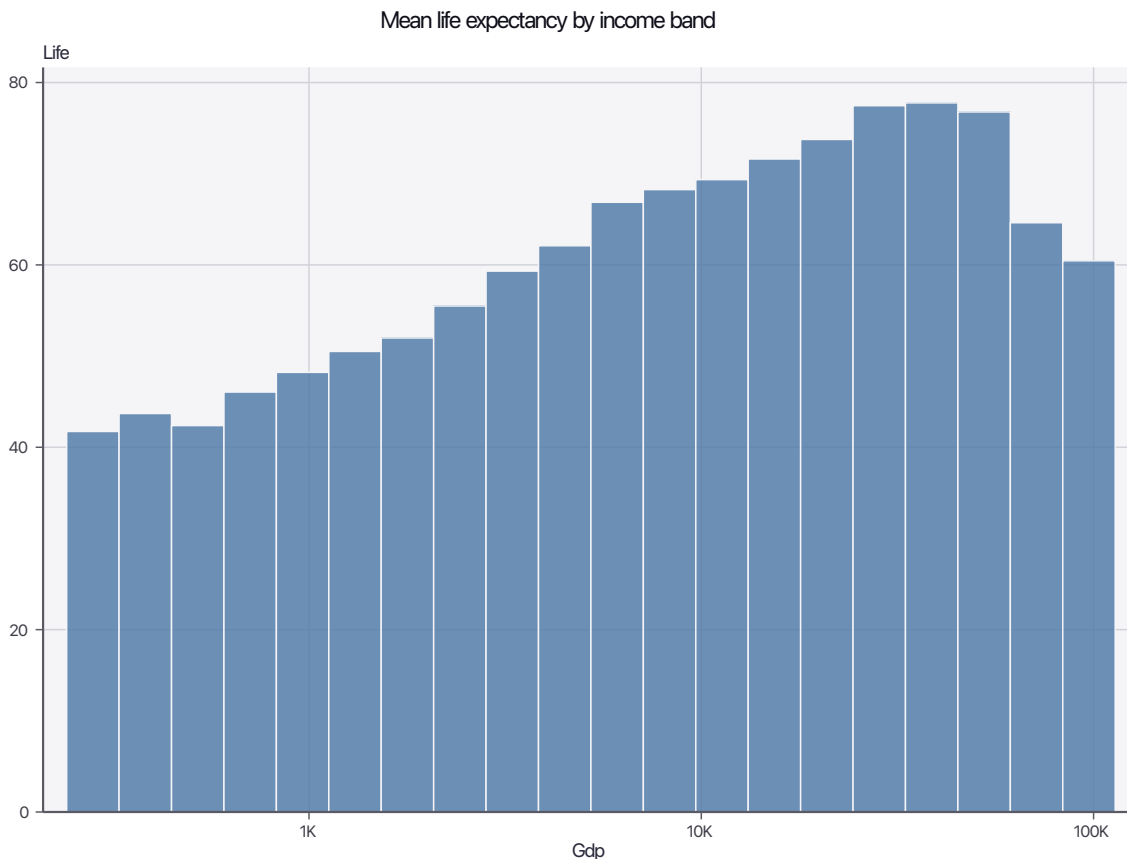
```
(data(gm_all) + bar * bin(20) * mean + x(col.gdp, scale = "log") +  
↪ y(col.life) +  
  title("Mean life expectancy by income band"))
```

Julia

```
data(gm_all) + bar * bin(20) * mean + x(:gdp, scale = "log") + y(:life) +  
  title("Mean life expectancy by income band")
```

JavaScript

```
plot(data(gm_all), layer(bar, bin(20), mean),  
  x(col.gdp, { scale: "log" }), y(col.life),  
  title("Mean life expectancy by income band"))
```



“Given all the gapminder years: bars derived by bin into 20 and mean, x is gdp on a log scale, y is life.”

Read the sentence as three separate claims. `bin(20)` cuts income into twenty bands. `mean` reduces `life` within each one. `bar` draws the result as a length from zero. The bars touch because the bands tile the axis, exactly as a histogram’s do, and the y axis is now labeled for the column that was actually reduced.

Any mark that takes both transforms takes the pair, so the same three claims read as points instead:

R

```
data(gm_all) + point * bin(20) * median + x(gdp, scale = "log") + y(life) +
  title("Median life expectancy by income band")
```

Python

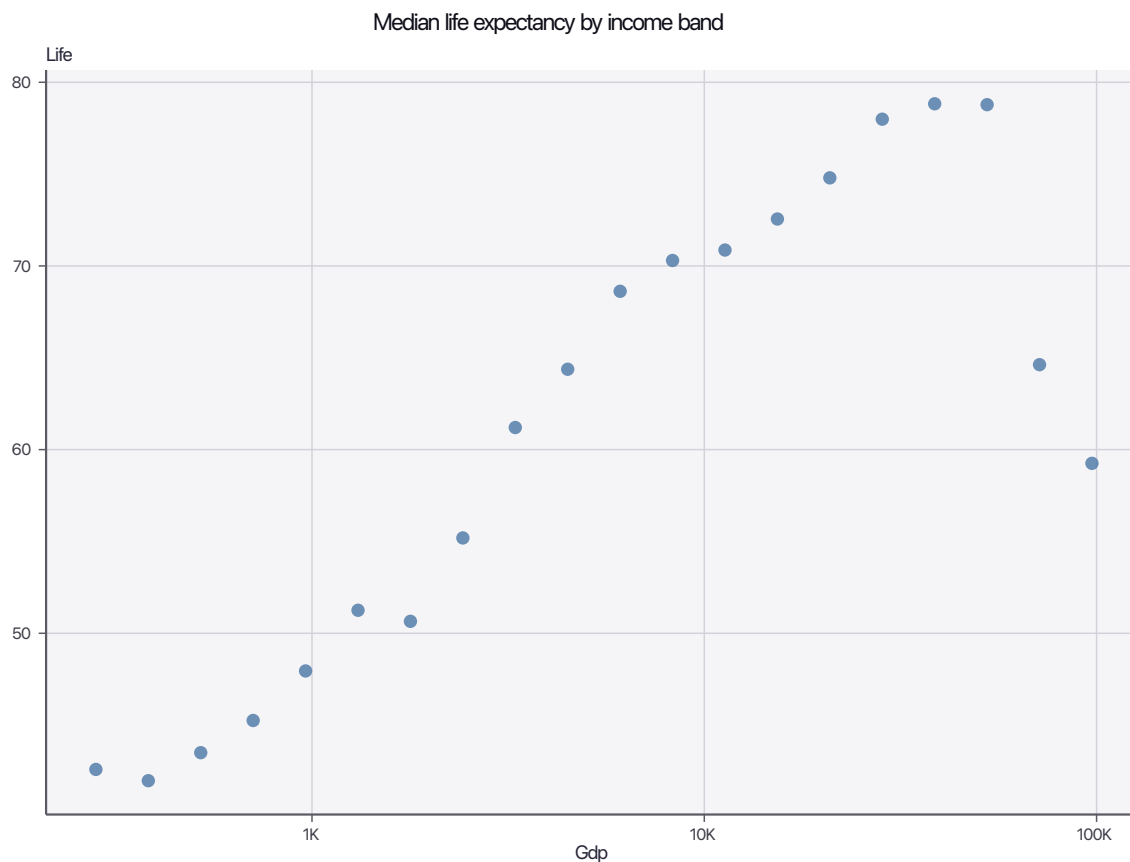

```
(data(gm_all) + point * bin(20) * median + x(col.gdp, scale = "log") +
  ↪ y(col.life) +
  title("Median life expectancy by income band"))
```

Julia

```
data(gm_all) + point * bin(20) * median + x(:gdp, scale = "log") +
  y(:life) + title("Median life expectancy by income band")
```

JavaScript

```
plot(data(gm_all), layer(point, bin(20), median),
  x(col.gdp, { scale: "log" }), y(col.life),
  title("Median life expectancy by income band"))
```



“Given all the gapminder years: points derived by bin into 20 and median, *x* is gdp on a log scale, *y* is life.”

25.7.1 What cannot be chained, and why

The other two inventing transforms cannot give their measurement up, because measuring is all they do. Take it away from count and nothing is left of it:

```
data(gm_all) + bar * count * mean + x(continent) + y(life)
```

Error:

```
! gog: `bar * count * mean` measures each cell twice - `count` supplies only
  ↳ a measurement: its cells are the slots the positions already own, so with
  ↳ `mean` measuring them too the cell is measured twice and `count` has
  ↳ nothing left to contribute. Keep whichever you meant: `bar * count` to
  ↳ measure what `count` computes, or `bar * mean` to reduce the column you
  ↳ name. To cut a continuous axis into cells and reduce a column inside
  ↳ each, `bin` is the transform that cuts without keeping the measurement:
  ↳ `bar * bin * mean + x(<number>)`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

density fails a different test. Its cells are the points where the estimate was sampled, and the estimate exists *between* your observations rather than in buckets holding them, so there are no rows inside one for a statistic to reduce:

```
data(gm_all) + bar * density * mean + x(gdp) + y(life)
```

Error:

```
! gog: `bar * density * mean` measures each cell twice - a `density` cell is
  ↳ a point where the estimate was sampled, not a bucket holding rows - the
  ↳ estimate exists *between* your observations - so there is nothing inside
  ↳ one for `mean` to reduce. Keep whichever you meant: `bar * density` to
  ↳ measure what `density` computes, or `bar * mean` to reduce the column you
  ↳ name. To cut a continuous axis into cells and reduce a column inside
  ↳ each, `bin` is the transform that cuts without keeping the measurement:
  ↳ `bar * bin * mean + x(<number>)`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

smooth is refused against all three, including bin, for a reason none of them share. It fits a curve through the rows and already averages locally as it goes, so cutting them into cells first changes nothing it was not doing:

```
data(gm_all) + bar * bin * smooth + x(gdp) + y(life)
```

Error:

! gog: `bar \* bin \* smooth` asks one question twice - `smooth` fits a curve
↳ through the rows and already averages locally as it goes, so cutting them
↳ into cells first changes nothing it was not doing. Keep whichever you
↳ meant: `bar \* smooth + x(<a>) + y(<b>)` for the fitted curve, or `bar *
↳ bin` for the shape `bin` measures. For a summary per cell rather than a
↳ fitted curve, name the statistic: `bar \* bin \* mean + x(<a>) + y(<b>)`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

And two transforms that each invent their own measurement are the same contradiction with neither able to give way, so a cell would be measured twice:

```
data(gm_all) + bar * bin * count + x(life)
```

Error:

! gog: `bar \* bin \* count` measures each cell twice - `bin` and `count` each
↳ invent their own measurement from the rows, and neither was handed a
↳ column to give way to, so there is no reading that keeps both. Keep
↳ whichever you meant: `bar \* bin` or `bar \* count`. To cut an axis into
↳ cells and measure something else inside them, the second transform has to
↳ be one you hand a column: `bar \* bin \* mean + x(<number>) + y(<column>)`.
↳ To read either as shares of the whole rather than as counts, `proportion`
↳ rescales whichever you keep: `bar \* bin \* proportion`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

Each refusal names both transforms you could drop, and the first two also name a third option: **bin** is the transform that cuts without keeping the measurement.

The same rule reaches the other three jobs, and it reads the same way each time. Two transforms that both produce a low and a high measure the cell twice. So **range** and **confidence** cannot stand together. A minimum and a maximum are not a 95% confidence interval, and one of the two would have to be discarded:

```
data(gm_all) + interval * range * confidence + x(continent) + y(life)
```

Error:

! gog: `interval \* range \* confidence` measures each cell twice - `range` and
 ↪ `confidence` both reduce the column you named, and a cell holds one
 ↪ answer, so drawing both would mean drawing one of them and discarding the
 ↪ other. Keep whichever you meant: `interval \* range` or `interval *
 ↪ confidence`. To show both, draw them as two layers: `interval * range +
 ↪ interval * confidence`.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

The same holds when one of the pair is a single value and the other is two of them:

```
data(gm_all) + line * sum * range + x(year) + y(population)
```

Error:

! gog: `line \* sum \* range` measures each cell twice - `sum` and `range` both
 ↪ reduce the column you named, and a cell holds one answer, so drawing both
 ↪ would mean drawing one of them and discarding the other. Keep whichever
 ↪ you meant: `line \* sum` or `line \* range`. To show both, draw them as two
 ↪ layers: `line \* sum + line \* range`.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

And two arrangements are the same contradiction on the fourth job. A bar cannot be set
 beside its neighbor and piled on top of it at once, because it sits in one place:

```
data(gm_all) + bar * count * dodge * stack + x(continent) + color(continent)
```

Error:

! gog: `bar \* dodge \* stack` arranges the same marks twice - `dodge` and
 ↪ `stack` each decide where colliding groups go, and a mark sits in one
 ↪ place, so one of the two would be discarded. Keep whichever you meant:
 ↪ `bar \* dodge` or `bar \* stack`. To show both readings, draw them as two
 ↪ plots side by side rather than as one layer.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A transform can also turn out to be unnecessary rather than contradictory, and that is a
 different answer. `proportion` divides a tally by its total, so it already counts the rows, which
 leaves `count` beside it with nothing to add. The plot is still exactly the plot you asked for, so
 the engine draws it:

R

```
data(gm_all) + bar * count * proportion + x(continent) +  
  title("Share of countries by continent")
```

Python

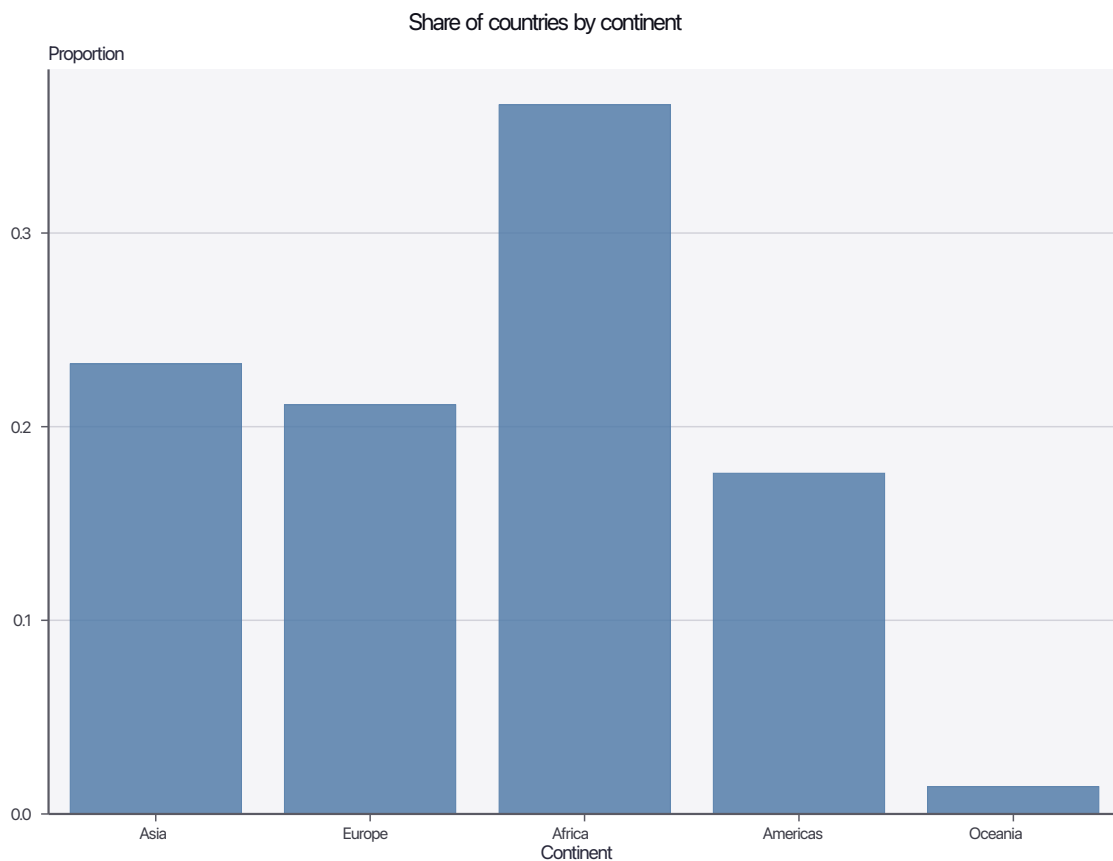
```
(data(gm_all) + bar * count * proportion + x(col.continent) +  
  title("Share of countries by continent"))
```

Julia

```
data(gm_all) + bar * count * proportion + x(:continent) +  
  title("Share of countries by continent")
```

JavaScript

```
plot(data(gm_all), layer(bar, count, proportion), x(col.continent),  
  title("Share of countries by continent"))
```



“Given all the gapminder years: bars derived by count and proportion, x is continent.”

It also prints a warning on your console. This page cannot show you that warning, because only refusals appear in a rendered book. It reads: *“gog: bar * count * proportion draws the same plot as bar * proportion — a share is a share of a tally, so proportion already counts the rows and count adds nothing. Drop it, or keep bar * count on its own to read the tallies themselves rather than their shares.”*

That is the line between the two answers. When keeping both transforms would mean throwing one away, the engine refuses. When both can stay and one of them simply changes nothing, it draws and says so. Neither answer is silence, which is the rule underneath both: a word you wrote is never read and then ignored.

25.8 Output columns

Every transform that computes a value writes its answer into a named column. This table gives those names.

Transform	Reads from	Writes to
<code>bin</code>	<code>x(field)</code>	<code>x(field)</code> (bin centers), <code>y(your_name)</code> (counts)
<code>count</code>	<code>x(field)</code>	<code>x(field)</code> (unique values), <code>y(your_name)</code> (counts)
<code>proportion</code>	whatever measured, or <code>x(field)</code> if nothing did	the same column, divided by its total (sums to 1)
<code>smooth</code>	<code>x(field)</code> , <code>y(field)</code>	same <code>x</code> , same <code>y</code> (smoothed values)
<code>density</code>	<code>x(field)</code>	<code>x(field)</code> (eval points), <code>y(your_name)</code> (density)
<code>sum / mean / median / max / min</code>	<code>x(field)</code> , <code>y(field)</code>	<code>x(field)</code> (unique), <code>y(field)</code> (aggregated)
<code>range</code>	<code>x(field)</code> , <code>y(field)</code>	<code>x(field)</code> (each group twice), <code>y(field)</code> (low, then high)
<code>confidence</code>	<code>x(field)</code> , <code>y(field)</code>	<code>x(field)</code> (each group twice), <code>y(field)</code> (low, then high), <code>center</code> (the mean)

For `bin`, `count`, `proportion`, and `density`, the output `y` column name is whatever you bind with `y()` in the spec, or an empty string when `y()` is omitted (the renderer finds it either way).

The y-axis label defaults to the transform name (“Count”, “Proportion”, “Density”) when `y()` is absent.

One row of that table changes when `bin` is composed with a statistic. Cutting is then all it does: it writes the bin centers to `x(field)` as before and leaves `y(field)` alone for the statistic to reduce, so the y column holds your column’s summary rather than a count.

25.9 smooth: LOESS trend line

The inventing transforms made new measurements. The **reading** transforms are the other half of the sort: each one reads a column you name and answers with a summary of it, and `smooth` is the first.

`smooth` fits a locally weighted linear regression (LOESS) through the (x, y) points and evaluates it at 100 evenly spaced x positions (Cleveland, 1979). The curve is the engine’s own, with no robustness pass, so it is identical in all four languages and differs from `geom_smooth()`, whose default fit is a local quadratic.

Combine it with `line` to overlay a trend line on a scatter plot:

R

```
data(gapminder_2007) + x(gdp) + y(life) +  
  point + color(continent) +  
  line * smooth +  
  title("GDP vs Life Expectancy with LOESS Smooth")
```

Python

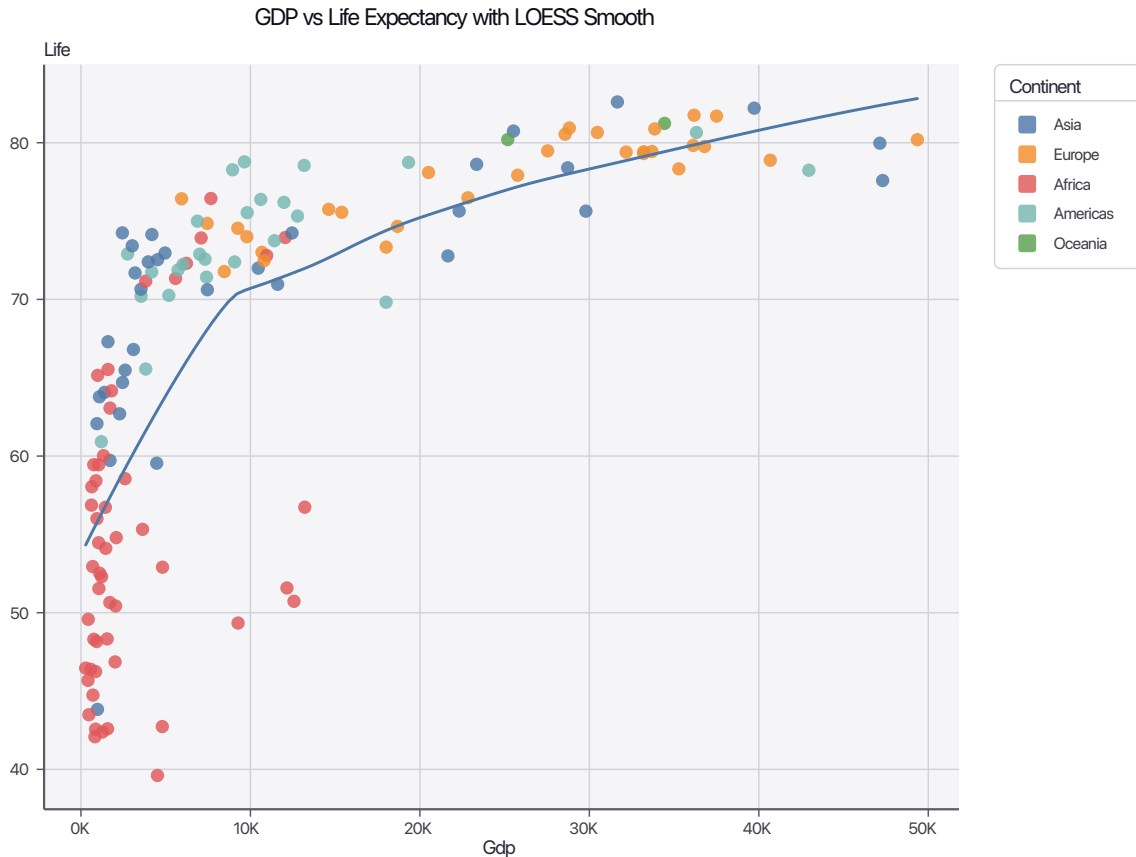
```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +  
  point + color(col.continent) +  
  line * smooth +  
  title("GDP vs Life Expectancy with LOESS Smooth"))
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + point + color(:continent) +  
  line * smooth + title("GDP vs Life Expectancy with LOESS Smooth")
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), point,  
     color(col.continent), layer(line, smooth),  
     title("GDP vs Life Expectancy with LOESS Smooth"))
```



“Given gapminder 2007: x is gdp, y is life, points colored by continent, and also a line derived by smooth.”

Span defaults to 0.75: 75% of the data is used for each local regression. A larger span gives a smoother curve; a smaller span follows the data more closely. **smooth** requires both x and y to be numeric columns, and at least three rows in each group it fits; both refusals are earlier in this chapter, under [what these three refuse](#). It preserves the field names, so no extra $y()$ binding is needed.

25.10 Aggregation family: sum, mean, median, max, min, quantile

The six summaries are the reading family at its plainest. When a table has multiple rows for the same x value (for example, medals from different years, all labeled “USA”), `bar` alone draws one bar per row, and the bars for one country are drawn over each other in one slot.

The aggregation family groups rows by x and reduces y to a single value per group. All six follow exactly the same grammar:

```
bar * sum      + x(country) + y(gold)
bar * mean     + x(country) + y(gold)
bar * median   + x(country) + y(gold)
bar * max      + x(country) + y(gold)
bar * min      + x(country) + y(gold)
bar * quantile(0.9) + x(country) + y(gold)
```

sum, total y per group, the most common aggregation for counts and amounts:

R

```
medals_dup <- data.frame(
  country = c("USA", "USA", "GBR", "GBR", "JPN"),
  gold    = c(10, 5, 8, 4, 6)
)
data(medals_dup) + bar * sum + x(country) + y(gold) +
  order(gold, desc = TRUE) +
  y_label("Total gold medals") + title("Sum per country: bar * sum")
```

Python

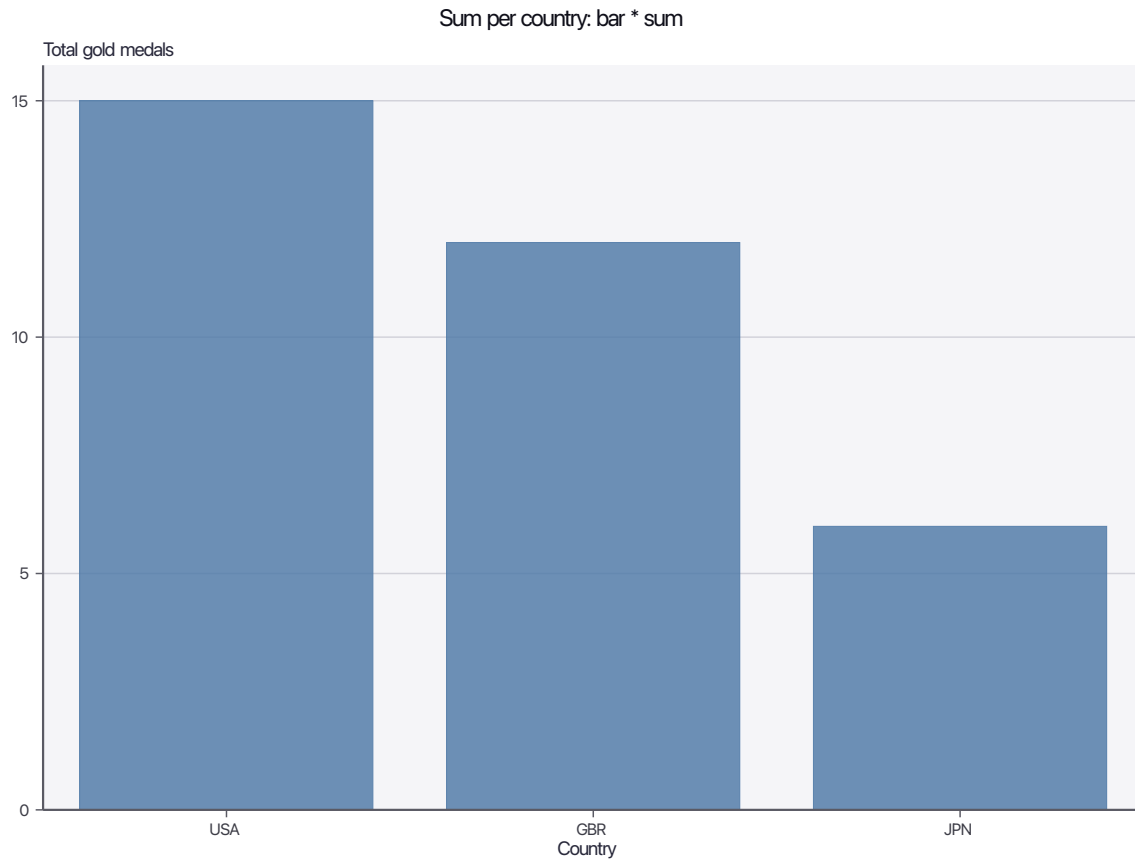
```
medals_dup = {"country": ["USA", "USA", "GBR", "GBR", "JPN"], "gold": [10, 5,
↪ 8, 4, 6]}
(data(medals_dup) + bar * sum + x(col.country) + y(col.gold) +
  order(col.gold, desc = True) +
  y_label("Total gold medals") + title("Sum per country: bar * sum"))
```

Julia

```
medals_dup = (country = ["USA", "USA", "GBR", "GBR", "JPN"], gold = [10, 5,
↪ 8, 4, 6],)
data(medals_dup) + bar * sum + x(:country) + y(:gold) +
  order(:gold, desc = true) + y_label("Total gold medals") +
  title("Sum per country: bar * sum")
```

JavaScript

```
const medals_dup = { country: ["USA", "USA", "GBR", "GBR", "JPN"], gold: [10,
  ↪ 5, 8, 4, 6] };
plot(data(medals_dup), layer(bar, sum), x(col.country), y(col.gold),
  order(col.gold, { desc: true }), y_label("Total gold medals"),
  title("Sum per country: bar * sum"))
```



“Given the medals dup table: bars derived by sum, x is country, y is gold, ordered by gold, largest first.”

mean, average y per group, useful for comparing typical values:

R

```
data(medals_dup) + bar * mean + x(country) + y(gold) +  
  order(gold, desc = TRUE) +  
  y_label("Mean gold medals") + title("Mean per country: bar * mean")
```

Python

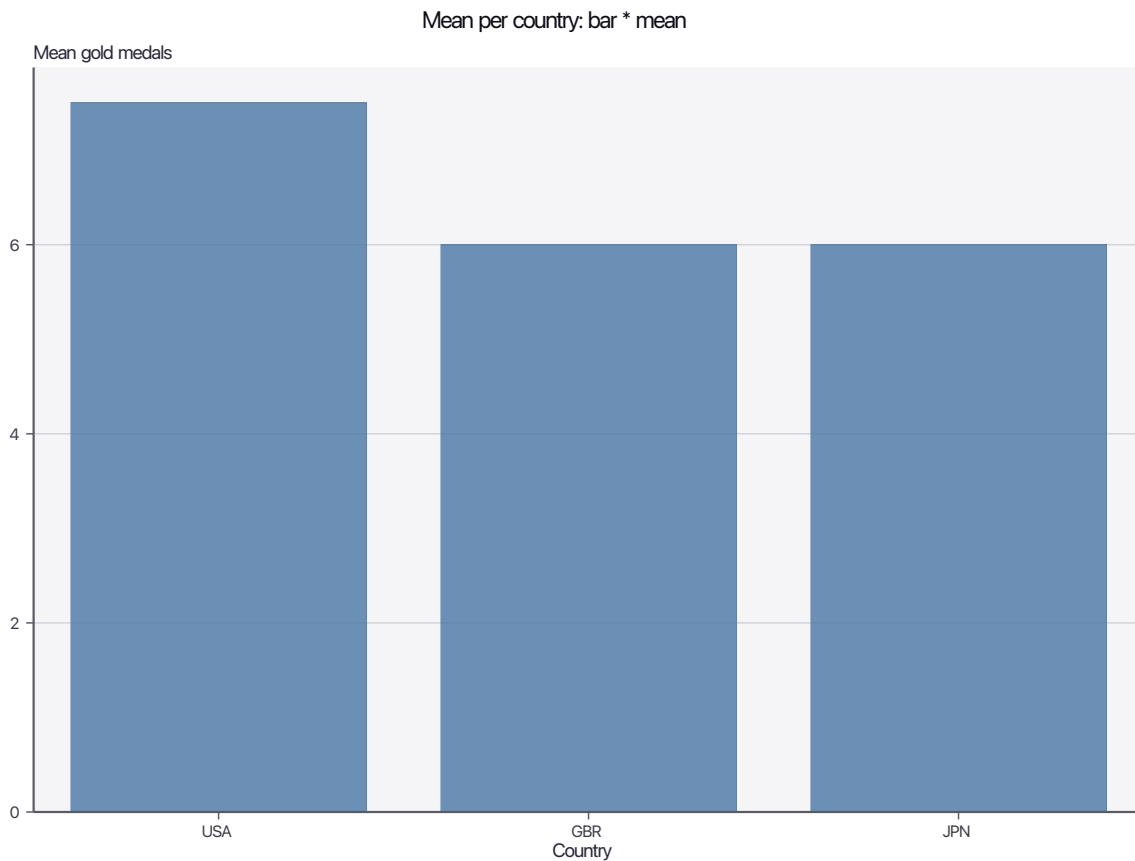
```
(data(medals_dup) + bar * mean + x(col.country) + y(col.gold) +  
  order(col.gold, desc = True) +  
  y_label("Mean gold medals") + title("Mean per country: bar * mean"))
```

Julia

```
data(medals_dup) + bar * mean + x(:country) + y(:gold) +  
  order(:gold, desc = true) + y_label("Mean gold medals") +  
  title("Mean per country: bar * mean")
```

JavaScript

```
plot(data(medals_dup), layer(bar, mean), x(col.country), y(col.gold),  
  order(col.gold, { desc: true }), y_label("Mean gold medals"),  
  title("Mean per country: bar * mean"))
```



median, the middle value, robust to outliers:

R

```
data(medals_dup) + bar * median + x(country) + y(gold) +
  order(gold, desc = TRUE) +
  y_label("Median gold") + title("Median per country: bar * median")
```

Python

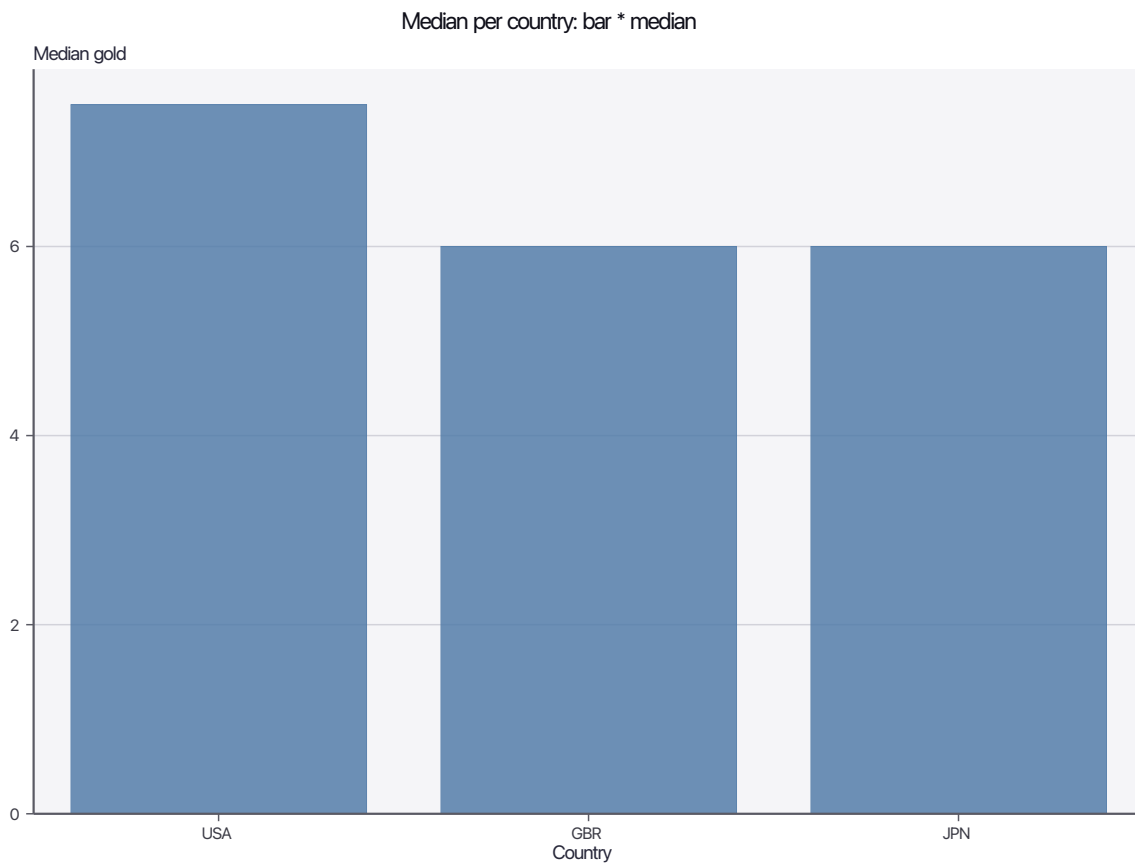
```
(data(medals_dup) + bar * median + x(col.country) + y(col.gold) +
  order(col.gold, desc = True) +
  y_label("Median gold") + title("Median per country: bar * median"))
```

Julia

```
data(medals_dup) + bar * median + x(:country) + y(:gold) +
  order(:gold, desc = true) + y_label("Median gold") +
  title("Median per country: bar * median")
```

JavaScript

```
plot(data(medals_dup), layer(bar, median), x(col.country), y(col.gold),
  order(col.gold, { desc: true }), y_label("Median gold"),
  title("Median per country: bar * median"))
```



max, the largest value within each group:

R

```
data(medals_dup) + bar * max + x(country) + y(gold) +  
  order(gold, desc = TRUE) +  
  y_label("Max gold") + title("Max per country: bar * max")
```

Python

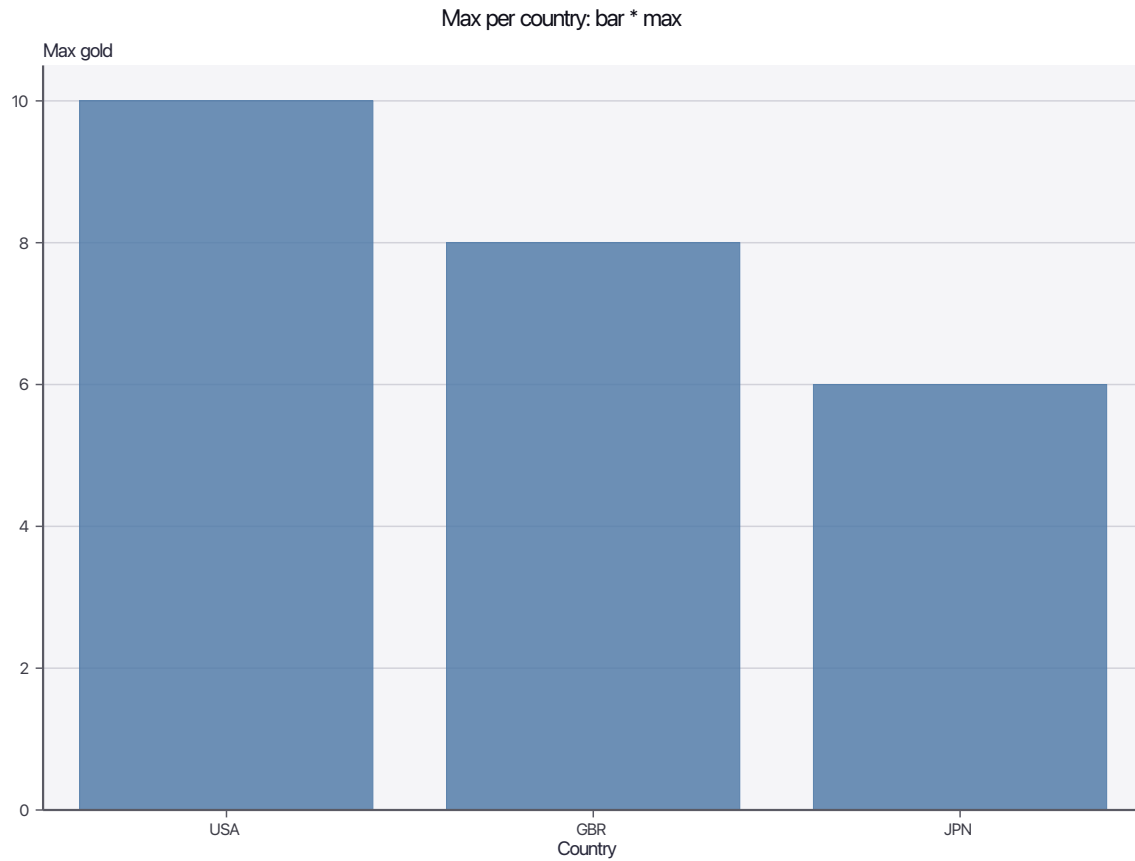
```
(data(medals_dup) + bar * max + x(col.country) + y(col.gold) +  
  order(col.gold, desc = True) +  
  y_label("Max gold") + title("Max per country: bar * max"))
```

Julia

```
data(medals_dup) + bar * max + x(:country) + y(:gold) +  
  order(:gold, desc = true) + y_label("Max gold") +  
  title("Max per country: bar * max")
```

JavaScript

```
plot(data(medals_dup), layer(bar, max), x(col.country), y(col.gold),  
  order(col.gold, { desc: true }), y_label("Max gold"),  
  title("Max per country: bar * max"))
```



“Given the medals_dup table: bars derived by max, x is country, y is gold, ordered by gold, largest first.”

min, the smallest value within each group:

R

```
data(medals_dup) + bar * min + x(country) + y(gold) +
  order(gold, desc = TRUE) +
  y_label("Min gold") + title("Min per country: bar * min")
```

Python

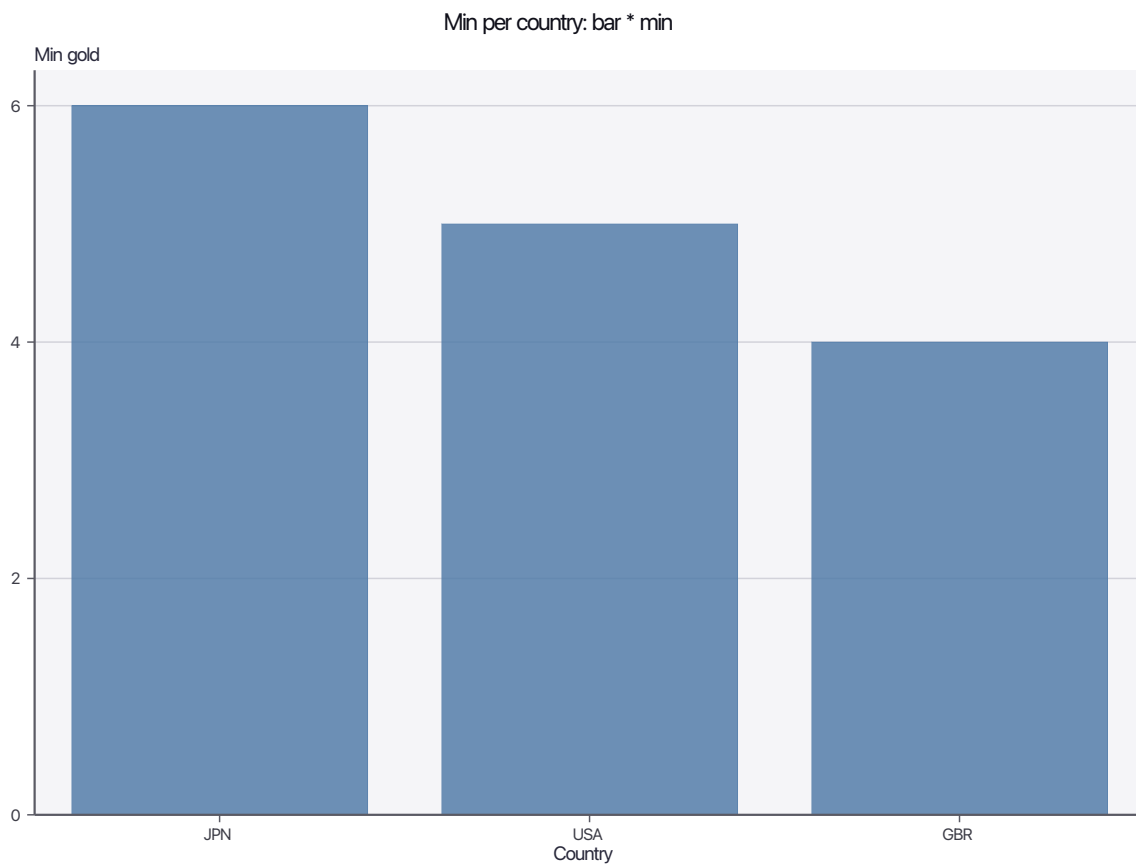
```
(data(medals_dup) + bar * min + x(col.country) + y(col.gold) +
  order(col.gold, desc = True) +
  y_label("Min gold") + title("Min per country: bar * min"))
```

Julia

```
data(medals_dup) + bar * min + x(:country) + y(:gold) +  
  order(:gold, desc = true) + y_label("Min gold") +  
  title("Min per country: bar * min")
```

JavaScript

```
plot(data(medals_dup), layer(bar, min), x(col.country), y(col.gold),  
  order(col.gold, { desc: true }), y_label("Min gold"),  
  title("Min per country: bar * min"))
```



“Given the medals dup table: bars derived by min, x is country, y is gold, ordered by gold, largest first.”

All six aggregations work with both string and numeric x columns. For string x, groups preserve first-appearance order (before any **order**); for numeric x, groups are sorted ascending.

Transform	What it computes per group
<code>sum</code>	total of y values
<code>mean</code>	arithmetic average of y
<code>median</code>	middle value of y
<code>max</code>	largest y value
<code>min</code>	smallest y value
<code>quantile(p)</code>	the y at probability p

25.10.1 The same six, grouping by a pair

`count` follows a bar onto a zone and tallies two axes instead of one, and these six follow it, for the same reason and by the same rule. A bar has a length to show the answer along, so it groups by one position and measures along the other; a **zone** measures nothing by length, so both its positions are free to be grouping columns and the answer goes to `color`:

R

```
data(winds) + zone * mean + x(direction) + y(season) + color(speed) +
  title("The same mean, grouped by a pair")
```

Python

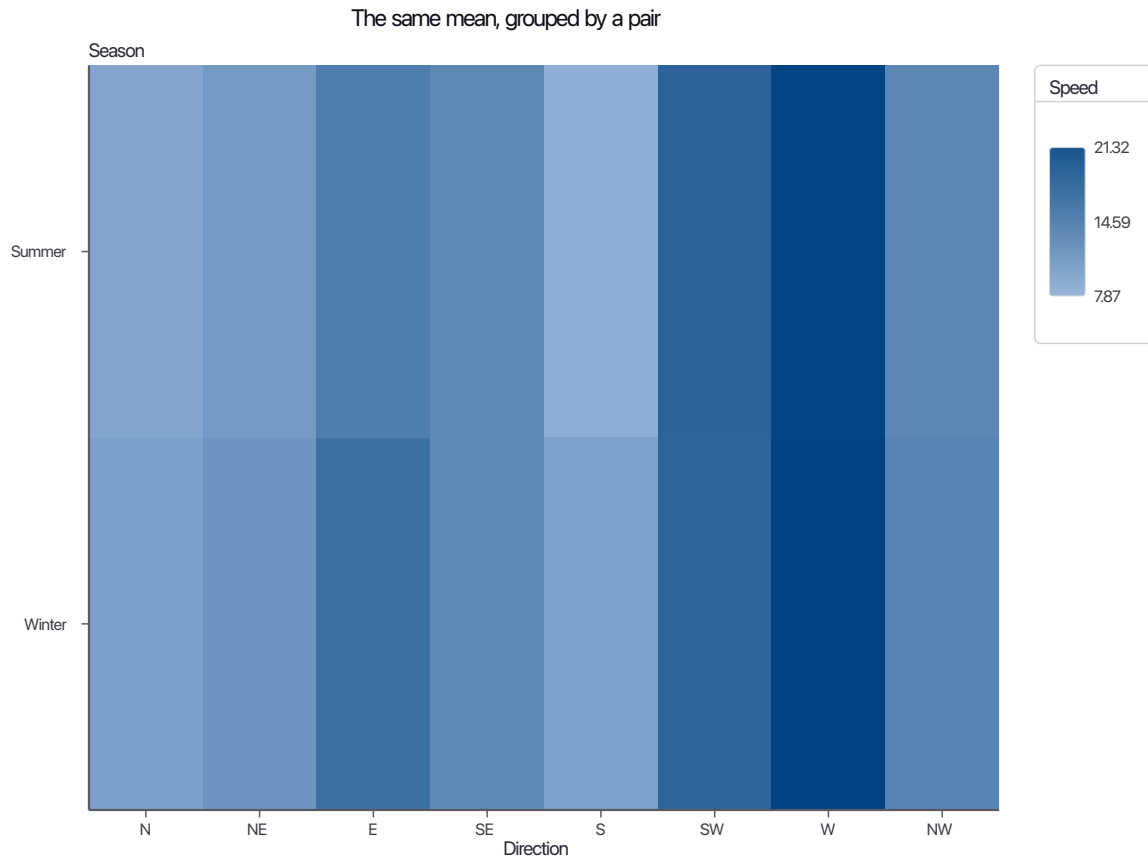
```
(data(winds) + zone * mean + x(col.direction) + y(col.season) +
  ↪ color(col.speed) +
  title("The same mean, grouped by a pair"))
```

Julia

```
data(winds) + zone * mean + x(:direction) + y(:season) + color(:speed) +
  title("The same mean, grouped by a pair")
```

JavaScript

```
plot(data(winds), layer(zone, mean), x(col.direction), y(col.season),
  color(col.speed), title("The same mean, grouped by a pair"))
```



What tells the transform which column to reduce is the same channel the answer comes back on. That is not a special case for the zone: a summary reduces **in place** everywhere, and on `bar * mean + x(country) + y(gold)` it is `y` that both names `gold` and receives the average. A zone measures by color, so color does both jobs there.

So the rule is one sentence, and it is the dimensional rule read a second time: a summary groups by every position the mark does not measure with, and reduces the column named on the one it does.

mark	positions	measures with	groups by	reduces
bar	x, y	y	x	y
bar in <code>space()</code>	x, y, z	z	x, y	z
surface	x, y, z	z	x, y	z
zone	x, y	color	x, y	color

The middle two rows are identical, and that is the rule working rather than a coincidence: a 3-D bar and a surface have the same three positions and measure along the same one, so

they group and reduce alike. What differs is only what each lays on the cell the pair names, a column or a plateau.

The division from **count**, **bin** and **density** is not about how many axes anything can reach: all of them read a mesh. It is about what each was handed. Those three invent their measurement, so they need no column named and publish their answer under a name of their own; these six are given a column, so the sentence has to say which one, and the answer replaces it. **proportion** sits outside the division because it is handed neither: it takes whatever measurement the sentence already made and divides it.

25.10.2 Why these names?

One naming question is worth settling before the six are used. **count**, **sum**, **mean**, **median**, **max** and **min** each name an operation on y, not a data type and not a chart name. The ***** operator makes this explicit: **bar * sum** reads as “a bar that sums”. Every aggregation is the word a statistics course already uses (Law 3: Plain Names).

25.11 quantile: one point in the distribution

sum, **mean**, **median**, **max** and **min** each answer a fixed question. **quantile** takes the question as its argument: **quantile(0.9)** is the value nine tenths of the group falls below, which is the **90th percentile**.

Many reported numbers have exactly that form. A service level is the 95th percentile of response time. A growth chart is a set of percentiles by age. A pay band is usually quoted at the 10th, 50th and 90th percentiles. Here are three of them across every country, year by year:

R

```
data(gm_all) + x(year) + y(life) +  
  line * quantile(0.9) + style(color = "steelblue") +  
  line * median + style(color = "gray") +  
  line * quantile(0.1) + style(color = "tomato") +  
  y_label("Life expectancy") +  
  title("The 90th percentile, the median, and the 10th")
```

Python

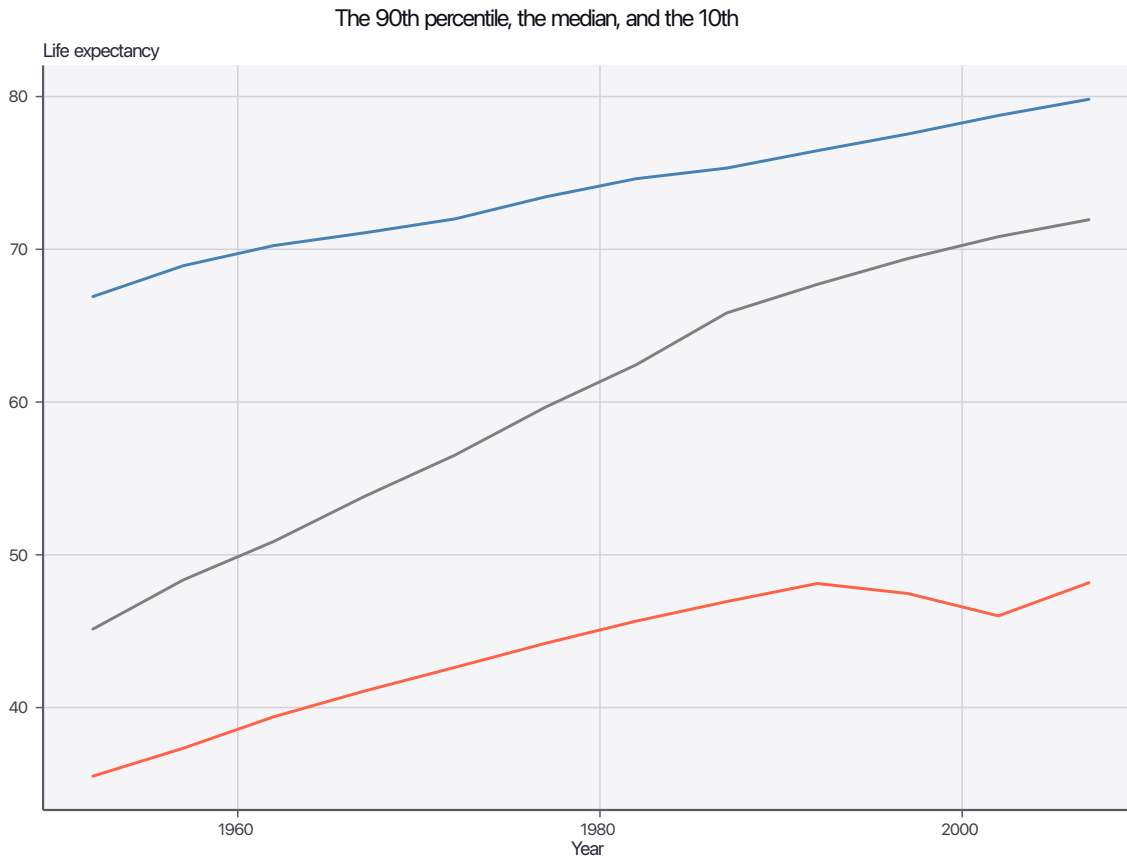
```
(data(gm_all) + x(col.year) + y(col.life) +
  line * quantile(0.9) + style(color = "steelblue") +
  line * median + style(color = "gray") +
  line * quantile(0.1) + style(color = "tomato") +
  y_label("Life expectancy") +
  title("The 90th percentile, the median, and the 10th"))
```

Julia

```
data(gm_all) + x(:year) + y(:life) + line * quantile(0.9) +
  style(color = "steelblue") + line * median + style(color = "gray") +
  line * quantile(0.1) + style(color = "tomato") +
  y_label("Life expectancy") +
  title("The 90th percentile, the median, and the 10th")
```

JavaScript

```
plot(data(gm_all), x(col.year), y(col.life), layer(line, quantile(0.9)),
  style({ color: "steelblue" }), layer(line, median),
  style({ color: "gray" }), layer(line, quantile(0.1)),
  style({ color: "tomato" }), y_label("Life expectancy"),
  title("The 90th percentile, the median, and the 10th"))
```



“Given all the gapminder years: x is year, y is life, a line derived by quantile at 0.9, colored steelblue, and also a line derived by median, colored gray, and also a line derived by quantile at 0.1, colored tomato.”

The three lines are three sentences over one table, and they say different things about the same 142 countries. The middle rose steadily. The top rose more slowly, because it was already high. The bottom rose and then declined for a decade, which neither of the others shows.

`quantile` has no default. Every transform whose parameter is a number works bare, and this one is refused:

```
data(gm_all) + line * quantile + x(year) + y(life)
```

Error:

! gog: `quantile` needs the probability it reduces to, e.g. `quantile(0.9)`

↪ for the 90th percentile. There is no default, because the only sensible

↪ one is the middle and that already has a plain name: `median`.

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

The only sensible default is the middle, and the middle already has a plain name. That is also why three particular values get a remark rather than a refusal:

R

```
data(gm_all) + line * quantile(0.5) + x(year) + y(life) +  
  y_label("Life expectancy") + title("The median, written as a quantile")
```

Python

```
(data(gm_all) + line * quantile(0.5) + x(col.year) + y(col.life) +  
  y_label("Life expectancy") + title("The median, written as a quantile"))
```

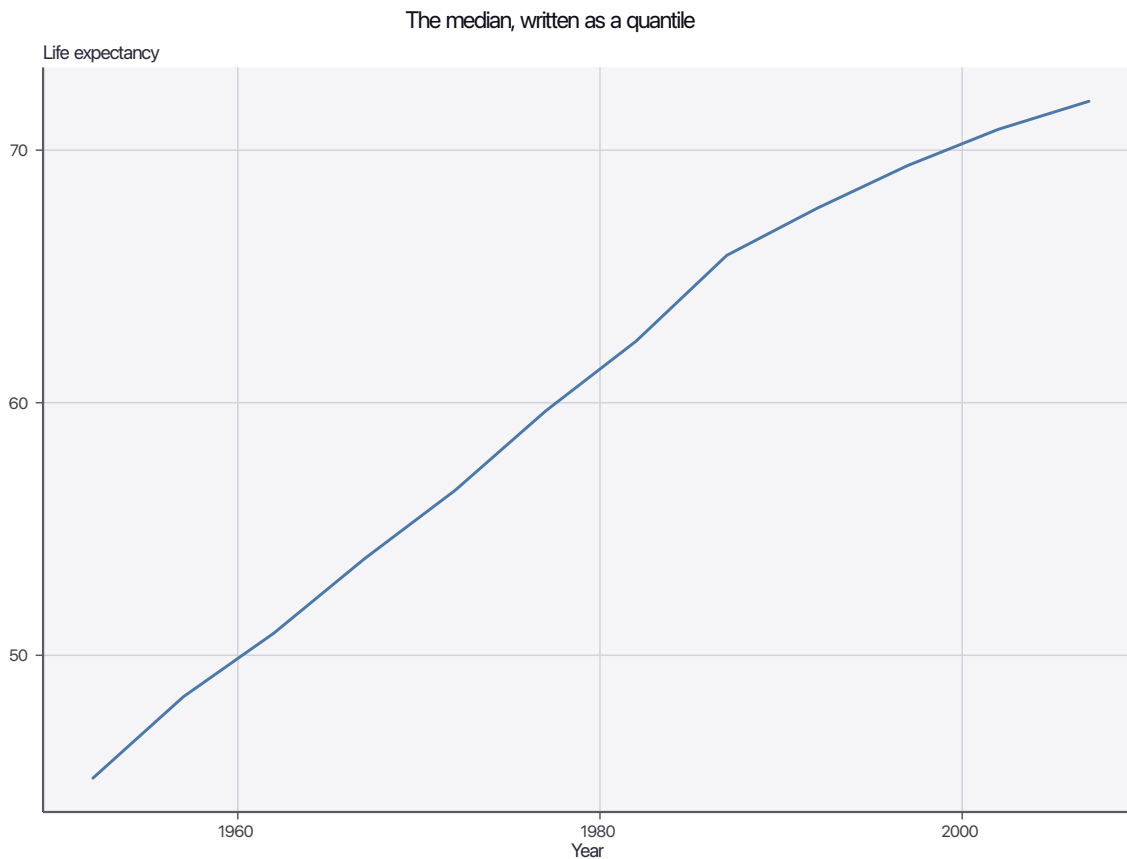
Julia

```
data(gm_all) + line * quantile(0.5) + x(:year) + y(:life) +  
  y_label("Life expectancy") + title("The median, written as a quantile")
```

JavaScript

```
plot(data(gm_all), layer(line, quantile(0.5)), x(col.year), y(col.life),  
  y_label("Life expectancy"), title("The median, written as a quantile"))
```

gog: `quantile(0.5)` is the median - write `median` for the plain name. Drawn
↪ as asked.



The plot draws, and `gog` says that `median` is the plain name for it. `quantile(0)` and `quantile(1)` say the same about `min` and `max`. They are not refused, because a program stepping through the deciles would stop at 0.5 for no reason. They are not silent either, because [Orthogonality](#) says no atom is redundant.

25.12 range: a low/high pair

Every transform above writes a single measured value per group. `range` is the first to write *two*: the minimum and the maximum of `y` within each group, emitted as a low then a high. That pair is exactly what an [interval](#) spans:

R

```
data(iris_flowers) + interval * range + x(species) + y(petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Petal length range per species")
```

Python

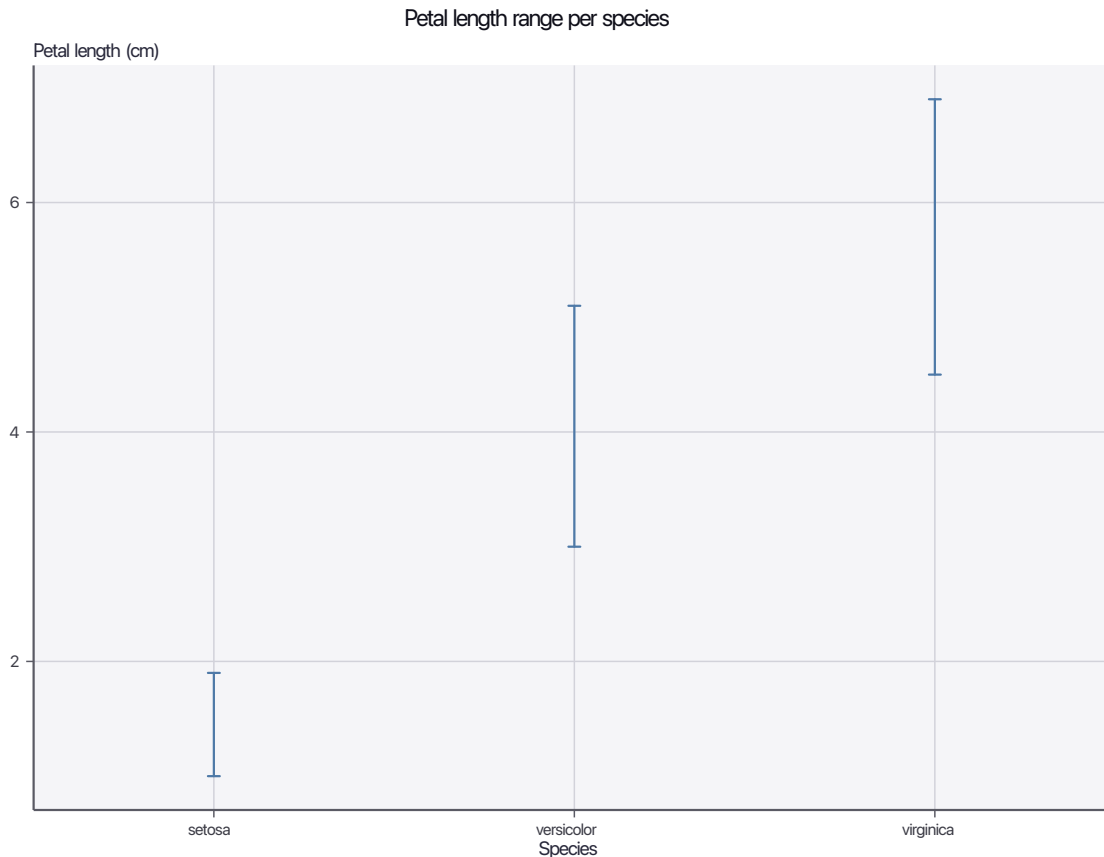
```
(data(iris_flowers) + interval * range + x(col.species) + y(col.petal_length)
↪ +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Petal length range per species"))
```

Julia

```
data(iris_flowers) + interval * range + x(:species) + y(:petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Petal length range per species")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, range), x(col.species),
  y(col.petal_length), x_label("Species"), y_label("Petal length (cm)"),
  title("Petal length range per species"))
```

“Given the iris flowers: intervals derived by range, x is species, y is petal length.”

`range` reads a `y` and groups by `x` exactly like the aggregation family: it needs a `y()` in scope and works on string or numeric `x` alike. What differs is the *output*: two rows per group instead of one. This low/high-rows shape is reused elsewhere: the `box` mark encodes its own five-number summary the same way (min and max as the pair of rows, the quartiles alongside), and the `ribbon` mark fills between the same pair as a continuous band. See [Interval](#) for the whisker, and for why its extents are invented here rather than bound to two extra channels the kernel would never otherwise have.

25.12.1 A band between two quantiles

The minimum and the maximum each come from a single row, so one unusual row moves the whole span. To show the middle of a group instead of its extremes, give `range` two **quantiles**:

R

```
data(iris_flowers) + interval * range(0.25, 0.75) + x(species) +
  ↪ y(petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Middle half of petal length per species")
```

Python

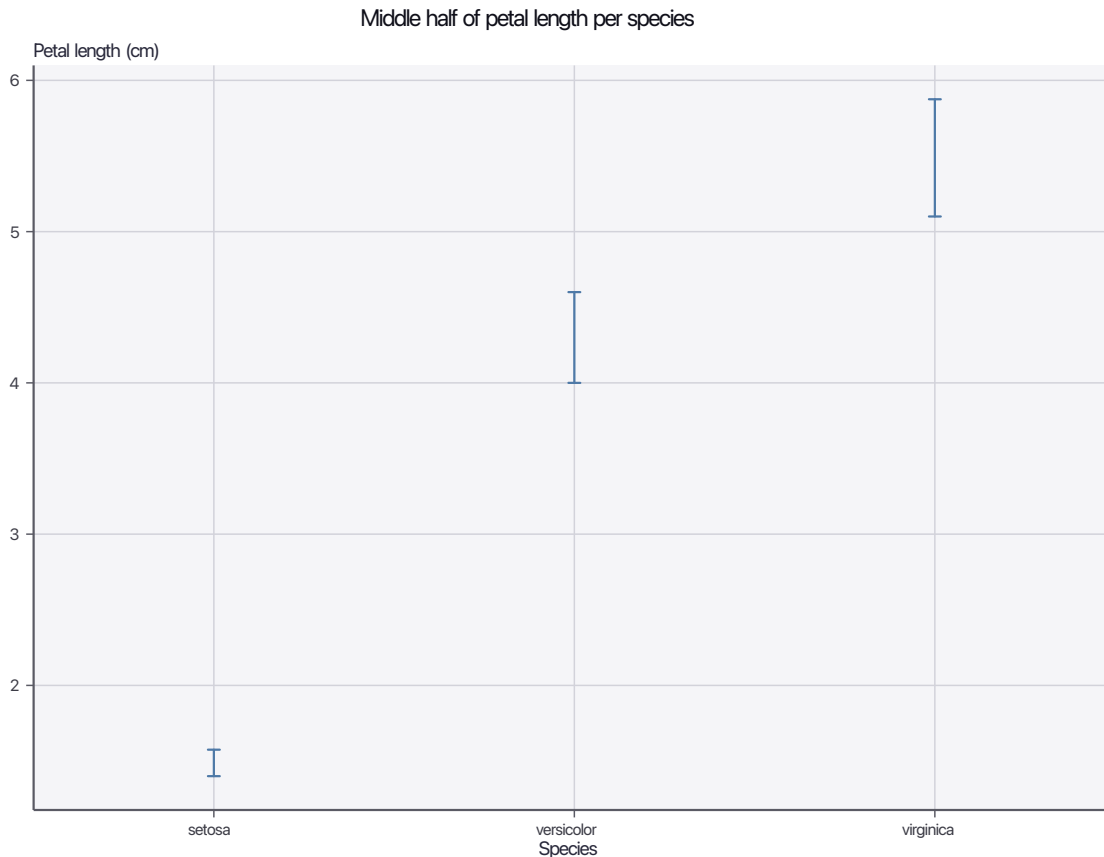
```
(data(iris_flowers) + interval * range(0.25, 0.75) + x(col.species) +
  ↪ y(col.petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("Middle half of petal length per species"))
```

Julia

```
data(iris_flowers) + interval * range(0.25, 0.75) + x(:species) +
  y(:petal_length) + x_label("Species") + y_label("Petal length (cm)") +
  title("Middle half of petal length per species")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, range(0.25, 0.75)),
  x(col.species), y(col.petal_length), x_label("Species"),
  y_label("Petal length (cm)",
  title("Middle half of petal length per species"))
```



“Given the iris flowers: intervals derived by range from 0.25 to 0.75, x is species, y is petal length.”

The two numbers are quantile probabilities, so `range(0.25, 0.75)` spans the first quartile to the third. That span is the **interquartile range**, and it is the same one a `box` draws as its body, computed by the same rule. Any other pair works the same way: `range(0.1, 0.9)` keeps the middle 80%, and `range(0.05, 0.95)` the middle 90%.

Bare `range` is `range(0, 1)`, because the quantile at 0 is the minimum and the quantile at 1 is the maximum. So `interval * range` is unchanged, and the two arguments only make its default visible. Naming one end leaves the other at its extreme, so `range(high = 0.9)` runs from the smallest value to the 90th percentile.

Both ends are probabilities. A number below 0 or above 1 is refused, not corrected:

```
data(iris_flowers) + interval * range(0.25, 75) + x(species) +
  ↪ y(petal_length)
```

Error:

```
! gog: `range(high = 75)` is not a probability - the band's ends are
  ↳ quantiles, so each is between 0 and 1. `range(0.25, 0.75)` is the middle
  ↳ half, `range(0.1, 0.9)` the middle 80 percent, and bare `range` the whole
  ↳ group.
```

For a low and a high you computed yourself, in the data's own units, use [bounds](#) instead.

25.13 confidence: the interval of the mean

`range` shows the *full* spread; often you want the **uncertainty of the mean** instead: an error bar. `confidence` computes the mean's confidence interval per group (mean $\pm t \cdot se$, with `se` the standard error: the t-interval), emitting a low, a high, *and* a center. That center is what turns the whisker into a pointrange:

R

```
data(iris_flowers) + interval * confidence(0.95) + x(species) +
  ↳ y(petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("95% CI of mean petal length")
```

Python

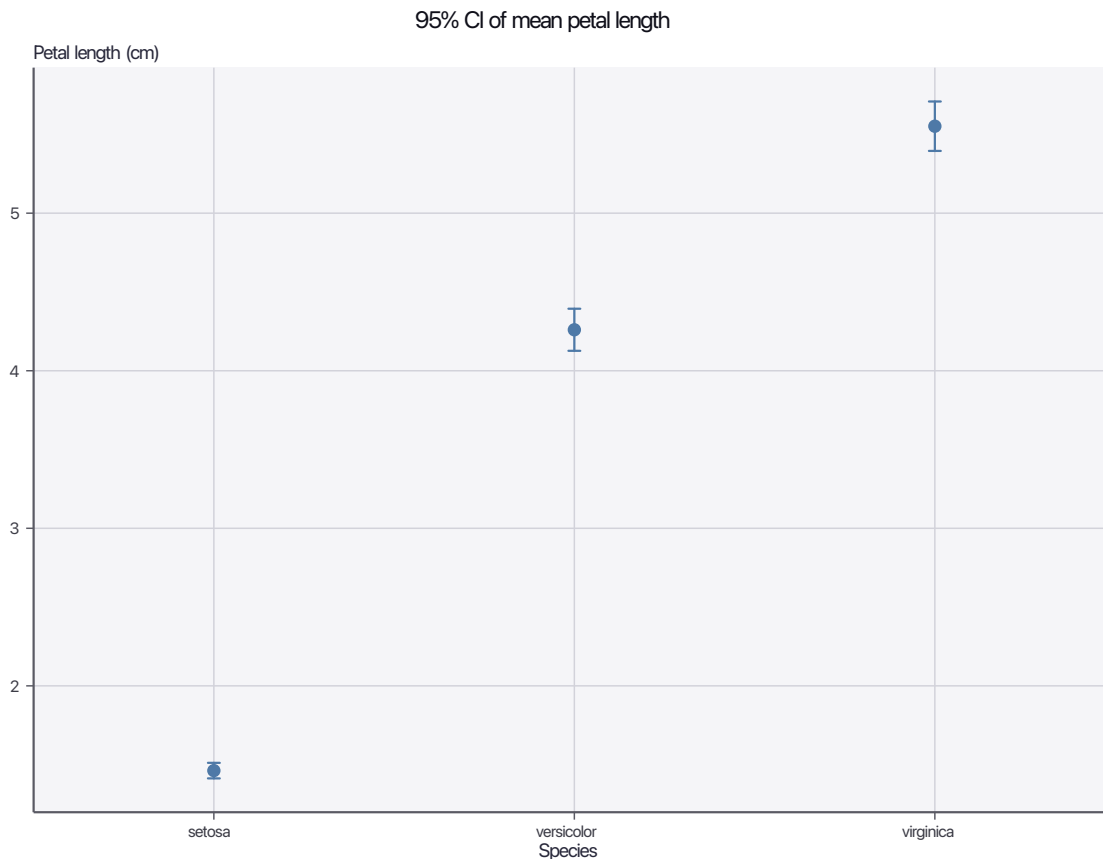
```
(data(iris_flowers) + interval * confidence(0.95) + x(col.species) +
  ↳ y(col.petal_length) +
  x_label("Species") + y_label("Petal length (cm)") +
  title("95% CI of mean petal length"))
```

Julia

```
data(iris_flowers) + interval * confidence(0.95) + x(:species) +
  y(:petal_length) + x_label("Species") + y_label("Petal length (cm)") +
  title("95% CI of mean petal length")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, confidence(0.95)),
     x(col.species), y(col.petal_length), x_label("Species"),
     y_label("Petal length (cm)", title("95% CI of mean petal length"))
```



“Given the iris flowers: intervals derived by confidence at 95%, x is species, y is petal length.”

The level is a parameter, exactly like `bin(30)` and `density(2)`: `confidence` alone is 95%, `confidence(0.99)` is wider. It uses the *t*-distribution (not a fixed 1.96), so a small group’s interval is honestly wider, and a group of one, no spread, collapses to a point.

The center dot appears because the statistic *has* a center; `range` has none, so it draws a bare whisker. That is the rule for the whole family: the transform decides whether there is a center, and the mark draws what it is given. So the difference between an error bar and a pointrange is which statistic you ask for, not another mark to remember.

25.14 deviation: the spread of the data

`confidence` says how precisely a mean is known. `deviation` says how spread the values behind it are, which is a different question with the same picture. It emits the same low, high and center, so it also draws a pointrange:

R

```
data(iris_flowers) + interval * deviation + x(species) + y(petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Mean petal length, with one standard deviation")
```

Python

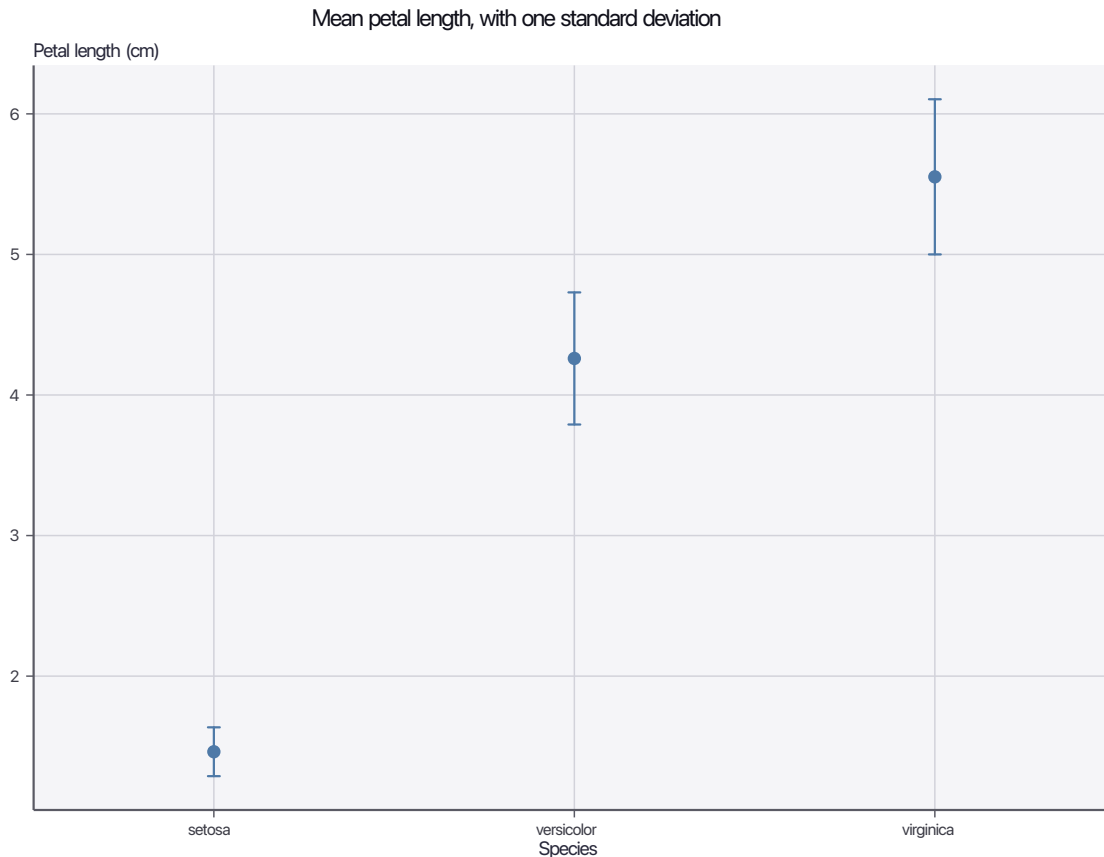
```
(data(iris_flowers) + interval * deviation + x(col.species) +  
↪ y(col.petal_length) +  
  x_label("Species") + y_label("Petal length (cm)") +  
  title("Mean petal length, with one standard deviation"))
```

Julia

```
data(iris_flowers) + interval * deviation + x(:species) +  
  y(:petal_length) + x_label("Species") + y_label("Petal length (cm)") +  
  title("Mean petal length, with one standard deviation")
```

JavaScript

```
plot(data(iris_flowers), layer(interval, deviation), x(col.species),  
  y(col.petal_length), x_label("Species"), y_label("Petal length (cm)"),  
  title("Mean petal length, with one standard deviation"))
```



“Given the iris flowers: intervals derived by deviation, x is species, y is petal length.”

The multiplier is a parameter, like **confidence**’s level: **deviation** alone reaches one standard deviation on each side of the mean, and **deviation(2)** reaches two, so that band is four standard deviations wide. The standard deviation is the sample one, dividing by $n - 1$, so a group of one has no spread to show and collapses to a point exactly as **confidence** does.

Neither transform makes the other redundant, and neither does **range**. On the iris flowers the confidence band is under a third as long as the deviation band: fifty measurements locate a mean precisely, and they still scatter as widely as they scatter. Adding rows narrows the first band and does not change the second. **Interval** draws the two side by side and says which sentence each one supports.

range(0.25, 0.75) answers a third version of the question. It is rank-based, so it reports where the middle half of the values actually lie, while **deviation** is moment-based and reports a distance from the mean. On a skewed group they disagree, and the disagreement is informative rather than a defect.

25.15 bounds: a *pre-computed* low/high pair

`range` and `confidence compute` the pair from raw values. `bounds` is their counterpart that computes nothing. You already have the low and the high in two columns, worked out upstream: a model's standard error, a psychometric SEM, a bootstrap interval. `bounds(lower, upper)` reshapes those two columns into the very same low/high pair, so a `ribbon` or `interval` draws them with no new machinery:

R

```
data(score_band) + ribbon * bounds(lower, upper) + x(score) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("A pre-computed band: ribbon with bounds")
```

Python

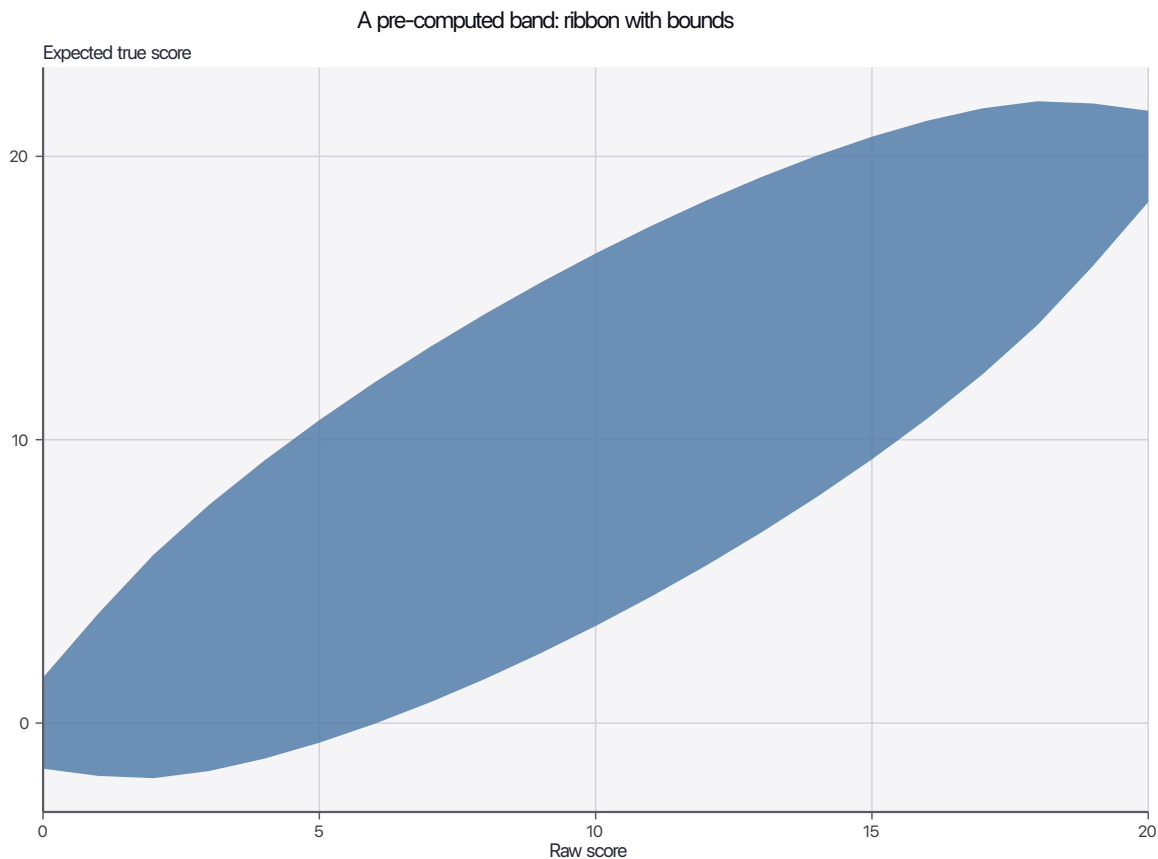
```
(data(score_band) + ribbon * bounds(col.lower, col.upper) + x(col.score) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("A pre-computed band: ribbon with bounds"))
```

Julia

```
data(score_band) + ribbon * bounds(:lower, :upper) + x(:score) +  
  y_label("Expected true score") + x_label("Raw score") +  
  title("A pre-computed band: ribbon with bounds")
```

JavaScript

```
plot(data(score_band), layer(ribbon, bounds(col.lower, col.upper)),  
  x(col.score), y_label("Expected true score"), x_label("Raw score"),  
  title("A pre-computed band: ribbon with bounds"))
```

“Given the score band: a ribbon from lower to upper, x is score.”

It differs from every transform above in two ways that both follow from *reshaping, not computing*. Its arguments are *column names*, not a number: it has to be told which columns hold the bounds. And it needs *no $y()$* : those two columns *are* the extents, so (like `count`, which invents its own y) the mark takes its low and high from `bounds` and the y -axis fits them. This is exactly where `gog` stops. It computes a deliberate few statistics; everything else you compute yourself, and `bounds` is how a band you already have gets drawn.

It is legal on the five marks that can draw a low/high pair, each a different geometry over the *same* statistic, the [No Exceptions](#) rule that gives one `bin` three histograms:

- `ribbon` fills the pair (a filled band),
- `interval` draws a **whisker** across it at each x ,
- `line` traces its **two boundaries** (the *unfilled* band),
- `step` traces them as **staircases** (stepped control limits), and
- `zone` shades the region **between them**, the band that names its own sides rather than tracking a curve.

So `line * bounds(lower, upper) + style(pattern = "dashed")` is the unfilled, dashed error band (the two $\pm$ curves with no fill between them) and it is the same pair, drawn by a different mark, that `ribbon * bounds` fills. On any mark that draws *neither* a pair nor a single value, `bounds` is refused with direction.

25.16 partition: a whole among nested parts

Nearly every transform so far reads a column and writes a summary of it; `bounds` alone reshapes what you already had. `partition` reads a **hierarchy** (two or three columns spelling the path down to each row) and writes the region each node of that hierarchy occupies: four edges and a center, which is exactly what `bin` publishes for a cell. That is what makes it a transform rather than a coordinate space, and it is why `zone` needs no new machinery to draw the result.

The levels can run two ways, and the difference is one parameter. **Nested** is the default: every level divides the same axis, and the other axis steps out one ring per level. On a flat plot that reading is the icicle. In `polar()` it is the sunburst, and that chapter works it through in full.

Crossed is the other, and it is the one this chapter draws, because it stays flat. The levels alternate: the first divides the width, the second divides the height *within* each of those columns, a third would divide the width again. The plot it makes is the **mosaic**.

25.16.1 The mosaic

`commutes` is a survey of how people get to work in four cities, one row per city and mode, with a count:

```
knitr::kable(head(commutes, 6))
```

city	mode	people
Ashford	Car	980
Ashford	Bus or tram	340
Ashford	Bicycle	110
Ashford	On foot	110
Brightwell	Car	420
Brightwell	Bus or tram	380

R

```
data(commutes) + zone * partition(city, mode, cross = TRUE) + x(people) +
  color(mode) + style(border_color = "white", border_size = 2) +
  x_label("People surveyed") + title("How four cities get to work")
```

Python

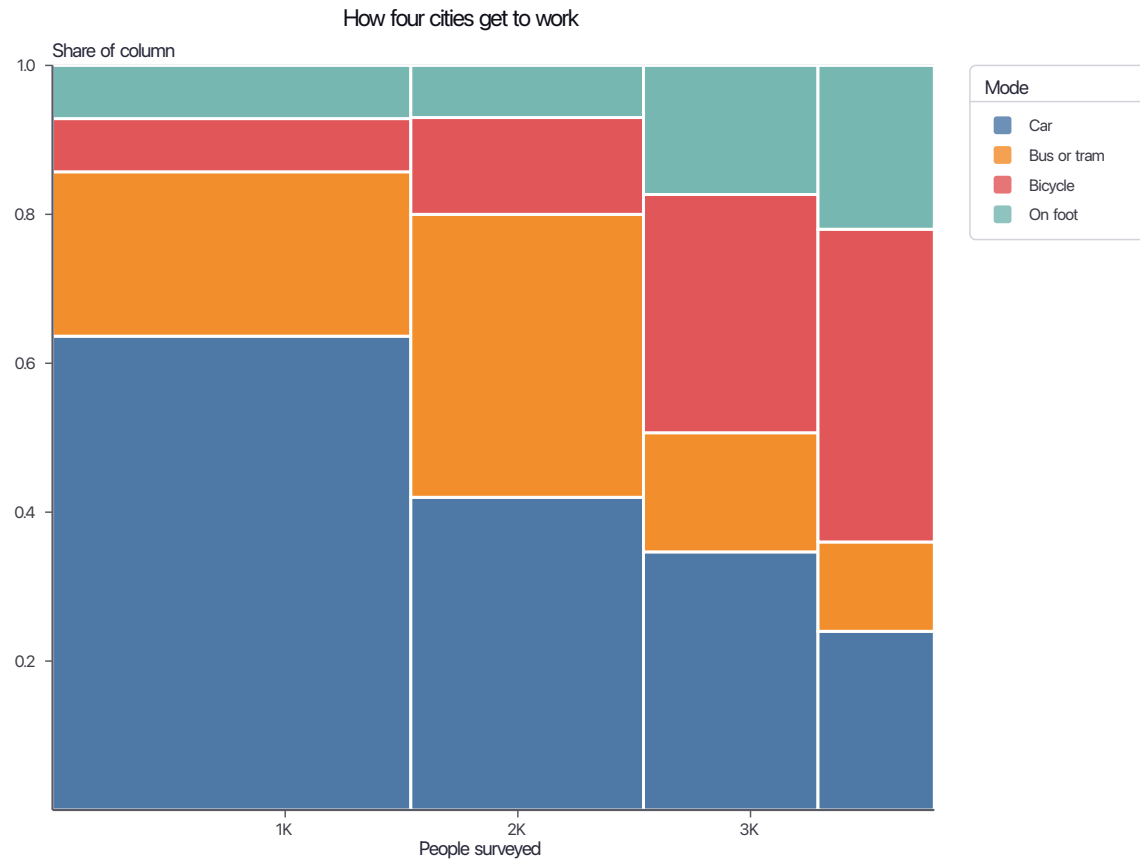
```
(data(commutes) + zone * partition(col.city, col.mode, cross = True) +
  ↪ x(col.people) +
  color(col.mode) + style(border_color = "white", border_size = 2) +
  x_label("People surveyed") + title("How four cities get to work"))
```

Julia

```
data(commutes) + zone * partition(:city, :mode, cross = true) +
  x(:people) + color(:mode) +
  style(border_color = "white", border_size = 2) +
  x_label("People surveyed") + title("How four cities get to work")
```

JavaScript

```
plot(data(commutes),
  layer(zone, partition(col.city, col.mode, { cross: true })),
  x(col.people), color(col.mode),
  style({ border_color: "white", border_size: 2 }),
  x_label("People surveyed"), title("How four cities get to work"))
```



“Given the commutes: zones derived by partition through city and mode, crossed, x is people, color by mode.”

Two variables, and the plot reads them in two directions at once. A column’s *width* is that city’s share of everyone surveyed, so Ashford’s column is three times Millbrook’s; a cell’s *height* is that mode’s share *within its own city*, so every column fills the panel whatever its width. The consequence is the one a mosaic is drawn for: a cell’s *area* is the number of people in it, and the eye compares areas across the whole panel without being told to.

That is also what makes Millbrook readable rather than misleading. Its cycling share is the largest of the four and its column is the narrowest, so the plot reports the finding and the weight of evidence behind it at once, which four equal-width stacked bars cannot do at all.

The two axes therefore carry different quantities, which is worth looking at before reading anything off them. The horizontal runs 0 to the total in the data’s own units, 3,790 people, exactly as the icicle’s measure axis does, so composing with **proportion** turns it into shares. The vertical runs 0 to 1 and can do nothing else, because what a height means is a share of its own column and the grand total is not the thing being divided.

`partition` writes a `name` column holding each node's own label, and a `text` layer reads it. The sunburst uses the same layer, for the same reason: a shallower partition of the same table lands its nodes in exactly the same places, so the second layer needs no filtering and no arithmetic of its own.

R

```
data(commutes) + x(people) +
  zone * partition(city, mode, cross = TRUE) + color(mode) +
  style(border_color = "white", border_size = 2) +
  text * partition(city, cross = TRUE) + label(name) +
  x_label("People surveyed") + title("The same plot, with its columns named")
```

Python

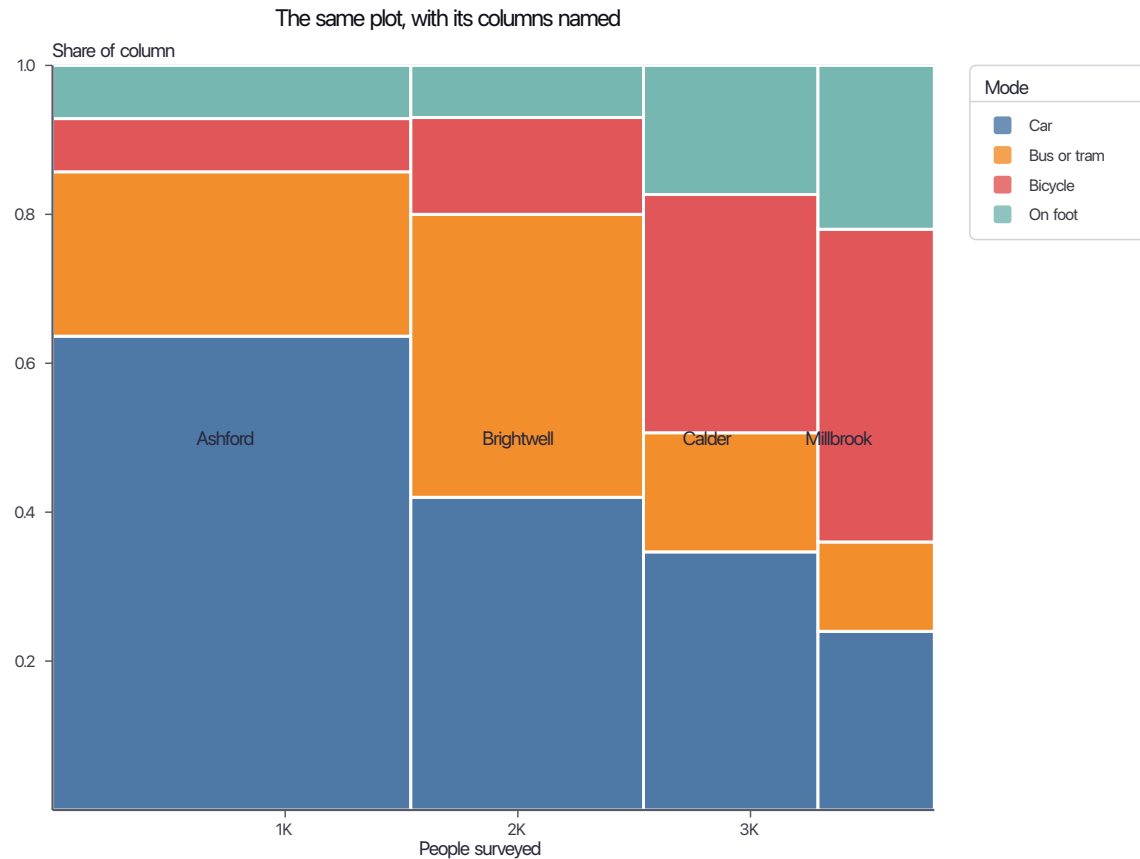
```
(data(commutes) + x(col.people) +
  zone * partition(col.city, col.mode, cross = True) + color(col.mode) +
  style(border_color = "white", border_size = 2) +
  text * partition(col.city, cross = True) + label(col.name) +
  x_label("People surveyed") + title("The same plot, with its columns
↪   named"))
```

Julia

```
data(commutes) + x(:people) +
  zone * partition(:city, :mode, cross = true) + color(:mode) +
  style(border_color = "white", border_size = 2) +
  text * partition(:city, cross = true) + label(:name) +
  x_label("People surveyed") +
  title("The same plot, with its columns named")
```

JavaScript

```
plot(data(commutes), x(col.people),
  layer(zone, partition(col.city, col.mode, { cross: true })),
  color(col.mode), style({ border_color: "white", border_size: 2 }),
  layer(text, partition(col.city, { cross: true })), label(col.name),
  x_label("People surveyed"),
  title("The same plot, with its columns named"))
```



“Given the commutes: x is people, zones derived by partition through city and mode, crossed, color by mode, and also text derived by partition through city, crossed, label by name.”

The labels sit at the middle of each column because that is where the center of a column is, and `style(nudge = )` is what moves them if you want them along the top. A mosaic’s column names belong on an axis, but `gog`’s category axis gives every slot the same width, which is the one thing a mosaic’s columns are not. So the names arrive as a layer.

The cell edges are a `style()` setting like any other, and they matter more here than anywhere else in the book. Without them, two neighboring cells that share a color merge into one shape and the column boundary disappears:

R

```
data(commutes) + zone * partition(city, mode, cross = TRUE) + x(people) +
  color(mode) + x_label("People surveyed") +
  title("The same plot with no edges")
```

Python

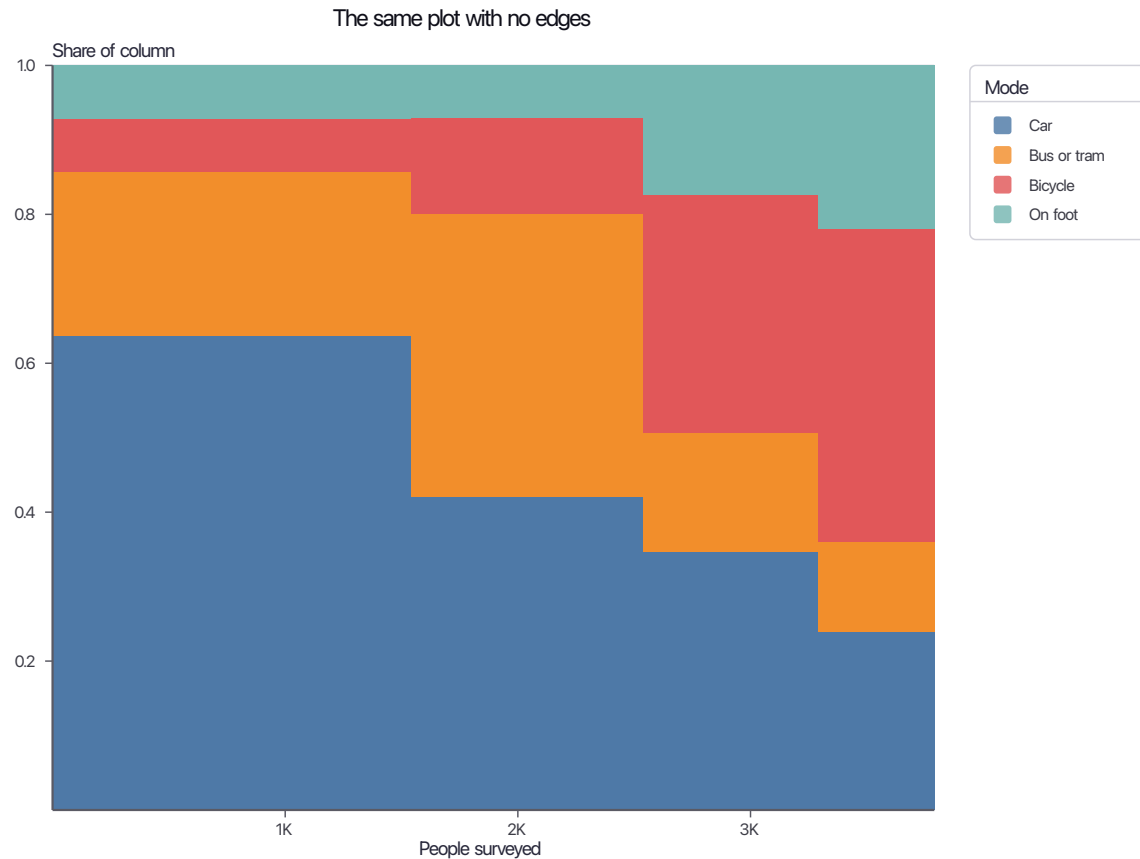
```
(data(commutes) + zone * partition(col.city, col.mode, cross = True) +  
  ↪ x(col.people) +  
  color(col.mode) + x_label("People surveyed") +  
  title("The same plot with no edges"))
```

Julia

```
data(commutes) + zone * partition(:city, :mode, cross = true) +  
  x(:people) + color(:mode) + x_label("People surveyed") +  
  title("The same plot with no edges")
```

JavaScript

```
plot(data(commutes),  
  layer(zone, partition(col.city, col.mode, { cross: true })),  
  x(col.people), color(col.mode), x_label("People surveyed"),  
  title("The same plot with no edges"))
```



Both plots draw the same numbers. The second is the reason a `zone` carries `style(border_color = , border_size = )` at all: the mark is both a highlight behind a line, where a frame would distract from the data, and the cells that *are* the data.

One level is legal and means something: the **spine plot**, columns of varying width and nothing dividing them, which is the marginal distribution on its own.

R

```
data(commutes) + zone * partition(city, cross = TRUE) + x(people) +
  color(city) + style(border_color = "white", border_size = 2) +
  x_label("People surveyed") + title("The margin alone: a spine plot")
```

Python

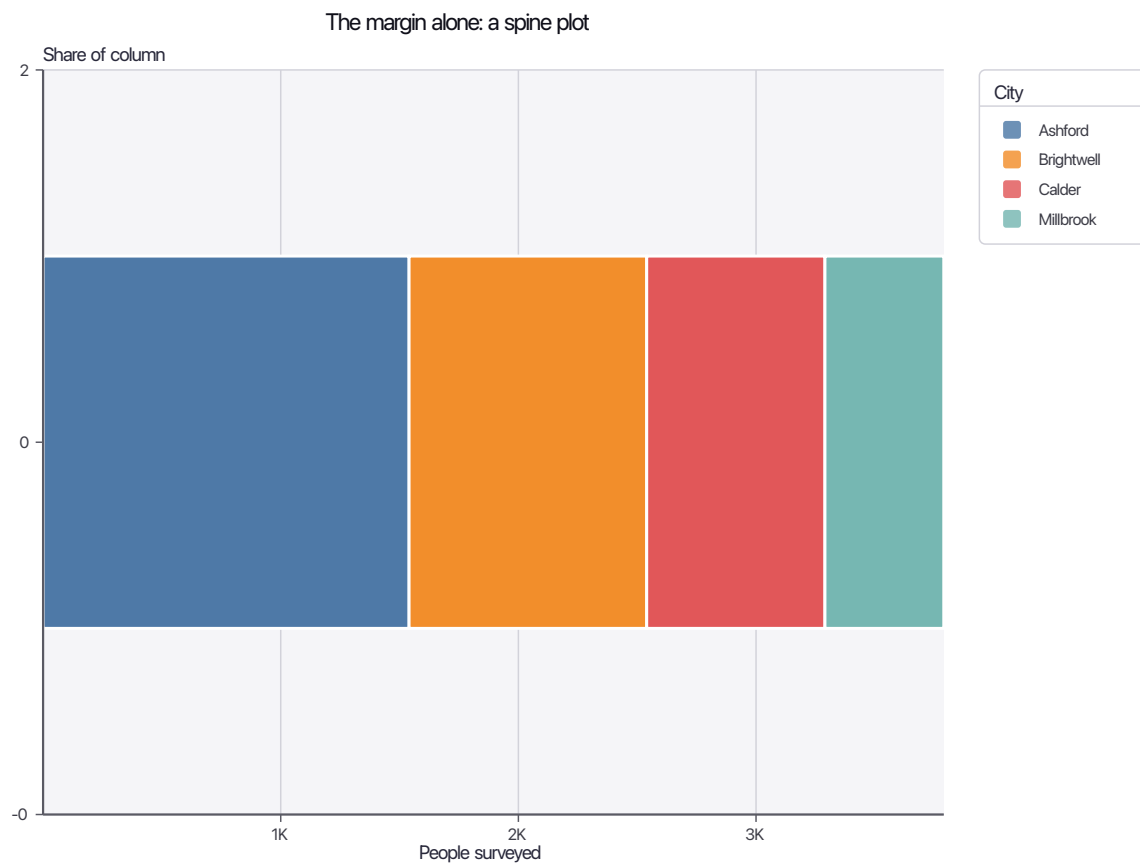

```
(data(commutes) + zone * partition(col.city, cross = True) + x(col.people) +
  color(col.city) + style(border_color = "white", border_size = 2) +
  x_label("People surveyed") + title("The margin alone: a spine plot"))
```

Julia

```
data(commutes) + zone * partition(:city, cross = true) + x(:people) +
  color(:city) + style(border_color = "white", border_size = 2) +
  x_label("People surveyed") + title("The margin alone: a spine plot")
```

JavaScript

```
plot(data(commutes), layer(zone, partition(col.city, { cross: true })),
  x(col.people), color(col.city),
  style({ border_color: "white", border_size: 2 }),
  x_label("People surveyed"), title("The margin alone: a spine plot"))
```



Everything **partition** refuses nested it refuses crossed, because the refusals are about the **tree** and not about the layout. A level must be a name rather than a number. A branch may stop early, but it may not have a hole in the middle. And an interior node carrying a value of its own is the one genuine ambiguity, so it is refused rather than guessed. The [sunburst chapter](#) meets each of them where they are easiest to see.

25.17 flow: a magnitude through its stages

flow is **partition**'s neighbor. Both read categorical columns named in the atom, both place their own marks, and both feed more than one mark from one computation. Where **partition** divides a whole among nested parts, **flow** carries a magnitude through a sequence of stages. One row of the table is one path through every stage the atom names.

Table 25.6: **titanic**: first 5 of 32 rows

class	sex	age	survived	n
1st	Male	Child	No	0
2nd	Male	Child	No	0
3rd	Male	Child	No	35
Crew	Male	Child	No	0
1st	Female	Child	No	0

R

```
data(titanic) + y(n) +  
  ribbon * flow(class, survived) + color(class) +  
  zone * flow(class, survived) +  
  style(color = "gray") +  
  title("Who survived, by class")
```

Python

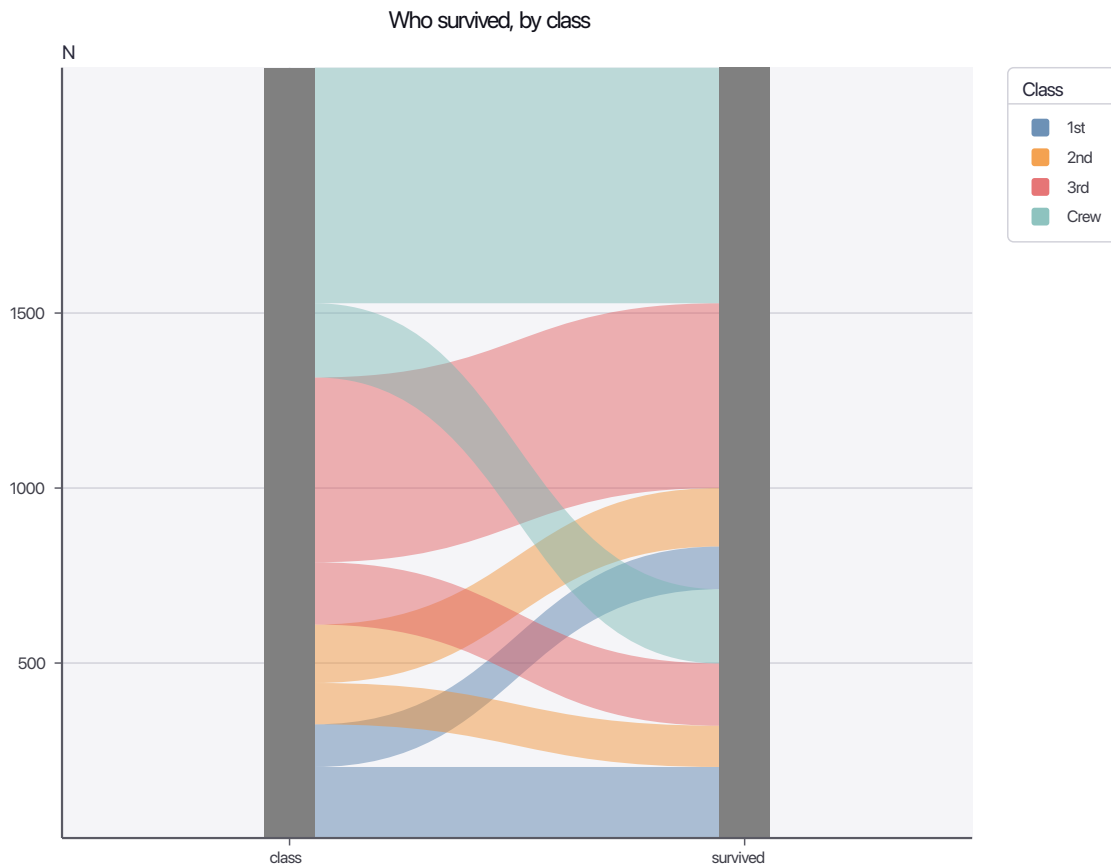
```
(data(titanic) + y(col.n) +  
  ribbon * flow(col["class"], col.survived) + color(col["class"])) +  
  zone * flow(col["class"], col.survived) +  
  style(color = "gray") +  
  title("Who survived, by class"))
```

Julia

```
data(titanic) + y(:n) + ribbon * flow(:class, :survived) + color(:class) +  
  zone * flow(:class, :survived) + style(color = "gray") +  
  title("Who survived, by class")
```

JavaScript

```
plot(data(titanic), y(col.n),  
  layer(ribbon, flow(col.class, col.survived)), color(col.class),  
  layer(zone, flow(col.class, col.survived)), style({ color: "gray" }),  
  title("Who survived, by class"))
```



“Given titanic: y is n, ribbons derived by flow through class and survived, color by class, and also zones derived by flow through class and survived.”

The **ribbon** layer draws the bands, one per path, as thick as the path's total at both ends. The **zone** layer draws each stage's slots, stacked with no padding, so the measure axis is drawn with its usual ticks and reads true counts. A **text** layer names the slots. The [Flow](#) chapter builds the full diagram and gives the rules; this section only places the atom in the family.

25.18 layout: a graph placed from an edge table

partition, **flow**, **layout** and **cluster** each compute a whole picture rather than a column. **layout** is the third of them, and the first whose output draws in a space of its own. **layout(from, to)** reads the two endpoint columns of an edge table and computes a position for every distinct name, in the engine, the same way in every language. Three marks read the placement inside **network()**: **edge** the connections, **point** the nodes, **text** their names. Its positions mean nothing as quantities, so its space draws no axes, and the [Network](#) chapter is where the family lives.

25.19 cluster: the closest leaves joined first

cluster is the fourth of the picture-computing transforms, and the question it answers is which things are most alike. It treats each category of an axis as a leaf of a tree, joins the two closest leaves, then the next two, and keeps joining until one tree holds them all. Each leaf is described by a profile: **cluster(amount, over = nutrient)** says that one leaf's profile is its **amount** in every category of **nutrient**.

Table 25.7: **nutrients**: first 5 of 40 rows

food	nutrient	amount
salmon	protein	25.4
salmon	fat	12.4
salmon	carbs	0.0
salmon	fiber	0.0
salmon	iron	0.5

R

```
data(nutrients) + path * cluster(amount, over = nutrient) + x(food) +  
  title("Eight foods, joined by what they are made of")
```

Python

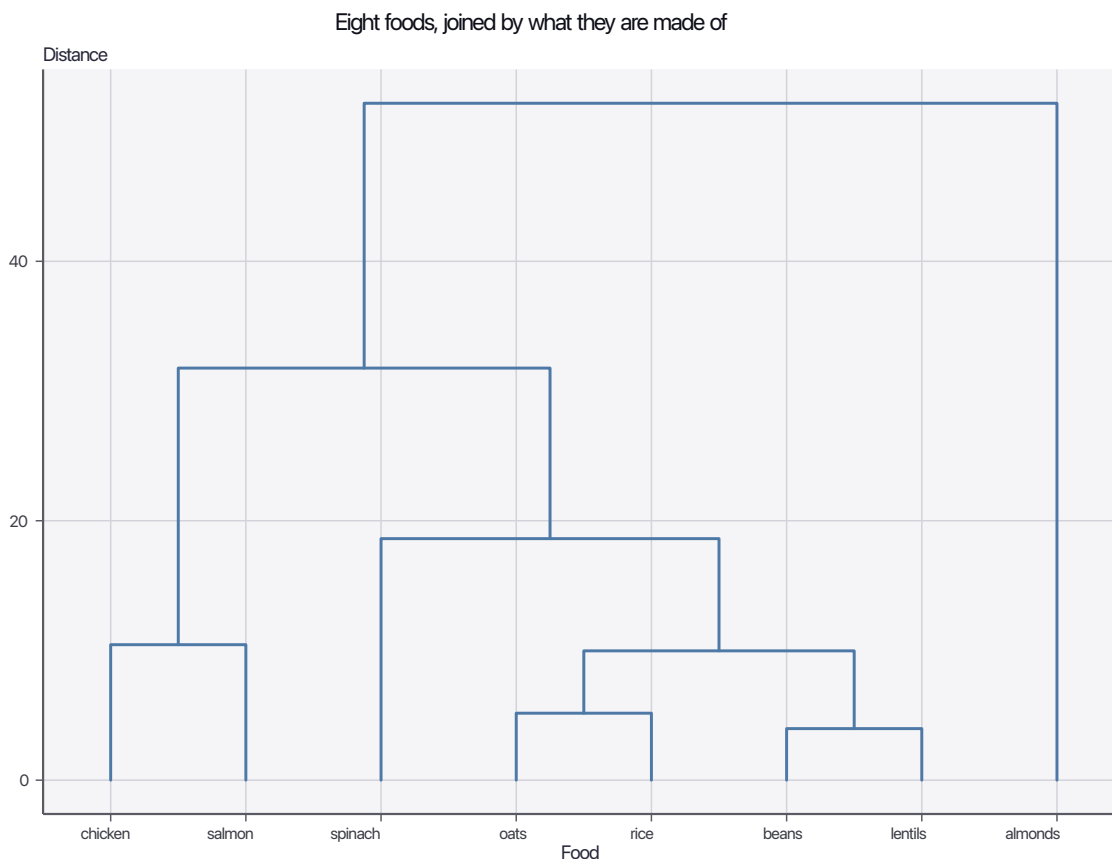
```
(data(nutrients) + path * cluster(col.amount, over=col.nutrient) +  
↪ x(col.food) +  
  title("Eight foods, joined by what they are made of"))
```

Julia

```
data(nutrients) + path * cluster(:amount, over = :nutrient) + x(:food) +  
  title("Eight foods, joined by what they are made of")
```

JavaScript

```
plot(data(nutrients),  
  layer(path, cluster(col.amount, { over: col.nutrient })), x(col.food),  
  title("Eight foods, joined by what they are made of"))
```



“Given nutrients: paths derived by cluster on amount, over nutrient, x is food.”

`path` draws the tree, and the height of a join says how different its two branches are. The same transform on `zone` reorders a tile plot’s slots to the tree’s leaf order. The [Cluster](#) chapter grows the tree, orders the tiles, and builds the clustered heatmap; this section only places the atom in the family.

25.20 dodge: side by side instead of on top

Every transform above answers a question about the data. The four that remain answer none. `dodge` is a different kind of atom: a **collision modifier**. It writes no new values; it *moves* the marks that a color split would otherwise stack at one position, setting them side by side instead.

When `color` splits a `bar`, a `box` or an `interval` into groups, the groups share their category's slot and pile up. For a histogram that reads well as translucent overlays, but for a grouped bar chart you want them separated. `dodge` does exactly that:

R

```
data(gm_eras) + bar * mean * dodge + x(continent) + y(life) + color(era) +  
  y_label("Mean life expectancy") +  
  title("Mean life expectancy by continent, 1957 vs 2007")
```

Python

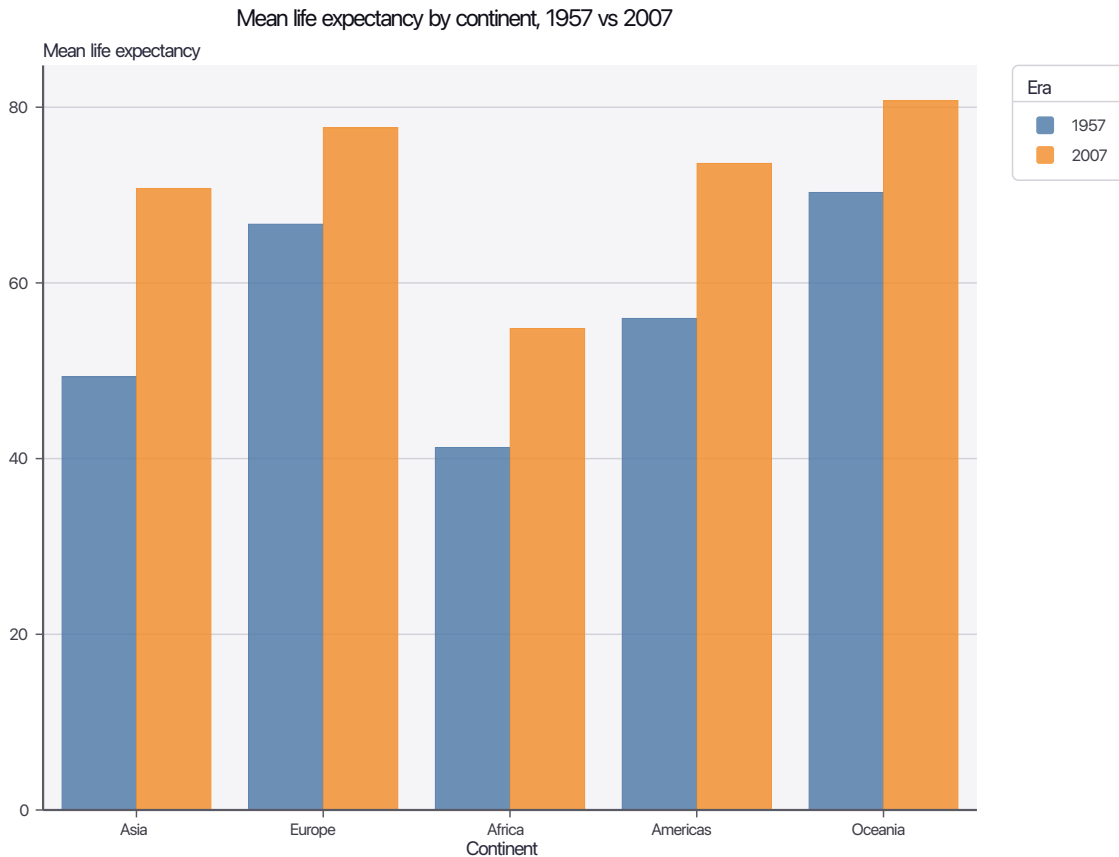
```
(data(gm_eras) + bar * mean * dodge + x(col.continent) + y(col.life) +  
  ↪ color(col.era) +  
  y_label("Mean life expectancy") +  
  title("Mean life expectancy by continent, 1957 vs 2007"))
```

Julia

```
data(gm_eras) + bar * mean * dodge + x(:continent) + y(:life) +  
  color(:era) + y_label("Mean life expectancy") +  
  title("Mean life expectancy by continent, 1957 vs 2007")
```

JavaScript

```
plot(data(gm_eras), layer(bar, mean, dodge), x(col.continent),  
  y(col.life), color(col.era), y_label("Mean life expectancy"),  
  title("Mean life expectancy by continent, 1957 vs 2007"))
```



“Given the gapminder eras: bars derived by mean and dodge, x is continent, y is life, color by era.”

The two eras sit next to each other in every continent, each bar half as wide, together filling exactly the slot one bar would; the gap between continents is untouched. There is no width to set: **dodge** derives it from the number of groups (**G** groups, each $1/G$ of the slot). And because the offset resolves the overlap outright, dodged bars draw **solid**: the translucent fill exists only to see through overlapping bars, and none overlap now.

It is written after ***** like any transform, so it composes cleanly: **bar * mean * dodge** aggregates first and dodges the result; **box * dodge** dodges a box’s own summary; **interval * range * dodge** dodges grouped whiskers. Here it sets each continent’s two distributions side by side:

R

```
data(gm_eras) + box * dodge + x(continent) + y(life) + color(era) +
  y_label("Life expectancy") +
  title("Life expectancy distribution, 1957 vs 2007")
```


Python

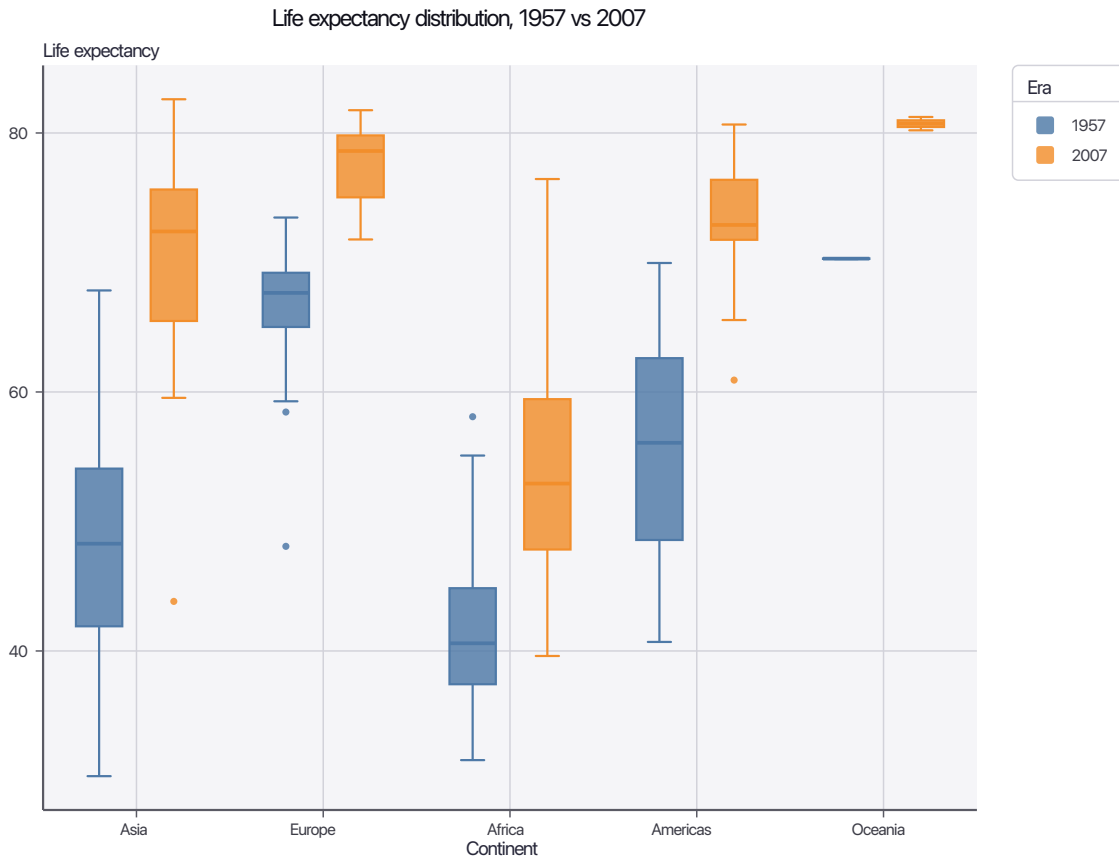
```
(data(gm_eras) + box * dodge + x(col.continent) + y(col.life) +  
  ↪ color(col.era) +  
  y_label("Life expectancy") +  
  title("Life expectancy distribution, 1957 vs 2007"))
```

Julia

```
data(gm_eras) + box * dodge + x(:continent) + y(:life) + color(:era) +  
  y_label("Life expectancy") +  
  title("Life expectancy distribution, 1957 vs 2007")
```

JavaScript

```
plot(data(gm_eras), layer(box, dodge), x(col.continent), y(col.life),  
  color(col.era), y_label("Life expectancy"),  
  title("Life expectancy distribution, 1957 vs 2007"))
```



“Given the gapminder eras: boxes derived by `dodge`, x is continent, y is life, color by era.”

Which marks `dodge` serves is decided by what a mark *is*. It subdivides a *width* (a bar’s thickness, a box’s body, a whisker’s slot) so it is defined for `bar`, `box` and `interval`. A `point` has no width to subdivide (its overlap answer is `jitter`, below); a `line` or `area` is offset by *accumulating* rather than subdividing (that is `stack`, next). Ask for `dodge` on one of those and `gog` refuses, naming the collision modifier that fits. That is the discipline the rest of the kernel follows: an atom combines with the marks it was defined for, and the refusal points at the right tool rather than drawing something wrong.

`dodge` also needs a split to separate: a `color` (or `group`) binding. Without one there is a single mark per slot and nothing to set beside anything, so `bar * dodge` on its own is refused toward adding the color, never accepted as a silent no-op.

25.21 stack: piled up, not side by side

`stack` is `dodge`'s sibling: the other way to resolve a color split's overlap. Where `dodge` sets the groups *beside* each other, `stack` piles them *on top* of each other along the measure axis, each group starting where the one below it ended. It is the collision modifier for the marks that accumulate: `bar` and `area` pile heights, and `point` piles rows.

Take the `dodge` example and change one atom:

R

```
data(gm_eras) + bar * sum * stack + x(continent) + y(population) + color(era)
↪ +
  y_label("Population") +
  title("Population by continent, 1957 and 2007 stacked")
```

Python

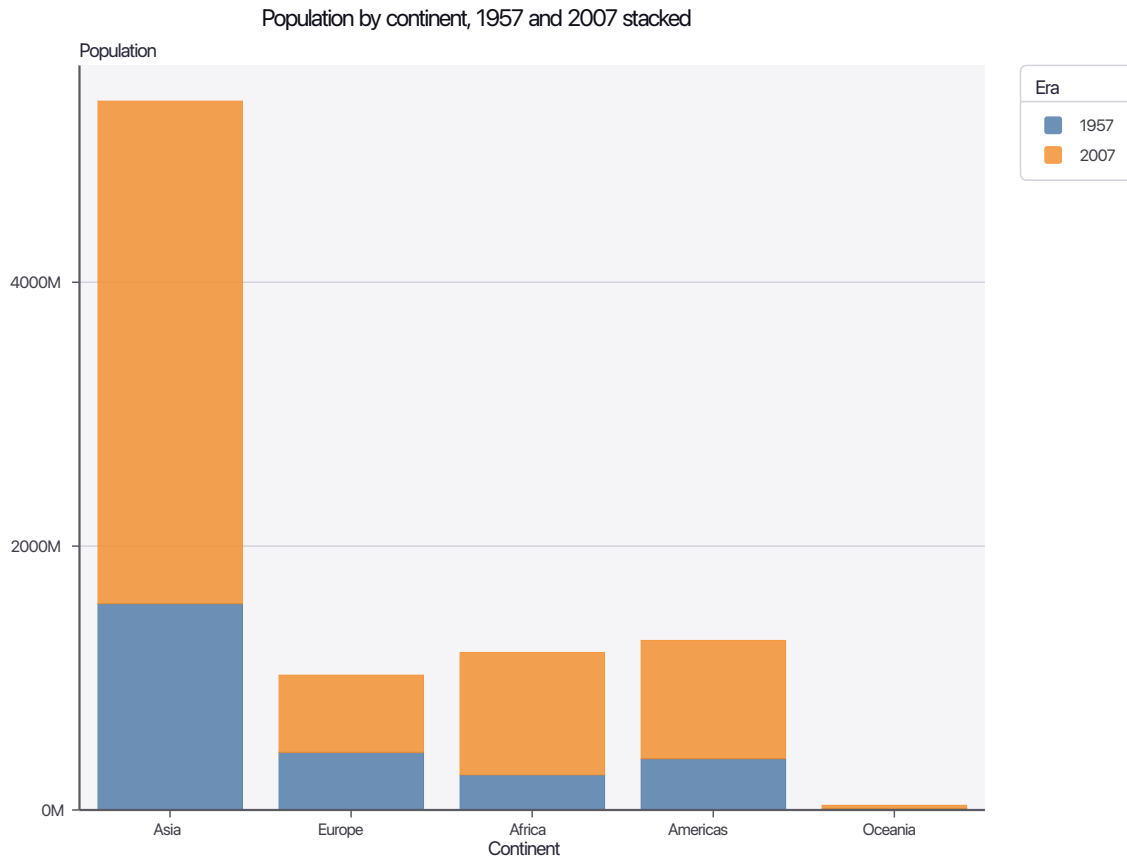
```
(data(gm_eras) + bar * sum * stack + x(col.continent) + y(col.population) +
↪ color(col.era) +
  y_label("Population") +
  title("Population by continent, 1957 and 2007 stacked"))
```

Julia

```
data(gm_eras) + bar * sum * stack + x(:continent) + y(:population) +
  color(:era) + y_label("Population") +
  title("Population by continent, 1957 and 2007 stacked")
```

JavaScript

```
plot(data(gm_eras), layer(bar, sum, stack), x(col.continent),
  y(col.population), color(col.era), y_label("Population"),
  title("Population by continent, 1957 and 2007 stacked"))
```



“Given the gapminder eras: bars derived by sum and stack, x is continent, y is population, color by era.”

Notice the measure also changed, from **mean life** to **sum population**. You stack quantities that *add up*: two populations sum to a total that means something, so the full bar reads as “everyone, both eras”. Two mean life expectancies do not add (a bar reaching 130 years is nonsense), so that comparison wants **dodge**, side by side. The grammar writes either; the data decides which is honest. Because the pile resolves the overlap, the bars draw **solid**, and the axis grows to the tallest *stack*: Asia passes five billion, the sum of its two eras.

25.21.1 Stacked areas: the case stack was built for

A split **area** is where overlap costs the most: filled regions hide one another, and no opacity setting recovers them (the [area chapter](#) shows the warning **gog** raises). **stack** is the answer. Each region sits on the cumulative height of the ones below, so they abut into one band instead of hiding each other, the classic “parts of a whole, over time”:

R

```
data(gm_all) + area * sum * stack + x(year) + y(population) +  
  ↪ color(continent) +  
  y_label("Population") +  
  title("World population by continent, stacked")
```

Python

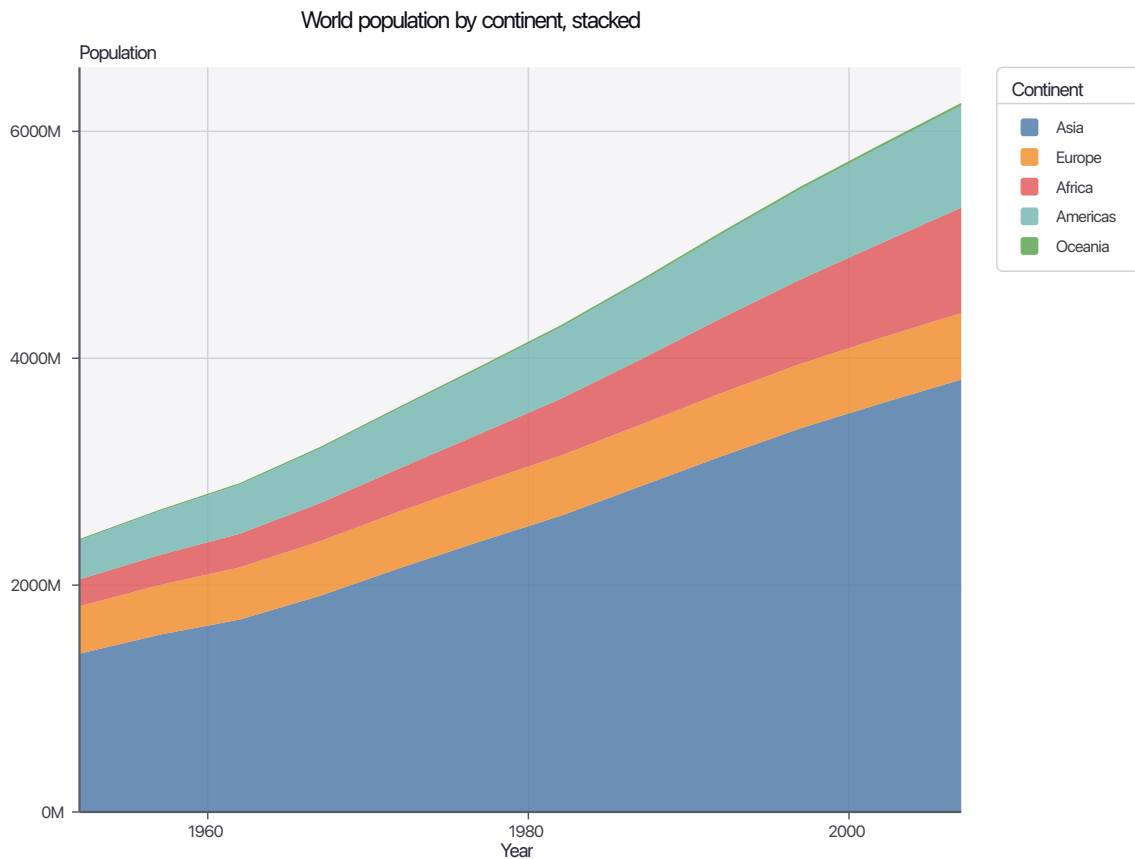
```
(data(gm_all) + area * sum * stack + x(col.year) + y(col.population) +  
  ↪ color(col.continent) +  
  y_label("Population") +  
  title("World population by continent, stacked"))
```

Julia

```
data(gm_all) + area * sum * stack + x(:year) + y(:population) +  
  color(:continent) + y_label("Population") +  
  title("World population by continent, stacked")
```

JavaScript

```
plot(data(gm_all), layer(area, sum, stack), x(col.year),  
  y(col.population), color(col.continent), y_label("Population"),  
  title("World population by continent, stacked"))
```



“Given all the gapminder years: areas derived by sum and stack, x is year, y is population, color by continent.”

Asia fills the floor; Europe, Africa, the Americas and Oceania pile on top, and the band’s upper edge is the world total climbing from 2.4 billion to 6.25. Read any one color’s *thickness* for that continent’s share.

25.21.2 `stack(share = TRUE)`: every pile filled to one

Reading a thickness is the hard part of the plot above, because each band’s baseline moves under it. When the composition is the question and the total is not, `stack(share = TRUE)` divides every pile by its own slot’s total, so all of them reach 1 and only the proportions are left:

R

```
data(gm_all) + area * sum * stack(share = TRUE) + x(year) + y(population) +  
  color(continent) + title("Every year's composition, filled to one")
```

Python

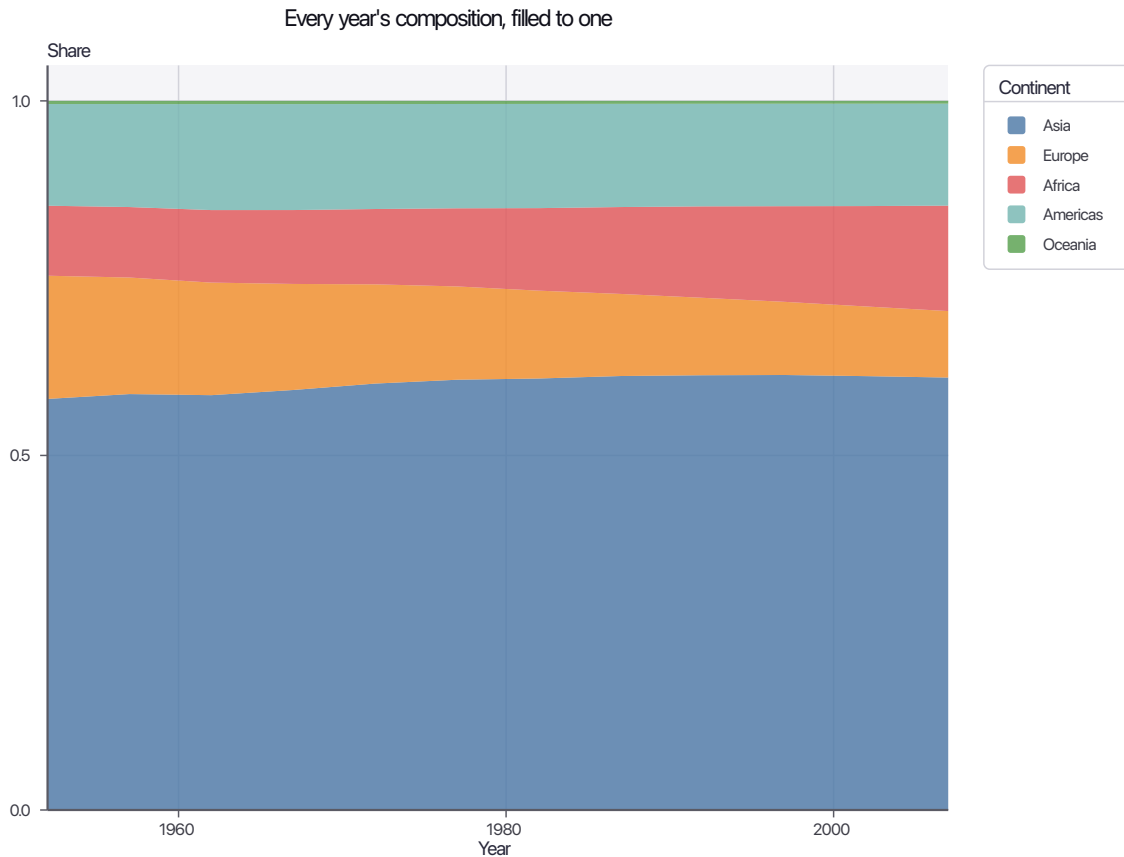
```
(data(gm_all) + area * sum * stack(share = True) + x(col.year) +  
  ↪ y(col.population) +  
  color(col.continent) + title("Every year's composition, filled to one"))
```

Julia

```
data(gm_all) + area * sum * stack(share = true) + x(:year) +  
  y(:population) + color(:continent) +  
  title("Every year's composition, filled to one")
```

JavaScript

```
plot(data(gm_all), layer(area, sum, stack({ share: true })), x(col.year),  
  y(col.population), color(col.continent),  
  title("Every year's composition, filled to one"))
```



“Given all the gapminder years: areas derived by sum and stack as shares, x is year, y is population, color by continent.”

Now Asia’s band holds about three fifths and Europe’s visibly narrows across the century, both of which the plot above hides inside a rising total. The axis says **Share**, because whatever the numbers were before, they are fractions of one now.

This is not a second name for **proportion**. The two divide by different totals. **proportion** divides by the **whole plot’s**, so its bars still say how big each slot is. This one divides by the **slot’s own**, which deliberately throws that away. And only this one composes with a measurement it was not given: **sum** of a population column is exactly the case **proportion** has no way to say. Filling is a *position* adjustment: it changes where the marks sit and what the scale reads, never what was counted, which is why it belongs to **stack** and not to a statistic.

A **bar** fills the same way, and that reading is the chart the name comes from: the **100% stacked bar**, worked through in the [bar chapter](#). One parameter, both geometries, exactly as plain **stack** hands both marks the same span and lets each draw it: [No Exceptions](#).

Which marks **stack** serves is decided by what a mark *is*: the same rule **dodge** follows. What **stack** computes is one thing for every mark, a span from a foot to a top along the measure axis, and what differs is how a geometry draws that span. A **bar** fills it, an **area** fills it across x, and a **point**, having no length to stretch across it, spends it on *how many dots there are* (the dot plot, just below). A **box** or **interval** is set beside its neighbors rather than piled (that is **dodge**); an unfilled **line** has nothing to fill and **stack** (use **area**); a **ribbon** already spans a low to a high, so it measures no height from a baseline. Ask for **stack** on one of those and **gog** refuses, naming the collision modifier that fits. For **bar** and **area** it also needs a **color** (or **group**) split to pile: **bar * stack** alone is refused toward adding the color, never accepted as a silent no-op.

25.21.3 `stack(baseline = )`: where the pile sits

A pile has two free choices. Its heights are one, and **share** is how you change them. Where the pile *sits* is the other, and **baseline** is how you change that.

The default is **"zero"**. Every pile sits on the axis, which is both plots above. That layout has a known weakness. Only the bottom band has a shape you can read. Every band above it rests on the sum of the bands below, so it inherits all of their movement as well as its own.

The weakness is easiest to see when each series peaks at a different time. Here are four music genres over half a year, in the **listening** table:

Table 25.8: **listening**: first 5 of 104 rows

week	genre	plays
1	Folk	38.0
2	Folk	36.4
3	Folk	38.6
4	Folk	39.5
5	Folk	38.3

R

```
data(listening) + area * stack + x(week) + y(plays) + color(genre) +
  y_label("Plays") + title("Stacked on zero")
```

Python

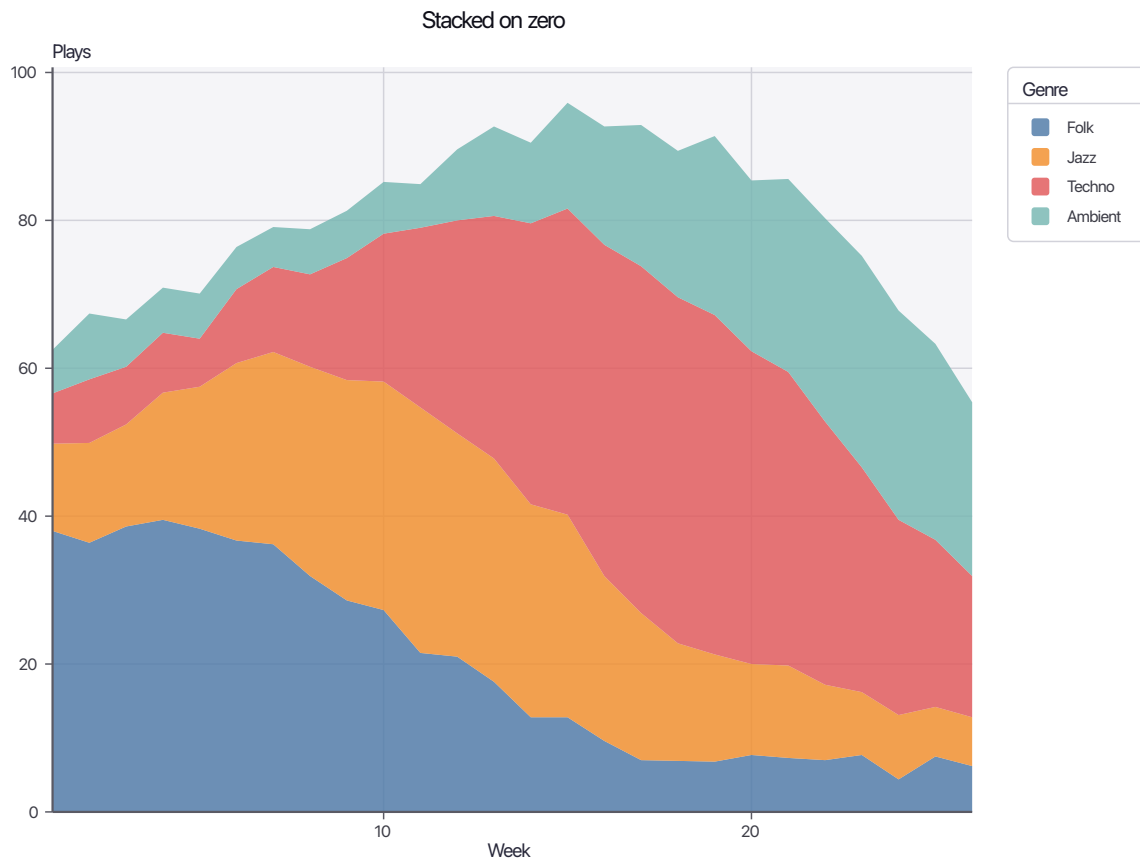
```
(data(listening) + area * stack + x(col.week) + y(col.plays) +
  ↪ color(col.genre) +
  y_label("Plays") + title("Stacked on zero"))
```

Julia

```
data(listening) + area * stack + x(:week) + y(:plays) + color(:genre) +  
  y_label("Plays") + title("Stacked on zero")
```

JavaScript

```
plot(data(listening), layer(area, stack), x(col.week), y(col.plays),  
  color(col.genre), y_label("Plays"), title("Stacked on zero"))
```



Techno is the red band. It looks like it grows to about week 17 and then collapses. Part of that shape is Techno, and part of it is Folk and Jazz shrinking underneath it. You cannot tell which part is which.

"center" places each pile so that its middle is at zero:

R

```
data(listening) + area * stack(baseline = "center") + x(week) + y(plays) +  
  color(genre) + title("Centered: the ThemeRiver")
```

Python

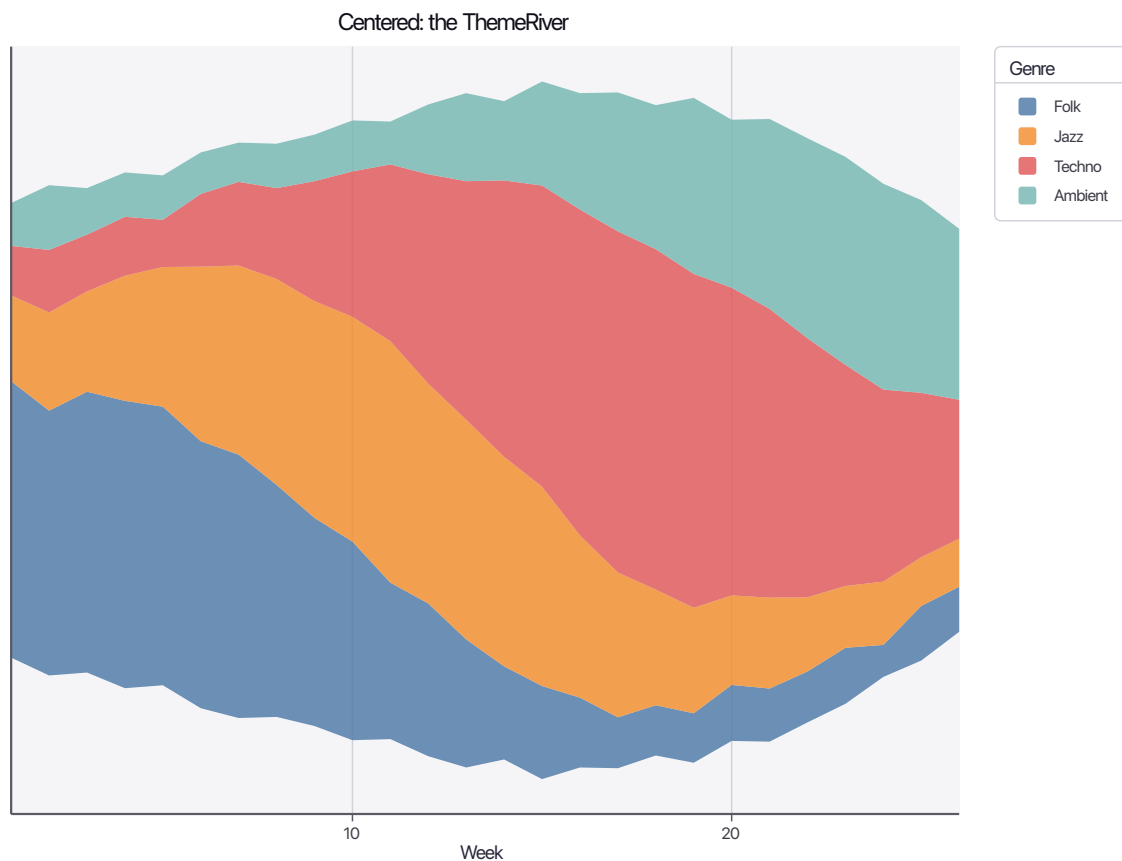
```
(data(listening) + area * stack(baseline = "center") + x(col.week) +  
  ↪ y(col.plays) +  
  color(col.genre) + title("Centered: the ThemeRiver"))
```

Julia

```
data(listening) + area * stack(baseline = "center") + x(:week) +  
  y(:plays) + color(:genre) + title("Centered: the ThemeRiver")
```

JavaScript

```
plot(data(listening), layer(area, stack({ baseline: "center" })),  
  x(col.week), y(col.plays), color(col.genre),  
  title("Centered: the ThemeRiver"))
```



“Given the listening table: areas derived by stack on the center baseline, x is week, y is plays, color by genre.”

That symmetric layout is the **ThemeRiver** (Havre et al., 2000), named for the plot it was built to draw: how much each theme is written about, over time.

"wiggle" instead picks the foot that makes the bands as flat as it can. Thick bands are held steadiest, because a reader looks at them first. Thin bands absorb the movement instead:

R

```
data(listening) + area * stack(baseline = "wiggle") + x(week) + y(plays) +
  color(genre) + title("Flattest: the streamgraph")
```

Python

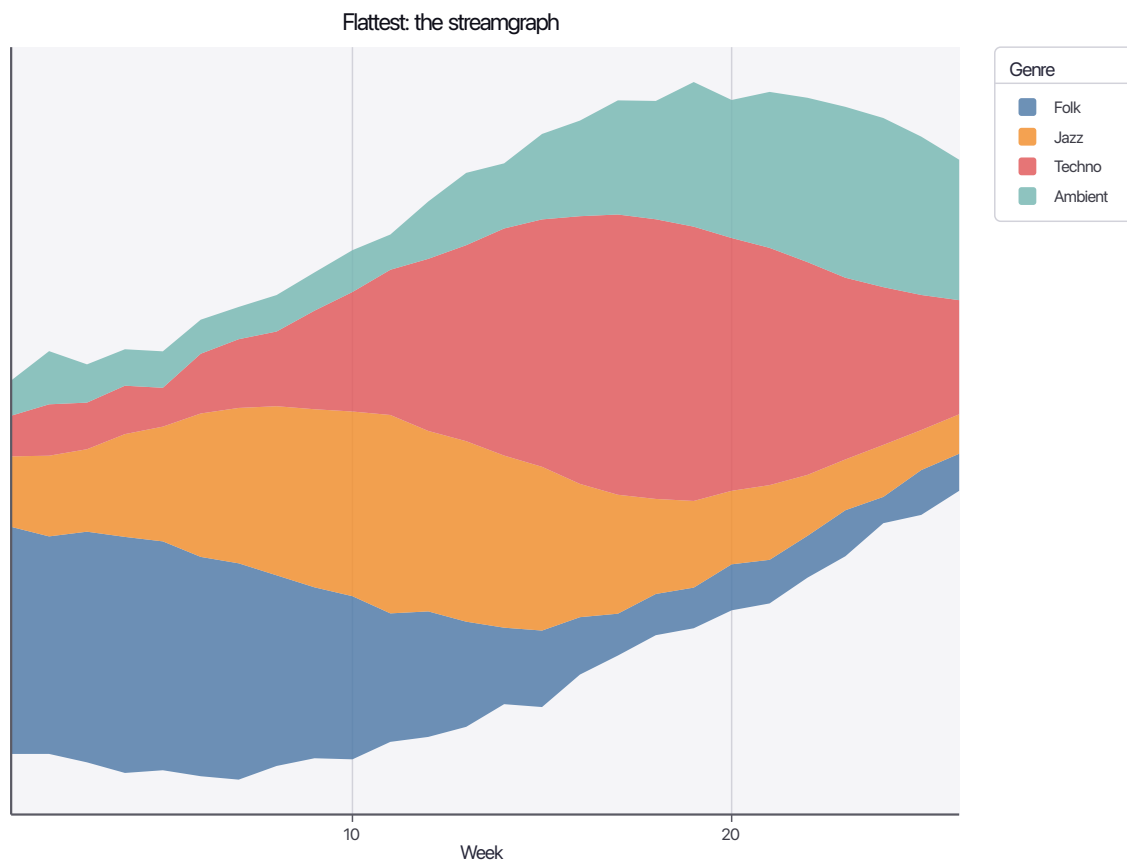
```
(data(listening) + area * stack(baseline = "wiggly") + x(col.week) +
  ↪ y(col.plays) +
  color(col.genre) + title("Flattest: the streamgraph"))
```

Julia

```
data(listening) + area * stack(baseline = "wiggly") + x(:week) +
  y(:plays) + color(:genre) + title("Flattest: the streamgraph")
```

JavaScript

```
plot(data(listening), layer(area, stack({ baseline: "wiggly" })),
  x(col.week), y(col.plays), color(col.genre),
  title("Flattest: the streamgraph"))
```



“Given the listening table: areas derived by stack on the wiggle baseline, x is week, y is plays, color by genre.”

Now Techno is one clean shape, widening and then narrowing, which is what the numbers actually say. Folk, the blue band, is a steady decline rather than a wedge. No band changed thickness between these three plots. Only the place each pile sits changed.

The last plot is a streamgraph (Byron & Wattenberg, 2008). The middle one is a ThemeRiver. The two are often confused and they are not the same layout: one is symmetric, the other is chosen to reduce movement. The streamgraph’s designers argue readers compare values more accurately on it, and that argument is the reason this option exists (Byron & Wattenberg, 2008).

Notice also that these two plots have no numbers on the measure axis. Once a pile has moved off zero, no value on that axis means anything. A band drawn from 12 to 20 says “this group is 8 here”, and the 12 and the 20 only record where the pile was placed. So `gog` draws no ticks and no axis name, for the same reason it leaves an empty heatmap cell unpainted. The *scale* is unchanged, so thicknesses stay comparable everywhere in the plot, and thickness is what a streamgraph asks you to read.

`baseline` is not an area-only parameter. A `bar` has a pile too, so it is placed the same way:

R

```
data(listening) + bar * stack(baseline = "center") + x(week) + y(plays) +  
  color(genre) + title("The same baseline, drawn as bars")
```

Python

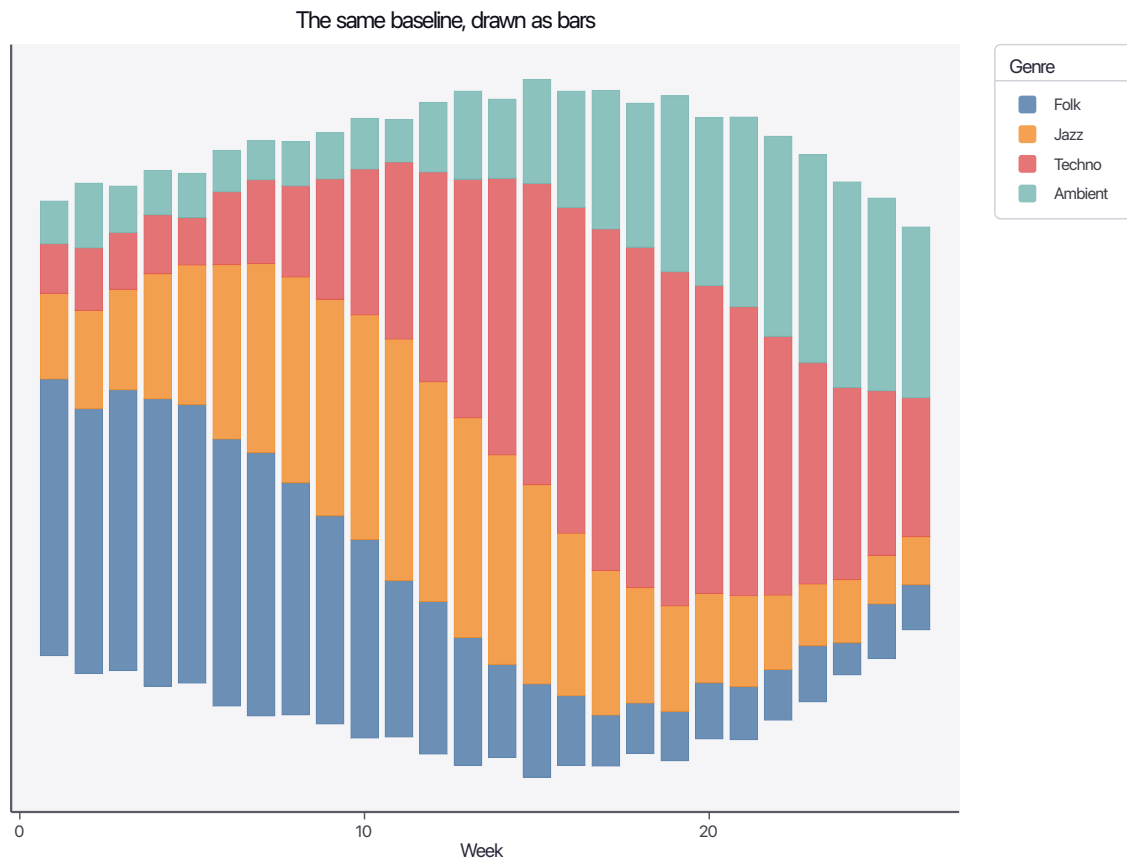
```
(data(listening) + bar * stack(baseline = "center") + x(col.week) +  
  ↪ y(col.plays) +  
  color(col.genre) + title("The same baseline, drawn as bars"))
```

Julia

```
data(listening) + bar * stack(baseline = "center") + x(:week) +  
  y(:plays) + color(:genre) + title("The same baseline, drawn as bars")
```

JavaScript

```
plot(data(listening), layer(bar, stack({ baseline: "center" })),  
  x(col.week), y(col.plays), color(col.genre),  
  title("The same baseline, drawn as bars"))
```



One parameter, both geometries. `share` follows the same rule, and so does plain `stack`, which hands both marks one span and lets each draw it: [No Exceptions](#).

Moving a pile gives up the origin of the measure axis. Only a flat plot can give that up, so the other spaces refuse:

```
data(listening) + area * stack(baseline = "wiggle") + x(week) + y(plays) +
  color(genre) + polar()
```

Error:

```
! gog: `stack(baseline = "wiggle")` moves every pile off the origin of the
  ↳ measure axis, and `polar()` has no origin to spare. A displaced pile is a
  ↳ statement about a plane: in `polar()` the measure is an angle or a radius
  ↳ or a height the space has already fixed, so moving the foot asks for a
  ↳ place that is not on the plot. Keep the space and drop the baseline
  ↳ (`stack` alone, or `stack(share = TRUE)` for composition), or keep the
  ↳ baseline and draw it flat.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

In `polar()` the measure is an angle or a radius. A radius of zero is the center of the circle, so moving a pile off it asks for a negative radius, which is not a place on the plot.

25.21.4 A pile has one direction

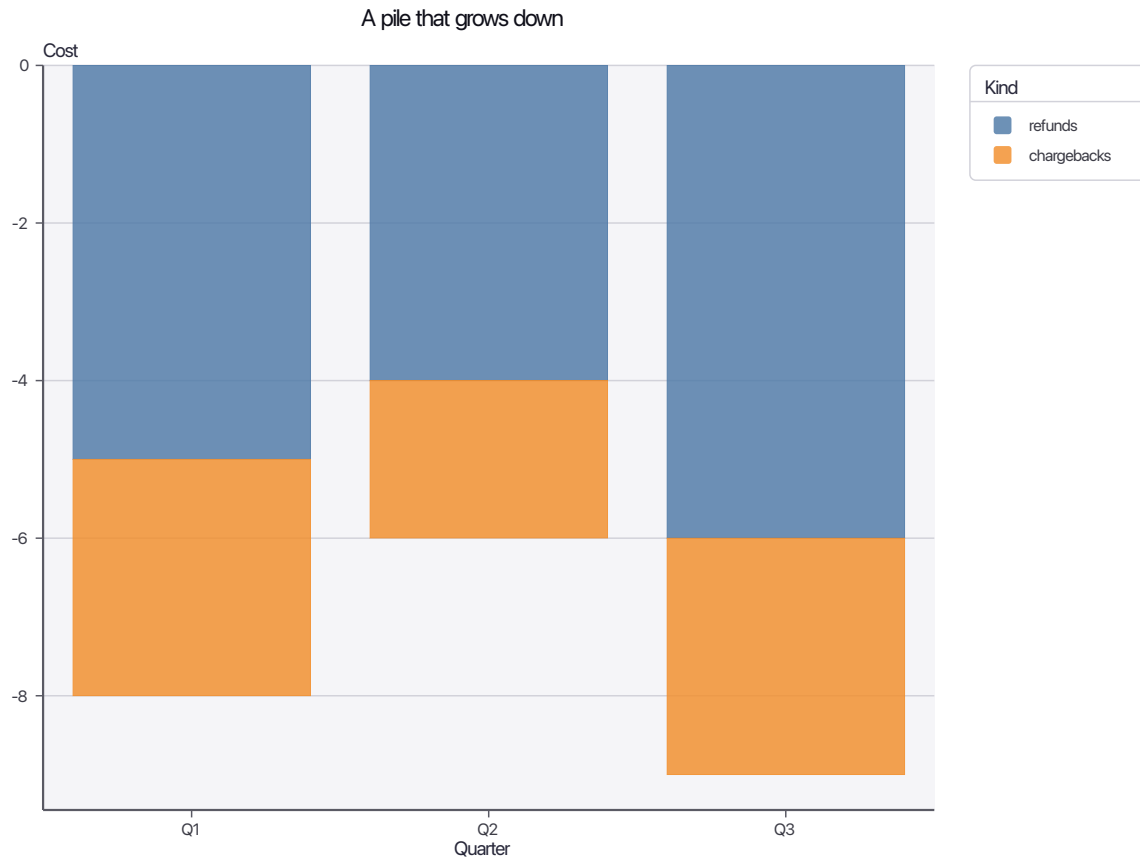
`stack` has a third condition. The first two are about the sentence. This one is about the numbers.

A pile puts its groups end to end. Every member must therefore point the same way.

Negative values are allowed. A pile of them grows *downward* from zero, exactly as a positive pile grows up:

```
drawdown <- data.frame(
  quarter = rep(c("Q1", "Q2", "Q3"), 2),
  cost     = c(-5, -4, -6, -3, -2, -3),
  kind     = rep(c("refunds", "chargebacks"), each = 3))

data(drawdown) + bar * stack + x(quarter) + y(cost) + color(kind) +
  y_label("Cost") + title("A pile that grows down")
```

A pile whose members disagree has no correct drawing. Suppose one group is 5 and another is -3 at the same position. The negative band would begin where the first band ended, at 5, and reach back down to 2. That region already belongs to the first group. A reader would see a solid block of length 3 where the number is negative. So `gog` refuses:

```
mixed <- data.frame(
  quarter = rep(c("Q1", "Q2", "Q3"), 2),
  amount  = c(5, 5, 5, 2, -3, 2),
  kind    = rep(c("sales", "returns"), each = 3))

data(mixed) + bar * stack + x(quarter) + y(amount) + color(kind)
```

Error:

! gog: ``bar * stack`` piles the groups at one position on top of one another,
 ↪ and at quarter = Q2 they disagree in sign - ``sales`` is 5 and ``returns`` is
 ↪ -3. A pile has one direction: the band going the other way is drawn
 ↪ `*inside*` the ones below it, so the plot would show a block of 3 where the
 ↪ number is negative and read its sign backwards. Either keep the sign and
 ↪ drop the pile - ``bar * dodge`` sets the groups side by side instead, and a
 ↪ dodged bar keeps its own baseline, so a negative one hangs below the axis
 ↪ where a reader can see that it is negative - or keep the pile and drop
 ↪ the sign, stacking magnitudes you took the absolute value of in the host.
 ↪ (A pile whose members agree stays legal both ways: all-negative grows
 ↪ downward from zero exactly as all-positive grows up.)
 gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

The message names the position, both groups, and both numbers. A message that said only “some values are negative” would make you search the whole table for a pile you cannot see on the page.

There are two ways out, and you choose by what you want the plot to show. If the sign matters, use `dodge`. Each bar then keeps its own baseline, so a negative bar hangs below the axis where a reader can see that it is negative. If the total matters, take the absolute value in the host first. You are then stacking magnitudes, and the sentence says so.

`stack(share = TRUE)` fails for the same reason, and the failure is easier to see. A share is a fraction of the pile’s total. Here the total is 5 plus -3, which is 2, so the two shares come out as 2.5 and -1.5. No rescaling fixes that. The problem is in the pile itself, not in how the pile is drawn.

Piles at *different* positions may still point different ways. Each pile is read on its own. One quarter can be positive and the next negative in the same plot.

25.21.5 The dot plot

A point has no length. Give it the same span every other mark gets and the only thing it can spend it on is *itself*: one dot per unit, piled up from the foot. The unit of a count is one row, so a pile of dots is a count you can count:

R

```
data(gapminder_2007) + point * bin * stack + x(life) +
  x_label("Life expectancy (years)") + title("One dot per country")
```

Python

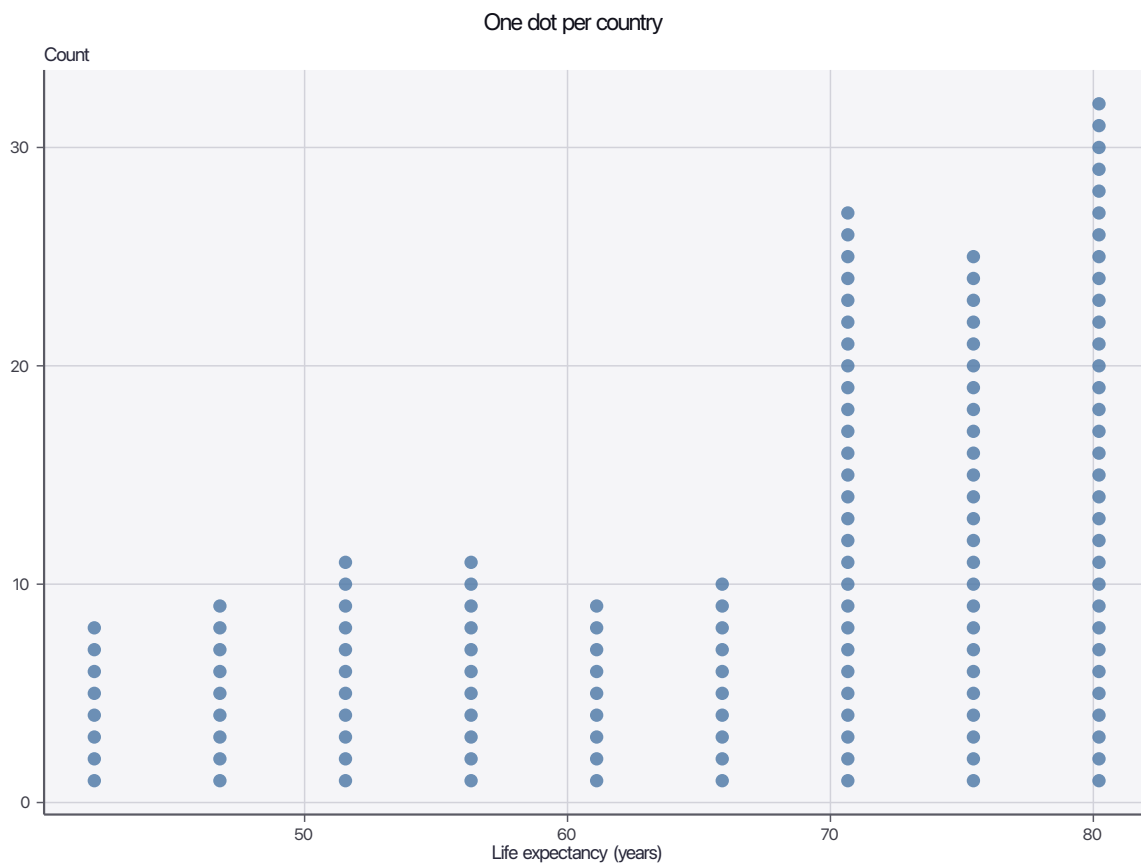
```
(data(gapminder_2007) + point * bin * stack + x(col.life) +  
  x_label("Life expectancy (years)") + title("One dot per country"))
```

Julia

```
data(gapminder_2007) + point * bin * stack + x(:life) +  
  x_label("Life expectancy (years)") + title("One dot per country")
```

JavaScript

```
plot(data(gapminder_2007), layer(point, bin, stack), x(col.life),  
  x_label("Life expectancy (years)"), title("One dot per country"))
```



“Given gapminder 2007: points derived by bin and stack, x is life.”

That is a dot plot (Wilkinson, 1999), and it is the same sentence as the histogram at the top of this chapter with one atom changed and one added. Compare the two: `bar * bin` draws each tally as a length, and you read the shape off the outline; `point * bin * stack` draws each tally as that many dots, and every country in the table is on the page. Count the tallest pile and you get its bin's count exactly, which no bar can offer.

Nothing here is a color split, and none is needed. `bar` and `area` pile *groups*, so they ask for one; a point piles *rows*, which every table already has. What it asks for instead is a transform that counts them, because a pile of 3.7 dots means nothing:

```
data(gapminder_2007) + point * mean * stack + x(continent) + y(life)
```

Error:

```
! gog: `point * stack` piles one dot per observation along the measure axis -  
  ↳ the dot plot - so the measure has to be a count of rows, and nothing here  
  ↳ counts them. Add the transform that does: `point * bin * stack` for a  
  ↳ continuous axis (dots per interval), `point * count * stack` for a  
  ↳ categorical one (dots per category).
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

The refusal names the two sentences that do count: `bin` for a continuous axis, and `count` for a categorical one. The second draws the bar chart's abstraction with the unit left visible, one dot per row:

R

```
data(gapminder_2007) + point * count * stack + x(continent) +  
  title("One dot per country, piled by continent")
```

Python

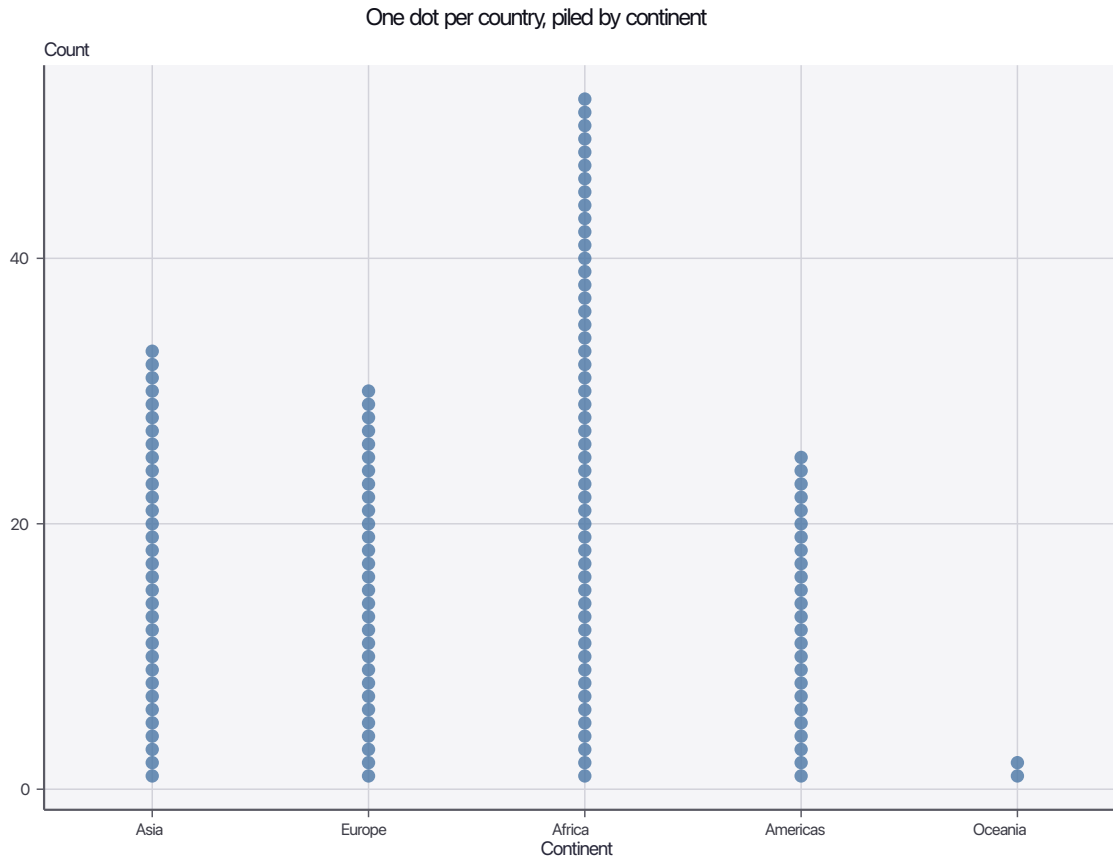
```
(data(gapminder_2007) + point * count * stack + x(col.continent) +  
  title("One dot per country, piled by continent"))
```

Julia

```
data(gapminder_2007) + point * count * stack + x(:continent) +  
  title("One dot per country, piled by continent")
```

JavaScript

```
plot(data(gapminder_2007), layer(point, count, stack), x(col.continent),
     title("One dot per country, piled by continent"))
```



The dots pack tightly here and loosely on a small table, and that is not a setting. The **rung** is the fixed vertical gap between stacked dots, and it is one count unit on the y axis. A taller pile therefore subdivides the panel more finely, and a pile of ten dots in a tall panel leaves wide gaps. `style(size = )` will enlarge the dots, but making them *touch* means fixing the panel's height to the tallest pile, and no setting does that. The count axis is real either way, and that choice is deliberate: the alternative is to draw the dots touching and leave the axis with no meaning.

25.21.6 How many rows is too many

Push the same sentence far enough and the dots stop being dots. Every year of gapminder rather than one is 1704 rows, and the tallest pile is 314 of them:

R

```
data(gm_all) + point * bin * stack + x(life) +  
  x_label("Life expectancy (years)") + title("Every year, every country: 1704  
  ↪ dots")
```

Python

```
(data(gm_all) + point * bin * stack + x(col.life) +  
  x_label("Life expectancy (years)") + title("Every year, every country: 1704  
  ↪ dots"))
```

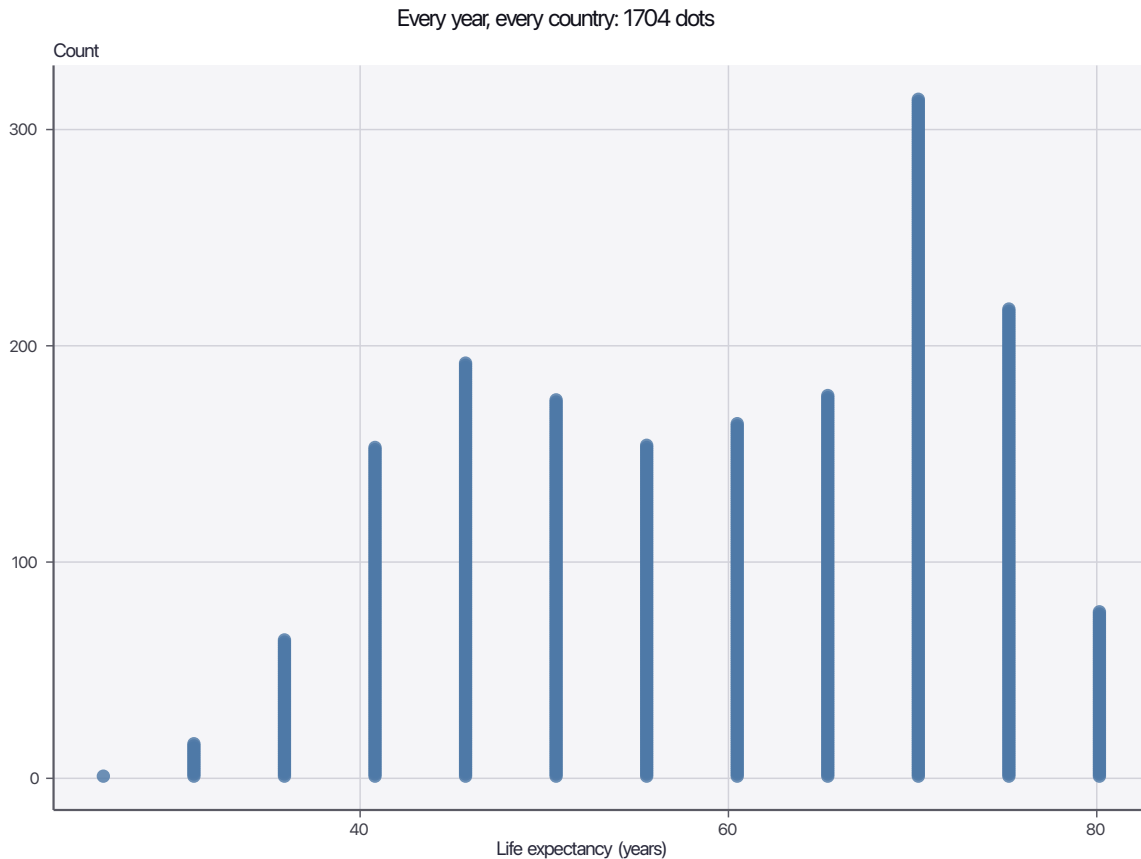
Julia

```
data(gm_all) + point * bin * stack + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("Every year, every country: 1704 dots")
```

JavaScript

```
plot(data(gm_all), layer(point, bin, stack), x(col.life),  
  x_label("Life expectancy (years)'),  
  title("Every year, every country: 1704 dots"))
```

gog: `point \* bin \* stack` piled 314 dots into one column, and at that height
 ↪ they overlap instead of counting - a dot plot says what it says by
 ↪ showing every observation separately, so this is a histogram drawn dot by
 ↪ dot. For this many rows, `bar \* bin` reads the same shape as a summary,
 ↪ and `line \* density` as a curve. `style(size = )` shrinks the dots, which
 ↪ buys a little room.



`gog` draws it, and says what happened. The piles have become solid sticks: at 314 rungs the gap between two of them is under a pixel, so the dots overlap and nothing can be counted, which was this plot’s entire claim. What you are looking at is a histogram drawn dot by dot, and a costly one: it draws 1704 circles where `bar * bin` draws twelve rectangles.

The warning is not a row limit, and that matters. It compares one rung’s gap against one dot’s diameter, both of which `gog` knows once the panel is laid out. So it fires exactly when the picture degrades and stays quiet when it does not. Shrink the dots with `style(size = )` and the same table draws no warning; widen the piles and a smaller table draws one. A number like “more than 500 rows” would be wrong at some panel size or dot size, and there is no need to guess one when the real condition is available.

Which also means the warning is advice, not a limit. Two atoms usually answer it without giving up a single dot, and the first is the one people forget: **more bins** spread the same rows over more piles, so every pile gets shorter without anything getting smaller. Then the dots themselves:

R

```
data(gm_all) + point * bin(40) * stack + x(life) + style(size = 1.5) +
  x_label("Life expectancy (years)") + title("The same 1704 rows, cut finer
↪ and drawn smaller")
```

Python

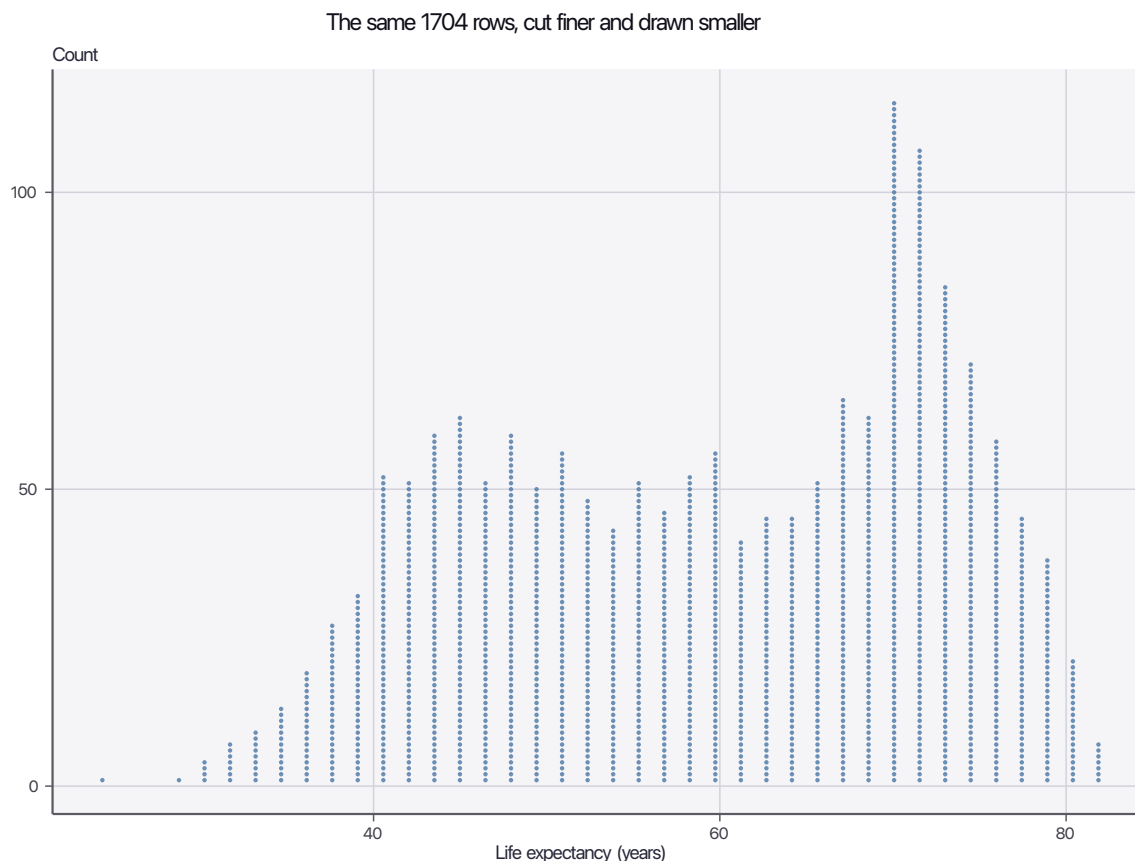
```
(data(gm_all) + point * bin(40) * stack + x(col.life) + style(size = 1.5) +
  x_label("Life expectancy (years)") + title("The same 1704 rows, cut finer
↪ and drawn smaller"))
```

Julia

```
data(gm_all) + point * bin(40) * stack + x(:life) + style(size = 1.5) +
  x_label("Life expectancy (years)") +
  title("The same 1704 rows, cut finer and drawn smaller")
```

JavaScript

```
plot(data(gm_all), layer(point, bin(40), stack), x(col.life),
  style({ size: 1.5 }), x_label("Life expectancy (years)",
  title("The same 1704 rows, cut finer and drawn smaller"))
```

“Given all the gapminder years: points derived by bin into 40 and stack, x is life, with size 1.5.”

The columns are made of visible dots again, and every one of the 1704 is still on the page. [Faceting](#) is the third way, since fewer rows per panel means shorter piles in each.

What none of them changes is the arithmetic. Counting needs at least one dot-diameter of axis per row, so the tallest honest pile is about the panel’s height divided by a dot: a few hundred with dots you can see, a few dozen if you mean to count them. Past that a summary is the better answer, for the reason that makes the dot plot worth having at all. A bin width or a bandwidth is a guess about structure when there are thirty rows. It is a reliable description when there are ten thousand. So the dot plot stops working at the same size where a summary starts working well.

25.21.7 One column, split into its parts

Every stacked bar so far has stood at a position, one pile per category on the x-axis. Take the position away and the split still has something to divide: a single column, showing how one total breaks up.

R

```
data(winds) + bar * count * stack + color(direction)
```

Python

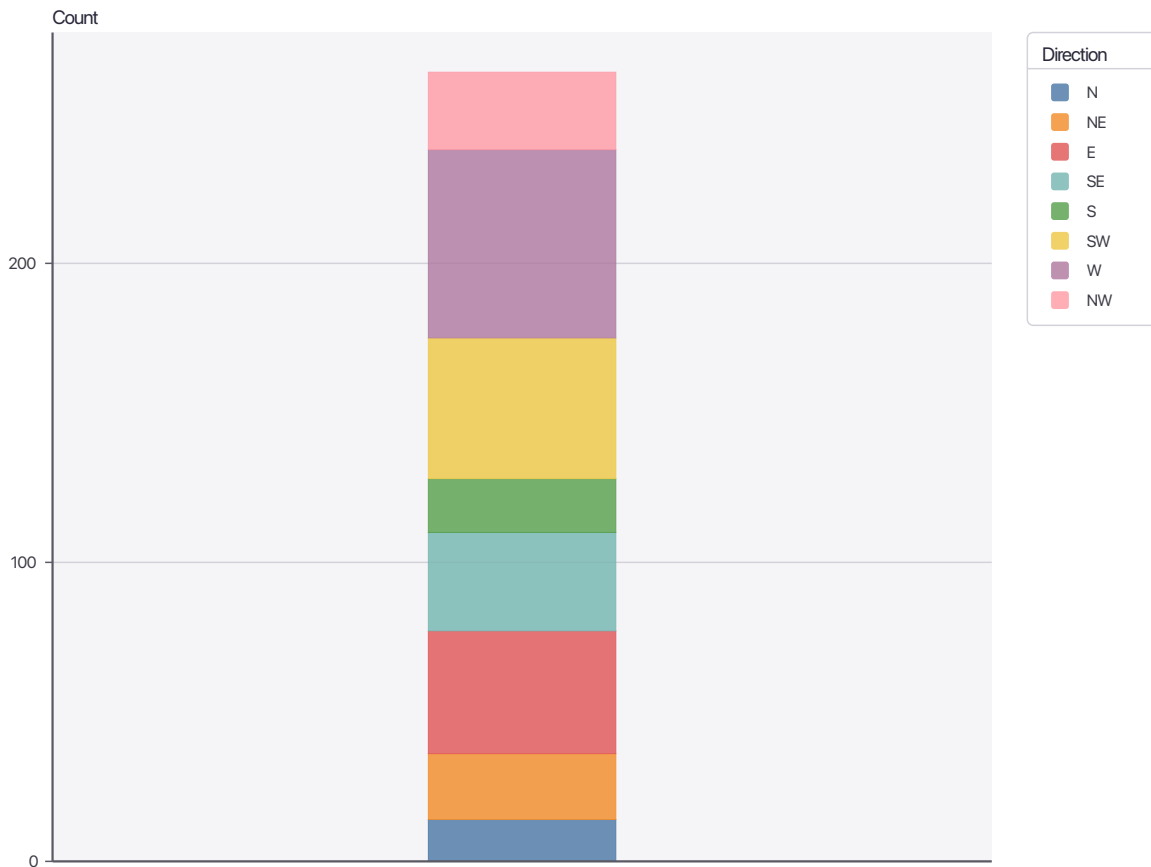
```
data(winds) + bar * count * stack + color(col.direction)
```

Julia

```
data(winds) + bar * count * stack + color(:direction)
```

JavaScript

```
plot(data(winds), layer(bar, count, stack), color(col.direction))
```



“Given the winds: bars derived by count and stack, color by direction.”

There is no `x()` in that sentence, and none is needed. A bar normally requires a position because it has to stand somewhere, but here the `split` is the segmentation and there is one slot for it to divide, so the position is no longer required. A required position may be dropped in only a few places in the grammar, and this is one of them. Bend this same plot into a circle and it is a pie, the subject of [Polar](#).

Without a `split` there is nothing to divide, so the position is required again and the refusal says so. The statistic has to be one that means something with nothing to group by, too: `count` and `sum` reduce a set of rows to one number, while `bin` and `density` describe how values spread *along* an axis, and there is no axis here.

25.22 jitter: spread sideways, along the category

`dodge` and `stack` resolve the overlap of *grouped* marks. The third collision modifier resolves a different overlap: not groups colliding, but a mark's *own* points landing on top of each other. A [strip plot](#), every country as a dot above its continent, stacks its points into a single vertical line, and heavy overlap hides how many there are:

R

```
data(gapminder_2007) + point + x(continent) + y(life) + style(opacity = 0.5)
↪ +
  y_label("Life expectancy") + title("Strip plot: the points pile onto one
↪ line")
```

Python

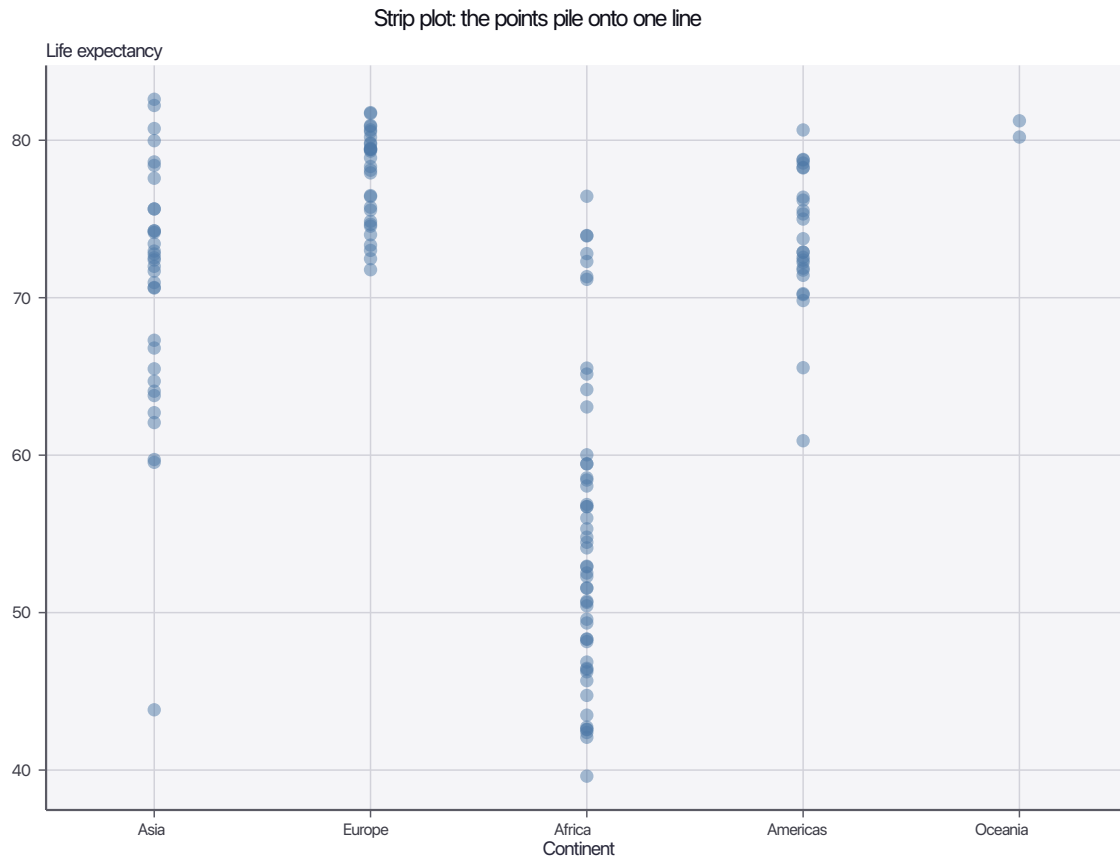
```
(data(gapminder_2007) + point + x(col.continent) + y(col.life) +
↪ style(opacity = 0.5) +
  y_label("Life expectancy") + title("Strip plot: the points pile onto one
↪ line"))
```

Julia

```
data(gapminder_2007) + point + x(:continent) + y(:life) +
  style(opacity = 0.5) + y_label("Life expectancy") +
  title("Strip plot: the points pile onto one line")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.continent), y(col.life),
  style({ opacity: 0.5 }), y_label("Life expectancy"),
  title("Strip plot: the points pile onto one line"))
```



`jitter` nudges each point a little to the side, so the pile spreads and the shape of each group shows:

R

```
data(gapminder_2007) + point * jitter + x(continent) + y(life) +
  ↪ style(opacity = 0.5) +
  y_label("Life expectancy") + title("point * jitter: the same points, spread
  ↪ to show density")
```

Python

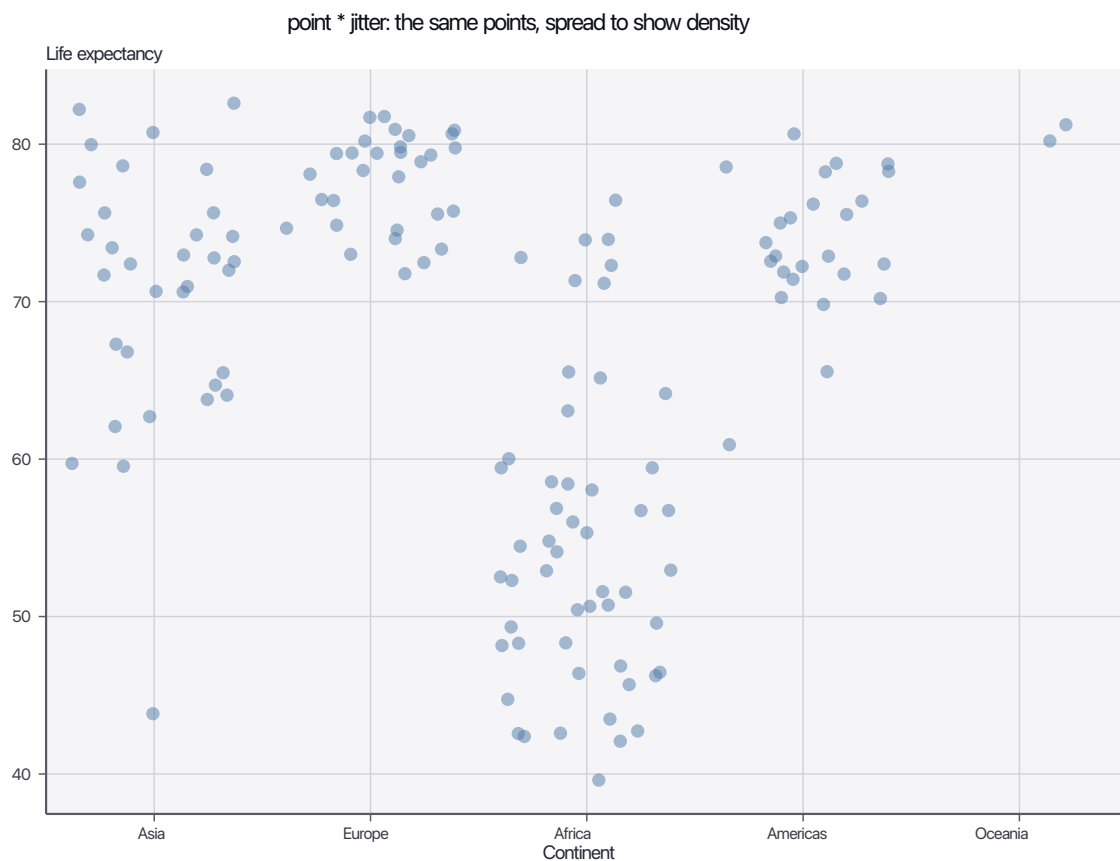
```
(data(gapminder_2007) + point * jitter + x(col.continent) + y(col.life) +
  ↪ style(opacity = 0.5) +
  y_label("Life expectancy") + title("point * jitter: the same points, spread
  ↪ to show density"))
```

Julia

```
data(gapminder_2007) + point * jitter + x(:continent) + y(:life) +  
  style(opacity = 0.5) + y_label("Life expectancy") +  
  title("point * jitter: the same points, spread to show density")
```

JavaScript

```
plot(data(gapminder_2007), layer(point, jitter), x(col.continent),  
  y(col.life), style({ opacity: 0.5 }), y_label("Life expectancy"),  
  title("point * jitter: the same points, spread to show density"))
```



“Given gapminder 2007: points derived by jitter, x is continent, y is life.”

Africa’s long tail of low values and Europe’s tight cluster near the top were both there before; the spread is what lets you read them.

`jitter` moves the *category*, never the *value*. Look again: every dot sits at exactly the same height as before; the spread is entirely sideways. This is the rule, not a coincidence. The horizontal axis is a category, and a category has no magnitude, so nudging a point along it changes nothing you could misread. The vertical axis is a *measurement*, and moving a point off its measured value would be a lie about the data, so `jitter` never touches it. (Spreading in both directions is the other way to do this. It moves a point off its measured value, which is why `gog` spreads along one axis only.) On a horizontal strip, with the category on `y`, the spread is vertical instead. `jitter` always spreads the categorical axis and leaves the measured one exact.

The spread is **deterministic**: it is computed from the data itself, not from a random-number generator or the clock, so the same plot drawn twice is pixel-for-pixel the same picture. A specification always means one plot.

25.22.1 How much to spread: `jitter(amount)`

The default spread fills a fraction of each category's slot: wide enough to show the pile, narrow enough to leave a gap between categories so no point strays into a neighbor. When you want more or less, `jitter(amount)` scales it: a plain number multiplying the default, so `jitter(0.5)` is half the spread and `jitter(2)` twice it (a bare `jitter` is `jitter(1)`).

R

```
data(gapminder_2007) + point * jitter(0.5) + x(continent) + y(life) +  
  style(opacity = 0.5) + y_label("Life expectancy") +  
  title("point * jitter(0.5): a tighter spread")
```

Python

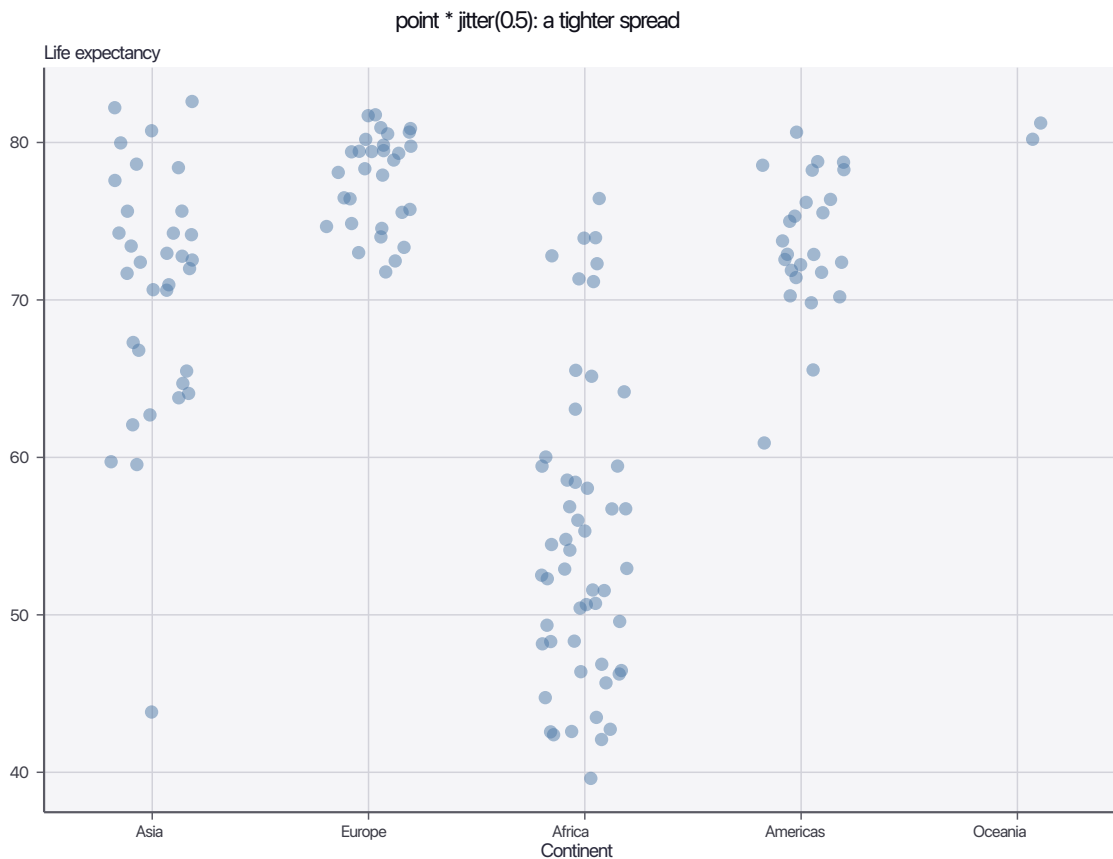
```
(data(gapminder_2007) + point * jitter(0.5) + x(col.continent) + y(col.life)  
↪ +  
  style(opacity = 0.5) + y_label("Life expectancy") +  
  title("point * jitter(0.5): a tighter spread"))
```

Julia

```
data(gapminder_2007) + point * jitter(0.5) + x(:continent) + y(:life) +  
  style(opacity = 0.5) + y_label("Life expectancy") +  
  title("point * jitter(0.5): a tighter spread")
```

JavaScript

```
plot(data(gapminder_2007), layer(point, jitter(0.5)), x(col.continent),
     y(col.life), style({ opacity: 0.5 }), y_label("Life expectancy"),
     title("point * jitter(0.5): a tighter spread"))
```



“Given gapminder 2007: points derived by jitter at 0.5, x is continent, y is life.”

The parameter has the same shape as **density**’s **adjust**, a dimensionless multiple of an automatic value. It is there for the same reason **density** and **bin** take a parameter while **dodge** does not. The jitter amount is a *legibility choice* with no single right value, whereas **dodge**’s width is *determined* by the group count. A value the writer must choose takes a parameter; a value **gog** can work out does not. (**jitter**(0) is legal and draws the plain strip plot, un-spread: **gog** never forbids the harmless.)

Two things set **jitter** apart from its siblings. It needs **no color split**: **dodge** and **stack** separate groups, but **jitter** separates a mark’s own coincident points, so a plain **point * jitter** with nothing but **x** and **y** is the whole expression. And it needs a **categorical position**

axis to spread within: ask for **jitter** when both **x** and **y** are continuous and **gog** refuses, because there is no band to spread across and no axis it could move without misplacing a value. The right tool for overplotting on two continuous axes is **style(opacity =)**, which reveals density without moving any point.

Which marks **jitter** serves is decided by what a mark *is*: the rule that divides the whole trio. **jitter** spreads a scatter of individual points, so it is defined for **point** alone. A **bar**, **box** or **interval** has a width, and its groups are set apart by **dodge**; a **line** or **area** is one connected shape, not a cloud of points. Ask for **jitter** on any of those and **gog** refuses, naming the collision modifier that fits.

Three of the four collision modifiers divide the marks by geometry **and by axis**: **dodge** subdivides a width, **stack** accumulates along the *measure* axis, **jitter** spreads along the *categorical* one. Every mark has an honest answer to overlap, and **point** has two, because it is the one mark that can collide on either axis: piled into a countable stack of dots along a measure, nudged apart within a slot along a category. The modifier follows the axis, which is why neither reading needs a third name.

25.23 **repel**: move the labels off each other

The fourth collision modifier answers a collision the first three cannot see. They all resolve marks that landed on one *position*, which is a fact about the data. A label is as wide as the word it draws, so two labels overlap at positions their points never shared, and no modifier computed from a position can find them. Thirty European countries, each named:

R

```
data(gm_europe) + point + style(color = "#9e9e9e") +
  text + x(gdp) + y(life) + label(country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The names overlap")
```

Python

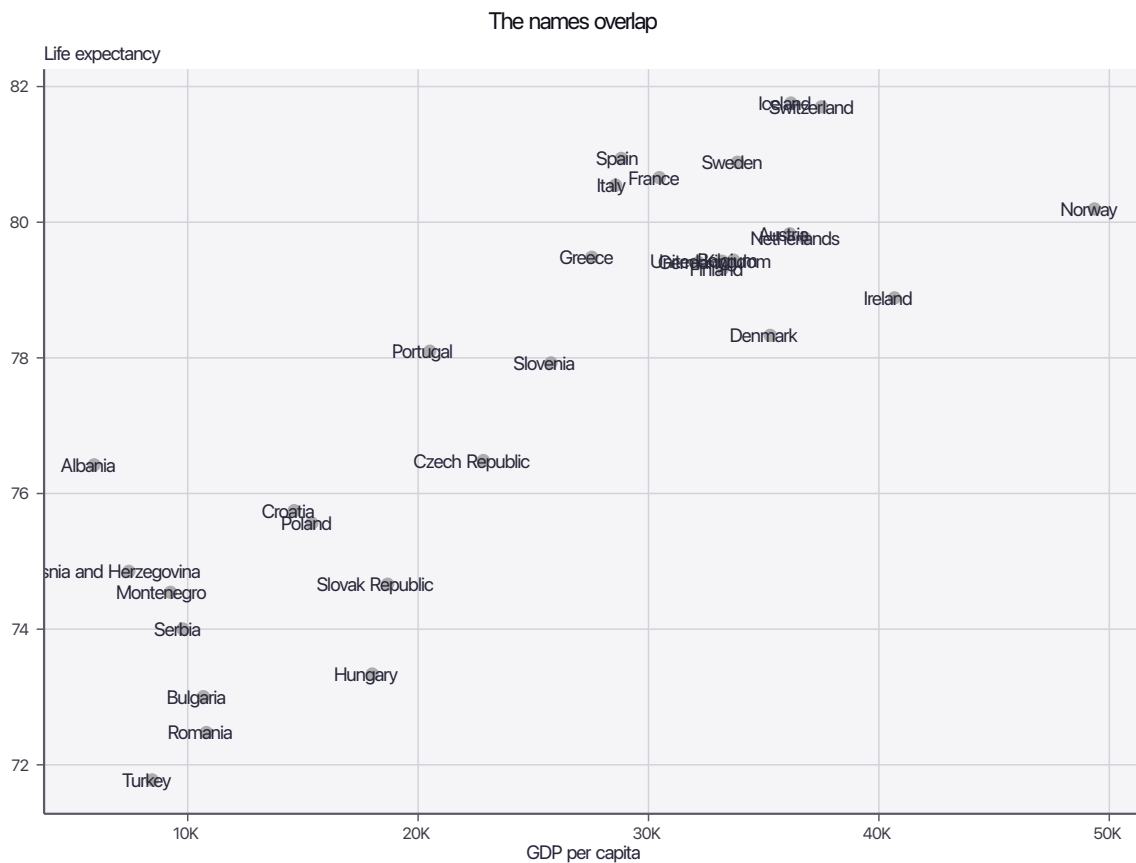
```
(data(gm_europe) + point + style(color = "#9e9e9e") +
  text + x(col.gdp) + y(col.life) + label(col.country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("The names overlap"))
```

Julia

```
data(gm_europe) + point + style(color = "#9e9e9e") + text + x(:gdp) +
  y(:life) + label(:country) + x_label("GDP per capita") +
  y_label("Life expectancy") + title("The names overlap")
```

JavaScript

```
plot(data(gm_europe), point, style({ color: "#9e9e9e" }), text,
  x(col.gdp), y(col.life), label(col.country), x_label("GDP per capita"),
  y_label("Life expectancy"), title("The names overlap"))
```



`text * repel` measures the ink each label will take, and moves the labels until no two words overlap:

R

```
data(gm_europe) + point + style(color = "#9e9e9e") +
  text * repel + x(gdp) + y(life) + label(country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("text * repel: the same names, moved apart")
```

Python

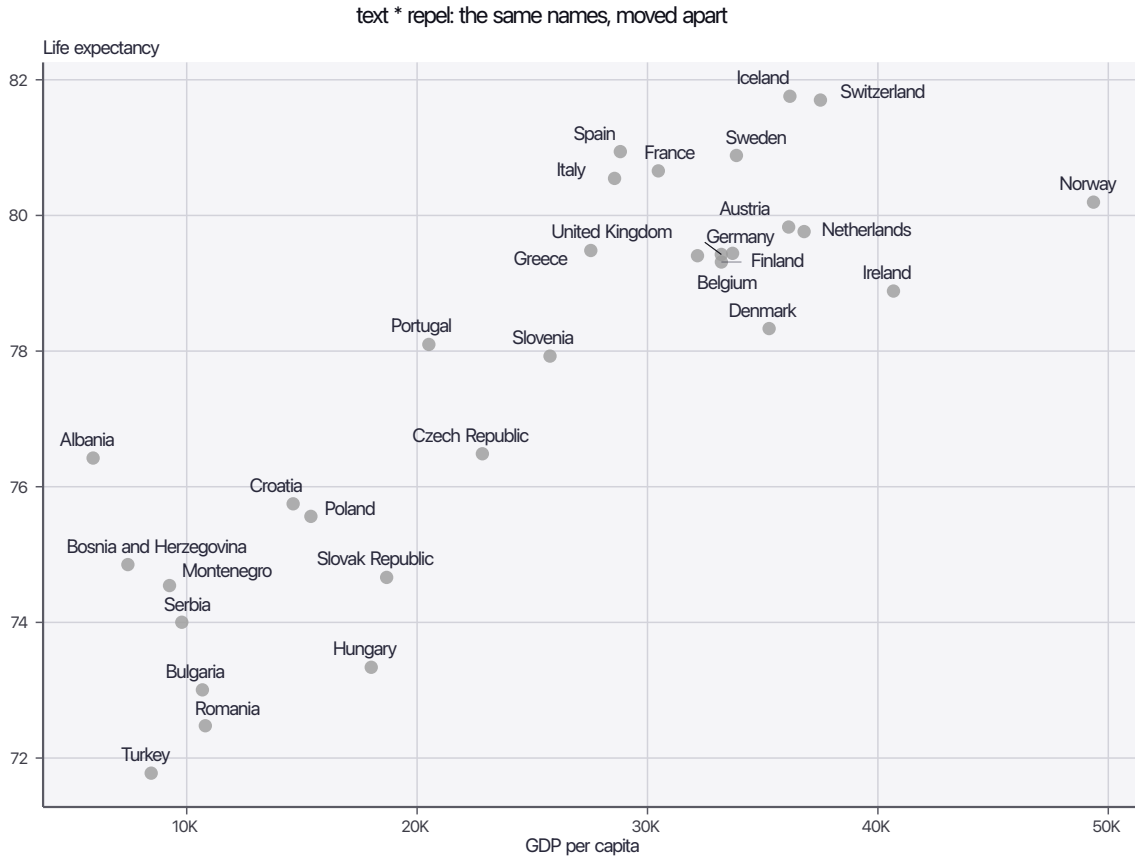
```
(data(gm_europe) + point + style(color = "#9e9e9e") +
  text * repel + x(col.gdp) + y(col.life) + label(col.country) +
  x_label("GDP per capita") + y_label("Life expectancy") +
  title("text * repel: the same names, moved apart"))
```

Julia

```
data(gm_europe) + point + style(color = "#9e9e9e") + text * repel +
  x(:gdp) + y(:life) + label(:country) + x_label("GDP per capita") +
  y_label("Life expectancy") +
  title("text * repel: the same names, moved apart")
```

JavaScript

```
plot(data(gm_europe), point, style({ color: "#9e9e9e" }),
  layer(text, repel), x(col.gdp), y(col.life), label(col.country),
  x_label("GDP per capita"), y_label("Life expectancy"),
  title("text * repel: the same names, moved apart"))
```



“Given gapminder Europe: points, and also text derived by repel, x is gdp, y is life, label by country.”

Every label ends up outside its own dot, and one that moved far keeps a thin line back to its point. Two rules set where each label rests. First, a label rests beside its dot, on whichever side has room. **repel** pulls the label toward its dot and pushes it away from every word and every dot, and the label stops where the two balance. A label with no neighbors sits just above its dot. One with neighbors moves to whichever side is free. Second, when a label is pushed by another label and by its own dot at the same time, it moves off the other label and may rest over its dot. A word over a dot is still readable. Two words over each other are not.

Like **jitter**, the placement is **deterministic**: it depends only on the labels and the rows, never on a random-number generator, so one specification always draws the same picture. Like **jitter**, it needs **no color split**, since what it separates is the layer’s own labels.

repel takes no parameter, and **jitter** takes one, for the same reason. A jitter’s spread has no single right value, so the writer chooses it. A label moves exactly as far as the overlap requires and no further. That distance is computed, the way **dodge** computes a width, so there

is nothing to set. The gap left between two labels is set by the font size, as the nudge's distance is.

Above some number of labels, no arrangement fits at all. **repel** draws every one of them anyway and reports how many still overlap. A plot missing the names it could not fit would read as a plot with fewer rows, and nobody looking at it would know.

repel is **text**-only. Each of the other three serves the marks whose geometry fits it, and a label is the only glyph made of a word. Ask for it on a **point** and **gog** refuses, naming **jitter**; on a **bar** it names **dodge**. Its counterpart among the settings is **style(nudge =)**, a *fixed* shift in a direction you name, described in [Text](#). The two compose: the nudge says which side a label prefers, and **repel** resolves the overlaps that remain.

The four collision modifiers are one family, and two questions divide it. Three of them ask *what geometry does this mark have*, and answer with a width, a measure or a band. The fourth asks *what is this mark made of*, and only one mark is made of words.

Part III

The sentences

Letters are not yet writing. The 1446 book, after explaining each letter class, devoted a chapter to assembly: how letters pack into syllable blocks, blocks into words, words into sentences.

This part is **gog**'s assembly chapter, and it moves through six groups.

First the mechanics. The **+** operator layers marks in one panel, and the ***** operator derives a new statement from a mark and a transform. Position in the sentence then decides which layer a channel binds to. Second, one pair of operators with two uses. **|** and **/** split one plot into a grid of panels, and the same pair arranges separate plots on a page. Third, two of the picture-computing transforms: **flow** carries a magnitude through its stages, and **cluster** builds the tree that puts an axis in order.

Fourth come the coordinate spaces. **space** stands a flat plot up by adding a single vowel, **z**. **polar** bends the same plane into a circle. **nest** packs a panel into regions. **map** flattens the earth onto the page, and **globe** keeps it round. Those five ask nothing new of the sentence. **network** does ask something new: it places a graph with no axes at all, from a transform that lays out its nodes. Fifth is time, where **play** cuts the plot into frames and runs them in order. Last is your own choice: **brush** selects rows with the mouse, and the plot dims what you did not select.

One law holds this part together, [Compositional Invariance](#): a composed expression means the same thing wherever it appears. What you build in one chapter can be carried into the next, and it still means the same thing. That is what makes sentences worth building.

26 Operators

How do you put two things in one picture? `gog` has exactly **four operators**. They are not atoms. They are the assembly rules: how atoms combine.

Think of them the way Hangeul (한글) works: consonants and vowels are the atoms; the rule “initial + vowel + optional final = syllable block” is the operator. The rule never changes, and the number of things you can build with it has no limit.

+	compose	(add a layer, a channel, or a label to a specification)
*	derive	(combine a mark with a transform into a compound mark)
	facet ↔	(small multiples, arranged in columns)
/	facet ↑	(small multiples, arranged in rows)

| and / also arrange whole plots on a page, and that second use has its own section below.

26.1 +: compose

+

 is the general assembly operator. Every atom enters a specification via +:

```
data(df) + point + x(col_a) + y(col_b) + color(group) + title("My plot")
```

Adding a **mark** (`point`, `line`, `bar`, ...) starts a new layer. Adding a **channel** (`x()`, `color()`, ...) or **label** (`title()`) attaches to the appropriate scope (plot-level or the nearest preceding mark, see [Encoding scope](#)).

26.1.1 Layering with +

If you have edited video, you already know this idea. A video is a stack of tracks: the picture, the sound, the captions, the logo, the effects. Each track is separate, they play at the same time, and you see one finished video. A plot layers the same way.

Multiple marks in the same expression produce multiple layers, rendered in order:

R


```
# Scatter plot: points, and a horizontal reference line at each band
data(gapminder_2007) + x(gdp) + y(life) +
  point + color(continent) +
  data(life_bands) + rule + style(color = "darkgray", pattern = "dashed") +
  title("GDP vs Life Expectancy, 2007")
```

Python

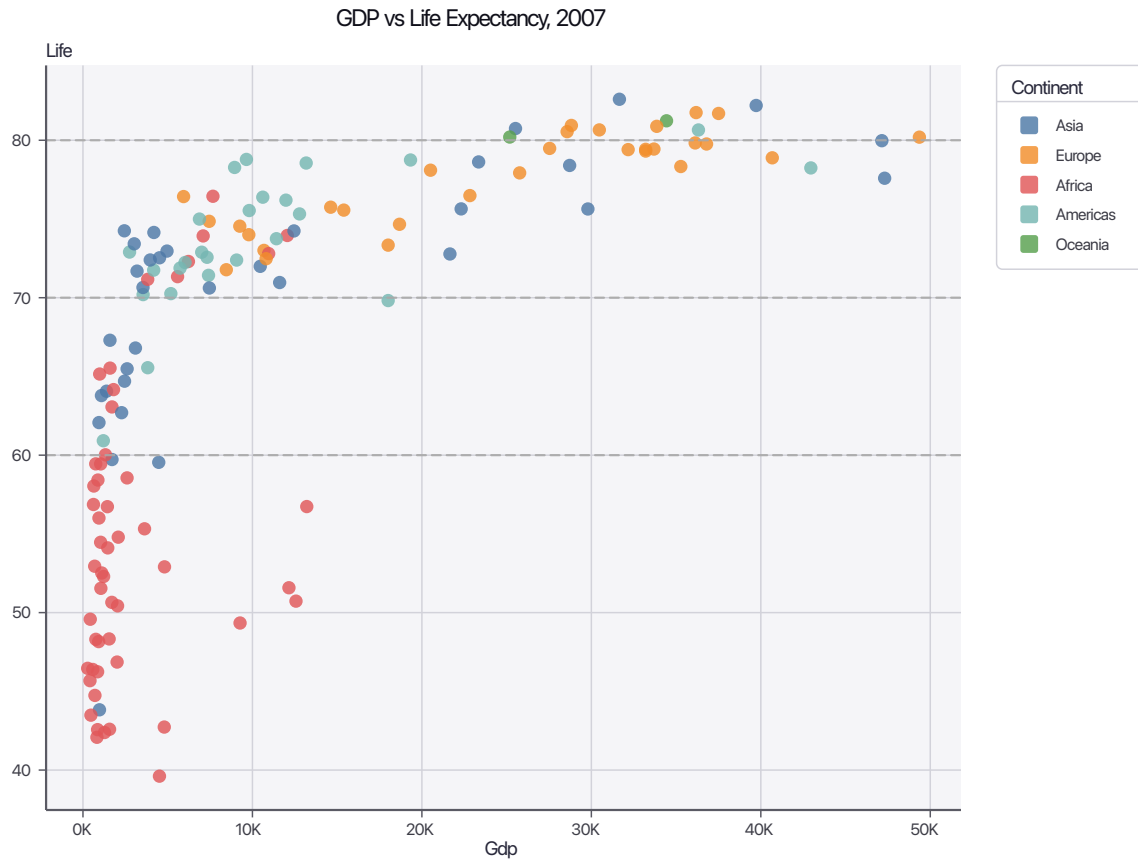
```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +
  point + color(col.continent) +
  data(life_bands) + rule + style(color = "darkgray", pattern = "dashed") +
  title("GDP vs Life Expectancy, 2007"))
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + point + color(:continent) +
  data(life_bands) + rule +
  style(color = "darkgray", pattern = "dashed") +
  title("GDP vs Life Expectancy, 2007")
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), point,
  color(col.continent), data(life_bands), rule,
  style({ color: "darkgray", pattern: "dashed" }),
  title("GDP vs Life Expectancy, 2007"))
```



“Given gapminder 2007: x is gdp , y is $life$, points colored by continent, and also rules from the life bands.”

Layers draw in the order they are written: the first mark sits at the bottom. Add a `line` after point to draw the line on top of the points.

26.1.2 The order of `+`: data, mark, positions, refinements

Inside a single layer the channels **commute**, and the mark commutes among them. Write the mark, `x` and `color` in any order and the plot is the same. Three things can be arranged six ways, so here are all six.

R

```
((data(iris_flowers) + bar * bin + x(petal_length) + color(species)) |
  (data(iris_flowers) + bar * bin + color(species) + x(petal_length))) /
```

```
((data(iris_flowers) + x(petal_length) + bar * bin + color(species)) |
  (data(iris_flowers) + x(petal_length) + color(species) + bar * bin)) /
((data(iris_flowers) + color(species) + bar * bin + x(petal_length)) |
  (data(iris_flowers) + color(species) + x(petal_length) + bar * bin))
```

Python

```
((data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species))
↪ |
  (data(iris_flowers) + bar * bin + color(col.species) +
↪ x(col.petal_length))) /
((data(iris_flowers) + x(col.petal_length) + bar * bin + color(col.species))
↪ |
  (data(iris_flowers) + x(col.petal_length) + color(col.species) + bar *
↪ bin)) /
((data(iris_flowers) + color(col.species) + bar * bin + x(col.petal_length))
↪ |
  (data(iris_flowers) + color(col.species) + x(col.petal_length) + bar *
↪ bin)))
```

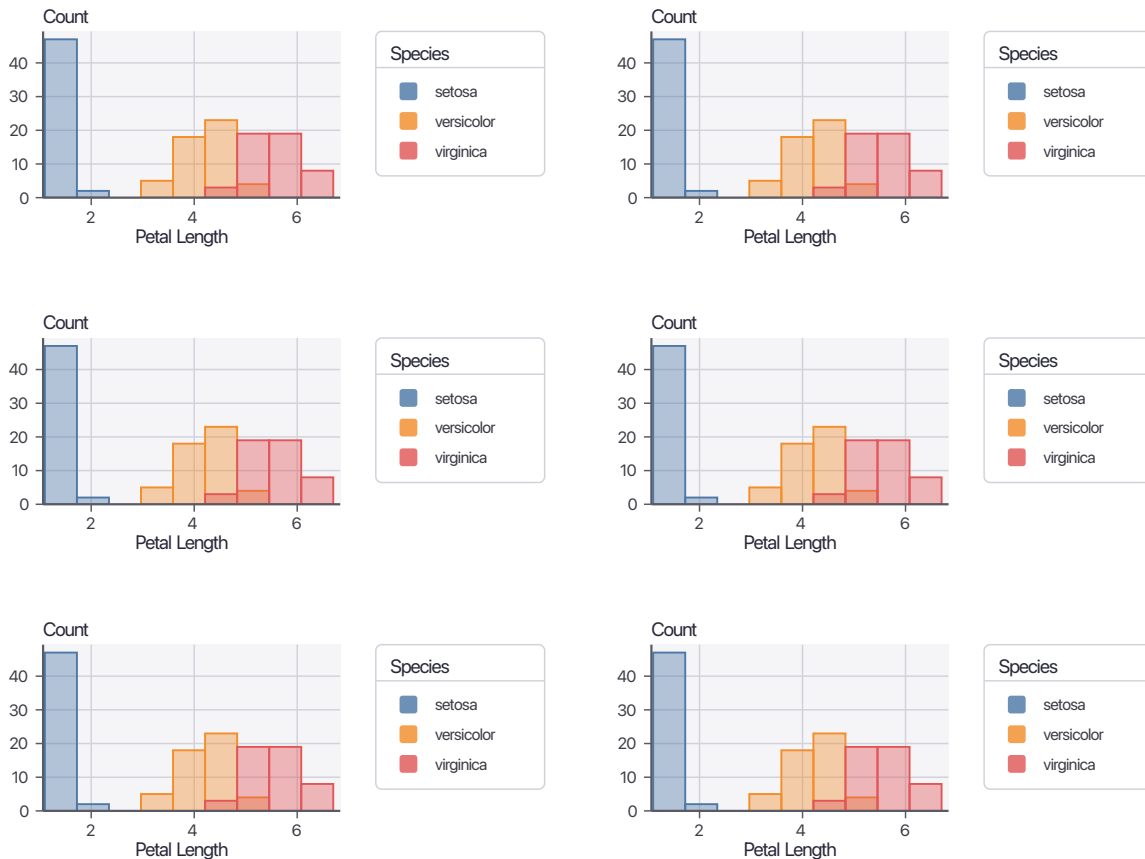
Julia

```
((data(iris_flowers) + bar * bin + x(:petal_length) + color(:species)) |
  (data(iris_flowers) + bar * bin + color(:species) + x(:petal_length))) /
((data(iris_flowers) + x(:petal_length) + bar * bin + color(:species)) |
  (data(iris_flowers) + x(:petal_length) + color(:species) + bar * bin)) /
((data(iris_flowers) + color(:species) + bar * bin + x(:petal_length)) |
  (data(iris_flowers) + color(:species) + x(:petal_length) + bar * bin))
```

JavaScript

```
below(below(beside(plot(data(iris_flowers), layer(bar, bin),
  x(col.petal_length), color(col.species)),
  plot(data(iris_flowers), layer(bar, bin), color(col.species),
  x(col.petal_length))),
  beside(plot(data(iris_flowers), x(col.petal_length), layer(bar, bin),
  color(col.species)),
  plot(data(iris_flowers), x(col.petal_length), color(col.species),
  layer(bar, bin)))))
  beside(plot(data(iris_flowers), color(col.species), layer(bar, bin),
```

```
x(col.petal_length)),
plot(data(iris_flowers), color(col.species), x(col.petal_length),
layer(bar, bin)))
```



“Given the iris flowers: bars derived by bin, x is petal length, color by species.”

Six sentences, and the same histogram six times. Not six plots that look alike: the engine emits the same bytes for every one of them.

So if the order does not change the picture, why does every plot in this book follow the same one: *data, mark, positions, refinements*? Because a fixed order lets a plot read like a sentence, and the sentence has a natural, subject-first shape:

- **data(...)** **first.** It names the table the columns come from. A bare name like **petal_length** means nothing until a table is bound (Bind-Once), so the table is named first. The sentence must start here.

- **The mark and its *-chain second.** This is the subject. A visual is a mark before it is anything else (Minimum Syllable): `bar * mean * dodge` says *what you are drawing* and *what was done to it*, as one unbroken word. It is the initial consonant of the block; everything after is said about it.
- **The positions third.** `x` and `y` say *where* the subject stands, and a mark cannot render without them (the vowel to the mark's consonant), so they follow immediately.
- **The refinements last.** `color`, `size`, `order`, `labels`, `title`: everything optional. Delete every refinement and a legal plot remains; delete the mark or a position and nothing renders. Writing the removable parts last is also how you *build* a plot: the minimum sentence first, then refine ([First plot](#)).

The order is a reading convention, not a grammar rule: the engine accepts any permutation that scopes the same way. Where position genuinely *does* matter (a channel written before or after a mark across two or more layers) is the subject of [Encoding scope](#). One order everywhere is worth learning: every plot in the book, and every plot you write, then reads the same way.

[Which orders matter](#) below sets out what the engine enforces under `+` and `*`, and what you are free to arrange.

26.1.2.1 A refinement far from what it refines

One part of the convention is easy to misread. A grouped bar chart pairs `dodge` with `color`, yet the sentence puts them at opposite ends, with the positions in between:

R

```
data(gm_eras) + bar * mean * dodge + x(continent) + y(life) + color(era) +
  title("Mean life expectancy by continent, 1957 vs 2007")
```

Python

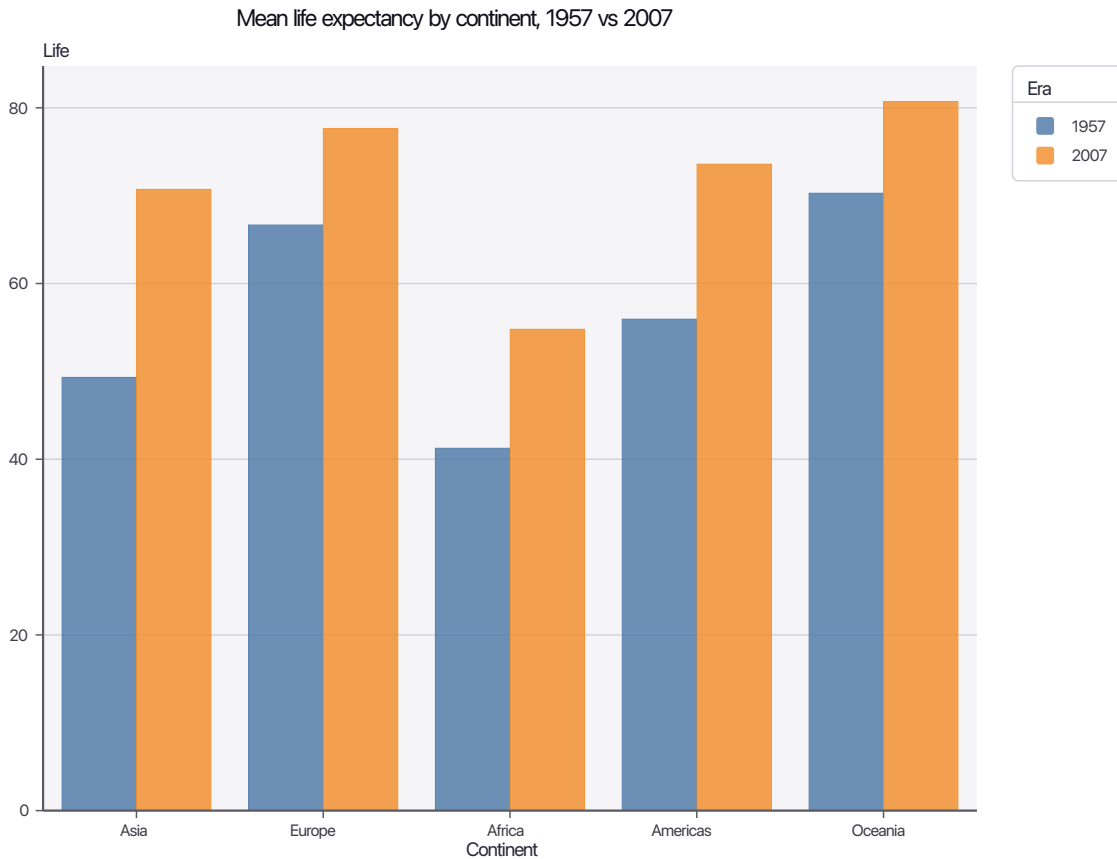
```
(data(gm_eras) + bar * mean * dodge + x(col.continent) + y(col.life) +
  ↪ color(col.era) +
  title("Mean life expectancy by continent, 1957 vs 2007"))
```

Julia

```
data(gm_eras) + bar * mean * dodge + x(:continent) + y(:life) +
  color(:era) + title("Mean life expectancy by continent, 1957 vs 2007")
```

JavaScript

```
plot(data(gm_eras), layer(bar, mean, dodge), x(col.continent),
     y(col.life), color(col.era),
     title("Mean life expectancy by continent, 1957 vs 2007"))
```



“Given the `gapminder` eras: bars derived by `mean` and `dodge`, x is continent, y is life, color by era.”

`dodge` and `color(era)` are plainly related: `dodge` has nothing to set side by side until a `color` (or `group`) names the groups. So why not write them together? Because the grammar deliberately gives them different operators. `dodge` belongs to `*`, because it modifies the mark. The same modifier works on `box * dodge` and `interval * range * dodge`, and neither of those involves color. `color` belongs to `+`: it binds a column to a channel. `color * dodge` cannot be written: `*` needs a mark on its left. Joining the two would make `dodge` depend on which channel splits the groups, and a modifier must not depend on that.

And the order is not a compromise; it is exactly right, once you see what `dodge` does. `dodge` subdivides the x slot: each continent’s space is cut into one sub-bar per era. So it needs *both*

neighbors: `x(continent)` says *which slot* to divide, `color(era)` says *into how many pieces*. The sentence lists them in the order **dodge** uses them: the axis it splits along, then the groups it splits into. The positions do not separate **dodge** from what it needs; one of them is the slot **dodge** divides.

You also never have to scan the line to check the split is there. `bar * mean * dodge` with no `color` or `group` does not draw a lone bar; it is refused, pointing you at the missing channel. The legality check requires both, so writing `color` far from **dodge** is safe: the two are bound whether or not they sit side by side on the page.

26.2 *: derive

`*` joins a transform to a mark, and the result is one **compound mark**.

```
bar * bin      # histogram
line * smooth # smoothed trend line
line * density # kernel density curve
```

Two rules govern `*`, and only two. The mark goes on the left, and the transforms go on the right. Then, among the transforms themselves, the order you write them in does not matter: any order draws the same plot. The rest of this section explains each rule.

26.2.1 Why the order is fixed

In Hangeul, a syllable is always *initial consonant + vowel*, never the reverse. `gog`'s `*` works the same way: the mark is the subject; the transform is what is done to it.

`bin * bar` is refused, just as $\vdash (a) + \neg (g)$ is not a valid Hangeul syllable.

```
bar * bin      # "a bar mark, derived by binning"
bin * bar      # Refused: `` is not defined for transform * mark
```

26.2.2 What * is rigid about

The rule that matters is the one above: the mark is on the left, always, and no transform may stand there. That is what makes `bar * bin` a compound mark rather than a function call, and it is why the atoms can be nouns.

Write the transforms in the order they happen. This is a reading convention: any order draws the same plot, and this one lets you follow the steps as you read. `bar * bin * mean` cuts the axis into bands, then averages inside each one. That is the order it runs in, and there is no other order it could run in.

The reason is what `mean` needs. A statistic reduces a **group** to one number, and a continuous column has no groups: every value stands alone, so its mean is itself. `bin` is what makes the groups, by turning many distinct numbers into one band. So the cut comes first because otherwise the mean has nothing to average.

So two transforms work together only when they do different jobs. One makes the groups, and the other measures inside them or rescales the result. A pair that answers the same question twice is refused rather than sequenced, so `bin` and `count` (both of them tallies with groups of their own) cannot both be in one layer:

```
data(gapminder_2007) + bar * bin(12) * count + x(life)
```

Error:

```
! gog: `bar * bin * count` measures each cell twice - `bin` and `count` each
  ↳ invent their own measurement from the rows, and neither was handed a
  ↳ column to give way to, so there is no reading that keeps both. Keep
  ↳ whichever you meant: `bar * bin` or `bar * count`. To cut an axis into
  ↳ cells and measure something else inside them, the second transform has to
  ↳ be one you hand a column: `bar * bin * mean + x(<number>) + y(<column>)`.
  ↳ To read either as shares of the whole rather than as counts, `proportion`
  ↳ rescales whichever you keep: `bar * bin * proportion`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The engine decides the running order from the transforms themselves, not from where you typed them, so you cannot get it wrong. Write it the way you mean it, and the sentence will read in the order it runs. [Chaining transforms](#) goes through the legal transform pairs one by one.

26.2.3 Live examples

R


```
# Histogram: life expectancy across all countries in 2007
data(gapminder_2007) + bar * bin + x(life) + y(count) +
  x_label("Life expectancy (years)") + y_label("Count") +
  title("Distribution of Life Expectancy, 2007")
```

Python

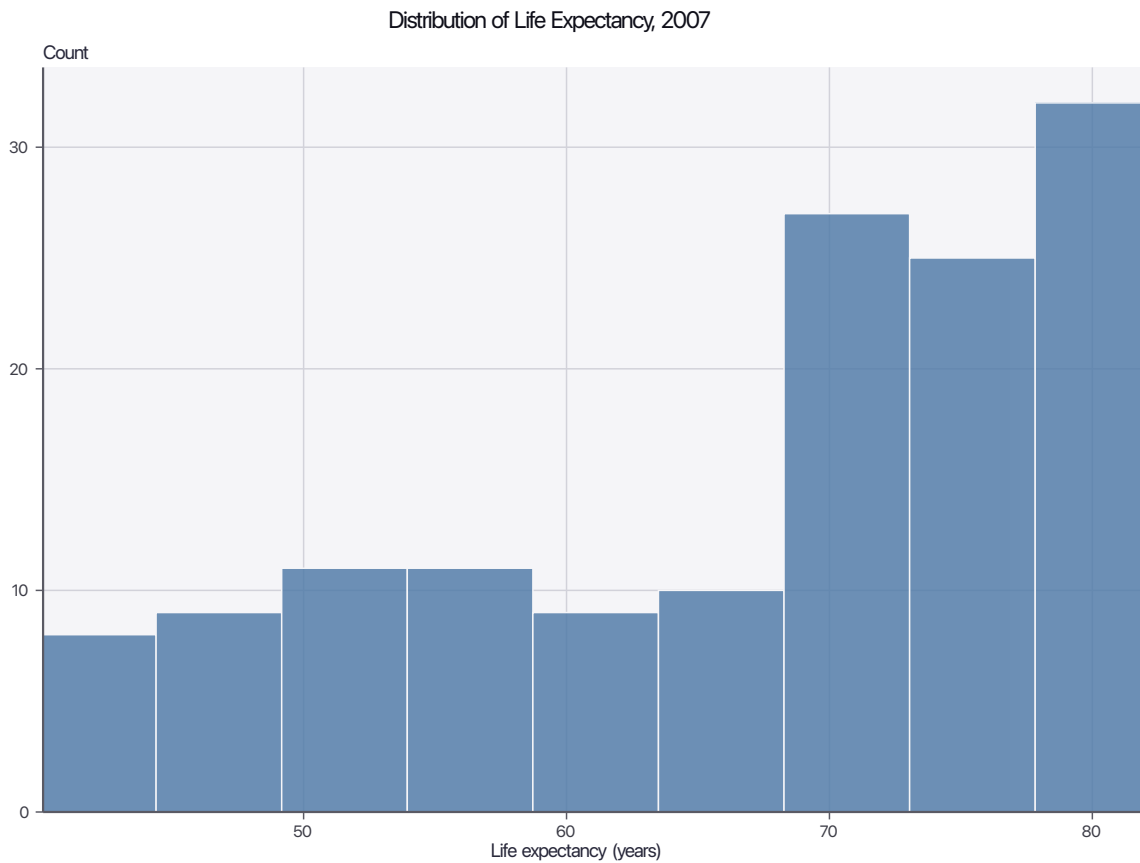
```
(data(gapminder_2007) + bar * bin + x(col.life) + y(col.count) +
  x_label("Life expectancy (years)") + y_label("Count") +
  title("Distribution of Life Expectancy, 2007"))
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) + y(:count) +
  x_label("Life expectancy (years)") + y_label("Count") +
  title("Distribution of Life Expectancy, 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life), y(col.count),
  x_label("Life expectancy (years)", y_label("Count"),
  title("Distribution of Life Expectancy, 2007"))
```



R

```
# Scatter + LOESS smooth layer on top
data(gapminder_2007) + x(gdp) + y(life) +
  point + color(continent) +
  line * smooth +
  title("GDP vs Life Expectancy with LOESS Smooth")
```

Python

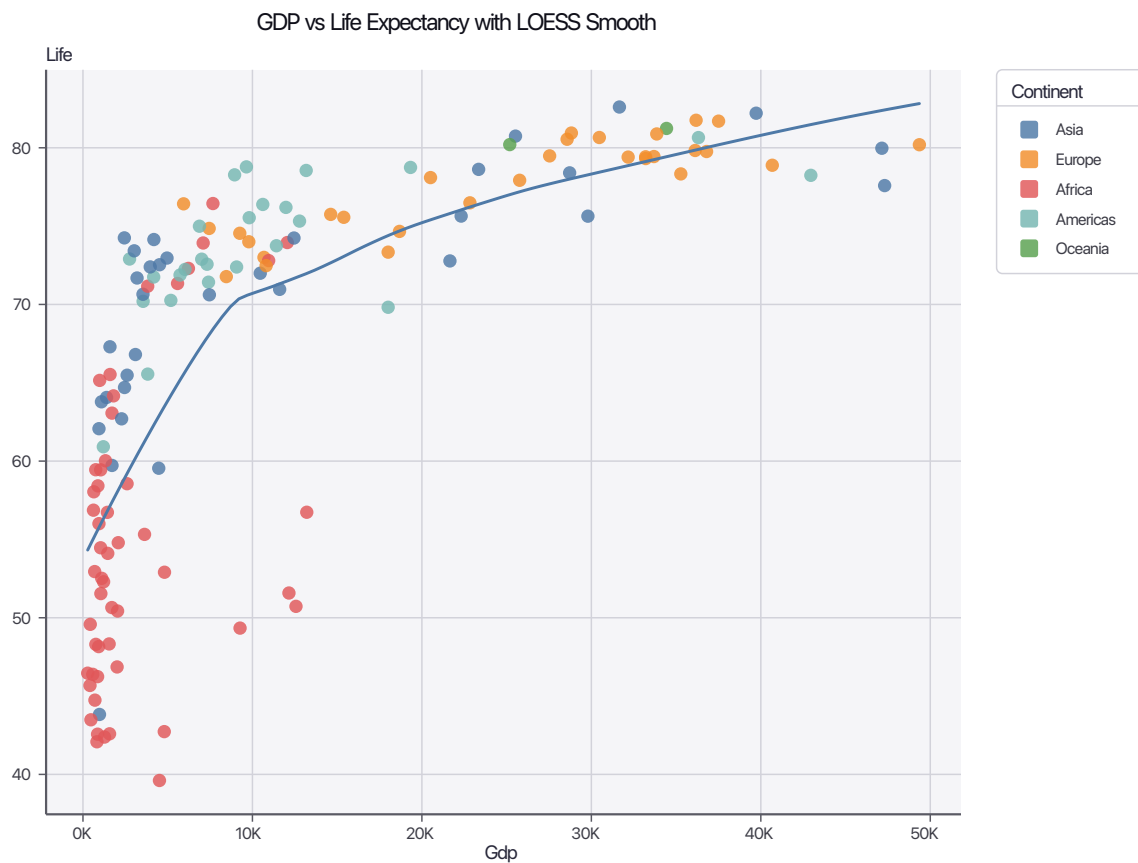
```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +
  point + color(col.continent) +
  line * smooth +
  title("GDP vs Life Expectancy with LOESS Smooth"))
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + point + color(:continent) +
  line * smooth + title("GDP vs Life Expectancy with LOESS Smooth")
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), point,
  color(col.continent), layer(line, smooth),
  title("GDP vs Life Expectancy with LOESS Smooth"))
```



“Given gapminder 2007: x is gdp, y is life, points colored by continent, and also a line derived by smooth.”

R

```
# Kernel density estimate
data(gapminder_2007) + line * density + x(life) + y(density) +
  x_label("Life expectancy (years)") + y_label("Density") +
  title("Kernel Density: Life Expectancy 2007")
```

Python

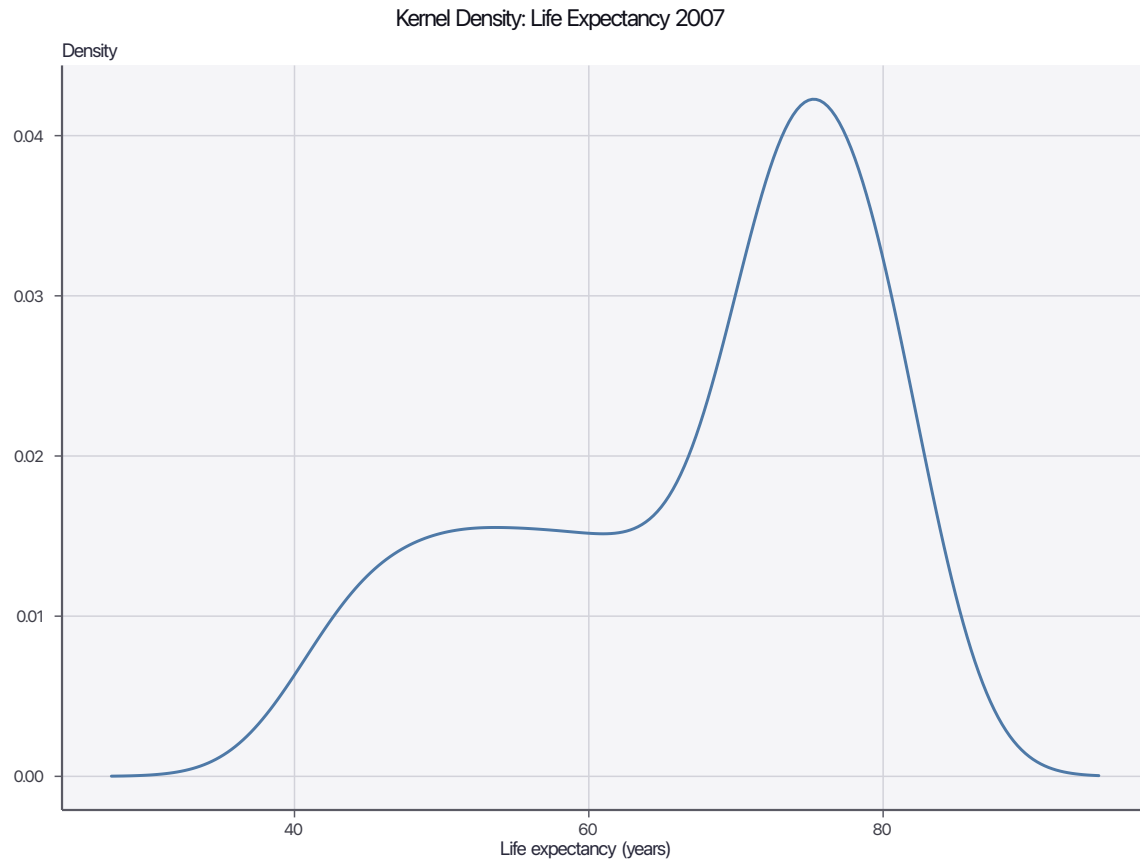
```
(data(gapminder_2007) + line * density + x(col.life) + y(col.density) +
  x_label("Life expectancy (years)") + y_label("Density") +
  title("Kernel Density: Life Expectancy 2007"))
```

Julia

```
data(gapminder_2007) + line * density + x(:life) + y(:density) +
  x_label("Life expectancy (years)") + y_label("Density") +
  title("Kernel Density: Life Expectancy 2007")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density), x(col.life),
  y(col.density), x_label("Life expectancy (years)", y_label("Density"),
  title("Kernel Density: Life Expectancy 2007"))
```



“Given gapminder 2007: a line derived by density, x is life, y is density.”

26.3 | and /: facet, or compose

| puts things side by side and / puts one above the other. What is written on the right decides which. A `facet()` splits one plot into panels; a second plot is placed beside or below the first.

With `facet()`, | produces small multiples arranged in columns, one panel per category of the column it names.

R

```
data(gapminder_2007) + point + x(gdp) + y(life) | facet(continent)
```

Python

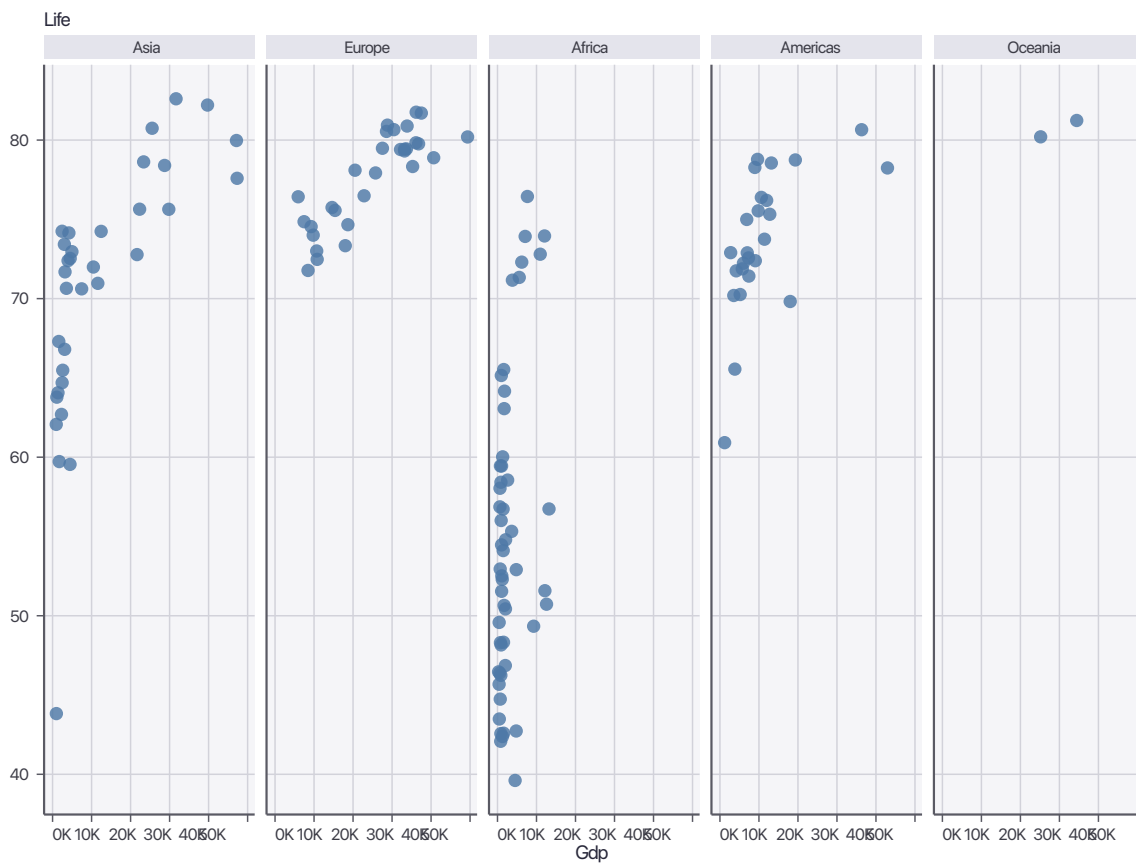
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) |  
  ↪ facet(col.continent)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) | facet(:continent)
```

JavaScript

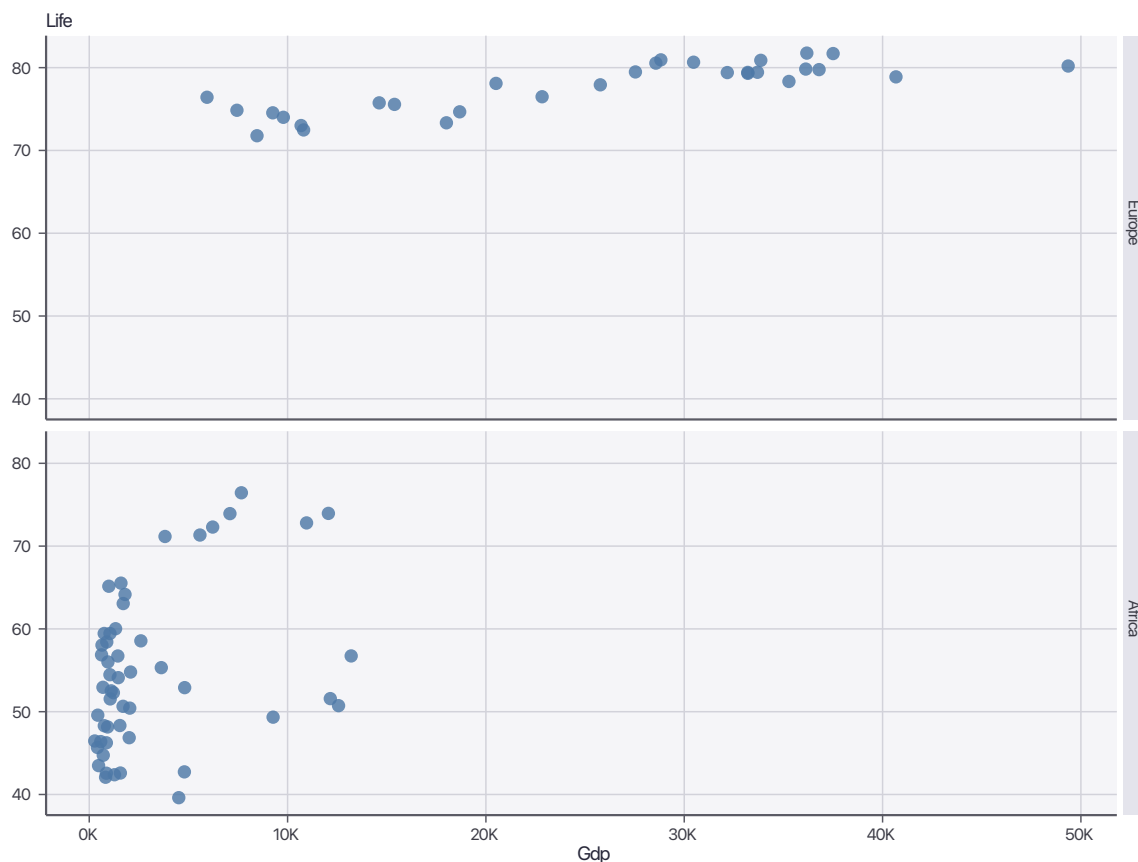
```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  across(col.continent))
```



“Given *gapminder 2007*: points, *x* is *gdp*, *y* is *life*, split into panel columns by continent.”

/ produces small multiples stacked in rows:

```
gm_two <- gapminder_2007[gapminder_2007$continent %in% c("Africa", "Europe"),  
  ↪ ]  
data(gm_two) + point + x(gdp) + y(life) / facet(continent)
```



“Given the *gm two* table: points, *x* is *gdp*, *y* is *life*, split into panel rows by continent.”

Write both to cross them into a grid: each operator applies to the facet written after it, read left to right. See [Faceting](#) for the grid, what the panels share, and why the facet column must be a category.

With a plot on the right, the same two operators arrange separate plots on one page. Each plot keeps its own coordinate space. Two plots side by side share the vertical axis when they

name the same column on it, and two plots one above the other share the horizontal axis the same way:

R

```
(data(gapminder_2007) + point + x(gdp) + y(life)) |  
  (data(gapminder_2007) + bar * count + x(continent))
```

Python

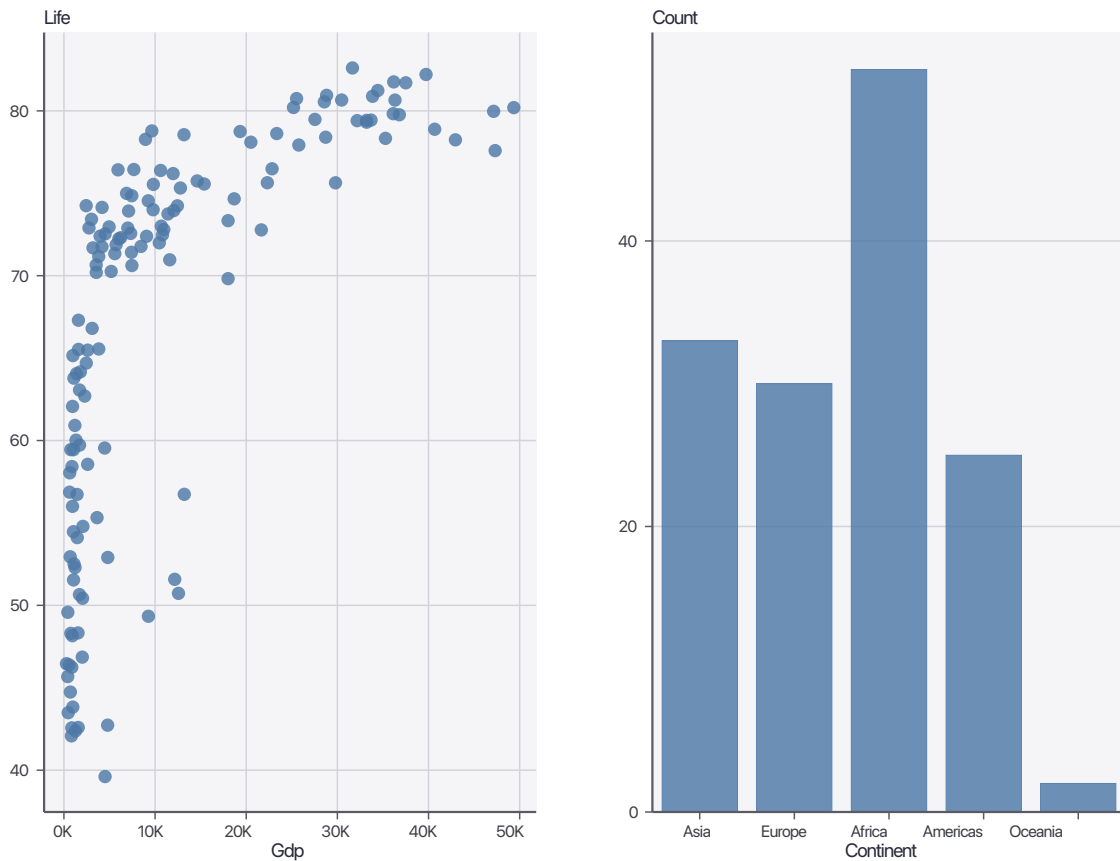
```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) |  
  (data(gapminder_2007) + bar * count + x(col.continent)))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp) + y(:life)) |  
  (data(gapminder_2007) + bar * count + x(:continent))
```

JavaScript

```
beside(plot(data(gapminder_2007), point, x(col.gdp), y(col.life)),  
  plot(data(gapminder_2007), layer(bar, count), x(col.continent)))
```

“Given gapminder 2007: points, x is gdp, y is life, beside bars derived by count, x is continent.”

One pair of operators, two meanings, and what you write on the right decides which one you get. See [Composition](#).

26.4 Which orders matter

This book writes every sentence the same way: data, mark, positions, refinements. That is a preference rather than a rule, and the engine enforces only part of it. The difference is worth knowing, because it tells you where to be careful. There are three kinds of ordering, and only one of them needs your attention.

Kind	What it means	Under +	Under *
Free	Either way draws exactly the same picture.	The mark and its positions, in any arrangement. <code>x</code> before <code>y</code> , or <code>y</code> before <code>x</code> . Refinements among themselves.	Two transforms after one mark. Write them in the order they happen; the engine works that order out either way.
Refused	The engine stops and names what to move.	A sentence that does not begin with <code>data()</code> . A <code>style()</code> written before any mark.	A transform on the left: <code>bin * bar</code> .
Meaningful	No error, and a different picture.	A channel before the marks, or after one of them, once a plot has two. Two marks swapped: the second draws on top.	Nothing. No order under <code>*</code> changes the picture.

Read the bottom right cell first. `*` is rigid about one thing only, which is that the mark stands on its left. Once the mark is there, no arrangement of what follows changes what you get.

Free is most of it. Every one of these draws the identical plot, and the book writes the first only because a fixed order is easier to read:

R

```
(data(gapminder_2007) + point + x(gdp) + y(life)) |
  (data(gapminder_2007) + x(gdp) + point + y(life))
```

Python

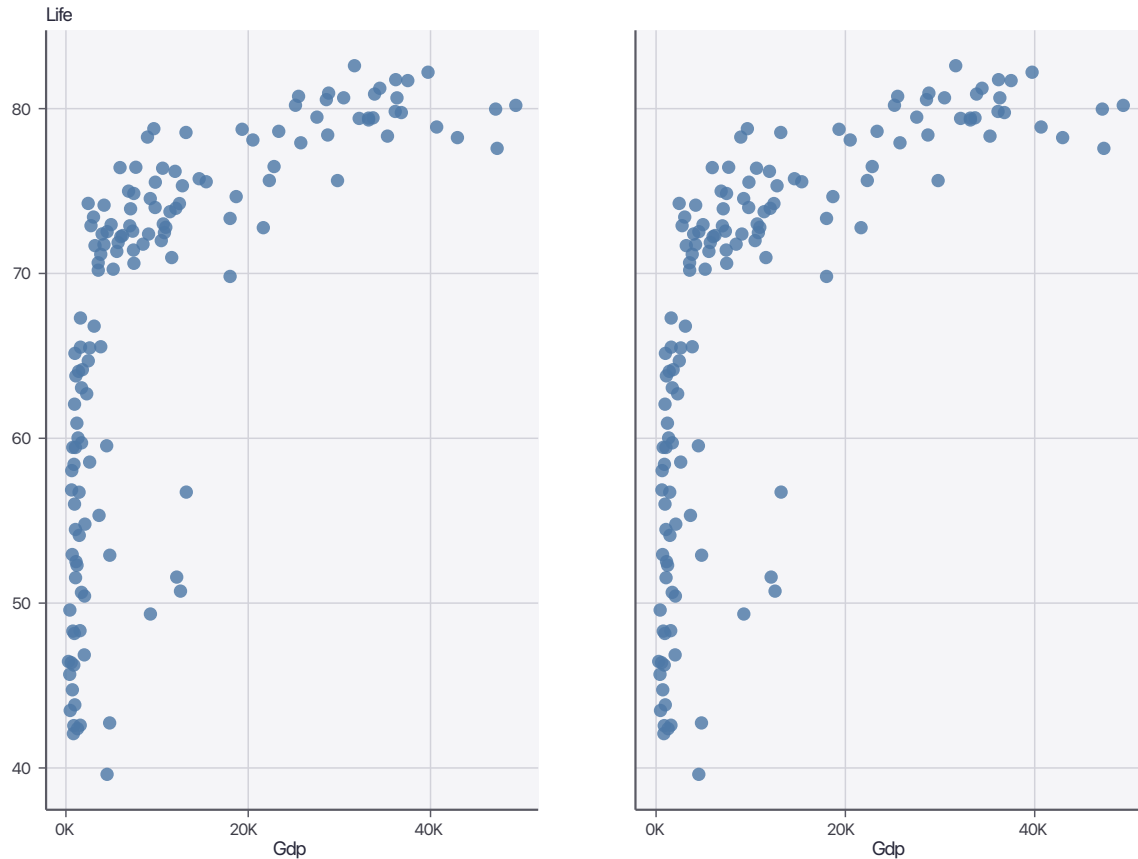
```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) |
  (data(gapminder_2007) + x(col.gdp) + point + y(col.life)))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp) + y(:life)) |
  (data(gapminder_2007) + x(:gdp) + point + y(:life))
```

JavaScript

```
beside(plot(data(gapminder_2007), point, x(col.gdp), y(col.life)),
  plot(data(gapminder_2007), x(col.gdp), point, y(col.life)))
```



Both name `life` on `y`, so the two panels share that axis and it is drawn once, on the left. Each panel still draws its own `gdp` axis underneath it.

Refused is the engine telling you the sentence has no subject yet. A setting needs a mark to attach to, and a plot needs its table first. `+` has to have a plot on its left before it can add anything:

```
data(gapminder_2007) + style(size = 3) + point + x(gdp) + y(life)
```

Error:

```
! gog: `style()` has no mark to style. Put it after a mark, e.g. `point +
  ↪ style(color = "tomato")`.
```

Meaningful is the only one to watch, because it is the only one that stays silent. With two marks in a sentence, a channel written before them belongs to the plot and reaches both, and a channel written after one belongs to that mark. Both are legal, neither warns, and they draw different pictures. That is the subject of [Encoding scope](#). It is also why this book keeps one order: when every sentence looks the same, a different arrangement is a deliberate signal.

The other silent one is the marks themselves. Two marks swapped draw in the order you wrote them, so the second sits on top of the first.

26.4.1 What groups first

The three kinds above are about where you may put a word. Precedence is a different question: how the words you did write get grouped. It is what lets a sentence like this one carry no parentheses at all.

R

```
data(gapminder_2007) + bar * bin + x(life) | facet(continent)
```

Python

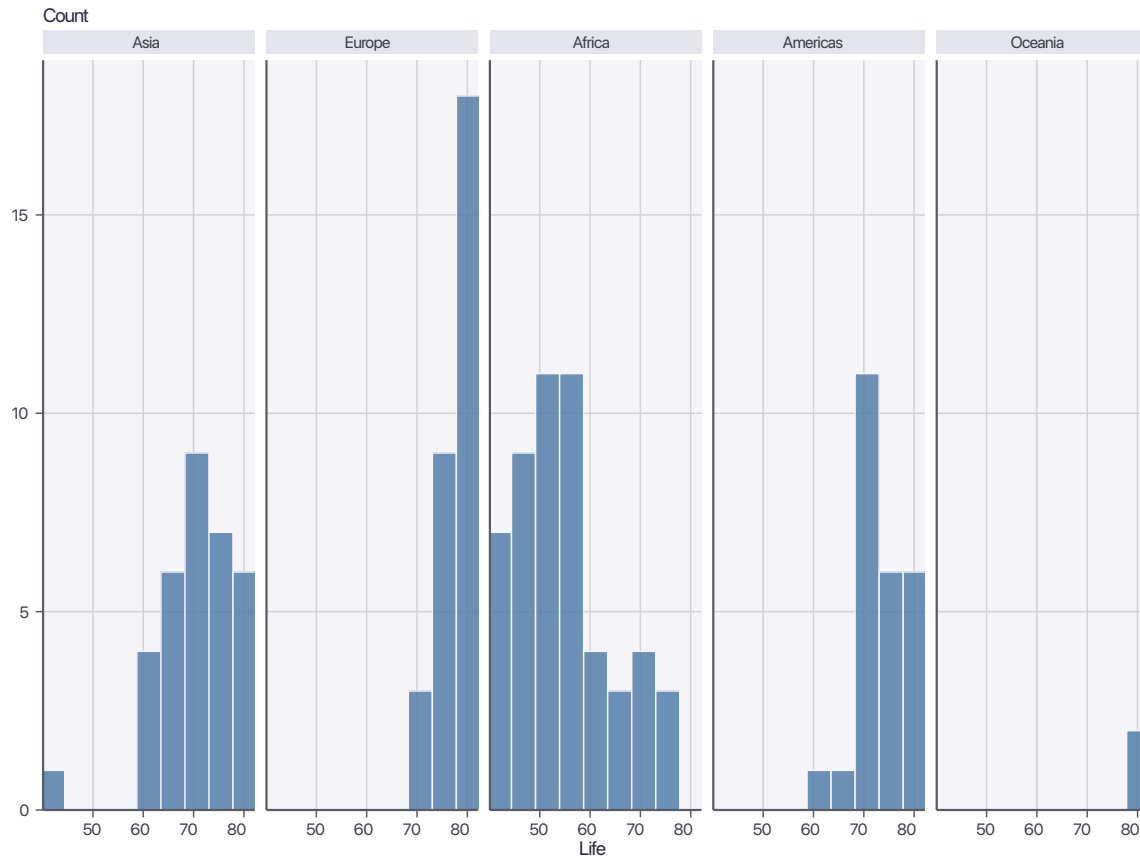
```
data(gapminder_2007) + bar * bin + x(col.life) | facet(col.continent)
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life),  
      across(col.continent))
```



* groups first, then +, then |. So the transform joins its mark first, the atoms join the plot next, and the facet splits the finished plot last. / groups before |; the next section has the rule.

Those are R's own precedences, and Python's and Julia's, so one sentence groups the same way in three of the four languages. JavaScript can give no operator a meaning of its own, so it spells the four as words, which makes the grouping something you see instead of something you remember ([JavaScript](#)).

26.4.2 What parentheses may group

Parentheses change which operator runs first, and that is the whole of what they do. They group **plots**, which is why a composed page uses them: / is applied before |, so `a | b / c` reads as `a | (b / c)` unless you say otherwise ([Composition](#)).

They do not group **marks**. A second `data()` applies to the mark written directly after it ([Data](#)). Parentheses around several marks do not extend that table to all of them:

```
data(actuals) + x(year) + y(sales) +  
  line +  
  (data(forecast) + point + area)
```

Error:

```
! gog: parentheses do not group marks, so everything inside these would be  
  ↳ dropped. Write the marks in sequence instead, and repeat `data()` before  
  ↳ each one that reads that table: `+ data(forecast) + point +  
  ↳ data(forecast) + area`. Parentheses compose whole plots, with `|` and  
  ↳ `/\`.
```

gog refuses that instead of drawing it. The refusal is worth having, because the alternative is worse than an error. The engine cannot give both marks inside the parentheses the second table. Its only other choice would be to discard both marks and draw a plot missing two layers, in silence. To put two marks on the second table, write `data()` before each of them.

The same refusal covers a position or a title inside the parentheses, for the same reason. In JavaScript, where the operators are words, the same mistake is a `plot()` handed to another `plot()`, and it is refused there too.

26.5 Why +, and not a pipe

Readers who know R's **tidyverse** packages ask why **gog** does not use a pipe. Hadley Wickham, **ggplot2**'s author, has said publicly that the pipe would have been the better interface (Wickham, 2018). **ggplot**, **ggplot2**'s predecessor, was written as function composition, and it would have used a pipe if one had existed. **ggplot2** used `+` instead.

He also answers the question a reader asks next. **ggplot2** cannot change now: its functions take the mapping and the data first, while a pipe needs the plot. And he judges that a new package built only to change this would not be worth what it would cost. So for **ggplot2** the matter is settled by history rather than by taste, and a grammar starting fresh has to answer it on its own. Three reasons are given for preferring the pipe. Two of them do not apply to **gog**, and the third is the one that settles the question the other way.

The first is about one language. A pipe is a single idea that works everywhere in R, so a grammar spelling composition differently is one more thing to learn, and switching between the two spellings is a frequent source of errors (his own included). That is true, and it is an argument about R and its packages. **gog** is one syntax spoken by four languages, and that changes the answer. `+` can be given a meaning in R, Python and Julia. The pipe cannot be given a **gog** meaning. Julia's `|>` and R's two pipes already mean something **gog** must not

change. Python has no pipe operator at all, and no way to add one: you can give new meanings only to the operators it already has. Piping would work in R and Julia and need some other spelling in Python. The languages would then have two different grammars, and this project exists to avoid that. JavaScript decides nothing here, because it cannot overload any operator at all, which is why it spells all four as words ([JavaScript](#)).

The second is about arithmetic, and it is the strongest. You expect $x + y$ to equal $y + x$, and `ggplot2`'s `+` does not always do that. That objection is answered a few sections up. Here the channels commute, and so does the mark among them: move `color` ahead of `x`, or write the positions before the mark, and the engine emits the same bytes. One thing does depend on order: two marks swapped. There the order is the drawing order you asked for, and the second mark draws on top.

The other half of that objection is grouping, whether $x + (y + z)$ means what $(x + y) + z$ means. The question cannot arise here, because $x + (y + z)$ cannot be written at all. `+` always takes a plot on its left, so two atoms with no table behind them never form a smaller expression to group. They are refused:

```
point + x(gdp)
```

Error:

```
! gog: these atoms have no plot to join - the sentence starts with the data:
  ↪ `data(df) + point + x(gdp) + ...`.
```

Atoms never collect on their own, so there is no grouping to get wrong.

The third reason is the one that decides it, and it is not an argument against the pipe. It is what a pipe would take away. A pipe applies functions: $x \mid> f$ means $f(x)$, so every atom would have to become a function taking a plot and returning one. Make `bar` a function, then ask what `bar * bin` could mean. Nothing at all: you cannot multiply two functions and get a histogram. The derive operator exists only because marks and transforms are *things* rather than actions. They are values, and they combine with each other before either one is joined to a plot.

That is the same decision this book opens with, arriving a second time. $\neg$ is a letter, not an instruction. You put it in a slot next to other letters, and the block is assembled from its pieces rather than performed in sequence. Hangeul's atoms are nouns, which is exactly why they compose, and `gog`'s are nouns for the same reason. A grammar built on the pipe is a grammar of verbs: it can layer, because layering really is a sequence of additions, but it cannot derive. Wickham's own pre-`ggplot2` experiment was function composition, and it had no `*` either.

So `+` here is a choice rather than something inherited. It is the one operator that can be given a meaning in R, Python and Julia. It commutes wherever order should not matter, and it leaves the atoms as nouns so that `*` can exist.

26.6 Summary

Operator	Reads as	Example
+	“...and also...”	<code>point + color(continent)</code>
*	“...derived by...”	<code>bar * bin</code>
	“...split into panel columns by...”	<code>plot facet(continent)</code>
/	“...split into panel rows by...”	<code>plot / facet(continent)</code>
	“...beside...”	<code>plot_a plot_b</code>
/	“...above...”	<code>plot_a / plot_b</code>

A finite set of rules. An unlimited number of plots.

27 Encoding scope

If you write one `x()` and then two marks, which marks use it? Both do. Each atom binds to a **scope**: the whole plot, or one layer. This chapter gives the rule that decides which.

27.1 The rule

Atom	Scope	Position-sensitive?
<code>x()</code> , <code>y()</code> , <code>z()</code> before any mark	Plot , every layer	Yes
the same positions after a mark	Layer , that mark's column for the shared axis	Yes
<code>data()</code> before any mark	Plot , default table	No
<code>data()</code> after a mark	Next layer only	Yes
<code>color()</code> , <code>size()</code> , <code>shape()</code> , <code>pattern()</code> , <code>opacity()</code> , <code>group()</code> , <code>label()</code> , <code>play()</code> before any mark	Plot , every layer that can accept it	Yes
the same channels after a mark	Layer , that mark alone	Yes
<code>style()</code>	Layer , nearest preceding mark; refused when no mark precedes it	Yes

Atom	Scope	Position-sensitive?
<code>title()</code> , <code>x_label()</code> , <code>y_label()</code> , <code>z_label()</code> , <code>palette()</code> , <code>theme()</code> , <code>brush()</code>	Plot	No

One rule covers the position-sensitive rows: a mark binds forward, so it takes every atom written after it. The rows marked **No** (the labels, `palette()`, and a `data()` before any mark) belong to the plot wherever they are written. Write a channel before any mark and it binds to the plot; write it after a mark and it binds to that mark.

Where two bindings could apply, the **nearest** one wins: the same rule that lets a mid-expression `data()` give the next layer its own table.

27.2 One axis, and which column each layer reads

Every layer of a plot shares the same axes. That is what makes it one plot rather than two drawn on top of each other. `gog` allows no second axis: one x scale, one set of ticks, one label. A table that does not belong on those axes does not belong in this plot. [Composition](#) covers the choice between layering, faceting, and a page of separate plots.

Sharing an axis is not the same as sharing a *column name*, and the two are easy to confuse. Bind the positions once before the marks and every layer reads them, which is what you want almost always:

R

```
# x and y defined once: both marks use them
data(actuals) + x(year) + y(sales) +
  bar +
  line
```

Python

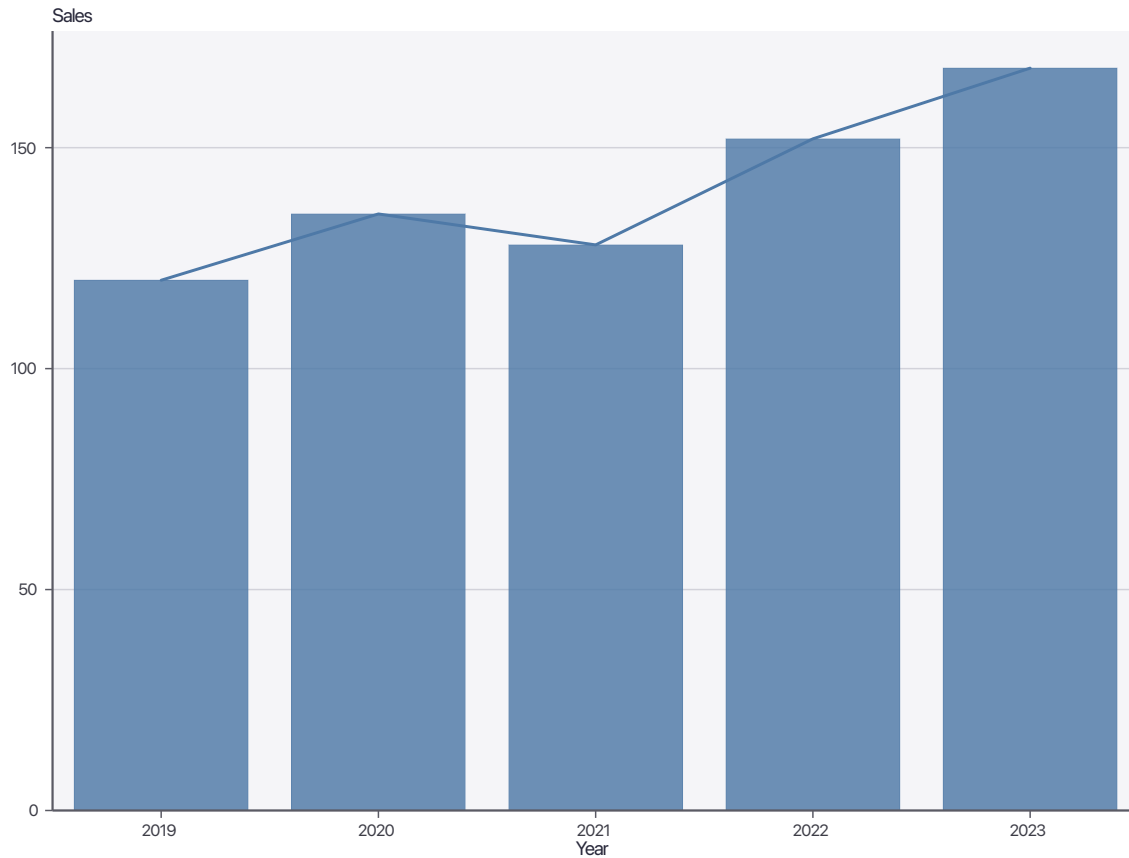
```
(data(actuals) + x(col.year) + y(col.sales) +
  bar +
  line)
```

Julia

```
data(actuals) + x(:year) + y(:sales) + bar + line
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), bar, line)
```



“Given the actuals: x is year, y is sales, bars and also a line.”

Write a position after a mark and it belongs to that mark, exactly as `color()` does. The axis is unchanged. The layer is only saying which of *its* columns supplies the values. That rule is what lets `milestones` in `Data` keep its own column names:

R

```
# `milestones` calls its columns `at` and `value`, not `gdp` and `life`  
data(gapminder_2007) + point + x(gdp) + y(life) +  
  data(milestones) + text + x(at) + y(value) + label(note)
```

Python

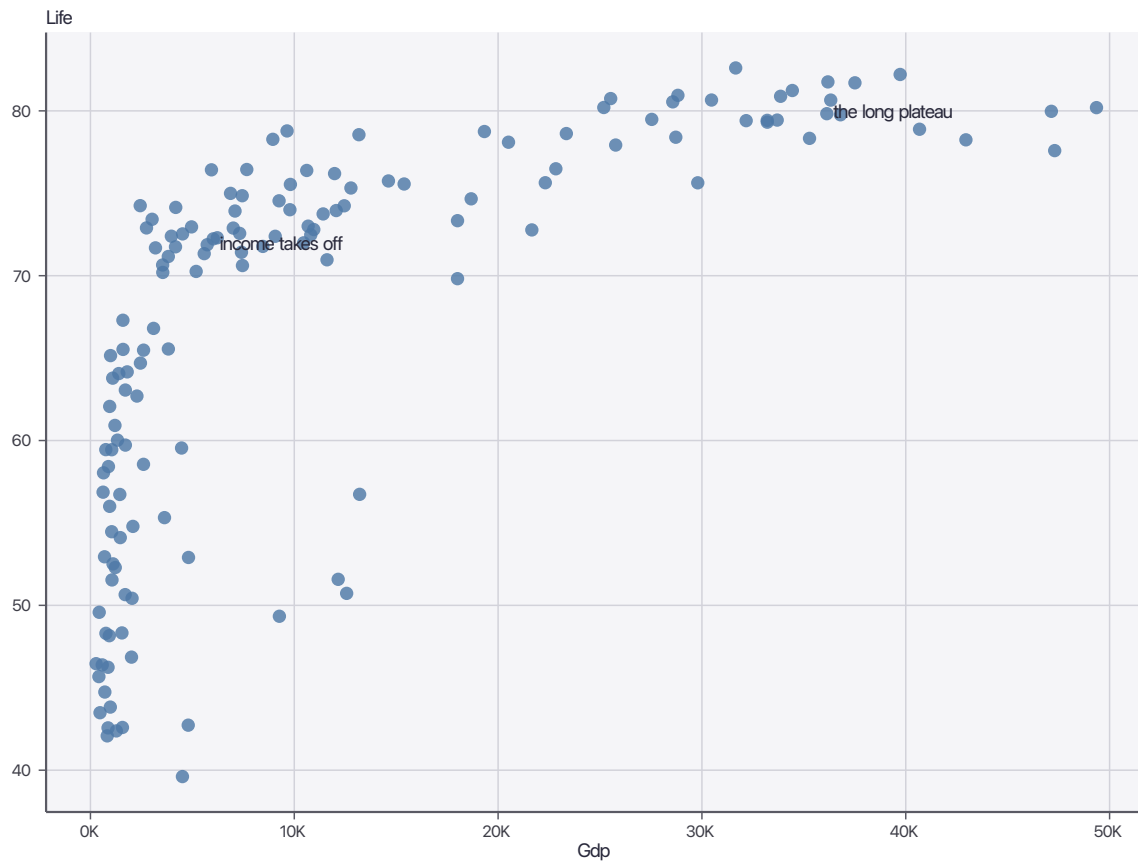
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  data(milestones) + text + x(col.at) + y(col.value) + label(col.note))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + data(milestones) +  
  text + x(:at) + y(:value) + label(:note)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  data(milestones), text, x(col.at), y(col.value), label(col.note))
```



“Given gapminder 2007: points, x is gdp, y is life; then given the milestones: text, x is at, y is value, label by note.”

Look at the axis labels: they still read **Gdp** and **Life**. The **milestones** column names never reach the axis labels, because the axis belongs to the plot. Only the column choice is local. Without this rule, **milestones** would have to rename **at** and **value** to **gdp** and **life**. That is easy when you are inventing a table, and tedious when the table already exists.

The same rule covers two columns of one table on one axis:

R

```
data(sessions) + x(session) +
  line + y(open) + style(color = "steelblue") +
  line + y(close) + style(color = "tomato", pattern = "dashed")
```

Python

```
(data(sessions) + x(col.session) +
  line + y(col.open) + style(color = "steelblue") +
  line + y(col.close) + style(color = "tomato", pattern = "dashed"))
```

Julia

```
data(sessions) + x(:session) + line + y(:open) +
  style(color = "steelblue") + line + y(:close) +
  style(color = "tomato", pattern = "dashed")
```

JavaScript

```
plot(data(sessions), x(col.session), line, y(col.open),
  style({ color: "steelblue" }), line, y(col.close),
  style({ color: "tomato", pattern: "dashed" }))
```



“Given the sessions: *x* is session, a line, *y* is open, colored steelblue, and also a line, *y* is close, colored tomato.”

Reshaping to tidy data is still usually better: one **value** column plus a **type** column, mapped with `color(type)`. That form earns a legend and handles more than two series. Name the columns per layer when you want the two lines styled differently. Do the same when the second set of values comes from another table.

What a layer may *not* do is bring its own scale:

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  data(milestones) + text + x(at, scale = "log") + y(value) + label(note)
```

Error:

```
! gog: this layer's `x(at, ...)` gives it its own scale, and the layers of one  
  ↳ plot share one x axis - two scales on it would be two coordinate spaces  
  ↳ in one panel, where every mark reads against an axis the expression never  
  ↳ named. Set the scale once for the plot, before the marks: `x(<column>`,  
  ↳ scale = "log")`. A layer may name its own *column* for the shared axis -  
  ↳ `x(at)` on its own is fine - but not its own scale. If the two series are  
  ↳ genuinely in different units, facet on the measure instead.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A second scale on one axis is a second axis. Every mark would then be measured against a scale you never wrote. This refusal and the plot before it differ by one word: a layer names its own *column*, never its own scale.

27.3 Channels: plot-scoped or layer-scoped, by position

Positions were the hard case, because every layer shares one axis. The other channels share nothing, and they follow the same position rule.

Written *after a mark*, a channel binds to that mark alone:

R

```
data(gapminder_2007) + x(gdp) + y(life) +  
  point + color(continent) + # color applies to point only  
  line   # line uses a single default color
```

Python

```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +  
  point + color(col.continent) +  
  line)
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + point + color(:continent) +  
  line
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), point,  
  color(col.continent), line)
```



“Given gapminder 2007: x is gdp, y is life, points colored by continent, and also a line.”

Written *before any mark*, a channel has no mark to bind to, so it binds to the plot and covers every layer:

R

```
data(gapminder_2007) + x(gdp) + y(life) + color(continent) +  
  point +  
  line
```

Python

```
(data(gapminder_2007) + x(col.gdp) + y(col.life) + color(col.continent) +  
  point +  
  line)
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + color(:continent) + point +  
  line
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), color(col.continent),  
  point, line)
```



“Given gapminder 2007: x is gdp, y is life, color by continent, points and also a line.”

That is the same position rule `x` and `y` follow. Written before the marks, they apply to every mark. Written after one mark, they belong to that mark. No channel has a scoping rule of its own. For `x`, `y` and `z`, a layer-scoped binding names a column, never a new axis.

27.3.1 Why binding only goes forward

Because a mark takes only the atoms written after it, the same atoms always mean the same thing:

R

```
# `size` is written after `point`, so it belongs to the points;
# the line beside them does not change, and keeps its own color
data(gapminder_asia) + x(year) + y(life) +
```

```
line + color(country) +  
point + size(population)
```

Python

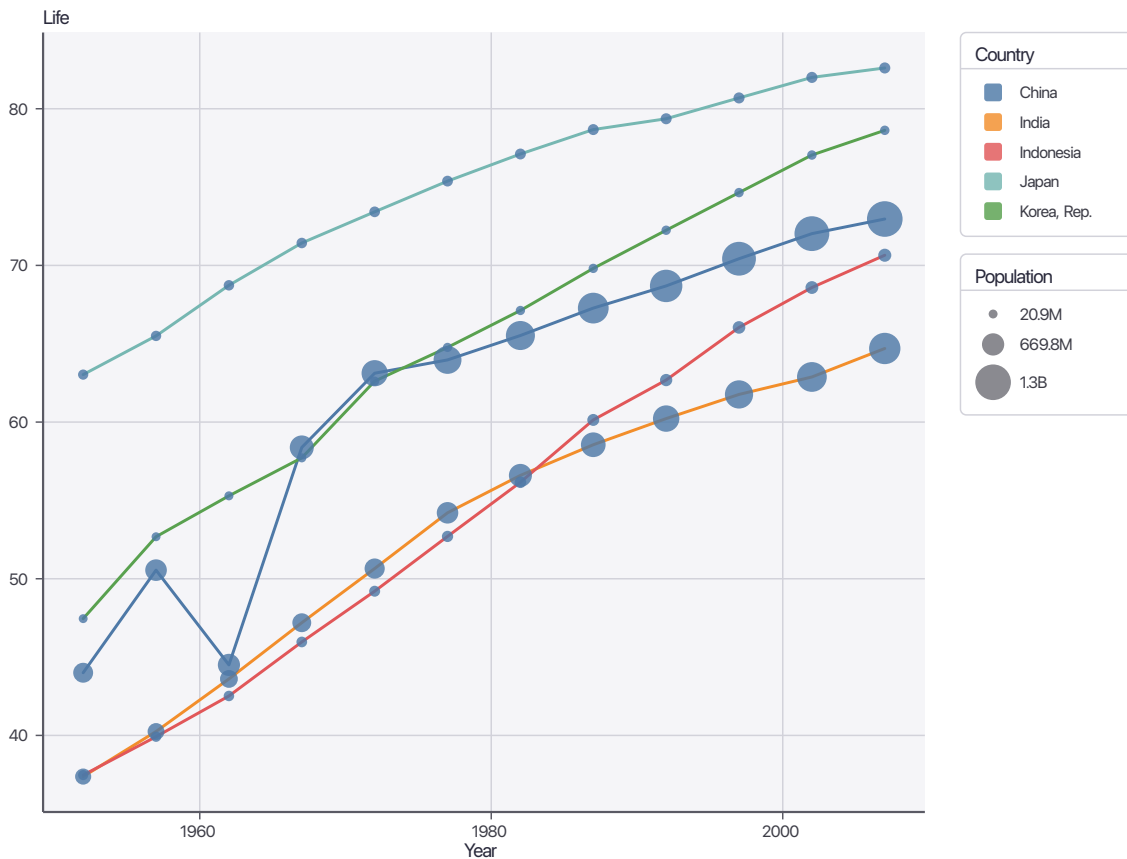
```
(data(gapminder_asia) + x(col.year) + y(col.life) +  
  line + color(col.country) +  
  point + size(col.population))
```

Julia

```
data(gapminder_asia) + x(:year) + y(:life) + line + color(:country) +  
point + size(:population)
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), line,  
  color(col.country), point, size(col.population))
```



“Given gapminder Asia: x is year, y is life, lines colored by country, and also points sized by population.”

A channel that bound backward could not draw this plot. It would put `size` on the line as well. A line has no size, so `gog` would refuse the whole plot, and you could not say what you meant.

27.3.2 A plot-scoped channel and the marks it skips

`size` written for the whole plot applies to the points, because a line has no size. `gog` applies it to every mark that has the feature and prints a message naming the marks it skipped. That message is an **assumption**: it reports a choice rather than refusing one, so the plot still renders:

R

```
data(gapminder_asia) + x(year) + y(life) + size(population) +  
  line + group(country) +  
  point
```

Python

```
(data(gapminder_asia) + x(col.year) + y(col.life) + size(col.population) +  
  line + group(col.country) +  
  point)
```

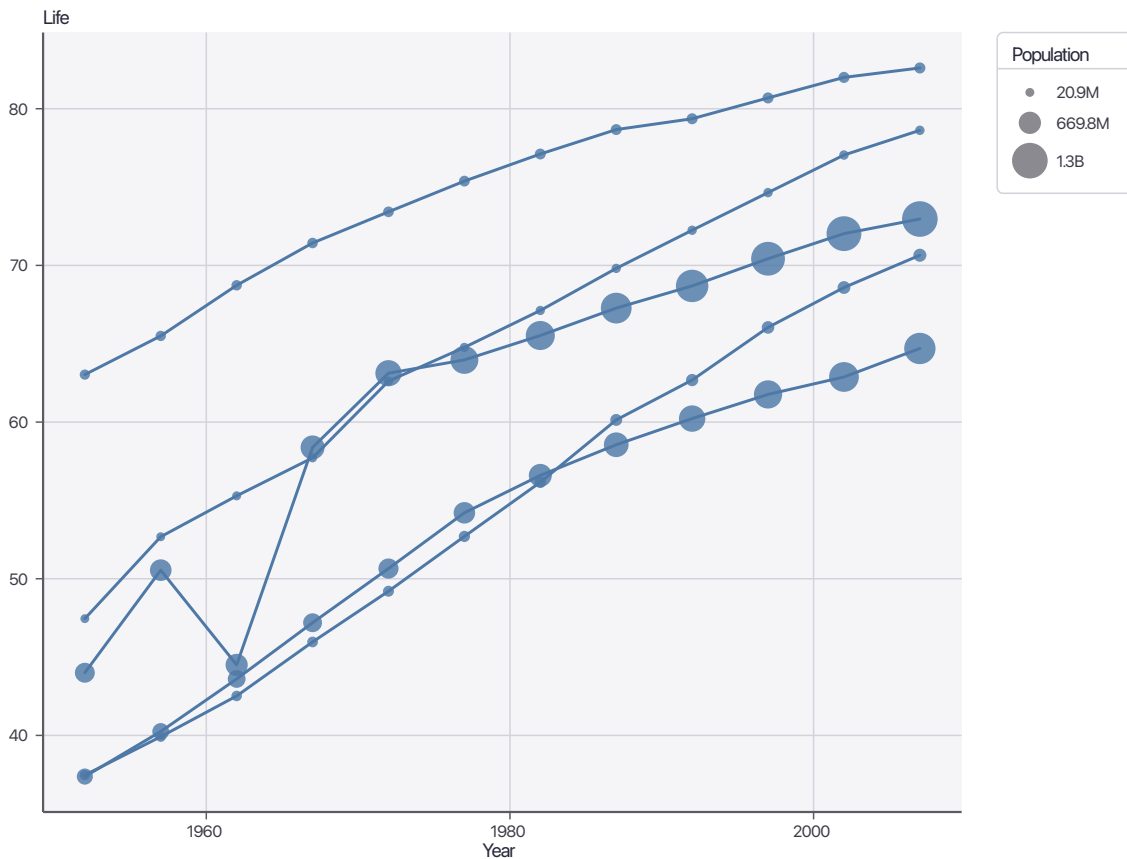
Julia

```
data(gapminder_asia) + x(:year) + y(:life) + size(:population) + line +  
  group(:country) + point
```

JavaScript

```
plot(data(gapminder_asia), x(col.year), y(col.life), size(col.population),  
  line, group(col.country), point)
```

gog: `size(population)` is written for the whole plot, so it applies to
 ↳ `point` - `line` has no size feature and is left unchanged. Move
 ↳ `size(population)` after a mark to bind it to that mark alone.



“Given gapminder Asia: x is year, y is life, size by population, lines grouped by country, and also points.”

If *no* mark in the plot can accept the channel, the binding does nothing at all, so **gog** refuses it:

```
data(gapminder_asia) + x(year) + y(life) + size(population) +
  line + group(country)
```

Error:

```
! gog: `size(population)` is written for the whole plot, but no mark here has
  ↳ a size feature (`line`). Remove it, or add a mark that does.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

27.4 data() mid-expression

Data states this rule, and it is a scope rule. The first `data()` is the plot's table. Each later `data()` applies to the mark written directly after it.

R

```
data(actuals) + x(year) + y(sales) +  
  line +  
  data(forecast) + point      # ← forecast applies to point, not to line
```

Python

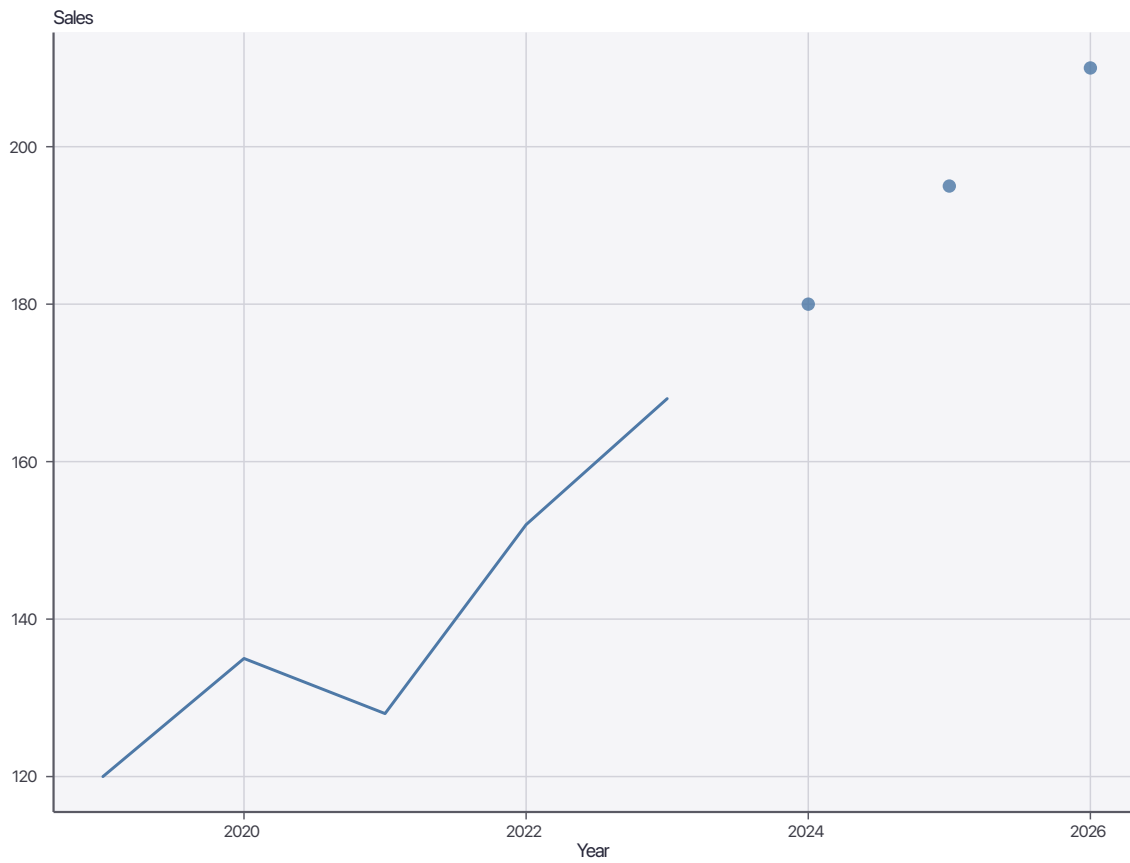
```
(data(actuals) + x(col.year) + y(col.sales) +  
  line +  
  data(forecast) + point)
```

Julia

```
data(actuals) + x(:year) + y(:sales) + line + data(forecast) + point
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), line, data(forecast),  
  point)
```



Position matters here because each `data()` marks where one layer's table ends and the next layer's begins.

28 Faceting

How do you compare the same plot across groups? A facet splits one plot into small multiples: one panel per category of a column. Wilkinson calls a facet a frame of frames. A **frame** is the space a plot is drawn in, with its own x and y axes. The facet column's categories make an outer frame, and every cell of it holds a copy of the inner one. Nothing else about the plot changes. The marks, channels, and transforms you have already learned draw each panel exactly as they would draw the whole.

The picture has been invented more than once, so you may know it under another name. Becker and Cleveland's group at Bell Labs built it into S as **Trellis display** (Becker et al., 1996). The name comes from the garden trellis, whose rows and columns it resembles, and most people first saw it in S-PLUS. R draws it that way still in the `lattice` package (Sarkar, 2026), and its own book is where the display is set out in full (Sarkar, 2008). Base R's `coplot()` is the same idea under a third name, the conditioning plot, because what a trellis conditions *on* is the facet column. That is also why the operator in the next section is `|`, the sign statistics already writes for conditioning. [Composition](#) finishes that history. Tufte's **small multiples** (Tufte, 1990) names the result rather than the operation, which is the division this book keeps: small multiples for the picture, **facet** for the thing you write. And **facet** is Wilkinson's own word (Wilkinson, 2005), from the Latin *facies*, a face: one side of a many-sided thing, like one face of a cut diamond. `gog` takes Wilkinson's word because it is the most general of them. To him a trellis is one arrangement a facet can take, beside the scatterplot matrix and the row plot. A grammar needs one term that covers all three. The word every one of these systems shares is **panel**, the cell a split produces, and this chapter uses it the same way.

28.1 |: panels side by side

The `facet()` atom names the column that does the splitting, and `|` arranges the panels in columns:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) | facet(continent)
```

Python

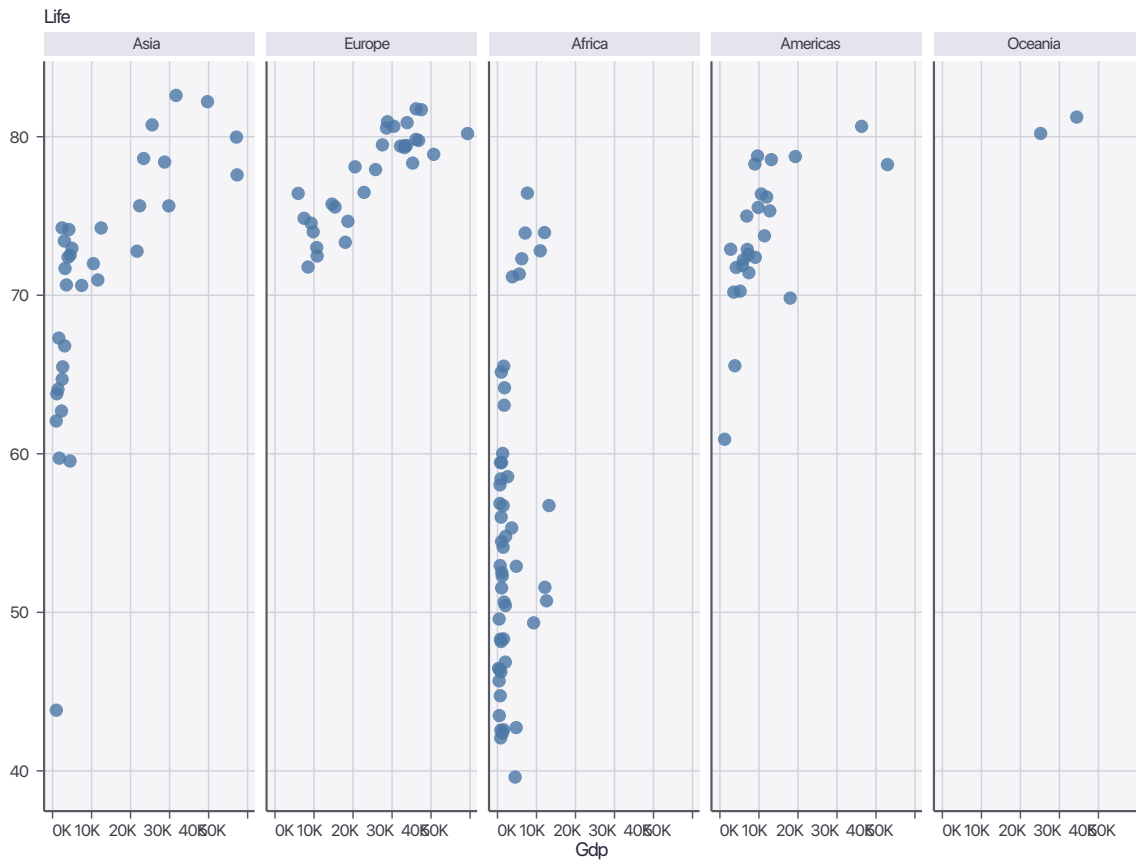
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) |
  ↪ facet(col.continent)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  across(col.continent))
```



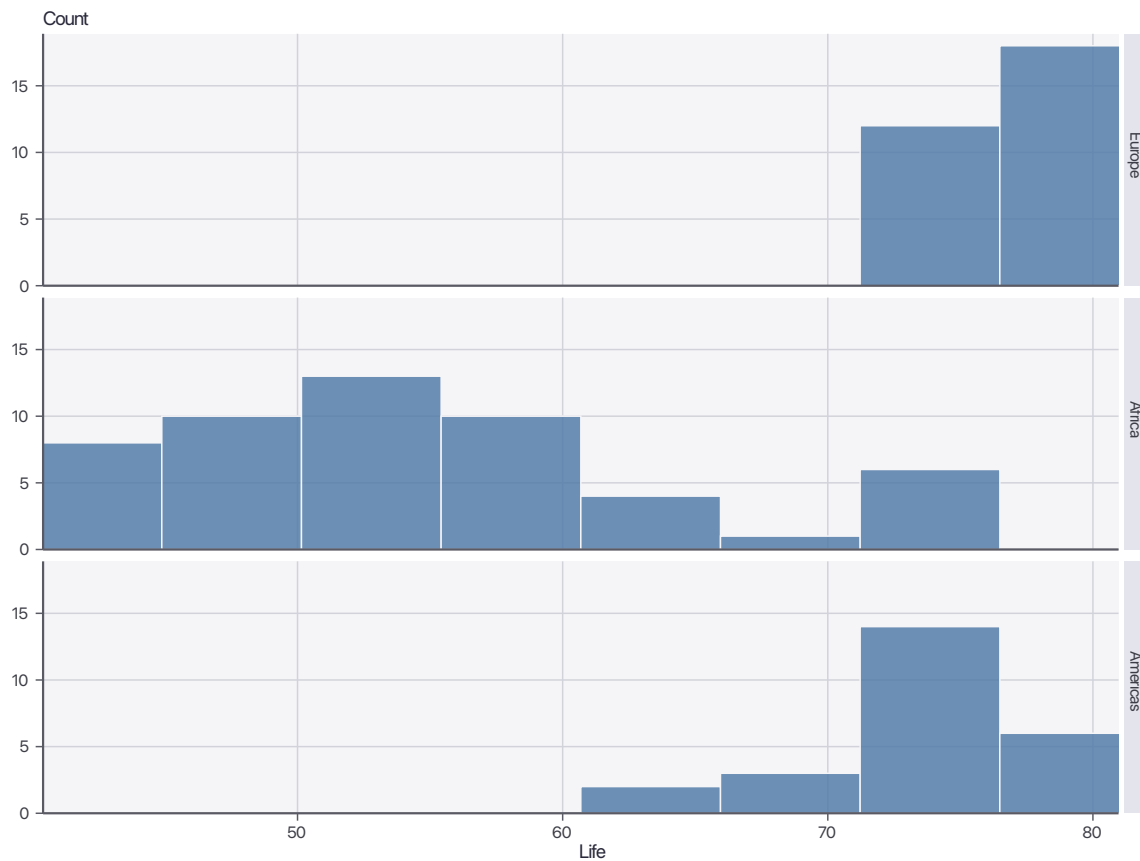
“Given gapminder 2007: points, x is gdp, y is life, split into panel columns by continent.”

Reading across, the panels share both axes, so a shift *between* continents reads as plainly as a pattern within one. The shared scale is what faceting is for. One plot per continent, each with its own scale, looks much the same and breaks the comparison, and nothing on the page warns the reader.

28.2 `/:` panels stacked

`/` arranges the panels in rows. Stacking aligns the x axis, which is the better direction when the comparison is *where things sit along x*:

```
gm_ae <- gapminder_2007[gapminder_2007$continent %in% c("Africa", "Americas",  
  ↪ "Europe"), ]  
data(gm_ae) + bar * bin + x(life) / facet(continent)
```



“Given the gm ae table: bars derived by bin, x is life, split into panel rows by continent.”

The three distributions line up bar for bar: Africa’s life expectancies spread wide toward the low end, and Europe’s cluster in a narrow band at the high end.

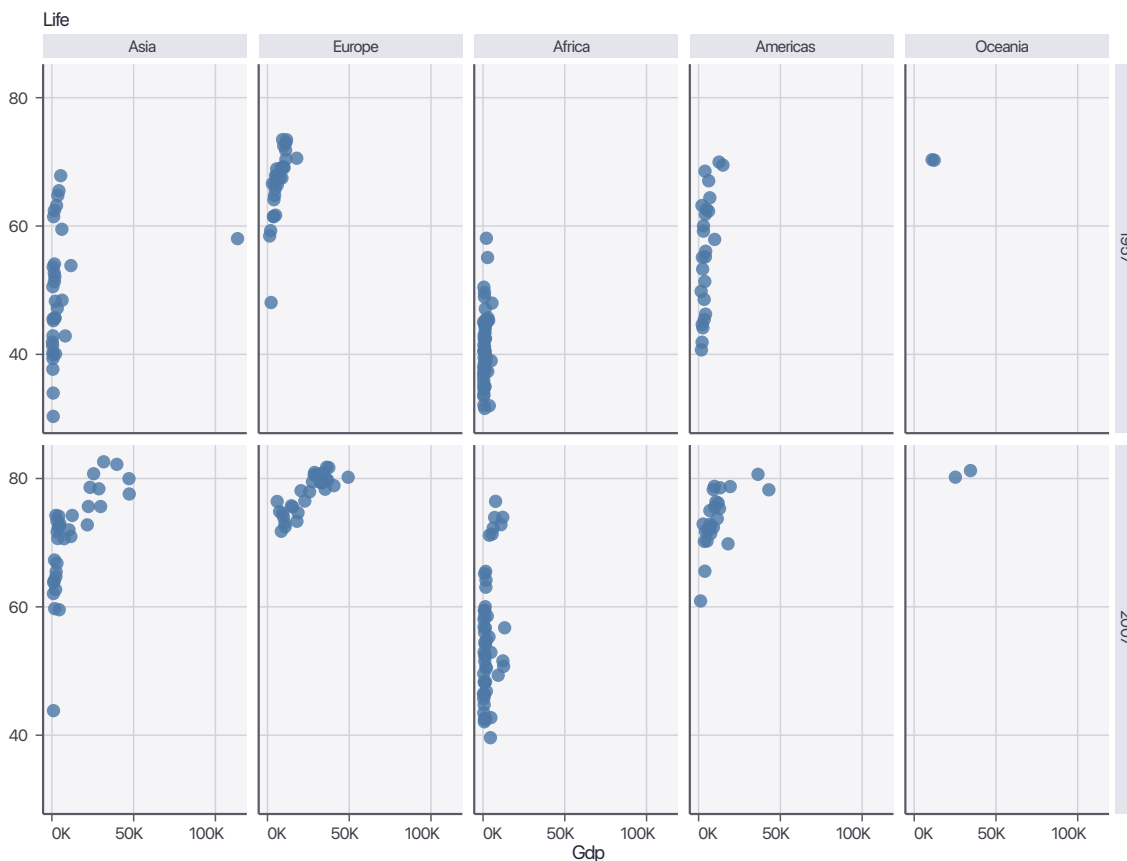
That reading only works because the bars are the same width in all three panels, and a histogram under a facet is where the count and the cut have to be decided separately. Each panel counts its own rows, because a statistic always sees the panel’s subset as if it were the whole table. That is true of every transform on every mark, which is the [No Exceptions](#) law applied one panel at a time. But `bin` is not only a statistic. It cuts the axis into cells *and* tallies what lands in them, and the cut is a description of the axis rather than a measurement of the rows, so it belongs with the shared scale. One set of bins is cut from all three panels’ rows at once, and each panel then counts only its own into them.

It helps to see what the alternative would draw. Cut per panel, Europe’s narrow spread gets narrow bins and Africa’s wide one gets wide bins, so bars of different widths stand against a single axis. A bar four years wide holding nine countries and a bar one year wide holding eight then draw at nearly the same height. The two rates per year differ by more than three times. The panels are still aligned and the axis still looks correct, and yet the comparison this chapter began with has stopped working.

28.3 Crossing into a grid

Write both operators to cross two columns into a grid. Each operator applies to the facet written after it, read left to right:

```
gm_years <- gm_all[gm_all$year %in% c(1957, 2007), ]
gm_years$period <- as.character(gm_years$year)
data(gm_years) + point + x(gdp) + y(life) | facet(continent) / facet(period)
```



“Given the *gm years* table: *points*, *x* is *gdp*, *y* is *life*, split into panel columns by continent and panel rows by period.”

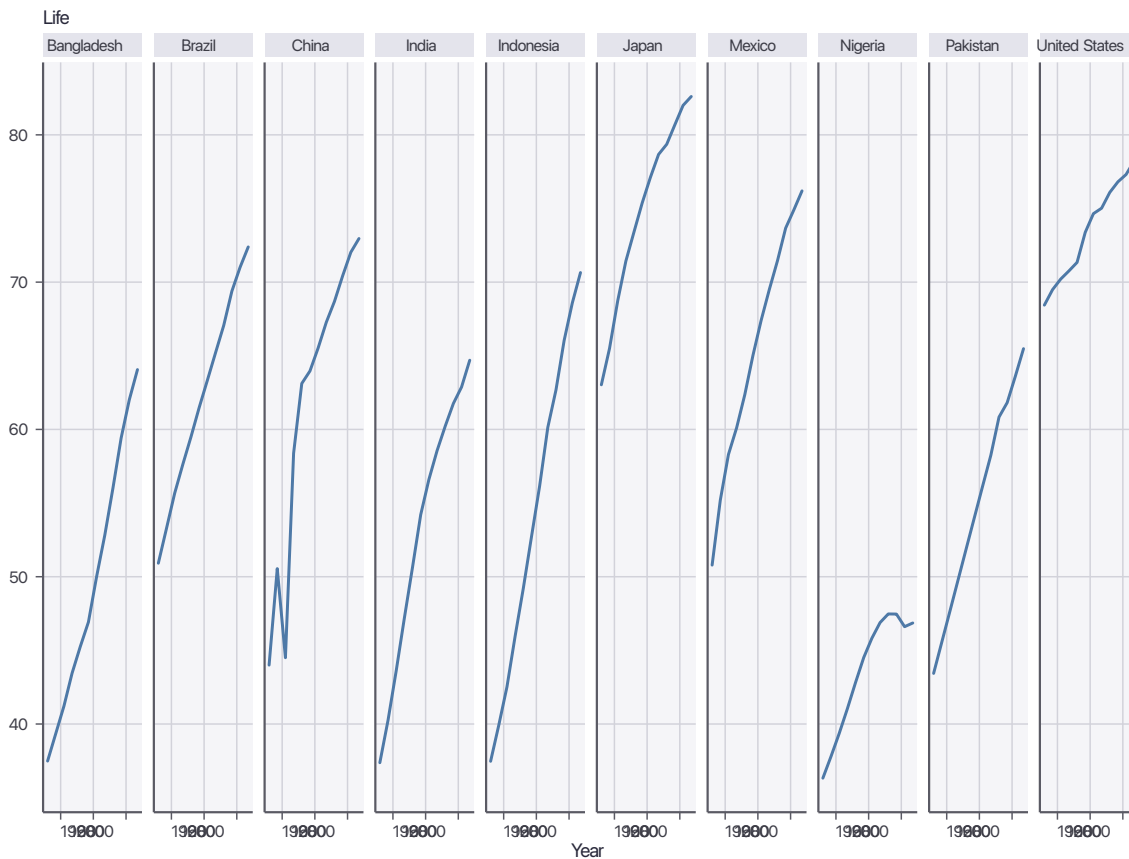
Fifty years and five continents in one grid: every panel column is a continent, and every panel row a period. The shared scale lets you measure the rise in life expectancy panel by panel.

The grid is a **crossing** in Wilkinson’s sense: every row $\times$ column combination gets a panel, and a combination with no rows still gets an empty one. The grid shows that the combination is *possible*, so an empty panel is information rather than an accident. Wilkinson’s other operation, *nesting*, draws only the combinations that exist. These operators do not do that.

28.4 Folding a long line of panels

One column with many categories makes a long line of panels, and a long line of panels makes each one narrow. Here are the ten most populous countries of 2007, each with its own half-century of life expectancy:

```
ranked <- gapminder_2007[base::order(-gapminder_2007$population), ]
gm_top <- gm_all[gm_all$country %in% utils::head(ranked$country, 10), ]
data(gm_top) + line + group(country) + x(year) + y(life) | facet(country)
```



“Given the gm_top table: lines, x is year, y is life, grouped by country, split into panel columns by country.”

The shapes are still readable and nothing else on the panel is. Read the axis: the tick labels touch and print as one run of digits, 196019802000, and the last panel’s name is wider than the strip that prints it. Ten categories is a common number, and one line of ten panels is already too narrow to read.

`wrap` folds that line into a rectangle. The number is how many panels to draw before the line turns:

R

```
data(gm_top) + line + group(country) + x(year) + y(life) | facet(country,  
  ↪ wrap = 4)
```

Python

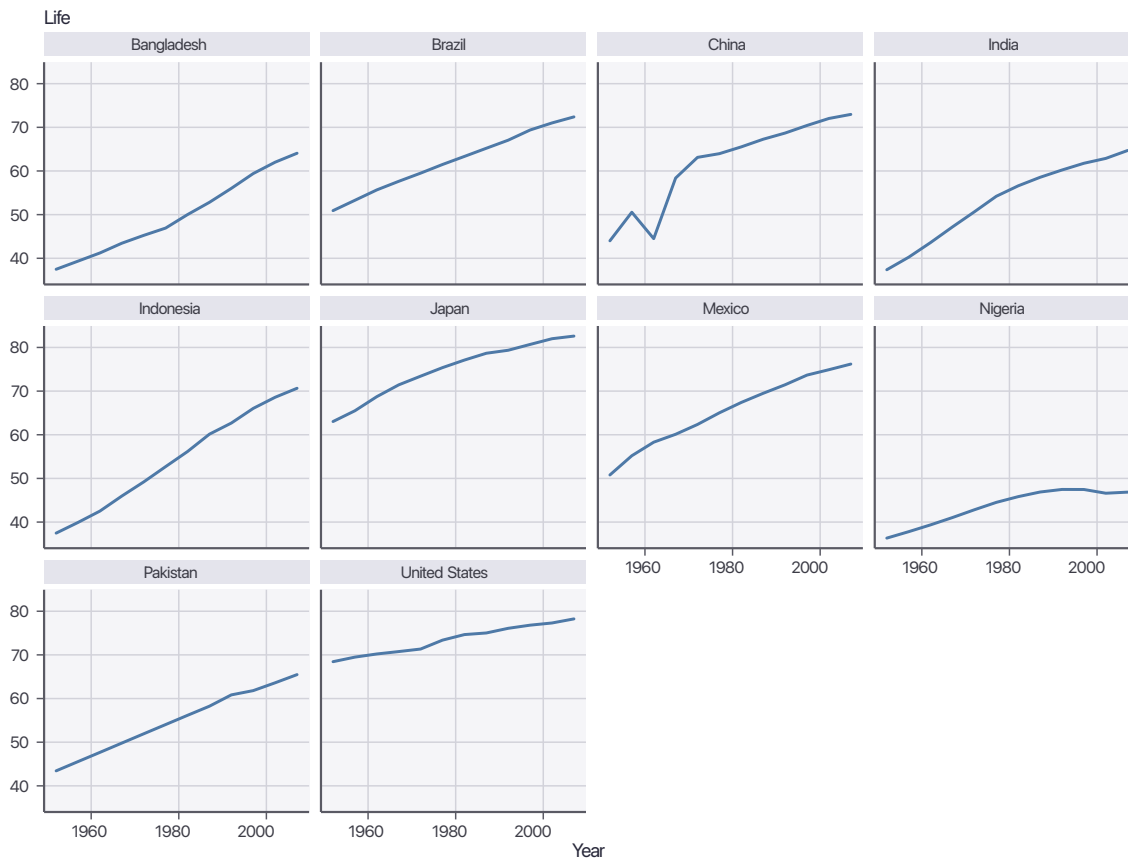
```
data(gm_top) + line + group(col.country) + x(col.year) + y(col.life) |  
  ↪ facet(col.country, wrap = 4)
```

Julia

```
data(gm_top) + line + group(:country) + x(:year) + y(:life) |  
  facet(:country, wrap = 4)
```

JavaScript

```
plot(data(gm_top), line, group(col.country), x(col.year), y(col.life),  
  across(col.country, { wrap: 4 } ))
```



“Given the gm_top table: lines, x is year, y is life, grouped by country, split into panel columns by country, wrapped at 4.”

Same ten panels, same shared scale, same sentence with one setting added. The years are readable, **United States** fits its strip, and each country’s rise is now large enough to look at.

Notice what **wrap** does *not* say: which way the panels run. The operator already settles it. | runs the line across, so four to a row; / runs it down, so four to a column:

R

```
data(gm_top) + line + group(country) + x(year) + y(life) / facet(country,
  ↪ wrap = 4)
```

Python

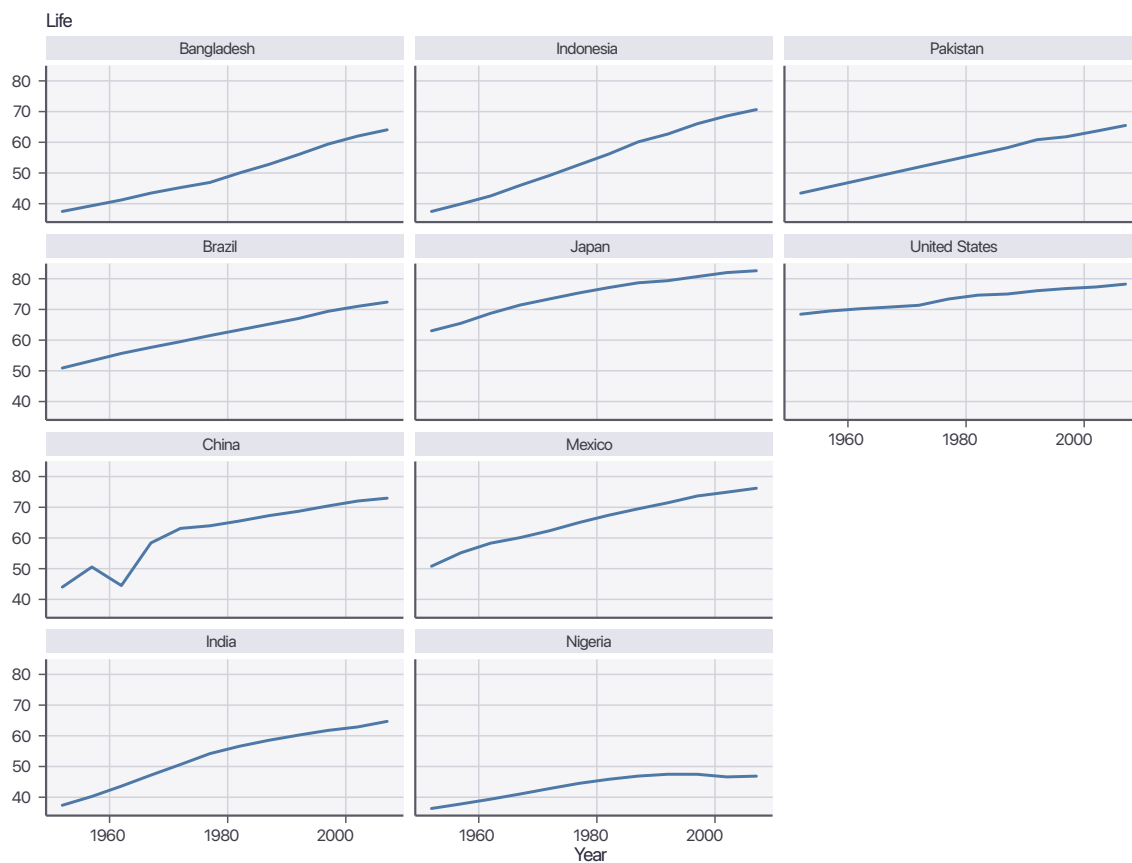

```
data(gm_top) + line + group(col.country) + x(col.year) + y(col.life) /
  ↪ facet(col.country, wrap = 4)
```

Julia

```
data(gm_top) + line + group(:country) + x(:year) + y(:life) /
  facet(:country, wrap = 4)
```

JavaScript

```
plot(data(gm_top), line, group(col.country), x(col.year), y(col.life),
  down(col.country, { wrap: 4 })))
```



The same number, the same ten countries, read down the columns instead of across the rows. One number is enough because the direction is already in the sentence, which is why there is no second setting for it. `ggplot2` spells the same idea as `nrow` and `ncol`.

Two details in the wrapped plots come from the fold itself. The first is the bottom right, where the last row stops after two panels: those two places are what is left over when ten panels are folded into a twelve-cell rectangle, so nothing is drawn in them. They are not the empty panels of the previous section. An empty panel in a crossing is a combination the frame says is possible and the data has no example of, which is information; here there is no combination at all, only leftover space.

The second is the axis under `Mexico` and `Nigeria`. Tick labels belong to the panels that touch the edge where the axis is drawn, which in a full rectangle is the bottom row. A folded line can leave the last row short, and then that rule no longer describes the plot. So `gog` states the rule in a wider form: a panel draws the x axis when no panel sits below it. `Mexico` and `Nigeria` sit above the gap, so they keep their year labels.

Wrapping a crossed grid is refused. The crossing already makes a rectangle from two columns, so a panel count beside it would answer the same question a second time:

```
render_svg(data(gm_years) + point + x(gdp) + y(life) |  
            facet(continent, wrap = 2) / facet(period))
```

Error:

```
! gog: `wrap` folds one line of panels into a rectangle, but `continent` and  
  ↪ `period` already cross into one. Drop `wrap`, or facet by one column and  
  ↪ let `wrap` shape it.
```

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

28.5 Giving a panel its own scale

Everything so far has used one scale across every panel, which is what makes the panels comparable. It is also what makes some of them unreadable. Here is the world's population by continent, one panel each:

R

```
data(gm_all) + line * sum + x(year) + y(population) | facet(continent, wrap =  
  ↪ 3)
```

Python

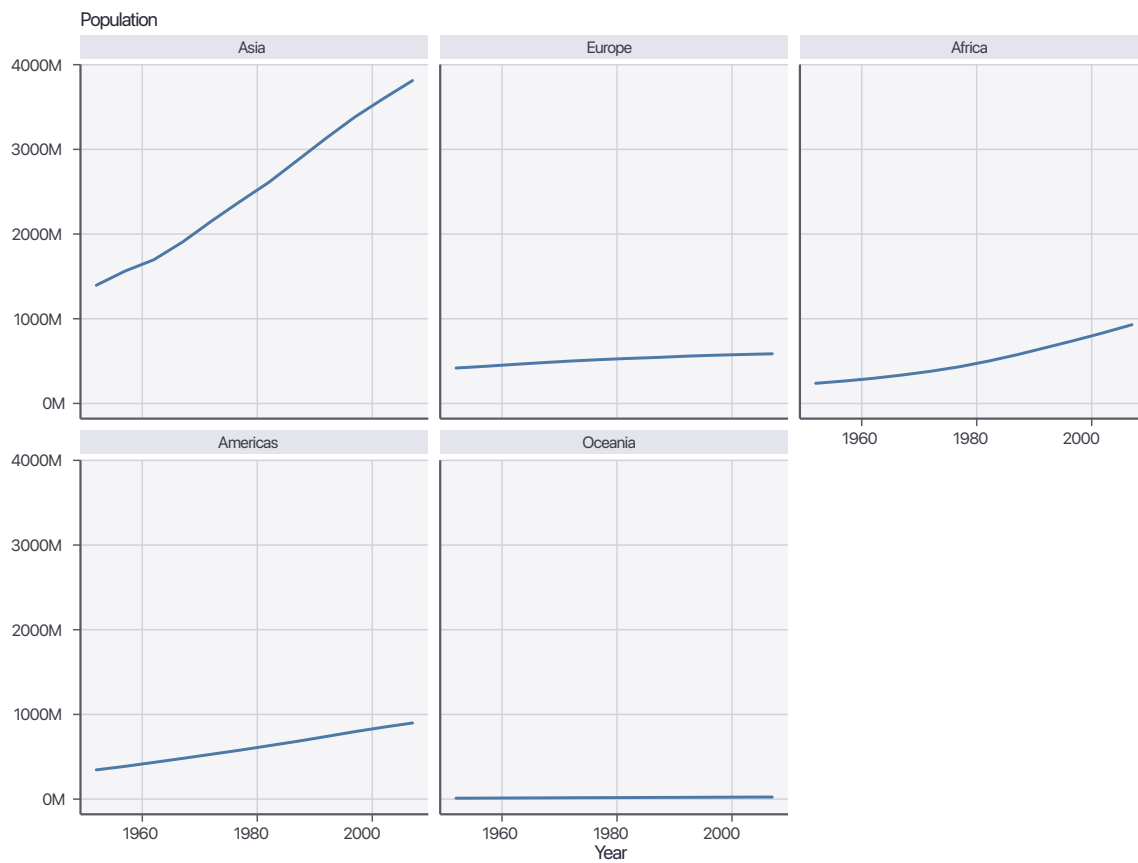
```
data(gm_all) + line * sum + x(col.year) + y(col.population) |
  ↪ facet(col.continent, wrap = 3)
```

Julia

```
data(gm_all) + line * sum + x(:year) + y(:population) |
  facet(:continent, wrap = 3)
```

JavaScript

```
plot(data(gm_all), layer(line, sum), x(col.year), y(col.population),
  across(col.continent, { wrap: 3 })))
```



“Given all the gapminder years: a line derived by sum, x is year, y is population, split into panel columns by continent, wrapped at 3.”

Asia is nearly four billion people and Oceania about twenty-five million, so the axis is sized for Asia and Oceania is a flat line along the bottom. Oceania's line is drawn correctly, but the axis is too tall for its shape to show. The plot is not wrong. It is reporting, correctly, that these five quantities are not the same size. But if the question was “how did each continent grow”, it has answered a different one.

`free = TRUE` on the position fits that axis from each panel's own rows:

R

```
data(gm_all) + line * sum + x(year) +  
  y(population, free = TRUE) | facet(continent, wrap = 3)
```

Python

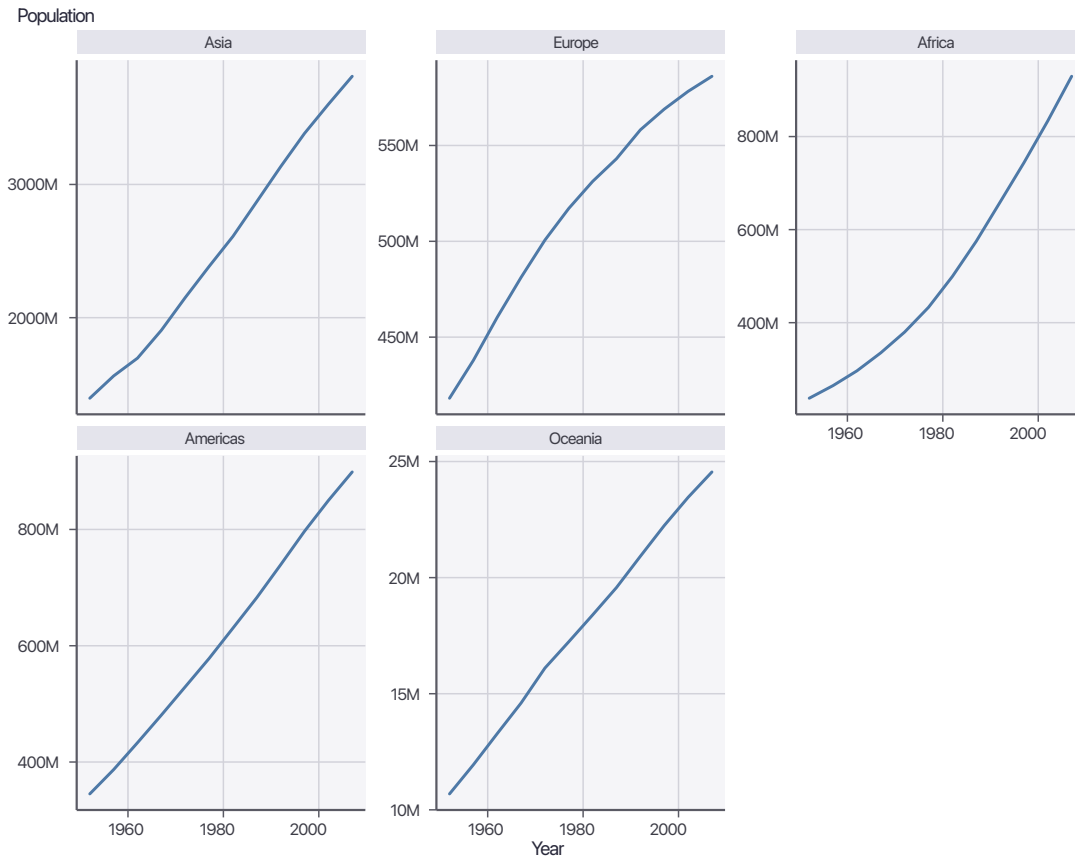
```
(data(gm_all) + line * sum + x(col.year) +  
  y(col.population, free = True) | facet(col.continent, wrap = 3))
```

Julia

```
data(gm_all) + line * sum + x(:year) + y(:population, free = true) |  
  facet(:continent, wrap = 3)
```

JavaScript

```
plot(data(gm_all), layer(line, sum), x(col.year),  
  y(col.population, { free: true }), across(col.continent, { wrap: 3 }))
```



“Given all the gapminder years: a line derived by sum, x is year, y is population, freed, split into panel columns by continent, wrapped at 3.”

Now every panel has a shape, and Oceania turns out to have more than doubled. Notice the y axis: each panel carries its own numbers, because one set of labels along the edge is enough only when the scale is shared.

Notice also what the second plot loses, because that loss is why a free scale is asked for and never assumed. All five panels now look alike: five lines rising left to right at about the same angle. Read side by side, nothing shows that one climb is measured in billions and another in tens of millions. A reader who looks at the shapes and not at the axes will draw the wrong conclusion. The first plot cannot mislead that way and the second can, so `gog` frees a scale only when you ask for it.

Which axis is freed is decided by where you write it. You write **free** inside `x()` or inside `y()`, so the sentence has already named the axis and **free** does not name it again. There is no **free_x** or **free_y**, and freeing both is writing it twice:

R

```
data(gm_all) + point + x(gdp, free = TRUE) + y(life, free = TRUE) |  
  facet(continent, wrap = 3)
```

Python

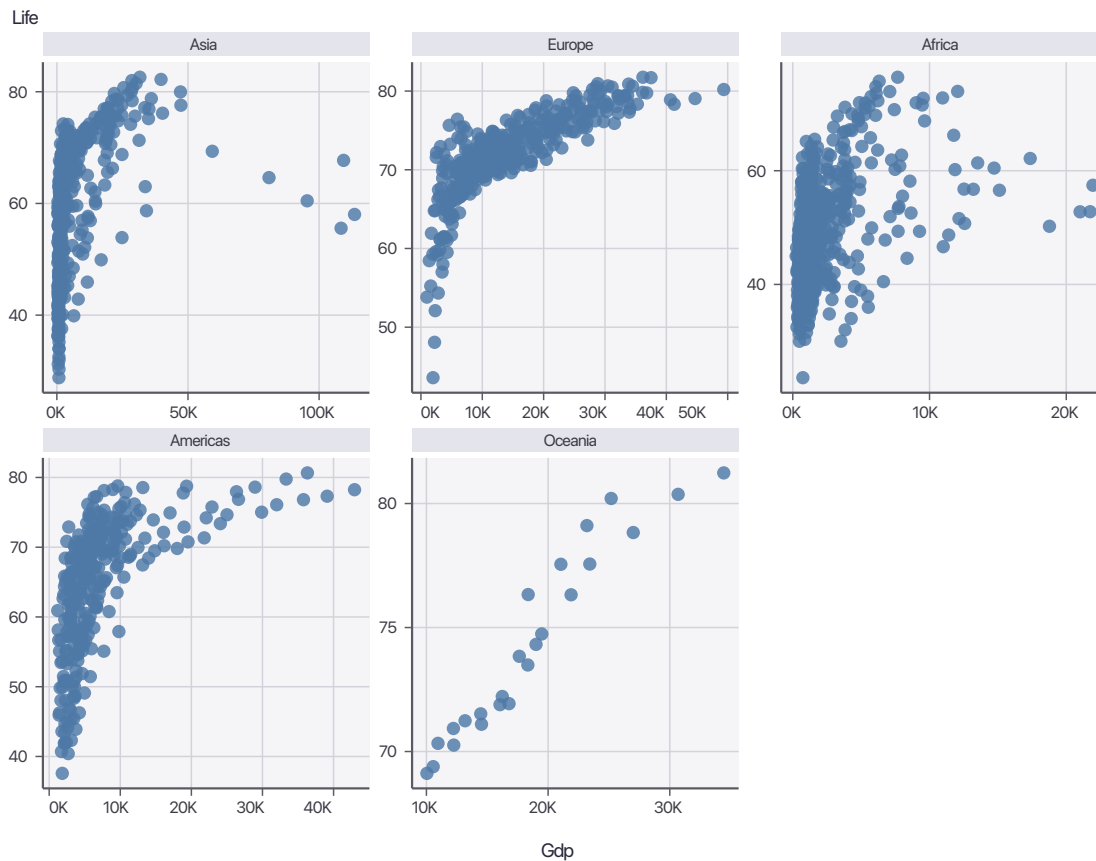
```
(data(gm_all) + point + x(col.gdp, free = True) + y(col.life, free = True) |  
  facet(col.continent, wrap = 3))
```

Julia

```
data(gm_all) + point + x(:gdp, free = true) + y(:life, free = true) |  
  facet(:continent, wrap = 3)
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { free: true }),  
  y(col.life, { free: true }), across(col.continent, { wrap: 3 }))
```



There is a third position, and it takes **free** the same way. A cube sits in a panel like any other plot ([Space](#)):

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  z(population, free = TRUE) + color(continent) | facet(continent)
```

Python

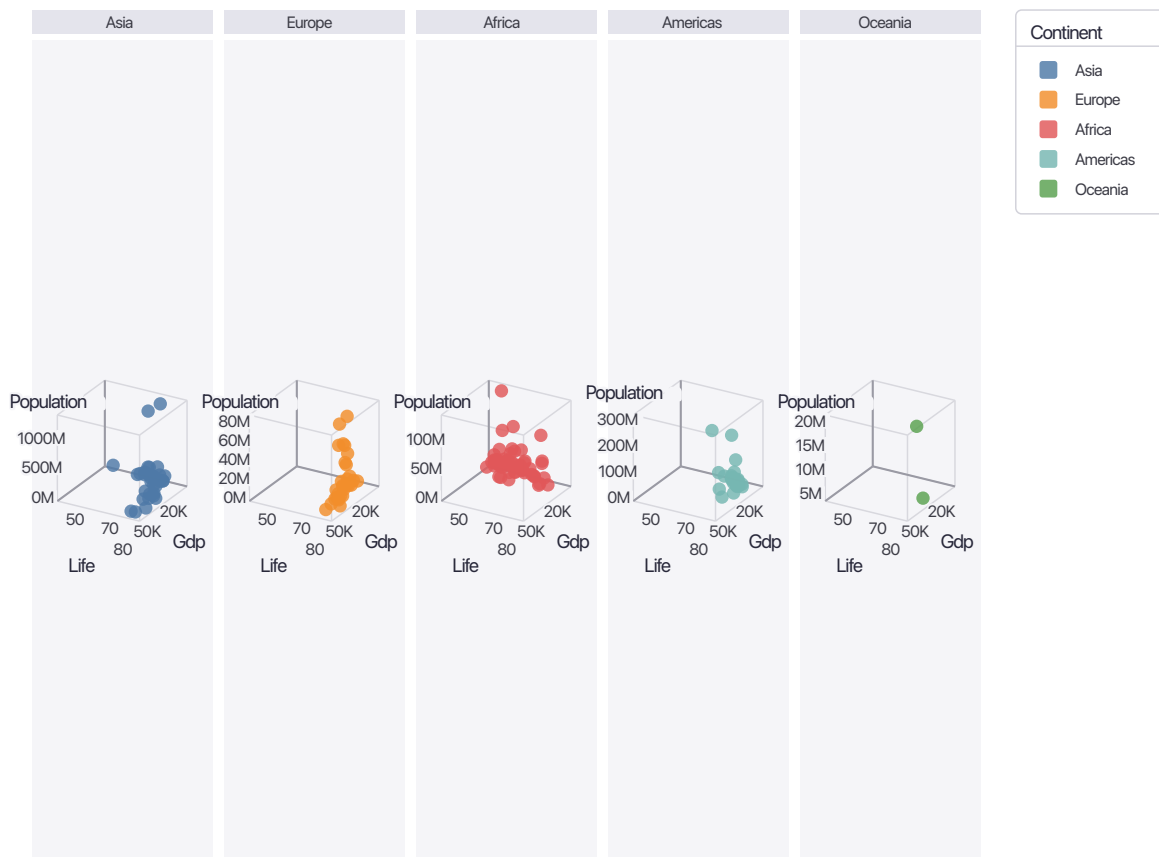
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  z(col.population, free = True) + color(col.continent) |
  ↪ facet(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  z(:population, free = true) + color(:continent) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  z(col.population, { free: true }), color(col.continent),  
  across(col.continent))
```



“Given gapminder 2007: points, x is gdp, y is life, z is population, freed, color by continent, split into panel columns by continent.”

If the height axis were shared, it would have to reach China, and Oceania’s two countries would sit at the very bottom with nothing to read. Freed, Oceania is measured in tens of millions and has a shape again. The loss is the one this section began with: the five cubes can no longer be

compared. That plot needs no new word in the grammar. `free` is written on a position and `z` is a position, so the third one accepts it like the other two.

A free scale brings two consequences and one limit with it, and none of them is a setting you can ask for. The first is the bin cut. The histogram earlier in this chapter shares one set of bin edges across every panel, because a cut describes the axis and the axis was shared. Free the axis and the edges follow it, for exactly the same reason:

R

```
data(gm_all) + bar * bin + x(gdp, free = TRUE) | facet(continent, wrap = 3)
```

Python

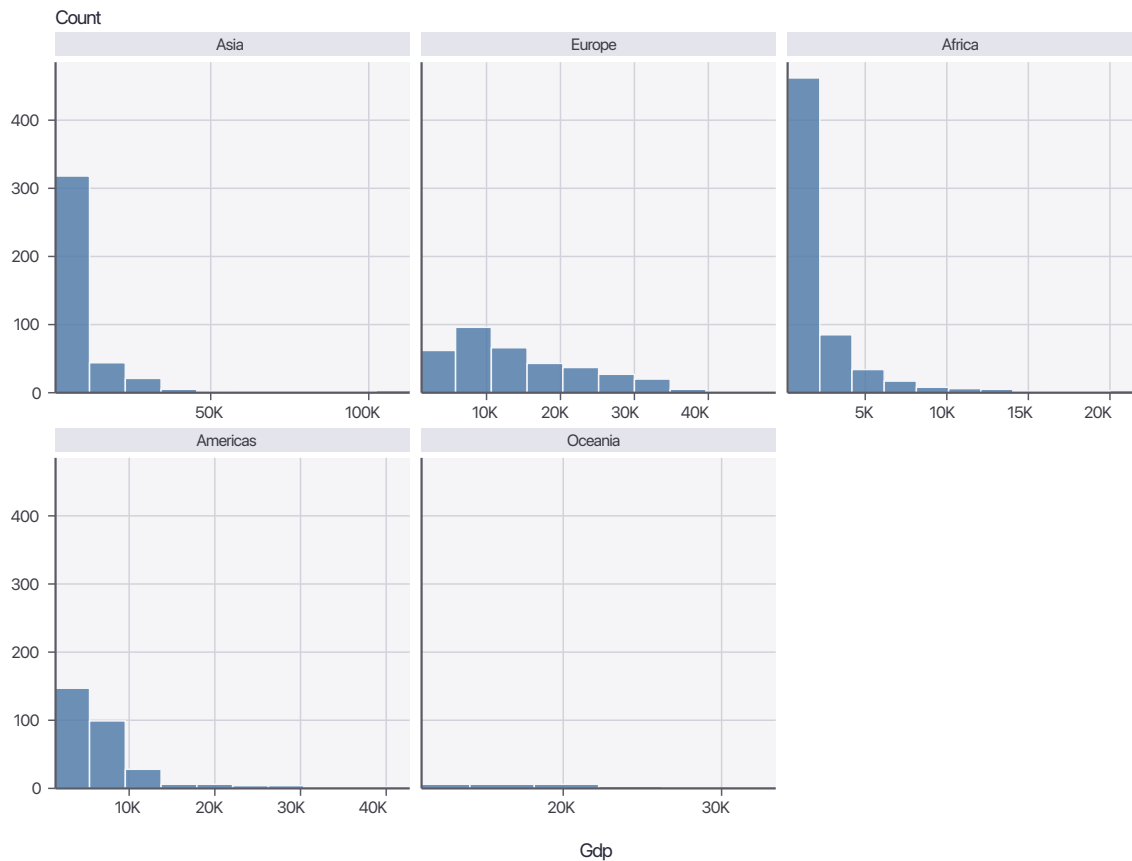
```
data(gm_all) + bar * bin + x(col.gdp, free = True) | facet(col.continent,  
↪ wrap = 3)
```

Julia

```
data(gm_all) + bar * bin + x(:gdp, free = true) |  
facet(:continent, wrap = 3)
```

JavaScript

```
plot(data(gm_all), layer(bar, bin), x(col.gdp, { free: true }),  
across(col.continent, { wrap: 3 }))
```



Each panel is cut from its own rows, so Africa gets bins the size of Africa’s spread rather than bins sized for the richest countries in the data. You cannot ask for a free scale over a shared cut. That picture would count each panel’s own rows into edges cut from every panel at once.

The second is the panel’s **guides**: its axis numbers and gridlines. A scale decides where the marks go and where the axis prints its numbers. Freeing the scale has to move both, or the numbers stop describing the picture. Look again at the freed population plot above. Its gridlines sit at different heights in every panel, and each panel’s labels sit beside its own. Shared, those same lines are drawn at one set of heights across all five. That is the whole of what a freed guide means: the lines a reader measures against belong to the scale that was freed.

The third is that a free scale belongs to a panel, never to an animation frame. `play` ([Animation](#)) splits rows the same way a facet does, so you can write `free` on a played plot. It is refused:

```
render_svg(data(gm_all) + point + x(gdp) + y(life, free = TRUE) + play(year))
```

Error:

```
! gog: `y(life, free = TRUE)` fits one scale per *panel*, and this plot has
  ↳ frames rather than panels. A frame replaces the one before it, so an axis
  ↳ refitted per frame would move under the data and the motion would be the
  ↳ scale's rather than the data's. Facet the plot to free a scale across
  ↳ panels, or leave the sequence on one scale.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The reason is in the message. Panels sit beside each other and a reader compares them. A frame *replaces* the one before it, so an axis refitted every frame would move under the data. The motion on screen would then be the scale's, not the data's. Across panels the freedom is safe. Across frames it would misreport the data.

Free scales are refused elsewhere for the same kind of reason. On a channel that is not a position there is no axis to free, only a legend, and that one legend decodes the whole plot. And with no facet there are no panels:

```
render_svg(data(gapminder_2007) + point + x(gdp) + y(life, free = TRUE))
```

Error:

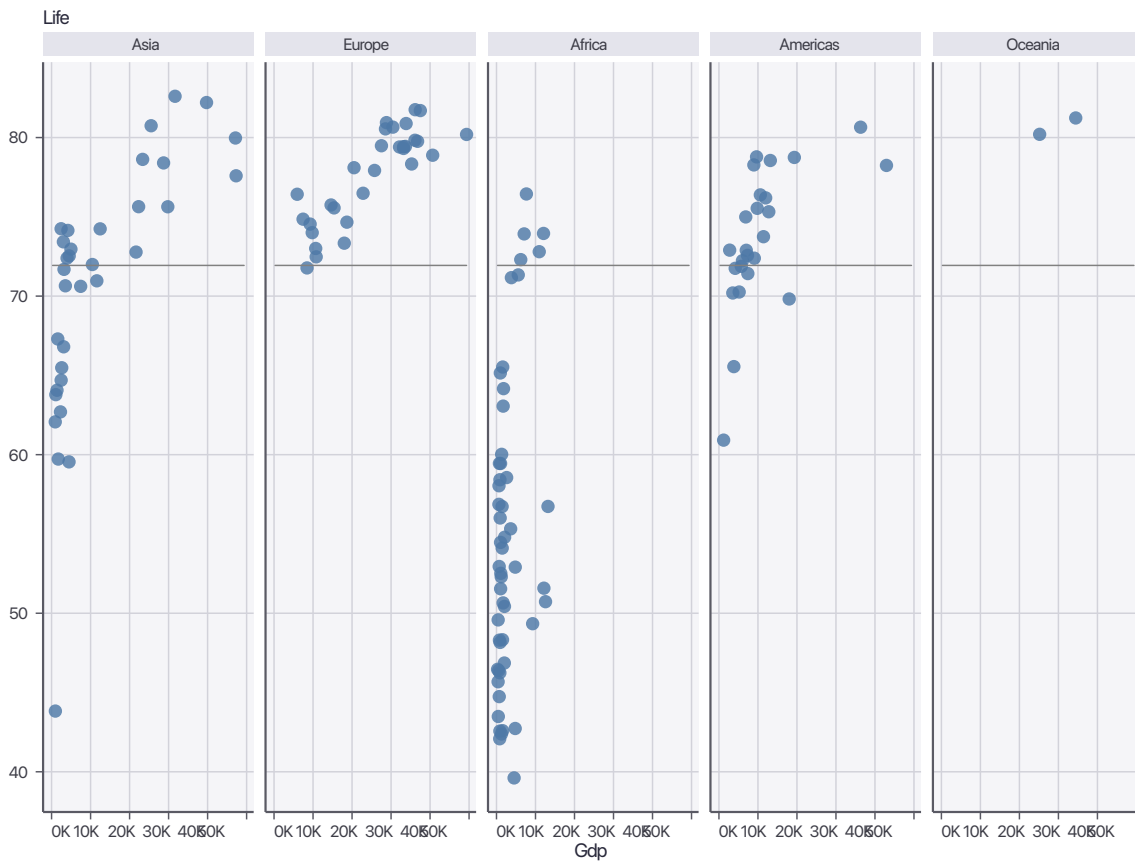
```
! gog: `y(life, free = TRUE)` fits one scale per panel, and this plot has one
  ↳ panel. Facet it - `plot | facet(<column>)` - or drop `free`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

28.6 A layer without the facet column

A layer whose table does not have the facet column is drawn in full in every panel. That is how one shared reference line is drawn behind small multiples:

```
world_median <- data.frame(
  gdp = c(min(gapminder_2007$gdp), max(gapminder_2007$gdp)),
  life = rep(median(gapminder_2007$life), 2)
)
data(gapminder_2007) + point + x(gdp) + y(life) +
  data(world_median) + line + style(color = "gray", size = 1) |
  facet(continent)
```

```
gog: table `world_median` has no column `continent`, so its layer is drawn in
  ↳ every panel. If it should be split too, add `continent` to that table.
```



“Given gapminder 2007: points, x is gdp, y is life, and also a line from the world median, split into panel columns by continent.”

The gray line, the world’s median life expectancy, repeats in every panel, so each continent is read against the same line. The engine reports this as an assumption rather than doing it in silence: the table `world_median` has no `continent` column, so its layer is drawn everywhere.

28.7 The facet column’s type

A facet column *names* the panels, so it must be a categorical column: a number is a position along an axis, not a name for a frame. Faceting by a numeric column refuses with direction:

```
render_svg(data(gapminder_2007) + point + x(gdp) + y(life) |
  ↪ facet(population))
```

Error:

```
! gog: `facet(population)` splits on a number column, but a facet variable
  ↳ names the panels, so it must be a category column. Make `population` text
  ↳ - in R, `factor(population)` - or cut it into named groups first.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

`shape` and `group` ask the same question of a column, and a facet asks it of the whole plot's panels. In R, `factor()`, or any explicit cut into named groups, is the way to say which panels you mean.

Trellis named that cut a **shingle**, and `lattice` still builds one with `shingle()` or `equal.count()`. A shingle's intervals are allowed to overlap, so one row can appear in two panels. That makes the change from panel to panel smoother, and it counts some rows twice. Cutting the column yourself with `cut()` gives up the smoothing and puts the boundaries somewhere a reader can check them, in the strip labels.

Faceting joins with the operators, never with `+`; a facet is a property of the whole plot's frame, not one more encoding in the chain. Writing `+ facet(continent)` refuses and points at `|` and `/`.

29 Composition

How do you put two different plots on one page? Faceting splits one plot into panels. Composition does the opposite: it takes plots that were never one plot and arranges them on a page. Wilkinson keeps the two apart, noting that “some tabled graphics are really two or more graphics glued together”. `gog` keeps them apart in the operators you have already learned. `|` and `/` place things side by side and one above the other, whatever those things are:

- `plot | facet(continent)`: one plot, split by a column, panels side by side
- `plot_a | plot_b`: two plots, arranged side by side
- `plot_a / plot_b`: two plots, one above the other

A facet splits one plot by a column of its data. A page arranges finished plots, and says nothing about any plot’s data. Which one you get is decided by what you write on the right of the operator, and by nothing else.

29.1 Two plots, one page

R

```
((data(gapminder_2007) + point + x(gdp) + y(life)) |  
  (data(gapminder_2007) + bar * count + x(continent) + theme(tick_angle =  
    ↪ 45)))
```

Python

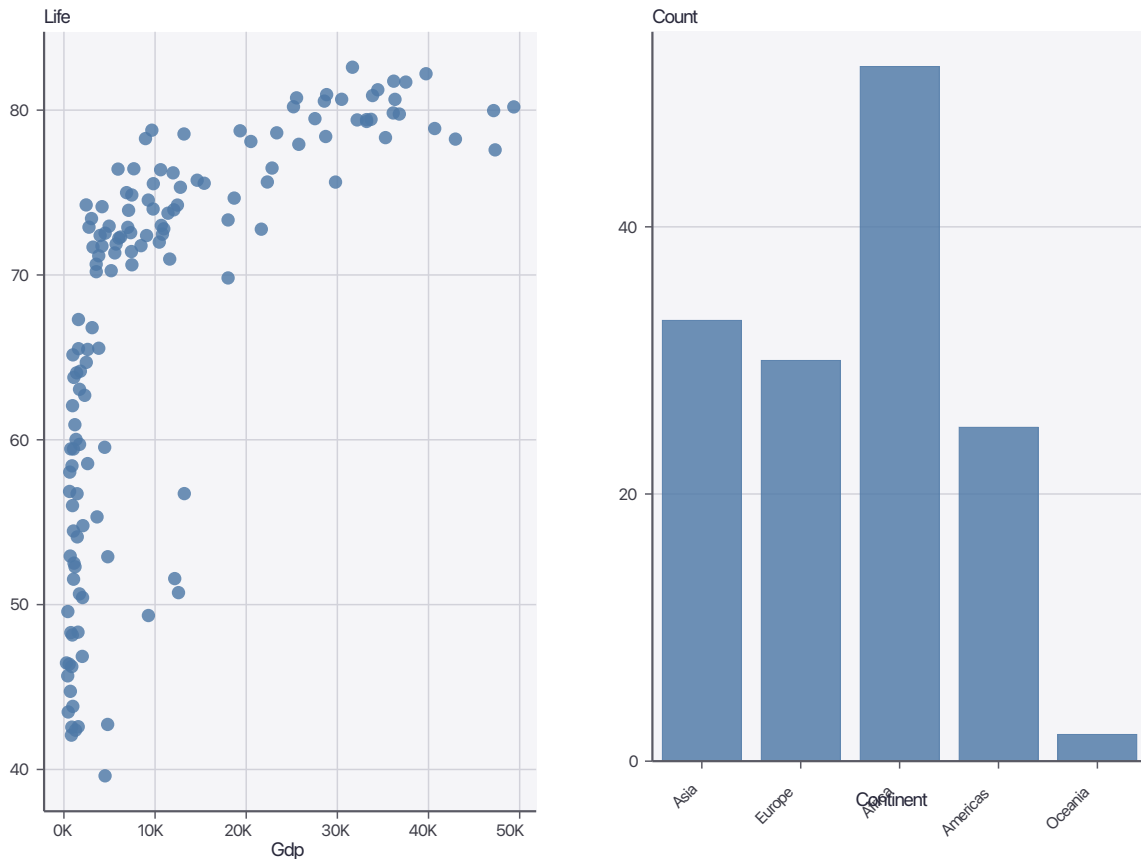
```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) |  
  (data(gapminder_2007) + bar * count + x(col.continent) + theme(tick_angle =  
    ↪ 45)))
```

Julia

```
((data(gapminder_2007) + point + x(:gdp) + y(:life)) |  
  (data(gapminder_2007) + bar * count + x(:continent) +  
    theme(tick_angle = 45)))
```

JavaScript

```
beside(plot(data(gapminder_2007), point, x(col.gdp), y(col.life)),  
  plot(data(gapminder_2007), layer(bar, count), x(col.continent),  
    theme({ tick_angle: 45 })))
```



“Given gapminder 2007: points, x is gdp, y is life, beside bars derived by count, x is continent.”

Two entirely different measurements: a country’s income against its life expectancy, and how many countries each continent holds. They share nothing, not a scale, not an axis, not a legend, because there is nothing to share. The page divides evenly between them.

/ places the same two one above the other:

R

```
(data(gapminder_2007) + point + x(gdp) + y(life)) /  
  (data(gapminder_2007) + bar * count + x(continent))
```

Python

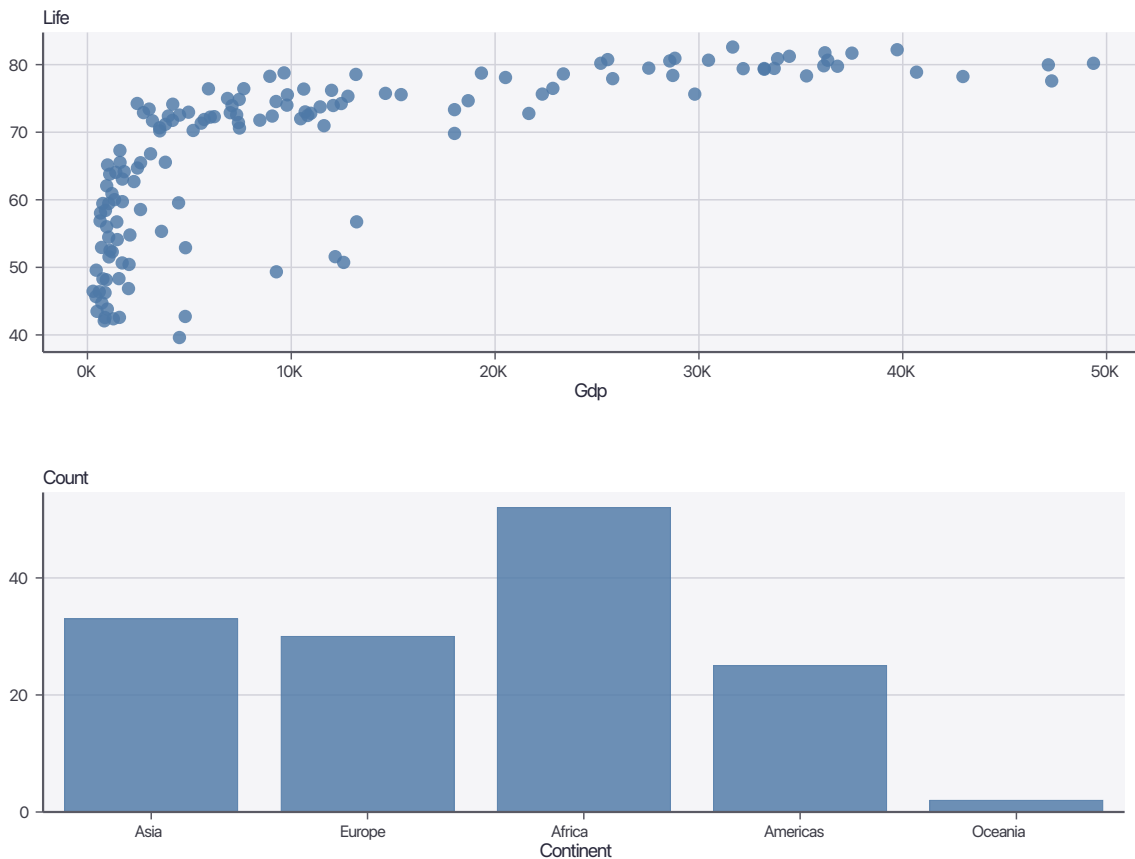
```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) /  
  (data(gapminder_2007) + bar * count + x(col.continent)))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp) + y(:life)) /  
  (data(gapminder_2007) + bar * count + x(:continent))
```

JavaScript

```
below(plot(data(gapminder_2007), point, x(col.gdp), y(col.life)),  
  plot(data(gapminder_2007), layer(bar, count), x(col.continent)))
```

“Given gapminder 2007: points, x is gdp, y is life, above bars derived by count, x is continent.”

Both operators take pages as well as plots, so a page can hold another page. $(a \mid b) / c$ is a row of two plots above a third. In R, Python and Julia, $/$ is applied before $\mid$, so $a \mid b / c$ reads as $a \mid (b / c)$. Add parentheses when the reading matters. JavaScript spells the operators as words, so there is no order to remember. Its chapter shows the spelling.

29.2 A shared column is a shared axis

Two composed plots are unrelated until they name the same column on the axis the arrangement lines up: x for plots one above the other, and y for plots side by side. When they do, that is not two axes that happen to look alike; it is one axis, and `gog` treats it as one:

The same column on the aligned axis of two composed plots is one axis: one scale, one panel extent, drawn once.

Here are two plots, both reading `gdp` on x. The `theme(height = 150)` on the first asks for 150 pixels of the page's height:

R

```
(data(gapminder_2007) + bar * bin + x(gdp) + theme(height = 150)) /  
  (data(gapminder_2007) + point + x(gdp) + y(life))
```

Python

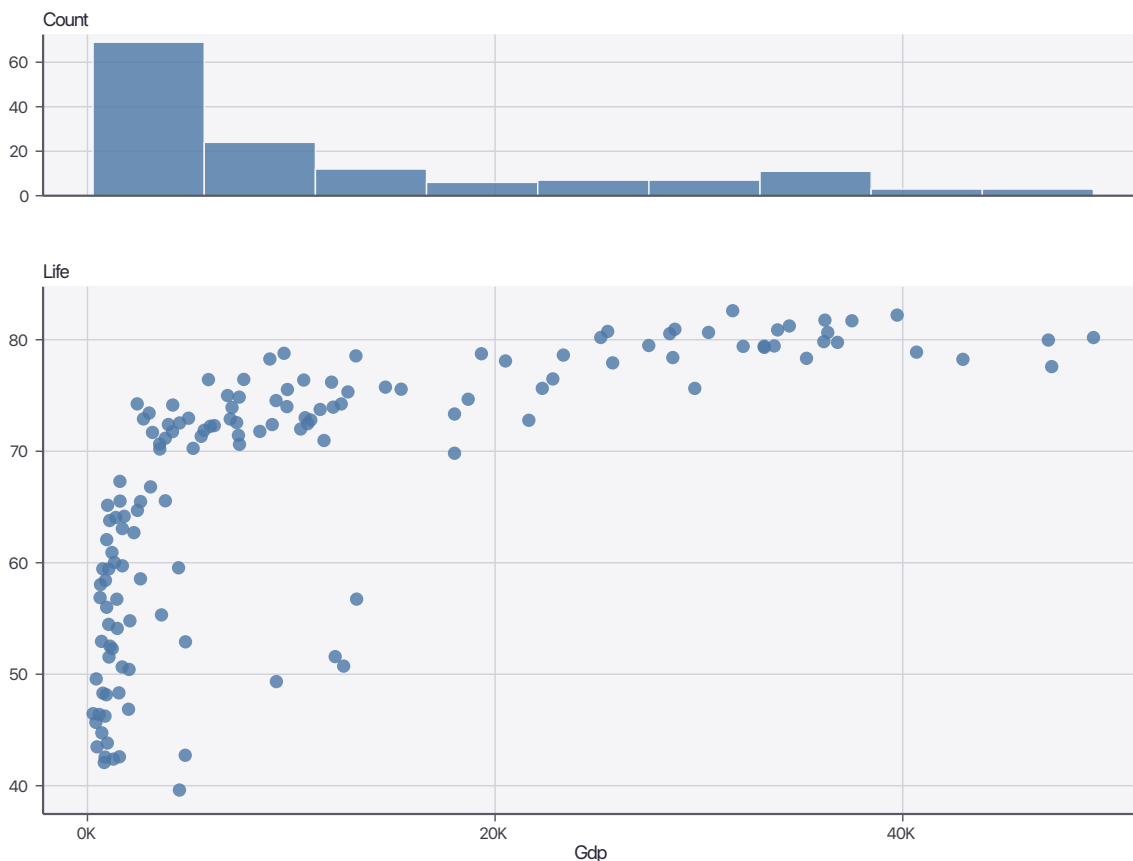
```
((data(gapminder_2007) + bar * bin + x(col.gdp) + theme(height = 150)) /  
  (data(gapminder_2007) + point + x(col.gdp) + y(col.life)))
```

Julia

```
(data(gapminder_2007) + bar * bin + x(:gdp) + theme(height = 150)) /  
  (data(gapminder_2007) + point + x(:gdp) + y(:life))
```

JavaScript

```
below(plot(data(gapminder_2007), layer(bar, bin), x(col.gdp),  
  theme({ height: 150 })),  
  plot(data(gapminder_2007), point, x(col.gdp), y(col.life)))
```



“Given gapminder 2007: bars derived by bin, x is gdp, above points, x is gdp, y is life.”

Look at what the rule did. The histogram’s panel is exactly as wide as the scatter’s, so a bar stands over the points it counts. Both plots measure **gdp** over the same range, so equal distances across the two panels stand for equal amounts of **gdp**. And the **gdp** axis is written once, along the bottom, because there is one axis: the same decision faceting makes when it puts tick labels only under the bottom row of panels.

That is why the histogram touches the scatter instead of standing a tick label’s height above it. A plot that draws no axis reserves no margin for one.

29.2.1 One axis, but not one legend

Compose two plots that both map **color** and you get two legends, one for each:

R

```
(data(gapminder_2007) + point + x(gdp) + y(life) + color(continent)) |
  (data(gapminder_2007) + point + x(population) + y(life) + color(continent))
```

Python

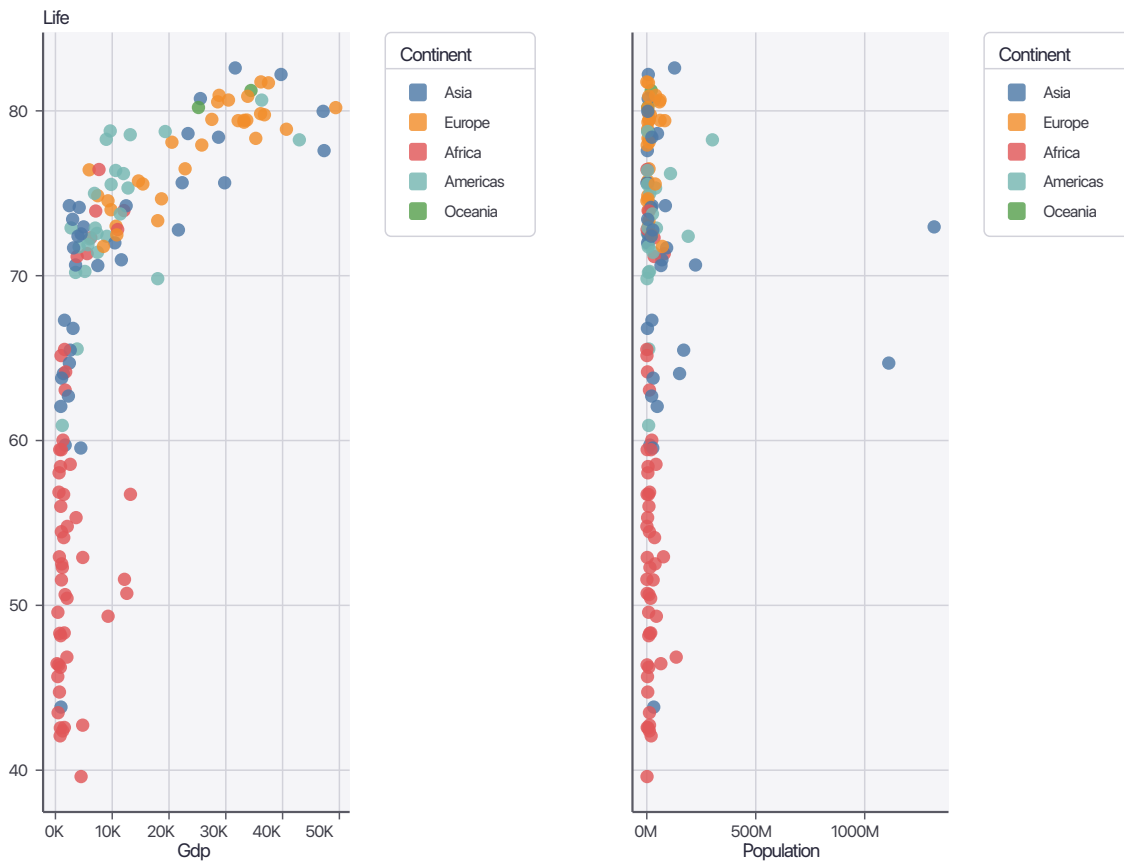
```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  ↪ color(col.continent)) |
  (data(gapminder_2007) + point + x(col.population) + y(col.life) +
  ↪ color(col.continent)))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent)) |
  (data(gapminder_2007) + point + x(:population) + y(:life) +
  color(:continent))
```

JavaScript

```
beside(plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  color(col.continent)),
  plot(data(gapminder_2007), point, x(col.population), y(col.life),
  color(col.continent)))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent, beside points, x is population, y is life, color by continent.”

Look at what that figure draws once, and what it draws twice. Both plots read **life** on **y**, so that axis is shared and written a single time, on the left. Both color by **continent** as well, and there you get *two* legends, saying exactly the same thing.

The asymmetry is deliberate. Axes merge because there is an exact test: the same column on the same axis means one scale is provably right for both plots. A legend has no such test. These two agree, but two composed plots are two plots, and nothing in the grammar makes them agree. The next pair may split by different columns, or map one column onto two scales. A single legend over that is wrong for one of them. Nothing in the picture shows the error, because it still looks like a legend.

gog merges no legend, because it cannot prove that one legend is right for both plots. Two identical legends side by side show two plots that agree.

When you want one legend over several panels, use [faceting](#) instead. A facet is *one* plot split by a column, so every panel is colored by the same scale. It draws a single legend. The choice between `|` and `facet()` is the choice between two plots and one.

29.3 The marginal plot

The histogram above the scatter is a **marginal**: one column's distribution, drawn along the edge of the plot that uses it. Put a second marginal on the right and the picture is the marginal plot, which most tools offer as a named chart type:

R

```
(data(gapminder_2007) + bar * bin + x(gdp) + theme(height = 130)) /  
  ((data(gapminder_2007) + point + x(gdp) + y(life)) |  
   (data(gapminder_2007) + bar * bin + y(life) + theme(width = 130)))
```

Python

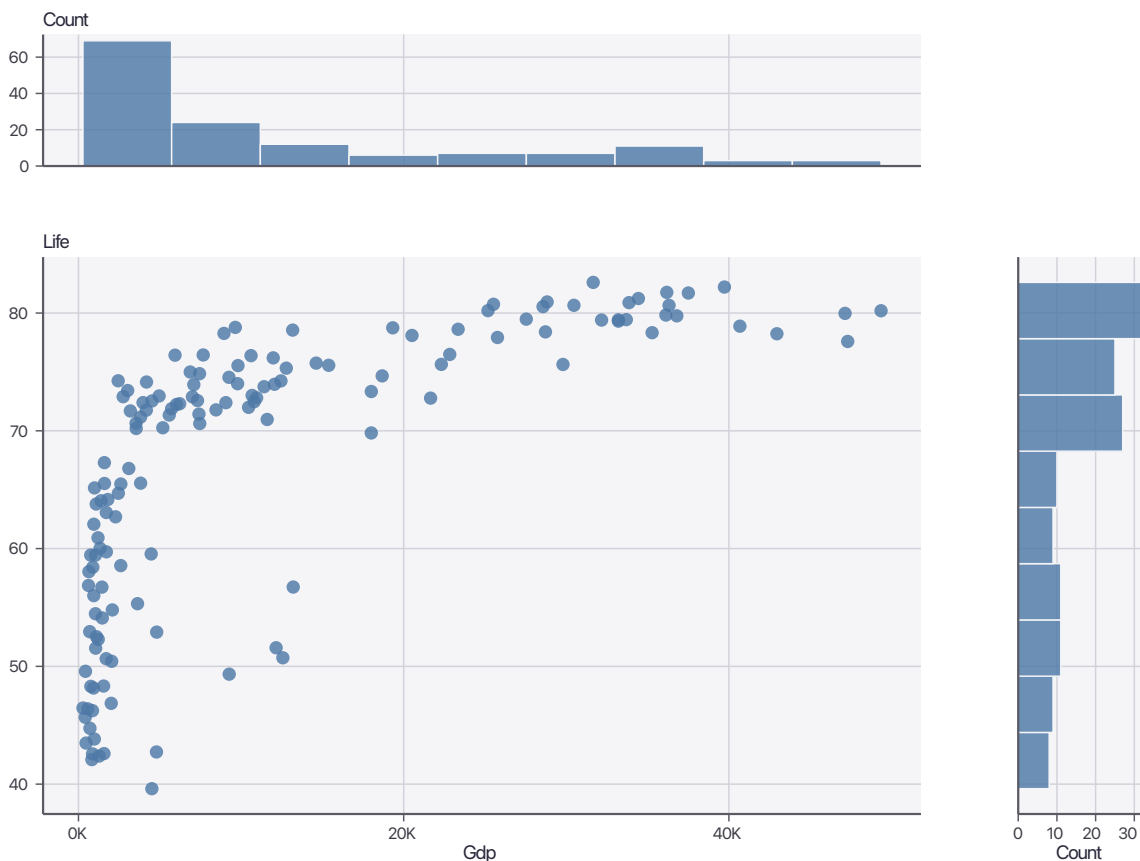
```
((data(gapminder_2007) + bar * bin + x(col.gdp) + theme(height = 130)) /  
  ((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) |  
   (data(gapminder_2007) + bar * bin + y(col.life) + theme(width = 130))))
```

Julia

```
(data(gapminder_2007) + bar * bin + x(:gdp) + theme(height = 130)) /  
  ((data(gapminder_2007) + point + x(:gdp) + y(:life)) |  
   (data(gapminder_2007) + bar * bin + y(:life) + theme(width = 130)))
```

JavaScript

```
below(plot(data(gapminder_2007), layer(bar, bin), x(col.gdp),  
  theme({ height: 130 })),  
  beside(plot(data(gapminder_2007), point, x(col.gdp), y(col.life)),  
    plot(data(gapminder_2007), layer(bar, bin), y(col.life),  
      theme({ width: 130 }))))
```



“Given gapminder 2007: bars derived by bin, x is gdp, above points, x is gdp, y is life, beside bars derived by bin, y is life.”

Three ordinary plots and two operators. The top histogram shares **gdp** with the scatter, so it is drawn exactly as wide as the scatter’s panel. The right-hand one shares **life**, so its panel is exactly as tall, and the **life** axis is drawn once on the left. The empty top-right corner has no atom of its own. It is the room left over once both shared axes have taken their width and height.

The marginals follow no special rule. `bar * bin + y(life)` is the histogram you already know, with its bars running left to right, because that is what binding the measure to **y** means. And the shared column need not be continuous: two plots reading the same *category* on **x** share its slots, so a box plot’s summary stands over the points it summarizes, one category at a time:

R

```
(data(gapminder_2007) + box + x(continent) + y(life) + theme(height = 150)) /
  (data(gapminder_2007) + point + x(continent) + y(gdp) + color(continent))
```

Python

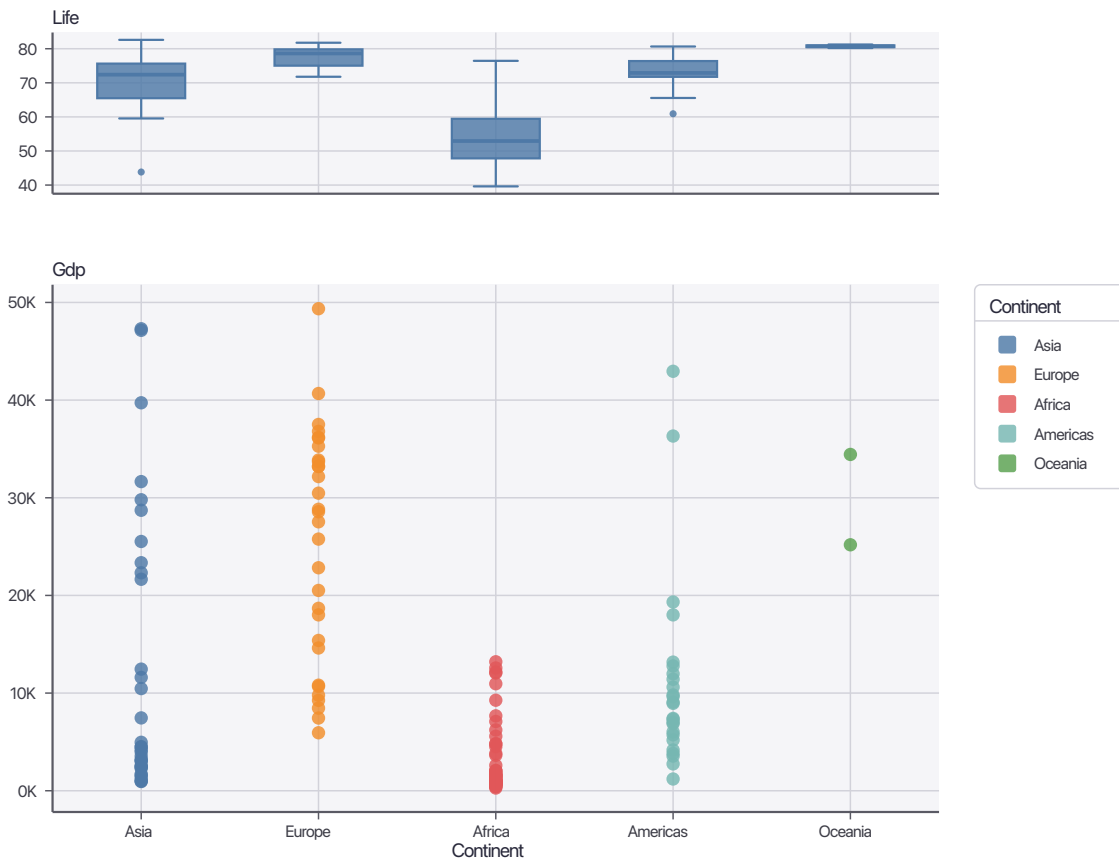
```
((data(gapminder_2007) + box + x(col.continent) + y(col.life) + theme(height
↪ = 150)) /
  (data(gapminder_2007) + point + x(col.continent) + y(col.gdp) +
↪ color(col.continent)))
```

Julia

```
(data(gapminder_2007) + box + x(:continent) + y(:life) +
  theme(height = 150)) /
  (data(gapminder_2007) + point + x(:continent) + y(:gdp) +
  color(:continent))
```

JavaScript

```
below(plot(data(gapminder_2007), box, x(col.continent), y(col.life),
  theme({ height: 150 })),
  plot(data(gapminder_2007), point, x(col.continent), y(col.gdp),
  color(col.continent)))
```

“Given gapminder 2007: boxes, x is continent, y is life, above points, x is continent, y is gdp, color by continent.”

29.4 A clustered panel decides the order

A categorical axis needs an order, and most plots take the one the data arrived in. A panel with `cluster` computes a better one. `cluster` builds a tree of similar rows, and reading its leaves from left to right gives an order that puts similar things next to each other. When such a panel shares its axis, the shared axis takes that order.

A shared categorical axis takes the order a clustered panel computed for it.

The panels beside the clustered one stay plain, and their slots arrive in the tree’s order. `gog` refuses two panels that derive different orders for one shared axis, because one axis cannot hold two orders. Give both the same columns to `cluster()`, or cluster in one panel alone. The

[Cluster](#) chapter uses this rule: a tile plot with a tree on two of its sides, each tree deciding one axis.

29.5 How thin is the marginal histogram? `theme(width =, height =)`

A plot states how much room it wants in pixels, and the plots that ask for nothing split what is left. A page that says nothing is 800 by 600, which is what a plot drawn alone takes. That is the whole of the sizing above: the marginal histogram, for example, asks for 130 pixels of height, the scatter asks for nothing and takes the rest.

The same two words also size a plot drawn on its own, where the room being divided is the whole image:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + theme(width = 420, height =  
  ↪ 320)
```

Python

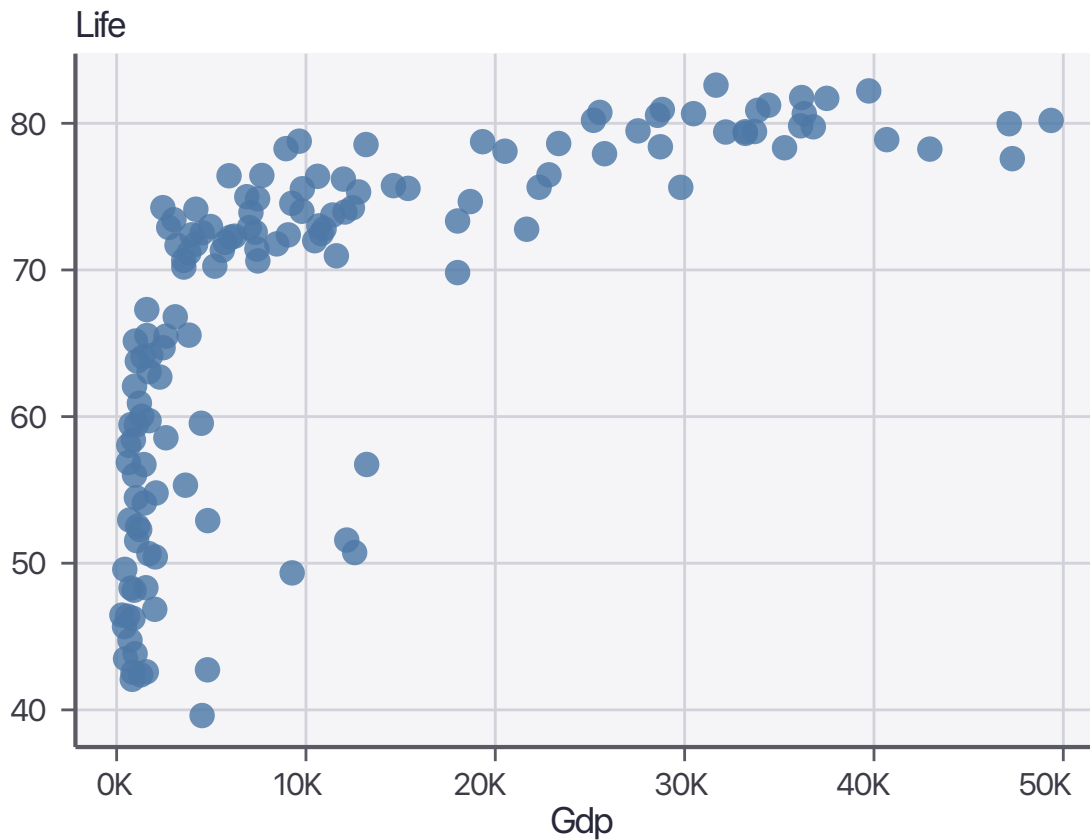
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) + theme(width = 420,  
  ↪ height = 320)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  theme(width = 420, height = 320)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  theme({ width: 420, height: 320 })))
```



One meaning in two places, which is Compositional Invariance: the sentence that says a plot is 130 pixels tall says it whether or not anything is composed with it. Do not confuse it with `theme(ratio = )`, which shapes the *panel* inside whatever room the plot was given and never resizes the image.

Plots that ask for more height than the page has are refused, and the message shows the arithmetic:

```
render_svg(
  (data(gapminder_2007) + point + x(gdp) + y(life) + theme(height = 500)) /
  (data(gapminder_2007) + point + x(gdp) + y(life) + theme(height = 500))
)
```

Error:

! gog: the plots below each other ask for 1000px of height between them, and
 ↳ the page has 600. A ``theme(height = )`` on a composed plot is how much of
 ↳ the page that plot takes, so the ones that state it must leave room for
 ↳ the ones that do not.
 gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

29.6 How big is the page itself?

Two plots set side by side divide the page's width, and each keeps the whole of its height. So a height written into one of them changes nothing. Each plot sits in a cell of the page, and a cell in a side-by-side page is already as tall as the page. The height belongs to the page, so the page is where it is written:

R

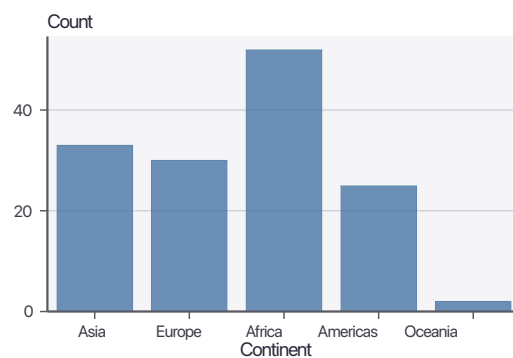
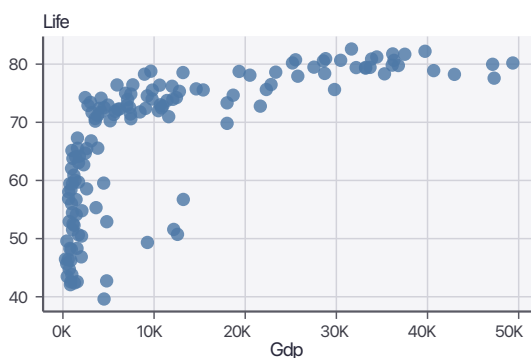
```
((data(gapminder_2007) + point + x(gdp) + y(life)) |
  (data(gapminder_2007) + bar * count + x(continent))) + theme(height = 260)
```

Python

```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) |
  (data(gapminder_2007) + bar * count + x(col.continent))) + theme(height =
  ↳ 260))
```

Julia

```
((data(gapminder_2007) + point + x(:gdp) + y(:life)) |
  (data(gapminder_2007) + bar * count + x(:continent))) +
  theme(height = 260)
```



The same two words, now written on the page instead of the plot. A plot states how much room it wants, and so does a page. Composed, the first is a cell of the second.

This matters most where a mark cannot be stretched to fill its panel. A `cube` is drawn with one scale in both screen directions, so its shape is never stretched. A tall, narrow cell therefore leaves it small, with empty bands above and below. The figure is the wrong shape, and the page is where you fix it.

Size is the only thing a page can say. Every other property `theme()` sets describes a panel: its gridlines, its frame, its background, the angle its tick labels are read at. A page has no panel, so each of those properties is refused, and the refusal names the plot it belongs to:

```
render_svg(((data(gapminder_2007) + point + x(gdp) + y(life)) |  
             (data(gapminder_2007) + bar * count + x(continent)))) +  
  ↪ theme(grid = "none"))
```

Error:

```
! gog: `theme(grid = )` describes a panel, and a page is plots arranged  
  ↪ rather than a panel of its own. On a page, `theme()` states how big the  
  ↪ figure is - `theme(width = )` and `theme(height = )` - and nothing else.  
  ↪ Write this into the plot it describes, before composing: `(plot +  
  ↪ theme(grid = )) | other_plot`.
```

29.7 What a page is not

A page is plots arranged, so anything belonging to *one* plot is written into that plot before it is composed. Apart from the size above, an atom added to a page refuses and says so:

```
render_svg(((data(gapminder_2007) + point + x(gdp) + y(life)) |  
             (data(gapminder_2007) + bar * count + x(continent)))) +  
  ↪ title("Two views"))
```

Error:

```
! gog: `title()` belongs to a plot, and the left side is a page of them.  
  ↪ Write it into the plot it describes, before composing: `(plot +  
  ↪ title("...")) | other_plot`.
```

And a page cannot be faceted, because a facet splits one plot by a column and a page is not one plot:

```
render_svg(((data(gapminder_2007) + point + x(gdp) + y(life)) |  
             (data(gapminder_2007) + bar * count + x(continent)))) |  
  ↪ facet(continent))
```

Error:

```
! gog: `|` faceted a page of plots, and a facet splits *one* plot by a
  ↪ column. Facet the plots before composing them: `(plot | facet(g)) |
  ↪ other_plot`.
```

Facet first, then compose: `(plot | facet(g)) | other_plot` is a faceted plot beside another plot, and it is perfectly legal.

29.8 Where these operators come from

Neither reading of `|` is this grammar’s invention, and the two were borrowed from different places.

The **facet** reading is the older. Trellis display wrote a conditioned plot as `y ~ x | g` (Becker et al., 1996), and **lattice** writes it that way still; the bar means “given”, which is the bar written in conditional probability. That is exactly what `plot | facet(continent)` says.

The **composition** reading comes from patchwork (Pedersen, 2019). That package writes `p1 | p2` for side by side and `p1 / p2` for one above the other. **gog** spells composition the same way on purpose: a reader who knows that package already knows this chapter, and a third spelling of a settled idea would help nobody.

Composing pictures with operators is older than either. Henderson’s *Functional Geometry* (Henderson, 1982) describes an algebra of pictures, and its two combining operations are **beside** and **above**. *Structure and Interpretation of Computer Programs* (SICP) teaches the same picture language, and Haskell’s diagrams library spells the two operations `|||` and `===`. The JavaScript binding has no operators to overload, so it names its two functions `beside()` and `below()` after that older algebra.

One divergence is worth naming, since Wilkinson settles most questions in this project. His graph algebra also has a `/`, and it means **nest** (`a/b`, “a within b”), not “below”. **gog**’s facet operators take his *crossing* and do not use his nesting, so `/` here can mean the page.

29.9 Layering, faceting, or composition?

The three answer different questions, so your data decides which one you want:

- **Layering** when the marks are *different tables on the same measurement*. One coordinate space, one pair of axes, and the second table annotates the first: a threshold, a band, a callout, a forecast. [Data](#) explains multi-table plots.
- **Faceting** when the panels are *the same measurement* on different subsets. One scale, one legend, and the comparison between panels is what you gain.

- **Composition** when the plots are *different measurements* that belong on one page. Each keeps its own space, and the only thing they may share is a column.

Layering is the one you have to verify, because it is the only one the engine cannot check for you. A second table that measures something the axes do not measure still draws when you layer it. The columns resolve, the marks are drawn, and the picture is wrong. Sales figures on an income axis are a legal sentence. So if the second table does not measure what the axes measure, compose two plots instead of layering one onto the other.

A marginal plot is composition because the histogram and the scatter measure different things: one counts countries, the other puts each country at its own position. They share **gdp**, and that shared column is exactly what the rule above matches.

30 Flow

Where did each part go? Some tables record movement: each row says that some amount started in one place and finished in another. A flow diagram draws that movement. The places are sorted into stages that stand side by side, and bands carry each amount from stage to stage. The wider the band, the larger the amount it carries. The world calls this picture a Sankey diagram, or an alluvial diagram, and as with the pie there is no atom that spells either name.

A **stage** is one of the table's categorical columns. Its distinct values are the possible places at that step, and each row sits in one of them. The **flow** transform computes the whole diagram from one table of such columns. This chapter builds the diagram one layer at a time, using the passengers and crew of the Titanic. **titanic** holds one row per group of people. The columns are **class**, **sex**, **age** (child or adult) and **survived**. The last column, **n**, is how many people the group held. The table has one row for every combination of those four columns, so some rows count zero people:

Table 30.1: **titanic**: first 5 of 32 rows

class	sex	age	survived	n
1st	Male	Child	No	0
2nd	Male	Child	No	0
3rd	Male	Child	No	35
Crew	Male	Child	No	0
1st	Female	Child	No	0

30.1 One row is one path

Name the stages in the atom, left to right, and bind the count to y:

R

```
data(titanic) + ribbon * flow(class, survived) + y(n) +  
  title("Who survived, by class")
```

Python

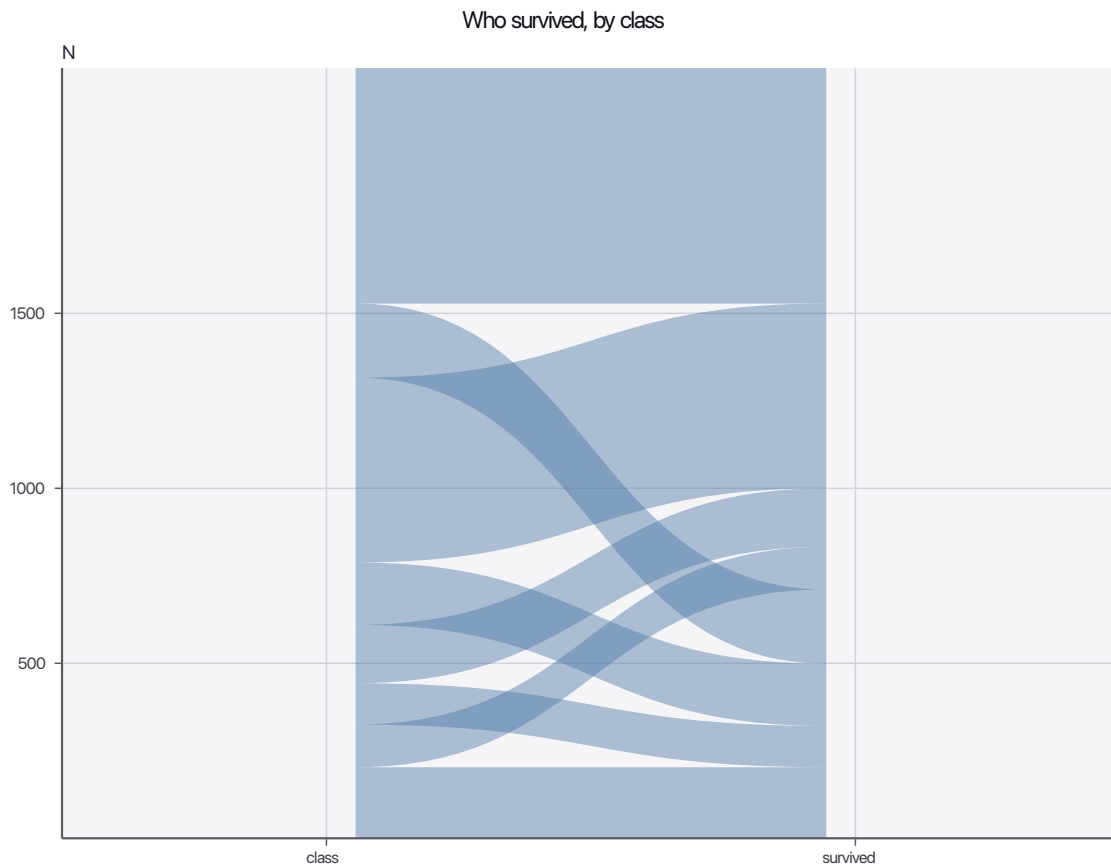

```
(data(titanic) + ribbon * flow(col["class"], col.survived) + y(col.n) +
  title("Who survived, by class"))
```

Julia

```
data(titanic) + ribbon * flow(:class, :survived) + y(:n) +
  title("Who survived, by class")
```

JavaScript

```
plot(data(titanic), layer(ribbon, flow(col.class, col.survived)),
  y(col.n), title("Who survived, by class"))
```



“Given titanic: ribbons derived by flow through class and survived, y is n.”

Each band is one path through the stages. This plot draws no names, so find a path by its position: the lowest band leaves the **1st** slot and arrives at the **Yes** slot. It carries the 203 first-class passengers who survived, and it holds that thickness at both ends. Rows that share a path are added together, so the four **1st** rows that end in **Yes**, one per sex and age, become one band. The same addition combines rows that differ only in the columns the atom does not name, **sex** and **age** here.

The stages stand on the horizontal axis, named by their columns. `x()` has nothing left to set: the atom's argument order is the order the stages are drawn in. The vertical axis is the count, with the same ticks any other plot would get. The bands at each stage stack from zero with no gaps, so the top edge of any band gives the total of that band and every band under it.

30.2 Adding a stage

Add a stage by adding a column. Each path now crosses three stages, and each band still carries one path's count at both of its ends:

R

```
data(titanic) + ribbon * flow(class, sex, survived) + y(n) +  
  title("Class, then sex, then survival")
```

Python

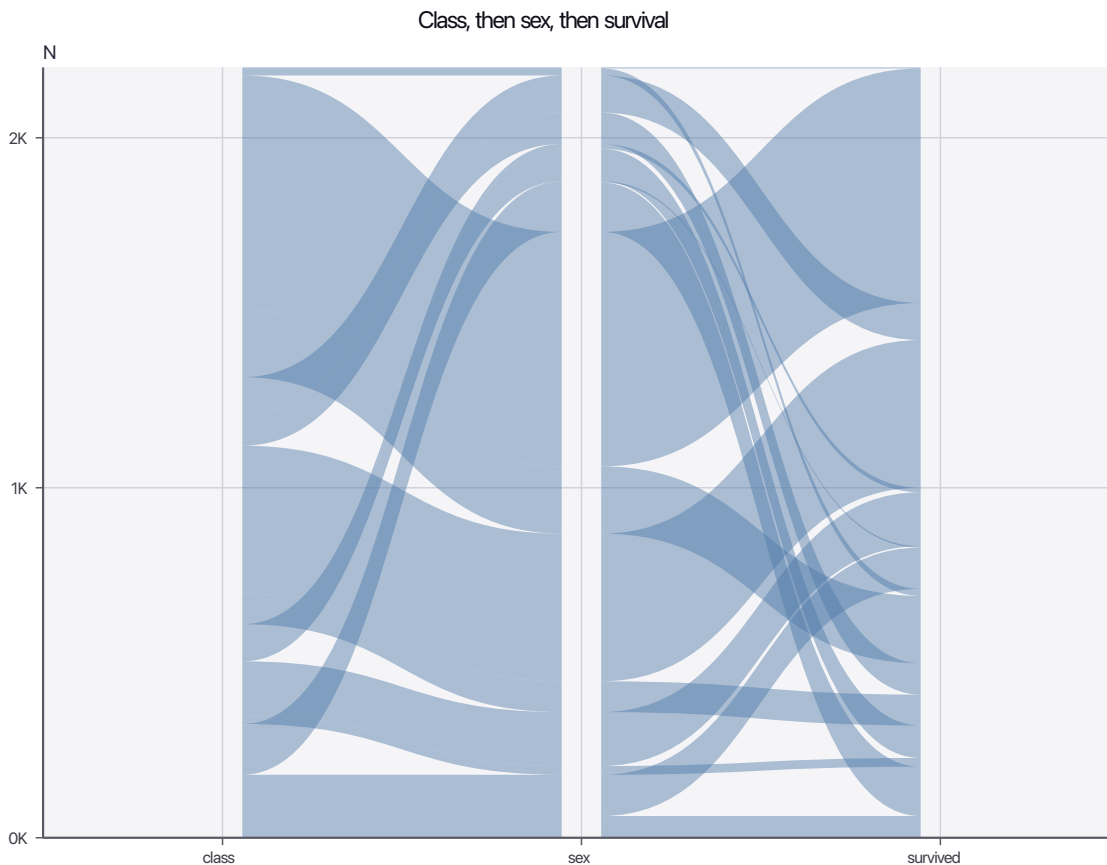
```
(data(titanic) + ribbon * flow(col["class"], col.sex, col.survived) +  
  ↪ y(col.n) +  
  title("Class, then sex, then survival"))
```

Julia

```
data(titanic) + ribbon * flow(:class, :sex, :survived) + y(:n) +  
  title("Class, then sex, then survival")
```

JavaScript

```
plot(data(titanic), layer(ribbon, flow(col.class, col.sex, col.survived)),  
  y(col.n), title("Class, then sex, then survival"))
```



“Given titanic: ribbons derived by flow through class, sex and survived, y is n.”

The three-stage picture answers a question the two-stage one could not: how many female passengers in first class survived? Follow any band and you follow one group of people. You see their class, whether they were male or female, and whether they survived.

30.3 Color comes from a stage

A band runs through every stage, so any stage can color it. Write `color()` after the `ribbon` layer and name the stage whose value gives each band its color:

R

```
data(titanic) + ribbon * flow(class, sex, survived) + y(n) + color(class) +
  title("The same paths, colored by class")
```

Python

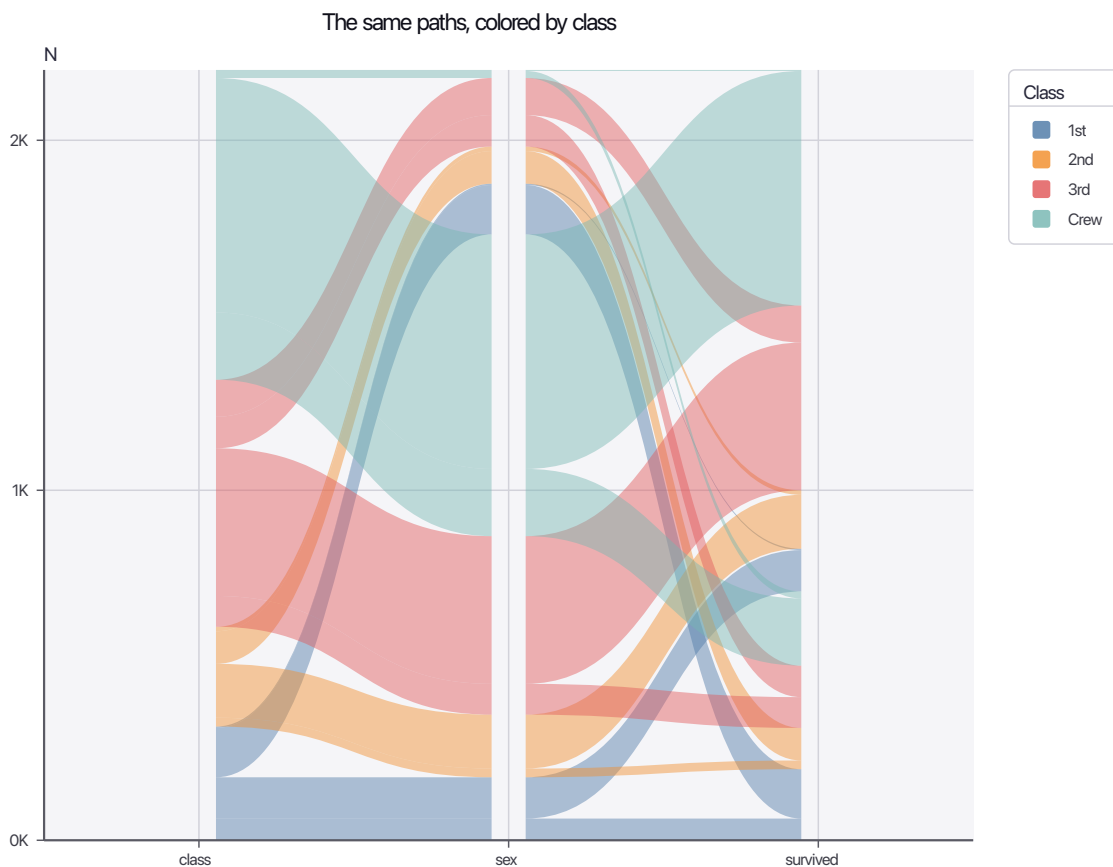
```
(data(titanic) + ribbon * flow(col["class"], col.sex, col.survived) +  
  ↳ y(col.n) + color(col["class"]) +  
    title("The same paths, colored by class"))
```

Julia

```
data(titanic) + ribbon * flow(:class, :sex, :survived) + y(:n) +  
  color(:class) + title("The same paths, colored by class")
```

JavaScript

```
plot(data(titanic), layer(ribbon, flow(col.class, col.sex, col.survived)),  
  y(col.n), color(col.class), title("The same paths, colored by class"))
```



“Given titanic: ribbons derived by flow through class, sex and survived, y is n, color by class.”

Coloring by `class` gives each band the color of the slot it starts from. You can then see which classes meet in each later slot. Coloring by `survived` would instead color every band by how its path ends. Only a stage can color a band, because a band is a whole path, and only a stage holds a value every path carries.

30.4 The slots and their names

Two more layers finish the diagram. A `zone` layer draws each stage’s slots, the stacked blocks the bands leave and arrive at. A `text` layer names them. The three layers read one computation, so they always agree about where everything sits:

R

```
data(titanic) + y(n) +  
  ribbon * flow(class, sex, survived) + color(class) +  
  zone * flow(class, sex, survived) + style(color = "gray") +  
  text * flow(class, sex, survived) + label(name) +  
  title("The full diagram: bands, slots, names")
```

Python

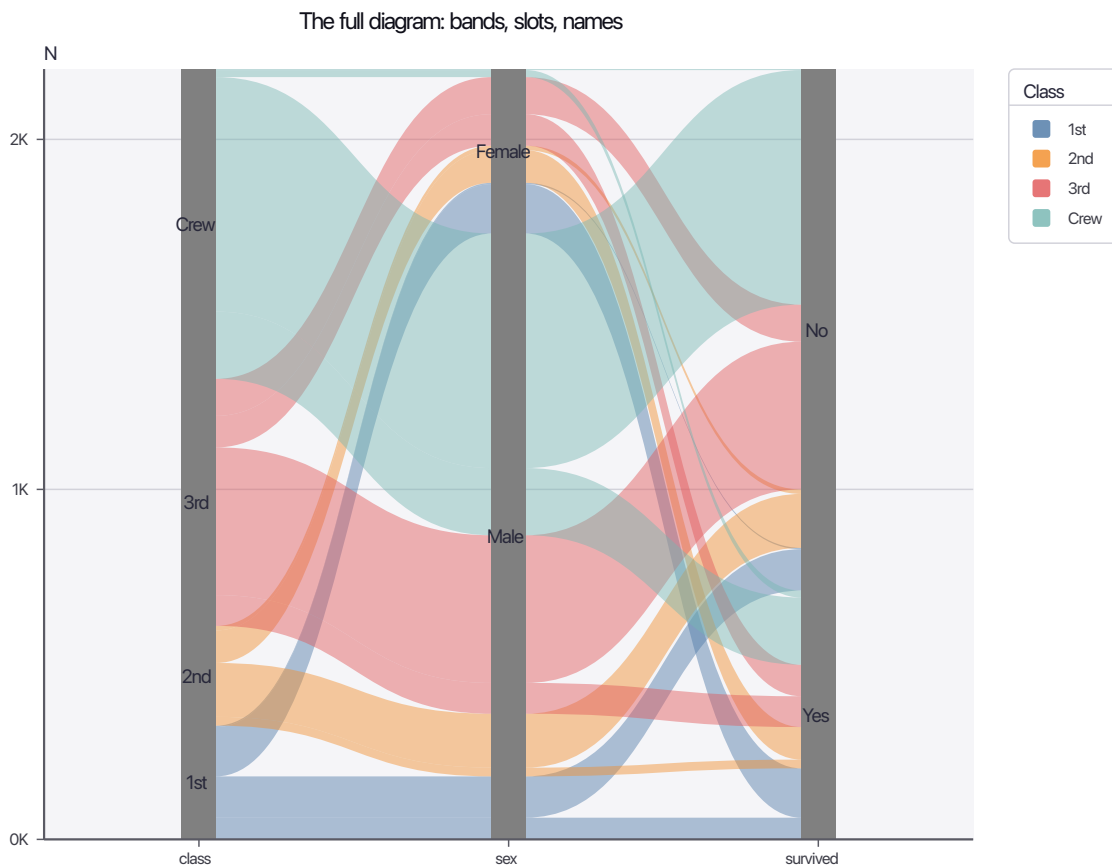
```
(data(titanic) + y(col.n) +  
  ribbon * flow(col["class"], col.sex, col.survived) + color(col["class"]) +  
  zone * flow(col["class"], col.sex, col.survived) + style(color = "gray") +  
  text * flow(col["class"], col.sex, col.survived) + label(col.name) +  
  title("The full diagram: bands, slots, names"))
```

Julia

```
data(titanic) + y(:n) + ribbon * flow(:class, :sex, :survived) +  
  color(:class) + zone * flow(:class, :sex, :survived) +  
  style(color = "gray") + text * flow(:class, :sex, :survived) +  
  label(:name) + title("The full diagram: bands, slots, names")
```

JavaScript

```
plot(data(titanic), y(col.n),
     layer(ribbon, flow(col.class, col.sex, col.survived)), color(col.class),
     layer(zone, flow(col.class, col.sex, col.survived)),
     style({ color: "gray" }),
     layer(text, flow(col.class, col.sex, col.survived)), label(col.name),
     title("The full diagram: bands, slots, names"))
```



“Given titanic: y is n , ribbons derived by flow through class, sex and survived, color by class, and also zones derived by flow through class, sex and survived, and text derived by flow through class, sex and survived, label by name.”

The slots take their color from `style()`, gray here so that only the bands carry color. The names come from `label(name)`: the transform writes each slot’s name into a column called `name`, exactly as `partition` does.

The count on `y` is optional. Bind nothing and every row counts as 1, so the diagram counts rows instead of summing a column. `partition` uses that default too, and `count` tallies rows the same way.

30.5 What flow refuses

A flow needs at least two stages, because movement needs a place to start and a place to finish:

```
data(titanic) + y(n) + ribbon * flow(class)
```

Error:

```
! gog: `flow()` needs at least two stage columns, in reading order -  
  ↳ `flow(class, sex, survived)` runs each row from its `class` to its  
  ↳ `survived`. One column has no between.
```

The stage axis is drawn from the atom's own columns, so a bound `x` has nothing left to set:

```
data(titanic) + x(n) + ribbon * flow(class, survived)
```

Error:

```
! gog: under `flow` the stage axis is drawn from the atom's own columns, so  
  ↳ `x(...)` has nothing left to say. Remove it - to reorder the stages,  
  ↳ reorder `flow(...)`'s arguments.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Only three marks can read what flow computes, and the refusal lists all three when another mark is used:

```
data(titanic) + y(n) + bar * flow(class, survived)
```

Error:

```
! gog: `flow` lays a path's magnitude through its stages - an interval at  
  ↳ each stage, and a band between adjacent ones - and `bar` has no reading  
  ↳ for that. Three marks do: `ribbon * flow(<a>, <b>)` draws the bands,  
  ↳ `zone * flow(<a>, <b>)` draws each stage's slots, and `text * flow(<a>,  
  ↳ <b>) + label(name)` names them. Bind `y(<amount>)` to weigh the paths, or  
  ↳ bind nothing and each row counts 1.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

And the column that colors a band must be one of the stages. A column outside the stages holds no value a whole path shares:

```
data(titanic) + y(n) + ribbon * flow(class, survived) + color(n)
```

Error:

```
! gog: `color(n)` under `flow` must name one of the atom's stages - a band is
  ↳ a path through them, and only a stage holds a value every band carries.
  ↳ Name one of `class`, `survived`, or set the paint for the whole layer
  ↳ with `style()`.
gog: `color(n)` maps a continuous (numeric) column, but `color` on `ribbon`
  ↳ needs a categorical (text) column.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

30.6 Where the limits are

Bending a flow into `polar()` is valid grammar. The bands bent around a circle are the chord diagram, which this engine does not draw:

```
data(titanic) + y(n) + ribbon * flow(class, survived) + polar()
```

Error:

```
! gog: `flow` in `polar()` is valid grammar - the bands bent round a rim are
  ↳ the chord diagram - but this engine does not draw it yet. Draw the flow
  ↳ flat, or wait for the feature.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The input has a limit too. A flow reads stage columns from one table, one row per path. Some tools accept a different shape: a list of nodes, which are these slots, and a list of links between them. That shape can also describe flows that merge, skip a stage, or return to a stage they already visited. `gog` does not read it. The stage-column shape settles three things by itself. Every path passes through every stage, the totals match at each one, and no path can return to a stage it already visited.

31 Cluster

Which things are most alike? A table of numbers holds the answer, but a reader cannot see it in the rows. The `cluster` transform computes it. It treats each category of an axis as a **leaf** of a tree. It joins the two closest leaves, then the next two, and keeps joining until one tree holds them all. The world calls the picture a dendrogram, and as with the pie there is no atom that spells the name.

This chapter builds the tree, then uses it to order a tile plot. `nutrients` holds one row per pair of food and nutrient: eight everyday foods, each measured on five nutrients:

Table 31.1: `nutrients`: first 5 of 40 rows

food	nutrient	amount
salmon	protein	25.4
salmon	fat	12.4
salmon	carbs	0.0
salmon	fiber	0.0
salmon	iron	0.5

31.1 The tree

Each leaf is described by a **profile**. `cluster(amount, over = nutrient)` says that one leaf's profile is its `amount` in every category of `nutrient`. The distance between two leaves is the straight-line distance between their two profiles. Two branches join at the average distance between their leaves.

R

```
data(nutrients) + path * cluster(amount, over = nutrient) + x(food) +  
  title("Eight foods, joined by what they are made of")
```

Python

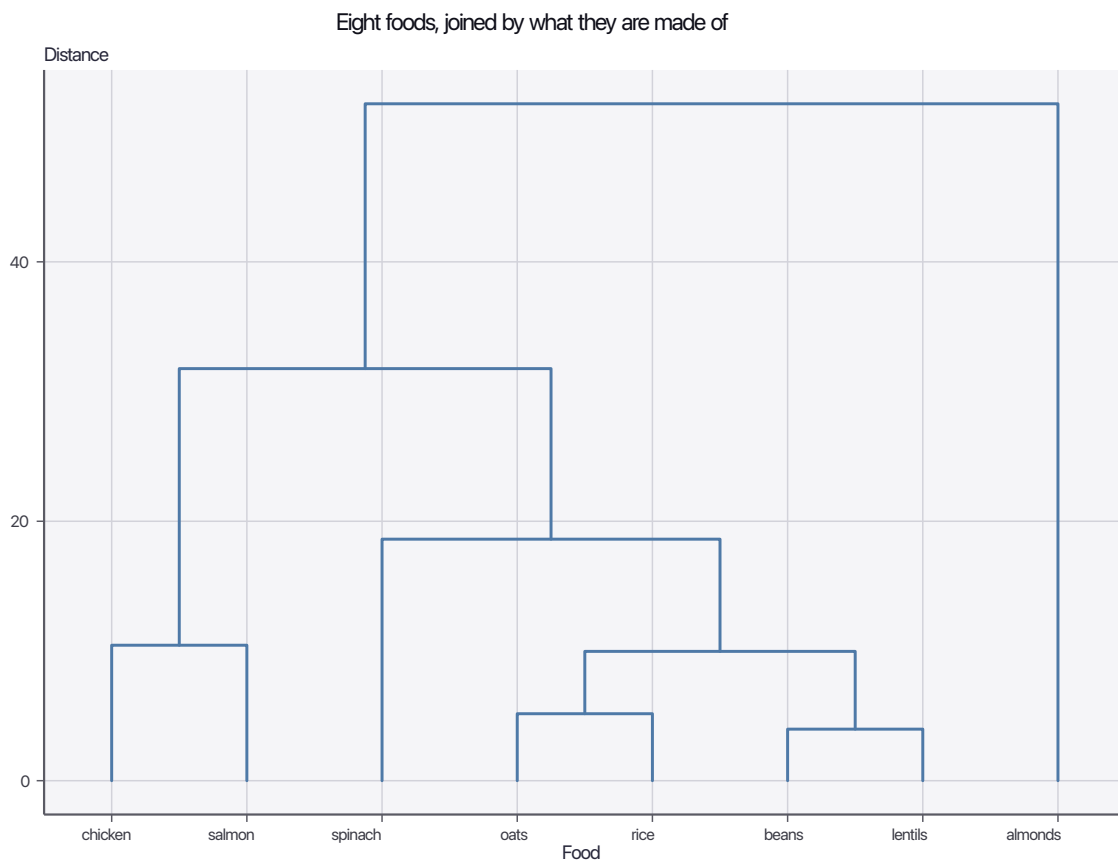
```
(data(nutrients) + path * cluster(col.amount, over=col.nutrient) +
  ↪ x(col.food) +
  title("Eight foods, joined by what they are made of"))
```

Julia

```
data(nutrients) + path * cluster(:amount, over = :nutrient) + x(:food) +
  title("Eight foods, joined by what they are made of")
```

JavaScript

```
plot(data(nutrients),
  layer(path, cluster(col.amount, { over: col.nutrient })), x(col.food),
  title("Eight foods, joined by what they are made of"))
```



“Given nutrients: paths derived by cluster on amount, over nutrient, x is food.”

`path` draws the tree. A tree is a set of corners joined in a fixed order, and `path` joins rows in the order the table gives them. The bound position is the leaf axis. The other position stays unbound: the tree computes the distance at which two branches join, and names that axis Distance. The place where two branches meet is a **join**, and its height says how different they are. Beans and lentils join lowest, rice and oats join just above them, and almonds join last of all.

Bind `y` instead of `x`, and the same tree lies on its side:

R

```
data(nutrients) + path * cluster(amount, over = nutrient) + y(food)
```

Python

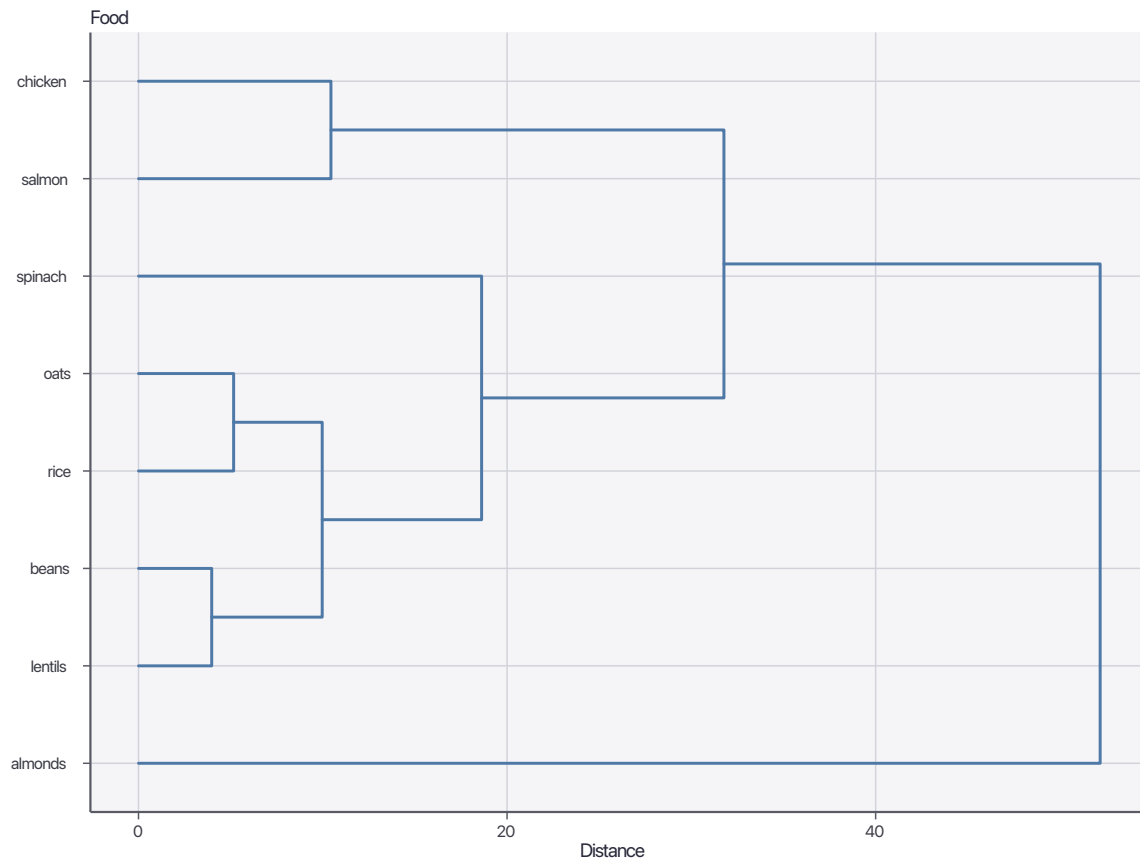
```
data(nutrients) + path * cluster(col.amount, over=col.nutrient) + y(col.food)
```

Julia

```
data(nutrients) + path * cluster(:amount, over = :nutrient) + y(:food)
```

JavaScript

```
plot(data(nutrients),  
  layer(path, cluster(col.amount, { over: col.nutrient })), y(col.food))
```



“Given nutrients: paths derived by cluster on amount, over nutrient, y is food.”

Three things about how `cluster` builds the tree cannot be changed. Distance is straight-line distance on the values as given, so a nutrient with large values changes the distance more than one with small values. Rescale where your data lives if that is not what you mean. Branches join at their average distance, so every join stands further from the leaves than the joins inside it. And two leaves with identical profiles join at zero, drawn as a join resting on the leaf axis.

31.2 The order

The tree can also be read as an order. A tile plot of the same table draws its slots in the order the rows arrived, and nothing chose that order:

R

```
data(nutrients) + zone + x(food) + y(nutrient) + color(amount) +  
  title("In the order the rows arrived")
```

Python

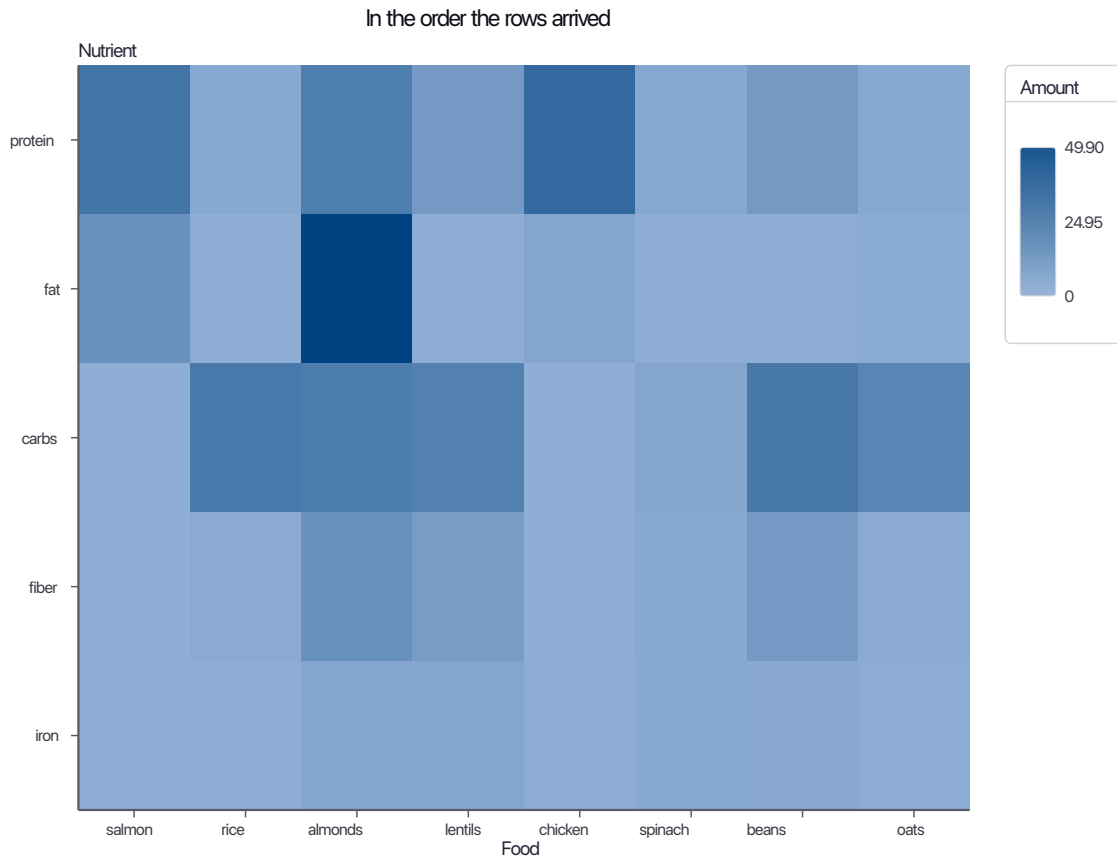
```
(data(nutrients) + zone + x(col.food) + y(col.nutrient) + color(col.amount) +  
  title("In the order the rows arrived"))
```

Julia

```
data(nutrients) + zone + x(:food) + y(:nutrient) + color(:amount) +  
  title("In the order the rows arrived")
```

JavaScript

```
plot(data(nutrients), zone, x(col.food), y(col.nutrient),  
  color(col.amount), title("In the order the rows arrived"))
```



“Given nutrients: zones, x is food, y is nutrient, color by amount.”

`zone * cluster` draws the same tiles with the leaf axis in the tree’s order. Chicken and salmon now stand together on the left, and the four foods that are mostly carbohydrate stand together between spinach and almonds:

R

```
data(nutrients) + zone * cluster(over = nutrient) +
  x(food) + y(nutrient) + color(amount) +
  title("In the tree's order")
```

Python

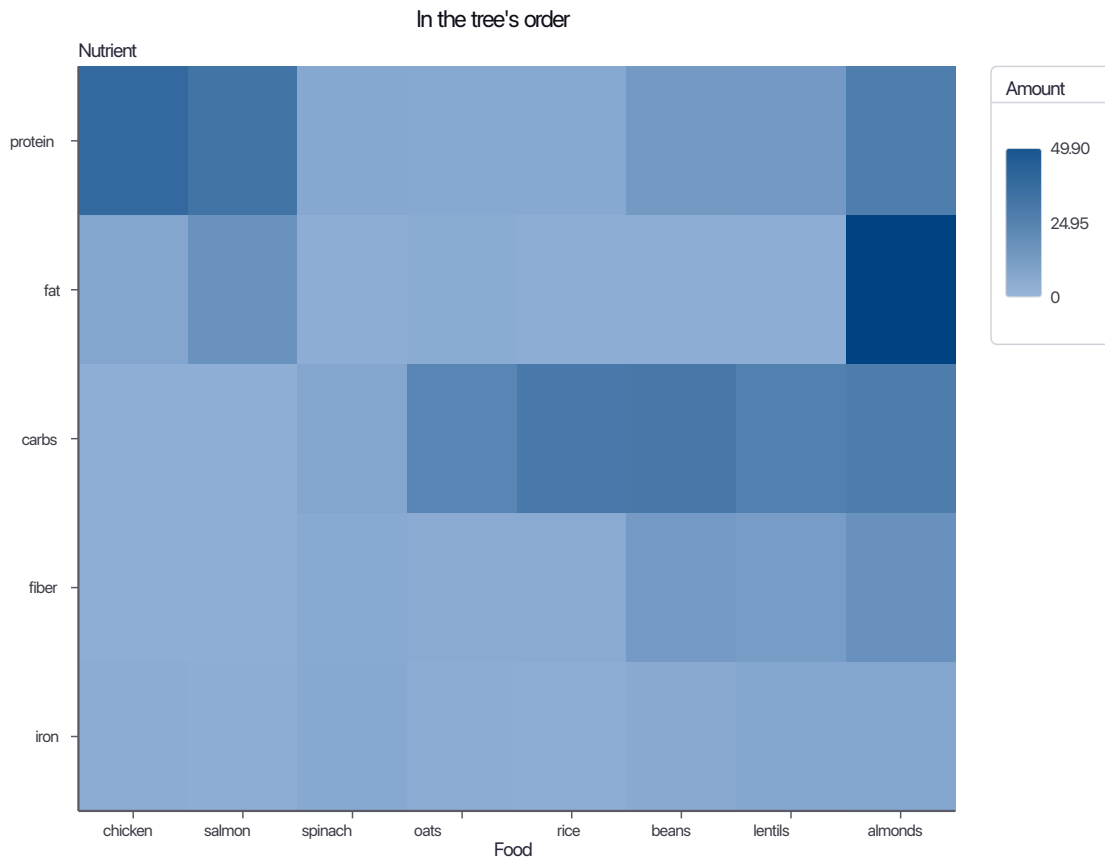
```
(data(nutrients) + zone * cluster(over=col.nutrient) +
  x(col.food) + y(col.nutrient) + color(col.amount) +
  title("In the tree's order"))
```

Julia

```
data(nutrients) + zone * cluster(over = :nutrient) + x(:food) +  
y(:nutrient) + color(:amount) + title("In the tree's order")
```

JavaScript

```
plot(data(nutrients), layer(zone, cluster({ over: col.nutrient })),  
x(col.food), y(col.nutrient), color(col.amount),  
title("In the tree's order"))
```



“Given nutrients: zones derived by cluster over nutrient, x is food, y is nutrient, color by amount.”

On a tile plot the values are already bound to `color`, so `cluster` need not name them again. `over` names the profile column, and the other categorical axis is the leaf axis the tree reorders.

31.3 The clustered heatmap

The full figure puts a tree on two sides of the tiles, so you can see where each order came from. The world calls it a clustered heatmap. It is three plots and two operators: a tree above, the tile plot, and a second tree on the right, beyond the legend. The alignment comes from one rule in [Composition](#): a shared categorical axis takes the order a clustered panel computed for it. The tile plot itself stays plain. Its column order arrives from the tree above, and its row order from the tree on the right.

R

```
(data(nutrients) + path * cluster(amount, over = nutrient) + x(food) +  
  theme(height = 150)) /  
  ((data(nutrients) + zone + x(food) + y(nutrient) + color(amount)) |  
    (data(nutrients) + path * cluster(amount, over = food) + y(nutrient) +  
      theme(width = 150)))
```

Python

```
((data(nutrients) + path * cluster(col.amount, over=col.nutrient) +  
  ↪ x(col.food) +  
    theme(height = 150)) /  
  ((data(nutrients) + zone + x(col.food) + y(col.nutrient) +  
  ↪ color(col.amount)) |  
    (data(nutrients) + path * cluster(col.amount, over=col.food) +  
  ↪ y(col.nutrient) +  
    theme(width = 150))))
```

Julia

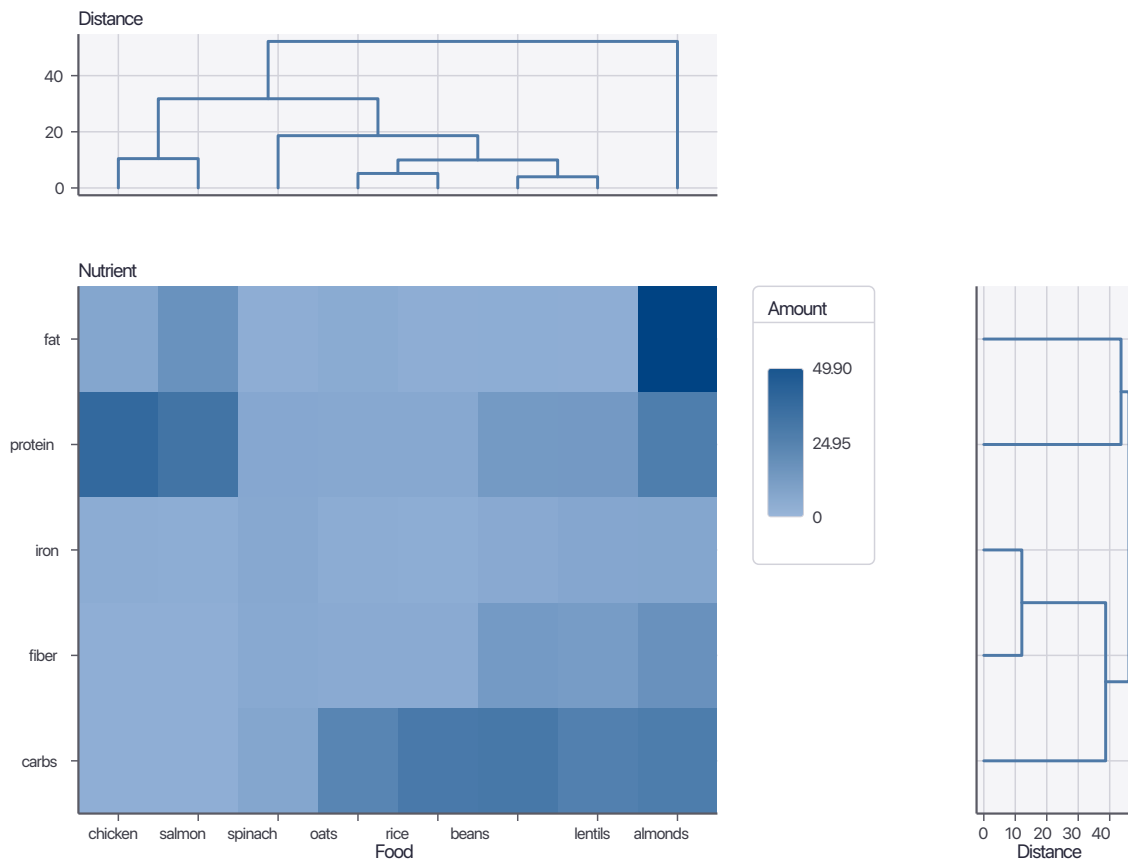
```
(data(nutrients) + path * cluster(:amount, over = :nutrient) + x(:food) +  
  theme(height = 150)) /  
  ((data(nutrients) + zone + x(:food) + y(:nutrient) + color(:amount)) |  
    (data(nutrients) + path * cluster(:amount, over = :food) +  
    y(:nutrient) + theme(width = 150)))
```

JavaScript


```

below(plot(data(nutrients),
  layer(path, cluster(col.amount, { over: col.nutrient })), x(col.food),
  theme({ height: 150 })),
beside(plot(data(nutrients), zone, x(col.food), y(col.nutrient),
  color(col.amount)),
plot(data(nutrients),
  layer(path, cluster(col.amount, { over: col.food })), y(col.nutrient),
  theme({ width: 150 }))))

```



“Given nutrients: paths derived by cluster on amount, over nutrient, x is food, above zones, x is food, y is nutrient, color by amount, beside paths derived by cluster on amount, over food, y is nutrient.”

Follow a column of tiles up into the tree, and the tree says which columns that food was joined with. The trees are not decoration: they decide which column stands next to which.

31.4 What cluster refuses

An axis holds one order. `order()` sorts by a column, `cluster` derives an order from the tree, and saying both is a contradiction:

```
data(nutrients) + path * cluster(amount, over = nutrient) + x(food) +  
  order(food)
```

Error:

```
! gog: `order(food)` and `cluster` both order the `food` axis - one sorts by  
  ↳ a column, the other derives the order from the tree, and an axis holds  
  ↳ one order. Drop one.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A profile holds one value for every category of the profile column, so that column must hold categories. A numeric column has none:

```
data(gm_all) + path * cluster(life, over = year) + x(continent)
```

Error:

```
! gog: `cluster(over = year)` names a numeric column, and a profile's entries  
  ↳ are indexed by names - every leaf holds one value per level of it. Make  
  ↳ it text where your data lives, or bin it there first.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A profile's entry is one value, so a pair that holds several rows is refused, and the message names the pair. In `gm_eras` every continent holds one row per country in each era. Aggregate to one row per pair first, where your data lives:

```
data(gm_eras) + path * cluster(life, over = era) + x(continent)
```

Error:

```
! gog: `continent` = Asia at `era` = 1957 holds 33 rows, and a profile's  
  ↳ entry is one value - the mean, the sum and the median are all defensible,  
  ↳ so gog will not choose. Aggregate to one row per pair where your data  
  ↳ lives.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

31.5 Where the limits are

A circular tree is valid grammar that this engine does not draw. Adding `polar()` says exactly that:

```
data(nutrients) + path * cluster(amount, over = nutrient) + x(food) + polar()
```

Error:

```
! gog: a circular cluster tree is valid grammar this engine does not draw yet
  ↳ - the treads would bend into arcs, and a path's segments draw straight.
  ↳ Drop `polar()`, or wait for the feature.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The grammar has no way to cut the tree into groups and color each group, and `color()` on the tree says so:

```
data(nutrients) + path * cluster(amount, over = nutrient) + x(food) +
  color(food)
```

Error:

```
! gog: `color(...)` under `cluster` has nothing to read - the tree's strokes
  ↳ are computed from every row, so no column is left to split them by.
  ↳ `style(color = )` paints the tree; coloring its branches by a cut is not
  ↳ designed.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

And one plot clusters one axis. The heatmap above orders both, because it composes two clustered plots, one for each axis.

32 Space

What do you do when a third measurement matters as much as the first two? Every plot so far has been drawn on a plane: a mark, an x , a y . The grammar has always held a third position channel, z . Binding it is enough to lift the mark off the page.

32.1 z : one more vowel

Here are three flower measurements. Two of them make an ordinary scatter, and the species already begin to separate, but *versicolor* and *virginica* still blur together along the diagonal:

R

```
data(iris_flowers) + point +  
  x(petal_length) + y(sepal_length) + color(species)
```

Python

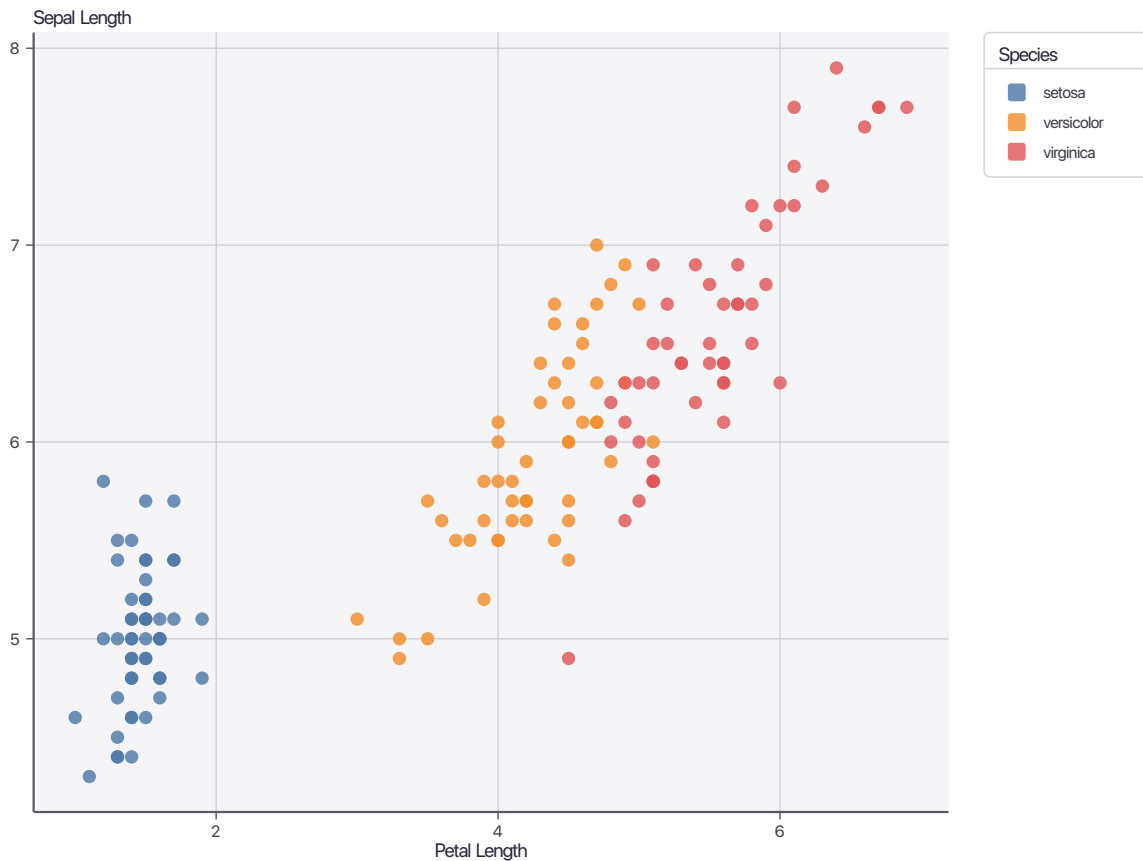
```
(data(iris_flowers) + point +  
  x(col.petal_length) + y(col.sepal_length) + color(col.species))
```

Julia

```
data(iris_flowers) + point + x(:petal_length) + y(:sepal_length) +  
  color(:species)
```

JavaScript

```
plot(data(iris_flowers), point, x(col.petal_length), y(col.sepal_length),  
  color(col.species))
```



“Given the iris flowers: points, x is petal length, y is sepal length, color by species.”

Now add the third measurement as **z**. Nothing else about the sentence changes (the same point, the same **x** and **y**, one more vowel), and the plot lifts into space:

R

```
data(iris_flowers) + point +
  x(petal_length) + y(sepal_length) + z(sepal_width) + color(species)
```

Python

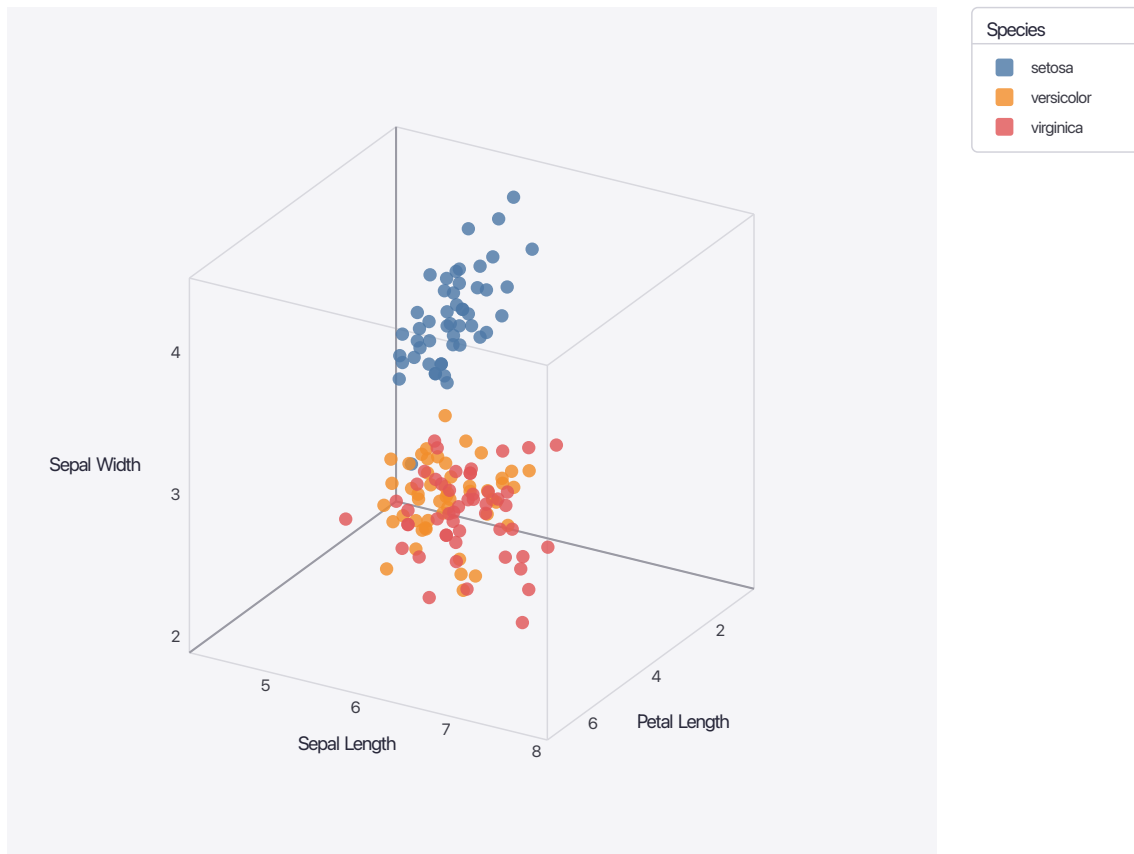
```
(data(iris_flowers) + point +
  x(col.petal_length) + y(col.sepal_length) + z(col.sepal_width) +
  ↪ color(col.species))
```

Julia

```
data(iris_flowers) + point + x(:petal_length) + y(:sepal_length) +  
  z(:sepal_width) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), point, x(col.petal_length), y(col.sepal_length),  
  z(col.sepal_width), color(col.species))
```



“Given the iris flowers: points, x is petal length, y is sepal length, z is sepal width, color by species.”

That is the whole idea. A 3-D scatter is not a different chart with a different name; it is **point** with its third position bound, the same way a horizontal bar is just **bar** with its categories on y. A flat plot is a spatial one with **z** held constant, so the same rules that place a mark on the page place it in the cube. Every flat plot you have read so far was the special case.

Drawing a 3-D plot on a flat page takes two steps, and neither one is new. First the engine turns the three coordinates of every point to match the angle you are looking from. Then it draws the turned points on the page, the way it draws any flat plot. Points nearer to you are drawn last, so they cover the points behind them. That overlap is what gives a flat picture its depth.

The faint box behind the cloud of points is a guide rather than part of the data. A bare cloud of points gives you no way to tell which direction is *x*, which is *y* and which is *z*. The box draws the edges of the space the points stand in, and each of the three axes puts its numbers along one of those edges.

On a screen you can turn this cube yourself. Drag it in any direction and watch *versicolor* and *virginica*. Some angles separate the two colors, and others leave them mixed together. When you find an angle that separates them, read the two numbers under the plot. They are called **turn** and **tilt**, and they describe the view you are looking at. The next section shows how to write those two numbers into the sentence, so the plot opens at the angle you found.

32.2 The viewing angle

A 3-D plot has to be seen from *somewhere*, and that angle is not part of the data. It is a property of the space the data sits in, the way a polar plot needs to know where its circle starts. [Polar](#) is the next chapter, and `polar(start = )` is the same kind of parameter. So it is not a channel (it maps no column) and not a mark. It is `space()`:

R

```
data(iris_flowers) + point +  
  x(petal_length) + y(sepal_length) + z(sepal_width) + color(species) +  
  space(turn = -50, tilt = 15)
```

Python

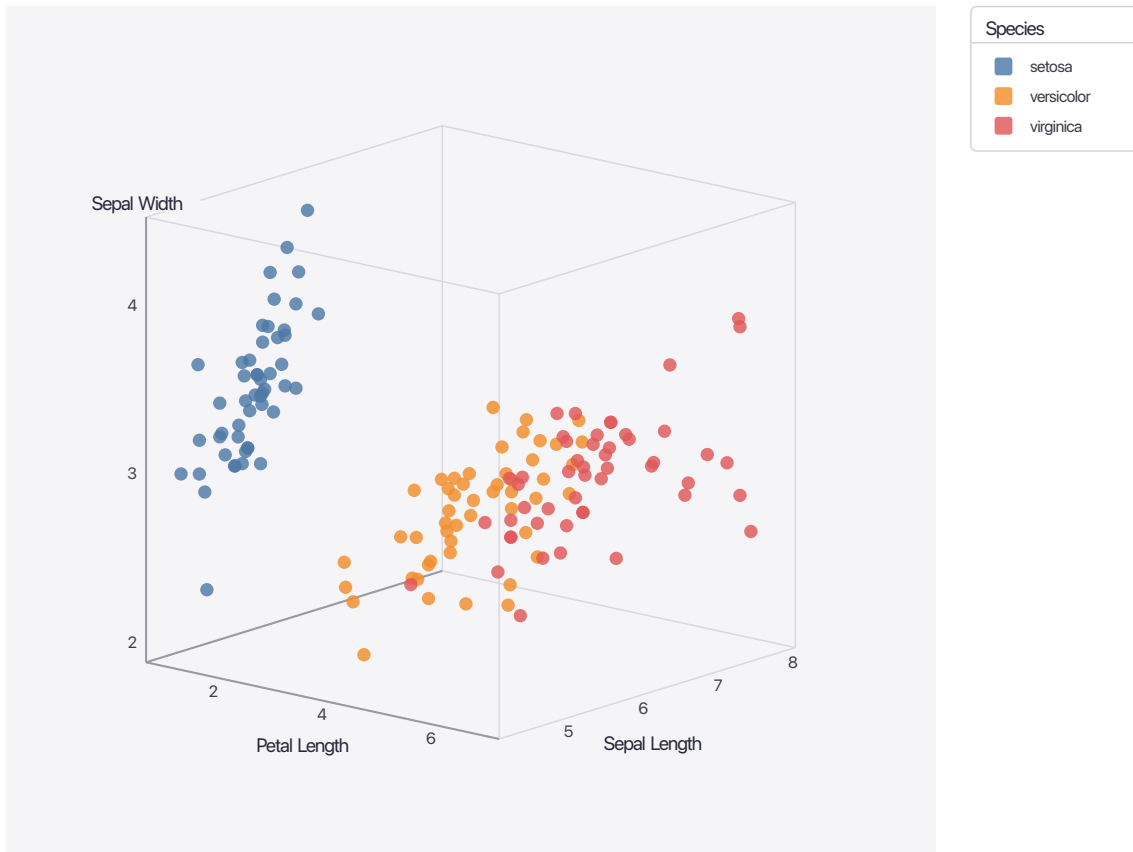
```
(data(iris_flowers) + point +  
  x(col.petal_length) + y(col.sepal_length) + z(col.sepal_width) +  
  ↪ color(col.species) +  
  space(turn = -50, tilt = 15))
```

Julia

```
data(iris_flowers) + point + x(:petal_length) + y(:sepal_length) +  
  z(:sepal_width) + color(:species) + space(turn = -50, tilt = 15)
```

JavaScript

```
plot(data(iris_flowers), point, x(col.petal_length), y(col.sepal_length),  
    z(col.sepal_width), color(col.species), space({ turn: -50, tilt: 15 })))
```



“Given the iris flowers: points, x is petal length, y is sepal length, z is sepal width, color by species, turned -50, tilted 15.”

Same data, same bindings: turned around and seen from lower down. The choice matters. From this angle the three species separate along *z* more cleanly than they did from the default view. A different angle again would mix them back together. Turning the plot changes *how you look*, never *what is drawn*, which is exactly why it belongs to the coordinate space and not to the grammar of the plot itself. Omit `space()` and the plot opens at a default angle; use `space()` when that default hides what you came to see.

Both angles are given in degrees, and both move your eye rather than the data.

turn says which side of the cube you look at. It moves your eye around the plot, the way you walk around an object to see another side of it. A positive turn moves you one way around, a negative turn moves you the other way, and a full circle is 360 degrees.

tilt says how high your eye is. At **tilt = 0** you are level with the floor and see the cube from the side. Raise it and you look down from further up. At **tilt = 90** you are directly above the cube, looking straight down. The floor fills the picture, and the **z** axis has nothing left to draw, so even its name disappears. A negative tilt puts your eye below the floor, looking up, as far as **tilt = -90** from directly underneath. Dragging stops at **tilt = 90** and **tilt = -90**, so the mouse reaches every angle you can write.

A plot with no **space()** opens at **turn = 30**, **tilt = 25**. That is far enough around and high enough up to show all three axes at once, which is why it is the default. Name one angle and the other keeps its default, so **space(tilt = 85)** looks down from high up without changing which side you stand on. Writing **space()** with nothing inside it takes both defaults, which is how the histogram sentences later in this chapter ask for a cube. (Astronomy calls these two angles *azimuth* and *elevation*, and the plain words say the same thing.)

32.3 Turning the plot yourself

Every plot drawn in a cube can be turned with the mouse. The cube follows your pointer. Drag right and the side facing you moves right; drag down and it tips down to show you the top. A line under each plot names the two angles of the view it is showing, and the numbers change as you drag. So you can find an angle by hand, then write it into the sentence as **space(turn = , tilt =)**.

A *reset* button sits beside those numbers. It returns the plot to the view it opened at, which is the angle the sentence asked for, or the default angle if the sentence named none. It is dimmed until you turn the plot, because until then there is nothing to undo.

Beside it sit the same boxes that every plot in this book has. They mean here what they mean everywhere, and the camera box saves the angle you turned the cube to, because that angle is already drawn into the picture.

The cube differs from a flat plot in two ways, and both follow from one fact: dragging turns the scene rather than moving the picture. So a zoomed cube cannot be dragged to one side, and the boxes do all the looking closer.

And the cube is the one plot with *two* boxes that undo something. There are two because they undo different things: the frame undoes the zoom, and *reset* undoes the angle. So zooming in never loses the angle you found, and straightening the angle never costs you your zoom. A flat plot has no angle to return, so it has the frame alone.

This changes nothing about the sentence. `space()` chooses where a plot opens, and the mouse takes you anywhere else from there. The data, the marks and the scales are the same at every angle. You are changing *how you look*, never *what is drawn*.

Dragging needs a web page to run in. On paper, and in the PDF edition, each plot is the still image its sentence asks for. So the angle is still worth naming, even though a reader can move it. The printed figure shows the view you chose, and a reader who can drag starts from it.

32.4 path: the route through the cube

So far the cube has held one mark. A scatter is a cloud: every dot stands alone and the table's order means nothing. When the order of the rows is what matters, the mark that says so is **path**, which joins rows in the order the table gives them. Give it three positions and the route runs through the cube. Here are two gliders circling one thermal (a rising column of warm air), sampled as they climb:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) + color(glider)
```

Python

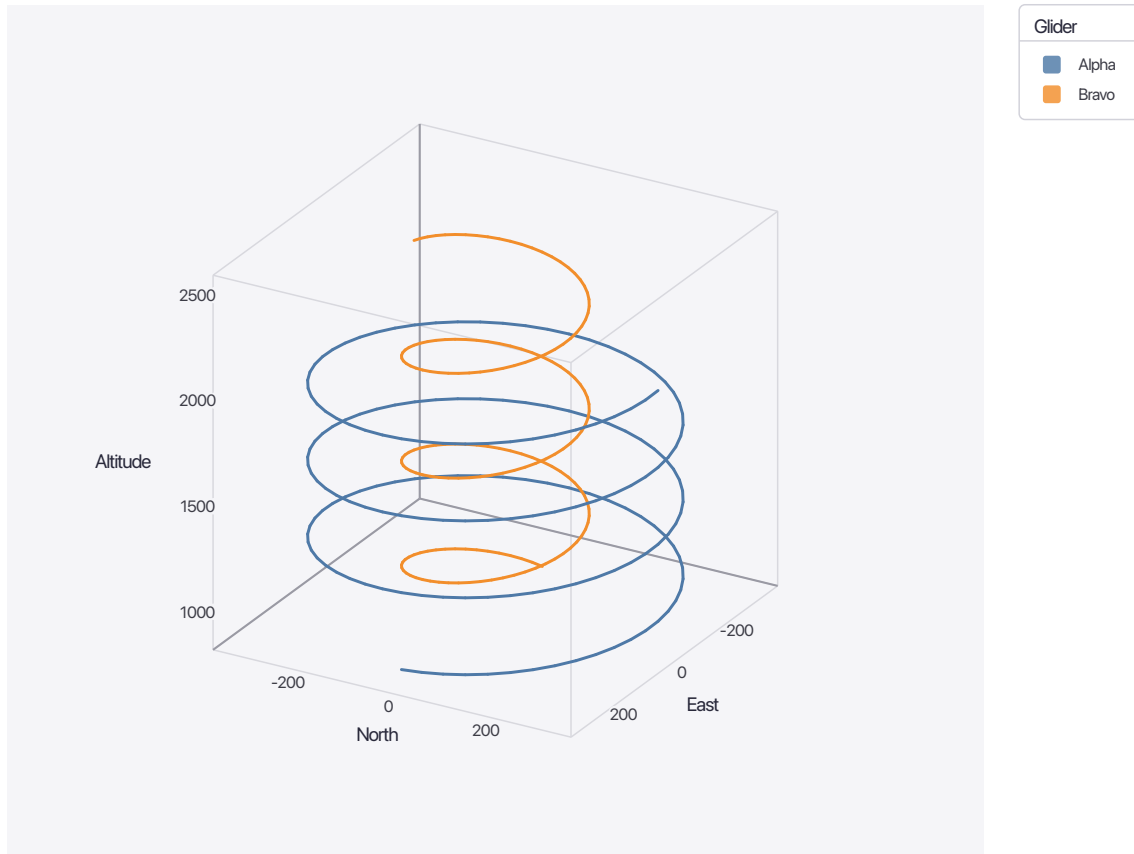
```
data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +  
    ↪ color(col.glider)
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +  
    color(:glider)
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),  
    color(col.glider))
```



“Given the thermals: paths, x is east, y is north, z is altitude, color by glider.”

Nothing in that sentence asks for a spiral. `path` was told to join the rows in order, and the gliders went round while they went up. The coil is what that looks like when it is drawn. The same three bindings would draw a straight line for an aircraft flying level, and a tangle for a tumbling one, because the route is whatever the rows did.

Look at where the two routes cross. The one in front covers the one behind, and which is which changes along the climb, so neither glider is simply “on top”. A stroke running through the cube has no single distance from the eye. Its far end and its near end are different distances. So the engine sorts the *segments* rather than the strokes, and sorts the segments of both routes together. Sorting whole strokes would put one glider entirely in front of the other: a picture of two coils that never meet, instead of two that thread through each other.

`path` is the only stroke that takes `z`, and the reason is the one that separates it from `line` on the flat page. An order belongs to no axis, so it means the same thing when the plane becomes a cube. A `line` sorts by `x` and draws one `y` for each, read left to right along a domain. The cube has no left to right. `x` is one of three equal positions, and at some angles it runs straight

into the page and becomes depth. A line in space would be sorted by an axis you cannot see, so the engine refuses it instead:

```
data(thermals) + line + x(east) + y(north) + z(altitude)
```

Error:

```
! gog: `line` reads a *domain* left to right - it sorts by `x` and draws one
  ↳ value for each - and a cube has no left to right: `x` is one of three
  ↳ equal positions, and at some viewing angles it runs into the page and
  ↳ becomes depth. A `line` in space would be sorted by an axis the reader
  ↳ cannot see, so this is refused rather than drawn. For a route through
  ↳ three dimensions use `path`, which is `line` with that sort removed:
  ↳ `path + x(<a>) + y(<b>) + z(altitude)`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

32.5 The 3-D histogram

Every cube so far bound a column to each of the three axes. The third axis can also carry something the data does not hold. The [heatmap](#) cuts a plane into cells and colors each one by how many rows fell in it. The cells already cover a floor, and the color represents a quantity. Bind that quantity to a height instead and the same counts are drawn as standing bars:

R

```
data(gm_all) + zone * bin(20) + x(gdp, scale = "log") + y(life)
```

Python

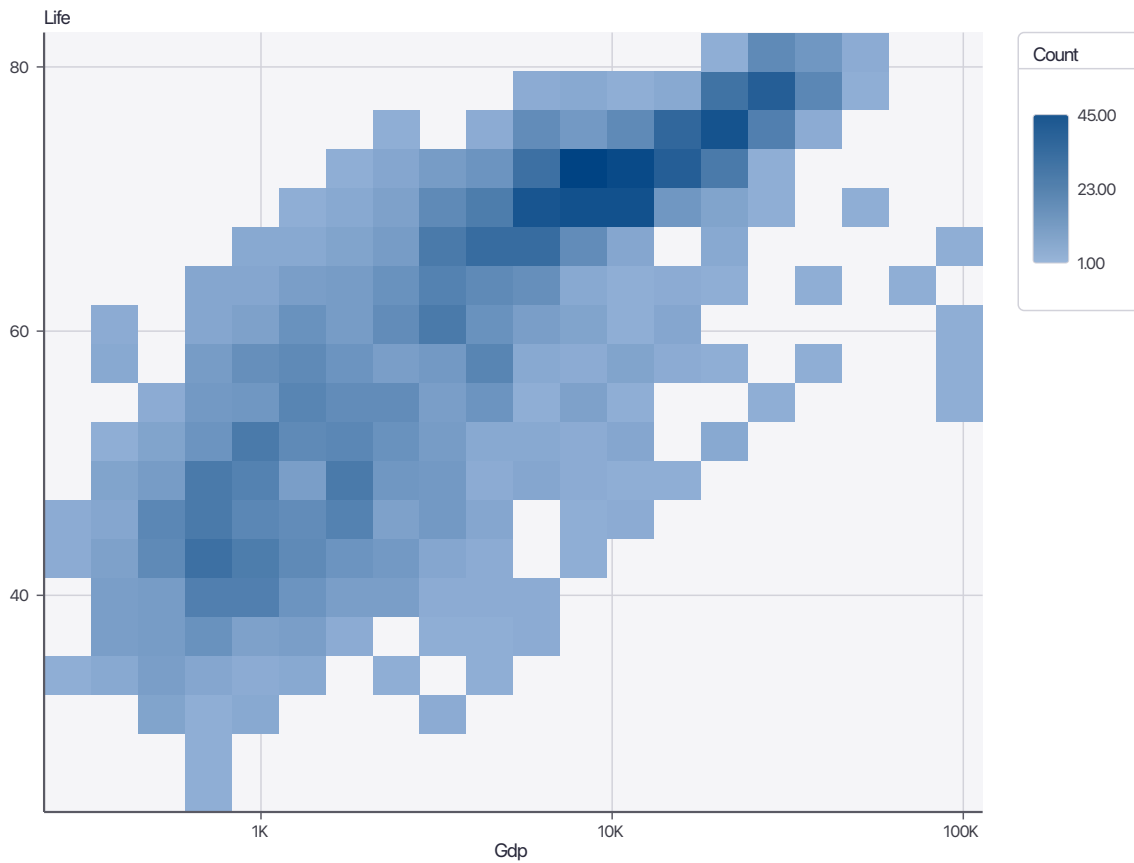
```
data(gm_all) + zone * bin(20) + x(col.gdp, scale = "log") + y(col.life)
```

Julia

```
data(gm_all) + zone * bin(20) + x(:gdp, scale = "log") + y(:life)
```

JavaScript

```
plot(data(gm_all), layer(zone, bin(20)), x(col.gdp, { scale: "log" }),
     y(col.life))
```



“Given all the gapminder years: zones derived by bin into 20, x is gdp on a log scale, y is life.”

R

```
data(gm_all) + bar * bin(20) + x(gdp, scale = "log") + y(life) + space() +
  style(color = "#2a9d8f")
```

Python

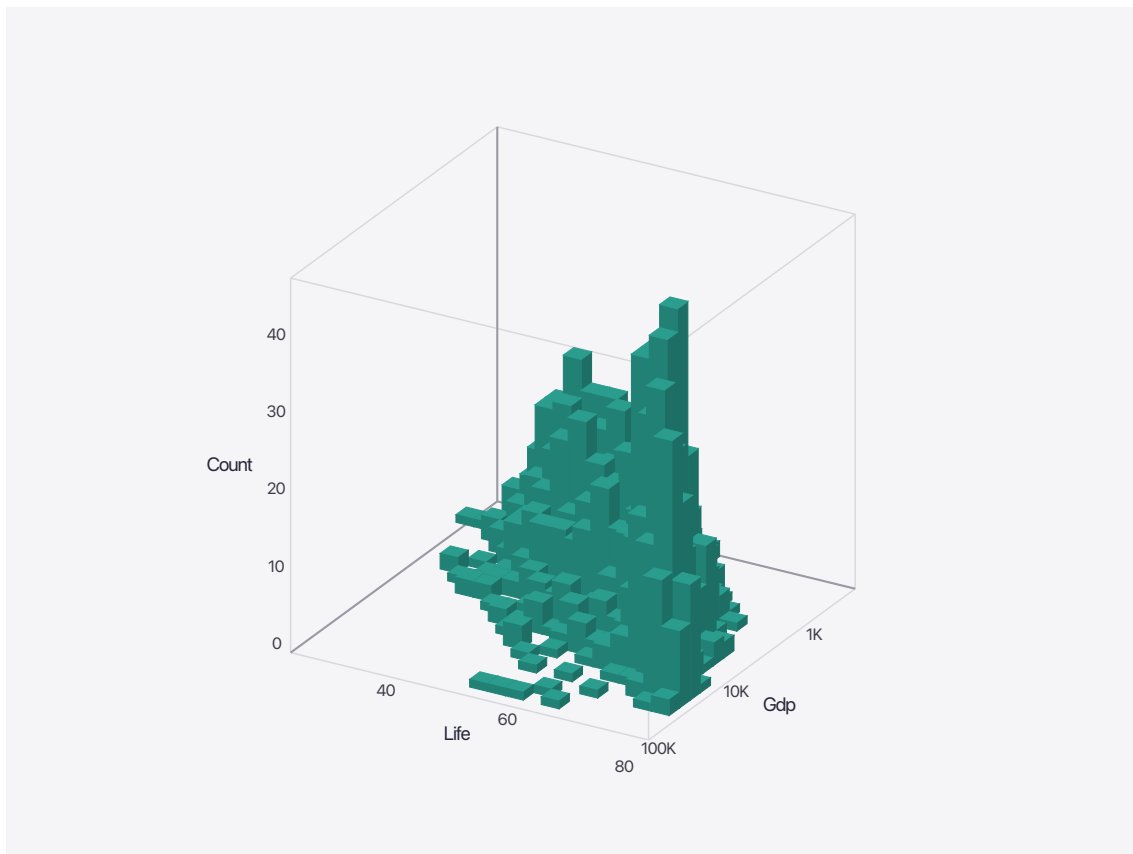
```
(data(gm_all) + bar * bin(20) + x(col.gdp, scale = "log") + y(col.life) +
  ↪ space() +
  style(color = "#2a9d8f"))
```

Julia

```
data(gm_all) + bar * bin(20) + x(:gdp, scale = "log") + y(:life) +
  space() + style(color = "#2a9d8f")
```

JavaScript

```
plot(data(gm_all), layer(bar, bin(20)), x(col.gdp, { scale: "log" }),
  y(col.life), space(), style({ color: "#2a9d8f" }))
```



“Given all the gapminder years: bars derived by bin into 20, x is gdp on a log scale, y is life, in the cube.”

The mark changed from **zone** to **bar**, and **space()** asks for the cube. The transform is the same **bin**, cutting the same two axes into the same cells, and the count it invents is the same count. What differs is the mark it was handed to, and therefore where the answer goes. A

zone measures nothing by length, so its count has to become a color. A **bar** *is* a length from a baseline, so its count becomes a height.

Nothing was added to the grammar to do that, and the rule that decided it was written for the flat page. How many axes a transform cuts is decided by the mark. **bar * bin** cuts one: a bar spends an axis on the domain and keeps the other to measure along. **zone * bin** cuts two, because a zone keeps none. A cube hands the bar a third axis. It can spend two on the floor and still have one to measure along, so the same sentence that cuts one axis on the page cuts two here. Read that way the 3-D histogram is not a feature. It is what the flat rule already said, applied to three axes.

Note also what you did *not* have to write. There is no **z()** in that sentence, for the same reason a flat histogram has no **y()**: the count is invented rather than bound, so naming it would be naming an output.

32.6 Transforms in the cube

That floor was cut from two continuous axes. A floor is made of cells, and the cells come from the two axes. It is the [zone](#) chapter's question one dimension up, with the same answers. A continuous axis has to be **cut** into cells before anything can be counted in them; a categorical axis arrives already cut, because a category *is* a cell. So which transform you want depends on which kind of axis you have. Two categorical axes arrive already cut:

R

```
data(gm_eras) + bar * count + x(continent) + y(era) + space() + style(color =  
  ↪ "#2a9d8f")
```

Python

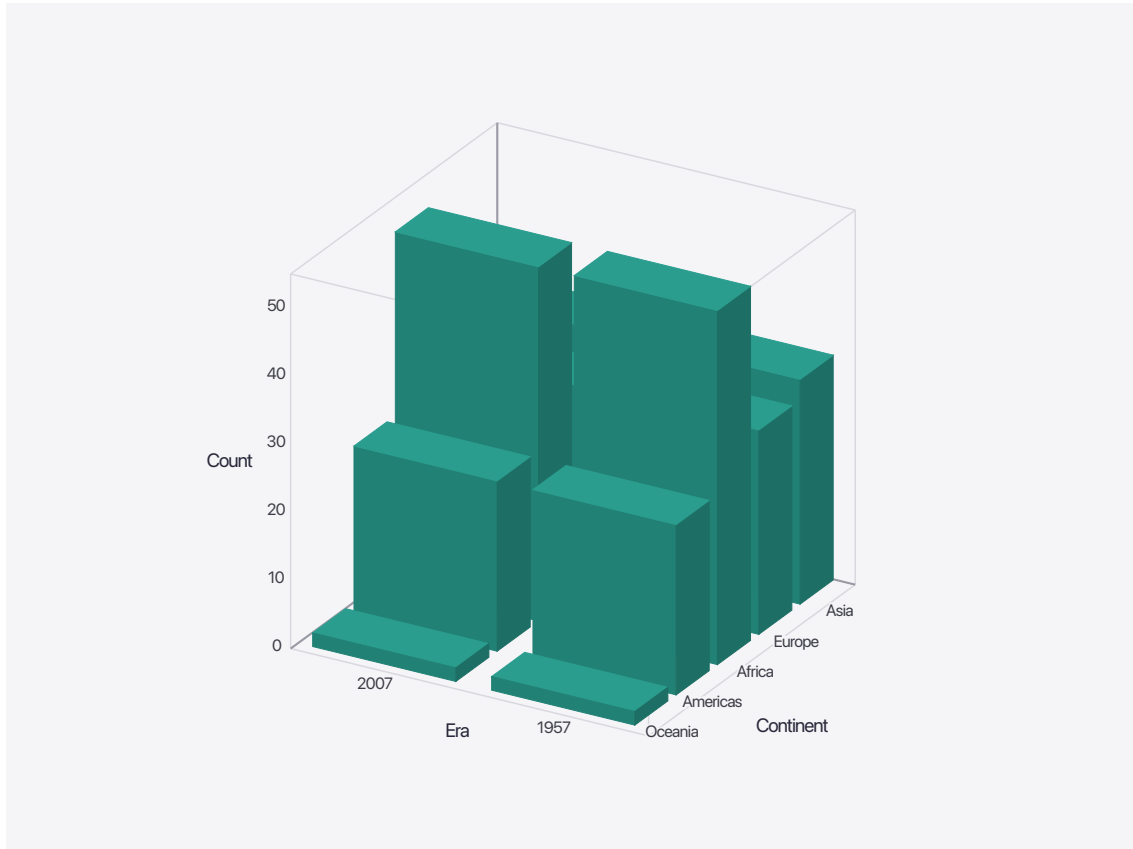
```
data(gm_eras) + bar * count + x(col.continent) + y(col.era) + space() +  
  ↪ style(color = "#2a9d8f")
```

Julia

```
data(gm_eras) + bar * count + x(:continent) + y(:era) + space() +  
  style(color = "#2a9d8f")
```

JavaScript

```
plot(data(gm_eras), layer(bar, count), x(col.continent), y(col.era),
     space(), style({ color: "#2a9d8f" })))
```



“Given the gapminder eras: bars derived by count, x is continent, y is era, in the cube.”

That one counts into slots on both axes: a table of counts, standing up. And a floor with one axis of each kind is a distribution per category, drawn as bars:

R

```
data(gm_all) + bar * bin(12) + x(life) + y(continent) + space() + style(color
  ↪ = "#2a9d8f")
```

Python

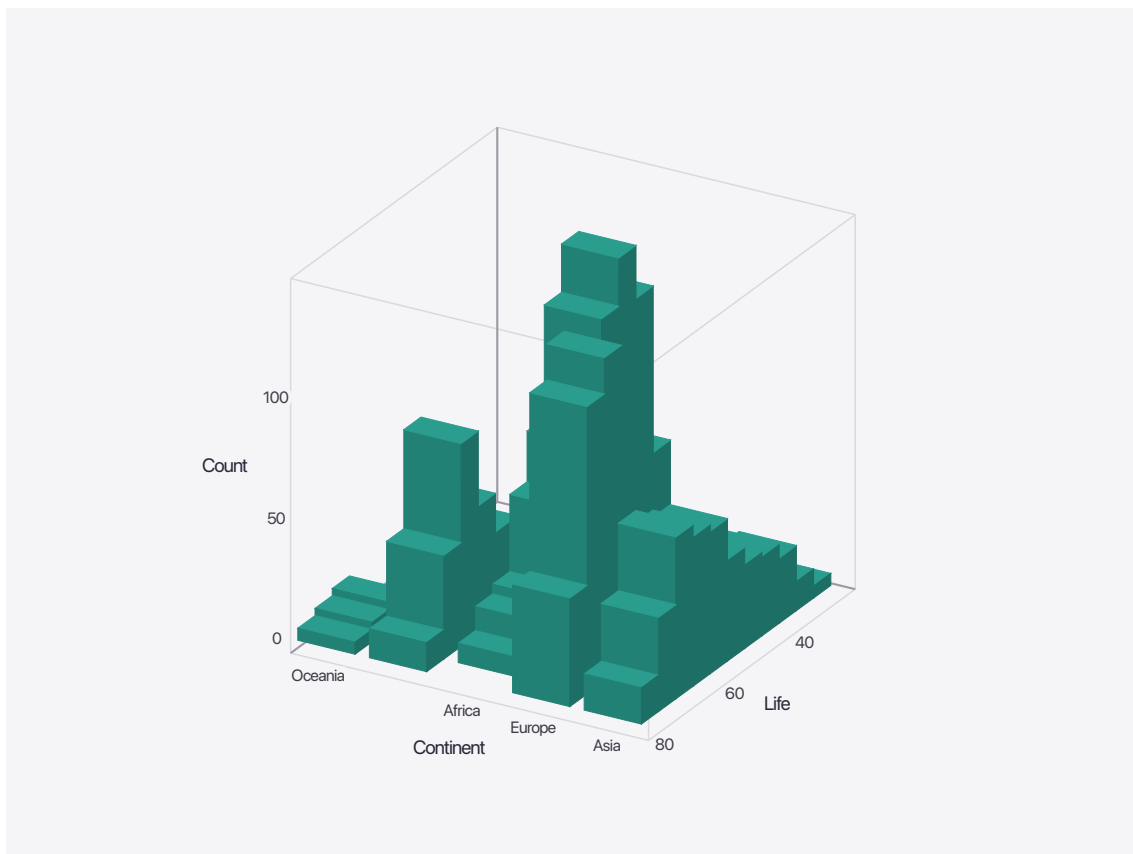

```
data(gm_all) + bar * bin(12) + x(col.life) + y(col.continent) + space() +
  ↪ style(color = "#2a9d8f")
```

Julia

```
data(gm_all) + bar * bin(12) + x(:life) + y(:continent) + space() +
  style(color = "#2a9d8f")
```

JavaScript

```
plot(data(gm_all), layer(bar, bin(12)), x(col.life), y(col.continent),
  space(), style({ color: "#2a9d8f" }))
```



Look at the two floors above, one after the other. On the binned axis the bars *touch*, and on the categorical axis they leave a gap. That is not a decision about how a 3-D plot should look;

it is the flat rule, per axis. A histogram's bins are adjacent intervals, so its bars fill them and meet. A category is one slot, and its neighbors are separate slots rather than adjacent intervals. The empty fifth of the slot is what says so. A floor cut on one axis and slotted on the other is contiguous one way and gapped the other, because each axis is answering only for itself.

Ask for a cut where there is nothing to cut and the engine names the other transform, exactly as it does on the plane:

```
data(gm_eras) + bar * bin + x(continent) + y(era) + space()
```

Error:

```
! gog: `bin` cuts a continuous axis into intervals, and `x(continent)` is
  ↪ categorical - a category is one slot, with no width to cut. To tally rows
  ↪ per category, `count` is the transform that does it: `bar * count`.
gog: `bin` cuts a continuous axis into intervals, and `y(era)` is categorical
  ↪ - a category is one slot, with no width to cut. To tally rows per
  ↪ category, `count` is the transform that does it: `bar * count`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Both of those transforms *invent* what they measure: you give `count` no column to reduce, because it counts rows. The other kind of statistic is the kind you point at a column: a mean, a sum, a median. Those work on a floor too, and the sentence names the column on the axis that measures:

R

```
data(gm_eras) + bar * mean + x(continent) + y(era) + z(life) + space() +
  style(color = "#2a9d8f")
```

Python

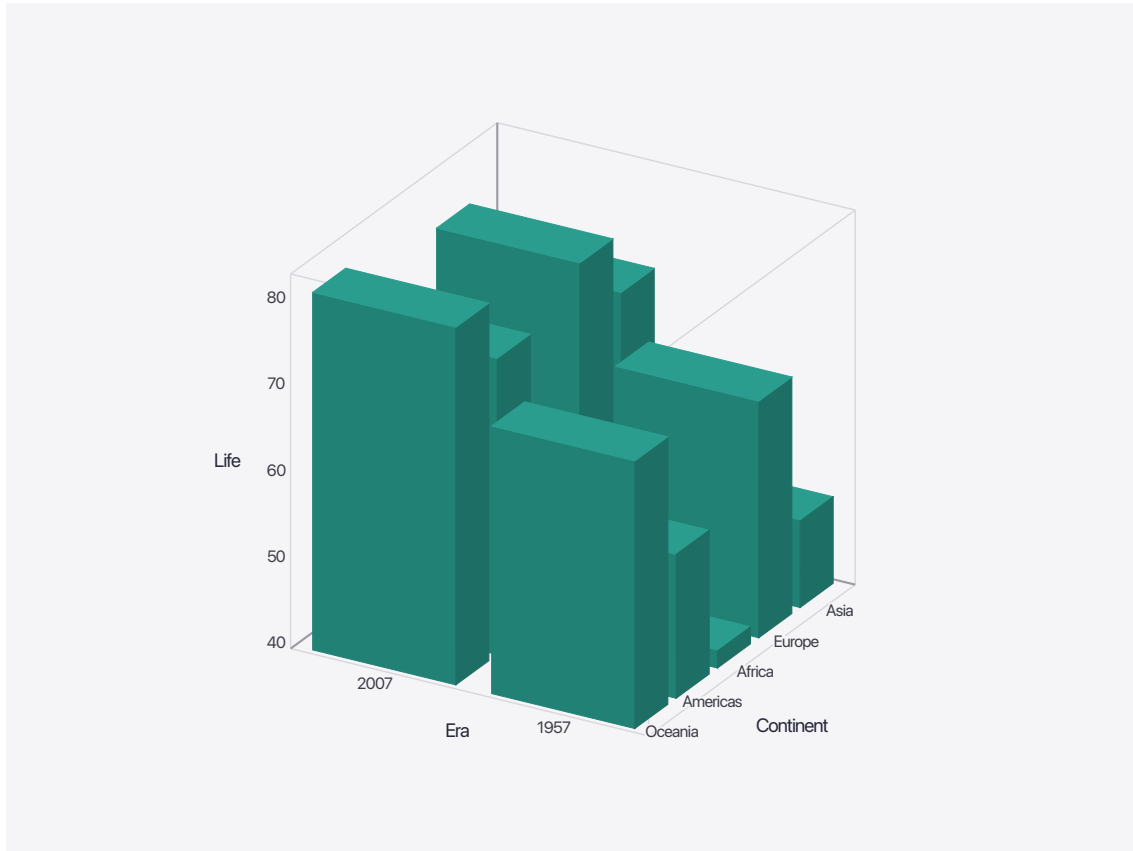
```
(data(gm_eras) + bar * mean + x(col.continent) + y(col.era) + z(col.life) +
  ↪ space() +
  style(color = "#2a9d8f"))
```

Julia

```
data(gm_eras) + bar * mean + x(:continent) + y(:era) + z(:life) +
  space() + style(color = "#2a9d8f")
```

JavaScript

```
plot(data(gm_eras), layer(bar, mean), x(col.continent), y(col.era),
     z(col.life), space(), style({ color: "#2a9d8f" })))
```



“Given the gapminder eras: bars derived by mean, x is continent, y is era, z is life.”

Read that beside the flat `bar * mean + x(continent) + y(life)`. The rule is the same in both. A summary reduces the column on the axis the mark measures along, and groups by the other positions. On the flat page, a bar measures with `y`, so it groups by `x` and averages `y`. In the cube it measures with `z`, so it groups by the *pair* and averages `z`. Nothing about `mean` changed; the mark simply has one more position to subtract from.

That is the same subtraction the histogram used, read a second time: the positions the space offers, minus the one the mark measures along. The first reading says how many axes `bin` cuts. This one says how many columns `mean` groups by. One rule, two questions, and the answer to both is read off the mark.

The two families differ in what you give them, not in what they can draw. `count` and `bin` invent a measurement, so they need no column, and color or height carries the answer. `mean`, `sum`, `median`, `max`, `min` and `quantile` reduce a column you name, so the sentence has to name it. Leave `z()` off and the engine says so rather than guessing.

One statistic really does stay on the plane, and it is the one that *fits* rather than reduces:

```
data(gm_eras) + bar * smooth + x(continent) + y(era) + z(life) + space()
```

Error:

```
! gog: `smooth` fits a curve of `y` against `x`, and `x(continent)` is
  ↳ categorical - a fit needs a number line to run along. For a typical value
  ↳ per category, `bar * mean` (or `point * mean`) says it directly.
gog: `smooth` fits a curve of `y` against `x`, and `y(era)` is categorical -
  ↳ a fit needs a number line to run along. For a typical value per category,
  ↳ `bar * mean` (or `point * mean`) says it directly.
gog: `bar * smooth` fits a curve of one column against another, and a curve
  ↳ needs a domain to run along - the cube's floor is a *pair* of positions
  ↳ with no left to right, which is why `line` and `area` cannot stand here
  ↳ either. Smooth on the plane, where there is a domain: `line * smooth +
  ↳ x(<a>) + y(<b>)` . To summarize a column within each pair of categories
  ↳ instead, the five reductions do read a floor: `bar * mean + x(<a>) +
  ↳ y(<b>) + z(<column>) + space()` .
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A smoother traces a curve of one column against another, and a curve needs a domain to run along. The cube's floor is a pair of positions with no left to right. That is why `line` and `area` cannot stand here either. It is the same reason, stated by a transform this time instead of by a mark.

32.7 interval and box in the cube

A bar is not the only mark that stands in a slot and measures along the other axis. A whisker spans a range and a box summarizes one. `bar`, `interval` and `box` ask the same question of the same pair of axes, so all three take the third dimension. Give the pair a cube and each stands on a *cell* of the floor, measuring along `z`:

R

```
data(gm_eras) + interval * range + x(continent) + y(era) + z(life) + space()
  ↳ +
  style(color = "#2a9d8f")
```

Python

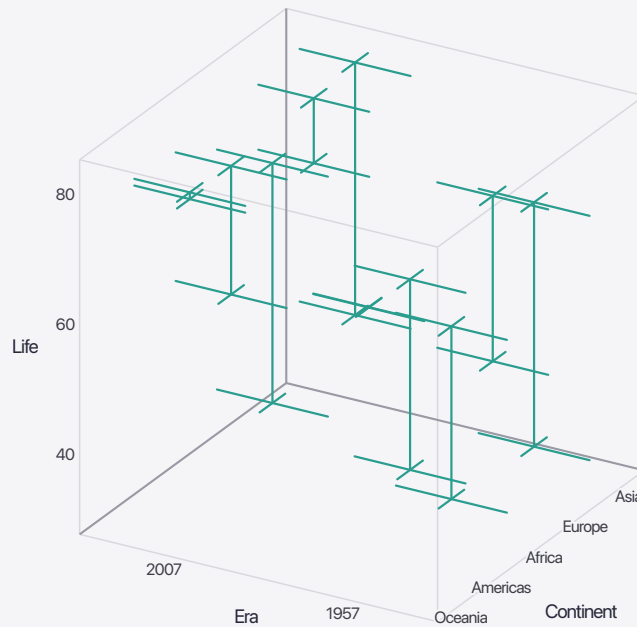
```
(data(gm_eras) + interval * range + x(col.continent) + y(col.era) +  
  ↪ z(col.life) + space() +  
  style(color = "#2a9d8f"))
```

Julia

```
data(gm_eras) + interval * range + x(:continent) + y(:era) + z(:life) +  
  space() + style(color = "#2a9d8f")
```

JavaScript

```
plot(data(gm_eras), layer(interval, range), x(col.continent), y(col.era),  
  z(col.life), space(), style({ color: "#2a9d8f" }))
```



“Given the gapminder eras: intervals derived by range, x is continent, y is era, z is life.”

Every cell’s whisker spans that cell’s own low and high. The caps are a **cross** rather than a single tick, and that is not decoration. On the flat page a cap runs across the slot in the one direction left over, and a cube leaves two. Picking one of them would be arbitrary, and a reader would look for a meaning that is not there. So the cap runs in both.

Ask for the interval that carries a center and `gog` draws the pointrange:

R

```
data(gm_eras) + interval * confidence + x(continent) + y(era) + z(life) +  
  space() + style(color = "#2a9d8f")
```

Python

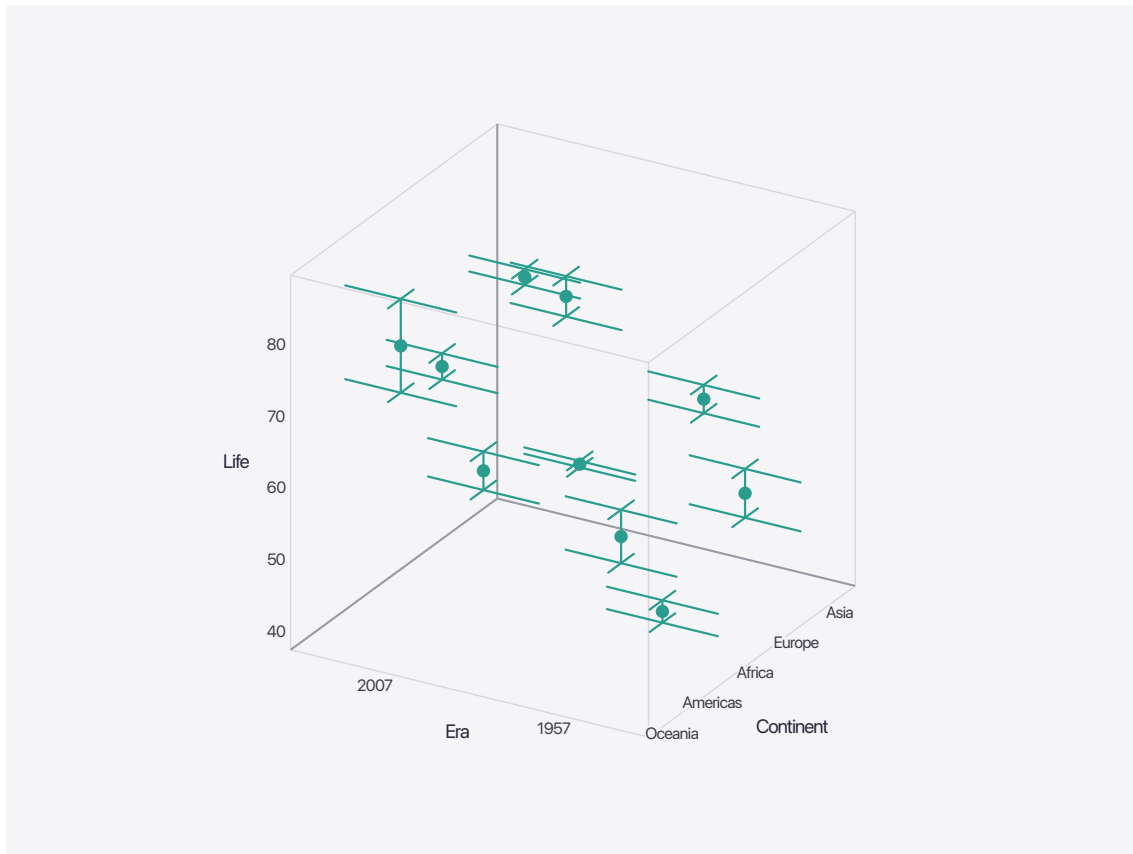
```
(data(gm_eras) + interval * confidence + x(col.continent) + y(col.era) +  
  ↪ z(col.life) +  
  space() + style(color = "#2a9d8f"))
```

Julia

```
data(gm_eras) + interval * confidence + x(:continent) + y(:era) +  
  z(:life) + space() + style(color = "#2a9d8f")
```

JavaScript

```
plot(data(gm_eras), layer(interval, confidence), x(col.continent),  
  y(col.era), z(col.life), space(), style({ color: "#2a9d8f" })))
```



“Given the gapminder eras: intervals derived by confidence, x is continent, y is era, z is life.”

And the five-number summary stands the same way, its box a solid between the quartiles with the whiskers reaching past it:

R

```
data(gm_eras) + box + x(continent) + y(era) + z(life) + space() + style(color
↪ = "#2a9d8f")
```

Python

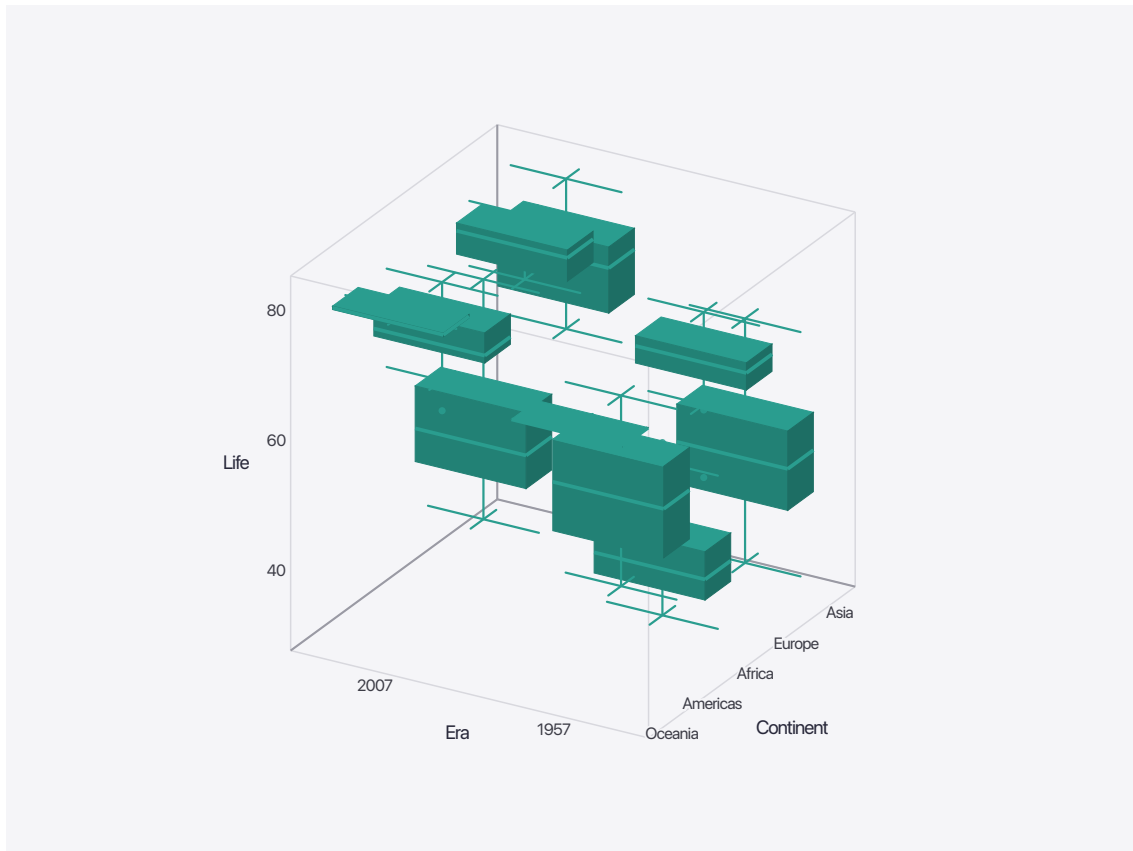
```
data(gm_eras) + box + x(col.continent) + y(col.era) + z(col.life) + space() +
↪ style(color = "#2a9d8f")
```

Julia

```
data(gm_eras) + box + x(:continent) + y(:era) + z(:life) + space() +
  style(color = "#2a9d8f")
```

JavaScript

```
plot(data(gm_eras), box, x(col.continent), y(col.era), z(col.life),
  space(), style({ color: "#2a9d8f" })))
```



“Given the gapminder eras: boxes, x is continent, y is era, z is life.”

Look at the median on each box. It is drawn only where the box’s own sides face you. The median is a plane *inside* an opaque solid, so the far half of it is behind the box. That is the back-face rule (a face turned away is not drawn), the same rule the solid’s own six faces follow. A group with a single observation is the exception. Its quartiles coincide, so there is no solid at all, and you see the whole median plane face-on. On the flat page, that same group draws as a line, which is what a box of zero height collapses to.

32.8 z_label(): naming the third axis

With the marks settled, the cube still needs its axes named. The cube's three axes take their names from the columns bound to them, and each can be overridden. Each of the three positions has its own label function, and `z_label()` is the third. Units are the usual reason to set all three at once:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) + color(glider) +  
  x_label("Meters east") + y_label("Meters north") + z_label("Altitude (m)")  
↪ +  
  title("Two gliders, one thermal")
```

Python

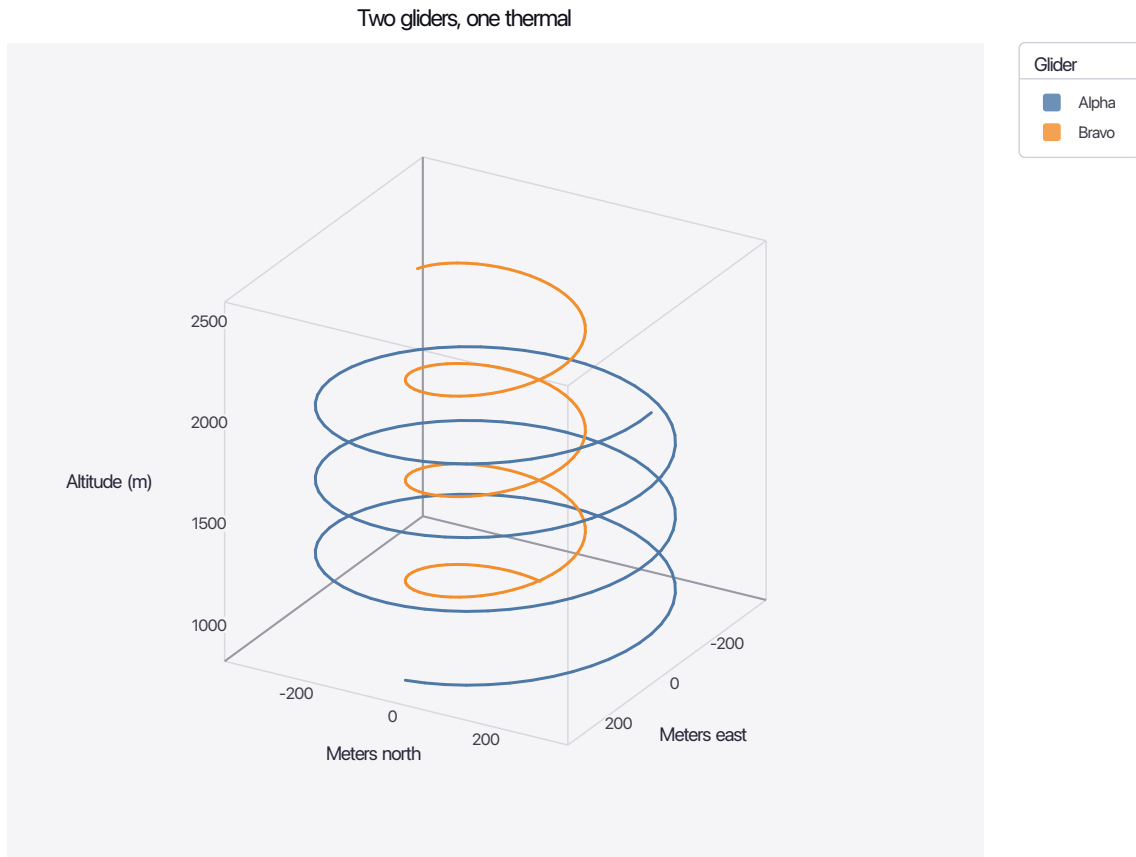
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +  
↪ color(col.glider) +  
  x_label("Meters east") + y_label("Meters north") + z_label("Altitude (m)")  
↪ +  
  title("Two gliders, one thermal"))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +  
  color(:glider) + x_label("Meters east") + y_label("Meters north") +  
  z_label("Altitude (m)") + title("Two gliders, one thermal")
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),  
  color(col.glider), x_label("Meters east"), y_label("Meters north"),  
  z_label("Altitude (m)"), title("Two gliders, one thermal"))
```



Ask for a `z_label()` on a plot with no `z` and the label has no axis to name, so `gog` says so and draws the plot anyway. It is guidance rather than a refusal, because nothing else about the plot is wrong.

[Path](#) is the chapter for the mark itself; the same two gliders appear in the [third-variable cookbook](#).

32.9 What the cube refuses

Everything above this point draws. What follows is the other half: what the cube will not do, and why. The box is drawn *behind* the marks, so the points are always in front of it. That is a fixed order rather than a calculation, and it means no point disappears behind the far face of the box. Its *labels* go on afterwards, over the marks. A guide is an annotation about the scene rather than a member of it, so no mark can cover the numbers that say what it measures.

Turning the plot is what makes a 3-D plot readable at all, and the mouse does it. Working out what hides what, piece by piece, is a separate and harder job. It costs far more work than turning does, and the renderer decides it rather than the grammar. So the box stays behind the marks however far you turn the scene.

A tilt past straight up or straight down is refused, and a turn past a full circle is not. The two angles read differently, which is the whole reason:

```
data(thermals) + point + x(east) + y(north) + z(altitude) + space(tilt = 95)
```

Error:

```
! gog: `space(tilt = 95)` is outside -90 to 90, which is where an elevation
↪ lives. At 90 the camera looks straight down at the floor and at -90
↪ straight up from underneath; past either the scene turns over. Use
↪ `space(tilt = 90)` for the nearest view, or `space(turn = )` to swing
↪ around the plot instead - a bearing wraps and an elevation does not.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

tilt is how high your eye is, and height runs out. At 90 you are directly above the cube and at -90 directly underneath, so there is nothing past either end to look from. Past either end the scene turns over, and an upside-down cube answers no question the sentence asked. Dragging stops at both ends, and writing an angle stops there too.

turn is which side you stand on, and you can keep walking. A full circle brings you back, so 390 is the same view as 30 and **gog** draws it without comment:

R

```
data(thermals) + point + x(east) + y(north) + z(altitude) + space(turn = 390)
```

Python

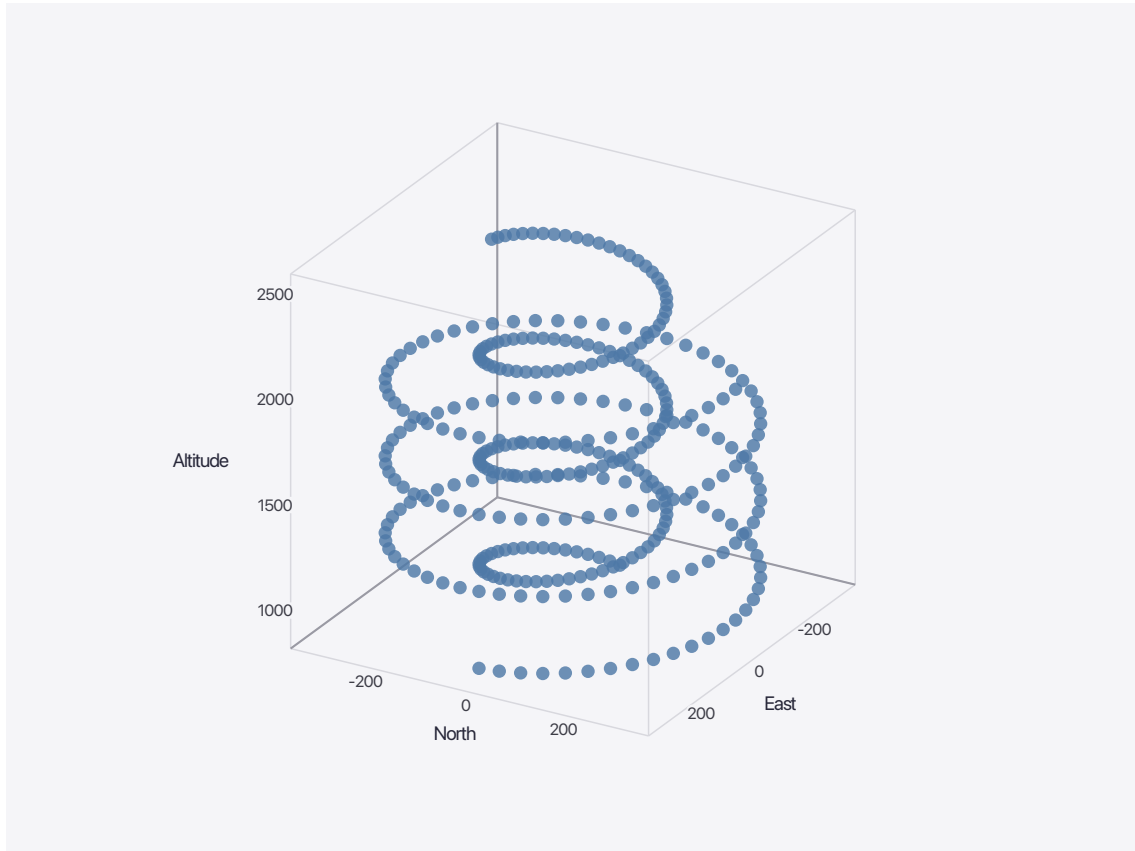
```
data(thermals) + point + x(col.east) + y(col.north) + z(col.altitude) +
↪ space(turn = 390)
```

Julia

```
data(thermals) + point + x(:east) + y(:north) + z(:altitude) +
space(turn = 390)
```

JavaScript

```
plot(data(thermals), point, x(col.east), y(col.north), z(col.altitude),
     space({ turn: 390 })))
```



The same picture as `space(turn = 30)`, to the byte. Two angles, two rules, and the difference is in what they measure rather than in a preference.

A mark the cube does not draw has three possible reasons, and the engine says which one you have.

Some are **decided**. `line`, `step`, `area` and `ribbon` read a *domain* left to right. They sort by `x` and draw one value for each, which is what makes a line a function rather than a route. A cube has no left to right, for the reason `path` gave above, so a line in space would be sorted by an axis you cannot see:

```
data(thermals) + line + x(east) + y(north) + z(altitude)
```

Error:

```
! gog: `line` reads a *domain* left to right - it sorts by `x` and draws one
↳ value for each - and a cube has no left to right: `x` is one of three
↳ equal positions, and at some viewing angles it runs into the page and
↳ becomes depth. A `line` in space would be sorted by an axis the reader
↳ cannot see, so this is refused rather than drawn. For a route through
↳ three dimensions use `path`, which is `line` with that sort removed:
↳ `path + x(<a>) + y(<b>) + z(altitude)`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

That is a rule rather than a gap, and the refusal says so: the route through three dimensions is [path](#), which is [line](#) with exactly that sort removed. Two marks that differ by one rule on the page turn out to differ by a whole dimension in the cube.

32.10 What the cube blocks

A mark can also be **blocked**, which is a different reason. A rule marks a value on one axis and spans the ones it does not name: in a cube that is *two* axes, so a rule here is a **plane**:

```
data(thermals) + rule + x(east) + z(altitude) + space()
```

Error:

```
! gog: `rule` marks a value on one axis and spans the ones it does not name -
↳ in a cube that is *two* axes, so a rule here is a plane. Marks in
↳ space are sorted by their footprint, and a plane's footprint is the whole
↳ floor, so it could only be drawn wholly in front of or wholly behind the
↳ data when its job is to cut through it. Drawing it would mean working
↳ out, piece by piece, what hides what, and the engine cannot do that yet.
↳ Draw it flat, or mark the threshold on a floor axis with `bar`/`point` at
↳ `z(altitude)`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Marks in space are painted far-to-near and sorted by the area they cover, and a plane covers the whole floor. It could only be drawn wholly in front of or wholly behind the data, but cutting *through* the data is the whole job of a reference plane. A **zone** is the same problem one dimension wider. Neither is drawn, because the engine places each mark by its whole footprint rather than piece by piece.

The *z* axis is linear, so a log depth is refused and the message names what to do instead. In every case the engine tells you what it will not do, because a plot that silently drops a dimension is the one failure this project refuses above all others.

32.11 Faceting: one cube per panel

A facet splits the rows into panels, and a panel is a rectangle. The cube is drawn inside a rectangle. So a faceted 3-D plot is one cube per panel, each projected into its own panel:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + z(population) +  
  color(continent) | facet(continent)
```

Python

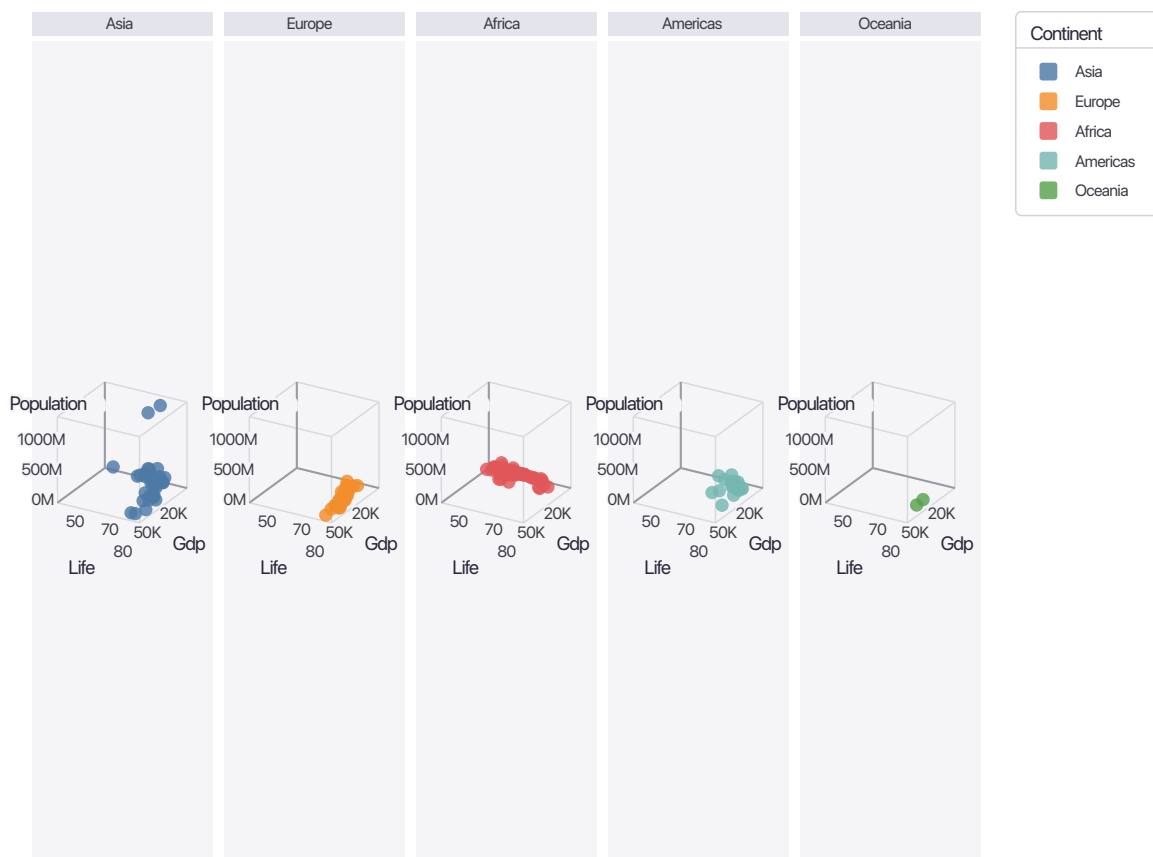
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) + z(col.population)  
  ↪ +  
  color(col.continent) | facet(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + z(:population) +  
  color(:continent) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  z(col.population), color(col.continent), across(col.continent))
```



“Given gapminder 2007: points, x is gdp, y is life, z is population, color by continent, split into panel columns by continent.”

The scales are shared, exactly as they are on the flat page, so the five cubes are the same cube five times and a height in one is a height in another. Oceania holds two countries, so its cube is nearly empty, and that is the correct reading. The shared scale is what makes the difference visible.

The same split, on a route rather than a cloud:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) + color(glider) |
  facet(glider)
```

Python

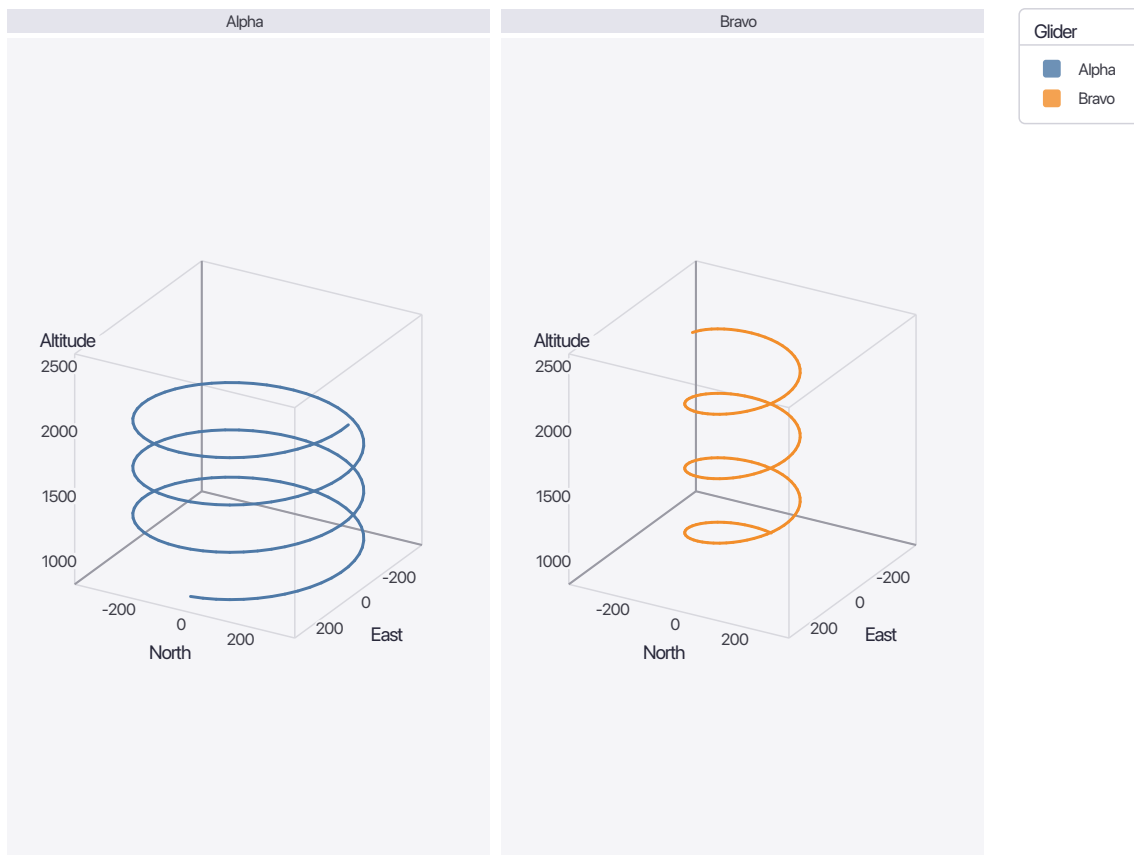
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +
  ↪ color(col.glider) |
  facet(col.glider))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +
  color(:glider) | facet(:glider)
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),
  color(col.glider), across(col.glider))
```



The guides are the one thing that needs explaining. Each cube draws its own tick numbers and its own three axis names. A faceted flat plot writes each axis once, along an edge of the page. That is a rule rather than an oversight. A guide drawn on the panel's boundary is drawn once, because the panels share that boundary. A guide drawn inside the panel appears in every panel, because there is no boundary to share it on. A faceted polar plot already works this way, with a category ring inside every panel. A cube's three axes are edges of the cube, and the cube sits inside the panel. A shared column of numbers beside the page would have nothing to point at. The tick reading -200 lands in a different place in every panel, because each panel projects from its own rectangle.

A second kind of split lays the panels out in time instead of across the page. [play](#) is that chapter, and it is where the two are combined.

Which edges carry the numbers is chosen per viewing angle, and the choice follows a rule. The cube offers four parallel edges per axis. The two domains take the floor edges nearest you, the pair that makes the V at the bottom of the picture, and the measurement takes whichever vertical edge sits on the cube's outline. All three lie outside the data, so nothing drawn inside the cube can be in front of them. Look at the left-hand edge of the tilted-up sheet in [Surface](#): you can still read heights off it while the mesh fills the rest of the picture. The labels are then measured against each other before any of them is drawn. So two axes cannot print their numbers in the same place where their edges meet, and an axis name cannot land on its own last tick.

One case is left: an axis that projects very short. Tilt the eye up and the measure axis is drawn shorter, until its numbers have nowhere to sit. The same summary at two tilts, and the second one is seen from almost straight above:

R

```
((data(gm_eras) + bar * mean + x(continent) + y(era) + z(life) + space() +
  style(color = "#2a9d8f") +
  z_label("Mean life expectancy") + title("The three-quarter view")) |
  (data(gm_eras) + bar * mean + x(continent) + y(era) + z(life) +
    space(tilt = 85) + style(color = "#2a9d8f") +
    z_label("Mean life expectancy") + title("Tilted up, with fewer
↪ numbers")) +
  theme(height = 430)
```

Python

```
((data(gm_eras) + bar * mean + x(col.continent) + y(col.era) + z(col.life) +
↪ space() +
  style(color = "#2a9d8f") +
  z_label("Mean life expectancy") + title("The three-quarter view")) |
```

```

(data(gm_eras) + bar * mean + x(col.continent) + y(col.era) + z(col.life)
↪ +
  space(tilt = 85) + style(color = "#2a9d8f") +
  z_label("Mean life expectancy") + title("Tilted up, with fewer
↪ numbers"))) +
  theme(height = 430))

```

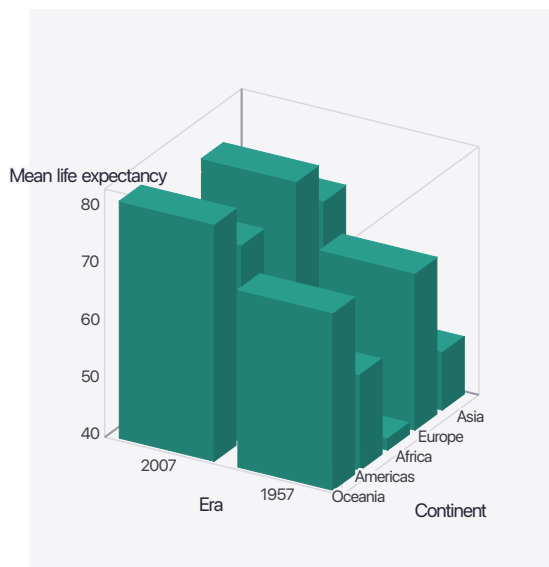
Julia

```

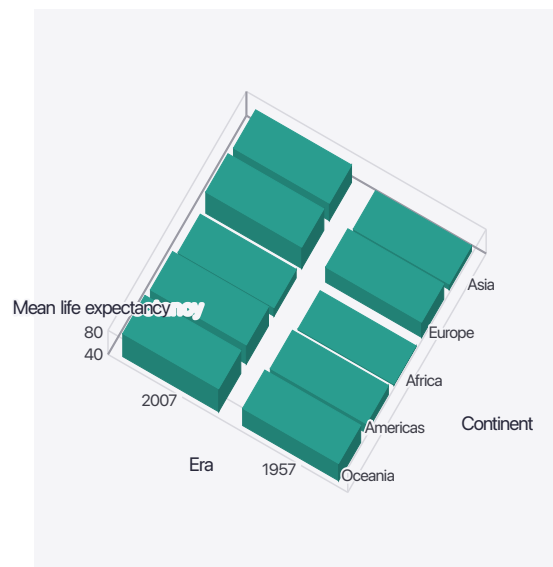
((data(gm_eras) + bar * mean + x(:continent) + y(:era) + z(:life) +
  space() + style(color = "#2a9d8f") + z_label("Mean life expectancy") +
  title("The three-quarter view"))) |
(data(gm_eras) + bar * mean + x(:continent) + y(:era) + z(:life) +
  space(tilt = 85) + style(color = "#2a9d8f") +
  z_label("Mean life expectancy") +
  title("Tilted up, with fewer numbers"))) + theme(height = 430)

```

The three-quarter view



Tilted up, with fewer numbers



“Given the gapminder eras: bars derived by mean, x is continent, y is era, z is life, beside the same tilted 85.”

The right-hand plot draws fewer numbers up its measure axis, and that is all it does about the crowding: it drops *labels* and never changes the scale. The other way to use the space is a

coarser tick step, and it is the wrong way. The step sets where the axis stops, so a bigger step raises the top of the axis and leaves the cube half empty. The numbers that survive are always a subset of the ones the left-hand plot shows, at exactly the spacing it uses, so a height read off either panel is the same height.

One more case, and it is about layers rather than marks. The depth sort runs *within* a layer, and layers are painted in the order you wrote them, exactly as they are on the flat page. So a **point** cloud layered over a **path** sits in front of the whole route rather than threading through it. That is the ordinary layering rule applied in space, not a new exception, so read a layered 3-D plot as layers, not as one depth-sorted scene.

33 Polar

Which way does the wind blow most often? Some questions do not fit a flat page. Every plot so far has been drawn on one: an *x* running across, a *y* running up. That is one coordinate space, *flat*, and the kernel names six more. It is also the wrong shape for a whole class of variables. Compass direction, the hour of the day, the month of the year: in each of them the last value is the neighbor of the first. A straight axis has no way to show that.

Two of the oldest statistical graphics already live in this space. Playfair's *Statistical Breviary* of 1801 cut a circle into sectors to show the Turkish empire's territory, and that is the first pie chart (Playfair, 1801). Florence Nightingale's 1858 diagram of what killed British soldiers in the Crimea (Nightingale, 1858) is a bar chart in polar coordinates, and it has three names: a **coxcomb**, a **rose**, or, most precisely, a **polar area diagram**. That third name is the one carrying information. Her wedges put each month's death count into the sector's *area*, so their radius grows as the square root of the number. A polar bar here maps the value straight to the radius. Both are honest drawings and they are not the same drawing, which is worth knowing before you set one beside the other.

The line version has collected even more names: **radar** plot, **spider** plot, **star** plot, and **Kiviat** diagram in engineering. As with the rose and the pie, none of them is an atom. Each is a mark and a channel or two with `polar()` at the end, which is what the rest of this chapter is about.

33.1 The axis that breaks a circle

Here is a season of wind observations as an ordinary bar chart, one bar per compass point:

R

```
data(winds) + bar * count + x(direction)
```

Python

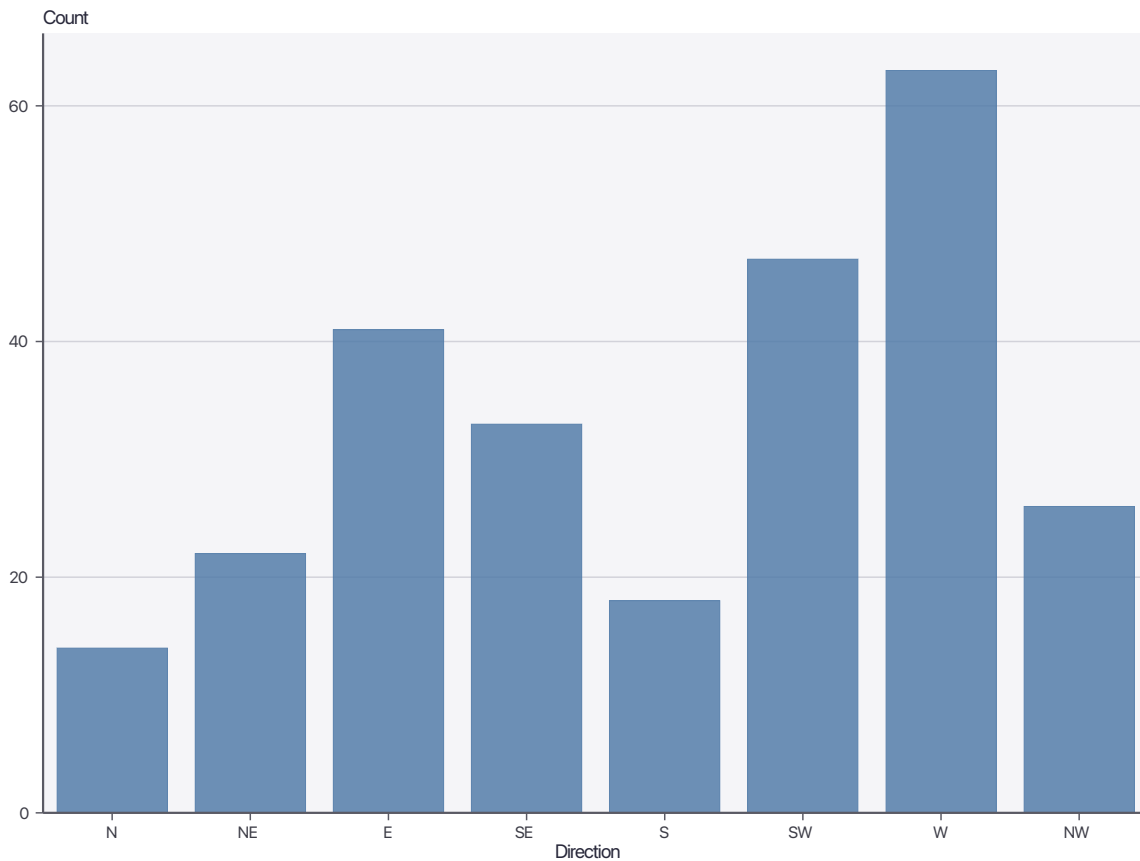
```
data(winds) + bar * count + x(col.direction)
```

Julia

```
data(winds) + bar * count + x(:direction)
```

JavaScript

```
plot(data(winds), layer(bar, count), x(col.direction))
```



“Given the winds: bars derived by count, x is direction.”

Read the two ends. North is at the far left and north-west at the far right, with the whole compass between them, when the two directions are 45 degrees apart in the world. The chart has cut the circle open and flattened it, and the cut is invisible: nothing in the picture says that the two ends belong together. A reader who did not already know the compass could not recover it from this.

Now add one atom to the same sentence:

R

```
data(winds) + bar * count + x(direction) + polar()
```

Python

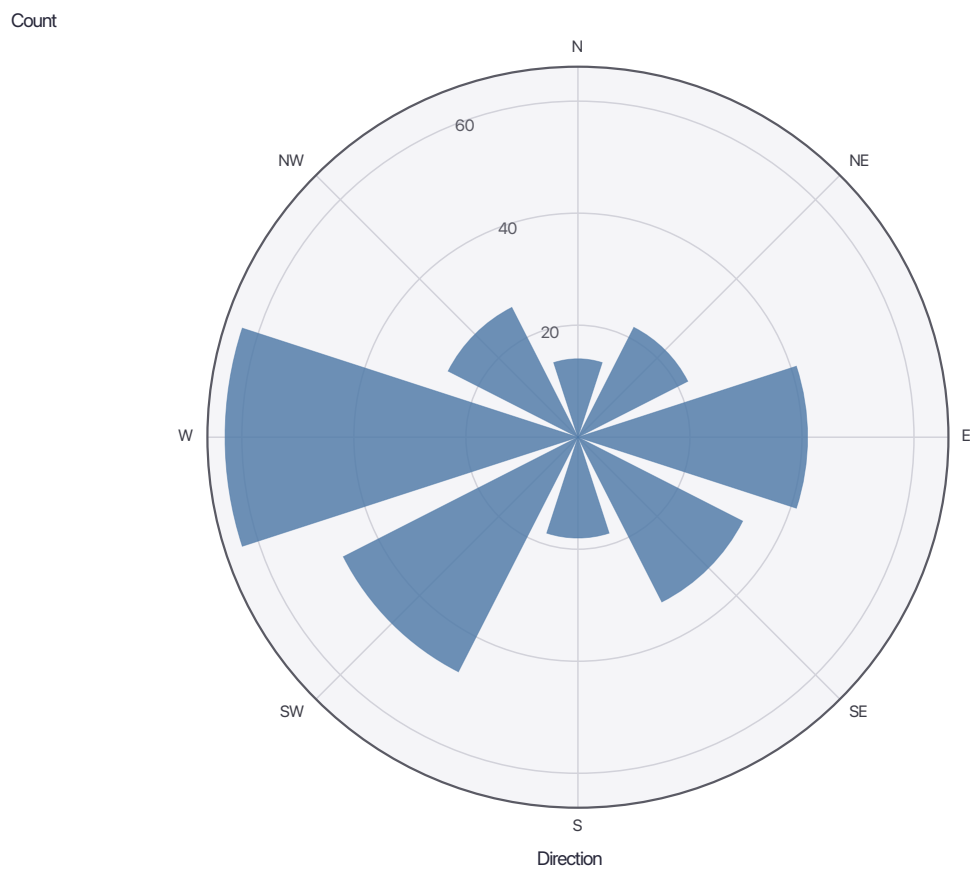
```
data(winds) + bar * count + x(col.direction) + polar()
```

Julia

```
data(winds) + bar * count + x(:direction) + polar()
```

JavaScript

```
plot(data(winds), layer(bar, count), x(col.direction), polar())
```



“Given the winds: bars derived by count, x is direction, in polar.”

Same mark, same transform, same binding, same data. The only thing that changed is the space the plot is drawn in, and north-west is beside north because on the compass it is beside north. The common west wind is now a long wedge pointing west, rather than a tall bar you find by reading the labels.

This is what a coordinate space is for, and it is why a pie chart, a rose and a radar plot are not chart types in this grammar. They are marks you already know, read in a circle.

33.2 What `polar()` changes, and what it leaves alone

`x` becomes the **angle** and `y` becomes the **radius**. That order is Wilkinson’s (Wilkinson, 2005), and his reason is that the first dimension of a plot is its domain: what you search in, which in a circle is a bearing. The second is the range, the amount, which becomes how far out from the center the mark reaches. (That is the rule when both positions are bound. Bind only one and there is nothing to choose, so the one you have becomes the angle: that is the pie, and it has its own section below.)

So `x(direction)` still says what it always said, that direction is the position each bar stands at. `polar()` only decides that positions are bearings rather than places along a line. A bar still runs from its baseline to its value, which is why the count still sets how far each wedge reaches: the center is zero.

Nothing else in the grammar knows the space changed, so nothing else has to be rewritten for it:

R

```
data(winds) + bar * count * dodge + x(direction) + color(season) + polar()
```

Python

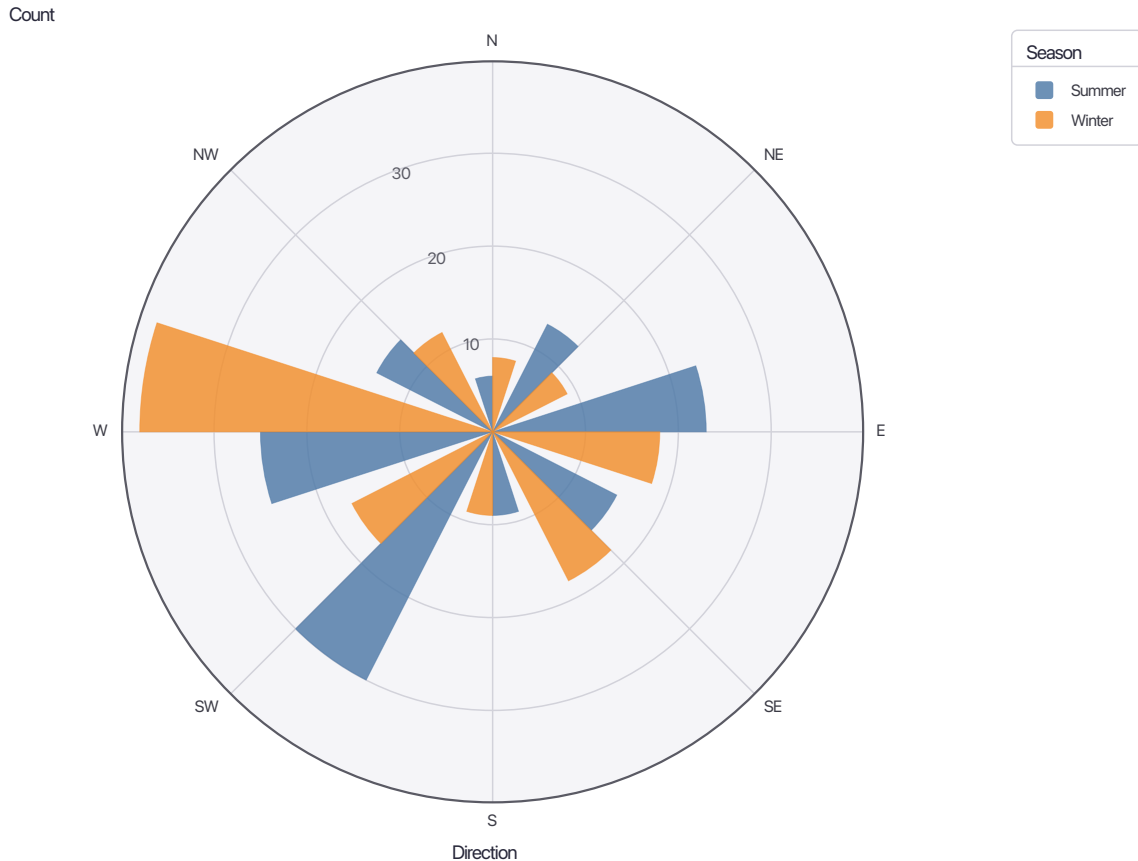
```
data(winds) + bar * count * dodge + x(col.direction) + color(col.season) +  
↪ polar()
```

Julia

```
data(winds) + bar * count * dodge + x(:direction) + color(:season) +  
polar()
```

JavaScript

```
plot(data(winds), layer(bar, count, dodge), x(col.direction),
     color(col.season), polar())
```



“Given the winds: bars derived by count and dodge, x is direction, color by season, in polar.”

`color` splits the count by season, and `dodge` sets the two halves side by side inside each slot instead of on top of each other. The legend decodes the colors. Every one of those works in a circle exactly as it works on a page, because none of them was ever written in terms of pixels.

`play` survives the bend too, and it is the one worth naming, because a rose that advances through time asks for nothing new. Each moment of the animation chooses the rows, and the space bends whatever it chose, so neither one changes the other. That chapter’s “Frames in polar” runs a rose and two pies through half a century. That chapter also reads the pie’s full turn as a measure axis, shared across every moment of the sequence.

Change the one word and the seasons pile outward from the center instead, one ring on top of another:

R

```
data(winds) + bar * count * stack + x(direction) + color(season) + polar()
```

Python

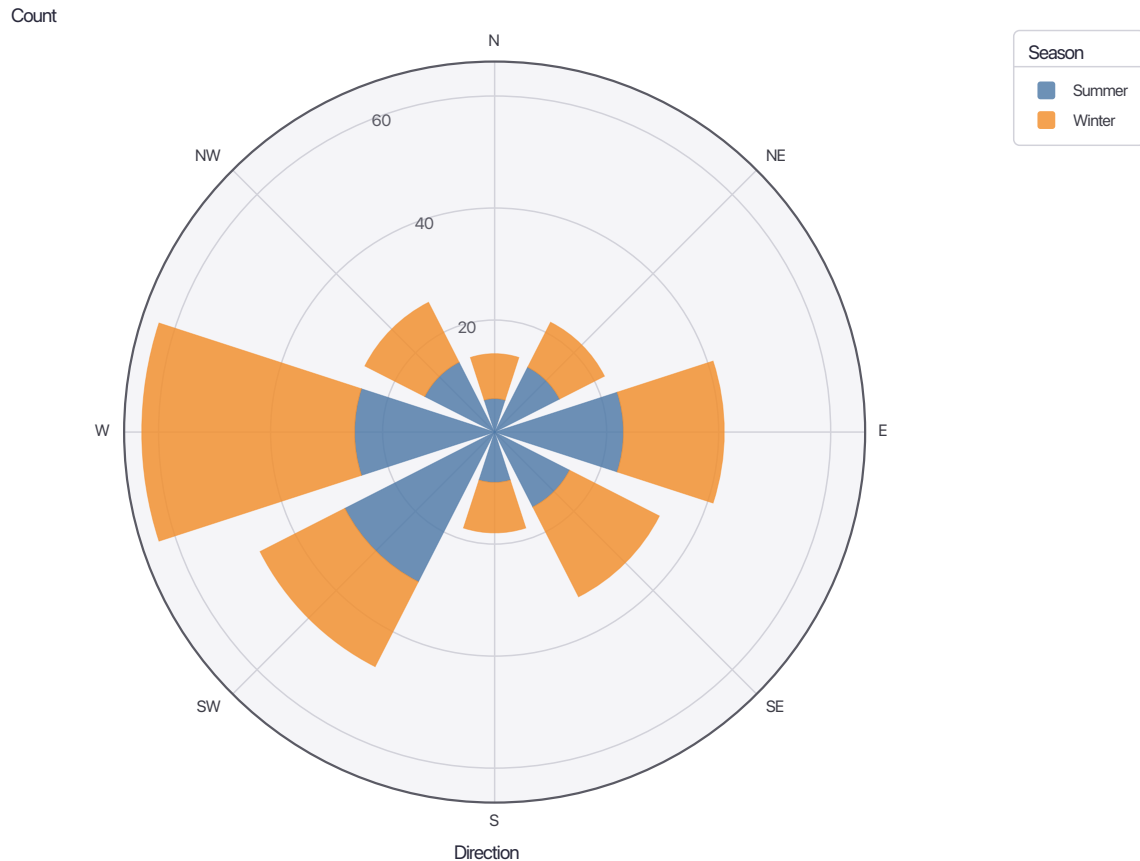
```
data(winds) + bar * count * stack + x(col.direction) + color(col.season) +  
    ↪ polar()
```

Julia

```
data(winds) + bar * count * stack + x(:direction) + color(:season) +  
    polar()
```

JavaScript

```
plot(data(winds), layer(bar, count, stack), x(col.direction),  
    color(col.season), polar())
```



“Given the winds: bars derived by count and stack, x is direction, color by season, in polar.”

Summer lies against the center and winter is stacked on top of it, so each wedge now reaches the season total. West runs out to 63. The dodged rose above stopped at 38, the taller of its two halves. The radial axis grew to fit, and that is why **stack** is a transform rather than a drawing option. The scale has to read the totals **stack** accumulates.

The pair is worth reading together, because each hides what the other shows. Stacked, the compass reads by total wind, and west is plainly the prevailing direction. Dodged, the seasons stand side by side, and you can see that the south-west blows in summer (30 against 17) while the west blows in winter (38 against 25). Neither is the correct rose. The rose is not a chart with its own options at all. It is **bar**, with its axis bent.

33.3 Where the circle starts

The angular axis begins at the top and runs clockwise, which is how a clock and a compass are both read. The first category sits *on* the top of the circle rather than beside it. That is why the rose above needed nothing said to it: north points straight up, because north is the first of the eight directions and the axis starts at the top.

The starting angle needed a decision, and the reason is worth knowing. A categorical scale runs from half a slot before the first category to half a slot after the last, so the scale's own origin is padding rather than any place in the data. A start angle aimed at the padding would put north at 22.5 degrees, leaving you to write $-180/8$ to correct it. A flat categorical axis already puts its tick at the category's center, so the circle agrees with it: **start** points at the category, not at the gap before it.

The `day_cycle` table holds one day of trips, hour by hour:

Table 33.1: `day_cycle`: first 5 of 25 rows

hour	trips
0	31
1	18
2	11
3	8
4	9

`polar(start = )` turns the whole space by a number of degrees:

R

```
data(day_cycle) + line + x(hour) + y(trips) + polar(start = -180)
```

Python

```
data(day_cycle) + line + x(col.hour) + y(col.trips) + polar(start = -180)
```

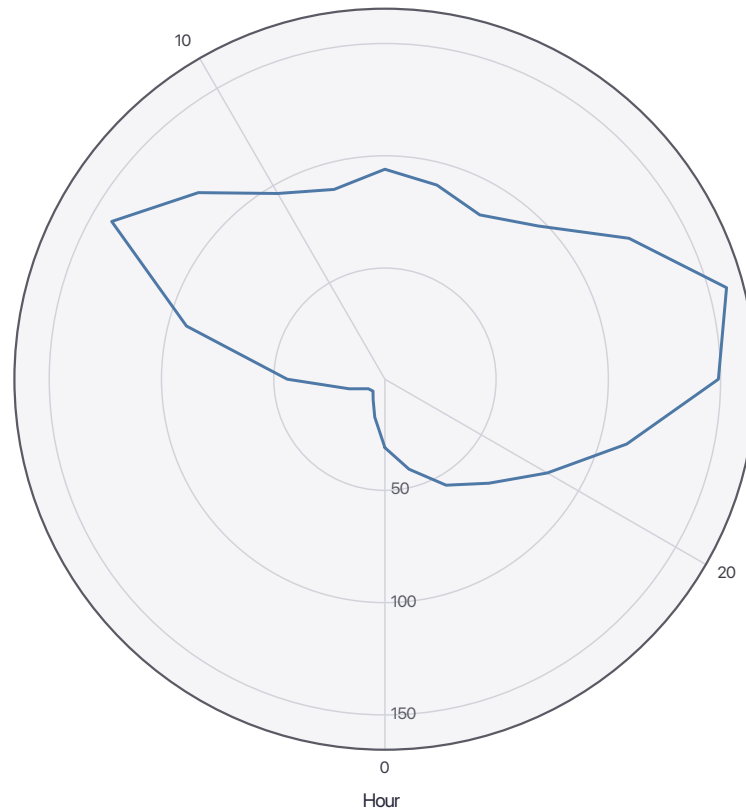
Julia

```
data(day_cycle) + line + x(:hour) + y(:trips) + polar(start = -180)
```

JavaScript

```
plot(data(day_cycle), line, x(col.hour), y(col.trips),
     polar({ start: -180 })))
```

Trips



“Given the day cycle: a line, x is hour, y is trips, in polar, starting at -180.”

Those are the same trips, turned half a circle. Noon is at the top now and midnight at the bottom, which is the right way up if you are reading the working day rather than the calendar one. `start` is a property of the space rather than of the data, the polar counterpart of the `space(turn =, tilt =)` that sets the angle a 3-D plot is viewed from (see [Space](#)). It changes how you look, never what is drawn.

33.4 One turn, whatever the data spans

The angular axis is **periodic**: one full turn covers exactly the range the axis covers, so its two ends land on the same spoke. That is what closes the circle with no seam and no empty wedge,

and it has a consequence worth saying out loud. The circle needs the whole cycle, or the two ends will meet in the wrong place.

Here is a day of trips, hour by hour. The table runs from hour 0 to hour 24 with *both* endpoints present, because midnight is where the day ends and also where it begins:

R

```
data(day_cycle) + line + x(hour) + y(trips) + polar()
```

Python

```
data(day_cycle) + line + x(col.hour) + y(col.trips) + polar()
```

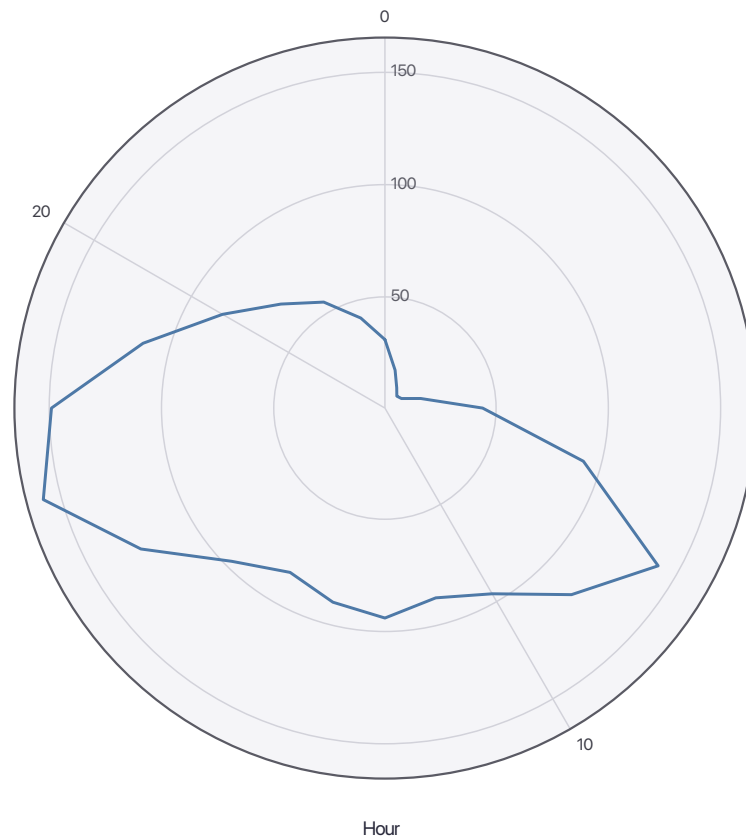
Julia

```
data(day_cycle) + line + x(:hour) + y(:trips) + polar()
```

JavaScript

```
plot(data(day_cycle), line, x(col.hour), y(col.trips), polar())
```

Trips



The curve closes at the top because the reading at 24 is the reading at 0, and the two sit on the same spoke. That is the first of three ways a circle closes. Most periodic data does not include both ends. A tide gauge reads every three hours and reaches neither midnight (hour 0 nor hour 24), so the axis it gets spans 1 to 22, and the failure that causes is easy to see:

R

```
data(tide) + line + x(hour) + y(height) + polar()
```

Python

```
data(tide) + line + x(col.hour) + y(col.height) + polar()
```

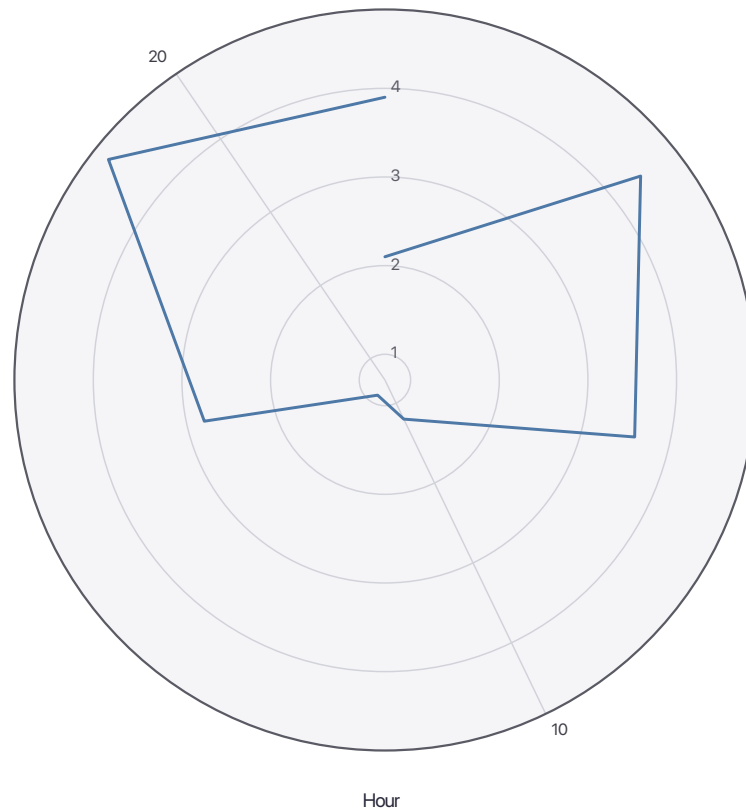
Julia

```
data(tide) + line + x(:hour) + y(:height) + polar()
```

JavaScript

```
plot(data(tide), line, x(col.hour), y(col.height), polar())
```

Height



One turn covers 1 to 22, so hour 1 and hour 22 land on the *same spoke*. The first and last readings are drawn on top of each other, three hours apart in the world and zero degrees apart on the page. Every reading between them sits at the wrong angle too, spread over 21 hours instead of 24.

A periodic axis cannot tell that your variable is periodic; nothing in a column of hours says the day wraps. So say it:

R

```
data(tide) + line + x(hour, limits = c(0, 24)) + y(height) + polar()
```

Python

```
data(tide) + line + x(col.hour, limits = [0, 24]) + y(col.height) + polar()
```

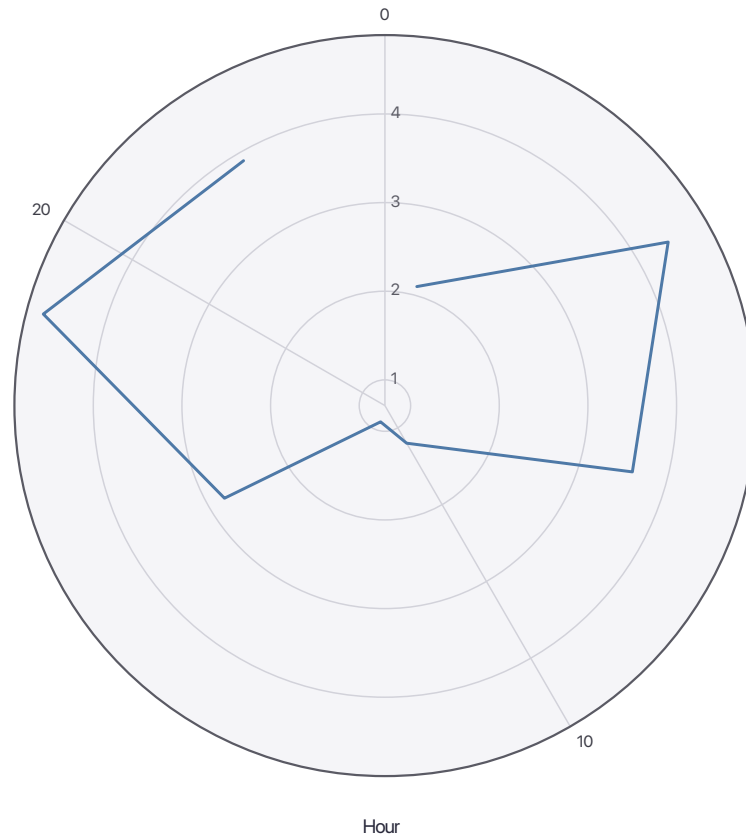
Julia

```
data(tide) + line + x(:hour, limits = [0, 24]) + y(:height) + polar()
```

JavaScript

```
plot(data(tide), line, x(col.hour, { limits: [0, 24] }), y(col.height),  
polar())
```

Height



“Given the tide: a line, x is hour from 0 to 24, y is height, in polar.”

Now each reading sits where a clock would put it. Hour 1 is a twenty-fourth of a turn past midnight, and hour 22 is a twelfth of a turn short of it. The two are 45 degrees apart, which is the three hours they really are.

The curve still does not close, which is the second case, and that is the correct picture. The gauge took no reading at midnight, and `limits` states a domain; it does not invent an observation. The gap you can see between hour 22 and hour 1 *is* the three-hour gap in the data, drawn at the size it is. Contrast the trips above, which close because a reading at 24 genuinely exists. A categorical angle is the third case and closes for a third reason, below: there every slot is filled, so the last category is adjacent to the first by construction rather than by luck.

`limits` states the domain the axis runs over rather than deriving it from the data, which is the general fix and not a polar one: the same words keep a color ramp fixed across facets. `Scales` has the rest of it. Note what it did *not* do here either: no reading was dropped, because 1 through 22 all lie inside 0 through 24. Stating a domain widens an axis as readily as it narrows one.

The same cycle filled is `area`, which closes on the baseline ring rather than on a straight line under the curve:

R

```
data(day_cycle) + area + x(hour) + y(trips) + polar()
```

Python

```
data(day_cycle) + area + x(col.hour) + y(col.trips) + polar()
```

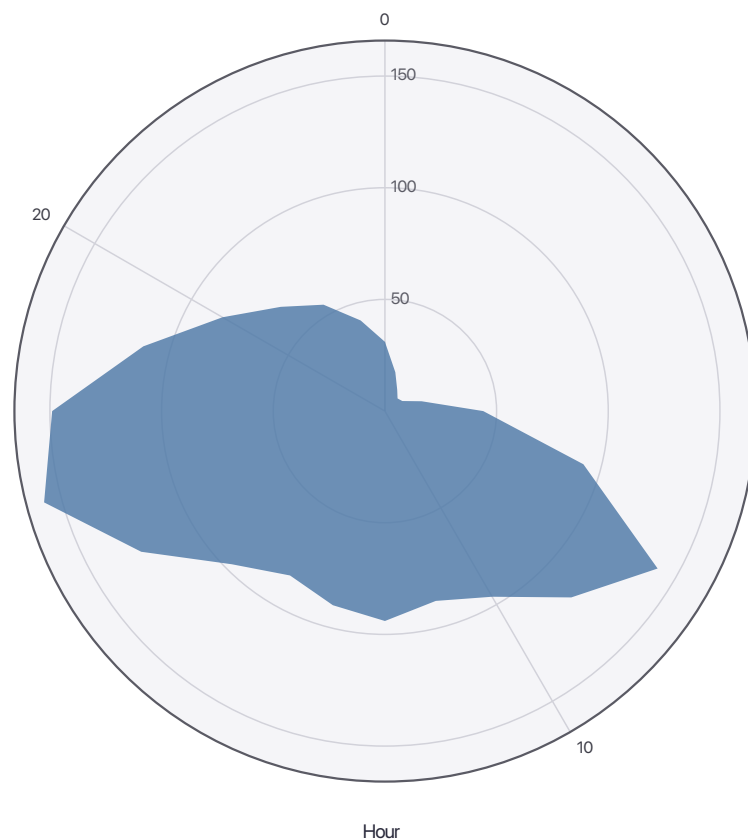
Julia

```
data(day_cycle) + area + x(:hour) + y(:trips) + polar()
```

JavaScript

```
plot(data(day_cycle), area, x(col.hour), y(col.trips), polar())
```

Trips



“Given the day cycle: an area, x is hour, y is trips, in polar.”

33.5 The radar: a categorical angle

A named angle closes differently, and the difference is worth following because the data causes it rather than the renderer. An hour axis closes because *you* gave it both ends of the cycle, 0 and 24, and the scale put them on one spoke. Categories cannot do that: each appears exactly once, by definition, so there is no repeated endpoint to land on. What they have instead is that together they fill the whole turn, which makes the last one genuinely adjacent to the first. So a path across categories closes:

R

```
data(gapminder_2007) + line * mean + x(continent) + y(life) + polar() +  
  title("line * mean + x(continent) + polar()")
```

Python

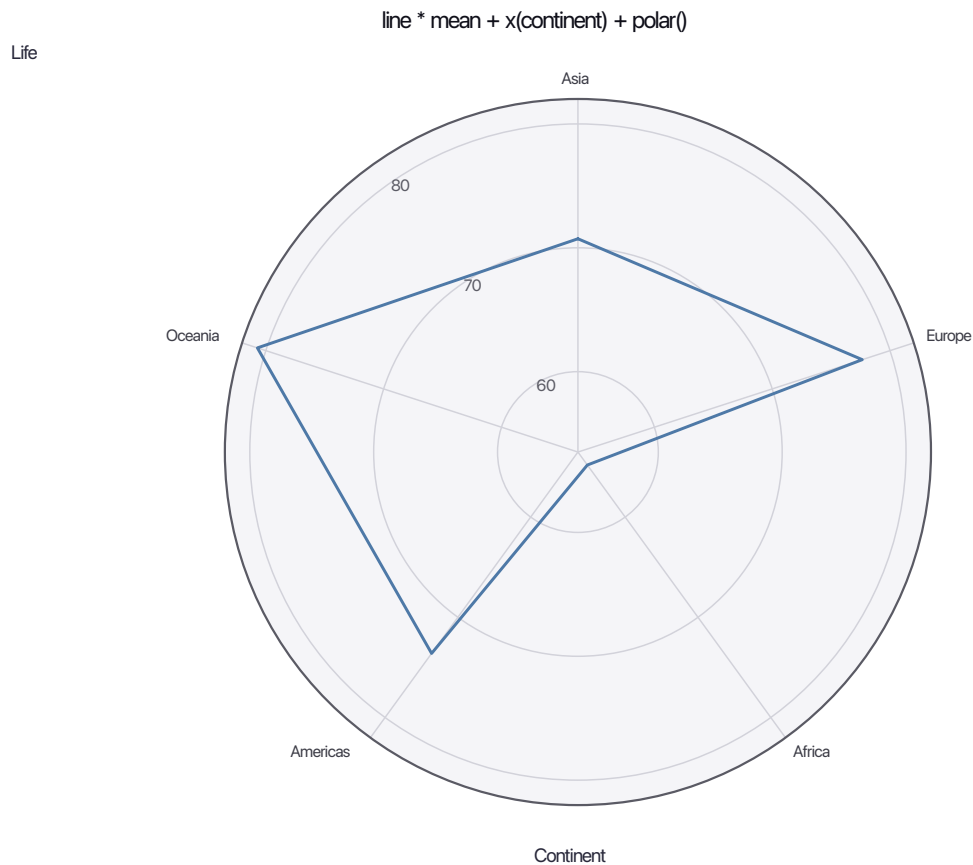
```
(data(gapminder_2007) + line * mean + x(col.continent) + y(col.life) +  
  ↪ polar() +  
  title("line * mean + x(continent) + polar()"))
```

Julia

```
data(gapminder_2007) + line * mean + x(:continent) + y(:life) + polar() +  
  title("line * mean + x(continent) + polar()")
```

JavaScript

```
plot(data(gapminder_2007), layer(line, mean), x(col.continent),  
  y(col.life), polar(), title("line * mean + x(continent) + polar()"))
```



“Given gapminder 2007: a line derived by mean, x is continent, y is life, in polar.”

That is a radar plot, and there is no **radar** atom in this grammar: the same sentence flat is the profile from the [Line](#) chapter, with a straight axis and therefore two ends. Bending the space is what joins them. Filled, it is the shape people usually mean by the name:

R

```
data(gapminder_2007) + area * mean + x(continent) + y(life) + polar() +  
  title("area * mean: the filled radar")
```

Python

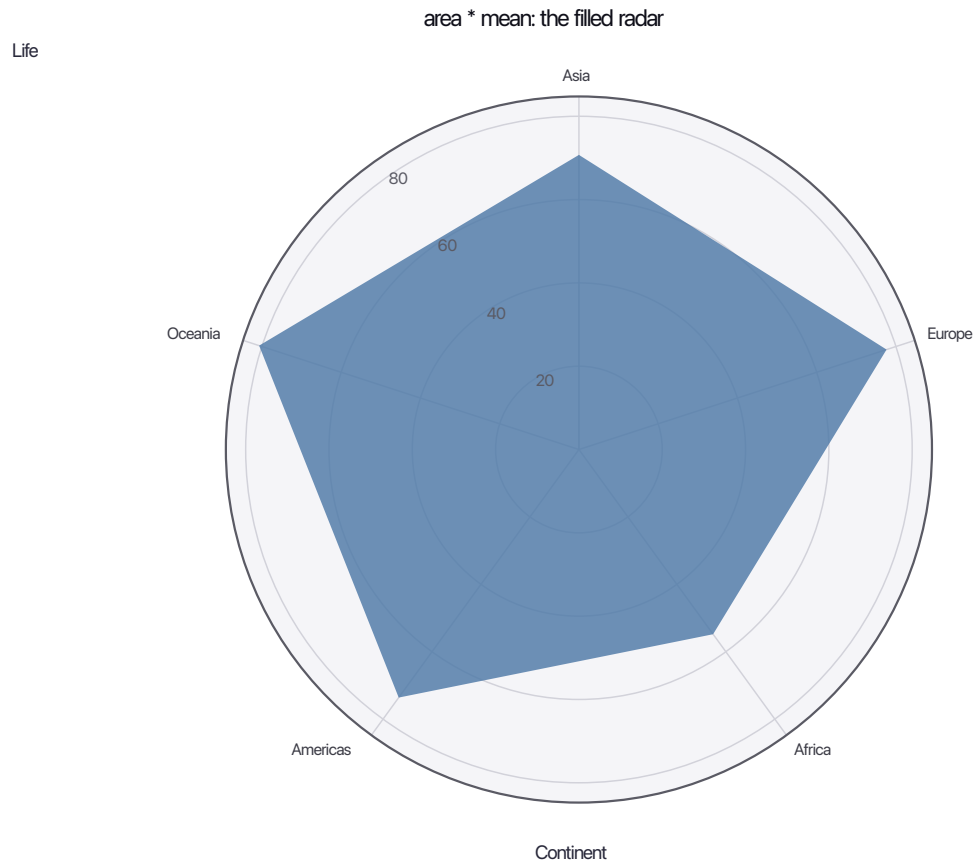
```
(data(gapminder_2007) + area * mean + x(col.continent) + y(col.life) +  
  ↪ polar() +  
  title("area * mean: the filled radar"))
```

Julia

```
data(gapminder_2007) + area * mean + x(:continent) + y(:life) + polar() +  
  title("area * mean: the filled radar")
```

JavaScript

```
plot(data(gapminder_2007), layer(area, mean), x(col.continent),  
  y(col.life), polar(), title("area * mean: the filled radar"))
```



Split it and each group is its own outline, and the split reads exactly as it does on a flat profile:

R

```
data(gm_eras) + line * mean + x(continent) + y(life) + color(era) + polar() +
  title("half a century apart, two rings")
```

Python

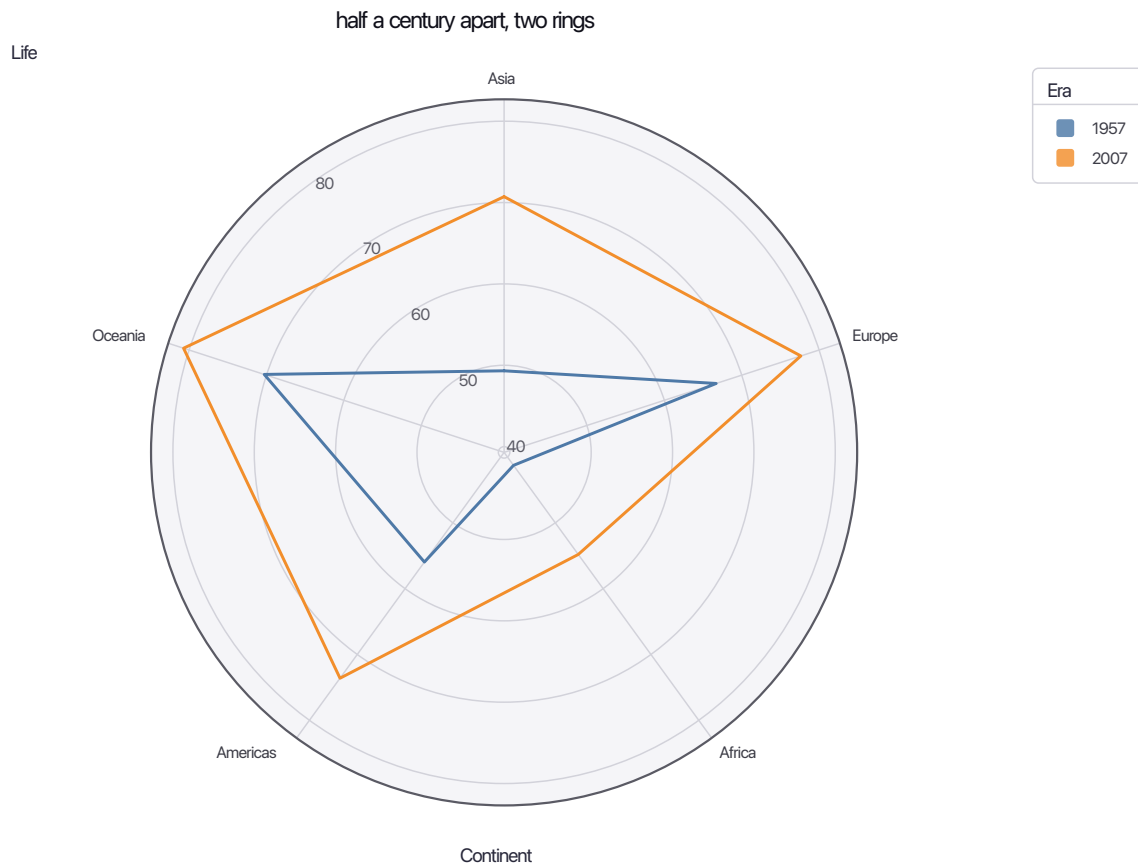
```
(data(gm_eras) + line * mean + x(col.continent) + y(col.life) +
  ⇨ color(col.era) + polar() +
  title("half a century apart, two rings"))
```

Julia

```
data(gm_eras) + line * mean + x(:continent) + y(:life) + color(:era) +
  polar() + title("half a century apart, two rings")
```

JavaScript

```
plot(data(gm_eras), layer(line, mean), x(col.continent), y(col.life),
  color(col.era), polar(), title("half a century apart, two rings"))
```



The same criticism applies to the radar as to the rose. A category's distance from the center is a radius, so its area grows as the square, and the enclosed shape covers an area that no number in the table gives. Use it when the categories really are a closed set with no first or last member, which is the case a straight axis misreports by having to pick an order. When the question is which continent is highest, the flat bar chart of the same sentence answers it precisely, and is two atoms away.

33.6 An angle that is measured, not named

An angular position does not have to be a category. `bearing` is the same wind observation recorded in degrees, a continuous position that happens to run around a circle:

R

```
data(winds) + point + x(bearing) + y(speed) + color(season) + polar()
```

Python

```
data(winds) + point + x(col.bearing) + y(col.speed) + color(col.season) +  
↪ polar()
```

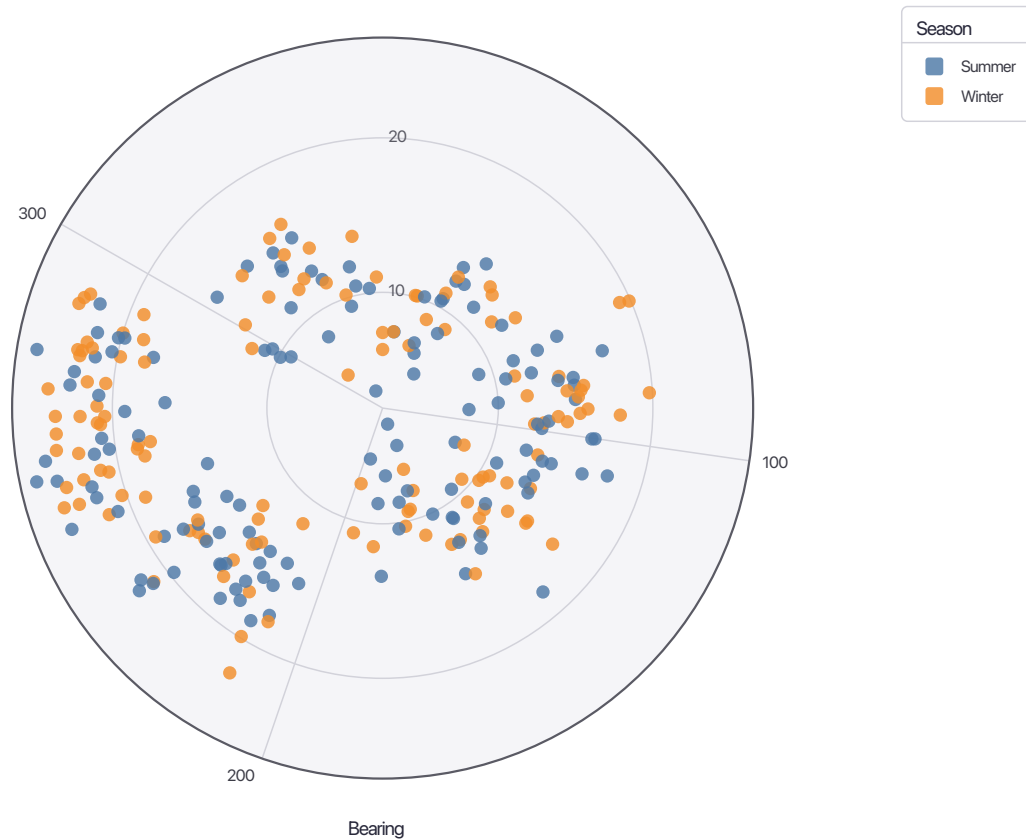
Julia

```
data(winds) + point + x(:bearing) + y(:speed) + color(:season) + polar()
```

JavaScript

```
plot(data(winds), point, x(col.bearing), y(col.speed), color(col.season),  
      polar())
```

Speed



“Given the winds: points, x is bearing, y is speed, color by season, in polar.”

Each dot sits at its own bearing, at the radius its speed sets. The observations were recorded at eight bearings, and those eight groups read as groups rather than as eight separate columns. The radial gridlines are read outward from the center, so a dot near the rim is a fast wind whichever way it was blowing.

`bin` cuts a measured angle into wedges the same way it cuts a flat axis into bars:

R

```
data(winds) + bar * bin + x(bearing) + polar()
```

Python

```
data(winds) + bar * bin + x(col.bearing) + polar()
```

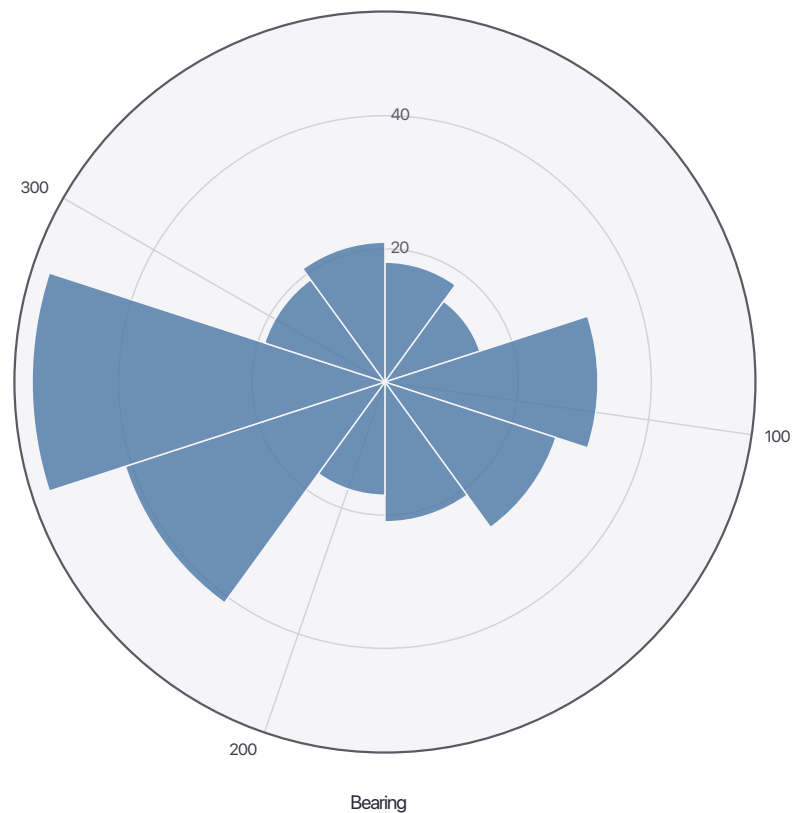

Julia

```
data(winds) + bar * bin + x(:bearing) + polar()
```

JavaScript

```
plot(data(winds), layer(bar, bin), x(col.bearing), polar())
```

Count



“Given the winds: bars derived by bin, x is bearing, in polar.”

That is the *rosa ventorum*, Latin for wind rose, which cartographers drew by hand from the 13th century. It is one atom different from a histogram: `bar * bin` in both cases, read on a line or read in a circle. The bins touch here as they touch there, because a histogram cuts a continuous axis into adjacent intervals, and that is true whichever shape the axis has been bent into.

A compass is a cycle, though, and the periodic axis cannot know that, just as with the hours. The range is fitted to what was observed, so the turn is the span of the *bins* rather than the 360 degrees a bearing actually runs over. The join between the two ends sits a little away from north. Stating the domain fixes it here exactly as it did there:

R

```
data(winds) + bar * bin + x(bearing, limits = c(0, 360)) + polar()
```

Python

```
data(winds) + bar * bin + x(col.bearing, limits = [0, 360]) + polar()
```

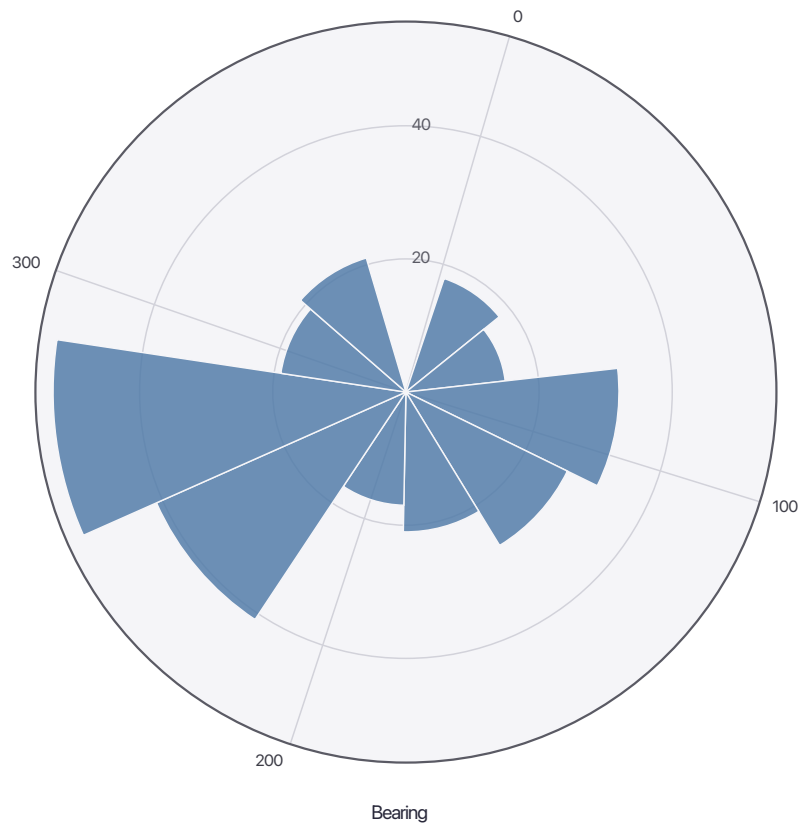
Julia

```
data(winds) + bar * bin + x(:bearing, limits = [0, 360]) + polar()
```

JavaScript

```
plot(data(winds), layer(bar, bin), x(col.bearing, { limits: [0, 360] }),  
    polar())
```

Count



The wedges now tile the compass rather than the data, so north on the plot is north.

33.7 The pie: where you put the category

Everything so far put the category on a *position*: `x(direction)` gave each compass point its own slot, and the count decided how far out the wedge reached. A pie does the opposite. Its categories are not positions at all; they are a split, and what goes around the circle is the measure itself.

That is the whole difference, and the grammar can say it. Take the category off `x` and give it to `color`, then say that the pieces lie end to end:

R

```
data(winds) + bar * count * stack + color(direction) + polar()
```

Python

```
data(winds) + bar * count * stack + color(col.direction) + polar()
```

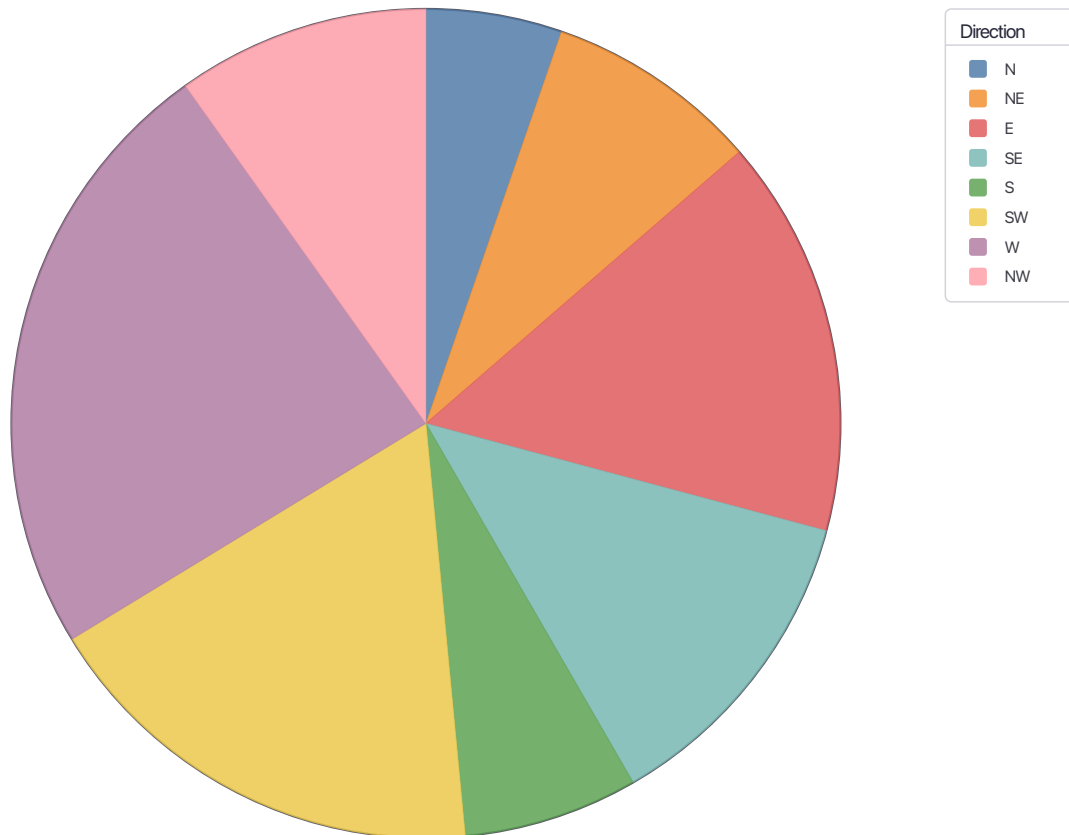
Julia

```
data(winds) + bar * count * stack + color(:direction) + polar()
```

JavaScript

```
plot(data(winds), layer(bar, count, stack), color(col.direction), polar())
```

Count



Read the sentence: a bar, counted, stacked, colored by direction, in polar. Not one word of it is about pies. The slices are the segments of a stacked bar, the angles are their shares, and the legend is the key.

The flat form is the same sentence without the last atom, and it is a real chart in its own right, the one column that shows how a total divides:

R

```
data(winds) + bar * count * stack + color(direction)
```

Python

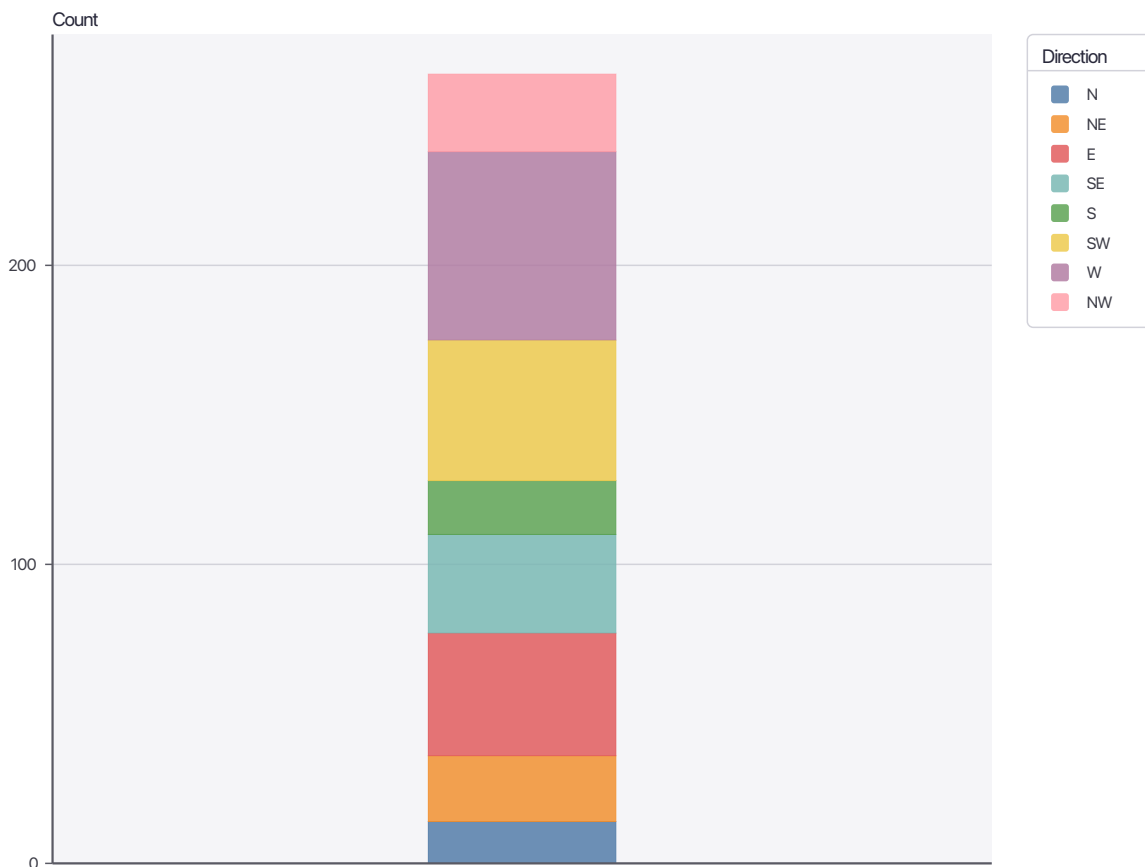
```
data(winds) + bar * count * stack + color(col.direction)
```

Julia

```
data(winds) + bar * count * stack + color(:direction)
```

JavaScript

```
plot(data(winds), layer(bar, count, stack), color(col.direction))
```



Those two pictures carry identical numbers. Wilkinson's second chapter is called *How To Make a Pie*, and his answer is exactly this: a pie chart is a stacked bar in polar coordinates. The column's height is the pie's full turn, and each segment's share of the height is its slice's share of the circle.

Nothing new was added to the grammar to reach it. There is no `pie()` atom, no `type = "pie"`, and no argument anywhere that says which chart to draw. Two things happen on their own. First, a bar divided only by its color split has one slot to stand in, so it needs no position axis, and dropping `x` is what says so. Second, a plot with one position bound has nothing to choose between: that position becomes the angle, and the radius is left as the constant that sets the pie's size. Both were in Wilkinson's book, where the one-argument polar function is a separate tool from the two-argument one.

The measure can be anything a group can be reduced to, not just a count. Swap `count` for `sum`, name the column it should total, and the same eight directions are weighted by how hard the wind blew rather than by how often:

R

```
data(winds) + bar * sum * stack + y(speed) + color(direction) + polar()
```

Python

```
data(winds) + bar * sum * stack + y(col.speed) + color(col.direction) +  
  ↪ polar()
```

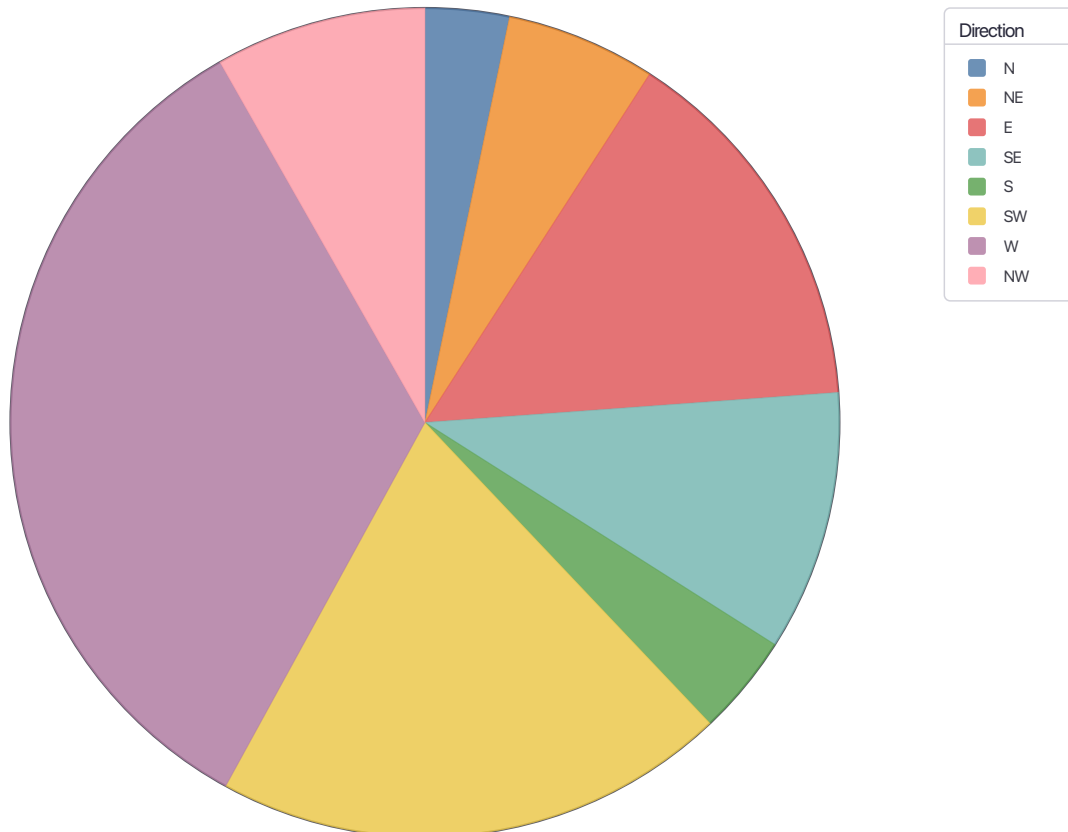
Julia

```
data(winds) + bar * sum * stack + y(:speed) + color(:direction) + polar()
```

JavaScript

```
plot(data(winds), layer(bar, sum, stack), y(col.speed),  
  color(col.direction), polar())
```

Speed



“Given the winds: bars derived by sum and stack, y is speed, color by direction, in polar.”

Compare it with the pie above: the same categories, a different answer. West is both the most common direction and the strongest, so its slice widens from about a quarter of the circle to a third. Winds from the north and south are the gentlest, so theirs shrink by roughly two fifths, even though exactly the same observations went into them. One atom decided which question the circle answers.

And `proportion` puts the shares on the axis rather than leaving them implied:

R

```
data(winds) + bar * proportion * stack + color(season) + polar()
```

Python

```
data(winds) + bar * proportion * stack + color(col.season) + polar()
```

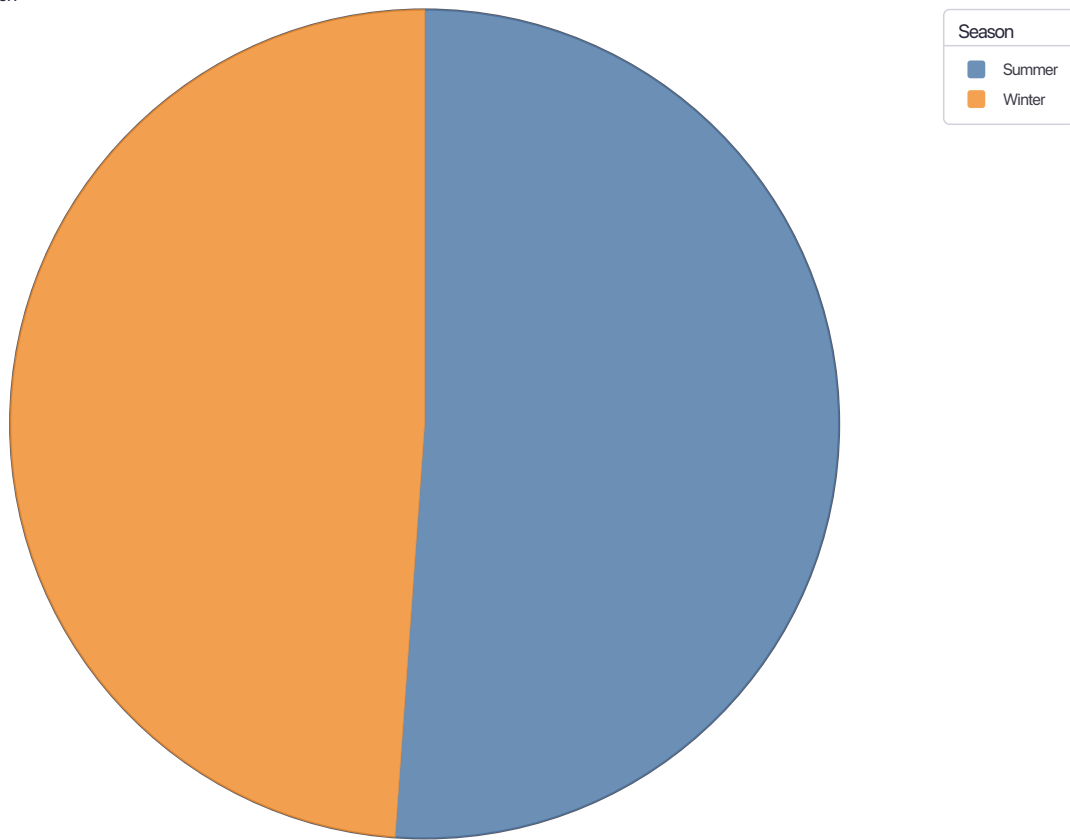
Julia

```
data(winds) + bar * proportion * stack + color(:season) + polar()
```

JavaScript

```
plot(data(winds), layer(bar, proportion, stack), color(col.season),  
      polar())
```


Proportion



“Given the winds: bars derived by proportion and stack, color by season, in polar.”

The split has to be there. A stack with nothing to pile, or a bar with neither a position nor a split, is refused rather than guessed at. So is a statistic that only means something *along* an axis:

```
render_svg(data(winds) + bar * bin * stack + color(season) + polar())
```

Error:

! gog: `bin` describes how values are spread along an axis, and this `bar`

↪ has no `x()` to spread them along. Add `x(<column>)` to draw the

↪ distribution, or use `count`/`sum` for one value per group.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

33.8 What a circle makes harder to read

The grammar draws every rose in this chapter, and drawing one is not the same as recommending it. A wedge's *area* grows with the square of its radius, so a count that doubles does not draw twice as much; it draws four times as much. Wilkinson says it plainly of the rose: the polar bar chart confounds area with radius, and growing the radius as the square root only partly repairs it. A reader who judges these by area, and most readers judge filled shapes by area, will read a stronger west wind than the numbers support.

That is [Law 8](#) working as intended: the engine guarantees a plot is well formed, never that it is well chosen. Use a circle when the variable really is a circle, which is the case a flat axis misreports. Use the flat bar chart when the comparison must be precise.

The pie avoids that criticism and has one of its own. Its slices are angles rather than radii, so a share of the total really is the same share of the circle, and readers judge that share well. What angles are bad at is *ranking*: two slices of 22% and 19% are hard to order in a circle and trivial to order in a column. Since both come from the same sentence, when ranking is the question, drop the last atom and read the bar.

33.9 The marks in a circle

There is no mark that draws on the page and refuses the circle. The summary marks read here exactly as they read flat: a box is still a five-number summary, a whisker still shows how far the values reach, a band is still a spread. Only the geometry changed.

R

```
data(winds) + box + x(direction) + y(speed) + polar()
```

Python

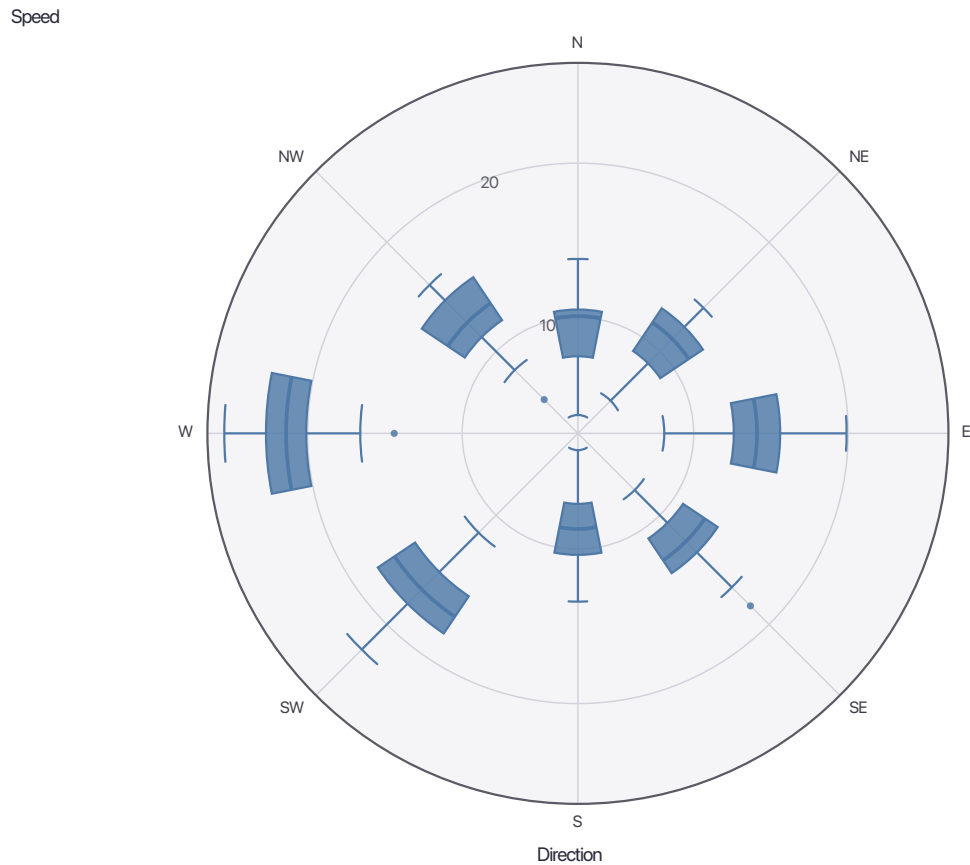
```
data(winds) + box + x(col.direction) + y(col.speed) + polar()
```

Julia

```
data(winds) + box + x(:direction) + y(:speed) + polar()
```

JavaScript

```
plot(data(winds), box, x(col.direction), y(col.speed), polar())
```



“Given the winds: boxes, x is direction, y is speed, in polar.”

Each box is a wedge of the ring between its quartiles, its median an arc across it, its whiskers running straight out from the center. Compare it with the same sentence flat and nothing has been added or taken away.

The whisker alone shows less, and it is the one to use when the quartiles are more detail than the question needs:

R

```
data(winds) + interval * range + x(direction) + y(speed) + polar()
```

Python

```
data(winds) + interval * range + x(col.direction) + y(col.speed) + polar()
```

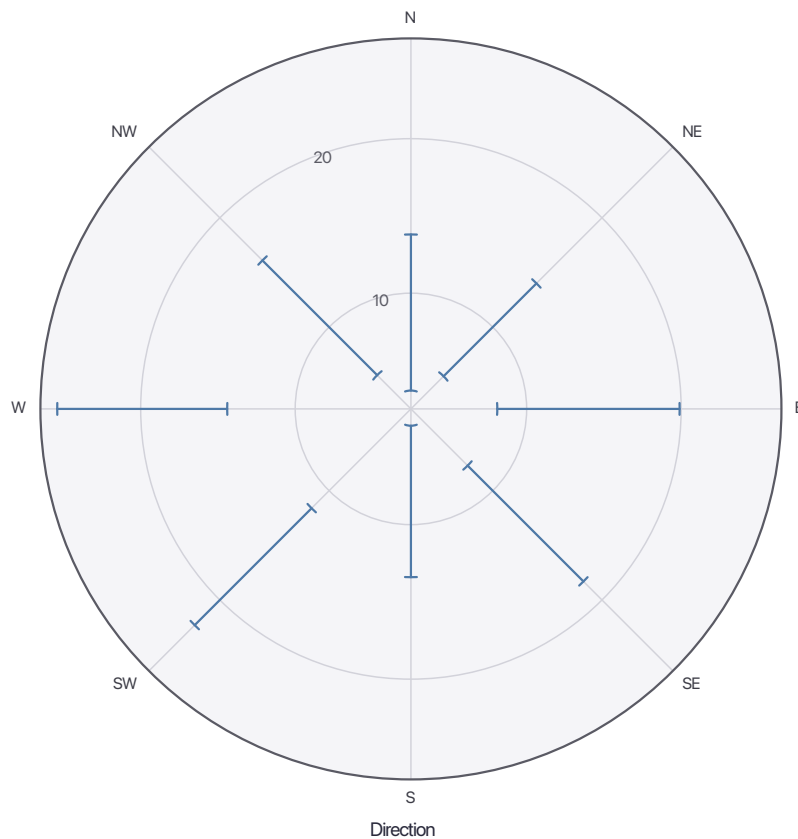
Julia

```
data(winds) + interval * range + x(:direction) + y(:speed) + polar()
```

JavaScript

```
plot(data(winds), layer(interval, range), x(col.direction), y(col.speed),  
     polar())
```

Speed



“Given the winds: intervals derived by range, x is direction, y is speed, in polar.”

A ribbon works the same way. Give each compass point a low and a high value, and `polar()` turns the ribbon into a **radar band**, the filled counterpart of the radar that `line` drew.

R

```
data(winds) + ribbon * range + x(direction) + y(speed) + polar()
```

Python

```
data(winds) + ribbon * range + x(col.direction) + y(col.speed) + polar()
```

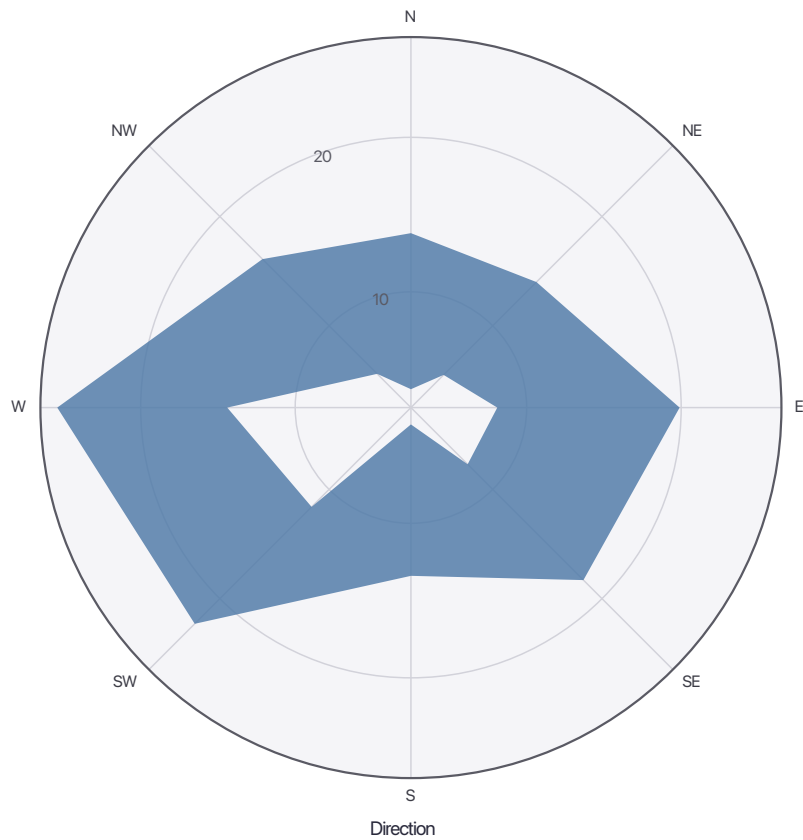
Julia

```
data(winds) + ribbon * range + x(:direction) + y(:speed) + polar()
```

JavaScript

```
plot(data(winds), layer(ribbon, range), x(col.direction), y(col.speed),  
      polar())
```

Speed



“Given the winds: a ribbon derived by range, x is direction, y is speed, in polar.”

A staircase holds each value across its own slot rather than sloping between them, so in a circle it is a ring of arcs joined by radial jumps. And because the categories exhaust the turn, the last one’s tread carries around to the first and the shape closes:

R

```
data(winds) + step * mean + x(direction) + y(speed) + polar()
```

Python

```
data(winds) + step * mean + x(col.direction) + y(col.speed) + polar()
```

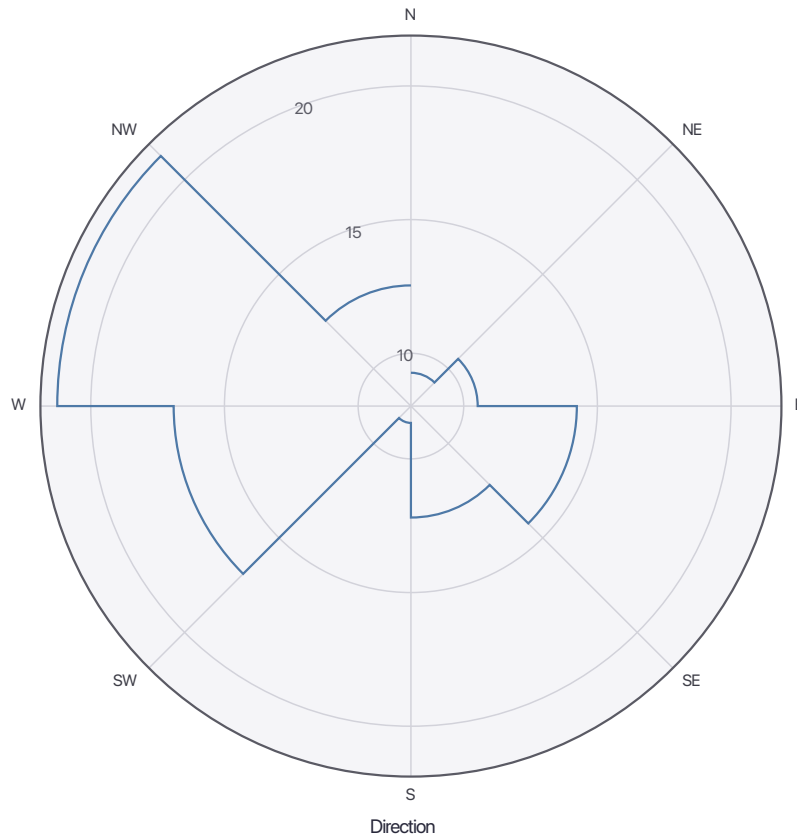
Julia

```
data(winds) + step * mean + x(:direction) + y(:speed) + polar()
```

JavaScript

```
plot(data(winds), layer(step, mean), x(col.direction), y(col.speed),  
  polar())
```

Speed



“Given the winds: a step outline derived by mean, x is direction, y is speed, in polar.”

Look at where the value changes. Every jump is a straight line out from the center, and every held value is a genuine arc rather than a chord cutting across it. That distinction is the whole of what these marks needed. A segment that *holds* a value across a span has to follow the ring. A straight line between the same two points falls inside the circle, and every point along it would be at a radius the data never took.

A region is a wedge, so a **zone** shades one. Two categorical axes give the tile plot bent into a disc, a cell per pair:

R

```
data(winds) + zone * count + x(direction) + y(season) + polar()
```

Python

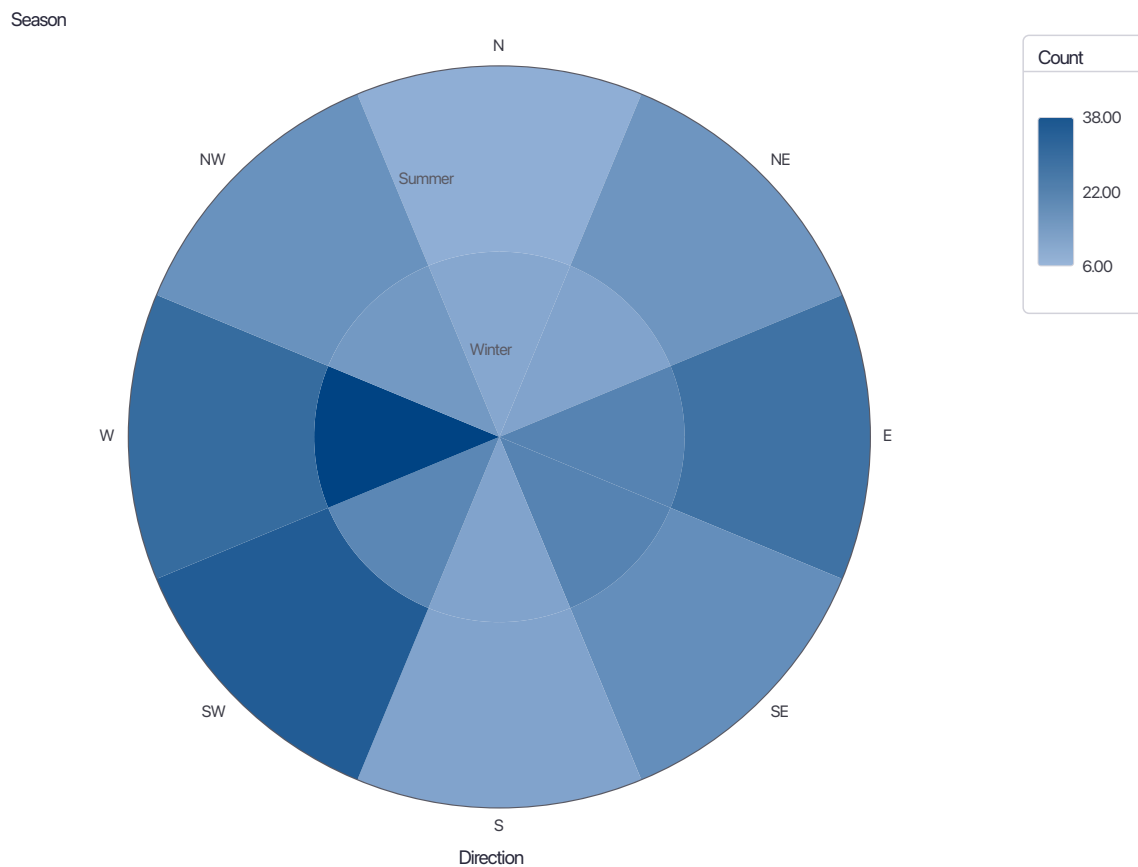
```
data(winds) + zone * count + x(col.direction) + y(col.season) + polar()
```

Julia

```
data(winds) + zone * count + x(:direction) + y(:season) + polar()
```

JavaScript

```
plot(data(winds), layer(zone, count), x(col.direction), y(col.season),  
polar())
```



“Given the winds: zones derived by count, x is direction, y is season, in polar.”

The one thing that does not bend is **surface**, and the circle is not what refuses it. A sheet through three positions has no reading in a space with two, which is why it does not draw on a

flat page either. A plot is also drawn in one space and not two, so asking for `polar()` and `z()` together is refused rather than one of the two being ignored.

One sentence is refused here, and the reason is worth reading, because it is the third form of one rule:

```
render_svg(data(gm_all) + zone * bin(tiling = "hex") + x(gdp) + y(life) +  
  ↪ polar())
```

Error:

```
! gog: `bin(tiling = "hex")` partitions a plane*, and `polar()` bends the  
  ↪ plane into a circle - where a cell is a sector, and a hexagon's six equal  
  ↪ sides are equal against a distance the space no longer has (a step of  
  ↪ angle is longer at the rim than at the center). Drop the tiling for  
  ↪ `rect`, whose cells are the sectors a bent rectangle already is, or drop  
  ↪ `polar()` to cut a flat plane.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

`bin(tiling = )` says how to partition a *plane*. It is refused on a one-dimensional bin because an interval is not a plane, and on a categorical axis because slots are an order rather than a distance. Here it is refused for the third form of the same problem. A bent plane has no distance for a hexagon to be regular against, because a step of angle at the rim is longer than the same step near the center. A hexagon exists to keep every cell the same size and shape, and a bent plane takes that away. A rectangle survives the bend, because a bent rectangle is a sector and that is a shape the engine draws.

Which marks draw in which coordinate space is stated in full, on one page, by the Mark $\times$ Space grid in [Combinations](#). It is generated from the engine's own table, so it cannot promise a plot the engine will not draw.

33.10 The sunburst: depth is the radius

A **sunburst** draws a tree: one ring per level, each arc as wide as its share of the whole and sitting inside its parent's. It has no mark of its own and needs none. An arc of a ring is a rectangle in angle and radius. **zone** is the mark bounded by a pair on each axis, and this space is what bends a rectangle into a sector. The arithmetic between a hierarchy and those four edges is **partition**.

The hierarchy arrives as **columns**. **spending** is a household budget with one row per leaf, meaning the last level of a branch, and the first three columns spell the path down to it:

```
knitr::kable(head(spending, 6))
```

group	item	detail	amount
Housing	Rent	NA	980
Housing	Utilities	Energy	140
Housing	Utilities	Water	35
Housing	Repairs	NA	120
Food	Groceries	NA	420
Food	Eating out	NA	185

A missing value is not missing data there. It says the branch stops: rent has no sub-parts worth naming, so it reaches the second level and no further.

R

```
data(spending) + zone * partition(group, item, detail) + x(amount) +  
  color(group) + polar()
```

Python

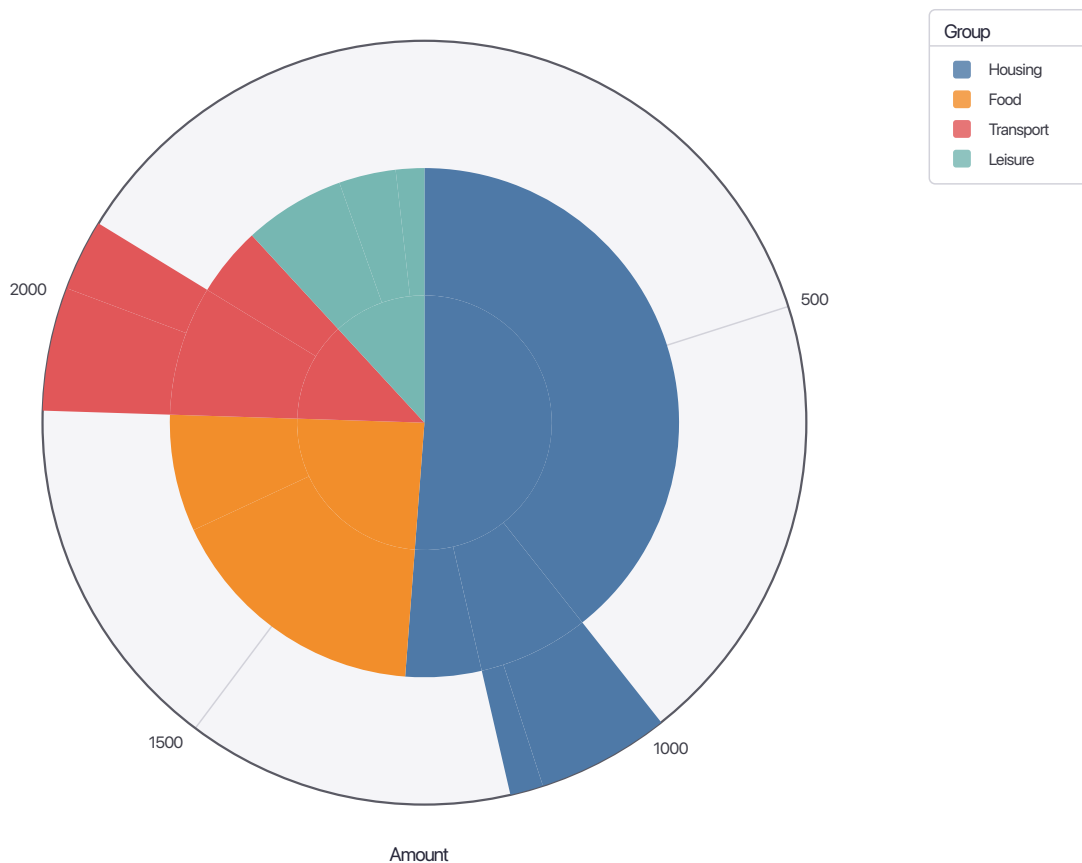
```
(data(spending) + zone * partition(col.group, col.item, col.detail) +  
  ↪ x(col.amount) +  
  color(col.group) + polar())
```

Julia

```
data(spending) + zone * partition(:group, :item, :detail) + x(:amount) +  
  color(:group) + polar()
```

JavaScript

```
plot(data(spending),  
  layer(zone, partition(col.group, col.item, col.detail)), x(col.amount),  
  color(col.group), polar())
```



“Given the spending table: zones derived by partition through group, item and detail, x is amount, color by group, in polar.”

`partition(group, item, detail)` names the levels from the top of the tree down, so **group** is the innermost ring and **detail** the rim. `x(amount)` is what each branch is weighed by. Nothing in the sentence is about drawing, and nothing in it is about circles.

Bind nothing to `x` and every leaf weighs 1, which is the tally `count` already does when nothing else is measured. The rings then divide by how many leaves each branch has rather than by what they cost:

R

```
data(spending) + zone * partition(group, item, detail) + color(group) +
  ↪ polar()
```

Python

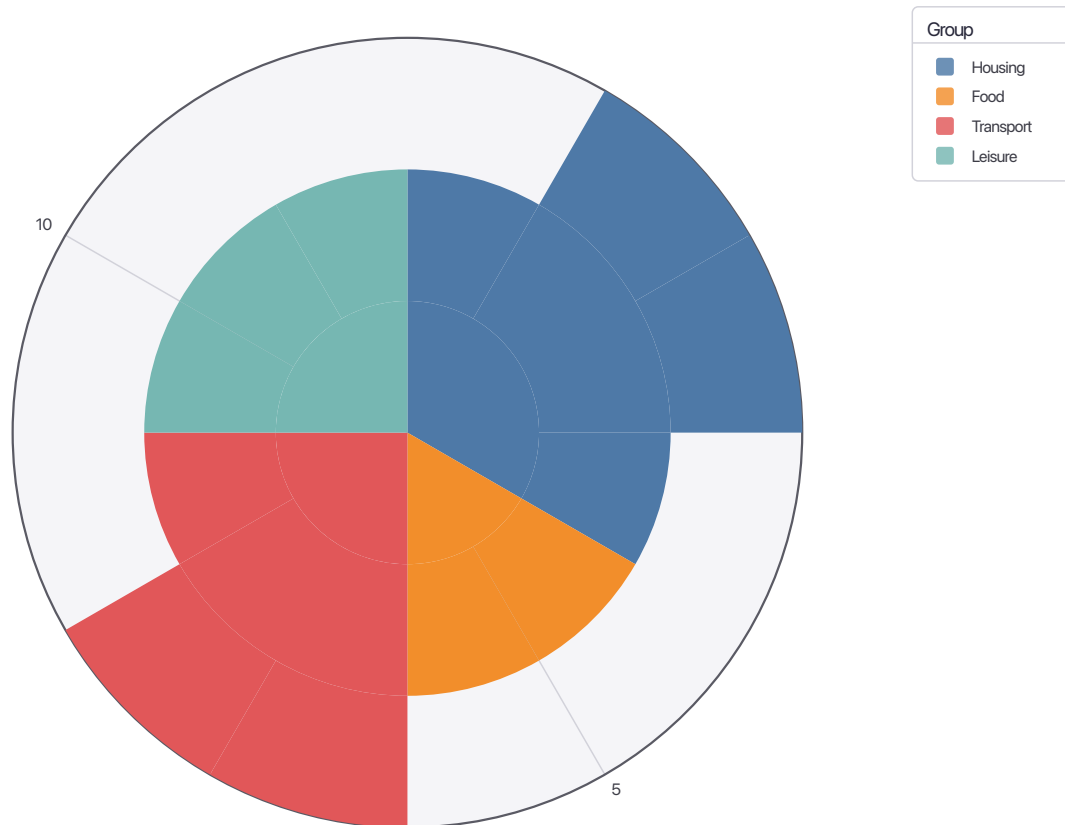
```
data(spending) + zone * partition(col.group, col.item, col.detail) +
  ↪ color(col.group) + polar()
```

Julia

```
data(spending) + zone * partition(:group, :item, :detail) +
  color(:group) + polar()
```

JavaScript

```
plot(data(spending),
  layer(zone, partition(col.group, col.item, col.detail)),
  color(col.group), polar())
```



Housing fills half the circle above and a third of it here, because it is 1,275 of the 2,490 spent and 4 of the 12 leaves that spend it. That is the same swap the pie made between `count` and `sum`, arriving at a tree.

Read the outer rim. It is solid above Utilities and Car and blank above every other item, because those two are the only items with a third level. A ragged rim is what a real hierarchy looks like, and it comes from the missing values rather than from anything the plot was told.

The radial axis carries **depth**, which is the level a node sits at: 1 for the groups on the inner ring, 2 for the items outside them, 3 for the details on the rim. The transform computes it and gives it that name. It is a column you can bind like any other, even though it is not in your table: `partition` is what put it there. Naming it is how you say what the radius runs over, and stating a domain that starts below the first ring is what hollows out the middle:

R

```
data(spending) + zone * partition(group, item, detail) + x(amount) +  
  y(depth, limits = c(0, 4)) + color(group) + polar()
```

Python

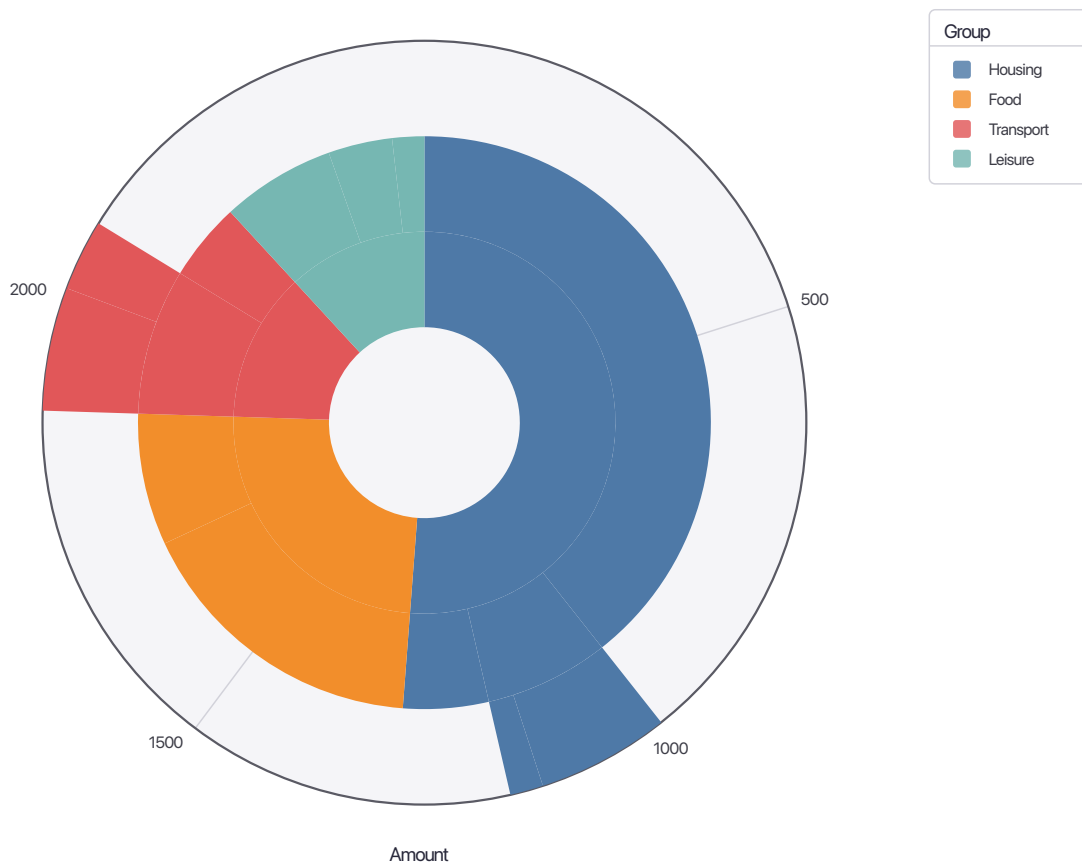
```
(data(spending) + zone * partition(col.group, col.item, col.detail) +  
  ↪ x(col.amount) +  
  y(col.depth, limits = [0, 4]) + color(col.group) + polar())
```

Julia

```
data(spending) + zone * partition(:group, :item, :detail) + x(:amount) +  
  y(:depth, limits = [0, 4]) + color(:group) + polar()
```

JavaScript

```
plot(data(spending),  
  layer(zone, partition(col.group, col.item, col.detail)), x(col.amount),  
  y(col.depth, { limits: [0, 4] }), color(col.group), polar())
```



That is worth a sentence, because it is not a setting. A hole is a stretch of the radial axis with nothing standing on it: the innermost ring starts at 1, and saying the axis runs from 0 leaves the first unit of radius empty. Widen the stretch and the hole widens with it, since `limits` states a domain and the rings keep the places they always had: `c(-1, 4)` gives up two fifths of the radius rather than a quarter.

33.11 The donut: one level is still a tree

A hierarchy one level deep is still a hierarchy, and everything above is true of it as well. One level is one ring, that ring is at depth 1, and an axis running from 0 to 2 leaves the first of its two units empty, which is a **donut**:

R

```
data(spending) + zone * partition(group) + x(amount) +  
  y(depth, limits = c(0, 2)) + color(group) + polar()
```

Python

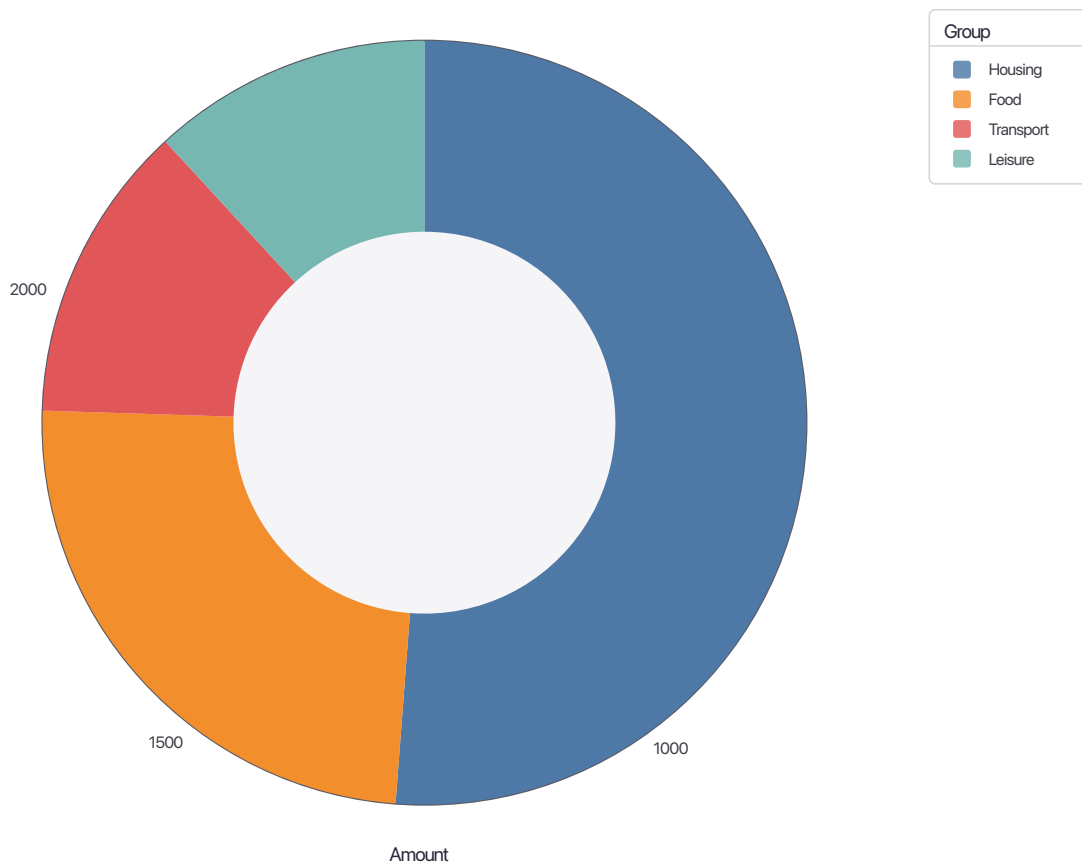
```
(data(spending) + zone * partition(col.group) + x(col.amount) +  
  y(col.depth, limits = [0, 2]) + color(col.group) + polar())
```

Julia

```
data(spending) + zone * partition(:group) + x(:amount) +  
  y(:depth, limits = [0, 2]) + color(:group) + polar()
```

JavaScript

```
plot(data(spending), layer(zone, partition(col.group)), x(col.amount),  
  y(col.depth, { limits: [0, 2] }), color(col.group), polar())
```



There is no `donut` atom, no `hole = setting`, and nothing in that sentence about donuts. It is the sunburst's sentence with one level named instead of three, drawn on a radial axis with one empty unit at the bottom of it.

The donut you usually see published writes its names in the ring, and that is the `text` layer from [Naming the nodes](#) (just below) with one level instead of two:

R

```
data(spending) + x(amount) + y(depth, limits = c(0, 2)) +
  zone * partition(group) + color(group) +
  text * partition(group) + label(name) + style(color = "white") +
  polar() + x_label("")
```

Python

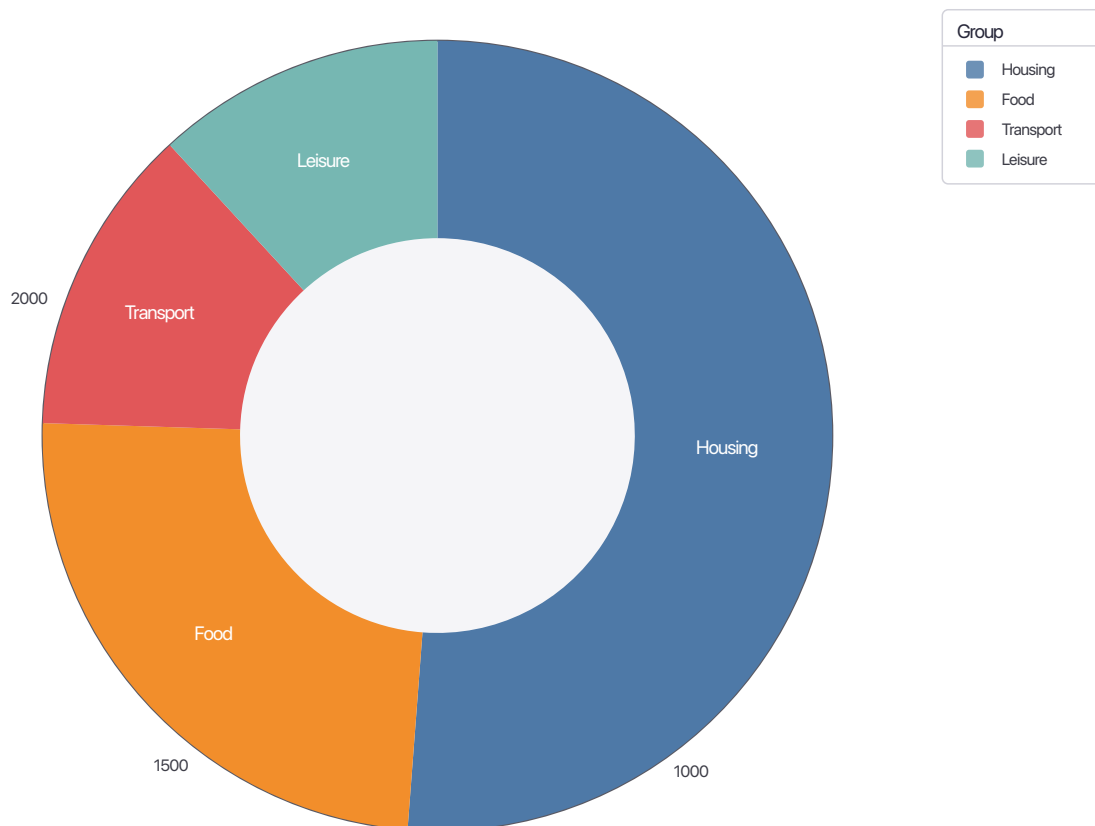

```
(data(spending) + x(col.amount) + y(col.depth, limits = [0, 2]) +
  zone * partition(col.group) + color(col.group) +
  text * partition(col.group) + label(col.name) + style(color = "white") +
  polar() + x_label(""))
```

Julia

```
data(spending) + x(:amount) + y(:depth, limits = [0, 2]) +
  zone * partition(:group) + color(:group) + text * partition(:group) +
  label(:name) + style(color = "white") + polar() + x_label("")
```

JavaScript

```
plot(data(spending), x(col.amount), y(col.depth, { limits: [0, 2] }),
  layer(zone, partition(col.group)), color(col.group),
  layer(text, partition(col.group)), label(col.name),
  style({ color: "white" }), polar(), x_label(""))
```



“Given the spending table: x is amount, y is depth from 0 to 2, zones derived by partition through group, color by group, and also text derived by partition through group, label by name, in polar.”

The legend now says twice what the ring says once, so a published plot would keep one of them. Both are kept here, so that the two ways of decoding a wedge stand side by side.

The pie is the one plot in this space that cannot have a hole, and that follows from what a pie binds. A [pie](#) puts its measurement on the angle and leaves the radius constant, so it has no radial domain to widen. A hole here would have to be a number given to the space, and a hole is never a number. The sentence that wants a hole is the sentence whose radius carries something, and that sentence is one atom away.

33.12 The same sentence, unbent

Take `polar()` off the three-level sunburst and change nothing else:

R

```
data(spending) + zone * partition(group, item, detail) + x(amount) +  
  y(depth, limits = c(0, 4)) + color(group)
```

Python

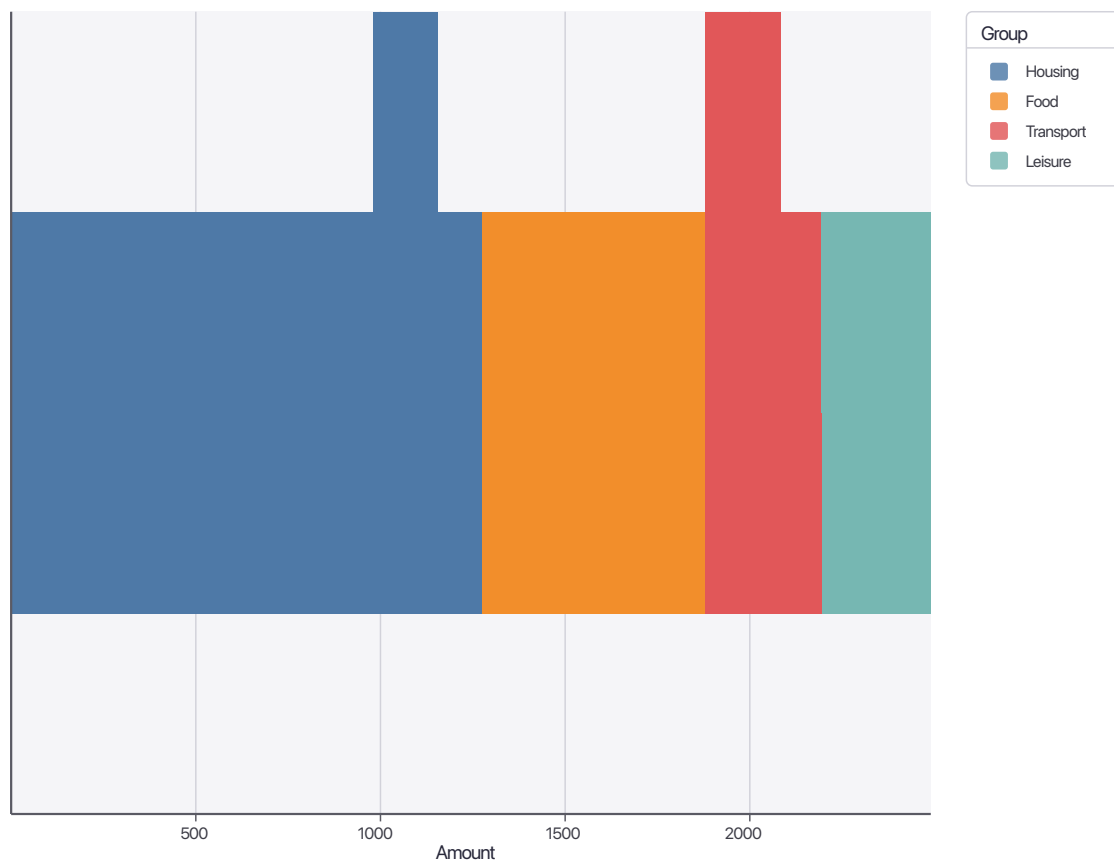
```
(data(spending) + zone * partition(col.group, col.item, col.detail) +  
  ↪ x(col.amount) +  
  y(col.depth, limits = [0, 4]) + color(col.group))
```

Julia

```
data(spending) + zone * partition(:group, :item, :detail) + x(:amount) +  
  y(:depth, limits = [0, 4]) + color(:group)
```

JavaScript

```
plot(data(spending),  
  layer(zone, partition(col.group, col.item, col.detail)), x(col.amount),  
  y(col.depth, { limits: [0, 4] }), color(col.group))
```



That is an **icicle chart**: depth going up instead of outward, share going across instead of around. The two are one coordinate space apart, and that is this chapter's point arriving at a chart with a name of its own. A rose and a bar chart are the same pair, and so are a pie and a stacked bar.

The empty band along the bottom is the hole, and seeing it flat is the clearest statement of what a hole is. It is not a round thing. It is a stretch of the depth axis with nothing on it, and bending the picture wraps that stretch into a disc at the center.

33.13 Naming the nodes

`partition` publishes each node's **center** as well as its four edges, which is why a second mark can read the same computation. `text` takes the center and labels the node:

R

```
data(spending) + x(amount) + y(depth, limits = c(0, 4)) +
  zone * partition(group, item, detail) + color(group) +
  text * partition(group, item) + label(name) + style(color = "white") +
  polar() + x_label("")
```

Python

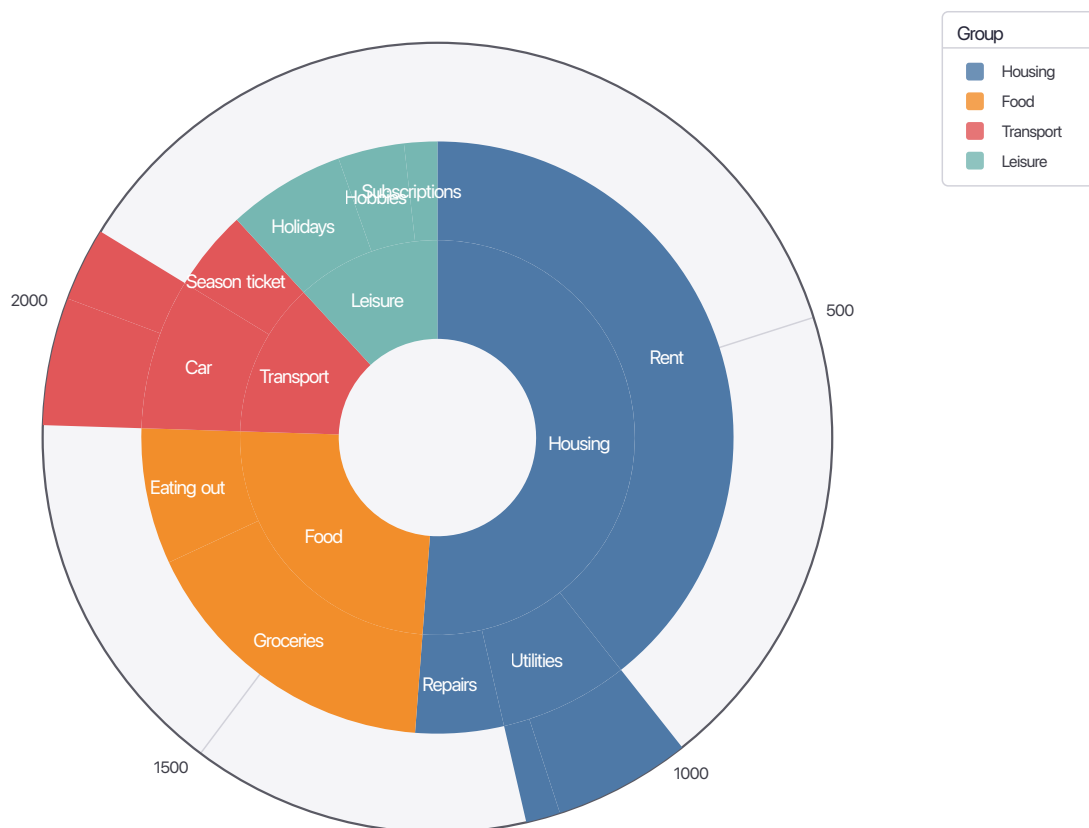
```
(data(spending) + x(col.amount) + y(col.depth, limits = [0, 4]) +
  zone * partition(col.group, col.item, col.detail) + color(col.group) +
  text * partition(col.group, col.item) + label(col.name) + style(color =
↪ "white") +
  polar() + x_label(""))
```

Julia

```
data(spending) + x(:amount) + y(:depth, limits = [0, 4]) +
  zone * partition(:group, :item, :detail) + color(:group) +
  text * partition(:group, :item) + label(:name) +
  style(color = "white") + polar() + x_label("")
```

JavaScript

```
plot(data(spending), x(col.amount), y(col.depth, { limits: [0, 4] }),
  layer(zone, partition(col.group, col.item, col.detail)),
  color(col.group), layer(text, partition(col.group, col.item)),
  label(col.name), style({ color: "white" }), polar(), x_label(""))
```



`label(name)` reads a column the transform wrote, the way `color(count)` reads a binned tally. And the text layer names *two* levels where the zone names three, which is how the rim is left unlabeled. A shallower partition of the same table puts its nodes in exactly the same places, because the arcs are decided by the leaves underneath them, and those did not change.

One thing about that plot is a limit of the engine rather than a choice. The names sit upright where a published sunburst runs them along their arcs, because `text` places a label at a point in this space and has no rotation to follow one with.

There is an `x_label("")` in that sentence and no `y_label("")` beside it, and the asymmetry is the whole of how these plots are labeled. The radial axis carries the depth the transform computed, and nobody reads a level off a scale, so it draws no ticks, no rings and no name of its own. That is decided from what the axis carries rather than asked for, the way a pie is recognized from its bindings. The measure keeps its guide, because an amount around a circle is a quantity in the data's own units, and the empty string is what removes *that* label.

33.14 What a tree lets the grammar check

A hierarchy has a rule that rectangles do not: every child lies inside its parent, and children of one parent do not overlap. Handed the four numbers directly the engine cannot know that, and will draw a wrong sunburst and say nothing. Handed the *tree*, it can, and one case is worth meeting because it is a real ambiguity rather than a mistake:

```
mixed <- data.frame(group = c("Housing", "Housing"),
                    item  = c(NA, "Rent"),
                    amount = c(50, 980))
render_svg(data(mixed) + x(amount) + zone * partition(group, item) + polar())
```

Error:

```
! gog: `Housing` has a value of its own *and* children with values, and those
  ↳ two readings draw different pictures: either its arc is the children's
  ↳ total (its own number already counted among them), or its own number sits
  ↳ beside them and widens it. gog will not pick - the arithmetic is the
  ↳ accounts', not the grammar's. Put every number on a **leaf** and let the
  ↳ parents be the sums, which is what a partition computes.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

Housing has a number of its own *and* a child with a number, and those two readings draw different pictures: either its arc is the child's total, or its own 50 sits beside the 980 and widens it. Plotly answers it with a setting called **branchvalues**. gog does not choose for you, because that arithmetic belongs to your table rather than to the grammar. Put every number on a leaf and let the parents be the sums, which is what a partition computes.

The other refusals are the same rule in smaller cases. A level must be a **name** that a branch shares, so a numeric column is refused and sent to **x()**, where a measurement belongs. A branch may stop early but may not skip a level, since nothing below a stop can be reached. And a mark with no reading for a region is sent to **zone** or **text**.

One shape of input stays outside this atom on purpose. A hierarchy written as columns, one level or several, is what **partition** reads and what **faceting** reads too. A tree of arbitrary depth arriving as a parent column and a child column is the shape a *network* has, and **Network** is where that shape is read: **layout(parent, child)** places it, in a space with its own rules. Each family answers the question it is built for.

34 Nest

How much of the whole does each part take? A share of a whole has to be carried by something you can see. Flat, it is a **length**: a segment's height as a fraction of the column it sits in. Bent into a circle, it is an **angle**, and the column becomes a pie. There is a third answer, and it is the one that fits the most categories into the least space: **area**.

That is what `nest()` draws. Each row's measure becomes the size of a region, and the regions fill the panel with no gaps. Dividing a panel that way is a **packing**. The plot has a name, **treemap**, and as with the pie there is no atom that spells it.

That name has an exact and ordinary origin. Ben Shneiderman's 80-megabyte hard disk was full in 1990, and a directory listing could not tell him what was filling it. So he wrote a display that packs a rectangle with nested rectangles, each sized by what it holds (Shneiderman, 1992). It is now the usual way to draw a share of a whole when there are more categories than a pie has room for. That display is the one this chapter teaches.

34.1 One sentence, three spaces

Here is the population of each continent as a single column, divided into its parts:

R

```
data(gapminder_2007) + bar * sum * stack + y(population) + color(continent) +  
  title("A column divided")
```

Python

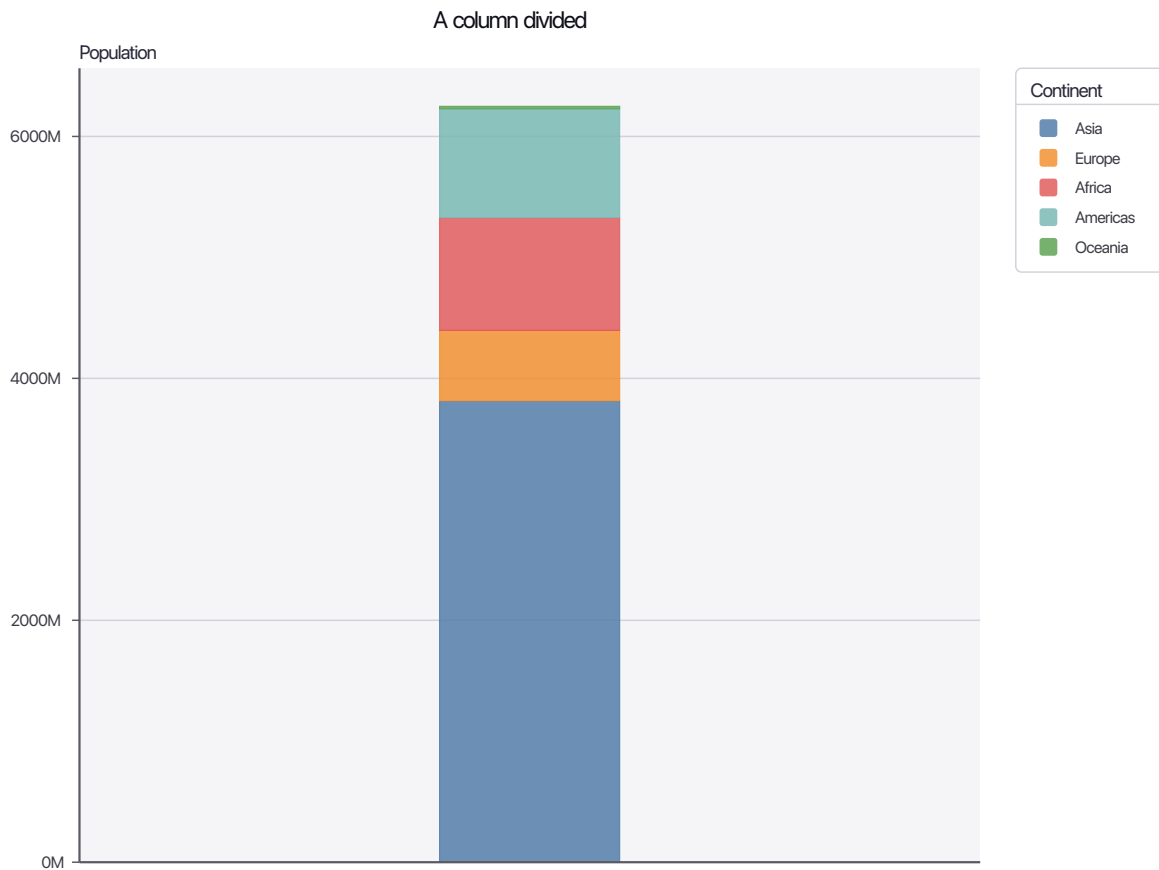
```
(data(gapminder_2007) + bar * sum * stack + y(col.population) +  
  ↪ color(col.continent) +  
  title("A column divided"))
```

Julia


```
data(gapminder_2007) + bar * sum * stack + y(:population) +
  color(:continent) + title("A column divided")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, sum, stack), y(col.population),
  color(col.continent), title("A column divided"))
```



“Given gapminder 2007: bars derived by sum and stack, y is population, color by continent.”

Now the same sentence in the circle. The column’s height becomes the whole turn, and each segment’s share of the height becomes its slice’s share of the circle:

R

```
data(gapminder_2007) + bar * sum * stack + y(population) + color(continent) +  
  polar() + title("The same shares, as angles")
```

Python

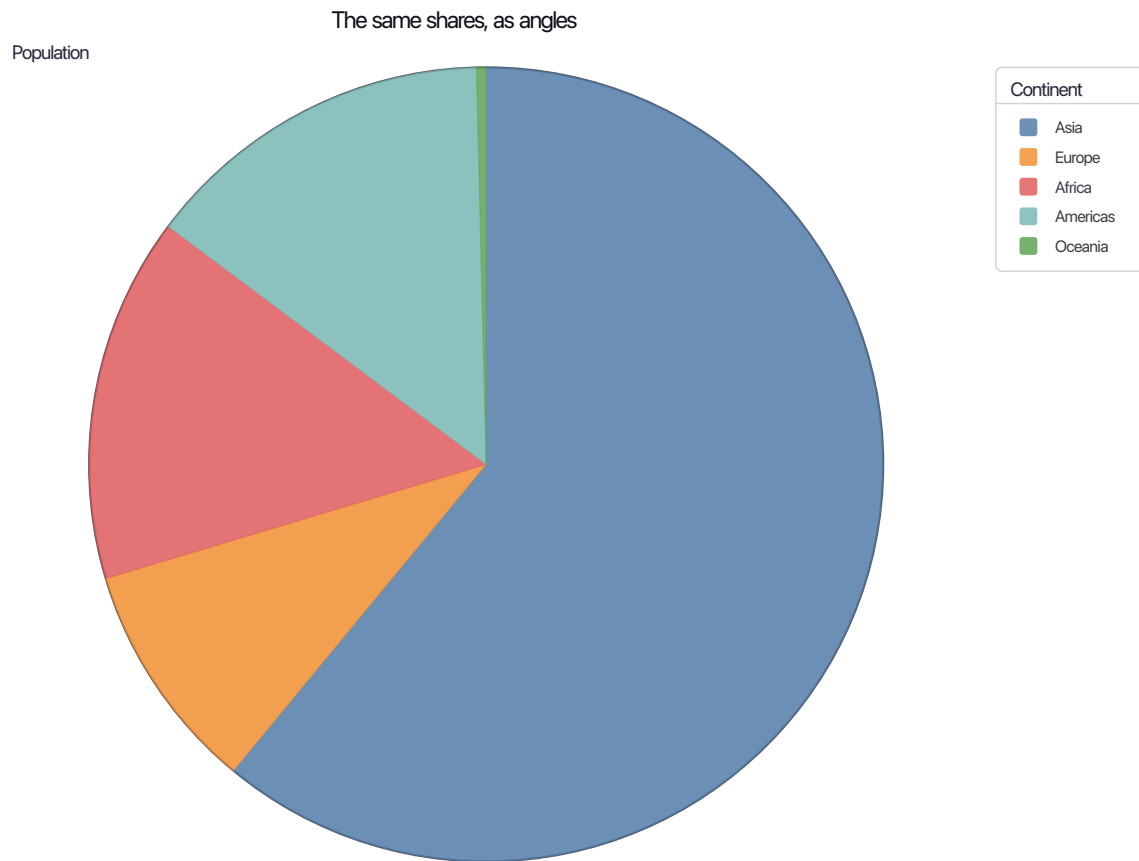
```
(data(gapminder_2007) + bar * sum * stack + y(col.population) +  
  ↪ color(col.continent) +  
  polar() + title("The same shares, as angles"))
```

Julia

```
data(gapminder_2007) + bar * sum * stack + y(:population) +  
  color(:continent) + polar() + title("The same shares, as angles")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, sum, stack), y(col.population),  
  color(col.continent), polar(), title("The same shares, as angles"))
```



“Given gapminder 2007: bars derived by sum and stack, y is population, color by continent, in polar.”

And the same sentence again, packed (one word is gone, and the paragraph below says why):

R

```
data(gapminder_2007) + bar * sum + y(population) + color(continent) +
  nest() + title("The same shares, as areas")
```

Python

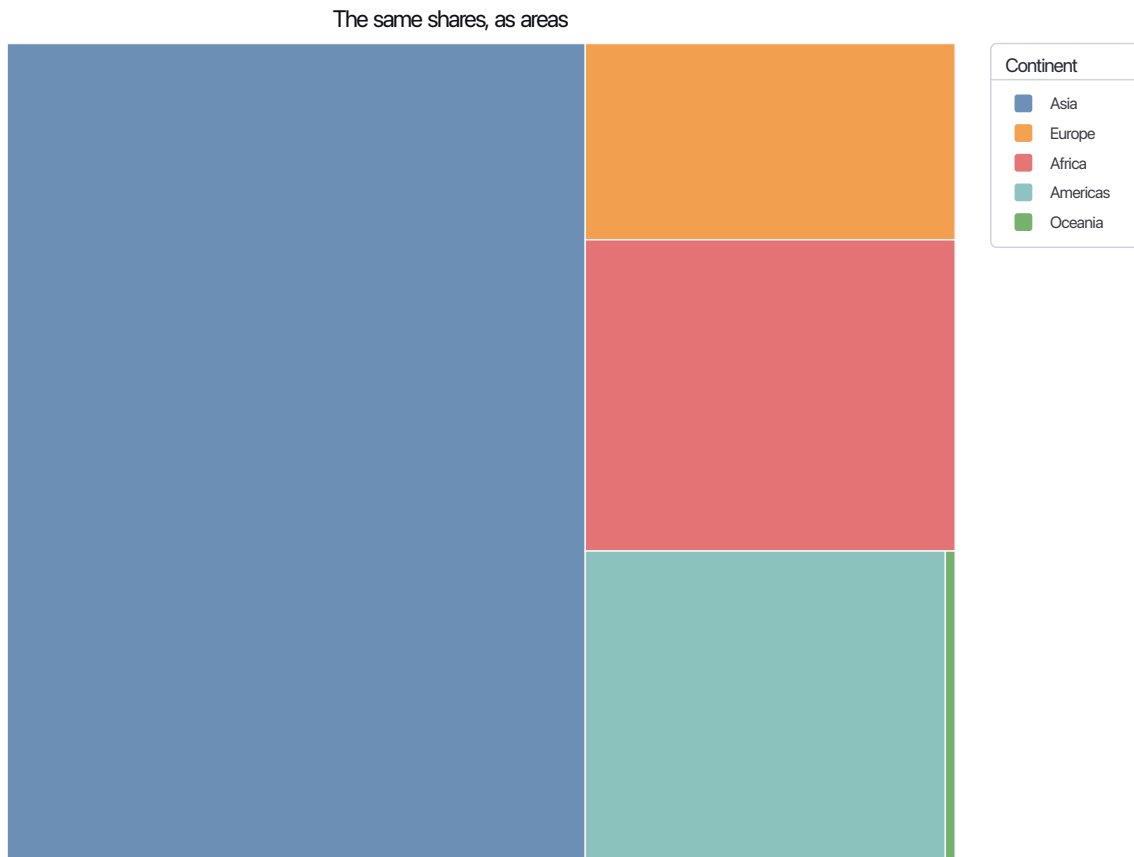
```
(data(gapminder_2007) + bar * sum + y(col.population) + color(col.continent)
  ↪ +
  nest() + title("The same shares, as areas"))
```

Julia

```
data(gapminder_2007) + bar * sum + y(:population) + color(:continent) +
  nest() + title("The same shares, as areas")
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, sum), y(col.population),
  color(col.continent), nest(), title("The same shares, as areas"))
```



“Given gapminder 2007: bars derived by sum, y is population, color by continent, in the nest.”

Three pictures, one grammar. Nothing in any of them names a chart: a bar, summed, colored by continent, and then a word for the space. Asia is three fifths of the world’s population in all three, and you can check that against the column, the pie and the treemap in turn.

That missing word is **stack**: the flat and polar forms carry it and the packed form does not. **stack** decides what happens when two marks land in the same place, so it assumes two marks

can share a place. A packing gives every region a place of its own, so there is no collision left to resolve, and `gog` refuses `stack` here rather than accepting a word with nothing to do.

34.2 Why area, when a column already works

Read the column again and look for Oceania. At 0.4% of the world it is about two pixels tall, so a quarter of that height is the border drawn around it. In the packing the same share is a region about six pixels wide, because a column has only its length to divide, and a packing divides both width and height.

Oceania's region is three times as wide as the bar was tall, and the share it reports is unchanged. With five parts a packing adds little room. It adds more as the parts multiply, because a column divides in one direction and a panel divides in two. A packing is for the case where there are *many* shares and the question is which of them are large. A column has one fixed length to divide, so it stops showing new categories long before a packing does.

Area has one weakness, the same one the pie has: two lengths can be compared exactly, and two areas cannot. If the comparison has to be precise, draw the flat column.

34.3 Two levels, from a position

A packing can nest one split inside another, and binding a position is what adds that second level. Each category gets a region sized by its own total, and the rows inside it are packed into that region:

R

```
data(gm_eras) + bar * sum + x(continent) + y(population) + color(era) +  
  nest() + title("Two eras, inside each continent")
```

Python

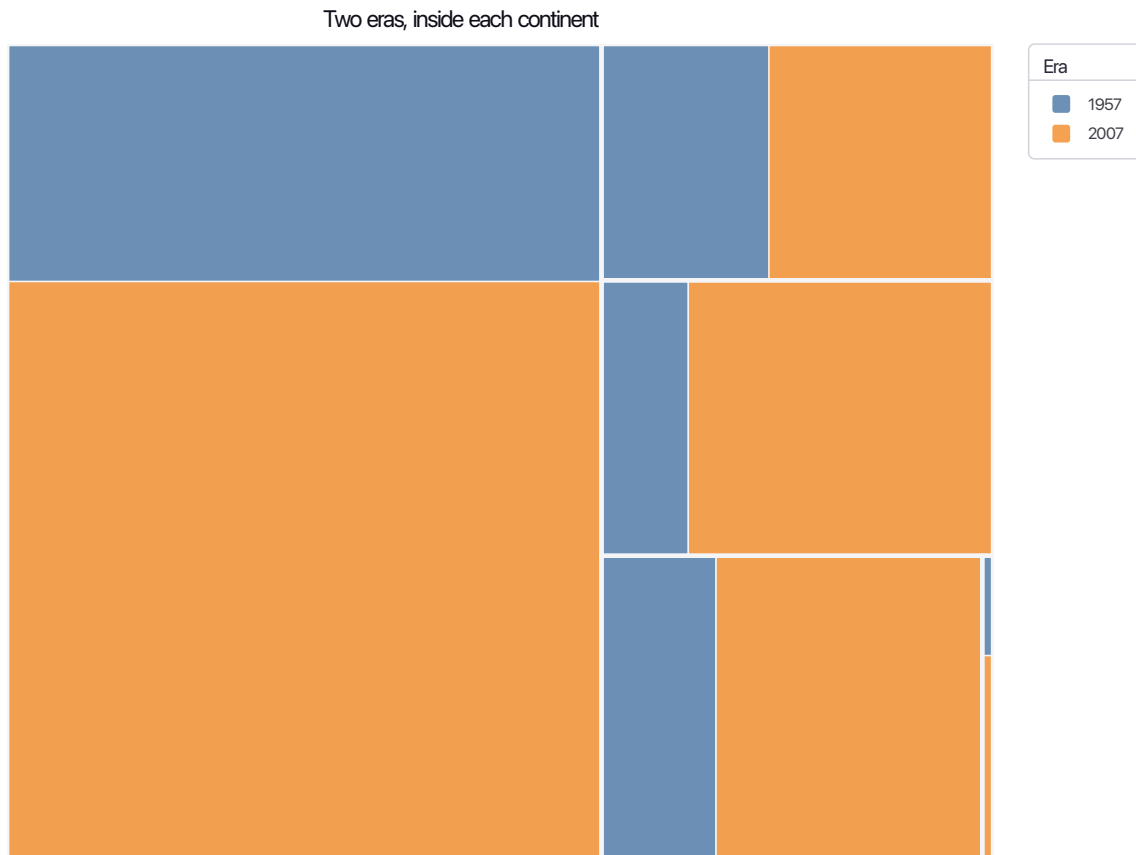
```
(data(gm_eras) + bar * sum + x(col.continent) + y(col.population) +  
  ↪ color(col.era) +  
  nest() + title("Two eras, inside each continent"))
```

Julia

```
data(gm_eras) + bar * sum + x(:continent) + y(:population) + color(:era) +  
  nest() + title("Two eras, inside each continent")
```

JavaScript

```
plot(data(gm_eras), layer(bar, sum), x(col.continent), y(col.population),  
     color(col.era), nest(), title("Two eras, inside each continent"))
```



“Given the gapminder eras: bars derived by sum, x is continent, y is population, color by era, in the nest.”

Each continent is a region sized by its total across both eras, and inside it the two eras are packed against each other. The heavier lines are the continents; the thinner ones divide what is inside them. Africa’s region is the most uneven: more than three quarters of it is the 2007 part.

The nesting is not new vocabulary. Wilkinson makes the point directly (Wilkinson, 2005): a **dimension hierarchy** (his name for levels inside levels) *is* a nested facet. So **x** here is doing what the outer level of a facet does, with the panel divided by share instead of into equal boxes.

The inner level comes from `color`, and the legend is what decodes it. So the second level needs few enough values to fit in the legend beside the plot. Countries inside continents, the treemap most often printed, would be a legend of 142 entries, and [Naming the regions](#) writes each name inside its own region instead.

34.4 What a packed panel does not have

Look for the axes. There are none, and that is what defines the space rather than a setting you can change.

A flat panel's horizontal direction *means* something: it is a variable, and a tick is a place on it. In a packing neither direction is a variable. Wilkinson states it exactly: the two dimensions of a treemap “have no intrinsic meaning related to the data because they can be reordered without changing the metric properties of the tree” (Wilkinson, 2005). Two regions that touch are not near each other in the data. Moving one to the other side of the panel would not change a single share the plot reports.

So there is nothing to tick and nothing to label, and `gog` refuses the atom that would say otherwise:

```
data(gapminder_2007) + bar * sum + y(population) + color(continent) + nest()
↪ +
  x_label("Population")
```

Error:

```
! gog: `x_label()` names an axis, and a `nest()` plot has none - its two
↪ directions carry no variable and can be reordered without changing what
↪ the plot says. Use `title()` for the plot's own name, and the color
↪ legend to say what the regions are.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

What decodes a packed panel is the color legend, exactly as it decodes a pie.

34.5 Naming the regions

A legend works while there are few enough entries to scan. With more entries it becomes a second thing to read, and 142 countries are far too many. So put the name where the region is. The sentence drops `sum` as well: without it each row is its own region, so every country gets one inside its continent.

R

```
data(gapminder_2007) + bar + x(continent) + y(population) + color(continent)
↪ +
  text + label(country) + style(color = "white") + nest() +
  title("Countries inside continents")
```

Python

```
(data(gapminder_2007) + bar + x(col.continent) + y(col.population) +
↪ color(col.continent) +
  text + label(col.country) + style(color = "white") + nest() +
  title("Countries inside continents"))
```

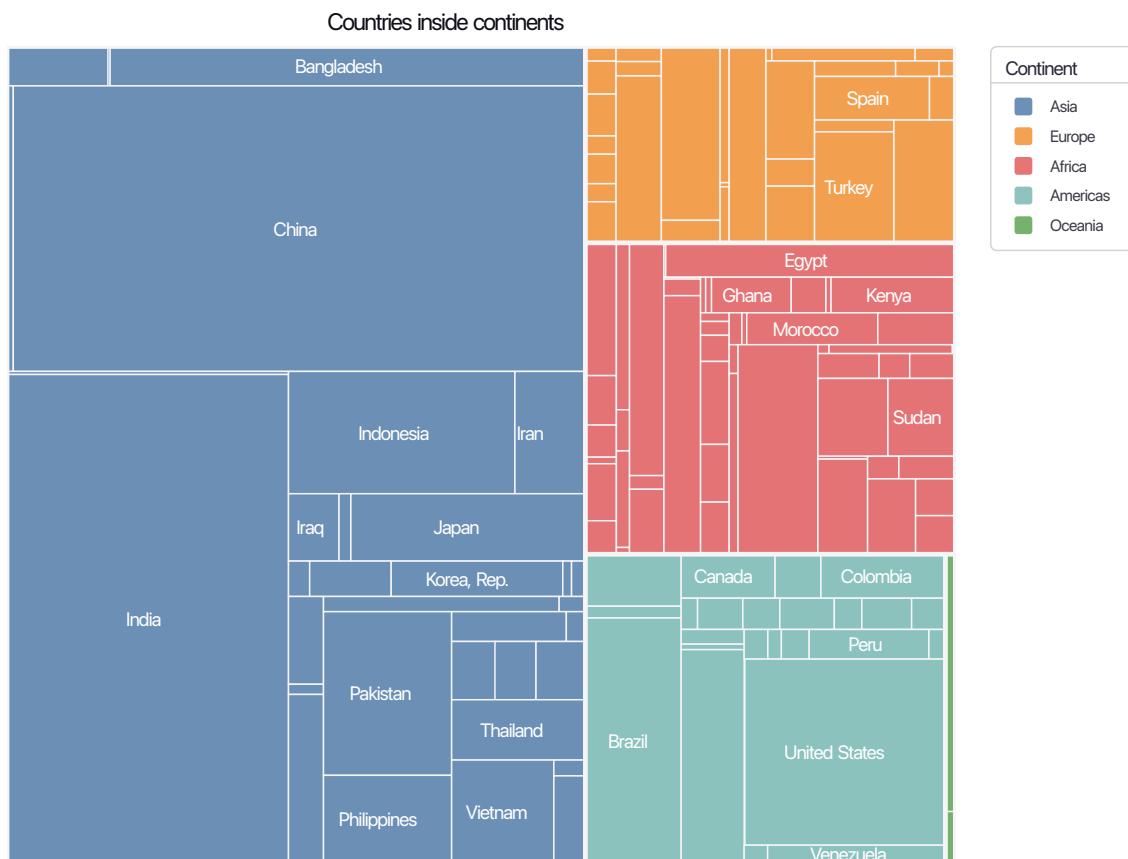
Julia

```
data(gapminder_2007) + bar + x(:continent) + y(:population) +
  color(:continent) + text + label(:country) + style(color = "white") +
  nest() + title("Countries inside continents")
```

JavaScript

```
plot(data(gapminder_2007), bar, x(col.continent), y(col.population),
  color(col.continent), text, label(col.country),
  style({ color: "white" }), nest(), title("Countries inside continents"))
```

gog: 25 of 142 labels are drawn - 116 are wider than the region they name,
 ↪ and one share is too small to have a region at all. The packing drew
 ↪ every share, so the names are what is missing. Fewer categories, a larger
 ↪ plot (`theme(width =, height =)`) or a smaller `style(size =)` fits more
 ↪ of them in.



“Given gapminder 2007: bars, x is continent, y is population, color by continent, and also text, label by country, in the nest.”

Two layers, and the second is the ordinary **text** mark: **label** says which column supplies the string, exactly as it does on a scatter. What it does *not* say is where to put it, because in this space nothing says where a mark goes. A label goes to the center of the region its own row was packed into. That is the region the bar drew, so both marks read one packing and no name is drawn in another country’s region.

Notice what is missing from that sentence: the text layer carries no **x** and no **y** of its own. The plot’s own **x(continent)** and **y(population)** do that work for it. They size the regions, and sizing a region is what places its name. On a flat panel a **text** mark needs two positions before it can be drawn at all. Here it needs a measure and a string, and the packing places it.

Count the names and there are far fewer than 142, which is what the message above the plot is for. Most of the world’s countries get a region narrower than their own name, so a label that does not fit its rectangle is not drawn. The engine says how many names it drew and why the rest are missing, rather than letting you read the named countries as though they were all

of them. A few African countries are named, and most of Africa is not. Every one of those unnamed countries still has a region whose area is exactly its share of the world. The picture is complete and the labeling is not, and the count above the plot is the one you can check against the names.

That message is why the missing names do not mislead you: a treemap that printed only the names that fit, and said nothing, would read as a complete list.

Three things change how many names are drawn, and you have used each of them already: fewer categories, a larger plot with `theme(width =, height =)`, or smaller text with `style(size = )`. None of them is a setting for *labels* in particular, and none of them will make a name fit a region narrower than the word.

34.6 The other refusals

A packing cannot hold a mark that is placed by a position, because it has no positions to place one at:

```
data(gapminder_2007) + point + x(gdp) + y(life) + nest()
```

Error:

```
! gog: `point` is placed by a position, and a packing has none to give it -  
  ↳ its two directions are not axes, and two neighboring regions are not near  
  ↳ each other in the data (Wilkinson §13.3.4.1). Drop `nest()` to draw  
  ↳ `point` flat, or use `bar` or `text`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

An area cannot be negative, so a measure that runs below zero has no packing. On the flat page, a bar below the baseline points down, and the reader can see the sign. A region has no direction to point in, and its parts would no longer sum to the whole.

```
data(cashflow) + bar * sum + y(delta) + color(step) + nest()
```

Error:

```
! gog: `delta` has negative values, and a `nest()` plot turns a measure into  
  ↳ an area, which cannot be negative. Flat, a bar below the baseline  
  ↳ reads as a loss; a region has no direction to run in. Filter or offset  
  ↳ the column, or draw it flat where the sign is readable.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

A log scale is refused too, and the reason is the one worth remembering: a region's size is its share of the total, which is arithmetic on the values themselves. Re-spacing them changes what a distance means, and a packing has no distances. Accepting the scale would draw exactly the same picture, and `gog` refuses a word that changes nothing.

```
data(gapminder_2007) + bar * sum + y(population, scale = "log") +  
  color(continent) + nest()
```

Error:

```
! gog: `y(scale = "log")` with `nest()` - a packed region's size is its  
  ↳ **share of the total**, which is arithmetic on the values themselves, so  
  ↳ a log scale would change nothing about the picture. Drop the scale, or  
  ↳ drop `nest()` to read the values against a log axis.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The packing method is not a choice you make. It is *squarified*, after Bruls, Huizing and van Wijk (Bruls et al., 2000): each group of regions is placed so its rectangles are as close to square as they can be. The simpler method divides one direction at a time, which makes long thin regions, and a long thin area is the hardest kind to judge. `gog` offers no choice here, because a second layout would add a word that draws nothing new.

34.7 Sorting the regions

Published treemaps usually sort by size, largest first, because the packing produces squarer rectangles that way. `gog` does not sort them for you. The regions follow the order the categorical axis already has, so the sort is something you write.

R

```
data(gm_eras) + bar * sum + x(continent) + y(population) + color(era) +  
  order(population, desc = TRUE) + nest() + title("Largest continent first")
```

Python

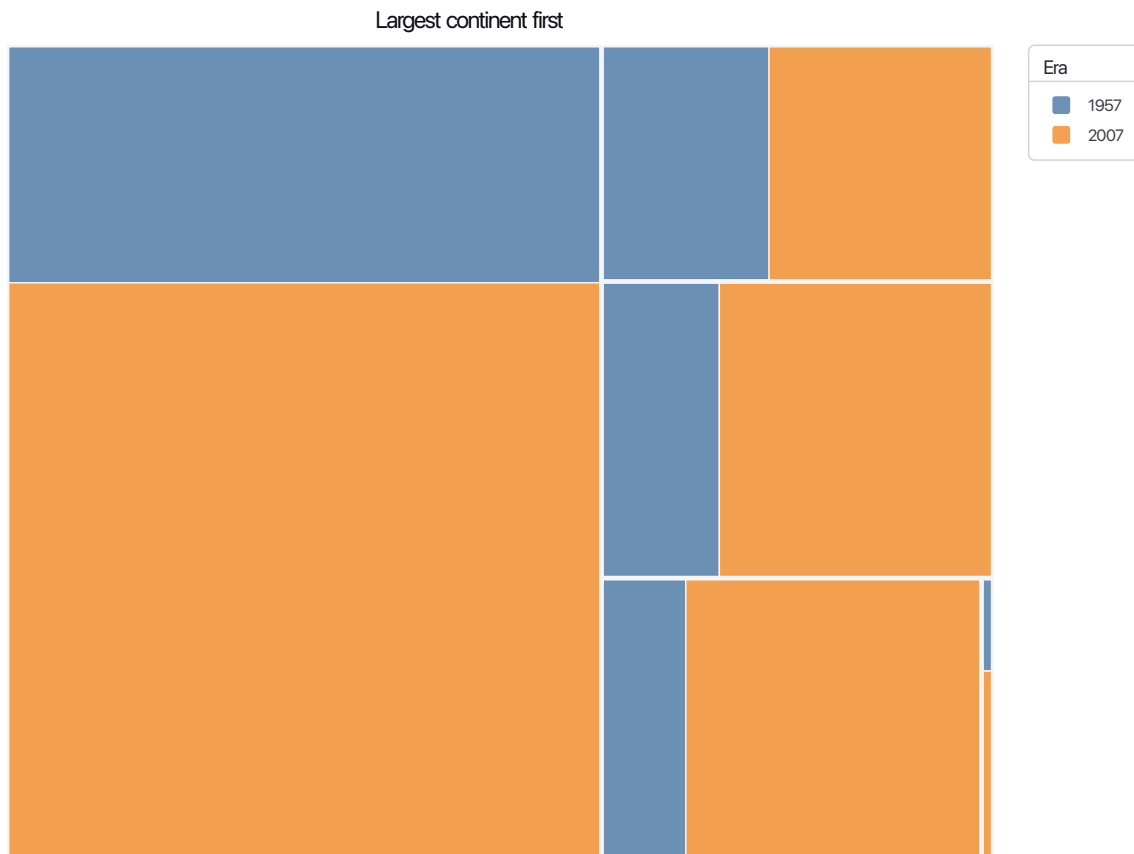
```
(data(gm_eras) + bar * sum + x(col.continent) + y(col.population) +  
  ↳ color(col.era) +  
  order(col.population, desc = True) + nest() + title("Largest continent  
  ↳ first"))
```

Julia

```
data(gm_eras) + bar * sum + x(:continent) + y(:population) + color(:era) +
  order(:population, desc = true) + nest() +
  title("Largest continent first")
```

JavaScript

```
plot(data(gm_eras), layer(bar, sum), x(col.continent), y(col.population),
  color(col.era), order(col.population, { desc: true }), nest(),
  title("Largest continent first"))
```



“Given the gapminder eras: bars derived by sum, x is continent, y is population, color by era, ordered by population, largest first, in the nest.”

That is the same `order()` that reorders a bar chart, doing the same job here. If the packing sorted on its own, `order()` would change nothing in this space, and the atom would promise more than it does.

`order()` sorts a categorical *position* axis, so a plot needs one before it can be sorted. The one-level treemap above binds no position at all, and there the atom has nothing to sort:

```
data(gapminder_2007) + bar * sum + y(population) + color(continent) +  
  order(population, desc = TRUE) + nest()
```

Error:

! gog: `order(population)` sorts a **category position axis**, and this

→ plot has none - so there is nothing for it to put in order. Bind a

→ category to `x` or `y`, or, if what you meant was the order of a color

→ split, set the column's factor levels where the data lives.

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

That refusal is not the packed space's. It is `order()`'s, and it applies in every space.

35 Map

Where did it happen? Some data answers that with a place on the earth, and a place is two numbers. **Longitude** says how far east or west, and **latitude** how far north or south. Both are angles, measured in degrees.

Every plot in this book has used two numbers for position. So a map is not a new kind of picture. It is the same sentence you already write, drawn in a space that treats the two numbers as angles on a sphere.

`quakes_fiji` records earthquakes near Fiji, one row per quake:

Table 35.1: `quakes_fiji`: first 5 of 1000 rows

east	north	elevation	magnitude	slab
181.62	-20.42	-562	4.8	540-630 km
181.03	-20.62	-650	4.2	630-720 km
184.10	-26.00	-42	5.4	0-90 km
181.66	-17.97	-626	4.1	540-630 km
181.96	-20.42	-649	4.0	630-720 km

R

```
data(quakes_fiji) + point + x(east) + y(north) + map()
```

Python

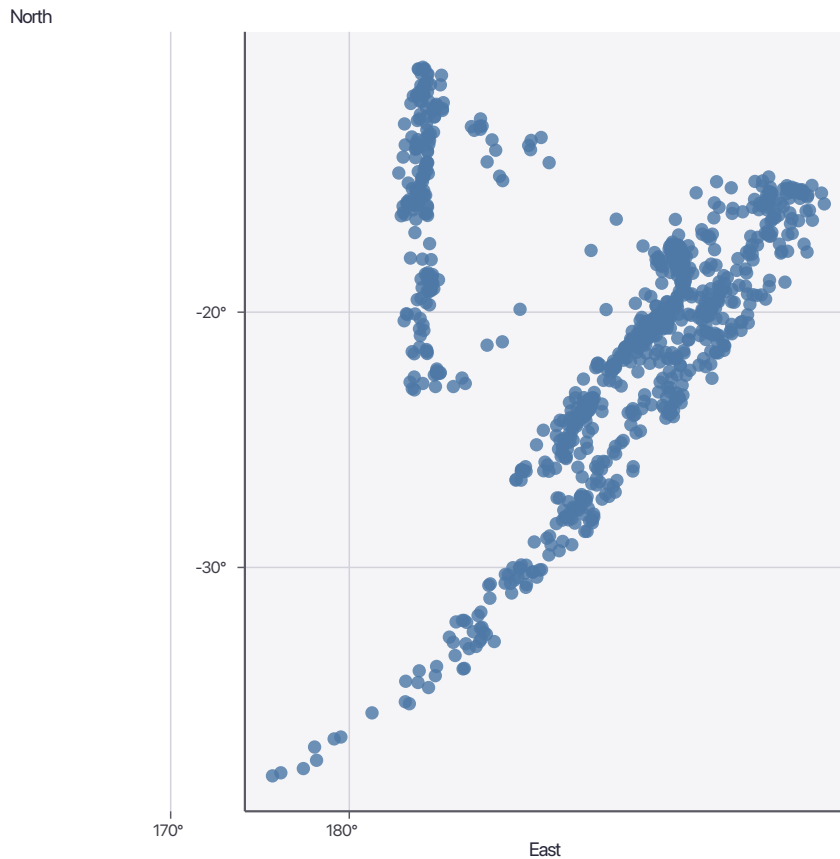
```
data(quakes_fiji) + point + x(col.east) + y(col.north) + map()
```

Julia

```
data(quakes_fiji) + point + x(:east) + y(:north) + map()
```

JavaScript

```
plot(data(quakes_fiji), point, x(col.east), y(col.north), map())
```



“Given quakes Fiji: points, x is east, y is north, in the map space.”

That is a thousand earthquakes recorded near Fiji. x is longitude and y is latitude, and `map()` is the whole of what makes it a map. The axes are labeled in degrees, because that is what the columns hold.

In the web edition, `+` magnifies a crowded part, `-` makes the picture smaller, and `fit` shows the whole map. A map needs that more than most plots do, because places stay crowded however far you zoom in. Zooming is a **view**: it magnifies the picture and changes nothing the plot claims.

35.1 Flattening the sphere loses something

The earth is round and the page is flat. No projection can flatten a sphere without distorting it, and the choice is not between a good projection and a bad one. You choose which distortion you accept.

Two properties matter most: a region's area, and a small shape's form. A projection can keep one of them, never both. `preserve` says which:

R

```
(data(world_borders) + path + x(lon) + y(lat) + group(piece) +  
  map(preserve = "area") + title("preserve = \"area\"")) /  
(data(world_borders) + path + x(lon) + y(lat) + group(piece) +  
  map(preserve = "angle") + title("preserve = \"angle\""))
```

Python

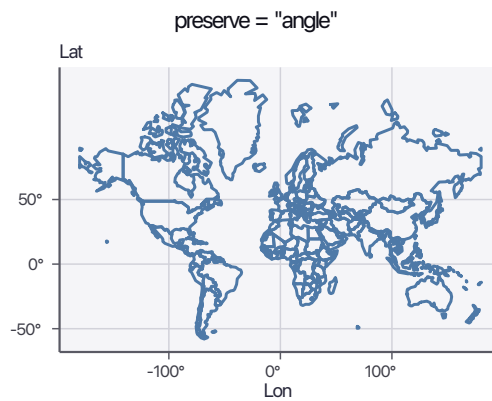
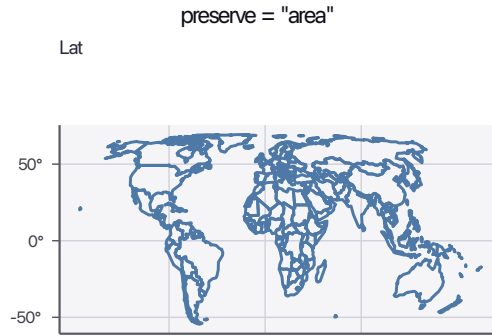
```
((data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +  
  map(preserve = "area") + title("preserve = \"area\"")) /  
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +  
  map(preserve = "angle") + title("preserve = \"angle\"")))
```

Julia

```
(data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +  
  map(preserve = "area") + title("preserve = \"area\"")) /  
(data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +  
  map(preserve = "angle") + title("preserve = \"angle\""))
```

JavaScript

```
below(plot(data(world_borders), path, x(col.lon), y(col.lat),  
  group(col.piece), map({ preserve: "area" })),  
  title("preserve = \"area\"")),  
plot(data(world_borders), path, x(col.lon), y(col.lat),  
  group(col.piece), map({ preserve: "angle" })),  
  title("preserve = \"angle\""))
```

“Given the world borders: paths, x is lon, y is lat, grouped by piece, in the map space preserving area, above the same preserving angle.”

Look at Greenland, at the top left of each map, and compare it with Africa. In the first, Greenland is small, which is correct: Africa is about fourteen times larger. In the second, Greenland has grown until it looks as large as Africa. The second map keeps every small shape’s true form and gets the sizes wrong.

Both plots name `lon` on `x`, so `/` gives them one axis and one width. The projection sets each height, and the two heights differ. Keeping every small shape’s form is what stretches the second one north and south.

`preserve = "area"` is the default, because a map is usually read by comparing how much space something takes. A projection that inflates the far north tells the reader something false about every number placed there.

The world map above is a table like any other, and the next section reads it column by column.

35.2 Drawing the outlines

The table has one row for every point along a coastline or a border:

```
knitr::kable(head(world_borders, 4))
```

lon	lat	country	continent	piece
66.519	37.363	Afghanistan	Asia	p001
69.196	37.151	Afghanistan	Asia	p001
70.807	38.486	Afghanistan	Asia	p001
71.845	36.738	Afghanistan	Asia	p001

`path` connects rows in the order the table gives them, which is exactly what tracing a coastline needs. `group(piece)` says where one outline ends and the next begins. A ring is one closed loop of points, ending where it returns to its starting point. A country is not always a single ring. An island is a ring of its own, and a country that encloses another needs a second ring for the hole. So `piece` counts rings rather than countries.

Without that split, `path` would join the last point of one country to the first point of the next, and draw a line across the ocean between them.

35.3 The second layer is the data

An outline on its own is only a background. It becomes a plot when something is drawn on top of it, and that is ordinary layering. The outline is one `data()` and one mark, and the earthquakes are another.

R

```
data(world_borders) + path + x(lon) + y(lat) + group(piece) +  
  style(color = "#c9ccd6") +  
  data(quakes_fiji) + point + x(east) + y(north) +  
  style(color = "tomato", opacity = 0.35, size = 3) +  
  map() + title("A thousand earthquakes, and where Fiji is")
```

Python

```
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +
  style(color = "#c9ccd6") +
  data(quakes_fiji) + point + x(col.east) + y(col.north) +
  style(color = "tomato", opacity = 0.35, size = 3) +
  map() + title("A thousand earthquakes, and where Fiji is"))
```

Julia

```
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +
  style(color = "#c9ccd6") + data(quakes_fiji) + point + x(:east) +
  y(:north) + style(color = "tomato", opacity = 0.35, size = 3) + map() +
  title("A thousand earthquakes, and where Fiji is")
```

JavaScript

```
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),
  style({ color: "#c9ccd6" }), data(quakes_fiji), point, x(col.east),
  y(col.north), style({ color: "tomato", opacity: 0.35, size: 3 }), map(),
  title("A thousand earthquakes, and where Fiji is"))
```

A thousand earthquakes, and where Fiji is



“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given quakes Fiji: points, x is east, y is north, in the map space.”

The earthquakes now have a coastline around them, so a reader can see where they are. Nothing in the sentence is new: two tables, two marks, and the second `data()` binds to the mark that follows it.

35.4 A quantity goes on a channel

A flat plot, a cube and a polar plot all leave you an axis to measure along. A bar stands on one axis and measures its length along the other. A map has no axis left, because longitude and latitude use both.

So a quantity moves to a **channel**. Size each place by it:

R

```
data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  data(quakes_fiji) + point + x(east) + y(north) + size(magnitude) +
  style(color = "tomato", opacity = 0.4) +
  map() + title("Sized by magnitude")
```

Python

```
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +
  style(color = "#c9ccd6") +
  data(quakes_fiji) + point + x(col.east) + y(col.north) +
  ↪ size(col.magnitude) +
  style(color = "tomato", opacity = 0.4) +
  map() + title("Sized by magnitude"))
```

Julia

```
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +
  style(color = "#c9ccd6") + data(quakes_fiji) + point + x(:east) +
  y(:north) + size(:magnitude) + style(color = "tomato", opacity = 0.4) +
  map() + title("Sized by magnitude")
```

JavaScript

```
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),
  style({ color: "#c9ccd6" }), data(quakes_fiji), point, x(col.east),
  y(col.north), size(col.magnitude),
  style({ color: "tomato", opacity: 0.4 }), map(),
  title("Sized by magnitude"))
```



“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given quakes Fiji: points, x is east, y is north, size by magnitude, in the map space.”

That plot has a name, the **proportional-symbol map**, and no single atom draws it. It is point with `size` bound, in this space.

Color works the same way, and takes a category rather than a quantity:

R

```
data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  color(continent) + map() +
  title("Each continent in its own color")
```

Python

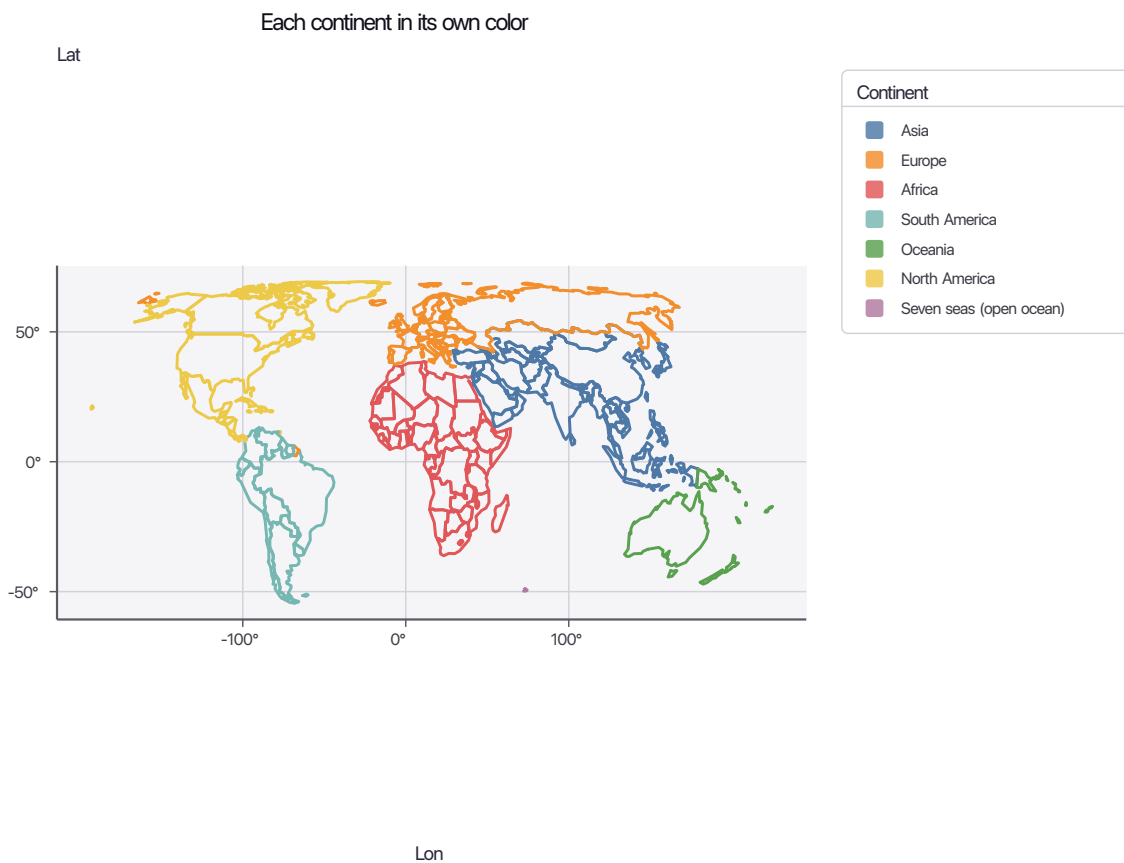
```
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +
  color(col.continent) + map() +
  title("Each continent in its own color"))
```

Julia

```
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +
  color(:continent) + map() + title("Each continent in its own color")
```

JavaScript

```
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),
  color(col.continent), map(), title("Each continent in its own color"))
```



“Given the world borders: paths, x is lon, y is lat, grouped by piece, color by continent, in the map space.”

35.5 Meridians and parallels

A **rule** draws a line at one position and spans the axis it does not name. In a flat space that is a reference line. Here, a rule at one latitude is a **parallel**, and a rule at one longitude is a **meridian**. The plot below draws one parallel, the equator:

R

```
lines_of <- data.frame(lat = 0)

data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  data(lines_of) + rule + y(lat) + style(color = "#d4674f") +
  map() + title("The equator")
```

Python

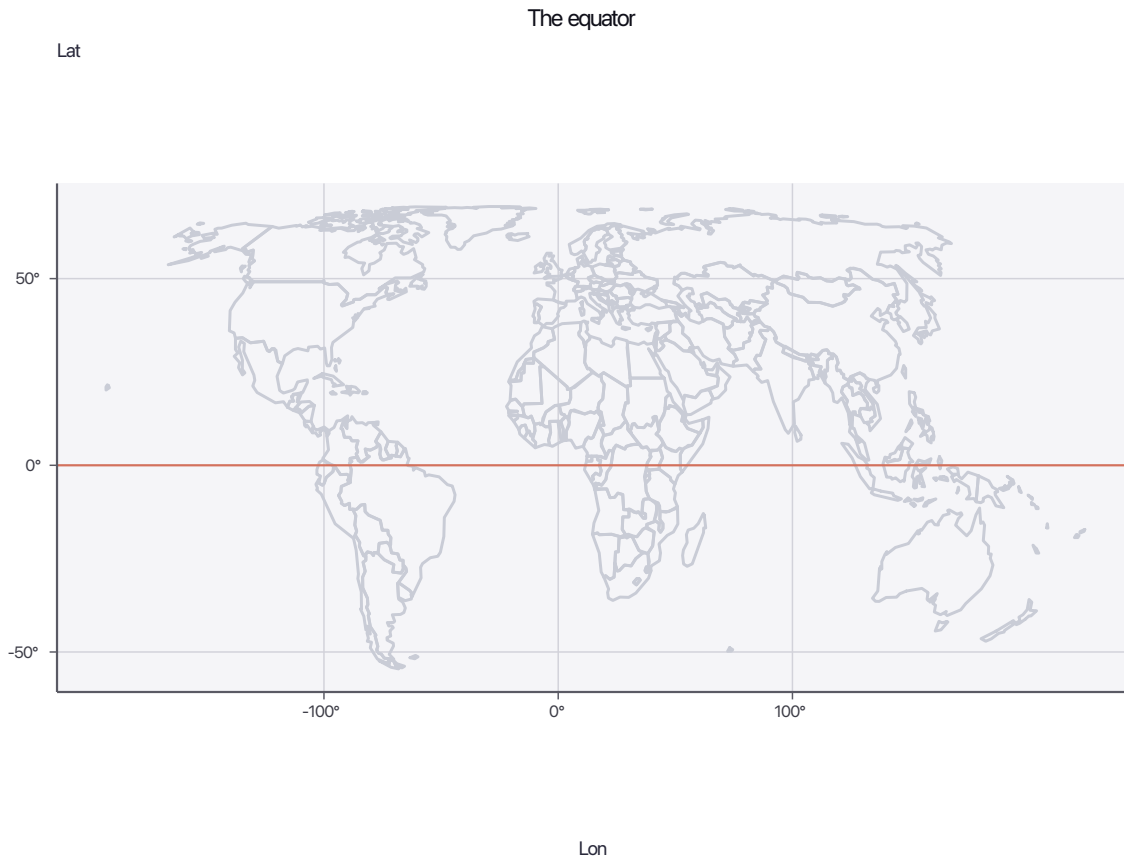
```
lines_of = {"lat": [0]}
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +
  style(color = "#c9ccd6") +
  data(lines_of) + rule + y(col.lat) + style(color = "#d4674f") +
  map() + title("The equator"))
```

Julia

```
lines_of = (lat = [0],)
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +
  style(color = "#c9ccd6") + data(lines_of) + rule + y(:lat) +
  style(color = "#d4674f") + map() + title("The equator")
```

JavaScript

```
const lines_of = { lat: [0] };
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),
  style({ color: "#c9ccd6" }), data(lines_of), rule, y(col.lat),
  style({ color: "#d4674f" }), map(), title("The equator"))
```

“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given the lines of table: a rule, y is lat, in the map space.”

A **rule** takes its position from a column, like every other mark, so the line above comes from a one-row table. That is the same way a reference line is drawn on any other plot.

Nothing was added to the grammar to draw that. The same sentence that puts a reference line on a scatter puts the equator on a map. The space gives the mark a new meaning, and the mark itself is unchanged.

35.6 Naming places

`text` writes a string at a position, and a position here is a place:

R

```

capitals <- data.frame(
  lon = c(-0.13, 139.69, -74.01, 151.21, 18.42),
  lat = c(51.51, 35.69, 40.71, -33.87, -33.92),
  name = c("London", "Tokyo", "New York", "Sydney", "Cape Town"))

data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  data(capitals) + point + x(lon) + y(lat) +
  style(color = "#3d5a80", size = 5) +
  data(capitals) + text + x(lon) + y(lat) + label(name) +
  style(nudge = "up", size = 11) +
  map() + title("Five cities")

```

Python

```

capitals = {"lon": [-0.13, 139.69, -74.01, 151.21, 18.42], "lat": [51.51,
↪ 35.69, 40.71, -33.87, -33.92], "name": ["London", "Tokyo", "New York",
↪ "Sydney", "Cape Town"]}
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +
  style(color = "#c9ccd6") +
  data(capitals) + point + x(col.lon) + y(col.lat) +
  style(color = "#3d5a80", size = 5) +
  data(capitals) + text + x(col.lon) + y(col.lat) + label(col.name) +
  style(nudge = "up", size = 11) +
  map() + title("Five cities"))

```

Julia

```

capitals = (lon = [-0.13, 139.69, -74.01, 151.21, 18.42], lat = [51.51,
↪ 35.69, 40.71, -33.87, -33.92], name = ["London", "Tokyo", "New York",
↪ "Sydney", "Cape Town"],)
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +
  style(color = "#c9ccd6") + data(capitals) + point + x(:lon) + y(:lat) +
  style(color = "#3d5a80", size = 5) + data(capitals) + text + x(:lon) +
  y(:lat) + label(:name) + style(nudge = "up", size = 11) + map() +
  title("Five cities")

```

JavaScript

```
const capitals = { lon: [-0.13, 139.69, -74.01, 151.21, 18.42], lat: [51.51,
↪ 35.69, 40.71, -33.87, -33.92], name: ["London", "Tokyo", "New York",
↪ "Sydney", "Cape Town"] };
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),
style({ color: "#c9ccd6" }), data(capitals), point, x(col.lon),
y(col.lat), style({ color: "#3d5a80", size: 5 }), data(capitals), text,
x(col.lon), y(col.lat), label(col.name),
style({ nudge: "up", size: 11 }), map(), title("Five cities"))
```



“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given the capitals: points, x is lon, y is lat; then given the capitals: text, x is lon, y is lat, label by name, in the map space.”

35.7 What a map refuses

A mark that measures along an axis has no axis here. `gog` says so rather than drawing something that looks like a map and is not one:

```
data(quakes_fiji) + bar * count + x(east) + map()
```

Error:

```
! gog: `bar` measures along an axis, and a `map()` plot has none to spare -
  ↳ longitude and latitude use both. Drop `map()` to draw `bar` flat, or use
  ↳ `point`, `text`, `path`, `rule`, or `zone`. To carry a quantity on a map,
  ↳ put it on a channel instead of an axis: `point + size(<column>)` sizes
  ↳ each place by it, and `color(<column>)` shades it.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The message names the marks that do draw, and it says what to do instead. Put the quantity on `size`, as the magnitude map above does, or on `color`.

A position on a map is a place, so a category cannot be one:

```
data(gapminder_2007) + point + x(continent) + y(life) + map()
```

Error:

```
! gog: `x(continent)` is a category, and a `map()` position is a place - `x`
  ↳ is longitude and `y` is latitude, both in degrees. A category has
  ↳ neither. Give the map the coordinates and put `continent` on a channel
  ↳ that decodes it: `color(continent)`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

And the panel's shape belongs to the projection:

```
data(quakes_fiji) + point + x(east) + y(north) + map() + theme(ratio = 1)
```

Error:

```
! gog: `theme(ratio = )` shapes a panel, and a `map()` panel is shaped by the
  ↳ projection - an equal-area map stretched to fit a box is no longer
  ↳ equal-area, though it still looks like a map. Drop it. To change how much
  ↳ of the page the plot takes, use `theme(width =, height =)`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

An equal-area map stretched to fit a box is no longer equal-area, though it still looks like a map. That is the kind of error a reader cannot see, so the engine refuses it instead of accepting the number and ignoring it.

35.8 Where Mercator stops

`preserve = "angle"` is the **Mercator** projection, the one most web maps use. It sends the poles infinitely far away, so it has to stop before it reaches them. It stops at 85.05 degrees. A row beyond that is drawn at the edge, and **gog** reports how many:

R

```
far_north <- data.frame(
  lon = c(0, 15, -45, 10),
  lat = c(60, 78, 83, 89))

data(far_north) + point + x(lon) + y(lat) + map(preserve = "angle")
```

Python

```
far_north = {"lon": [0, 15, -45, 10], "lat": [60, 78, 83, 89]}
data(far_north) + point + x(col.lon) + y(col.lat) + map(preserve = "angle")
```

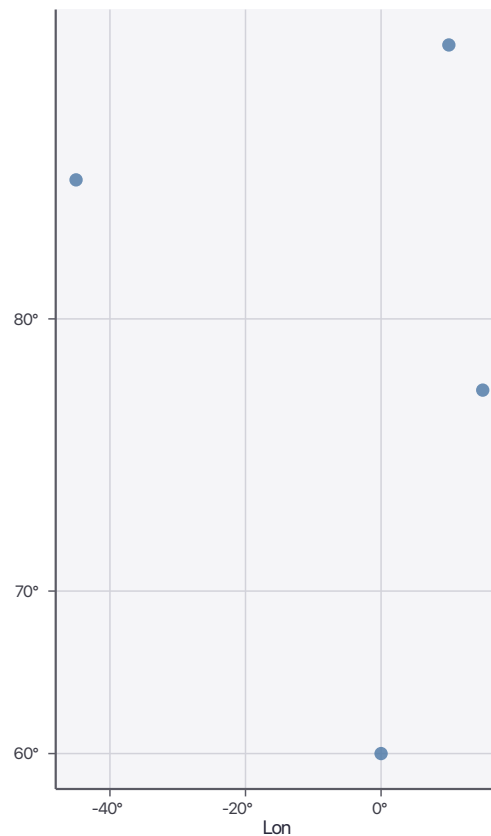
Julia

```
far_north = (lon = [0, 15, -45, 10], lat = [60, 78, 83, 89],)
data(far_north) + point + x(:lon) + y(:lat) + map(preserve = "angle")
```

JavaScript

```
const far_north = { lon: [0, 15, -45, 10], lat: [60, 78, 83, 89] };
plot(data(far_north), point, x(col.lon), y(col.lat),
  map({ preserve: "angle" }))
```

Lat



Read the message above the plot. One row sits beyond the limit and was drawn at the edge instead of where it belongs. Nothing was dropped, because dropping it would misreport how many places there are. `preserve = "area"` reaches both poles and has no such limit.

35.9 Filling the regions: the choropleth

Every country so far has been drawn as an *outline*. Filling each country by a value is the most familiar kind of map, and it has a name: the **choropleth**.

The mark is **zone**, which shades a region. Everywhere else in this book a zone takes its sides from its own row: a category's slot, **bounds**, **bin** or **density**. A coastline is a fifth way, and it is the first that gives many rows for one shape. So the sentence needs a word for which rows belong to which region, and **group** is that word already:

R

```
data(world_borders) + zone + x(lon) + y(lat) + group(country) +  
  color(continent) + map() + title("Every country, filled by continent")
```

Python

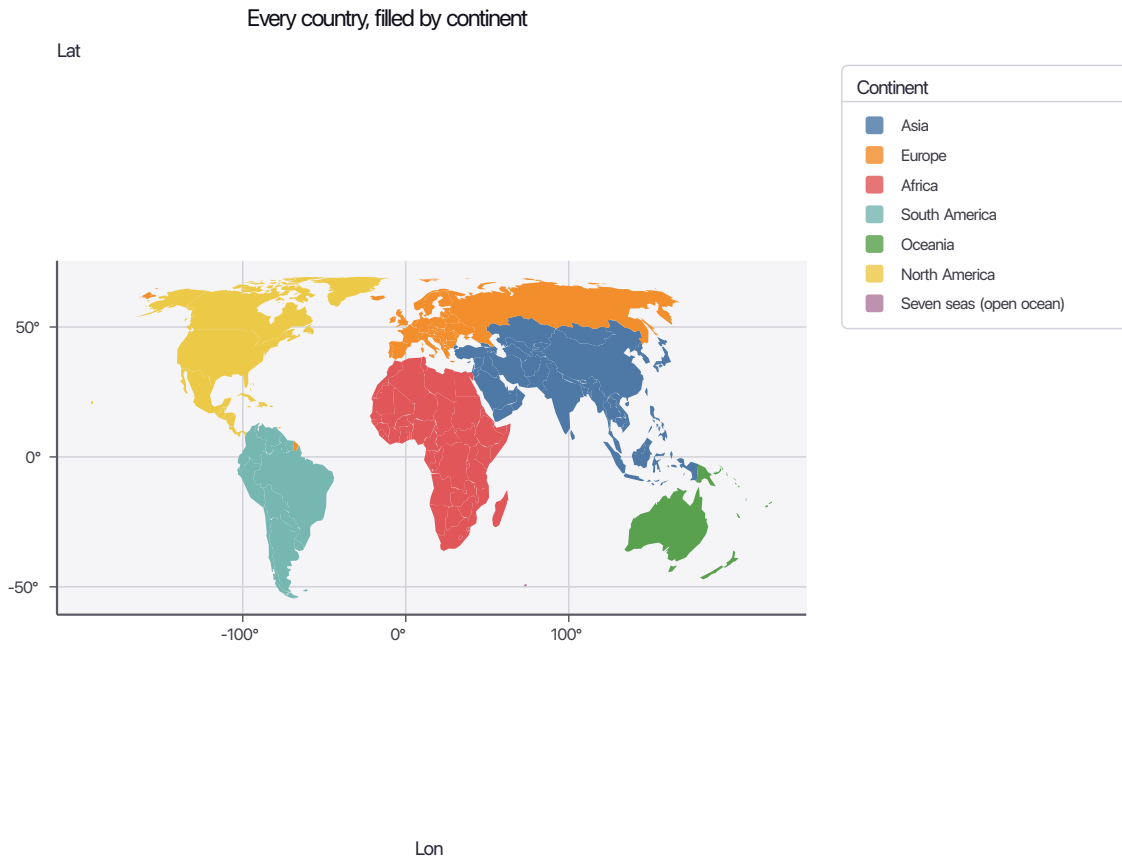
```
(data(world_borders) + zone + x(col.lon) + y(col.lat) + group(col.country) +  
  color(col.continent) + map() + title("Every country, filled by continent"))
```

Julia

```
data(world_borders) + zone + x(:lon) + y(:lat) + group(:country) +  
  color(:continent) + map() + title("Every country, filled by continent")
```

JavaScript

```
plot(data(world_borders), zone, x(col.lon), y(col.lat),  
  group(col.country), color(col.continent), map(),  
  title("Every country, filled by continent"))
```



“Given the world borders: zones, x is lon, y is lat, grouped by country, color by continent, in the map space.”

Compare that with the earlier map that colored each continent. Only two things changed: `path` became `zone`, and `group(piece)` became `group(country)`. A `path` traces one ring at a time, so it groups by ring. A `zone` fills a whole country, so it groups by country.

35.10 A value on the map

`color` reads a number as readily as a category, so the same sentence shades each country by a value. The value comes from a table you already know: `gapminder_2007`, the same 142 countries [A first plot](#) draws.

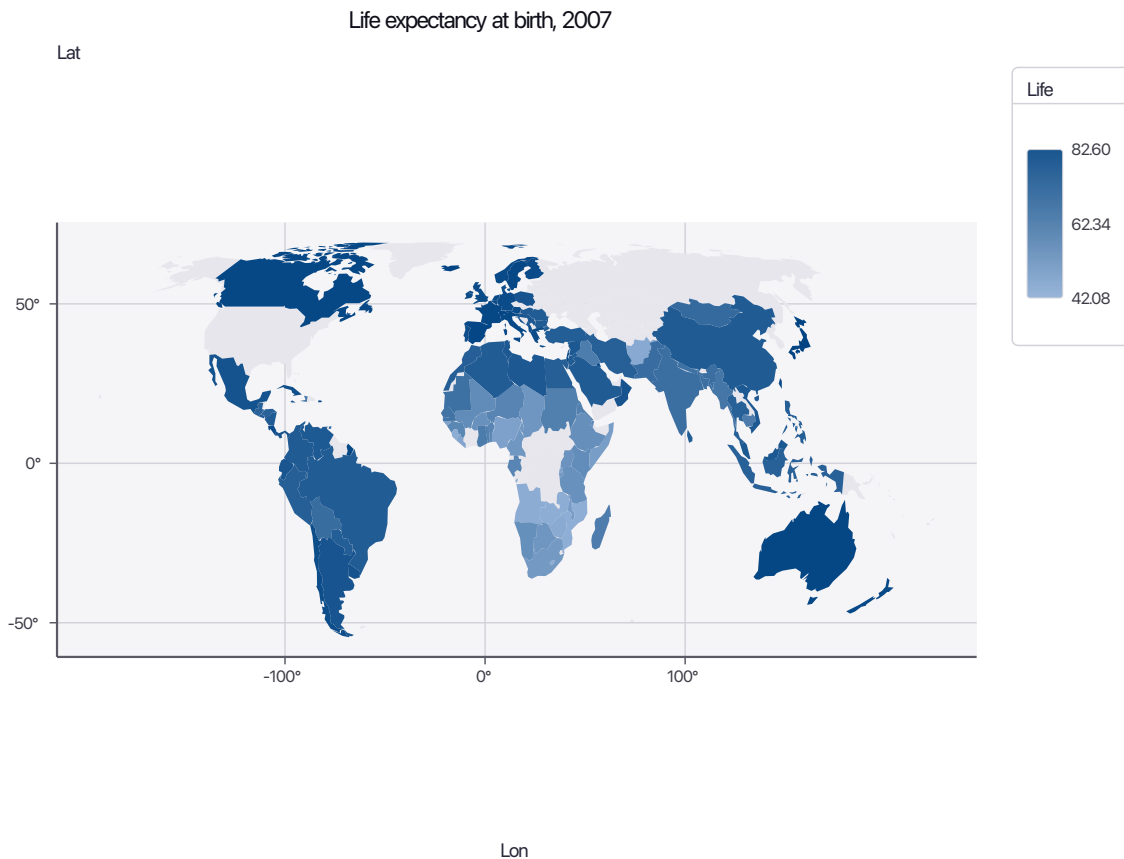
Joining two tables is the host language’s job, not the grammar’s. In R the join itself is one `match()` call. Every host language has its own one-call lookup. The plot sentence under the join is the same in all four languages:


```

filled <- world_borders
filled$life <- gapminder_2007$life[match(filled$country,
                                         gapminder_2007$country)]
filled <- filled[!is.na(filled$life), ]

data(world_borders) + zone + x(lon) + y(lat) + group(country) +
  style(color = "#e6e6ec") +
  data(filled) + zone + x(lon) + y(lat) + group(country) + color(life) +
  map() + title("Life expectancy at birth, 2007")

```



Two layers again, and the first one is not decoration. Of the 176 countries in the outline table, 120 have a number in `gapminder_2007`. The pale gray layer underneath is what the reader sees for the other 56. Without it those countries would simply be absent, and a reader could not tell “no data” from “no country”.

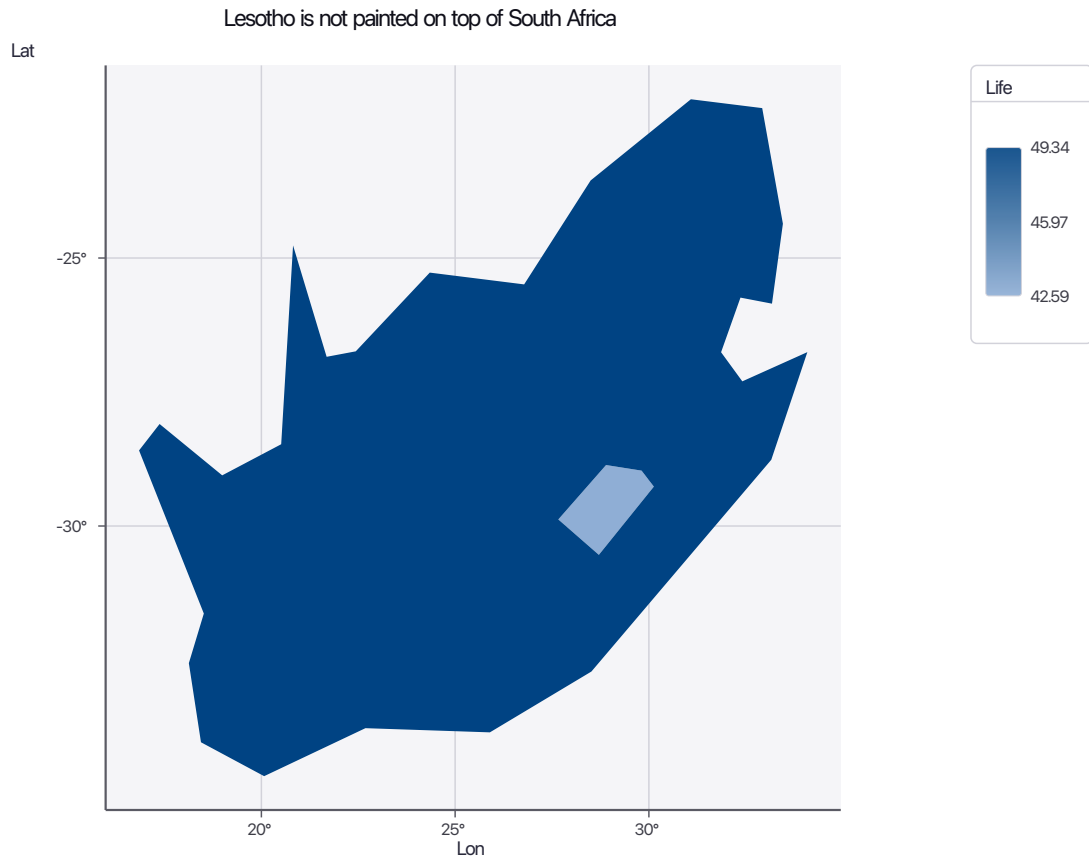
Read the map and the scatter together. The chapter [A first plot](#) shows life expectancy against income as a cloud of points. Here is the same column, and the map answers a question the scatter cannot: *where*.

35.11 Rings, and the country inside another country

A region is made of rings, and the rings are found by a rule rather than by a column: a ring ends where it returns to the point it started from. Boundary data is written that way, so nothing extra has to be said.

That rule is what lets a country contain a hole. Lesotho lies entirely inside South Africa, and South Africa's boundary has a second ring around Lesotho:

```
enclave <- world_borders[world_borders$country %in%  
                          c("South Africa", "Lesotho"), ]  
enclave$life <- gapminder_2007$life[match(enclave$country,  
                                          gapminder_2007$country)]  
  
data(enclave) + zone + x(lon) + y(lat) + group(country) + color(life) +  
  map() + title("Lesotho is not painted on top of South Africa")
```



Lesotho is a hole in South Africa, not a patch laid over it. The difference matters because the two countries have different values. A patch is correct only when it is drawn second, and nothing fixes the drawing order.

If a ring in your data does not close, **gog** cannot tell where one ring ends and the next begins. It draws the region, warns you that the ring ends had to be guessed, and names what to fix.

Every sentence in this chapter also draws on the sphere itself. That is **Globe**: the same marks at the same places, seen from one side.

36 Globe

What does the data look like on the earth itself? The map chapter flattened the sphere to fit a page. This chapter keeps it round, and turns it toward you.

Here is the map chapter's first sentence, and beside it the same sentence with the space swapped:

R

```
(data(quakes_fiji) + point + x(east) + y(north) + map() +  
  title("map()")) |  
(data(quakes_fiji) + point + x(east) + y(north) +  
  globe(turn = 178, tilt = -18) + title("globe(turn = 178, tilt = -18)"))
```

Python

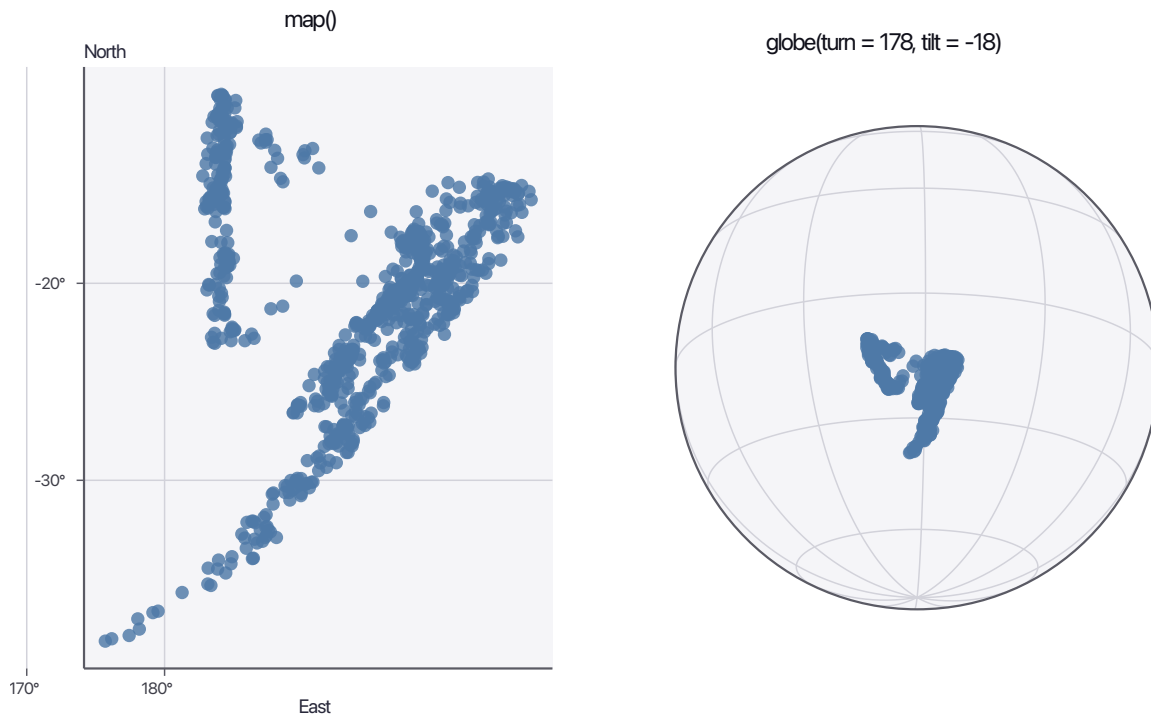
```
((data(quakes_fiji) + point + x(col.east) + y(col.north) + map() +  
  title("map()")) |  
(data(quakes_fiji) + point + x(col.east) + y(col.north) +  
  globe(turn = 178, tilt = -18) + title("globe(turn = 178, tilt = -18)")))
```

Julia

```
(data(quakes_fiji) + point + x(:east) + y(:north) + map() +  
  title("map()")) |  
(data(quakes_fiji) + point + x(:east) + y(:north) +  
  globe(turn = 178, tilt = -18) + title("globe(turn = 178, tilt = -18)"))
```

JavaScript

```
beside(plot(data(quakes_fiji), point, x(col.east), y(col.north), map(),  
  title("map()")),  
plot(data(quakes_fiji), point, x(col.east), y(col.north),  
globe({ turn: 178, tilt: -18 }),  
title("globe(turn = 178, tilt = -18)")))
```



“Given quakes Fiji: points, x is east, y is north, on the globe.”

x is longitude and y is latitude, exactly as on the map. What changed is what a position means on the page. Each point now stands at its place on the sphere, and the sphere is drawn as the disk you would see looking at it.

36.1 The view

A globe shows one half of the earth, so which half matters. `turn` and `tilt` name the place the view faces, in the words `space()` already uses: `turn` is the longitude at the center of the disk, and `tilt` is the latitude. The plot above faces Fiji. Leave them out and the view faces zero and zero, where the equator crosses the prime meridian.

`turn` is a bearing, so any number is a view: 190 and -170 face the same place. `tilt` is a latitude, and it stops at the poles.

In the web edition the written angle is only where the globe starts. Drag it and it turns, the way the cube in [Space](#) turns, and the far half of the earth comes around; *reset* returns to the view the sentence asked for. On paper a plot keeps its one view, which is why this chapter also draws some data at a second angle.

Half the earth faces away, and data on that half is hidden behind the sphere. *gog* counts what a view hides rather than dropping it in silence:

R

```
data(world_borders) + path + x(lon) + y(lat) + group(piece) +  
  style(color = "#c9ccd6") +  
  data(quakes_fiji) + point + x(east) + y(north) +  
  style(color = "tomato", opacity = 0.35, size = 3) +  
  globe(turn = 178, tilt = -18) + title("The map chapter's layers, kept  
↪ round")
```

Python

```
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +  
  style(color = "#c9ccd6") +  
  data(quakes_fiji) + point + x(col.east) + y(col.north) +  
  style(color = "tomato", opacity = 0.35, size = 3) +  
  globe(turn = 178, tilt = -18) + title("The map chapter's layers, kept  
↪ round"))
```

Julia

```
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +  
  style(color = "#c9ccd6") + data(quakes_fiji) + point + x(:east) +  
  y(:north) + style(color = "tomato", opacity = 0.35, size = 3) +  
  globe(turn = 178, tilt = -18) +  
  title("The map chapter's layers, kept round")
```

JavaScript

```
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),  
  style({ color: "#c9ccd6" }), data(quakes_fiji), point, x(col.east),  
  y(col.north), style({ color: "tomato", opacity: 0.35, size: 3 }),  
  globe({ turn: 178, tilt: -18 }),  
  title("The map chapter's layers, kept round"))
```

```
gog: 3192 of 4150 row(s) face away from `globe(turn = 178, tilt = -18)` and
  ↳ are hidden behind the sphere. Turn to face them, or put a second view
  ↳ beside this one with `|`.
```

The map chapter's layers, kept round



“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given quakes Fiji: points, x is east, y is north, on the globe.”

Read the message above the plot. The coastline table covers the whole earth, so most of its rows face away from any one view, while every earthquake faces this one. Turning the globe brings the hidden rows around:

R

```
data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  globe(turn = 20, tilt = 5) + title("The other side")
```

Python

```
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +  
  style(color = "#c9ccd6") +  
  globe(turn = 20, tilt = 5) + title("The other side"))
```

Julia

```
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +  
  style(color = "#c9ccd6") + globe(turn = 20, tilt = 5) +  
  title("The other side")
```

JavaScript

```
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),  
  style({ color: "#c9ccd6" }), globe({ turn: 20, tilt: 5 }),  
  title("The other side"))
```

The other side



36.2 The graticule

The gray lines on the sphere are the **graticule**: a meridian every thirty degrees of longitude, and a parallel every thirty degrees of latitude. They are this space's gridlines, the same lines every panel draws, bent onto the sphere the way polar bends them into rings and spokes. There are no axes beside the disk, because a sphere has no edge to write one on; the graticule is the reference instead. `theme(grid = )` reaches it like any other grid:

R

```
data(quakes_fiji) + point + x(east) + y(north) +  
  globe(turn = 178, tilt = -18) + theme(grid = "none") +  
  title("No graticule")
```

Python

```
(data(quakes_fiji) + point + x(col.east) + y(col.north) +  
  globe(turn = 178, tilt = -18) + theme(grid = "none") +  
  title("No graticule"))
```

Julia

```
data(quakes_fiji) + point + x(:east) + y(:north) +  
  globe(turn = 178, tilt = -18) + theme(grid = "none") +  
  title("No graticule")
```

JavaScript

```
plot(data(quakes_fiji), point, x(col.east), y(col.north),  
  globe({ turn: 178, tilt: -18 }), theme({ grid: "none" }),  
  title("No graticule"))
```

No graticule



36.3 A route bends with the sphere

On the map, `path` joins places with straight strokes. On the globe a straight route does not exist. The shortest way between two places bends with the surface, along a great circle, and a `path` here follows it:

R

```
flight <- data.frame(lon = c(139.69, -118.24), lat = c(35.69, 34.05))

data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  data(flight) + path + x(lon) + y(lat) +
  style(color = "#3d5a80", arrow = "end") +
  globe(turn = 178, tilt = -18) + title("Tokyo to Los Angeles")
```

Python

```
flight = {"lon": [139.69, -118.24], "lat": [35.69, 34.05]}
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +
 style(color = "#c9ccd6") +
 data(flight) + path + x(col.lon) + y(col.lat) +
 style(color = "#3d5a80", arrow = "end") +
 globe(turn = 178, tilt = -18) + title("Tokyo to Los Angeles"))
```

Julia

```
flight = (lon = [139.69, -118.24], lat = [35.69, 34.05],)
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +
 style(color = "#c9ccd6") + data(flight) + path + x(:lon) + y(:lat) +
 style(color = "#3d5a80", arrow = "end") +
 globe(turn = 178, tilt = -18) + title("Tokyo to Los Angeles")
```

JavaScript

```
const flight = { lon: [139.69, -118.24], lat: [35.69, 34.05] };
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),
 style({ color: "#c9ccd6" }), data(flight), path, x(col.lon), y(col.lat),
 style({ color: "#3d5a80", arrow: "end" }),
 globe({ turn: 178, tilt: -18 }, title("Tokyo to Los Angeles"))
```

Tokyo to Los Angeles



Two rows describe that route, and the sphere sets the curve between them. The arc bends toward the pole because the shortest way across an ocean bends there too.

36.4 Many routes are a network

One table can hold every route at once. Give each route a name, give each of its two rows a place, and `group()` splits the table into one arc per route, exactly as it split the coastline into pieces. A `point` layer marks the cities and a `text` layer names them. The picture this draws is a route network, and it needs no new word, because every position in it is a place the data already knows:

```
cities <- data.frame(  
  city = c("Seoul", "Tokyo", "Singapore", "Delhi", "Cairo",  
           "London", "Anchorage", "Sydney"),  
  lon  = c(126.98, 139.69, 103.82, 77.21, 31.24, -0.13, -149.90, 151.21),
```

```

    lat = c(37.57, 35.68, 1.35, 28.61, 30.04, 51.51, 61.22, -33.87)
  )
legs <- data.frame(
  route = rep(c("Seoul-Tokyo", "Seoul-Singapore", "Seoul-Delhi",
               "Delhi-Cairo", "Cairo-London", "Tokyo-Anchorage",
               "Singapore-Sydney"), each = 2),
  city = c("Seoul", "Tokyo", "Seoul", "Singapore", "Seoul", "Delhi",
           "Delhi", "Cairo", "Cairo", "London", "Tokyo", "Anchorage",
           "Singapore", "Sydney")
)
routes <- merge(legs, cities, by = "city", sort = FALSE)

data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  data(routes) + path + x(lon) + y(lat) + group(route) +
  style(color = "#3d5a80", opacity = 0.6) +
  data(cities) + point + x(lon) + y(lat) +
  data(cities) + text + x(lon) + y(lat) + label(city) + style(nudge = "up") +
  globe(turn = 95, tilt = 30) + title("Routes between cities")

```

gog: 897 of 4150 row(s) face away from `globe(turn = 95, tilt = 30)` and are
 ↪ hidden behind the sphere. Turn to face them, or put a second view beside
 ↪ this one with `|`.

Routes between cities



“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given the routes: paths, x is lon, y is lat, grouped by route; then given the cities: points, x is lon, y is lat; then given the cities: text, x is lon, y is lat, label by city, on the globe.”

Looking up each city’s longitude and latitude is the host language’s job, the same join a later section does for country centers. What the sentence adds is only the layers: the routes, then the cities, then their names.

Every arc bends with the surface. The count above the plot reports what this view hides: the coastlines on the far side, and no city. Dragging the globe in the web edition brings those coastlines around. A network between places asks nothing new of the grammar. The places carry the positions, `group()` carries the connections, and the sphere carries the shortest ways between them.

36.5 Meridians and parallels

A **rule** draws a line at one position and spans the axis it does not name. On the map that drew the equator as a straight line. Here the same sentence bends it around the sphere:

R

```
lines_of <- data.frame(lat = 0)

data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  data(lines_of) + rule + y(lat) + style(color = "#d4674f") +
  globe(turn = 20, tilt = 30) + title("The equator, bent around the sphere")
```

Python

```
lines_of = {"lat": [0]}
(data(world_borders) + path + x(col.lon) + y(col.lat) + group(col.piece) +
  style(color = "#c9ccd6") +
  data(lines_of) + rule + y(col.lat) + style(color = "#d4674f") +
  globe(turn = 20, tilt = 30) + title("The equator, bent around the sphere"))
```

Julia

```
lines_of = (lat = [0],)
data(world_borders) + path + x(:lon) + y(:lat) + group(:piece) +
  style(color = "#c9ccd6") + data(lines_of) + rule + y(:lat) +
  style(color = "#d4674f") + globe(turn = 20, tilt = 30) +
  title("The equator, bent around the sphere")
```

JavaScript

```
const lines_of = { lat: [0] };
plot(data(world_borders), path, x(col.lon), y(col.lat), group(col.piece),
  style({ color: "#c9ccd6" }), data(lines_of), rule, y(col.lat),
  style({ color: "#d4674f" }), globe({ turn: 20, tilt: 30 }),
  title("The equator, bent around the sphere"))
```

The equator, bent around the sphere



“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given the lines of table: a rule, y is lat, on the globe.”

A rule on y is a parallel, a circle of constant latitude. A rule on x is a meridian, and it runs pole to pole.

36.6 The choropleth, kept round

The map chapter filled each country by its continent. The same sentence fills the countries on the sphere:

R

```
data(world_borders) + zone + x(lon) + y(lat) + group(country) +  
  color(continent) + globe(turn = 20, tilt = 5) +  
  title("Every country, filled by continent")
```


Python

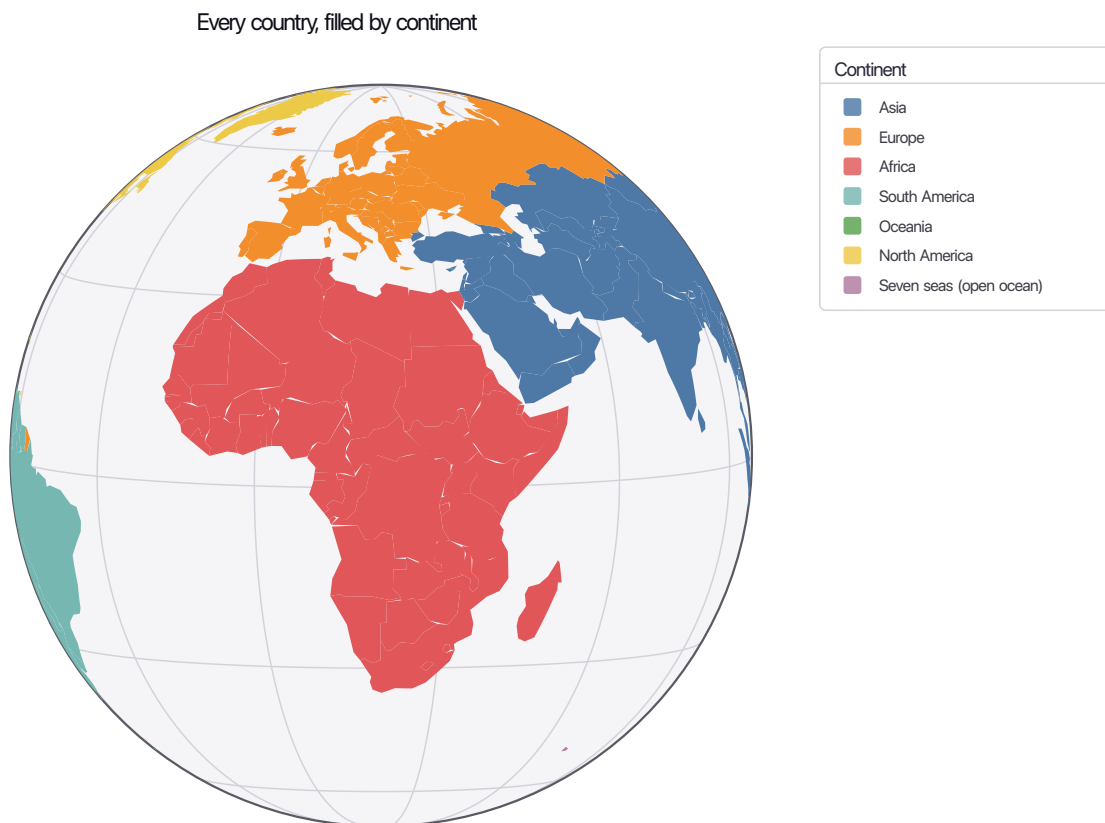
```
(data(world_borders) + zone + x(col.lon) + y(col.lat) + group(col.country) +  
color(col.continent) + globe(turn = 20, tilt = 5) +  
title("Every country, filled by continent"))
```

Julia

```
data(world_borders) + zone + x(:lon) + y(:lat) + group(:country) +  
color(:continent) + globe(turn = 20, tilt = 5) +  
title("Every country, filled by continent")
```

JavaScript

```
plot(data(world_borders), zone, x(col.lon), y(col.lat),  
group(col.country), color(col.continent), globe({ turn: 20, tilt: 5 } ),  
title("Every country, filled by continent"))
```



“Given the world borders: zones, x is lon, y is lat, grouped by country, color by continent, on the globe.”

Look at the countries the edge of the disk cuts through. `gog` closes each cut shape along that edge, so a half-visible country is still a filled region rather than an open outline.

36.7 A measure stands on the radius

The flattened map has no axis to spare. The sphere has exactly one: the radius, which runs straight out from the surface at every place. A `bar` here stands at its place and measures outward along it, a spike from the surface, and `z` names its measure:

```
centers <- do.call(rbind, lapply(split(world_borders[, c("lon", "lat")],
                                     world_borders$country), colMeans))
spikes <- data.frame(country = rownames(centers), centers)
matched <- match(spikes$country, gapminder_2007$country)
spikes$people <- gapminder_2007$population[matched]
spikes$continent <- gapminder_2007$continent[matched]
spikes <- spikes[!is.na(spikes$people), ]
spikes$reach <- sqrt(spikes$people)

data(world_borders) + path + x(lon) + y(lat) + group(piece) +
  style(color = "#c9ccd6") +
  data(spikes) + bar + x(lon) + y(lat) + z(reach) + color(continent) +
  globe(turn = 20, tilt = 12) +
  title("World population, standing on its places")
```

World population, standing on its places



“Given the world borders: paths, x is lon, y is lat, grouped by piece; then given the spikes: bars, x is lon, y is lat, z is reach, color by continent, on the globe.”

Finding a country’s center and joining the population are the host language’s job, as the join was on the map. Shaping the measure is its job too. The square root keeps China’s spike from making the rest look tiny, and a scale parameter would only be a second way of writing that one line.

Look at the edge of the disk. The sphere itself clips a spike, so one standing just behind that edge still shows its top when it is tall enough. The tallest spikes there are seen from the side. A spike facing you foreshortens toward a point, which is why a plot like this is read by turning it.

36.8 The depth goes to the cube

Each earthquake here carries a third number: `elevation`, its depth below the surface. That is not a length to raise from the surface but a third place coordinate, and on the globe `z` belongs to the spike alone. So `gog` refuses the sentence below rather than guessing at it. The refusal names three ways forward: add a `bar` layer, drop `z()`, or draw the same table in the cube.

```
data(quakes_fiji) + point + x(east) + y(north) + z(elevation) +  
  globe(turn = 178, tilt = -18)
```

Error:

```
! gog: `z(...)` on a globe is the radius, and only a `bar` reads it - a spike  
  ↳ standing at its place, measuring outward from the surface. Add a `bar`  
  ↳ layer to raise spikes, drop `z(...)` to keep the surface marks, or drop  
  ↳ `globe()` for the cube, where every position mark reads `z`.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A place with a depth belongs in the cube, the space with a third position. `space()` names the view with the words `globe()` uses:

R

```
data(quakes_fiji) + point + x(east) + y(north) + z(elevation) +  
  color(slab) + space(turn = 130, tilt = 25) +  
  title("The same earthquakes, read by depth")
```

Python

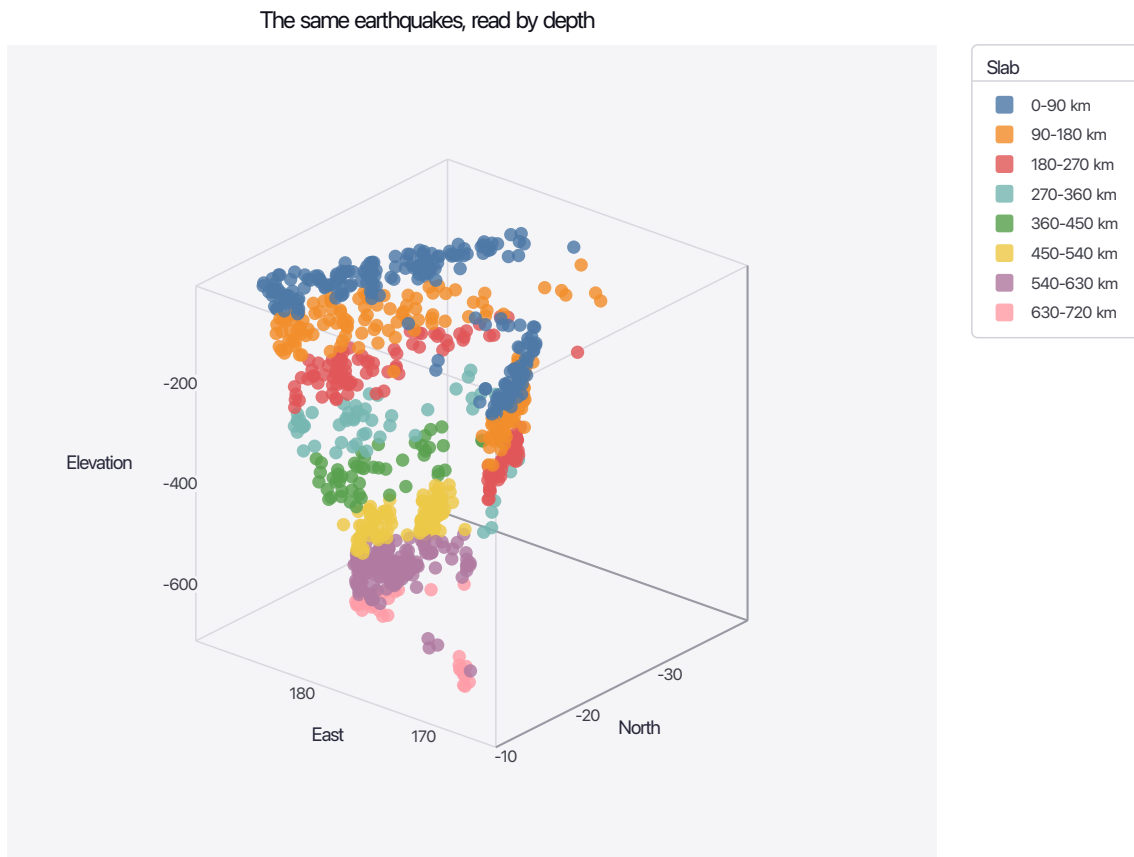
```
(data(quakes_fiji) + point + x(col.east) + y(col.north) + z(col.elevation) +  
  color(col.slab) + space(turn = 130, tilt = 25) +  
  title("The same earthquakes, read by depth"))
```

Julia

```
data(quakes_fiji) + point + x(:east) + y(:north) + z(:elevation) +  
  color(:slab) + space(turn = 130, tilt = 25) +  
  title("The same earthquakes, read by depth")
```

JavaScript

```
plot(data(quakes_fiji), point, x(col.east), y(col.north),
     z(col.elevation), color(col.slabs), space({ turn: 130, tilt: 25 } ),
     title("The same earthquakes, read by depth"))
```



“Given quakes Fiji: points, x is east, y is north, z is elevation, color by slab.”

Drag this one too: the globe turns the earth, and the cube turns the cloud, one gesture for both. `slab` names the depth band each quake falls in. The earlier plots looked straight down on these quakes; this one shows the sinking sheet of ocean floor they sit on.

36.9 What a globe refuses

Both positions carry the place, exactly as on the map, so a mark that measures along an axis has no axis left:

```
data(quakes_fiji) + line + x(east) + y(north) + globe()
```

Error:

```
! gog: `line` measures along an axis, and a `globe()` plot has none to spare
  ↳ - longitude and latitude use both. Drop `globe()` to draw `line` flat, or
  ↳ use `point`, `bar`, `text`, `path`, `rule`, or `zone`. To carry a
  ↳ quantity on the globe, put it on a channel instead of an axis: `point +
  ↳ size(<column>)` sizes each place by it, and `color(<column>)` shades it.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The quantity moves to a channel instead, `size()` or `color()`, or it stands on the radius as a spike, the way the population spikes above do.

A latitude past a pole names no place, so a view cannot face that latitude:

```
data(quakes_fiji) + point + x(east) + y(north) + globe(tilt = 100)
```

Error:

```
! gog: `globe(tilt = 100)` is outside -90 to 90, which is where a latitude
  ↳ lives. At 90 the view faces straight down on the north pole and at -90 on
  ↳ the south; there is no place past either. Use `globe(tilt = 90)` for the
  ↳ nearest view, or `globe(turn = )` to swing around the earth instead - a
  ↳ bearing wraps and a latitude does not.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

And a globe draws no axes at all, so naming one is refused rather than accepted and dropped:

```
data(quakes_fiji) + point + x(east) + y(north) + globe() +
  ↳ x_label("Longitude")
```

Error:

```
! gog: an axis label names an axis, and a `globe()` plot draws none - a
  ↳ sphere has no edge to write one on, and the graticule is the reference
  ↳ instead. Drop it; `title()` still names the plot.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The map and the globe are one meaning with two pictures of it. The map trades the sphere for a page a reader sees whole. The globe keeps the sphere and trades away the far half. Moving between them costs one word.

37 Network

What is connected to what? Some tables record relations rather than measurements: each row says that one thing is related to another. A network diagram draws those relations. Each thing becomes a **node**, one marked place in the picture, and each relation becomes a stroke between two nodes. Nodes with many relations are drawn near the middle.

Nothing in such a table says where any node should sit. That is the whole difficulty, and this chapter shows how `gog` answers it. The engine computes a position for every node. A space named `network()` draws the result, with no axes at all.

`trade_partners` holds one row per trade relation: who exports, who imports, and the value of the goods, in billions of dollars, rounded:

Table 37.1: `trade_partners`: first 5 of 22 rows

exporter	importer	billions
China	United States	500
Mexico	United States	455
Canada	United States	420
United States	Mexico	320
United States	Canada	355

37.1 One row is one edge

Write the `edge` mark derived by `layout()`, name the two endpoint columns in order, and say the space:

R

```
data(trade_partners) + edge * layout(exporter, importer) + network()
```

Python

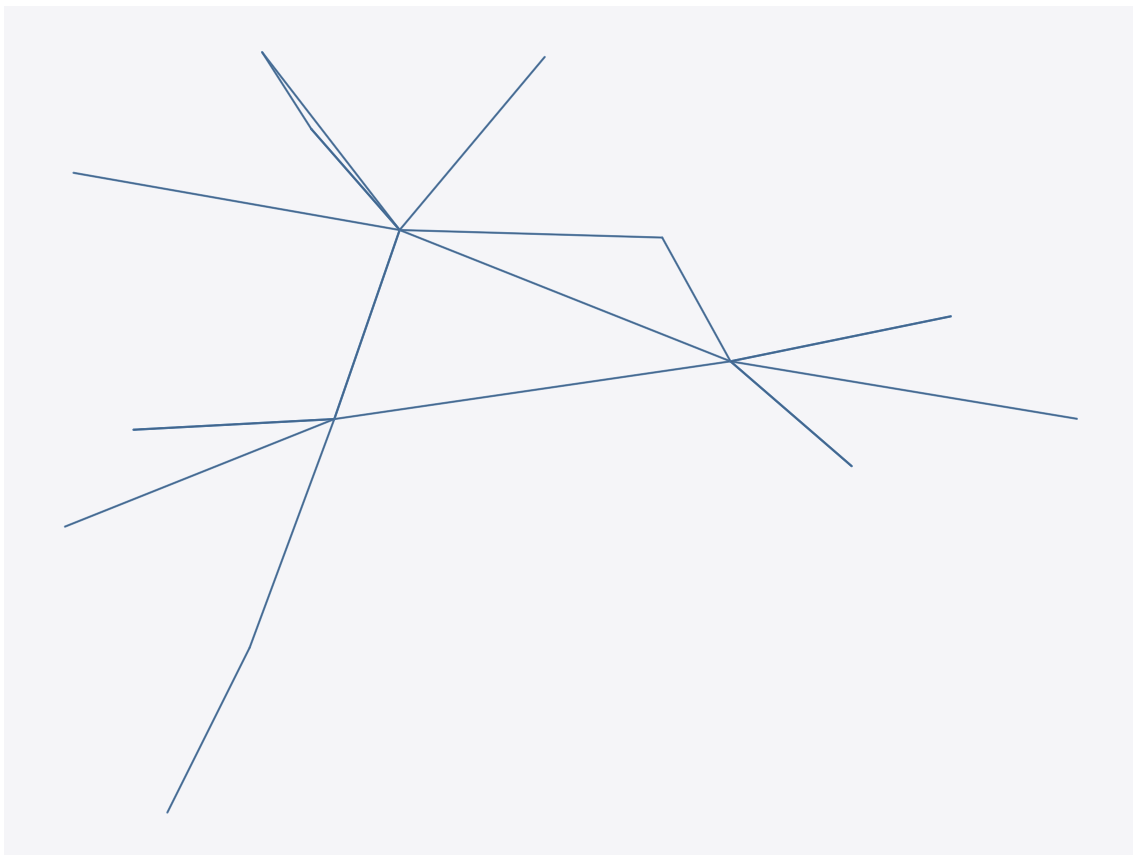
```
data(trade_partners) + edge * layout(col.exporter, col.importer) + network()
```

Julia

```
data(trade_partners) + edge * layout(:exporter, :importer) + network()
```

JavaScript

```
plot(data(trade_partners),  
      layer(edge, layout(col.exporter, col.importer)), network())
```



“Given trade partners: edges derived by layout from exporter to importer, in the network.”

Each stroke is one row of the table. The `layout` reads both columns and finds every distinct name. All nodes push apart, and nodes that share a row also pull together, so the connected

ones settle near each other. The engine computes every position itself, and computes it the same way everywhere, so this sentence draws the same picture in R, Python, Julia and JavaScript, to the byte.

The panel has no axes, no ticks and no grid, because `network()` draws none. A layout's positions mean nothing as quantities. Flip the picture left to right and it means the same, so a number on an axis would state nothing about the data. A layout therefore draws in `network()` and in no other space.

37.2 The nodes and their names

Two more layers finish the diagram. A `point` layer draws the nodes, and a `text` layer names them. Each one writes the same `layout`, and the engine computes it once per layer. Every computation lands on the same positions, because the placement is deterministic. The layout publishes two columns about each node: its `name`, and its `degree`, the count of its relations. `size(degree)` draws the nodes with the most relations largest. The `text` layer also composes `repel`, the modifier that moves labels off each other, so a name rests beside its dot instead of covering it:

R

```
data(trade_partners) +  
  edge * layout(exporter, importer) +  
  point * layout(exporter, importer) + size(degree) +  
  text * layout(exporter, importer) * repel + label(name) +  
  network() +  
  title("Who trades with whom")
```

Python

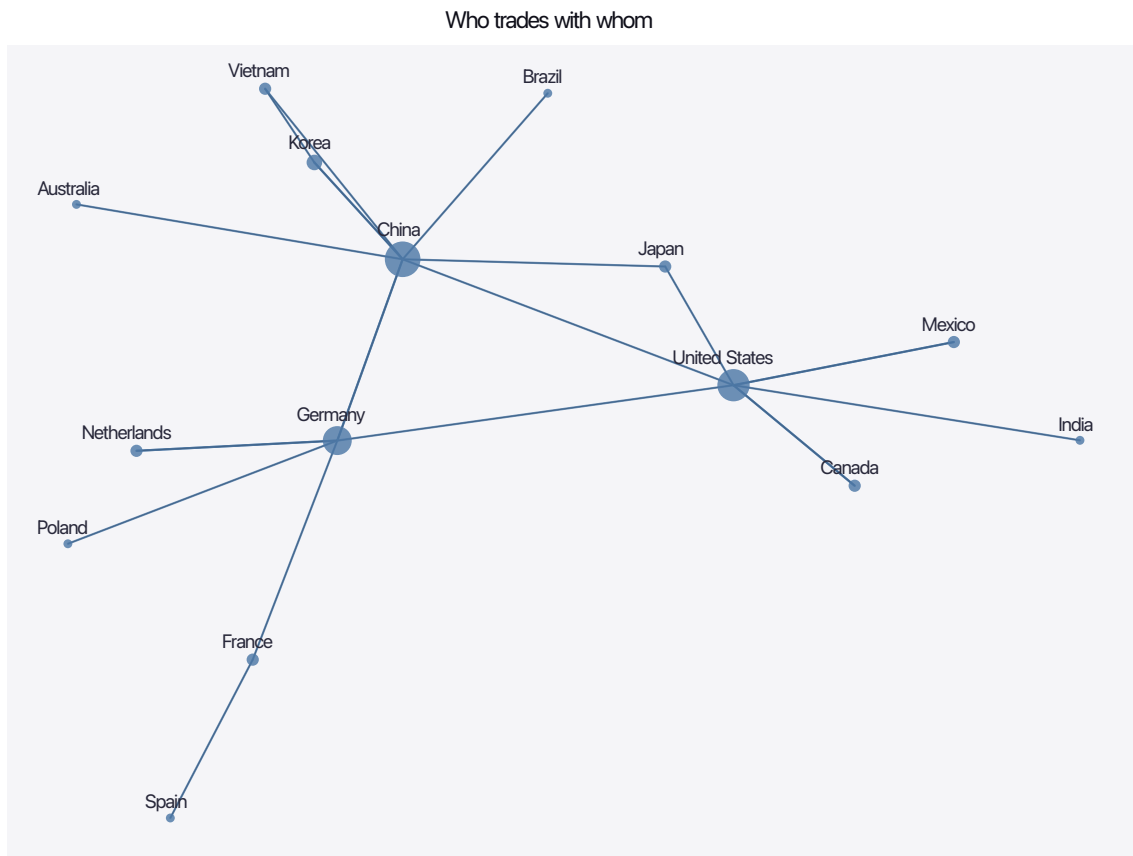
```
(data(trade_partners) +  
  edge * layout(col.exporter, col.importer) +  
  point * layout(col.exporter, col.importer) + size(col.degree) +  
  text * layout(col.exporter, col.importer) * repel + label(col.name) +  
  network() +  
  title("Who trades with whom"))
```

Julia

```
data(trade_partners) + edge * layout(:exporter, :importer) +
  point * layout(:exporter, :importer) + size(:degree) +
  text * layout(:exporter, :importer) * repel + label(:name) + network() +
  title("Who trades with whom")
```

JavaScript

```
plot(data(trade_partners),
  layer(edge, layout(col.exporter, col.importer)),
  layer(point, layout(col.exporter, col.importer)), size(col.degree),
  layer(text, layout(col.exporter, col.importer), repel), label(col.name),
  network(), title("Who trades with whom"))
```



“Given trade partners: edges derived by layout from exporter to importer, and also points derived by layout from exporter to importer, size by degree, and also text derived by layout from exporter to importer and repel, label by name, in the network.”

The nodes with the most relations sit near the middle. No row of the table says that the United States, China and Germany are centers of trade; saying it is what the layout is for.

37.3 An edge carries its own columns

An edge is one row, so its color and its opacity can follow any column of the table. `opacity(billions)` maps each stroke's opacity to its value, so the heavier relations draw darker:

R

```
data(trade_partners) +  
  edge * layout exporter, importer + opacity(billions) +  
  point * layout exporter, importer + size(degree) +  
  text * layout exporter, importer * repel + label(name) +  
  network() +  
  title("Heavier trade, darker stroke")
```

Python

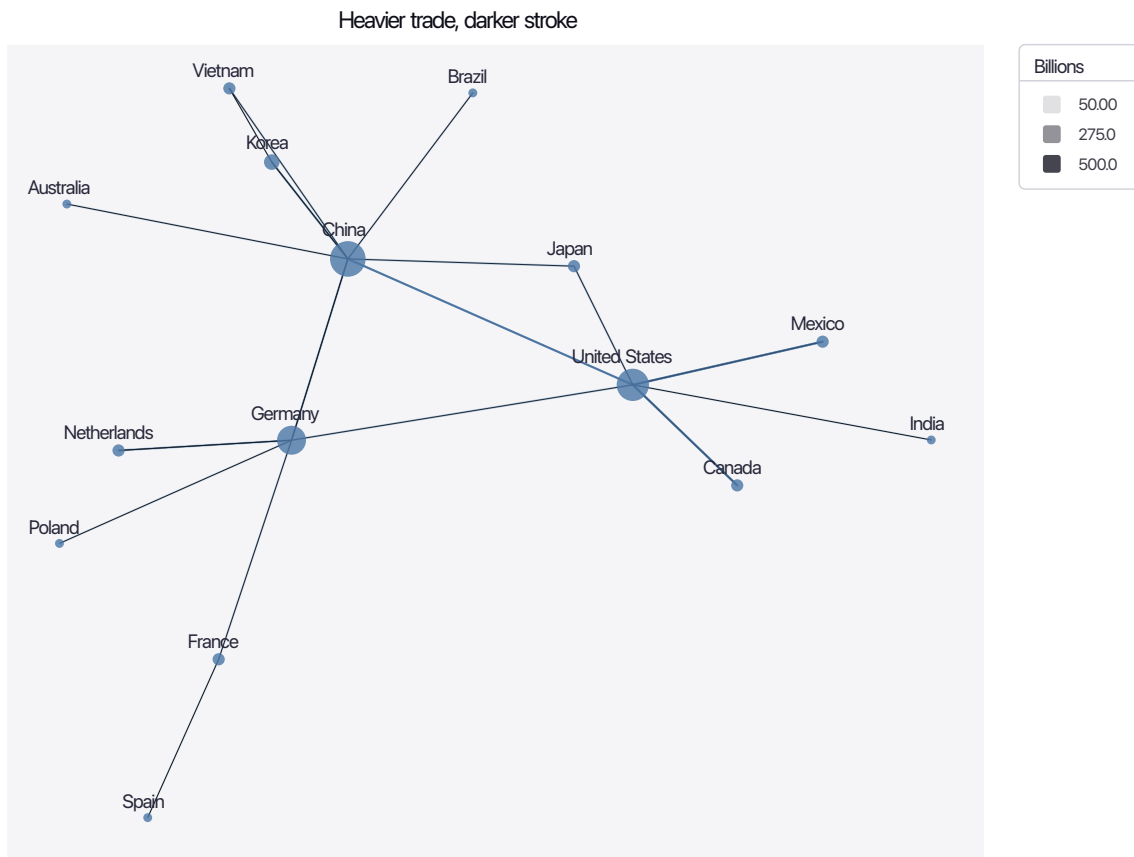
```
(data(trade_partners) +  
  edge * layout(col.exporter, col.importer) + opacity(col.billions) +  
  point * layout(col.exporter, col.importer) + size(col.degree) +  
  text * layout(col.exporter, col.importer) * repel + label(col.name) +  
  network() +  
  title("Heavier trade, darker stroke"))
```

Julia

```
data(trade_partners) + edge * layout(:exporter, :importer) +  
  opacity(:billions) + point * layout(:exporter, :importer) +  
  size(:degree) + text * layout(:exporter, :importer) * repel +  
  label(:name) + network() + title("Heavier trade, darker stroke")
```

JavaScript

```
plot(data(trade_partners),  
  layer(edge, layout(col.exporter, col.importer)), opacity(col.billions),  
  layer(point, layout(col.exporter, col.importer)), size(col.degree),  
  layer(text, layout(col.exporter, col.importer), repel), label(col.name),  
  network(), title("Heavier trade, darker stroke"))
```



“Given trade partners: edges derived by layout from exporter to importer, opacity by billions, and also points derived by layout from exporter to importer, size by degree, and also text derived by layout from exporter to importer and repel, label by name, in the network.”

A node is different. A node stands for many rows, and those rows carry different values. So a node layer maps only **name** and **degree**. Every other column belongs on the **edge** layer, where one row carries one value.

37.4 An angle asks for the cube

Bare `network()` computes the layout in the plane. State a viewing angle and the same edges are laid out in a cube instead. Every node takes a new position, and the picture is drawn from the angle you asked for:

R

```
data(trade_partners) +
  edge * layout exporter, importer + opacitybillions +
  point * layout exporter, importer + sizedegree +
  network(turn = 35, tilt = 20) +
  title("The same network, given depth")
```

Python

```
(data(trade_partners) +
  edge * layout(col.exporter, col.importer) + opacity(col.billions) +
  point * layout(col.exporter, col.importer) + size(col.degree) +
  network(turn = 35, tilt = 20) +
  title("The same network, given depth"))
```

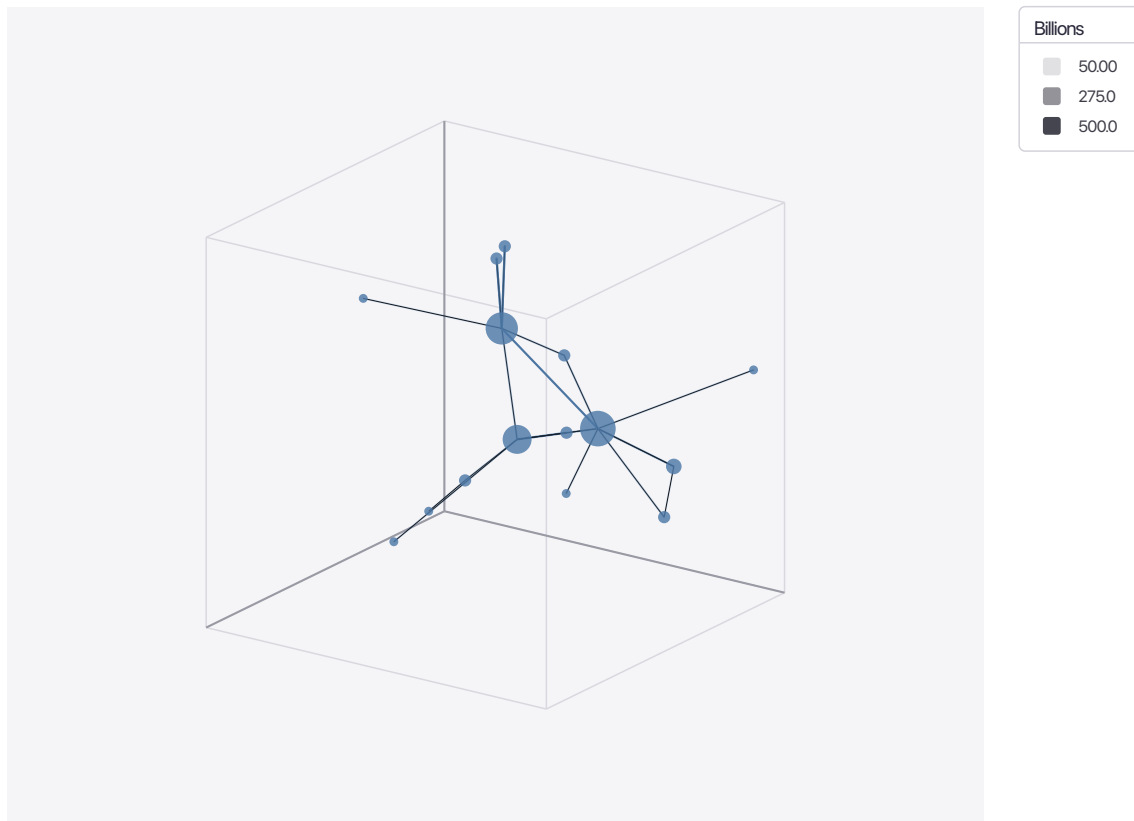
Julia

```
data(trade_partners) + edge * layout(:exporter, :importer) +
  opacity(:billions) + point * layout(:exporter, :importer) +
  size(:degree) + network(turn = 35, tilt = 20) +
  title("The same network, given depth")
```

JavaScript

```
plot(data(trade_partners),
  layer(edge, layout(col.exporter, col.importer)), opacity(col.billions),
  layer(point, layout(col.exporter, col.importer)), size(col.degree),
  network({ turn: 35, tilt: 20 }), title("The same network, given depth"))
```

The same network, given depth



“Given trade partners: edges derived by layout from exporter to importer, opacity by billions, and also points derived by layout from exporter to importer, size by degree, in the network, turned 35, tilted 20.”

Nothing in the table gives a node a third position, so there is nothing to bind to **z**. The third dimension belongs to the space, exactly as the viewing angle does, so asking for the angle is asking for the cube. The box is drawn for depth and carries no numbers, for the same reason the flat form has no axes.

In the web edition the cube turns with a drag, the way the cube in [Space](#) turns, and *reset* returns to the angle the sentence asked for. On paper it keeps the view it was given.

Naming a node in the cube is valid grammar. This engine does not draw those names:

```
data(trade_partners) +  
  edge * layout(exporter, importer) +  
  text * layout(exporter, importer) * repel + label(name) +  
  network(turn = 35, tilt = 20)
```

Error:

! gog: names in the network's cube are valid grammar this engine does not
↪ draw yet. Drop the `text` layer, or drop the viewing angle and label the
↪ flat form.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

37.5 What the network refuses

A layout outside its space would draw axes, and the numbers along them would mean nothing:

```
data(trade_partners) + edge * layout(exporter, importer)
```

Error:

! gog: a `layout`'s positions are the graph's, not the data's - an axis under
↪ them would number something no reader can mean. Draw it in the network:
↪ `+ network()` flat, or `+ network(turn = 30, tilt = 25)` for the cube.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

Only three marks read a layout. Write any other mark and the refusal names all three:

```
data(trade_partners) + bar * layout(exporter, importer) + network()
```

Error:

! gog: `layout` places a graph - a position per node, computed from an edge
↪ table - and `bar` has no reading for that. Three marks do: `edge *
↪ layout(<from>, <to>)` draws the connections, `point * layout(...)` the
↪ nodes, and `text \* layout(...) + label(name)` names them, all inside
↪ `network()`.
gog: `bar` does not stand in `network()` - it reads an axis, and this space
↪ has none. The layout's readers are `edge`, `point` and `text`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

The layout has already placed every node, so x() has nothing left to set:

```
data(trade_partners) + x(billions) +  
  edge * layout(exporter, importer) + network()
```

Error:

! gog: under `layout` the nodes place themselves, so `x(...)` has nothing
↪ left to say. Remove it - the third dimension, when you want it, is the
↪ space's: `network(turn = 30, tilt = 25)`.

gog: `x` cannot be bound to `edge` - an edge has no x feature. Remove the
↪ `billions` mapping from `x`, or use a mark that has one.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

And `network()` with no layout has nothing to place, so it refuses:

```
data(trade_partners) + point + network()
```

Error:

! gog: `network()` draws what a `layout` places, and nothing here carries
↪ one. Compose the syllable: `edge \* layout(from, to) + network()`, with
↪ `point \* layout(...)` for the nodes and `text \* layout(...)` +
↪ `label(name)` for their names.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

37.6 Where the limits are

A row from a node to itself is refused. Such a stroke would have no length, and a fact about one node belongs on the node. A row missing an endpoint is left out, and the engine says how many it left out.

gog does not read a second table of node facts. A node shows only what the edges give it: its name and its degree. Joining more columns onto the nodes is a table operation, so do it in the host language, before the plot. A **brush** is refused too, and its answer is also upstream: a brush names a range on the data's axes, and a network has none. Select the rows in your table first, then color the edges they make.

38 Play

How did the whole picture change from year to year? `play` cuts the plot into frames and shows them in sequence. The shortest way to say what `play` does is that it is [faceting](#) read in time. A facet splits the rows by a column's distinct values and lays the pieces out across the page; `play` splits them exactly the same way and lays them out in time, one after another.

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + color(continent) +  
  play(year)
```

Python

```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +  
  ↪ color(col.continent) +  
  play(col.year))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + play(:year)
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  color(col.continent), play(col.year))
```



“Given all the gapminder years: points, x is gdp on a log scale, y is life, color by continent, played by year.”

Twelve frames, one per year in the data, running from 1952 to 2007. The strip above the panel names the one you are looking at.

38.1 The same sentence, in space

Everything `play` does, `facet` does too, and swapping one operator for the other is the whole difference:

```
gm_four <- gm_all[gm_all$year %in% c(1952, 1972, 1992, 2007), ]
gm_four$period <- factor(gm_four$year)
data(gm_four) + point + x(gdp, scale = "log") + y(life) + color(continent) |
  facet(period)
```



“Given four of the gapminder years: points, x is gdp on a log scale, y is life, color by continent, split into panel columns by period.”

R

```
data(gm_four) + point + x(gdp, scale = "log") + y(life) + color(continent) +
  play(period)
```

Python

```
(data(gm_four) + point + x(col.gdp, scale = "log") + y(col.life) +
  ↪ color(col.continent) +
  play(col.period))
```

Julia

```
data(gm_four) + point + x(:gdp, scale = "log") + y(:life) +
  color(:continent) + play(:period)
```

JavaScript

```
plot(data(gm_four), point, x(col.gdp, { scale: "log" }), y(col.life),
  color(col.continent), play(col.period))
```



Four panels, then four frames, over identical rows. The grid is better for comparing two moments carefully, because both are in front of you and your eye can go back and forth. The sequence is better for seeing *movement*, because a point that shifts left is a point you watch move rather than one you have to find twice. Neither is a different plot; they are one plot read two ways, which is why they are one grammar with one column bound differently.

On paper only the first frame is printed: in the PDF this page shows 1952 with its strip, while the web edition plays. That is why the faceted version is here beside it: the grid is what a reader holding the printed book sees instead of the motion.

38.2 The column's order

`play` and `facet` split the rows the same way, so you might expect the same columns to suit both. The same columns do not suit both. The test is not the column's type. It is whether the column has an order.

A facet lays its panels out in space, and you read space in any order you like. Your eye goes to any panel you choose, then comes back. A sequence lays its pieces out in time, and time arrives in one order only. So a sequence claims something a grid never claims: this frame comes after that one.

Bind a column with no order, and the plot shows movement the data does not have:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(continent) + play(continent)
```

Python

```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.continent) + play(col.continent))
```

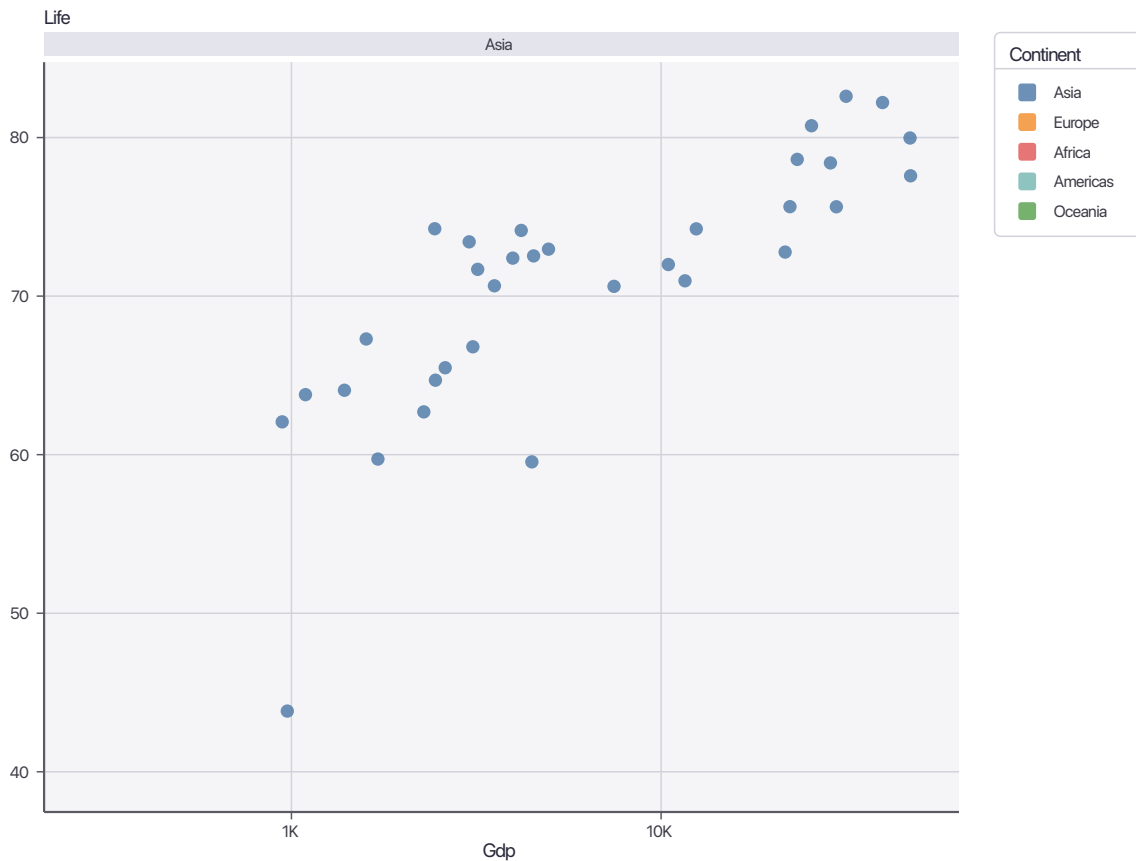
Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + play(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.continent), play(col.continent))
```

`gog: `play(continent)`` has no stated order, so each frame is a snapshot of
↳ one `continent` and the frames run in the order the rows arrive: Asia,
↳ Europe, Africa, Americas, Oceania. A sequence claims that one frame comes
↳ after another. Set the column's factor levels where the data lives, or
↳ use ``facet(continent)``, which claims no order.



Watch the strip: Asia, Europe, Africa, Americas, Oceania. That order is not alphabetical, and it is not geographic. It is the order the continents first appear in the table, because the rows are sorted by country and Afghanistan comes before Albania. The sequence is showing you how a file was sorted. The points jump between frames, and none of the jumps mean anything.

`gog` says so above the plot, and it draws it anyway. Nothing about the sentence is malformed, so the sequence is yours to keep if you meant it. The message names the order it had to choose, and the two answers to it: declare the column's levels, or facet instead. Give the column its levels and the frames follow them. Bind `facet(continent)` instead and the panels claim no order at all.

This is the rainbow-ramp mistake ([Channels](#)) in reverse. There the data had an order the hues could not carry; here the frames carry an order the data does not have, so a reader looks for a pattern that was never there. Written as `| facet(continent)`, the five groups sit side by side and claim nothing about sequence.

Ordered does not mean numeric. A year is ordered, and so is a date. So is a text column whose levels you declared: months in calendar order, or a rating from low to high. `gog` runs

the frames in the order the levels declare, the same way it runs an axis ([Data](#)). A text column with no declared levels takes the order its values first appear in, which is the trap above.

How many values there are is a separate question, and the next section answers it. Twelve frames and a hundred and seventy frames are both worth animating, as long as each one says something new after the one before it. A long sequence costs the reader time, not meaning, and [speed](#) is there for that.

38.3 Too many panels for one page

Whether a column has an order decides whether it can be played. How many values it has decides something else: whether a facet of it is still readable. A facet has to fit every piece on one page. A sequence does not.

Nothing limits how many pieces a sequence shows, because it shows them one at a time.

Having an order is not on its own a reason to play a column. A short sequence should move only when its frames hold the same things in different states. Then a reader can follow one of them across the frames. The four frames above are that case: the same countries at four dates, and the cloud climbs. Four frames holding four different sets of things are not that case. Nothing moves between them. The reader waits, and then has to remember what a facet would have shown at once. This is worth saying plainly, because animation is the choice most people want to make.

The limit is the point where panels stop being readable. They share one page, so every panel added makes every panel smaller. `wrap` folds a long line into a rectangle, which makes each panel wider and shorter ([Faceting](#)). The rectangle is still one page.

Here are the two gliders from [the cube](#), cut into one panel per second instead:

```
thermal_moments <- thermals
thermal_moments$moment <- factor(thermal_moments$second)
data(thermal_moments) + point + x(east) + y(north) + color(glider) |
  facet(moment, wrap = 13)
```



“Given the thermal moments: points, x is east, y is north, color by glider, split into panel columns by moment, wrapped at 13.”

170 panels, already folded into a rectangle. The shapes are still there: two dots in every cell, one for each glider. Their positions are not readable. The tick labels have printed over one another. A dot cannot be read against them, and its position is all this plot measures. A reader can see that something changes, and cannot see what.

The same rows advanced in time instead, one frame per second of flight (speed is explained under How fast, below):

R

```
data(thermals) + point + x(east) + y(north) + color(glider) +
  play(second, speed = 3)
```

Python


```
(data(thermals) + point + x(col.east) + y(col.north) + color(col.glider) +
  play(col.second, speed = 3))
```

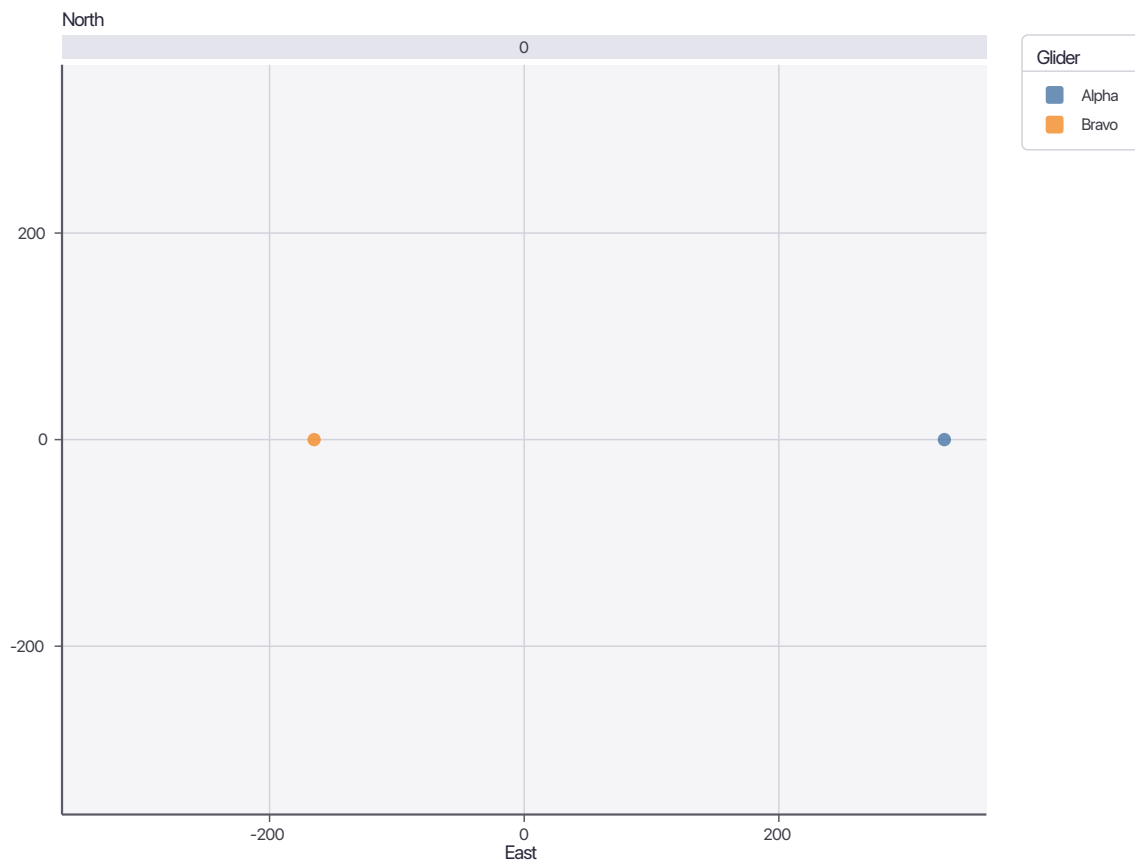
Julia

```
data(thermals) + point + x(:east) + y(:north) + color(:glider) +
  play(:second, speed = 3)
```

JavaScript

```
plot(data(thermals), point, x(col.east), y(col.north), color(col.glider),
  play(col.second, { speed: 3 })))
```

gog: `play(second)` cuts 170 frames, so the animation loops every 45 seconds.
 ↪ Run it faster with `play(second, speed = 6)`, or bind a coarser column.



“Given the thermals: points, x is east, y is north, color by glider, played by second at speed 3.”

One panel at full size, one pair of axes, and the two gliders circling through all 170 frames.

gog drew both of those sentences. It refuses neither, and it never will. A plot is refused for not being well formed, never for being a poor idea ([Design laws](#)). Which of the two says more about gliders is your judgment, not the engine’s. What the engine does instead is name the one cost it can measure. The message above counts the frames and reports how long one loop takes. It offers two answers: [speed](#), or a column with fewer values.

That judgment is worth making before anyone else sees the plot. A facet that was the wrong choice costs a reader one glance. A sequence asks a reader to watch, and keeps them for as long as the loop runs. If it then says nothing, that time was spent for nothing, and a reader can usually tell.

The choice is easiest to see with a large number. Imagine a year of hourly temperatures for New York, drawn as one curve for each day. As a facet that is 365 panels, and no fold leaves a curve large enough to read. As a sequence it is 365 frames, and a reader watches a year of weather pass. Below about ten pieces the panels usually read better, and [Faceting](#) shows ten of them wrapped and readable. Well above that the panels stop being readable, as the 170 above were, and a sequence is the picture that still works.

Holding more does not make a sequence the better picture. What it loses is what a facet does best: two pieces side by side, where a reader compares them instead of remembering them.

38.4 What every frame shares

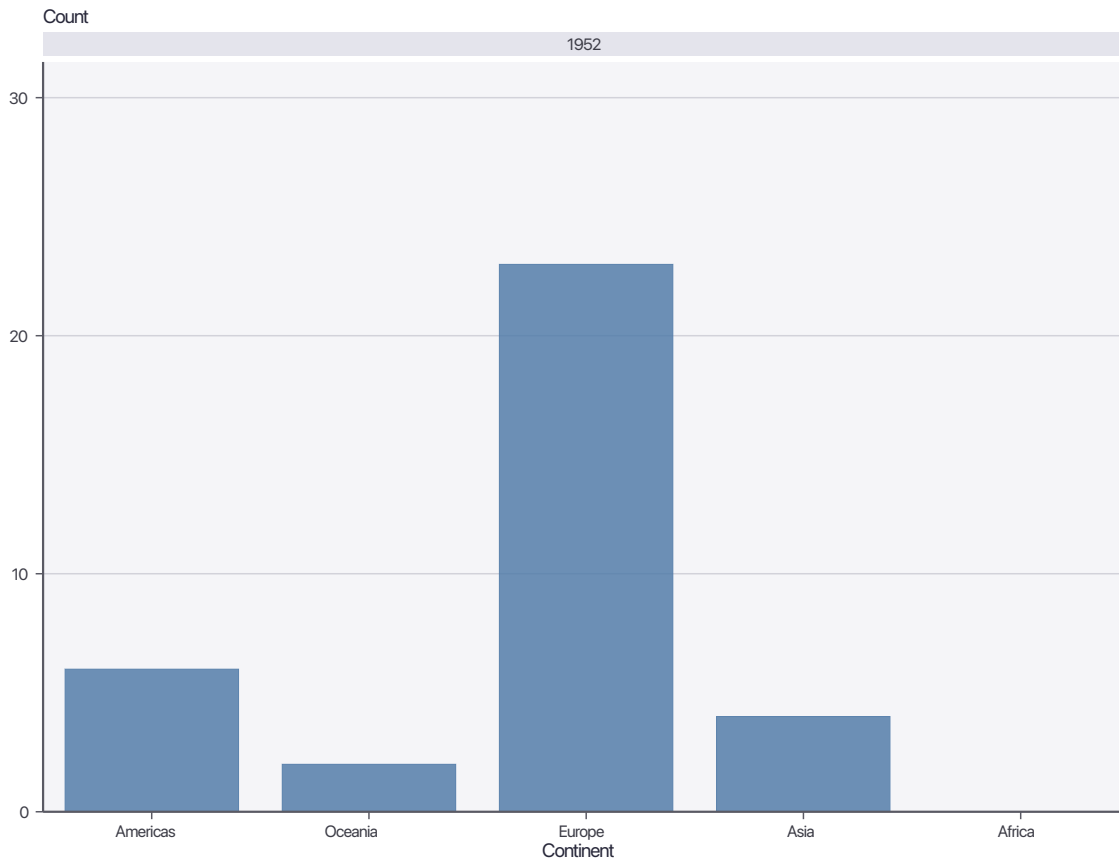
Every scale, the color palette and every legend are fitted across the whole sequence at once, never per frame. That is the property that makes a sequence worth trusting, and it is worth seeing what its absence would look like. If each frame fitted its own axis, 1952’s poorest countries and 2007’s would both sit at the left edge. A world whose incomes tripled would draw as a world that stood still. The only thing actually moving would be the axis, silently, with nothing on the page to say so.

Because the frames share their scales, the opposite reads correctly: in the plot above, the cloud genuinely moves right and up, and the countries that do not move are countries that did not.

The color palette is shared for a second reason. A reader follows a point by its color, so a palette that changed partway through would take away the one thing they are following. Every continent keeps its color in every frame, including frames where it happens to have no rows.

The statistics are not shared. A transform sees each frame’s rows as if they were the whole table, exactly as it sees a facet panel’s rows: the No Exceptions law applied to frames:

```
gm_past60 <- gm_all[gm_all$life > 60, ]
data(gm_past60) + bar * count + x(continent) + play(year)
```



“Given the gapminder past 60 table: bars derived by count, x is continent, played by year.”

Each bar is that year’s tally of the countries that had passed sixty years of life expectancy. Asia climbs from four to thirty, Africa opens with none and ends with twelve, and Europe starts at twenty-three and then runs out of countries to add.

A tally can also have nothing to report, and it is worth seeing that once, because a still sequence is sometimes an answer rather than a fault:

R

```
data(gm_all) + bar * count + x(continent) + play(year)
```

Python

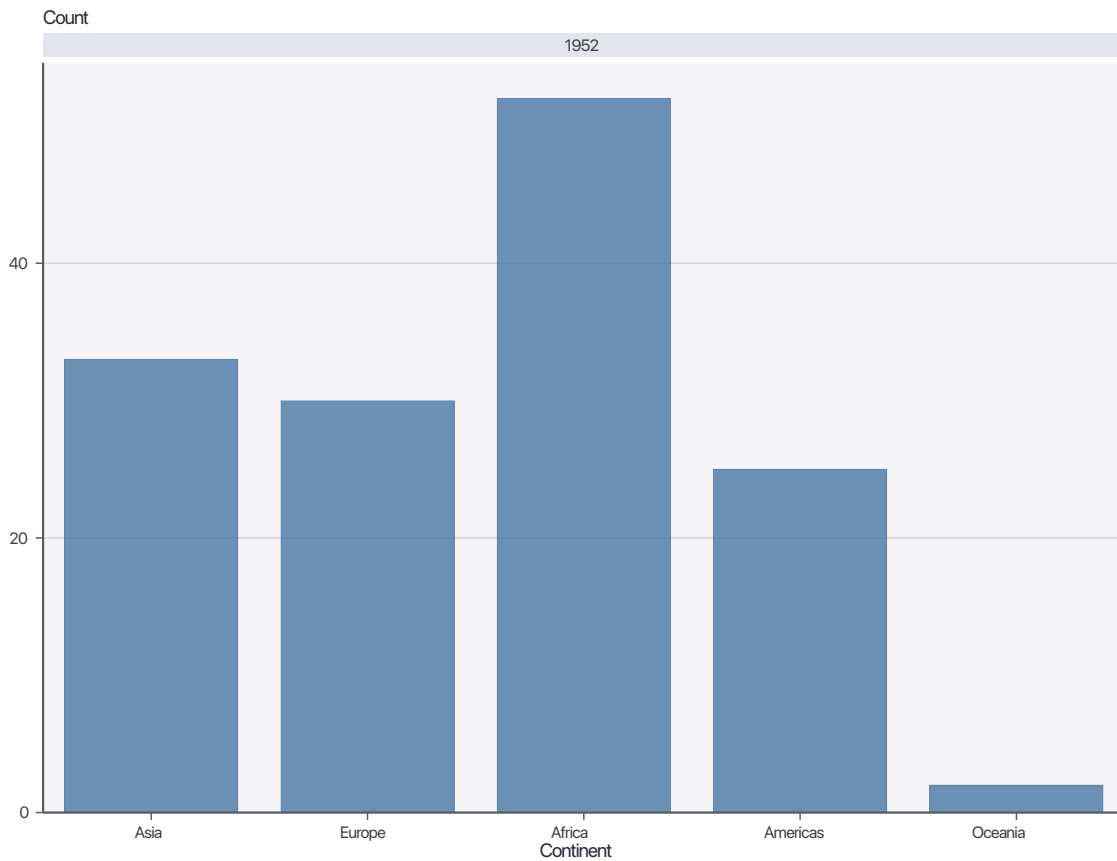
```
data(gm_all) + bar * count + x(col.continent) + play(col.year)
```

Julia

```
data(gm_all) + bar * count + x(:continent) + play(:year)
```

JavaScript

```
plot(data(gm_all), layer(bar, count), x(col.continent), play(col.year))
```



Twelve frames run and not a bar moves. The statistic is being recomputed in every one of them; it keeps arriving at the same number, because gapminder holds the same 142 countries

in every year, so counting rows per continent counts the *table* and not the year. Only the strip changes, which is how the plot shows it. Filtering first, as the sentence before this one does, is what turns a count of countries into a count of something that happened.

What the frames do share is the *cut*, when there is one. A `bin`'s cells describe the axis rather than measuring the rows. One set of bins is cut from every frame's rows at once, and each frame then counts only its own into them. [Faceting](#) explains why, and the rule is the same word for word with “panel” read as “frame”.

R

```
data(gm_all) + bar * bin + x(life) + play(year)
```

Python

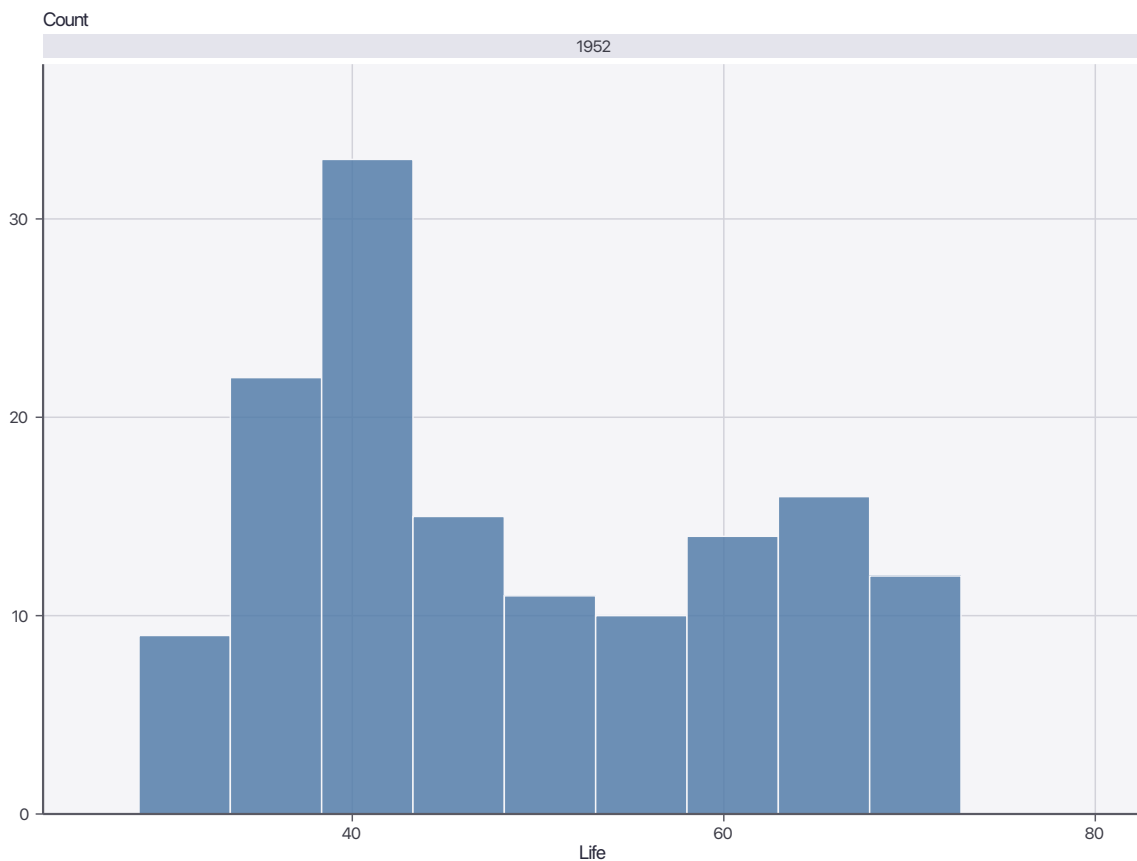
```
data(gm_all) + bar * bin + x(col.life) + play(col.year)
```

Julia

```
data(gm_all) + bar * bin + x(:life) + play(:year)
```

JavaScript

```
plot(data(gm_all), layer(bar, bin), x(col.life), play(col.year))
```



“Given all the gapminder years: bars derived by bin, x is life, played by year.”

The distribution moves right across half a century, and it does so against bars of one width, which is the only way that movement is legible.

38.5 A layer with no frames

`play` is a channel, so it obeys the ordinary `scope` rule: written before any mark it reaches every layer, and written after one it binds that layer alone. A layer that never names it is drawn whole in every frame, which is how a fixed reference sits behind something that moves.

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + play(year) +
  data(life_bands) + rule + y(life) + style(color = "#c0392b")
```

Python

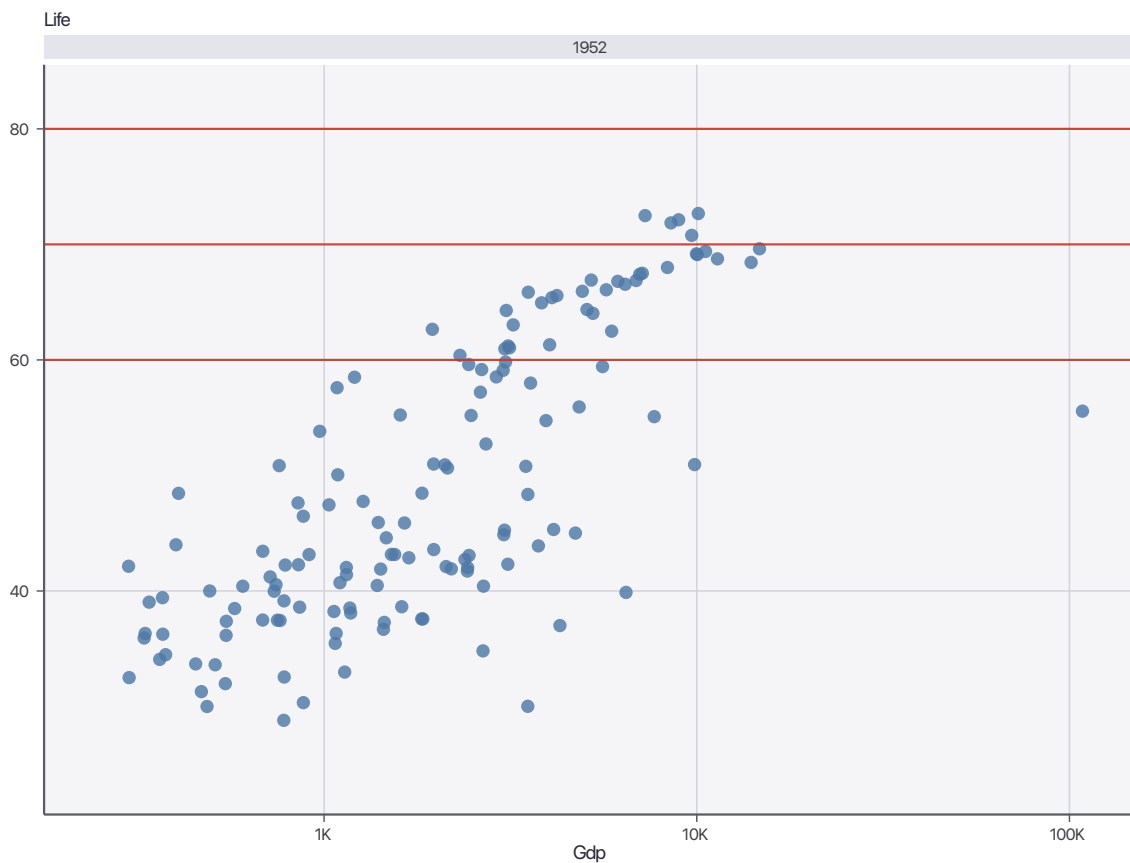
```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +  
  ↪ play(col.year) +  
  data(life_bands) + rule + y(col.life) + style(color = "#c0392b"))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) + play(:year) +  
  data(life_bands) + rule + y(:life) + style(color = "#c0392b")
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  play(col.year), data(life_bands), rule, y(col.life),  
  style({ color: "#c0392b" }))
```



“Given all the gapminder years: points, *x* is gdp on a log scale, *y* is life, played by year; then given the life bands: rules, *y* is life.”

The three lines hold still while the cloud climbs past them. Nothing was added to the grammar to allow that: the reference layer simply never mentioned `year`, and a layer with no `play` binding has no frames to be cut into.

The same scope rule works in the cube and across panels together. `thermal_marks` holds one row per glider per instant, saying where that glider was at that moment:

Table 38.1: `thermal_marks`: first 5 of 34 rows

east	north	altitude	glider	instant
330.0000	0.0000	900.0000	Alpha	0
117.0196	308.5554	970.0817	Alpha	20
-247.0085	218.8305	1040.1634	Alpha	40
-292.2005	-153.3586	1110.2450	Alpha	60
39.7771	-327.5939	1180.3267	Alpha	80

The route is the whole flight, drawn once; the marker is where the glider actually is:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) + color(glider) +
  data(thermal_marks) + point + play(instant, speed = 3) | facet(glider)
```

Python

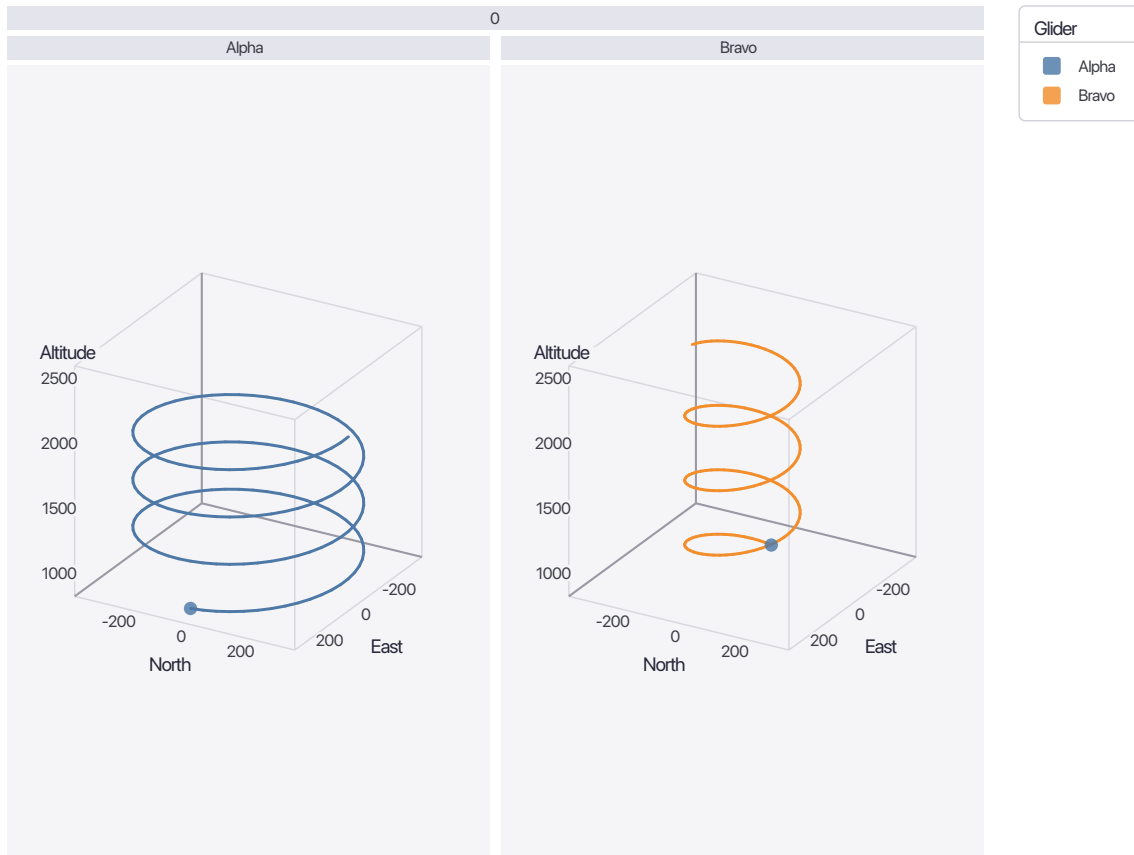
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +
  ↪ color(col.glider) +
  data(thermal_marks) + point + play(col.instant, speed = 3) |
  ↪ facet(col.glider))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +
  color(:glider) + data(thermal_marks) + point +
  play(:instant, speed = 3) | facet(:glider)
```

JavaScript


```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),
     color(col.glider), data(thermal_marks), point,
     play(col.instant, { speed: 3 }), across(col.glider))
```



“Given the thermals: paths, x is east, y is north, z is altitude, color by glider; then given the thermal marks: points, played by instant at speed 3, split into panel columns by glider.”

`play` is bound to the second layer because it is written after `point`, and a channel written after a mark reaches that mark alone. So the coil stands still in both panels while a dot climbs it, and you can see the glider with the tighter coil climbing faster than the wider one, which is the whole reason the two tables are drawn together.

One detail of the second table is worth knowing, because it decides how many frames the plot has rather than what it means. `thermal_marks` calls its time column `instant` where `thermals` calls the same measurement `second`. The frames are gathered from every table that carries the played column, not only from the layer that bound `play`, on the same rule that gives the

plot one scale rather than one per layer. Had the marker also called its column **second**, the backdrop's 170 samples would have set the frame count and the sequence would have run 170 frames instead of 17.

38.6 How fast

speed is a multiple of the normal pace, written on the binding beside **scale** and **limits**, because it belongs to **play** and to nothing else:

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + color(continent) +  
  play(year, speed = 3)
```

Python

```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +  
  ↪ color(col.continent) +  
  play(col.year, speed = 3))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + play(:year, speed = 3)
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  color(col.continent), play(col.year, { speed: 3 }))
```



These are the same twelve frames the chapter opened with, running three times as fast. `speed = 0.5` would run them at half pace. Long sequences are the reason it exists. Past about thirty frames, `gog` reports how many seconds the loop will take. The length of a loop is a default you did not choose and cannot see.

38.7 The column that cannot do both jobs

One sentence is worth knowing, because it looks so reasonable. Bind the same column to `x` and to `play`, on a mark that reads a function along `x`, and every frame then holds a single position:

```
data(gapminder_asia) + line + x(year) + y(life) + color(country) +
  play(year, speed = 3)
```

Error:

```
! gog: `line` reads a function along `x`, but `year` both cuts the plot into
  ↳ frames and supplies `x` - so every frame holds a single `year`, and a
  ↳ line needs at least two positions to read between. Animate a different
  ↳ column - `play` is what the frames advance through, and `x(year)` is what
  ↳ the line draws along. Or use `point`, which places each row on its own
  ↳ and so needs no domain.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A line needs at least two positions to read between. If each frame is one year, there is no domain left in it, and what you get is axes and a legend around an empty panel. The refusal matters because the alternative is silent: a frame strip counting up over an empty panel, with nothing to say why.

The same is true of the same column split across the page rather than the clock, `| facet(year)`, and for the same reason; the engine refuses both, because [panels and frames are one split asked twice](#). Marks that place each row on its own (`point`, `bar`, `box`) have no domain to lose, so they draw in every frame:

R

```
data(gapminder_asia) + point + x(year) + y(life) + color(country) +
  ↳ play(year)
```

Python

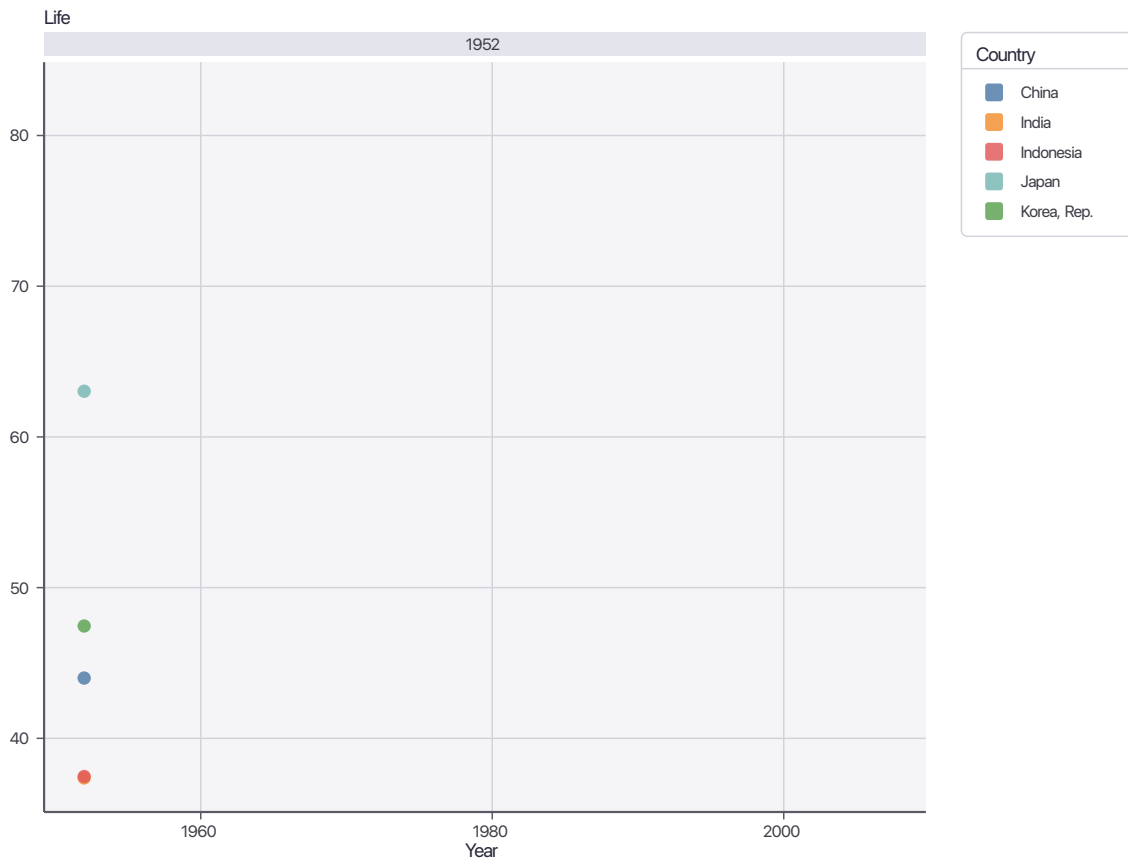
```
data(gapminder_asia) + point + x(col.year) + y(col.life) + color(col.country)
  ↳ + play(col.year)
```

Julia

```
data(gapminder_asia) + point + x(:year) + y(:life) + color(:country) +
  play(:year)
```

JavaScript

```
plot(data(gapminder_asia), point, x(col.year), y(col.life),
  color(col.country), play(col.year))
```



38.8 Crossed with a facet

Panels split the page and frames split the clock, so the two compose and every panel advances together:

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + play(year) |
  facet(continent)
```

Python

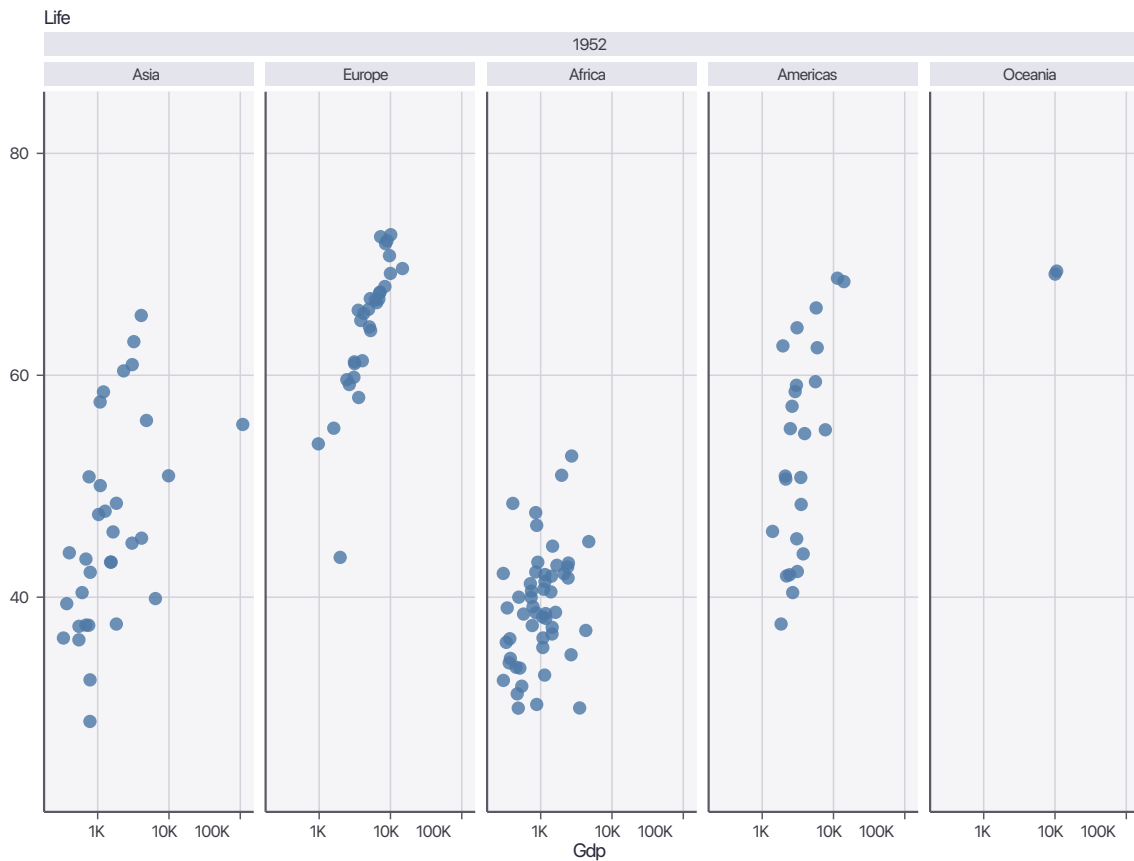
```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +
  ↪ play(col.year) |
  facet(col.continent))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) + play(:year) |  
  facet(:continent)
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  play(col.year), across(col.continent))
```



Five continents, twelve frames, one shared pair of axes. Each panel is its own continent, and the strip above them all names the year they share.

38.9 Frames in the cube

Nothing about `play` belongs to the plane. A frame is a subset of the rows, and the [third position](#) is a way of looking at whatever rows you have, so the two compose, and neither needed anything added for the other:

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + z(population) +  
  color(continent) + play(year)
```

Python

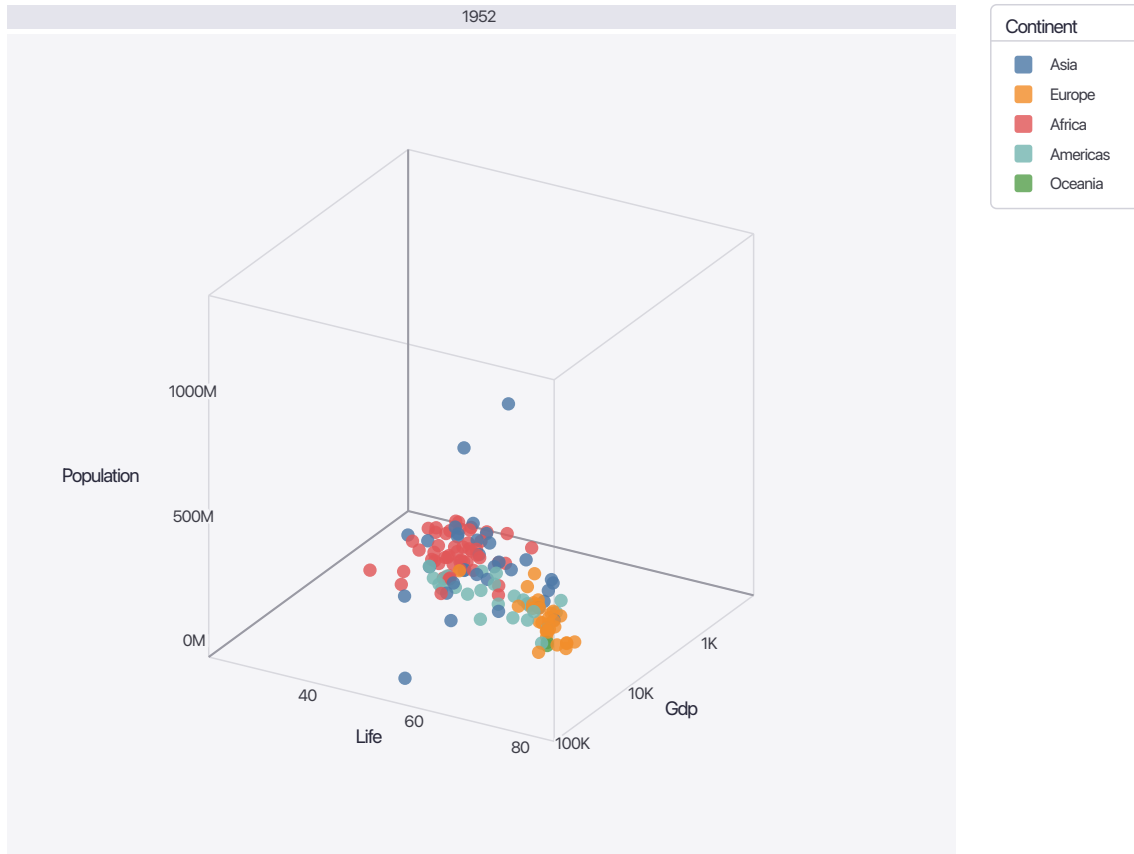
```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +  
  ↪ z(col.population) +  
  color(col.continent) + play(col.year))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) +  
  z(:population) + color(:continent) + play(:year)
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  z(col.population), color(col.continent), play(col.year))
```



“Given all the gapminder years: points, x is gdp on a log scale, y is life, z is population, color by continent, played by year.”

That one is worth reading slowly, because it is the sentence this kernel was designed around. Three of the kernel’s ideas were designed to compose before any of them was drawn: the third position, the sequence, and the selection. The sentence above spends two of them at once. [Selection](#) adds the third to this very sentence, so all three stand in one line.

The cube is fitted once across the whole sequence, exactly as a flat plot’s axes are, and it matters more in a sequence than in a still picture. A box that resized itself per frame would swing its own corners around a cloud that had not moved, and every apparent motion would belong to the guide rather than to the countries. Fitted once, the height axis has room for 2007’s China from the first frame onward, which is why 1952 opens with the cloud pressed low and only China and India standing well clear of the floor.

A transform inside the cube is the same transform. The floor is cut once from every frame’s rows and each frame then counts only its own into the cells, which is this chapter’s shared-cut rule read with a second axis present:

R

```
data(gm_all) + bar * bin(12) + x(life) + y(continent) + color(continent) +  
  palette("okabe") + space() + play(year)
```

Python

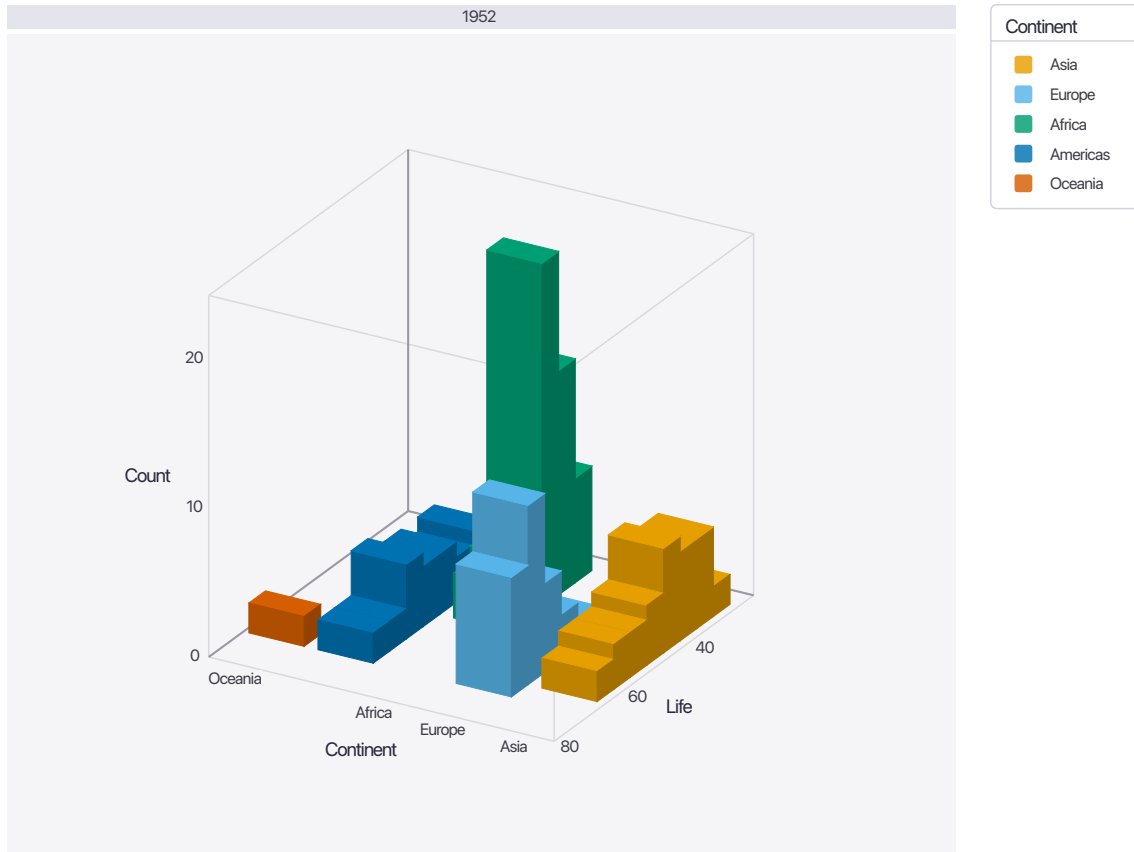
```
(data(gm_all) + bar * bin(12) + x(col.life) + y(col.continent) +  
  ↪ color(col.continent) +  
  palette("okabe") + space() + play(col.year))
```

Julia

```
data(gm_all) + bar * bin(12) + x(:life) + y(:continent) +  
  color(:continent) + palette("okabe") + space() + play(:year)
```

JavaScript

```
plot(data(gm_all), layer(bar, bin(12)), x(col.life), y(col.continent),  
  color(col.continent), palette("okabe"), space(), play(col.year))
```



“Given all the gapminder years: bars derived by bin into 12, x is life, y is continent, color by continent, with the okabe palette, in the cube, played by year.”

Follow one continent’s row of bars rather than the whole floor. Each pile slides toward longer lives as the sequence runs, and it slides against cells that never move, so what you are watching is the countries changing which cell they fall in. `color(continent)` names the same column the y axis does, and it is there so that following one row is possible at all. A floor axis in a cube runs diagonally into the picture. Picking a row off it by eye means tracing a receding edge while five rows of bars block the view. The legend names the row at once; the axis makes you trace it.

The palette is a choice the cube makes for you, and it is the argument the `okabe set` was built on, reached a second way. A solid in space is shaded per face, so one category is already three tones before any two categories are compared, and a palette whose hues sit close together loses the difference that shading has already reduced. Okabe and Ito’s eight are spaced for readers who cannot separate some hues at all, which turns out to be the same requirement as separating them after a third of the light has been taken out.

What the frames do not touch is the viewing angle. `space(turn = , tilt = )` is a property of the space rather than of the data, so one angle serves the whole sequence and every frame is seen from the same place. That is a design choice, not a gap: a viewpoint that swung while the sequence played would be motion that no column asked for, and a reader would then have to subtract it from the motion that does carry one. `play` moves the data past a fixed viewpoint.

You can still turn the plot yourself while it runs, and that is a different thing. The design choice above is about a viewpoint that moves on its own, with no column behind it. A reader who drags knows which motion is theirs, and can stop it. This is worth more here than anywhere else in the book. Something is moving inside a solid shape. The only way to see what the near edge hides is to look around it.

Panels and frames still compose in the cube, and the plot below shows it. They are one split asked twice, so crossing them costs nothing: the panels divide the page, the frames divide the clock, and each panel projects the cube its own rectangle has room for.

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + z(population) +  
  color(continent) + play(year) | facet(continent, wrap = 3)
```

Python

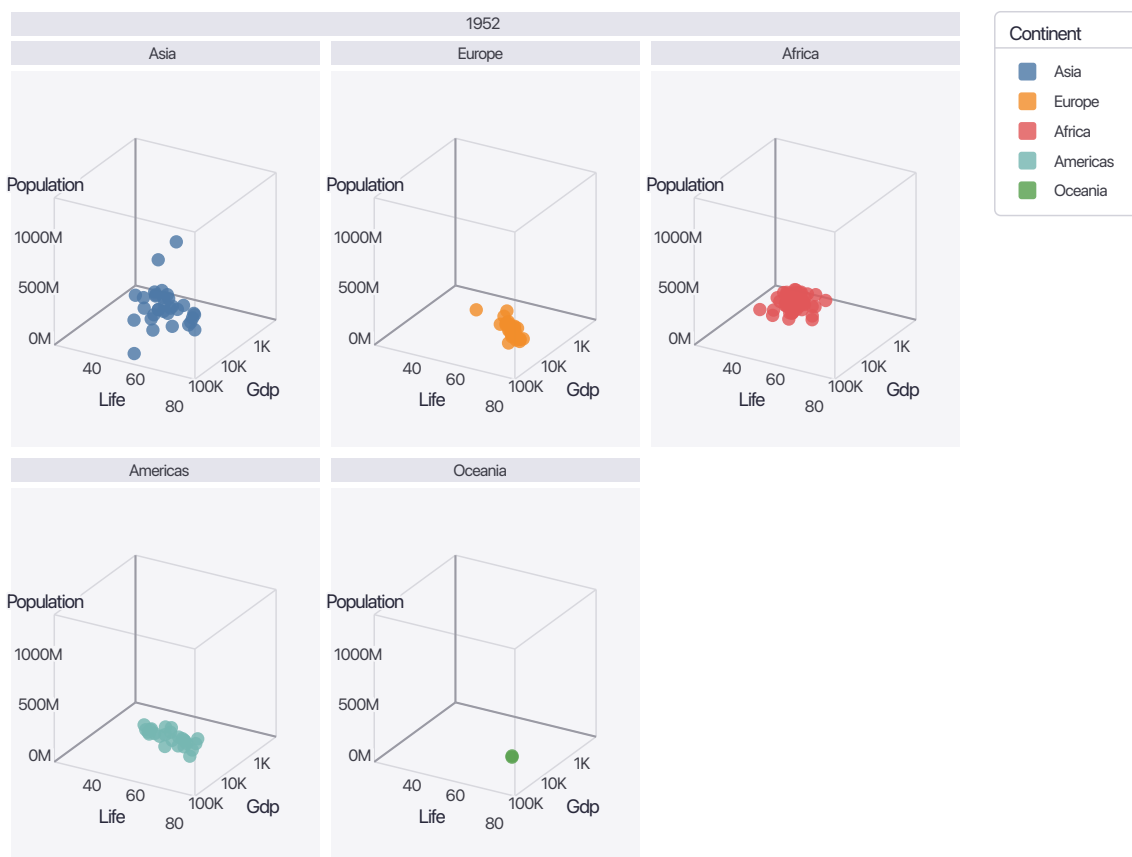
```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +  
  ↪ z(col.population) +  
  color(col.continent) + play(col.year) | facet(col.continent, wrap = 3))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) +  
  z(:population) + color(:continent) + play(:year) |  
  facet(:continent, wrap = 3)
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  z(col.population), color(col.continent), play(col.year),  
  across(col.continent, { wrap: 3 })))
```



Five cubes, twelve frames, and one strip above them all naming the year, because a frame is the whole plot, while a panel is one piece of it. That is four of the kernel's ideas in one sentence: the third position, the panels, the frames, and a scale shared across every one of them so the cubes stay comparable while they run.

And the sheet moves too. `ripples` describes a wave tank tapped in one corner, computed on a grid at twelve instants, in a shallow tank and a deep one.

Table 38.2: `ripples`: first 5 of 5400 rows

across	along	height	tank	instant
20	20	3.547	Shallow	1
40	20	5.845	Shallow	1
60	20	4.353	Shallow	1
80	20	-0.178	Shallow	1
100	20	-4.265	Shallow	1

One row is one grid node at one instant in one tank, so `instant` counts frames from 1 to 12 rather than measuring a time, and `height` is a signed height in millimeters. Depth sets how fast water carries a wave, so the same tapping makes crests 140 mm apart in the shallow tank and 220 mm apart in the deep one:

R

```
data(ripples) + surface + x(across) + y(along) + z(height) +  
  color(height) + palette("plasma") + play(instant, speed = 4) | facet(tank)
```

Python

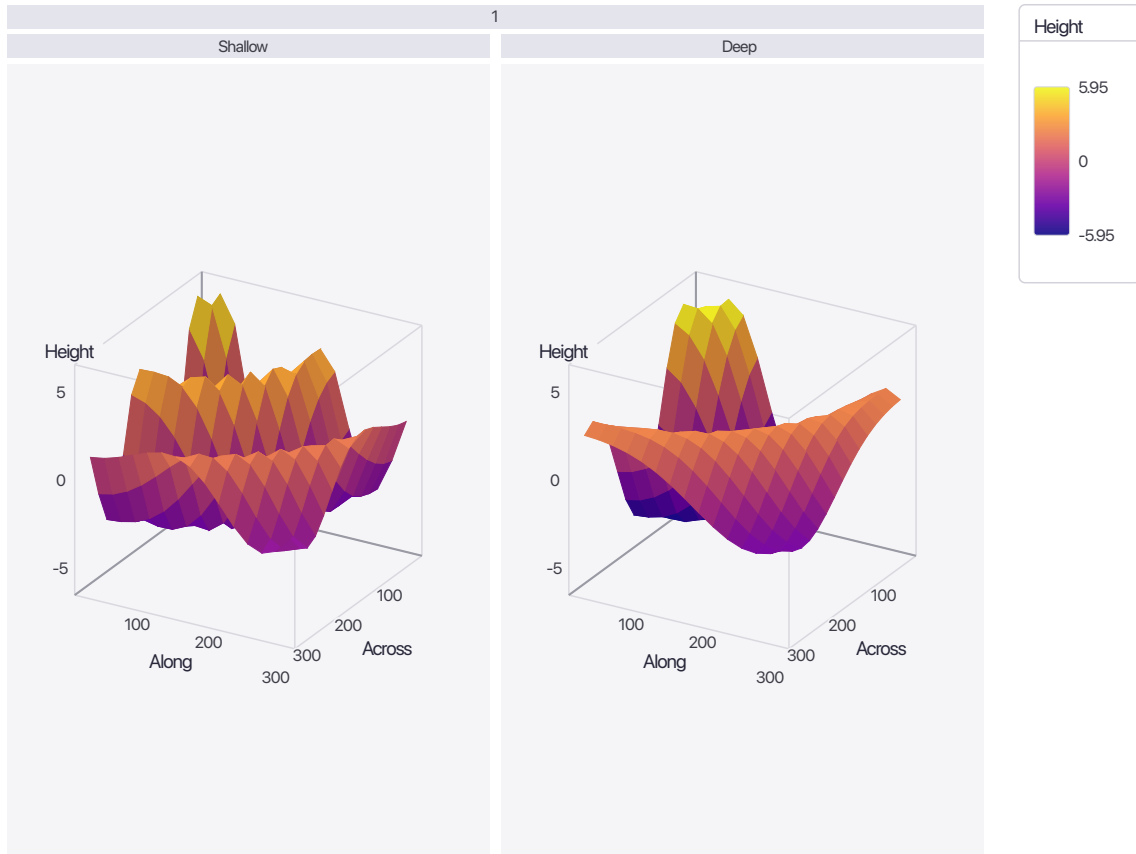
```
(data(ripples) + surface + x(col.across) + y(col.along) + z(col.height) +  
  color(col.height) + palette("plasma") + play(col.instant, speed = 4) |  
  ↪ facet(col.tank))
```

Julia

```
data(ripples) + surface + x(:across) + y(:along) + z(:height) +  
  color(:height) + palette("plasma") + play(:instant, speed = 4) |  
  facet(:tank)
```

JavaScript

```
plot(data(ripples), surface, x(col.across), y(col.along), z(col.height),  
  color(col.height), palette("plasma"), play(col.instant, { speed: 4 }),  
  across(col.tank))
```



“Given the ripples: a surface, x is across, y is along, z is height, color by height, with the plasma palette, played by instant at speed 4, split into panel columns by tank.”

Watch one arc leave the corner and cross the floor. It is the same tapping in both panels and it is drawn against one shared height scale, so the difference you see is a wavelength and not a scaling. **speed = 4** matters here: at the normal pace twelve frames take nearly ten seconds, which is a slideshow of a wave rather than a wave.

The height is said twice, once as the third position and once as the color, and that is deliberate rather than a redundancy. A sheet seen from one angle hides part of itself: the far corner is foreshortened and the near crests stand in front of the troughs behind them, so a shape that is plain at the front is guesswork at the back. Bright crest bands survive the projection where the height alone does not, and the wavelength this plot exists to compare is the spacing of those bands. `palette("plasma")` runs dark blue through magenta and orange to yellow; [Channels](#) has the rest, including two diverging ramps whose neutral color would sit at the still-water line if the sign of the height were the point.

38.10 Frames in polar

The next coordinate space works the same way, and for the same reason. `polar()` bends the panel, the frames choose the rows, and neither has to know about the other:

R

```
data(gm_all) + bar * mean + x(continent) + y(life) + polar() + play(year)
```

Python

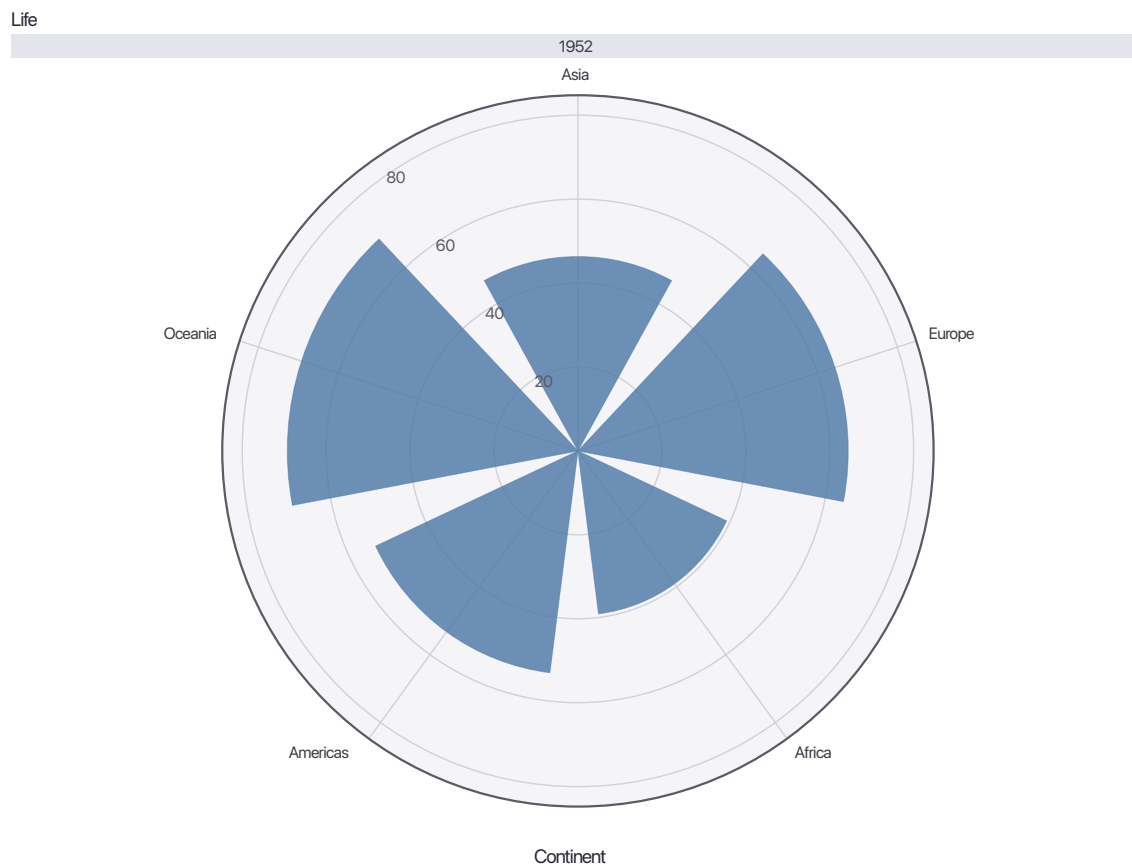
```
data(gm_all) + bar * mean + x(col.continent) + y(col.life) + polar() +  
  ↪ play(col.year)
```

Julia

```
data(gm_all) + bar * mean + x(:continent) + y(:life) + polar() +  
  play(:year)
```

JavaScript

```
plot(data(gm_all), layer(bar, mean), x(col.continent), y(col.life),  
  polar(), play(col.year))
```



“Given all the gapminder years: bars derived by mean, x is continent, y is life, in polar, played by year.”

A rose of mean life expectancy, one petal per continent, opening across half a century. The radial axis is fitted over the whole sequence exactly as a flat one is, which is why the ring at 80 is already drawn in 1952 with nothing reaching it: the petals grow inside a circle that does not resize under them. Asia’s is the one to watch, from 46 years to 71. Africa’s climbs steadily until the late 1980s and then stops for fifteen years, which is a pause the average reports rather than smooths away.

The pie is where that shared axis says something surprising, and it is worth following, because it is the flat rule bent rather than a rule of its own:

R

```
data(gm_all) + bar * sum * stack + y(population) + color(continent) + polar()
  ↪ +
  play(year)
```


Python

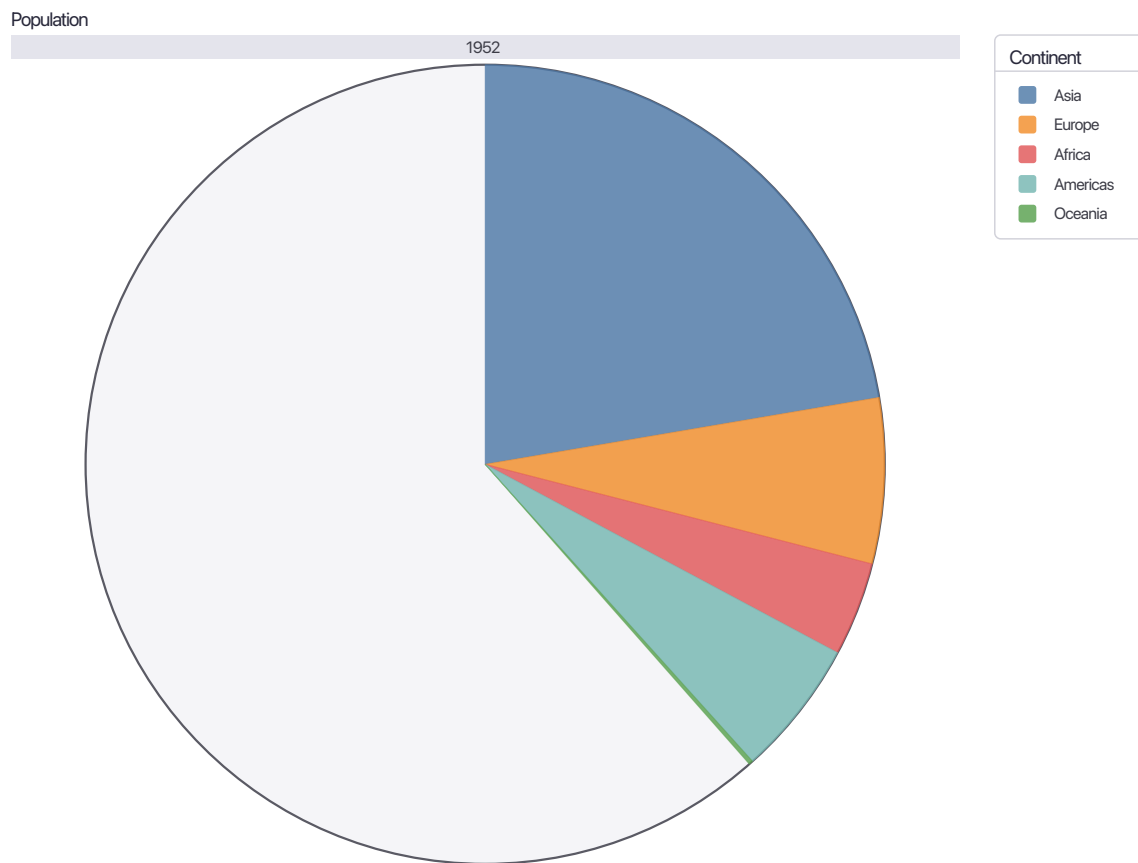
```
(data(gm_all) + bar * sum * stack + y(col.population) + color(col.continent)
↪ + polar() +
play(col.year))
```

Julia

```
data(gm_all) + bar * sum * stack + y(:population) + color(:continent) +
polar() + play(:year)
```

JavaScript

```
plot(data(gm_all), layer(bar, sum, stack), y(col.population),
color(col.continent), polar(), play(col.year))
```



“Given all the gapminder years: bars derived by sum and stack, y is population, color by continent, in polar, played by year.”

The circle does not close until the last frame. A pie’s turn *is* its measure axis, that axis is fitted across the sequence like every other, and 1952’s world population is a little under two fifths of 2007’s, so 1952 sweeps a little under two fifths of the turn. The identical sentence without `polar()` draws a stacked column that stops well short of the top of its axis, and nobody finds that surprising. A pie that does not close is that stacked bar bent.

If the question is each year’s *shares* rather than each year’s total, the sentence can say so, and then every frame closes:

R

```
data(gm_all) + bar * sum * stack(share = TRUE) + y(population) +  
  ↪ color(continent) +  
  polar() + play(year)
```

Python

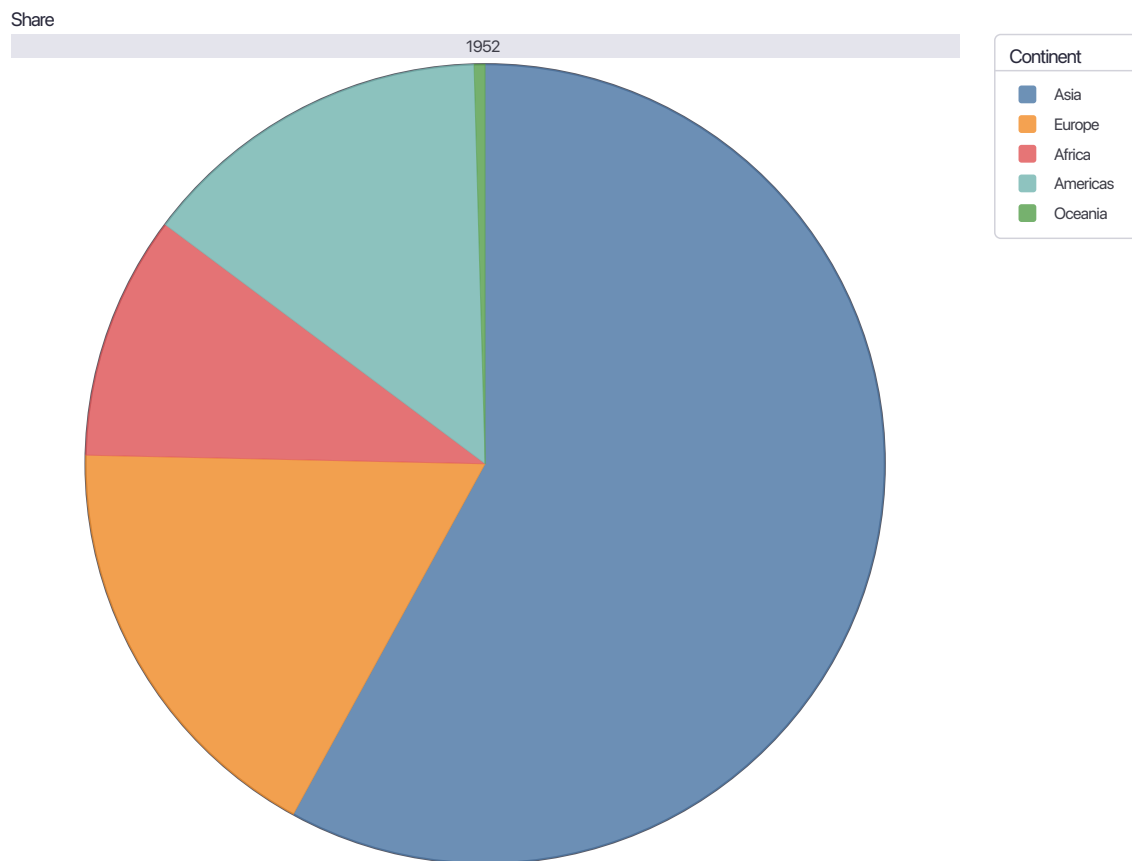
```
(data(gm_all) + bar * sum * stack(share = True) + y(col.population) +  
  ↪ color(col.continent) +  
  polar() + play(col.year))
```

Julia

```
data(gm_all) + bar * sum * stack(share = true) + y(:population) +  
  color(:continent) + polar() + play(:year)
```

JavaScript

```
plot(data(gm_all), layer(bar, sum, stack({ share: true })),  
  y(col.population), color(col.continent), polar(), play(col.year))
```



“Given all the gapminder years: bars derived by sum and stack as shares, y is population, color by continent, in polar, played by year.”

The turn is one whole in every frame now, and what moves is how it divides: Asia widens a little from 58% to 61%, Europe falls from 17% to 9%, and Africa grows by half, from 10% to 15%. Two sentences, one atom apart, answering two different questions about the same column. Choosing between them is the reader’s job and the grammar’s business is to keep both sayable.

38.11 Taking the sequence with you

Every plot on this page moves because the SVG carries its own timing. A browser reads that timing. Most other places do not. A message to a friend, a slide in a talk, a post on a social site: each of them shows the first frame and stops.

`save_gif()` writes the same sequence as a file that those places will play:

```

wave <- file.path(tempdir(), "wave.gif")
save_gif(data(ripples) + surface + x(across) + y(along) + z(height) +
         color(height) + palette("plasma") + play(instant, speed = 4) |
         facet(tank),
         wave, scale = 2)
cat(basename(wave), round(file.size(wave) / 1024), "KB\n")

```

wave.gif 856 KB

Nothing has to be installed first. The engine converts and encodes the file itself, so the same line works on every machine that can draw the plot at all.

scale multiplies the canvas. A plot is 800 by 600 pixels unless its theme says otherwise, which is small for a post, so **scale = 2** doubles both numbers. The pace is kept without being restated: this sequence was written **speed = 4**, and each frame holds for a fifth of a second in the file as it does on the page.

The file cannot disagree with the plot beside it, and the reason is worth one paragraph, because it is the reason [Design laws](#) gives for having one renderer. The frames are not drawn again one at a time. The plot is drawn once, with every scale, the color map and each legend fitted across the whole sequence, and what you see at any time is that single drawing with one frame left showing. So there is no second set of decisions that could come out different.

Two sentences are refused. A plot with no `play()` has no frames to write, and saying so is more useful than writing a file of one frame:

```

still <- data(gapminder_2007) + point + x(gdp) + y(life)
save_gif(still, file.path(tempdir(), "still.gif"))

```

Error:

```

! gog: this plot does not play, so it has no moments to write. Bind a column
  ↳ with an order to `play` - `play(year)` - or save the still picture it
  ↳ already is.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

```

And the path has to end in `.gif`, because the name of a file is a claim about what is inside it:

```

moving <- data(gm_all) + point + x(gdp) + y(life) + play(year)
save_gif(moving, "wave.png")

```

Error:

```

! gog: `save_gif()` writes a GIF, so the path ends in `.gif` - `save_gif(p,
  ↳ "wave.gif")`.

```

One thing changes when the file is written, and it is worth knowing before you send the file rather than after. A GIF is pixels. The text stops being text, so it no longer sharpens when a reader zooms in, and the colors are fitted to a palette of 256. The SVG remains the plot itself. The GIF is a copy of it made for somewhere that cannot read the original.

38.11.1 Why a GIF and not a video

A video file would be smaller and sharper than a GIF, and a few places accept video and nothing else. `gog` writes no video, and the reason is about what a package can carry rather than about what a plot is.

Making a video needs a video encoder. The one nearly everybody uses is more than ten times the size of this whole engine, once for every platform a package supports, and its license does not allow it to travel inside a package licensed the way this one is. The other route is to require the reader to install it separately. That sounds harmless until you count the readers who cannot. A reader who installs a prebuilt package rather than building from source gets whatever the package carries, so a tool that cannot fit inside it never reaches them at all. The feature would work for some readers and refuse for others, which is a worse answer than not having it.

A GIF asks for nothing. The engine writes it on every machine that can draw the plot, in every one of the four languages. When the file looks soft, raise `scale`. When you genuinely need a video, converting a GIF into one is easy, and free converters online do it in one step.

38.12 What it refuses

One column names the panels or names the frames, and not both. If it were drawn, one panel's rows would fall in one frame and every other panel would be empty:

```
data(gm_four) + point + x(gdp) + y(life) + play(period) | facet(period)
```

Error:

```
! gog: `period` cannot both name the frames and name the panels - every panel
  ↳ but one would be empty in every frame. Split the page by one column and
  ↳ the time by another, or drop `| facet(period)` and let `play(period)`
  ↳ carry it.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A speed that is not a pace is refused at the binding that carries it:

```
data(gm_all) + point + x(gdp) + y(life) + play(year, speed = 0)
```

Error:

```
! gog: `speed = 0` - a speed is a multiple of the normal pace, so it has to  
↪ be above zero. `speed = 2` is twice as fast, `speed = 0.5` half.
```

`speed` belongs to `play` and to nothing else, because nothing else in the grammar takes time to draw: a color is drawn all at once, so there is no duration for a multiplier to scale. A refusal for that mistake is not shown here, and the reason is worth saying. None of the four packages gives any other channel a `speed` argument at all, so `color(continent, speed = 2)` fails in R (or in Python, or Julia, or JavaScript) before a specification is ever built. The engine still checks it, because a specification can also be written by hand, but you will never see that message from a sentence: the language you are writing in refuses first, which is the earlier and better place for it.

38.13 What the page adds

A played plot on a web page carries three more buttons. They sit on the same line as the four every plot has, a little to the right of them, and they control the clock:

Control	What it does
the left arrow	Goes back one frame, and stops the clock
the pause	Stops the clock. It becomes a play arrow, which starts it again
the right arrow	Goes forward one frame, and stops the clock

Stepping stops the clock, and it has to. A running clock would move past the frame you just asked for, in less than a second. So the arrows stop it, and the middle button is how you start it again.

The two ends are connected. Go back from the first frame and you arrive at the last. The sequence already runs that way: it loops, so the last frame is followed by the first, every time. Buttons that stopped at the ends would disagree with the sequence they control.

Nothing displays the frame you are on, because the strip above the panel already names it. That strip is the sequence's guide, drawn on every played plot, the way an axis is drawn for `x`.

None of the three is in the sentence, and that is the rule rather than an omission. Every frame is already in the file: the whole sequence is drawn once, and a clock decides which frame you see at each moment. Stepping moves that clock. It chooses no rows, hides no rows, and changes

nothing the sentence said. So it is like turning a cube ([Space](#)), which changes how you look and never what is drawn. Neither one needs a word in the grammar.

Stop the clock and the camera photographs the frame you stopped on. That is the camera's own rule holding rather than a rule about animation: it saves what you are looking at, the same way it saves how far you have zoomed in.

On paper, and in the PDF edition, a played plot is the first frame with its strip. The buttons are the page's, so a printed sequence has none, and none is missing there.

38.14 Cuts between frames

A sequence cuts. Frames do not glide into one another: a point in 1957 and a point in 1962 are two complete pictures shown in turn, not one point eased between two positions. Easing needs a **key**, something that says Afghanistan in one frame is the same country as Afghanistan in the next. The grammar has no such word. **group** looks like the word until you notice that a **point** does not take it, and the point everyone pictures gliding is exactly that: a **point**. So the difference is a word the vocabulary does not have, not a picture the engine cannot draw.

39 Selection

Which of these points are the rich countries, and where do they sit? Finding them among 142 dots takes a long time. Add one word and the reader can draw a box around any group, and the rest are dimmed:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent) +  
  theme(width = 620) + brush
```

Python

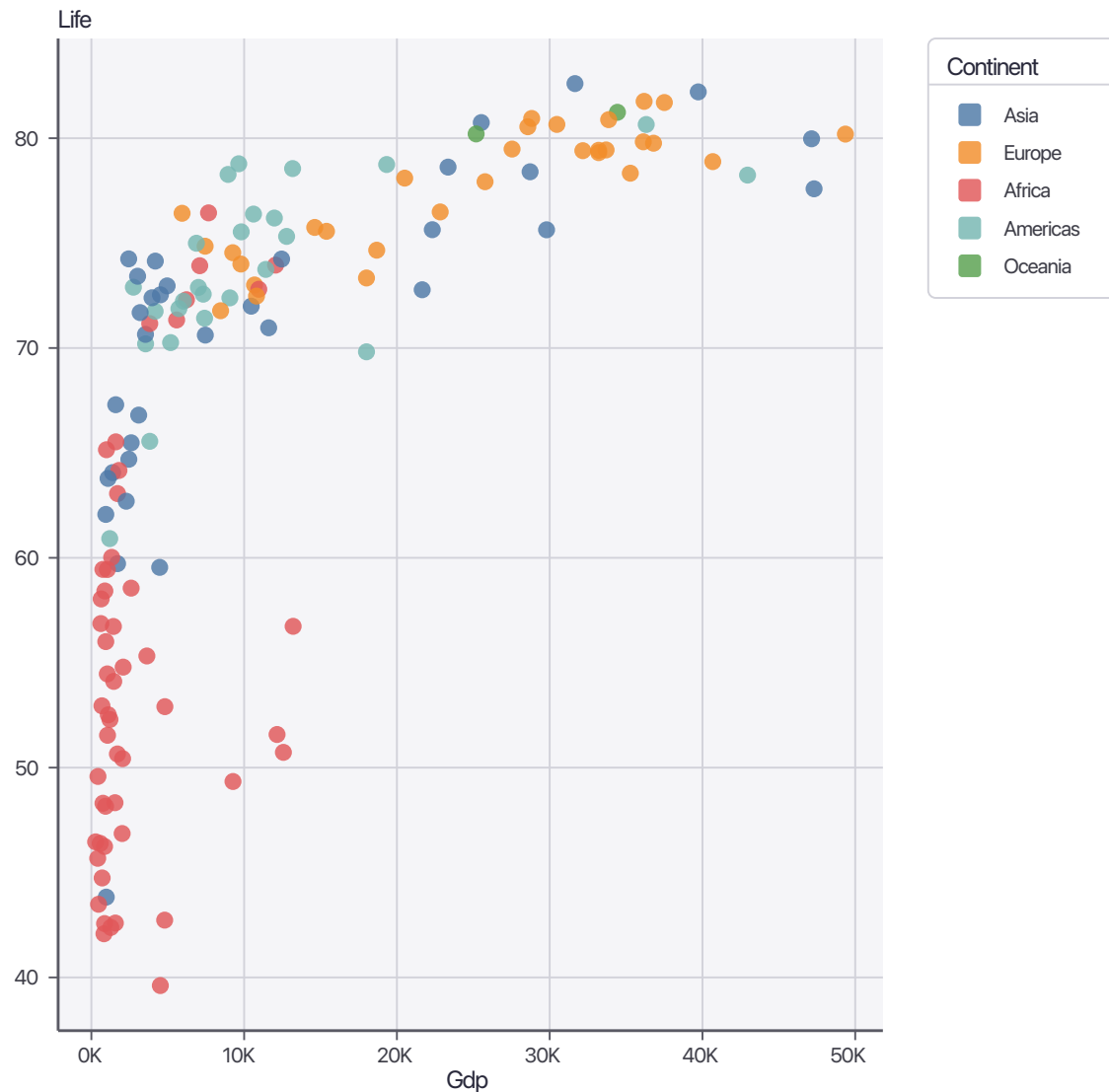
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ color(col.continent) +  
  theme(width = 620) + brush)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  theme(width = 620) + brush
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent), theme({ width: 620 }), brush)
```

“Given gapminder 2007: points, x is gdp, y is life, color by continent, and let the reader select.”

On a page that can run the engine, drag across that plot. On paper it is the scatter itself, because nothing is selected until someone selects it.

Bare **brush** means *the positions this plot binds*, so here it is a region over **gdp** and **life** together. It says the plot is explorable without deciding in advance what is worth exploring.

Every country is still drawn while you drag. The ones inside your box keep their color and the rest are dimmed, so the group is read *against* what it was taken from. Remove the other rows

and the plot no longer shows that the rich countries sit at the top right of the whole world.

39.1 Saying where the selection starts

Name a column and the brush states a **bound** on it: the two values the selection starts between. That is how you point at something rather than let a reader look for it:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent) +  
  theme(width = 620) + brush(gdp, at = c(20000, 60000))
```

Python

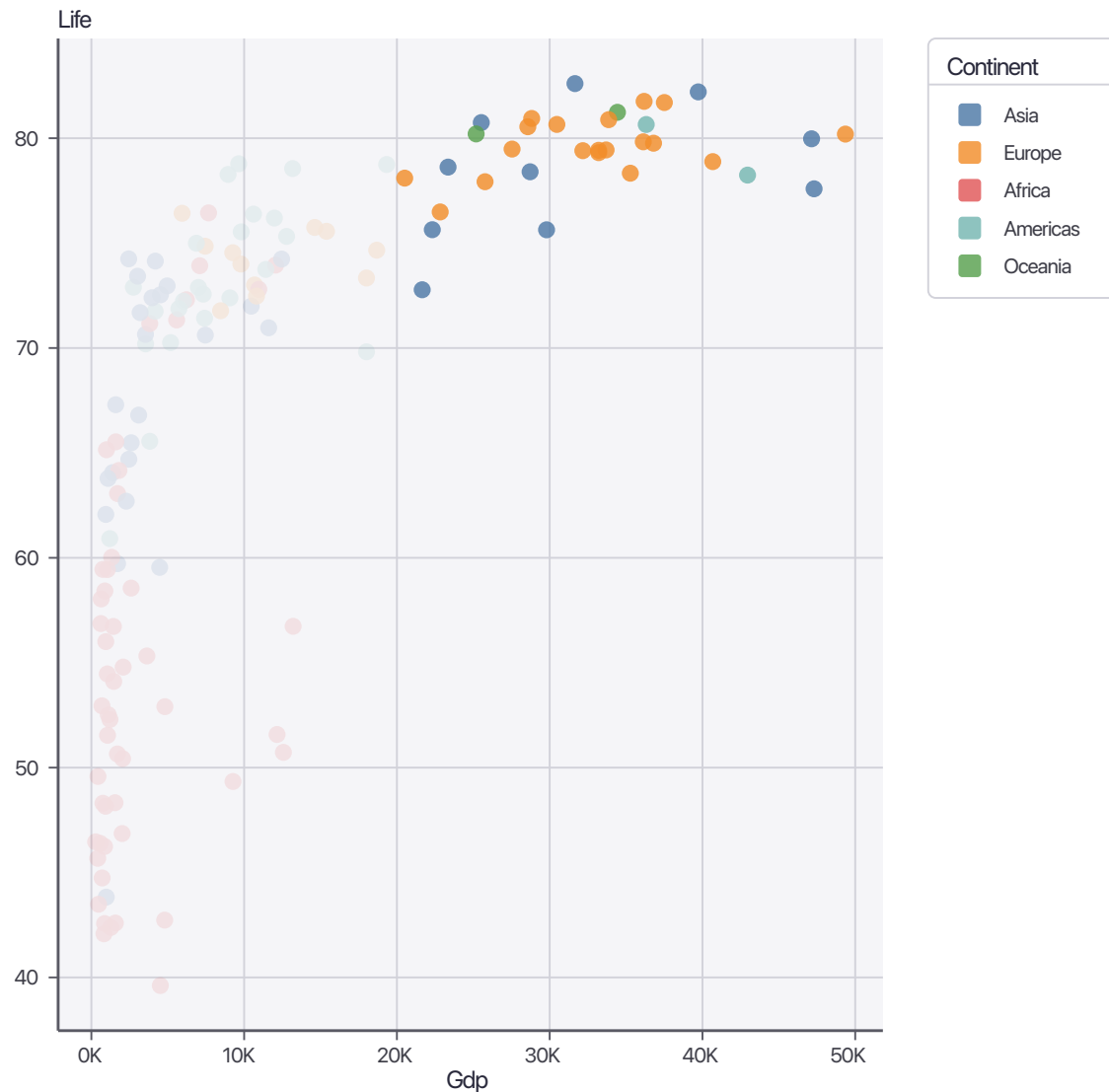
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ color(col.continent) +  
  theme(width = 620) + brush(col.gdp, at = [20000, 60000]))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +  
  theme(width = 620) + brush(:gdp, at = [20000, 60000])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  color(col.continent), theme({ width: 620 }),  
  brush(col.gdp, { at: [20000, 60000] }))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent, and let the reader select gdp, starting from 20000 to 60000.”

The thirty or so countries above 20,000 dollars keep their color, and you can see at once that they sit almost entirely above 75 years. That is closer to an annotation than to a tool: the sentence has done the selecting, and a reader can still drag to move the bound.

The two forms answer different questions, and the difference is who chooses. **brush** hands the choice to the reader. **brush(gdp, at = ...)** makes the choice in the sentence, so the selection appears on paper too.

39.2 Highlighting, never removing rows

This is the one rule to learn, and it separates **brush** from an atom you already know. **limits** also takes a range in a column's own units. It does something different with it.

limits runs *before* the statistics. It removes rows, and the transform then sees the survivors as if they were the whole data. **brush** runs *after* the picture is composed. It removes nothing and recomputes nothing.

One range, one mark, two outcomes:

R

```
data(gapminder_2007) + point + x(gdp, limits = c(20000, 60000)) + y(life) +  
  theme(width = 620, height = 300) + title("limits: 109 countries gone")
```

Python

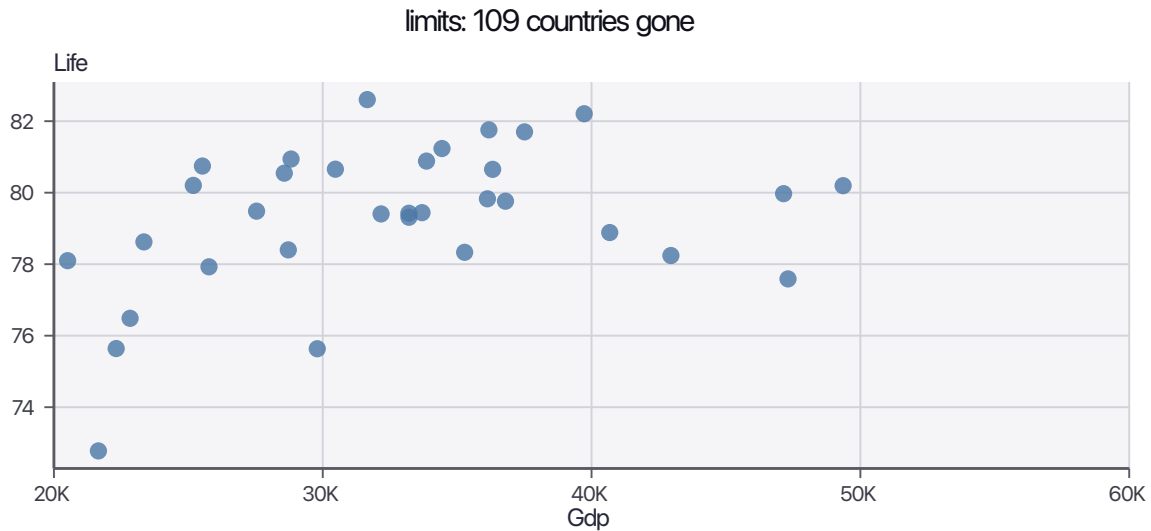
```
(data(gapminder_2007) + point + x(col.gdp, limits = [20000, 60000])) +  
  ↪ y(col.life) +  
  theme(width = 620, height = 300) + title("limits: 109 countries gone")
```

Julia

```
data(gapminder_2007) + point + x(:gdp, limits = [20000, 60000]) +  
  y(:life) + theme(width = 620, height = 300) +  
  title("limits: 109 countries gone")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { limits: [20000, 60000] }),  
  y(col.life), theme({ width: 620, height: 300 }),  
  title("limits: 109 countries gone"))
```



“Given gapminder 2007: points, x is gdp from 20000 to 60000, y is life.”

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  theme(width = 620, height = 300) + brush(gdp, at = c(20000, 60000)) +
  title("brush: all 142 still here")
```

Python

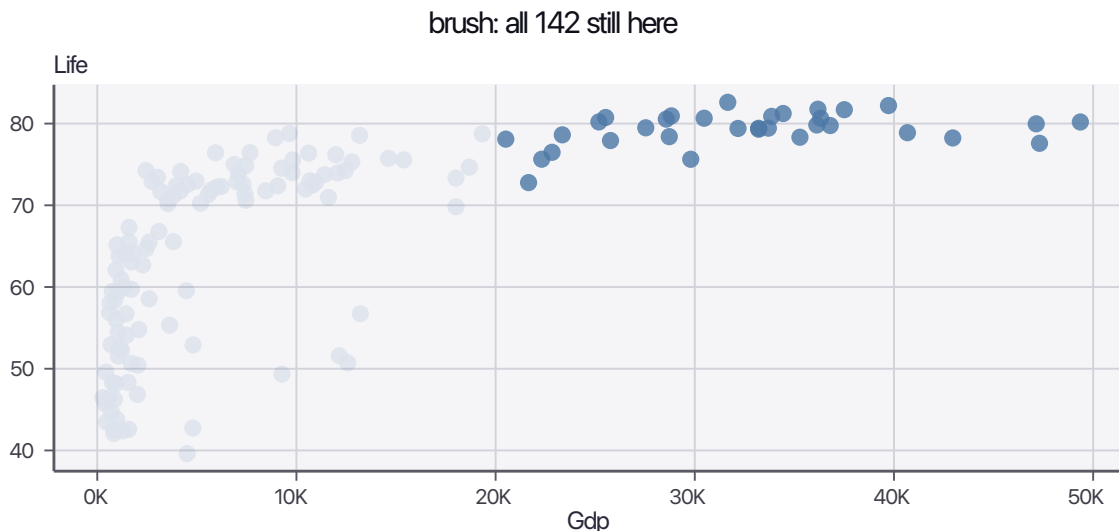
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  theme(width = 620, height = 300) + brush(col.gdp, at = [20000, 60000])) +
  title("brush: all 142 still here"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +
  theme(width = 620, height = 300) + brush(:gdp, at = [20000, 60000]) +
  title("brush: all 142 still here")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
     theme({ width: 620, height: 300 }),
     brush(col.gdp, { at: [20000, 60000] } ),
     title("brush: all 142 still here"))
```



The first plot is a different plot. Its axis runs from 20,000 to 60,000, only the countries inside that range are drawn, and both axes were fitted to the ones that survived. Look at its y axis: the lowest number on it is 74, because every country with a low life expectancy was removed. `gog` prints how many it removed rather than removing them in silence.

The second is the whole world, with one group keeping its color. Its axes are the axes of all 142 countries, so the selection sits where it always sat. Stacked rather than side by side on purpose: the two share no fitted axis, and setting them next to each other invites a reader to compare positions that were fitted to different data.

Which of the two you want is a genuine choice and the grammar keeps both sayable. Ask *how does this subgroup look on its own* and you want `limits`. Ask *where does this subgroup sit inside the whole* and you want `brush`.

39.3 Two bounds make a rectangle

One `brush` is one column. Write two, and a row has to satisfy both:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) + color(continent) +
  theme(width = 620) + brush(gdp, at = c(20000, 60000)) + brush(life, at =
↪ c(78, 85))
```

Python

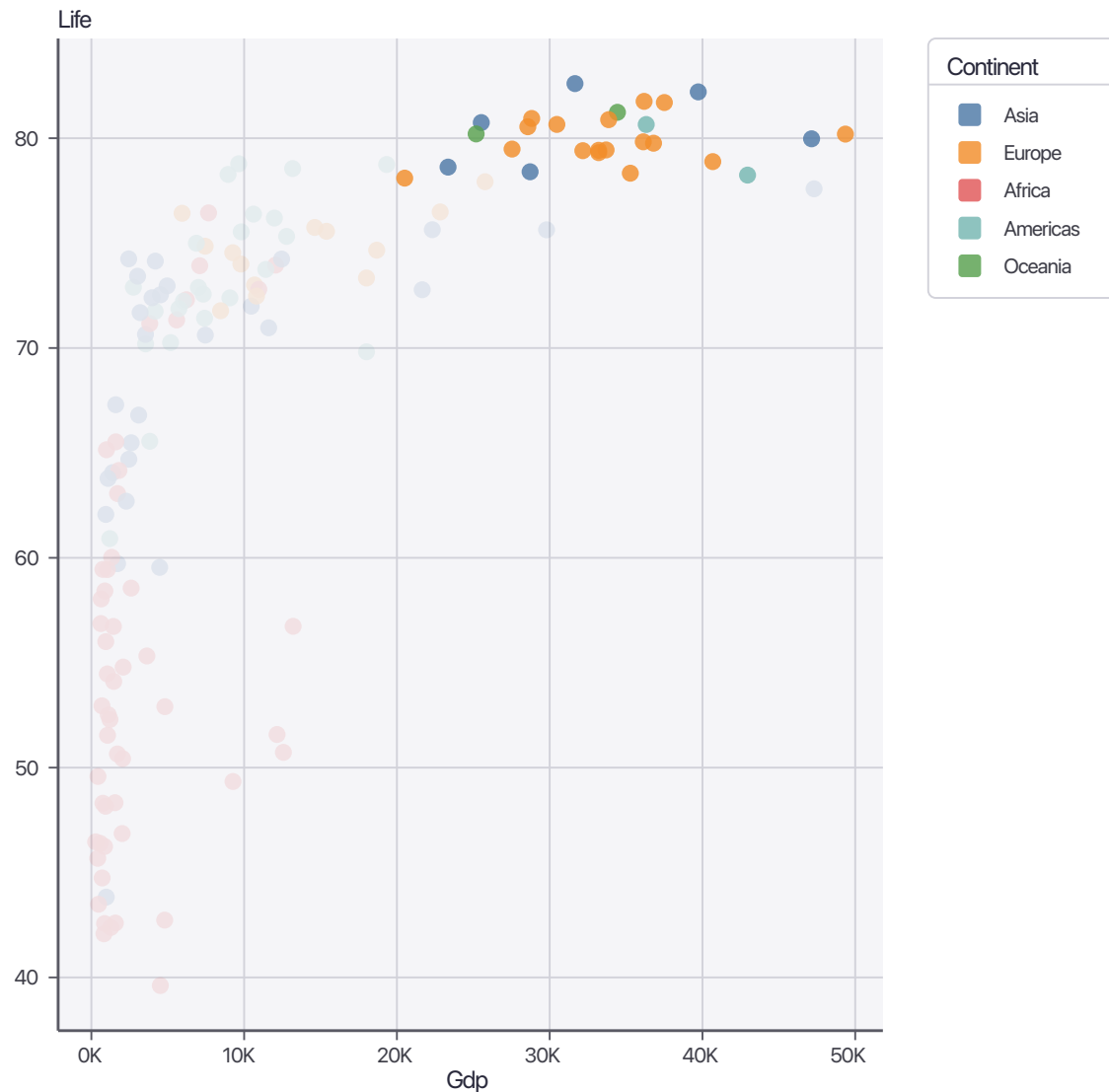
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
↪ color(col.continent) +
  theme(width = 620) + brush(col.gdp, at = [20000, 60000]) + brush(col.life,
↪ at = [78, 85]))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent) +
  theme(width = 620) + brush(:gdp, at = [20000, 60000]) +
  brush(:life, at = [78, 85])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  color(col.continent), theme({ width: 620 }),
  brush(col.gdp, { at: [20000, 60000] }),
  brush(col.life, { at: [78, 85] }))
```



“Given gapminder 2007: points, x is gdp, y is life, color by continent, and let the reader select gdp, starting from 20000 to 60000, and life, starting from 78 to 85.”

The rich countries that also live longest. Six of the countries that survived the first bound do not survive this one, and you can see exactly which six, because they are still on the page.

Writing it as two atoms rather than one with two arguments follows the rest of the grammar. A position is one column at a time, and so is a bound.

39.4 A column of categories selects slots

`at` is where the selection starts. The column decides which reading you get, in the same way it decides whether `color` uses a ramp or a palette. Numbers are a range. Names are a set of slots:

R

```
data(gapminder_2007) + point + x(continent) + y(life) + color(continent) +  
  theme(width = 620) + brush(continent, at = c("Europe", "Oceania"))
```

Python

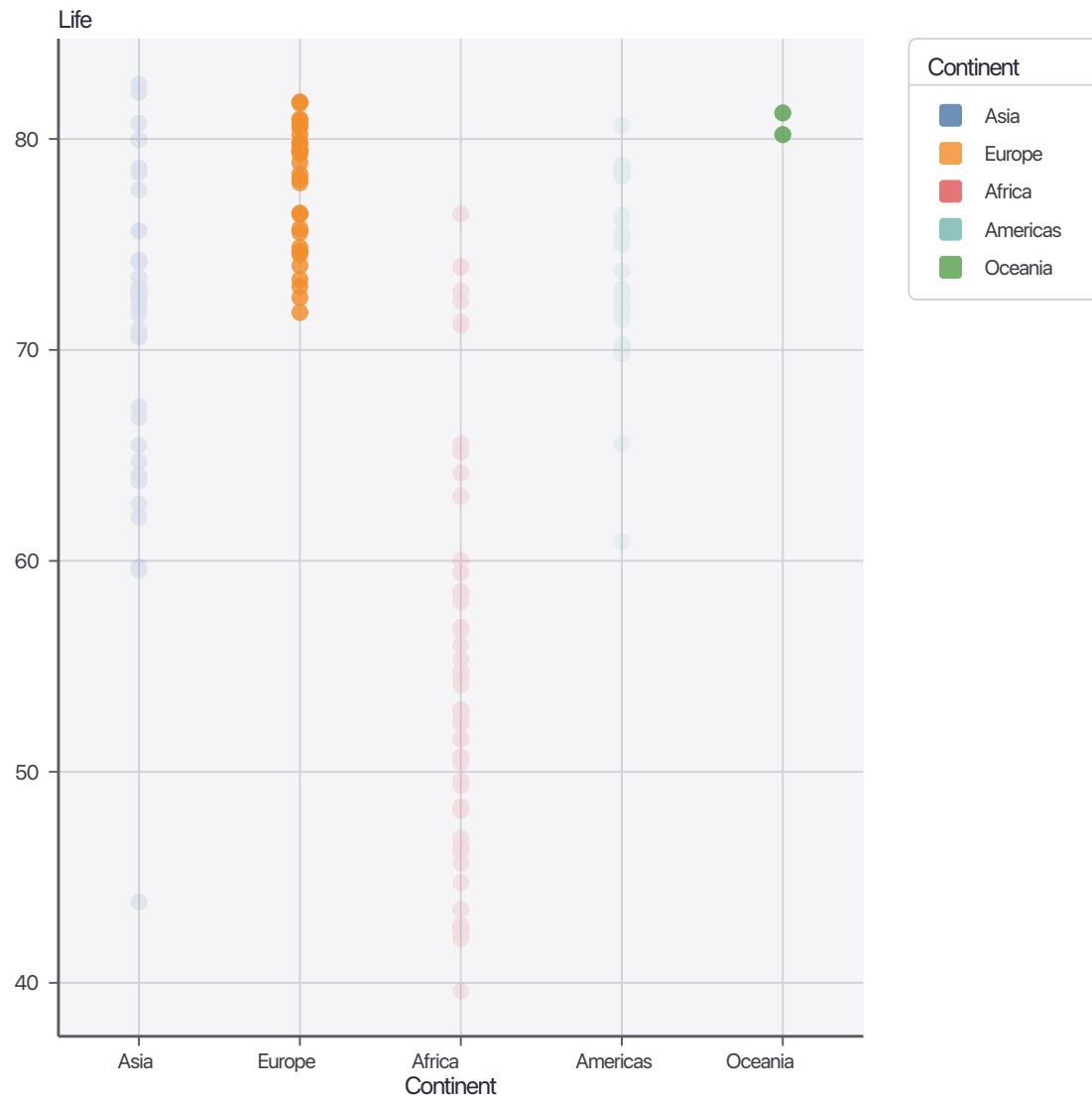
```
(data(gapminder_2007) + point + x(col.continent) + y(col.life) +  
  ↪ color(col.continent) +  
  theme(width = 620) + brush(col.continent, at = ["Europe", "Oceania"]))
```

Julia

```
data(gapminder_2007) + point + x(:continent) + y(:life) +  
  color(:continent) + theme(width = 620) +  
  brush(:continent, at = ["Europe", "Oceania"])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.continent), y(col.life),  
  color(col.continent), theme({ width: 620 }),  
  brush(col.continent, { at: ["Europe", "Oceania"] })))
```



“Given gapminder 2007: points, x is continent, y is life, color by continent, and let the reader select continent, starting with Europe and Oceania.”

One word, two readings, decided by the column rather than by a second atom. Drag across the slots and the selection follows them, whole slots at a time, because a category has no half.

The column is on the x axis here and that is not decoration. A drag can only reach a column the panel places. Write it over the scatter this chapter opened with, where `continent` is only a color. The sentence is still legal and still dims the right countries. But there is nowhere to drag, because no axis is measuring continents. The sentence has chosen the selection, and dragging cannot change it.

Which axis places it does not matter. Here is the same sentence with `x` and `y` exchanged, in a shorter panel:

R

```
data(gapminder_2007) + point + x(life) + y(continent) + color(continent) +  
  theme(width = 620, height = 320) + brush(continent, at = c("Europe",  
  ↪ "Oceania"))
```

Python

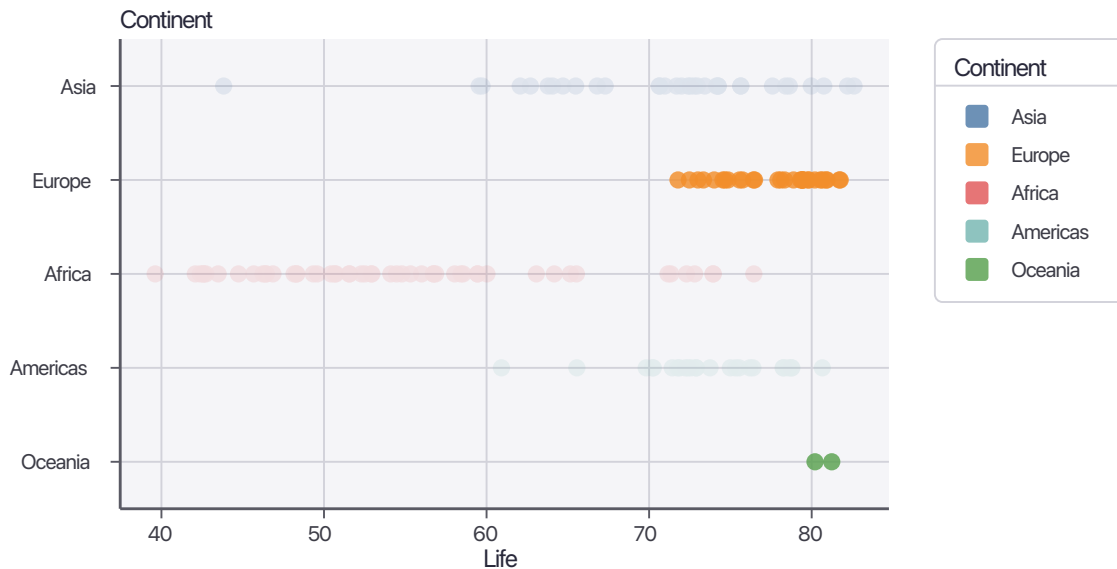
```
(data(gapminder_2007) + point + x(col.life) + y(col.continent) +  
  ↪ color(col.continent) +  
  theme(width = 620, height = 320) + brush(col.continent, at = ["Europe",  
  ↪ "Oceania"]))
```

Julia

```
data(gapminder_2007) + point + x(:life) + y(:continent) +  
  color(:continent) + theme(width = 620, height = 320) +  
  brush(:continent, at = ["Europe", "Oceania"])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.life), y(col.continent),  
  color(col.continent), theme({ width: 620, height: 320 }),  
  brush(col.continent, { at: ["Europe", "Oceania"] })))
```



The slots stack up the side now, and the same two continents keep their color. Drag on it and the band lies across the panel instead of standing in it, whole slots at a time as before. The bound never mentions an axis. It names a column, so turning the plot moves the drag and leaves the sentence alone.

39.5 Two views, one selection

This is what a brush is most useful for, and the sections above prepared for it. Selecting inside one plot shows you what the plot already showed. The question worth asking is *where does this group sit in a different picture*, and that needs a second picture:

R

```
(data(gapminder_2007) + point + x(gdp) + y(life) +
  brush(gdp, at = c(20000, 60000)) + theme(height = 230)) /
(data(gapminder_2007) + point + x(gdp) + y(population, scale = "log") +
  brush(gdp, at = c(20000, 60000)) + theme(height = 200))
```

Python

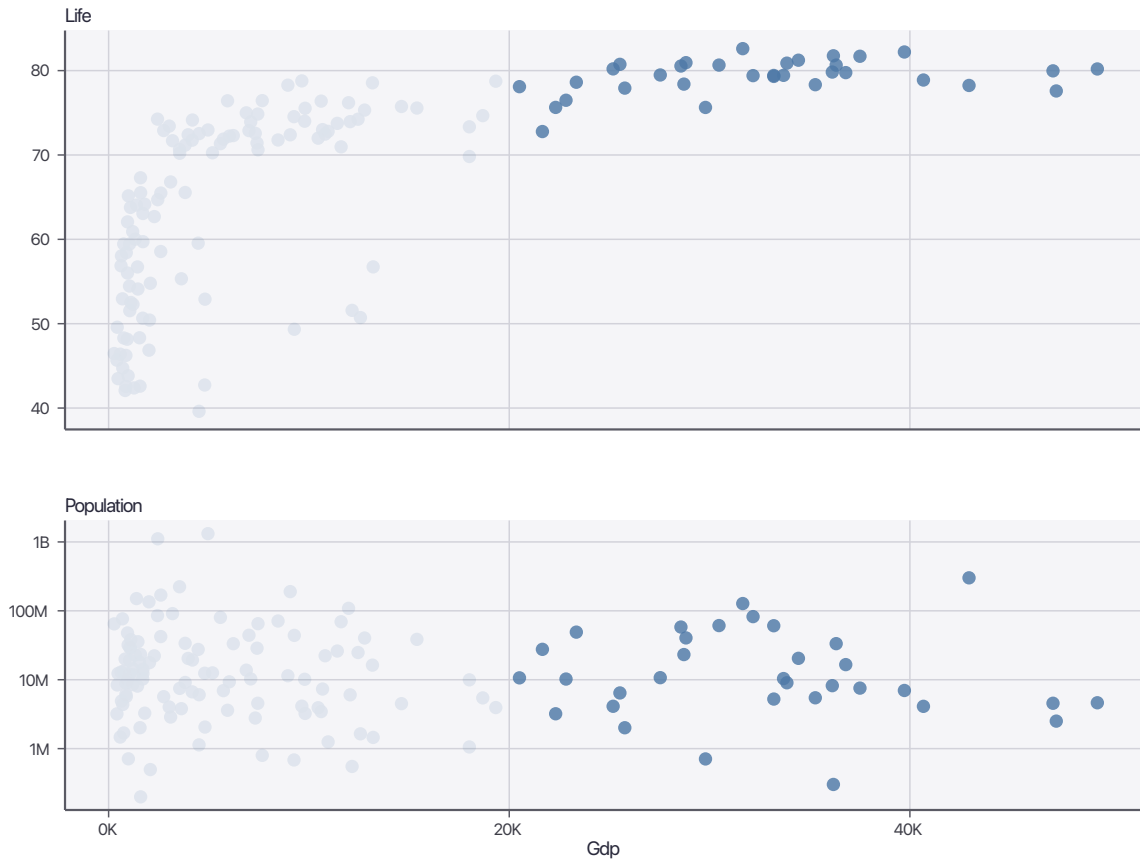
```
((data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  brush(col.gdp, at = [20000, 60000]) + theme(height = 230)) /
  (data(gapminder_2007) + point + x(col.gdp) + y(col.population, scale =
↪ "log") +
    brush(col.gdp, at = [20000, 60000]) + theme(height = 200)))
```

Julia

```
(data(gapminder_2007) + point + x(:gdp) + y(:life) +
  brush(:gdp, at = [20000, 60000]) + theme(height = 230)) /
  (data(gapminder_2007) + point + x(:gdp) +
  y(:population, scale = "log") + brush(:gdp, at = [20000, 60000]) +
  theme(height = 200))
```

JavaScript

```
below(plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
  brush(col.gdp, { at: [20000, 60000] })), theme({ height: 230 })),
  plot(data(gapminder_2007), point, x(col.gdp),
  y(col.population, { scale: "log" })),
  brush(col.gdp, { at: [20000, 60000] })), theme({ height: 200 })))
```



“Given gapminder 2007: points, x is gdp, y is life, and let the reader select gdp, starting from 20000 to 60000, above points, x is gdp, y is population on a log scale, and let the reader select gdp, starting from 20000 to 60000.”

Two plots, one page, and one bound keeping the same countries in color in both. Drag in either and the other follows. The rich countries live long, which the top plot says; their populations run from under a million to over three hundred million, which only the bottom one says, and neither plot could have told you both.

Nothing was added to the grammar to link them. Look at what the two sentences share: the column **gdp**, named on **x** in each. A bound is a fact about a column’s values rather than about a panel, so two plots that read that column are already answering the same question. There is no name to declare, no identifier to match up, and no third atom joining them.

That is **composition**’s one rule again, in a new place. Two composed plots naming the same column on the same axis share that axis, which is why these two are the same width and the **gdp** numbers are written once. The selection is shared for the same reason.

39.6 One bound reaches every panel

Faceting is the other way to get more than one panel, and a brush treats it the same way. The bound is a fact about a column, and every panel reads that column, so every panel answers:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  brush(gdp, at = c(20000, 60000)) | facet(continent)
```

Python

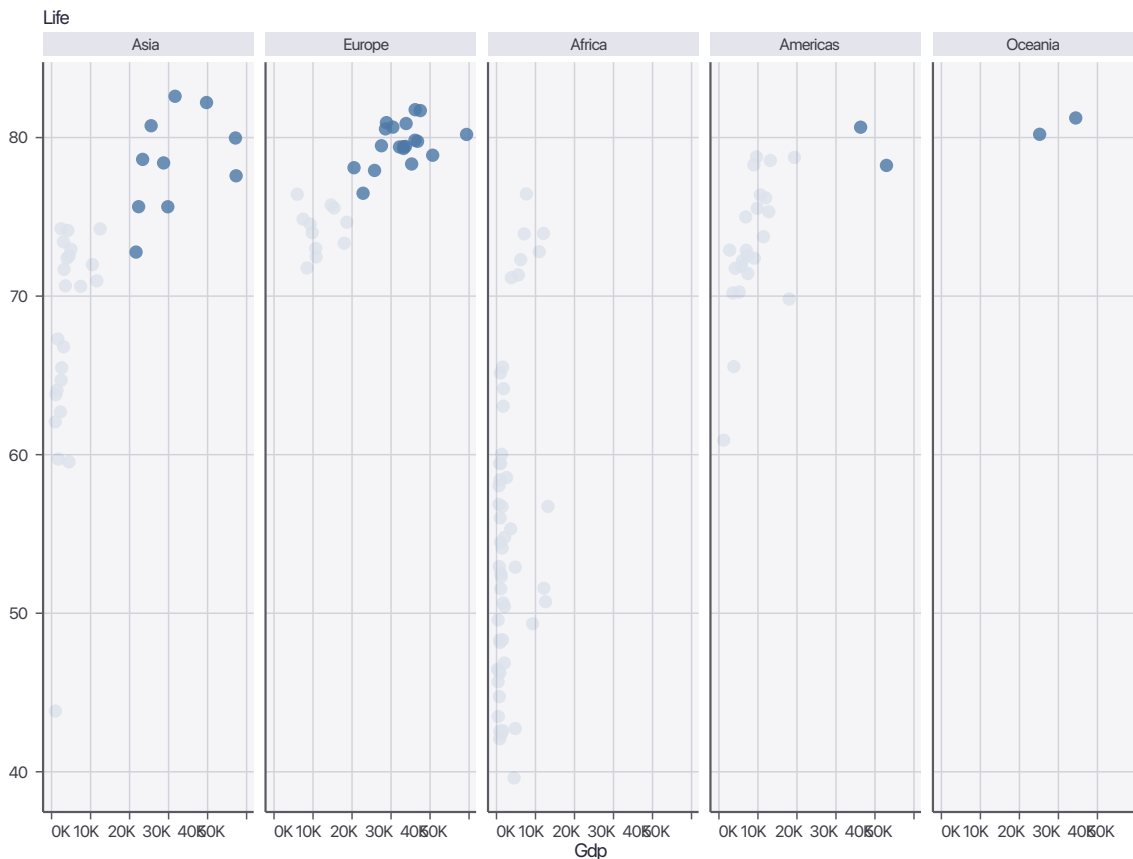
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  brush(col.gdp, at = [20000, 60000])) | facet(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  brush(:gdp, at = [20000, 60000]) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  brush(col.gdp, { at: [20000, 60000] }), across(col.continent))
```



“Given gapminder 2007: points, x is gdp, y is life, and let the reader select gdp, starting from 20000 to 60000, split into panel columns by continent.”

Five panels, one bound, and the rich countries keeping their color. Europe has many and Africa has none, which is the comparison the split was made for. Again nothing was added: **facet** divides the rows and **brush** dims the rest, and neither atom knows about the other.

The panels differ in one way worth knowing. A bound is shared, because it names a column. Pointing is not. In the web edition, resting the pointer on a mark shows that row’s values. They come from the panel you are pointing at, never from a row another panel drew.

39.7 Which one is that?

The other thing people want from a selection is a name. Layer **text** over the points and the brush decides which labels are legible:

R


```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  text + label(country) + style(size = 7) +  
  brush(gdp, at = c(30000, 60000)) + theme(width = 620)
```

Python

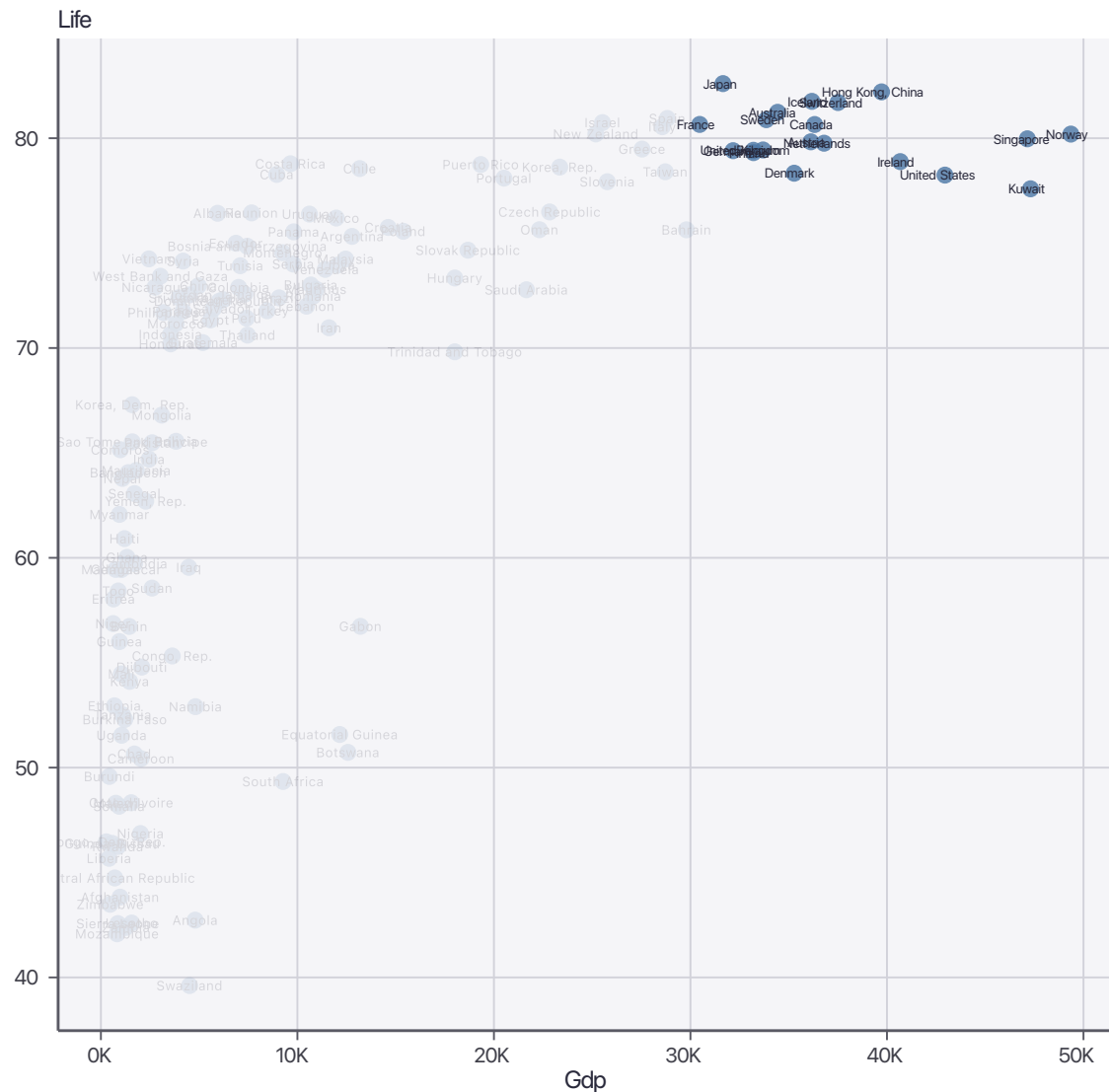
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  text + label(col.country) + style(size = 7) +  
  brush(col.gdp, at = [30000, 60000])) + theme(width = 620))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + text +  
  label(:country) + style(size = 7) + brush(:gdp, at = [30000, 60000]) +  
  theme(width = 620)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life), text,  
  label(col.country), style({ size: 7 } ),  
  brush(col.gdp, { at: [30000, 60000] } ), theme({ width: 620 } ))
```



“Given gapminder 2007: points, x is gdp, y is life, and also text, label by country, and let the reader select gdp, starting from 30000 to 60000.”

A hundred and forty-two labels is unreadable, and that is what selection answers: the ones you did not select are dimmed with their points, and about twenty names keep their color. Drag the bound and a different twenty are readable.

This works because `text` is one of the marks a brush reaches, on the same test as `point`: one row is one label. No rule joins selection to labels. Each follows its own rule, and this plot is where the two meet.

39.8 What can be brushed

A selection picks out rows, so a mark can be brushed when *one row is one shape*: `point`, `text`, `rule` and `zone`.

A plain `bar` passes that test and still refuses: one row is one bar, but a bar's thickness is measured from the gap between neighbors, so selected and unselected bars would draw at different widths. The engine refuses it, and the message says so. A jittered `point` refuses for the same kind of reason, because its offset is seeded from the row's place in the table, and a repelled `text` layer is drawn whole instead, with a note that says why.

A `line` is the other case. It draws one shape *through* many rows, so a row is a corner of the line rather than a thing of its own. Selecting half of a line would mean cutting it in two, and cutting a line in two already has a word:

```
data(gapminder_asia) + line + x(year) + y(life) + group(country) +  
  brush(life, at = c(60, 80))
```

Error:

```
! gog: a `line` draws one shape through many rows, so there is no single row  
  ↳ to select. Use `group()` to split it, or brush a mark that draws one  
  ↳ shape per row: `point`, `text`, `rule` or `zone`.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

The refusal names two ways out. This sentence has already taken the first: `group()` splits the line, and each piece still runs through twelve years. So take the second, and brush a mark that draws one shape per row.

A summarized layer refuses for a different reason. A bar of `bar * count` stands for as many as fifty-two countries at once, so “twelve of these are selected” has no honest picture: dimming the whole bar would be wrong, and dimming part of it would invent a mark nobody asked for. `gog` says so instead of guessing:

```
data(gapminder_2007) + bar * count + x(continent) +  
  brush(gdp, at = c(20000, 60000))
```

Error:

```
! gog: `bar * count` cannot be brushed, because `count` summarizes many rows  
  ↳ into one and the engine cannot say which of them you selected. Brush the  
  ↳ layer that draws the rows themselves, or drop the brush.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

When a layer that *can* answer is present, neither of these is a refusal. The plot draws, and gog reports which layer it left whole:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  line * smooth +  
  brush(gdp, at = c(20000, 60000))
```

Python

```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  line * smooth +  
  brush(col.gdp, at = [20000, 60000]))
```

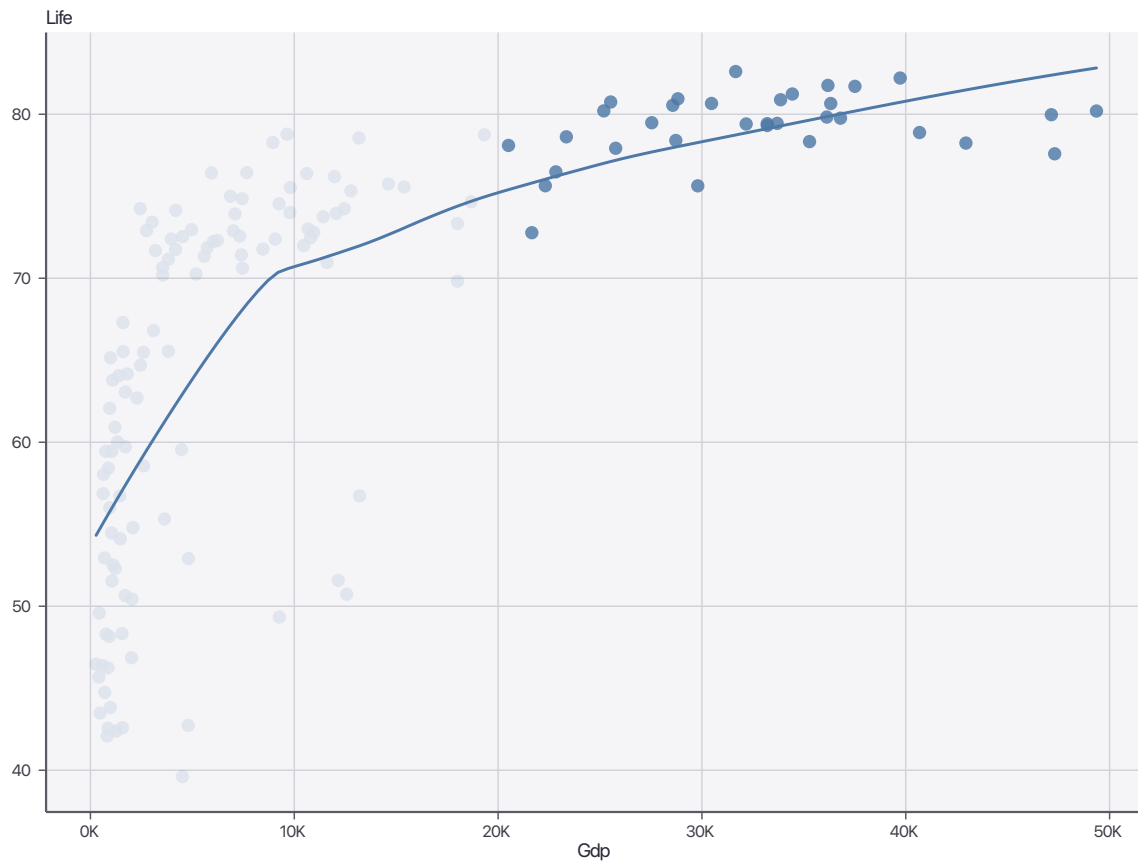
Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + line * smooth +  
  brush(:gdp, at = [20000, 60000])
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  layer(line, smooth), brush(col.gdp, { at: [20000, 60000] })))
```

gog: the `line` layer is drawn whole, because it draws one shape through many
↪ rows and there is no single row to select. The selection still reads on
↪ the rest of the plot.



“Given gapminder 2007: points, x is gdp, y is life, and also a line derived by smooth, and let the reader select gdp, starting from 20000 to 60000.”

The points answer the selection and the trend line does not, so the line is drawn once, undimmed, through all 142 countries. That is the right answer: a trend fitted to the selection alone would be a different statistic, and the sentence did not ask for one.

A range that does not run upward selects nothing, so it is a mistake rather than an empty selection:

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  brush(gdp, at = c(60000, 20000))
```

Error:

```
! gog: `brush(gdp, at = c(60000, 20000))` selects nothing, because the range  
  ↳ does not run upward. Write the smaller number first.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

39.9 Selecting while the plot plays

A frame is a subset of the rows, and a bound is a test on a column's values, so the two compose and neither needed anything added for the other. The bound is tested again inside every frame:

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + color(continent) +  
  theme(width = 620) + play(year) + brush(gdp, at = c(3000, 20000))
```

Python

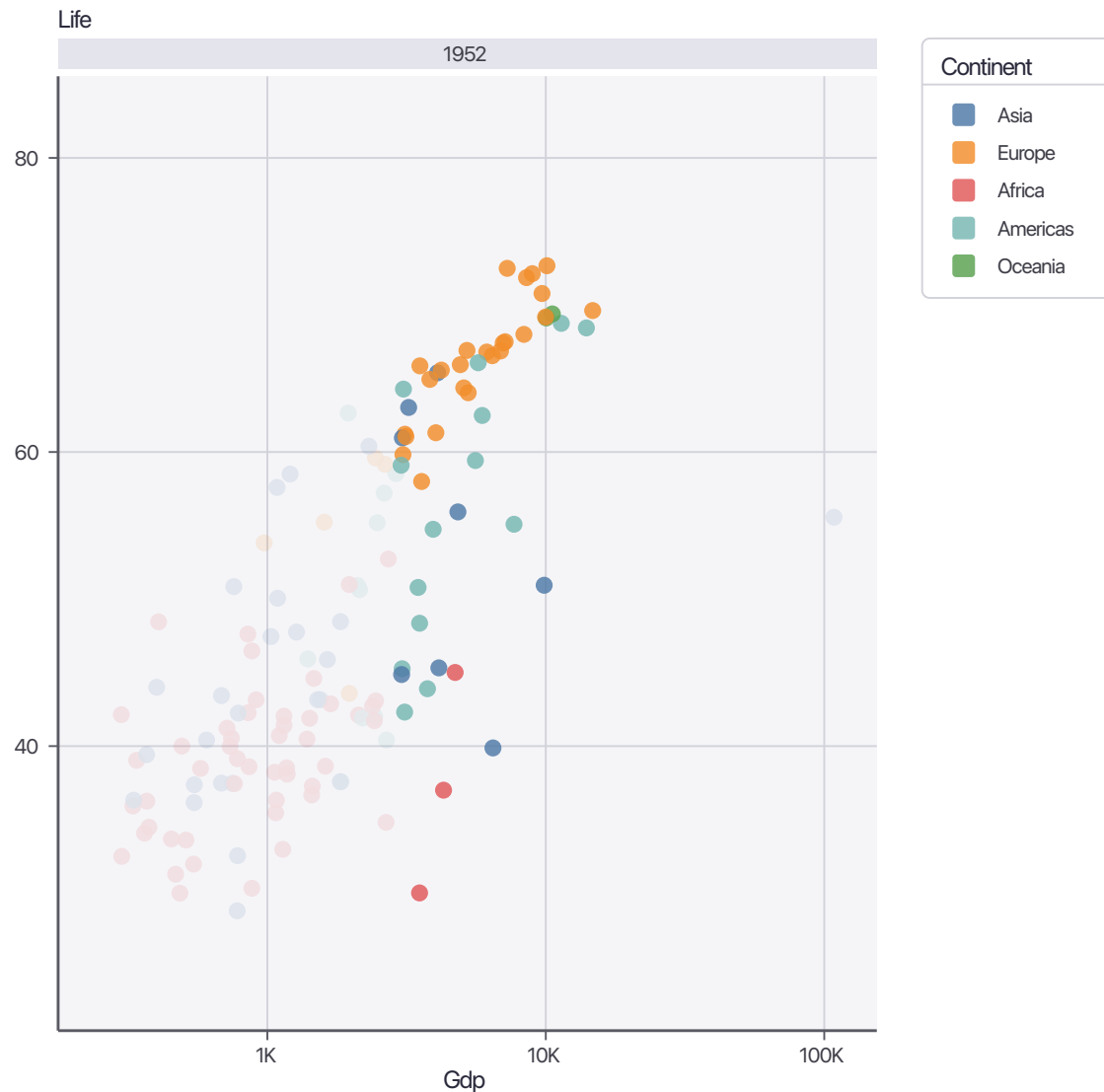
```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +  
  ↪ color(col.continent) +  
  theme(width = 620) + play(col.year) + brush(col.gdp, at = [3000, 20000]))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + theme(width = 620) + play(:year) +  
  brush(:gdp, at = [3000, 20000])
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  color(col.continent), theme({ width: 620 }), play(col.year),  
  brush(col.gdp, { at: [3000, 20000] }))
```



“Given all the gapminder years: points, x is gdp on a log scale, y is life, color by continent, played by year, and let the reader select gdp, starting from 3000 to 20000.”

Watch countries cross into the selection, frame by frame. The bound stays at 3,000 to 20,000 dollars while the countries move. In 1952 the selected group is that year’s middle-income countries. By 2007 many have grown past 20,000 and poorer ones have grown into the range. Nothing pauses, and no frame is skipped. The same test simply gets a different answer in each one. The bound stays still, and the countries move across it.

39.9.1 What the count includes

Stop the clock, draw a small box, and the count under the plot (the line reading 17 of 1704 selected) looks far too large. The number is right, and the reason is one sentence: a bound is a test on a column's values, so it holds or fails for every row in the table. It does not stop at the frame you can see. The count and **show rows** (a button under the plot, listed below) report the whole selection, across every frame. The picture shows you one frame of it.

The plot above makes that concrete. `gm_all` is 1,704 rows, which is 142 countries in each of twelve years, and one frame draws 142 of them. Stop on 1982 and draw a box around four countries in the low corner, and the count reads far more than four. The same box tests every year's rows, not only 1982's, and in earlier years other countries sat in that corner too. The four you can see are 1982's members of a set that runs through the whole sequence.

show rows is where you check it. The table lists the **year** of every row it found, beside the columns the sentence maps, so you can see which frame each row belongs to and pick out the four from the one you are looking at.

A faceted plot never raises this question. There every selected row is on the page at once, so a count of the whole selection is a count of what you see. A sequence puts eleven twelfths of it out of sight, in time rather than off the edge of the page. The rule is the same one; only the picture differs.

39.9.2 Choosing a bound the data crosses

The start of a sequence is where an animation misleads. Count what a bound selects in each frame before you write it down. From 3,000 to 20,000 dollars selects between 52 and 74 countries in every frame here. From 20,000 to 60,000 dollars selects none at all in 1952 and 1957, one in 1962, and four by 1972. The second is a true picture and it reads as a broken plot, because a reader cannot tell *nothing is selected here* from *this is not working*. Drag the bound upward and you will watch it happen.

One column cannot do both jobs. Brushing the column the frames are cut on would select frames rather than rows inside one, and that is a control for moving through time rather than a selection:

```
data(gm_all) + point + x(gdp) + y(life) + play(year) + brush(year, at =  
  ↪ c(1980, 2007))
```

Error:

```
! gog: `year` is already the column `play()` cuts the frames on, so  
  ↪ `brush(year)` would select frames rather than rows inside one. That is a  
  ↪ scrub bar, which belongs to the viewer and not to the sentence. Brush a  
  ↪ column the frames are not cut on.
```


gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

Moving the clock belongs to the page a plot is shown on, not to the sentence that describes the plot. That is what the buttons under a played plot do, and [Play](#) says why they have no word here.

39.10 All three at once

The [third position](#), the frames and the selection are three separate ideas, and none of them was taught about the others. So they compose:

R

```
data(gm_all) + point + x(gdp, scale = "log") + y(life) + z(population) +  
  color(continent) + play(year) + brush(gdp, at = c(3000, 20000))
```

Python

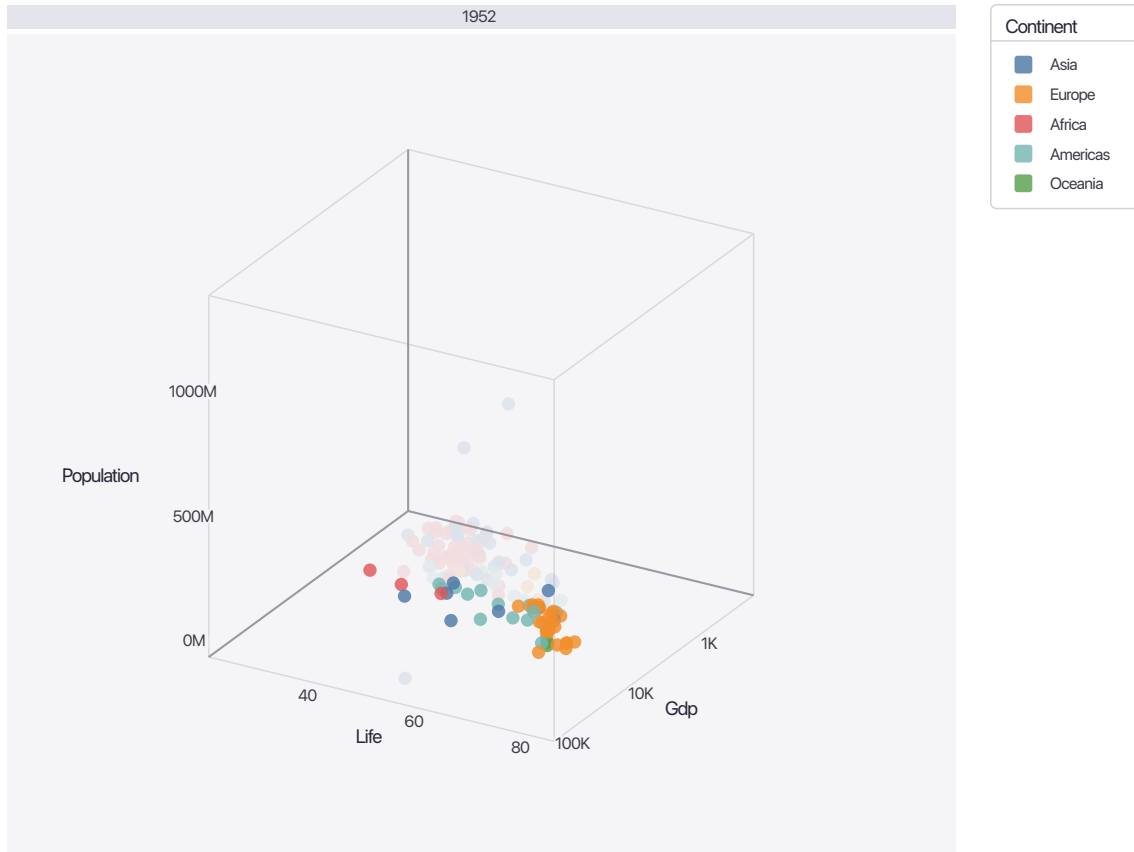
```
(data(gm_all) + point + x(col.gdp, scale = "log") + y(col.life) +  
  ↪ z(col.population) +  
  color(col.continent) + play(col.year) + brush(col.gdp, at = [3000, 20000]))
```

Julia

```
data(gm_all) + point + x(:gdp, scale = "log") + y(:life) +  
  z(:population) + color(:continent) + play(:year) +  
  brush(:gdp, at = [3000, 20000])
```

JavaScript

```
plot(data(gm_all), point, x(col.gdp, { scale: "log" }), y(col.life),  
  z(col.population), color(col.continent), play(col.year),  
  brush(col.gdp, { at: [3000, 20000] }))
```



“Given all the gapminder years: points, x is gdp on a log scale, y is life, z is population, color by continent, played by year, and let the reader select gdp, starting from 3000 to 20000.”

A cube of 142 countries, twelve frames, and one bound tested inside every one of them. Drag the plot to turn it and the selection turns with the cloud, because the bound picks rows and the angle picks a viewpoint, and neither asks the other anything.

There is one difference here, and it is worth knowing rather than working out. In the cube the drag already turns the plot, so name the bound in the sentence: **brush** on its own would declare a region with no way to draw one. That is the same rule as everywhere else in this chapter, arriving from the other side. A drag means one thing at a time, and in the cube it means turn.

39.11 Moving the bound yourself

Every brushed plot in this chapter after the first states where its selection starts. On a page that can run the engine, you can drag the panel to move the bound, and the plot redraws under your hand.

This changes nothing about the sentence, and it is the same arrangement the cube uses. `space(turn =, tilt =)` chooses the angle a plot starts at and the pointer takes you anywhere else; `brush(gdp, at = )` chooses the bound a plot starts with and the pointer does the same. Drag to find a selection, then write it into the sentence.

On paper, and in the PDF edition, each plot shows the bound its sentence asked for. That is why naming it is worth doing even though a reader can move it. The printed figure shows the selection you chose, and a reader who can drag starts from it.

Written with no `at` at all, `brush` selects nothing. The plot then draws exactly as it would with no `brush` in the sentence, which is the resting state a page shows before the first drag.

39.12 What the page adds

A brushed plot on a web page carries a line of controls under it. None of them are in the sentence, and that is the rule rather than an omission. They report a selection, or they change how you look at one. Not one of them changes what the plot claims about the data, so none of them needs a word in the grammar. It is the same reason a plot in the cube has a line of controls and `space()` has no word for the pointer.

They come in two lines, and the first is the one every plot carries:

Control	What it does
- +	Makes the picture smaller or larger
the frame (the box that shows the whole plot)	Returns to the whole picture
the camera	Saves what you are looking at as a picture file

It sits first, directly under the picture, on a plain plot and a brushed one alike. This matters. A reader who has found the magnifiers once finds them in the same place under every plot in this book, rather than somewhere that shifts with whatever else the plot needed.

A plot that also plays carries three more buttons at the end of that same line, which stop the clock and step it. [Play](#) sets them out.

The second line is what a brush adds:

Control	What it does
drag: and its icons	Says what dragging the plot will do, and switches it: a rectangle, a free shape, or moving the picture
17 of 142 selected	How many rows the current bound selects. On a played plot this counts every frame, not the one on screen
show rows	Opens a table of those rows; while the table is open the button reads hide rows . A long selection arrives ten at a time, with < and > under the table
clear	Puts the selection back to what the sentence asked for
unstamp	Takes the stamps off, and counts them once there are several. Absent until you have made one

A control drawn as an icon names itself when the pointer rests on it. The ones with a word on them need no name.

39.12.1 Selecting

Drag across the panel. A dotted box follows the pointer, and it shows *exactly* what is bound. On a plot with one brush the box spans the whole panel in the other direction, because that direction was not selected. On a plot with two, it is a rectangle. A box that narrowed an axis nobody bound would be claiming a range you did not choose.

Dragging in either direction gives the same bound, so you can start from any corner. A click on empty space clears the selection. A click that lands on a mark does something else, and the section below is about that.

39.12.2 Drawing around a group

A rectangle is what a sentence can say. Some groups are not rectangles. A cloud of points often has a shape: a narrow tail to one side, a curve, a group sitting apart from the rest. A box drawn around such a group includes points you did not mean.

The second icon beside **drag:** is the **free shape**. Choose it, then draw around the group with the pointer held down. The outline closes itself when you let go, and everything inside it is selected. The count and the row table read exactly as they do for a box, because a shape and a bound ask the same question: which rows do you mean.

Nothing about the sentence changes. You do not type a shape, and the printed page still shows the bound the sentence named. Drawing one is a way of looking, like turning a cube, rather than a claim about the data.

That is also why the grammar has no word for it. Other tools call the gesture a *lasso*, and *gog* has no such word. A shape can only be written down as a long list of corners, and a specification is something you can write down and print. You can still draw the shape; you cannot write it.

Two restrictions are worth knowing. A free shape needs two axes that measure, so where one axis carries categories the drag stays a rectangle. A category has no half, and whole slots are what you wanted there anyway. And **clear** removes the shape instead of restoring it, because a shape has no resting form for a sentence to state.

39.12.3 Reading what you selected

The count is always there. **show rows** opens a table underneath, listing the columns the sentence maps rather than every column in the table, because the columns you did not plot are not what you are looking at.

A long selection arrives ten rows at a time. The line under the table says which rows you are reading, as 11–20 of 40, and the two arrows beside it move you through the rest. Every row you selected is reachable, and none of them pushes the plot off the screen. A selection short enough to fit shows no arrows at all.

The table names every row you selected, in one place, away from the picture. Draw a bound tight around one point and it names that point alone.

For a quick look without selecting anything, move the pointer over a mark and that row's values appear beside it. Nothing is selected and nothing changes.

39.12.4 Stamping a point

The pointer names one row at a time and forgets it as soon as you move. That is right for a quick look and wrong for a comparison. Three points stand out on a scatter, and reading them one after another means remembering the first two while you look at the third.

Click a mark and its values stay. That is a **stamp**: a dot on the mark, a short line, and a small card saying what the pointer said. Click as many as you want to compare. The card sits on the picture beside the mark it belongs to, which is the difference between a stamp and the row table. One names rows, the other labels the plot.

A card needs space on the picture. On a crowded scatter it covers the very points you stamped it to read. So drag a card and it goes where you put it. The line stretches to follow, and a

long line ends in an arrow head pointing at the row. Move four cards out to the edges and the picture underneath stays whole.

Where you put a card is where it stays. Zoom in, pan across, or drag a new bound: the card holds its place beside its point while the picture moves under it. Zoom far enough that the point leaves the panel and the whole stamp waits out of sight. It has nothing left to point at, and a card left over an unrelated corner would claim a point that is not there. Bring the point back and the card is where you left it.

A played plot works the same way, for the same reason. A row there is one country in one year, so a stamp made in 1972 names something the 1987 frame does not draw. It waits while the other frames run, and comes back on its point when the loop returns. The card names the year it holds. Stamp the same country in three frames and each card appears in its own.

There are three ways to take a stamp off. The `*` in the corner of a card removes that card. Clicking the card itself does the same, as long as you did not move it. If the pointer traveled, you moved the card. If it did not, the card comes off. `unstamp` removes every card at once.

`clear` leaves them alone, because a stamp is not a selection: clear a bound to read your stamped rows against the whole data and they are all still there. `clear`, `unstamp` and the frame each undo one thing, which the section below sets out.

The camera saves them. Arrange the cards, press the camera, and the picture file carries the plot with your annotations on it, where they sit and pointing where they point. That is the camera's rule rather than a rule about stamps: it saves what you are looking at, the same way it saves how far you have zoomed in and the angle a cube is turned to.

You do not type a stamp, and the printed page carries none.

A stamp is for deciding which points are worth naming. A plot with a name on every point is unreadable, and a plot with a name on the few that matter is the one you wanted. Stamping lets you try a few before you commit to them. To put those names in the picture itself, where the page and the PDF will both carry them, write them down: that is `text`, and it is a layer like any other.

Arranging the cards by hand has a written form too. `text * repel` does the same job inside the sentence. Every label ends up outside its own dot, and one that moved far keeps a thin line back to the point it names. That is the stamp's line again, written down rather than dragged. Stamp to decide what is worth naming, then write the labels down and let `repel` place them.

The stamp is for looking. `text` is for saying.

Both can go into a presentation, and they are not the same there. A saved picture is one session at one angle and cannot be redrawn. A sentence with `text` in it draws again for anyone who has the data, in the PDF and on the page and in all four languages. Stamp and save to keep this one picture. Write the labels down to keep the plot itself.

39.12.5 Where pointing cannot answer

The page computes where each row was drawn, from the row's own value and the two ends of the axis the panel records. That is exact wherever a mark stands at its value, and some plots put a mark somewhere else on purpose.

`jitter` sets each point beside its value so that a crowded group can be read. `dodge` moves marks aside to clear their neighbors, and `stack` sets each on top of the one below. A summary such as `bin` or `mean` draws one shape for many rows, so there is no single row under the pointer to name. A polar plot bends both axes around a circle, and a map turns two columns into places on the page before it draws anything.

On any plot like those the pointer stays quiet rather than naming a row that is not there, and a line under the plot says which reason applied. Nothing is stamped either, for the same reason. The page computes a position rather than reading one, so it stays quiet where it cannot compute. That is why some plots name no row.

39.12.6 Looking closer

`+` and `-` magnify the picture and the frame returns to the whole of it. Zooming in switches the drag to panning, because a reader who has just looked closer usually wants to move around inside it; the frame hands the drag back to selecting. You can switch by hand at any time with the button on the left, and the pointer tells you which mode you are in: a crosshair to draw a box, an open hand to move the picture.

This is a *view*, so it *recomputes nothing*. The bars are the same bars and the ticks are the same ticks, drawn larger. Zoom far enough into a histogram and you will see the same bins, not finer ones. If you want the statistics re-run over a narrower range, that is `limits`, the atom this chapter compared with `brush`.

39.12.7 Three words, three things

`clear` acts on the **selection**, `unstamp` acts on the **stamps**, and the frame acts on the **view**. Three controls that undo three different things, so each is named for what it acts on rather than all three being called reset. The frame sits a line above the other two, beside the zoom buttons. It puts back the view they changed.

The `×` on a card is a fourth undo, and it is not in this line. It belongs to the card rather than to the plot. A control under the picture acts on all of something. A control on a card acts on that card.

None of this exists on paper, and paper loses nothing by it. The PDF shows the bound the sentence named.

39.12.8 Your data on the page

The browser answers a brush, so the table has to be in the page. It is the whole table, including columns the sentence never maps. The table that **show rows** opens lists only the mapped columns, and the page carries every column.

This matters when a table has a column you would not publish. Bind the columns you plot and drop the rest, as you would check any file before sharing it. [What the file carries](#) states the rule for both kinds of plot, with the one line that applies it.

Part IV

The cookbook

The 1446 book ends its explanations with a list: ninety-four everyday words, written out in the new letters. Eagle, ice, brook, unripe rice. Not rules, but usage: proof that the system could write ordinary life.

This part is that list. It is organized the way questions actually arrive, by the shape of your data and the thing you want to know, never by chart name. You will not find a chapter called “histograms”; you will find *how is a measurement distributed?*, and the sentence that answers it. (If you already know the chart’s name, the [chart names appendix](#) translates it into a sentence.)

Every recipe has the same shape: the question, the sentence that answers it, the plot, and a line or two on why it works. Most then show what is one atom away, and that is the real lesson. A cookbook of finished charts teaches you charts. This one teaches you that the recipes were never separate: swap one atom and you have the next one.

The recipes use only the cast of tables from the Preface, and only atoms taught in Parts I to III. Most link back to the chapter that owns their pieces.

40 One variable

You have one column. What can it tell you? The answer depends on the column's type: text asks how many of each, a measurement asks how the values spread.

40.1 How many of each?

A text column's first question. Counting rows is a derivation, not a different kind of chart, so the sentence is `bar` with `* count`:

R

```
data(gapminder_2007) + bar * count + x(continent)
```

Python

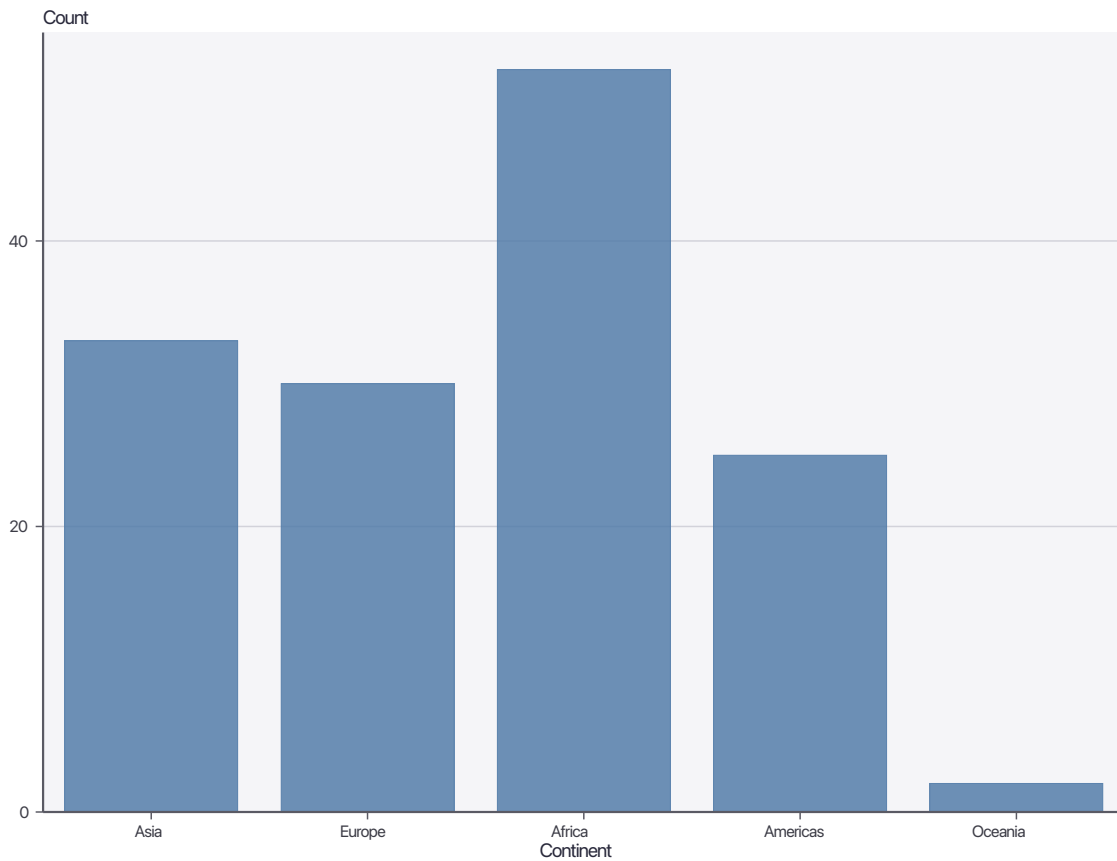
```
data(gapminder_2007) + bar * count + x(col.continent)
```

Julia

```
data(gapminder_2007) + bar * count + x(:continent)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, count), x(col.continent))
```



“Given gapminder 2007: bars derived by count, x is continent.” No `y()` is bound because the count is invented, and the engine labels the axis for you.

One atom away. Long category names? Move the categories to `y` and the bars lie down. The sentence is otherwise the same.

R

```
data(gapminder_2007) + bar * count + y(continent)
```

Python

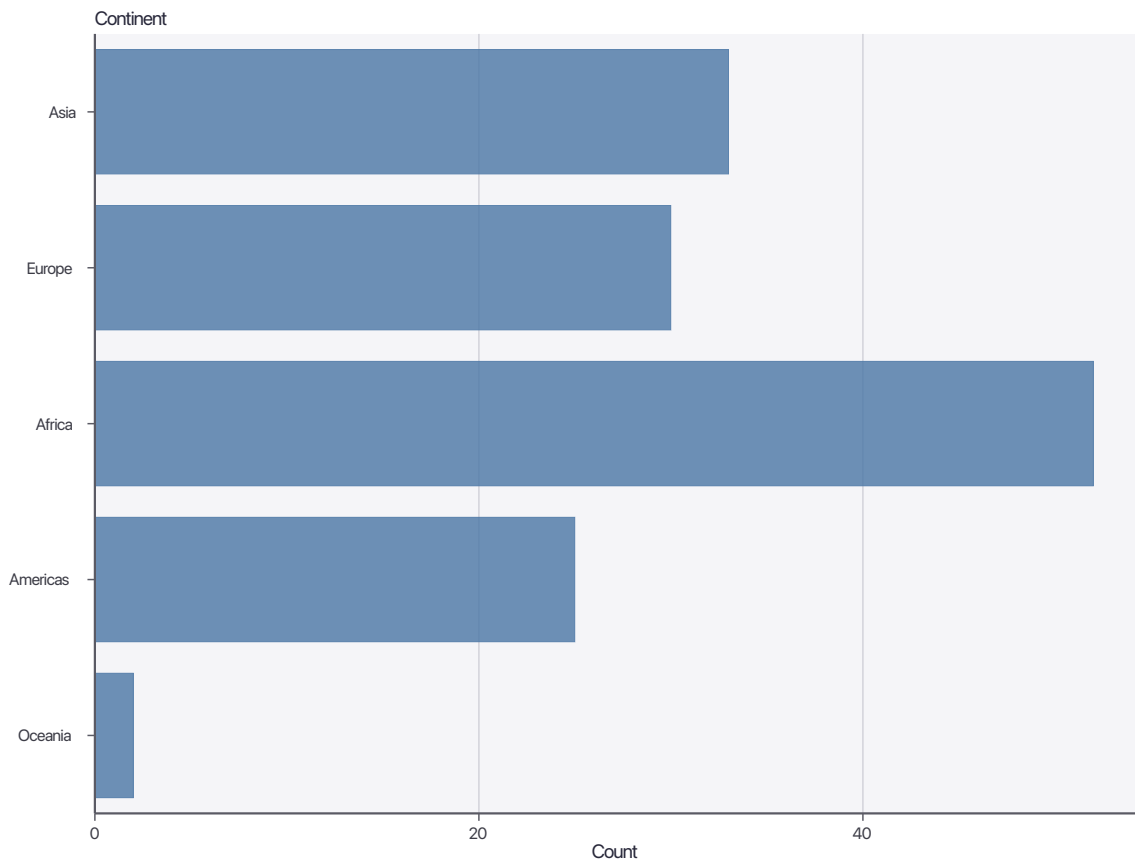
```
data(gapminder_2007) + bar * count + y(col.continent)
```

Julia

```
data(gapminder_2007) + bar * count + y(:continent)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, count), y(col.continent))
```



“Given gapminder 2007: bars derived by count, y is continent.”

The count still measures along the invented axis, which is now x. Owing chapters: [Bar](#), [Transforms](#).

40.2 What share of the whole is each?

Swap `count` for `proportion` and the counts become fractions that sum to one:

R

```
data(gapminder_2007) + bar * proportion + x(continent)
```

Python

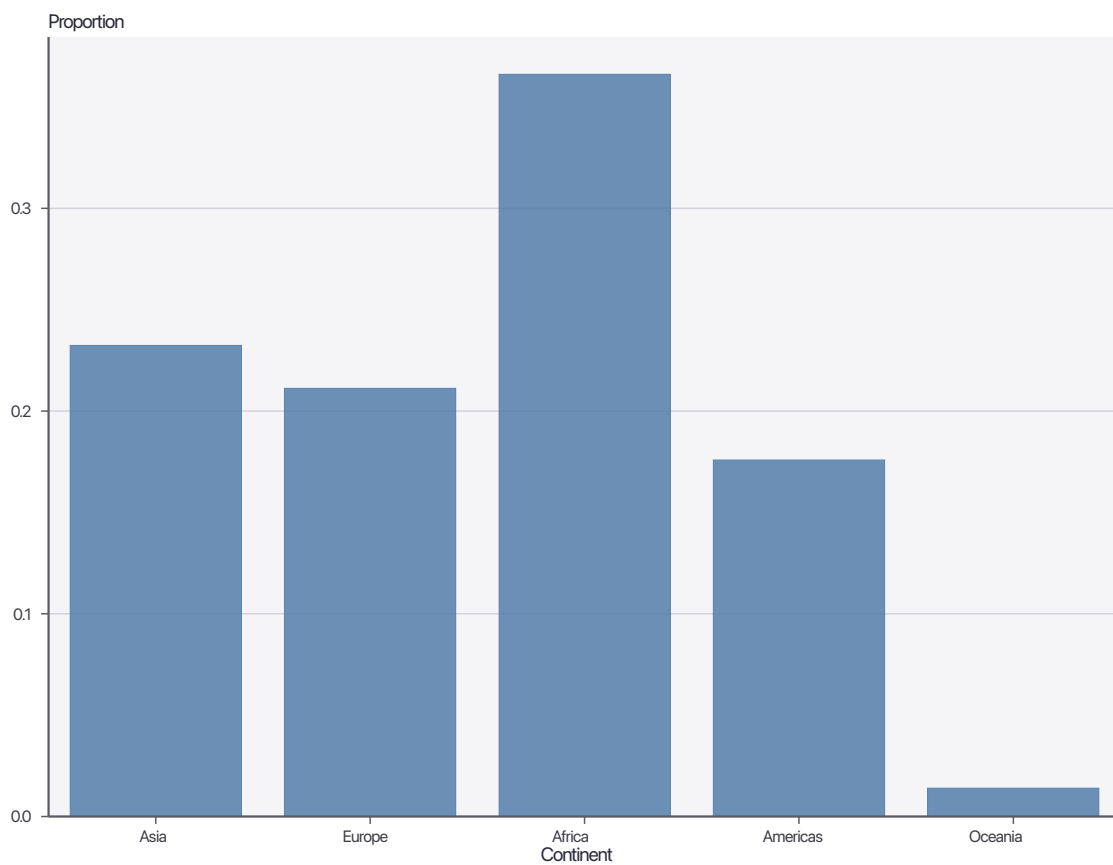
```
data(gapminder_2007) + bar * proportion + x(col.continent)
```

Julia

```
data(gapminder_2007) + bar * proportion + x(:continent)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, proportion), x(col.continent))
```



“Given gapminder 2007: bars derived by proportion, x is continent.”

One atom away. Back to raw counts: `* count`. `order()` reorders the categories by a column the table holds, never by the invented share.

40.3 How is a measurement distributed?

The continuous column’s first question. `bin` cuts the axis into equal-width bins and counts the rows in each; `bar` draws them touching, so a gap means a bin with no rows. The result is the **histogram**:

R

```
data(iris_flowers) + bar * bin + x(petal_length)
```

Python

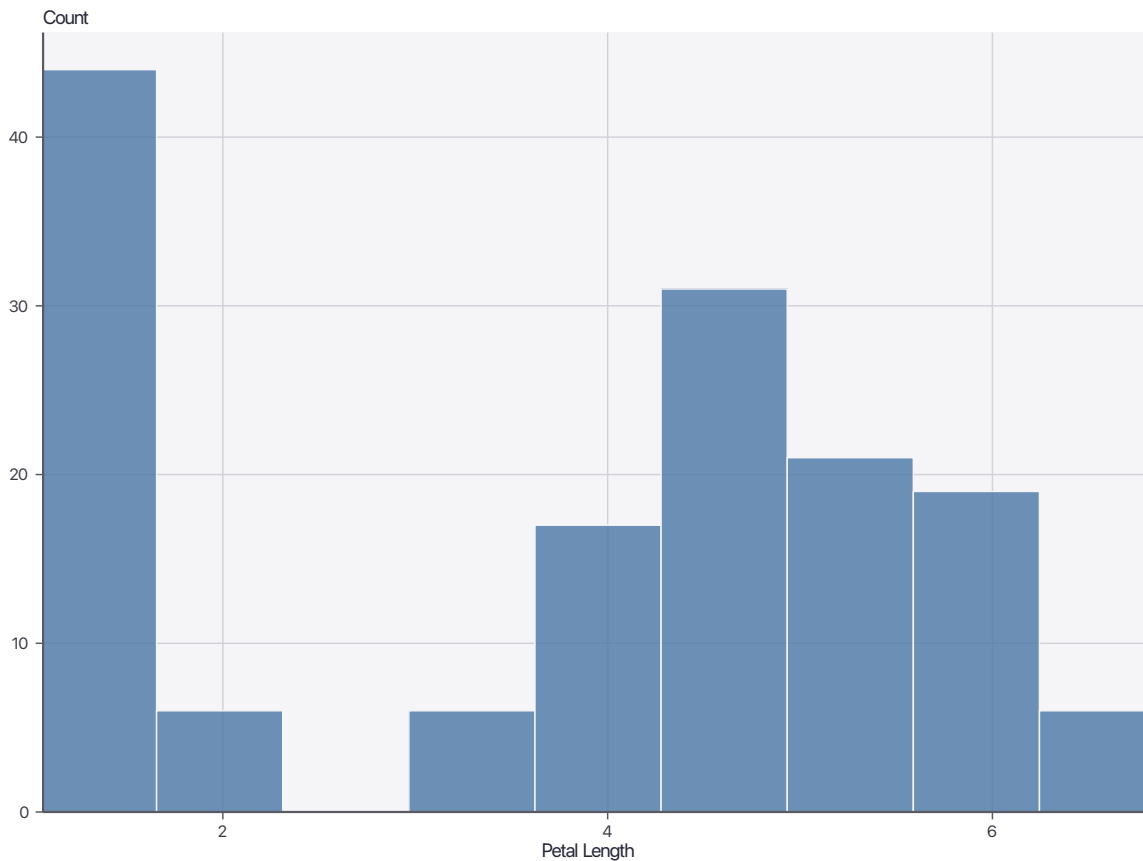
```
data(iris_flowers) + bar * bin + x(col.petal_length)
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length)
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length))
```



“Given the iris flowers: bars derived by bin, x is petal length.”

One atom away. `bin` takes a count or a width when the default does not suit the data. `bin(30)` asks for thirty bins; `bin(width = 0.5)` asks for bins half a unit wide:

R

```
data(iris_flowers) + bar * bin(width = 0.5) + x(petal_length)
```

Python

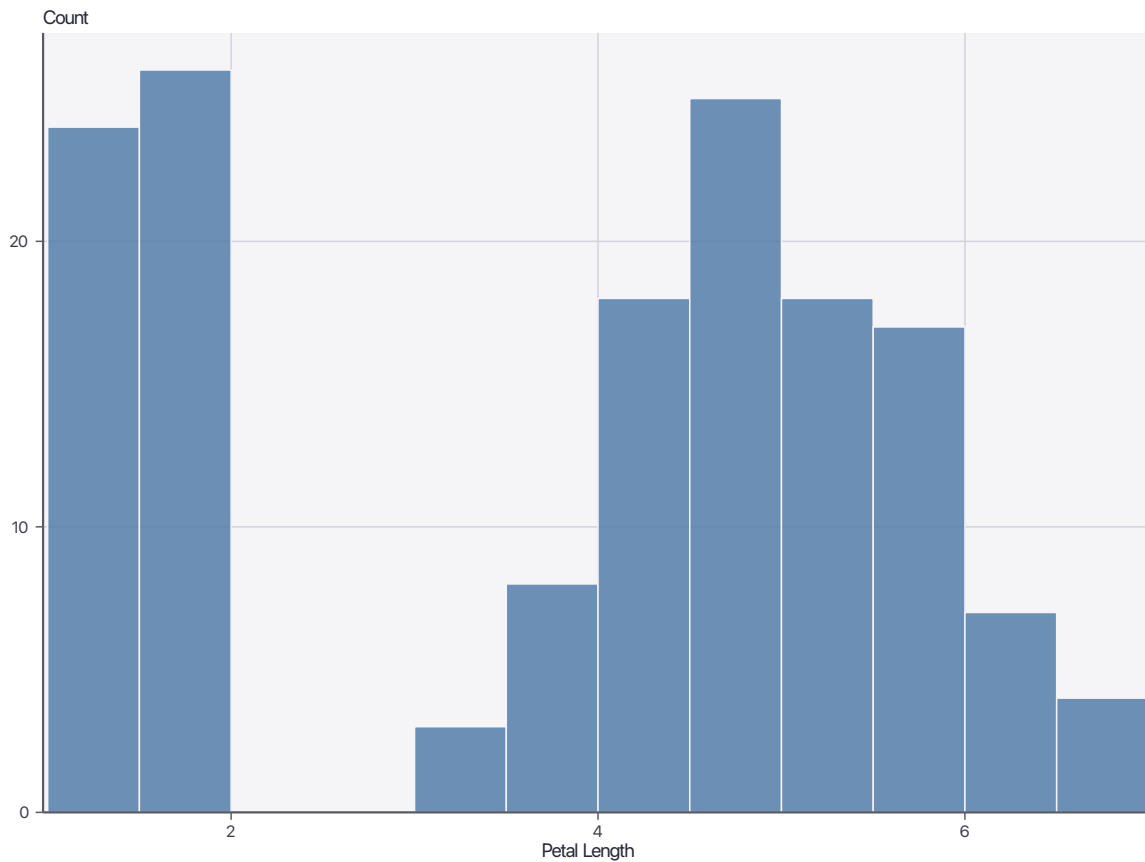
```
data(iris_flowers) + bar * bin(width = 0.5) + x(col.petal_length)
```

Julia


```
data(iris_flowers) + bar * bin(width = 0.5) + x(:petal_length)
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin({ width: 0.5 })),  
  x(col.petal_length))
```



“Given the iris flowers: bars derived by bin at width 0.5, x is petal length.”

A `line * bin` is the frequency polygon: the same counts, joined through the bin centers. Owing chapters: [Bar](#), [Transforms](#).

40.4 What is its smooth shape?

The binned view depends on where the bin edges fall; the density estimate removes that decision:

R

```
data(gapminder_2007) + line * density + x(life)
```

Python

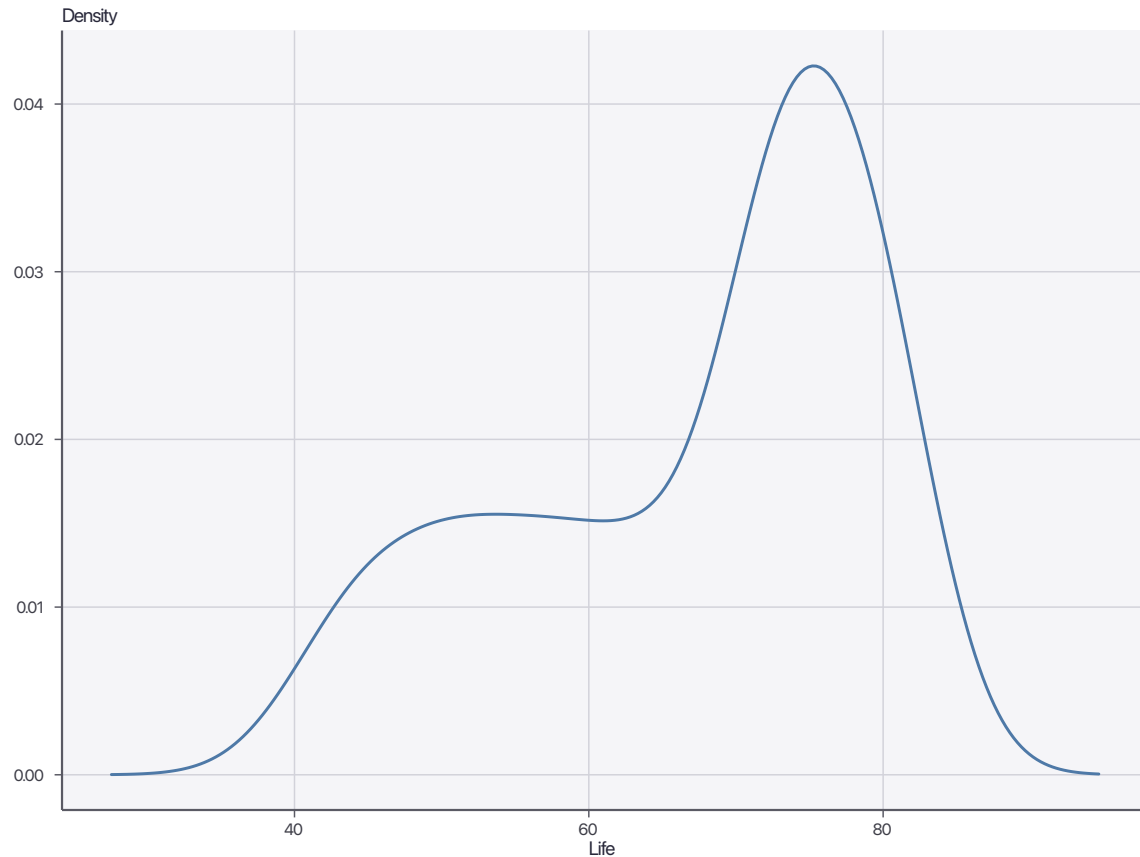
```
data(gapminder_2007) + line * density + x(col.life)
```

Julia

```
data(gapminder_2007) + line * density + x(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(line, density), x(col.life))
```



“Given gapminder 2007: a line derived by density, x is life.” The same question the histogram answered, with one derivation changed. Draw both when the number of peaks is the thing you want to show. One atom tunes the smoothing: `density(0.5)` sharpens the peaks, `density(2)` flattens them, and `density(bandwidth = 1)` fixes the kernel width in years. Owing chapters: [Line](#), [Transforms](#).

40.5 How does the distribution look in steps?

A third mark on the same `bin`: a staircase silhouette, tracing the bin tops with no fill. It is most useful when several distributions share an axis, as in [Many groups](#), and it reads well alone:

R

```
data(gapminder_2007) + step * bin + x(life)
```

Python

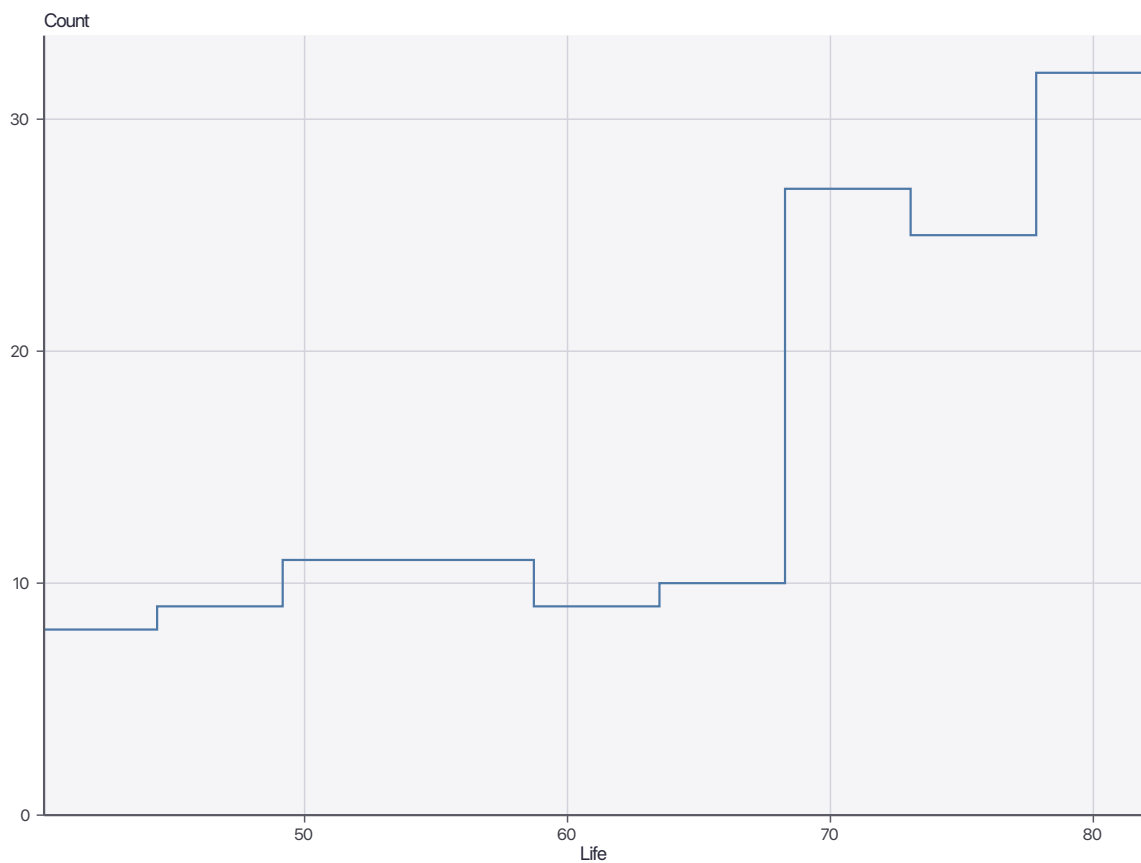
```
data(gapminder_2007) + step * bin + x(col.life)
```

Julia

```
data(gapminder_2007) + step * bin + x(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(step, bin), x(col.life))
```



“Given gapminder 2007: a step outline derived by bin, x is life.”

Two marks (`bar`, `step`) and one `bin`, with `line * bin` above making three. The transform never changed. Only the geometry that draws its output changed. That is the [No Exceptions law](#) doing its job. Owing chapters: [Step](#), [Transforms](#).

40.6 When there are only thirty rows

Every answer above summarizes: `bin` needs a bin width, `density` needs a bandwidth, and both choices matter more when there are fewer rows to average over. With thirty numbers the bins can stay and every row still shows. Add `stack` and a `point` piles one dot per observation:

R

```
data(gm_europe) + point * bin * stack + x(life) +  
  x_label("Life expectancy (years)") + title("Thirty countries, thirty dots")
```

Python

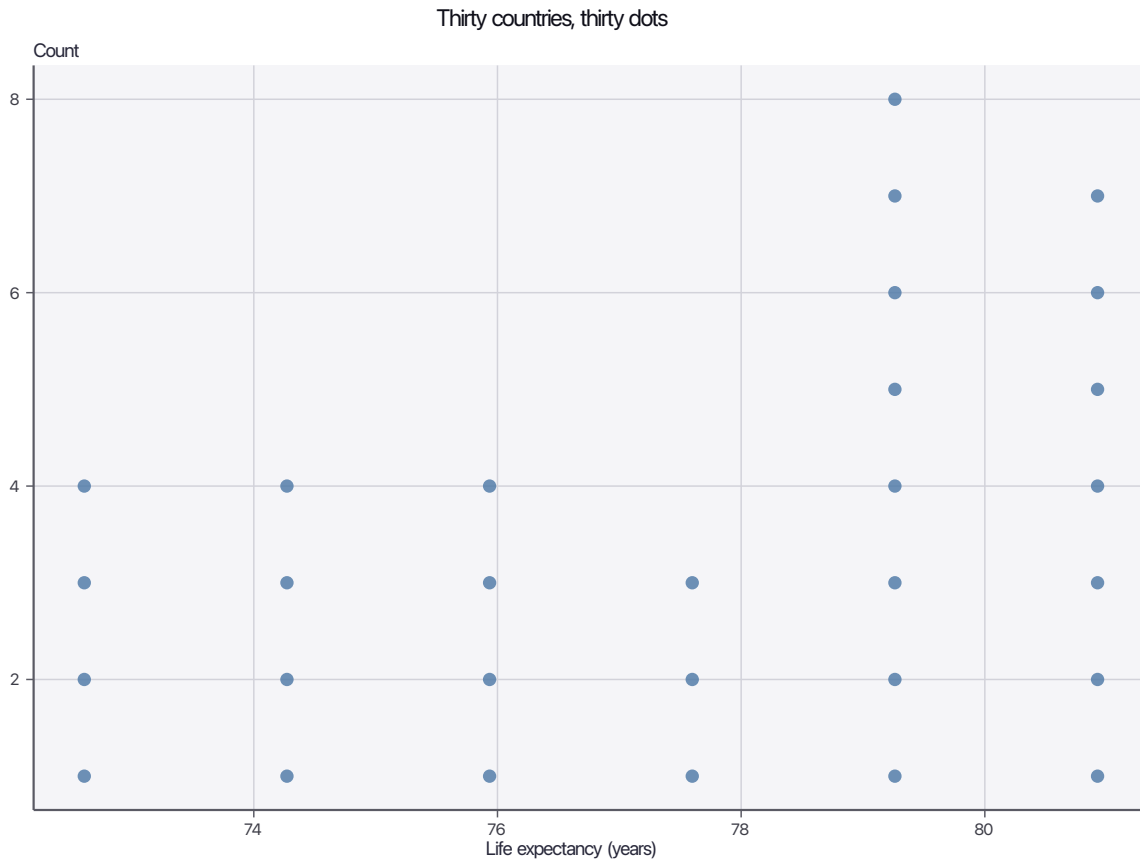
```
(data(gm_europe) + point * bin * stack + x(col.life) +  
  x_label("Life expectancy (years)") + title("Thirty countries, thirty  
  ↪ dots"))
```

Julia

```
data(gm_europe) + point * bin * stack + x(:life) +  
  x_label("Life expectancy (years)") +  
  title("Thirty countries, thirty dots")
```

JavaScript

```
plot(data(gm_europe), layer(point, bin, stack), x(col.life),  
  x_label("Life expectancy (years)"),  
  title("Thirty countries, thirty dots"))
```



“Given *gapminder Europe*: points derived by bin and stack, *x* is life.” The pile’s height is a count, so the tallest pile can be read off the axis or counted dot by dot. No row has been averaged into anything. Compare the same thirty as bars, where each pile becomes one length and the rows disappear into it:

R

```
data(gm_europe) + bar * bin + x(life) +
  x_label("Life expectancy (years)") + title("The same thirty, summarized")
```

Python

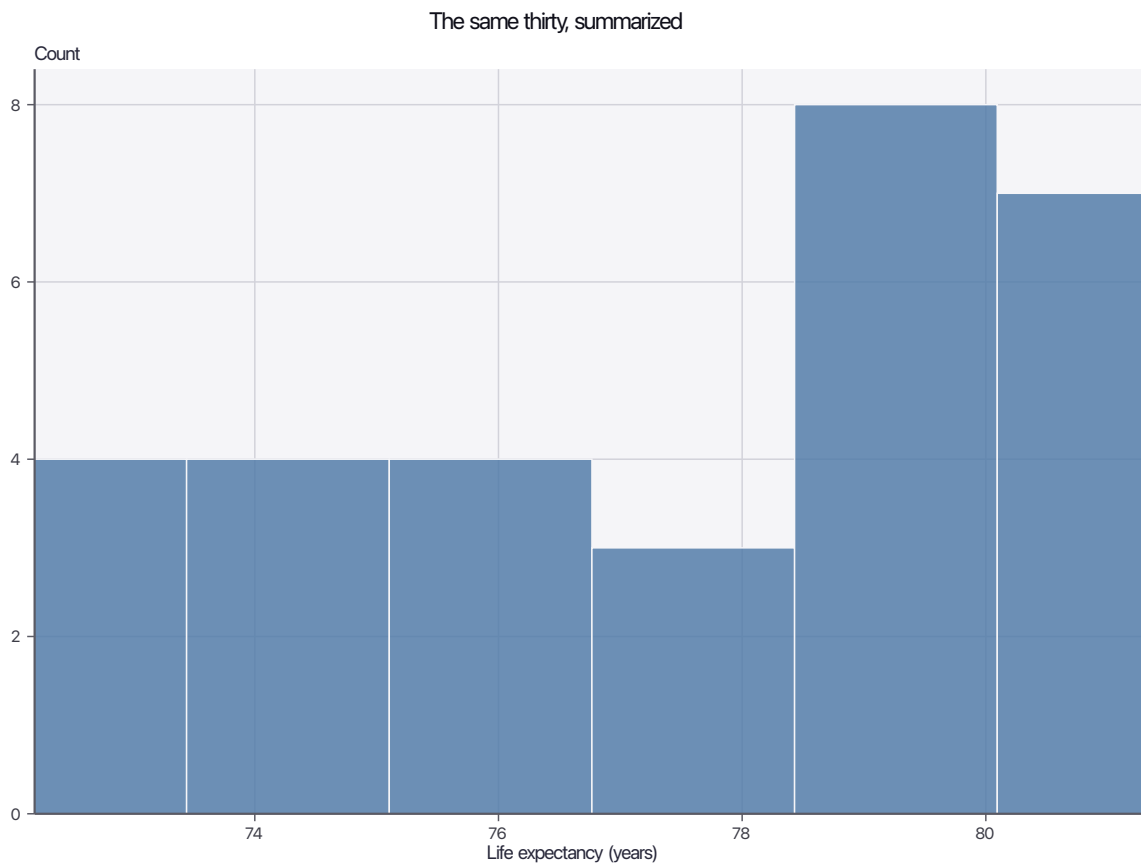
```
(data(gm_europe) + bar * bin + x(col.life) +
  x_label("Life expectancy (years)") + title("The same thirty, summarized"))
```

Julia

```
data(gm_europe) + bar * bin + x(:life) +
  x_label("Life expectancy (years)") +
  title("The same thirty, summarized")
```

JavaScript

```
plot(data(gm_europe), layer(bar, bin), x(col.life),
  x_label("Life expectancy (years)",
  title("The same thirty, summarized"))
```



“Given gapminder Europe: bars derived by bin, x is life.”

One atom away. A categorical column instead: `point * count * stack` piles one dot per row in each category. Owning chapter: [Transforms](#).

40.7 What this section refuses

A distribution question needs a continuous column. Give `density` a category and the refusal names the column type it needs. The categorical version of this question is the share recipe above:

```
data(gapminder_2007) + line * density + x(continent)
```

Error:

```
! gog: `density` estimates a continuous distribution, and `x(continent)` is  
  ↳ categorical - there is no number line for the curve to spread along. For  
  ↳ the share of rows in each category, that is `bar * proportion`.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```


41 Two variables

You have two columns. What can they tell you together? Their types decide the question. Two measurements ask about *relationship*: do high values of one go with high values of the other? A category and a measurement ask about *comparison*: is the measurement larger in some groups than in others?

41.1 Are two measurements related?

Two measurements, one on each position, and nothing else:

R

```
data(gapminder_2007) + point + x(gdp) + y(life)
```

Python

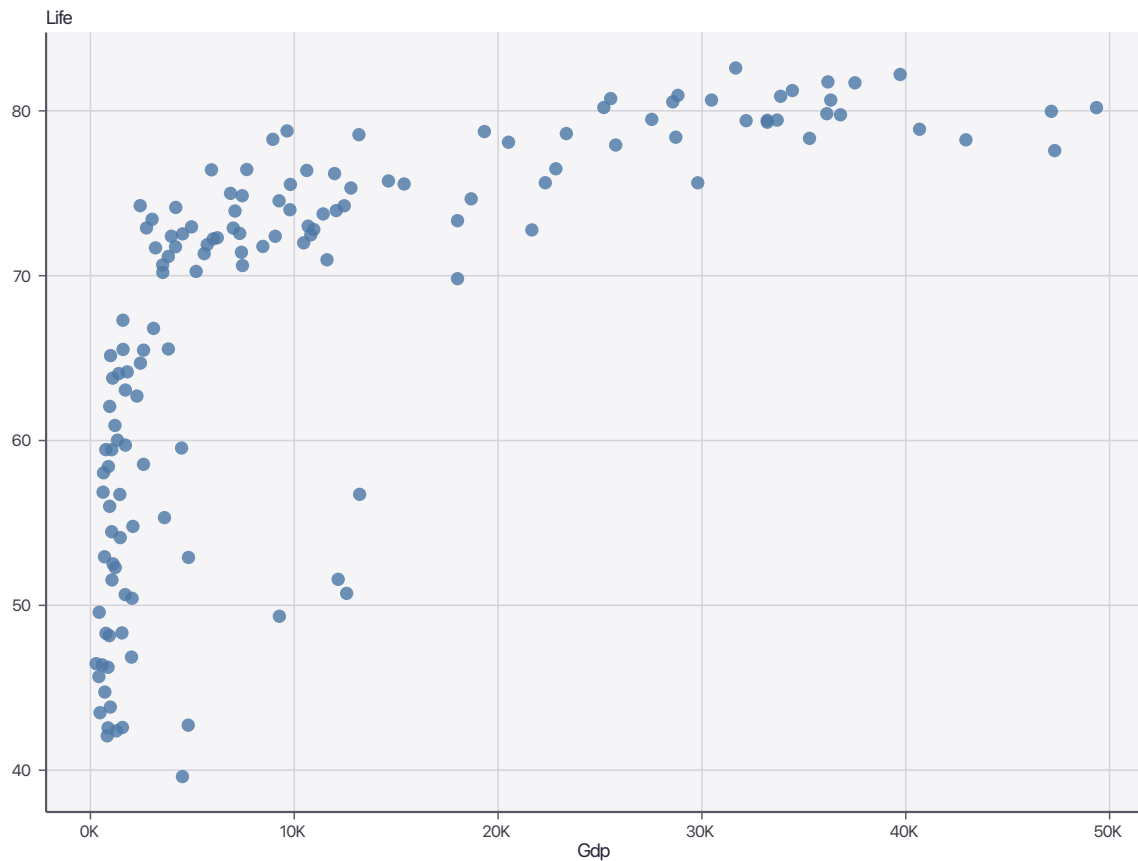
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life))
```



“Given gapminder 2007: points, x is gdp, y is life.” The scatter is the plainest sentence in the grammar: a mark and its two positions.

One atom away. When one column is spread across orders of magnitude, put its axis on a log scale. The scale is written on the binding, not as an atom of its own:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life)
```

Python

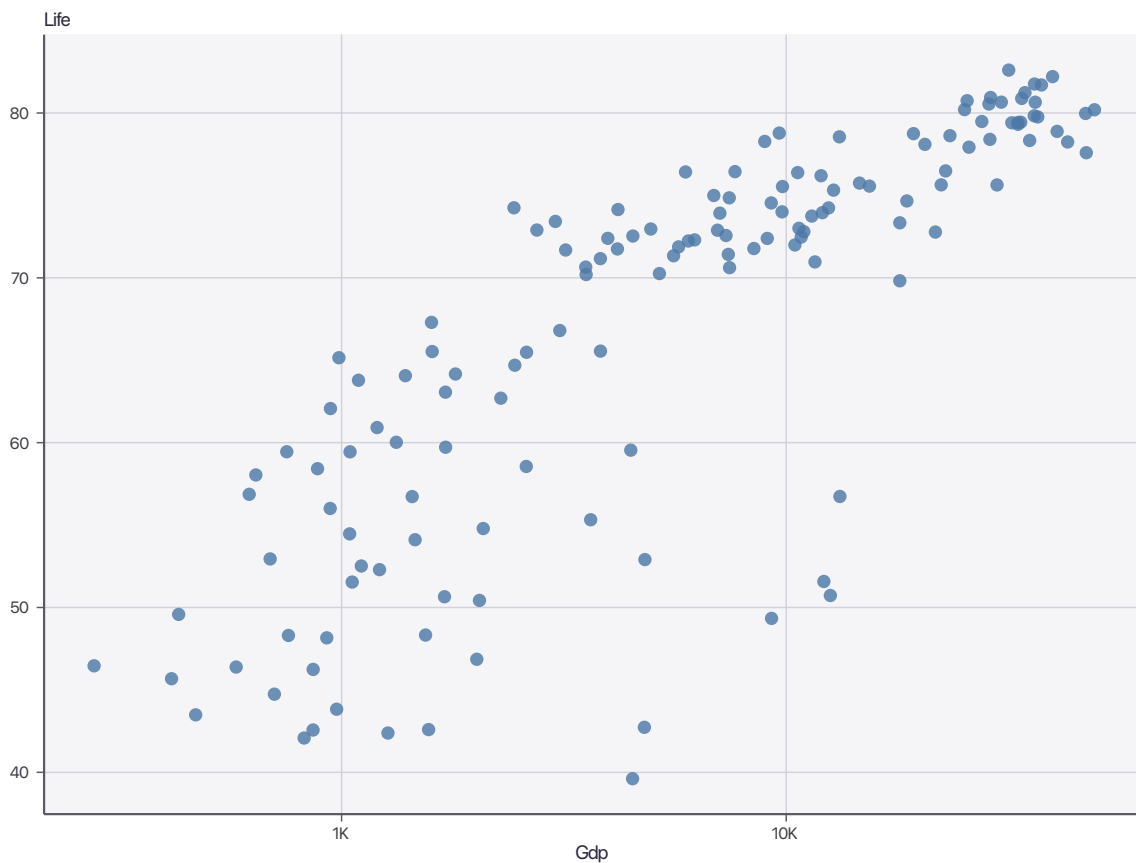
```
data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
y(col.life))
```



“Given gapminder 2007: points, x is gdp on a log scale, y is life.”

Owning chapters: [Point](#), [Scales](#).

41.2 What is the trend through the cloud?

Layer a derived line over the points. `+` puts both marks in one panel, and `* smooth` fits a curve through the middle of the cloud:

R

```
data(gapminder_2007) + x(gdp) + y(life) +  
  point + color(continent) +  
  line * smooth
```

Python

```
(data(gapminder_2007) + x(col.gdp) + y(col.life) +  
  point + color(col.continent) +  
  line * smooth)
```

Julia

```
data(gapminder_2007) + x(:gdp) + y(:life) + point + color(:continent) +  
  line * smooth
```

JavaScript

```
plot(data(gapminder_2007), x(col.gdp), y(col.life), point,  
  color(col.continent), layer(line, smooth))
```



“Given *gapminder 2007*: *x* is *gdp*, *y* is *life*, points colored by continent, and also a line derived by *smooth*.”

The positions are written before any mark, so both layers share them; *color* is written after point, so only the points split. *Transforms* owns *smooth*; *Encoding scope* owns that rule.

41.3 How much was there at each time?

A line says *where the value went*; an area says *how much there was*. It fills to zero, so the filled area is the amount:

R

```
data(actuals) + area + x(year) + y(sales)
```

Python

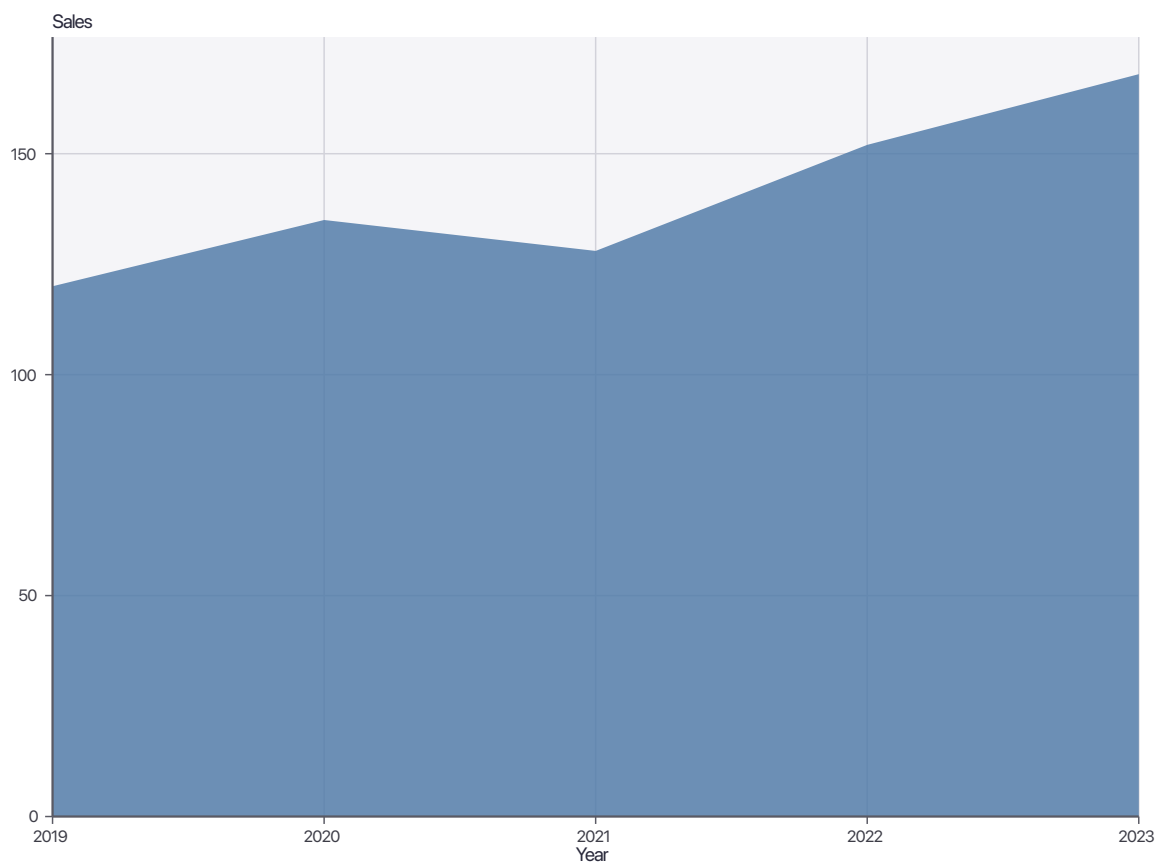
```
data(actuals) + area + x(col.year) + y(col.sales)
```

Julia

```
data(actuals) + area + x(:year) + y(:sales)
```

JavaScript

```
plot(data(actuals), area, x(col.year), y(col.sales))
```



“Given the actuals: an area, x is year, y is sales.”

One atom away. If your column has no meaningful zero, you want [line](#), not [area](#). Owning chapters: [Line](#), [Area](#).

41.4 What is the typical value per group?

A category and a measurement, summarized. The aggregation family is `sum`, `mean`, `median`, `max`, `min` and `quantile`. Each one reads a bound `y` and reduces every group to a single number. `quantile` also needs a probability, written `quantile(0.9)`:

R

```
data(gapminder_2007) + bar * mean + x(continent) + y(life)
```

Python

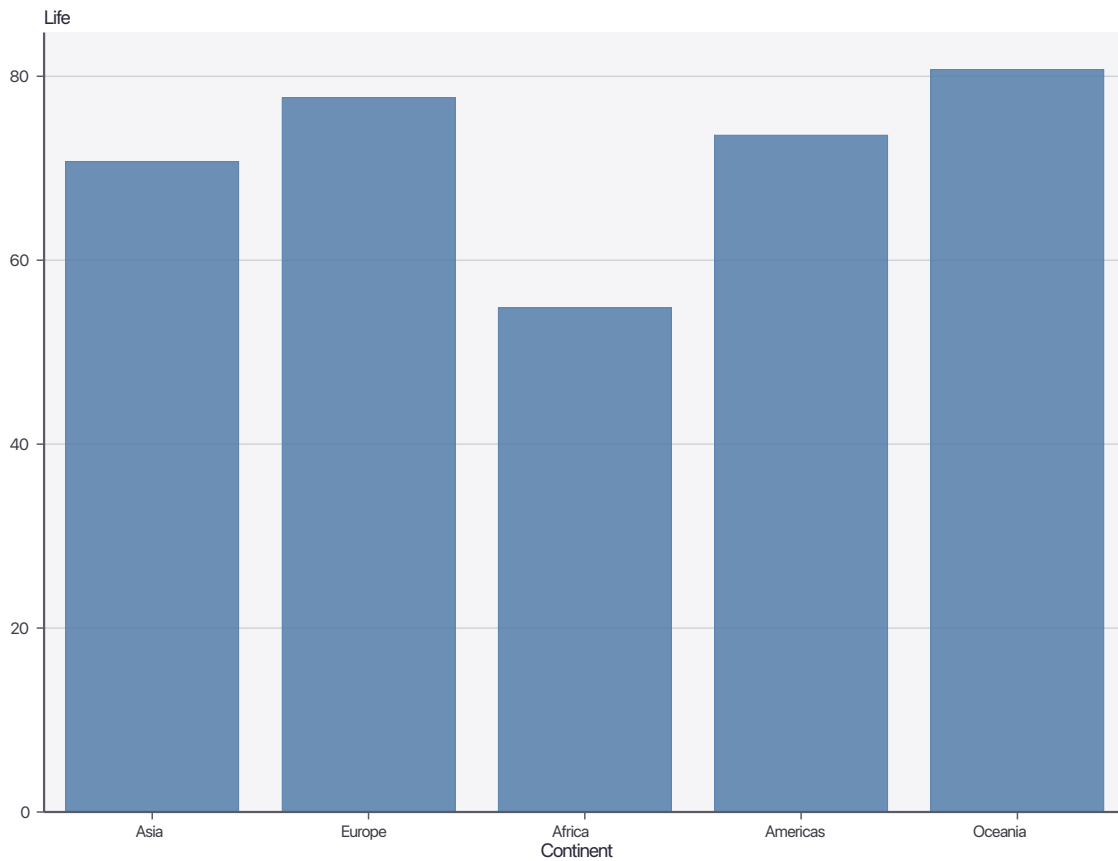
```
data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life)
```

Julia

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent),  
      y(col.life))
```



“Given gapminder 2007: bars derived by mean, x is continent, y is life.”

One atom away. The median is robust to outliers, so swap mean for median:

R

```
data(gapminder_2007) + bar * median + x(continent) + y(life)
```

Python

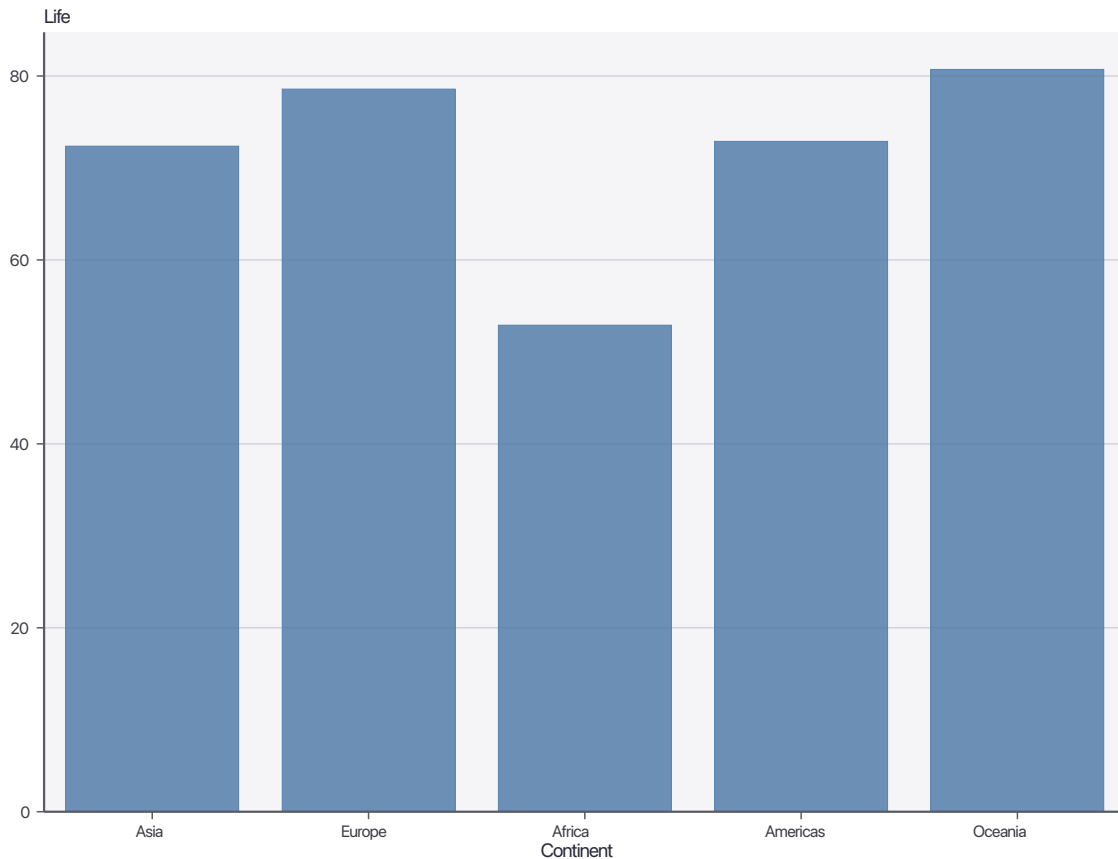
```
data(gapminder_2007) + bar * median + x(col.continent) + y(col.life)
```

Julia


```
data(gapminder_2007) + bar * median + x(:continent) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, median), x(col.continent),  
y(col.life))
```



“Given gapminder 2007: bars derived by median, x is continent, y is life.” Owing chapters: [Bar](#), [Transforms](#).

41.5 What does every value per group look like?

Summaries hide spread. The strip plot drops the derivation and changes the mark: `point` where `bar * mean` stood. Each row is one dot, at its own height in its category’s slot:

R

```
data(gapminder_2007) + point + x(continent) + y(life)
```

Python

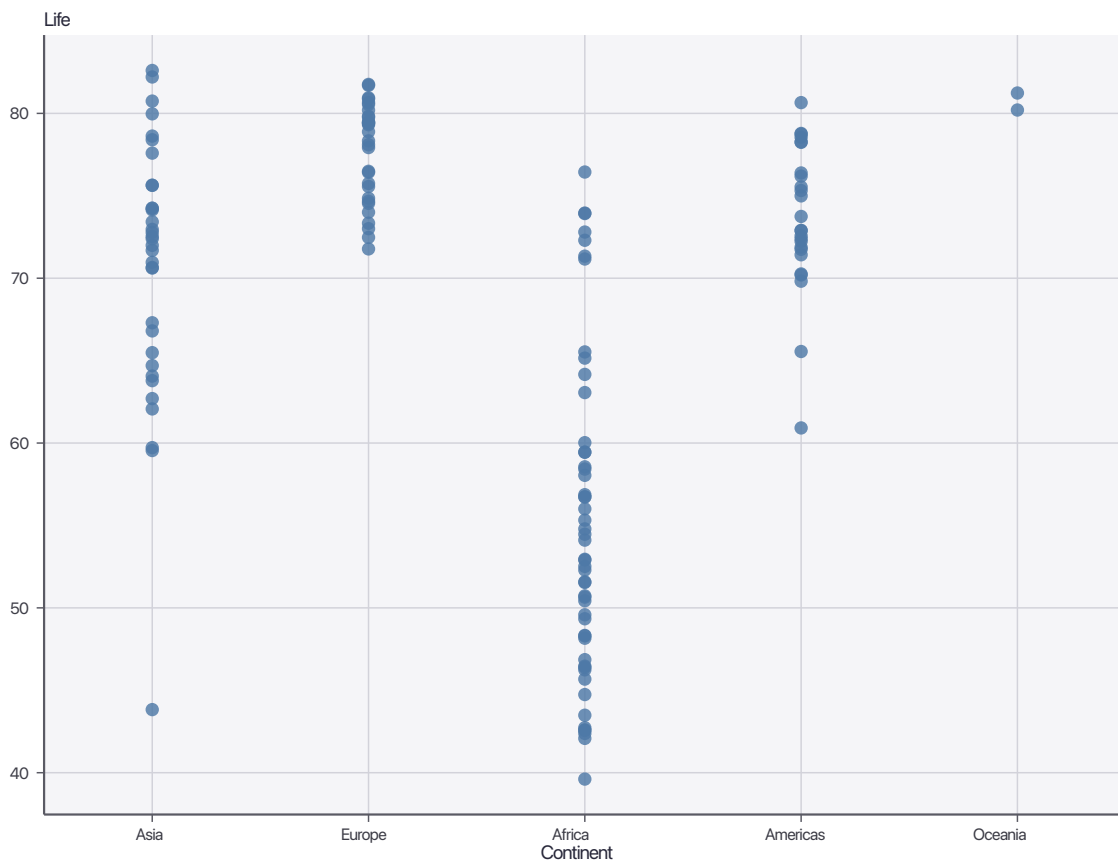
```
data(gapminder_2007) + point + x(col.continent) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:continent) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.continent), y(col.life))
```



“Given gapminder 2007: points, x is continent, y is life.”

When the dots pile onto one line, * `jitter` spreads them sideways, along the category axis only, so every dot keeps its exact height:

R

```
data(gapminder_2007) + point * jitter + x(continent) + y(life) +  
  ↪ style(opacity = 0.6)
```

Python

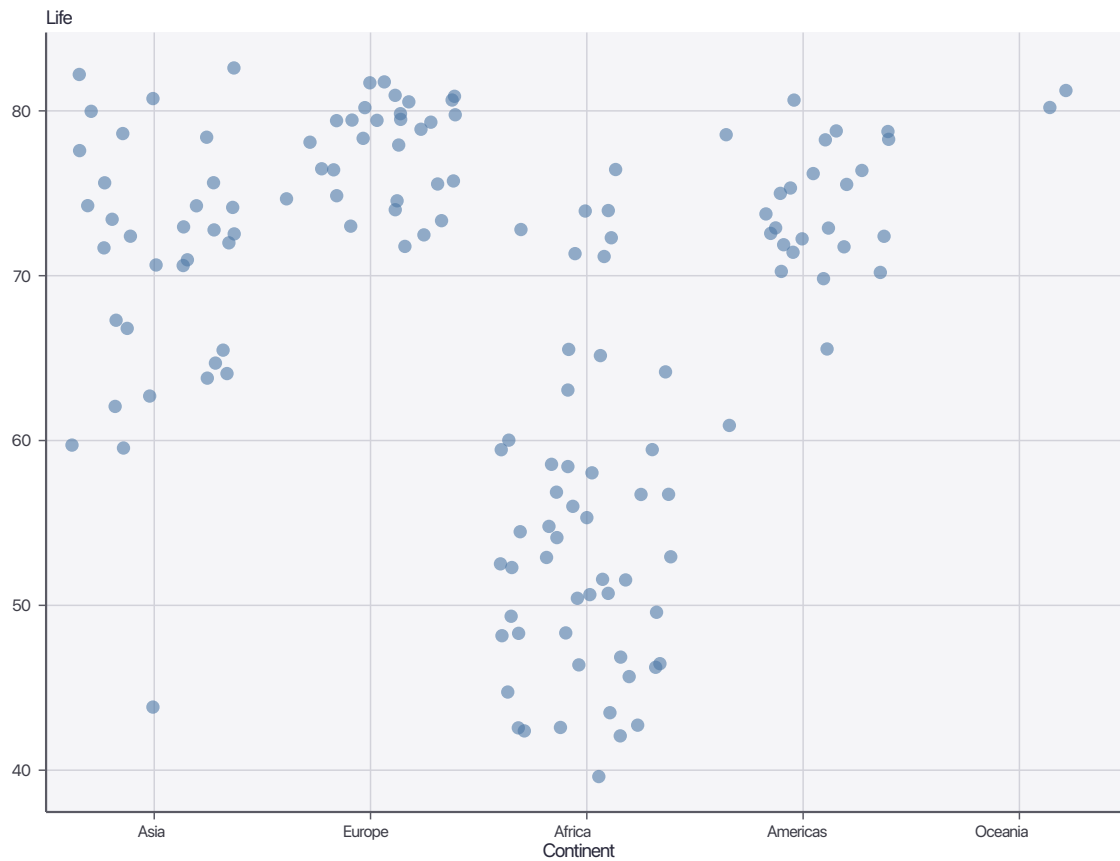
```
data(gapminder_2007) + point * jitter + x(col.continent) + y(col.life) +  
  ↪ style(opacity = 0.6)
```

Julia

```
data(gapminder_2007) + point * jitter + x(:continent) + y(:life) +  
  style(opacity = 0.6)
```

JavaScript

```
plot(data(gapminder_2007), layer(point, jitter), x(col.continent),  
  y(col.life), style({ opacity: 0.6 }))
```



“Given gapminder 2007: points derived by jitter, x is continent, y is life.”

One atom away. Swap the positions and the strips lie down, useful when the category names are long. See [Point](#) and [jitter](#).

41.6 What if the table already has the answer?

When the table already holds one value per category, no transform is involved; **bar** draws data as it arrives:

R

```
data(medals) + bar + x(country) + y(bronze)
```

Python

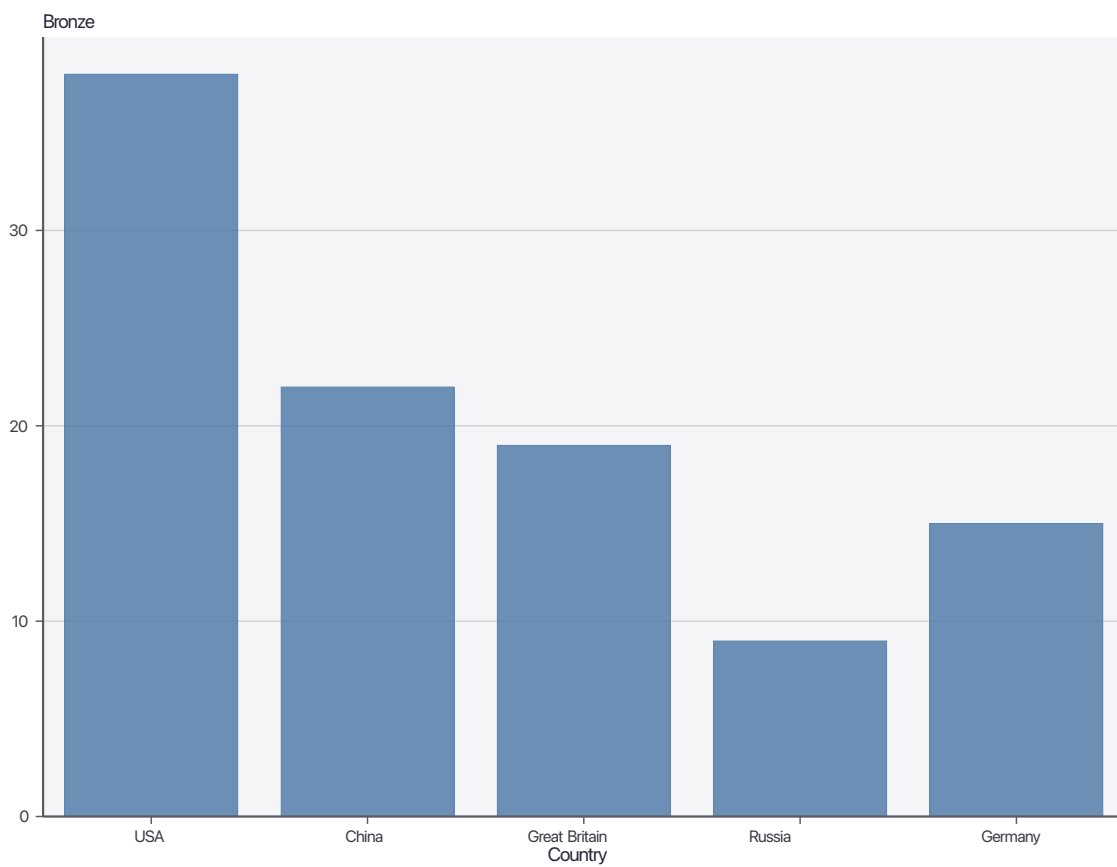
```
data(medals) + bar + x(col.country) + y(col.bronze)
```

Julia

```
data(medals) + bar + x(:country) + y(:bronze)
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.bronze))
```



“Given the medals: bars, x is country, y is bronze.”

One atom away. Ranked, because a comparison is easiest to read when it is sorted:

R

```
data(medals) + bar + x(country) + y(bronze) + order(bronze, desc = TRUE)
```

Python

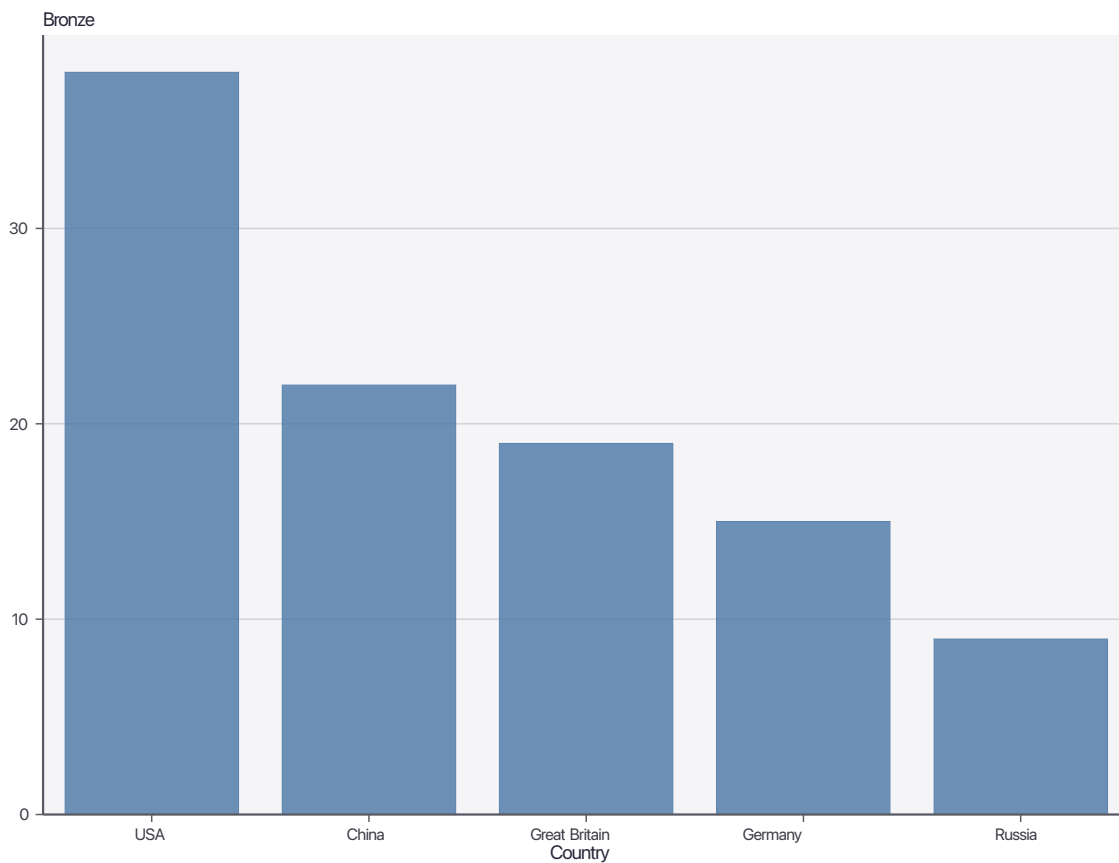
```
data(medals) + bar + x(col.country) + y(col.bronze) + order(col.bronze, desc  
↪ = True)
```

Julia

```
data(medals) + bar + x(:country) + y(:bronze) +  
  order(:bronze, desc = true)
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.bronze),  
  order(col.bronze, { desc: true })))
```



“Given the medals: bars, x is country, y is bronze, ordered by bronze, largest first.”

There is no hidden counting here to surprise you: if you want a computation, you write its name. That is why this recipe and [How many of each?](#) are different sentences.

41.7 What this section refuses

A bar needs at least one of its axes to be a number: that number is the length of the bar. Two categories leave it nothing to measure, so the refusal names `count`:

```
data(gapminder_2007) + bar + x(country) + y(continent)
```

Error:

```
! gog: `bar` has categorical columns on both axes - `x(country)` and  
  ↳ `y(continent)` - so there is nothing for it to measure. One axis must be  
  ↳ a number: that is the length of the bar. To count rows per category  
  ↳ instead, use `bar * count`.  
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

42 A third variable, and more

You have more columns than the flat page has positions. Where do the rest go? `x` and `y` are already taken. Every further variable is carried by a refining channel (`color`, `size`, `shape`, `pattern`, `opacity`) or by the third position, `z`. Each channel answers one kind of question, and a refining channel earns the legend that decodes it.

42.1 Which group does each mark belong to?

A category is carried by `color`, and the palette gives each category one hue:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  color(continent)
```

Python

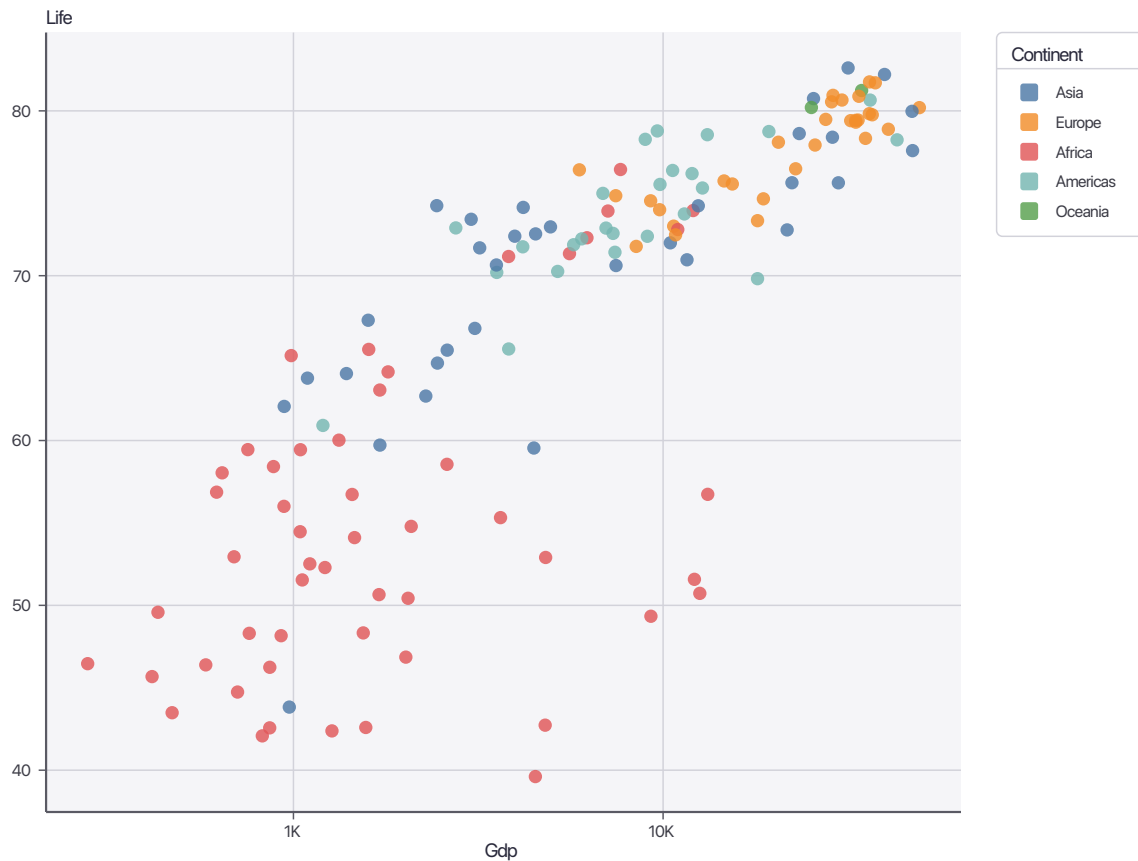
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.continent))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.continent))
```

“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by continent.”

One atom away. The same split without color, when you want the grouping but not the hues, is the `group` channel; see [Channels](#).

42.2 Will it read in gray?

Bind `color` *and* `shape` to the same column and each group is said twice, readable for colorblind readers, in grayscale print, and in poor light:

R

```
data(iris_flowers) + point + x(petal_length) + y(sepal_length) +
  color(species) + shape(species) +
  palette("okabe")
```

Python

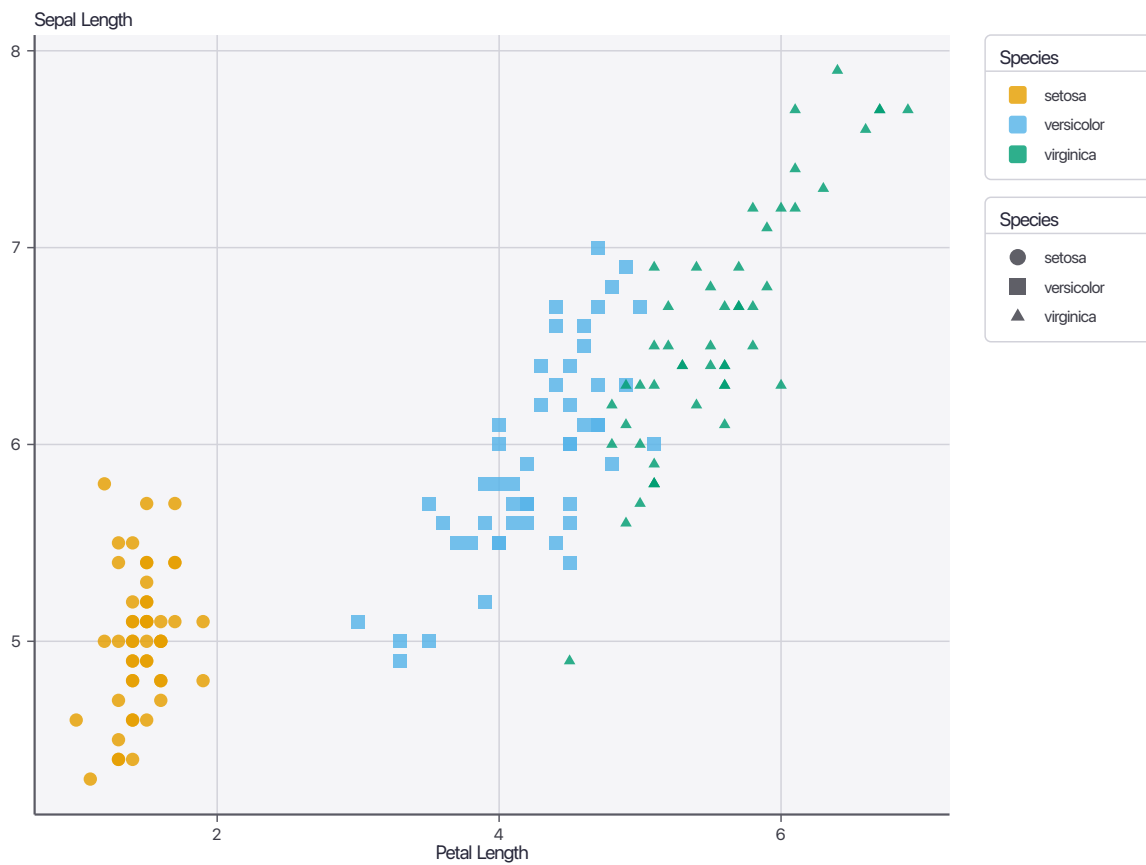
```
(data(iris_flowers) + point + x(col.petal_length) + y(col.sepal_length) +  
  color(col.species) + shape(col.species) +  
  palette("okabe"))
```

Julia

```
data(iris_flowers) + point + x(:petal_length) + y(:sepal_length) +  
  color(:species) + shape(:species) + palette("okabe")
```

JavaScript

```
plot(data(iris_flowers), point, x(col.petal_length), y(col.sepal_length),  
  color(col.species), shape(col.species), palette("okabe"))
```



“Given the iris flowers: points, x is petal length, y is sepal length, color by species, shape by species, with the okabe palette.”

`palette()` chooses which colors the `color` channel uses: here the Okabe and Ito colorblind-safe set. Saying one column twice is redundant encoding. Owing chapters: [Channels](#), [Setting vs mapping](#).

42.3 How big is each observation?

A magnitude is carried by `size`; the channel’s question is *how much*, so it only accepts a numeric column:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +  
  size(population)
```

Python

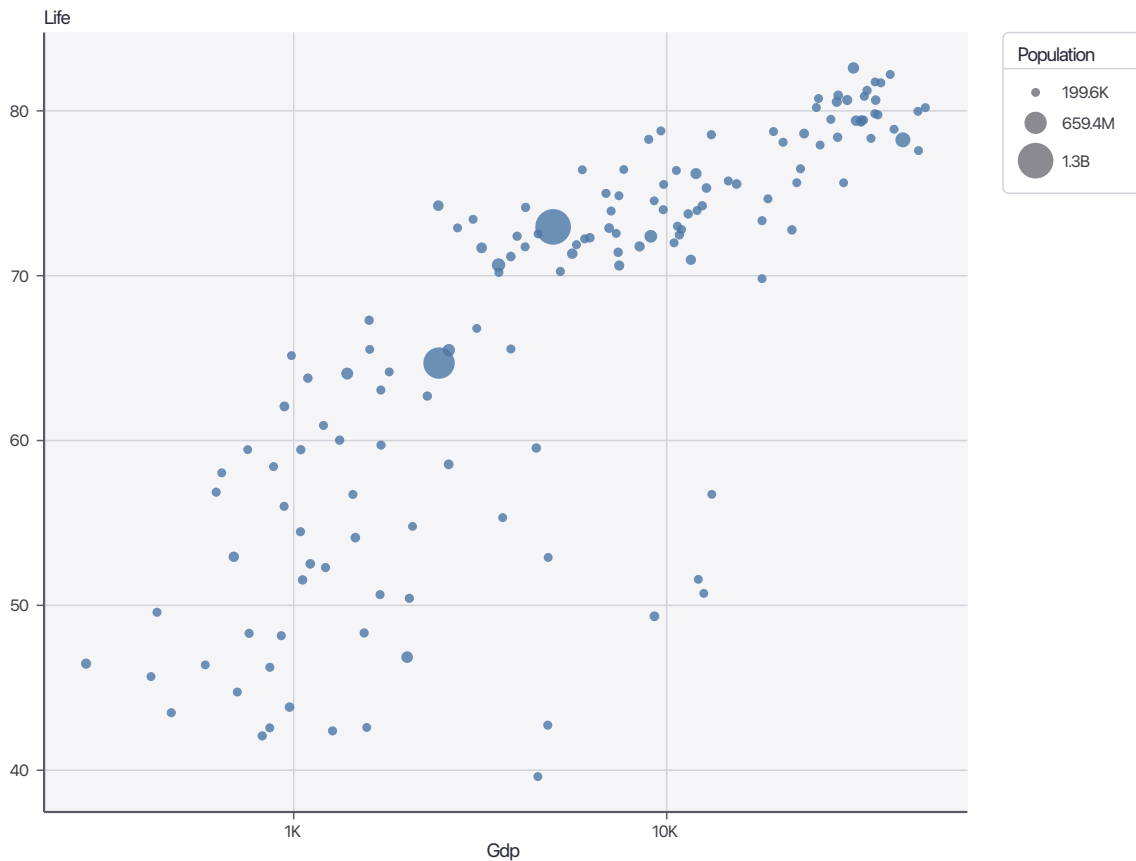
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  size(col.population))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  size(:population)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), size(col.population))
```



“Given *gapminder 2007*: *points*, *x* is *gdp* on a *log* scale, *y* is *life*, size by *population*.” A legend decodes the radius. Owning chapter: [Channels](#).

42.4 What if four variables matter?

Channels compose: every one of them works beside every other. That rule is [Orthogonality](#). Wealth, health, population, and continent in one sentence:

R

```
data(gapminder_2007) + point +
  x(gdp, scale = "log") + y(life) +
  color(continent) + size(population) +
  title("The health and wealth of nations, 2007")
```

Python

```
(data(gapminder_2007) + point +  
  x(col.gdp, scale = "log") + y(col.life) +  
  color(col.continent) + size(col.population) +  
  title("The health and wealth of nations, 2007"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:continent) + size(:population) +  
  title("The health and wealth of nations, 2007")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.continent), size(col.population),  
  title("The health and wealth of nations, 2007"))
```



“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by continent, size by population.”

Every channel keeps its own rule and its own legend; none of them changed meaning by being combined. Four variables is usually the most a single plot can carry. Past that, use [panels](#) instead of more channels.

42.5 What if the third variable is a number?

`color` accepts either kind of column. A numeric column gets a sequential ramp instead of a palette, and a skewed one reads better on a log ramp, exactly like a skewed axis:

R

```
data(gapminder_2007) + point + x(gdp, scale = "log") + y(life) +
  color(population, scale = "log")
```

Python

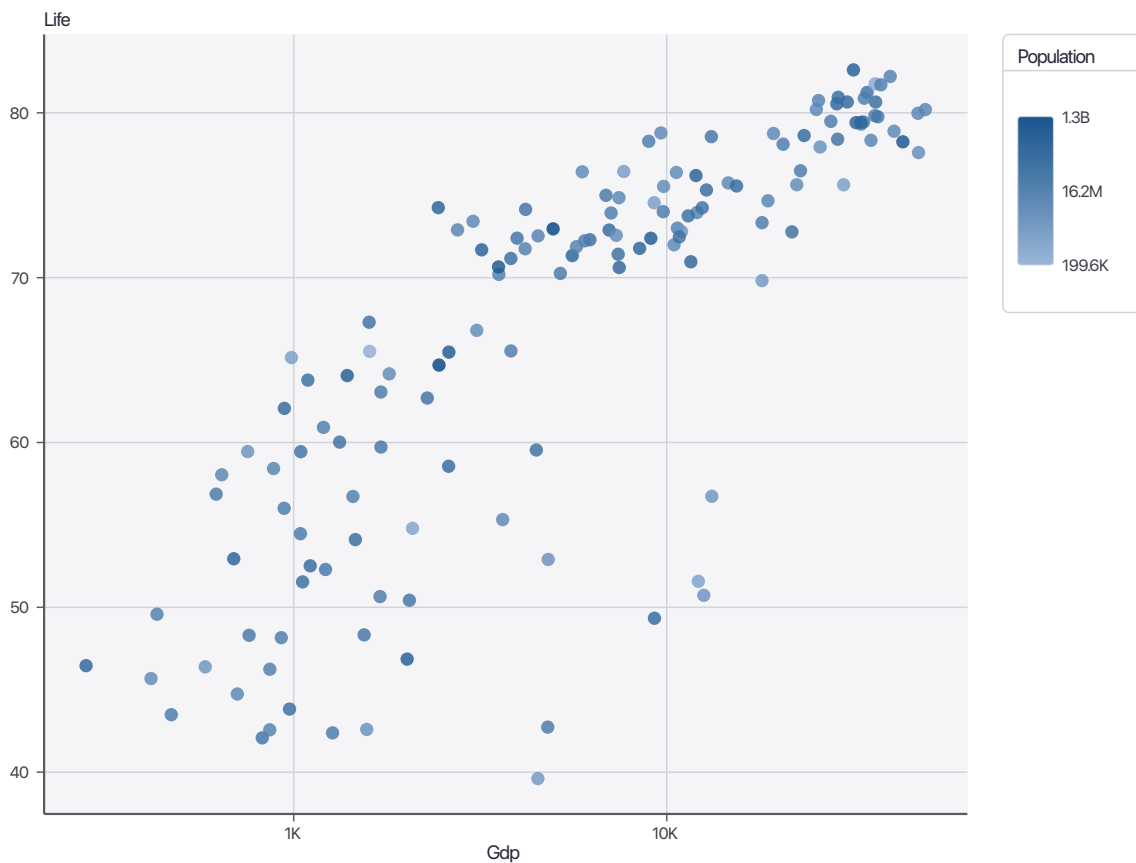
```
(data(gapminder_2007) + point + x(col.gdp, scale = "log") + y(col.life) +  
  color(col.population, scale = "log"))
```

Julia

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  color(:population, scale = "log")
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: "log" }),  
  y(col.life), color(col.population, { scale: "log" }))
```



“Given *gapminder 2007*: *points*, *x* is *gdp* on a log scale, *y* is *life*, color by population on a log scale.”

The legend becomes a labeled gradient strip: same channel, same rule, different column type. `Scales` owns the log scale.

42.6 What if three measurements matter equally?

When three *measurements* matter equally, the third one is a position, not a decoration. One more vowel and the plot stands up:

R

```
data(iris_flowers) + point +  
  x(sepal_length) + y(sepal_width) + z(petal_length) +  
  color(species)
```

Python

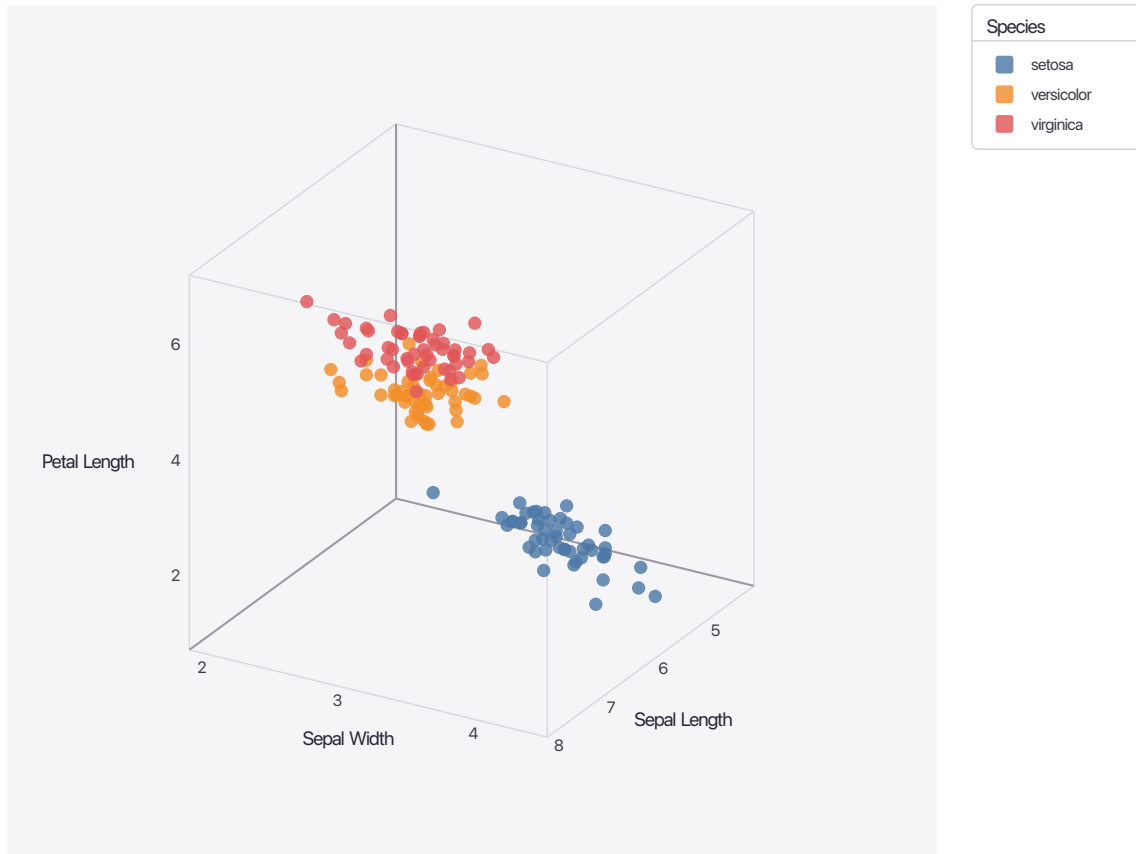
```
(data(iris_flowers) + point +  
  x(col.sepal_length) + y(col.sepal_width) + z(col.petal_length) +  
  color(col.species))
```

Julia

```
data(iris_flowers) + point + x(:sepal_length) + y(:sepal_width) +  
  z(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), point, x(col.sepal_length), y(col.sepal_width),  
  z(col.petal_length), color(col.species))
```

“Given the iris flowers: points, x is sepal length, y is sepal width, z is petal length, color by species.”

One atom away. The viewing angle is a property of the space, not of the data; **turn** rotates the scene sideways, **tilt** raises the viewing height:

R

```
data(iris_flowers) + point +
  x(sepal_length) + y(sepal_width) + z(petal_length) +
  color(species) +
  space(turn = -50, tilt = 15)
```

Python

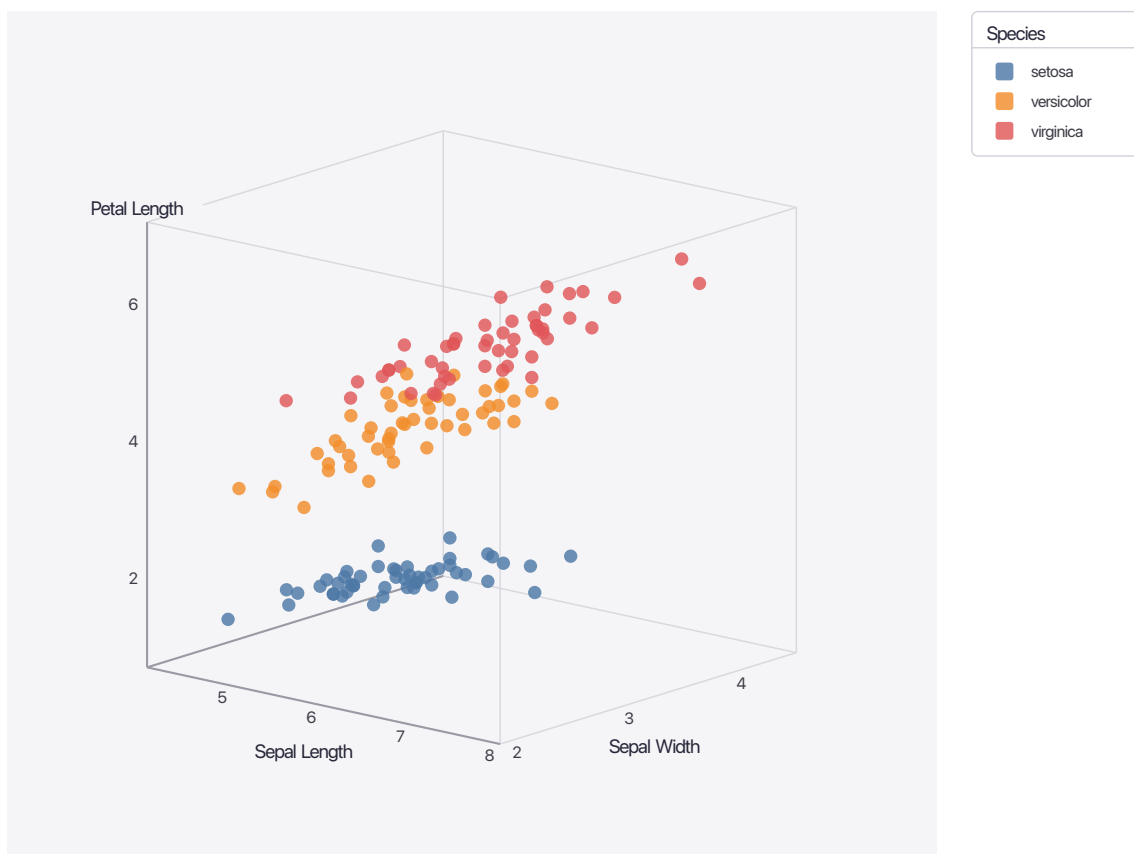
```
(data(iris_flowers) + point +
  x(col.sepal_length) + y(col.sepal_width) + z(col.petal_length) +
  color(col.species) +
  space(turn = -50, tilt = 15))
```

Julia

```
data(iris_flowers) + point + x(:sepal_length) + y(:sepal_width) +
  z(:petal_length) + color(:species) + space(turn = -50, tilt = 15)
```

JavaScript

```
plot(data(iris_flowers), point, x(col.sepal_length), y(col.sepal_width),
  z(col.petal_length), color(col.species), space({ turn: -50, tilt: 15 })))
```



“Given the iris flowers: points, x is sepal length, y is sepal width, z is petal length, color by species, turned -50, tilted 15.”

Space owns the third dimension.

42.7 What if the rows form a route?

The scatter above is a cloud: each flower is its own dot and the table’s order means nothing. Sometimes the order is the point. Here are the two gliders from [Path](#) again. Each row records where that glider was next, so the marks belong joined in the order they were recorded. The mark that joins rows in table order is `path`:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) +  
  color(glider) +  
  title("Two gliders, one thermal")
```

Python

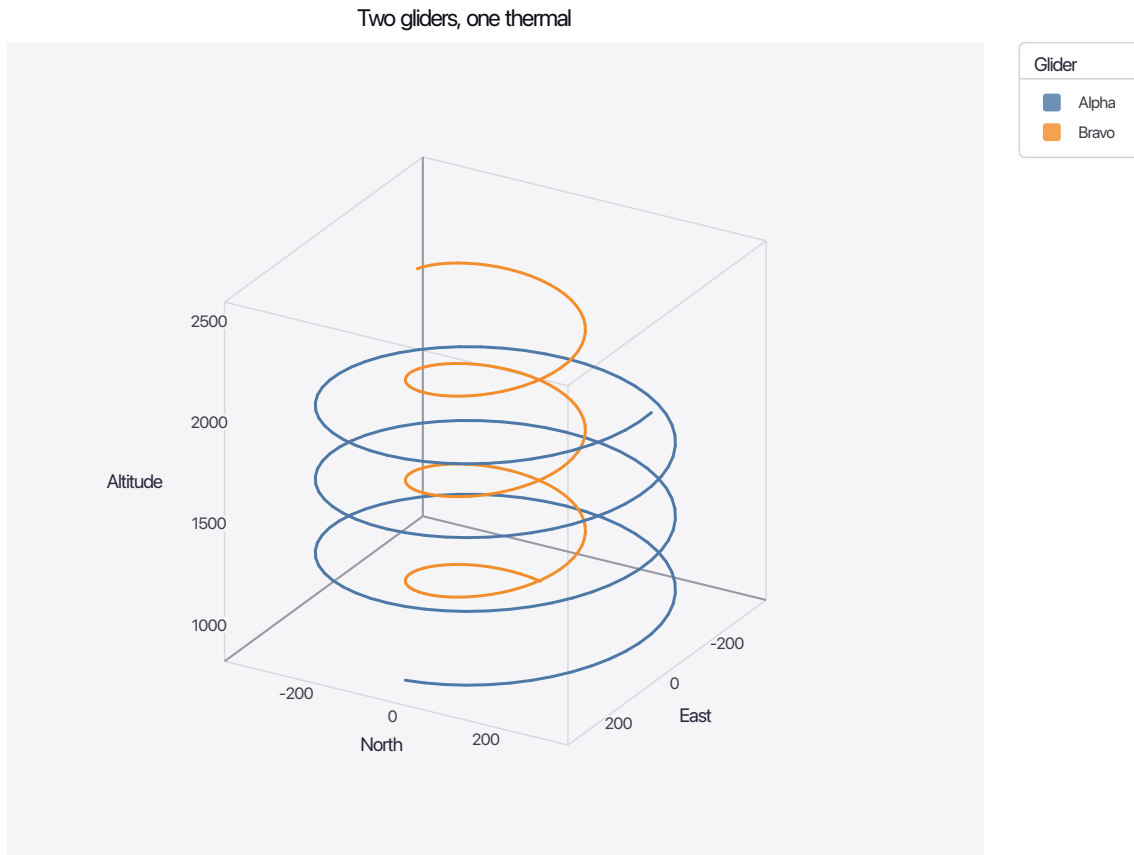
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +  
  color(col.glider) +  
  title("Two gliders, one thermal"))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +  
  color(:glider) + title("Two gliders, one thermal")
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),  
  color(col.glider), title("Two gliders, one thermal"))
```



“Given the thermals: paths, x is east, y is north, z is altitude, color by glider.”

Each route rises because the gliders do. Nothing in the sentence draws a spiral: **path** was told to connect the rows in order, the rows happen to go around while they go up, and the coil is the result. Watch where the two routes cross, too. Whichever route is nearer covers the other, and the nearer one changes as they climb, so neither glider is simply drawn on top.

Turn the view and the same route answers a different question, with no binding changed:

R

```
data(thermals) + path + x(east) + y(north) + z(altitude) +
  color(glider) +
  space(turn = 20, tilt = 80) +
  title("The same two routes, seen from overhead")
```

Python

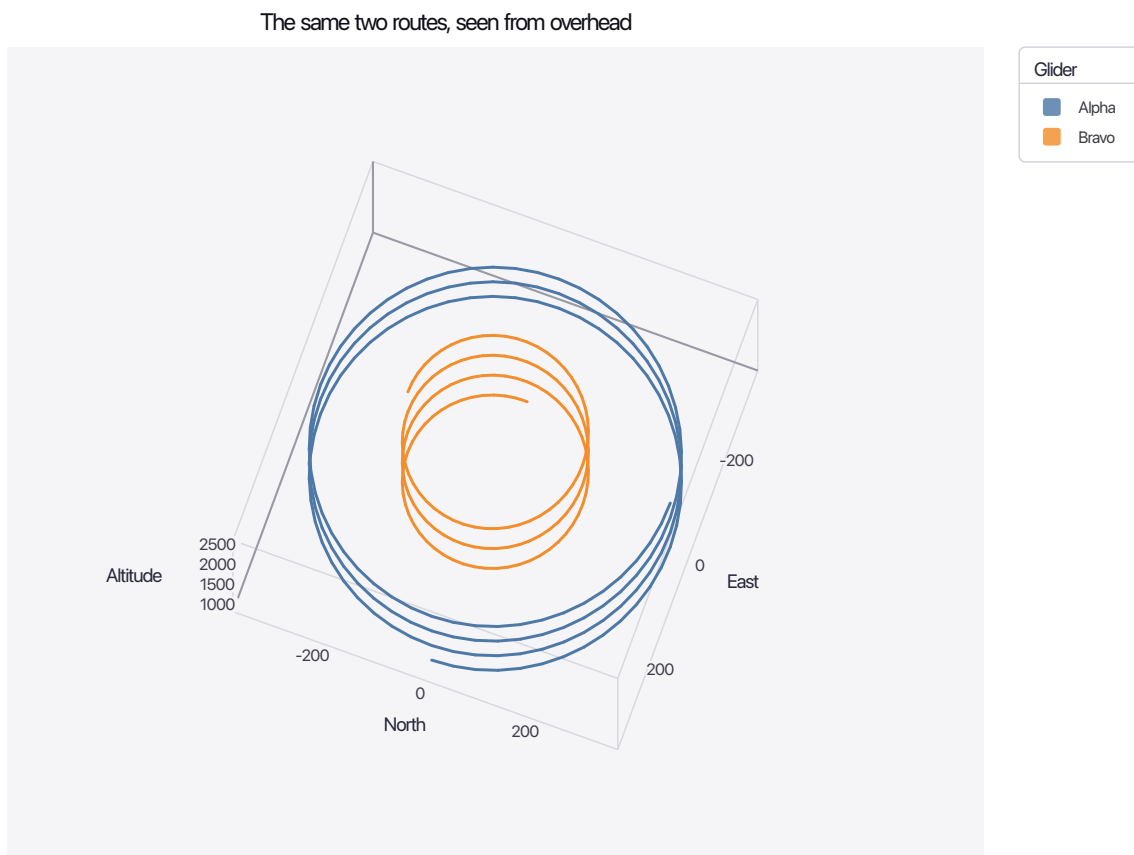
```
(data(thermals) + path + x(col.east) + y(col.north) + z(col.altitude) +
  color(col.glider) +
  space(turn = 20, tilt = 80) +
  title("The same two routes, seen from overhead"))
```

Julia

```
data(thermals) + path + x(:east) + y(:north) + z(:altitude) +
  color(:glider) + space(turn = 20, tilt = 80) +
  title("The same two routes, seen from overhead")
```

JavaScript

```
plot(data(thermals), path, x(col.east), y(col.north), z(col.altitude),
  color(col.glider), space({ turn: 20, tilt: 80 }),
  title("The same two routes, seen from overhead"))
```



“Given the thermals: paths, x is east, y is north, z is altitude, color by glider, turned 20, tilted 80.”

From nearly overhead the climb collapses and you are looking down at two circles. The plot has become a picture of how wide each circle is, and it shows what the first view hid: Bravo is flying a far tighter circle than Alpha. The side view above showed Bravo climbing faster; this view shows the tighter turn that goes with it.

Of **line**, **step**, **area**, **ribbon** and **path**, only **path** takes **z**, and the reason is the one that separates it from **line** in the plane. A path joins rows in the table’s order, and an order belongs to no axis, so it survives the third dimension untouched. A **line** sorts by **x**, and the cube has no left to right: at some angles **x** runs straight into the page and becomes depth rather than position, which would leave a line sorted by an axis you cannot see. So a rising route is **path**, and **line** refuses to pretend otherwise:

```
data(thermals) + line + x(east) + y(north) + z(altitude) + color(glider)
```

Error:

```
! gog: `line` reads a *domain* left to right - it sorts by `x` and draws one
  ↳ value for each - and a cube has no left to right: `x` is one of three
  ↳ equal positions, and at some viewing angles it runs into the page and
  ↳ becomes depth. A `line` in space would be sorted by an axis the reader
  ↳ cannot see, so this is refused rather than drawn. For a route through
  ↳ three dimensions use `path`, which is `line` with that sort removed:
  ↳ `path + x(<a>) + y(<b>) + z(altitude)`.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

42.8 What if the shape needs three dimensions?

quakes_fiji is the thousand earthquakes from [Map](#), each with a place on the globe and a depth below it. The reason it is here is that **east**, **north** and **elevation** show together what no pair of them shows alone:

R

```
data(quakes_fiji) + point + x(east) + y(north) + z(elevation) +
  color(magnitude) + palette("viridis") + space(turn = 120, tilt = 20) +
  title("A thousand earthquakes, and the sheet they lie on")
```

Python

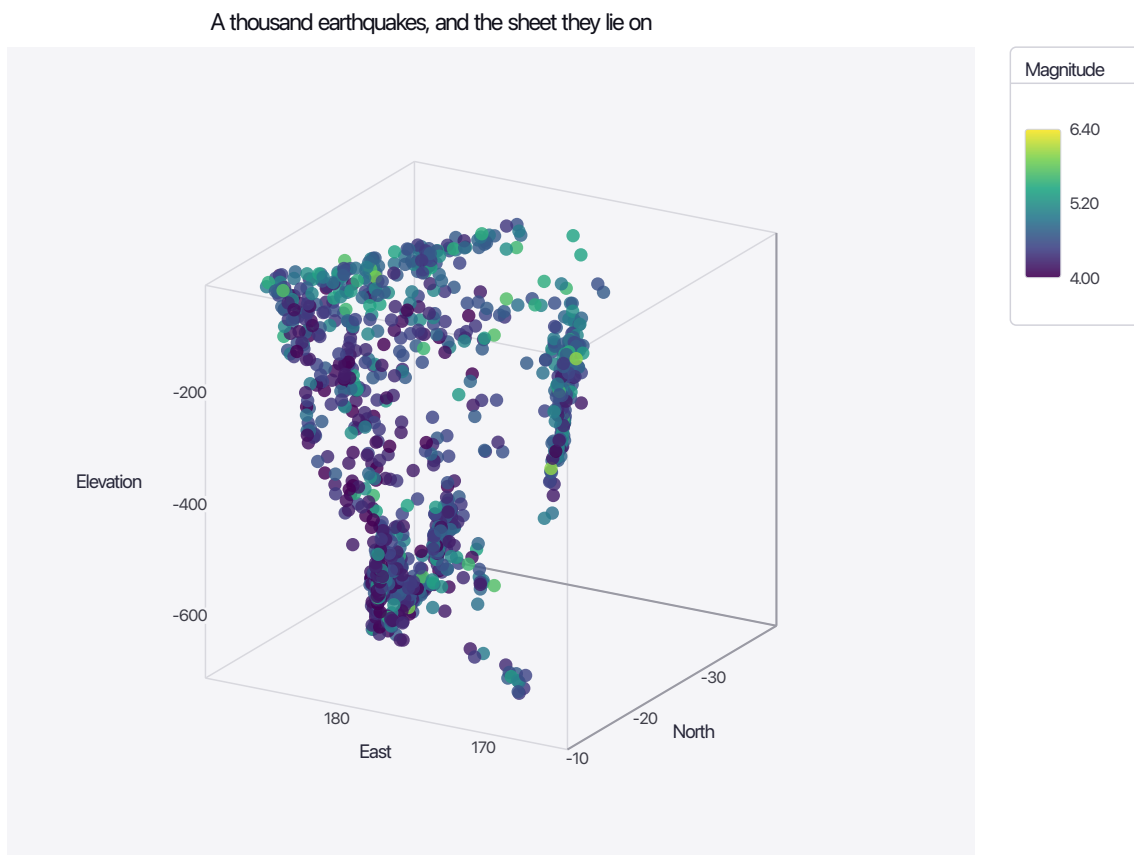
```
(data(quakes_fiji) + point + x(col.east) + y(col.north) + z(col.elevation) +
  color(col.magnitude) + palette("viridis") + space(turn = 120, tilt = 20) +
  title("A thousand earthquakes, and the sheet they lie on"))
```

Julia

```
data(quakes_fiji) + point + x(:east) + y(:north) + z(:elevation) +
  color(:magnitude) + palette("viridis") + space(turn = 120, tilt = 20) +
  title("A thousand earthquakes, and the sheet they lie on")
```

JavaScript

```
plot(data(quakes_fiji), point, x(col.east), y(col.north),
  z(col.elevation), color(col.magnitude), palette("viridis"),
  space({ turn: 120, tilt: 20 }),
  title("A thousand earthquakes, and the sheet they lie on"))
```



“Given quakes Fiji: points, x is east, y is north, z is elevation, color by magnitude.”

The column is named `elevation`, so a measurement below sea level is a negative number: the deepest quake in the table sits at -680 km. The quakes are not scattered through the cube. They trace a sheet of ocean floor sinking into the earth. It starts shallow in the east and runs down past 600 km to the west. That is a shape, and a shape needs three positions to be a shape.

Now cut the quakes into slices of depth and run the slices one after another. `play(slab)` steps through `slab`, a column that holds the depth in 90 km bands:

R

```
data(quakes_fiji) + point + x(east) + y(north) + z(elevation) +  
  space(turn = 120, tilt = 20) + play(slab)
```

Python

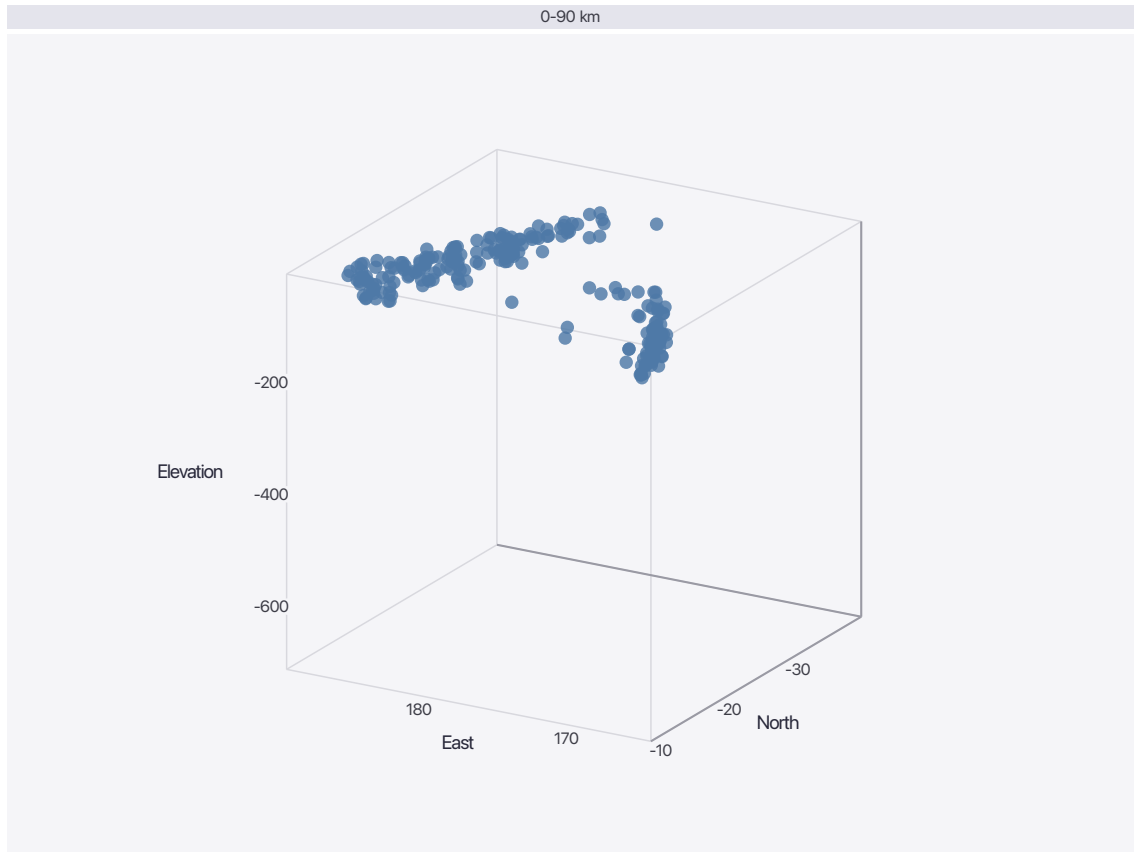
```
(data(quakes_fiji) + point + x(col.east) + y(col.north) + z(col.elevation) +  
  space(turn = 120, tilt = 20) + play(col.slab))
```

Julia

```
data(quakes_fiji) + point + x(:east) + y(:north) + z(:elevation) +  
  space(turn = 120, tilt = 20) + play(:slab)
```

JavaScript

```
plot(data(quakes_fiji), point, x(col.east), y(col.north),  
  z(col.elevation), space({ turn: 120, tilt: 20 }), play(col.slab))
```

“Given quakes Fiji: points, x is east, y is north, z is elevation, played by slab.”

The band of points moves down and west as the sequence runs, which is the slope of that sheet read one slice at a time. The frames are not time here. **play** advances through the levels of an ordered column, and whether that column measures years or kilometers is the column’s business, not the channel’s. Depth happens to be the one variable a cross-section is usually cut on, so this is the ordinary cross-section view, spelled with the atom that also animates a year.

The same slices laid out across the page instead of shown one after another in time:

R

```
data(quakes_fiji) + point + x(east) + y(north) + z(elevation) +
  space(turn = 120, tilt = 20) | facet(slab, wrap = 4)
```

Python

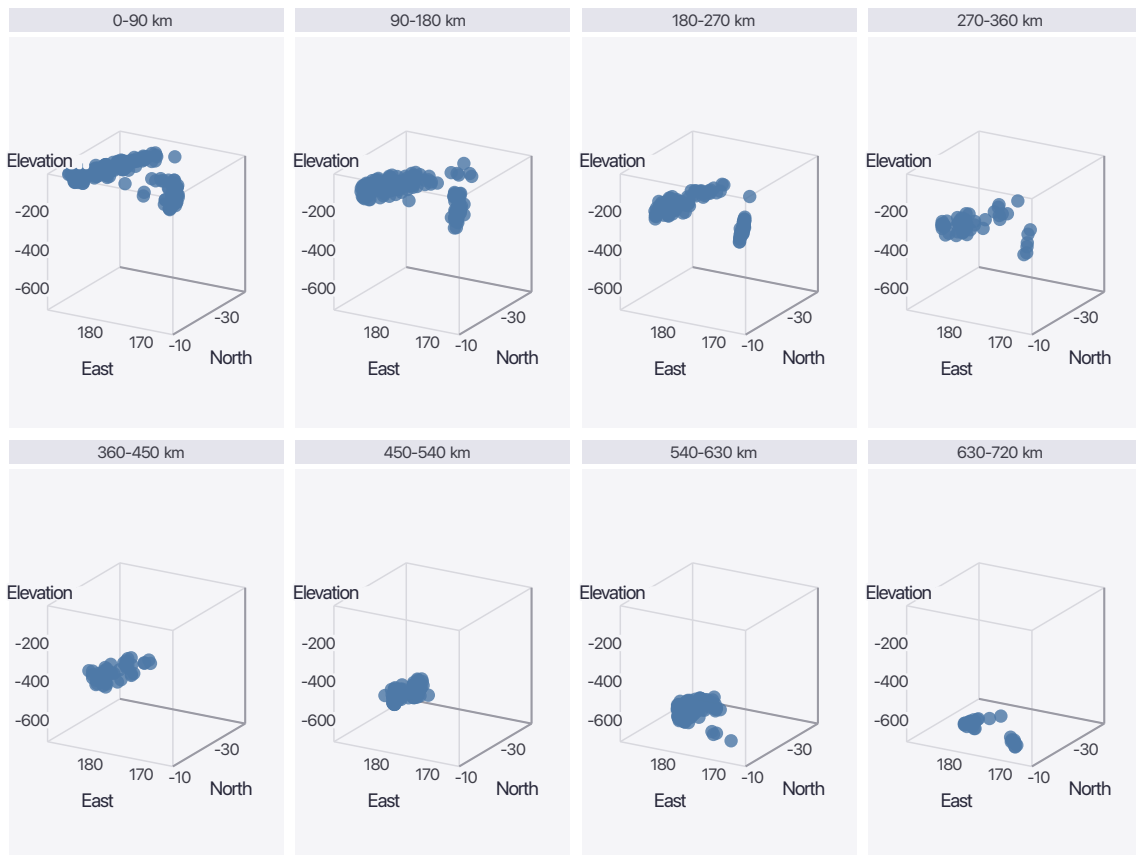
```
(data(quakes_fiji) + point + x(col.east) + y(col.north) + z(col.elevation) +
  space(turn = 120, tilt = 20) | facet(col.slab, wrap = 4))
```

Julia

```
data(quakes_fiji) + point + x(:east) + y(:north) + z(:elevation) +
  space(turn = 120, tilt = 20) | facet(:slab, wrap = 4)
```

JavaScript

```
plot(data(quakes_fiji), point, x(col.east), y(col.north),
  z(col.elevation), space({ turn: 120, tilt: 20 }),
  across(col.slab, { wrap: 4 })))
```



“Given quakes Fiji: points, x is east, y is north, z is elevation, split into panel columns by slab, wrapped at 4.”

Eight cubes, on one shared scale, so the reader compares them side by side instead of remembering them. Choosing between them is a question about the reader: panels let them look back, frames let them see the movement. [Play](#) owns that choice.

42.9 Which rows and columns are alike?

Two categorical axes and one value make a tile plot, and nothing in the data chose its slot order. Build a [cluster](#) tree on each axis, then compose the three plots: the shared axes take the trees’ leaf order, and tiles of similar color end up next to each other.

R

```
(data(nutrients) + path * cluster(amount, over = nutrient) + x(food) +  
  theme(height = 150)) /  
((data(nutrients) + zone + x(food) + y(nutrient) + color(amount)) |  
  (data(nutrients) + path * cluster(amount, over = food) + y(nutrient) +  
    theme(width = 150)))
```

Python

```
((data(nutrients) + path * cluster(col.amount, over=col.nutrient) +  
  ↪ x(col.food) +  
  theme(height = 150)) /  
  ((data(nutrients) + zone + x(col.food) + y(col.nutrient) +  
  ↪ color(col.amount)) |  
    (data(nutrients) + path * cluster(col.amount, over=col.food) +  
  ↪ y(col.nutrient) +  
    theme(width = 150))))
```

Julia

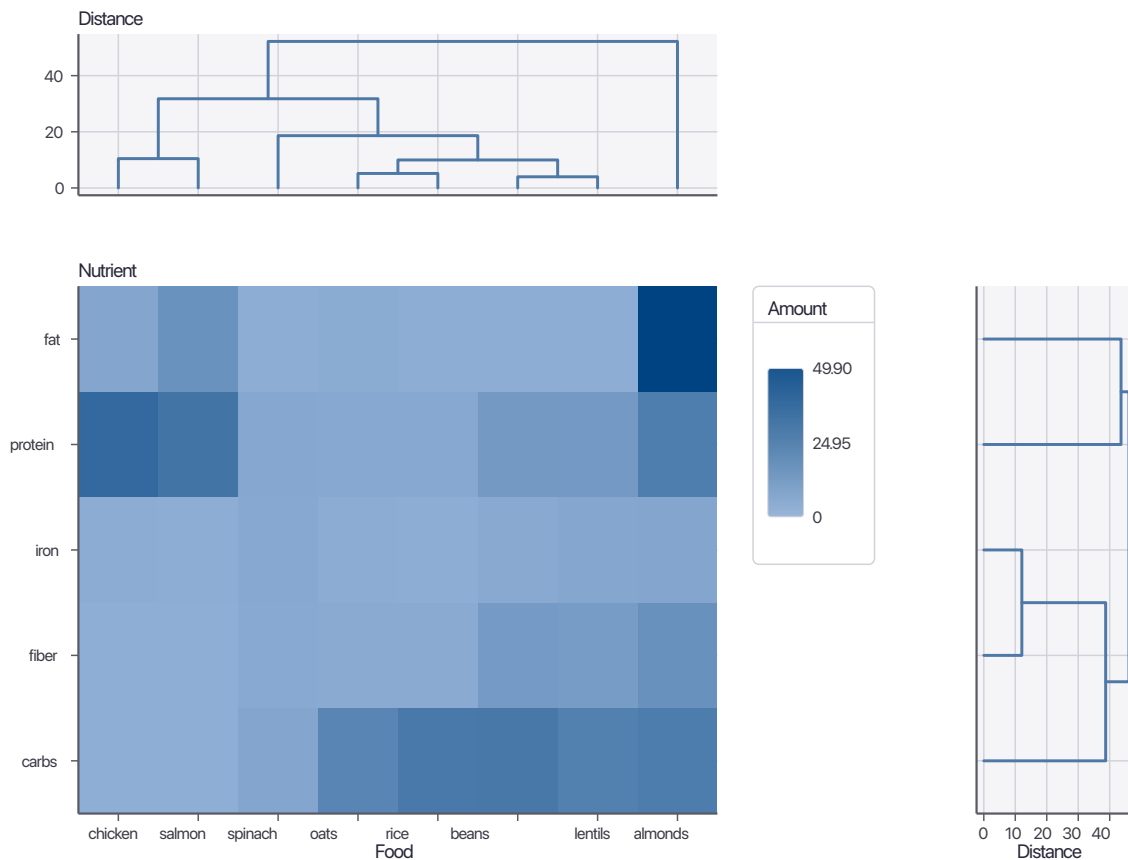
```
(data(nutrients) + path * cluster(:amount, over = :nutrient) + x(:food) +  
  theme(height = 150)) /  
((data(nutrients) + zone + x(:food) + y(:nutrient) + color(:amount)) |  
  (data(nutrients) + path * cluster(:amount, over = :food) +  
  y(:nutrient) + theme(width = 150)))
```

JavaScript

```

below(plot(data(nutrients),
  layer(path, cluster(col.amount, { over: col.nutrient })), x(col.food),
  theme({ height: 150 })),
beside(plot(data(nutrients), zone, x(col.food), y(col.nutrient),
  color(col.amount)),
plot(data(nutrients),
  layer(path, cluster(col.amount, { over: col.food })), y(col.nutrient),
  theme({ width: 150 }))))

```



“Given nutrients: paths derived by cluster on amount, over nutrient, x is food; above zones, x is food, y is nutrient, color by amount, beside paths derived by cluster on amount, over food, y is nutrient.” Owing chapters: [Cluster](#), [Zone](#).

42.10 What this section refuses

Five glyph shapes have no order from smallest to largest, so `shape` refuses a numeric column and names the channels that do carry magnitude:

```
data(gapminder_2007) + point + x(gdp) + y(life) + shape(population)
```

Error:

```
! gog: `shape(population)` maps a continuous (numeric) column, but `shape` on  
  ↳ `point` needs a categorical (text) column. Use `size` or `color` to show  
  ↳ a numeric column.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

43 Over time

How did it change over time? Time is not a chart type in this grammar; it is a column, and you choose which channel carries it. On `x` it makes the familiar time series, and every recipe below puts it there.

43.1 How did it change?

R

```
data(actuals) + line + x(year) + y(sales)
```

Python

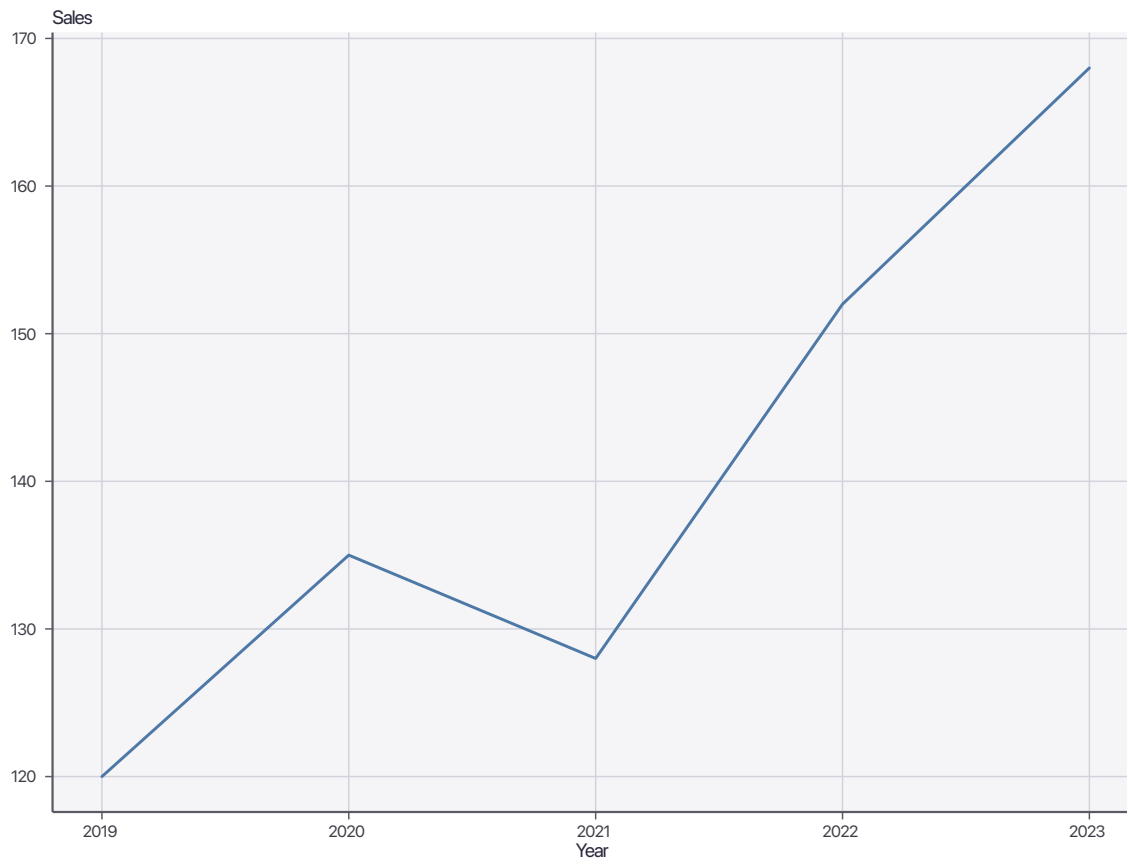
```
data(actuals) + line + x(col.year) + y(col.sales)
```

Julia

```
data(actuals) + line + x(:year) + y(:sales)
```

JavaScript

```
plot(data(actuals), line, x(col.year), y(col.sales))
```



“Given the actuals: a line, x is year, y is sales.” A **line** connects the rows in x order; with years on x , that order is time.

One atom away. If the amount matters more than the rise and fall, swap **line** for **area**: it fills to zero, so the filled area is the amount.

43.2 How did several things change?

A categorical **color** splits the rows into one line per category and colors them at the same time:

R

```
data(gapminder_asia) + line + x(year) + y(life) + color(country)
```

Python

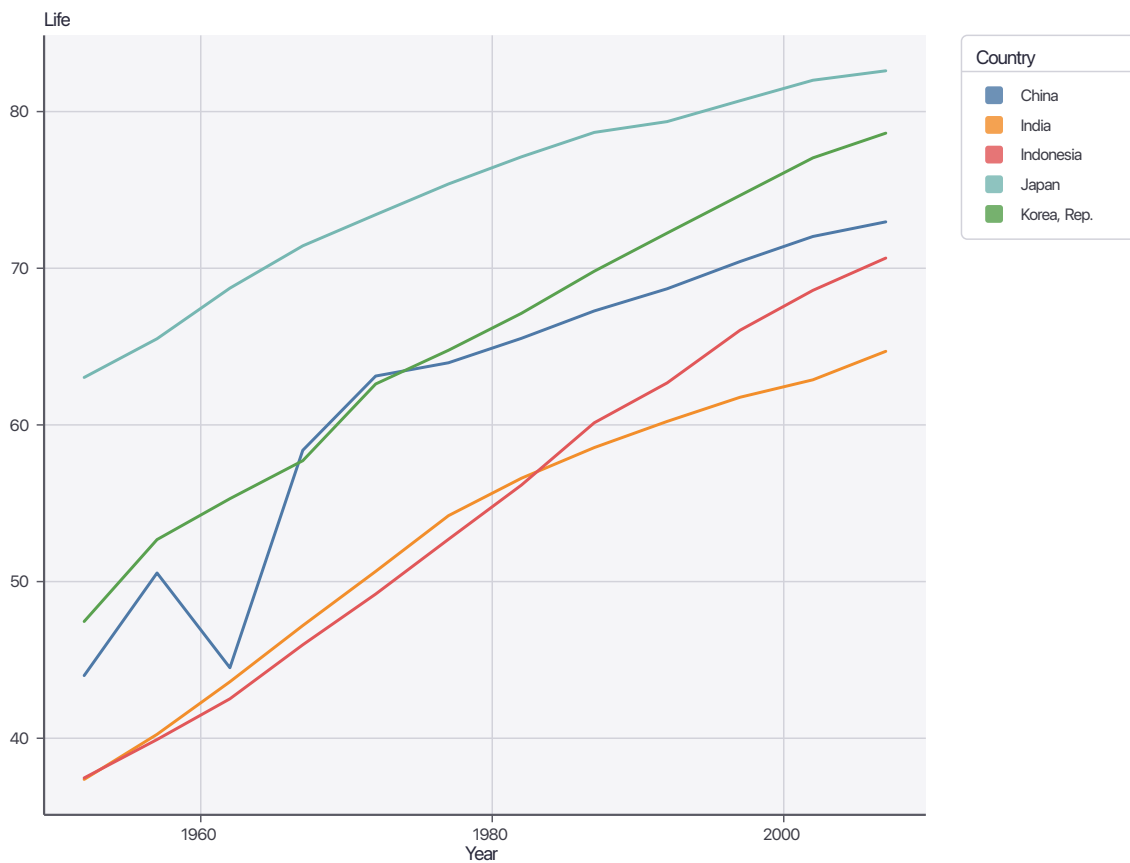
```
data(gapminder_asia) + line + x(col.year) + y(col.life) + color(col.country)
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + color(:country)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
     color(col.country))
```



“Given gapminder Asia: lines, x is year, y is life, color by country.”

One atom away. Too many categories for a legend to stay readable? `group` splits without coloring; see [Many groups](#).

43.3 What about a date column?

The `year` above is a plain number. A `Date` column *is* temporal, so you get a calendar axis without asking for one. Ticks land on calendar boundaries, and weekly steps land on Mondays:

R

```
data(six_weeks) + line + x(day) + y(orders)
```

Python

```
data(six_weeks) + line + x(col.day) + y(col.orders)
```

Julia

```
data(six_weeks) + line + x(:day) + y(:orders)
```

JavaScript

```
plot(data(six_weeks), line, x(col.day), y(col.orders))
```



“Given the six weeks table: a line, x is day, y is orders.”

You never have to write `scale = "time"`: the column’s type already says it. [Scales](#) owns the calendar’s rules: three-month steps that read as quarters, and a midnight tick labeled with the day’s name.

43.4 What happened, and what comes next?

One table holds the history, another holds the expectation. The first `data()` is the plot’s table; a later one applies to the mark written after it.

R

```
data(actuals) + x(year) + y(sales) +
  line +
  data(forecast) + point
```

Python

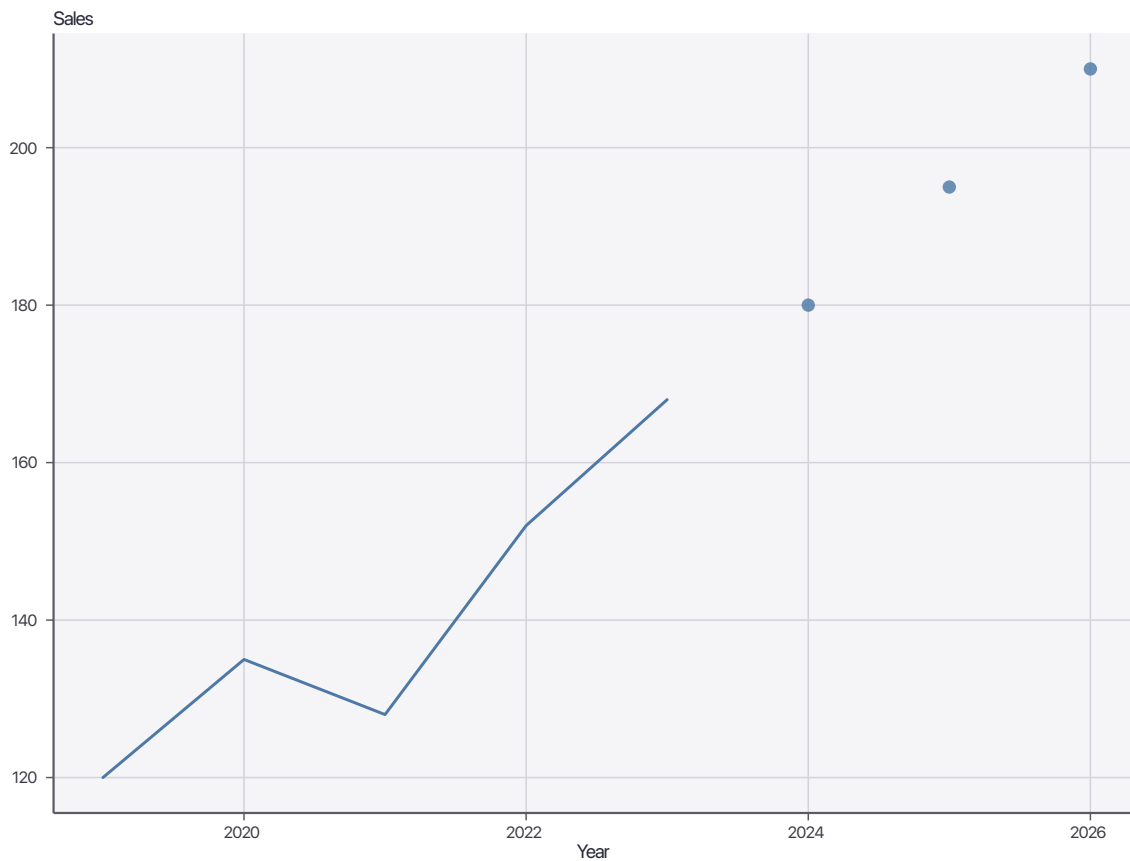
```
(data(actuals) + x(col.year) + y(col.sales) +  
    line +  
    data(forecast) + point)
```

Julia

```
data(actuals) + x(:year) + y(:sales) + line + data(forecast) + point
```

JavaScript

```
plot(data(actuals), x(col.year), y(col.sales), line, data(forecast),  
    point)
```



“Given the actuals: x is year, y is sales, a line; then given the forecast: points.” The line ends at 2023, the dots start at 2024, and each layer reads its own rows. [Data](#) owns multi-table plots.

43.5 What this section refuses

One panel per year is faceting, but a facet variable *names* its panels, and **year** here is a number. The refusal says what to do with a year you mean as a label:

```
data(gapminder_asia) + point + x(gdp) + y(life) | facet(year)
```

Error:

```
! gog: `facet(year)` splits on a number column, but a facet variable names  
  ↳ the panels, so it must be a category column. Make `year` text - in R,  
  ↳ `factor(year)` - or cut it into named groups first.
```

gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.

44 Many groups

How do several groups compare? There are two ways to answer. You can split the marks *inside* one panel, with `color` or `group`. You can split the plot *into* panels, with `|` or `/`. The recipes below use both.

44.1 How do the groups' distributions compare?

The overlaid histogram: `bin` cuts one set of intervals for everyone, so the groups sit on shared edges, each a translucent fill under its own solid outline:

R

```
data(iris_flowers) + bar * bin + x(petal_length) + color(species)
```

Python

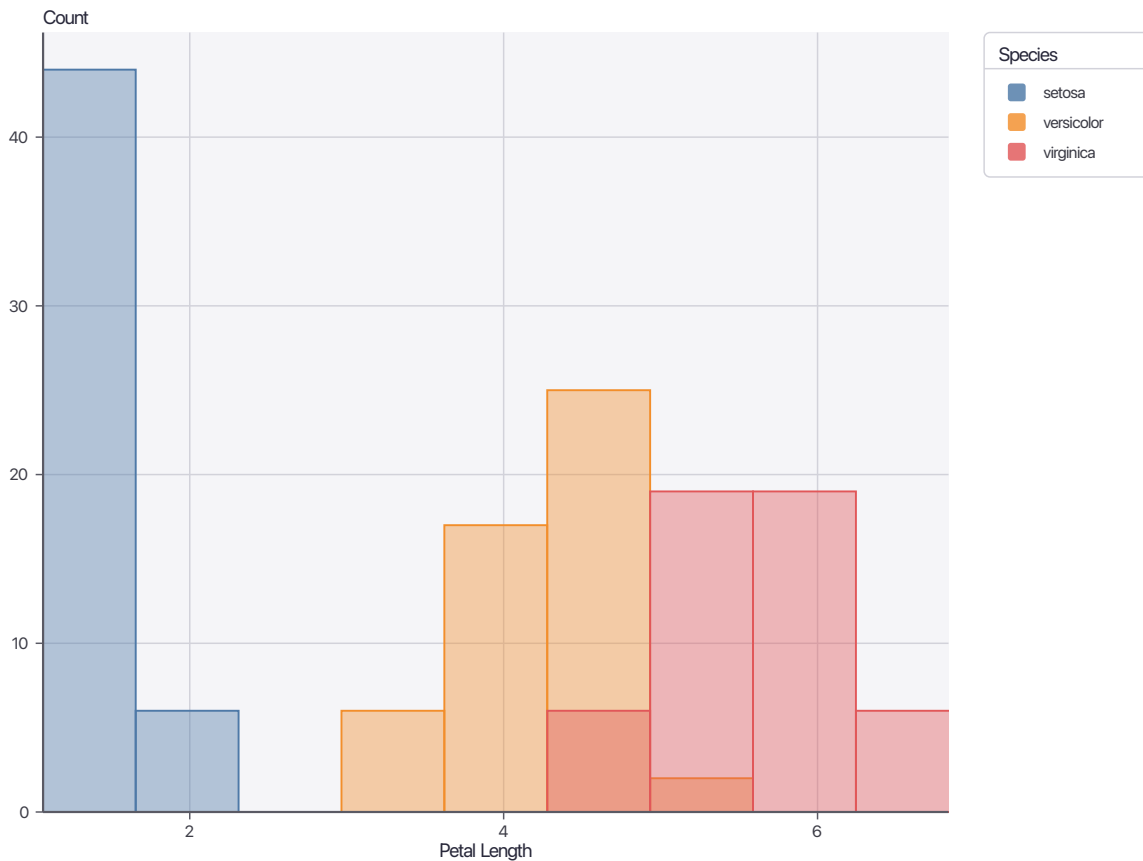
```
data(iris_flowers) + bar * bin + x(col.petal_length) + color(col.species)
```

Julia

```
data(iris_flowers) + bar * bin + x(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), layer(bar, bin), x(col.petal_length),  
     color(col.species))
```



“Given the iris flowers: bars derived by bin, x is petal length, color by species.” The outline is what keeps each species’ silhouette readable where the fills overlap. Owing chapters: [Bar](#), [Transforms](#), [Channels](#).

44.2 What if the overlap gets worse?

Drop the fills entirely: `step * bin` traces each group’s silhouette as an unfilled staircase, so nothing can hide anything:

R

```
data(iris_flowers) + step * bin + x(petal_length) + color(species)
```

Python

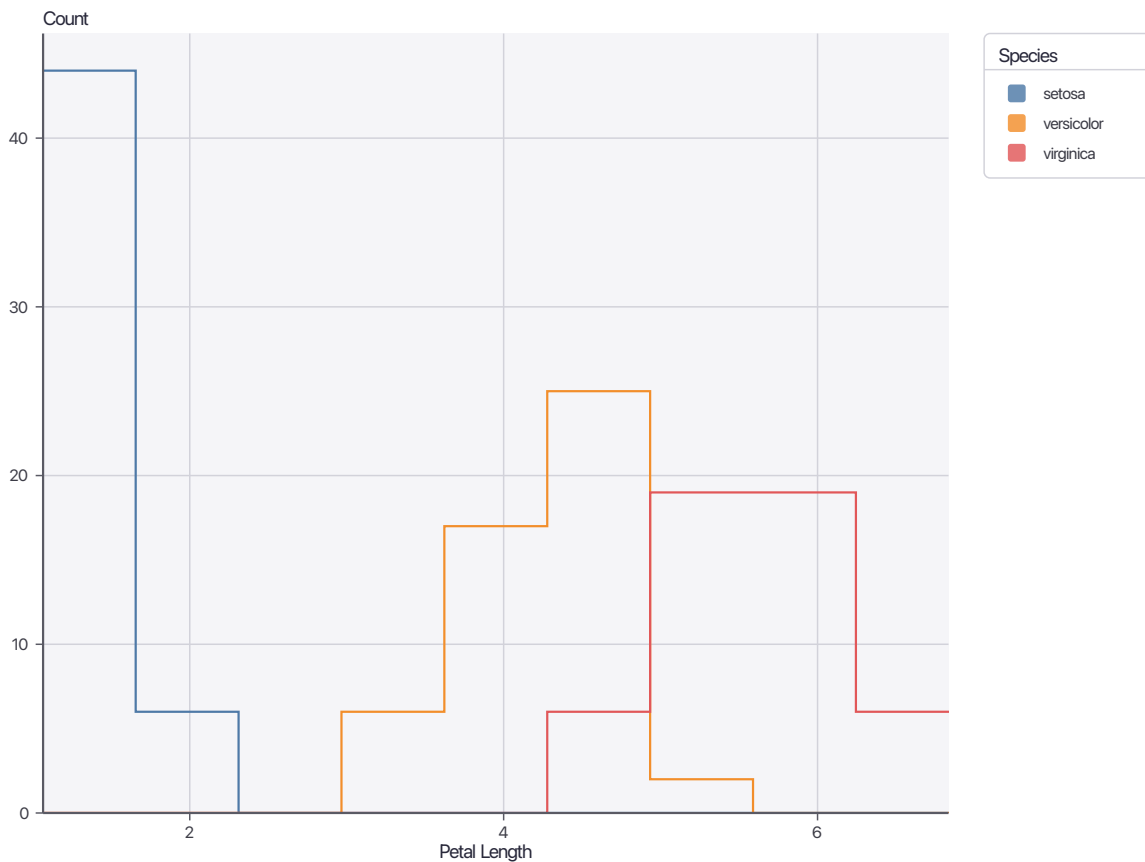
```
data(iris_flowers) + step * bin + x(col.petal_length) + color(col.species)
```

Julia

```
data(iris_flowers) + step * bin + x(:petal_length) + color(:species)
```

JavaScript

```
plot(data(iris_flowers), layer(step, bin), x(col.petal_length),  
color(col.species))
```



“Given the iris flowers: step outlines derived by bin, x is petal length, color by species.”

One mark swapped, nothing else. [Step](#) owns the staircase.

44.3 Grouping without a legend

Sometimes the grouping matters and the identities do not: you want the *shape* of many series, not a legend naming each. `group` splits without coloring:

R

```
data(gapminder_asia) + line + x(year) + y(life) + group(country)
```

Python

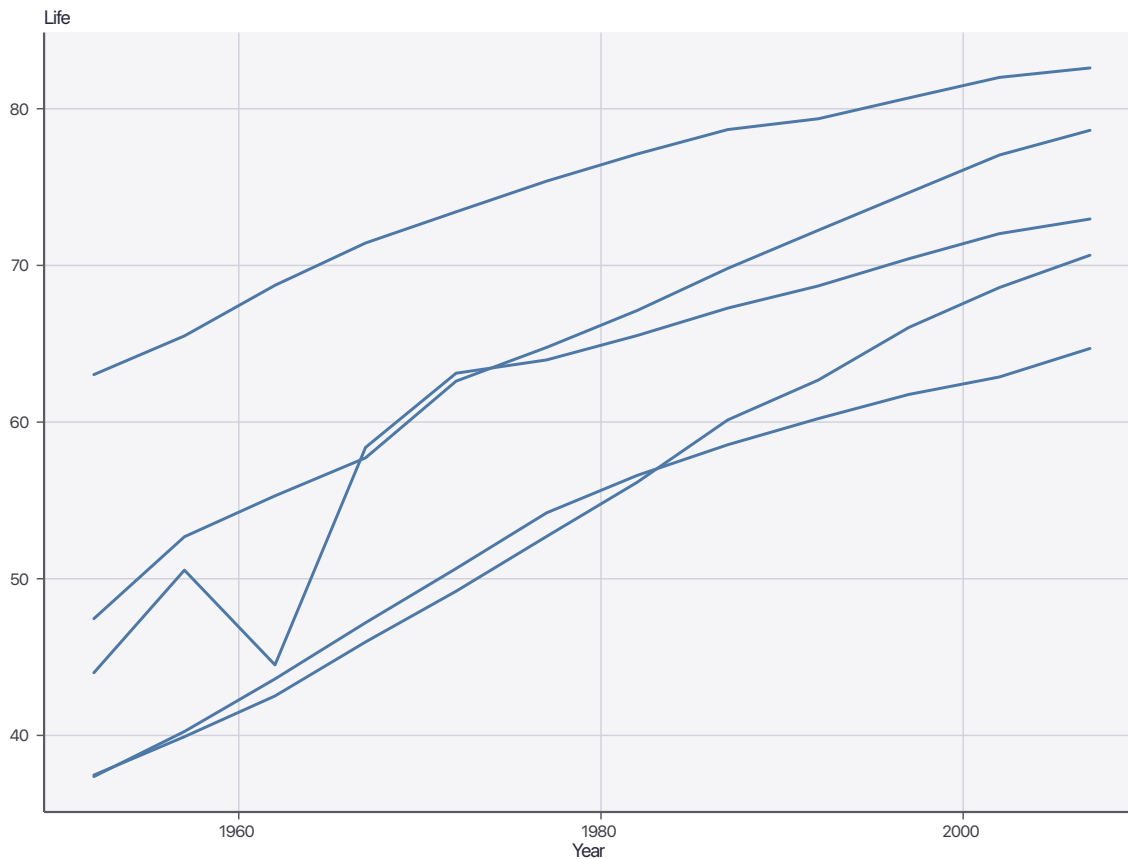
```
data(gapminder_asia) + line + x(col.year) + y(col.life) + group(col.country)
```

Julia

```
data(gapminder_asia) + line + x(:year) + y(:life) + group(:country)
```

JavaScript

```
plot(data(gapminder_asia), line, x(col.year), y(col.life),  
      group(col.country))
```

“Given *gapminder Asia*: lines, *x* is year, *y* is life, grouped by country.”

Five countries, five lines, no legend. The shape the five lines make together is the answer. Bind **color** instead when the reader needs to know which is which.

44.4 One panel per group

When each group needs its own pair of axes, split the plot into panels instead of splitting the marks. Statistics run *within* each panel; the scale stays shared so panels stay comparable:

R

```
data(gapminder_2007) + bar * bin + x(life) | facet(continent)
```

Python

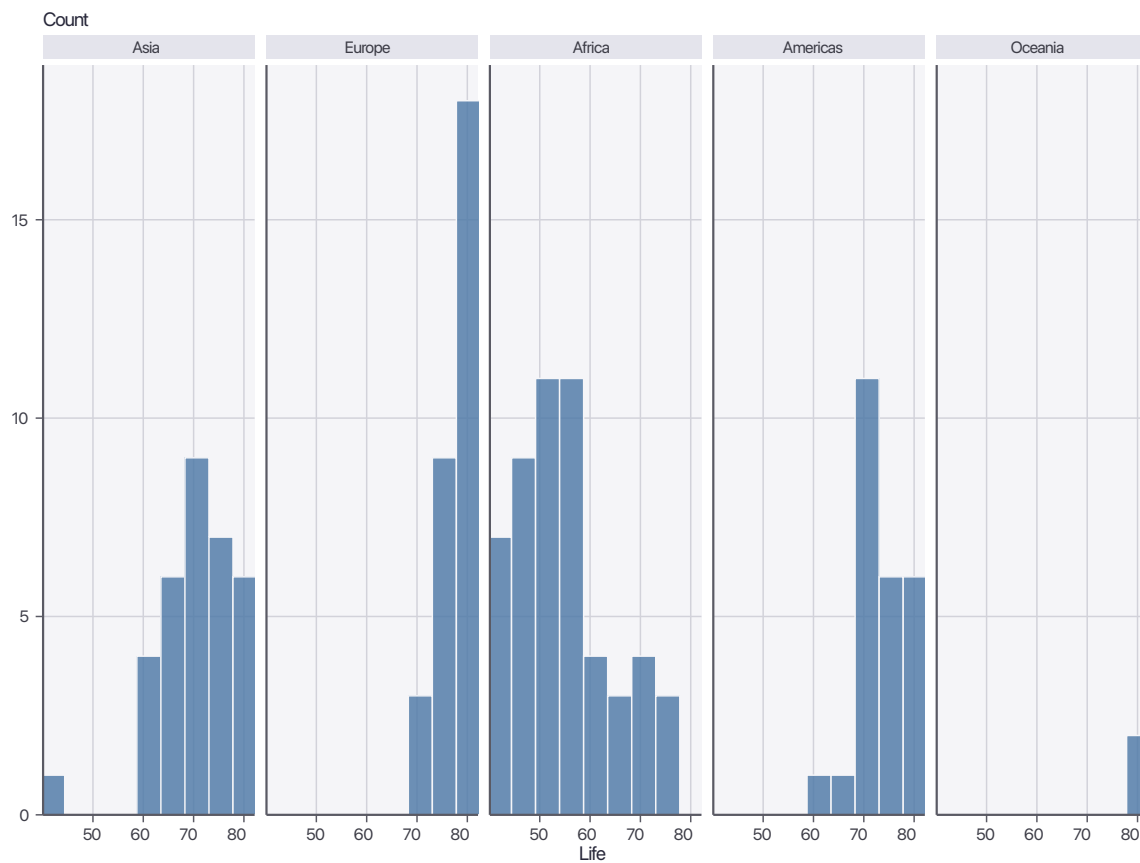
```
data(gapminder_2007) + bar * bin + x(col.life) | facet(col.continent)
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life),  
  across(col.continent))
```



“Given gapminder 2007: bars derived by bin, x is life, split into panel columns by continent.”

Five distributions on one common axis, cut into one common set of bins and counted continent by continent. Compare the panels one at a time. Africa’s bars sit at the low end of the axis.

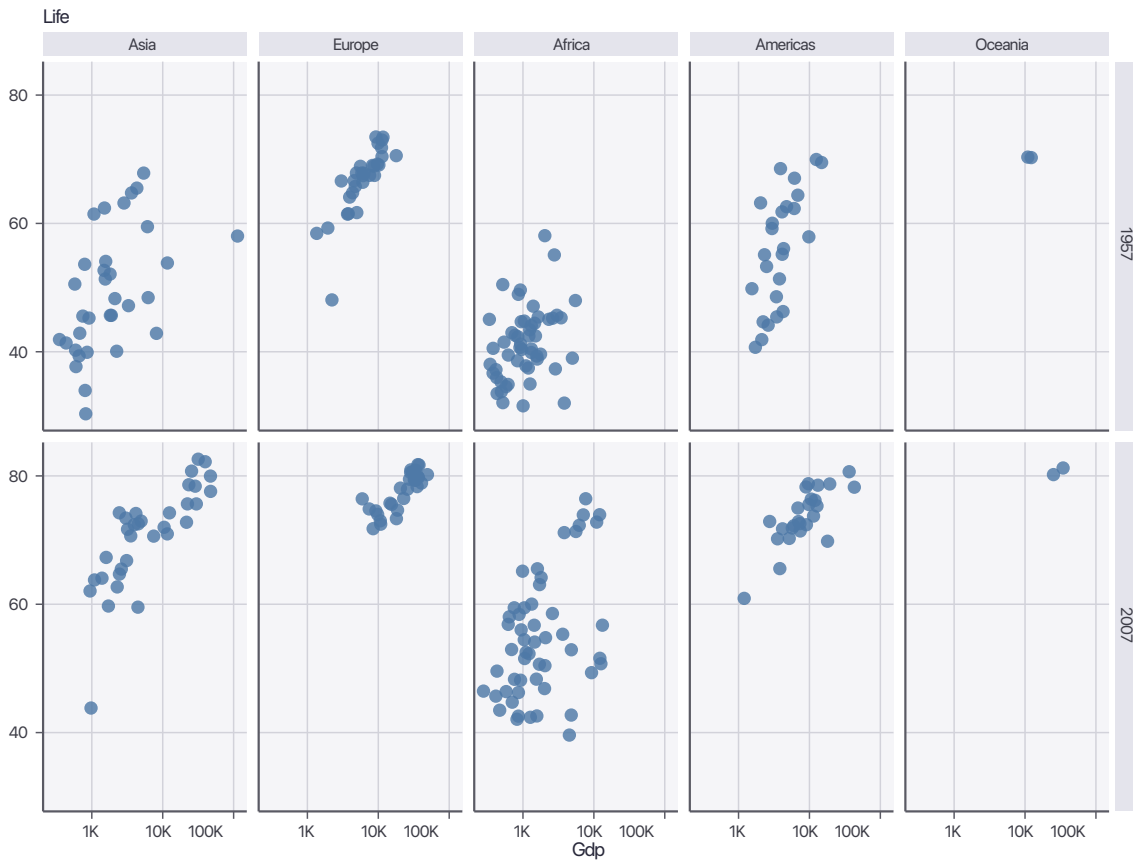
Europe's bars sit at the high end. Bins and the count axis are shared, so equal heights mean equal counts. [Faceting](#) owns the rules, including why the cut is shared when the count is not.

44.5 Two grouping variables at once

| splits into panel columns, / into panel rows; written together they cross into a grid. Every combination gets a panel. The two years are picked out in R, because subsetting a table is the host language's job, and the other three bindings do it their own way:

```
gm_years <- gm_all[gm_all$year %in% c(1957, 2007), ]
gm_years$period <- as.character(gm_years$year)

data(gm_years) + point + x(gdp, scale = "log") + y(life) |
  facet(continent) / facet(period)
```



“Given the gm years table: points, x is gdp on a log scale, y is life, split into panel columns by continent and panel rows by period.”

Fifty years of change, read down each continent’s column. `year` is a number, and a facet variable has to name its panels, so `period` is `year` written as text. That is what the [Over time](#) refusal is about.

44.6 Which group is biggest?

A comparison is easiest to read when it is sorted, and a text column carries no order of its own. `order()` is how you give it one:

R

```
data(medals) + bar + x(country) + y(gold) + order(gold, desc = TRUE)
```

Python

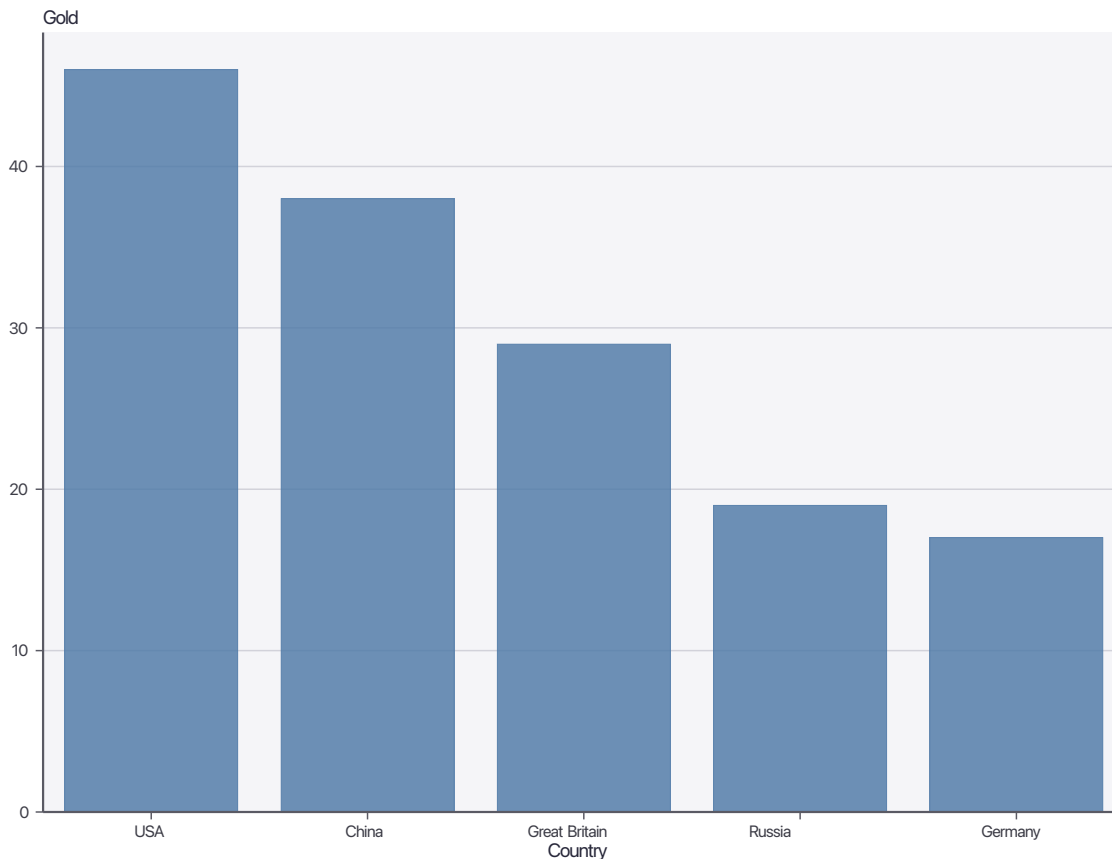
```
data(medals) + bar + x(col.country) + y(col.gold) + order(col.gold, desc =  
↪ True)
```

Julia

```
data(medals) + bar + x(:country) + y(:gold) + order(:gold, desc = true)
```

JavaScript

```
plot(data(medals), bar, x(col.country), y(col.gold),  
order(col.gold, { desc: true })))
```



“Given the medals: bars, x is country, y is gold, ordered by gold, largest first.”

One atom away. `desc = FALSE`, the default, sorts smallest first. `order()` names **gold** here, not the category column **country**: that is how a category is sorted by a value. It overrides a [declared order](#) as well.

44.7 Two groups, mirrored

A **population pyramid** is a named chart type, and nothing in the kernel knows its name. It follows from rules that were already there, for other reasons.

The **census** table holds 36 rows: an **age** band, a **city** (Busan or Seoul), and a **population** for each of 18 bands per city. It is invented data, shaped like a census.

Table 44.1: census: first 5 of 36 rows

age	city	population
0	Busan	5200
5	Busan	3100
10	Busan	2600
15	Busan	9000
20	Busan	38500

The mirror comes from a minus sign: one city's populations are negated so its bars grow the other way. That is arithmetic, so it happens in the table, before the sentence:

```
mirrored <- census
mirrored$population <- ifelse(mirrored$city == "Busan",
                             -mirrored$population, mirrored$population)
```

Table 44.2: mirrored: first 5 of 36 rows

age	city	population
0	Busan	-5200
5	Busan	-3100
10	Busan	-2600
15	Busan	-9000
20	Busan	-38500

The arithmetic is written in R, because it is R's job, and the other three bindings negate a column the same way. What follows is the sentence, and it has four spellings like every other:

R

```
data(mirrored) + bar + y(age) + x(population) + color(city) +
  order(age, desc = TRUE)
```

Python

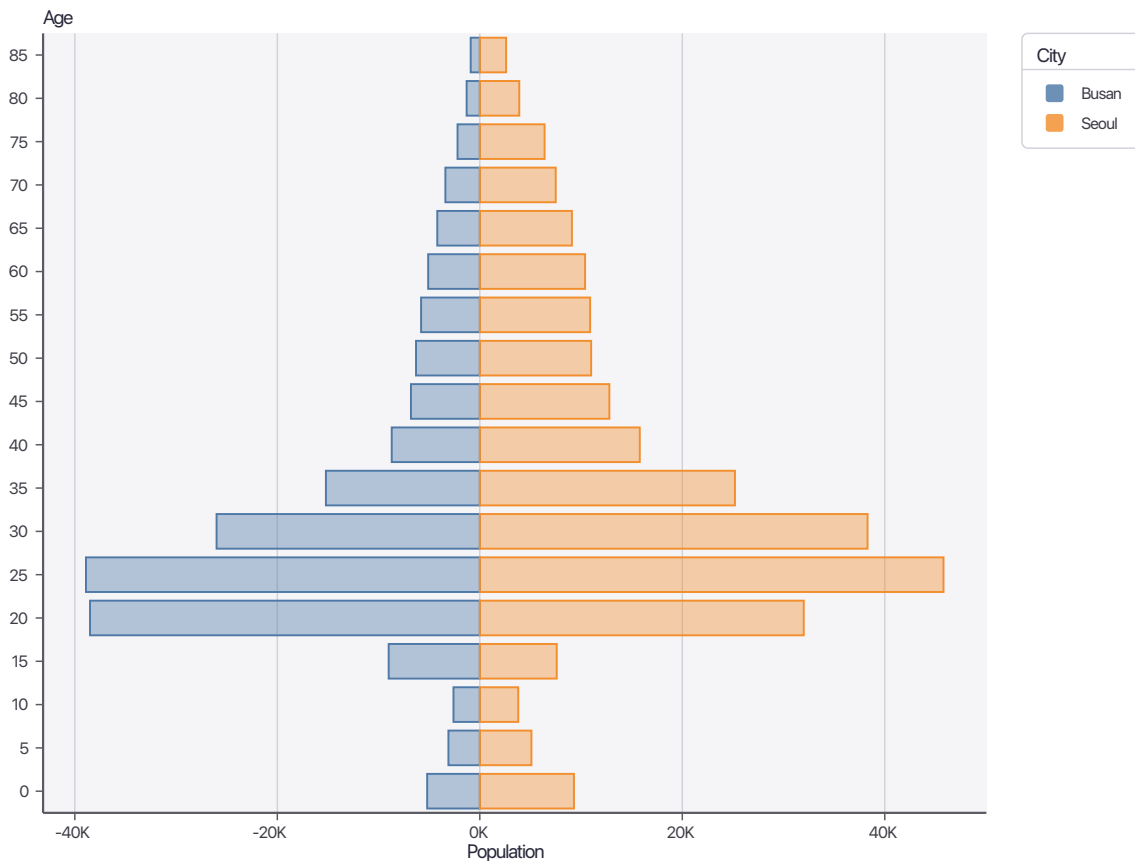
```
(data(mirrored) + bar + y(col.age) + x(col.population) + color(col.city) +
  order(col.age, desc = True))
```

Julia

```
data(mirrored) + bar + y(:age) + x(:population) + color(:city) +
  order(:age, desc = true)
```

JavaScript

```
plot(data(mirrored), bar, y(col.age), x(col.population), color(col.city),
  order(col.age, { desc: true })))
```



“Given the mirrored table: bars, y is age, x is population, color by city, ordered by age, largest first.”

No new atom, and none was needed. The pyramid comes from three rules the grammar already states. Orientation is read off the bindings, so a category on **y** lays the bars down. There is no **flip** atom, because that would be a second way to say one thing. A bar runs from the zero baseline to its value, in whichever direction that value lies. A negative bar is not a special case.

It is the same rectangle, pointing the other way. And `stack` is something you ask for rather than something that happens, so two bars sharing one slot sit back to back instead of piling up.

`order(age, desc = TRUE)` is the atom from the section above, doing exactly what it did there. `gog` puts the first category at the top of a horizontal chart, so age 0 would sit at the top. A pyramid needs its youngest band at the bottom, and `desc = TRUE` turns the list over.

Now look at the left half of the population axis. It reads -40K, not 40K.

This chart is often drawn with the tick text replaced by its absolute value, which hides the minus sign the bars were placed with. `gog` keeps the coordinates it drew, and this is the clearest case for why. Relabeled, the axis carries two different ticks both reading 40K, and only a reader who knows the convention can see which is which. That reader did not need the help. The reader who did need it is now given a plot that disagrees with its own axis. `gog` has no way to set your own tick text, and [Coverage](#) lists it among the capabilities with no decision recorded. The paragraph above is the case against adding one.

44.8 What this section refuses

A bar is a rectangle; it has no glyph to vary, so `shape` on a `bar` is refused outright. The table of what each mark accepts is one table, with no per-mark exceptions:

```
data(medals) + bar + x(country) + y(gold) + shape(country)
```

Error:

```
! gog: `shape` cannot be bound to `bar` - a bar has no shape feature. Remove  
  ↳ the `country` mapping from `shape`, or use a mark that has one.
```

```
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```


Part V

The reasons

The most remarkable chapter of the 1446 book is the one that explains *why*: why $\neg$ (*g*) is shaped like the back of the tongue blocking the throat, why the vowels build from three primitives. That chapter was lost for five hundred years, so the world had the letters but not the reasons. A surviving copy came to light in 1940. The letters worked the whole time. But a system whose reasons are lost cannot be extended, only used.

This part is *gog*'s reasons, published with the package on purpose. It states the nine laws the package never breaks, and the one shortcut they all exist to refuse. It says why the engine refuses what it refuses, and why every refusal comes with a direction. None of it is required to *use gog*. The earlier parts cite these laws by name, and never ask you to have read them. It is here for the moment you want to know why the grammar is shaped the way it is. It also predicts what the engine will do in a case no chapter covers.

The laws all refuse the same thing, and it has a name: the expert's shortcut, the saved keystroke that takes a rule away from the beginner.

45 Design laws

`gog` is governed by nine laws. They are not preferences or style guides; they are rules the package *never* breaks.

Every new feature, every argument, every naming decision is checked against these rules. If a proposed addition breaks one, it is rejected.

Laws 1–5 govern the daily grammar you write. Laws 6–8 take the Hangeul (한글) analysis furthest, and govern the grammar’s structure: what composes, what renders, and what the engine may refuse. Law 9 says what a specification is, and what it must never contain.

45.1 Law 1: Orthogonality

Every compatible atom combines with every other atom. No atom is redundant.

If `color` works with `point`, it works with `line`, `bar`, `area`, and every other mark. Where a channel makes sense for a mark, it is there. `color` is never withheld from one mark and given to its neighbor.

The rule that follows: if you want a mark-specific version of a channel (`point_color` vs `bar_color`), that is a violation. The channel must be generalized, not specialized.

45.2 Law 2: No Exceptions

A transform behaves identically on every mark. > No per-mark special cases.

`bin` cuts the same buckets and counts the same rows whichever mark reads it. Change only the mark, and what changes is the geometry:

R

```
((data(gapminder_2007) + bar * bin + x(life)) |  
  (data(gapminder_2007) + line * bin + x(life))) /  
((data(gapminder_2007) + area * bin + x(life)) |  
  (data(gapminder_2007) + step * bin + x(life)))
```

Python

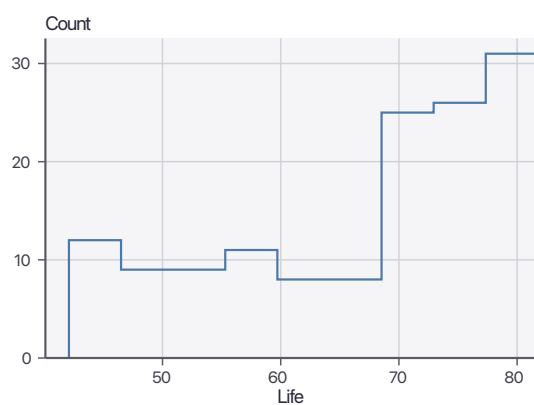
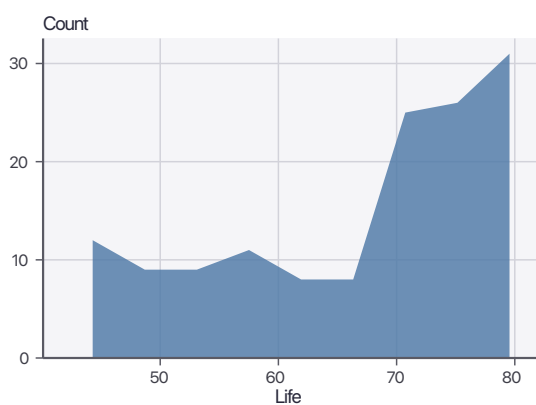
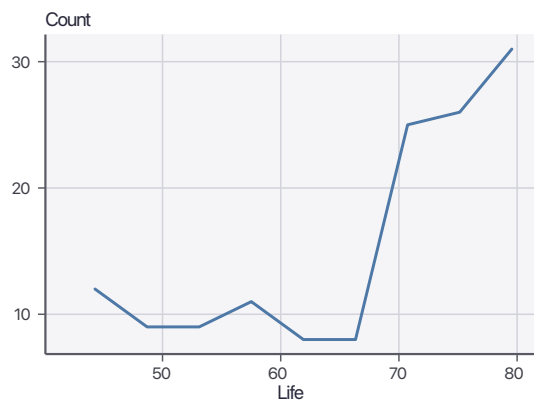
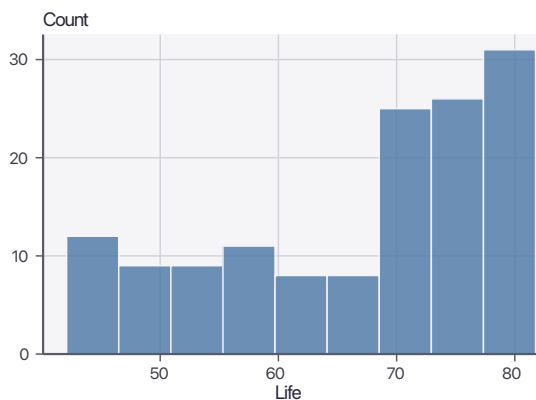
```
((data(gapminder_2007) + bar * bin + x(col.life)) |  
 (data(gapminder_2007) + line * bin + x(col.life))) /  
((data(gapminder_2007) + area * bin + x(col.life)) |  
 (data(gapminder_2007) + step * bin + x(col.life)))
```

Julia

```
((data(gapminder_2007) + bar * bin + x(:life)) |  
 (data(gapminder_2007) + line * bin + x(:life))) /  
((data(gapminder_2007) + area * bin + x(:life)) |  
 (data(gapminder_2007) + step * bin + x(:life)))
```

JavaScript

```
below(beside(plot(data(gapminder_2007), layer(bar, bin), x(col.life)),  
 plot(data(gapminder_2007), layer(line, bin), x(col.life))),  
 beside(plot(data(gapminder_2007), layer(area, bin), x(col.life)),  
 plot(data(gapminder_2007), layer(step, bin), x(col.life))))
```



“Given gapminder 2007: bars derived by bin, x is life, beside a line derived by bin, x is life; above an area derived by bin, x is life, beside a step outline derived by bin, x is life.”

One cut and one set of counts, four times over: rectangles, a line through the bin centers, that same line filled, and the counts as a staircase. Read any bar’s height, then read the same bin in the panel beside it: the number is the same. The transform never asks which mark will draw its result.

This is the hardest law to keep as the package grows. Every user request that begins “but for maps I need...” or “histograms are special because...” is a pressure to break it. What keeps it is that the permitted bindings sit in *one table* inside the engine, covering every mark $\times$ channel pair. A mark cannot leave a channel out, because there is no place to leave it out.

45.3 Law 3: Plain Names

Every taught name is a common English word. No acronyms. No abbreviations. Two words maximum, joined by `_`. Only `x`, `y`, `z` are literal exceptions.

The name you may know	The plain name here
<code>alpha</code>	<code>opacity</code>
<code>loess, stat_smooth</code>	<code>smooth</code>
<code>coord_polar</code>	<code>polar</code>
<code>geom_point, aes</code>	<code>point</code> , and channels called by their plain names: <code>x()</code> , <code>color()</code>

`color_by, size_by | color, size |`

If a new user has to look up what a name means, it fails this law.

The last row is the one where a word was *removed* rather than replaced, and it repays a second reading because the word is not lost. A plotting library could reasonably write `color_by(species)` and `size_by(population)`. The suffix marks the argument as a column, which says nothing here, because every atom that takes an argument takes a column. Now read `color(continent)` aloud. It is “*color by continent*”. The word *by* is already in how you read the sentence, so writing it into the name says it twice. That is a silent letter by Law 2’s measure, and every channel would have had to carry one.

45.3.1 One spelling of English

`gog` writes American English everywhere: `color`, `center`, `gray`, and every word of the kernel. There is no British alternative. `ggplot2` takes both spellings, so `scale_colour_manual()` and `scale_color_manual()` are one function there; here there is one spelling and no second one to learn.

Two spellings for one word is a silent letter, and Law 2 refuses silent letters in the vocabulary as well as in the grammar. The cost is invisible to the person who already knows both forms, and everyone else pays it. A reader of your code must recognize two shapes for one idea, a search for `color(` finds only half the code, and every document that lists the vocabulary lists it twice.

Typing `colour` is the likeliest way to meet this law, so the refusal names the word to write:

```
data(gapminder_2007) + point + x(gdp) + y(life) + style(colour = "tomato")
```

Error:

```
! gog: `style(colour = )` is not a setting. gog spells it `color`: American  
  ↪ English is the grammar's only spelling, and unlike ggplot2 there is no  
  ↪ British alternative.
```

That is the setting. The channel is refused the same way, and `colour` exists in the package for no other purpose: it is not a channel, it is a word whose only job is to name the channel you meant.

```
data(gapminder_2007) + point + x(gdp) + y(life) + colour(continent)
```

Error:

```
! gog: there is no `colour()` channel. gog spells it `color(<column>)`:  
  ↪ American English is the grammar's only spelling, and unlike ggplot2 there  
  ↪ is no British alternative.
```

The same holds for `border_colour` and `centre`. Everywhere else in the grammar, the American spelling is simply the only one there has ever been.

45.4 Law 4: Bind Once

A table is named once. Columns are then bare names. The nearest `data()` wins.

R

```
# Correct: table named once, columns bare  
data(gapminder_2007) + point + x(gdp) + y(life)
```

Python

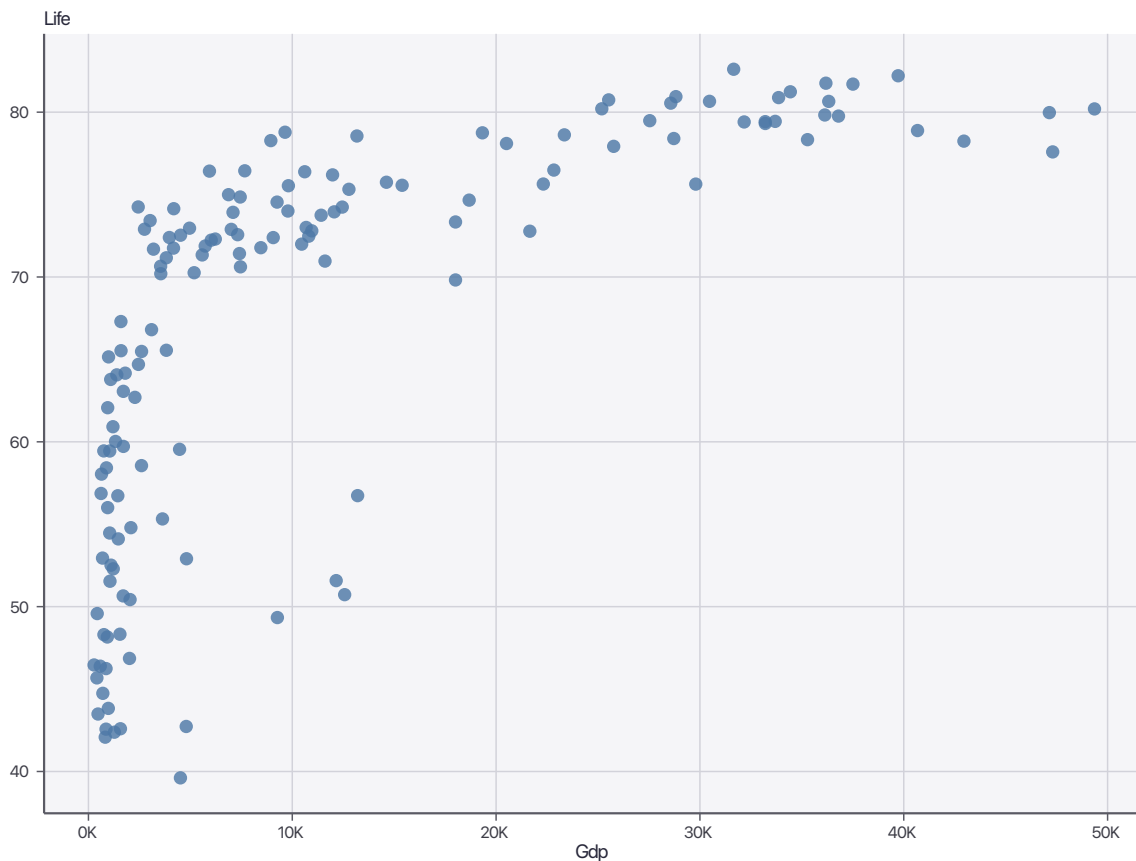
```
data(gapminder_2007) + point + x(col.gdp) + y(col.life)
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life))
```



“Given gapminder 2007: points, x is gdp, y is life.”

Quoting a column name is a habit brought from other plotting libraries rather than a typo, so gog says which fix it wants:

```
data(gapminder_2007) + point + x("gdp") + y("life")
```

Error:

```
! gog: `x("gdp")` binds a *value*, and a channel takes a *column*: `x(gdp)`  
↪ maps the column called `gdp`.
```

This law makes a plot specification readable as an English sentence. It also lets your session offer column names as you type, and lets a mistake be refused while you are still writing rather than when the plot is drawn.

45.5 Law 5: Explicit Over Implicit

Short is better than long, unless short is ambiguous. Then say it out loud.

Implicit behavior saves typing now and costs clarity later. `gog` chooses the other way: when brevity would leave a reader guessing what the engine decided, the engine makes you say it.

This law is also what the error messages are for. A refusal must tell you *what `gog` assumed* or *what `gog` cannot do*, never merely that something went wrong.

45.5.1 Setting is not mapping

The tempting short spelling for a constant color is `color("red")`, reusing the channel and letting the quotes decide the meaning. `gog` does not reuse the channel. Quoting a column name is a habit rather than a mistake, so `color("continent")` would be genuinely ambiguous: a column, or a color named “continent”. Deciding by “*is it a column in this data?*” would make the same expression mean different things against different tables.

So `gog` says it out loud. Mapping is a channel; setting is `style()`:

R

```
data(gapminder_2007) + point + x(gdp) + y(life) +  
  style(color = "seagreen", opacity = 0.5)
```

Python

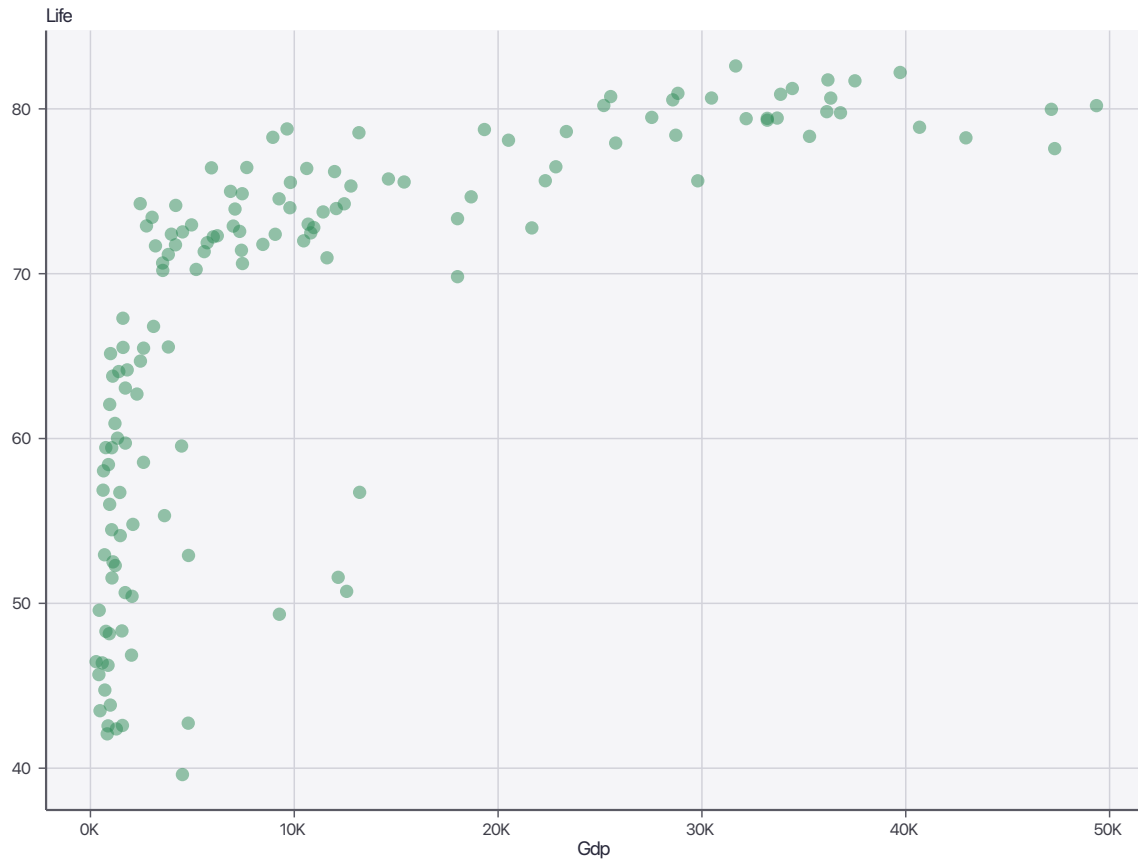
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  style(color = "seagreen", opacity = 0.5))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) +  
  style(color = "seagreen", opacity = 0.5)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  style({ color: "seagreen", opacity: 0.5 })))
```



“Given gapminder 2007: points, x is gdp, y is life, colored seagreen, with opacity 0.5.”

This also gives the two operations the different rules they deserve. A mapping earns a guide and a setting does not, and `line`, which cannot *map* `opacity` because one stroke has one opacity, can still *set* it.

`ggplot2` puts the same boundary in a different place. There the boundary is a pair of parentheses: `color = "red"` inside `aes()` maps, and the same words outside it set. `gog` moves that boundary from a punctuation mark to a word, so the atom you write says which one you meant. The parentheses have one failure that a word cannot have. Writing `aes(color = "red")` is legal, so it maps every row to a category with a single level. You get a legend with one key labeled “red”, and points drawn in the palette’s first color rather than in red.

45.5.2 A missing name must not become a default

A palette name that matches nothing could be answered two ways: refuse it, or quietly draw in the default colors. `gog` refuses, and the message lists every name it would have accepted:

```
data(gapminder_2007) + point + x(gdp) + y(life) +
  color(continent) + palette("red")
```

Error:

```
! gog: `palette("red")` is not a known palette. Named palettes are gog,
  ↳ okabe, soft for categories; blue, viridis, magma, inferno, plasma,
  ↳ cividis, gray for numbers; blue_red, brown_tea for numbers that diverge
  ↳ from a center. `red` is a color, not a palette. To paint every mark one
  ↳ color use `style(color = "red")`; to give each category its own color
  ↳ pass a vector: `palette(c("red", ...))`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

A default is the quiet way software breaks this law: the plot still appears, so nothing looks wrong, and the person reading the plot is the one misled.

45.5.3 Illegal is not the same as unsupported

gog separates two refusals, because they ask different things of you:

Refusal	Meaning	What to do
Illegal	The grammar forbids it. It will never work.	Rewrite the expression.
Unsupported	The grammar allows it; this engine does not draw it.	Use another atom that renders.

Calling a valid combination “illegal” would teach you a rule that does not exist. So gog says which it is.

```
# Illegal: size encodes magnitude, and a category has none
data(gapminder_2007) + point + x(gdp) + y(life) + size(continent)
```

Error:

```
! gog: `size(continent)` maps a categorical (text) column, but `size` on
  ↳ `point` needs a continuous (numeric) column. Use `color`, `shape`, or
  ↳ `pattern` to distinguish categories.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

```
# Illegal: a bar has no size feature at all; its extent is y
data(medals) + bar + x(country) + y(gold) + size(gold)
```

Error:

```
! gog: `size` cannot be bound to `bar` - a bar has no size feature. Remove
  ↳ the `gold` mapping from `size`, or use a mark that has one.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

```
# Illegal: `z` stands a `point`, `path` or `bar` up into the cube (see
  ↳ Space),
# but a `line` is read along its x axis, and a cube has no left to right
data(medals) + line + x(country) + y(gold) + z(silver)
```

Error:

```
! gog: `line` reads a *domain* left to right - it sorts by `x` and draws one
  ↳ value for each - and a cube has no left to right: `x` is one of three
  ↳ equal positions, and at some viewing angles it runs into the page and
  ↳ becomes depth. A `line` in space would be sorted by an axis the reader
  ↳ cannot see, so this is refused rather than drawn. For a route through
  ↳ three dimensions use `path`, which is `line` with that sort removed:
  ↳ `path + x(<a>) + y(<b>) + z(silver)`.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

That third one is worth reading closely, because it is the kind that is easy to mistake for a missing feature. A 3-D line is not unbuilt. It is ruled out, and the refusal says why: a `line` sorts by `x` and draws one value for each. A cube has no left to right, so there is no axis for that sort to run along. The direction it gives is `path`, which is `line` with that sort removed, and it does draw in space.

An Unsupported refusal looks different. Here is one:

```
# Unsupported: `z` is valid grammar for `text`; this engine has not built it
data(medals) + text + x(country) + y(gold) + z(silver) + label(country)
```

Error:

```
! gog: `z` is valid grammar for `text`, but this engine does not draw it yet
  ↳ - `z(silver)` would have no visual effect. Remove it, or use a channel
  ↳ that renders.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

In every case `gog` *refuses to render*. It does not quietly drop the channel and hand you a plot that looks finished. A silently ignored binding is a silent letter in software: written, and with no effect on what comes out.

The four refusals above are *live*: the engine prints them as this page builds. The last one is a gap the engine can close; the first three are rulings that never close. That is the whole distinction. A check re-runs every refusal in this book and fails if one has started drawing instead.

45.5.4 Drawing it anyway

Every refusal the engine makes ends with the same offer: fix it, *or set `GOG_STRICT=0` to draw anyway*. That switch turns refusals into warnings. You get the same message, in the same words, and you get the plot underneath it.

Set it on a line of its own, before the sentence you want drawn:

Language	Set it with
R	<code>Sys.setenv(GOG_STRICT = 0)</code>
Python	<code>os.environ["GOG_STRICT"] = "0"</code>
Julia	<code>ENV["GOG_STRICT"] = "0"</code>
JavaScript	<code>process.env.GOG_STRICT = "0"</code>
Shell	<code>export GOG_STRICT=0</code>

Only 0 works. `GOG_STRICT=false` and `GOG_STRICT=no` still refuse, because a switch this simple is better with one spelling than with a list of words to guess at.

It then stays on for the rest of your session, which is the part worth remembering: every later refusal is a warning too, and readers stop reading warnings. Clear the variable again once you have seen the plot you wanted.

You get the plot with the refused part left out, so `bar + size(gold)` draws the bars and no size.

The switch cannot reach a refusal your binding makes before the engine is asked, because there is no half-drawn plot to offer. Your binding catches a quoted column name, a British spelling, a sentence that does not begin with `data()`, and a malformed atom such as `box(whiskers = "middle")`.

45.6 Law 6: Compositional Invariance

A composed sub-expression means the same thing in every context. No enclosing expression may silently reinterpret an inner one.

The Korean syllable `하` (*ha*) is pronounced identically in `하늘` (*haneul*, sky), `하루` (*haru*, a day), and `하지만` (*hajiman*, but). Its reusability comes from stability, not from meaning. English composition is not stable: “ough” is a different sound in *though*, *through*, *thought*, *tough*, *cough*, and *bough*. A `gog` expression must be `하`, never “ough”: layering, faceting, or animating a plot may add to it, but may not change what the existing part meant.

You can see the law by using one sub-expression in two sentences. `bar * bin` is one sub-expression: a mark and its transform, composed once.

R

```
data(gapminder_2007) + bar * bin + x(life)
```

Python

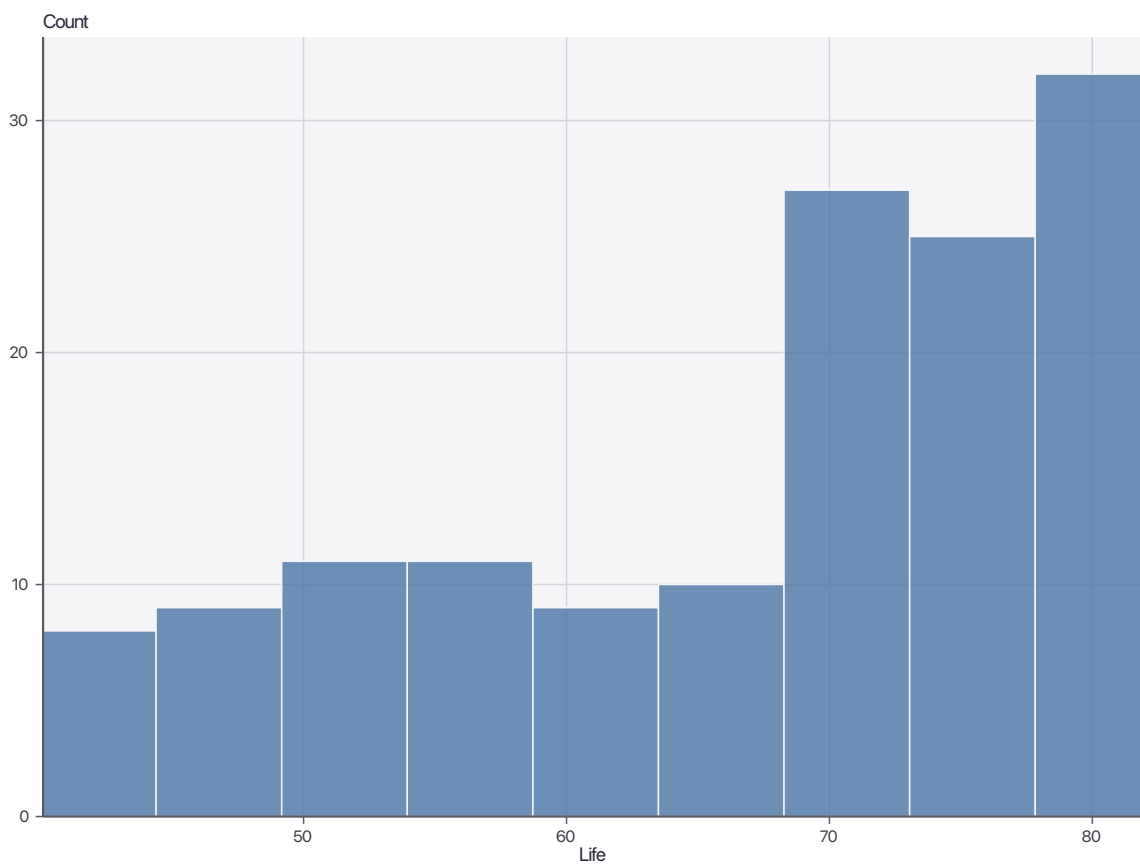
```
data(gapminder_2007) + bar * bin + x(col.life)
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life))
```



Now put that same block inside a faceted sentence:

R

```
data(gapminder_2007) + bar * bin + x(life) | facet(continent)
```

Python

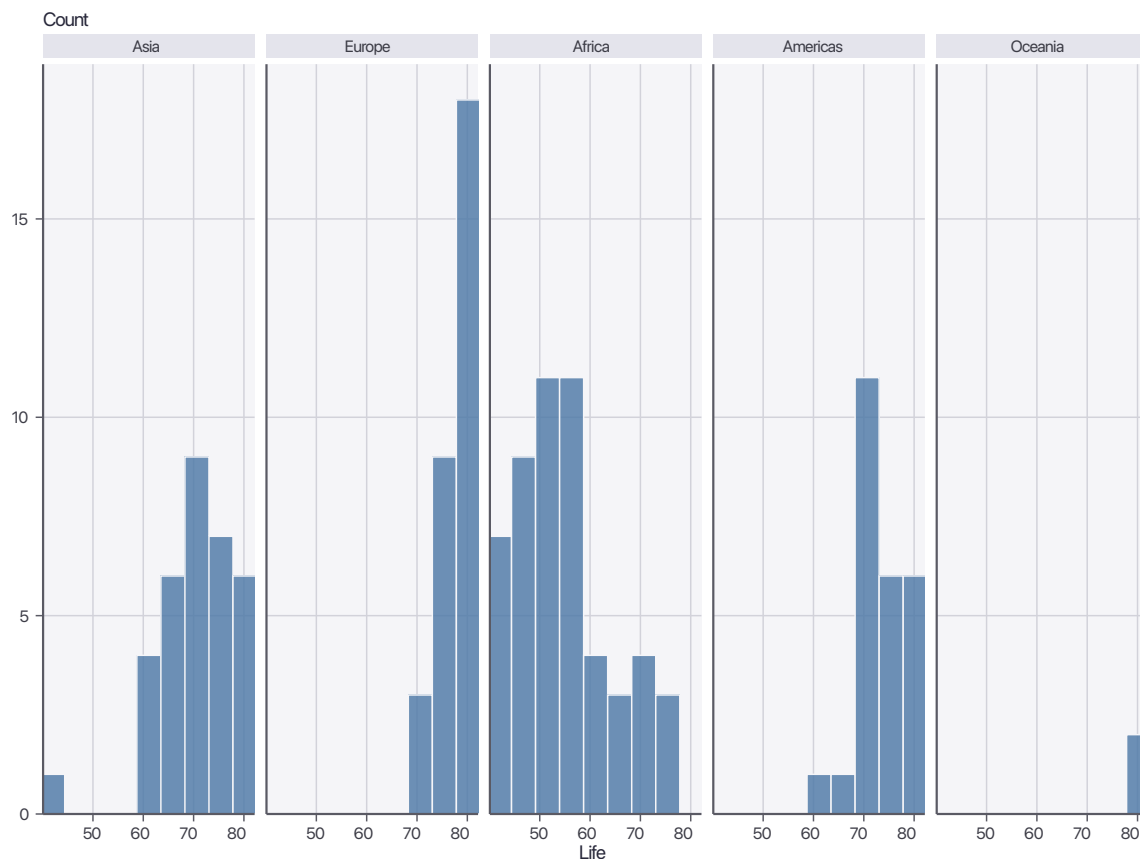
```
data(gapminder_2007) + bar * bin + x(col.life) | facet(col.continent)
```

Julia

```
data(gapminder_2007) + bar * bin + x(:life) | facet(:continent)
```

JavaScript

```
plot(data(gapminder_2007), layer(bar, bin), x(col.life),  
      across(col.continent))
```



“Given gapminder 2007: bars derived by bin, x is life, split into panel columns by continent.”

The enclosing expression added panels. It did not reinterpret the sub-expression: each panel bins and counts exactly as the single plot did. That is the whole law, and it is what lets you read a long sentence one piece at a time. A sub-expression can be held in a variable too, `piled <- bar * bin`, and spoken wherever you like.

45.7 Law 7: Minimum Syllable

A visible plot is a mark plus its required positions. A mark alone is silent; a position alone has no carrier.

In Hangeul the smallest pronounceable unit is a consonant plus a vowel: $\neg$ (*g*) alone is silent, and $\neg\text{ㅏ}$ (*ga*) is the minimum syllable. A bare vowel cannot stand alone either: ㅏ (*a*) must be written ㅇㅏ (*a*), on a silent carrier.

A mark is the silent consonant: `point` with no positions draws nothing, because there is nowhere to draw it. A position is the vowel that needs its carrier: `x(gdp)` without a mark answers no question. It does not say what sits at that x. `gog` refuses the incomplete syllable, and the refusal names every missing half:

```
data(gapminder_2007) + point
```

Error:

```
! gog: `point` needs `x()` but none is set. Add `x(<column>)` - a point
  ↳ cannot be drawn without it.
gog: `point` needs `y()` but none is set. Add `y(<column>)` - a point cannot
  ↳ be drawn without it.
gog: nothing was rendered. Fix the above, or set GOG_STRICT=0 to draw anyway.
```

45.8 Law 8: Pronounceable $\neq$ Usable

The grammar guarantees well-formedness, never good taste. Grammar is law; taste is advice.

꺤 (*kkwek*) obeys every Hangeul syllable rule, you can pronounce it, yet no Korean word uses it. Plots work the same way. This renders, because it is grammatically sound; whether it *says* anything is your judgment, not the engine's:

R

```
# Legal, and nearly invisible: the engine's job is to obey, not to veto
data(gapminder_2007) + point + x(gdp) + y(life) +
  style(opacity = 0.05)
```

Python

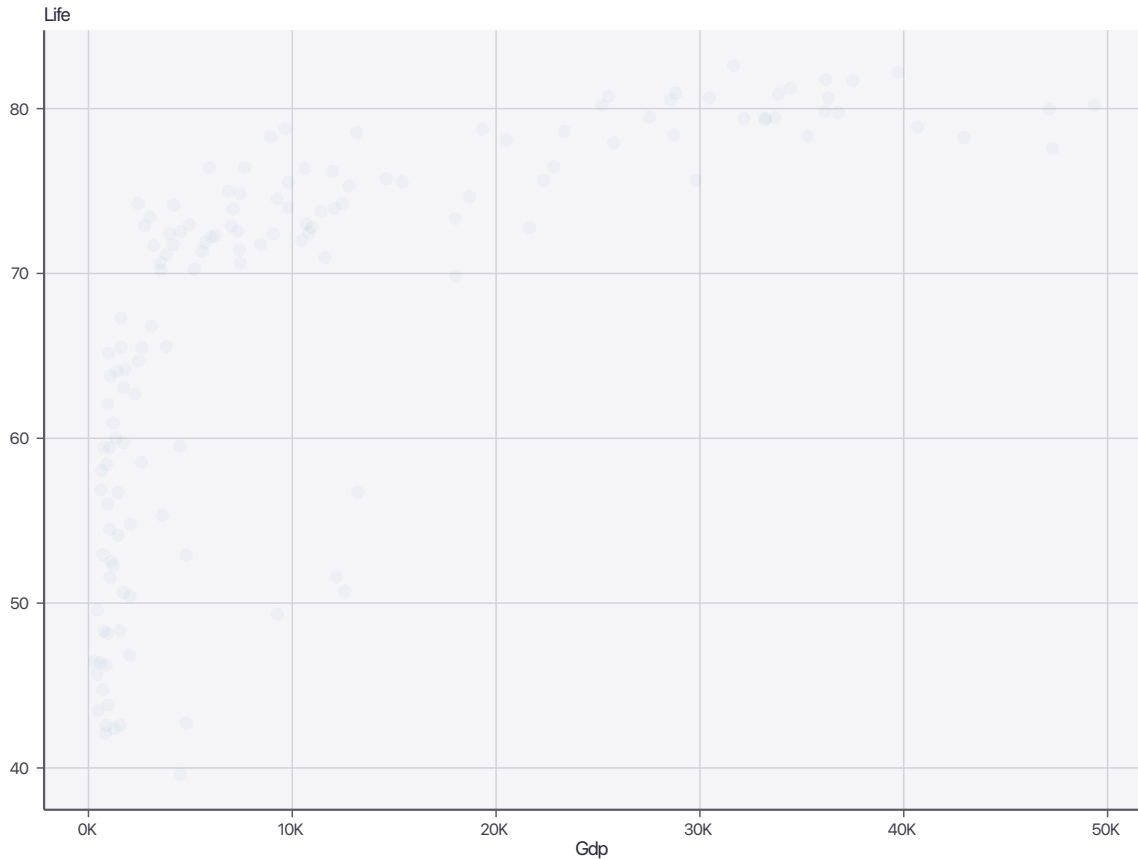
```
(data(gapminder_2007) + point + x(col.gdp) + y(col.life) +
  style(opacity = 0.05))
```

Julia

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + style(opacity = 0.05)
```

JavaScript

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
      style({ opacity: 0.05 })))
```



It draws, and almost nothing is visible. No legal combination is ever refused for being ugly: one that looks pointless today may be a real plot tomorrow, so refusing it now on taste grounds removes something nobody has thought of yet.

The law has a second half, and the two viewing angles of a cube show both at once. `space(turn = 390)` is a full circle plus 30 degrees, so it is the same view as `space(turn = 30)`, and `gog` draws it without comment. `space(tilt = 95)` is refused. The difference is not that one is uglier. A bearing wraps, so 390 names a view you can stand in. A tilt runs out at 90, where you are looking straight down, so 95 names no view at all. Taste is guided softly and never blocked; well-formedness is enforced hard. `Space` draws both.

The engine will not judge, so here is the advice instead. `gog` exists so that you can say a visualization in clean, minimal syntax. It will let you combine anything the grammar allows,

and that includes combinations that hide what you wanted to show. Be like a good writer, and do not let decoration distract from the main idea.

That advice is about the picture, and it applies to the sentence too. Two sentences can build the identical plot while one is much harder to follow. A second `data()` applies to a single mark, so a mark written after it returns to the plot's first table. A reader who has not learned that rule will read such a sentence wrong. `gog` accepts either arrangement, so the fix is a habit rather than a rule, and [Data](#) gives it.

45.9 Law 9: Universal Transcription

A specification describes the visual, never one renderer's drawing commands.

A specification is **declarative**: you say what you want to see, never how to draw it. The form is far older than software. “And God said, Let there be light: and there was light” (*King James Bible*, 1769, Genesis 1:3) states an outcome and nothing else. There is no procedure in it, no order of steps, and no mention of how the light is made. Every declarative sentence leaves that part out, and something else has to supply it. In `gog` the engine does, and every stroke on these pages answers a sentence that never mentioned strokes.

What you write becomes a small, renderer-neutral specification (which marks, which columns on which channels, which transforms), and *that* is what renders. One renderer reads it, and it produces the SVGs on these pages. The same specification, unchanged, is what the Python, Julia and JavaScript bindings build. It is also why a specification names no shape and no stroke: it says what to see, and the renderer decides how to draw it.

45.9.1 Why that renderer writes SVG

The law says the specification is neutral. It does not say which format the renderer writes, and **SVG** is a choice with reasons.

The first reason is the one you can see. An SVG stores shapes and coordinates, not a grid of colored dots. A plot therefore redraws at any size it is given, and stays sharp at every one. Print one at poster size and its curves are still curves.

The second is that text stays text. Every axis label, tick number and title is written as characters in the file, not drawn as a picture of characters. You can select them, a search can find them, and a screen reader can read them aloud. It is also what lets a plot in this book carry Hangeul, or any other writing system, without the engine knowing anything about the letters.

The third reason is the one this project depends on most, and it is easy to miss. An SVG is a text file, so two of them can be compared character by character. That is what makes the

promise of four bindings checkable: every sentence in this book is drawn from R, Python, Julia and JavaScript, and the four files must match exactly. Comparing pictures instead would mean comparing colored dots, which changes with the version of whatever drew them. An exact test would become an approximate one, and a small disagreement between two languages could pass as a rounding difference.

The fourth is motion. A `play` animation is written into the same file as the plot, using SVG's own timing elements. A time-lapse is one picture that moves, with no video file and no JavaScript beside it.

45.9.2 Why there is one renderer

`gog` has exactly one renderer, and the reason is what a second one would have to copy.

A renderer decides five things the specification does not name: layout, tick selection, palettes, axis ranges, and label formatting. Write a second renderer beside the first and those five decisions exist twice. Two copies of a decision do not stay equal. One would bin and count, the other would draw the raw rows, and `bar * bin` would then mean two different pictures from one specification.

That would be Law 9 broken by the renderer rather than by the specification: a renderer deciding things the specification is supposed to own, with nothing in the design to make it visible. So there is exactly *one* renderer. A second one is possible only once those five decisions live in a single shared layer that both would read.

Raster output comes from converting the SVG rather than from a second renderer. `render_svg()` hands you the plot as ordinary text, in every binding:

```
svg <- render_svg(data(gapminder_2007) + point + x(gdp) + y(life))
cat(substr(svg, 1, 51))
```

```
<svg xmlns="http://www.w3.org/2000/svg" width="800"
```

Those are the first 51 characters; `svg` holds the whole plot, about 14,000 of them. Writing it to a file saves it, and any tool that reads SVG turns it into another format: `rsvg-convert -w 1600 plot.svg -o plot.png` gives you a raster image, and `rsvg-convert -f pdf` is what places each plot in this book's PDF. Converting is the deliberate answer rather than a missing feature. It keeps every decision about what a plot looks like in the one place that is allowed to make them, which is the whole of the law above.

The engine does that conversion itself in one case, and the case is worth seeing because it shows where the boundary sits. A plot that binds `play()` moves in a browser and nowhere else, so `save_gif()` writes the same sequence as a picture that a slide or a message will play. That is still conversion in the sense this section means. The frames come out of the one renderer, and

the GIF writer only turns them into pixels. It chooses no tick, no color and no layout, so there is nothing for it to drift from.

45.10 The settable rule

A **setting** (`style(...)`) is an atom like any other, so the first two laws govern it as well: a setting is available on every mark whose geometry can carry it, behaves the same across them, and is absent exactly where composition already expresses the feature more capably. The first clause is No Exceptions: no mark-specific gap you have to memorize; the second is Orthogonality's *non-redundant* half: no setting for what marks already do by combining.

That is why `style(border_color =, border_size =)`, an outline distinct from the fill, is available on all five closed-glyph fills (`bar`, `box`, `point`, `zone`, `surface`), not one of them: their perimeter is a shape you cannot trace by composition, so a setting is the only way to draw it. And why `style(pattern =)`, the paint's **texture**, is realized once per geometry: on the six path strokes (`line`, `step`, `interval`, `path`, `rule`, `edge`) as a **dash**, and on the five fills (`bar`, `box`, `area`, `ribbon`, `zone`) as a **hatch**: one setting, one form per geometry, the way `color` is a stroke on a line and a fill on a bar. A curve fill (`area`, `ribbon`) takes the hatch (a region can be textured) but no *border*: its edge is a data curve, which already *is* a `line` (`area + line`, `line * bounds`), more capable than any border. The classes, not the individual marks, decide.

The failure this prevents is the quiet surprise: learning a setting on one mark and finding it missing on its sibling for no reason you can see. Where a class is incomplete, `gog` says so with an **Unsupported** message rather than a silent nothing, so the gap is visible.

45.11 Why these laws matter

Each law refuses the same enemy, **the expert's shortcut**.

The requests for a special case are always reasonable ones:

- “Can *bar* have a different color scale than *point*?”
- “Can *smooth* have a *span* parameter?”
- “Can I just write *alpha*? Everyone knows what it means.”

Each request sounds reasonable in isolation. Each one costs more than the one before it: the tenth exception makes the first nine harder to remember.

The laws are the answer: not “no, because we don't want to,” but “no, because that would break Law 2 / Law 3 / ...” The rule, not the maintainer, says no.

Part VI

The host language

The 1446 book had no host language to explain: its letters were the whole system. `gog`'s letters live inside four programming languages, and this part is what each language adds.

Everything so far has been the grammar, and the grammar is the same wherever you write it. One engine reads one specification, so the same sentence draws the same plot in every language.

This part holds what the grammar chapters could not say. Some things belong to the language you type in rather than to the grammar, and they cannot be stated once for everybody. A table has to reach `data()`. A plot has to display in a notebook. And `gog`'s `+` has to work beside the operators the language already has. Those are real questions, and answering them in the grammar chapters would answer them wrongly for three readers out of four.

Each of the four has a chapter here (Bezanson et al., 2017; Ecma International, 2025; R Core Team, 2026; Van Rossum & Drake, 2009). The grammar chapters say nothing about any one language, which is what keeps this part separate.

They are ordered R, Python, Julia, JavaScript, and the order has a reason. The first three write the same sentence three times. The operators are identical, and only the column name moves: a bare `gdp`, then `col.gdp`, then `:gdp`. If you have read the R in this book you can read the Python and the Julia already. JavaScript is the one that genuinely reads differently. It is the only one of the four that cannot give `+`, `*`, `|` and `/` new meanings, so it spells them as words. It is last for that reason, so the pattern the other three share is read first.

The four chapters are also the proof. Every sentence in this book was translated and drawn by every binding. The four produce byte-identical plots and refuse the same sentences in the same words. They differ only where a chapter is about the host language itself.

46 R

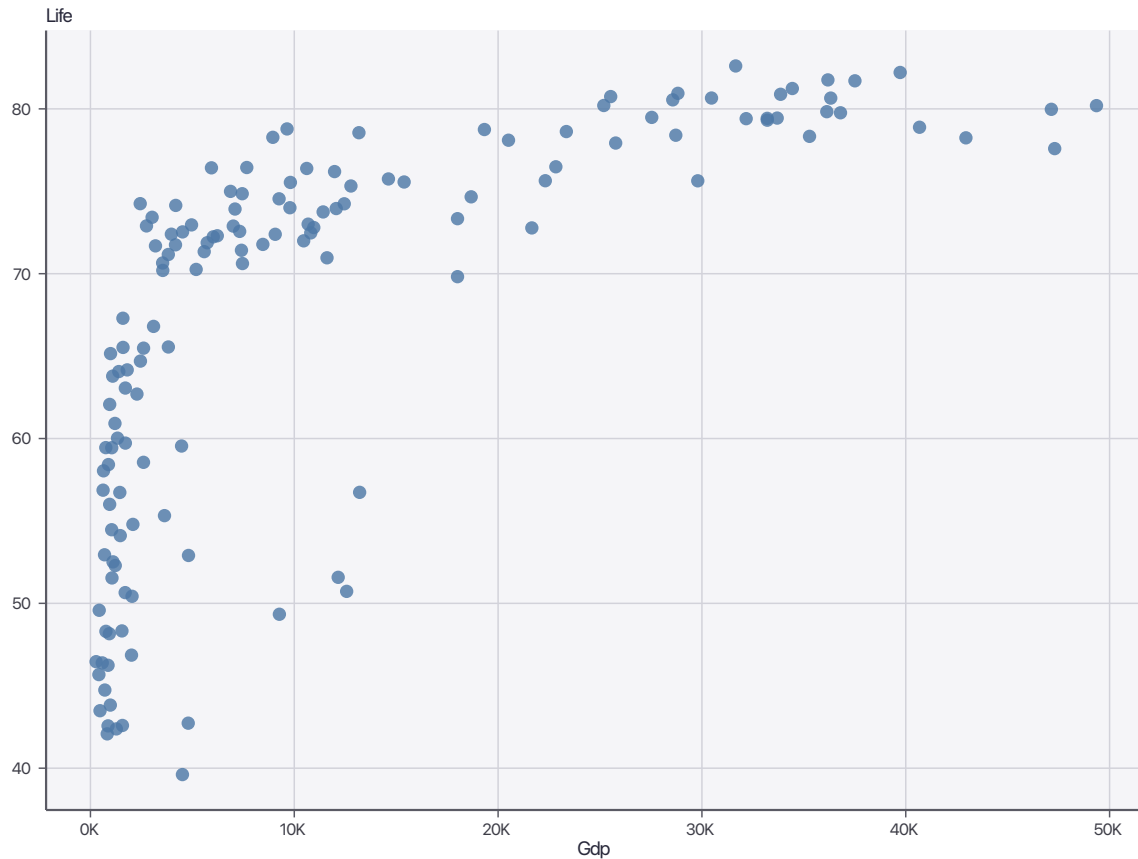
Every sentence in this book so far was written in R, so this chapter adds nothing to the grammar. What it adds is the four questions only R raises: how the two pipes reach `data()`, which of `gog`'s words base R already uses, how a database connection arrives, and how the package installs. What a table is, and how one is passed, is in [Data](#). A table is an ordinary R data frame, and needs no package of its own.

Every block marked `{r}` below was run when this page was built, through the same engine every other chapter uses. The install commands at the end are shown, not run.

46.1 Pipes

R has two pipes in common use, and R users reasonably want to start a plot with a table they already have. The native pipe works, and draws exactly the plot the unpiped form draws.

```
gapminder_2007 |> data() + point + x(gdp) + y(life)
```

“Given *gapminder 2007*: *points*, *x* is *gdp*, *y* is *life*.”

That is not a second way of saying it, and `gog` does not implement it. By the time the engine sees anything, the two forms are the *same expression*. You can watch R rewrite one into the other, because `quote()` shows what the parser built rather than what you typed:

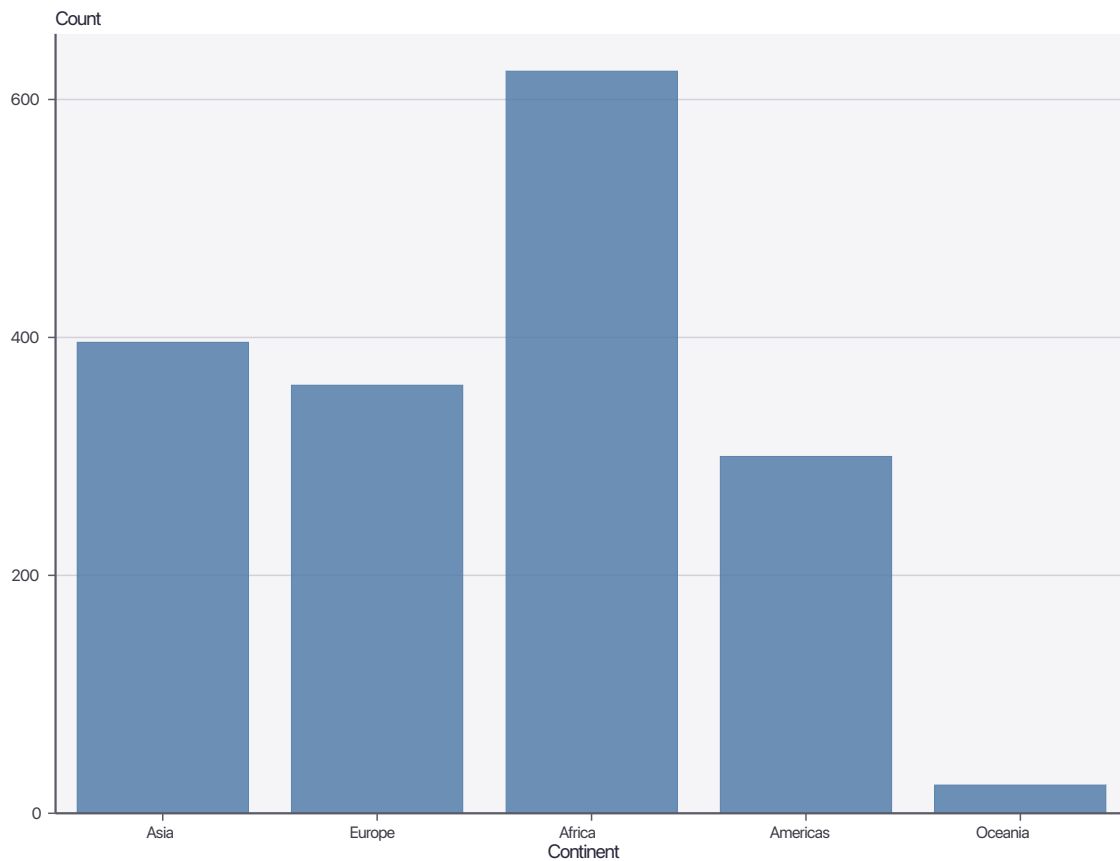
```
quote(gapminder_2007 |> data() + point + x(gdp) + y(life))
```

```
data(gapminder_2007) + point + x(gdp) + y(life)
```

Two facts about R combine to make that work, and both have to hold.

The first is precedence. `|>` binds more tightly than `+`, so the pipe finishes before the sentence begins: it takes the table into `data()` and hands the finished result to `+`. Derivation with `*` sits between the two, which is where it needs to be, so a transform still binds to its mark first:

```
gm_all |> data() + bar * count + x(continent)
```

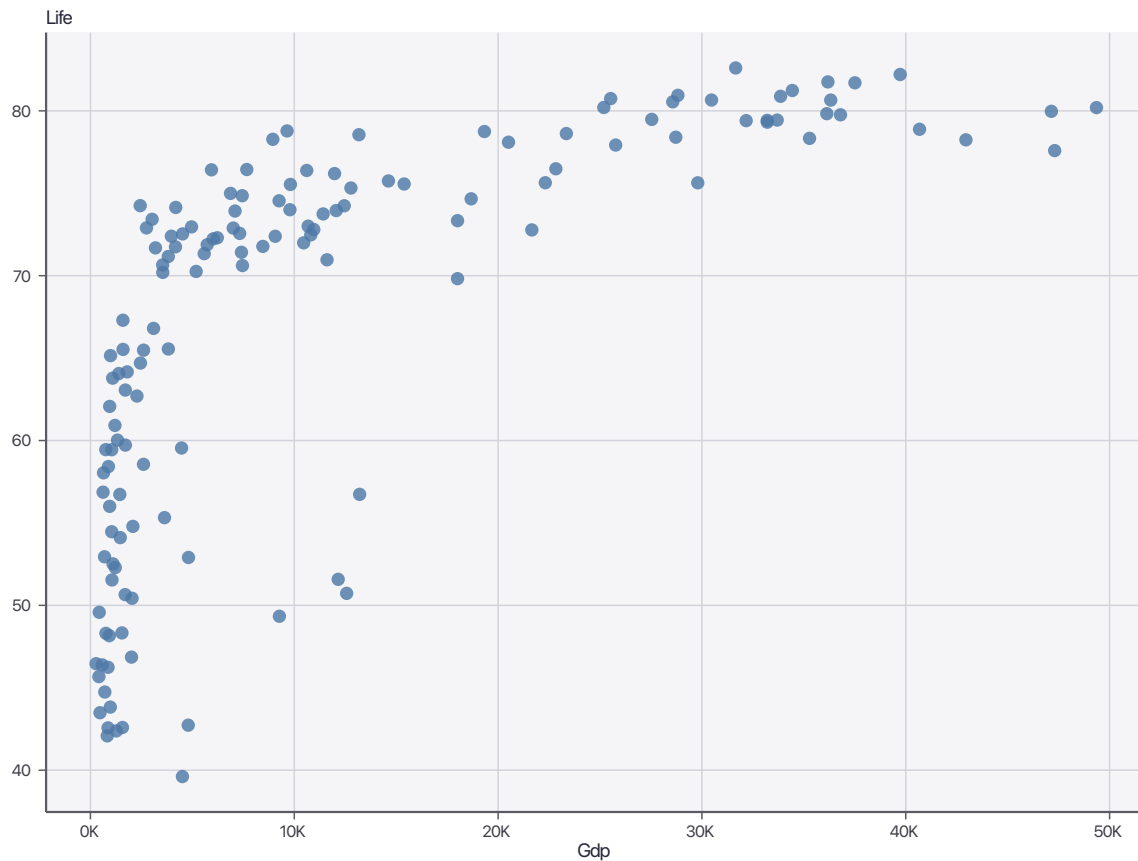


“Given all the gapminder years: bars derived by count, x is continent.”

The second is *when* it happens. The native pipe is a parser transformation: writing `df |> data()` produces the code `data(df)` before any of it runs. That timing decides everything, because `data()` reads the table’s name off the expression you wrote. The pipe leaves it a plain name, so nothing is lost. Naming the table is the whole job of `data()` (see [Data](#)), and it is what a later layer resolves its bare columns against.

Arguments still land where you would expect, since the piped value fills the first one:

```
gapminder_2007 |> data(name = "gapminder") + point + x(gdp) + y(life)
```

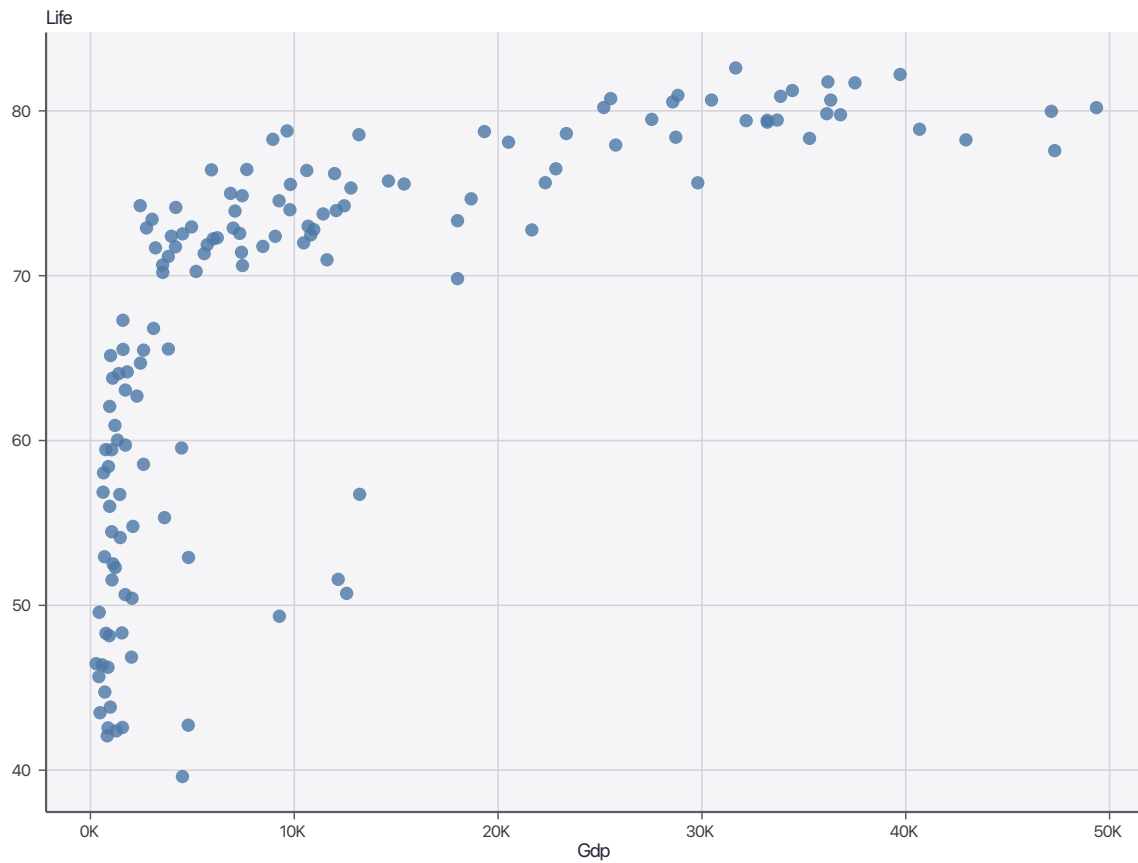


46.2 What %>% does to the table's name

The `magrittr` pipe, `%>%`, is the one most R code still uses, and here it behaves differently. It is an ordinary function rather than a parser rule, so it *evaluates* the left side and passes the value along under its own placeholder, `..`. The name never reaches `data()`:

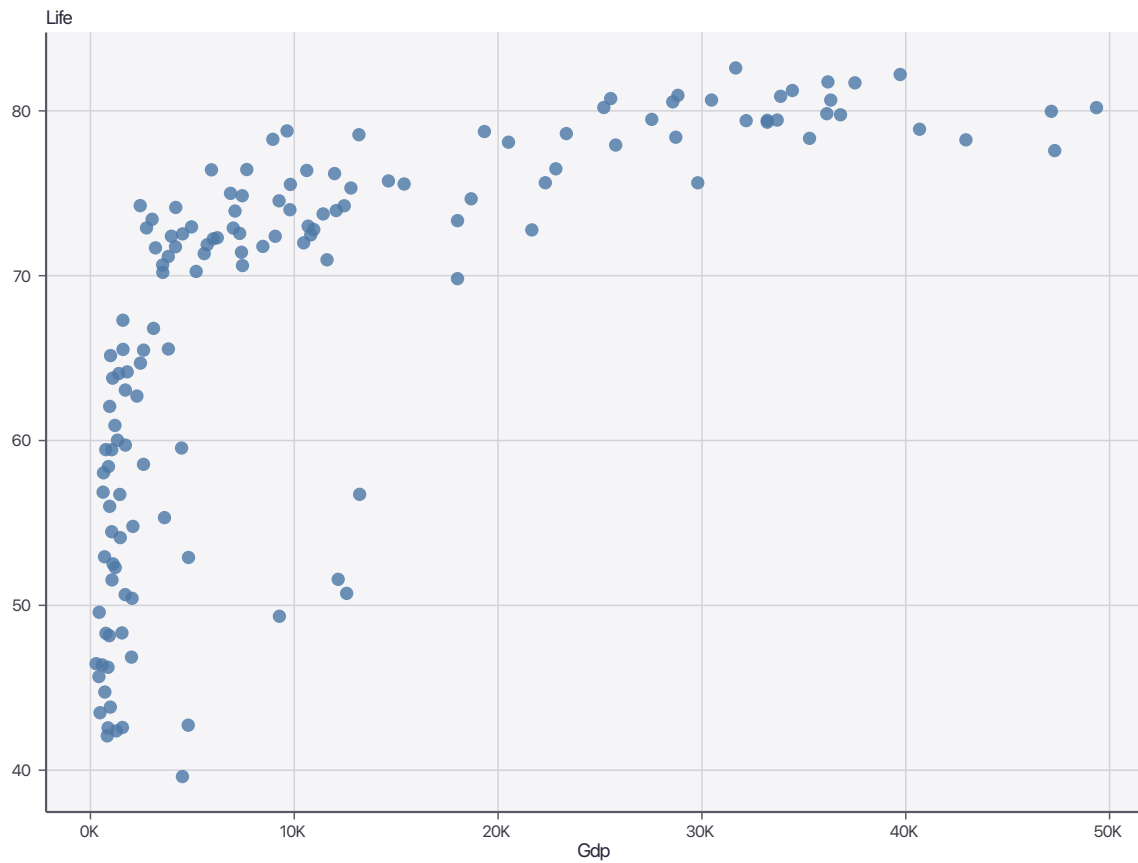
```
library(magrittr)
gapminder_2007 %>% data() + point + x(gdp) + y(life)
```

Warning: gog: magrittr's `%>%` replaced this table with its placeholder `..` before `data()` could read the name, so the table is called `data`. A layer resolves its bare columns against the nearest table *by name*, so two tables piped this way collide. Either name it - `data(name = "...")` - or use R's native pipe, which keeps the name: `df |> data()` reads as `data(df)`.



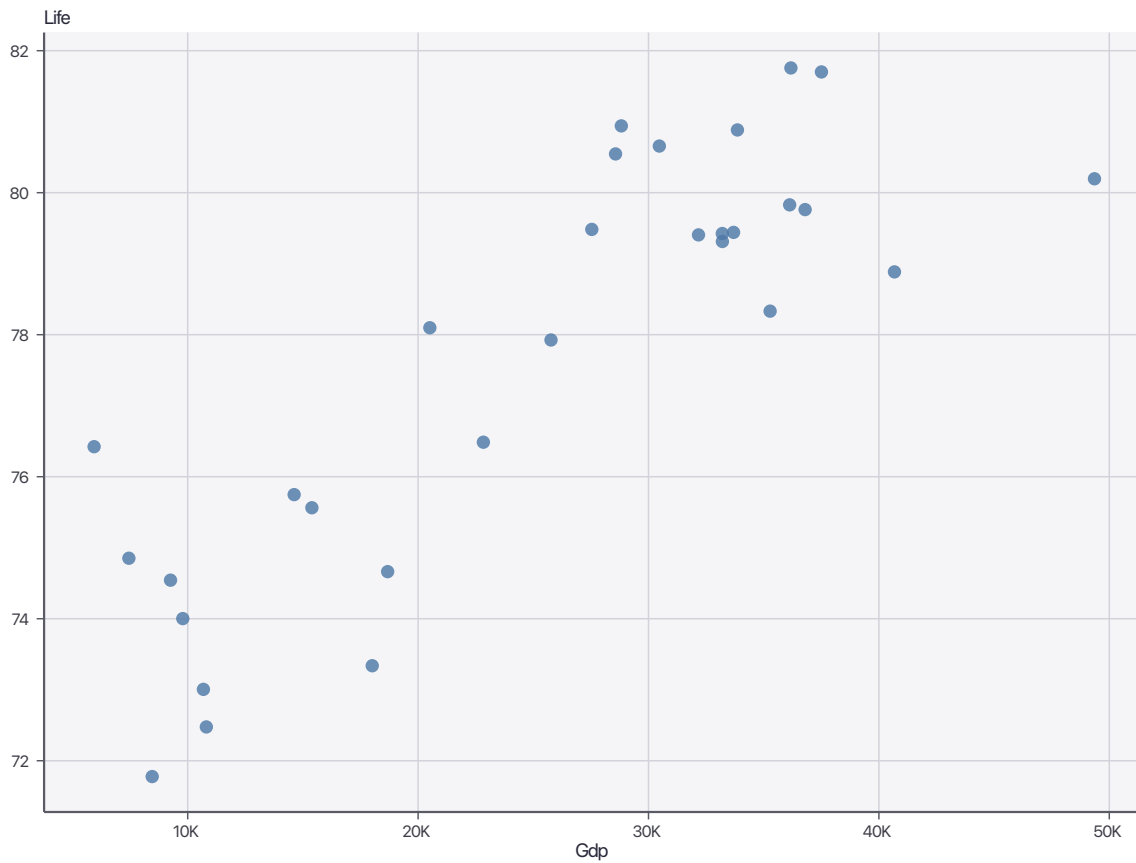
The plot is right, and for a single table it stays right. What is gone is the name, so `gog` says so and tells you how to put it back. Either use the native pipe, or name the table yourself:

```
gapminder_2007 %>% data(name = "gapminder_2007") + point + x(gdp) + y(life)
```



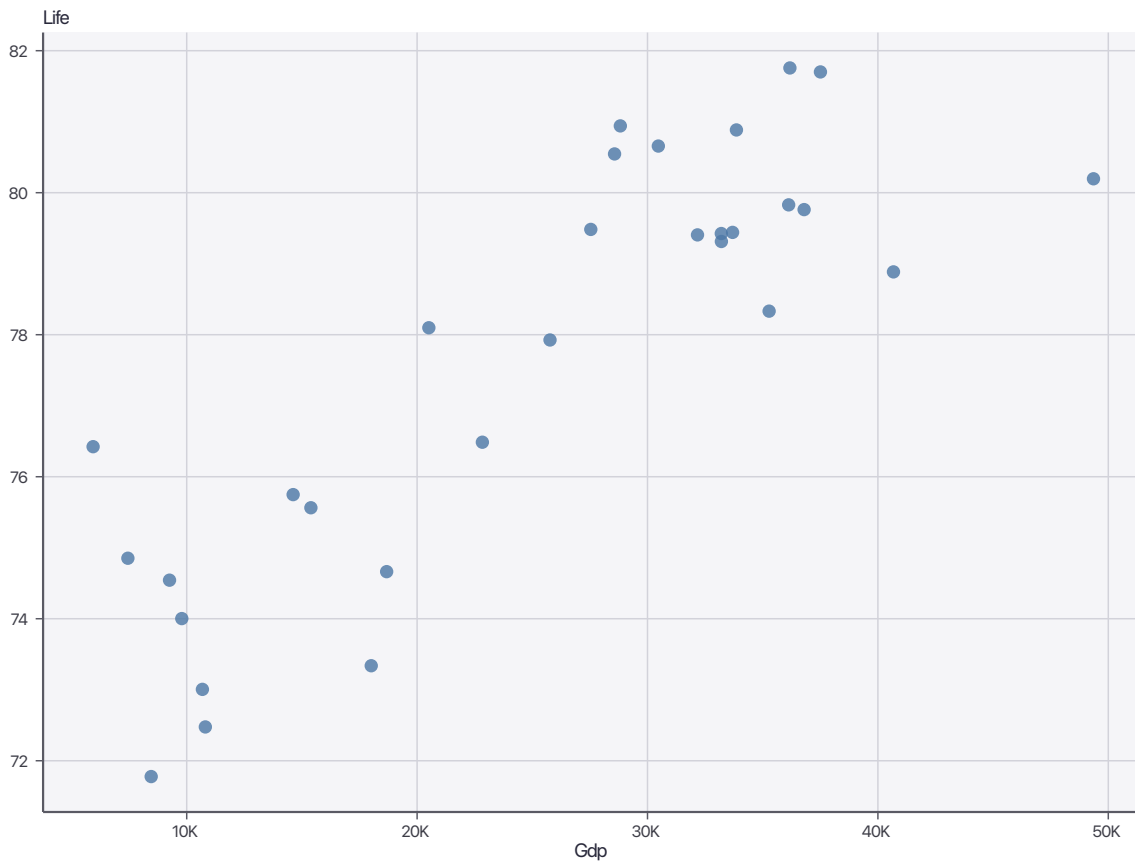
Only the placeholder raises that warning, so it is worth saying what does not raise it. Filtering before `data()` is the native pipe's most useful form, and it raises no warning. The expression is not a plain name, but it is still distinct, so `gog` uses it as the table's name.

```
gm_all |> subset(year == 2007 & continent == "Europe") |> data() +  
  point + x(gdp) + y(life)
```



The table there is called `subset(gm_all, year == 2007 & continent == Europe)`, which is ugly and still works. `name =` gives the table a shorter name, and is worth using once a second table is involved:

```
gm_all |> subset(year == 2007 & continent == "Europe") |> data(name =  
  ↪ "europe") +  
  point + x(gdp) + y(life)
```



46.3 Why the name is worth a warning

An unnamed table is fine until there are two of them. A second `data()` gives a layer its own table, and “nearest table wins” is decided *by name*, so two tables under one name leave the rule nothing to choose between:

```
data(actuals, name = "series") + x(year) + y(sales) + line +
  data(forecast, name = "series") + point
```

Error:

```
! gog: two different tables are both called `series` - a layer resolves its
  ↳ bare columns against the nearest table by name, so one of these can never
  ↳ be reached. Give them distinct names: `data(name = "...")`.
```

Without that refusal the plot would draw the first table twice and report the second table's columns as misspellings, which blames the reader for something the binding lost. Two `%>%` pipes in one expression reach that state, because both tables arrive as `..`

46.4 What the pipe does not change

The pipe feeds `data()`. It does not replace `+`, and there is no piped form of the rest of the sentence: marks, channels, and transforms compose with `+` and `*` as they do everywhere else in this book. `gog` has no pipe of its own, because that would be a second spelling for something the language already spells.

This section is about R specifically. Julia's `|>` is a function-application operator with different behavior, and Python and JavaScript have no pipe of their own, so the question does not arise in the other three chapters.

46.5 Twenty names base R already uses

Twenty of the package's exports are also names in the packages R attaches for you, and `library(gog)` warns you about eleven of them. Here are all twenty:

from	names
stats	density line median quantile smooth step
graphics	box layout text title
grDevices	palette
utils	data stack
base	jitter max mean min order range sum

Nine of the twenty cannot affect you, which is why `library(gog)` leaves them out, and the reason is a rule in R itself. A mark that takes no argument is an *object* rather than a function, and so is a transform that takes no parameter. When R evaluates a call, it looks for a function of that name and skips any binding that is not one. So base R still answers, even though `library(gog)` put its own `mean` in front:

```
is.function(mean)  # gog's `mean` is an object, so a call cannot reach it
```

```
[1] FALSE
```



```
mean(1:10)           # and base R's function answers anyway
```

```
[1] 5.5
```

That holds for the functions that take another function as an argument, because they resolve the name the same way:

```
sapply(gapminder_2007["life"], mean)
```

```
      life  
67.00742
```

The other eleven are functions, and a call does reach **gog**'s: **box**, **data**, **density**, **jitter**, **layout**, **order**, **palette**, **quantile**, **range**, **stack** and **title**. Qualify the one you want, and both remain available:

you write	you get	for base R's, write
<code>order(population)</code>	the plot's sort order	<code>base::order(df\$population)</code>
<code>data(gapminder_2007)</code>	the plot's table	<code>utils::data(diamonds)</code>
<code>density(2)</code>	the smoothing transform	<code>stats::density(x)</code>
<code>jitter(0.5)</code>	the offset transform	<code>base::jitter(x)</code>
<code>quantile(0.9)</code>	the 90th percentile transform	<code>stats::quantile(df\$gdp)</code>
<code>range(0.25, 0.75)</code>	the band transform	<code>base::range(df\$gdp)</code>
<code>stack(share = TRUE)</code>	the piling transform	<code>utils::stack(df)</code>
<code>palette("soft")</code>	the plot's colors	<code>grDevices::palette()</code>
<code>title("Life expectancy")</code>	the plot's title	<code>graphics::title(main = "...")</code>
<code>box()</code>	the box-and-whisker mark	<code>graphics::box()</code>

range is a function because it takes a band, so the skip rule above does not save you. That is what writing `range(0.25, 0.75)` costs you. The refusal reads the argument it was given: a vector where a probability belongs means you wanted base R's, and the message says so.

```
range(c(3, 1, 4, 1, 5))
```

Error:

```
! gog: `range()` takes the band's two ends, each one number between 0 and 1,  
  ↪ e.g. `range(0.25, 0.75)`. It was given 5 numbers. For the smallest and  
  ↪ largest of a vector, gog masks that name: use `base::range()`.
```

`order` is the one that breaks the most ordinary code. Sorting a table beside a plot is ordinary work. `gog`'s `order` returns an atom, not the row positions that `[]` expects. Left alone, the failure arrives far from the line that caused it and names neither `order` nor `gog`. Whenever the argument is not a bare name, the refusal says which function you reached and how to reach the other one:

```
gapminder_2007[order(gapminder_2007$population), ]
```

Error:

```
! gog: `order(gapminder_2007$population)` is not a column name. gog's
  ↳ `order()` takes a bare column, as in `order(population, desc = TRUE)`. To
  ↳ sort a vector, the base function is still there as
  ↳ `base::order(gapminder_2007$population)`.
```

Qualified, it sorts:

```
ranked <- gapminder_2007[base::order(-gapminder_2007$population), ]
head(ranked[c("country", "population")], 3)
```

	country	population
25	China	1318683096
59	India	1110396331
135	United States	301139947

This book does that in several other chapters, for exactly this reason.

Three of the eleven refuse nothing, because nothing separates the two readings. `data(mtcars)` is well formed in both readings. It returns a plot's table and no complaint, so write `utils::data()` when you mean to load a dataset. `title("...")` is the same case: a string is what both want. So is `box()`, which both readings accept with no argument at all. The other eight report the mistake, because the argument you passed is one `gog` would refuse. `order` is the partial case: it catches `order(df$population)` and not `order(pop)`, where `pop` is a bare name holding a vector.

The names stay as they are. A grammar keeps its own vocabulary, which is Law 3, and the question is settled. Renaming `order` to `sort_by` would replace a word every reader knows with one only this package uses. Where another package defines a name `gog` also defines, attach `gog` after that package, so R finds `gog`'s name first, or write `gog::interval`. Every language meets this problem, and each one asks you to solve it differently; [the grammar chapter](#) compares them.

46.6 Five spellings that still need a prefix

The nine objects above are safe in ordinary code, and safe inside `sapply()`, `lapply()`, `tapply()`, `mapply()`, `Map()`, `Reduce()` and `apply()`. Each of those asks R for a function of the name it was handed, which finds base R's.

Five spellings still fail, for two different reasons. The first is a name that is passed on twice before anyone looks it up:

```
aggregate(life ~ continent, gapminder_2007, mean)
```

```
Error in `get()`:  
! object 'FUN' of mode 'function' was not found
```

By the time the lookup happens, the name being searched for is `FUN` and not `mean`, so the message names neither `mean` nor `gog`. `by()` and `ave(..., FUN = mean)` fail the same way. Writing `base::mean` fixes all three. The data frame form of `aggregate()` looks the name up one step earlier, so `aggregate(gapminder_2007["life"], gapminder_2007["continent"], mean)` works unchanged.

The second reason is a name used as a value rather than called. `do.call(sum, list(x))` wants a function and receives an object, so write `do.call("sum", list(x))` or `do.call(base::sum, list(x))`. And `f <- mean` copies the object, so `f(x)` reports that it could not find a function called `f`.

That is the whole cost, and it is worth stating as one rule. If you hand one of those nine words to another function *by name*, and that function hands it on again, write `base::` in front of it.

46.7 Reading from a database

`query()` takes a **DBI** connection. DBI is R's database standard, so the same call reaches SQLite, Postgres, MariaDB, BigQuery, and anything with an ODBC driver.

```
con <- DBI::dbConnect(RSQLite::SQLite(), "sales.db")  
  
query(con, "SELECT status, revenue FROM orders") + bar + x(status) +  
  ↪ y(revenue)
```

DBI and its drivers are **Suggests**, not hard dependencies. A reader who never writes SQL installs neither. If you do write SQL, install DBI and the one driver your database needs:

```
install.packages(c("DBI", "RSQLite"))    # or RPostgres, RMariaDB, odbc,  
↪ bigrquery
```

`gog` never opens a connection and never stores a password. You connect the way you already do, and hand the open connection over.

46.8 Getting it

A plot is drawn by the engine, which is a compiled Rust binary, so the R package has to be able to find one. An installed copy carries its own. The package's `configure` script bundles the engine at install time, and refuses to install at all rather than leave you a package that cannot draw.

```
install.packages("gog", repos = c("https://psychometrician.r-universe.dev",  
                                   "https://cloud.r-project.org"))
```

That command works today. `gog` is published on [r-universe](https://psychometrician.r-universe.dev). On macOS and Windows it builds a binary with the engine inside it, so installing needs no Rust toolchain.

On Linux the same command installs from source, and the source build compiles the engine. That needs Rust and a network connection. If the machine does not have Rust, install it first and then run the command above:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y  
source "$HOME/.cargo/env"
```

The build runs once, while the package installs.

The `repos` argument names two addresses because `r-universe` is not CRAN. The first address is where `gog` lives. The second is an ordinary CRAN mirror, and it is there so your other packages still install as usual. `gog` is not on CRAN, so both addresses are needed.

There is another route, and it installs straight from the source repository:

```
remotes::install_github("psychometrician/gog", subdir = "r-pkg/gog")
```

A tarball from GitHub carries the engine's Rust sources, and `configure` compiles them during the install. So this route needs a Rust toolchain, which the macOS and Windows builds above do not. It also needs the `remotes` package, which `install.packages("remotes")` adds if you

do not have it. Take it when you want one particular branch or commit rather than the latest build.

You can also work from a checkout. On that route nothing installs other packages for you, so `jsonlite` and `pkgload` have to be installed first. Outside base R, `gog` needs one package, `jsonlite`. It turns the sentence you write into the JSON that the engine reads. `install.packages()` and `remotes::install_github()` both install it along with `gog`. Loading from a checkout also needs `pkgload`, which loads a package from a folder instead of from a library. Install both if you do not have them:

```
install.packages(c("jsonlite", "pkgload"))
```

If one is missing, the step that needs it stops immediately, and the message names the missing package.

Then build the engine once and load the package from where it sits:

```
cargo build --release -p gog-cli
```

```
pkgload::load_all("r-pkg/gog")
```

Installing from that same checkout is the better test, because it runs `configure` and proves the bundling works:

```
R CMD INSTALL r-pkg/gog
```

The engine is looked for in five places, in this order: the `GOG_CLI_PATH` environment variable, a `target/` build in a surrounding checkout, the copy inside the installed package, `gog-cli` on your `PATH`, and one last search upward from the working directory. A checkout is searched before the bundled copy on purpose. Inside a checkout the bundled engine is a leftover build, and using it would give a stale engine to the person editing the engine.

47 Python

The grammar does not change when the language does. One engine reads one specification, so a sentence means the same thing no matter who wrote it. Everything in this book up to here is as true in Python as it is in R, and what differs is short enough to state in one chapter.

Every Python block with a plot under it was executed, through the same engine every other chapter uses.

47.1 A column is written `col.gdp`

R writes a column as a bare name, Julia as a symbol, Python through a small accessor:

```
data(gapminder_2007) + point + x(col.gdp, scale='log') + y(col.life) +  
  ↪ color(col.continent)
```

1K 10K 40 50 60 70 80 Life Gdp Continent Asia Europe Africa Americas Oceania

“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by continent.”

The R form of that sentence is the one from [A first plot](#), with `col.` in front of each column. Those four characters are not decoration, and the next paragraph says what they do. In R a bare `gdp` cannot be anything except a name; Python has no bare names, and in this grammar a plain string is how you spell a *value*: `style(color="tomato"), title("..."), palette("okabe")`. Without the accessor a column and a value would look identical, and the rule that a channel takes a column, never a value, would be invisible in Python.

So a string where a column belongs is refused, and told what to write:

```
data(gapminder_2007) + point + x('gdp') + y(col.life)
```

gog: ``x("gdp")`` binds a **value**, and a channel takes a **column**. Python has
 ↪ no bare names, so a column is written with the accessor: ``x(col.gdp)``
 ↪ maps the column called ``gdp``.

The message names the column spelling, `x(col.gdp)`. On a channel you could also set, it names both, because one *maps* a column and earns a legend while the other *sets* one value and earns none ([Setting vs mapping](#)). The mistake in the other direction is refused the same way:

```
data(gapminder_2007) + point + x(col.gdp) + y(col.life) +  
  ↪ style(color=col.continent)
```

gog: ``style(color=)`` fixes one value for the whole layer, and ``col.continent``
↪ is a column. To **map** it - one value per category, with a legend to
↪ decode it - that is a channel: ``color(col.continent)``.

A name Python cannot spell after a dot takes the bracket form instead: `col["life exp"]` for a space, `col["gdp.2007"]` for a dot. A non-Latin column name needs nothing special either: `col.국가` works, and the bracket form covers the rest.

47.2 A sentence over several lines takes parentheses

R continues a line that ends in `+`. Python ends the statement there, so a sentence spanning several lines is wrapped in parentheses:

```
(data(gapminder_2007) + point + x(col.gdp, scale='log') + y(col.life)  
  + color(col.continent) + size(col.population)  
  + title('Wealth and life expectancy, 2007'))
```

1K 10K 40 50 60 70 80 Wealth and life expectancy, 2007 Life Gdp Continent Asia Europe
Africa Americas Oceania Population 199.6K 659.4M 1.3B

“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by continent, size by population.”

That is Python’s rule about lines rather than anything of gog’s. With the accessor, it covers almost everything you will type differently.

47.3 The table

A table is a dict of columns, so a first plot needs nothing installed:

```
data({'level': ['Low', 'Medium', 'High'], 'count': [30.0, 20.0, 10.0]},  
     name='small') + bar + x(col.level) + y(col.count)
```

Low Medium High 0 10 20 30 Count Level

“Given the small table: bars, x is level, y is count.”

Anything carrying `.columns` and `df[name]` is also a table, which covers `pandas` and `polars`. Neither is imported, so neither is a dependency of the binding, and you read a file the way you already do. `GAPMINDER_CSV` here is the path to the book’s `gapminder_2007.csv`, which [The book’s data](#) says where to get:

```
import pandas as pd

gapminder_2007 = pd.read_csv(GAPMINDER_CSV)
print(gapminder_2007.shape, list(gapminder_2007.columns))
```

```
(142, 6) ['country', 'continent', 'year', 'life', 'population', 'gdp']
```

Every plot on this page reads that table. Types convert the way you would expect. Numbers become positions, text becomes categories, and `datetime64`, `date` and `datetime` all become a calendar axis. `NaN` and `None` are missing values: the engine leaves out the whole row and says how many it left out. A `pandas Categorical` carries its declared order across, which is the counterpart of R’s `factor(x, levels = ...)`. A plain dict of lists declares the same order with `ordered(values, levels)`, which [Data](#) describes.

47.4 Computing a column first

A channel takes a column name and never an expression. This is true in all four languages, and `pandas` is where a Python reader meets it. There is no `x(log(col.gdp))`, and `gog` has no small language for deriving values, because Python is already one: compute the column, then bind it by name.

```
europe = gapminder_2007[gapminder_2007.continent == 'Europe'].copy()
europe['gdp_thousands'] = europe.gdp / 1000
europe = europe.sort_values('life', ascending=False)
print(europe[['country', 'gdp_thousands',
↪  'life']].head(3).to_string(index=False))
```

```
country  gdp_thousands  life
Iceland      36.180789  81.757
Switzerland  37.506419  81.701
Spain        28.821064  80.941
```


That result is a table like any other, and the sentence over it is the sentence you already know:

```
(data(europe) + point + x(col.gdp_thousands) + y(col.life)
+ size(col.population) + style(color='steelblue')
+ x_label('GDP per capita (thousands)')
+ title('Europe, 2007'))
```

10 20 30 40 50 72 74 76 78 80 82 Europe, 2007 Life GDP per capita (thousands) Population
301.9K 41.4M 82.4M

The division stayed in `pandas`, where you can see it, and the plot names a column that exists. That split is deliberate rather than a missing feature. A plot is a mapping from a table ([Data](#)), so a channel names one of its columns, and anything you would rather compute is computed first.

47.5 Showing the plot

In a Jupyter notebook a plot displays itself, as it does in the R console. A script displays nothing on its own, so say where the plot goes:

```
plot = data(gapminder_2007) + point + x(col.gdp) + y(col.life)
print(plot)
print(plot.save('/tmp/life.svg'))
```

```
<gog plot: point on gapminder_2007>
/tmp/life.svg
```

Printing a plot gives one line naming its mark and its table, not the picture. `save()` writes the SVG, `show()` opens it in a browser, and `render()` returns it as a string. R's `print()` opens the viewer without being asked; Python waits to be told, which is the convention each language already has.

47.6 Six names Python already uses

`bin`, `map`, `max`, `min`, `range` and `sum` are `gog`'s words here and builtins there, so `from gog import *` puts `gog`'s over Python's for the rest of the file. The R package does the same to base R, with `max`, `min`, `range` and `sum` among them: a grammar keeps its own vocabulary. Calling five of the six says so, and names the fix. Only `bin` is silent: `bin(5)` returns a transform where Python would have returned a string.

```
try:
    range(10)
except GogError as error:
    print(error)
```

gog: ``range()`` takes the band's two ends, each one number between 0 and 1,
 ↪ e.g. ``range(0.25, 0.75)``. For a sequence of integers, gog shadows that
 ↪ name: use ``builtins.range`` for Python's.

The worst of the six is `map`, because it is the only one whose error does not mention gog. Python reports the wrong number of arguments and names nothing you can act on.

A module that needs both can `import gog` and write `gog.point`, or take the builtin back with `from builtins import sum`.

47.7 What does not change

The four operators, including how tightly they bind. `*` groups before `+`, `+` before `|`, and `/` sits beside `*`. Those are Python's own precedences and R's alike, so a derived layer and a crossed facet group exactly as they do in [Operators](#):

```
(data(gapminder_2007) + bar * mean + x(col.continent) + y(col.life)
 + order(col.life, desc=True)
 + title('Mean life expectancy, highest first'))
```

Oceania Europe Americas Asia Africa 0 20 40 60 80 Mean life expectancy, highest first Life
Continent

“Given gapminder 2007: bars derived by mean, x is continent, y is life, ordered by life, largest first.”

The vocabulary does not change either: every mark, transform, channel, setting and plot-level atom in [the kernel card](#) exists in Python under the same name. Neither does an engine refusal, which reaches you word for word. A refusal the binding raises is the one that differs, because it teaches Python's own spelling:

```
data(gapminder_2007) + bar + x(col.continent)
```

gog: ``bar`` needs ``y()`` but none is set. Add ``y(<column>)`` - a bar cannot be
 ↪ drawn without it.

gog: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

The distance between the two spellings is measured rather than argued. Every sentence in this book was translated by rule and run through the Python binding: 632 drew a *byte-identical* plot, 166 earned a word-identical refusal, and 19 differed only in the example inside the message, where each binding teaches its own syntax. Eleven sentences do not carry over at all. Nine are in [R](#), about R-only spellings, and two are the GIF exports in [Play](#). Two rewrites account for nearly all the rest: the accessor, 812 times, and the parentheses, 554.

47.8 Reading from a database

Python has a database standard, and `query()` speaks it. It also speaks Spark, because that is where a large share of Python's data is stored.

The standard is DB-API, also called PEP 249. Anything with a `.cursor()` method works: the built-in `sqlite3`, `DuckDB`, `psycpg`, `pyodbc`, Snowflake's connector, and the Databricks SQL connector.

```
import sqlite3
from gog import query, bar, x, y, col

con = sqlite3.connect("sales.db")
plot = query(con, "SELECT status, revenue FROM orders") + bar + x(col.status)
    ↪ + y(col.revenue)
plot.save("orders.svg")
```

A Spark session works too. Hand `query()` the session itself, and the table name is whatever your catalog calls it. This is the route to a Databricks table:

```
plot = query(spark, "SELECT status, revenue FROM main.sales.orders") + bar +
    ↪ x(col.status) + y(col.revenue)
```

In a Databricks notebook the session is already there, under the name `spark`. `gog` installs from PyPI with its engine included, so `%pip install gog` is the only setup step.

Neither route adds a dependency to `gog`. You bring the connection; `gog` reads rows from it.

47.9 Getting it

A plot is drawn by the engine, which is a compiled Rust binary, so the Python package has to be able to find one. The released wheel carries it: one wheel per platform, each holding the engine built for that platform, so installing needs no Rust toolchain.

```
pip install gog
```

That command works today. `gog` is on [PyPI](#). Five wheels cover macOS and Linux on both common processor families, and Windows on Intel. The other three bindings install from their own registries, and the [preface](#) lists all four commands.

To work from a checkout instead, build the engine once and import the package from where it sits:

```
cargo build --release -p gog-cli
```

```
import sys
sys.path.insert(0, "py-pkg/gog")

from gog import *
```

The engine is looked for in four places, in this order: the `GOG_CLI_PATH` environment variable, the copy inside the installed package, `gog-cli` on your `PATH`, and a local `target/release` build. The first three are how an installed copy works. This page was rendered through the first: the book points `GOG_CLI_PATH` at a local build before any Python runs.

48 Julia

The grammar does not change when the language does. One engine reads one specification, so a sentence means the same thing no matter who wrote it, and everything in this book up to here is as true in Julia as it is in R.

Of the four languages, Julia is the one that changes least about the sentence you already know. Like R and Python, it gives `+`, `*`, `|` and `/` a meaning of their own, and like R it keeps a column apart from a value in the syntax: a symbol is not a string. So a sentence differs from the R above it by one colon:

```
data(gm) + bar * bin + x(life) | facet(era)
```

```
data(gm) + bar * bin + x(:life) | facet(:era)
```

Every Julia block with a plot under it was executed, through the same engine every other chapter uses.

48.1 A column is a symbol

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) +  
  ↪ color(:continent)
```

1K 10K 40 50 60 70 80 Life Gdp Continent Asia Europe Africa Americas Oceania

“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by continent.”

That is the sentence from [A first plot](#), with a colon in front of each column. In this grammar a plain string spells a *value*, as in `style(color = "tomato")` or `palette("okabe")`. A symbol is visibly not a string, so the colon does the same work as R’s bare name and Python’s `col.` accessor. The grammar’s oldest rule, that a channel takes a column and never a value, is therefore in the syntax rather than in a check.

So a string where a column belongs is refused, and told what to write:

```
data(gapminder_2007) + point + x("gdp") + y(:life)
```

gog: ``x("gdp")`` binds a **value**, and a channel takes a **column**. In Julia a
 ↪ column is a symbol, which is what keeps the two apart: ``x(:gdp)`` maps the
 ↪ column called ``gdp``.

The mistake in the other direction is refused the same way, and for the same reason ([Setting vs mapping](#)):

```
data(gapminder_2007) + point + x(:gdp) + y(:life) + style(color = :continent)
```

gog: ``style(color = )`` fixes one value for the whole layer, and ``:continent``
 ↪ is a column. To **map** it - one value per category, with a legend to
 ↪ decode it - that is a channel: ``color(:continent)``.

A column whose name is not a Julia identifier is written `Symbol("life exp")`. A name in another writing system needs no such thing: `:지역` (*jiyeok*, region) is an ordinary symbol, because Julia identifiers are Unicode.

48.2 The operators, and their precedence

Julia's precedence table is not R's. R puts `|` below `+`; Julia puts it in the *addition* tier, alongside `+`. That could have broken every faceted sentence in this book, so it was measured rather than assumed:

Written	Julia builds	R builds
<code>a + b + c \ d</code>	<code>(a + b + c) \ d</code>	the same
<code>a \ b / c</code>	<code>a \ (b / c)</code>	the same
<code>a * b + c</code>	<code>(a * b) + c</code>	the same
<code>a \ b + c</code>	<code>(a \ b) + c</code>	<code>a \ (b + c)</code>

Only the last row differs, and it is the shape where something is written *after* a facet. No sentence in this book has it, and R refuses that shape anyway, so every sentence this book teaches parses identically in both languages. The first row is safe because Julia reads operators of one tier from left to right: everything left of the `|` is joined before the `|` is reached.

Faceting therefore reads exactly as it does in R:

```
data(gapminder_2007) + point + x(:gdp, scale = "log") + y(:life) |  
  ↪ facet(:continent)
```

1K 10K 40 50 60 70 80 1K 10K 1K 10K 1K 10K 1K 10K Asia Europe Africa Americas Oceania
Life Gdp

“Given gapminder 2007: points, x is gdp on a log scale, y is life, split into panel columns by continent.”

And so does a transform, with `*` binding tighter than `+`:

```
data(gapminder_2007) + bar * mean + x(:continent) + y(:life)
```

Asia Europe Africa Americas Oceania 0 20 40 60 80 Life Continent

“Given gapminder 2007: bars derived by mean, x is continent, y is life.”

48.3 Nine names Julia already uses

Nine of the kernel’s words are also names in `Base`: `count`, `sum`, `min`, `max`, `range`, `size`, `step`, `stack` and `map`. Julia will not silently choose between two modules that export the same name, and every Julia programmer has met that rule before. Writing one of them without the import gives an `UndefVarError` with a hint, rather than the wrong function, which is the good outcome, but it does mean a second import line:

```
println(bar * bin)
println(Base.sum([1, 2, 3]))
```

```
<gog bar>
6
```

Those two lines open every block on this page:

```
using GrammarOfGraphics
using GrammarOfGraphics: count, sum, min, max, range, size, step, stack, map
```

`Base.sum` is still there, as the second line of output shows. `bin` is not in the import line above, because `Base.bin` is not exported and so there is no second `bin` to choose between.

The import line treats the nine alike, and one of them costs more than the others. `map` is the one most likely to break code you already wrote. `Base.map` is everyday Julia. `Base.step` and `Base.stack` are not. A script that called `map` before loading this package will stop working, and the error will name a line that did not change.

Every language meets this problem, and each one asks you to solve it differently: R’s `range`, `sum` and `min` belong to base R, Python’s `bin`, `sum` and `max` are builtins, and Julia’s belong to

Base. A grammar keeps its own vocabulary; what changes is how each language asks you to say which one you meant.

48.4 The table

A table is a named tuple of columns, so a first plot needs nothing installed:

```
data((person = ["Ada", "Alan", "Grace"], cm = [160.0, 172.0, 181.0]),
     name = "heights") + bar + x(:person) + y(:cm)
```

Ada Alan Grace 0 50 100 150 Cm Person

A `Dict` works too, and so does anything that answers `names` and `[!, col]`, which is what a `DataFrame` does. `DataFrames` is never imported, so a reader who does not use one installs nothing for it. `gog_table("gapminder_2007")` fetches any of the book's shared tables, as it does in the other three languages.

Julia has both a `Date` and a `DateTime`, as R and Python do. A `Date` column gets day ticks, and a `DateTime` column keeps its hours and minutes. The engine reads the unit off the type, so nothing is guessed.

R reads a table's name off the expression you wrote, and Julia cannot. An unnamed table gets a generated name instead, such as `data2`. Write `data(df, name = "notes")` when a message should say notes ([Data](#)).

48.5 Reading from a database

`query()` takes a `DBInterface.jl` connection. That is Julia's database standard, and it covers `SQLite.jl`, `LibPQ.jl`, `MySQL.jl` and `DuckDB.jl`.

```
using DBInterface, SQLite, GrammarOfGraphics

con = SQLite.DB("sales.db")
query(con, "SELECT status, revenue FROM orders") + bar + x(:status) +
  ↪ y(:revenue)
```

One detail is specific to this binding. `GrammarOfGraphics` does not depend on `DBInterface`. Every dependency the package declares is a Julia standard library, and a database package would be the first that is not, installed for every reader although many never write SQL. So the package looks `DBInterface` up in your session instead.

The practical consequence is one line: you must write `using DBInterface` yourself. Without it, `query()` names the missing package and the line that fixes it, rather than failing later with a message about something else.

48.6 Getting it

The Julia package is named `GrammarOfGraphics`. The name is longer than the one R and Python use, and the registry is the reason. Julia requires a package name of at least five letters, in capitalized words, so a lowercase `gog` cannot be registered.

Installing takes two commands here, and the engine comes first. A plot is drawn by the engine, which is a compiled Rust binary. The Python and JavaScript packages carry an engine built for your computer, and the R package carries one on macOS and Windows. The Julia package carries none, so this is the binding that asks you to install it:

```
cargo install --git https://github.com/psychometrician/gog gog-cli
```

That command needs Rust, which `rustup` installs in one step. `cargo install` then puts `gog-cli` on your `PATH`, which is one of the places the package looks. If you started Julia from an editor or a notebook, that session may not see your `PATH`, so set `ENV["GOG_CLI_PATH"]` to the binary instead. Then the package:

```
using Pkg
Pkg.add("GrammarOfGraphics")
```

The order is advice rather than a rule. The package loads without an engine and looks for one only when it draws, so the other order works as well. Installing the engine first means you never meet the message that says the engine is missing.

To work from a checkout instead, add the package from the path where it sits:

```
using Pkg
Pkg.develop(path = "jl-pkg/GrammarOfGraphics")

using GrammarOfGraphics
```

The engine is looked for in four places, in this order: the `GOG_CLI_PATH` environment variable, the copy inside the installed package, `gog-cli` on your `PATH`, and a local `target/release` build. The Julia package carries no engine of its own, so the third and the fourth places are the ones that find it.

`render_svg(plot)` returns the SVG as a string, and a plot shown in a notebook draws itself, because the binding hands the notebook a picture when it asks for one.

49 JavaScript

The grammar does not change when the language does. One engine reads one specification, so a sentence means the same thing no matter who wrote it, and everything in this book up to here is as true in JavaScript as it is in R.

The *sentence*, though, is spelled differently here, and the operators are why. Every other binding writes the same words joined by the same four operators. JavaScript cannot join them, because `+`, `*`, `|` and `/` cannot be given a meaning on objects: `a + b` glues two objects into text, `a * b` is NaN, and `a | b` is a bitwise integer. There is no version of the language in which `data(gm) + point` builds a plot.

So JavaScript spells the four operators as four words. That is the change this chapter starts from, and it is worth reading the two lines side by side first:

```
data(gm) + bar * bin + x(life) | facet(era)
```

```
plot(data(gm), layer(bar, bin), x(col.life), across(col.era))
```

Every JavaScript block with a plot under it was executed, through the same engine every other chapter uses.

49.1 The comma is +

A gog sentence is a list of words, and in JavaScript it is a list of arguments. `plot()` is what gives the comma its meaning. A named argument such as `scale = "log"` gathers into an object at the end of the call, which the section on named arguments below sets out in full:

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: 'log' }), y(col.life),  
  ↪ color(col.continent))
```

1K 10K 40 50 60 70 80 Life Gdp Continent Asia Europe Africa Americas Oceania

“Given gapminder 2007: points, x is gdp on a log scale, y is life, color by continent.”

That is [A first plot](#) exactly: the same words, in the same order, reading the same way. Position still decides scope ([Encoding scope](#)), so a channel written after a mark belongs to that mark and one written before belongs to the plot. The commas change nothing about that; they are the + signs.

49.2 layer() is *

In R, * binds tighter than +, which is how `bar * bin + x(life)` reads as a binned bar rather than a bar plus a bin. JavaScript has no precedence to use here, so the tighter binding becomes nesting:

```
plot(data(gapminder_2007), layer(bar, mean), x(col.continent), y(col.life))
```

Asia Europe Africa Americas Oceania 0 20 40 60 80 Life Continent

“Given gapminder 2007: bars derived by mean, x is continent, y is life.”

Nesting is the clearer of the two. A reader who does not already know that * binds tighter than + has to be told; a reader looking at `layer(bar, mean)` can see it.

Most sentences need no `layer()` at all. Just over half the sentences in this book are a flat list of words, and the ones that do use it usually hold two:

```
plot(data(gapminder_2007), layer(bar, count), x(col.continent), style({  
  ↪ color: 'steelblue' })))
```

Asia Europe Africa Americas Oceania 0 20 40 Count Continent

“Given gapminder 2007: bars derived by count, x is continent.”

49.3 across() and down() are | and /

Faceting in R is `plot | facet(era)`, panels side by side, or `plot / facet(era)`, panels stacked. The | and the `facet()` were always one phrase, because `facet()` means nothing without one of them, so JavaScript writes the phrase as one word:

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: 'log' }), y(col.life),  
  ↪ across(col.continent))
```

1K 10K 40 50 60 70 80 1K 10K 1K 10K 1K 10K 1K 10K Asia Europe Africa Americas Oceania
Life Gdp

“Given *gapminder 2007*: points, *x* is gdp on a log scale, *y* is life, split into panel columns by continent.”

`down()` stacks them instead, and writing both crosses the two into a grid, which is what `plot | facet(a) / facet(b)` does in R.

Because the pair is one word here, there is no separate `facet()` to write. Writing `facet()` from memory gets a refusal that names the word to use instead:

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),  
  ↪ facet(col.continent))
```

gog: JavaScript has no ``facet()``, because it has no ``|`` or ``/`` to join it

↪ with `-` the pair is one word here. Panels side by side:

↪ ``across(col.continent)``. Panels stacked: ``down(col.continent)``.

49.4 Composing two plots: `beside()` and `below()`

R’s `|` means two things, and what is on its right decides which: `plot | facet(g)` splits one plot into panels, `plot_a | plot_b` arranges two plots on a page. With no operator to overload there is nothing to interpret, so the second meaning gets its own word: `beside()`, and `below()` for `/`:

```
below(plot(data(gapminder_2007), layer(bar, bin), x(col.gdp), theme({ height:  
  ↪ 130 })), plot(data(gapminder_2007), point, x(col.gdp), y(col.life)))
```

0 20 40 60 Count

0K 20K 40K 40 50 60 70 80 Life Gdp

“Given *gapminder 2007*: bars derived by *bin*, *x* is gdp, above points, *x* is gdp, *y* is life.”

The two words come from an older algebra: Henderson’s *Functional Geometry* (Henderson, 1982) combines pictures with **beside** and **above**, and that is the picture language SICP teaches ([Composition](#) gives its full name). Both take pages as well as plots, so `below(top, beside(main, right))` is the marginal plot, exactly as `top / (main | right)` is in R. What the composed plots share is decided by the engine and not by the spelling: the same column on the same axis in two of them is one axis. See [Composition](#).

49.5 A column is written col.gdp

R writes a column as a bare name, Julia as a symbol, Python and JavaScript through a small accessor. The two languages with no bare names give the same answer, and they give it for the same reason: in this grammar a plain string is how you spell a *value*, as in `style({ color: "tomato" })`, `title("...")`, `palette("okabe")`. Without the accessor a column and a value would look identical, and the grammar's oldest rule (a channel takes a column, never a value) would be invisible in the two languages that most need it stated.

So a string where a column belongs is refused, and told what to write:

```
plot(data(gapminder_2007), point, x('gdp'), y(col.life))
```

gog: ``x("gdp")`` binds a **value**, and a channel takes a **column**. JavaScript
↪ has no bare names, so a column is written with the accessor: ``x(col.gdp)``
↪ maps the column called ``gdp``.

The message names the spelling to write, because a string here is a *value* and a channel takes a *column* ([Setting vs mapping](#)). The mistake in the other direction is refused the same way:

```
plot(data(gapminder_2007), point, x(col.gdp), y(col.life), style({ color:
↪ col.continent })))
```

gog: ``style({ color: ... })`` fixes one value for the whole layer, and
↪ ``col.continent`` is a column. To **map** it - one value per category, with a
↪ legend to decode it - that is a channel: ``color(col.continent)``.

A column whose name is not a JavaScript identifier, because it holds a space or a dot, takes the bracket form instead: `col["life exp"]`. Names in other writing systems need no such thing: `col.지역` (*jiyeok*, region) is an ordinary identifier.

49.6 A named argument is one trailing object

R writes `x(gdp, scale = "log")`. JavaScript has no syntax for a named argument, so the named ones gather into a single object at the end while the positional ones stay where they are:

```
plot(data(gapminder_2007), point, x(col.gdp, { scale: 'log', base: 2 }),
↪ y(col.life), title('Doublings of income'))
```

256 512 1024 2048 4096 8192 16384 32768 40 50 60 70 80 Doublings of income Life Gdp

“Given gapminder 2007: points, x is gdp on a log scale in base 2, y is life.”

`style()` is the case where every argument is named, so it is written as one object throughout the book: `style({ color: "tomato", size: 3 })`.

49.7 The table, and its name

A table is an object of columns, `{ person: ['Ada', 'Alan', 'Grace'], cm: [160, 172, 181] }`, so a first plot needs nothing installed:

```
plot(data(heights), bar, x(col.person), y(col.cm))
```

Ada Alan Grace 0 50 100 150 Cm Person

R reads the table’s *name* off the expression you wrote, and Python off the variable you passed. JavaScript can do neither, and the name matters: a layer resolves its columns against the nearest table by that name ([Data](#)). So an unnamed table is given a unique one, which is enough for two tables in one sentence to stay apart, and `data(df, { name: "notes" })` is there for when a message should say `notes` rather than `data2`.

49.8 A sub-expression is a value

This spelling makes one law easy to see. A piece of a sentence is an ordinary JavaScript value, so it can be named, reused, and passed around, and it means the same thing everywhere it lands:

```
const piled = layer(point, bin, stack);
console.log(String(plot(data(gapminder_2007), piled, x(col.life))));
```

<gog plot: point on data>

That law is Compositional Invariance, the sixth of the [nine laws](#), and JavaScript keeps it with no help from the binding.

49.9 Reading from a database

Node.js, where JavaScript runs outside a browser, has no database standard, so `query()` accepts a connection by the shape of its methods rather than by its type. A connection that can prepare a statement and return all its rows works:

```
import { DatabaseSync } from "node:sqlite";
import { plot, data, query, bar, x, y, col, render_svg } from
  ↪ "grammar-of-graphics";

const con = new DatabaseSync("sales.db");
render_svg(plot(query(con, "SELECT status, revenue FROM orders"),
  bar, x(col.status), y(col.revenue)));
```

`node:sqlite` is built into Node.js, behind a flag from version 22 and without one from 23.4, so that example needs nothing installed. `better-sqlite3` has the same shape and works the same way.

49.9.1 Asynchronous drivers are refused

`render_svg()` is synchronous. It returns the SVG rather than a Promise, which is what lets a plot be written inline in an expression. A driver whose query returns a Promise cannot be awaited inside it.

So `pg` and `mysql2` do not work with `query()`, and `gog` says so rather than letting a Promise reach the engine. The direction it gives is the fix: fetch the rows yourself, then bind them as an ordinary table.

```
const { rows } = await con.query("SELECT status, revenue FROM orders");
render_svg(plot(data(rows), bar, x(col.status), y(col.revenue)));
```

That works with every driver, asynchronous ones included. `data()` takes the rows an async driver returns, and the sentence after it is unchanged. This is the only place where a language rule changes what a sentence can *do* rather than how it is spelled, and it costs one `await`.

49.10 Getting it

A plot is drawn by the engine, which is a compiled Rust binary, so the npm package has to be able to find one.


```
npm install grammar-of-graphics
```

That command works today. The package is on [npm](#), and the name is spelled out because npm already had a package called `gog`. The registry also forbids capital letters in a new name, so this is Julia's `GrammarOfGraphics` written the way npm writes names.

The engine arrives with it. There are five more packages on npm, one per platform, each holding the engine compiled for that platform, and the main package lists all five as optional dependencies. Installing takes the one that matches your computer and skips the other four. So there is nothing to put on your `PATH`, nothing to build, and no Rust toolchain to install.

The binding itself only builds the specification and hands it to that engine, and it depends on nothing else:

```
import { plot, data, point, x, y, col, render_svg } from
  ↪ "grammar-of-graphics";

const gm = { gdp: [1000, 2000, 3000], life: [60, 70, 80] };
console.log(render_svg(plot(data(gm), point, x(col.gdp), y(col.life))));
```

`render_svg()` returns the SVG as a string, which in a browser goes straight into the DOM and on a server goes straight into a response. Nothing has to open a viewer window and nothing has to become pixels first, which is where JavaScript sits closest to the engine.

To work from a checkout instead, build the engine once and import the package from where it sits:

```
cargo build --release -p gog-cli
```

```
import { plot, data, point, x, y, col, render_svg } from
  ↪ "../js-pkg/gog/src/index.js";
```

The engine is looked for in four places, in this order: the `GOG_CLI_PATH` environment variable, the platform package installed beside this one, `gog-cli` on your `PATH`, and a `target/release` build in a surrounding checkout. The second and third are how an installed copy works; `GOG_CLI_PATH` is how this page was rendered.

49.11 A plot for a web page: `html_block()`

The string `render_svg()` returns is the drawing and nothing else. For a web page, `html_block()` returns the same drawing with everything around it: the line of controls under the picture, and, for some plots, a copy of the engine compiled for the browser, which redraws them. A plot in the cube, a plot on the globe, a turned network, and a plot with a brush all carry that engine. Those are the plots of [Space](#), [Globe](#), [Network](#) and [Selection](#).

```
const cube = plot(data(gapminder_2007), point,  
                  x(col.gdp), y(col.life), z(col.population));  
console.log(html_block(cube).split("\n")[0]);
```

```
<div class="gog-plot" id="gog-00007g442g" style="text-align:center;">
```

The function returns HTML as a string. Put it in a page, a notebook cell, or a dashboard panel. The picture is written first, so it is on the page before that copy of the engine loads. A page whose security policy forbids WebAssembly, the format that copy ships in, still shows the plot. What it loses is the dragging: the cube and the globe stop turning, and the brush stops moving.

R, Python and Julia never call this function. A notebook in those languages asks the plot object how it would be displayed, and each binding answers: R with `repr_html`, Python with `_repr_html_`, Julia with a `show` method. JavaScript has no such request to answer, so in JavaScript it is a function you call.

Afterword

The 1446 book closed with a postface making two claims for its little alphabet: that a wise man could learn it before the morning was over, and a slow one within ten days, and that it could write down any sound at all: wind, crane, cock, dog. This book closes by restating both, because they are its own measures of success.

The ten-day challenge

You have finished the morning; here is the rest of the promise. For the next ten days, bring one question a day from *your own data*, not the cast's, and answer it in one sentence. Start where the cookbook starts: what shape is the data, and what do you want to know? By day three the recipes will feel like spellings of things you already say. By day ten you should be writing sentences this book never showed you, which is the only outcome that would prove the grammar, rather than the examples, is what you learned.

If a sentence surprises you (renders when you expected a refusal, or refuses with a direction you disagree with), that is worth reporting. A grammar this small stays correct only when readers argue with it.

Growth without new words

The kernel you learned is the whole vocabulary, and nothing in it is waiting to be drawn. Every word draws. Every mark, every space, every channel, and the one selection.

Growth arrives as new pictures rather than new words. The **choropleth** is the pattern: no atom was added for it. It shades each country by a value, and what it needed was geometry. **zone** finds its sides five ways, and a **boundary** is one of them, drawn by the data itself. It cost no new word, because **group** already says which rows belong together. [Map](#) draws it with the words the kernel already had.

The same sentence you write becomes a small, engine-neutral specification inside the package. Every binding speaks it, with the same words and its own way of naming a column. Each has a short chapter of its own, a difference list: [R](#), [Python](#), [Julia](#), [JavaScript](#).

```
# Python
(data(gapminder_2007) + point + x(col.gdp) + y(col.life)
 + color(col.continent))
```

```
# Julia
data(gapminder_2007) + point + x(:gdp) + y(:life) + color(:continent)
```

```
// JavaScript, the one binding that cannot overload the operators
plot(data(gapminder_2007), point, x(col.gdp), y(col.life),
 ↪ color(col.continent))
```

One goal left

One goal is bigger than the rest, and half of it is already here. The engine runs inside the page: that is how a 3-D plot turns under your mouse in the web edition. What is left is the other half, written down here so you can check later whether the project did it: every example in this book editable in place. Change `color(continent)` to `shape(continent)` and watch the plot redraw. The book you are reading is already ready for that change, because every example in it is a live specification.

Writing quickly is not writing well

I think Hangeul (한글) is a perfect writing system. I should admit that I may be biased, because it is the alphabet I write with. What cannot be argued is that it is fast to read and fast to write.

Writing quickly is not the same as writing well. There is no shortcut to good writing: you read good writing, and you practice. Visualization is no different. This tool is new and it has problems, and what I hope it can be for now is something worth practicing on.

King Sejong made the alphabet. He did not write the books that were written with it. I am not a great author of plots either. What I do have is twenty years in this field as a user, watching and listening and reading. That is what I built from: a tool that is easy to learn and easy to use.

I hope good work comes out of it.

The gap

A specification is a sentence said before the picture exists. Everyone who uses this grammar lives in the moment between declaring what they want to see and finding out whether the engine agrees. The package is **gog**. The state of working with it is *agog*. We hope the book kept you there.

Be agog. Use gog.

References

- Becker, R. A., Cleveland, W. S., & Shyu, M.-J. (1996). The visual design and control of trellis display. *Journal of Computational and Graphical Statistics*, 5(2), 123–155. <https://doi.org/10.1080/10618600.1996.10474701>
- Bertin, J. (1967). *Sémiologie graphique: Les diagrammes, les réseaux, les cartes*. Gauthier-Villars.
- Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1), 65–98. <https://doi.org/10.1137/141000671>
- Bruls, M., Huizing, K., & van Wijk, J. J. (2000). Squarified treemaps. In W. C. de Leeuw & R. van Liere (Eds.), *Data visualization 2000: Proceedings of the joint Eurographics and IEEE TCVG symposium on visualization* (pp. 33–42). Springer. https://doi.org/10.1007/978-3-7091-6783-0_4
- Bryan, J. (2025). *gapminder: Data from Gapminder*. R package version 1.0.1, published 12 June 2025. <https://CRAN.R-project.org/package=gapminder>
- Byron, L., & Wattenberg, M. (2008). Stacked graphs – geometry & aesthetics. *IEEE Transactions on Visualization and Computer Graphics*, 14(6), 1245–1252. <https://doi.org/10.1109/TVCG.2008.166>
- Carr, D. B., Littlefield, R. J., Nicholson, W. L., & Littlefield, J. S. (1987). Scatterplot matrix techniques for large N. *Journal of the American Statistical Association*, 82(398), 424–436. <https://doi.org/10.1080/01621459.1987.10478445>
- Cleveland, W. S. (1979). Robust locally weighted regression and smoothing scatterplots. *Journal of the American Statistical Association*, 74(368), 829–836. <https://doi.org/10.1080/01621459.1979.10481038>
- Ecma International. (2025). *ECMAScript 2025 language specification* (16th ed.). Ecma International. <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>
- Friendly, M., & Denis, D. (2005). The early origins and development of the scatterplot. *Journal of the History of the Behavioral Sciences*, 41(2), 103–130.
- Havre, S., Hetzler, B., & Nowell, L. (2000). ThemeRiver: Visualizing theme changes over time. *Proceedings of the IEEE Symposium on Information Visualization 2000*, 115–123. <https://doi.org/10.1109/INFVIS.2000.885098>
- Henderson, P. (1982). Functional geometry. In D. P. Friedman & D. S. Wise (Eds.), *Conference record of the 1982 ACM symposium on lisp and functional programming* (pp. 179–187). ACM Press.

- Hintze, J. L., & Nelson, R. D. (1998). Violin plots: A box plot–density trace synergism. *The American Statistician*, 52(2), 181–184. <https://doi.org/10.1080/00031305.1998.10480559>
- Kaplan, E. L., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 457–481. <https://doi.org/10.1080/01621459.1958.10501452>
- King James Bible*. (1769). Oxford University Press.
- Klabnik, S., & Nichols, C. (2023). *The Rust programming language* (2nd ed.). No Starch Press. <https://www.rust-lang.org/>
- Munzner, T. (2014). *Visualization analysis and design*. CRC Press.
- Nightingale, F. (1858). *Notes on matters affecting the health, efficiency, and hospital administration of the british army*.
- Okabe, M., & Ito, K. (2008). *Color universal design (CUD): How to make figures and presentations that are friendly to colorblind people*. J*FLY Data Depository for Drosophila Researchers. <https://jfly.uni-koeln.de/color/>
- Ooms, J. (2014). The jsonlite package: A practical and consistent mapping between JSON data and R objects. *arXiv:1403.2805 [Stat.CO]*. <https://arxiv.org/abs/1403.2805>
- Pearson, K. (1895). Contributions to the mathematical theory of evolution. II. Skew variation in homogeneous material. *Philosophical Transactions of the Royal Society of London A*, 186, 343–414. <https://doi.org/10.1098/rsta.1895.0010>
- Pedersen, T. L. (2019). *patchwork: The composer of plots*. R package; first CRAN release 1 December 2019. <https://patchwork.data-imaginist.com/>
- Playfair, W. (1786). *The commercial and political atlas*.
- Playfair, W. (1801). *The statistical breviary*.
- Posit Software, PBC. (2026). *Ggsql: A SQL extension for declarative data visualisation based on the grammar of graphics* (Version 0.3.0 (alpha)). <https://ggsql.org/>
- R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Sampson, G. (1985). *Writing systems: A linguistic introduction*. Stanford University Press.
- Sarkar, D. (2008). *Lattice: Multivariate data visualization with R*. Springer. <https://doi.org/10.1007/978-0-387-75969-2>
- Sarkar, D. (2026). *lattice: Trellis graphics for R*. R package version 0.22-9, published 9 February 2026. <https://CRAN.R-project.org/package=lattice>
- Sejong the Great and the scholars of the Hall of Worthies. (1446). 훈민정음 (*hunminjeongeum*): *The correct sounds for the instruction of the people, with the haerye commentary*.
- Shneiderman, B. (1992). Tree visualization with tree-maps: 2-d space-filling approach. *ACM Transactions on Graphics*, 11(1), 92–99.
- Silverman, B. W. (1986). *Density estimation for statistics and data analysis*. Chapman; Hall.
- Spear, M. E. (1952). *Charting statistics*. McGraw-Hill.
- Sturges, H. A. (1926). The choice of a class interval. *Journal of the American Statistical Association*, 21(153), 65–66. <https://doi.org/10.1080/01621459.1926.10502161>
- Tufte, E. R. (1990). *Envisioning information*. Graphics Press.
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace. <https://www.python.org/doc/essential/reference-manual/>

- [//www.python.org/](https://www.python.org/)
- Wickham, H. (2008). *Practical tools for exploring data and models* [PhD thesis, Iowa State University]. <https://dr.lib.iastate.edu/entities/publication/9210139e-102e-4537-8b6e-7d8197e2f3d8>
- Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-319-24277-4>
- Wickham, H. (2018). *Answer on why ggplot2 uses + rather than the pipe*. Posit (RStudio) Community forum, 29 January 2018.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science: Import, tidy, transform, visualize, and model data* (2nd ed.). O'Reilly Media. <https://r4ds.hadley.nz/>
- Wickham, H., Chang, W., Henry, L., Pedersen, T. L., Takahashi, K., Wilke, C., Woo, K., Yutani, H., Dunnington, D., & Brand, T. van den. (2026). *ggplot2: Create elegant data visualisations using the grammar of graphics*. R package version 4.0.3, published 22 April 2026. <https://CRAN.R-project.org/package=ggplot2>
- Wilkinson, L. (1999). Dot plots. *The American Statistician*, 53(3), 276–281. <https://doi.org/10.1080/00031305.1999.10474474>
- Wilkinson, L. (2005). *The grammar of graphics* (2nd ed.). Springer. <https://doi.org/10.1007/0-387-28695-0>

A The kernel card

The whole grammar, on one page. The functions around it are in [Functions around the grammar](#). This card is extracted live from [The kernel](#) as the book builds, so it cannot disagree with the chapter it comes from.

Kind	The words
Sources	data query
Marks	point line area bar step interval box ribbon text path rule zone surface edge
Channels	x y z color size shape pattern opacity group label play
Transforms	bin smooth count density proportion sum mean median max min quantile range confidence deviation bounds partition flow layout cluster, and the four collision modifiers, dodge stack jitter repel
Operators	+ layers, * derives, and / arrange
Scales	linear log time category order
Spaces	flat space polar nest map globe network
Settings	style theme palette
Labels	title x_label y_label z_label
Facets	facet
Selections	brush

The minimum sentence is a source, a mark, and the mark's required position channels:

```
data(table) + mark + x(column) + y(column)
```

The order of + is a convention, not a rule: every sentence in this book reads left to right as *data*, *mark*, *positions*, *refinements*. A channel written before any mark is shared by every layer that can accept it; written after a mark, it belongs to that mark alone. Each later `data()` applies to the next mark.

Deriving is what `*` does: it puts a transform on a mark, and the mark is always on the left. Two transforms are written in the order they happen: `bar * bin * mean` cuts the axis into bands, then averages inside each.

Precedence decides what groups first: `*`, then `/`, then `+`, then `|`. A single plot then needs no parentheses; a page of plots does. `data(t) + bar * bin + x(a) | facet(g)` derives the mark, builds the plot, then splits it.

When gog refuses, the message names the rule and the fix. **Illegal** means the grammar forbids it: rewrite. **Unsupported** means the engine does not draw it: write it another way. A warning means `gog` drew the plot and something is worth checking: read it and decide.

B Functions around the grammar

Every word in the kernel belongs to a sentence. `point` is a mark, `x` is a channel, `bin` is a transform, and none of them means anything on its own. They are words, and a word needs a sentence.

The functions in this appendix are not words. They take a plot you have already written and act on it, or they hand you a table to write a plot about. None of them joins a sentence with `+` or `*`, and none of them is part of the grammar.

The name alone will not tell you which is which. `x_label` is a word of the grammar and joins two words with an underscore, exactly as `render_svg` does. What separates them is where each one can stand. A word of the grammar stands inside a sentence, and these three never do. That is why none of them appears in [The kernel card](#).

	What it gives you
<code>render_svg(plot)</code>	The plot as text, in one string
<code>save_gif(plot, path)</code>	A played plot as a file that moves
<code>gog_table(name)</code>	One of this book's example tables

The list is short. Everything on it follows the same rule: it acts on a finished plot, or it hands you a table, and it has no place in a sentence.

B.1 `render_svg()`

A plot is a specification until something draws it. `render_svg()` draws it and gives you the result as ordinary text:

```
picture <- render_svg(data(gapminder_2007) + point + x(gdp) + y(life))
cat(substr(picture, 1, 51), "\n")
```

```
<svg xmlns="http://www.w3.org/2000/svg" width="800"
```

```
cat(nchar(picture), "characters\n")
```

14581 characters

The first line of output shows where that text begins. The second line gives its whole length. So the picture you have been looking at is also this: a run of characters that starts `<svg` and states a width. After that it names every point, every tick and every label.

That text is the whole plot. It is what every notebook, every web page and this book itself receives. Writing it to a file with a `.svg` name gives you a picture that any browser opens and a vector drawing program edits.

The function has the same name in all four languages, and takes the plot as its one argument.

B.1.1 What the file carries

An SVG file is text, so anyone who opens it can read it. If your table holds something private, open the file and read it yourself before you publish. The file carries the drawing and nothing else.

A plot draws the columns you map. Columns you never map are absent from the file, and so are their names:

```
private <- data.frame(
  patient = c("A. Kim", "B. Lee", "C. Park", "D. Choi"),
  email   = c("ak@ex.com", "bl@ex.com", "cp@ex.com", "dc@ex.com"),
  weight  = c(61.2, 74.8, 55.4, 68.0)
)

picture <- render_svg(data(private) + bar * bin + x(weight))
sapply(c("A. Kim", "email", "ex.com", "61.2"), grepl, x = picture, fixed =
  ↪ TRUE)
```

```
A. Kim  email ex.com  61.2
FALSE  FALSE  FALSE  FALSE
```

Every search returns `FALSE`. The sentence maps one column, so the drawing used one column. Even the weights are gone, because `bin` counts rows into intervals and the bars show the counts.

The missing weights come from the transform rather than from a general rule. A mark that summarizes draws no individual row. A mark that does not summarize draws one glyph per row, and each position is computed from one value:

```
picture <- render_svg(data(private) + point + x(weight) + y(patient))
lengths(gregexpr("<circle", picture))
```

[1] 4

Four rows, four circles, and the four names printed along the axis. Every plotting tool works this way, and it is what you asked for. A scatter plot draws each value you gave it.

So the rule is not that the file is safe. It is narrower and more useful. What you map is what you publish. A column you never map cannot be recovered from the file, and a private column that stays out of the sentence stays private.

B.1.2 Plots that carry the table

One case works differently, and you should know it before you publish.

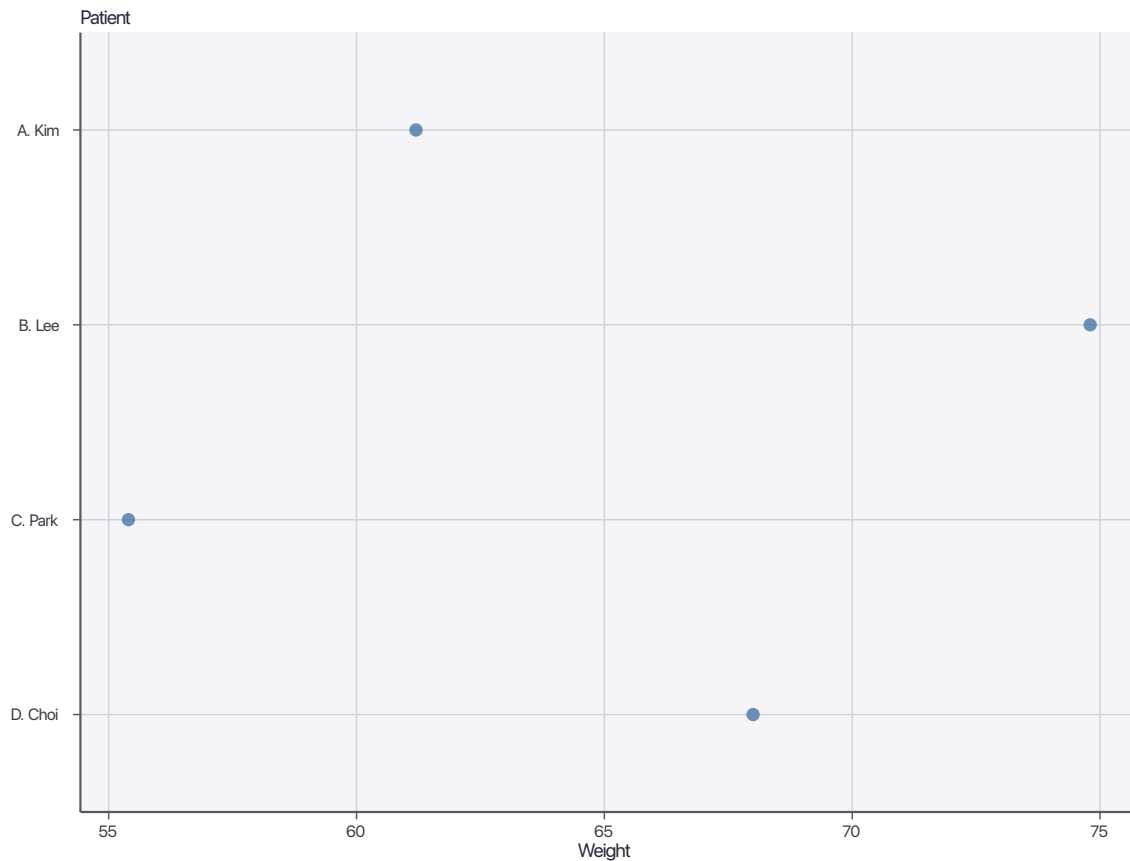
Four kinds of plot are drawn twice: one that binds `brush()`, one in the `cube`, one on the `globe`, and a `network` given an angle. Once when you write it, and once again in the reader's browser, because the reader can move a selection or turn the view. The second drawing needs the numbers, so the page carries the table beside the picture.

It carries the whole table, including columns the sentence never maps. The table that `show rows` (a button under the plot, explained in [Selection](#)) opens lists only the mapped columns, and the page itself carries every column.

An SVG file never carries the table, even when its sentence names a brush. `render_svg()` gives you the picture and stops.

One line of code keeps a private column out of the page. Bind the columns you plot and drop the rest:

```
shown <- private[, c("patient", "weight")]
data(shown) + point + x(weight) + y(patient) + brush(weight)
```



“Given the shown table: points, x is weight, y is patient, and let the reader select weight.”

That sentence names a brush, so this page carries a table. The table it carries is **shown**, which has two columns. Bind **private** instead and **email** would be in this page too.

B.1.3 What “byte for byte” means

This book says in several places that the four languages draw the same picture byte for byte. **render_svg()** is the function that claim is about, and the idea behind it is simpler than the phrase sounds.

A plot looks like a picture on your screen. Underneath, it is the text you just saw. Two plots are the same when that text is the same, byte for byte. Not similar. Not close enough to look the same. The same.

That is how the four languages are kept in step. Nearly every sentence in this book is written in R, in Python, in Julia and in JavaScript. Each one that draws is turned into its text, and

the four texts are compared byte by byte. One wrong byte counts as a disagreement and has to be explained.

The reason for such a strict test is that a loose one would miss things. Nobody looks at four pictures and decides whether they match. A person cannot see a bar that is half a pixel too wide, or a color one step away from the right one. This check sees both, and it compares the four texts the same way every time.

B.2 `save_gif()`

A plot that binds `play()` moves in a browser, because the SVG carries its own timing. A message, a slide in a talk and a post on a social site cannot play that timing. `save_gif()` writes the same sequence as a file that they do play:

```
ring <- file.path(tempdir(), "ring.gif")
save_gif(data(gm_all) + bar * mean + x(continent) + y(life) + polar() +
         play(year),
         ring, scale = 2)
cat(basename(ring), round(file.size(ring) / 1024), "KB\n")
```

ring.gif 752 KB

Nothing has to be installed. `scale` multiplies the canvas, which is 800 by 600 pixels unless the plot's theme says otherwise.

Three of the four languages take `scale` as a named argument. JavaScript has no named arguments, so it takes the same setting in the options object it uses everywhere else:

R	<code>save_gif(p, "ring.gif", scale = 2)</code>
Python	<code>save_gif(p, "ring.gif", scale=2)</code>
Julia	<code>save_gif(p, "ring.gif", scale = 2)</code>
JavaScript	<code>save_gif(p, "ring.gif", { scale: 2 })</code>

A plot with no `play()` is refused rather than written as a file of one frame. The [Play](#) chapter has that refusal, and the reason there is no video version.

B.3 gog_table()

Every shared table this book plots is available to you by name, so you can run any example here without writing a data reader first:

```
gapminder_2007 <- gog_table("gapminder_2007")  
data(gapminder_2007) + point + x(gdp) + y(life)
```

That block does not run on this page, and the reason is worth one line. The call reads the table over the network. A page that fetched on every build would leave the book unable to render without a working connection.

[The book's data](#) is the chapter for this function. It lists every table in the book and shows the call in each of the four languages.

`gog_table()` exists for reading this book rather than for your own work. When the data is yours, bind it with `data()` and skip this entirely.

C Combinations

Every mark down the side, every channel across the top: the whole of what the grammar lets you combine, on one page. This page holds four crossings. The first is what a mark lets you *map* (the channels); the second is what a mark lets you *derive* (the transforms); the third is which two transforms can *stand together* on one mark; the fourth is where a mark can be *drawn* (the coordinate spaces). Not only what a mark takes, but what it does not, and the surprising-but-legal pairings you would never think to try.

These grids are the orthogonality law made visible. [Law 1](#) says every compatible atom combines with every other; a table is that promise enumerated, one cell at a time. It is the direct analog of the *jamo* (자모) chart a Hangeul (한글) learner reads. The *jamo* are Hangeul’s individual letters: its consonants and its vowels. A syllable block is what they assemble into. This book has used that pairing from the start: a mark is a consonant and a position is a vowel, and neither draws alone. The chart puts the consonants down the side and the vowels across the top. The whole syllable space appears in the crossing.

Legal is not the same as useful. [Law 8](#), Pronounceable $\neq$ Useable, guarantees only that the engine draws what a cell allows. Whether it should be drawn is your judgment. A cell the engine refuses teaches as much as one it draws. It marks a real limit of the geometry, not an oversight.

Every grid here is generated live from the engine’s own rule table, each time the book builds. That is the same table the engine consults to accept or refuse your plot. A grid that is *described* can stop matching the code; a grid that is *generated* cannot. Add a mark, a channel, a transform or a space and these pages widen by themselves.

C.1 Marks and channels

Down the side, the marks; across the top, the channels. A cell says whether you may map a column to that channel on that mark.

Mark	x	y	z	color	size	shape	pattern	opacity	group	label	play
point	●	●					—		—	—	
line	●	●			○	—		○		—	
area	●	●			—	—		○		—	
bar	●	●			—	—			—	—	

Mark	x	y	z	color	size	shape	pattern	opacity	group	label	play
step	●	●			○	—		○		—	
interval	●	●			○	—		○		—	
box	●	●			○	—		○		—	
ribbon	●	●			—	—		○		—	
text	●	●				—	—		—	●	
path	●	●			○	—		○		—	
rule					○	—		○	—	—	
zone					—	—		○		—	
surface	●	●	●		—	—	—			—	
edge	—	—	—		○	—			—	—	

Reading the five glyphs:

- **must**: a required position. The mark does not render without it. A **point** needs an **x** and a **y**; a **text** needs its **label**.
- maps: bind a column here and the engine draws it today. The ordinary case, a channel doing its job.
- **set**: the mark has this feature, but not row by row. A **line** is one stroke, so its **size** and **opacity** are a single value for the whole layer, fixed with `style()` rather than mapped. Those constants have their own [Mark × Setting grid](#).
- not drawn**: legal grammar the engine refuses today, with a direction rather than a wrong picture. One glyph, three verdicts, and the refusal message tells you which you have. **Owed**: a mapped **size** or **opacity** on **text**, **z** on **text**, and **pattern** on **edge**. **Blocked** on infrastructure: **z** on **rule** and on **zone**, whose 3-D forms are a plane and a slab with no footprint to sort by, so they need an engine that can tell what hides what. **Decided**: **z** on **line**, **step**, **area** and **ribbon**, which read a domain left to right where a cube has none, and which name **path** as the direction. The Mark × Space grid below is the same three readings on its own blanks.
- **none**: the mark has no such feature. A **bar** has no glyph to **shape**, a **point** nothing to **group** into a line. Binding it means nothing, and the engine says so.

Five things the grid shows that prose has to say one at a time:

- group splits a line, not a bar**. Run down the **group** column: — on the marks that draw one shape per row (**point**, **bar**, **text**, **rule**, **edge**), — on every mark made of a connected run. A bar is split into groups by **color**, then piled or set side by side with **stack** / **dodge**; a bare **group** has no line to divide.
- label belongs to text alone**. Its column is ● on **text** and — everywhere else. A labeled scatter is the superposition **point** + **text**, never a **label** hung on the **point**.
- Categories on x and on y**. A category on **x** reaches every mark but **surface**, and on **y** it reaches far fewer. Read the **x** and **y** columns against each other; which types

each accepts is in [Channels](#), and the split divides the marks with positions in two. The glyph marks (`point`, `text`) and the marks whose slot width does the measuring (`bar`, `interval`, `box`) take either type on *both* positions, which is why a strip plot, a horizontal bar and a categorical box all follow from one rule. So do `path`, which sorts nothing and therefore has no domain, `rule`, which measures nothing on either axis, and `zone`. A `zone` measures nothing by length, which is what makes a heatmap cell a zone rather than a second kind of bar. The path and region marks (`line`, `step`, `area`, `ribbon`) take either type on *x* alone: their *x* is the domain the path is read along and their *y* is the quantity it traces, and those are not the same kind of axis. A glyph’s two axes are. `surface` stands outside the split: both of its positions measure, because a sheet is a height read at a place.

- **Three rows have no ● at all.** `rule` and `zone` show `on` on both positions and ● on neither, because each needs *one* of the two and no per-channel cell can say “either, but not both”. `edge` is the third, and it takes no position at all. Those obligations are stated once in the engine instead. Two `s` and no ● means the mark needs one position, and the panel supplies the other ([Rule](#), [Zone](#)).
- **zone and surface take either type on color.** They are the two fills that do: every other region’s fill takes a category, and these two take a number as well, reading the ramp.

C.2 Marks and transforms

The second crossing: not what a mark maps, but what it *derives*. A transform enters through `*` (`bar * bin`, `interval * range`, `point * jitter`), and which transform a mark takes is decided by the mark’s geometry against the transform’s kind. There are three kinds, and each has its own grid below.

Three glyphs read every cell:

- **combines:** the transform applies to this mark and renders.
- **● required:** the transform’s *kind* is the mark’s minimum syllable. An `interval` and a `ribbon` float between a low and a high, so they will not draw until *one* transform of that kind supplies the pair. The ● cells in a row are a choice, not a list: pick one. A `zone` shows ● wherever a transform is one of the ways it gets its sides.
- — **none:** the mark has no use for this kind of transform, and the engine refuses it with a direction toward the transform its geometry does want.

C.2.1 Statistics: a value per group

Most summarizing transforms reduce the data to one value at each *x*: a bin’s count, a group’s mean, a smoothed height. Any mark that draws a value at each *x* can draw that summary, so

the whole block moves together. That is why a distribution has three spellings rather than one histogram with a shape option: `bar * bin` fills it, `line * bin` is the frequency polygon, `step * bin` the staircase.

Mark	bin	smooth	count	density	proportion	posterior	mean	median	max	min	quantile	partition	flow	layout	cluster
point												—	—		—
line												—	—	—	—
area												—	—	—	—
bar												—	—	—	—
step												—	—	—	—
interval	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
box	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
ribbon	—	—	●	—	—	—	—	—	—	—	—	—	●	—	—
text	—	—	—	—	—	—	—	—	—	—	—				—
path	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
rule	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
zone	●	—	●	●	●							●	●	—	
surface	—	—	—	—	—						—	—	—	—	—
edge	—	—	—	—	—	—	—	—	—	—	—	—	—	●	—

Five marks (`point`, `line`, `area`, `bar`, `step`) take the whole family of summaries. `interval`, `box` and `rule` take none, and each for a reason the next two grids give. A span mark needs a *pair*, not a single value. A `box` carries its own summary: the five numbers are injected by the mark rather than composed, so no statistic column applies to it. A `rule` handed a whole axis to the panel, so there is no measure to compute.

Four more transforms build a *shape* rather than a summary, and they fill most of the rows in between. `partition` reads a hierarchy (`zone`, `text`), `flow` reads a path through stages (`ribbon`, `zone`, `text`), `layout` places a network (`edge`, `point`, `text`), and `cluster` orders leaves by similarity (`path`, `zone`). Each has a chapter of its own.

C.2.2 Ranges and bands

Four transforms produce a *pair* of values, a low boundary and a high one, per group. `range`, `confidence` and `deviation` compute the pair from your data; `bounds` reshapes a pair you computed already. Five marks draw such a pair: `interval` shows it as a whisker, `ribbon` fills it, `line` and `step` trace its two boundaries, and `zone` reads it as two sides.

Mark	range	confidence	deviation	bounds
point	—	—	—	—
line				

Mark	range	confidence	deviation	bounds
area	—	—	—	—
bar	—	—	—	—
step				
interval	●	●	●	●
box	—	—	—	—
ribbon	●	●	●	●
text	—	—	—	—
path	—	—	—	—
rule	—	—	—	—
zone	—	—	—	●
surface	—	—	—	—
edge	—	—	—	—

The ● cells on **interval**, **ribbon** and **zone** are a requirement, not a permission. An **interval** and a **ribbon** have no baseline to start from, so neither renders until a pair transform gives it a low and a high: **interval * range** is the whole minimum syllable, a bare **interval** is refused. A **line** or a **step** already draws fine on its own, so the same transform is optional there, a second way to use the mark. Everywhere else the pair has nowhere to go: on a **point** or a **bar** it is refused toward the whisker, on an **area** toward the **ribbon** that is its filled twin.

C.2.3 Collision modifiers: an offset, not a statistic

When marks collide at one position, four offsets pull them apart, and which one a mark takes is decided by its geometry and by the *axis* the collision is on. **dodge** subdivides a *width*, so it serves the marks that have one (**bar**, **box**, **interval**). **stack** accumulates along the *measure* axis, so it serves the marks that can spend a span: **bar** and **area** fill it with length, and a **point** has no length to stretch, so it uses the span to count dots, which is the dot plot. **jitter** spreads a mark with *no* width along a *categorical* axis, which is **point** alone. **repel** divides on a different question: not what geometry a mark has, but what it is made of. A label is as wide as the word it draws, so two labels overlap where their points do not, and **text** is the only mark made of words.

Mark	dodge	stack	jitter	repel
point	—			—
line	—	—	—	—
area	—		—	—
bar			—	—
step	—	—	—	—
interval		—	—	—

Mark	dodge	stack	jitter	repel
box		—	—	—
ribbon	—	—	—	—
text	—	—	—	
path	—	—	—	—
rule	—	—	—	—
zone	—	—	—	—
surface	—	—	—	—
edge	—	—	—	—

Six marks take an offset, and two of them take two. **bar** has both a width and a measure axis, so it takes **dodge** and **stack**; **point** has neither, so it takes **stack** and **jitter**. **stack** answers its overlap along the measure axis (a pile of dots, which can be counted); **jitter** answers the same mark’s overlap along a categorical one (a nudge, because a category has no magnitude to spend). Everywhere else the division is exclusive, which is why the engine can always name the right transform when you ask for the wrong one. Ask for **point * dodge** and it names **jitter**; ask for **area * dodge** and it names **stack**. A **box** shows the split cleanly: it takes no statistic (its own is built in) yet still takes **dodge**, because setting grouped boxes side by side is a question about position, orthogonal to the summary each box already carries.

C.3 Transforms and transforms

The third crossing is the only one where a mark does not appear. Two transforms can follow one mark: **bar * bin * mean** cuts the axis into bands, then averages inside each. Which pairs can do that is decided by the transforms alone, and by one rule.

A transform has a **job**. There are four of them, and every transform in **gog** fills at least one, except **repel**:

job	the question it answers	the transforms that answer it
extent	where are the cells?	bin , density , smooth , partition , and bounds on a zone
measure	what is in them?	count , sum , mean , median , max , min , quantile , range , confidence , deviation , bounds
scale	what scale is the answer read on?	proportion , and stack (share = TRUE)
position	where do the marks sit?	dodge , stack , jitter

`repel` fills no job at all. It moves a label after every job has run, so it contradicts nothing and chains with all three offsets. That is the whole `repel` row in the grid below. `flow`, `layout` and `cluster` fill all four jobs at once. Each computes the whole picture, so only `repel` chains with one.

Two transforms compose when they do different jobs, and contradict when they do the same one. A cell holds one extent, one measure, one scale and one position. So a second transform doing a job the first already did leaves the engine no reading that keeps both, and it refuses rather than drawing one and discarding the other.

One transform is the exception, and it is the exception that makes chaining useful. `bin` does two jobs: it cuts the axis, and it counts what lands in each band. The count is a by-product of the cut, so `bin` can give it up, and it gives it up to a transform you handed a *column*. That is why `bar * bin * mean` composes: `bin` supplies the cells and `mean` supplies their value. And it is why `bar * bin * count` does not, since `count` was never handed a column to reduce.

	bin	smooth	count	density	proportion	sum	mean	median	max	min	quantile	range	confidence	deviation	bounds	partition	flow	layout	cluster	dodge	stack	jitter	repel
bin	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
smooth	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
count	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
density	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
proportion	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
sum	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
mean	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
median	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
max	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—	—
min	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—	—
quantile	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—	—
range	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—	—
confidence	—	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—	—
deviation	—	—	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—	—
bounds	—	—	—	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—	—
partition	—	—	—	—	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—	—
flow	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—	—
layout	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—	—
cluster	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	·	·	—	—	—	—
dodge																	—	—	·	·	—	—	—
stack																	—	—	—	·	·	—	—
jitter																	—	—	—	—	·	·	—
repel																						·	·

The grid is symmetric, and that is the rule showing rather than being claimed. Whether two transforms contradict is a fact about their jobs. It has nothing to do with which one you typed

first. So order is yours to choose: write them in the order they happen, and the sentence reads the way it runs.

Read the block of - in the middle. Eleven transforms measure, so no two of them combine, and that block is the whole reason `bar * count * mean` is refused, and `bar * mean * sum`, and `interval * range * confidence`. `bin` is the only extent transform with a where `smooth` and `density` have -. That is the exception above, in one cell.

Two collisions the grid cannot show, because both depend on something outside the pair. `stack(share = TRUE)` divides each pile by its own total, which is a *scale*. `proportion` divides again, and the second division has no whole left to be a share of. And `bounds` says where a *zone*'s sides are. On that one mark it does the *extent* job, so it contradicts `bin`. On every other mark it names the low and the high on the measure axis. The engine reads both cases correctly, and the grid above shows the reading that applies everywhere else.

Chaining stops at four, and nobody chose the number. Four jobs, one transform each, so a fifth has to repeat a job on the same mark, and repeating one is the contradiction this whole section is about.

C.3.1 The shapes a long chain can take

Three and four transforms sound like a large space, and they are not. Once each job is filled at most once, and the marks are asked which transforms they take, every chain of three or four takes one of the shapes below, and there are not many. Two slots stand for a family rather than a transform: `<aggregation>` is any of `sum`, `mean`, `median`, `max`, `min`, `quantile`, and `<collision modifier>` is `dodge`, `stack` or `jitter`, whichever the mark takes.

how many	the shape	the jobs it fills
3	<code><aggregation> *</code> <code>proportion *</code> <code><collision modifier></code>	measure, scale, position
3	<code>bin * <aggregation> *</code> <code><collision modifier></code>	extent, measure, position
3	<code>bin * <aggregation> *</code> <code>proportion</code>	extent, measure, scale
3	<code>bin * deviation *</code> <code>proportion</code>	extent, measure, scale
3	<code>bin * proportion *</code> <code><collision modifier></code>	extent, scale, position
3	<code>count * proportion *</code> <code><collision modifier></code>	measure, scale, position

how many	the shape	the jobs it fills
4	<code>bin * <aggregation> * proportion * <collision modifier></code>	extent, measure, scale, position

Read the four-transform row first, because it is the longest chain there is. It fills each job exactly once, and only one of its slots has any real choice in it. `bin` is the only extent transform that hands its measurement over. `proportion` is the only one that rescales. The family in the last slot is picked by the mark. So the aggregation is the only free decision, which is why four is reachable and rare.

The shapes say what the *transforms* allow. A mark can still refuse one of them for a reason about its own geometry, and the grids earlier in this chapter are where that is decided. `point * proportion * stack` is the clearest case. A dot plot piles one whole dot per observation, and a share is less than one dot. So the chain is refused on `point` and drawn on `bar`.

C.4 Marks and spaces

The fourth crossing is not a channel of the mark at all. It is the coordinate space the whole plot sits in, and it asks a different question of every mark: once the plane is bent, or stood up into a cube, can this geometry still be drawn? A `polar()` bar is a wedge, a `polar()` line is a closed curve, and a `z` scatter is a projected cloud, all from the same sentences that draw them flat.

Two glyphs here, not five. Either the engine draws this mark in this space today, or it does not and says so:

- **drawn:** the mark renders in this space.
- — **not drawn:** legal to write and refused with direction, never half-drawn. As on the channel grid, one glyph covers owed, blocked and decided alike; which one a blank is, is the subject of the paragraphs under the table.

Mark	flat	space	polar	nest	globe	map	network
point				—			
line		—		—	—	—	—
area		—		—	—	—	—
bar						—	—
step		—		—	—	—	—
interval				—	—	—	—
box				—	—	—	—
ribbon		—		—	—	—	—

Mark	flat	space	polar	nest	globe	map	network
text		—					
path				—			—
rule		—		—			—
zone		—		—			—
surface	—		—	—	—	—	—
edge	—	—	—	—	—	—	

flat is full but for **surface** and **edge**, and **surface** is the one to read first: it does not draw in the plane at all. Every other mark draws there, and some also stand up in the cube, while a surface is a sheet through three positions and there is no such thing without the third, so its minimum syllable includes the cube. It is the only mark that draws in the cube and not in the plane ([Surface](#)). An **edge** is the other kind of blank: it takes no position at all, and the space that places its nodes is the last column of this grid.

space holds six marks, and which six is a reading rather than a list. A **point** is a place, a **path** is an order, a **surface** is a sheet. **bar**, **interval** and **box** are the three marks that stand in a slot and measure along the other axis: a length, a span and a summary, which the grammar treats as one family. Each means in a cube exactly what it meant on the page.

The blanks in that column are not one kind of thing, and the refusal message says which each is. **line**, **step**, **area** and **ribbon** read a *domain* left to right, and a cube has no left to right, so they are **decided**, a ruling rather than work still owed, with **path** as the direction. **rule** and **zone** are **blocked**: their 3-D forms are a plane and a slab, neither of which has a footprint to sort by, so placing them among the data needs an engine that can tell what hides what ([Space](#)). **text** is **owed**: a label in the cube is not built yet, and the message says so rather than drawing one flat ([Space](#)).

polar matches **flat**: the same twelve marks draw, and the same two do not. Every mark that draws on the page draws in the circle, because every mark whose edges could be curved draws there, a band's boundaries being the same chords a **line** draws. The two blanks are **surface** and **edge**, and neither is a polar refusal: a sheet needs the cube, and an edge needs the space that places its nodes ([Polar](#)).

nest is a column of the opposite shape: two cells filled and the rest blank. Those blanks are not work waiting to be done. This space asks a different question from the other five. Everywhere else a mark bends because its geometry survives a transformation of the plane; a nesting is not a transformation of the plane at all, so what it can draw is whatever it can hand a *rectangle* to. **bar** can take one, because a bar is a measure and this space is the third thing a measure can become, after a length and an angle. **text** can too, and it is the only mark here that names a region rather than filling one: the rectangle is already computed, so a label goes to its center, which is what a published treemap has and what a legend cannot do once the split runs past a hundred regions ([Nest](#)).

The blanks share one reason, and it is not that the space is unfinished. They are the marks placed by a *position*. **point** needs somewhere to sit. **line**, **step**, **path**, **area** and **ribbon** need an order along an axis, and two touching regions here are not near each other in the data. **rule** and **zone** span an axis that is not there, and **box**, **interval** and **surface** measure along one. **edge** is placed by a layout that this space does not compute. **text** is not among them because it is not placed by a position here either; it is placed by the region, which is the same thing that places the bar.

map, the sixth column, fills five cells, and the shape of its column comes from one fact: longitude and latitude spend both positions on the place. The marks that draw are the ones that place something. A **point** is a place, a **path** is a route, a **text** is a name at a place, a **rule** at one position spans the other and becomes a meridian or a parallel, and a **zone** takes its sides from a boundary, which is the choropleth ([Map](#)). The blanks share one reason: they measure along an axis, and a map has no axis to spare. The quantity moves to **size** or **color** instead, which is what cartography does with a measure once both positions are gone.

globe, the column before it, fills six cells: **map**'s five, and one the flattening cannot have. The sphere and its flattening answer one question: where does a place go. So the marks that place something stand on both, and only the reader's view changes. The far half of the earth is hidden behind the sphere, a route bends along a great circle, and a rule becomes the whole meridian or parallel it holds. The sixth cell is **bar**, because the sphere alone has a direction to spare: the radius, which runs straight out from the surface at every place, where a bar stands as a spike with **z** naming its measure ([Globe](#)). The blanks read a domain along an axis, and a sphere has none to read.

network holds three cells, and one of its marks draws in no other space. **edge** draws nowhere else, because an edge is placed by neither position: the space computes where every node sits and the edge follows the two it joins ([Network](#)). **point** and **text** join it there, as the nodes and their names.

Three of the four crossings have a mark on every row, and reading one mark across them is worth doing. A path takes nearly every channel on Mark $\times$ Channel and almost nothing on Mark $\times$ Transform, because a path *is* its rows in order and every statistic replaces them. **rule** reads the same way on the transform grid, for a related reason. It hands a whole axis to the panel, so there is no measure to compute. And **surface** has a row of the transforms that cover a plane: **density**, which estimates a value at every node, and **bin**, which cuts adjacent cells, with **sum**, **mean**, **median**, **max** and **min** naming the column each cell holds. **path** derives two things. With **density** it draws as a contour the field a **surface** lifts into a sheet, and with **cluster** it draws the tree.

With these grids the orthogonality law is stated in full: what each mark maps, what it sets ([the Mark \$\times\$ Setting grid](#)), what it derives, and where it can be drawn.

D Chart names, translated

`gog` has no chart-type atoms: a histogram is a sentence, not a word. But people look for plots by name, so this appendix is the index of names. It is the one place organized by chart name; the [cookbook](#) organizes the same territory by question, which is usually the better place to start.

Column names here are placeholders: put your own in their place.

D.1 Names with a one-line answer

The name you know	The sentence
Scatter plot	<code>point + x(a) + y(b)</code>
Bubble chart	<code>point + x(a) + y(b) + size(c)</code>
Strip plot	<code>point + x(category) + y(measure)</code> , or swap the axes to draw it sideways
Bar chart	<code>bar + x(category) + y(measure)</code>
Horizontal bar chart	<code>bar + x(measure) + y(category)</code>
Hatched (print-safe) bars	<code>bar + x(category) + y(measure) + style(pattern = "hatch")</code> , also <code>"crosshatch"/"grid"/"dots"</code>
Count (frequency) chart	<code>bar * count + x(category)</code>
Proportion chart	<code>bar * proportion + x(category)</code>
Histogram	<code>bar * bin + x(measure)</code> , <code>bin(30)</code> or <code>bin(width = 5)</code> to tune
Frequency polygon	<code>line * bin + x(measure)</code>
Outline (step) histogram	<code>step * bin + x(measure)</code>
Overlaid histograms	<code>bar * bin + x(measure) + color(group)</code>
Density plot	<code>line * density + x(measure)</code>
Line chart / time series	<code>line + x(time) + y(measure)</code>
Multi-series lines by dash (no color)	<code>line + x(time) + y(measure) + pattern(series)</code> , add <code>color(series)</code> for redundant encoding
Step chart	<code>step + x(a) + y(b)</code>

The name you know	The sentence
Empirical CDF	<code>step</code> over cumulative shares, worked example in Step
Area chart	<code>area + x(a) + y(b)</code>
Spread / range band	<code>ribbon * range + x(a) + y(b)</code> , split by group with <code>color(group)</code>
Confidence band (computed)	<code>ribbon * confidence(0.95) + x(a) + y(b)</code> , often under a <code>line * mean</code> trend
Confidence band (pre-computed)	<code>ribbon * bounds(lower, upper) + x(a)</code> , bounds you already have (a model's standard error, a standard error of measurement)
Unfilled (dashed) error band	<code>line * bounds(lower, upper) + style(pattern = "dashed")</code> , the two $\pm$ edges, no fill
Min-max envelope, as two lines	<code>line * range + x(a) + y(b)</code> , the unfilled counterpart to <code>ribbon * range</code>
Dashed / dotted line	any <code>line/step + style(pattern = "dashed")</code> (or <code>"dotted"</code>)
Trend line / smoother / LOESS	<code>line * smooth + x(a) + y(b)</code>
Summary bars (mean per group)	<code>bar * mean + x(category) + y(measure)</code> , also <code>sum</code> , <code>median</code> , <code>max</code> , <code>min</code>
Grouped bars (side by side)	<code>bar * mean * dodge + x(category) + y(measure) + color(group)</code> , <code>* dodge</code> separates the split
Colorblind-safe grouped bars	add <code>+ pattern(group)</code> to the grouped bars: hue <i>and</i> hatch, one merged legend
Ranked bars	add <code>+ order(measure, desc = TRUE)</code>
Small multiples / trellis / conditioning plot / facets	<code>plot \l facet(category)</code> , <code>plot / facet(category)</code> , or both for a grid
Small multiples of 3-D plots	any cube sentence <code>\l facet(category)</code> : one projected box per panel
3-D scatter	<code>point + x(a) + y(b) + z(c)</code> , view angle via <code>space(turn = , tilt =)</code>
3-D trajectory / space curve	<code>path + x(a) + y(b) + z(c)</code> , the rows joined in table order, in the cube
Surface plot / wireframe	<code>surface + x(a) + y(b) + z(c)</code> over a table with one row per (x, y) crossing (<code>expand.grid()</code> 's shape); <code>style(border_color = "white")</code> draws the mesh lines, <code>color(c)</code> shades it by height
Terrain / elevation / bathymetry	the same sentence: a height over a plane is a <code>surface</code> , whatever the height means

The name you know	The sentence
Response surface	<code>surface + x(factor1) + y(factor2) + z(response)</code> over the design grid; from a fitted model, predict onto a grid first and hand over the result
3-D density surface	<code>surface * density + x(a) + y(b) + space()</code> , the estimate raised as height, the third geometry of the same field <code>zone * density</code> paints and <code>path * density</code> traces
Profile plot (categories joined)	<code>line * mean + x(category) + y(measure)</code> , one summary per category with the path across them; <code>step</code> stays flat between categories instead of sloping
Filled profile	<code>area * mean + x(category) + y(measure)</code> , the same boundary filled to the baseline
Connected scatterplot	<code>path + x(a) + y(b)</code> , the rows joined in the table's order rather than sorted by x; <code>style(arrow = "end")</code> says which end is last
Trajectory / route / hysteresis loop	the same sentence: any reading whose <i>order</i> is the data's is <code>path</code> , not <code>line</code>
Annotation arrow / callout	<code>data(two_rows) + path + style(arrow = "end")</code> , layered over the plot; the note itself is <code>data(one_row) + text + label(note)</code>
Rose / coxcomb / wind rose	<code>bar * count + x(direction) + polar()</code> , the bar chart read in a circle
Radar / spider plot	<code>line * mean + x(category) + y(measure) + polar()</code> , the profile bent into a circle; <code>area</code> fills it
Pie chart	<code>bar * count * stack + color(category) + polar()</code> , a stacked bar in polar coordinates; the category is a <i>split</i> , not a position

The name you know	The sentence
Sunburst	<code>zone * partition(a, b, c) + x(measure) + polar()</code> , worked example in Polar . One ring per level of a tree, each arc as wide as its share; the hierarchy is columns , outermost first, and a blank level ends that branch early, which is what gives the rim its ragged edge. <code>y(depth, limits = c(0, n))</code> puts the hole in the middle, and <code>text * partition(...) + label(name)</code> names the nodes, reading the same centers the zone computed
Icicle / partition chart	the sunburst's sentence with <code>polar()</code> taken off, which is the whole difference: depth up the y axis, share across the x. The two names are one coordinate space apart, exactly as a rose and a bar chart are, or a pie and a stacked bar
Treemap	<code>bar * sum + y(value) + color(category) + nest()</code> , the same shares carried by area rather than by angle (Nest)
Nested treemap	the same, plus <code>x(group)</code> : one region per group, its rows packed inside
Labeled treemap	the same, plus a <code>text</code> layer: <code>+ text + label(name)</code> writes each region's name at its center, which is what a packing needs once the split has too many levels for a legend. A name wider than the region it belongs to is left out, and the plot reports how many (Nest)
Pie of a measure	<code>bar * sum * stack + y(amount) + color(category) + polar()</code> , also <code>proportion</code> for shares on the axis
Share-of-total column	the same sentence without <code>polar()</code> , one segmented bar
Polar histogram	<code>bar * bin + x(bearing) + polar()</code> , bins as wedges of a measured angle
Circular (cyclical) line	<code>line + x(hour) + y(measure) + polar()</code> , both ends of the cycle in the data
Polar scatter	<code>point + x(bearing) + y(measure) + polar()</code>

The name you know	The sentence
Contour plot / density contours	<code>path * density + x(a) + y(b)</code> , the contours of a density estimated over the plane; <code>density(levels =)</code> says how many, <code>group(category)</code> estimates one field per group. <code>zone * density(levels =)</code> fills the bands between the same lines, and a crater (a ring-shaped cluster with a hollow middle) is the one field that filled reading draws wrongly, since a band around a hollow is a shape with a hole and <code>gog</code> has no mark for one
2-D density heatmap	<code>zone * density + x(a) + y(b)</code> , the same field painted as cells instead of traced as lines
Filled contour / filled density	<code>zone * density(levels = 8) + x(a) + y(b)</code> , the bands between eight contours; the same <code>levels</code> a <code>path</code> traces
Binned heatmap	<code>zone * bin + x(a) + y(b)</code> , cells colored by how many rows fell in each
Hexbin	<code>zone * bin(tiling = "hex") + x(a) + y(b)</code> , the staggered mesh that stops the eye reading the mesh's own alignment as data
Heat strip / binned distribution per group	<code>zone * bin + x(number) + y(category)</code> , the mixed mesh: the continuous axis cut into cells, one row of them per category, every row cut on the same edges
Choropleth	<code>zone</code> with <code>group(country)</code> and <code>map()</code> : the boundary is the fifth way a zone finds its sides (Map)
Candlestick / OHLC chart	two layers over one table, worked example in Zone : <code>interval * bounds(low, high) + color(dir)</code> is the whisker and <code>zone * bounds(body_lo, body_hi, start = left, end = right) + color(dir) + style(opacity = 1)</code> the body, over <code>x(session)</code> . Four numbers a session, and no new channels for them: the pair a band mark reads is the pair a rectangle's sides are. The <code>opacity</code> and the named sides both replace a zone default meant for highlighting: here the rectangle <i>is</i> the data

The name you know	The sentence
Waterfall / bridge chart	<code>zone * bounds(base, top, start = left, end = right) + x(step) + color(dir) + style(opacity = 1),</code> worked example in Zone . The running total is host arithmetic (<code>cumsum</code>), because <code>stack</code> accumulates <i>within</i> a position and a waterfall accumulates <i>across</i> positions; the subtotal bars start at 0, and <code>left/right</code> sit just inside each slot's edges, which is what leaves a gap between the bars
Funnel chart	<code>zone * bounds(lo, hi) + y(stage) + style(opacity = 1),</code> worked example in Zone , with <code>text + label(n) + x(mid)</code> for the numbers inside. The columns <code>lo</code> and <code>hi</code> hold $-n/2$ and $n/2$, computed in the host, which is the same arithmetic the waterfall uses for its running total. A funnel is a bar chart whose blocks are centered on zero, and centering is two columns rather than a chart type. The measure lands on <code>x</code> because <code>y(stage)</code> is a category; nothing had to ask for it. The sloped connectors between stages are deliberately absent. Each one spans the gap between two categories, where the axis means nothing, so no scale gives its size a meaning

D.2 Names that need more

These names need a paragraph rather than a line. Nearly all of them draw, and the entry gives the sentence. Where one does not, the entry says so first and then says what the near miss costs you.

The name	The sentence, and what to know
Box plot	<code>box + x(group) + y(value),</code> the five-number summary per group

The name	The sentence, and what to know
Violin plot	<code>ribbon * density + x(group) + y(value)</code> , the shape the box plot's five numbers summarize. <code>density(compare =)</code> says whether the widths carry each group's row count (the default) or draw every violin to the same area
Half violin	<code>area * density + x(group) + y(value)</code> : the same estimate closed on the slot's center line instead of on its own reflection, which is the difference between those two marks everywhere else
Ridgeline plot / joyplot	<code>area * density(reach = 2.5) + x(value) + y(group)</code> , the half violin with its axes exchanged, one density per category, each on its own line, sharing one measure axis. <code>reach</code> is in <code>slots</code> , so past 0.5 the ridges leave their own slots, which is the joyplot's look. Its edge is a layer: <code>line * density + style(color = "black")</code> reads the same estimate and traces the edge of the fill
Jittered strip plot	<code>point * jitter + x(group) + y(value)</code> , the <code>jitter</code> collision modifier spreads coincident points along the category axis
Labeled scatter, names not overlapping	<code>point + text * repel + x(a) + y(b) + label(name)</code> , the <code>repel</code> collision modifier moves each word off the other words and off the dots, and draws a thin line back to the point when a label has moved far
Stacked bars / areas	<code>bar * sum * stack / area * sum * stack</code> with a <code>color split</code> , the <code>stack</code> collision modifier (grouped bars are its sibling <code>dodge</code>)
100% stacked bar / filled area	<code>bar * count * stack(share = TRUE) + x(a) + color(b)</code> , every pile divided by its own slot's total so all of them reach 1 and only the composition is left. Any measurement fills the same way, <code>sum</code> included

The name	The sentence, and what to know
Streamgraph	<code>area * stack(baseline = "wiggle") + x(t) + y(v) + color(g)</code> , worked through in Transforms . The foot of each pile is placed to keep the bands as flat as possible, so a band's shape is its own rather than the sum of everything below it. <code>share</code> and <code>baseline</code> are the pile's two free choices and compose
ThemeRiver	<code>area * stack(baseline = "center")</code> , the same sentence with the <i>symmetric</i> layout instead (Havre et al., 2000). Often confused with the streamgraph and not the same plot: this one centers every pile, that one minimizes movement, and its designers argue readers compare values more accurately on the streamgraph (Byron & Wattenberg, 2008)
Alluvial diagram	<code>ribbon * flow(a, b, c) + y(count)</code> , with <code>zone * flow(...)</code> for the stage slots and <code>text * flow(...) + label(name)</code> for their names, worked through in Flow . One row of the table is one path through the stages, and one band carries that path's magnitude between each adjacent pair
Sankey diagram	Not drawn in the general form. The alluvial gives the merged-band look: group the table down to one pair of stages at a time and the bands are the flows between them. The general many-source form, nodes and links given as an edge list, is a different input shape and is not read (Flow says where the limits are)
Network diagram / node-link graph	<code>edge * layout(from, to) + network()</code> , with <code>point * layout(...)</code> for the nodes, <code>size(degree)</code> sizing each node by its relations, and <code>text * layout(...) + label(name)</code> for their names, worked through in Network . The engine computes every position, so the picture is the same in all four languages; state an angle, <code>network(turn = 35, tilt = 20)</code> , and the same layout is a cube that drags in the web edition

The name	The sentence, and what to know
Relative-frequency histogram	<code>bar * bin * proportion + x(number)</code> , the histogram's own counts read as fractions of <code>n</code> , on the mesh <code>bar * bin</code> cuts
Heatmap	Drawn on either kind of axis, and on one of each. Which transform you want depends on which. A <i>continuous</i> axis needs cutting, so <code>zone * bin</code> counts the rows per cell: both axes for the classic heatmap, one for the mixed mesh. <code>zone * density</code> estimates a value at each cell of a plane. A <i>categorical</i> axis arrives cut, so on two of them <code>zone * count</code> tallies into the slots, and <code>zone + color(value)</code> draws a table you already have
Summary heatmap / matrix of means	<code>zone * mean + x(a) + y(b) + color(column)</code> reduces the named column within every cell two categories cross, and <code>sum/median/max/min</code> read the same mesh. The column is named on <code>color</code> because that is the channel a zone measures with, and a summary reduces in place, the same binding that both names and receives it on <code>bar * mean + x + y</code>
Tile plot / calendar heatmap	<code>zone + x(a) + y(b) + color(value)</code> , one cell per pair of categories, filling the slot each one owns
Confusion matrix	<code>zone * count + x(actual) + y(predicted)</code> tallies the raw rows; <code>zone * proportion</code> reads them as shares of the whole. Normalizing per <i>true class</i> , a share within each row of the grid, is not one of these: <code>stack(share =)</code> divides a pile by its slot's total, and a grid has no pile to divide
Correlation matrix / corrplot	Drawn as a tile plot, once the correlations are a table: compute <code>cor()</code> where your data lives, reshape it long, and <code>zone + x(var1) + y(var2) + color(r)</code> draws it. <code>gog</code> computes no correlations; that is the host's job, as <code>x(sin(t))</code> is

The name	The sentence, and what to know
Donut chart	<code>zone * partition(category) + x(measure) + y(depth, limits = c(0, 2)) + polar()</code> , worked example in Polar . A donut is a sunburst of a tree one level deep, and the hole is not a round thing at all; it is a stretch of the radial axis with nothing standing on it, which is what the stated <code>limits</code> leave empty. The plain <i>pie</i> (<code>bar * count * stack</code>) is the one reading that cannot say it, having spent its measurement on the angle and left its radius a constant with no domain to widen; that is a fact about the reading rather than a gap, so there is no <code>polar(hole =)</code> and no plan for one
Animated (time-lapse) chart	any sentence + <code>play(year)</code> : faceting read in time
Marginal / joint plot (scatter with histograms in its margins)	<code>top / (main right)</code> : three ordinary plots and the two composition operators. No chart type, no <code>ggMarginal()</code> : the marginals align and share their scale because both name the same column on the same axis, which is the one rule composition adds (Composition). Any mark can be the marginal: a histogram, a box, a violin, a dot plot
Dendrogram / cluster tree	<code>path * cluster(value, over = profile) + x(leaves)</code> joins the closest leaves one pair at a time, with each join's distance on an axis of its own; <code>y(leaves)</code> lays it on its side (Cluster). <i>Dendrogram</i> is Greek for tree drawing, which is what <i>cluster tree</i> says in English
Clustered heatmap	<code>zone * cluster(over = profile)</code> reorders a tile plot's slots to the tree's leaf order, and the full figure is the marginal plot, with trees above and beside the tiles; the shared axes take the trees' order (Cluster)

The name	The sentence, and what to know
3-D surface	The same sentence as Surface plot above, surface + x(a) + y(b) + z(c) over a grid table, or surface * density + x(a) + y(b) + space() to raise an estimated field. A <i>scatter</i> cannot be one, and is refused toward point rather than drawn as an empty sheet: a surface's faces span the gaps between its samples, so it needs samples on a grid
Terraced / stepped surface (one value per cell)	surface * bin * mean + x(a) + y(b) + z(v) cuts the floor into cells and lays a flat plateau on each, with a riser at every step. The sentence for a design that measures one number per condition, where an interpolated sheet would assert a gradient nobody observed (Surface)

If the name you came with is in neither table, it is usually a styling of a name that is listed. Ask what question the chart answers, and start from the [cookbook](#). If you came from **ggplot2** and the name was one of its function names, [Coverage](#) is the same territory indexed that way. It sorts each name into four: refused, the host language's job, missing with the direction settled, and not decided either way.

E The ggplot2 phrasebook

`ggplot2` proved that a grammar of graphics could be a working tool and not only a theory. Most readers of this book arrive from it. `gog` makes its own choices inside that same idea, and this appendix translates between the two: which word to use, and what will feel different.

It assumes the plot you want can be drawn. To find out whether it can, read [Coverage](#). That appendix reads `ggplot2`'s whole reference index against the kernel and says what `gog` refuses and what it does not draw.

E.1 Side by side

	ggplot2	gog
Languages	R	R, Python, Julia and JavaScript, one engine under all of them
Naming	<code>geom_point</code> , <code>aes</code> , <code>alpha</code>	<code>point</code> , channels by plain name, <code>opacity</code>
Where <code>x/y</code> are written	<code>aes()</code> at the plot or the layer, and a layer may override the plot's <code>aes()</code>	resolved by where the channel is written in the sentence
The histogram	<code>geom_histogram()</code> <i>or</i> <code>stat_bin(geom = "bar")</code>	<code>bar * bin</code>
Log axis	<code>scale_x_log10()</code>	<code>x(gdp, scale = "log")</code> , written on the binding
3-D	external packages	one more position: <code>z</code>
Animation	<code>gganimate</code> , a separate package	<code>play</code> is a channel of the kernel: <code>+ play(year)</code>

E.2 Translations

You write in ggplot2	You write in gog
<code>ggplot(df, aes(x = a, y = b))</code>	<code>data(df) + x(a) + y(b) + ...</code>
<code>geom_point()</code>	<code>point</code>
<code>geom_line()</code>	<code>line</code>
<code>geom_step()</code>	<code>step</code>
<code>geom_area()</code>	<code>area</code>
<code>geom_col()</code>	<code>bar</code>
<code>geom_bar()</code> , which counts the rows for you	<code>bar * count</code> , where the count is a word in the sentence
<code>geom_histogram(bins = 30)</code>	<code>bar * bin(30)</code>
<code>geom_freqpoly()</code>	<code>line * bin</code>
<code>geom_density()</code>	<code>line * density</code>
<code>geom_smooth()</code>	<code>line * smooth</code>
<code>stat_summary(fun = mean, geom = "bar")</code>	<code>bar * mean</code>
<code>aes(color = g)</code>	<code>color(g)</code>
<code>aes(size = v)</code>	<code>size(v)</code>
<code>aes(shape = g)</code>	<code>shape(g)</code>
<code>aes(alpha = v)</code>	<code>opacity(v)</code>
<code>aes(group = g)</code>	<code>group(g)</code>
<code>scale_x_log10()</code>	<code>x(v, scale = "log")</code>
<code>scale_color_viridis_c()</code>	<code>color(v) + palette("viridis")</code>
<code>facet_grid(r ~ c)</code>	<code>\ facet(c) / facet(r)</code>
<code>facet_wrap(~ g, ncol = 4)</code>	<code>\ facet(g, wrap = 4);</code> / runs the same line down instead, so one number replaces <i>nrow and ncol</i>
<code>facet_(scales = "free_y")</code>	<code>y(v, free = TRUE)</code> , written on the axis it frees, so there is no <code>free_x/free_y/free</code> to choose between
patchwork's <code>p1 p2, p1 / p2</code>	<code>plot_a plot_b, plot_a / plot_b</code> : the same two operators, and the operand decides which <code>theme(width =, height =)</code> on the plot that needs more room; the blank corner of a marginal plot is the space a shared axis does not use
patchwork's <code>plot_spacer()</code> , <code>plot_layout(widths =)</code>	<code>text * repel</code> , the collision modifier for overlapping labels, beside <code>dodge</code> , <code>stack</code> and <code>jitter</code>
ggrepel's <code>geom_text_repel()</code>	<code>top / (main right)</code> , three ordinary plots (Composition)
ggExtra's <code>ggMarginal()</code>	swap the <code>x</code> and <code>y</code> bindings: orientation is read, not switched
<code>coord_flip()</code>	

You write in <code>ggplot2</code>	You write in <code>gog</code>
<code>labs(title = , x = , y =)</code>	<code>title(), x_label(), y_label(), z_label()</code>
<code>color = "red", alpha = 0.5</code> inside a <code>geom_*()</code>	<code>style(color = "red", opacity = 0.5)</code>
<code>theme(...)</code>	<code>theme()</code> sets the page; <code>style()</code> sets a layer's constants
<code>ggsave("plot.svg")</code>	the rendered plot <i>is</i> SVG; <code>render_svg()</code> returns it as text to write to a file, spelled the same way in all four languages

E.3 Six differences to expect

- **+ is still +, and on purpose.** `ggplot2`'s author has said the pipe would have been the better interface (Wickham, 2018), so it is fair to ask why `gog` kept `+`. The short answer has two parts. Python has no pipe operator of its own, so a pipe could not spell one syntax in all four languages, and a pipe makes every atom a function, which would leave `bar * bin` meaning nothing. [Operators](#) has the long version, including what commutes in `gog` that did not commute in `ggplot2`.e.
- **Nothing is computed unless you name it.** `bar` draws your numbers as they arrive; if you want counting, you write `count`Where a `geom` counts for you, `gog` computes nothing until you write the word.d.
- **There is one spelling.** No `geom/stat` pairs, and no choice of which function to use. If you can say a plot two ways here, one of them is a bug. Use "Report an issue" in the margin.
- **The scale belongs to the binding.** You name a channel once, and its scale is a property of that binding, not a second sentence about it.
- **Setting is not mapping.** A constant maps no column, gets no legend, and belongs in `style()`. The quoted-string ambiguity (`color("red")`) does not exist here: it is refused, with the fix named.
- **Refusals give direction.** A sentence `gog` cannot draw is refused, and the message names the sentence it would accept. If you are used to a warning and a plot, expect a stop instead. The errors are part of the book, and it renders them on purpose.

F Coverage, both directions

The [phrasebook](#) answers “how do I spell this in gog”. This appendix answers the question underneath it: *can* it be spelled. Every entry in `ggplot2`’s reference index was read against the kernel. The result is two lists of very different lengths, and a table of differences between them.

Each table below has a `ggplot2` column, and it is dated on purpose, because nothing here can check it: reference index of `ggplot2` 4.0.3, read 2026-07-26 (Wickham et al., 2026). Every `gog` claim below is either a live plot on this page or a refusal the build re-runs, and `ggplot2` is deliberately not a dependency of this book, so its column is the one that can go out of date.

The short list is the one people expect to be long, so it goes first. The long list is the interesting one, because every item on it is one of four things, and three of the four were decided on purpose.

F.1 What gog draws that ggplot2 does not

Two of these are pictures `ggplot2` cannot draw. The third is about the vocabulary rather than about a picture.

F.1.1 Anything in the cube

Binding `z` gives a plot a third position, and `surface` draws a height over a plane:

R

```
data(maunga_whau) + surface + x(east) + y(north) + z(elevation) +  
  color(elevation) + palette("plasma") +  
  title("Maunga Whau, one row per crossing")
```

Python

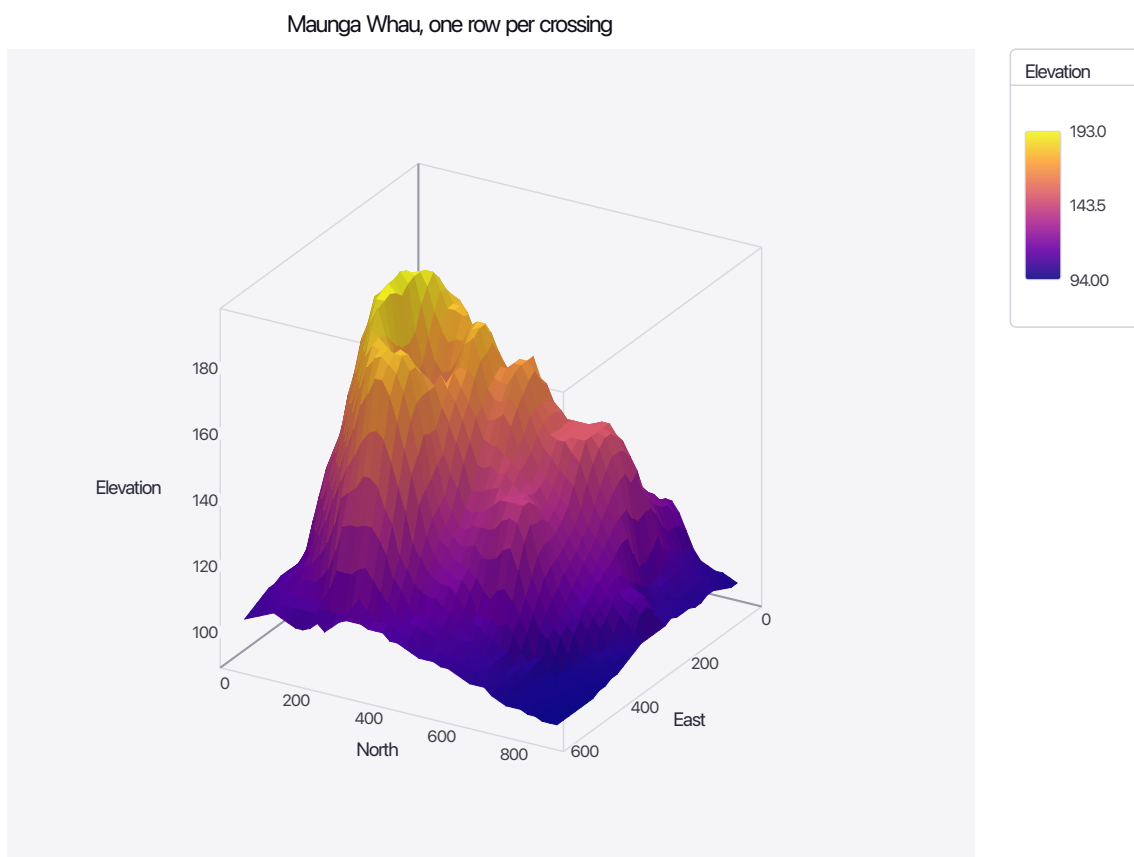
```
(data(maunga_whau) + surface + x(col.east) + y(col.north) + z(col.elevation)
↪ +
  color(col.elevation) + palette("plasma") +
  title("Maunga Whau, one row per crossing"))
```

Julia

```
data(maunga_whau) + surface + x(:east) + y(:north) + z(:elevation) +
  color(:elevation) + palette("plasma") +
  title("Maunga Whau, one row per crossing")
```

JavaScript

```
plot(data(maunga_whau), surface, x(col.east), y(col.north),
  z(col.elevation), color(col.elevation), palette("plasma"),
  title("Maunga Whau, one row per crossing"))
```



“Given Maunga Whau: a surface, x is east, y is north, z is elevation, color by elevation.”

Six marks draw in the cube: `point`, `path`, `bar`, `interval`, `box` and `surface`. `space(turn = , tilt = )` sets the viewing angle. `facet` and `play` split a cube’s rows as they split any other plot’s: `| facet(g)` gives one projected box per panel, `+ play(t)` runs the sequence inside a single box. `ggplot2` has no depth position and no coordinate system that would read one, and says so outright in the first chapter of its book, so none of this is expressible in it: in R these plots come from `plotly`, `rgl` or `lattice`, each with a grammar of its own. Here `z` is one more channel in the same sentence, and [Space](#) is the chapter.

F.1.2 Texture, mapped from a column

`pattern` maps a categorical column to a hatch on fills and a dash on strokes. Put it on the same column as `color` and each series is marked twice, by hue and by hatch, in one merged legend:

R

```
data(iris_flowers) + bar * mean + x(species) + y(petal_length) +  
  color(species) + pattern(species) +  
  title("Hue and hatch, one legend")
```

Python

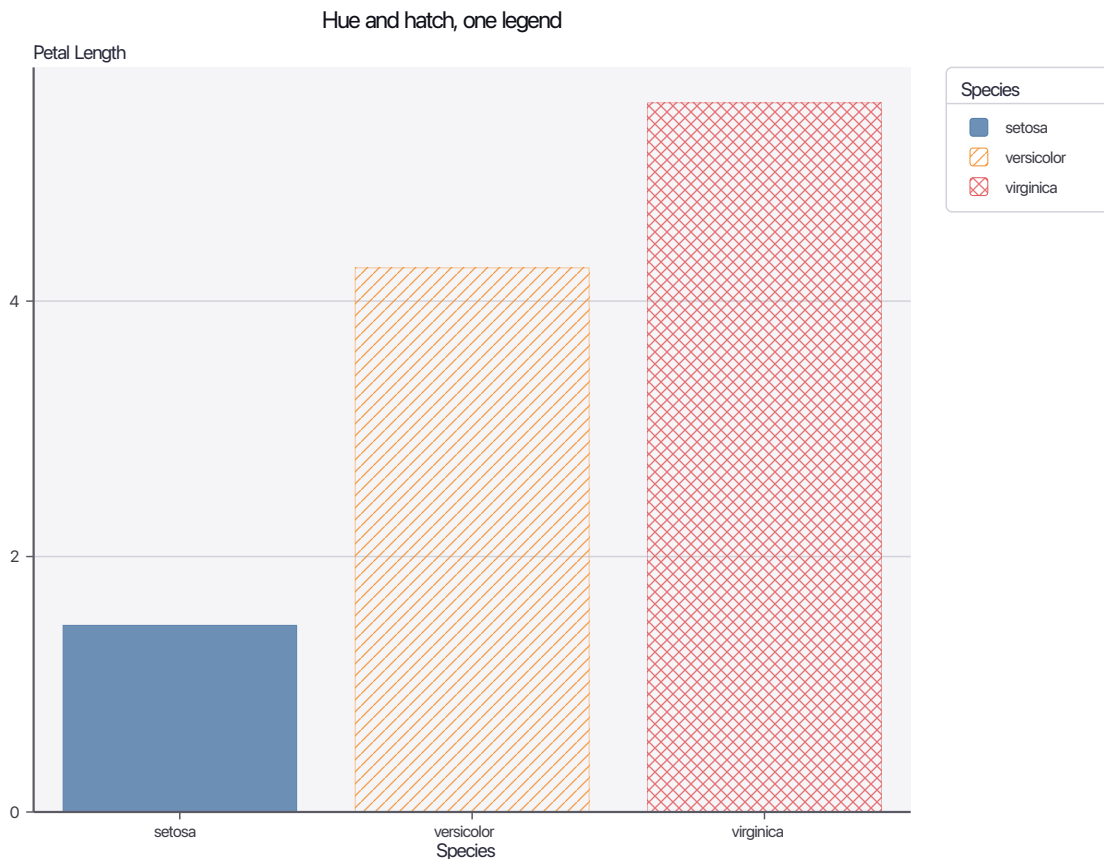
```
(data(iris_flowers) + bar * mean + x(col.species) + y(col.petal_length) +  
  color(col.species) + pattern(col.species) +  
  title("Hue and hatch, one legend"))
```

Julia

```
data(iris_flowers) + bar * mean + x(:species) + y(:petal_length) +  
  color(:species) + pattern(:species) + title("Hue and hatch, one legend")
```

JavaScript

```
plot(data(iris_flowers), layer(bar, mean), x(col.species),  
  y(col.petal_length), color(col.species), pattern(col.species),  
  title("Hue and hatch, one legend"))
```



“Given the iris flowers: bars derived by mean, x is species, y is petal length, color by species, pattern by species.”

The plot above stays legible in grayscale and under color blindness, because hue and hatch say the same thing twice. `ggplot2` has `linetype` for strokes and nothing for the inside of a filled shape, so hatching a bar there means the third-party `ggpattern`.

F.1.3 The same plot, without a word for it

Not a picture `ggplot2` lacks, but one this grammar draws without being taught it. `geom_violin()` is a mark of its own there, with `stat_ydensity()` beside it; here it is `ribbon * density`, the mark that closes on a second boundary, given the transform that estimates a distribution, over a category:

R

```
data(gapminder_2007) + ribbon * density + x(continent) + y(life) +  
  y_label("Life expectancy") + title("ribbon * density: no new word  
  ↪ required")
```

Python

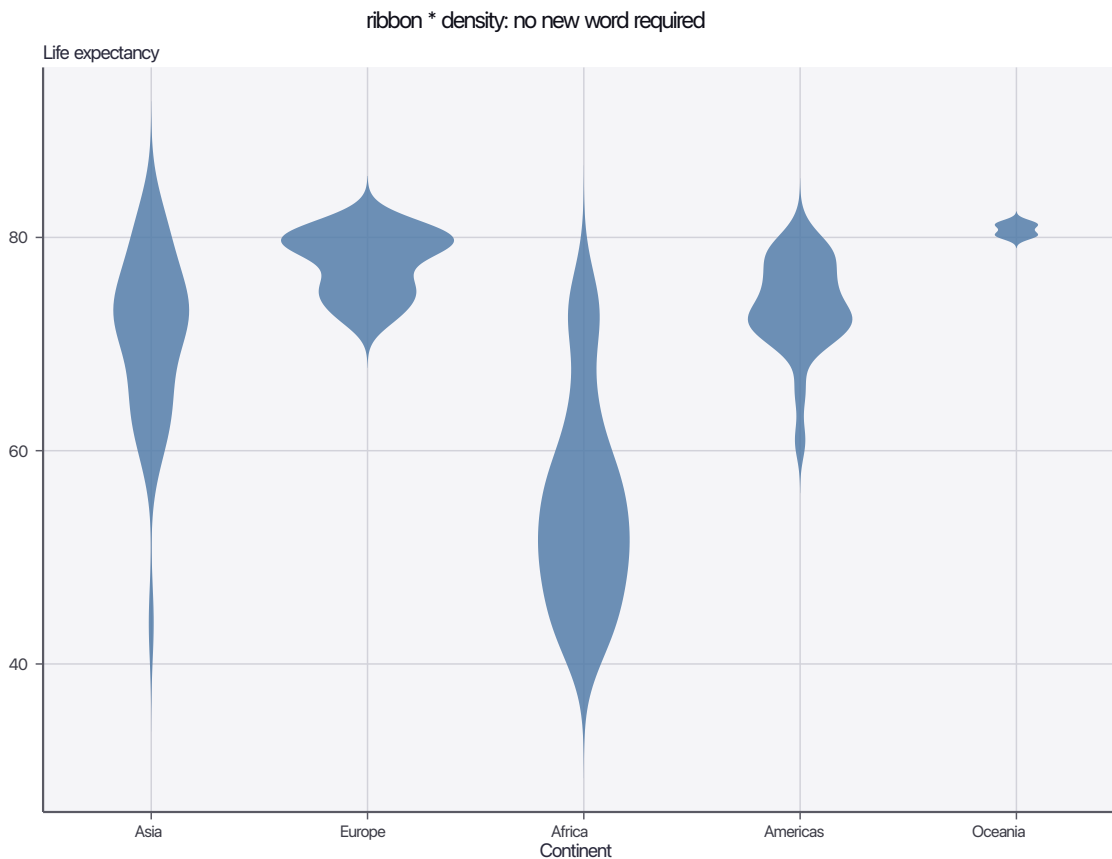
```
(data(gapminder_2007) + ribbon * density + x(col.continent) + y(col.life) +  
  y_label("Life expectancy") + title("ribbon * density: no new word  
  ↪ required"))
```

Julia

```
data(gapminder_2007) + ribbon * density + x(:continent) + y(:life) +  
  y_label("Life expectancy") +  
  title("ribbon * density: no new word required")
```

JavaScript

```
plot(data(gapminder_2007), layer(ribbon, density), x(col.continent),  
  y(col.life), y_label("Life expectancy"),  
  title("ribbon * density: no new word required"))
```



“Given gapminder 2007: ribbons derived by density, x is continent, y is life.”

The half violin comes with it, because `area` and `ribbon` already differ by where a region closes: `area * density + x(continent) + y(life)`. That is what the orthogonality is *for*: not that the plot is possible, but that the vocabulary did not grow to make it so.

The donut is the same case arriving from a *parameter* rather than a *geom*. `ggplot2` spells it `coord_radial(inner.radius = )`, a number given to the space; here the hole is not a property of the circle at all. It is a stretch of the radial axis with nothing standing on it, so a tree one level deep, drawn on an axis that starts below the first ring, is a donut:

R

```
data(spending) + zone * partition(group) + x(amount) +
  y(depth, limits = c(0, 2)) + color(group) + polar() +
  title("a donut is a sunburst of a one-level tree")
```

Python

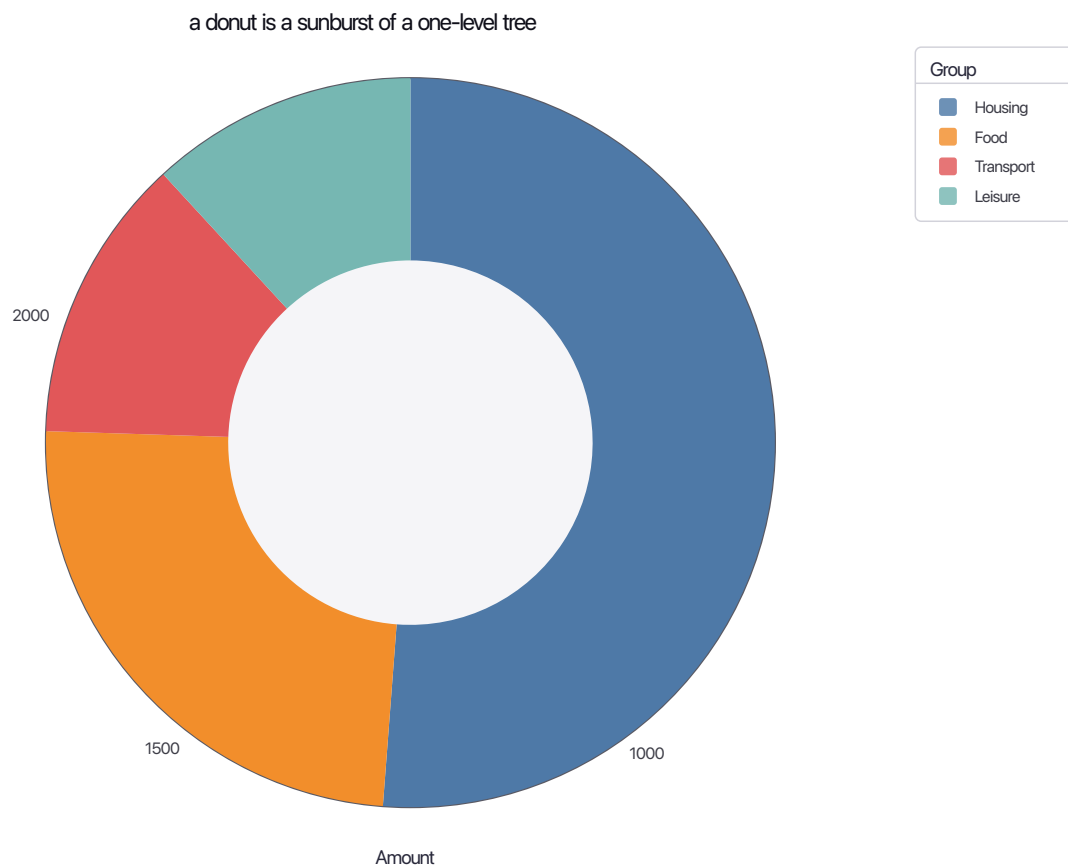
```
(data(spending) + zone * partition(col.group) + x(col.amount) +
  y(col.depth, limits = [0, 2]) + color(col.group) + polar() +
  title("a donut is a sunburst of a one-level tree"))
```

Julia

```
data(spending) + zone * partition(:group) + x(:amount) +
  y(:depth, limits = [0, 2]) + color(:group) + polar() +
  title("a donut is a sunburst of a one-level tree")
```

JavaScript

```
plot(data(spending), layer(zone, partition(col.group)), x(col.amount),
  y(col.depth, { limits: [0, 2] }), color(col.group), polar(),
  title("a donut is a sunburst of a one-level tree"))
```



“Given the spending table: zones derived by partition through group, x is amount, y is depth from 0 to 2, color by group, in polar.”

Nothing there is about donuts, and the same `limits` that hollow this out hollow the three-level `sunburst` and leave an empty band under the flat icicle, which is what tells you the hole belongs to the scale rather than to the bend. The one reading that cannot state a hole is the plain pie, whose radius is a constant with no domain to widen.

The marginal plot is the same again, this time between whole plots. `ggExtra` adds `ggMarginal()` to `ggplot2` and `patchwork` adds a layout algebra beside it; here a page is `|` and `/` between two plots, and the marginals follow from one rule about what a shared column means ([Composition](#)):

R

```
(data(gapminder_2007) + bar * bin + x(gdp) + theme(height = 130)) /  
  ((data(gapminder_2007) + point + x(gdp) + y(life)) |  
   (data(gapminder_2007) + bar * bin + y(life) + theme(width = 130)))
```

Python

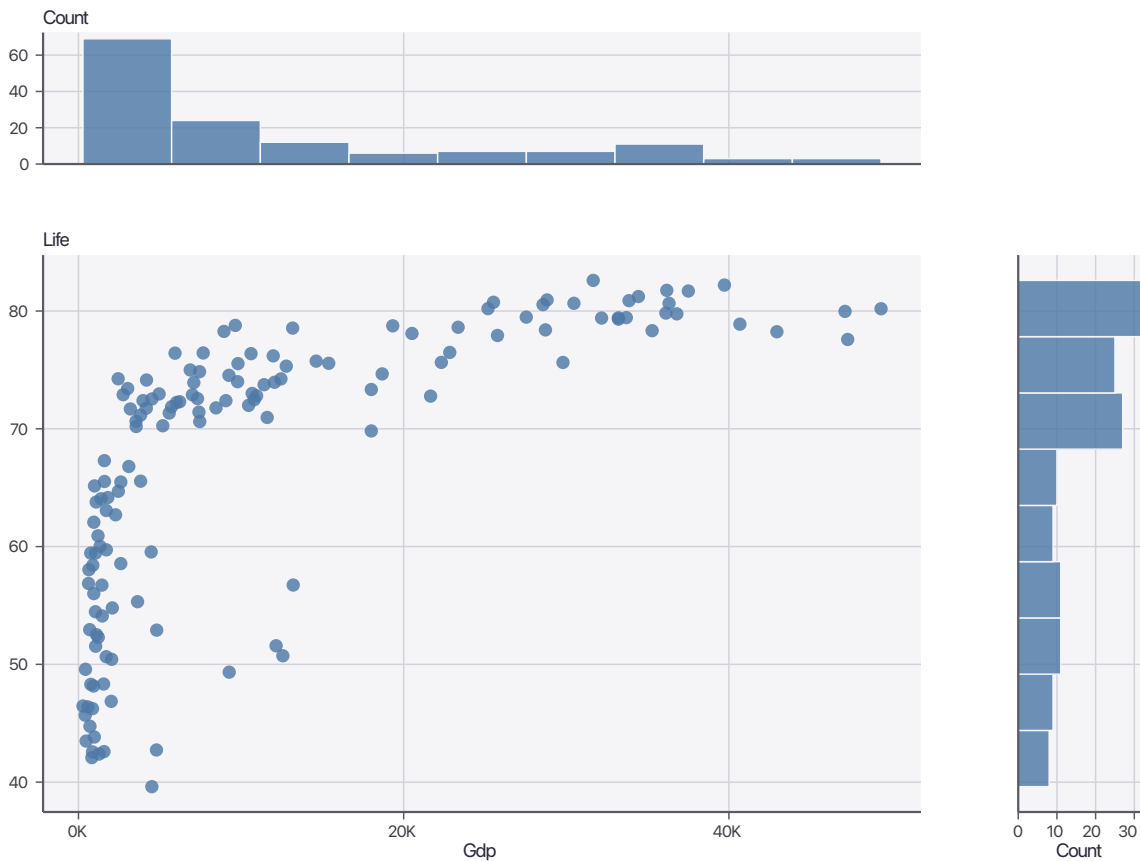
```
((data(gapminder_2007) + bar * bin + x(col.gdp) + theme(height = 130)) /  
  ((data(gapminder_2007) + point + x(col.gdp) + y(col.life)) |  
   (data(gapminder_2007) + bar * bin + y(col.life) + theme(width = 130))))
```

Julia

```
(data(gapminder_2007) + bar * bin + x(:gdp) + theme(height = 130)) /  
  ((data(gapminder_2007) + point + x(:gdp) + y(:life)) |  
   (data(gapminder_2007) + bar * bin + y(:life) + theme(width = 130)))
```

JavaScript

```
below(plot(data(gapminder_2007), layer(bar, bin), x(col.gdp),  
  theme({ height: 130 })),  
  beside(plot(data(gapminder_2007), point, x(col.gdp), y(col.life)),  
    plot(data(gapminder_2007), layer(bar, bin), y(col.life),  
      theme({ width: 130 }))))
```



“Given gapminder 2007: bars derived by bin, x is gdp; above points, x is gdp, y is life, beside bars derived by bin, y is life.”

That is the whole list. `ggplot2` draws everything else `gog` draws, and usually with more settings.

F.2 What is different rather than missing

The rest is not about pictures at all. These rows are differences in how the two packages work: each one follows from what its own package was built for.

	ggplot2	gog
Where it runs	R	R, Python, Julia and JavaScript, one engine, the same SVG out of each

	ggplot2	gog
What a plot <i>is</i>	R closures, alive in the session	an engine-neutral JSON specification you can store, compare, send or generate
A sentence the package cannot draw	a warning, and a plot with those rows removed (Removed 12 rows containing missing values)	no plot, and a message naming the sentence it would accept
What combines with what	documented per geom	one rule table, every pair decided, dumped from the engine into Combinations
Picking out a subset with the mouse	outside the package: the plot is a still image, and the answer is plotly or shiny	brush is a word in the sentence, so the printed page shows the selection too. It dims what you did not pick and removes no rows. Selection is the chapter
Moving overlapping labels apart	ggrepel , a separate package, with a geom of its own to use instead	text * repel , a collision modifier beside dodge , stack and jitter , so it composes with the text mark that was already there. It draws a thin line back to the point when a label has moved far, and where no arrangement fits, it still draws every label and reports how many overlap
Spellings per plot	often two (geom_bar / geom_col , geom_histogram / stat_bin)	one
Saying “log”	scale_x_log10() , a scale function per transformation	x(gdp, scale = "log") , written on the binding
Non-positive values on a log axis	dropped, with a warning	refused, with the fix named
A mean under a log axis	the geometric mean	computed in the data’s own units

	ggplot2	gog
Centering a diverging ramp	<code>scale_color_gradient2(midpoint = 0)</code> , which fixes the midpoint and stretches each side to the data's own ends	<code>limits = c(-k, k)</code> , the atom that already states a domain. The middle of a domain <i>is</i> a midpoint, so no parameter is added, and the ramp is linear on each side of the middle.

F.3 What ggplot2 draws that gog does not

The table below is sorted by the reason, because the reason is the useful part. A gap with a written ruling behind it is a different thing from a gap nobody has examined.

F.3.1 Refused, with the ruling on record

These will not arrive. Each has a written ruling, and the ruling names the direction to go instead. [Design laws](#) is where the laws behind them live.

In ggplot2	The ruling
<code>sec_axis()</code> , <code>dup_axis()</code>	A second y-axis is two coordinate spaces drawn in one panel. Facet on the measure, or layer if the units really are shared.
<code>annotate()</code> at page coordinates, <code>annotation_custom()</code> , <code>annotation_raster()</code>	A shape at pixel (120, 340) is a drawing command, and the specification has to mean the same thing at another size, in a facet, or on a sphere. Annotation in <i>data</i> space needs no new word: layer a mark over a small table.
<code>ggproto()</code> , user-written geoms and stats	Every promise on the Combinations grid depends on the vocabulary being fixed. An atom missing from the rule table has no row on that grid, breaks No Exceptions, and cannot be written into the JSON. Compose instead.
<code>coord_flip()</code>	There is no <code>flip</code> atom, because orientation is read off which axis carries the categories. <code>ggplot2</code> has since superseded it.

In ggplot2	The ruling
<code>geom_bar()</code> beside <code>geom_col()</code> ; <code>geom_histogram()</code> beside <code>stat_bin(geom = "bar")</code> <code>geom_smooth(method = "lm")</code>	<code>gog</code> spells each of these one way. <code>bar</code> draws your numbers, <code>bar * count</code> counts them, <code>bar * bin</code> bins them. A family of fits behind one argument is exactly the enumeration this grammar refuses. A least-squares line is a different statistic from a local smoother, so it needs its own plain name.
<code>scale_x_log10()</code> , <code>scale_x_sqrt()</code> as <i>names</i> <code>coord_polar(theta = "y")</code>	One parameter derives the family: <code>base = 2</code>, <code>base = 10</code>, <code>base = exp(1)</code>. Which axis is the angle is read off the bindings, not chosen by an argument.
<code>qplot()</code> , and plotting bare vectors	A plot is a mapping from a table. Put the vectors in one.
<code>geom_vline()</code> , <code>geom_hline()</code> as separate marks	Three marks for one idea. They are one rule, which takes a <i>column</i>, so a table of thresholds draws all of them at once, and which is also the rug via <code>style(reach = "edge")</code>.
<code>plotmath</code> , <code>label_parsed()</code> , <code>label_bquote()</code>	A TeX engine belongs to one output format, not to the grammar. Unicode covers what readers need: superscript powers on log ticks, Greek, sub- and superscripts in a label.
<code>coord_cartesian(xlim =)</code>	One name here covers two different operations. <i>Narrowing the domain and running the statistics again</i> is limits, which already counts the rows it left out aloud. <i>Magnifying the finished picture</i> changes how you look and not what the plot claims, so it needs no word: it is a button under every plot in the web edition, <code>-</code>, <code>+</code> and <i>fit</i>.

In ggplot2	The ruling
plotly-style interaction: click a point, and read what it is	This needs no word in the kernel, and the survey of what other tools do is what decided it. Plotly hands your application an <i>event</i> ; Vega-Lite puts a predicate over named fields in the specification; D3 asks the author for a row identity, and uses it for animation rather than for selection. <code>gog</code> answers each of the three elsewhere. Reading a row belongs to the page, so the pointer does it, and in the web edition a click leaves that reading on the plot as a stamp . A predicate over a column is brush . A row identity is what an animation needs, and the grammar has no word for it (Animation).
a free-shape selection, drawn around some points with the mouse	There is no lasso word in the kernel. Every other selection can be written down: <code>brush(gdp, at = c(1200, 45000))</code> names a range, and the printed page then shows the plot that range asks for. A free shape can only be written as a long list of corners, which nobody writes in a sentence. The <i>act</i> was not refused, only the word: in the web edition you draw the shape with the mouse and the sentence still says brush .

The refusal a `ggplot2` reader is most likely to meet is the first one, so here it is, live. Two series in different units, the second asking for its own scale:

```
data(quarterly) + x(year) + y(sales) + line +
  point + y(sales, scale = "log")
```

Error:

```
! gog: this layer's `y(sales, ...)` gives it its own scale, and the layers of
↪ one plot share one y axis - two scales on it would be two coordinate
↪ spaces in one panel, where every mark reads against an axis the
↪ expression never named. Set the scale once for the plot, before the
↪ marks: `y(<column>, scale = "log")`. A layer may name its own *column*
↪ for the shared axis - `y(sales)` on its own is fine - but not its own
↪ scale. If the two series are genuinely in different units, facet on the
↪ measure instead.
```

`gog`: nothing was rendered. Fix the above, or set `GOG_STRICT=0` to draw anyway.

Every refusal in `gog` has that shape: it says what the rule is, and what to write instead.

F.3.2 The host language's job

These are computations, not pictures. `gog` draws what you hand it, and the fitted value, the theoretical quantile and the reshaped model belong in R, Python, Julia or JavaScript, before the plot, where you already have better tools for them.

In <code>ggplot2</code>	Where it goes
<code>stat_ecdf()</code>	compute the cumulative shares, then step
<code>stat_qq()</code> , <code>stat_qq_line()</code>	compute the theoretical quantiles, then point and line
<code>stat_ellipse()</code>	compute the ellipse, then path
<code>stat_quantile()</code>	fit the quantile regression, then line
<code>stat_function()</code> , <code>geom_function()</code>	evaluate the function onto a grid, then line
<code>stat_unique()</code>	de-duplicate the table
<code>cut_interval()</code> , <code>cut_number()</code> , <code>cut_width()</code>	cut the column, or use bin
<code>mean_cl_boot()</code>	bootstrap a lower and an upper bound, then ribbon * bounds(lower, upper) . Two helpers documented beside it are <code>gog</code> atoms instead: <code>mean_sdl()</code> is deviation , and <code>median_hilow()</code> is range(0.025, 0.975) with point * median
<code>fortify()</code> , <code>autoplot()</code> , <code>map_data()</code>	get the model or the map boundaries in the host, then hand over a table
a correlation matrix	<code>cor()</code> where the data lives, reshape it long, then zone + x(var1) + y(var2) + color(r)

The boundary is not as clean as that table makes it look, and pretending otherwise would be dishonest. `gog` *does* compute: **bin** cuts, **density** estimates, **smooth** fits a local regression, **confidence** builds an interval, **deviation** measures spread, and **quantile** cuts a distribution. The test is whether the computation is the picture. A histogram has no bars until something bins. A quantile regression is a separate answer that is then drawn as a line. Where the test lands in a new case is not settled.

F.3.3 Missing, with the direction already settled

gog does not draw these. Each one belongs to a family the grammar already has, and nothing about them is disputed.

In ggplot2	Here
<code>labs(subtitle =, caption =)</code>	more of the label family; <code>title()</code> is the one that exists
<code>theme(legend.position =)</code>	<code>theme()</code> is where this would be written. Every legend sits to the right of the plot area, and no setting moves it
<code>geom_smooth(se = TRUE)</code>	the band around a smoother. <code>confidence</code> is the pair transform for a mean; a smoother has no matching one
<code>geom_sf()</code> , <code>geom_map()</code> , <code>coord_sf()</code> , <code>borders()</code>	the <code>map</code> space draws, so a mark at a place on a projected earth is an ordinary sentence, and <code>zone</code> with <code>group(country)</code> fills each region from a boundary the data carries, holes included (Map). <code>gog</code> reads no spatial file. A boundary arrives as an ordinary table of points, which is the shape <code>map_data()</code> returns
<code>geom_dotplot()</code>	a mesh that follows the data's clusters, and a second rule for choosing bin edges
<code>scale_x_binned()</code> , equal-count bins	the other cutpoint rule, the same parameter

F.3.4 Not decided either way

None of these has a ruling in either direction, so this table is the whole record of them.

In ggplot2	What is missing here
<code>scale_*_continuous(breaks =, labels =)</code>	tick <i>positions</i> and tick <i>text</i> . <code>tick_count</code> asks for a number of ticks and the engine chooses them, so “tick at 0, 50 and 100” and relabeling a category cannot be written.
<code>scale_*_manual(values = c(setosa = "red"))</code>	binding a color to a level <i>by name</i> , shown below
<code>scale_y_reverse()</code>	a reversed continuous axis. <code>order(desc = TRUE)</code> reverses a categorical one.
<code>scale_x_sqrt()</code> , <code>coord_trans()</code>	a power scale. The <code>log10</code> -style <i>naming</i> is refused; the capability never was.

In ggplot2	What is missing here
<code>guides(color = "none"), show.legend = FALSE</code>	turning a legend off. Every mapped channel earns one, always.
<code>linewidth</code> as an aesthetic	a column mapped to stroke width. <code>size</code> can be set on a stroke for the whole layer, not mapped per row.
<code>labs(alt =), get_alt_text()</code>	alt text on the rendered plot
<code>geom_label()</code>	text in a box, rather than bare <code>text</code>
<code>geom_curve()</code>	a curved connector between two points; <code>path</code> joins them straight
<code>after_stat(), after_scale(), stage()</code> , and <code>geom_count()</code> with them	sending a transform's <i>output</i> to a channel rather than to a position. <code>point * count + size(count)</code> is refused, because the count a transform invented is not a column the sentence can name, so sizing a point by how many rows landed on it has no spelling. One question, and it is the widest one on this list.
<code>geom_contour()</code> over a supplied <code>z</code>	iso-lines of a field you computed. <code>path * density</code> traces a field <code>gog</code> estimated.
<code>geom_text(angle =),</code> and ggplot2's polar text generally	a rotated label. <code>text</code> places a name at a point in every space, upright, so a sunburst cannot run its labels along their arcs and small wedges collide. The gap is general to polar rather than belonging to that one plot.
<code>theme(axis.text = element_blank())</code> and friends	asking for a plot with no axis text. <code>theme(grid =)</code> removes gridlines and <code>tick_count = 0</code> is refused, so silence has to follow from what an axis carries rather than being asked for. A pie is recognized from its bindings. A packing has no axes at all. A partition's radial axis carries the depth the transform invented, so it draws no ticks, no rings and no name. Three recognitions do one general job. What is left over is the reader who wants a <i>real</i> quantity's axis silent: the measure around a sunburst takes <code>x_label("")</code> and keeps its numbers.

The color one is worth seeing, because it is not a missing setting so much as a missing *kind* of statement. Three species, three colors:

R

```
data(iris_flowers) + point + x(petal_length) + y(sepal_length) +  
  color(species) + palette(c("tomato", "steelblue", "seagreen")) +  
  title("Rows in the order the table has them")
```

Python

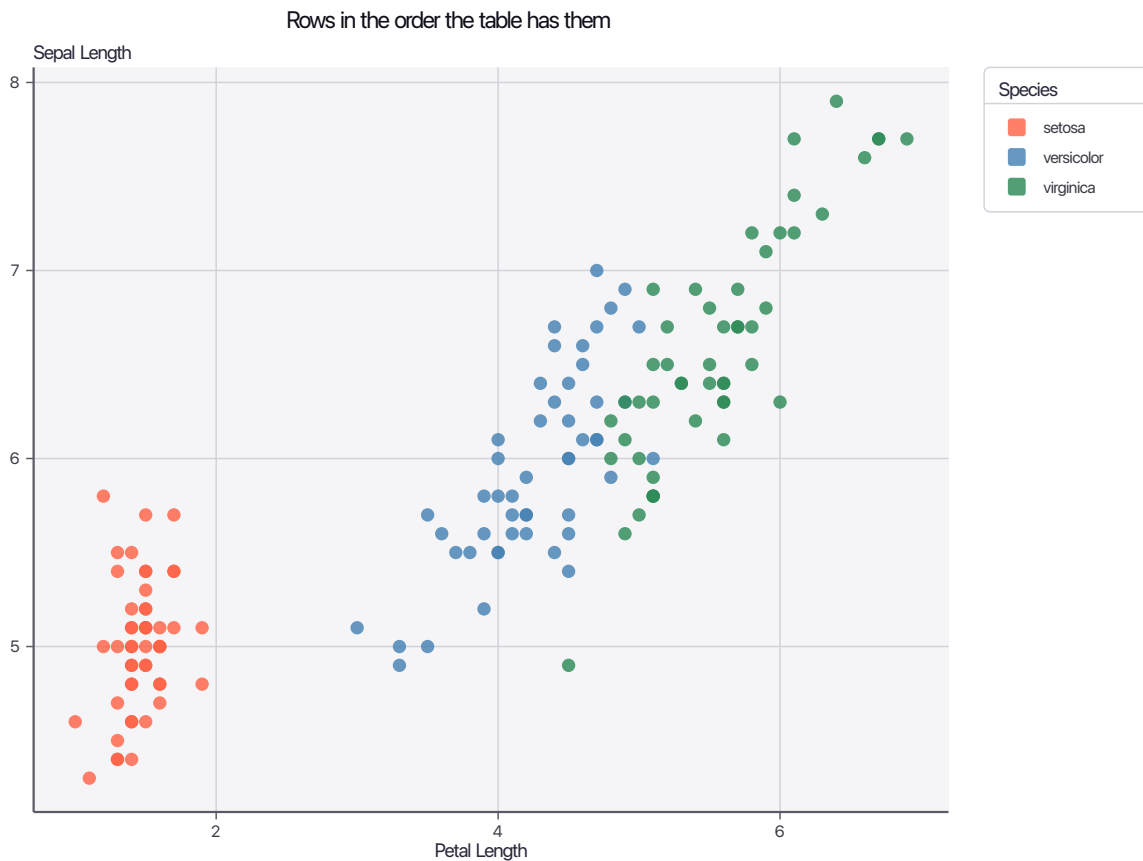
```
(data(iris_flowers) + point + x(col.petal_length) + y(col.sepal_length) +  
  color(col.species) + palette(["tomato", "steelblue", "seagreen"])) +  
  title("Rows in the order the table has them"))
```

Julia

```
data(iris_flowers) + point + x(:petal_length) + y(:sepal_length) +  
  color(:species) + palette(["tomato", "steelblue", "seagreen"]) +  
  title("Rows in the order the table has them")
```

JavaScript

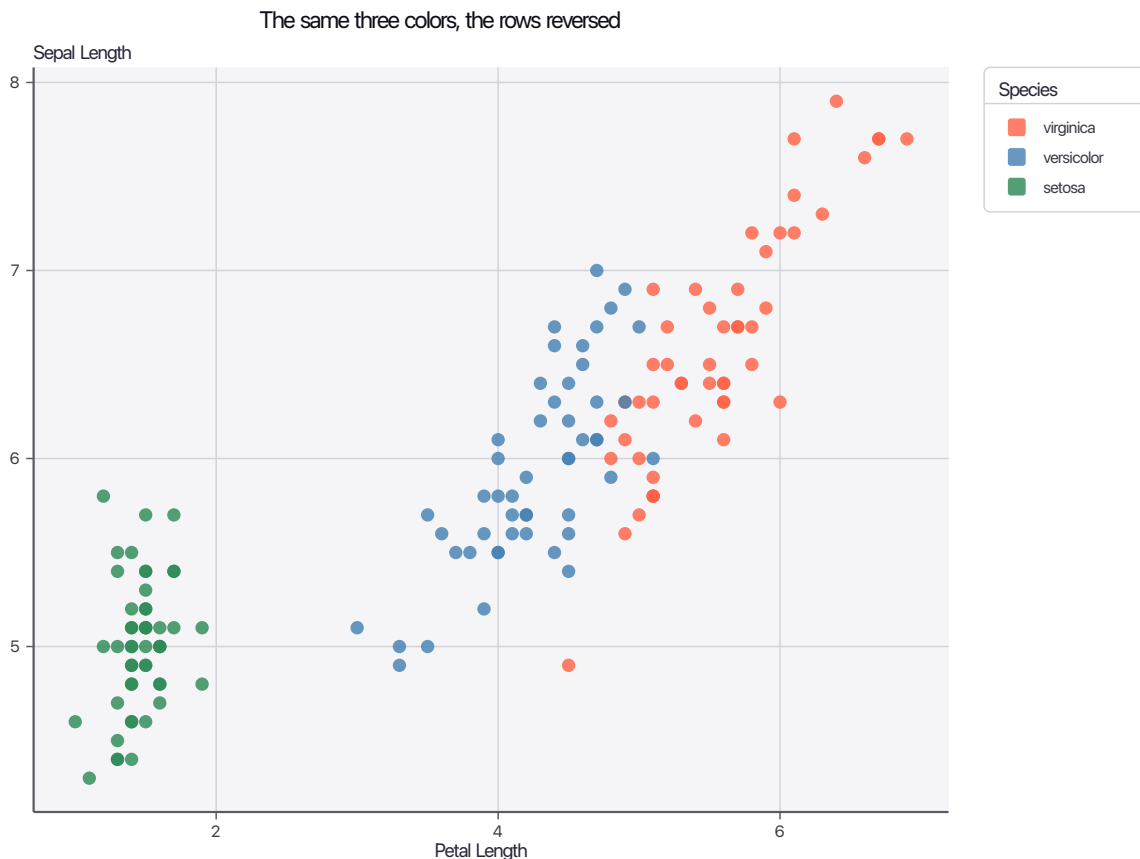
```
plot(data(iris_flowers), point, x(col.petal_length), y(col.sepal_length),  
  color(col.species), palette(["tomato", "steelblue", "seagreen"]),  
  title("Rows in the order the table has them"))
```



“Given the iris flowers: points, x is petal length, y is sepal length, color by species, with a palette of tomato, steelblue and seagreen.”

Now the same sentence and the same palette, over the same rows sorted the other way. Watch the legend:

```
by_species <- iris_flowers[base::order(iris_flowers$species, decreasing =
  ↪ TRUE), ]
data(by_species) + point + x(petal_length) + y(sepal_length) +
  color(species) + palette(c("tomato", "steelblue", "seagreen")) +
  title("The same three colors, the rows reversed")
```



Nothing is wrong here. A palette gives out its colors in the categories' order, and that order is the axis's order. For a text column, the axis order is the order in which the values first appear. A declared order is honored when you supply one. It is consistent and it is documented. There is no way to say *setosa is tomato*. So a category cannot keep its color across two plots whose category order differs, and a report that puts two such plots side by side usually wants exactly that.

F.4 If your plot is not here

The sweep was of `ggplot2`'s reference index, so a plot that lives in one of its extensions may be missing from both columns. Networks are the largest such family, and `gog` draws them: [Network](#) is the node-link diagram, and [Flow](#) is the alluvial. One family has no drawing here: a filled band around a hollow, which is what a ring-shaped density needs. `path * density` traces that field correctly; `zone * density`, which fills the bands between the same lines, cannot.

For a plot you can name, [Chart names](#) is the dictionary and says which ones are drawn. For a question rather than a name, start at the [cookbook](#).